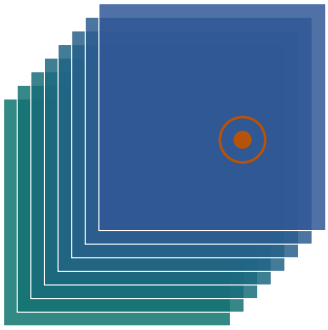
Finding What Does Not Belong

The Mathematics of Hyperspectral Anomaly Detection and the Indradhanu Foundation Model

A step-by-step textbook built from the Allotrope training courses



Study material, subordinate to the source code and to `docs/01-orientation.md` through `docs/10-code-style.md`.

Compiled from the courses as of 2026-10-29

Contents

How to read this book **xlii**

Notation **xlvi**

I The domain and the data **1**

1 The problem: finding what does not belong **2**
1.1 The situation . . . 2
1.2 Why a classifier will not work . . . 3
1.3 The reframe: model normality, score the residue . . . 4
1.4 Two ways to model normality . . . 5
1.5 From a score map to a shortlist: thresholds are percentiles . . . 7
1.6 Why satellites make this harder . . . 9
1.7 Notation for the rest of the book . . . 10
1.8 Common confusions . . . 11
1.9 Check yourself . . . 11

2 Light, bands and the hyperspectral cube **12**
2.1 Why three numbers are not enough . . . 12
2.2 Light and wavelength . . . 12
2.3 Radiance, irradiance and reflectance . . . 13
2.4 What a band records . . . 15
2.5 A spectrum is a vector . . . 17
2.6 The cube . . . 18
2.7 Thermal images: the one-band cousin . . . 20
2.8 Validity: not every number is a measurement . . . 21
2.9 Common confusions . . . 22
2.10 Check yourself . . . 23

3 Five sensors and the 165-band grid **24**
3.1 Why this chapter exists . . . 24
3.2 The roster . . . 24
3.3 Three ways to lay a cube out in memory . . . 25
3.4 From stored integers to physical units . . . 26

3.5 Quality information, and reading bits . . . 27
3.6 How the code hides the differences . . . 28
3.7 Sensors that do not agree on wavelengths . . . 28
3.8 Building the grid . . . 29
3.9 Two design questions . . . 33
3.10 The grid is a wire format . . . 33
3.11 Common confusions . . . 34
3.12 Check yourself . . . 35

4 The product: scenes, projects, actions and the vendable 36
4.1 Why this comes before any model . . . 36
4.2 The four nouns . . . 36
4.3 Actions form a directed acyclic graph . . . 37
4.4 Two processes and one queue . . . 39
4.5 Where Indradhanu lives . . . 41
4.6 The vendable . . . 42
4.7 No spatial reference, on purpose . . . 45
4.8 Vendables are pickled: class names are a wire format . . . 46
4.9 The thermal vendable and its two cloud masks . . . 46
4.10 Common confusions . . . 46
4.11 Check yourself . . . 47

5 Making sensors comparable: band filtering, interpolation and normalisation 48
5.1 The problem . . . 48
5.2 Stages 1–4: deciding which bands to trust . . . 49
5.3 Stages 5 and 6: deciding which pixels to keep . . . 50
5.4 Linear interpolation, from the equation of a line . . . 51
5.5 Why the wavelengths must go up: a preview of a real bug . . . 54
5.6 Stage 7: filling holes without inventing features . . . 55
5.7 Stage 8: onto the grid . . . 56
5.8 Per-band normalisation inside the model . . . 57
5.9 The nearest-valid pixel fill . . . 61
5.10 Common confusions . . . 63
5.11 Check yourself . . . 63

II Statistical detection 65

6 The mathematical toolkit: vectors, projection, covariance, eigenvectors, PCA and whitening 66
6.1 Why a chapter of linear algebra . . . 66
6.2 Vectors, the dot product, length and angle . . . 67
6.3 Projection: the best multiple of one vector . . . 69
6.4 Matrices, the matrix-vector product and the transpose . . . 72

6.5 Mean, centring and the covariance matrix . . . 74
6.6 Eigenvalues and eigenvectors . . . 77
6.7 Principal component analysis . . . 79
6.8 Whitening . . . 81
6.9 Common confusions . . . 83
6.10 Check yourself . . . 83

7 Anomaly as surprise: the Gaussian, Mahalanobis distance and the RX detector 85
7.1 Anomaly as surprise . . . 85
7.2 The one-dimensional Gaussian and the z-score . . . 86
7.3 Why many bands need more than many z-scores . . . 87
7.4 The multivariate Gaussian . . . 88
7.5 Mahalanobis distance . . . 89
7.6 The RX detector . . . 90
7.7 How large is large? The chi-square distribution . . . 91
7.8 A full worked example in three bands . . . 94
7.9 Why global RX fails on real scenes . . . 96
7.10 Regularisation . . . 98
7.11 The thermal case: RX in one band . . . 100
7.12 Common confusions . . . 100
7.13 Check yourself . . . 101

8 Local RX, regularisation and MNF-RX 102
8.1 Two failures, two repairs . . . 102
8.2 The dual window . . . 103
8.3 Why the guard window exists . . . 104
8.4 The local RX algorithm . . . 105
8.5 The cost of local RX . . . 107
8.6 Choosing a better space: why not PCA . . . 107
8.7 Deriving MNF . . . 108
8.8 MNF-RX . . . 111
8.9 PCA-RX, the evaluation baseline . . . 113
8.10 Global and local together . . . 114
8.11 Common confusions . . . 114
8.12 Check yourself . . . 115

III Reconstruction, the Indradhanu architecture and Chakshu 116

9 Reconstruct, then subtract: why a pixel must never help reconstruct itself 117
9.1 The problem: no labels, and a background too varied for one Gaussian . . . 117
9.2 An autoencoder is a function from a cube to a cube . . . 118
9.3 The trap: the identity is the perfect answer to the wrong question . . . 120

9.4 A one-dimensional example you can do by hand . . . 121
9.5 The bottleneck makes copying hard, not impossible . . . 123
9.6 L2 teaches the model to reconstruct anomalies; L1 does not . . . 124
9.7 Masking: make the answer absent . . . 126
9.8 Inference: every pixel predicted while hidden . . . 128
9.9 Where the rule is enforced, and the whole model on one page . . . 129
9.10 Common confusions . . . 131
9.11 Check yourself . . . 132

10 Tensors, shapes, layers and the arithmetic of convolutions 133
10.1 Tensors and how to read a shape . . . 133
10.2 Two layouts of the same numbers . . . 135
10.3 The linear layer . . . 137
10.4 The convolution . . . 138
10.5 The output-size formula, derived by counting . . . 140
10.6 Receptive field . . . 142
10.7 The 1×1 convolution is a linear layer at every pixel . . . 143
10.8 Counting parameters: a convolutional encoder by hand . . . 143
10.9 The whole forward pass as a table of shapes . . . 145
10.10 Common confusions . . . 145
10.11 Check yourself . . . 146

11 The five convolutional autoencoders 147
11.1 Two building blocks: the stride-2 convolution and its transpose . . . 147
11.2 The five side by side . . . 148
11.3 Pratibimba: the plain autoencoder . . . 149
11.4 Antardhana and Tirohita: hide pixels, keep the class . . . 149
11.5 Asanskrita: tell the network where the holes are . . . 150
11.6 Drashta: the masks and the normalisation . . . 151
11.7 What the product does with them . . . 151
11.8 Common confusions . . . 152
11.9 Check yourself . . . 153

12 The spectral compressor 154
12.1 The problem: 165 channels into a 32-wide transformer . . . 154
12.2 The compressor is a matrix applied at every pixel . . . 155
12.3 BatchNorm . . . 156
12.4 The decompressor, and why it must have no normalisation . . . 159
12.5 A linear bottleneck: everything passes through 32 directions . . . 159
12.6 Counting the parameters . . . 161
12.7 It is MNF-shaped, but it is not MNF . . . 162
12.8 Why D is 32, not 8 and not 128 . . . 162
12.9 Common confusions . . . 163

12.10Check yourself 163

13 Patch embedding **165**
13.1 The problem: pixels are too many tokens 165
13.2 A strided convolution makes the tokens 166
13.3 LayerNorm: each token standardised on its own 168
13.4 Four stages, two kinds of window 169
13.5 Parameter counts, derived and checked 172
13.6 Common confusions 173
13.7 Check yourself 173

14 Attention, from one query to many heads **175**
14.1 The problem: a token needs its surroundings 175
14.2 Similarity: the dot product 176
14.3 From scores to proportions: the softmax 176
14.4 One query against many keys 180
14.5 Queries, keys and values are learned projections 181
14.6 Attention in matrix form 181
14.7 Why divide by the square root of d 185
14.8 Attention ignores order: permutation equivariance 187
14.9 Many heads 188
14.10Self-attention and cross-attention 190
14.11The cost problem 191
14.12Common confusions 191
14.13Check yourself 191

15 Efficient self-attention with spatial reduction **193**
15.1 The cost of plain attention, counted 193
15.2 The strided convolution that summarises 195
15.3 The data flow, with shapes at every step 196
15.4 The 16-token summary 198
15.5 What the reduction saves, in work and memory 199
15.6 What the reduction costs in parameters 200
15.7 The fallback: when the grid is incomplete 201
15.8 Common confusions 202
15.9 Check yourself 203

16 Mix-FFN, LayerNorm, residuals and the transformer block **204**
16.1 Why attention alone is not enough 204
16.2 LayerNorm 205
16.3 Residual connections 207
16.4 Pre-norm, not post-norm 208
16.5 Dropout 209
16.6 Mix-FFN 210

16.7 The block . . . 214
16.8 Common confusions . . . 216
16.9 Check yourself . . . 216

17 The four-stage encoder 218
17.1 Why one resolution is not enough . . . 218
17.2 The pyramid, stage by stage . . . 219
17.3 The attention budget per stage . . . 222
17.4 How far each token sees: the receptive field . . . 223
17.5 One stage, in code . . . 224
17.6 Hiding happens at stage 1 only . . . 225
17.7 The complete masked walk-through . . . 226
17.8 Three things that look like bugs and are not . . . 226
17.9 What the encoder returns . . . 227
17.10Common confusions . . . 227
17.11Check yourself . . . 228

18 Masked autoencoding: validity, targets, remove-and-restore, erosion 229
18.1 The problem, and why the obvious fixes fail . . . 229
18.2 Step 1: which tokens hold real data . . . 230
18.3 Step 2: choosing which valid tokens to hide . . . 232
18.4 Step 3: gather and scatter, remove and restore . . . 236
18.5 Step 4a: prediction targets and the loss mask . . . 239
18.6 Step 4b: erosion of the mask border . . . 240
18.7 All four masks on one patch . . . 242
18.8 Common confusions . . . 244
18.9 Check yourself . . . 245

19 The decoder, pixel shuffle and one full forward trace 246
19.1 The problem, and why the obvious fixes fail . . . 246
19.2 Step 1: one width for all four maps . . . 247
19.3 Step 2: one size for all four maps, by bilinear interpolation . . . 248
19.4 Steps 3 and 4: stack, then fuse . . . 249
19.5 Step 5: refine at quarter resolution, then pixel shuffle . . . 250
19.6 Decompress and denormalise . . . 253
19.7 One patch, all the way through . . . 253
19.8 Common confusions . . . 255
19.9 Check yourself . . . 256

20 The parameter budget, counted by hand 257
20.1 What a parameter is, and what it is not . . . 257
20.2 Three counting rules . . . 258
20.3 The configuration being counted . . . 258
20.4 The front end: the spectral compressor . . . 259

20.5 The encoder . . . 259
20.6 The decoder and the back end . . . 261
20.7 The grand total, and the buffers . . . 261
20.8 Where the parameters live . . . 262
20.9 Parameters are not compute . . . 263
20.10 Parameters a masked pass never trains . . . 264
20.11 Other models: which figure belongs to which . . . 264
20.12 Common confusions . . . 266
20.13 Check yourself . . . 266

21 Chakshu: the same machine for one thermal band 268
21.1 Why a thermal model, and why this one . . . 269
21.2 The input: one band, in Celsius . . . 269
21.3 The configuration . . . 271
21.4 One forward pass at full size . . . 271
21.5 The parameters, counted by hand . . . 273
21.6 Training: 70% hidden, the trimmed L1 . . . 275
21.7 Inference on a scene . . . 277
21.8 From reconstruction to score . . . 278
21.9 The HotSat-1 units caveat . . . 279
21.10 Where Chakshu sits in the product . . . 280
21.11 Common confusions . . . 281
21.12 Check yourself . . . 281

IV The spectral stem 283

22 Why the compressor is a ceiling: cross-attention and a spectrum as tokens 284
22.1 The compressor you already know . . . 284
22.2 Consequence 1: one recipe for every pixel, proved with rank . . . 285
22.3 Consequence 2: a broken band cannot be left out . . . 287
22.4 Consequence 3: tied to 165 positions . . . 288
22.5 Cross-attention: the same four steps, someone else asking . . . 289
22.6 A spectrum as tokens that know their wavelength . . . 294
22.7 Common confusions . . . 299
22.8 Check yourself . . . 300

23 The cross-attention compressor, dead bands and what it costs 302
23.1 The data flow for one pixel . . . 302
23.2 Predict the parameter count . . . 304
23.3 The 1x1 convolution is a special case . . . 306
23.4 Skipping a dead band, exactly . . . 308
23.5 What it costs in arithmetic . . . 311
23.6 The memory wall . . . 312

23.7 Common confusions . . . 316
23.8 Check yourself . . . 316

24 The spectral decoder, the traps, per-pixel gating and spectral reduction 318
24.1 The way back out: a decoder that answers for any wavelength . . . 318
24.2 One pixel start to finish, and the three traps . . . 322
24.3 Per-pixel gating: the cheap rival . . . 325
24.4 Spectral reduction: attention over wavelength bins . . . 329
24.5 Common confusions . . . 341
24.6 Check yourself . . . 341

V Training 343

25 Losses: L1, the spectral angle and the ramp 344
25.1 What a loss is, and what it must do . . . 344
25.2 The masked mean absolute error . . . 345
25.3 Why L1 and not squared error . . . 349
25.4 The 40% rule and sample counting . . . 350
25.5 The spectral angle . . . 351
25.6 The chain rule . . . 354
25.7 The derivative of arccos, and where it blows up . . . 355
25.8 The gradient of the spectral angle . . . 356
25.9 Why the code does not call arccos . . . 358
25.10 The combined loss and the ramp . . . 360
25.11 Common confusions . . . 364
25.12 Check yourself . . . 365

26 Training data and the training loop: gradients, AdamW, warmup and cosine schedules 366
26.1 Vocabulary: parameter, gradient, batch, step, epoch . . . 367
26.2 Gradient descent, by hand . . . 367
26.3 Momentum . . . 368
26.4 Adam . . . 369
26.5 Weight decay, L2 regularisation and AdamW . . . 371
26.6 Learning-rate schedules . . . 372
26.7 Gradient clipping . . . 375
26.8 Mixed precision . . . 375
26.9 Gradient accumulation . . . 376
26.10 The training data . . . 377
26.11 The loop . . . 380
26.12 Common confusions . . . 383
26.13 Check yourself . . . 384

27 Pretraining on SpectralEarth **385**
27.1 What this chapter is about . . . 385
27.2 SpectralEarth in numbers . . . 386
27.3 A ruler that does not move . . . 387
27.4 One band list is not any band list . . . 390
27.5 F001: two hundred epochs on rotating data . . . 395
27.6 Three sensors at their own bands . . . 399
27.7 Common confusions . . . 404
27.8 Check yourself . . . 404

28 The segmentation head, BCE plus Dice, and auditing labels **406**
28.1 Why a segmentation head . . . 406
28.2 The head: the decoder of an earlier chapter, smaller . . . 407
28.3 Sigmoid or softmax: one yes/no per class . . . 409
28.4 Binary cross-entropy from the coin flip . . . 410
28.5 Overlap: the Dice coefficient and soft Dice . . . 413
28.6 Why BCE plus Dice: class imbalance with numbers . . . 414
28.7 Measuring it: intersection over union . . . 415
28.8 Labels you cannot fully trust: the known mask . . . 416
28.9 Checking labels when there is no ground truth . . . 418
28.10 Common confusions . . . 422
28.11 Check yourself . . . 423

29 Fine-tuning: the curriculum, the BatchNorm see-saw and mixed batches **424**
29.1 Why a curriculum . . . 424
29.2 The stages and their gates . . . 425
29.3 Starting from F001 . . . 426
29.4 Parameters, buffers and BatchNorm . . . 426
29.5 Mixing sensors by share . . . 428
29.6 The trained-wavelength record . . . 429
29.7 The see-saw and the one BatchNorm . . . 430
29.8 Mixed batches: three band lists, one encoder pass . . . 432
29.9 Joint training: reconstruction and segmentation together . . . 436
29.10 How segmentation was meant to help detection . . . 439
29.11 Common confusions . . . 440
29.12 Check yourself . . . 441

VI From model to answer **442**

30 Inference on a whole scene **443**
30.1 Two problems inference has that training does not . . . 443
30.2 Loading the model . . . 444
30.3 Two-pass masking . . . 445

30.4 The sliding window . . . 449
30.5 Accumulate, then average . . . 452
30.6 Shapes from scene to reconstruction . . . 454
30.7 From reconstruction to score map . . . 455
30.8 One scene through the whole product . . . 456
30.9 Common confusions . . . 458
30.10Check yourself . . . 458

31 From residual to score: residual Mahalanobis, the ring and integral images 460
31.1 Size scores, and the big errors that are not anomalies . . . 461
31.2 A residual is a direction . . . 462
31.3 Whitening: rotate, divide, measure . . . 465
31.4 Regularisation: why only the top 30 directions . . . 467
31.5 The ring: a local mean with a global covariance . . . 469
31.6 Summing a ring fast: integral images . . . 471
31.7 Eleven scores, fixed before the run . . . 475
31.8 Common confusions . . . 478
31.9 Check yourself . . . 479

32 Naming the material, export, and using Indradhanu V2 in a chain 480
32.1 Why a threshold is a person's job . . . 480
32.2 From score maps to an anomaly mask . . . 481
32.3 Naming the material . . . 483
32.4 A redesign: natural first, then the class . . . 485
32.5 Export: where geography comes back . . . 490
32.6 Using Indradhanu V2 in an action chain . . . 492
32.7 Common confusions . . . 495
32.8 Check yourself . . . 495

VII Evaluation 497

33 Detection measures: ROC, pAUC, TPR at a fixed FPR, objects caught and chance 498
33.1 Why a new set of measures was needed . . . 498
33.2 Thresholds and the confusion matrix . . . 499
33.3 Six pixels by hand: the ROC curve . . . 499
33.4 The area under the curve, by the trapezoid rule . . . 500
33.5 AUC is a probability: the Mann–Whitney count . . . 501
33.6 Why AUC is decided by the bulk . . . 503
33.7 Partial AUC up to a false-alarm limit . . . 504
33.8 Exact ROC versus the product's sampled ROC . . . 506
33.9 Detection rate at 0.1% false alarms . . . 507
33.10Objects caught at a fixed budget . . . 507

33.11The chance level of objects caught . . . 508
33.12The 3D-ROC terms, and why they are degenerate here . . . 509
33.13The measures side by side . . . 509
33.14Common confusions . . . 510
33.15Check yourself . . . 510

34 Benchmarks: the public sets and the implant bench (contrast z, foreignness, PSF) 511
34.1 Why a bigger benchmark was needed . . . 511
34.2 What a model needs from each scene . . . 512
34.3 HAD100 . . . 512
34.4 ABU . . . 513
34.5 The implant idea, and what real objects look like . . . 514
34.6 The ring and the contrast z . . . 514
34.7 Linear mixing: what an implant does to a pixel . . . 516
34.8 Solving for the strength on the realised contrast . . . 517
34.9 bench-v1, and how it failed . . . 518
34.10Foreignness . . . 519
34.11The halo and the Gaussian point-spread function . . . 520
34.12bench-v2: solve on the object as it ends up . . . 522
34.13Accepting a benchmark: the ordering test . . . 523
34.14bench-v2.1 . . . 525
34.15Common confusions . . . 525
34.16Check yourself . . . 526

35 Running an evaluation, and what a mask costs 527
35.1 Why one evaluator . . . 527
35.2 One config . . . 528
35.3 The scene-major loop . . . 529
35.4 The cache and its key . . . 530
35.5 Chance, per object, and the report . . . 530
35.6 Stage 3: the numbers held . . . 531
35.7 The go-kart: PCA-RX wins on implants and loses on real objects . . . 531
35.8 Exclusion, and why ROC-AUC falls while the top improves . . . 532
35.9 Scope: cloud excluded, water kept . . . 533
35.10Common confusions . . . 535
35.11Check yourself . . . 535

36 How small a difference can we see: the paired object bootstrap 537
36.1 The problem: 909 objects is not many . . . 537
36.2 Sampling variation and the standard error . . . 538
36.3 The bootstrap idea, by hand . . . 538
36.4 The percentile interval . . . 539

36.5 Why resample objects, not pixels . . . 540
36.6 pAUC rebuilt from per-pixel contributions . . . 541
36.7 The paired design . . . 543
36.8 What it does not cover: only four scenes . . . 544
36.9 The rule, and the intervals so far . . . 544
36.10Common confusions . . . 546
36.11Check yourself . . . 546

VIII The experiments, told as lessons 548

37 The sorting bug that rewrote the scoreboard 549
37.1 What was believed . . . 549
37.2 Linear interpolation, derived . . . 550
37.3 The contract of `np.interp` . . . 551
37.4 Resampling a cube is a matrix . . . 552
37.5 What it did to real PRISMA band lists . . . 555
37.6 What it did to the signature implants . . . 555
37.7 How it hid in plain sight . . . 556
37.8 The cross-check that found it . . . 557
37.9 The scoreboard, read correctly: bench-v2.1 . . . 557
37.10Lessons for trusting a benchmark . . . 560
37.11Common confusions . . . 560
37.12Check yourself . . . 561

38 FT01 to FT05: what each fine-tuning run taught 562
38.1 Where the runs start . . . 562
38.2 FT01: the probe measures what the encoder already knows . . . 563
38.3 FT02: better reconstruction, a gate missed by 0.006 . . . 565
38.4 Edges: why correcting the score map fails . . . 567
38.5 The options not taken . . . 571
38.6 Stage 3: FT03, masks, and why the implants had to change first . . . 573
38.7 FT05: implant-denoising . . . 575
38.8 The scoreboard . . . 579
38.9 FT04: pseudo-labels, rare classes and the release candidate . . . 581
38.10What the runs taught . . . 585
38.11Common confusions . . . 585
38.12Check yourself . . . 586

39 Shipping Indradhanu V2 587
39.1 From a checkpoint to a product . . . 587
39.2 Why the codename had to change . . . 588
39.3 A manifest that carries the whole configuration . . . 589
39.4 One band filter, two kinds of model: the grid check . . . 590

39.5 `read_bands`: tell the model what it is reading . . . 591
39.6 Score first, then zero the cloud . . . 592
39.7 The parity run: the guard . . . 594
39.8 What did not change . . . 596
39.9 Common confusions . . . 596
39.10 Check yourself . . . 596

40 Improving the score on a fixed model, and gated scene adaptation 598
40.1 The question: change the score, not the model . . . 598
40.2 A1: the spatial matched filter . . . 600
40.3 A2: rank fusion with PCA-RX . . . 603
40.4 A3: a mixture of backgrounds . . . 604
40.5 Expectations, a cache bug, and the verdict . . . 606
40.6 Gated scene adaptation . . . 607
40.7 Common confusions . . . 612
40.8 Check yourself . . . 613

41 The uncertainty head and the Laplace likelihood 614
41.1 Where we were, and the gap the score leaves . . . 614
41.2 A probability model for errors: the Laplace distribution . . . 615
41.3 Likelihood, and the negative log-likelihood . . . 617
41.4 The best scale is the error: minimising over b . . . 618
41.5 Predicting $\log b$, and the detached residual . . . 619
41.6 The head itself, counted by hand . . . 620
41.7 The one rule that keeps it honest: hidden pixels only . . . 621
41.8 The score: dividing by b, with a floor . . . 622
41.9 Spearman's rank correlation . . . 622
41.10 The run: FT06 . . . 623
41.11 Why: the head expects large errors exactly where targets are . . . 625
41.12 What might rescue it, and what it taught . . . 627
41.13 Common confusions . . . 627
41.14 Check yourself . . . 628

42 Gated losses, block masking, the decoder bottleneck and FT09 629
42.1 The question behind FT07 and FT08 . . . 629
42.2 Gating the worst 1% . . . 630
42.3 Block masking . . . 632
42.4 FT07 and FT08: results . . . 634
42.5 Is the decoder the bottleneck? A diagnosis without training . . . 636
42.6 FT09: the progressive decoder . . . 640
42.7 The cumulative lesson since FT04 . . . 646
42.8 Common confusions . . . 646
42.9 Check yourself . . . 646

43 Indradhanu Heavy: capacity, a fair control and the haze 648
43.1 Why capacity was the untested question . . . 648
43.2 The architecture . . . 649
43.3 Why parameters grow like width squared . . . 649
43.4 One forward pass of Heavy, at full size . . . 652
43.5 Pretraining on a rented H100 . . . 657
43.6 The curve: did it overfit? . . . 659
43.7 Pretrained detection, and the decision . . . 660
43.8 The fair test: a control and three checkpoints . . . 660
43.9 The mask failure: haze in the thin-cloud head . . . 661
43.10FTH2, FTB2 and FTH3: a gain that holds . . . 663
43.11The lesson: reconstruction did not separate the models . . . 664
43.12Into the product . . . 665
43.13Common confusions . . . 667
43.14Check yourself . . . 668

IX Naming and delivering 669

44 The spectral namer as built 670
44.1 What the namer answers . . . 671
44.2 The architecture: a Siamese, two-tower network . . . 671
44.3 The stem in full, at the namer's shapes . . . 675
44.4 The ring mean . . . 679
44.5 The generator . . . 680
44.6 The class score . . . 683
44.7 Level 1 by matching . . . 685
44.8 One training step, end to end . . . 689
44.9 The loss, the optimiser and the schedule . . . 692
44.10The library, the classes and the splits . . . 693
44.11The test it must pass . . . 695
44.12Inference: naming the pixels of a mask . . . 696
44.13Common confusions . . . 697
44.14Check yourself . . . 697

45 Producing a submission 699
45.1 The rules, and the one bundle that meets them . . . 700
45.2 The PRISMA grid . . . 700
45.3 Native bands by default . . . 701
45.4 Three results per scene . . . 702
45.5 The per-object table . . . 702
45.6 The reflectance writer . . . 703
45.7 Common confusions . . . 703

45.8 Check yourself . . . 704

X A foundation model that is wrong for this job: HyperSL 705

46 What HyperSL is, and what it does well 706
46.1 Why look at an outside model at all . . . 706
46.2 What it is, read from the code . . . 707
46.3 What it was trained on . . . 711
46.4 Did we have the right model? The reproduction . . . 711
46.5 What it is good at . . . 713
46.6 Common confusions . . . 714
46.7 Check yourself . . . 715

47 Per-scene min-max, and a library with nowhere to stand 716
47.1 The scaling, as a formula . . . 716
47.2 A three-pixel scene, by hand . . . 717
47.3 A convolution over band index . . . 720
47.4 What happened when it was tried . . . 722
47.5 What we do instead . . . 723
47.6 Common confusions . . . 724
47.7 Check yourself . . . 724

48 Anomaly detection: a model that completes the anomaly 725
48.1 Two ways to predict a pixel . . . 725
48.2 The measurement on the labelled bench (2026-10-31) . . . 727
48.3 HyperSL as a false-alarm filter . . . 731
48.4 Common confusions . . . 732
48.5 Check yourself . . . 732

49 Unmixing and sub-pixel targets 733
49.1 The words . . . 733
49.2 What was compared . . . 734
49.3 Blind unmixing: the classical method wins where truth is exact . . . 734
49.4 Sub-pixel targets: smoothed away . . . 736
49.5 Cost, and the decision . . . 739
49.6 Common confusions . . . 739
49.7 Check yourself . . . 739

50 Cloud masking 741
50.1 The experiment . . . 741
50.2 The results, in full . . . 742
50.3 Why thin cloud is hard for this model . . . 744
50.4 Common confusions . . . 745

50.5 Check yourself . . . 745

51 The verdict, and what would have to change 746

51.1 The decision table . . . 746
51.2 What it costs . . . 748
51.3 What would have to change . . . 748
51.4 Common confusions . . . 749
51.5 Check yourself . . . 749

A Answers to the Check-yourself exercises 750

Chapter 1: The problem: finding what does not belong . . . 750
Chapter 2: Light, bands and the hyperspectral cube . . . 751
Chapter 3: Five sensors and the 165-band grid . . . 752
Chapter 4: The product: scenes, projects, actions and the vendable . . . 753
Chapter 5: Making sensors comparable: band filtering, interpolation and normalisation 754
Chapter 6: The mathematical toolkit: vectors, projection, covariance, eigenvectors, PCA and whitening . . . 756
Chapter 7: Anomaly as surprise: the Gaussian, Mahalanobis distance and the RX detector . . . 757
Chapter 8: Local RX, regularisation and MNF-RX . . . 758
Chapter 9: Reconstruct, then subtract: why a pixel must never help reconstruct itself 760
Chapter 10: Tensors, shapes, layers and the arithmetic of convolutions . . . 762
Chapter 11: The five convolutional autoencoders . . . 763
Chapter 12: The spectral compressor . . . 764
Chapter 13: Patch embedding . . . 765
Chapter 14: Attention, from one query to many heads . . . 767
Chapter 15: Efficient self-attention with spatial reduction . . . 768
Chapter 16: Mix-FFN, LayerNorm, residuals and the transformer block . . . 769
Chapter 17: The four-stage encoder . . . 771
Chapter 18: Masked autoencoding: validity, targets, remove-and-restore, erosion . . . 772
Chapter 19: The decoder, pixel shuffle and one full forward trace . . . 773
Chapter 20: The parameter budget, counted by hand . . . 774
Chapter 21: Chakshu: the same machine for one thermal band . . . 775
Chapter 22: Why the compressor is a ceiling: cross-attention and a spectrum as tokens 777
Chapter 23: The cross-attention compressor, dead bands and what it costs . . . 778
Chapter 24: The spectral decoder, the traps, per-pixel gating and spectral reduction . 780
Chapter 25: Losses: L1, the spectral angle and the ramp . . . 781
Chapter 26: Training data and the training loop: gradients, AdamW, warmup and cosine schedules . . . 783
Chapter 27: Pretraining on SpectralEarth . . . 785
Chapter 28: The segmentation head, BCE plus Dice, and auditing labels . . . 787
Chapter 29: Fine-tuning: the curriculum, the BatchNorm see-saw and mixed batches . 788
Chapter 30: Inference on a whole scene . . . 789

Chapter 31: From residual to score: residual Mahalanobis, the ring and integral images 791
Chapter 32: Naming the material, export, and using Indradhanu V2 in a chain 792
Chapter 33: Detection measures: ROC, pAUC, TPR at a fixed FPR, objects caught and chance 794
Chapter 34: Benchmarks: the public sets and the implant bench (contrast z, foreignness, PSF) 795
Chapter 35: Running an evaluation, and what a mask costs 797
Chapter 36: How small a difference can we see: the paired object bootstrap 798
Chapter 37: The sorting bug that rewrote the scoreboard 799
Chapter 38: FT01 to FT05: what each fine-tuning run taught 801
Chapter 39: Shipping Indradhanu V2 802
Chapter 40: Improving the score on a fixed model, and gated scene adaptation 803
Chapter 41: The uncertainty head and the Laplace likelihood 804
Chapter 42: Gated losses, block masking, the decoder bottleneck and FT09 806
Chapter 43: Indradhanu Heavy: capacity, a fair control and the haze 807
Chapter 44: The spectral namer as built 808
Chapter 45: Producing a submission 810
Chapter 46: What HyperSL is, and what it does well 811
Chapter 47: Per-scene min-max, and a library with nowhere to stand 811
Chapter 48: Anomaly detection: a model that completes the anomaly 812
Chapter 49: Unmixing and sub-pixel targets 813
Chapter 50: Cloud masking 813
Chapter 51: The verdict, and what would have to change 814

B Glossary **815**

Bibliography **873**

Index **877**

List of worked examples

The “always normal” rule . . . 3
A one-band scene with one hot pixel . . . 5
A reconstruction residual on three numbers . . . 7
Why a fixed cut-off fails . . . 8
A top-20% cut on ten scores . . . 8
An unmasked zero becomes the top anomaly . . . 9
Radiance to reflectance . . . 14
A band as a weighted average of three samples . . . 15
The weights of a 10 nm band . . . 16
Two pixels of the same material . . . 18
Slicing a 3-band, 2-by-2 cube . . . 20
The size of a real cube . . . 20
A Landsat digital number to Celsius . . . 21
Two ways to collapse validity . . . 22
One element, three addresses . . . 25
Reading BIL as if it were BSQ . . . 26
Two reflectance conversions . . . 27
Unpacking two QA values . . . 27
Counting the five windows . . . 30
Which wavelength is band 43, and which band is 1,000 nm? . . . 32
A topological order for one Project . . . 39
When is a job reaped? . . . 40
Both summaries on a 3-band, 2-by-2 validity cube . . . 44
Pixel to map, with a real transform and with the identity . . . 45
Filtering a ten-band toy sensor . . . 50
Kept or wiped? . . . 51
Three bands by hand . . . 52
Resampling one spectrum onto four grid wavelengths . . . 53
The same three samples, listed backwards . . . 54
A spline overshoots a step . . . 55
PCHIP on the same step . . . 56
Normalising the 3-band, 2-by-2 cube . . . 57
Real statistics, band 0 . . . 59
Why the moments are computed in float64 . . . 60

Filling a row and a 3-by-3 patch . . . 62
The dot product of two 3-band spectra . . . 68
The length of a spectrum . . . 68
The angle between two spectra . . . 69
Projection in two dimensions . . . 71
Projecting a pixel onto a reference spectrum . . . 71
A matrix times a vector . . . 72
The transpose of a product, on 2×2 matrices . . . 73
Inverting a 2×2 matrix by hand . . . 74
A covariance matrix from four pixels and two bands . . . 75
Eigenvalues and eigenvectors of the covariance matrix . . . 77
Rebuilding the covariance from its eigenvectors . . . 78
Checking that the first principal direction wins . . . 80
Turning the four pixels onto their principal directions . . . 80
Whitening the four pixels . . . 81
Per-band z-scores miss the pattern . . . 88
Two ways to the same Mahalanobis distance . . . 90
RX by hand on three bands . . . 94
The same score through the eigenvectors . . . 96
A mixture hides an anomaly from the global mean . . . 96
Two highly correlated bands . . . 97
Loading the correlated pair . . . 99
The default ring, and the ring at an edge . . . 103
A 3×3 target with and without a guard . . . 104
The masked mean must divide by the real count . . . 106
How much arithmetic one PRISMA scene costs . . . 107
MNF against PCA on two bands . . . 109
A noise covariance from a strip of five pixels . . . 110
Eigenvalues of PCA and MNF on a synthetic cube . . . 111
PCA-RX on a tiny whitened patch . . . 113
A three-band residual and its score . . . 119
Leave-one-out prediction of six pixels . . . 122
The self-including average hides a third of the anomaly . . . 122
Mean versus median when one value is an anomaly . . . 125
Tracing a checkerboard through two passes . . . 128
Finding a value in memory . . . 135
A $2 \times 2 \times 2$ cube through flatten, transpose and a wrong reshape . . . 136
A linear layer from three inputs to two outputs . . . 137
A 3×3 input and a 2×2 kernel, by hand . . . 138
Every stage of Indradhanu's encoder . . . 141
Why stage 1 must set its padding explicitly . . . 141
The convolutional autoencoder and a 100×100 patch . . . 142

Receptive field of the four patch embeddings . . . 142
The SpatialEncoder's 165,024 parameters . . . 144
A transposed convolution in one dimension . . . 148
Four bands into two . . . 156
BatchNorm on four values . . . 157
One training step of running statistics, then evaluation . . . 158
A 3-to-2-to-3 bottleneck and what it loses . . . 160
The spectral bottleneck's 10,821 parameters . . . 161
Four tokens from a 4×4 image . . . 167
LayerNorm on a four-number token . . . 168
How many windows see each pixel . . . 171
All four patch embeddings . . . 172
Three dot products . . . 176
Softmax of three scores . . . 177
The stable softmax . . . 178
Gradients of a healthy and a saturated softmax . . . 179
One query, three tokens of dimension 2 . . . 180
Full attention on three tokens, identity projections . . . 182
Full attention with real projections . . . 183
The variance of a 2-dimensional dot product, exactly . . . 186
Swapping two tokens . . . 187
Splitting a 4-number token into two heads . . . 188
The projection parameters at every stage . . . 190
The cost of plain attention at stage 1 . . . 194
The memory of one stage-1 score tensor . . . 194
Summarising a 4×4 grid with $R = 2$ . . . 196
One query against the four summary tokens . . . 197
The cost of reduced attention at stage 1 . . . 200
The stage-1 score tensor, reduced . . . 200
The stage-1 attention module, parameter by parameter . . . 201
Is the fallback affordable? . . . 202
LayerNorm of a 4-number token . . . 205
Twenty layers with and without residuals . . . 208
Dropout with $p = 0.4$ . . . 209
Ordinary against depthwise at stage 1 . . . 211
A depthwise filter sees the border . . . 211
GELU at -1, 0 and 1 . . . 212
Mix-FFN at stage 1 . . . 213
A stage-1 block: 79,616 parameters . . . 215
A stage-4 block: 800,000 parameters . . . 215
The four grid sizes from the formula . . . 219
Receptive fields of the four stages . . . 223

Token validity on a 4 × 4 pixel grid with 2 × 2 tokens . . . 231
Six tokens at $\rho = 0.65$: what the rounding does . . . 233
Hiding three of six tokens by hand . . . 234
Building ids_restore for six tokens . . . 235
The course example: two invalid tokens and the +2 . . . 235
The selection matrix for three kept tokens of six . . . 237
Remove and restore on six tokens . . . 237
Leakage in a batch of two . . . 238
The loss mask on the 4 × 4 example . . . 240
Erosion in one dimension, kernel 5 . . . 241
Erosion in two dimensions, kernel 3 . . . 241
Counting the graded pixels in Figure 18.5 . . . 244
A 1 × 1 projection at one position . . . 247
Upsampling a row of two values to four . . . 248
Bilinear upsampling of a 2 × 2 grid to 4 × 4 . . . 249
Pixel shuffle by hand, and PyTorch's order . . . 251
A one-pixel anomaly, two ways . . . 252
Denormalising one value . . . 253
The rules on a tiny convolution . . . 258
The four stages from (20.4) . . . 259
Where the manifest's 5,507,354 comes from . . . 262
Multiply-adds of three layers . . . 263
Two Landsat pixels into degrees . . . 270
Normalising the two pixels, and an invalid one . . . 270
How many tokens a training pass hides . . . 272
Every stage attends to 64 keys . . . 273
Chakshu's four blocks . . . 274
Trimming 20 errors at 5% . . . 276
Tiles over two scene sizes . . . 277
A HotSat number under the baked statistics . . . 279
Rank and null space of the toy compressor . . . 286
A dead band filled with zero . . . 287
One query reading three tokens . . . 291
Stretching one band value . . . 294
The tag of two wavelengths at one period . . . 296
Cosine similarity of 660 and 670 nm, all sixteen periods . . . 297
Building one band token . . . 298
The cross-attention compressor's parameters by hand . . . 305
One channel of a one-head compressor . . . 307
Masking a dead band among four . . . 308
Counting the compressor's multiply-adds . . . 311
Bytes of attention for one patch . . . 313

The cross-attention decoder's parameters by hand . . . 320
A per-pixel L1 over valid bands only . . . 322
Reading the mean curve at 864 nm . . . 324
Four values of the sigmoid . . . 326
Gating the toy pixel . . . 327
The gate's parameters at 202 bands . . . 328
Counting the bins and placing three bands . . . 330
A dip and its mirror image collide . . . 332
Fix 1 separates the dip from its mirror . . . 332
The skip path on the toy pixel . . . 333
The skip path with a dead band holding 99 . . . 334
Decoding one band . . . 335
The reduced compressor, 14,721 parameters . . . 337
The wavelength-linear decoder, 4,257 parameters . . . 337
Reading the throughput table . . . 339
Masked L1 on three bands and two graded pixels . . . 347
The gradient of L1 for the two-pixel example . . . 348
A glitch pulls the mean, not the median . . . 349
The spectral angle of pixel A, by hand . . . 352
A reconstruction twice as bright . . . 353
The chain rule on a tiny function . . . 354
The gradient of the angle for pixel A . . . 357
The naive formula producing NaN . . . 358
The atan2 form on pixel A, and on a perfect reconstruction . . . 359
The v0.2.0 ramp, epoch by epoch . . . 362
The combined loss at the configuration's epoch-100 baseline . . . 363
Three steps of gradient descent . . . 367
Three steps with momentum . . . 368
The bias-correction factors for t = 1, 2, 3 . . . 370
Three steps of Adam on the toy loss . . . 370
The same decay, two different shrinks . . . 372
The v0.2.0 schedule at five epochs . . . 373
Clipping a 3-vector . . . 375
Why the loss stays in float32 . . . 376
The row coordinates of a 300-pixel scene . . . 378
Why the averaging is weighted . . . 382
The size of one patch . . . 386
The same model scored two ways . . . 388
One spectrum, two sensors, two answers . . . 391
The toy again, with widths . . . 394
Coverage per epoch and per run . . . 395
One pass of the driver after a crash . . . 397

From a PRISMA digital number to a stored value . . . 401
Delta-encoding one pixel . . . 402
Aligning a sample onto a band list . . . 403
The head's parameter count . . . 408
Why the softmax cannot say "water and thin cloud" . . . 410
BCE on three pixels . . . 411
Soft Dice on a 2×2 mask . . . 413
A rare class and a common one under BCE and Dice . . . 414
IoU of two pixel sets, and why the tally pools . . . 415
The known-mask loss on a 2×2 tile . . . 417
Four spectra through the cloud test . . . 419
Why the curve is frozen . . . 426
A "frozen" encoder that drifts . . . 428
Growing the record . . . 429
The end of an epoch moves the stored mean . . . 431
Alternating sensors never settle; mixed batches do . . . 435
Where the running mean settles with an EnMAP-only second pass . . . 436
Pooled against per-class scoring on a coastal scene . . . 439
Selecting from two passes on four tokens . . . 446
The checkerboard on a 4×4 token grid . . . 447
Keep mask for a row of four tokens with one invalid . . . 448
Tiles along one axis of a 1000-pixel scene . . . 450
A real PRISMA frame, 1210×1219 . . . 450
Averaging three tiles, one of them eroded . . . 453
The combined score and its fragile denominator . . . 455
The funnel from 1.47 million pixels to a shortlist . . . 457
Residual Mahalanobis on three bands . . . 463
The same scores by rotate and divide . . . 466
Building an integral image on a 4×4 grid . . . 471
A box sum and a ring mean from the table . . . 473
A local covariance from five box sums . . . 474
A two-model composite at one pixel . . . 481
Flagging the top 30% of ten pixels, and the top 0.5% of a scene . . . 482
SAM names the grass around a roof . . . 484
The product of the two levels . . . 487
Near the natural average, off the natural line . . . 487
ACE on three bands, at two fills . . . 488
An axis-aligned transform from a bounding box (the fallback) . . . 491
The excluded percentage . . . 494
The ROC of six pixels . . . 499
The AUC of the six pixels, strip by strip . . . 500
The six pixels, counted as pairs . . . 502

A tie, both ways . . . 502
Two detectors whose AUC and top-of-list disagree . . . 503
Partial AUC of the six pixels, with a limit that cuts a segment . . . 505
Chance for four object shapes at the top 1% . . . 508
Top-of-atmosphere reflectance . . . 512
The test's z, by hand . . . 515
A pixel on the mixing line . . . 516
One band: the simple formula misses, the realised solve hits . . . 518
Foreignness of $(3, 4, 0)$ . . . 520
Matching $(4, 0, 2)$ to foreignness 0.1 . . . 520
The PSF weights and the abundance map on a 3×3 footprint . . . 521
An unforeign signal on a foreign spot . . . 523
Counting the work of stage 3 . . . 529
A bench's chance row . . . 531
Exclusion in miniature . . . 532
Two water rules on one turbid pixel . . . 533
Bootstrapping the mean of five numbers . . . 539
Percentiles of the six bootstrap means . . . 539
An effective sample size . . . 541
A tiny scene, both ways . . . 542
Five objects, paired and unpaired . . . 543
Two paired draws on the tiny scene . . . 543
Interpolating the repo's own test spectrum . . . 551
The same three samples, listed the other way round . . . 552
One row of the resample matrix, right and wrong . . . 553
The sorted resample, by hand . . . 554
A signature flattened . . . 556
Objects the bug hid . . . 557
How much of v0.2.0's jump is the fix? . . . 558
Is 35.5% against 34.9% a difference? . . . 559
The selection effect behind the easy tier . . . 560
The segmentation head's parameters . . . 563
Reading FT02's gate . . . 565
Two edge measures on a 3×3 patch . . . 567
Rank minus the median rank of its bin . . . 568
The same blur, two losses . . . 570
The repo's `match_foreignness` test case . . . 575
Which prediction does each target reward? . . . 575
Times five moves 0.15% to 0.76% . . . 576
Right singular vectors are eigenvectors of $\mathbf{X}^\top\mathbf{X}$ . . . 577
Criterion 1 at epoch 5 . . . 578
Four pixels through `pseudo_targets` . . . 581

Weighted BCE on a batch with no snow . . . 582
FT04's criteria . . . 583
Main effects of model and mask . . . 583
Which manifest wins "Indradhanu"? . . . 588
Where 5,886,451 comes from . . . 590
How close is "on the grid"? . . . 591
Three bands against a record ending 2395, 2405, 2440 . . . 592
One pixel, two cloud rules . . . 593
The same pixel, twice the score . . . 593
excluded_percent on a small tile . . . 594
A two-band matched filter . . . 601
The promise and its price, on a 3-tap kernel . . . 601
A strip of 12 pixels, plain and blurred . . . 602
Rank mean and rank max on five pixels . . . 603
Responsibilities and a weighted score in one band . . . 605
k-means on six numbers . . . 605
Targets that become their type's normal . . . 606
Growth of a gate under two passes . . . 608
Leave-one-scene-out on the real numbers . . . 610
The loss at three scales . . . 617
The derivative at three points . . . 619
Four pixels, one scale . . . 619
Gradients with respect to b and to $\log b$ . . . 620
The uncertainty head's 95,632 parameters . . . 621
The floor at work . . . 622
Spearman on five pairs . . . 623
How often a halo leaks out of its token . . . 625
Dividing by b, by hand . . . 626
The top 1% of 100 loss pixels, with its halo . . . 630
Loss share against gradient share . . . 631
An upper bound on a real batch . . . 632
A 5 px footprint on a 4 px and an 8 px grid . . . 632
How much a 128 px tile hides . . . 633
Half of each block taken from a pass that saw it . . . 634
FT04's decoder, 2,164,736 parameters . . . 637
The floor with 3 bands and 1 curve . . . 637
Two pixels through one number . . . 640
The progressive decoder's 1,504,096 parameters . . . 641
Set1's contrast change . . . 643
Reading an interval as a margin . . . 644
Stage-1 block, FT04 and Heavy . . . 650
Heavy's progressive decoder, 4,279,616 parameters . . . 652

What the masked stage 1 costs and holds . . . 653
Dealing shards to workers . . . 657
The learning rate at epochs 40 and 60 . . . 658
Where the top-0.1% loss sits . . . 660
Switching off a scene's targets . . . 662
Mean gain over FT04 at three checkpoints . . . 664
Parity, to the digit . . . 665
Four times faster from eight times the threads . . . 666
The cost of a new library . . . 674
Counting the namer's parameters . . . 674
Normalising one band . . . 675
Three entries of enc(1000 nm) . . . 676
Widths of four bands, and of a band beside a water gap . . . 676
Tokens by the matrix product, on paper . . . 677
One query, three bins, one empty . . . 678
One channel through the frozen BatchNorm . . . 679
A ring that ignores the halo . . . 680
How much training goes to small fills . . . 682
Noise from four pixels . . . 682
Planting one pixel . . . 683
Recognising the sample, or naming the material . . . 684
Who gets the pull . . . 685
Level 1 by matching, three library entries . . . 687
Four points no line can split . . . 688
From one row of **S** to both levels . . . 689
The schedule at a few steps . . . 693
Two metals, two sides . . . 694
Recall at a fixed false-alarm rate . . . 695
Set-1's grid by hand . . . 701
Two objects or three . . . 702
Writing five values . . . 703
The wavelength code at 450 nm . . . 709
HyperSL's 9,027,841 trainable parameters . . . 709
How much does pretraining buy? . . . 713
Scaling scene S_1 . . . 717
The same spectrum in scene S_2 . . . 717
Scene S_1 after a bright pixel arrives . . . 718
The angle between $\mathbf{p}_2$ and $\mathbf{p}_3$, before and after . . . 719
A 5-band window on PRISMA's 149-band list . . . 720
Self-completion of band 2 . . . 726
Spatial completion of the same pixels . . . 726
How many bands should never be hidden in 4 draws? . . . 728

Reading the bootstrap rows 730
What 0.069 against 0.199 radians means 736
One target pixel at fill a, averaged over 3×3 737
Why a TV penalty sets small targets to zero 738
Cirrus recall and precision, from the tables 743
One cirrus layer, two scenes 744

List of Figures

1 The chapter dependency map. Each box is a chapter number; an arrow means "read this first". Inside a part, read in order. The long arrows show the few places where a later part leans on a specific earlier chapter: the linear algebra of Chapter 6 under the tensor arithmetic of Chapter 10, the local detectors of Chapter 8 under the residual Mahalanobis score of Chapter 31, and the fine-tuning machinery of Chapter 29 under the experiment chapters. Part X leans on the namer (Chapter 44) and, through it, on the reconstruction, masking, spectral-stem and evaluation chapters. xliv

1.1 Reconstruct, then subtract. Blocks of the observed cube are hidden and the model, trained only on ordinary scenes, redraws them from their surroundings. Subtracting the reconstruction from the observation, band by band, gives a per-pixel error map; where the redrawing is most wrong (amber) is where something unusual is. The amber square stands for an object the model has never seen. (Course figure, Indradhanu part 1.) 7

2.1 A Gaussian spectral response function with FWHM 10 nm, drawn from Equation (2.7). The response falls to one half exactly 5 nm either side of the centre; the three amber points are the samples used in worked example 2.2. 17

2.2 The mean reflectance of each of the 165 common-grid bands over the normalisation corpus (teal line), with one standard deviation either side (shaded). Real numbers from `app/constants/hyperspectral.json`, plotted at the grid wavelengths; the four breaks are the atmospheric windows removed from the grid. 19

2.3 From photograph to spectrum. (a) A colour photograph stores three bands per pixel. (b) A hyperspectral cube stores one image per wavelength, with shape `(C, H, W)`. (c) Reading one pixel down the band axis gives its spectrum. (Course figure, Indradhanu part 2; the curve is illustrative.) 19

3.1 The common grid drawn to scale. Ticks are the 200 ladder rungs from 460 to 2,450 nm; grey rungs fall inside the hatched atmospheric windows and are removed ($6 + 2 + 11 + 16 = 35$), teal rungs are kept. Braces give the number of kept bands in each of the five segments ($46 + 16 + 19 + 34 + 50 = 165$). The window 0–450 nm lies below the ladder and removes nothing. 32

3.2 The same grid as drawn in the Indradhanu course (part 3): a ladder of 200 wavelengths loses 35 rungs to four atmospheric windows, leaving 165 bands in five segments of 46, 16, 19, 34 and 50. 32

4.1 The shipped Action graph, edges read from each Action type's declared inputs. Each box writes its outputs (monospace) to its own folder. `anomaly_scoring` has two possible parents besides the optional annotation, so the structure is a graph, not a chain; Indradhanu is one model ticked *inside* `anomaly_scoring`. Export re-reads the raw scene file to recover map coordinates. 38

4.2 The same chain as drawn in the Indradhanu course (part 5), emphasising that Indradhanu is one option inside the `anomaly_scoring` Action, not an Action of its own. 38

4.3 The hyperspectral vendable as a data structure: two arrays of identical shape (the reflectance cube and its validity cube) and five lists with one entry per band. There is no field for map coordinates. 43

4.4 How a vendable reaches Indradhanu (course figure, Indradhanu part 4). Of the filtered vendable's fields the model uses the cube and the validity cube; a pixel is spatially valid if it is valid in at least one band. 44

5.1 Resampling one spectrum onto the grid (worked example 5.4). Native samples (teal) at 1,003, 1,011, 1,019, 1,027 nm; grid values (amber) at 1,000, 1,010, 1,020, 1,030 nm, each read off the straight line between its two neighbours. Below the first sample the value is held constant; above the last, the code extends the last line. 54

5.2 Why each band is normalised separately (course figure, Indradhanu part 9). (a) Raw reflectance puts a dim blue band and a bright near-infrared band on very different scales. (b) Subtracting each band's mean and dividing by its standard deviation puts every band on one scale, centred at 0 with spread 1. Distributions are illustrative. 61

6.1 Projecting $\mathbf{x} = (3, 1)$ onto the direction $\mathbf{u} = (1, 1)$. The closest point of the line $t\mathbf{u}$ is $\mathbf{p} = 2\mathbf{u} = (2, 2)$; the residual $\mathbf{r}$ meets the line at a right angle. The grey segments go to two other points of the line, which are further away. 70

6.2 Before and after whitening. (a) Sixty points drawn from a Gaussian with the covariance $\begin{pmatrix} 2 & 33 \\ 10 & \end{pmatrix}$ of worked example 6.9 (grey; an illustrative sample, seed 0, `tools/compute_linalg.py`) and the four worked pixels (teal). The ellipses are the points at whitened distance 1 and 2; the arrows are the eigenvectors scaled by the square roots of their eigenvalues, 3.32 and 1. (b) The same points after $\mathbf{z} = \mathbf{W}\mathbf{d}$: the ellipses have become circles, and the eigenvectors the coordinate axes. 82

7.1 The standard normal density. The light and dark tails together hold 1% of the area (beyond ± 2.576); the dark tails alone hold 0.1% (beyond ± 3.291). They are thin, so the right panel magnifies the grey box on the right tail. The thermal pixel of Section 7.1 sits at $z = 3.42$, beyond both thresholds. 87

7.2 Chi-square densities for $C = 1, 2, 3, 10$ degrees of freedom. The right panel magnifies the grey box: its shaded tails hold 1% of the area for $C = 3$ (beyond 11.34) and for $C = 10$ (beyond 23.21). As C grows the whole distribution moves right (mean C) and spreads (standard deviation $\sqrt{2C}$), so a score of 15 is alarming in 3 dimensions and ordinary in 10. 93

8.1 A dual window with outer half-size $o = 4$ and inner half-size $i = 1$. The background is the 9×9 outer square minus the 3×3 guard: $81 - 9 = 72$ pixels. The product's defaults, $o = 25$ and $i = 5$, give a 51×51 square and an 11×11 guard. 103

9.1 The six-pixel signal of Equation (9.11), its leave-one-out predictions and the residuals, drawn to scale. The anomaly at pixel 4 gets the largest residual; its two neighbours get a halo of residual because their predictions used pixel 4. 123

9.2 Why the model must not see what it predicts. (a) Asked to reproduce what it can see, a model learns to copy; the residual is zero everywhere and nothing is detected. (b) A masked autoencoder hides about 65% of the tokens (dashed) and is graded only on them; it must predict from context, so at the hidden anomaly (amber) it paints ordinary background and leaves a large error. (Course figure IW 6.1.) 127

9.3 The Indradhanu forward pass 130

10.1 The two layouts of the same numbers. (a) Spatial `(B, C, H, W)`: a grid of positions per channel, as convolutions need. (b) `flatten(2)` joins the two spatial axes into $N = H \times W$ positions. (c) `transpose(1, 2)` makes each position a row of C numbers, a token, as attention and LayerNorm need. A reshape alone would scramble the order. (Course figure IW 7.1.) 135

10.2 A 3×3 kernel sliding with stride 2 over a 4×4 input padded by one ring of zeros, the setting of encoder stages 2 to 4. Consecutive windows share a column (the overlap is $k - s = 1$). The corner windows see only 4 real pixels; the bottom-right one sees 9. 140

10.3 A 1×1 convolution is a matrix-vector product per pixel. At every pixel the 4-band vector $\mathbf{x}$ is multiplied by the same 2×4 matrix $\mathbf{W}$, giving two output channels, $y_0 = 1.1$ and $y_1 = -0.5$ (the numbers of worked example 12.1). The spatial grid is untouched; only the channel axis changes. (Course figure IW 8.1.) 144

12.1 The spectral bottleneck around the SegFormer. A 1×1 convolution with BatchNorm mixes each pixel's 165 normalised bands into 32 channels (5,376 parameters); the spatial model runs on those 32; a second 1×1 convolution, without normalisation, expands them back to 165 (5,445 parameters). (Course figure IW 10.1.) 155

13.1 Turning pixels into tokens. (a) At stage 1 a `Conv2d` with kernel 4 and stride 4 summarises each 4×4 block of the 32-channel input as one 32-number token (amber: one block and its token), giving a 32×32 token grid that is flattened to 1,024 tokens. (b) Stage 1's windows tile exactly, so each pixel feeds one token, which hiding requires; stages 2 to 4 use kernel 3, stride 2, padding 1, whose windows overlap by one. (Course figure IW 11.1.) 170

14.1 The attention matrix $\mathbf{A}$ of worked example 14.6, drawn as a heat map (darker plum = larger weight). Row i is how query i shares its attention among the three keys; every row sums to 1, columns need not. Token 3, which resembles both others, puts half its attention on itself. 184

14.2 The same example as the course draws it: (a) three 2-number tokens with $\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$; (b) scaled scores $\mathbf{q} \cdot \mathbf{k}/\sqrt{2}$; (c) softmax along each row; (d) each output is the weighted average of the values. Every number matches worked example 14.6. 184

14.3 Where the questions come from. (a) In self-attention the tokens produce queries, keys and values. (b) In cross-attention an outside set of queries reads the tokens' keys and values. Right: the two score matrices for the spectral case of Chapter 22, drawn to scale. 190

15.1 The toy reduction of worked example 15.3 with the averaging kernel. Each output token summarises one $R \times R$ block of the input (outlined); the queries are still made from all 16 input tokens. 196

15.2 Spatial reduction at stage 1, as the course draws it. Queries keep all 1,024 tokens; keys and values are first shrunk by a strided convolution (kernel = stride = $R = 8$) to a 4×4 grid of 16 summary tokens. The score matrix is 1024×16 instead of 1024×1024, 64 times smaller (the hatched square is not drawn to scale). With $R = 8, 4, 2, 1$ every stage attends over the same 4×4 summary. 198

15.3 Score entries per head per patch, plain against spatially reduced, on a logarithmic axis (each gridline is ten times the one below). The reduction factor R^2 is 64, 16, 4 and 1. 199

16.1 The feed-forward half of a block. Tokens are expanded fourfold, reshaped back to their grid, mixed with their eight neighbours by a depthwise 3×3 convolution (one private filter per channel), passed through GELU and projected back. The depthwise convolution is what gives the model a sense of position. 210

16.2 GELU against ReLU. GELU is a smooth ReLU: near zero for very negative inputs, a soft bend through the origin, near-linear for large inputs. The curve is drawn with the tanh approximation, which differs from the exact form by less than 0.001 on this range. 213

16.3 One SegFormer block. Each of the two sublayers, efficient self-attention (plum) and Mix-FFN (indigo), normalises its input first (pre-norm), transforms it, applies dropout and adds the result back to its input through a residual connection. The shape (B, N, C) is unchanged; each encoder stage stacks two blocks. 214

17.1 The encoder pyramid drawn to scale. Each square covers the same 128×128 pixels of ground; its grid lines are the stage's tokens (stage 1: one token per 4×4 pixels, stage 4: one per 32×32). The bar under each square is the token width, drawn at one scale (256 channels = the full square width). Grey cells in F1 are hidden tokens at a 65% hiding rate (an illustrative pattern); only F1 has them. Arrows are patch embeddings (stride 2, kernel 3). 221

17.2 The course's view of the same pyramid: each patch embedding (arrow) shrinks the grid, by 4 at stage 1 and by 2 after, and widens the channels, producing F1 to F4 at 1/4 to 1/32 of the input resolution. Hidden tokens (hatched) exist at stage 1 only. Front-face size and depth are compressed for drawing; Figure 17.1 is to scale. 221

18.1 Token validity on an 8×8 pixel mask with the real stage-1 geometry (kernel 4, stride 4). (a) The pixel mask split into four 4×4 blocks. (b) Average pooling gives each block's valid fraction: 16/16, 4/16, 12/16, 8/16. (c) The strict threshold > 0.5 gives the token mask $[1, 0, 1, 0]$; the exactly-half block is invalid. 231

18.2 The worked example of worked example 18.5 as one picture. Invalid tokens (hatched) get +2 so they sort to the back. After an ascending sort the first $\lfloor 4 \times 0.5 \rfloor = 2$ places (amber) are flagged and scattered back: tokens 0 and 4 are hidden, and the keep mask is the exact complement of the prediction mask. . . . 236

18.3 Hidden tokens are removed, not zeroed. The gather keeps only the visible tokens (here 4 of 6), so the stage-1 blocks never see the hidden ones; after the blocks the scatter writes the encoded tokens back into a zero tensor at their original positions. 239

18.4 Erosion as a max pool, with the 1-D example of worked example 18.10 (kernel 5). Flipping makes invalid pixels 1; a stride-1 max pool grows that region by $\lfloor 5/2 \rfloor = 2$ pixels on each side; flipping back leaves the valid region shrunk by 2 pixels per edge. 243

18.5 The four masks on one 16×16 patch with 4×4 tokens, in the book's colours (indigo spatial tokens, amber loss, grey invalid). A cloud (dashed) covers rows 10–15, columns 11–15. (a) Tokens 11 and 15 are invalid (valid fractions 8/16 and 0/16); the other 14 are valid and $\lfloor 0.65 \times 14 \rfloor = 9$ of them are hidden (an illustrative draw). (b) The loss mask with erosion radius 1 (kernel 3): hidden pixels count only when valid and not next to the cloud. Worked in worked example 18.12. . . 243

19.1 The decoder. The four encoder maps are projected to 256 channels by 1×1 convolutions, resized to F1's 32×32, concatenated to 1,024 channels, fused and refined, and expanded to $512 = 32 \times 4 \times 4$ channels. PixelShuffle(4) rearranges each group of 16 channels into a 4×4 block of pixels (inset, with $r = 2$), returning `(B, 32, 128, 128)` without any arithmetic. 252

19.2 The seven steps of `forward()` for a batch of four 128×128 patches. Spectral steps are teal, spatial steps indigo. Step 4 expands into the four encoder stages (hiding at stage 1 only). The model stops at the reconstruction; the trainer computes the loss (amber) in raw reflectance. 255

20.1 Where Indradhanu's 5,506,629 parameters live, drawn to scale from the counts of this chapter. Spectral layers teal, encoder indigo, decoder plum. Stage 4's two blocks (16 tokens) hold more than any other component; the decoder's two refine convolutions hold 1,770,240 between them; the spectral compressor and decompressor are invisible at this scale (0.2% together). 262

20.2 The course's view of the same budget by component, bar widths to scale: the blocks (51.4%) and decoder (39.3%) carry almost all of it. 263

21.1 Chakshu's forward pass on a masked batch of eight 256×256 Landsat patches, with the shape after each box (recorded by forward hooks on the real model [C]). Blue: encoder; plum: decoder; amber: the two ends, in degrees Celsius. The chapters on the left explain each box. 274

21.2 Where Chakshu's 406,496 parameters live, drawn to scale from worked example 21.5 and the sums above (blue encoder, plum decoder). The largest single layer is the decoder's refine convolution, 20% of the model; the stage-4 patch embedding (55,584) alone outweighs the first three blocks together (stage-end norms, 416, not drawn). 276

21.3 An 8×8-pixel window around the 3×3 spot of Section 21.8.2. Thick lines are stage-1 token borders. In pass 1 the grey tokens are hidden, but the two teal tokens still show four of the spot's nine hot pixels to the encoder, so the hidden hot pixels are predicted warm; pass 2 swaps the colours. 279

22.1 The ceiling of a fixed compressor. (a) Every pixel – forest, desert or shadow – is mixed by the same trained matrix $\mathbf{W}$. (b) A dead band filled with 0.0 still receives its weight: with the toy numbers the output moves from -0.50 to -0.30 (worked example 22.2). 289

22.2 Shapes in cross-attention. The token count N is an inner size in both products, so it never reaches the output: M queries always give M answers. 290

22.3 Where the questions come from, and what it costs. (a) In self-attention the band tokens produce queries, keys and values. (b) In cross-attention an outside set of D queries reads the tokens' keys and values. Right: score matrices for one pixel and head with 202 bands, drawn to scale: 40,804 against 6,464 scores. 293

22.4 The cosine similarity of two wavelength tags, Equation (22.11), plotted exactly from the sixteen periods of the source. It starts at 1 for $\Delta = 0$, is still about 0.6 at 20–30 nm, and beyond about 100 nm it fluctuates around a mean of about 0.06, never above 0.49 nor below -0.35: tags of distant wavelengths are close to orthogonal. The two marked points are worked example 22.6. 298

22.5 Building one band token. (a) The band's value is stretched to d numbers by `Linear(1, d)` and added to a fixed sinusoidal tag of its wavelength. (b) The real 32-number tag for three wavelengths (teal positive, amber negative, shade = magnitude): 660 and 670 nm share most of their pattern (cosine similarity 0.671), 660 and 1650 nm almost none (0.013). 299

23.1 The per-band cross-attention compressor for one pixel. Each of 202 bands becomes a token; keys and values are projected from the tokens; 32 learned queries, shared by every pixel, attend over them with 4 heads, and each answer becomes one output channel. None of the 5,345 parameters depends on the band count. . 303

23.2 Minus infinity before softmax. Left: three valid scores $[2, 1, 0]$ give weights $[0.6652, 0.2447, 0.0900]$. Middle: marking band 3 dead sets its score to $-\infty$, its weight to exactly 0, and the other two renormalise to $[0.7311, 0.2689]$. Right: the same mask lets a 202-band SpectralEarth pixel share a batch with 239-band PRISMA pixels, its 37 padding tokens masked out. 309

23.3 Why training, not inference, hits the wall. (a) For backpropagation the per-band stem keeps about 19 intermediates of shape (16,384 pixels, 202 bands, 32), 202 MiB each in bf16: about 3.75 GiB per 128×128 patch, matching the measured 3.7 GiB. (b) The reduced stem of Chapter 24 never builds a band-by-32 tensor; its largest per-pixel tensor has 53 bin slots, 53 MiB. 314

23.4 Memory per patch of the quantities computed in this section, to scale (linear axis). All are this book's arithmetic from the shapes; the training bar is 19×202 MiB, matching the measured 3.7 GiB. The self-attention bar, 9.96 GiB, is why the design is cross-attention. 315

24.1 The decoder turned around. The pixel's 32 latent channels, each stretched and given a learned identity, supply keys and values; every requested wavelength becomes a query through its tag. Attention and a final Linear(32, 1) produce one reflectance per requested wavelength, so the same 6,401 parameters reconstruct any band list. 320

24.2 The trace in one row. The new spectral parts (plum) and wavelength normalisation work strictly inside each pixel; hiding, encoder and decoder are the unchanged spatial model; the loss counts valid bands only. Because hidden pixels are removed before any spatial attention, a pixel cannot rebuild its own spectrum. 323

24.3 Variant B. A small network reads each pixel's own spectrum and emits one score per band; sigmoid turns the scores into gates between 0 and 1, the bands are scaled by them, and today's fixed 1×1 compressor mixes the result. Each pixel therefore gets the recipe $\mathbf{W}\,\mathrm{diag}(\mathbf{g})$, today's $\mathbf{W}$ with rescaled columns, at three times today's arithmetic. 328

24.4 SpectralEarth's 202 bands counted into the 53 fixed 40 nm bins, to scale, from the real wavelength table (this book's script). Grey crosses mark the 11 empty bins, masked out of attention. The two bins of 9 (920–1000 nm) are where EnMAP's two detectors overlap. 331

24.5 The fixed bin grid as the course draws it. The 400 to 2500 nm axis is cut into 53 bins of 40 nm (top); each tick below is one of SpectralEarth's 202 band centres, and the bar height is the number of bands in that bin (1 to 9). 42 bins hold at least one band; the 11 empty bins (hatched) fall in the two water-vapour gaps and at the long end. 331

24.6 The reduced compressor for one pixel. Left: band values (teal) are masked by validity (grey hatching marks a dead band) and multiplied by a per-band weight vector $k(\lambda)$ computed from each band's wavelength; the products are averaged within each 40 nm bin and given the bin centre's tag, giving about 42 bin tokens. Centre: 32 learned queries attend over the bin tokens (plum), empty bins masked, and each answer becomes one channel. Bottom: the skip path multiplies every valid band by $W(\lambda)$ and averages, the 1×1 compressor with wavelength-computed columns. The two halves add into the 32-channel latent z. 335

24.7 The wavelength-linear decoder. Each requested wavelength (any list: 202 SpectralEarth, 224 EnMAP, 239 PRISMA bands) passes through one small network that emits a 32-number weight row $U(\lambda)$ and a bias $c(\lambda)$; the band's value is the dot product of that row with the pixel's latent z, plus the bias. No attention and no per-band stored weights: the same 4,257 parameters serve every band list. . . 336

24.8 The L40S measurements of the drift box, as bars on linear axes (to scale). Left: training throughput; the per-band stem is the short plum bars, the reduced stem the dark plum. Right: peak memory. "OOM": the per-band stem ran out of memory at batch 32. 339

24.9 The same measurements as the course draws them. Left: patches per second against batch size (log scale). Right: peak memory. The per-band cross-attention stem runs out of memory at batch 32; the reduced stem is about six times faster at each stem's best batch and needs about a quarter of the memory per added patch. 339

25.1 Which pixels are graded. The pixel prediction mask (hidden tokens, amber) is multiplied by the eroded validity mask (indigo; the cloud and its collar hatched). Multiplying 0/1 masks is a logical AND, so only pixels that were hidden, valid and away from invalid regions enter the loss. Right: the L1 term for one graded pixel. 346

25.2 Two complementary errors. (a) Treating a spectrum as a vector, the spectral angle is the angle between reconstruction and truth, and L1 the average per-band distance. A reconstruction that is the truth scaled by 2 (amber) has an angle of exactly 0 but a large L1; a reconstruction close in level but slightly rotated (indigo) has a small L1 and a small angle. (b) The three-band numbers of worked example 25.4. 354

25.3 The size of the derivative of arccos, drawn to scale from $1/\sqrt{1-u^2}$, on a logarithmic vertical axis. Dots are the four table rows. (a) Against u itself: flat until about $u = 0.9$, then a wall. (b) Against $1-u$ on a log axis, reading right to left as $u \to 1$: a straight line of slope $-\frac{1}{2}$, because $1/\sqrt{1-u^2} \approx 1/\sqrt{2(1-u)}$ there. The shaded band, $u \geq 0.999$, is where a well-trained model's pixels sit. 356

25.4 The SAM-weight ramp of both shipped configurations, drawn to scale from Equation (25.8). The weight is held for a whole epoch, so each ramp is a staircase; the thin amber line is the formula $\lambda = 0.05\,t$ that the stairs follow. Both configurations happen to rise at the same rate, 0.05 per epoch; v0.2.0 stops at half the height. After the ramp both stay flat for the rest of training (to $t = 199$ or 499). 362

25.5 The course's drawing of the v0.2.0 ramp: the weight on the angle rises linearly from 0 at epoch index 0 to 0.5 at epoch index 10 and then stays flat, so L1 leads early training and the angle is phased in. 362

26.1 The learning-rate schedules of v0.2.0 and F001 374

26.2 The training-data pipeline, one job per stage. A raw scene becomes a 165-band vendable; PatchPlanGenerator decides where to cut 128 × 128 patches (stride 64 in training, so neighbours overlap by half, the last snapping to the scene edge); the patcher cuts them; they are packed into .tar shards on S3 and streamed by the DataLoader as batches. 377

26.3 The training loop of `FoundationTrainer` with Indradhanu's loss, as the source runs it. The left column is one training step, repeated until the epoch's sample budget is spent; the right column runs once per epoch: validation, the scheduler step, logging and the periodic checkpoint. The SAM weight $\lambda(t)$ and the learning rate $\eta(t)$ both change only at the epoch boundary. 381

26.4 The course's drawing of one epoch: a training pass with random masking and gradient accumulation, a validation pass, a scheduler step, logging, and a periodic checkpoint. The dashed frame repeats once per epoch. 381

27.1 The four epochs of worked example 27.2 scored two ways. With the training ramp (amber, dashed) the validation score climbs from 0.055 to 0.099 although L1 and SAM both fall; epoch 1 stays "best" for ever. At the full SAM weight (teal) the score falls from 0.155 to 0.099. The two meet at epoch 11, where the ramp ends. 389

27.2 Left: how each sensor's bands divide between the visible, near-infrared and shortwave infrared. PRISMA's native list gives the visible 16% of its bands against SpectralEarth's 26%. Right: E004's error on water, on SpectralEarth test patches with all 202 bands, on the same pixels thinned to PRISMA's spacing, and with the skip path reweighted at inference. 392

27.3 (a) The ten 30,000-patch blocks along F001's 200 epochs, under the one cosine learning-rate curve that runs through all of them. (b) The loop in `rotate_f001.sh`: every decision is computed from `metrics.csv` and two marker files, so the script can be killed and restarted at any point. 396

27.4 F001's records drawn to scale from its README (validation rows at epochs 6, 13, 20, 38, 60, 109; bench at 7, 14, 20, 38, 60, 109). (a) Validation L1 falls at every checkpoint until epoch 60 and then flattens. (b) The bench under the `combined` score jumps above production's 0.662 by epoch 20 and then stays flat; residual Mahalanobis, measured at epochs 60 and 109, sits about 0.13 higher. (c) Validation SAM follows L1. 398

27.5 The course's version of the same records, including the residual-Mahalanobis measurement at epoch 52 (0.865) reported in the scoring comparison of 2026-10-15. . 399

27.6 (a) The three shard sets as they stood on 2026-10-16 (colours identify the sensor only). (b) The one rule that lets mixed band lists share a trainer: match by wavelength within 0.5 nm; a target band with no match becomes nodata, a source band with no target is dropped. 400

28.1 The head in context: the shared stem and encoder feed two heads. Pass 1 removes 65% of the tokens and reconstructs; pass 2 keeps every token and segments. Which passes run depends on the training mode (Chapter 29). 408

28.2 The worked example. Hatched cells are unknown and contribute nothing. Cloud, unknown on a dry scene, drops out of the class average; the loss is the mean of water's and shadow's terms. 418

28.3 (a) Share of cirrus labels that are surface-driven, by the scene's water vapour, for cirrus level 1 and level 2 or more; below $0.75\,\mathrm{g/cm^2}$ (dashed) nearly all are. (b) Candidate policies: share of suspect labels each removes against share of plausible labels it also removes. The adopted rule removes 62.6% of suspect labels for 13.6% of plausible ones. 421

29.1 (a) The six stages, what each trains, and the gate (amber) it must pass; stage 4 is optional. (b) Batches are drawn by fixed shares (top), not in proportion to each sensor's tile count (bottom), which would give AVIRIS-NG 43% of training. . . . 425

29.2 (a) The last micro-batches of an epoch, one sensor each, and the running mean they drag around; validation uses wherever it ends. (b) In training each sensor is normalised by its own batch mean (top); at evaluation every sensor is normalised by the one running mean (bottom), so each arrives with an offset the encoder never saw. 431

29.3 Three sensors, three stem passes (teal), one concatenated batch through the single BatchNorm, encoder and decoder (indigo), split by row ranges and decoded at each sensor's own wavelengths. 434

29.4 One mixed batch of 8 tiles goes through the shared BatchNorm and encoder twice: masked, to the decoder, for the reconstruction loss on all 8; unmasked, to the segmentation head, for the segmentation loss on the 3 EnMAP tiles only (the other 5 rows are normalised but not scored). Total loss = reconstruction $+0.1\times$ segmentation. 437

30.1 Two-pass checkerboard inference (course figure 29.1). Each token's final value is taken from the pass that hid it, so the combined reconstruction is built entirely from predictions. See the source check below for which pass hides the top-left token. 447

30.2 Sliding-window inference (course figure 29.2). 128×128 tiles start every 32 pixels, so a 1000×1000 scene needs $29 \times 29 = 841$ tiles; an interior pixel lies in $128/32 = 4$ tiles along each axis and so in $4 \times 4 = 16$ tiles in all. 451

30.3 Tiles covering each column of a 1000-pixel axis 452

30.4 The `combined` scoring method (course figure 30.1): per-pixel L1 and SAM maps are each divided by their maximum over the valid pixels and blended with weight 0.5. Since 2026-10-15 this is no longer Indradhanu's default. 455

30.5 The product's action chain for a hyperspectral scene (AD13). Teal steps transform spectra, plum runs the model, amber is where a person decides. 457

31.1 Rotate, divide, measure (course figure 45.1, the course's two-band version of the example above). (a) Ordinary residuals (grey) spread along the shared diagonal far more than across it; one test pixel lies further out along the diagonal, the other across it. (b) The same points turned onto the eigenvectors. (c) After dividing by each direction's spread the ordinary pixels sit on one circle and the across-the-diagonal pixel is far outside it. 466

31.2 Size versus direction (anomaly course figure 12.3). Left: ordinary residuals spread widely along direction 1 and barely along direction 2. Right: after whitening the cloud is round; the pixel far out along the common direction is inside it, the one a short way along the rare direction is far outside. 467

31.3 The ring, and a box sum from four look-ups (course figure 46.1). (a) The 61×61 window minus the hatched 21×21 guard leaves a ring of 3,280 pixels whose mean is subtracted from the centre pixel. (b) A 4×4 grid and its zero-led integral image: the shaded 2×2 box sums to 6 from four entries of the table. 470

31.4 The comparison drawn to scale from the table (F001, epoch 52, 2026-10-15). Amber: scores built on the residual's size; teal: scores built on its direction (C, D and the filters applied to C). The axis starts at 0.65 to separate the bars; the gap from A to C is $0.849 - 0.715 = 0.134$. 476

31.5 The same comparison as a dot plot (course figure 46.2), with object counts where they were recorded. 476

31.6 Per-scene ROC-AUC of A, C and D, to scale (F001, epoch 52). D beats C on every scene; the axis starts at 0.6. 477

32.1 Before 2026-10-31, comparing the two Indradhanu models on one scene took two branches, each with its own band filter and scoring action. Since then one native band filter and one scoring action serve both. A classical detector can ride along in either. 493

33.1 The ROC of the six pixels of worked example 33.1 (indigo staircase), the chance diagonal (grey, dashed), and the region counted by a partial AUC with limit FPR ≤ 0.5 (amber). Drawn to scale from the worked numbers. The full area under the staircase is 7/9; the amber area is 5/18, which normalised by the limit gives 5/9 (worked example 33.6). 501

33.2 The course's figure for this chapter. (a) The five-pixel ROC of the course's own example (targets at 0.9 and 0.7 among five pixels), AUC 5/6. (b) Detectors A and B: B has the larger total area; inside the first 1% of false alarms (the amber strip, stretched, not to scale) A is twenty times higher. 503

33.3 Detectors A (teal) and B (indigo, dashed) of worked example 33.5, drawn to scale. Left: the whole curve, where B's larger area comes from FPR 0.01–0.30. Right: the first 1.2% of false alarms, where A's area (amber, up to FPR 0.01) is twenty times B's. The chance diagonal (grey) is nearly flat on the right-hand scale. . . . 504

33.4 Left: a two-pixel object (amber) and the six pixels its 1-pixel tolerance adds (indigo); a flagged pixel anywhere in these $m = 8$ cells catches it. Right: two target pixels touching only at a corner are two objects under 4-connectivity (the book's script confirms `ndimage.label` returns 2). 508

34.1 What each benchmark supplies: the four things a model needs, plus platform. Amber cells are the reason a benchmark cannot test this product as it is. 514

34.2 The diamond ring around a one-pixel object. Distance 1 (grey) is skipped; distances 2 (8 pixels) and 3 (12 pixels) form the 20-pixel ring (teal). The dashed square shows the 40-pixel ring a 3×3 dilation would have made. 515

34.3 The mixing line of worked example 34.3 in the plane of two bands: every mixture of background $\mathbf{b}$ and material $\mathbf{t}$ lies on the straight segment between them, at a distance from $\mathbf{b}$ proportional to the abundance a. 517

34.4 $z(s)$ for worked example 34.4. The simple formula's $s = 0.5$ lands at $z = 0.6$ (medium); the solver follows the curve through its minimum at $s = 0.2$ and crosses the target at $s = 0.7$. 519

34.5 The abundance map of worked example 34.7: a one-pixel bench-v2 implant at strength 0.1 with a Gaussian PSF of $\sigma = 1.4$, shown on its 3×3 footprint (darker amber = more material). The blur continues beyond the window, weaker with distance. 522

34.6 pAUC to 1% false alarms for FT03 (amber) and PCA-RX (teal) on three benchmarks. The real PRISMA bench puts FT03 first; bench-v1 reversed that order; bench-v2 restores it. 524

34.7 What bench-v2 had to match: medians on PRISMA, real objects (grey) against bench-v1 implants (amber), overall and in the easy tier, on the diagnosis ruler; the shaded band is within 25% of the real value. 524

35.1 pAUC to 1% false alarms. On implant bench-v1 (a) the model-free PCA-RX (teal) is highest; on the real PRISMA bench (b) it is lowest, and excluding water and cloud (hatched) helps the model on both. 532

35.2 The 39 labelled objects lying wholly inside FT03's water / cloud mask, by kind. Excluding water and cloud hides all 39 (amber); excluding cloud only hides the 4 in set4 that sit under haze FT03 calls cloud. 534

36.1 The exact bootstrap distribution of the mean of five numbers: every one of the $5^5 = 3{,}125$ draws with replacement, counted by the mean it gives (computed by `research/textbook/tools/compute_bootstrap.py`). The teal band is the percentile 95% interval. The distribution is lumpy because only 43 distinct means are possible, and skewed right because one value (12) is far from the rest. 540

36.2 The paired-bootstrap intervals of Table 36.1, drawn to scale: point value (dot) and 95% interval (bar) of the pAUC difference $B - A$. Teal: interval wholly above zero (a resolved gain); amber: wholly below zero (a resolved loss); grey: crosses zero (a tie). FT09b's interval clears zero by 0.0002. 545

37.1 pAUC by training stage, as read on 2026-10-17: the implanted bench (amber, easy and medium tiers) rises from v0.2.0 to FT02c, the real bench (grey) wanders with no order. The v0.2.0 points are the bug this chapter dissects. 550

37.2 The same three samples (500, 600, 700 nm; values 1, 2, 4). (a) Listed ascending, `np.interp` draws the line: 1.5 at 550 nm and 3.0 at 650 nm. (b) Listed descending, it returns the last listed value, 1, for every wavelength above 500 nm: the spectrum goes flat. 553

37.3 Objects caught at the top 1% on the three implant benches, drawn to scale. The wavelength-reading models (F001, FT03) and PCA-RX move a few points with each rebuild; v0.2.0, whose reading was the only one fixed, jumps 13.4 points from bench-v2 to bench-v2.1. Its bench-v1 and bench-v2 bars measured a broken resample. 558

38.1 FT01's IoU by epoch, drawn to scale from Table 38.2. Water, cloud and shadow climb; thin cloud (grey) stays between 0.06 and 0.09 for the whole run. 565

38.2 FT02's bench by epoch against the gate. Residual Mahalanobis (amber) starts above the 0.861 gate at epoch 1 and settles below it; `combined` (grey) climbs from F001's 0.706 to 0.824. best.pt is epoch 8, chosen by validation reconstruction, not by the bench. 566

38.3 (a) Edge-strength correction (amber) pushes down everything on strong edges, the field boundary and the target alike. (b) Under the blind spot: a boundary passing through a hidden token (grey hatching) can be continued from its visible ends; a target wholly inside the hidden token leaves no trace in the context. . . . 570

38.4 The four options discussed after FT02, what each would add, its main cost, and where it stood. 572

38.5 Medians on PRISMA, real objects (grey) against bench-v1 implants (amber), overall and in the easy tier; the shaded band is within 25% of the real value. . . . 574

38.6 One FT05 step. The loader workers (CPU) implant half the tiles and keep the clean copy; the model (GPU) reads the implanted tile and is graded against the clean one, ×5 near the implants. The CPU side sets the pace. 577

38.7 Share of objects caught at the top 1% on the real bench (grey) and bench-v2.1 (amber), drawn to scale from Table 38.8 and Table 37.2. Every model is far above chance and above PCA-RX; the models tie each other. 580

38.8 The course's version of the same board: share caught at the top 1% for each model and PCA-RX, on the real bench (grey) and bench-v2.1 (amber). 580

38.9 (a) How one teacher probability becomes a target: taught "yes" above 0.9, "no" below 0.1, nothing between, and a confident "water" over a bright NIR turned into a known not-water. (b) The real-bench pAUC from the shipped v0.2.0 to the release candidate: the model adds 0.003, cloud exclusion 0.008. 584

39.1 The worker's path for Indradhanu V2, in order. Every box is one of the decisions of this chapter; the parity run (Section 39.7) runs the same boxes outside the worker and compares with the evaluator. 595

40.1 Pixel pAUC against objects caught, real PRISMA bench, drawn to scale from Table 40.1. The dashed lines cross at plain RM. Nothing lies above the horizontal line: the A1 blurs (amber) move right and down - more pixel pAUC, fewer objects. 599

40.2 Per-scene pAUC of FT04 and its gated adaptations, drawn to scale. Set2 gains at every N; set1 and set4 lose at every N; nothing peaks at small N. 611

41.1 The Laplace density for three scales, drawn to scale from Equation (41.1), with a standard Gaussian in grey for comparison. The Laplace has a sharp peak at zero and heavier tails: a large error is less surprising under a Laplace than under a Gaussian of similar width. 616

41.2 The per-pixel loss of Equation (41.3) for $|r| = 0.2$, drawn to scale, with the three points of worked example 41.1. The curve rises steeply to the left (overconfidence) and slowly to the right (vagueness). 618

41.3 The three pixels of worked example 41.9, drawn to scale. Dividing by b with a target/ring ratio of 1.19 drops the target below an ordinary flat-field pixel; the square root keeps it a little above. 627

42.1 The top-left token of a hidden 2×2 block (thick outlines are blocks). Three of its 8 neighbours are in its own block and are certainly hidden; the other 5 lie in three neighbouring blocks. 633

42.2 The progressive decoder. The stem output reaches the decoder twice: through the encoder (top) and through the masked skip (bottom), at half and full resolution. Channel counts are written as "channels @ size". 640

42.3 Table 42.7 drawn to scale: point values with their 95% paired-bootstrap intervals. Only cloud exclusion lies wholly right of zero; FT09b touches it, and FT09c falls back. 645

43.1 Parameter growth of a stage-1 block with width, drawn to scale. The curve hugs k^2; the gap is the linear (bias and norm) terms. 651

43.2 Indradhanu Heavy as served, one masked pass over two 128×128 PRISMA tiles with 185 bands, drawn from the shapes the forward hooks recorded [C]. Teal: spectral stem and output; blue: encoder; plum: progressive decoder; dashed teal: the skip from the stem output with hidden tokens zeroed; amber: the segmentation head, fed by a separate, unmasked encoder pass. 656

43.3 Indradhanu Heavy's pretraining, all 80 epochs drawn from `metrics.csv` [M: `F002-heavy/metrics.csv`]. Train L1 lies above validation L1 throughout (see the text), and both keep falling. 659

43.4 Heavy against the control (indigo squares) and against shipped FT04 (amber circles), at three neighbouring checkpoints each, drawn to scale; cloud exclusion (teal) for reference. Only FTH1 epoch 9 touches zero. Sources: [M: `FTH1-heavy-joint/README.md`], [M: `FTH2-heavy-joint-more/bootstrap.txt`], [M: `FTH3-heavy-joint-more2/bootstrap.txt`]. 664

44.1 The publication schematic of the namer, from `research/experiments/spectral-namer/figures/namer_network.pdf`, redrawn on 31 October 2026 for level 1 by matching: the pixel tower has two heads (embedding and fill), and the class-matching box pools the natural library entries as one more class, $P(\text{man-made}) = \sigma(\text{LSE}_{c<7}\, S_c - S_{\text{natural}} + b)$ (Section 44.7, Figure 44.2). The old Linear level-1 head is still built in the model but not used, so it is not drawn. 672

44.2 Tensor shapes through one training step for one sensor, with **level 1 by matching** (the route through column 7 of the pooled scores). Grey: the generator; teal: the shared stem, run three times; blue: the pixel tower; green: the library tower; plum: matching; amber: losses. The dashed grey head is N001's linear level 1, still built, no longer read. $B = 16$, $P = 8192$, $N = 200$ [A], $L = 378$. 691

46.1 Chikusei clustering: HyperSL features against raw spectra and PCA 713

47.1 The span of HyperSL's kernels along PRISMA's 149-band list 721

48.1 pAUC on the PRISMA bench. HyperSL's own residual (L2, loss) sits near zero; scored with residual Mahalanobis it lands below PCA-RX, which uses no model at all. Data `figdata/hsl_detect_bar.dat`. 729

49.1 mSAD on the four exact-truth sets. The classical method is near zero on DC1, DC2 and EGU; every HyperSL preset is 0.08–0.26 rad. The last two bars, pretrained and random backbone, are nearly the same height. 735

49.2 HySU object AUC against target size. Every method finds targets of 1 m and more when told the material; below one pixel, HyperSL anchored falls to 0.68 and 0.77 while per-pixel ACE stays at 0.97–1.00. 737

50.1 Share of each EnMAP-labelled class the classifier finds, out of fold over 47 datatakes. HyperSL's features lose most on cirrus, the thin class, and on snow. 743

How to read this book

What this book is

This book turns two training courses into one mathematical textbook. The first course, *Finding anomalies in satellite imagery* (`docs/training/anomaly-detection/`, 34 parts), teaches the domain, the classical detectors and the evaluation. The second, *Indradhanu, from zero* (`indradhanu workings/`, 71 parts), teaches the hyperspectral foundation model the product uses to find anomalies, and the long history of experiments that shaped it. Where the two courses covered the same ground, this book merges them into one chapter.

The courses already explain every idea in words and with small numbers. This book keeps that and adds the mathematics underneath: each formula is derived one operation at a time, each rule is checked on a tiny example the first time it is used, and each result is followed by a worked example small enough to do on paper.

It is study material. It is subordinate to two things:

- **The source code.** When this book and the code disagree, the code is right. Wherever a writer found a course and the code disagreeing, the chapter says so in a red *Source check* box and follows the code.
- **The reference documentation**, `docs/01-orientation.md` through `docs/10-code-style.md`, and the low-level designs in `docs/lld/`.

The mathematics of naming a material from its spectrum (spectral library matching) has its own companion volume in `research/math/spectral-matching/`; this book only points to it.

Who it is for

You need school algebra: you can expand $(a + b)^2$, solve $2x + 3 = 11$, and read a sum written with $\sum$. Everything else - vectors, matrices, eigenvalues, probability densities, derivatives of functions of many variables, the parts of a neural network - is introduced from the beginning, in the chapter that first needs it. If you learn slowly and carefully, the book is written for you.

How every chapter is built

Every chapter has the same skeleton, so that you always know where you are.

1. A green box, **The one thing this chapter teaches**, in one or two sentences.
2. The motivation: what problem exists, and why the obvious fix fails.
3. The mechanism, in numbered sections, with definitions before use, derivations one step per line, and purple **Worked example** boxes with every arithmetic step written out.
4. **Common confusions**: the mistakes people actually make, as questions and answers.

5. **Check yourself**: about five exercises, at least one by hand. Full solutions are in Appendix A.

Other boxes you will meet:

- a red **Source check** box, where a claim was checked against the code, or where a course and the code disagree;
- an **Analogy** box, one per chapter, tied to something outside software;
- an amber key-point box, the one sentence to remember from a section;
- a grey **Values that will drift** box, around numbers read from configurations, checkpoints and runs, with the date they were true.

Where the numbers come from. A number about the project carries a small grey tag: [M: `file`] means *measured*, and names the file it was read from; [C] means *computed* for this book, by hand in the text or by a script in `research/textbook/tools/`; [A] means *assumed* for illustration, with the assumption stated. Plain arithmetic inside a worked example is not tagged.

Do the arithmetic

The worked examples are deliberately tiny - three bands, a 2×2 matrix, six tokens - so that you can do them in a minute with a pencil. Reading them is not the same as doing them. Cover the right-hand side of each line, compute it, then uncover it.

The parts, and what depends on what

The book has ten parts. Part I is the domain: light, sensors and the data structure the product hands to every algorithm. Part II is classical statistics: the linear algebra of covariance and whitening, and the detectors built on it. Part III builds the Indradhanu network piece by piece, with tensor shapes at every step and every parameter counted by hand; on the way it tours the five convolutional autoencoders that came before it (Chapter 11), and it ends with Chakshu, the same machine for one thermal band (Chapter 21). Part IV replaces Indradhanu's spectral front end. Part V trains it. Part VI turns its output into an answer a user sees. Part VII measures how good that answer is. Part VIII tells the history of the experiments as lessons, because most of what the team knows was learned from runs that failed. Part IX covers what was built on 2026-10-31: the spectral namer, a second model on the Indradhanu stem, and the first evaluation submission. Part X answers a question the team kept being asked: why not use HyperSL, a published spectral foundation model with open weights? It shows what HyperSL is and does well, then measures it on each of the product's jobs: library matching, anomaly detection, unmixing and cloud masking.

Figure 1 shows which chapters you need before which. A solid arrow means "read this first". Read the book in order if you can; the map is for when you want to reach a chapter sooner and need to know the shortest path to it.

Five reading paths

The whole book (recommended).
Chapters 1 to 45 in order. A fortnight of steady study.

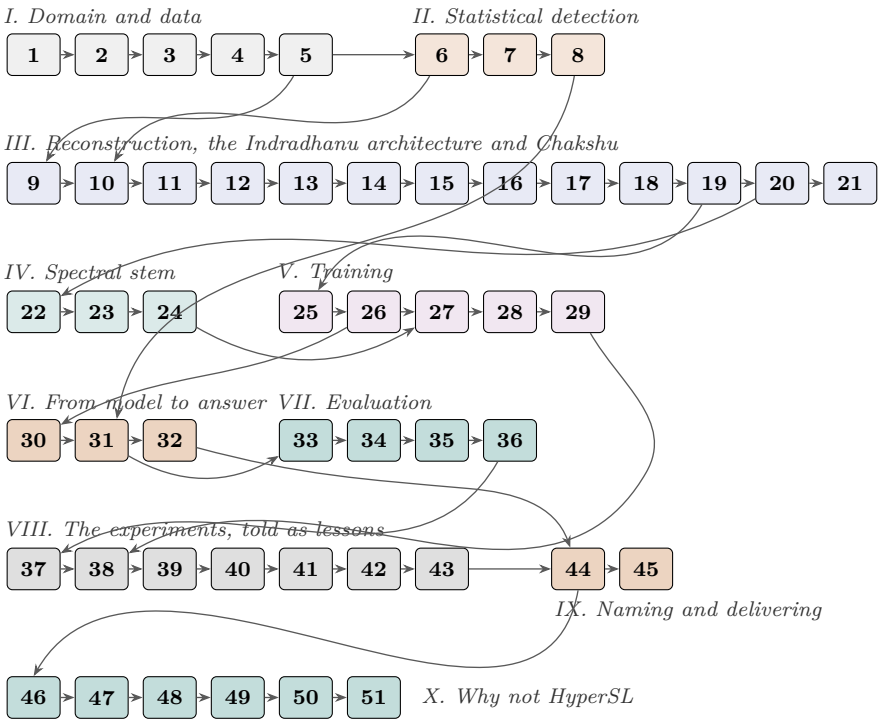


Figure 1: The chapter dependency map. Each box is a chapter number; an arrow means "read this first". Inside a part, read in order. The long arrows show the few places where a later part leans on a specific earlier chapter: the linear algebra of Chapter 6 under the tensor arithmetic of Chapter 10, the local detectors of Chapter 8 under the residual Mahalanobis score of Chapter 31, and the fine-tuning machinery of Chapter 29 under the experiment chapters. Part X leans on the namer (Chapter 44) and, through it, on the reconstruction, masking, spectral-stem and evaluation chapters.

Detection only.
Chapters 1–8, then 9, 31, 33–36. You learn the statistics of detection and how detections are measured, without the network.

The network only.
Chapters 1–5 for the data, 6 for the linear algebra, then 9–29.

Why not an outside foundation model.
Chapters 1–5, 9, 18, 24, 31, 32, 33 and 36, then 46–51. The case against HyperSL rests on why a pixel must not help rebuild itself, why statistics must be fixed by wavelength, and how detections are measured.

The thermal models.
Chapters 1–5, then 9–21: every architecture chapter, because Chakshu reuses them all; 11 and 21 are the thermal ones.

Numbers that will drift

Architecture and mathematics age slowly. Run results do not: every pAUC, every validation loss, every count of objects caught belongs to a run, a benchmark build and a date. Those numbers sit in grey *Values that will drift* boxes. The course indexes (`docs/training/anomaly-detection/README.md` and `indradhanu workings/00-START-HERE.md`) list, part by part, which numbers to re-verify; the experiment READMEs under `research/experiments/` are authoritative for every result.

The index and the lists

After the table of contents come a *List of worked examples* - use it to find an example to redo when a later chapter assumes a calculation - and a list of figures. At the back are the full solutions to every exercise (Appendix A), a glossary of every technical term with the chapter that defines it (Appendix B), the bibliography of the papers the courses cite, and an index.

Notation

This table collects the symbols used across the book. Each is defined again where it first appears; come back here when a later chapter uses a symbol you have forgotten. Two collisions inherited from the code are worth learning now: C means *input bands* in the data chapters but *channel width* inside the network (written C_i for stage i), and d means the *per-head width* in attention but the *token width* in the spectral stem. The chapter always says which it means.

Numbers, vectors and matrices

Symbol	Meaning
a, b, t	plain numbers (scalars), in italic
$\mathbf{x}, \mathbf{u}$	vectors, in bold lower case; a column of numbers
x_k	the k-th entry of $\mathbf{x}$
$\mathbf{A}, \mathbf{\Sigma}$	matrices, in bold upper case
$\mathbf{A}^\top$	the transpose of $\mathbf{A}$: rows become columns
$\mathbf{A}^{-1}$	the inverse of $\mathbf{A}$, with $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$
$\mathbf{I}$	the identity matrix: ones on the diagonal, zeros elsewhere
$\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^\top \mathbf{b}$	the dot product $\sum_k a_k b_k$
$\|\mathbf{x}\|$	the Euclidean length $\sqrt{\mathbf{x}^\top \mathbf{x}}$
$\|x\|$	absolute value
$\mathbb{R}^C$	the set of all lists of C real numbers
$\mathrm{diag}(\lambda_1, \ldots, \lambda_C)$	a diagonal matrix with these entries
$\mathrm{tr}(\mathbf{A})$	the trace: the sum of the diagonal entries
$\lambda_k, \mathbf{u}_k$	the k-th eigenvalue and eigenvector of a symmetric matrix
$\mathbf{U}, \mathbf{\Lambda}$	eigenvectors as columns, eigenvalues on a diagonal: $\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$
$\mathbf{W}$	a whitening matrix, or (in a network) a weight matrix; the chapter says which

Data

Symbol	Meaning
λ	wavelength, in nanometres (nm). Context separates it from an eigenvalue
C	number of spectral bands (165 on the common grid)
H, W	image height and width in pixels
$\mathbf{x} \in \mathbb{R}^C$	one pixel's spectrum: a reflectance per band
$\hat{\mathbf{x}}$	the model's reconstruction of $\mathbf{x}$
$\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}}$	the residual: what the reconstruction got wrong

Symbol	Meaning
$\boldsymbol{\mu}$	a mean vector (of the background)
$\boldsymbol{\Sigma}$	a covariance matrix; $\boldsymbol{\Sigma}_r$ is the covariance of residuals
m	a validity mask: 1 where the measurement is real, 0 where it is not

Detection and statistics

Symbol	Meaning
$\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$	the Gaussian (normal) distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$
z	a z-score: (value − mean) / standard deviation
$D^2(\mathbf{x})$	squared Mahalanobis distance $(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})$; the RX score
χ^2_C	the chi-square distribution with C degrees of freedom
ϵ	a small regularising constant (for example in $\boldsymbol{\Sigma} + \epsilon\mathbf{I}$)
θ	a spectral angle, in radians
$s(p)$	the anomaly score of pixel p
TPR, FPR	true-positive rate and false-positive rate at a threshold
pAUC	partial area under the ROC curve, up to a maximum FPR
a	an abundance (fill fraction) in the linear mixing model $\mathbf{x} = (1-a)\mathbf{b} + a\mathbf{t}$

The network

Symbol	Meaning
B	batch size: how many patches are processed together
D	width after the spectral compressor or stem (32; 64 in Indradhanu Heavy)
N	number of tokens at a stage
C_i	channel width at encoder stage i (32, 64, 160, 256 for the base model; 96, 192, 480, 768 for Heavy; 16, 32, 64, 96 for Chakshu)
R_i	spatial-reduction ratio for attention at stage i (8, 4, 2, 1)
h, d	number of attention heads, and width per head
E	expansion ratio of the feed-forward network (4; 2 in Chakshu)
$\mathbf{Q}, \mathbf{K}, \mathbf{V}$	queries, keys and values
k, s, p	convolution kernel size, stride and padding
ρ	the mask ratio: the fraction of valid tokens hidden
$\mathcal{L}$	a loss: the number training makes smaller
η	the learning rate
β_1, β_2	Adam's decay rates for its two moving averages
GELU, LN	the GELU activation and Layer Normalisation
`(B, C, H, W)`	a tensor shape in the spatial layout; `(B, N, C)` is the sequence layout
P	in the spectral stem, the number of pixel rows, $B \times H \times W$
k, s, p (transposed)	a transposed convolution's kernel, stride and padding; output size $(H-1)s - 2p + k$

The thermal models

Symbol	Meaning
T, $\hat{T}$	a pixel's land surface temperature in °C, and the model's reconstruction of it
μ_T, σ_T	the thermal normalisation constants stored in the checkpoints, 24.5756 and 13.5744 °C
τ	the trim fraction of Chakshu's trimmed L1 (0.05). In the namer chapter τ is a temperature that scales cosine scores; the chapter says which
$m_{\text{valid}}, m_{\text{input}}$	the validity and input-mask channels of Asanskrita and Drashta
DN	a digital number: the integer a sensor records before calibration

HyperSL (Part X)

Symbol	Meaning
m_b, M_b	the smallest and largest value of band b over a scene's pixels
x'_b	a value after per-scene min-max scaling, $(x_b - m_b)/(M_b - m_b)$
s_b, c_b	the scale and offset of a per-band affine map, $x'_b = s_b x_b + c_b$
p	HyperSL's wavelength coordinate $(\lambda - 400)/10$, λ in nm
ω_j	the j-th frequency of HyperSL's position code, $100^{-2j/256}$
k, h	a kernel size over band index and its half-width $(k-1)/2$
$\mathbf{E}, \mathbf{a}$	endmembers as columns, and their abundances, in $\mathbf{x} = \mathbf{E}\mathbf{a} + \mathbf{n}$
a'	a reported fill, against the true fill a
λ (Part X)	also the weight of a total-variation penalty; the chapter says which
δ	a small additive brightening, such as thin cirrus

Part I

The domain and the data

Chapter 1

The problem: finding what does not belong

The one thing this chapter teaches

We never teach the computer what an anomaly looks like. We teach it what *normal* looks like in a scene, give every pixel a number that says how badly it fits that normal, and hand the highest-scoring pixels to a person.

This chapter has no heavy mathematics. Its job is to fix the question that the rest of the book answers, to show with arithmetic why the obvious answer fails, and to set up the notation that every later chapter uses. Read it slowly anyway: almost every design decision in the forty chapters that follow is a consequence of the three facts in Section 1.2.

1.1 The situation

A satellite flies over a patch of ground and records an image. Somebody at a mining ministry, an environmental agency or an insurance company wants to know one thing: *is there anything unusual down there?*

Unusual might mean an illegal mine, a leaking pipeline, a chemical dumped in a field, an unusually hot piece of machinery, a vehicle parked where no vehicle should be, or a mineral outcrop that could be worth money. The software this book describes is called **Allotrope**. It is a web application: a user uploads a satellite file, runs a chain of processing steps on it, picks a cut-off on a map of scores, and downloads a bundle naming the most unusual places with their coordinates. Chapter 4 describes the product; this chapter is about the question it answers.

Before any mathematics, three words.

Definition 1.1 (Pixel)**.** A **pixel** is one cell of the grid an image is made of. In a satellite image a pixel stands for a patch of ground; for the sensors in this book that patch is about 30 metres on a side (Chapter 2). We name a pixel by its row i and column j, both counted from 0.

Definition 1.2 (Scene)**.** A **scene** is one complete satellite acquisition: one patch of the Earth, at one moment, as one file (or folder of files).

Definition 1.3 (Anomaly)**.** An **anomaly** is a pixel (or a small group of pixels) that is unlike the rest of its scene. The word says nothing about *what* the pixel is. In this codebase "anomaly", "outlier" and "novelty" are used interchangeably; the code says "anomaly" throughout.

1.2 Why a classifier will not work

If you have met machine learning before, your instinct is a **classifier**: a program that takes a pixel and answers with one of a fixed list of labels. You would collect thousands of pictures of illegal mines, label them "illegal mine", *train* the classifier on them (adjust its internal numbers until its answers on the labelled examples are right), and run it on new imagery.

That approach is standard, and here it fails for three separate reasons. Each one alone would sink it, and each closes a different escape route, so it is worth keeping all three.

1.2.1 Reason 1: there are no labels

A **label** is the correct answer attached to a training example. To label a satellite pixel "illegal mine" somebody has to know what is on the ground under it, which in practice means somebody visited the spot. Nobody has done that for thousands of anomalies across every terrain and season. The number of labelled anomalies available is, for practical purposes, zero. A classifier with no labelled examples has nothing to learn from.

1.2.2 Reason 2: you do not know what you are looking for

A classifier can only answer with labels it was given in advance. Train it on "illegal mine" and "oil spill" and it will find illegal mines and oil spills. The valuable find is usually the one nobody anticipated: a chemical nobody had seen before, a structure nobody expected. A classifier is blind to unnamed classes *by construction*, and the unnamed classes are the point of surveying.

1.2.3 Reason 3: the interesting thing is vanishingly rare

Anomalies are a tiny fraction of the pixels. That rarity breaks the ordinary way of measuring whether a classifier works.

Definition 1.4 (Accuracy)**.** The **accuracy** of a yes/no rule over N pixels is the fraction of pixels it answers correctly:

$$\text{accuracy} = \frac{\text{number of correct answers}}{N}. \tag{1.1}$$

Worked example 1.1: The "always normal" rule

A scene has $N = 20{,}000{,}000$ pixels, of which 40 are anomalous. Consider the rule that ignores its input and answers "normal" for every pixel. Work out its accuracy on paper before reading on.

The rule is right on every normal pixel and wrong on every anomalous one. Count the normal pixels:

$$\begin{aligned}\text{normal pixels} &= 20{,}000{,}000 - 40 \\ &= 19{,}999{,}960.\end{aligned}$$

Every one of those is a correct answer, so put that count into Equation (1.1):

$$\begin{aligned}\text{accuracy} &= \frac{19{,}999{,}960}{20{,}000{,}000} \\ &= 0.999998 \\ &= 99.9998\%.\end{aligned}$$

The rule finds nothing and scores 99.9998%. The fraction of anomalous pixels, often called the **base rate**, is

$$\frac{40}{20{,}000{,}000} = \frac{4}{2{,}000{,}000} = 0.000002 = 2 \times 10^{-6}.$$

Whenever the base rate is this small, accuracy is close to 1 for every rule that says "normal" almost all the time, useful or not.

A classifier trained on such data learns the same lesson as the "always normal" rule, because that rule is nearly always right. Accuracy is therefore useless here; Chapter 33 builds the measures used instead.

No labels, an open-ended target and extreme rarity together rule out supervised classification; any one of them alone would be enough.

1.3 The reframe: model normality, score the residue

Stop asking "is this pixel a mine?", which needs labels. Ask instead:

Given everything else in this scene, how surprising is this pixel?

That question needs no labels at all. It needs only a description of what normal looks like here, and every scene carries its own definition of normal in its own pixels. It also catches things nobody anticipated, because "unlike everything else here" does not require knowing what the thing is. The sentence to remember from this chapter is:

We never model anomalies. We model normality, and call the residue anomaly.

Everything in the rest of this book is machinery for two jobs: building a model of normality, and measuring the residue.

Definition 1.5 (Anomaly score and score map)**.** An **anomaly score** is a function s that gives each pixel (i, j) of a scene a number $s(i, j) \geq 0$, where larger means "fits the model of normal worse". Laid out on the pixel grid, the scores form a **score map**: one number per pixel, the same height and width as the image.

The score map is the product's central output. It is not a yes/no answer and it is not a list of objects. A person looks at it, chooses a cut-off, and decides what the highest-scoring pixels are. The system's output is **a shortlist for an analyst, not a verdict**.

Analogy

A proofreader working in a language they know well has no list of the typos in this book. What they have is a deep sense of what correct words look like. At "the cat sat on the mta" their eye snags: they did not recognise a known error, they *failed to recognise a normal word*. That snag is the anomaly score. The proofreader can tell you *where* something is off without being able to say what the author meant, and in the same way the system can say *where* without saying *what*. You give up naming the thing in exchange for not needing to be told what to look for. The analogy stops working at one point: a proofreader corrects the typo, while the system never proposes what the pixel "should" have been as a finding, only how far it is from what it expected.

1.4 Two ways to model normality

This codebase has exactly two families of method. Both ship in the product, and a user can run both on the same scene and compare.

1.4.1 Family 1: classical detectors, fitted to each scene

A **classical detector** computes statistics of the scene in front of it: the average pixel, and how much pixels typically vary. It then measures how far each pixel is from that average, in units of the typical variation. Nothing is trained in advance and nothing is stored between scenes; the statistics are computed fresh from each scene's own pixels.

The simplest version of this idea uses one number per pixel, which is what a thermal sensor gives (Chapter 2). Write x for a pixel's value, μ ("mu") for the average over the scene, and σ ("sigma") for the typical spread. The distance from the average in units of spread is

$$z = \frac{x - \mu}{\sigma}, \tag{1.2}$$

and a score can be its size, $s = |z|$. Read Equation (1.2) in words: subtract the average to get "how far above or below average", then divide by the spread to measure that distance in standard units.

Definition 1.6 (Mean and standard deviation)**.** For n numbers $x_1, \ldots, x_n$, the **mean** and the **standard deviation** are

$$\mu = \frac{1}{n}\sum_{k=1}^{n} x_k, \qquad \sigma = \sqrt{\frac{1}{n}\sum_{k=1}^{n}(x_k - \mu)^2}.$$

For the mean, add the numbers up and divide by how many there are. For the standard deviation, take each number's distance from the mean, square it so negatives do not cancel positives, average the squares, and take the square root to return to the original units.

Worked example 1.2: A one-band scene with one hot pixel

A tiny thermal scene has five pixels with surface temperatures, in degrees Celsius,

$$20,\ 21,\ 19,\ 20,\ 30.$$

Do each line on paper. First the mean:

$$\begin{aligned} \text{sum} &= 20 + 21 + 19 + 20 + 30 = 110, \\ \mu &= 110/5 = 22. \end{aligned}$$

Next each difference from the mean, and its square:

$$\begin{aligned} 20 - 22 &= -2, & (-2)^2 &= 4, \\ 21 - 22 &= -1, & (-1)^2 &= 1, \\ 19 - 22 &= -3, & (-3)^2 &= 9, \\ 20 - 22 &= -2, & (-2)^2 &= 4, \\ 30 - 22 &= 8, & 8^2 &= 64. \end{aligned}$$

Average the squares, then take the square root:

$$\text{sum of squares} = 4 + 1 + 9 + 4 + 64 = 82,$$
$$\sigma^2 = 82/5 = 16.4,$$
$$\sigma = \sqrt{16.4} \approx 4.050.$$

Now the score of each pixel, $s = |x - \mu| / \sigma$:

$$s(20) = 2/4.050 \approx 0.49, \qquad s(21) = 1/4.050 \approx 0.25, \qquad s(19) = 3/4.050 \approx 0.74,$$
$$s(30) = 8/4.050 \approx 1.98.$$

The hot pixel scores about four times higher than any other. Notice also that the hot pixel pulled the mean up from 20 to 22 and inflated the spread; a detector that computes its statistics from the very pixels it scores lets an anomaly partly hide itself. Chapter 7 and Chapter 8 return to this.

The classical method in this book is called **RX** [36]. It is Equation (1.2) for many numbers per pixel at once, using a distance (the Mahalanobis distance, Chapter 7) that accounts for the fact that some directions in the data vary more than others. The version shipped for hyperspectral data is **MNF-RX**, which first transforms the data into a smaller, better-behaved space [8].

Values that will drift

History, dated 2026-05-11. Plain RX on the 165-band hyperspectral data was deliberately dropped on that date: with that many bands the covariance matrix it inverts is nearly singular and the distances blew up to around 10^{11}. MNF-RX replaced it. Do not reintroduce plain RX on hyperspectral data; it has been tried. (Source: the project instructions `CLAUDE.md` and the Indradhanu course, part 1.) Chapters 7 and 8 explain why the inversion fails.

1.4.2 Family 2: foundation models, trained once to redraw imagery

The second family uses a **neural network**: a function built from many simple layers whose behaviour is set by a large list of adjustable numbers called **parameters** (Chapter 10 defines layers precisely). The network is **trained**, once and offline, on tens of thousands of ordinary satellite images, to do a task that needs no labels: hide part of an image and redraw the hidden part from its surroundings. Training produces a file of learned parameters ("weights"); running the model on a new scene loads that file.

In this repository such a network is called a **foundation model**. Careful with that phrase: in the wider industry it usually means an enormous language model with hundreds of billions of parameters. Here it means something much smaller and more specific, a network trained to reconstruct imagery whose reconstruction error is used as the anomaly score. The hyperspectral model in this book, **Indradhanu** (Sanskrit for "rainbow"), had 5,507,354 [M: `indradhanuworkings/05-where-it-lives.md`] parameters in the version the courses describe; Chapter 20 counts them by hand. Its thermal sibling is **Chakshu** ("the eye").

The rule that turns a reconstruction into a score is "reconstruct, then subtract" (Figure 1.1). Where the model redraws the scene well, the difference between what was observed and what was redrawn is small. Where the model, trained only on ordinary ground, meets something it has never seen, it paints ordinary background there instead, and the difference is large.

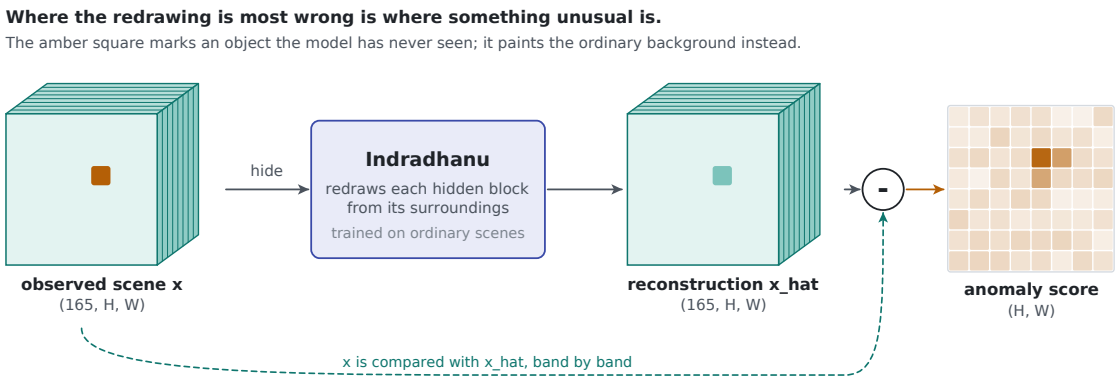


Figure 1.1: Reconstruct, then subtract. Blocks of the observed cube are hidden and the model, trained only on ordinary scenes, redraws them from their surroundings. Subtracting the reconstruction from the observation, band by band, gives a per-pixel error map; where the redrawing is most wrong (amber) is where something unusual is. The amber square stands for an object the model has never seen. (Course figure, Indradhanu part 1.)

Worked example 1.3: A reconstruction residual on three numbers

A pixel is described by three numbers (three "bands", Chapter 2): observed $\mathbf{x} = (0.10,\ 0.20,\ 0.30)$. The model's redrawing of it is $\hat{\mathbf{x}} = (0.11,\ 0.18,\ 0.30)$. These values are illustrative [A]. One simple score is the mean absolute difference over the three numbers. Do each step.

Subtract band by band:

$$0.10 - 0.11 = -0.01,$$
$$0.20 - 0.18 = 0.02,$$
$$0.30 - 0.30 = 0.00.$$

Take absolute values (drop the signs, because over- and under-shooting are equally bad):

$$0.01, \quad 0.02, \quad 0.00.$$

Add them and divide by 3:

$$0.01 + 0.02 + 0.00 = 0.03,$$
$$s = 0.03/3 = 0.01.$$

If a second pixel had the same observation but the model redrew it as $(0.10,\ 0.10,\ 0.10)$, the differences would be 0.00, 0.10, 0.20, their sum 0.30, and the score $0.30/3 = 0.10$: ten times larger. That pixel is the one the model could not explain. Chapter 9 explains why a pixel must never be allowed to help reconstruct itself, and Chapters 25 and 31 replace this simple average with better scores.

1.5 From a score map to a shortlist: thresholds are percentiles

A score map ranks pixels; someone still has to decide where to cut. The obvious rule is an absolute one: "flag every pixel scoring above 0.05". It fails, and the reason is worth seeing with numbers.

Table 1.1: The two families of anomaly model in this codebase.

	Classical detectors	Foundation models
Model of normal	scene mean and spread	a trained network
Trained?	no, fitted per scene	yes, once, offline
Stored file of weights	none	a checkpoint (`.pt`)
Score	distance from the mean	reconstruction error
Examples here	MNF-RX	Indradhanu, Chakshu

Worked example 1.4: Why a fixed cut-off fails

Two scenes are scored by the same model. The numbers are illustrative [A], of the size the Indradhanu course quotes.

- Scene A, a calm desert: typical score 0.002, highest score 0.04.
- Scene B, a hazy coastal city: typical score 0.08.

Apply the rule "flag if $s > 0.05$". In scene A the highest score is 0.04 and $0.04 < 0.05$, so nothing is flagged, including the real anomaly. In scene B the typical pixel already scores $0.08 > 0.05$, so most of the image is flagged. The typical score differs between the scenes by a factor of

$$0.08/0.002 = 40,$$

so no single number can serve both.

The cure is to make the cut relative to the scene in front of you.

Definition 1.7 (Percentile threshold)**.** Sort a scene's N valid scores from largest to smallest. The **top-q threshold**, for a fraction q such as 0.001 ("the top 0.1%"), keeps the k highest-scoring pixels, where k is qN rounded to a whole number. Equivalently, the cut-off value is the $(1-q)$ **percentile** of the scores: the value below which a fraction $1-q$ of them lie.

Worked example 1.5: A top-20% cut on ten scores

Ten pixels have scores

$$0.3,\ 0.1,\ 0.9,\ 0.2,\ 0.4,\ 0.2,\ 0.7,\ 0.1,\ 0.3,\ 0.2.$$

Keep the top $q = 0.2$. Do it on paper.
How many pixels to keep:

$$k = qN = 0.2 \times 10 = 2.$$

Sort the scores from largest to smallest:

$$0.9,\ 0.7,\ 0.4,\ 0.3,\ 0.3,\ 0.2,\ 0.2,\ 0.2,\ 0.1,\ 0.1.$$

Keep the first two: the pixels scoring 0.9 and 0.7. Any cut-off strictly between 0.4 and 0.7 gives the same set. If every score in the scene were multiplied by 10, the sorted order would not change and the same two pixels would be kept; that indifference to the overall scale is exactly what a fixed cut-off lacked.
For a real scene of $1{,}210 \times 1{,}219$ pixels, all valid, the top 0.1% is

$$N = 1{,}210 \times 1{,}219 = 1{,}474{,}990,$$
$$k = 0.001 \times 1{,}474{,}990 = 1{,}474.99 \approx 1{,}475 \text{ pixels.}$$

In the product a person makes this choice by dragging a slider over the score map, in a separate step that deliberately pauses for them (Chapter 4). Separating "score" from "cut" also means one expensive scoring run can be tried at five different thresholds.

Thresholds in this system are relative to the scene (percentiles), never absolute numbers.

1.6 Why satellites make this harder

Two complications follow you through the whole book.

1.6.1 Scale

A single scene from the PRISMA satellite (Chapter 3) is about 1,210 by 1,219 pixels with 239 measurements per pixel. Both figures are quoted in module docstrings, the first in the PRISMA reflectance transformer in `app/utils/data_transformations/` [M: `prs_l2d_dn_to_surface_reflectance_transformer.py`] and the second in the PRISMA dataset builder in `app/utils/dataset_builder/` [M: `prisma_dataset_builder.py`]. Count the numbers:

$$1{,}210 \times 1{,}219 = 1{,}474{,}990 \text{ pixels},$$
$$1{,}474{,}990 \times 239 = 352{,}522{,}610 \text{ numbers}.$$

Stored as 4-byte floating-point values that is

$$352{,}522{,}610 \times 4 = 1{,}410{,}090{,}440 \text{ bytes} \approx 1.41 \text{ GB}$$

for one copy of one scene, which is why the code worries about how many copies exist at once.

1.6.2 Not every pixel is real

Sensors have dead detectors. Clouds block the ground. A satellite sweeps a strip, so a rectangular file has ragged, empty edges where the strip did not reach. A large fraction of any scene is data you must actively ignore.

Definition 1.8 (Validity mask)**.** A **validity mask** is a companion array of the same grid as the image holding 1 where the pixel is a genuine measurement and 0 where it is not.

Forgetting the mask is the single most common source of false detections in this codebase. Invalid pixels typically hold 0 or a special "fill" value, and a detector looking for unusual values finds a large block of identical extreme values and ranks it top.

Worked example 1.6: An unmasked zero becomes the top anomaly

Take the five temperatures of worked example 1.2 and add a sixth pixel that is off the edge of the swath, stored as 0. If the detector forgets the mask, the scene is

$$20,\ 21,\ 19,\ 20,\ 30,\ 0.$$

Mean:

$$\text{sum} = 20 + 21 + 19 + 20 + 30 + 0 = 110,$$
$$\mu = 110/6 \approx 18.33.$$

Squared differences from 18.33:

$$\begin{aligned}(20-18.33)^2 &= 1.67^2 \approx 2.79,\\ (21-18.33)^2 &= 2.67^2 \approx 7.13,\\ (19-18.33)^2 &= 0.67^2 \approx 0.45,\\ (20-18.33)^2 &= 1.67^2 \approx 2.79,\\ (30-18.33)^2 &= 11.67^2 \approx 136.19,\\ (0-18.33)^2 &= (-18.33)^2 \approx 335.99.\end{aligned}$$

Sum, average, root:

$$\begin{aligned}2.79+7.13+0.45+2.79+136.19+335.99 &= 485.34,\\ \sigma^2 &= 485.34/6 \approx 80.89,\\ \sigma &\approx 8.99.\end{aligned}$$

Scores of the two extreme pixels:

$$\begin{aligned}s(30) &= 11.67/8.99 \approx 1.30,\\ s(0) &= 18.33/8.99 \approx 2.04.\end{aligned}$$

The fake zero now outranks the genuinely hot pixel, and its presence has halved the hot pixel's score (from 1.98 to 1.30). With the mask applied, the zero is excluded from both the statistics and the ranking, and worked example 1.2 is recovered.

The validity mask appears in Chapter 2 as a "validity cube", becomes binary per pixel in Chapter 5, and decides which tokens a network may see in Chapter 18.

1.7 Notation for the rest of the book

The symbols below are fixed from here on. Later chapters add to the list but do not change it.

Table 1.2: Notation used throughout the book.

Symbol	Meaning
C, H, W	number of bands, rows (height) and columns (width) of an image
c	a band index, $0 \le c \le C-1$
(i, j)	a pixel at row i, column j, both counted from 0
$\mathbf{X}$	a whole image ("cube"), an array of shape `(C, H, W)`
$x_c(i, j)$	the value of band c at pixel (i, j)
$\mathbf{x}$, $\mathbf{x}_{ij}$	one pixel's C values as a column vector (its spectrum, Chapter 2)
$\hat{\mathbf{x}}$	a model's reconstruction of $\mathbf{x}$
$v(i, j)$	validity: 1 for a real measurement, 0 otherwise
$s(i, j)$	anomaly score of pixel (i, j), larger means more unusual
μ, σ	a mean and a standard deviation; $\boldsymbol{\mu}$, $\boldsymbol{\Sigma}$ their many-band versions
λ	wavelength, in nanometres (Chapter 2)

Indices start at 0, as they do in Python, because the book quotes Python code constantly. A cube with $C = 165$ bands has bands 0 to 164.

1.8 Common confusions

So the model is trained to detect anomalies?
No. A foundation model is trained to redraw ordinary imagery; it has no concept of an anomaly. Detection happens afterwards, by subtracting. A classical detector is not trained at all.

Does it need examples of anomalies to work?
No, and that is the point. Neither family is ever shown an anomaly on purpose. Labelled anomalies are used only to *measure* a detector afterwards (Chapter 33), never to build one.

Is a high score a guarantee of something interesting?
No. A high score means "the model could not explain this". Sensor glitches, cloud edges and unusual-but-boring terrain also score high. Turning a high score into a finding is the job of later steps and of a person.

Anomaly detection is outlier detection, then?
Related, but where the comparison is made matters. A pixel can hold a perfectly ordinary temperature for the scene as a whole and still be anomalous because everything right around it is 20 degrees colder. Several detectors score a pixel against its neighbours rather than the whole scene (Chapter 8).

Why not pick one good absolute threshold by experiment?
Because score magnitudes change with the scene by an order of magnitude or more (worked example 1.4). A cut that works on one scene flags nothing or everything on another. Percentiles do not care about the overall scale.

Does "foundation model" here mean a large language model?
No. Here it means a small network (a few million parameters) trained to reconstruct imagery, whose reconstruction error is the score.

1.9 Check yourself

Exercise 1.1. Give the three reasons a supervised classifier is the wrong tool here, one sentence each, and say which escape route each one closes.

Exercise 1.2. A scene has 1,000,000 pixels and 10 are anomalous. Compute, step by step, the accuracy of the rule "always answer normal" and the base rate.

Exercise 1.3. A one-band scene has the four values 10, 12, 14, 20. Compute the mean, the standard deviation (dividing by n), and the score $|x - \mu| / \sigma$ of the pixel reading 20. Show every line.

Exercise 1.4. A scene has 2,000,000 valid pixels. How many pixels does a top 0.05% cut keep? If every score is doubled, does the kept set change? Why?

Exercise 1.5. A three-band pixel is observed as (0.05, 0.25, 0.40); the model reconstructs it as (0.07, 0.22, 0.40). Compute the mean absolute difference. Then explain in two sentences why a pixel stored as 0 because it lies off the swath must be masked out before any such score is ranked.

Next: Chapter 2, Light, bands and the hyperspectral cube.

Chapter 2

Light, bands and the hyperspectral cube

The one thing this chapter teaches

A hyperspectral pixel is not a colour, it is a curve: one reflectance value per narrow band of wavelengths, which you can treat as a vector of C numbers. That curve identifies what the pixel is made of, and a whole image is a three-dimensional array of such curves.

2.1 Why three numbers are not enough

A digital photograph stores three numbers per pixel: how much red, how much green and how much blue. Your eye works the same way, with three kinds of colour receptor, so three numbers are enough to fool it. Three numbers are also very little information. Fresh grass and a sheet of green-painted metal can produce the same red, green and blue triple, and once the three numbers match, nothing distinguishes them.

The sensors in this book record *hundreds* of numbers per pixel instead of three, most of them for light your eye cannot see. With those extra numbers grass and green paint stop looking alike, because grass has a sharp rise in reflected light immediately beyond red and paint does not. This chapter builds the vocabulary for that statement, carefully and with arithmetic: what a wavelength is, what "reflectance" measures, what a band actually records, and how all of it is stored as an array.

2.2 Light and wavelength

Light is an electromagnetic wave. Its **wavelength**, written λ (the Greek letter "lambda"), is the distance from one crest of the wave to the next. Different wavelengths are what we perceive as different colours, and beyond the ends of the visible range the same physics continues invisibly.

Wavelengths of light are tiny, so they are measured in **nanometres** (nm):

$$1 \text{ nm} = 10^{-9} \text{ m} = 0.000\,000\,001 \text{ m}.$$

So 1,650 nm is $1{,}650 \times 10^{-9}$ m $= 1.65 \times 10^{-6}$ m, which is 1.65 micrometres. Remote-sensing papers often quote micrometres (μm); this codebase uses nanometres everywhere.

The hyperspectral sensors in this book cover roughly 400 to 2,500 nm and the shared grid of Chapter 3 keeps 460 to 2,450 nm. Most of that is invisible. Two of the five sensors are thermal: they record emitted heat near 10,000 nm and are treated separately in Section 2.7.

Table 2.1: The wavelength ranges used in this book. Boundaries between the named regions are conventions, not physics.

Region	Range (nm)	What it shows
Visible (VIS)	about 400–700	colour as you see it
Near infrared (NIR)	about 700–1,000	leaf structure, the "red edge"
Shortwave infrared (SWIR)	about 1,000–2,500	water, minerals, clays
Thermal infrared (TIR)	about 10,000–12,000	emitted heat (temperature)

2.3 Radiance, irradiance and reflectance

A satellite does not measure "what the ground is". It measures light arriving at its detector. To turn that into a property of the ground you need three quantities, which are easy to confuse because all three are "amounts of light".

Definition 2.1 (Irradiance)**. Irradiance** $E(\lambda)$ is the power of sunlight *arriving* on a surface, per square metre of surface and per nanometre of wavelength. Units: $\mathrm{W\,m^{-2}\,nm^{-1}}$.

Definition 2.2 (Radiance)**. Radiance** $L(\lambda)$ is the power *leaving* a surface in one particular direction, per square metre, per unit of solid angle (a "steradian", sr, the three-dimensional version of an angle), and per nanometre. Units: $\mathrm{W\,m^{-2}\,sr^{-1}\,nm^{-1}}$. A sensor looking down at the ground measures radiance.

Definition 2.3 (Reflectance)**. Reflectance** $\rho(\lambda)$ (Greek "rho") is the fraction of the arriving light that the surface sends back, at wavelength λ. Because it is a ratio of two amounts of light, it has no units.

2.3.1 Reflectance as a ratio

In its simplest form reflectance is "what came back divided by what arrived". If 1.20 units of light arrive at some wavelength and 0.30 units come back,

$$\rho = \frac{0.30}{1.20} = 0.25,$$

so the surface reflects a quarter of the light at that wavelength. A value of 0 would mean the surface absorbs everything (perfectly black); 1 would mean it returns everything.

The ratio is what makes reflectance useful. If the sun is lower, both the arriving and the returned light fall, and their ratio stays the same. If a different satellite measures the same field, its electronics differ but the surface's ratio does not. Reflectance is a property of the *surface*, which is what lets one model work across sensors and lets a pixel be compared with a laboratory measurement of a known material.

2.3.2 From radiance to reflectance: where the π comes from

The sensor measures radiance in one direction, but the surface sends light in all directions. To compare "what came back" with "what arrived" you need the total returned over the whole sky above the surface. The standard simplifying assumption is a **Lambertian** surface: one that looks equally bright from every viewing direction, so its radiance L is the same in all directions. Matt surfaces such as soil are close to this; water and metal are not.

The total power per square metre leaving such a surface, called its exitance M, adds up the radiance over every direction of the upper hemisphere, weighting each direction by $\cos\theta$ (a beam

leaving at angle θ from vertical crosses a horizontal square metre at a slant, so only $\cos\theta$ of it counts). Measure directions by θ from vertical (0 to $\pi/2$) and φ around the vertical (0 to 2π); a small patch of directions has solid angle $\sin\theta\, d\theta\, d\varphi$. Then:

$$M = \int_0^{2\pi}\int_0^{\pi/2} L\cos\theta\sin\theta\, d\theta\, d\varphi. \tag{2.1}$$

L is the same in every direction (Lambertian), so take it outside both integrals:

$$M = L\int_0^{2\pi} d\varphi \int_0^{\pi/2} \cos\theta\sin\theta\, d\theta. \tag{2.2}$$

The φ integral is the length of the range, 2π. For the θ integral note that the derivative of $\frac{1}{2}\sin^2\theta$ is $\sin\theta\cos\theta$, so

$$\int_0^{\pi/2} \cos\theta\sin\theta\, d\theta = \left[\tfrac{1}{2}\sin^2\theta\right]_0^{\pi/2} = \tfrac{1}{2}(1)^2 - \tfrac{1}{2}(0)^2 = \tfrac{1}{2}. \tag{2.3}$$

Multiply the two pieces:

$$M = L \times 2\pi \times \tfrac{1}{2} = \pi L. \tag{2.4}$$

Sunlight arriving from a sun at angle θ_s from vertical is spread over a larger patch of ground than when the sun is overhead, so the irradiance on a horizontal surface is $E\cos\theta_s$, where E is the irradiance on a surface facing the sun. Reflectance is returned over arrived:

$$\rho = \frac{M}{E\cos\theta_s} = \frac{\pi L}{E\cos\theta_s}. \tag{2.5}$$

Worked example 2.1: Radiance to reflectance

The numbers here are illustrative [A], chosen to be of a realistic size. At 860 nm a sensor measures radiance $L = 0.10\ \mathrm{W\,m^{-2}\,sr^{-1}\,nm^{-1}}$; the solar irradiance is $E = 1.50\ \mathrm{W\,m^{-2}\,nm^{-1}}$ and the sun is $\theta_s = 30°$ from vertical. Do each step on paper.
The cosine of the sun angle:

$$\cos 30° \approx 0.8660.$$

Numerator of Equation (2.5):

$$\pi L = 3.1416 \times 0.10 = 0.31416.$$

Denominator:

$$E\cos\theta_s = 1.50 \times 0.8660 = 1.2990.$$

Divide:

$$\rho = 0.31416/1.2990 \approx 0.242.$$

The surface returns about 24% of the light at 860 nm, typical of vegetation in the near infrared. If the sun dropped to 60° ($\cos 60° = 0.5$) and the surface were unchanged, the arriving light would fall to $1.50 \times 0.5 = 0.75$ and the measured radiance would fall in proportion to $L = 0.242 \times 0.75/\pi \approx 0.0578$; the ratio ρ would still come out as 0.242.

This is background physics. In this codebase the satellite providers have already done the correction for the atmosphere and the sun: PRISMA's *L2D* and EnMAP's *L2A* products are surface reflectance stored as integers, and the code's job is only to undo the integer scaling (Chapter 3). The "L2" prefix means "level 2", a processing level at which the provider delivers a physical surface quantity rather than raw counts.

2.3.3 What real reflectance looks like

Real ground sits mostly between about 0.02 and 0.30: most surfaces return between 2% and 30% of the light. You can check this against the project's own data. The file `app/constants/hyperspectral.json` holds the average of every band over the training corpus (Chapter 5 uses it). Its first value, for 460 nm, is 0.04301 [M: `app/constants/hyperspectral.json`], and the largest of its 165 averages is 0.2775 at 1,270 nm [C] (computed by `research/textbook/tools/compute_light.py`). Both are in the range quoted above.

Values above 1 are not automatically bugs. Sunlight glinting off water or metal can be brighter than a perfect matt reflector would give, and after atmospheric correction it can exceed 1. Do not add a hard clamp at 1 assuming otherwise.

2.4 What a band records

Definition 2.4 (Band)**.** A **band** is one measurement channel of a sensor: the amount of light it records in one narrow range of wavelengths. "Red" is a band; so is "light between 1,645 and 1,655 nm". In deep-learning code the same thing is called a **channel**; this codebase uses both words, often in the same file.

A band is not a knife-edge at one wavelength. A detector element responds to a small range of wavelengths, most strongly at its centre and less towards the sides. The shape of that response is the band's **spectral response function** (SRF), a curve $w(\lambda) \geq 0$. What the band reports is a *weighted average* of the scene's spectrum, with the SRF as the weights:

$$b = \frac{\int w(\lambda)\,\rho(\lambda)\,d\lambda}{\int w(\lambda)\,d\lambda} \approx \frac{\sum_k w_k\,\rho(\lambda_k)}{\sum_k w_k}. \tag{2.6}$$

The right-hand form samples the curve at a few wavelengths λ_k with weights $w_k = w(\lambda_k)$. Read it in words: multiply each reflectance by how strongly the band responds at that wavelength, add the products, and divide by the total weight so that a flat spectrum comes back unchanged.

Worked example 2.2: A band as a weighted average of three samples

A band is centred at 1,650 nm. Sample its response at three wavelengths: 1,645, 1,650 and 1,655 nm, with weights 0.5, 1, 0.5 (half as sensitive 5 nm either side of the centre; worked example 2.3 shows where those halves come from). The surface's reflectance at the three wavelengths is 0.20, 0.22, 0.30 (illustrative [A]). Compute the band value.
Multiply each reflectance by its weight:

$$\begin{aligned} 0.5 \times 0.20 &= 0.10, \\ 1 \times 0.22 &= 0.22, \\ 0.5 \times 0.30 &= 0.15. \end{aligned}$$

Add the products, and add the weights:

$$\begin{aligned} 0.10 + 0.22 + 0.15 &= 0.47, \\ 0.5 + 1 + 0.5 &= 2. \end{aligned}$$

Divide:

$$b = 0.47/2 = 0.235.$$

Compare three possible answers: the reflectance at the centre alone is 0.22; the plain average of the three samples is $(0.20 + 0.22 + 0.30)/3 = 0.72/3 = 0.24$; the band reports 0.235. The band is pulled towards the steeply rising side of the spectrum, but less than an unweighted

average would be. Check the normalisation: if all three reflectances were 0.22, the numerator would be $0.5 \times 0.22 + 0.22 + 0.5 \times 0.22 = 0.44$ and the band would report $0.44/2 = 0.22$, unchanged.

2.4.1 Band width: the full width at half maximum

A common model of an SRF is the bell-shaped **Gaussian** curve

$$w(\lambda) = \exp\left(-\frac{(\lambda - \lambda_c)^2}{2\sigma^2}\right), \tag{2.7}$$

centred at the band's **centre wavelength** λ_c, with width set by σ. At the centre $w = \exp(0) = 1$. Sensor documentation does not quote σ; it quotes the **full width at half maximum** (FWHM): the distance between the two wavelengths where the response has fallen to half its peak. The two are related by a fixed factor, which you can derive.

Find the offset $\Delta = \lambda - \lambda_c$ where the response is one half:

$$\exp\left(-\frac{\Delta^2}{2\sigma^2}\right) = \frac{1}{2}.$$

Take the natural logarithm of both sides (the logarithm undoes the exponential):

$$-\frac{\Delta^2}{2\sigma^2} = \ln \tfrac{1}{2} = -\ln 2.$$

Multiply both sides by $-2\sigma^2$:

$$\Delta^2 = 2\sigma^2 \ln 2.$$

Take the positive square root:

$$\Delta = \sigma\sqrt{2\ln 2}.$$

The half-maximum points sit at $\lambda_c - \Delta$ and $\lambda_c + \Delta$, so the full width is twice Δ:

$$\mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma.$$

Now the number. $\ln 2 \approx 0.6931$, so $2\ln 2 \approx 1.3863$, its square root is about 1.1774, and twice that is about 2.3548:

$$\mathrm{FWHM} \approx 2.3548\,\sigma, \qquad \sigma \approx \frac{\mathrm{FWHM}}{2.3548}. \tag{2.8}$$

Worked example 2.3: The weights of a 10 nm band

A band has FWHM 10 nm. Find σ, then the Gaussian weight 5 nm from the centre.
By Equation (2.8):

$$\sigma = 10/2.3548 \approx 4.2466 \text{ nm}.$$

At $\Delta = 5$ nm, square the offset and the width:

$$\Delta^2 = 25,$$
$$2\sigma^2 = 2 \times 4.2466^2 = 2 \times 18.034 = 36.068.$$

Divide and exponentiate:

$$\Delta^2/(2\sigma^2) = 25/36.068 \approx 0.6931,$$
$$w = \exp(-0.6931) \approx 0.5000.$$

Exactly one half, as it must be: 5 nm is half of the FWHM, and the FWHM is by definition where the response is half. Those are the weights 0.5, 1, 0.5 used in worked example 2.2. For PRISMA, whose bands are about 12 nm wide, $\sigma \approx 12/2.3548 \approx 5.096$ nm [C].

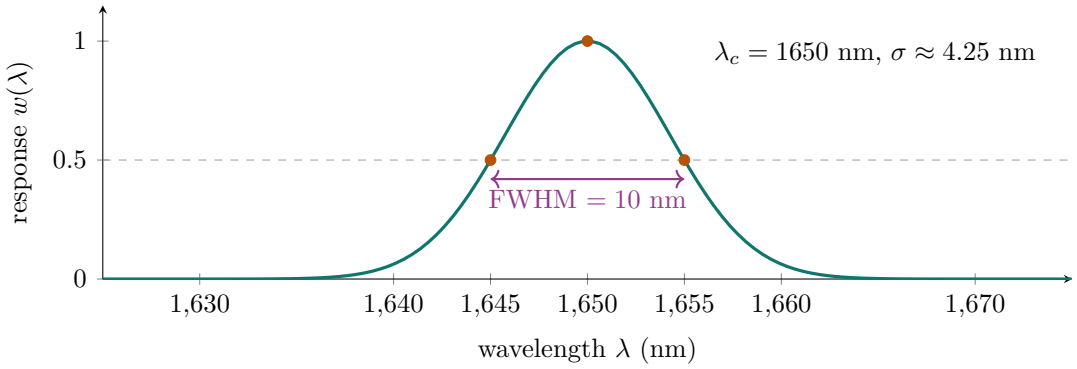


Figure 2.1: A Gaussian spectral response function with FWHM 10 nm, drawn from Equation (2.7). The response falls to one half exactly 5 nm either side of the centre; the three amber points are the samples used in worked example 2.2.

Every band therefore carries two numbers of metadata: its centre wavelength and its FWHM. The codebase stores them as lists, one entry per band (Chapter 4). FWHM matters most for naming materials: comparing a satellite spectrum with a laboratory spectrum means simulating how the laboratory curve would look through *this* sensor's bands, which is Equation (2.6) applied to the laboratory curve. The companion volume on spectral matching works that convolution in full. A scene without FWHM values cannot be material-matched at all.

2.5 A spectrum is a vector

Pick one pixel and read its value in every band in order. You get C numbers. Plotted against the bands' centre wavelengths they form a curve, the pixel's **spectrum**. For mathematics it is more useful to treat those C numbers as a single object.

Definition 2.5 (Vector)**.** A **vector** in $\mathbb{R}^C$ is an ordered list of C real numbers, written as a column,

$$\mathbf{x} = \begin{pmatrix} x_0 \\ x_1 \\ \vdots \\ x_{C-1} \end{pmatrix},$$

where x_c is its c-th **component**. $\mathbb{R}^C$ ("R to the C") is the set of all such lists.

A pixel's spectrum on the shared grid is a vector $\mathbf{x} \in \mathbb{R}^{165}$, and the component x_c is its reflectance in band c. Two operations on vectors are used constantly, and both act component by component.

- **Subtraction**: $(\mathbf{x} - \mathbf{y})_c = x_c - y_c$. The residual of worked example 1.3 was $\mathbf{x} - \hat{\mathbf{x}}$.

- **Scaling** by a number a: $(a\mathbf{x})_c = a\,x_c$.

Worked example 2.4: Two pixels of the same material

Two pixels on the same field have spectra over three bands

$$\mathbf{x} = (0.04,\ 0.24,\ 0.12), \qquad \mathbf{y} = (0.02,\ 0.12,\ 0.06)$$

(illustrative [A]; written as rows to save space). The second pixel lies in the shadow of a hill. Compare them.
Divide each component of $\mathbf{y}$ by the matching component of $\mathbf{x}$:

$$0.02/0.04 = 0.5,$$
$$0.12/0.24 = 0.5,$$
$$0.06/0.12 = 0.5.$$

Every ratio is 0.5, so $\mathbf{y} = 0.5\,\mathbf{x}$: the same shape, half the brightness. Their difference is

$$\mathbf{x} - \mathbf{y} = (0.04 - 0.02,\ 0.24 - 0.12,\ 0.12 - 0.06) = (0.02,\ 0.12,\ 0.06),$$

which is not small, even though the material is identical. A score built on plain differences calls the shadowed pixel different; a score built on the *shape* (the direction of the vector) does not. Chapter 25 builds that second kind of score, the spectral angle.

2.5.1 What a real spectrum looks like

Figure 2.2 plots a real spectrum: the average of each of the 165 grid bands over the corpus used to fit Indradhanu's normalisation (Chapter 5), with a shaded band one standard deviation either side. Three features are worth finding on it.

- **The red edge**. Between 680 and 760 nm the mean rises from 0.0777 to 0.1831 [C], more than doubling in 80 nm. Chlorophyll absorbs red light, and leaf structure scatters near-infrared light strongly, so vegetated pixels jump sharply here. Nothing else does that, which is why it separates grass from green paint.
- **Water absorption**. The curve is cut at four places, around 940, 1,140, 1,400 and 1,875 nm. Water vapour in the air absorbs so strongly there that the bands are removed from the grid (Chapter 3). Next to the gaps the curve dips, for example to 0.1341 at 1,480 nm [C].
- **SWIR fall-off**. Beyond about 1,700 nm reflectance falls, with small dips such as the one near 2,200 nm where clay minerals absorb.

2.6 The cube

Stack one image per band and you get a three-dimensional block of numbers.

Definition 2.6 (Array, tensor and shape)**.** An **array** is a block of numbers indexed by one or more whole-number positions. Its **shape** lists how many positions each index has. A one-index array is a vector, a two-index array is a matrix, and arrays with more indices are commonly called **tensors**. In this book "tensor" means a multi-index array of numbers, nothing more.

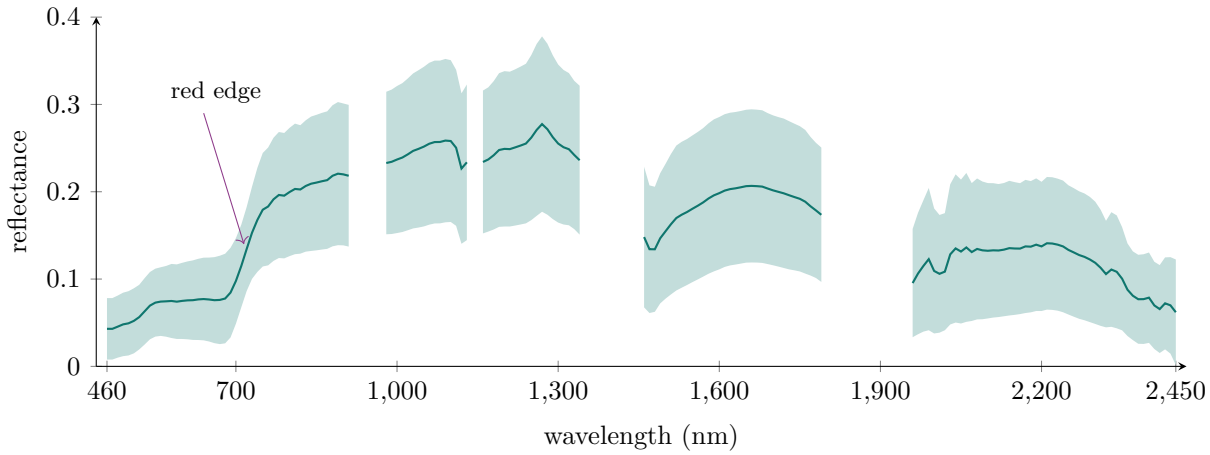


Figure 2.2: The mean reflectance of each of the 165 common-grid bands over the normalisation corpus (teal line), with one standard deviation either side (shaded). Real numbers from `app/constants/hyperspectral.json`, plotted at the grid wavelengths; the four breaks are the atmospheric windows removed from the grid.

Definition 2.7 (Hyperspectral cube)**.** A **hyperspectral cube** is an array $\mathbf{X}$ of shape `(C, H, W)`: C bands, H rows, W columns. Its entry $\mathbf{X}[c, i, j] = x_c(i, j)$ is the reflectance of band c at pixel (i, j). A sensor that records 200 or more narrow bands is called **hyperspectral**.

"Hyperspectral" means high *spectral* resolution: many narrow wavelength ranges. It does not mean sharp images. Spatially these sensors are coarse; a PRISMA or EnMAP pixel covers about 30 metres on the ground. The "cube" is not three-dimensional on the ground either: every layer is a picture of the same flat ground, and the third axis is wavelength, not height.

The cube can be read in two ways, both correct and both used.

- **A stack of images.** Fix the band c and take $\mathbf{X}[c, :, :]$, an `(H, W)` array: the whole scene at one wavelength, a grey picture. Visualisation uses this reading.
- **A grid of spectra.** Fix the pixel (i, j) and take $\mathbf{X}[:, i, j]$, a vector of length C: one pixel at every wavelength. Detectors use this reading.

In Python the colon "`:`" means "every position along this axis".

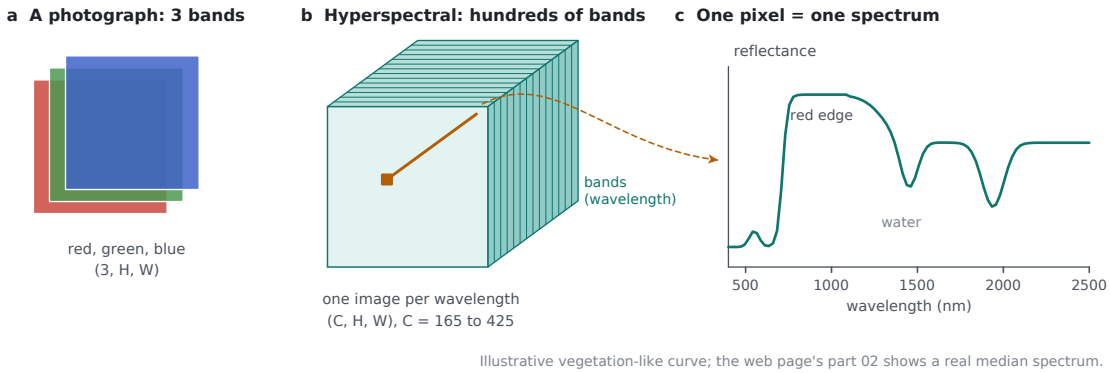


Figure 2.3: From photograph to spectrum. (a) A colour photograph stores three bands per pixel. (b) A hyperspectral cube stores one image per wavelength, with shape `(C, H, W)`. (c) Reading one pixel down the band axis gives its spectrum. (Course figure, Indradhanu part 2; the curve is illustrative.)

Worked example 2.5: Slicing a 3-band, 2-by-2 cube

This small cube is used again in Chapter 5, so copy it down. It has $C = 3$ bands, at 460, 860 and 1,650 nm, and $H = W = 2$ (values illustrative [A]):

$$\mathbf{X}[0] = \begin{pmatrix} 0.03 & 0.05 \\ 0.05 & 0.03 \end{pmatrix}, \quad \mathbf{X}[1] = \begin{pmatrix} 0.20 & 0.30 \\ 0.30 & 0.20 \end{pmatrix}, \quad \mathbf{X}[2] = \begin{pmatrix} 0.14 & 0.10 \\ 0.14 & 0.10 \end{pmatrix}.$$

Each matrix is one band; rows are $i = 0, 1$ and columns $j = 0, 1$.
A band. $\mathbf{X}[1, :, :]$ is the 860 nm image, the middle matrix.
A spectrum. For pixel $(0, 1)$ (row 0, column 1) read position $(0, 1)$ of each matrix in turn:

$$\mathbf{X}[0, 0, 1] = 0.05,$$
$$\mathbf{X}[1, 0, 1] = 0.30,$$
$$\mathbf{X}[2, 0, 1] = 0.10,$$

so $\mathbf{x}_{01} = (0.05,\ 0.30,\ 0.10)$. The other three spectra are $\mathbf{x}_{00} = (0.03,\ 0.20,\ 0.14)$, $\mathbf{x}_{10} = (0.05,\ 0.30,\ 0.14)$ and $\mathbf{x}_{11} = (0.03,\ 0.20,\ 0.10)$.
Counting. The cube holds $3 \times 2 \times 2 = 12$ numbers: 3 bands of 4 pixels, or equivalently 4 spectra of 3 values.

Worked example 2.6: The size of a real cube

A PRISMA cube has shape `(239, 1210, 1219)` stored as `float32` (4 bytes per number). How large is it? Count the pixels, then the numbers, then the bytes:

$$1{,}210 \times 1{,}219 = 1{,}474{,}990 \text{ pixels},$$
$$1{,}474{,}990 \times 239 = 352{,}522{,}610 \text{ numbers},$$
$$352{,}522{,}610 \times 4 = 1{,}410{,}090{,}440 \text{ bytes} \approx 1.41 \text{ GB}.$$

After resampling to the 165-band grid of Chapter 3 the same scene needs

$$1{,}474{,}990 \times 165 \times 4 = 973{,}493{,}400 \text{ bytes} \approx 0.97 \text{ GB}.$$

Table 2.2: Vocabulary to pin down now. The two easiest to confuse are band and spectrum: a band is a horizontal slice (one wavelength, all pixels), a spectrum a vertical skewer (one pixel, all wavelengths).

Word	What it is	Shape
band	one wavelength layer, a grey picture	`(H, W)`
spectrum	one pixel's values across all bands	`(C,)`
cube	the whole stack	`(C, H, W)`
patch	a small square cut from a cube for training	`(165, 128, 128)`
scene	one complete acquisition	`(C, about 1000, about 1000)`
validity cube	the 0/1 companion of the cube (Section 2.8)	`(C, H, W)`

2.7 Thermal images: the one-band cousin

Two of the five sensors, Landsat 9 and HotSat-1, are **thermal**: they record one band in the thermal infrared, which measures how much heat the ground *emits*, and so how hot it is. A thermal image is the same cube structure with $C = 1$, shape `(1, H, W)`. "Hotter than its

surroundings" is exactly what a fire, a gas flare or a running machine looks like, so anomaly detection is still useful, but there is no spectrum to compare. The thermal model Chakshu shares almost all its code with Indradhanu; everywhere this book says "165 bands", Chakshu says "1 band".

A sensor does not store physics directly. It stores **digital numbers** (DN): integers proportional to the energy that hit the detector, scaled by the electronics. DN are not comparable across sensors, dates or sun angles, so the first processing step converts them to a physical quantity. For Landsat's surface-temperature product the conversion is a straight line with two published constants [M: `l2sp_dn_to_temperature_transformer.py`], in `app/utils/data_transformations/l2sp_dn_to_temperature_transformer.py`:

$$T_{\mathrm{K}} = 0.00341802 \times \mathrm{DN} + 149.0, \qquad T_{^\circ\mathrm{C}} = T_{\mathrm{K}} - 273.15. \tag{2.9}$$

The first gives temperature in kelvin (K), the scale that starts at absolute zero; subtracting 273.15 converts to degrees Celsius.

Worked example 2.7: A Landsat digital number to Celsius

A Landsat pixel reports DN = 30,000. Do each step on paper.
Multiply by the scale factor:

$$0.00341802 \times 30{,}000 = 102.5406.$$

Add the offset to get kelvin:

$$T_{\mathrm{K}} = 102.5406 + 149.0 = 251.5406 \text{ K}.$$

Subtract 273.15:

$$T_{^\circ\mathrm{C}} = 251.5406 - 273.15 = -21.6094 \approx -21.61\ ^\circ\mathrm{C}.$$

Minus 21.6 degrees: plausibly the top of a cold cloud.

2.8 Validity: not every number is a measurement

Chapter 1 introduced the validity mask. For hyperspectral data the codebase keeps one validity value per band per pixel.

Definition 2.8 (Validity cube)**.** The **validity cube V** has the same shape `(C, H, W)` as the cube, with $\mathbf{V}[c, i, j] = 1$ if band c of pixel (i, j) is a real measurement and 0 if it is not (dead detector, cloud, off-swath).

After the processing of Chapter 5, hyperspectral validity is *all-or-nothing per pixel*: a pixel is valid in all bands or in none. That lets code collapse the cube to one `(H, W)` map, and the codebase does it in two different ways. The training code reads band 0 alone (`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, line 160):

```
# Use band 0 as spatial validity proxy: (B, C, H, W) -> (B, 1, H, W)
return validity[:, 0:1, :, :].float()
```

The scoring code sums over bands and asks whether any band is valid (`backend/allotrope/action_types/_anomaly_scoring_run.py`, line 174):

```
spatial_valid = (validity_np.sum(axis=0) > 0).astype(np.uint8)    # (H, W)
```

Here `B` is the **batch** size, the number of image patches processed together in one step; `sum(axis=0)` adds along the first axis (the bands).

Worked example 2.8: Two ways to collapse validity

A 3-band, 1×3 image (one row, three pixels) has validity

$$\mathbf{V}[0] = (1,\ 0,\ 0), \quad \mathbf{V}[1] = (1,\ 0,\ 1), \quad \mathbf{V}[2] = (1,\ 0,\ 1).$$

Sum over bands, pixel by pixel:

$$\begin{aligned} \text{pixel } 0: &\ 1+1+1 = 3, \\ \text{pixel } 1: &\ 0+0+0 = 0, \\ \text{pixel } 2: &\ 0+1+1 = 2. \end{aligned}$$

"Sum > 0" gives $(1,\ 0,\ 1)$. "Band 0 only" gives $(1,\ 0,\ 0)$. They disagree on pixel 2, which is valid in two bands but not in band 0. That pixel is *not* all-or-nothing, so this validity cube has not been through the pipeline. After the pipeline, pixel 2 would have been either filled to $(1, 1, 1)$ or wiped to $(0, 0, 0)$ (Chapter 5), and both methods would agree. The training code's shortcut reads one band instead of 165, which is 165 times less memory traffic on an operation done for every batch; it is safe only because of the all-or-nothing guarantee.

The two collapses agree only on cubes that have been through the band pipeline; on raw validity the band-0 shortcut is wrong.

2.9 Common confusions

Is a band the same as a channel?
In practice yes. Remote sensing says band, deep learning says channel; the C in `(C, H, W)` is the band count either way.

Does hyperspectral mean high resolution?
It means high *spectral* resolution, many narrow bands. Spatially a pixel is about 30 m, coarse by photographic standards.

Is the band value the reflectance at the centre wavelength?
Not exactly. It is a weighted average over the band's response function (Equation (2.6)). For narrow bands and a smooth spectrum the two are close; near a sharp feature they differ (worked example 2.2).

Are more bands always better?
No. Adjacent bands 10 nm apart carry nearly the same information, each narrow band is noisier, and many bands relative to usable pixels make some statistics numerically unstable (Chapter 7). Indradhanu's first layer compresses the 165 bands for exactly this reason (Chapter 12).

Reflectance is between 0 and 1, so values above 1 are bugs?
Usually, not always. Specular glint off water or metal can exceed 1. Do not clamp blindly.

Radiance or reflectance: which does the model see?
Reflectance. The providers' level-2 products are already surface reflectance (as scaled integers); the codebase undoes the integer scaling and never handles radiance.

If validity is all-or-nothing, why is the validity cube C layers deep?
Because the same type is used at earlier pipeline stages where validity genuinely varies per band. By the time a model sees it, the layers agree.

2.10 Check yourself

Exercise 2.1. A band has FWHM 12 nm. Compute σ to three decimal places, then the Gaussian weight 3 nm from the centre. Show every step.

Exercise 2.2. A band samples wavelengths 1,000, 1,005, 1,010 nm with weights 0.5, 1, 0.5; the reflectances there are 0.30, 0.26, 0.18. Compute the band value and compare it with the reflectance at the centre.

Exercise 2.3. A Landsat pixel reports DN = 41,000. Compute its temperature in kelvin and in Celsius.

Exercise 2.4. Using the cube of worked example 2.5, write the spectrum of pixel $(1, 0)$ and the 1,650 nm band as a matrix. Is pixel $(1, 0)$ a scaled copy of pixel $(0, 0)$? Check by dividing component by component.

Exercise 2.5. With the numbers of worked example 2.1, a second surface gives radiance $L = 0.05$ under the same sun. What is its reflectance? Then explain in one or two sentences why reflectance, not radiance, is what lets one model be used on several sensors.

Next: Chapter 3, Five sensors and the 165-band grid.

Chapter 3

Five sensors and the 165-band grid

The one thing this chapter teaches

Five sensors hand you five incompatible things, and one decision fixes the worst of the mismatch: resample every hyperspectral sensor onto one list of wavelengths, 10 nm apart from 460 to 2,450 nm with four atmospheric windows removed. That list has exactly 165 entries, and you can derive the 165 on paper.

3.1 Why this chapter exists

Chapter 2 described a hyperspectral cube as if every sensor produced the same thing. They do not. The five sensors this product accepts disagree about file format, about the order of the axes in memory, about how stored integers become physical units, about which quality information is shipped, and, most damagingly, about *which wavelengths* they measure.

The last disagreement is the one that matters for a model. Band number 47 of one sensor and band number 47 of another sit at different wavelengths. A network has one set of learned weights per input position, so if you hand it both sensors unchanged, the weights for position 47 have to learn two contradictory things. The fix is a shared list of wavelengths, the **common grid**, and the second half of this chapter derives it. The first half is the mess that the fix hides, because every awkward corner of the ingestion code exists to hide it.

3.2 The roster

Table 3.1: The five sensors. Band counts drift as products are reprocessed; the structural differences do not. Sources: module docstrings of the dataset builders and `docs/03-data-pipeline.md`.

Sensor	Kind	Bands	Upload	Units after onboarding
PRISMA	hyperspectral	239 (66 + 173)	one `.he5` file	reflectance
EnMAP	hyperspectral	224	a folder	reflectance
AVIRIS-NG	hyperspectral	about 425	a folder	reflectance
Landsat 9	thermal	1 (band B10)	one `.tif` file	degrees Celsius
HotSat-1	thermal	1	a folder	raw DN

VNIR and SWIR are the names of a hyperspectral sensor's two detectors: one array for the visible and near infrared, one for the shortwave infrared (Table 2.1). PRISMA stores the two as separate datasets inside one file, and its builder stitches them into one cube. For AVIRIS-NG the fine-tuning shards use the 372 bands its bad-band list marks good [M: `docs/03-data-pipeline.md`].

File shape. PRISMA and Landsat are single files; EnMAP, AVIRIS-NG and HotSat-1 are folders in which the cube, the masks and the metadata are separate files. So "the path to a scene" sometimes means a file and sometimes a directory, resolved per sensor in `backend/allotrope/sensors/source_path.py`.

3.3 Three ways to lay a cube out in memory

A computer's memory is one long line of numbers. A three-index array has to be flattened onto that line, and the order in which the three indices are flattened is called its **layout**. Hyperspectral data uses three.

Table 3.2: The three cube layouts.

Layout	Axes	Contiguous in memory	Who uses it
BSQ, band-sequential	`(C, H, W)`	one whole band image	EnMAP, Landsat, every model
BIL, band-interleaved-by-line	`(H, C, W)`	one image row of one band	PRISMA (native)
BIP, band-interleaved-by-pixel	`(H, W, C)`	one pixel's whole spectrum	visualisation, spectral matching

3.3.1 Where an element lives: the offset formula

NumPy, PyTorch and this codebase store arrays in **row-major** order: the *last* index changes fastest. For an array of shape `(A, B, D)`, the element at index (a, b, d) sits at position

$$\text{offset}(a, b, d) = a \cdot (B \cdot D) + b \cdot D + d \tag{3.1}$$

counted from 0. Read it as a mileage count: to reach element (a, b, d) you first skip a whole blocks of size $B \cdot D$, then b rows of size D, then d single numbers. The three layouts are three choices of which cube index plays a, b and d:

$$\begin{aligned} \text{BSQ } (C, H, W): &\quad \text{offset} = c\,HW + i\,W + j, \\ \text{BIL } (H, C, W): &\quad \text{offset} = i\,CW + c\,W + j, \\ \text{BIP } (H, W, C): &\quad \text{offset} = i\,WC + j\,C + c. \end{aligned}$$

Worked example 3.1: One element, three addresses

The cube of worked example 2.5 has $C = 3$, $H = 2$, $W = 2$. Find where the element "band $c = 1$, row $i = 0$, column $j = 1$" (the value 0.30) lives in each layout. Do it on paper.
BSQ: $HW = 2 \times 2 = 4$, so

$$\text{offset} = 1 \times 4 + 0 \times 2 + 1 = 4 + 0 + 1 = 5.$$

BIL: $CW = 3 \times 2 = 6$, so

$$\text{offset} = 0 \times 6 + 1 \times 2 + 1 = 0 + 2 + 1 = 3.$$

BIP: $WC = 2 \times 3 = 6$, so

$$\text{offset} = 0 \times 6 + 1 \times 3 + 1 = 0 + 3 + 1 = 4.$$

The same number lives at position 5, 3 or 4 depending on the layout. Nothing in the file says which layout it is; the reader has to know.

Worked example 3.2: Reading BIL as if it were BSQ

Write out the whole BIL memory line of that cube. BIL loops over rows i (outer), then bands c, then columns j (inner). Row 0: band 0 gives $0.03, 0.05$; band 1 gives $0.20, 0.30$; band 2 gives $0.14, 0.10$. Row 1: $0.05, 0.03$; $0.30, 0.20$; $0.14, 0.10$. So memory holds

$$0.03,\ 0.05,\ 0.20,\ 0.30,\ 0.14,\ 0.10,\ 0.05,\ 0.03,\ 0.30,\ 0.20,\ 0.14,\ 0.10.$$

A reader who wrongly assumes BSQ takes the first $HW = 4$ numbers as band 0:

$$\text{“band 0”} = \begin{pmatrix} 0.03 & 0.05 \\ 0.20 & 0.30 \end{pmatrix}.$$

Its bottom row is really near-infrared data. Every “band” it builds is a mixture of rows of different bands, and on a real scene the result looks like plausible but wrong static. If a cube ever looks like structured noise, check the axis order before suspecting the data.

The fix is a **transpose**: reorder the axes without changing any value. BIL `(H, C, W)` to BSQ `(C, H, W)` swaps the first two axes, in NumPy `np.transpose(cube,(1,0,2))`. The codebase does it in `ImageCubeOperations.convert_cube` (`app/utils/image_transformation/image_cube_operations.py`) with a permutation of the axes.

The layouts are not arbitrary. BIP puts one pixel's whole spectrum next to itself in memory, so a per-pixel spectral calculation is a fast linear walk; BSQ puts a whole band image together, so a spatial filter over one band is. Choosing the wrong one costs nothing in correctness and a great deal in speed.

3.4 From stored integers to physical units

Every sensor stores integers. There is no shared formula for turning them into physics.

EnMAP. One uniform gain for all 224 bands, set as `GAIN:float=0.0001` in the EnMAP reflectance transformer [M: `enmap_l2a_dn_to_surface_reflectance_transformer.py`]:

$$\rho = \mathrm{DN} \times 0.0001. \tag{3.2}$$

PRISMA. A per-detector linear map read from the file's own metadata (`app/utils/data_transformations/prs_l2d_dn_to_surface_reflectance_transformer.py`):

$$\rho = \frac{\mathrm{DN}}{65535}\left(\mathrm{L2ScaleMax} - \mathrm{L2ScaleMin}\right) + \mathrm{L2ScaleMin}. \tag{3.3}$$

$65535 = 2^{16} - 1$ is the largest 16-bit unsigned integer, so DN/65535 runs from 0 to 1, and the two scale attributes stretch that onto the reflectance range the provider chose for this file.

Landsat 9. The straight line of Equation (2.9), to Celsius.

HotSat-1. No conversion: it ships raw 14-bit digital numbers around 5,000 with a spread of about 400 [M: `app/models/training/inference_config.py`]. Chapter 5 shows what that does to a model fitted on other units.

Worked example 3.3: Two reflectance conversions

EnMAP. A pixel stores DN = 4,250. By Equation (3.2),

$$\rho = 4{,}250 \times 0.0001 = 0.425.$$

That is 42.5% reflectance, plausible for bright bare soil or a light roof; vegetation in red light would be far lower, around 0.05.
PRISMA. Take L2ScaleMin = 0 and L2ScaleMax = 0.7 (illustrative values [A]; each real file carries its own). A pixel stores DN = 20,000. Step by step:

$$\begin{aligned} \text{DN}/65535 &= 20{,}000/65{,}535 \approx 0.30518, \\ \text{L2ScaleMax} - \text{L2ScaleMin} &= 0.7 - 0 = 0.7, \\ 0.30518 \times 0.7 &\approx 0.21363, \\ \rho &= 0.21363 + 0 \approx 0.2136. \end{aligned}$$

3.5 Quality information, and reading bits

The sensors also disagree about which quality information they ship. EnMAP ships ready-made masks for cloud, cirrus, haze, cloud shadow and snow, one `(H, W)` layer each. PRISMA's L2D product ships none. Landsat ships one integer layer, `QA_PIXEL`, in which each *bit* is a separate yes/no flag. That asymmetry is why the band-filter setting `quality_masks_to_apply` exists and applies to EnMAP alone (Chapter 5).

Definition 3.1 (Bit flag)**.** An integer written in base 2 is a row of binary digits (bits), bit k worth 2^k. A **bit flag** uses bit k as a separate yes/no answer. To read bit k of a value q, shift q right by k places (divide by 2^k and drop the remainder) and keep the last bit (the remainder on division by 2). In code: `(q>>k)&1`.

The Landsat builder (`app/utils/dataset_builder/landsat_dataset_builder.py`, lines 187–207) marks a pixel unusable if any of bits 0 (fill), 1 (dilated cloud), 3 (cloud), 4 (cloud shadow) or 15 (high bit of the cirrus confidence) is set.

Worked example 3.4: Unpacking two QA values

Take two example values [A], q = 21,824 and q = 22,280. First write 21,824 as a sum of powers of two, subtracting the largest power that fits each time:

$$\begin{aligned} 21{,}824 - 16{,}384 &= 5{,}440 \quad (2^{14}), \\ 5{,}440 - 4{,}096 &= 1{,}344 \quad (2^{12}), \\ 1{,}344 - 1{,}024 &= 320 \quad (2^{10}), \\ 320 - 256 &= 64 \quad (2^{8}), \\ 64 - 64 &= 0 \quad (2^{6}). \end{aligned}$$

So bits $14, 12, 10, 8, 6$ are set. None of bits $0, 1, 3, 4, 15$ is set, so the pixel is *valid*. (Bit 6 is the "clear" flag.)

Now 22,280:

$$\begin{aligned} 22{,}280 - 16{,}384 &= 5{,}896 \quad (2^{14}),\\ 5{,}896 - 4{,}096 &= 1{,}800 \quad (2^{12}),\\ 1{,}800 - 1{,}024 &= 776 \quad (2^{10}),\\ 776 - 512 &= 264 \quad (2^{9}),\\ 264 - 256 &= 8 \quad (2^{8}),\\ 8 - 8 &= 0 \quad (2^{3}). \end{aligned}$$

Bit 3 (cloud) is set, so the pixel is *flagged*. Check bit 3 the way the code does: $22{,}280 \gg 3$ is $\lfloor 22{,}280/8 \rfloor = 2{,}785$, and 2,785 is odd, so $2{,}785 \,\&\, 1 = 1$.

3.6 How the code hides the differences

Two layers keep sensor-specific knowledge in one place.

- A **FileHelper** wraps one physical file format and hands back raw arrays and metadata: `HE5Helper` (PRISMA), `TIFHelper` (Landsat), `EnmapHelper`, `ENVIHelper` (AVIRIS-NG), `HotSatHelper`.
- A **template** is a dictionary telling a helper where each logical component lives inside the file, by one of three strategies (`app/models/hyperspectral_concepts/references.py`): `FILE_REFERENCE` (an internal path inside the container, such as PRISMA's SWIR cube dataset), `ROOT_METADATA_FIELD` (a root-level attribute, such as PRISMA's list of SWIR centre wavelengths) and `DIRECT_PROPERTY_DEFINITION` (a property on the opened dataset, such as rasterio's `crs`).

Templates are *injected, not looked up*: the helper never contains a sensor-specific path, which is what lets one contract serve HDF5, GeoTIFF and ENVI files. Only PRISMA, Landsat and EnMAP have templates; AVIRIS-NG and HotSat-1 parse their own sidecar files, an inconsistency the source notes honestly.

Three traps will each cost you an afternoon.

- **rasterio band indices start at 1.** Every other array in the codebase counts from 0. Asking rasterio for band 0 is an error.
- **PRISMA keeps its bad bands.** A band the sensor flagged unusable still occupies a slice of the cube, with FWHM = 0.0. Filter on the validity flag before trusting anything about a band.
- **The AVIRIS-NG sample cubes are not in the repository.** `samples/aviris_samples/` holds the ENVI `.hdr` headers and `.sha256` checksums; the `.bin` data files are absent, so the AVIRIS-NG path cannot run locally without them.

3.7 Sensors that do not agree on wavelengths

Now the disagreement that matters for a model. PRISMA, EnMAP and AVIRIS-NG all "measure the spectrum", but at different wavelengths, with different spacing and different totals. In made-up but representative numbers [A]:

PRISMA	…, 1,000, 1,012, 1,024, … nm
EnMAP	…, 1,003, 1,011, 1,019, … nm
AVIRIS-NG	…, 1,001, 1,006, 1,011, … nm

Training data is the bottleneck for a model. One model per sensor gives each model a third of the data, triples the work of training and maintenance, and makes every new sensor a new model. One model for all three gives it all the data, and lets a new sensor join without retraining, but only if position c of the input means the same wavelength for every sensor.

The fix is to agree, once, on a fixed list of wavelengths, and to **resample** every sensor onto it before anything else touches the data. Resampling means: "you measured at 1,001 and 1,006; I need a value at 1,000; estimate one from your neighbours". The estimating is interpolation, derived in Chapter 5. This chapter builds the list.

Analogy

Three weather stations log temperature at different times: one at 06:07, 06:19, 06:31; another at 06:03, 06:11, 06:19; a third every five minutes. To put all three on one chart you agree on the times you will report (06:00, 06:10, 06:20, …) and estimate each station's temperature at those times from its nearest readings. Two decisions remain: how far apart the reporting times are (not finer than the slowest station's readings, or you invent detail), and which times to leave out entirely (say, the hour each night when a station is switched off, because any value reported there would be made up). The common grid makes exactly these two decisions for wavelengths.

3.8 Building the grid

The list is `DEFAULT_COMMON_WAVELENGTH_GRID` in `app/models/dataset/vendables.py`, built by this function (lines 35–62, docstring trimmed):

```
def build_common_wavelength_grid(start_nm=460.0, end_nm=2450.0,
                                 spacing_nm=10.0, exclusion_ranges=None):
    if exclusion_ranges is None:
        exclusion_ranges = [
            (0, 450),       # low SNR
            (912, 978),     # water vapor
            (1131, 1152),   # water vapor
            (1350, 1450),   # water vapor absorption
            (1800, 1950),   # water vapor + CO2
        ]
    full_grid = np.arange(start_nm, end_nm + spacing_nm / 2, spacing_nm)  # (200,)
    mask = np.ones(len(full_grid), dtype=bool)
    for lo, hi in exclusion_ranges:
        mask &= ~((full_grid >= lo) & (full_grid <= hi))   # drop lo <= x <= hi
    return full_grid[mask]                                 # (165,)
```

Two decisions, then: a ladder of evenly spaced wavelengths, and a list of ranges cut out of it. Each can be counted by hand.

3.8.1 Step 1: the ladder, and the fencepost formula

The ladder starts at 460, steps by 10, and stops at 2,450:

$$460,\ 470,\ 480,\ \ldots,\ 2{,}440,\ 2{,}450.$$

How many rungs? Number them: the rung at wavelength x is rung number $k = (x - 460)/10$, so 460 is rung 0, 470 is rung 1, and 2,450 is rung $(2{,}450 - 460)/10 = 199$. Rungs 0 to 199 are 200 rungs. In general:

$$\text{number of rungs} = \frac{\text{last} - \text{first}}{\text{step}} + 1. \tag{3.4}$$

The "+1" is the **fencepost** correction: a fence 10 metres long with a post every metre has 11 posts, because the division counts gaps, not posts.

$$\begin{aligned} \frac{2{,}450 - 460}{10} + 1 &= \frac{1{,}990}{10} + 1 \\ &= 199 + 1 \\ &= 200 \text{ rungs.} \end{aligned}$$

Source check: the half-step in `np.arange`

`np.arange(start,stop,step)` stops *before* `stop`, so `np.arange(460,2450,10)` would end at 2,440 and have 199 entries. Adding half a step makes the limit 2,455, which includes 2,450 without adding 2,460. Verified: `len(np.arange(460.0,2450.0,10.0))` is 199 and the code's ladder has 200 entries (`research/textbook/tools/compute_sensors_output.txt`).

3.8.2 Step 2: count the rungs inside each window

The atmosphere is not transparent. At some wavelengths water vapour and carbon dioxide absorb the light almost completely before it reaches the satellite; a band there measures noise, and noise whose strength changes with humidity, so it looks like structure. Five ranges are removed, each with both ends included ("$lo \le x \le hi$", the last line of the loop).

Every rung is a multiple of 10. The multiples of 10 inside $[lo, hi]$ run from the first multiple at or above lo to the last at or below hi. Writing $\lceil y \rceil$ for "round y up" and $\lfloor y \rfloor$ for "round y down", those are $10\lceil lo/10 \rceil$ and $10\lfloor hi/10 \rfloor$, and by Equation (3.4) with step 10 (dividing through by 10):

$$\text{rungs removed} = \left\lfloor \frac{hi}{10} \right\rfloor - \left\lceil \frac{lo}{10} \right\rceil + 1, \tag{3.5}$$

provided the range lies on the ladder (a range entirely below 460 removes nothing). Do each window on paper.

Worked example 3.5: Counting the five windows

Window $(0, 450)$. Every rung is at least $460 > 450$. Nothing removed: 0.
Window $(912, 978)$.

$$\begin{aligned} \lceil 912/10 \rceil &= \lceil 91.2 \rceil = 92, \\ \lfloor 978/10 \rfloor &= \lfloor 97.8 \rfloor = 97, \\ 97 - 92 + 1 &= 6. \end{aligned}$$

The six rungs are $920, 930, 940, 950, 960, 970$. Check the edges: $910 < 912$ is kept; $980 > 978$ is kept.
Window $(1131, 1152)$.

$$\lceil 113.1 \rceil = 114, \qquad \lfloor 115.2 \rfloor = 115,$$
$$115 - 114 + 1 = 2 \quad (1{,}140 \text{ and } 1{,}150).$$

Window $(1350, 1450)$. Both ends land on rungs and both are included:

$$\lceil 135 \rceil = 135, \qquad \lfloor 145 \rfloor = 145,$$
$$145 - 135 + 1 = 11 \quad (1{,}350, 1{,}360, \ldots, 1{,}450).$$

Window $(1800, 1950)$.

$$180 \text{ and } 195: \quad 195 - 180 + 1 = 16 \quad (1{,}800, \ldots, 1{,}950).$$

Total.

$$\text{removed} = 0 + 6 + 2 + 11 + 16 = 35,$$
$$\text{remaining} = 200 - 35 = 165.$$

Source check: 165 checked against the source

Running the code's own mask over the ladder gives the same per-window counts $0, 6, 2, 11, 16$ and `len(DEFAULT_COMMON_WAVELENGTH_GRID)` = 165, first value 460.0, last 2,450.0 (`research/textbook/tools/compute_sensors.py` and its saved output) [C]. Nothing about 165 is chosen directly: it is a consequence of the ladder and the five windows.

$165 = 200 - (0 + 6 + 2 + 11 + 16)$: a 10 nm ladder from 460 to 2,450 nm minus the rungs the atmosphere ruins.

3.8.3 The five segments

The surviving rungs fall into five unbroken runs, 10 nm apart within each run, with jumps between runs. Count each with Equation (3.4):

$$\begin{aligned}
460 \text{ to } 910: &\quad (910 - 460)/10 + 1 = 45 + 1 = 46,\\
980 \text{ to } 1{,}130: &\quad (1{,}130 - 980)/10 + 1 = 15 + 1 = 16,\\
1{,}160 \text{ to } 1{,}340: &\quad (1{,}340 - 1{,}160)/10 + 1 = 18 + 1 = 19,\\
1{,}460 \text{ to } 1{,}790: &\quad (1{,}790 - 1{,}460)/10 + 1 = 33 + 1 = 34,\\
1{,}960 \text{ to } 2{,}450: &\quad (2{,}450 - 1{,}960)/10 + 1 = 49 + 1 = 50.
\end{aligned}$$

Check: $46 + 16 + 19 + 34 + 50 = 165$. The four jumps (from 910 to 980, 1,130 to 1,160, 1,340 to 1,460, 1,790 to 1,960) are the four windows that touch the ladder. Figure 3.1 draws the whole grid to scale.

Source check: "spectral families" in the vendable

The Indradhanu course (part 3) calls the five segments "spectral families" and says the vendable records which of the five each band is in, in `spectral_family_order`. The source disagrees: `spectral_family_order` holds values of the enum `SpectralFamily` (`app/models/hyperspectral_concepts/spectral_family.py`), whose members are `VNIR`, `SWIR` and `PANCHROMATIC`. It records which *detector* a band came from, not which of the five segments it lies in. On the common grid each band is labelled with the detector of its nearest native band (`_assign_families` in `app/utils/data_transformations/spectral_resampler.py`). The five segments have no name in the code; they exist only as the jumps in the grid. Chapter 5 reports a problem with that nearest-band labelling.

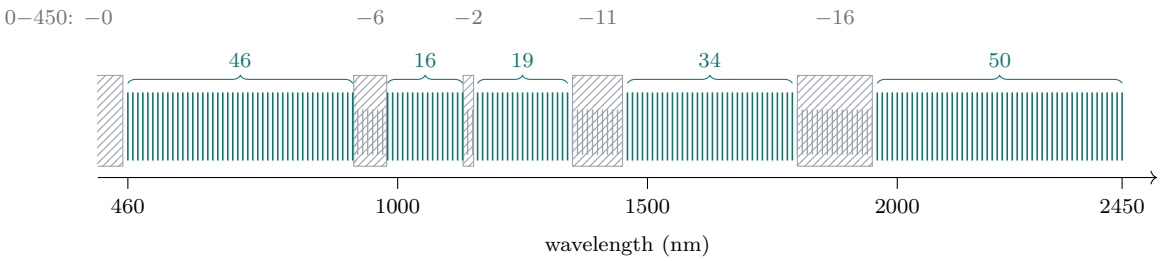


Figure 3.1: The common grid drawn to scale. Ticks are the 200 ladder rungs from 460 to 2,450 nm; grey rungs fall inside the hatched atmospheric windows and are removed ($6+2+11+16=35$), teal rungs are kept. Braces give the number of kept bands in each of the five segments ($46+16+19+34+50=165$). The window 0–450 nm lies below the ladder and removes nothing.

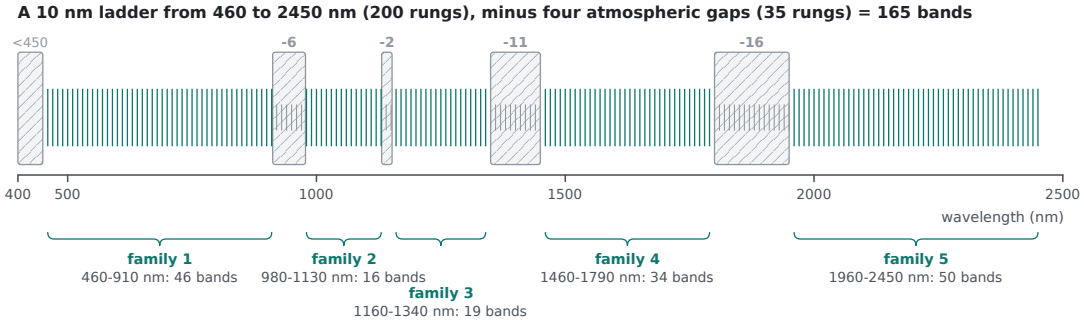


Figure 3.2: The same grid as drawn in the Indradhanu course (part 3): a ladder of 200 wavelengths loses 35 rungs to four atmospheric windows, leaving 165 bands in five segments of 46, 16, 19, 34 and 50.

3.8.4 From band index to wavelength and back

Because the segments are evenly spaced inside, the wavelength of grid band c ($0 \le c \le 164$) is a piecewise formula. Inside a segment that starts at index c_0 and wavelength λ_0, each step of the index adds 10 nm:

$$\lambda(c) = \lambda_0 + 10\,(c - c_0), \qquad (c_0, \lambda_0) \in \{(0, 460),\ (46, 980),\ (62, 1{,}160),\ (81, 1{,}460),\ (115, 1{,}960)\}. \tag{3.6}$$

The segment starts are the running totals of the segment sizes: 0, then $0+46=46$, then $46+16=62$, then $62+19=81$, then $81+34=115$.

Worked example 3.6: Which wavelength is band 43, and which band is 1,000 nm?

Band 43. $43 < 46$, so it is in the first segment, $(c_0, \lambda_0) = (0, 460)$:

$$\lambda(43) = 460 + 10 \times (43 - 0) = 460 + 430 = 890 \text{ nm}.$$

1,000 nm. It lies in the second segment (980 to 1,130). Invert Equation (3.6):

$$\begin{aligned} c &= c_0 + (\lambda - \lambda_0)/10 \\ &= 46 + (1{,}000 - 980)/10 \\ &= 46 + 2 = 48. \end{aligned}$$

Band 100. $81 \le 100 < 115$, so $(c_0, \lambda_0) = (81, 1{,}460)$ and

$$\lambda(100) = 1{,}460 + 10 \times 19 = 1{,}650 \text{ nm}.$$

All three agree with the code's array (`research/textbook/tools/compute_sensors_output.txt`) [C].

Source check: a numeric slip in the course

The Indradhanu course (part 3) illustrates the "wire format" warning below with the phrase "band 43 means 1000 nm". On the real grid band 43 is 890 nm and 1,000 nm is band 48 (worked example 3.6). The warning itself is correct; the example pair is not a real pair.

3.9 Two design questions

3.9.1 Why 10 nm and not 5

The grid has to respect the *coarsest* contributing sensor. PRISMA's bands are about 12 nm wide (the code comment above `DEFAULT_COMMON_WAVELENGTH_GRID` says "PRISMA ~12nm"). A value every 5 nm would claim detail PRISMA never measured: the interpolator would happily produce numbers, but they would be invented, and a model would learn to reproduce invented structure. 10 nm is fine enough to be useful and coarse enough that every grid value is supported by real measurements.

3.9.2 Why cut the windows out of the grid, rather than interpolate and discard later

Suppose 1,350 to 1,450 nm were left in the grid. After band filtering there is no real sensor data between 1,340 and 1,460 (Chapter 5 drops those bands first). An interpolator does not know that; asked for values at $1{,}350, 1{,}360, \ldots, 1{,}450$, it draws a line or curve from the last real sample below to the first real sample above, and returns eleven values that were never measured. Those values would go into training, the model would learn to reproduce them, and at inference a genuine feature in that region would be compared against fiction. Removing the wavelengths from the grid means the interpolator is never asked the question: the problem is designed out rather than cleaned up afterwards.

Source check: which interpolator resamples onto the grid

The Indradhanu course (part 3) says the resampling onto the grid uses PCHIP. The source says otherwise: `app/utils/data_transformations/spectral_resampler.py` uses *linear* interpolation, and its docstring explains why ("the native bands are densely sampled (6-12nm apart) and the target grid is 10nm … PCHIP offers no practical quality gain but costs significantly more memory"). PCHIP is used one step earlier, to fill gaps in partly valid spectra (`spectral_interpolator.py`). The argument above holds for either interpolator: a straight line across a 120 nm gap is as invented as a curve. Chapter 5 derives both.

3.10 The grid is a wire format

Nowhere in a saved model file does it say "input position 48 is 1,000 nm". The link is purely positional: input position c means whatever the $(c+1)$-th entry of the grid is. Three things downstream are lined up with this exact list in this exact order:

- the model's input width, `in_channels:int=165` in `HyperspectralSegFormerMAEConfig` (`app/models/training/training_config.py`, line 88);
- the 165 means and 165 standard deviations in `app/constants/hyperspectral.json` (Chapter 5);

- every training shard, in which PRISMA and EnMAP patches are mixed and, at the tensor level, indistinguishable.

AVIRIS-NG was added later and needed no model change, because it comes out of the band pipeline as the same 165-band cube; the resolver says so in a comment (`backend/allotrope/foundation_models/resolver.py`, near line 259).

So if someone edited the exclusion ranges tomorrow, the model would still load (still 165 inputs), the normalisation statistics would still load (still 165 numbers), and every band would silently refer to the wrong wavelength. No error, no warning, wrong answers. Treat the grid like a database schema: changing it invalidates everything already stored against it.

Values that will drift

Dated 2026-10-29. The 165-band grid is what Indradhanu v0.2.0 consumes. The model shipped as Indradhanu V2 reads each sensor's *native* bands instead: its resolver entry sets `band_grid="native"` (`backend/allotrope/foundation_models/resolver.py`), and the `band_filter_apply` option `use_common_wavelength_grid` is documented as "Required by Indradhanu (v0.2.0); Indradhanu V2 needs it false (native bands)" (`backend/allotrope/action_types/band_filter_apply.py`). Chapters 22 and 23 explain the model that makes this possible. The grid remains the default band-filter output and the input of the classical detectors.

Source check: two different default exclusion lists

Two default lists of windows exist. `BandFilterConfig.exclusion_ranges` and `build_common_wavelength_grid` use $(1350, 1450)$ and $(1800, 1950)$. The older `DEFAULT_EXCLUSION_RANGES` in `app/utils/data_transformations/spectral_band_filter.py` uses wider ones, $(1328, 1492)$ and $(1784, 1967)$, labelled "Standard PRISMA exclusion ranges". The wider list is used only when `SpectralBandFilter` is built with `exclusion_ranges=None`; every builder in this book passes `band_filter_config.exclusion_ranges` explicitly, so the product uses the narrower list. Separately, the description of `BandFilterConfig.common_wavelength_grid` in `app/models/dataset/vendables.py` still says "(450–2450nm, 201 bands)"; the grid the code builds has 165 bands starting at 460 nm. The description is stale; the code wins.

3.11 Common confusions

Are the 165 bands evenly spaced?
Within each of the five segments, yes, 10 nm apart. Across the whole list, no: there are four jumps. The list is ascending but not uniform.

Does resampling lose information?
A little, always. That is the price of comparability, and the grid is chosen (at 10 nm) to keep the loss small.

Could we feed each sensor's native bands and pad to a fixed length?
Then position c would mean different wavelengths in different rows of one batch, which is exactly what stops one fixed-position model from serving three sensors. (A model that is told each band's wavelength can avoid this; Chapter 22 builds one.)

The window list starts with $(0, 450)$, yet nothing is removed by it. Why keep it?
It guards the band *filter*, which runs on native sensor bands, some of which lie below 450 nm. On the grid, which starts at 460, it has no effect.

BIL, BSQ, BIP: is one of them correct?
No. Each is fastest for a different access pattern. The models all expect BSQ `(C, H, W)`.

Are the two thermal sensors interchangeable?
No. Landsat 9 arrives in degrees Celsius; HotSat-1 arrives as raw DN around 5,000. A model normalised for one sees the other as wildly out of range (Chapter 5).

3.12 Check yourself

Exercise 3.1. How many rungs does a ladder from 400 to 2,500 nm in steps of 10 nm have? How many in steps of 5 nm? Use Equation (3.4) and show each line.

Exercise 3.2. Suppose the window $(1131, 1152)$ were widened to $(1125, 1160)$. Use Equation (3.5) to count the rungs it removes, and give the new length of the grid.

Exercise 3.3. Using Equation (3.6), find the wavelength of grid band 70 and of band 164, and the index of 2,210 nm.

Exercise 3.4. A cube has shape $C = 4$, $H = 3$, $W = 5$. Compute the memory offset of the element at band 2, row 1, column 3 in BSQ, BIL and BIP layouts.

Exercise 3.5. A Landsat `QA_PIXEL` value is $q = 21{,}840$. Write it as a sum of powers of two and decide whether the builder marks the pixel valid. An EnMAP pixel stores DN $= 1{,}830$: what is its reflectance?

Next: Chapter 4, The product: scenes, projects, actions and the vendable.

Chapter 4

The product: scenes, projects, actions and the vendable

The one thing this chapter teaches

Allotrope is a web application with four nouns (Scene, Project, Action, Export). Every piece of analysis in this book runs as an Action, Actions link into a directed acyclic graph, and every Action consumes one standard object, the *vendable*: a cube in physical units, a validity cube of the same shape, and per-band bookkeeping, with no map coordinates at all.

4.1 Why this comes before any model

You are about to spend many chapters on detectors and neural networks. If you have only ever seen machine-learning code, you will picture a script that takes a file and prints a result, and you will then be unable to work out why the code is shaped the way it is: why a model is "one option inside a step", why a file called `filtered_vendable.pkl` must exist before anything can be scored, why renaming a class can break every stored result. Each of those follows from the product's structure, so this chapter lays the structure out first.

A user logs in through a browser, uploads a satellite file, clicks through a chain of steps, drags a threshold slider, and downloads a result. Behind that sits a Postgres database, a login system, a React front end and a background worker process. The models are components inside it.

4.2 The four nouns

Definition 4.1 (Scene, Project, Action, Export)**.** A **Scene** is one uploaded raw satellite acquisition, onboarded into the standard form of Section 4.6. A **Project** is a piece of work bound to exactly one Scene. An **Action** is one processing step inside a Project; it reads the outputs of earlier Actions and writes its own outputs to its own folder (an **ActionOutput**). An **Export** is the deliverable: a zip bundle with a GeoTIFF, a Shapefile and a CSV naming every candidate anomaly with its coordinates.

The relationships are worth stating exactly, because they are asked about constantly.

- One Scene can back many Projects; a Project binds to exactly one Scene. Analysing one scene two ways means one Scene and two Projects. Comparing two scenes means two Projects.

- A Project holds many Actions. An Action names its inputs by referring to earlier Actions' outputs.

On the wire (in every API request and response), identifiers are never bare database keys. They carry a type prefix, `scene_<uuid>`, `action_<uuid>`, `output_<uuid>`, where a **UUID** is a 128-bit random identifier written as 32 hexadecimal digits. The helpers `to_prefixed` and `parse_prefixed_id` are in `backend/allotrope/api/wireformat.py`. A prefixed id cannot be passed where an id of another type is expected without the mistake being caught.

4.3 Actions form a directed acyclic graph

4.3.1 Graphs, briefly

Definition 4.2 (Directed graph, path, cycle, DAG)**.** A **directed graph** is a set of **nodes** and a set of **edges**, each edge an arrow from one node to another. A **path** follows arrows from node to node. A **cycle** is a path that returns to where it started. A **directed acyclic graph** (DAG) is a directed graph with no cycles.

In a Project, each Action is a node, and there is an edge from Action P to Action Q when Q reads an output of P. The graph is acyclic by construction: an Action can only name outputs that already exist when it is created, so an arrow always points from an older Action to a newer one, and a path that only moves forward in time can never come back to its start.

4.3.2 The shipped chain, read from the source

Each Action type declares the inputs it accepts in its `META` object, as a list of `ActionInputSpecs` whose `producing_action_types` field says which earlier types may feed it (`backend/allotrope/action_types/_meta.py`). Reading those declarations gives the edges of Figure 4.1:

- `band_filter_apply` reads the Scene itself and writes `filtered_vendable.pkl`.
- `scene_segmentation` (hyperspectral) reads a `band_filter_apply` output and writes `keep_mask.tif`, the map of pixels to keep (not cloud, not shadow). `cloud_mask` plays the same role for thermal scenes.
- `anomaly_scoring` reads a `band_filter_apply` output (required for hyperspectral scenes) and, optionally, a `scene_segmentation` or `cloud_mask` output and a ground-truth annotation (`anomaly_scoring.py`, lines 250–296). It writes one folder per model, `models/<codename>/`, holding `anomaly_score.tif` and `reconstruction.tif` among others.
- `anomaly_detection_prep` reads an `anomaly_scoring` output, rescales and combines the per-model score maps into `composite_score.tif`, and stops in the state `needs_threshold` (below).
- `spectral_library_match` (hyperspectral only) reads a committed `anomaly_detection_prep` output and names the material of each anomalous pixel.
- Export (`POST/actions/\{id\}/export`) bundles the result, recovering map coordinates on the way (Section 4.7).

Source check: the chain's outputs in the source

The course figure lists `matches.parquet` as the output of `spectral_library_match`. The current source writes that file and `match_map.tif`, and also the per-family answer `family_matches.parquet` and `family_map.tif` (`backend/allotrope/action_`

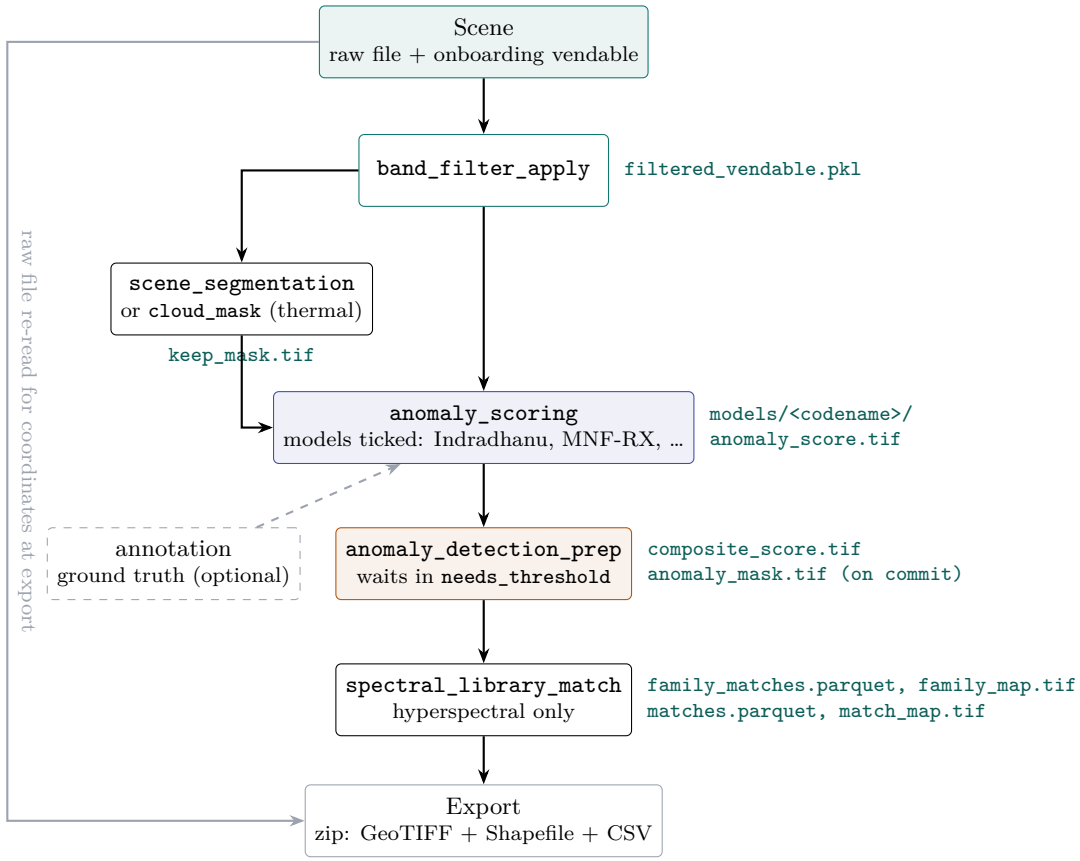


Figure 4.1: The shipped Action graph, edges read from each Action type's declared inputs. Each box writes its outputs (monospace) to its own folder. `anomaly_scoring` has two possible parents besides the optional annotation, so the structure is a graph, not a chain; Indradhanu is one model ticked *inside* `anomaly_scoring`. Export re-reads the raw scene file to recover map coordinates.

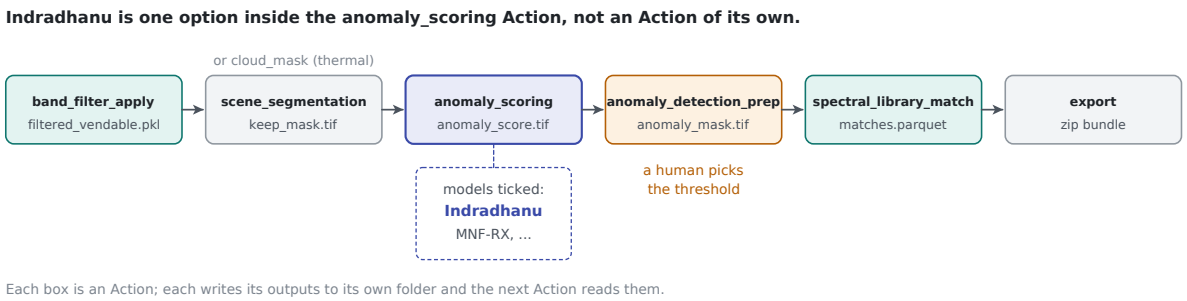


Figure 4.2: The same chain as drawn in the Indradhanu course (part 5), emphasising that Indradhanu is one option inside the `anomaly_scoring` Action, not an Action of its own.

`types/_spectral_library_match_run.py`, lines 46–47); the project instructions call the family map "the answer". The companion volume on spectral matching covers both. Second, the docstring of `anomaly_detection_prep.py` (line 22) says the threshold is locked by "a separate `anomaly_detection_commit` action". In the current source the commit is an API endpoint, `POST/actions/\{id\}/anomaly_detection_commit`, in `backend/allotrope/api/action_threshold.py`; it writes `anomaly_mask.tif`, which `spectral_library_match` then reads.

4.3.3 An order in which a graph can run: topological sorting

A DAG always has at least one order in which every node comes after all of its parents, called a **topological order**. Such an order is what lets a user (or a script) create Actions one at a time, each with its inputs already finished. The standard way to find one is Kahn's algorithm: repeatedly pick a node with no unfinished parents, list it, and remove its outgoing arrows.

Worked example 4.1: A topological order for one Project

A Project has six Actions: B (`band_filter_apply`), G (`scene_segmentation`), S (`anomaly_scoring`), P_1 and P_2 (two `anomaly_detection_prep` Actions trying different weightings on the same scoring run) and M (`spectral_library_match`, from P_2). The edges are

$$B \to G, \quad B \to S, \quad G \to S, \quad S \to P_1, \quad S \to P_2, \quad P_2 \to M.$$

Count each node's parents (its **in-degree**):

$$B{:}\,0, \quad G{:}\,1, \quad S{:}\,2, \quad P_1{:}\,1, \quad P_2{:}\,1, \quad M{:}\,1.$$

Run Kahn's algorithm on paper.

1. Only B has in-degree 0. List B; remove $B \to G$ and $B \to S$. Now $G{:}\,0$, $S{:}\,1$.
2. Only G has in-degree 0. List G; remove $G \to S$. Now $S{:}\,0$.
3. List S; remove $S \to P_1$ and $S \to P_2$. Now $P_1{:}\,0$, $P_2{:}\,0$.
4. Two choices. List P_1 (it has no outgoing arrows).
5. List P_2; remove $P_2 \to M$. Now $M{:}\,0$.
6. List M.

One valid order is B, G, S, P_1, P_2, M. Another is B, G, S, P_2, M, P_1: step 4 had a choice, so topological orders are not unique. What every order shares is that S comes after both B and G, because S has two parents.

There is no scheduler and no automatic re-run in the product: a user creates each Action. The graph is the record of what was derived from what, which is why scoring once and trying several thresholds (two prep Actions sharing one scoring parent, as above) costs one expensive run.

4.4 Two processes and one queue

Two programs run. The **api** is the web server: it handles requests, reads and writes the database, and hands out work, but never does heavy computation. The **worker** does the analysis and has no web interface. They communicate through a `jobs` table in Postgres: there is no Redis, no

Celery and no message broker.

- The api inserts a job row with status `queued`.
- A worker *claims* one with `SELECT\dots\FORUPDATESKIPLOCKED` (`backend/allotrope_worker/claim.py`), sets it to `running` and commits immediately. "Skip locked" means two workers asking at once never get the same row: each skips rows another transaction is holding.
- While it works, the worker writes a **heartbeat** timestamp to the row at a fixed interval.
- A **reaper** flips any `running` job whose last heartbeat is older than a timeout to `failed` (`backend/allotrope_worker/reaper.py`), so a crashed worker does not leave a job "running" forever.

Worked example 4.2: When is a job reaped?

The defaults are a heartbeat every 5 seconds and a timeout of 60 seconds [M: `backend/allotrope_worker/runner.py`], overridable by the environment variables `WORKER_HEARTBEAT_INTERVAL_SEC` and `WORKER_HEARTBEAT_TIMEOUT_SEC`. A job's last heartbeat was written at 10:00:00 and the worker then crashed. The reaper computes a cutoff of "now minus 60 s" and reaps rows whose heartbeat is older than the cutoff.

- At 10:00:59 the cutoff is 09:59:59. The heartbeat (10:00:00) is newer: kept.
- At 10:01:01 the cutoff is 10:00:01. The heartbeat is older: reaped to `failed`.

How many heartbeats must a healthy worker miss before that happens?

$$60/5 = 12 \text{ intervals.}$$

The source comment asks for the timeout to be at least about three heartbeat intervals "to absorb jitter"; the defaults give twelve.

The consequence you will feel: **the api can be perfectly healthy while the worker is dead**. The api's health check talks to the database, not the worker. This has happened twice in this repository's history, and jobs sat in `queued` forever. When something does not run, check the worker first.

4.4.1 The step that waits for a person

`anomaly_detection_prep` is the one Action type that does not finish in `complete`. It writes its composite score map and stops in `needs_threshold`, waiting for a person to drag a threshold slider in the viewer. Each "Apply" recomputes a preview mask; the commit endpoint locks the threshold and writes `anomaly_mask.tif`. Section 1.5 gave the reason: no fixed cut-off works across scenes, so the cut is a human judgement made per scene.

4.4.2 Where the code lives, and the lazy-import rule

The models' architectures, trainers and inferencers live in `app/`. They know nothing about users, jobs, HTTP or Postgres, which is why the same model code runs under a training script on a rented GPU machine and under the worker in a container.

Action modules come in pairs: `anomaly_scoring.py` (light: name, metadata, configuration validation) and `_anomaly_scoring_run.py` (heavy: the implementation). Action modules must not import `app.*`, `torch` or `rasterio` at the top of the file, only inside their functions. The reason: the api imports every Action module at start-up to list the available Actions, and a

Table 4.1: The layers. The rule in the last column is enforced by review, not by tooling.

Layer	Path	Rule
Portable science	`app/`	numpy, torch, rasterio only. No database, no web framework.
Shared backend	`backend/allotrope/`	database models, routes, authentication, the Action registry; used by api and worker.
Worker only	`backend/allotrope_worker/`	the claim loop, heartbeat, reaper and job handlers.

top-level `importtorch` would drag hundreds of megabytes of deep-learning library into the web server to render a drop-down menu. A side effect worth remembering: a broken import in a worker-only module does not stop the api starting, so the api stays green while the worker cannot start.

4.5 Where Indradhanu lives

Indradhanu is not an Action. It is one selectable model *inside* `anomaly_scoring`: the user ticks one or more models by name in the scoring dialog, and the Action loops over them (`forcodenameincfg["model_codenames"]:`), writing each model's outputs to its own `models/<codename>/` folder. You can run Indradhanu and MNF-RX in one Action and compare their score maps side by side.

Every foundation model carries an Indic codename, and the backend selects models *by codename*, not by class name.

Table 4.2: The foundation-model roster (Indradhanu course, part 5; project instructions).

Codename	Meaning	Architecture	Sensor
Pratibimba	reflection	spatial autoencoder	thermal
Antardhana	disappearance	masked autoencoder	thermal
Tirohita	vanished	masked autoencoder, L1 loss	thermal
Asanskrita	unrefined	unnormalised variant	thermal
Drashta	seer	normalised masked autoencoder	thermal
Chakshu	the eye	`segformer_mae`	thermal
Indradhanu	rainbow ("Indra's bow")	`hyperspectral_segformer_mae`	hyperspectral

A codename becomes a running model through three cooperating pieces: a **manifest** (`current.json`) beside the weights file under `allotrope_models/<architecture>/` saying which file is current; the **resolver** (`backend/allotrope/foundation_models/resolver.py`), a static table of what each architecture can do (which scoring methods to offer, default patch size and stride, where its normalisation statistics live); and the **inferencer factory** (`app/foundation_models/inferencers/inferencer_factory.py`) mapping the architecture to the Python class that runs it. Adding a model means four registrations: the architecture class, the trainer factory, the inferencer factory, and a resolver capabilities entry plus a manifest. The fourth is the one people forget, and it fails silently: the model catalogue skips architectures with no capabilities entry, so a model that trained perfectly never appears in the picker.

Values that will drift

Manifest values quoted by the Indradhanu course (part 5). `current.json` for `hyperspectral_segformer_mae` listed file `hyperspectral_segformer_mae_v0.2.0_epoch200.pt`, version v0.2.0, epoch 200, 5,507,354 parameters, encoder widths $[32, 64, 160, 256]$, compression width $D = 32$, decoder width 256, validation loss

0.04349 [M: indradhanuworkings/05-where-it-lives.md]. The manifest itself is not in this checkout (only the `.pt` file is), so these could not be re-read for this book. Since 2026-10-29 the shipped hyperspectral model is Indradhanu V2 ("Indradhanu Heavy"), whose resolver entry reads native bands (Chapter 3, Chapters 39 and 43).

4.6 The vendable

Raw satellite files are unpleasant (Chapter 3): every vendor its own layout, quality flags packed into bits, some files reflectance and some raw counts, detector quirks everywhere. If every detector had to cope with all of that, every detector would change whenever a sensor was added. So the first thing the system does with a scene is convert it, once, into a standard object.

Definition 4.3 (Vendable)**.** A **vendable** is the standard in-memory form of a scene: a cube in physical units, a validity cube of the same shape, and per-band metadata. It is produced by a **DatasetBuilder** from a raw file. Every detector, model and Action consumes a vendable; nothing downstream reads a raw sensor file. The classes are in `app/models/dataset/vendables.py`.

Read the odd name as "the thing this scene can *vend*": what it dispenses to anything that asks, once cleaned up.

Analogy

A shipping port receives cargo in every kind of packaging (sacks, crates, barrels, loose bales) and repacks it into standard containers. After that, cranes, trucks and ships need to handle one shape only, and a new kind of cargo needs a new repacking procedure, not new trucks. The vendable is the container; the dataset builders are the repacking sheds. The analogy also predicts the vendable's best-known limitation: a container's label says what is inside, not where it came from on a map, and the vendable likewise carries no geographic location (Section 4.7).

4.6.1 Three variants and their fields

Table 4.3: The three vendable classes.

Class	Sensors	Cube holds
`VendableHyperspectralDataset`	PRISMA, AVIRIS-NG (and SpectralEarth patches)	reflectance
`VendableEnmapHyperspectralDataset`	EnMAP	reflectance, plus five quality masks
`VendableThermalDataset`	Landsat 9, HotSat-1	Celsius (HotSat-1: raw DN)

Source check: the EnMAP class is not a subclass

The Indradhanu course (part 4) says EnMAP has "a subclass" of `VendableHyperspectralDataset` with five extra quality masks. In the source, `VendableEnmapHyperspectralDataset` is a separate class deriving directly from Pydantic's `BaseModel` (`app/models/dataset/vendables.py`, line 233), with the same core fields re-declared plus the masks, a land/water class layer and the two detectors' channel

Table 4.4: Fields of `VendableHyperspectralDataset` (`app/models/dataset/vendables.py`, lines 141–178).

Field	Type, shape	Meaning
`normalized_hyperspectral_cube`	array `(C, H, W)`	reflectance; what a model sees
`validity_cube`	array `(C, H, W)`	1 real measurement, 0 not
`spectral_family_order`	list of C enums	detector of each band (VNIR or SWIR)
`band_cw_order`	list of C floats	centre wavelength of each band, nm
`band_fwhm_order`	list of floats	FWHM of each band; empty on the common grid
`band_validity_by_position`	list of C ints	per-band validity flag
`band_level_validity_score`	list of C floats	percentage of valid pixels per band

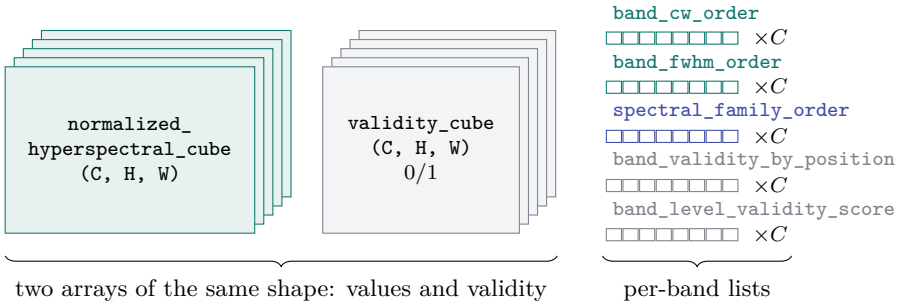


Figure 4.3: The hyperspectral vendable as a data structure: two arrays of identical shape (the reflectance cube and its validity cube) and five lists with one entry per band. There is no field for map coordinates.

indices. Code that checks `isinstance(v,VendableHyperspectralDataset)` will therefore *not* accept an EnMAP onboarding vendable. (After `band_filter_apply`, the EnMAP builder's output on the common grid is the same class as every other hyperspectral vendable.)

4.6.2 Two vendables in a scene's life

A hyperspectral scene has two vendables. Scene onboarding builds the **onboarding vendable**, with all the sensor's native bands. The `band_filter_apply` Action builds the **filtered vendable**, on the common grid, and pickles it to `filtered_vendable.pkl` (Chapter 5 is that Action's pipeline). Indradhanu v0.2.0 and the classical detectors read the filtered one; spectral matching reads the onboarding one, because it needs native bands and their FWHM. If `band_filter_apply` has not run, the scoring Action refuses a hyperspectral scene with `"anomaly_scoringonahyperspectralsceneneedsinput_band_filter_output_id."` Thermal scenes skip band filtering and are scored from the onboarding vendable.

4.6.3 Per-band and per-pixel validity, computed

Two summaries of the validity cube $\mathbf{V}$ appear everywhere. The *per-band* score, stored in `band_level_validity_score`, is the percentage of pixels valid in band c:

$$s_c = 100 \times \frac{1}{HW} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \mathbf{V}[c, i, j]. \tag{4.1}$$

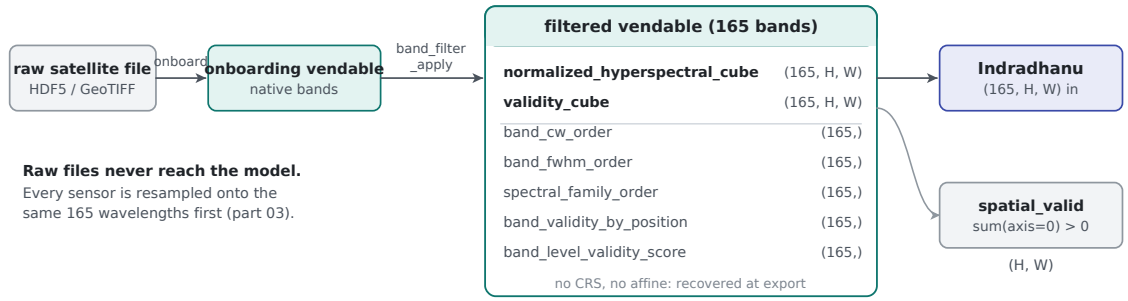


Figure 4.4: How a vendable reaches Indradhanu (course figure, Indradhanu part 4). Of the filtered vendable's fields the model uses the cube and the validity cube; a pixel is spatially valid if it is valid in at least one band.

The *per-pixel* map used by the scoring code is

$$m(i,j) = \begin{cases} 1 & \text{if } \sum_{c=0}^{C-1} \mathbf{V}[c,i,j] > 0, \\ 0 & \text{otherwise.} \end{cases} \tag{4.2}$$

Worked example 4.3: Both summaries on a 3-band, 2-by-2 validity cube

A validity cube after the band pipeline (all-or-nothing per pixel) has

$$\mathbf{V}[0] = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{V}[1] = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{V}[2] = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}.$$

Per band, by Equation (4.1) with $HW = 4$:

$$s_0 = 100 \times (1+1+0+1)/4 = 100 \times 3/4 = 75.0,$$

and likewise $s_1 = s_2 = 75.0$.
Per pixel, by Equation (4.2), sum down the bands at each position:

$$(0,0):\ 1+1+1 = 3 > 0 \Rightarrow 1, \qquad (0,1):\ 1+1+1 = 3 \Rightarrow 1,$$
$$(1,0):\ 0+0+0 = 0 \Rightarrow 0, \qquad (1,1):\ 1+1+1 = 3 \Rightarrow 1.$$

So $m = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$, and the fraction of valid pixels is $3/4$, the same 75%, as it must be when validity is all-or-nothing.

4.6.4 The contract a model receives

From the filtered vendable, the scoring code builds exactly two tensors for a hyperspectral model (`_anomaly_scoring_run.py`, lines 285–286):

```
scene_tensor = torch.from_numpy(cube_np).float()                # (165, H, W)
    reflectance
mask_tensor  = torch.from_numpy(spatial_valid[None]).float()    # (1, H, W)   1.0
    valid
```

`[None]` inserts a new axis of size 1 at the front, turning `(H, W)` into `(1, H, W)`. Notice what is

not in the contract: no wavelengths, no coordinates, no sensor name. The model does not know which satellite took the picture; that is the payoff of the common grid.

Source check: a naming collision

The field is called `normalized_hyperspectral_cube`, but "normalized" here means "cleaned and standardised by the intake pipeline" (physical units, BSQ layout, common grid). It is *not* the per-band z-score of Chapter 5, which happens inside the model at run time. The vendable's values are reflectance, around 0.02 to 0.30.

4.7 No spatial reference, on purpose

A vendable does not know where on Earth it is. There is no coordinate reference system, no latitude or longitude, no transform field. To see what is missing, you need one definition.

Definition 4.4 (Affine geotransform)**.** A **geotransform** maps a pixel's column j and row i to map coordinates (x, y) in some coordinate reference system (CRS):

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a & b \\ d & e \end{pmatrix} \begin{pmatrix} j \\ i \end{pmatrix} + \begin{pmatrix} c \\ f \end{pmatrix}. \tag{4.3}$$

a and e are the pixel width and height in map units, b and d are rotation terms (usually 0), and (c, f) is the map position of the top-left corner. The **identity transform** has $a = e = 1$ and $b = c = d = f = 0$, so $(x, y) = (j, i)$: "pixel $(0, 0)$ is at $(0, 0)$ and each pixel is one unit". Meaningless as geography.

The matrix-times-vector product in Equation (4.3) means: the first output is $a \cdot j + b \cdot i$, the second is $d \cdot j + e \cdot i$; then add c and f.

Worked example 4.4: Pixel to map, with a real transform and with the identity

A scene in a UTM projection (map units are metres) has 30 m pixels, top-left corner at $x =$ 500,000, $y =$ 2,000,000, and no rotation (illustrative values [A]). Rows increase southwards, so $e = -30$. The transform is $a = 30$, $b = 0$, $c =$ 500,000, $d = 0$, $e = -30$, $f =$ 2,000,000. Where is the pixel at row $i = 10$, column $j = 20$?

$$x = a\,j + b\,i + c = 30 \times 20 + 0 \times 10 + 500{,}000 = 600 + 0 + 500{,}000 = 500{,}600,$$
$$y = d\,j + e\,i + f = 0 \times 20 + (-30) \times 10 + 2{,}000{,}000 = -300 + 2{,}000{,}000 = 1{,}999{,}700.$$

With the identity transform the same pixel lands at $(x, y) = (20, 10)$: twenty metres east and ten north of wherever $(0, 0)$ is in the file's CRS. Opened in a GIS, an intermediate GeoTIFF therefore lands as a tiny image near the origin (for a geographic CRS, off the coast of West Africa). That is not a bug.

Every GeoTIFF an Action writes carries the identity transform. The real georeferencing is recovered once, at export, by re-reading the original raw file (`app/georef/`). Carrying geographic metadata through every step would mean every step had a chance to corrupt it; deriving it once, from the source, at the end, means it is right. PRISMA is the awkward case: its L2D product is swath data with a latitude and longitude per pixel, and the export fits an axis-aligned affine transform to those, discarding the swath's curvature (`docs/03-data-pipeline.md`). A scene whose georeferencing cannot be resolved fails at export, not at analysis, with the error `422crs_missing`.

Practical consequences: do not add a transform field to the vendable expecting it to be filled in, and do not trust the coordinates of an intermediate `.tif`.

4.8 Vendables are pickled: class names are a wire format

`band_filter_apply` writes the filtered vendable to disk with Python's `pickle` module, and later Actions (and the api) read it back. A pickle stores an object's data together with the *full import path* of its class, for example `app.models.dataset.vendables.VendableHyperspectralDataset`. Loading the file imports that exact path. Rename the class or move the module, and every vendable already on disk becomes unloadable: not corrupted, unreadable, because the recorded path no longer exists. The module's own docstring says it: "Treat the class names and module path as a wire format." Adding a field is safe; renaming or moving one is a breaking change, like changing a database schema.

4.9 The thermal vendable and its two cloud masks

`VendableThermalDataset` holds `normalized_thermal_cube` (degrees Celsius) and a validity cube, plus two kinds of cloud information kept apart on purpose: a `cloud_mask` modelled from the scene's own temperature distribution (a five-component Gaussian mixture, `docs/03-data-pipeline.md`) and the provider's `QA_PIXEL`-derived masks (worked example 3.4). The provider's mask is authoritative but conservative and not always present; the modelled one is always available and adapts to the scene. Merging them would throw away their disagreement, which is often the interesting part.

Source check: HotSat-1 units are silently dropped

The anomaly-detection course (part 5) notes, and the source confirms, that the HotSat-1 builder passes a `units` value that `VendableThermalDataset` does not declare. Pydantic models ignore undeclared fields by default, so the value is discarded, and the scoring code's check `getattr(vendable,"units",None)` finds nothing. The guard meant to treat HotSat's raw DN differently from Celsius (Chapter 5) is therefore not in effect through this path. Source over comments.

4.10 Common confusions

Can one Project cover several Scenes?
No. A Project binds to exactly one Scene. Two scenes, two Projects.

Is the Action graph a workflow engine?
Lighter than that. Actions name their inputs; there is no scheduler and no automatic re-run. A user creates each Action.

Why is scoring separate from thresholding?
Scoring is deterministic and expensive; thresholding is a judgement call and cheap. Splitting them lets you try several thresholds on one scoring run.

Is Indradhanu a service I can call?
No. It is a Python class loaded inside the worker, on demand, for the duration of one scoring job, as one of the models ticked in `anomaly_scoring`.

Is the vendable the file on disk or the object in memory?
Both, at different moments. It is a Python object; `filtered_vendable.pkl` is that object pickled.

The intermediate GeoTIFF sits at (0,0). Is georeferencing broken?
No. Action outputs carry the identity transform by design; coordinates are recovered at

export from the raw file.

The api is healthy but nothing runs. Why?
Almost always the worker: crashed or crash-looping, often on an import error. The api's health check does not look at the worker.

4.11 Check yourself

Exercise 4.1. A user wants to analyse one scene with two band-filter settings, score each with Indradhanu and MNF-RX in one scoring Action, and try two thresholds on each scoring run. How many Scenes, Projects and Actions (counting only `band_filter_apply`, `anomaly_scoring` and `anomaly_detection_prep`) are needed if each band-filter setting has its own Project?

Exercise 4.2. For the graph with edges $B \to G$, $B \to S$, $G \to S$, $S \to P$, $P \to M$, $S \to Q$, write the in-degree of each node and give two different topological orders. Explain why adding an edge $M \to B$ is impossible in the product.

Exercise 4.3. A validity cube of shape $(2, 2, 3)$ has $\mathbf{V}[0] = \mathbf{V}[1] = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}$. Compute s_0 and the per-pixel map m, and the fraction of valid pixels.

Exercise 4.4. With the transform of worked example 4.4, where is pixel row $i = 0$, column $j = 0$? Row 100, column 0? And which pixel is at map position $x = 501{,}500$, $y = 1{,}998{,}500$? (Invert Equation (4.3) by hand.)

Exercise 4.5. With heartbeat interval 5 s and timeout 60 s, a worker freezes for 45 seconds and then resumes. Is its job reaped? What if the operator sets the timeout to 12 s?

Next: Chapter 5, Making sensors comparable: band filtering, interpolation and normalisation.

Chapter 5

Making sensors comparable: band filtering, interpolation and normalisation

The one thing this chapter teaches

Eight stages turn a sensor's own bands into 165 bands that mean the same thing on every hyperspectral sensor, using nothing more advanced than the equation of a straight line (and one shape-preserving curve). Inside the model, a second and different "normalisation" rescales each band to a z-score, and its exact inverse turns the model's output back into reflectance.

5.1 The problem

Chapter 3 fixed the destination: 165 wavelengths from 460 to 2,450 nm. This chapter is the journey. A PRISMA cube arrives with 239 bands at PRISMA's wavelengths, some flagged bad by the sensor, some inside atmospheric windows, some noisy at detector edges, and pixels that are valid in some bands and not others. To reach the grid you must decide which bands to trust, decide which pixels to keep, fill the holes in the kept pixels' spectra, and then estimate each spectrum's value at the grid wavelengths.

The obvious shortcut is to skip the first three decisions and interpolate the raw spectrum straight onto the grid. It fails, because an interpolator trusts every number it is given: a bad band becomes a spike in the resampled spectrum, an invalid zero drags its neighbours down, and a detector-edge artefact is spread across two grid bands. Interpolation is only as good as its inputs, so the pipeline cleans first and interpolates last.

The pipeline is driven entirely by one configuration object, `BandFilterConfig` (`app/models/dataset/vendables.py`, lines 71–138), and run by each sensor's dataset builder; the shared composition is visible in one function, `to_common_grid` (`app/utils/data_transformations/common_grid_pipeline.py`).

After the eight stages, the `band_filter_apply` Action applies one more step, the nearest-valid pixel fill (Section 5.9), and pickles the result.

Source check: "four stages" or "eight"

The docstring of `BandFilterConfig` says it "controls four filtering stages"; the courses and `docs/03-data-pipeline.md` speak of eight. Both are right about different things: the four

Table 5.1: The eight stages and their defaults. Stages 1–4 are `SpectralBandFilter` (`spectral_band_filter.py`); stage 7 is `interpolate_spectral_gaps` (`spectral_interpolator.py`); stage 8 is `resample_to_common_grid` (`spectral_resampler.py`).

Stage	Acts on	What it does	Setting (default)
1	bands	drop bands the sensor flagged bad	always
2	bands	drop bands inside atmospheric windows	`exclusion_ranges` (5 windows)
3	bands	drop the first and last bands of each detector	`edge_bands_to_trim` (3)
4	bands	drop bands with too few valid pixels	`min_valid_pixel_pct` (20.0)
5	pixels	invalidate pixels flagged by quality masks (EnMAP only)	`quality_masks_to_apply`
6	pixels	invalidate pixels with too many invalid bands	`max_invalid_voxel_fraction` (0.4)
7	values	fill the remaining holes along wavelength (PCHIP)	–
8	values	resample onto the common grid (linear)	`common_wavelength_grid`

are the band-dropping stages inside `SpectralBandFilter`, and the eight count the pixel and value stages the builders run afterwards with the same configuration. This book uses the eight.

5.2 Stages 1–4: deciding which bands to trust

`SpectralBandFilter.get_good_band_indices` walks the sensor's bands once and keeps band k only if it passes all four tests, in order:

```
for i in range(len(self._wavelengths)):
    if self._flags[i] != 1: continue                                    # 1: sensor
    flag
    wl = self._wavelengths[i]
    if any(lo <= wl <= hi for lo, hi in self._exclusion_ranges): continue  # 2:
    window
    if i in edge_set: continue                                          # 3: detector
    edge
    if self._validity_scores[i] < self._min_valid_pixel_pct: continue  # 4: coverage
    good.append(i)
```

Stage 1, the sensor already knows. Detectors fail, and the sensor ships a flag per band. A flagged PRISMA band still occupies a slice of the cube, with FWHM = 0.0 (Chapter 3); it is present but meaningless.

Stage 2, the atmosphere is opaque in places. The five windows of Chapter 3 remove every native band whose centre wavelength lies inside one of them, ends included.

Stage 3, detector edges are unreliable. A detector's response falls off at the ends of its array, so the first and last `edge_bands_to_trim` bands of each detector (VNIR and SWIR) are dropped. In the source the edge set is computed once, from the *full* list of each detector's band indices, before any other stage: a band that stage 1 or 2 already dropped can still count as one of the three edge bands.

Stage 4, a mostly invalid band is not worth keeping. The per-band score s_c of

Equation (4.1) must be at least 20%: a band valid in fewer than a fifth of the scene's pixels is dropped.

Worked example 5.1: Filtering a ten-band toy sensor

A toy sensor has ten bands (all values illustrative [A]). Bands 0–4 are on detector VNIR, 5–9 on SWIR. Use `edge_bands_to_trim` = 1 to keep it small, and the real windows and 20% threshold.

band k	0	1	2	3	4	5	6	7	8	9
wavelength (nm)	440	600	800	940	990	1000	1400	1600	2200	2480
flag	1	1	0	1	1	1	1	1	1	1
detector	V	V	V	V	V	S	S	S	S	S
s_k (%)	99	99	99	99	99	99	99	12	99	99

First the edge set (stage 3), from the full per-detector index lists: VNIR is $\{0, \ldots, 4\}$, so its edges are 0 and 4; SWIR is $\{5, \ldots, 9\}$, edges 5 and 9. Edge set = $\{0, 4, 5, 9\}$. Now walk the bands.

- Band 0: flag 1; 440 lies in $(0, 450)$, dropped at stage 2.
- Band 1: passes stages 1 and 2; not an edge; $99 \geq 20$. **Kept.**
- Band 2: flag 0, dropped at stage 1.
- Band 3: 940 lies in $(912, 978)$, dropped at stage 2.
- Band 4: 990 is in no window; but 4 is an edge, dropped at stage 3.
- Band 5: edge, dropped at stage 3.
- Band 6: 1,400 lies in $(1350, 1450)$, dropped at stage 2.
- Band 7: passes 1–3; $12 < 20$, dropped at stage 4.
- Band 8: 2,200 is in no window, not an edge, $99 \geq 20$. **Kept.**
- Band 9: edge, dropped at stage 3.

Two bands survive, 1 and 8. Note band 0: it was both a window band and an edge band; it is counted as dropped at stage 2 because the tests run in order, but because the edge set came from the full list, trimming "one band at the VNIR edge" removed nothing beyond what stage 2 had already removed. The edges are positions in the file, not "the first surviving bands".

For a real PRISMA cube about 188 of the 239 bands survive stages 1–4 (`docs/03-data-pipeline.md`).

5.3 Stages 5 and 6: deciding which pixels to keep

Stage 5 applies only to EnMAP, the one sensor that ships quality masks. Any pixel flagged (> 0) by one of the masks named in `quality_masks_to_apply` (default cloud, cloud shadow, haze) has its whole validity column set to 0: `validity[:,flagged]=0`.

Stage 6 makes validity binary. Let C' be the number of bands that survived stages 1–4 and $n(i, j)$ the number of those bands in which pixel (i, j) is valid. The pixel's invalid fraction is

$$f(i, j) = 1 - \frac{n(i, j)}{C'}, \tag{5.1}$$

and the pixel is invalidated in every band if $f(i, j) > 0.4$ (`max_invalid_voxel_fraction`); note

the strict "greater than". Pixels that pass keep their partial validity, and stage 7 fills their holes. After stage 7, every pixel is valid in all bands or in none: **validity is binary**. That single fact removes a whole category of special cases from every detector downstream: none of them ever has to handle "valid in these 140 bands but not those 25".

Worked example 5.2: Kept or wiped?

Take $C' = 188$ surviving bands. How many invalid bands may a pixel have and still be kept? It is kept when $f \le 0.4$, that is when the number of invalid bands $u = C' - n$ satisfies

$$\frac{u}{188} \le 0.4$$
$$u \le 0.4 \times 188 = 75.2.$$

u is a whole number, so $u \le 75$. Check both sides of the line:

$$u = 75: \quad 75/188 \approx 0.3989 \le 0.4 \;\Rightarrow\; \text{kept, its 75 holes filled at stage 7,}$$
$$u = 76: \quad 76/188 \approx 0.4043 > 0.4 \;\Rightarrow\; \text{invalid in every band.}$$

Source check: the denominator is the surviving bands, not 165

The anomaly-detection course (parts 5 and 7) works this rule as "55/165" and "61/165". In the source the fraction is taken over the bands that survived stages 1–4 (`num_bands=bsq_validity.shape[0]` after band filtering, `prisma_dataset_builder.py`, lines 358–362; `validity.mean(axis=0)` on the filtered cube in `common_grid_pipeline.py`). Stage 6 runs *before* resampling, when the cube has C' native bands (about 188 for PRISMA), not 165. The arithmetic of the course examples is right; the denominator is not.

5.4 Linear interpolation, from the equation of a line

Stages 7 and 8 both estimate a spectrum's value at a wavelength where it was not measured. The simplest estimator, and the one stage 8 uses, is linear interpolation. You can derive it from the equation of a straight line.

5.4.1 Two-point form

You know the spectrum at two wavelengths: value y_0 at λ_0 and y_1 at λ_1, with $\lambda_0 < \lambda_1$. Assume the spectrum is a straight line between them. A straight line has a constant **slope**, the rise divided by the run:

$$m = \frac{y_1 - y_0}{\lambda_1 - \lambda_0}. \tag{5.2}$$

Starting at (λ_0, y_0) and moving a distance $\lambda - \lambda_0$ to the right, the line rises by slope times distance:

$$y(\lambda) = y_0 + \frac{y_1 - y_0}{\lambda_1 - \lambda_0}\,(\lambda - \lambda_0). \tag{5.3}$$

This is the **two-point form** of the line. Check it at both ends. At $\lambda = \lambda_0$ the bracket is 0, so $y = y_0$. At $\lambda = \lambda_1$ the fraction times $(\lambda_1 - \lambda_0)$ is $y_1 - y_0$, so $y = y_0 + y_1 - y_0 = y_1$.

5.4.2 Weight form

Name the fraction of the way from λ_0 to λ_1:

$$t = \frac{\lambda - \lambda_0}{\lambda_1 - \lambda_0}. \tag{5.4}$$

$t = 0$ at λ_0, $t = 1$ at λ_1, $t = 0.5$ halfway. Substitute t into Equation (5.3):

$$y = y_0 + (y_1 - y_0)\, t. \tag{5.5}$$

Multiply out the bracket:

$$y = y_0 + y_1 t - y_0 t. \tag{5.6}$$

Collect the two y_0 terms:

$$y = (1 - t)\, y_0 + t\, y_1. \tag{5.7}$$

This is the **weight form**: the estimate is a weighted average of the two neighbours, with weights $1 - t$ and t that add to 1. The nearer neighbour gets the larger weight. It is exactly the line the product's resampler computes (`spectral_resampler.py`, lines 115–118):

```
t = np.float32((twl - wl_left) / (wl_right - wl_left))           # scalar weight t
out_2d[i, valid_cols] = (valid_spectra[idx_left] * (1.0 - t)
                         + valid_spectra[idx_right] * t)          # (n_valid,): (1-t)
    y0 + t y1
```

Here `twl` is the target wavelength and `valid_spectra` is `(C_src,n_valid)`: one column per valid pixel, so one line computes the interpolated value for every pixel at once.

Worked example 5.3: Three bands by hand

A spectrum is known at three wavelengths: 500, 600 and 700 nm, with values 1, 2 and 4 (the numbers used by the repository's own interpolation test, quoted in the anomaly-detection course, part 22). Estimate it at 550, 650 and 680 nm. Do every line on paper.
At 550 nm the neighbours are $(500, 1)$ and $(600, 2)$:

$$t = (550 - 500)/(600 - 500) = 50/100 = 0.5,$$
$$y = (1 - 0.5) \times 1 + 0.5 \times 2 = 0.5 + 1.0 = 1.5.$$

At 650 nm the neighbours are $(600, 2)$ and $(700, 4)$:

$$t = (650 - 600)/(700 - 600) = 50/100 = 0.5,$$
$$y = 0.5 \times 2 + 0.5 \times 4 = 1.0 + 2.0 = 3.0.$$

At 680 nm, same neighbours:

$$t = (680 - 600)/100 = 0.8,$$
$$y = 0.2 \times 2 + 0.8 \times 4 = 0.4 + 3.2 = 3.6.$$

Check 680 with the two-point form instead: slope $= (4 - 2)/(700 - 600) = 0.02$ per nm, and $y = 2 + 0.02 \times 80 = 2 + 1.6 = 3.6$. Same answer, as the algebra promised.

5.4.3 Finding the neighbours, and what happens at the ends

For each target wavelength the resampler finds the bracketing pair with a binary search, `np.searchsorted(src_wl_sorted, twl, side="right")`, which returns the index of the first source wavelength strictly greater than `twl`; call it r, and $\ell = r - 1$. Both are clamped into the array: $r \le C_{\text{src}} - 1$ and $\ell \ge 0$. If the clamping makes $\ell = r$, the target lies beyond an end of the sensor's range and the resampler copies the nearest end value (**constant extrapolation**).

Worked example 5.4: Resampling one spectrum onto four grid wavelengths

A pixel's native samples (illustrative [A], roughly EnMAP-like spacing) are

native wavelength (nm)	1003	1011	1019	1027
reflectance	0.240	0.248	0.252	0.250

Resample onto the grid wavelengths 1,000, 1,010, 1,020 and 1,030 nm, following the code's steps. Indices of the native samples are $0, 1, 2, 3$.

1,000 nm. No native wavelength is $\le$ 1,000; the first one greater is index 0, so $r = 0$, $\ell = \max(-1, 0) = 0$. $\ell = r$: constant extrapolation, $y = 0.240$.

1,010 nm. The first native wavelength greater than 1,010 is 1,011 (index 1), so $r = 1$, $\ell = 0$:

$$t = (1{,}010 - 1{,}003)/(1{,}011 - 1{,}003) = 7/8 = 0.875,$$
$$y = 0.125 \times 0.240 + 0.875 \times 0.248 = 0.0300 + 0.2170 = 0.2470.$$

1,020 nm. First greater is 1,027 (index 3), so $r = 3$, $\ell = 2$:

$$t = (1{,}020 - 1{,}019)/(1{,}027 - 1{,}019) = 1/8 = 0.125,$$
$$y = 0.875 \times 0.252 + 0.125 \times 0.250 = 0.2205 + 0.03125 = 0.25175.$$

1,030 nm. Nothing is greater, so `searchsorted` returns 4; clamped to $r = 3$, and $\ell = 2$. Now $\ell \ne r$, so the code does *not* extrapolate as a constant: it extends the last segment's line,

$$t = (1{,}030 - 1{,}019)/(1{,}027 - 1{,}019) = 11/8 = 1.375,$$
$$y = (1 - 1.375) \times 0.252 + 1.375 \times 0.250 = -0.0945 + 0.34375 = 0.24925.$$

With $t > 1$ the "weight" $1 - t$ is negative: this is *linear* extrapolation past the last sample, not the constant extrapolation the docstring promises. At the low end, by contrast, the clamping produces a true constant. Figure 5.1 draws the four results.

Source check: the resampler's upper end extrapolates linearly

The docstring of `resample_to_common_grid` says it uses "constant edge extrapolation". Tracing the code (`spectral_resampler.py`, lines 103–118) shows constant extrapolation only below the first native wavelength; above the last, `idx_right` is clamped to the last index and `idx_left` to the one before it, so the last segment's line is extended (with $t > 1$), as in worked example 5.4. On the real grid this rarely matters: the grid ends at 2,450 nm and the sensors' last bands lie close to or beyond that. It is recorded here because a reader trusting the docstring would compute a different number.

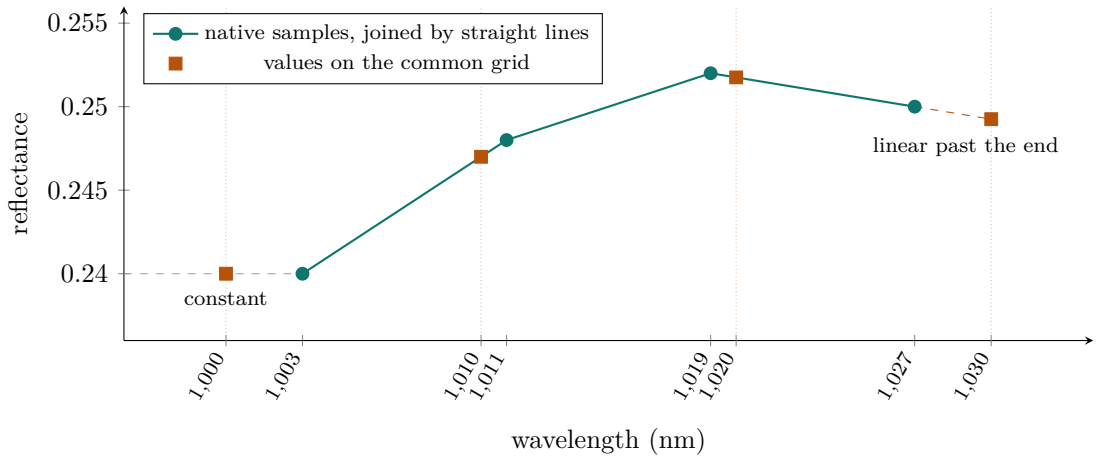


Figure 5.1: Resampling one spectrum onto the grid (worked example 5.4). Native samples (teal) at 1,003, 1,011, 1,019, 1,027 nm; grid values (amber) at 1,000, 1,010, 1,020, 1,030 nm, each read off the straight line between its two neighbours. Below the first sample the value is held constant; above the last, the code extends the last line.

5.5 Why the wavelengths must go up: a preview of a real bug

Binary search, and therefore both `np.searchsorted` and NumPy's `np.interp`, assumes the sample wavelengths are in *ascending* order. NumPy's documentation for `np.interp` says the sequence "is expected to be increasing, but this is not explicitly enforced", and that otherwise "results are meaningless" (quoted in the anomaly-detection course, part 22). Nothing raises an error.

PRISMA lists its bands going *down* in wavelength within each detector, and its builder concatenates SWIR before VNIR (`prisma_dataset_builder.py`, "We will arrange as SWIR followd by VNIR"). What happens when descending samples meet `np.interp`?

Worked example 5.5: The same three samples, listed backwards

Give `np.interp` the samples of worked example 5.3 in descending order: wavelengths $(700, 600, 500)$, values $(4, 2, 1)$. Ask for 520, 550, 650 and 690 nm.
The routine's first check, made for a sorted list, is "is the query at or beyond the last sample wavelength?" The last listed wavelength is 500. Every query, 520, 550, 650 and 690, is at least 500, so for every query the answer is "yes, beyond the end", and it returns the last listed value, 1. All four answers are 1 (`research/textbook/tools/compute_bandpipeline_output.txt`) [C], where the correct answers are 1.2, 1.5, 3.0 and 3.8. No single answer looks absurd; only the fact that they are all equal gives it away. On a whole cube, every resampled band becomes a copy of one band.

The product's resampler and gap-filler both sort first (`np.argsort` in `spectral_resampler.py`, lines 70–78, and `spectral_interpolator.py`), so the product was never affected. The evaluation code was: from 2026-10-17 to 2026-10-18 the evaluator's own resample did not sort, turned every PRISMA cube it resampled into copies of one or two bands, and rewrote a scoreboard before the bug was found and fixed. Chapter 37 tells that story.

Before any interpolation, sort the sample wavelengths ascending and reorder the values with them; NumPy will not warn you if you forget.

Source check: family labels are not re-ordered with the wavelengths

`resample_to_common_grid` sorts the source wavelengths but passes the *unsorted* family list to `_assign_families` together with the sorted wavelengths (`spectral_resampler.py`, lines 71 and 125). Inside, the check `np.all(np.diff(source_wl_sorted)>0)` is always true for sorted wavelengths, so the family list is used in its original order while being indexed by sorted position. For a sensor whose bands are not already ascending (PRISMA: SWIR first, each detector descending), the labels on the grid come out wrong. Running the function on sources $(2000, 1500, 800, 500)$ nm with families (SWIR, SWIR, VNIR, VNIR) labels 500 nm as SWIR and 2,000 nm as VNIR (`research/textbook/tools/compute_bandpipeline_output.txt`) [C]. Only the bookkeeping field `spectral_family_order` is affected, not the reflectance values; the one consumer found while writing this book is the destriper option of the classical `StatisticalEnsembler` (`app/detectors/statistical_ensembler.py`, line 140), whose behaviour with wrong labels was not traced.

5.6 Stage 7: filling holes without inventing features

After stage 6 a kept pixel may still be invalid in up to 40% of its bands. Stage 7 fills those holes from the same pixel's valid bands, along wavelength. Why not a straight line here too, or a smoother curve? The danger is a curve that *overshoots*.

A **cubic spline** joins the samples with cubic polynomials chosen to be as smooth as possible. Smoothness has a price: given samples that rise and then level off, a spline can swing below the lowest sample or above the highest before settling. An invented dip looks exactly like a narrow absorption feature, which is the very evidence used to name a material.

Worked example 5.6: A spline overshoots a step

Four samples: $(0, 0)$, $(1, 0)$, $(2, 1)$, $(3, 1)$, a step up. With four points, SciPy's default spline (the "not-a-knot" condition) is the single cubic through all four. Find it by hand. The cubic is 0 at $x = 0$ and $x = 1$, so it has factors x and $(x - 1)$:

$$p(x) = x(x - 1)(\alpha x + \beta).$$

Use the other two points:

$$p(2) = 2 \times 1 \times (2\alpha + \beta) = 1 \;\Rightarrow\; 2\alpha + \beta = 0.5,$$
$$p(3) = 3 \times 2 \times (3\alpha + \beta) = 1 \;\Rightarrow\; 3\alpha + \beta = 1/6.$$

Subtract the first equation from the second: $\alpha = 1/6 - 0.5 = -1/3$. Then $\beta = 0.5 - 2\alpha = 0.5 + 2/3 = 7/6$. Evaluate between the samples:

$$p(0.5) = 0.5 \times (-0.5) \times \left(-\tfrac{1}{3} \times 0.5 + \tfrac{7}{6}\right) = -0.25 \times \left(-\tfrac{1}{6} + \tfrac{7}{6}\right) = -0.25 \times 1 = -0.25,$$
$$p(2.5) = 2.5 \times 1.5 \times \left(-\tfrac{1}{3} \times 2.5 + \tfrac{7}{6}\right) = 3.75 \times \left(-\tfrac{5}{6} + \tfrac{7}{6}\right) = 3.75 \times \tfrac{1}{3} = 1.25.$$

The spline dips to -0.25, below every sample, and peaks at 1.25, above every sample. SciPy gives the same two numbers (`research/textbook/tools/compute_bandpipeline_output.txt`) [C].

PCHIP (piecewise cubic Hermite interpolating polynomial) also uses one cubic per interval, but chooses the slope at each sample to preserve the data's shape. Where the data is flat on one side or changes direction, PCHIP sets the slope there to 0, so it never creates a minimum

or maximum that is not in the data. On one interval $[x_0, x_1]$ with values y_0, y_1 and end slopes both 0, the Hermite cubic reduces to

$$y(t) = y_0 + (y_1 - y_0)\,(3t^2 - 2t^3), \qquad t = \frac{x - x_0}{x_1 - x_0}, \tag{5.8}$$

which starts at y_0 ($t = 0$), ends at y_1 ($3 - 2 = 1$ at $t = 1$) and is flat at both ends.

Worked example 5.7: PCHIP on the same step

The slopes between neighbouring samples (the secants) are 0 on $[0, 1]$, 1 on $[1, 2]$, 0 on $[2, 3]$. At $x = 1$ one neighbouring secant is 0, so PCHIP's slope there is 0; the same at $x = 2$; at the two ends the secants are 0, so the end slopes are 0. Every interval therefore uses Equation (5.8).

- On $[0, 1]$: $y_0 = y_1 = 0$, so $y = 0$ everywhere, including $x = 0.5$. No dip.
- On $[1, 2]$ at $x = 1.5$: $t = 0.5$, $3t^2 - 2t^3 = 0.75 - 0.25 = 0.5$, so $y = 0 + 1 \times 0.5 = 0.5$.
- On $[2, 3]$: $y_0 = y_1 = 1$, so $y = 1$, including $x = 2.5$. No bump.

SciPy's `PchipInterpolator` gives 0, 0.5 and 1 at the three points [C]. PCHIP gives up some smoothness (its second derivative jumps at the samples) to guarantee it never invents a feature. In a system that hunts for unusual spectral features, that trade is not close.

The filler works per pixel on the wavelength-sorted bands, only on pixels with at least 2 valid bands and at least one hole; fully valid and fully invalid pixels are left alone. Holes beyond the first or last valid band are filled with that band's value (constant extrapolation).

Source check: PCHIP only for common hole patterns

`interpolate_spectral_gaps` groups pixels by their exact pattern of invalid bands and runs PCHIP once per group. Its docstrings say groups of at least 100 pixels get PCHIP. The code uses `MIN_GROUP_SIZE=5000` (`spectral_interpolator.py`, line 64); pixels whose hole pattern is shared by fewer than 5,000 pixels are filled by per-pixel *linear* interpolation (`np.interp`) instead. So "stage 7 is PCHIP" is true for common hole patterns (a dead detector column across the scene) and false for rare ones.

5.7 Stage 8: onto the grid

With every kept pixel complete, stage 8 resamples each one onto the 165 grid wavelengths by the linear interpolation of Section 5.4. The resampler's docstring gives its reason for not using PCHIP here: "the native bands are densely sampled (6-12nm apart) and the target grid is 10nm [...] the sample points barely move, so PCHIP offers no practical quality gain but costs significantly more memory". Invalid pixels stay 0 with validity 0 in every band; valid pixels get validity 1 in every band. FWHM has no meaning on the resampled grid, so `band_fwhm_order` is left empty; spectral matching reads the native onboarding vendable for that reason.

After stage 8 a PRISMA cube and an EnMAP cube have identical shape and identical wavelengths. One model can train on shards mixing both, and a laboratory spectrum resampled onto the grid compares against either. Resampling loses a little native detail; the pipeline accepts that deliberately, because cross-sensor comparability is worth more.

5.8 Per-band normalisation inside the model

The vendable's cube is called `normalized_hyperspectral_cube`, but that "normalized" meant "cleaned by the intake pipeline". A second, different normalisation happens inside the model, at run time, in `app/foundation_models/components/pixel_normalization.py`.

5.8.1 Why normalise

Neural networks learn badly when inputs are tiny, huge, or on very different scales from each other. Reflectance is small (around 0.04 in the blue) and the bands differ: vegetation is dark in blue and bright in the near infrared, so band 0 and band 40 have different typical values and different spreads (Figure 2.2). Fed raw, a network would spend its early training learning that "band 40 is usually bigger than band 0", a fact we already know. Normalisation tells it.

Definition 5.1 (Per-band z-score)**.** For band c with mean μ_c and standard deviation σ_c over a reference corpus, the **z-score** of value $x_c(i, j)$ is

$$z_c(i, j) = \frac{x_c(i, j) - \mu_c}{\sigma_c}. \tag{5.9}$$

It says how many standard deviations the value is above ($z > 0$) or below ($z < 0$) its band's typical value. Each band uses its own μ_c and σ_c.

Analogy

Exam marks from different subjects are not comparable: a 62 in a hard paper may be better than an 80 in an easy one. Converting each mark to "standard deviations above that subject's class average" makes them comparable. Here each band is a subject, the corpus is the class, and the z-score is the comparable mark. The analogy also explains the inverse: if you know a student's z-score and the subject's average and spread, you can recover the raw mark exactly.

Worked example 5.8: Normalising the 3-band, 2-by-2 cube

Use the cube of worked example 2.5:

$$\mathbf{X}[0] = \begin{pmatrix} 0.03 & 0.05 \\ 0.05 & 0.03 \end{pmatrix}, \quad \mathbf{X}[1] = \begin{pmatrix} 0.20 & 0.30 \\ 0.30 & 0.20 \end{pmatrix}, \quad \mathbf{X}[2] = \begin{pmatrix} 0.14 & 0.10 \\ 0.14 & 0.10 \end{pmatrix}.$$

For this exercise, take the "corpus" to be these four pixels and divide by $n = 4$ in the standard deviation. Work each band separately, on paper.
Band 0. Mean:

$$0.03 + 0.05 + 0.05 + 0.03 = 0.16, \qquad \mu_0 = 0.16/4 = 0.04.$$

Deviations $-0.01, +0.01, +0.01, -0.01$; squares 0.0001 each; sum 0.0004; divided by 4, 0.0001; root $\sigma_0 = 0.01$.
Band 1. $0.20 + 0.30 + 0.30 + 0.20 = 1.00$, $\mu_1 = 0.25$. Deviations ± 0.05; squares 0.0025 each; sum 0.0100; divided by 4, 0.0025; $\sigma_1 = 0.05$.
Band 2. $0.14 + 0.10 + 0.14 + 0.10 = 0.48$, $\mu_2 = 0.12$. Deviations ± 0.02; squares 0.0004 each; sum 0.0016; divided by 4, 0.0004; $\sigma_2 = 0.02$.
Z-scores. Pixel $(0, 0)$, spectrum $(0.03, 0.20, 0.14)$:

$$\begin{aligned} z_0 &= (0.03 - 0.04)/0.01 = -0.01/0.01 = -1, \\ z_1 &= (0.20 - 0.25)/0.05 = -0.05/0.05 = -1, \\ z_2 &= (0.14 - 0.12)/0.02 = +0.02/0.02 = +1. \end{aligned}$$

The other pixels the same way:

$$\mathbf{Z}[0] = \begin{pmatrix} -1 & +1 \\ +1 & -1 \end{pmatrix}, \quad \mathbf{Z}[1] = \begin{pmatrix} -1 & +1 \\ +1 & -1 \end{pmatrix}, \quad \mathbf{Z}[2] = \begin{pmatrix} +1 & -1 \\ +1 & -1 \end{pmatrix}.$$

Raw, band 1's values (0.20 to 0.30) were five times band 0's (0.03 to 0.05); after normalisation every band is centred on 0 with spread 1. Note what normalisation does *not* do: pixel $(0, 0)$'s raw spectrum rose from band 0 to band 1, and its z-scores do not; the z-scores describe each value relative to its own band, not the spectrum's shape.

Source check: n or $n - 1$

The script that produced `app/constants/hyperspectral.json`, `scripts/compute_hyperspectral_pixel_stats.py`, finishes with `std=np.sqrt(m2/(n-1))` (line 110): the *sample* standard deviation, dividing by $n - 1$. With 15,264,937 valid pixels per band [M: `app/constants/hyperspectral.json`] the difference from dividing by n is about one part in thirty million, invisible. In the four-pixel example it would matter (dividing by 3 gives $\sigma_0 \approx 0.01155$), which is why the example states its convention.

5.8.2 Denormalisation is the inverse map

At the end of the model, `PixelDenormalize` turns the reconstruction back into reflectance with $x = z\sigma_c + \mu_c$. Derive it by solving Equation (5.9) for x:

$$z = \frac{x - \mu_c}{\sigma_c}.$$

Multiply both sides by σ_c (allowed because $\sigma_c > 0$):

$$\sigma_c z = x - \mu_c.$$

Add μ_c to both sides:

$$x = \sigma_c z + \mu_c.$$

Call the two maps $N(x) = (x - \mu_c)/\sigma_c$ and $D(z) = \sigma_c z + \mu_c$. Composing them in either order gives back what you started with:

$$D(N(x)) = \sigma_c \cdot \frac{x - \mu_c}{\sigma_c} + \mu_c = (x - \mu_c) + \mu_c = x,$$
$$N(D(z)) = \frac{(\sigma_c z + \mu_c) - \mu_c}{\sigma_c} = \frac{\sigma_c z}{\sigma_c} = z.$$

So D is the **inverse** of N. The requirement $\sigma_c > 0$ is real: a band with zero spread cannot be divided by, which is why the per-scene override below clamps tiny spreads to 1. With pixel $(0, 0)$ band 2 of worked example 5.8: $D(+1) = 0.02 \times 1 + 0.12 = 0.14$, the original value.

The model is trained and scored in reflectance, after D, not in z-scores: an L1 error of 0.008 then means "the average band is off by 0.008 reflectance", a statement about the world (Chapter 25).

5.8.3 The code, and broadcasting

```
class PixelNormalize(nn.Module):
    def __init__(self, mean: list[float], std: list[float]):
        super().__init__()
        # (1, C, 1, 1) for broadcasting against (B, C, H, W)
        self.register_buffer("mean", torch.tensor(mean, dtype=torch.float32).view(1,
    -1, 1, 1))
        self.register_buffer("std",  torch.tensor(std,  dtype=torch.float32).view(1,
    -1, 1, 1))
    def forward(self, x):                          # x: (B, 165, H, W)
        return (x - self.mean) / self.std          # (B, 165, H, W)
```

`.view(1,-1,1,1)` reshapes the list of 165 numbers to shape `(1, 165, 1, 1)` without changing any value (the -1 means "work this size out"). Then **broadcasting** lines the shapes up axis by axis: where one array has size 1 and the other a larger size, the size-1 array is repeated along that axis.

	`(B, 165, H, W)`	the batch of cubes
$-$	`(1, 165, 1, 1)`	the means, repeated over B, H, W
	`(B, 165, H, W)`	each band minus its own mean

Band 7's mean is subtracted from every pixel of band 7 in every patch, and from nothing else, with no loop over bands.

The statistics are registered as **buffers**, which matters. A plain attribute is neither saved nor trained; an `nn.Parameter` is saved and trained; a buffer is saved in the checkpoint and moved to the GPU with the model, but never trained. So the 165 means and 165 standard deviations travel *inside* the `.pt` file. Normalise and denormalise are separate modules with their own copies, so a checkpoint holds

$$2 \text{ modules} \times 2 \text{ statistics} \times 165 = 660 \text{ numbers},$$

and you cannot swap the statistics without rebuilding the model: loading checks that every saved entry has a slot of the right shape, and a length mismatch fails loudly at load time. Chapter 20 uses the 660 to reconcile two parameter counts.

5.8.4 The order of operations, and where invalid pixels land

The model's `forward` masks first and normalises second (`app/foundation_models/components/hyperspectral_seg_former_mae.py`, lines 126–133):

```
if mask is not None:
    x = x * mask            # (B, 165, H, W): invalid pixels become 0.0
if self.normalize is not None:
    x = self.normalize(x)   # (B, 165, H, W): z-scores
```

An invalid pixel therefore enters the network at $z_c = (0 - \mu_c)/\sigma_c = -\mu_c/\sigma_c$, not at 0.

Worked example 5.9: Real statistics, band 0

`app/constants/hyperspectral.json` gives band 0 (460 nm) $\mu_0 = 0.04301$ and $\sigma_0 =$

0.03524 [M: app/constants/hyperspectral.json].

$$\begin{aligned}
\text{typical pixel, } 0.04301: &\quad (0.04301 - 0.04301)/0.03524 = 0/0.03524 = 0.00,\\
\text{bright pixel, } 0.07825: &\quad (0.07825 - 0.04301)/0.03524 = 0.03524/0.03524 = 1.00,\\
\text{dark pixel, } 0.00777: &\quad (0.00777 - 0.04301)/0.03524 = -0.03524/0.03524 = -1.00,\\
\text{invalid pixel, } 0: &\quad (0 - 0.04301)/0.03524 = -0.04301/0.03524 \approx -1.22.
\end{aligned}$$

Over all 165 bands, an invalid zero lands between -2.90 (band 27, 730 nm, where the mean is large relative to the spread) and -1.03 (band 164, 2,450 nm) [C].

Had the code normalised first and masked second, invalid pixels would sit at $z = 0$, which means "perfectly typical", indistinguishable from an ordinary pixel. Masking first puts them at a consistent value outside the typical range, which the network can learn to recognise as "not a measurement".

5.8.5 When the corpus statistics do not fit: the per-scene override

Some sensors do not deliver the units a model was fitted on. HotSat-1 ships raw 14-bit numbers around $5{,}000 \pm 400$ [M: app/models/training/inference_config.py]. A model whose buffers expect values near 0.04 would compute $z \approx (5{,}000 - 0.043)/0.035 \approx 143{,}000$: input like nothing it has seen. `PixelStatsOverride` (`app/models/training/inference_config.py`) lets the caller replace the buffers with the scene's own per-band mean and spread, so the input arrives roughly centred with unit spread. The price is that scores become *scene-relative*: comparable within that scene but not across scenes, and the scoring Action records `normalization_mode="per_scene_dn_zscore"` so the interface can say so. For Indradhanu the override does not apply: band-filtered reflectance matches the training distribution, and the mode stays `"baked"`.

Two details of the override are worth copying into your own code. Spreads below 10^{-6} (or not finite) are clamped to 1.0, so a constant scene cannot cause a division by zero. And the moments are computed in 64-bit floating point, for a reason worth seeing in numbers.

Worked example 5.10: Why the moments are computed in float64

Variance can be computed as $\mathbb{E}[x^2] - (\mathbb{E}[x])^2$, the mean of the squares minus the square of the mean. For values $5{,}000 \pm 400$: $\mathbb{E}[x] = 5{,}000$, $(\mathbb{E}[x])^2 = 25{,}000{,}000$, and $\mathbb{E}[x^2] = 25{,}000{,}000 + 400^2 = 25{,}160{,}000$. The answer, 160,000, is the small difference of two large, nearly equal numbers.

A 32-bit float carries 24 binary digits of precision. Between $2^{24} = 16{,}777{,}216$ and 2^{25} the representable numbers are 2 apart, so a single squared value near 25,000,000 is stored to the nearest even integer: harmless. But the code sums squares over a million pixels. That running sum reaches about $10^6 \times 2.5 \times 10^7 = 2.5 \times 10^{13}$, which lies between $2^{44} \approx 1.76 \times 10^{13}$ and 2^{45}, where float32 numbers are $2^{44-23} = 2^{21} = 2{,}097{,}152$ apart. Each new term of about 25,000,000 is added to a total that can only move in steps of about two million, so each addition can be off by up to about a million. After a million additions the sum of squares is wrong by far more than the $160{,}000 \times 10^6$ that separates $\mathbb{E}[x^2]$ from $(\mathbb{E}[x])^2$ in total. The subtraction then returns noise. This is **catastrophic cancellation**. A 64-bit float carries 53 binary digits; at 2.5×10^{13} its spacing is $2^{44-52} = 2^{-8} \approx 0.004$, and the problem disappears.

Source check: does the override ever fire?

The scoring code builds the override when the vendable's `units` string starts with `"DN_"`. As Chapter 4 showed, `VendableThermalDataset` declares no `units` field, so the HotSat-1 builder's `units="DN_14bit_relative"` is discarded by Pydantic and `getattr(vendable, "units",None)` returns `None` (checked by constructing one; see Section 4.9). Through this path the override therefore does not fire, despite the comments in `resolver.py` and `_anomaly_scoring_run.py` that describe it as active.

Values that will drift

Dated 2026-10-29. Indradhanu V2 reads native bands and normalises with `WavelengthStats` (same file, `pixel_normalization.py`): the mean and spread are stored as curves over wavelength, measured on SpectralEarth, and read at each sensor's own band wavelengths by linear interpolation, after sorting the curve because SpectralEarth's band order is not monotonic near 900–1,000 nm. It is Equation (5.9) with μ and σ looked up by wavelength instead of by position (Chapters 23 and 27).

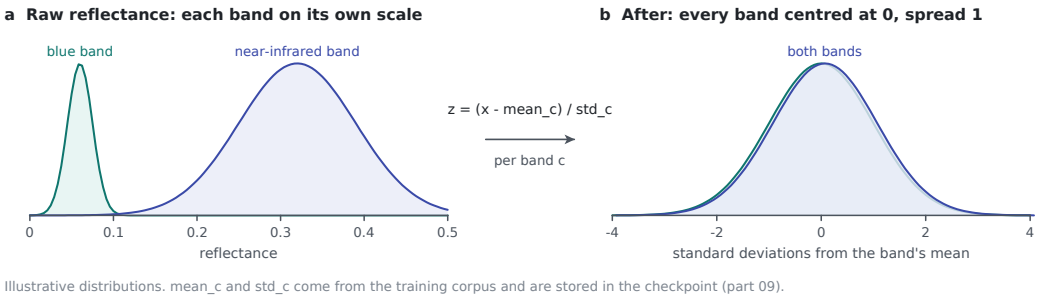


Figure 5.2: Why each band is normalised separately (course figure, Indradhanu part 9). (a) Raw reflectance puts a dim blue band and a bright near-infrared band on very different scales. (b) Subtracting each band's mean and dividing by its standard deviation puts every band on one scale, centred at 0 with spread 1. Distributions are illustrative.

5.9 The nearest-valid pixel fill

After the eight stages, invalid pixels hold 0 in every band. The `band_filter_apply` Action then (by default, `apply_nearest_valid_fill:true`) replaces each invalid pixel's values with those of its nearest valid pixel, leaving the validity cube unchanged (`backend/allotrope/action_types/band_filter_apply.py`, lines 372–381; `app/utils/pixel_fill/nearest_valid_fill.py`).

Definition 5.2 (Nearest-valid fill)**.** For each invalid pixel (i, j), find the valid pixel (i^*, j^*) that minimises the Euclidean distance $\sqrt{(i-i^*)^2 + (j-j^*)^2}$, and copy all C band values from (i^*, j^*) to (i, j). Valid pixels are their own nearest valid pixel (distance 0) and are unchanged. Ties are broken by the implementation, SciPy's `distance_transform_edt`.

The motivation given in the module's docstring: a patch embedding whose kernel straddles a boundary between valid pixels and invalid zeros sees a cliff, produces contaminated tokens there, and attention spreads the contamination, so boundaries light up in the error map and outrank real detections. Copying a real neighbouring measurement turns the cliff into a smooth continuation, and every filled value is physically plausible because it is a genuine measurement, never an average or an interpolation. The masks are untouched, so any loss still excludes those pixels.

Worked example 5.11: Filling a row and a 3-by-3 patch

One row (the docstring's example; one band): values $30, 32, 0, 0, 0, 25, 28$ with validity $1, 1, 0, 0, 0, 1, 1$. Positions 2, 3, 4 are invalid.

- Position 2: valid neighbours at distance 1 (position 1, value 32) and 3 (position 5). Nearest is position 1: 32.
- Position 3: distance 2 to position 1 and 2 to position 5, a tie. The implementation picked position 1: 32.
- Position 4: distance 1 to position 5: 25.

Result $30, 32, 32, 32, 25, 25, 28$ [C].
A 3-by-3 patch (one band, illustrative [A]):

$$\text{values} = \begin{pmatrix} 0.10 & 0.12 & 0 \\ 0.11 & 0 & 0 \\ 0 & 0 & 0.30 \end{pmatrix}, \qquad \text{validity} = \begin{pmatrix} 1 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}.$$

- $(0, 2)$: nearest valid is $(0, 1)$ at distance 1: 0.12.
- $(1, 1)$: $(0, 1)$ and $(1, 0)$ both at distance 1, a tie; the implementation picked $(1, 0)$: 0.11.
- $(1, 2)$: $(2, 2)$ at distance 1 ($(0, 2)$ is itself invalid): 0.30.
- $(2, 0)$: $(1, 0)$ at distance 1: 0.11.
- $(2, 1)$: $(2, 2)$ at distance 1: 0.30.

$$\text{filled} = \begin{pmatrix} 0.10 & 0.12 & 0.12 \\ 0.11 & 0.11 & 0.30 \\ 0.11 & 0.30 & 0.30 \end{pmatrix}$$

(confirmed by running the function, `research/textbook/tools/compute_bandpipeline_output.txt`) [C].

Source check: the kernel, the cliff, and whether the fill reaches Indradhanu

Three corrections, all from reading the source.

1. *The kernel.* The fill's docstring, the anomaly-detection course (part 7) and `docs/03-data-pipeline.md` speak of the SegFormer's "7x7 kernel". The encoder that both Chakshu and Indradhanu use builds its first patch embedding with kernel 4 and stride 4, and the later stages with kernel 3 (`app/foundation_models/components/seg_former_encoder.py`, lines 110–111). The 7 is the default in `overlap_patch_embedding.py`'s docstring, not what the encoder passes. Chapter 13 covers the patch embedding.
2. *The cliff.* "Invalid pixels normalise to about -2.3" is the thermal model's number (an invalid 0 °C pixel). For Indradhanu an invalid zero lands at -1.22 in band 0 and between -2.90 and -1.03 across bands (worked example 5.9).
3. *The re-zeroing.* The scoring code passes the model `mask_tensor` built from the same spatial validity used by the fill (`_anomaly_scoring_run.py`, line 286), and the model's `forward` multiplies by that mask before normalising (lines 127–128 of `hyperspectral_seg_former_mae.py`; the inferencer calls `self.model(tensor, mask=mask, ...)`, `segformer_mae_inferencer.py`, line 222). The filled pixels are therefore set back to 0 before Indradhanu v0.2.0 sees them, and enter at $-\mu_c/\sigma_c$ as in worked example 5.9.

As far as this book could trace, the fill does not change Indradhanu v0.2.0's input on this path; it does change what reaches code that reads the cube without re-masking it (the evaluator also applies it, `app/evaluation/datasets.py`, line 136). Which consumers benefit was not established here.

5.10 Common confusions

Does resampling lose information?
Yes, a little, deliberately: cross-sensor comparability is worth more than the last few nanometres of native resolution. Spectral matching avoids the loss by reading the native onboarding vendable.

Why not interpolate across the atmospheric windows too?
There is no signal there to interpolate from; the gap is up to 170 nm wide. Filling it would invent spectrum. The grid keeps the hole.

188 bands survive stage 4 but the grid has 165. Where did the rest go?
Nowhere: they are different axes. The 188 are surviving sensor bands at sensor wavelengths; the 165 are grid slots. Stage 8 resamples one onto the other.

Is the 0.4 in stage 6 a fraction of the cube or of the pixel?
Of that pixel's surviving bands. A pixel with more than 40% of its own bands invalid is wiped.

Is PCHIP or linear interpolation used?
Both. Stage 7 (gap filling) uses PCHIP for common hole patterns and linear for rare ones; stage 8 (onto the grid) is linear.

Is the vendable's "normalized" cube the z-score?
No. The vendable holds reflectance; the z-score is computed inside the model from statistics stored in the checkpoint.

Why denormalise at the end instead of comparing in z-score space?
So the loss and the score are in reflectance units, which mean something physically.

5.11 Check yourself

Exercise 5.1. Linear interpolation: a spectrum is 0.20 at 1,600 nm and 0.26 at 1,612 nm. Estimate it at 1,610 nm using the weight form, then check with the two-point form. Show every step.

Exercise 5.2. With $C' = 200$ surviving bands and `max_invalid_voxel_fraction` $= 0.4$, what is the largest number of invalid bands a kept pixel can have? Is a pixel with exactly 80 invalid bands kept? Why does the answer depend on "$>$" versus "$\geq$"?

Exercise 5.3. Give `np.interp` wavelengths $(900, 800, 700)$ and values $(0.3, 0.2, 0.1)$, and ask for 750 nm. What does it return, and what is the correct answer? Explain the wrong one using the "beyond the last sample" check.

Exercise 5.4. A band has corpus mean 0.25 and standard deviation 0.05. Normalise the values 0.15, 0.25, 0.40 and an invalid pixel stored as 0. Then denormalise $z = -0.6$ and show algebraically that denormalising the z-score of 0.40 returns 0.40.

Exercise 5.5. Apply nearest-valid fill (one band) to the row $0, 0.20, 0, 0, 0, 0.35$ with validity $0, 1, 0, 0, 0, 1$. Which position is a tie? Then explain in two sentences why the fill copies a measurement rather than averaging the neighbours.

Next: Chapter 6, The mathematical toolkit: vectors, projection, covariance, eigenvectors, PCA and whitening.

Part II

Statistical detection

Chapter 6

The mathematical toolkit: vectors, projection, covariance, eigenvectors, PCA and whitening

The one thing this chapter teaches

A pixel's spectrum is an arrow in a space with one axis per band, and almost every calculation in this book is one of a handful of operations on such arrows: measuring length and angle, projecting one arrow onto another, and turning a cloud of arrows so that it is round instead of stretched. That last operation, whitening, is what the statistical detectors of Chapters 7 and 8 and the residual score of Chapter 31 are built on.

6.1 Why a chapter of linear algebra

The next two chapters build the classical anomaly detectors: the RX detector, its local version, and MNF-RX. The chapter after them in Part VI builds the score that Indradhanu ships with, residual Mahalanobis. All four do the same three things to a cloud of spectra:

1. find the centre of the cloud (a *mean*);
2. find the directions in which the cloud is stretched, and how much (a *covariance* and its *eigenvectors*);
3. rescale every direction so that the cloud becomes round (*whitening*), and measure how far a pixel sits from the centre of the round cloud.

You could try to skip the machinery and score each band on its own: subtract the band's average, divide by its spread, add up. Chapter 7 shows in detail why that fails, but the short version is this. Bands are not independent. Two neighbouring bands of a vegetation spectrum rise and fall together across a scene, so a pixel that is a little high in both is ordinary, and a pixel that is high in one and low in the other is strange, even if neither value alone is extreme. Seeing the second case needs the arithmetic of how bands vary *together*, which is covariance, and the geometry of directions in band space, which is this chapter.

The anomaly-detection course gives this material as a warm-up of five operations (dot product, norm, angle, matrix product, softmax). This chapter keeps the first four, raises them to a derivation, and adds what the detectors need: projection, transpose, the inverse, covariance, eigenvectors, PCA and whitening. Softmax and the 1×1 convolution belong to the neural network chapters; they are taught in Chapters 10 and 14.

Analogy

Think of exam marks. A raw mark of 70 means little until you know the class average and how spread out the class was: 70 in a test where everyone scored between 68 and 72 is unremarkable, and 70 in a test where the average was 40 with a spread of 10 is three spreads above the average. Converting marks to "spreads above average" is the one-dimensional version of whitening. With two subjects that are correlated (students good at algebra tend to be good at geometry) you cannot convert each subject separately; you must first turn the axes to the directions in which the class actually varies, then convert. That turning-then-converting is what this chapter builds.

6.2 Vectors, the dot product, length and angle

Definition 6.1 (Vector)**.** A *vector* with C entries is an ordered list of C numbers, written as a column

$$\mathbf{x} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_C \end{pmatrix}.$$

The set of all such lists is written $\mathbb{R}^C$. You can picture a vector as an arrow from the origin to the point whose coordinates are its entries.

A pixel's spectrum is a vector: x_1 is its reflectance in band 1, x_2 in band 2, and so on. On the 165-band common grid of Chapter 3 a spectrum lives in $\mathbb{R}^{165}$. You cannot draw 165 axes, but every formula below works the same in 2, 3 or 165 dimensions, so we draw the 2-D case and trust the algebra for the rest.

Two operations are defined entry by entry. Adding two vectors adds their entries, $(\mathbf{x}+\mathbf{y})_i = x_i + y_i$; multiplying a vector by a number t (a *scalar*) multiplies every entry, $(t\mathbf{x})_i = tx_i$. Geometrically, $t\mathbf{x}$ is the same arrow stretched by t; for a spectrum it is the same shape at t times the brightness.

6.2.1 The dot product

Definition 6.2 (Dot product)**.** The *dot product* (or inner product) of two vectors of the same length is the number

$$\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^\top \mathbf{b} = a_1 b_1 + a_2 b_2 + \cdots + a_C b_C = \sum_{i=1}^{C} a_i b_i. \tag{6.1}$$

The notation $\mathbf{a}^\top \mathbf{b}$ anticipates Section 6.4: a row times a column. The result is a single number, not a vector. Three properties follow directly from the definition, and you will use all three:

- symmetry: $\mathbf{a}^\top \mathbf{b} = \mathbf{b}^\top \mathbf{a}$, because $a_i b_i = b_i a_i$;
- scaling: $(t\mathbf{a})^\top \mathbf{b} = t(\mathbf{a}^\top \mathbf{b})$, because t factors out of every term of the sum;
- adding: $(\mathbf{a}+\mathbf{c})^\top \mathbf{b} = \mathbf{a}^\top \mathbf{b} + \mathbf{c}^\top \mathbf{b}$, because $(a_i + c_i)b_i = a_i b_i + c_i b_i$.

Worked example 6.1: The dot product of two 3-band spectra

Take the two spectra of the course's warm-up, $\mathbf{a} = (0.1, 0.2, 0.3)$ and $\mathbf{b} = (0.3, 0.2, 0.1)$: one rises towards the long wavelengths, the other falls. Do this on paper. Multiply position by position:

$$\begin{aligned} \text{band 1:} \quad & 0.1 \times 0.3 = 0.03 \\ \text{band 2:} \quad & 0.2 \times 0.2 = 0.04 \\ \text{band 3:} \quad & 0.3 \times 0.1 = 0.03 \end{aligned}$$

then add:

$$\mathbf{a}^\top \mathbf{b} = 0.03 + 0.04 + 0.03 = 0.10.$$

6.2.2 Length

Definition 6.3 (Norm)**.** The *length* or *Euclidean norm* of a vector is

$$\|\mathbf{a}\| = \sqrt{\mathbf{a}^\top \mathbf{a}} = \sqrt{a_1^2 + a_2^2 + \cdots + a_C^2}. \tag{6.2}$$

In two dimensions this is Pythagoras: the arrow to $(3, 4)$ is the hypotenuse of a right triangle with sides 3 and 4, so its length is $\sqrt{9 + 16} = 5$. In more dimensions you apply Pythagoras repeatedly, one axis at a time, and get the same formula. For a spectrum, the length is essentially its overall brightness.

Worked example 6.2: The length of a spectrum

For $\mathbf{a} = (0.1, 0.2, 0.3)$, square each entry:

$$\begin{aligned} 0.1^2 &= 0.01 \\ 0.2^2 &= 0.04 \\ 0.3^2 &= 0.09 \end{aligned}$$

add them, $0.01 + 0.04 + 0.09 = 0.14$, and take the square root: $\|\mathbf{a}\| = \sqrt{0.14} = 0.37417$. The spectrum $\mathbf{b} = (0.3, 0.2, 0.1)$ has the same three squares in a different order, so $\|\mathbf{b}\| = 0.37417$ as well: the order of the bands does not affect the length.

6.2.3 Angle

The dot product and the lengths together give the angle between two arrows.

Definition 6.4 (Angle between vectors)**.** For non-zero $\mathbf{a}$ and $\mathbf{b}$ the angle θ between them satisfies

$$\cos\theta = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|\,\|\mathbf{b}\|}, \qquad 0 \le \theta \le \pi. \tag{6.3}$$

Where does Equation (6.3) come from? Draw the triangle with sides $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{a} - \mathbf{b}$. The school *law of cosines* says the third side satisfies

$$\|\mathbf{a} - \mathbf{b}\|^2 = \|\mathbf{a}\|^2 + \|\mathbf{b}\|^2 - 2\,\|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta.$$

Now compute the left side with the dot product instead. Expand the square using the adding

and symmetry properties:

$$\begin{aligned} \|\mathbf{a}-\mathbf{b}\|^2 &= (\mathbf{a}-\mathbf{b})^\top(\mathbf{a}-\mathbf{b}) && \text{definition of the norm} \\ &= \mathbf{a}^\top\mathbf{a} - \mathbf{a}^\top\mathbf{b} - \mathbf{b}^\top\mathbf{a} + \mathbf{b}^\top\mathbf{b} && \text{multiply out, term by term} \\ &= \|\mathbf{a}\|^2 - 2\,\mathbf{a}^\top\mathbf{b} + \|\mathbf{b}\|^2 && \text{symmetry: } \mathbf{b}^\top\mathbf{a} = \mathbf{a}^\top\mathbf{b}. \end{aligned}$$

Set the two expressions for $\|\mathbf{a}-\mathbf{b}\|^2$ equal. The $\|\mathbf{a}\|^2$ and $\|\mathbf{b}\|^2$ terms cancel, leaving $-2\,\mathbf{a}^\top\mathbf{b} = -2\,\|\mathbf{a}\|\,\|\mathbf{b}\|\cos\theta$. Divide both sides by $-2\,\|\mathbf{a}\|\,\|\mathbf{b}\|$ and you have Equation (6.3).

Worked example 6.3: The angle between two spectra

Every piece is already computed in worked examples 6.1 and 6.2:

$$\begin{aligned} \|\mathbf{a}\|\,\|\mathbf{b}\| &= 0.37417 \times 0.37417 = 0.14 \\ \cos\theta &= \frac{0.10}{0.14} = 0.71429 \\ \theta &= \arccos(0.71429) = 0.7752 \text{ rad}. \end{aligned}$$

To convert radians to degrees multiply by $180/\pi$: $0.7752 \times 180/3.14159 = 44.4°$. The two spectra point in noticeably different directions.
Now replace $\mathbf{b}$ by $2\mathbf{a} = (0.2, 0.4, 0.6)$, the same shape at twice the brightness:

$$\begin{aligned} \mathbf{a}^\top(2\mathbf{a}) &= 0.1 \times 0.2 + 0.2 \times 0.4 + 0.3 \times 0.6 = 0.02 + 0.08 + 0.18 = 0.28 \\ \|2\mathbf{a}\| &= \sqrt{0.04 + 0.16 + 0.36} = \sqrt{0.56} = 0.74833 \\ \cos\theta &= \frac{0.28}{0.37417 \times 0.74833} = \frac{0.28}{0.28} = 1 \\ \theta &= \arccos(1) = 0. \end{aligned}$$

The angle is exactly zero although every entry differs. Angle measures shape and is blind to brightness. That is the spectral angle of Chapter 25, and the reason the training loss pairs it with an L1 term that does notice brightness.

Two arrows with $\mathbf{a}^\top\mathbf{b} = 0$ have $\cos\theta = 0$, so $\theta = \pi/2$: they are at right angles. The word for this is *orthogonal*. A vector of length 1 is a *unit vector*; dividing any non-zero vector by its length makes one, $\mathbf{u} = \mathbf{a}/\|\mathbf{a}\|$.

6.3 Projection: the best multiple of one vector

The single most useful operation in this book is *projection*: given a vector $\mathbf{x}$ and a direction $\mathbf{u}$, find the multiple $t\mathbf{u}$ that comes closest to $\mathbf{x}$. You will meet it again as "the coordinate of a pixel along a principal direction" (this chapter), "the part of a residual inside the scene's main directions" (the foreign fraction of Chapter 34), and as the heart of PCA.

6.3.1 Derivation by minimising a squared distance

Write the squared distance from $\mathbf{x}$ to $t\mathbf{u}$ as a function of the unknown number t:

$$f(t) = \|\mathbf{x} - t\mathbf{u}\|^2 .$$

Expand it, exactly as in the law-of-cosines derivation above:

$$\begin{aligned} f(t) &= (\mathbf{x} - t\mathbf{u})^\top (\mathbf{x} - t\mathbf{u}) && \text{definition of the norm} \\ &= \mathbf{x}^\top\mathbf{x} - t\,\mathbf{x}^\top\mathbf{u} - t\,\mathbf{u}^\top\mathbf{x} + t^2\,\mathbf{u}^\top\mathbf{u} && \text{multiply out; } t \text{ factors out of each term} \\ &= \mathbf{x}^\top\mathbf{x} - 2t\,\mathbf{u}^\top\mathbf{x} + t^2\,\mathbf{u}^\top\mathbf{u} && \text{symmetry of the dot product.} \end{aligned} \tag{6.4}$$

This is a parabola in t: a constant, a term linear in t, and t^2 times the positive number $\mathbf{u}^\top\mathbf{u}$. A parabola that opens upwards has its lowest point where its slope is zero. Differentiate with respect to t (the constant $\mathbf{x}^\top\mathbf{x}$ has slope 0; the derivative of $-2t\,c$ is $-2c$; the derivative of $t^2 c$ is $2tc$):

$$\frac{df}{dt} = -2\,\mathbf{u}^\top\mathbf{x} + 2t\,\mathbf{u}^\top\mathbf{u}.$$

Set the slope to zero and solve for t:

$$\begin{aligned} -2\,\mathbf{u}^\top\mathbf{x} + 2t^\star\,\mathbf{u}^\top\mathbf{u} &= 0 \\ t^\star\,\mathbf{u}^\top\mathbf{u} &= \mathbf{u}^\top\mathbf{x} && \text{add } 2\mathbf{u}^\top\mathbf{x} \text{ to both sides, divide by 2} \\ t^\star &= \frac{\mathbf{u}^\top\mathbf{x}}{\mathbf{u}^\top\mathbf{u}} && \text{divide by } \mathbf{u}^\top\mathbf{u} > 0. \end{aligned} \tag{6.5}$$

Definition 6.5 (Projection onto a vector)**.** The *projection* of $\mathbf{x}$ onto the direction of $\mathbf{u}$ is

$$\mathbf{p} = t^\star\mathbf{u} = \frac{\mathbf{u}^\top\mathbf{x}}{\mathbf{u}^\top\mathbf{u}}\,\mathbf{u}, \tag{6.6}$$

and the *residual* of the projection is $\mathbf{r} = \mathbf{x} - \mathbf{p}$. If $\mathbf{u}$ is a unit vector, $\mathbf{u}^\top\mathbf{u} = 1$ and the formula shortens to $\mathbf{p} = (\mathbf{u}^\top\mathbf{x})\,\mathbf{u}$: the number $\mathbf{u}^\top\mathbf{x}$ is then the *coordinate* of $\mathbf{x}$ along $\mathbf{u}$.

The residual is always orthogonal to $\mathbf{u}$. Check it with the properties of the dot product:

$$\mathbf{u}^\top\mathbf{r} = \mathbf{u}^\top\mathbf{x} - t^\star\,\mathbf{u}^\top\mathbf{u} = \mathbf{u}^\top\mathbf{x} - \frac{\mathbf{u}^\top\mathbf{x}}{\mathbf{u}^\top\mathbf{u}}\,\mathbf{u}^\top\mathbf{u} = \mathbf{u}^\top\mathbf{x} - \mathbf{u}^\top\mathbf{x} = 0.$$

So $\mathbf{x} = \mathbf{p} + \mathbf{r}$ splits $\mathbf{x}$ into a part along $\mathbf{u}$ and a part at right angles to it, and by Pythagoras $\|\mathbf{x}\|^2 = \|\mathbf{p}\|^2 + \|\mathbf{r}\|^2$.

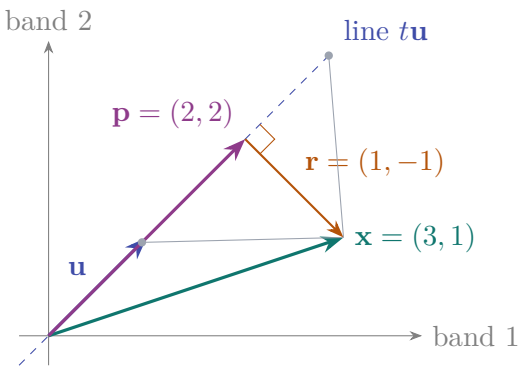


Figure 6.1: Projecting $\mathbf{x} = (3, 1)$ onto the direction $\mathbf{u} = (1, 1)$. The closest point of the line $t\mathbf{u}$ is $\mathbf{p} = 2\mathbf{u} = (2, 2)$; the residual $\mathbf{r}$ meets the line at a right angle. The grey segments go to two other points of the line, which are further away.

Worked example 6.4: Projection in two dimensions

Take $\mathbf{x} = (3, 1)$ and $\mathbf{u} = (1, 1)$, as in Figure 6.1. Do this on paper.

$$\begin{aligned}
\mathbf{u}^\top \mathbf{x} &= 1 \times 3 + 1 \times 1 = 3 + 1 = 4 \\
\mathbf{u}^\top \mathbf{u} &= 1 \times 1 + 1 \times 1 = 2 \\
t^\star &= 4/2 = 2 \\
\mathbf{p} &= 2\,(1, 1) = (2, 2) \\
\mathbf{r} &= (3, 1) - (2, 2) = (1, -1) \\
\mathbf{u}^\top \mathbf{r} &= 1 \times 1 + 1 \times (-1) = 0.
\end{aligned}$$

The residual is orthogonal to $\mathbf{u}$, as promised. Check that $t^\star = 2$ really is the minimum by evaluating $f(t) = \|\mathbf{x} - t\mathbf{u}\|^2$ at a few values of t:

t	$\mathbf{x} - t\mathbf{u}$	squares	$f(t)$
1	$(2, 0)$	$4 + 0$	4
1.5	$(1.5, -0.5)$	$2.25 + 0.25$	2.5
2	$(1, -1)$	$1 + 1$	2
2.5	$(0.5, -1.5)$	$0.25 + 2.25$	2.5
3	$(0, -2)$	$0 + 4$	4

The values fall to 2 at $t = 2$ and rise symmetrically on either side: a parabola with its bottom at $t^\star$. Finally, Pythagoras: $\|\mathbf{x}\|^2 = 9 + 1 = 10$, and $\|\mathbf{p}\|^2 + \|\mathbf{r}\|^2 = (4 + 4) + (1 + 1) = 10$.

Worked example 6.5: Projecting a pixel onto a reference spectrum

A reference spectrum has the shape $\mathbf{u} = (0.1, 0.2, 0.3)$. A pixel reads $\mathbf{x} = (0.12, 0.24, 0.30)$. How much of the pixel is "the reference, scaled", and what is left over? Compute the two dot products:

$$\begin{aligned}
\mathbf{u}^\top \mathbf{x} &= 0.1 \times 0.12 + 0.2 \times 0.24 + 0.3 \times 0.30 = 0.012 + 0.048 + 0.090 = 0.150 \\
\mathbf{u}^\top \mathbf{u} &= 0.01 + 0.04 + 0.09 = 0.14.
\end{aligned}$$

The best multiple is

$$t^\star = \frac{0.150}{0.14} = 1.0714,$$

so the projection is

$$\mathbf{p} = 1.0714 \times (0.1, 0.2, 0.3) = (0.10714,\ 0.21429,\ 0.32143)$$

and the residual is

$$\begin{aligned}
\mathbf{r} = \mathbf{x} - \mathbf{p} &= (0.12 - 0.10714,\ 0.24 - 0.21429,\ 0.30 - 0.32143) \\
&= (0.01286,\ 0.02571,\ -0.02143).
\end{aligned}$$

Check orthogonality: $0.1 \times 0.01286 + 0.2 \times 0.02571 + 0.3 \times (-0.02143) = 0.001286 + 0.005143 - 0.006429 = 0$. The pixel is the reference at 107% brightness, plus a small residual that is a little high in bands 1 and 2 and a little low in band 3. Its squared length, $0.01286^2 + 0.02571^2 + 0.02143^2 = 0.000165 + 0.000661 + 0.000459 = 0.001286$, is under 1% of $\|\mathbf{x}\|^2 = 0.0144 + 0.0576 + 0.09 = 0.162$: the pixel is almost entirely the reference shape.

Projection onto a unit vector is one dot product, and what it leaves behind is at right angles to the direction you projected on.

6.4 Matrices, the matrix-vector product and the transpose

Definition 6.6 (Matrix)**.** A *matrix* with m rows and n columns is a rectangular table of numbers; A_{ij} is the entry in row i, column j. A vector of length n is a matrix with n rows and one column.

6.4.1 Matrix times vector

Definition 6.7 (Matrix-vector product)**.** For an $m \times n$ matrix $\mathbf{A}$ and a vector $\mathbf{x}$ of length n, the product $\mathbf{y} = \mathbf{A}\mathbf{x}$ is the vector of length m with

$$y_i = \sum_{j=1}^{n} A_{ij}\, x_j = (\text{row } i \text{ of } \mathbf{A})^\top \mathbf{x}. \tag{6.7}$$

Read Equation (6.7) in two ways. Row by row, each output entry is the dot product of one row of $\mathbf{A}$ with $\mathbf{x}$, so each output is a weighted mixture of all the inputs with its own weights. If the rows of $\mathbf{A}$ are unit vectors, the outputs are the coordinates of $\mathbf{x}$ along those directions (Section 6.3); this is exactly how a pixel is turned onto its principal directions later in the chapter.

Worked example 6.6: A matrix times a vector

Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{v} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$.

$$y_1 = (\text{row } 1)^\top \mathbf{v} = 1 \times 1 + 2 \times (-1) = 1 - 2 = -1$$
$$y_2 = (\text{row } 2)^\top \mathbf{v} = 3 \times 1 + 4 \times (-1) = 3 - 4 = -1$$

so $\mathbf{A}\mathbf{v} = (-1, -1)$.

6.4.2 Matrix times matrix

Definition 6.8 (Matrix product)**.** For an $m \times n$ matrix $\mathbf{A}$ and an $n \times p$ matrix $\mathbf{B}$, the product $\mathbf{AB}$ is the $m \times p$ matrix with

$$(\mathbf{AB})_{ik} = \sum_{j=1}^{n} A_{ij} B_{jk} = (\text{row } i \text{ of } \mathbf{A})^\top (\text{column } k \text{ of } \mathbf{B}).$$

The inner sizes must agree (n and n). Applying $\mathbf{AB}$ to a vector is the same as applying $\mathbf{B}$ first and then $\mathbf{A}$: $(\mathbf{AB})\mathbf{x} = \mathbf{A}(\mathbf{B}\mathbf{x})$.

Definition 6.9 (Transpose)**.** The *transpose* $\mathbf{A}^\top$ of an $m \times n$ matrix is the $n \times m$ matrix with the rows and columns swapped: $(\mathbf{A}^\top)_{ij} = A_{ji}$. A column vector transposed is a row vector, which is why $\mathbf{a}^\top \mathbf{b}$ (a row times a column) is the dot product. A matrix equal to its own transpose, $\mathbf{S}^\top = \mathbf{S}$, is *symmetric*.

The rule you will use constantly is that transposing a product reverses the order:

$$(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top. \tag{6.8}$$

The proof is one line of index juggling. Entry (i,k) of the left side is entry (k,i) of $\mathbf{AB}$, which is $\sum_j A_{kj}B_{ji}$. Entry (i,k) of the right side is $\sum_j (\mathbf{B}^\top)_{ij}(\mathbf{A}^\top)_{jk} = \sum_j B_{ji}A_{kj}$. The two sums contain the same products, so they are equal. Order matters because matrix products do not commute in general: $\mathbf{AB} \neq \mathbf{BA}$.

Worked example 6.7: The transpose of a product, on 2×2 matrices

Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 0 & 1 \\ 5 & 2 \end{pmatrix}$. Do this on paper. First $\mathbf{AB}$, entry by entry (row of $\mathbf{A}$ with column of $\mathbf{B}$):

$$\begin{aligned} (1,1): &\quad 1\times 0 + 2\times 5 = 0 + 10 = 10 \\ (1,2): &\quad 1\times 1 + 2\times 2 = 1 + 4 = 5 \\ (2,1): &\quad 3\times 0 + 4\times 5 = 0 + 20 = 20 \\ (2,2): &\quad 3\times 1 + 4\times 2 = 3 + 8 = 11 \end{aligned}$$

so $\mathbf{AB} = \begin{pmatrix} 10 & 5 \\ 20 & 11 \end{pmatrix}$ and $(\mathbf{AB})^\top = \begin{pmatrix} 10 & 20 \\ 5 & 11 \end{pmatrix}$. Now $\mathbf{B}^\top = \begin{pmatrix} 0 & 5 \\ 1 & 2 \end{pmatrix}$ and $\mathbf{A}^\top = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$, and

$$\begin{aligned} (1,1): &\quad 0\times 1 + 5\times 2 = 0 + 10 = 10 \\ (1,2): &\quad 0\times 3 + 5\times 4 = 0 + 20 = 20 \\ (2,1): &\quad 1\times 1 + 2\times 2 = 1 + 4 = 5 \\ (2,2): &\quad 1\times 3 + 2\times 4 = 3 + 8 = 11 \end{aligned}$$

so $\mathbf{B}^\top\mathbf{A}^\top = \begin{pmatrix} 10 & 20 \\ 5 & 11 \end{pmatrix}$, equal to $(\mathbf{AB})^\top$. The wrong order gives something else: $\mathbf{A}^\top\mathbf{B}^\top = \begin{pmatrix} 1\times 0 + 3\times 1 & 1\times 5 + 3\times 2 \\ 2\times 0 + 4\times 1 & 2\times 5 + 4\times 2 \end{pmatrix} = \begin{pmatrix} 3 & 11 \\ 4 & 18 \end{pmatrix}$.

6.4.3 The identity and the inverse

The *identity matrix* $\mathbf{I}$ has ones on the diagonal and zeros elsewhere; $\mathbf{Ix} = \mathbf{x}$ for every $\mathbf{x}$. A square matrix $\mathbf{A}$ is *invertible* if there is a matrix $\mathbf{A}^{-1}$ with $\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$. The inverse undoes the matrix: if $\mathbf{y} = \mathbf{Ax}$ then $\mathbf{x} = \mathbf{A}^{-1}\mathbf{y}$.

Definition 6.10 (Inverse of a 2×2 matrix)**.** For $\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$ with *determinant* $\det \mathbf{A} = ad - bc \neq 0$,

$$\mathbf{A}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}. \tag{6.9}$$

You do not need to memorise a derivation; you need to be able to check it. Multiply:

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix} = \begin{pmatrix} ad - bc & -ab + ba \\ cd - dc & -cb + da \end{pmatrix} = \begin{pmatrix} ad - bc & 0 \\ 0 & ad - bc \end{pmatrix} = (ad - bc)\,\mathbf{I}.$$

Dividing by $ad - bc$ gives $\mathbf{I}$. The recipe is: swap the diagonal, negate the off-diagonal, divide by the determinant. When the determinant is zero there is no inverse: the matrix squashes some direction flat and nothing can undo that. When the determinant is *nearly* zero, the inverse exists but contains huge numbers. Chapter 7 is in part a chapter about that second case.

Worked example 6.8: Inverting a 2×2 matrix by hand

Take $\mathbf{S} = \begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}$, which is the covariance matrix you will build in worked example 6.9.

$$\det \mathbf{S} = 2 \times 10 - 3 \times 3 = 20 - 9 = 11$$
$$\mathbf{S}^{-1} = \frac{1}{11} \begin{pmatrix} 10 & -3 \\ -3 & 2 \end{pmatrix} = \begin{pmatrix} 0.9091 & -0.2727 \\ -0.2727 & 0.1818 \end{pmatrix}.$$

Check the first column of $\mathbf{S}\mathbf{S}^{-1}$ (using the fractions, not the decimals): $\frac{1}{11}(2\times10+3\times(-3)) = \frac{1}{11}(20-9) = 1$ and $\frac{1}{11}(3\times10+10\times(-3)) = \frac{1}{11}(30-30) = 0$. Do the second column yourself; you should get 0 and 1.

6.5 Mean, centring and the covariance matrix

Now the statistics. Suppose a scene has N valid pixels with spectra $\mathbf{x}_1, \ldots, \mathbf{x}_N$, each in $\mathbb{R}^C$.

Definition 6.11 (Mean vector)**.** The *mean* spectrum is the band-by-band average

$$\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^{N} \mathbf{x}_n. \tag{6.10}$$

Definition 6.12 (Centring)**.** *Centring* subtracts the mean from every pixel: $\mathbf{d}_n = \mathbf{x}_n - \boldsymbol{\mu}$. The centred vectors (*deviations*) always add to the zero vector, because $\sum_n \mathbf{d}_n = \sum_n \mathbf{x}_n - N\boldsymbol{\mu} = N\boldsymbol{\mu} - N\boldsymbol{\mu} = \mathbf{0}$.

The *variance* of one band is the average squared deviation in that band. The *covariance* of two bands i and j is the average of the product of their deviations, $d_{n,i}\, d_{n,j}$. When the two bands tend to be above their means together and below together, most products are positive and the covariance is positive; when one tends to be up while the other is down, it is negative; when they are unrelated, positive and negative products cancel and it is near zero. The variance is the covariance of a band with itself.

All C^2 of these numbers fit into one matrix, and the neat way to build it is from *outer products*.

Definition 6.13 (Outer product)**.** For a column vector $\mathbf{d}$ of length C, the outer product $\mathbf{d}\mathbf{d}^\top$ is the $C \times C$ matrix whose entry (i, j) is $d_i d_j$. (Compare the dot product $\mathbf{d}^\top\mathbf{d}$, a single number: the same symbols in the other order.)

Entry (i, j) of $\mathbf{d}_n\mathbf{d}_n^\top$ is exactly the product $d_{n,i}d_{n,j}$ that the covariance of bands i and j averages. So averaging the outer products averages every pairwise product at once:

Definition 6.14 (Covariance matrix)**.** The (sample) *covariance matrix* of the pixels is

$$\boldsymbol{\Sigma} = \frac{1}{N-1} \sum_{n=1}^{N} (\mathbf{x}_n - \boldsymbol{\mu})(\mathbf{x}_n - \boldsymbol{\mu})^\top = \frac{1}{N-1} \sum_{n=1}^{N} \mathbf{d}_n\mathbf{d}_n^\top. \tag{6.11}$$

Its diagonal holds the band variances; entry (i, j) off the diagonal holds the covariance of bands i and j. Because $d_i d_j = d_j d_i$, it is symmetric.

Why $N-1$ and not N? The deviations are measured from the sample's own mean, which sits a little closer to the sample than the true mean does, so the average squared deviation slightly

underestimates the true spread. Dividing by $N-1$ instead of N corrects that on average. The deeper reason, which Chapter 8 needs, is that the N deviations add up to zero, so only $N-1$ of them are free. With millions of pixels the difference between the two divisors is negligible; with a handful it is not, and you must know which one the code uses.

Source check: which divisor the code uses

`np.cov` divides by $N-1$, and every multi-band detector in this book computes its covariance that way: the `spectral` library behind `spectral.rx` calls `np.cov` on array input (`spectral/algorithms/algorithms.py`, function `mean_cov`, version 0.23.1), `app/detectors/_local_background.py` line 68 divides by `counts_f-1`, and `app/utils/anomaly_detection/residual_mahalanobis.py` line 44 calls `np.cov`. The one exception is the single-band thermal detector, `app/detectors/thermal_grx_detector.py` line 100, which uses `vals.var()`, numpy's default divisor N. On a scene of millions of pixels the two agree to many digits.

Worked example 6.9: A covariance matrix from four pixels and two bands

Four pixels, two bands, reflectance in percent. Do every line on paper.

pixel	band 1	band 2
1	8	18
2	10	17
3	11	24
4	11	21

Step 1, the mean.

$$\mu_1 = (8+10+11+11)/4 = 40/4 = 10$$
$$\mu_2 = (18+17+24+21)/4 = 80/4 = 20.$$

Step 2, centre. Subtract $(10, 20)$ from each pixel:

$$\mathbf{d}_1 = (8-10,\ 18-20) = (-2,-2)$$
$$\mathbf{d}_2 = (10-10,\ 17-20) = (0,-3)$$
$$\mathbf{d}_3 = (11-10,\ 24-20) = (1,4)$$
$$\mathbf{d}_4 = (11-10,\ 21-20) = (1,1).$$

Check that they add to zero: band 1, $-2+0+1+1=0$; band 2, $-2-3+4+1=0$.
Step 3, the outer products. Entry (i,j) is $d_i d_j$:

$$\mathbf{d}_1\mathbf{d}_1^\top = \begin{pmatrix}(-2)(-2) & (-2)(-2)\\ (-2)(-2) & (-2)(-2)\end{pmatrix} = \begin{pmatrix}4 & 4\\ 4 & 4\end{pmatrix}$$
$$\mathbf{d}_2\mathbf{d}_2^\top = \begin{pmatrix}0\times 0 & 0\times(-3)\\ (-3)\times 0 & (-3)(-3)\end{pmatrix} = \begin{pmatrix}0 & 0\\ 0 & 9\end{pmatrix}$$
$$\mathbf{d}_3\mathbf{d}_3^\top = \begin{pmatrix}1\times 1 & 1\times 4\\ 4\times 1 & 4\times 4\end{pmatrix} = \begin{pmatrix}1 & 4\\ 4 & 16\end{pmatrix}$$
$$\mathbf{d}_4\mathbf{d}_4^\top = \begin{pmatrix}1\times 1 & 1\times 1\\ 1\times 1 & 1\times 1\end{pmatrix} = \begin{pmatrix}1 & 1\\ 1 & 1\end{pmatrix}.$$

Step 4, add them entry by entry:

$$\begin{aligned} (1,1): &\quad 4+0+1+1=6 \\ (1,2)=(2,1): &\quad 4+0+4+1=9 \\ (2,2): &\quad 4+9+16+1=30. \end{aligned}$$

Step 5, divide by $N-1=3$:

$$\boldsymbol{\Sigma} = \frac{1}{3}\begin{pmatrix} 6 & 9 \\ 9 & 30 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}.$$

Read it: band 1 has variance 2 (spread $\sqrt{2}=1.41$), band 2 has variance 10 (spread $\sqrt{10}=3.16$), and their covariance is $+3$: they tend to rise and fall together. The *correlation*, the covariance divided by both spreads, is $3/(\sqrt{2}\times\sqrt{10}) = 3/4.472 = 0.67$. Had you divided by $N=4$ you would have got $\begin{pmatrix} 1.5 & 2.25 \\ 2.25 & 7.5 \end{pmatrix}$, smaller by the factor $3/4$.

In matrix form the whole calculation is one line. Stack the centred pixels as the rows of an $N \times C$ matrix $\mathbf{D}$. Then $\mathbf{D}^\top\mathbf{D}$ is $C \times C$, and its entry (i,j) is $\sum_n d_{n,i}d_{n,j}$: exactly the sum of outer products. So

$$\boldsymbol{\Sigma} = \frac{1}{N-1}\,\mathbf{D}^\top\mathbf{D}. \tag{6.12}$$

This is the line `cov=dX.transpose(-1,-2)@dX` in `app/detectors/_local_background.py`, done for a whole batch of pixels at once.

6.5.1 How a linear map changes the covariance

Almost everything in the rest of Part II transforms the pixels by a matrix and asks what happens to their mean and covariance. The answer is short and worth deriving once. Let $\mathbf{y}_n = \mathbf{A}\mathbf{x}_n$ for a fixed matrix $\mathbf{A}$.

The mean of the $\mathbf{y}$'s is $\mathbf{A}\boldsymbol{\mu}$, because a matrix product distributes over the sum: $\frac{1}{N}\sum_n \mathbf{A}\mathbf{x}_n = \mathbf{A}\,\frac{1}{N}\sum_n \mathbf{x}_n$. So the deviations are $\mathbf{y}_n - \mathbf{A}\boldsymbol{\mu} = \mathbf{A}\mathbf{d}_n$, and

$$\begin{aligned} \boldsymbol{\Sigma}_y &= \frac{1}{N-1}\sum_n (\mathbf{A}\mathbf{d}_n)(\mathbf{A}\mathbf{d}_n)^\top && \text{definition, Equation (6.11)} \\ &= \frac{1}{N-1}\sum_n \mathbf{A}\mathbf{d}_n\mathbf{d}_n^\top\mathbf{A}^\top && \text{transpose of a product, Equation (6.8)} \\ &= \mathbf{A}\left(\frac{1}{N-1}\sum_n \mathbf{d}_n\mathbf{d}_n^\top\right)\mathbf{A}^\top && \mathbf{A} \text{ is the same in every term; factor it out} \\ &= \mathbf{A}\,\boldsymbol{\Sigma}\,\mathbf{A}^\top. && \end{aligned} \tag{6.13}$$

> If $\mathbf{y} = \mathbf{A}\mathbf{x}$, then $\boldsymbol{\Sigma}_y = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$.

A special case is a single direction. Take $\mathbf{A}$ to be one row, $\mathbf{u}^\top$ with $\mathbf{u}$ a unit vector. Then $y_n = \mathbf{u}^\top\mathbf{x}_n$ is the coordinate of each pixel along $\mathbf{u}$, and its variance is the 1×1 matrix

$$\mathrm{Var}(\mathbf{u}^\top\mathbf{x}) = \mathbf{u}^\top\boldsymbol{\Sigma}\,\mathbf{u}. \tag{6.14}$$

The covariance matrix therefore knows the spread of the cloud in *every* direction, not only along the band axes. For the matrix of worked example 6.9 and $\mathbf{u}=(1,0)$ you get $\mathbf{u}^\top\boldsymbol{\Sigma}\mathbf{u} = \Sigma_{11} = 2$,

the variance of band 1. For the diagonal $\mathbf{u} = (1,1)/\sqrt{2}$:

$$\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} = \tfrac{1}{2}(1,1)\begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = \tfrac{1}{2}(1,1)\begin{pmatrix} 5 \\ 13 \end{pmatrix} = \tfrac{1}{2}(5+13) = 9.$$

Along the diagonal the cloud spreads more than along either axis. Which direction spreads the most? That is the question eigenvectors answer.

6.6 Eigenvalues and eigenvectors

Definition 6.15 (Eigenvector and eigenvalue)**.** A non-zero vector $\mathbf{u}$ is an *eigenvector* of a square matrix $\mathbf{S}$, with *eigenvalue* λ, if

$$\mathbf{S}\mathbf{u} = \lambda \mathbf{u}. \tag{6.15}$$

The matrix does not turn an eigenvector; it only stretches it by λ.

Most vectors are turned by a matrix. For $\mathbf{S} = \begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}$, the vector $(1,0)$ goes to $(2,3)$, a different direction. We want the special directions that are only stretched.

6.6.1 Finding them by hand: the characteristic polynomial

Rewrite Equation (6.15) as $\mathbf{S}\mathbf{u} - \lambda\mathbf{u} = \mathbf{0}$, that is $(\mathbf{S} - \lambda\mathbf{I})\mathbf{u} = \mathbf{0}$. A non-zero $\mathbf{u}$ is sent to zero, so the matrix $\mathbf{S} - \lambda\mathbf{I}$ squashes a direction flat, and (Equation (6.9)) its determinant must be zero. For a 2×2 matrix $\mathbf{S} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ (symmetric: both off-diagonal entries are b):

$$\begin{aligned} \det(\mathbf{S} - \lambda\mathbf{I}) &= (a-\lambda)(c-\lambda) - b^2 \\ &= \lambda^2 - (a+c)\lambda + (ac - b^2) = 0. \end{aligned} \tag{6.16}$$

This quadratic is the *characteristic polynomial*. Its coefficients are the *trace* $a + c$ (the sum of the diagonal) and the determinant $ac - b^2$. Its two roots are the eigenvalues:

$$\lambda_{1,2} = \frac{(a+c) \pm \sqrt{(a-c)^2 + 4b^2}}{2}. \tag{6.17}$$

(Expand $(a+c)^2 - 4(ac - b^2) = a^2 - 2ac + c^2 + 4b^2 = (a-c)^2 + 4b^2$ to get the square root.) The quantity under the root is a sum of squares, so it is never negative: a symmetric 2×2 matrix always has two real eigenvalues. For each eigenvalue, the first row of $(\mathbf{S} - \lambda\mathbf{I})\mathbf{u} = \mathbf{0}$, namely $(a - \lambda)u_1 + bu_2 = 0$, gives the direction.

Worked example 6.10: Eigenvalues and eigenvectors of the covariance matrix

For $\boldsymbol{\Sigma} = \begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}$: trace $2 + 10 = 12$, determinant $2 \times 10 - 3 \times 3 = 11$. The characteristic polynomial is

$$\lambda^2 - 12\lambda + 11 = 0.$$

It factorises: $(\lambda - 11)(\lambda - 1) = \lambda^2 - 11\lambda - \lambda + 11 = \lambda^2 - 12\lambda + 11$. So $\lambda_1 = 11$ and $\lambda_2 = 1$. (The formula agrees: $\sqrt{(2-10)^2 + 4 \times 9} = \sqrt{64 + 36} = 10$, and $(12 \pm 10)/2 = 11$ or 1.)
Eigenvector for $\lambda_1 = 11$. The first row gives $(2 - 11)u_1 + 3u_2 = 0$, so $-9u_1 + 3u_2 = 0$, so $u_2 = 3u_1$. Take $(1,3)$, length $\sqrt{1+9} = \sqrt{10}$, and divide: $\mathbf{u}_1 = (1,3)/\sqrt{10} = (0.3162, 0.9487)$. Check without the scaling: $\boldsymbol{\Sigma}(1,3) = (2 \times 1 + 3 \times 3,\ 3 \times 1 + 10 \times 3) = (11, 33) = 11 \times (1,3)$.

Eigenvector for $\lambda_2 = 1$. $(2-1)u_1 + 3u_2 = 0$, so $u_1 = -3u_2$. Take $(3, -1)$ and divide by $\sqrt{10}$: $\mathbf{u}_2 = (3,-1)/\sqrt{10} = (0.9487, -0.3162)$. Check: $\boldsymbol{\Sigma}(3,-1) = (6-3,\ 9-10) = (3,-1) = 1 \times (3,-1)$.
The two eigenvectors are orthogonal: $(1,3)^\top(3,-1) = 3 - 3 = 0$.

The orthogonality is not luck. For any symmetric $\mathbf{S}$, take eigenvectors $\mathbf{u}$ and $\mathbf{v}$ with different eigenvalues $\lambda \neq \nu$. Compute $\mathbf{v}^\top \mathbf{S}\mathbf{u}$ two ways: as $\mathbf{v}^\top(\lambda\mathbf{u}) = \lambda\,\mathbf{v}^\top\mathbf{u}$, and, using $\mathbf{S}^\top = \mathbf{S}$ and Equation (6.8), as $(\mathbf{S}\mathbf{v})^\top\mathbf{u} = \nu\,\mathbf{v}^\top\mathbf{u}$. So $(\lambda - \nu)\,\mathbf{v}^\top\mathbf{u} = 0$, and since $\lambda \neq \nu$ the dot product must be zero.

6.6.2 The spectral theorem

Definition 6.16 (Spectral theorem for symmetric matrices)**.** Every real symmetric $C \times C$ matrix $\mathbf{S}$ has C real eigenvalues $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_C$ and a matching set of C orthogonal unit eigenvectors $\mathbf{u}_1, \ldots, \mathbf{u}_C$. Put the eigenvectors as the columns of $\mathbf{U}$ and the eigenvalues on the diagonal of $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_C)$. Then

$$\mathbf{U}^\top\mathbf{U} = \mathbf{I} \qquad \text{and} \qquad \mathbf{S} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top = \sum_{k=1}^{C} \lambda_k\,\mathbf{u}_k\mathbf{u}_k^\top. \tag{6.18}$$

A square matrix with $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$ is *orthogonal*: it only rotates (or reflects), never stretches, and its inverse is its transpose.

We prove the two-by-two case by multiplication in the worked example below and take the general case on trust; it is proved in every linear algebra text. The decomposition is what `np.linalg.eigh` computes; the `h` stands for Hermitian, the complex-number word for symmetric. It returns the eigenvalues in *ascending* order, which is why every piece of code in this repository that uses it reverses the order with `np.argsort(evals)[::-1]`.

A covariance matrix has one more property: its eigenvalues are never negative. By Equation (6.14), $\lambda_k = \mathbf{u}_k^\top\boldsymbol{\Sigma}\mathbf{u}_k$ is the variance of the pixels along $\mathbf{u}_k$, and a variance is an average of squares. A matrix with all eigenvalues ≥ 0 is *positive semi-definite*; if all are > 0 it is *positive definite* and invertible.

Worked example 6.11: Rebuilding the covariance from its eigenvectors

With $\mathbf{u}_1 = (1,3)/\sqrt{10}$, $\mathbf{u}_2 = (3,-1)/\sqrt{10}$, $\lambda_1 = 11$, $\lambda_2 = 1$, use the sum form of Equation (6.18). The outer products are

$$\mathbf{u}_1\mathbf{u}_1^\top = \frac{1}{10}\begin{pmatrix} 1 & 3 \\ 3 & 9 \end{pmatrix}, \qquad \mathbf{u}_2\mathbf{u}_2^\top = \frac{1}{10}\begin{pmatrix} 9 & -3 \\ -3 & 1 \end{pmatrix}.$$

Weight them by the eigenvalues and add:

$$11 \times \frac{1}{10}\begin{pmatrix} 1 & 3 \\ 3 & 9 \end{pmatrix} + 1 \times \frac{1}{10}\begin{pmatrix} 9 & -3 \\ -3 & 1 \end{pmatrix} = \frac{1}{10}\begin{pmatrix} 11+9 & 33-3 \\ 33-3 & 99+1 \end{pmatrix} = \frac{1}{10}\begin{pmatrix} 20 & 30 \\ 30 & 100 \end{pmatrix} = \begin{pmatrix} 2 & 3 \\ 3 & 10 \end{pmatrix}.$$

The covariance is back. Read the sum form as a statement about the cloud: it is the sum of a spread of 11 along $\mathbf{u}_1$ and a spread of 1 along $\mathbf{u}_2$, and nothing else.

6.7 Principal component analysis

6.7.1 PCA as variance maximisation

Principal component analysis (PCA) asks: along which unit direction $\mathbf{u}$ do the pixels spread the most? By Equation (6.14) the spread along $\mathbf{u}$ is $\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u}$, so the question is the constrained maximisation

$$\max_{\mathbf{u}} \ \mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} \qquad \text{subject to} \qquad \mathbf{u}^\top \mathbf{u} = 1. \tag{6.19}$$

The constraint matters: without it you could make $\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u}$ as large as you like by making $\mathbf{u}$ long.

The standard tool for maximising under a constraint is a *Lagrange multiplier*. The idea in one sentence: at the best point you cannot increase the objective by moving along the constraint, so the objective's gradient must point straight out of the constraint surface, which means it is a multiple λ of the constraint's gradient. Formally, build

$$L(\mathbf{u}, \lambda) = \mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} - \lambda \left(\mathbf{u}^\top \mathbf{u} - 1 \right)$$

and set its derivatives with respect to every entry of $\mathbf{u}$ to zero.

We need the derivative of a quadratic form. Work it out on two bands first. With $\boldsymbol{\Sigma} = \begin{pmatrix} a & b \\ b & c \end{pmatrix}$ and $\mathbf{u} = (u_1, u_2)$, multiplying out gives

$$\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} = a u_1^2 + 2b\, u_1 u_2 + c u_2^2.$$

Differentiate with respect to u_1 (treat u_2 as a constant) and with respect to u_2:

$$\frac{\partial}{\partial u_1} \left(a u_1^2 + 2b u_1 u_2 + c u_2^2 \right) = 2a u_1 + 2b u_2,$$
$$\frac{\partial}{\partial u_2} \left(a u_1^2 + 2b u_1 u_2 + c u_2^2 \right) = 2b u_1 + 2c u_2.$$

Stacked, these are $2 \begin{pmatrix} a & b \\ b & c \end{pmatrix} \begin{pmatrix} u_1 \\ u_2 \end{pmatrix} = 2\boldsymbol{\Sigma}\mathbf{u}$. The same holds in any number of dimensions for a symmetric matrix: the gradient of $\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u}$ is $2\boldsymbol{\Sigma}\mathbf{u}$. With $\boldsymbol{\Sigma} = \mathbf{I}$ it says the gradient of $\mathbf{u}^\top \mathbf{u}$ is $2\mathbf{u}$.

Now set the gradient of L to zero:

$$\begin{aligned} 2\boldsymbol{\Sigma}\mathbf{u} - 2\lambda\mathbf{u} &= \mathbf{0} && \text{gradient of each term} \\ \boldsymbol{\Sigma}\mathbf{u} &= \lambda\mathbf{u} && \text{add } 2\lambda\mathbf{u}\text{, divide by 2.} \end{aligned} \tag{6.20}$$

The best direction is an eigenvector of $\boldsymbol{\Sigma}$. Which one? Multiply Equation (6.20) on the left by $\mathbf{u}^\top$ and use $\mathbf{u}^\top \mathbf{u} = 1$:

$$\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} = \lambda\, \mathbf{u}^\top \mathbf{u} = \lambda.$$

The variance along an eigenvector is its eigenvalue, so the maximum is reached at the eigenvector with the *largest* eigenvalue. That eigenvector is the *first principal direction*, and the coordinate of a pixel along it, $\mathbf{u}_1^\top (\mathbf{x} - \boldsymbol{\mu})$, is its *first principal component*.

The second principal direction maximises the variance among unit directions orthogonal to the first. Repeating the argument with a second multiplier for the constraint $\mathbf{u}^\top \mathbf{u}_1 = 0$ gives the eigenvector with the second-largest eigenvalue, and so on: the principal directions are the eigenvectors of $\boldsymbol{\Sigma}$, in order of decreasing eigenvalue.

Worked example 6.12: Checking that the first principal direction wins

For the covariance of worked example 6.9, compute the spread $\mathbf{u}^\top \boldsymbol{\Sigma} \mathbf{u} = 2u_1^2 + 6u_1u_2 + 10u_2^2$ along unit directions at a few angles ϕ from the band-1 axis, $\mathbf{u} = (\cos\phi, \sin\phi)$:

ϕ	$\mathbf{u}$	$2u_1^2 + 6u_1u_2 + 10u_2^2$	spread
$0°$	$(1, 0)$	$2 + 0 + 0$	2.0
$45°$	$(0.7071, 0.7071)$	$1 + 3 + 5$	9.0
$71.57°$	$(0.3162, 0.9487)$	$0.2 + 1.8 + 9$	11.0
$90°$	$(0, 1)$	$0 + 0 + 10$	10.0
$135°$	$(-0.7071, 0.7071)$	$1 - 3 + 5$	3.0

The line for $71.57°$ is $\mathbf{u}_1 = (1, 3)/\sqrt{10}$ (its angle is $\arctan 3$): there $2 \times 0.1 = 0.2$, $6 \times 0.3 = 1.8$ and $10 \times 0.9 = 9$, using $u_1^2 = 0.1$, $u_1u_2 = 0.3$, $u_2^2 = 0.9$. No direction beats 11, the largest eigenvalue, and the smallest possible spread is the other eigenvalue, 1, at $\mathbf{u}_2$ (angle $-18.43°$).

6.7.2 Explained variance

Because $\mathbf{U}$ only rotates, the total variance is the same in the rotated coordinates as in the bands: it is the trace, $\operatorname{tr} \boldsymbol{\Sigma} = \sum_i \Sigma_{ii} = \sum_k \lambda_k$. (For the example, $2 + 10 = 12 = 11 + 1$.) The share carried by component k is its *explained variance ratio*

$$\rho_k = \frac{\lambda_k}{\lambda_1 + \lambda_2 + \cdots + \lambda_C}. \tag{6.21}$$

In the example $\rho_1 = 11/12 = 0.917$ and $\rho_2 = 1/12 = 0.083$: one direction carries 92% of the spread. Real hyperspectral scenes are far more extreme, because neighbouring bands of a 165-band spectrum are nearly copies of each other; a few components carry almost all the variance and the remaining hundred-odd carry almost none. Keep that sentence in mind: it is the whole of Chapter 7's numerical problem.

Worked example 6.13: Turning the four pixels onto their principal directions

The principal components of each centred pixel are its dot products with $\mathbf{u}_1$ and $\mathbf{u}_2$. Working with $\sqrt{10}\,\mathbf{u}_1 = (1, 3)$ and $\sqrt{10}\,\mathbf{u}_2 = (3, -1)$ keeps integers; divide by $\sqrt{10}$ at the end.

pixel	$\mathbf{d}$	$(1, 3)^\top \mathbf{d}$	$(3, -1)^\top \mathbf{d}$
1	$(-2, -2)$	$-2 - 6 = -8$	$-6 + 2 = -4$
2	$(0, -3)$	$0 - 9 = -9$	$0 + 3 = 3$
3	$(1, 4)$	$1 + 12 = 13$	$3 - 4 = -1$
4	$(1, 1)$	$1 + 3 = 4$	$3 - 1 = 2$

Variance of the first component (the mean is $(-8 - 9 + 13 + 4)/\sqrt{10} = 0$):

$$\frac{1}{3} \cdot \frac{(-8)^2 + (-9)^2 + 13^2 + 4^2}{10} = \frac{1}{3} \cdot \frac{64 + 81 + 169 + 16}{10} = \frac{1}{3} \cdot \frac{330}{10} = 11.$$

Variance of the second: $\frac{1}{3} \cdot \frac{16+9+1+4}{10} = \frac{1}{3} \cdot \frac{30}{10} = 1$. Covariance between them: $\frac{1}{3} \cdot \frac{(-8)(-4)+(-9)(3)+(13)(-1)+(4)(2)}{10} = \frac{1}{3} \cdot \frac{32-27-13+8}{10} = 0$. In the rotated coordinates the covariance is $\operatorname{diag}(11, 1) = \boldsymbol{\Lambda}$: the components are uncorrelated, with variances equal to the eigenvalues. That is Equation (6.13) with $\mathbf{A} = \mathbf{U}^\top$: $\mathbf{U}^\top \boldsymbol{\Sigma} \mathbf{U} = \mathbf{U}^\top \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^\top \mathbf{U} = \boldsymbol{\Lambda}$.

6.8 Whitening

After PCA the cloud's axes line up with the coordinate axes, but it is still stretched: spread 11 one way, 1 the other. *Whitening* also divides each rotated coordinate by its own standard deviation, so the cloud becomes round with spread 1 in every direction. The name comes from signal processing: "white" noise has equal power at every frequency, and a whitened cloud has equal variance in every direction.

Definition 6.17 (Whitening matrix)**.** With $\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ and all $\lambda_k > 0$, let

$$\mathbf{\Lambda}^{-1/2} = \operatorname{diag}(1/\sqrt{\lambda_1}, \ldots, 1/\sqrt{\lambda_C}).$$

The *whitening matrix* is

$$\mathbf{W} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top, \qquad \mathbf{z} = \mathbf{W}(\mathbf{x} - \boldsymbol{\mu}). \tag{6.22}$$

Row k of $\mathbf{W}$ is $\mathbf{u}_k^\top/\sqrt{\lambda_k}$: "take the coordinate along $\mathbf{u}_k$, then divide by that direction's standard deviation".

Why the square root? The eigenvalue λ_k is a variance, a squared spread. The coordinate $\mathbf{u}_k^\top \mathbf{d}$ is in the data's own units, so it must be divided by a spread in the same units: $\sqrt{\lambda_k}$.

Claim. If $\mathbf{z} = \mathbf{W}(\mathbf{x} - \boldsymbol{\mu})$ with $\mathbf{W}$ as in Equation (6.22), then $\mathbf{\Sigma}_z = \mathbf{I}$: the whitened pixels have identity covariance.

Proof. By Equation (6.13) with $\mathbf{A} = \mathbf{W}$ (subtracting $\boldsymbol{\mu}$ does not change a covariance):

$$\begin{aligned}
\mathbf{\Sigma}_z &= \mathbf{W}\mathbf{\Sigma}\mathbf{W}^\top \\
&= \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top\, \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top\, (\mathbf{\Lambda}^{-1/2}\mathbf{U}^\top)^\top && \text{substitute } \mathbf{W} \text{ and the spectral theorem} \\
&= \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top\mathbf{U}\mathbf{\Lambda}^{-1/2} && \text{Equation (6.8); a diagonal matrix is its own transpose} \\
&= \mathbf{\Lambda}^{-1/2}\,\mathbf{I}\,\mathbf{\Lambda}\,\mathbf{I}\,\mathbf{\Lambda}^{-1/2} && \mathbf{U}^\top\mathbf{U} = \mathbf{I} \\
&= \mathbf{\Lambda}^{-1/2}\mathbf{\Lambda}\mathbf{\Lambda}^{-1/2} && \text{the identity changes nothing} \\
&= \mathbf{I} && \text{diagonal entries: } \tfrac{1}{\sqrt{\lambda_k}}\,\lambda_k\,\tfrac{1}{\sqrt{\lambda_k}} = 1.
\end{aligned}$$

□

Worked example 6.14: Whitening the four pixels

For the running example,

$$\mathbf{W} = \begin{pmatrix} 1/\sqrt{11} & 0 \\ 0 & 1 \end{pmatrix} \frac{1}{\sqrt{10}} \begin{pmatrix} 1 & 3 \\ 3 & -1 \end{pmatrix} = \begin{pmatrix} 1/\sqrt{110} & 3/\sqrt{110} \\ 3/\sqrt{10} & -1/\sqrt{10} \end{pmatrix} = \begin{pmatrix} 0.0953 & 0.2860 \\ 0.9487 & -0.3162 \end{pmatrix}.$$

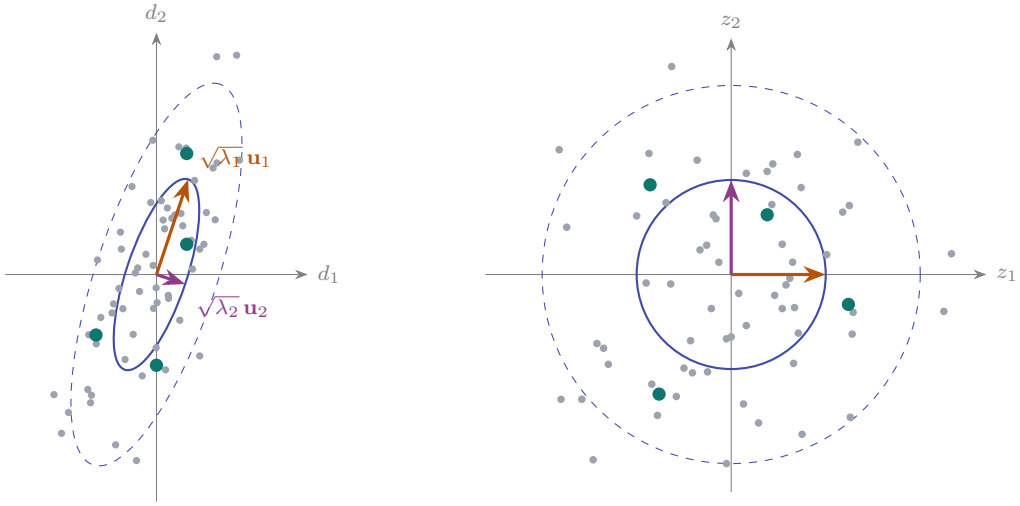
The whitened pixels are the principal components of worked example 6.13 divided by $\sqrt{\lambda_k}$: the first by $\sqrt{11} = 3.317$, the second by $\sqrt{1} = 1$. Pixel 1, for example, had components $-8/\sqrt{10} = -2.530$ and $-4/\sqrt{10} = -1.265$, so

$$\mathbf{z}_1 = (-2.530/3.317,\ -1.265/1) = (-0.763,\ -1.265).$$

All four:

pixel	$z_1 = (1,3)^\top\mathbf{d}/\sqrt{110}$	$z_2 = (3,-1)^\top\mathbf{d}/\sqrt{10}$
1	$-8/10.488 = -0.763$	$-4/3.162 = -1.265$
2	$-9/10.488 = -0.858$	$3/3.162 = 0.949$
3	$13/10.488 = 1.240$	$-1/3.162 = -0.316$
4	$4/10.488 = 0.381$	$2/3.162 = 0.632$

Variance of z_1: $\frac{1}{3}(64+81+169+16)/110 = \frac{1}{3}\cdot 330/110 = \frac{1}{3}\cdot 3 = 1$. Variance of z_2: $\frac{1}{3}(16+9+1+4)/10 = \frac{1}{3}\cdot 3 = 1$. Their covariance is $\frac{1}{3}\cdot\frac{0}{\sqrt{1100}} = 0$ from the same products as before. The whitened covariance is $\mathbf{I}$, as the proof said it must be.



(a) centred pixels

(b) whitened, $\mathbf{z} = \mathbf{W}\mathbf{d}$

Figure 6.2: Before and after whitening. (a) Sixty points drawn from a Gaussian with the covariance $\left(\begin{smallmatrix} 2 & 3 \\ 3 & 10 \end{smallmatrix}\right)$ of worked example 6.9 (grey; an illustrative sample, seed 0, `tools/compute_linalg.py`) and the four worked pixels (teal). The ellipses are the points at whitened distance 1 and 2; the arrows are the eigenvectors scaled by the square roots of their eigenvalues, 3.32 and 1. (b) The same points after $\mathbf{z} = \mathbf{W}\mathbf{d}$: the ellipses have become circles, and the eigenvectors the coordinate axes.

Figure 6.2 shows the whole chapter in one picture. On the left, the cloud is a tilted ellipse: long along $\mathbf{u}_1$, short along $\mathbf{u}_2$. On the right, after rotating by $\mathbf{U}^\top$ and dividing by $\sqrt{\lambda_k}$, it is round. Two points the same Euclidean distance from the centre on the left can be very different distances on the right: a step of 1 unit along $\mathbf{u}_1$ becomes $1/3.32 = 0.30$ whitened units, while the same step along $\mathbf{u}_2$ stays 1. Distance in the whitened picture is the *Mahalanobis distance*, and Chapter 7 takes it from here.

Whitening is rotate onto the principal directions, then divide each by its standard deviation; afterwards the background cloud is round and every direction counts equally.

6.8.1 Two practical details you will meet in the code

Whitening with fewer directions. If you keep only the top d eigenvectors (the first d rows of $\mathbf{W}$), $\mathbf{z}$ has d entries and is still white: its covariance is the $d \times d$ identity, by the same proof with $\mathbf{U}$ replaced by its first d columns (Exercise 6.5). The residual score of Chapter 31 and the PCA-RX baseline of Chapter 8 do exactly this with $d = 30$.

A floor under the eigenvalues. Dividing by $\sqrt{\lambda_k}$ is impossible when $\lambda_k = 0$ and dangerous when it is tiny. Code that whitens therefore raises tiny eigenvalues to a floor before the square root: `np.maximum(evals[order],1e-12)` in `app/utils/anomaly_detection/residual_mahalanobis.py` line 46, and `np.maximum(noise_eigvals,1e-10)` in `app/detectors/mnf_compression_detector.py` line 321. Why tiny eigenvalues are dangerous, and not only zero ones, is the subject of Section 7.9.2.

Two whitening matrices. The matrix $\mathbf{W} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$ rotates and then scales. Multiplying it on the left by $\mathbf{U}$ gives $\mathbf{U}\mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$, which rotates, scales, and rotates back. It also whitens (the extra $\mathbf{U}$ is a rotation, and $\mathbf{U}\mathbf{I}\mathbf{U}^\top = \mathbf{I}$), and it is the symmetric *inverse square root* $\mathbf{\Sigma}^{-1/2}$, because multiplying it by itself gives $\mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^\top = \mathbf{\Sigma}^{-1}$. The MNF code builds this second form from the noise covariance (`whiten=noise_eigvecs@np.diag(1.0/np.sqrt(noise_eigvals))@noise_eigvecs.T`, line 323); Chapter 8 explains why it does not matter which form you use there.

6.9 Common confusions

Is the dot product the same as multiplying two vectors?
No. Multiplying entry by entry gives another vector. The dot product adds those products into a single number. The outer product $\mathbf{d}\mathbf{d}^\top$ is a third thing again: a whole matrix of products.

Why does projection divide by $\mathbf{u}^\top\mathbf{u}$?
To make the answer independent of the length of $\mathbf{u}$. Doubling $\mathbf{u}$ doubles $\mathbf{u}^\top\mathbf{x}$ and quadruples $\mathbf{u}^\top\mathbf{u}$, so $t^\star$ halves and $t^\star\mathbf{u}$ is unchanged. With a unit vector the division is by 1 and disappears.

Why $N-1$ in the covariance?
Because the deviations are measured from the sample's own mean and add to zero, so only $N-1$ of them are free. The code uses $N-1$ (`np.cov`). The difference matters only for small samples, such as the local windows of Chapter 8 and the tiny examples here.

Is an eigenvector unique?
Only up to sign and length. If $\mathbf{u}$ is an eigenvector, so is $-\mathbf{u}$ and $5\mathbf{u}$. We always take unit length; the sign is arbitrary, and `np.linalg.eigh` may return either. Squared coordinates, and therefore every score in this book, do not care about the sign.

Does PCA find the most interesting directions?
It finds the directions of greatest variance. Whether those are interesting depends on what produces the variance. If the noisiest bands have the largest variance, PCA keeps the noise. Chapter 8 builds MNF to fix exactly that.

Whitening makes everything the same size, so does it destroy information?
No. It is an invertible change of coordinates (when all eigenvalues are kept and positive): you can undo it with $\mathbf{U}\mathbf{\Lambda}^{1/2}$. What changes is which differences look large. That change is the whole point.

6.10 Check yourself

Exercise 6.1. Compute the angle in degrees between $(1, 2, 2)$ and $(2, 1, 2)$. Then project $(1, 2, 2)$ onto $(2, 1, 2)$ and check that the residual is orthogonal to $(2, 1, 2)$.

Exercise 6.2. Three pixels in two bands: $(1, 2)$, $(3, 2)$, $(2, 5)$. Compute the mean, the deviations, the outer products and the covariance matrix (divide by $N-1$). What is the correlation of the two bands?

Exercise 6.3. Find the eigenvalues and unit eigenvectors of $\begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}$ by hand, and the explained variance ratio of the first component. Rebuild the matrix from $\sum_k \lambda_k \mathbf{u}_k \mathbf{u}_k^\top$.

Exercise 6.4. Using the matrix of Exercise 6.3, write down the whitening matrix $\mathbf{W} = \mathbf{\Lambda}^{-1/2}\mathbf{U}^\top$ and verify by multiplication that $\mathbf{W}\mathbf{\Sigma}\mathbf{W}^\top = \mathbf{I}$.

Exercise 6.5. Let $\mathbf{U}_d$ hold only the first d eigenvectors of a $C \times C$ covariance $\mathbf{\Sigma}$ as columns, and $\mathbf{\Lambda}_d$ the first d eigenvalues. Show that $\mathbf{z} = \mathbf{\Lambda}_d^{-1/2}\mathbf{U}_d^\top(\mathbf{x} - \boldsymbol{\mu})$ has covariance $\mathbf{I}_d$. Which fact about the columns of $\mathbf{U}$ does the proof use?

Exercise 6.6. Show that $(\mathbf{ABC})^\top = \mathbf{C}^\top\mathbf{B}^\top\mathbf{A}^\top$ using Equation (6.8) twice, and explain in one sentence why $\mathbf{A\Sigma A}^\top$ is symmetric whenever $\mathbf{\Sigma}$ is.

Next: Chapter 7, Anomaly as surprise: the Gaussian, Mahalanobis distance and the RX detector.

Chapter 7

Anomaly as surprise: the Gaussian, Mahalanobis distance and the RX detector

The one thing this chapter teaches

Every detector in this book answers one question, "how badly does this pixel fit the pattern of normal?", and the RX detector answers it by modelling normal as one Gaussian cloud and scoring a pixel by its Mahalanobis distance from the centre: Euclidean distance after whitening. On 165 correlated bands the covariance that whitening divides by is nearly singular, the distances explode, and that is why plain RX was removed from the product on 2026-05-11.

7.1 Anomaly as surprise

Chapter 1 turned "is this a gas leak?" into "is this pixel unlike its surroundings?". This chapter makes that computable. Every detector in this repository, statistical or neural, follows the same three steps:

1. build a model of what *normal* looks like;
2. measure how badly each pixel *fits* that model;
3. rank the pixels; the worst-fitting are the candidates.

The two families of detectors differ only in step 1. The *statistical* family describes normal with statistics computed from the scene itself (its mean spectrum, and how its bands vary together) and needs no training, no checkpoint and no labels. The *reconstruction* family (Chapter 9) trains a network to redraw ordinary imagery and treats a bad redrawing as a sign of something unusual. This chapter and the next build the statistical family, and with it every piece of mathematics the residual score of Chapter 31 needs.

Every detector outputs a *score map*: one number per pixel, the height and width of the scene, higher meaning more anomalous. It does not output a yes/no decision. A threshold turns scores into a mask later, chosen by a person in a separate Action (Chapter 4), and invalid pixels are excluded from scoring and from every statistic computed over the map.

7.1.1 Start in one dimension

A thermal scene from Landsat 9 has one number per pixel, a temperature. The shipped thermal statistics are a mean of 24.58 °C [M: `app/constants/thermal_pixel_consts.json`] and a standard deviation of 13.57 °C [M: `app/constants/thermal_pixel_consts.json`]. How unusual is a pixel at 71 °C? Count how many standard deviations it sits above the mean:

$$\begin{aligned} z &= \frac{71 - 24.58}{13.57} \\ &= \frac{46.42}{13.57} \\ &= 3.42. \end{aligned}$$

Three and a half standard deviations above the mean is unusual. That is the whole idea, and the rest of the chapter generalises it to many bands. To generalise it properly you need to know *how* unusual 3.42 is, which needs a model of how the values are spread: the Gaussian.

Analogy

A doctor reading a blood panel does not flag a test because its number is large; she flags it because it is large *for that test*, measured against the normal range. When several tests move together for an ordinary reason (dehydration raises many concentrations at once), she reads them as one finding, not many. One value far outside its own normal range while the rest are ordinary is the alarm. The RX detector reads a spectrum the same way: it measures each pixel against the normal range of the scene, it discounts movements that the bands always make together, and it raises the alarm for a movement the scene does not normally make.

7.2 The one-dimensional Gaussian and the z-score

Definition 7.1 (Gaussian distribution)**.** A quantity x has a *Gaussian* (or *normal*) distribution with mean μ and standard deviation $\sigma > 0$, written $x \sim \mathcal{N}(\mu, \sigma^2)$, if its probability density is

$$p(x) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2} \right). \tag{7.1}$$

A *probability density* is a curve whose area over an interval is the probability of landing in that interval; the total area under it is 1. The Gaussian is the familiar bell: highest at μ, symmetric, falling away fast. Two pieces of Equation (7.1) deserve a sentence each. The factor $1/(\sqrt{2\pi}\sigma)$ is only there to make the total area 1; it does not depend on x. All the shape is in the exponent, $-(x-\mu)^2/(2\sigma^2)$, which is minus one half of the squared number of standard deviations from the mean.

Definition 7.2 (z-score)**.** The *z-score* of a value is its distance from the mean in units of standard deviation, $z = (x - \mu)/\sigma$. If $x \sim \mathcal{N}(\mu, \sigma^2)$ then $z \sim \mathcal{N}(0, 1)$, the *standard normal*, with density $\varphi(z) = \frac{1}{\sqrt{2\pi}} e^{-z^2/2}$.

Substituting z into Equation (7.1) shows the exponent is $-z^2/2$: the density of a value depends on it only through z^2. Values with the same z^2 (the same distance from the mean, on either side) are equally likely, and a larger z^2 means an exponentially less likely value. So z^2 is a natural surprise score in one dimension. Remember that sentence: the multi-band score is the same statement with z^2 replaced by its multi-band version.

7.2.1 Tail probabilities and thresholds

The probability that a standard normal lands more than t from zero in either direction is the area in its two tails, $P(|z| > t)$. There is no closed form for it, but the values are tabulated (and computed by `scipy.stats.norm.sf`):

t	$P(\|z\| > t)$	roughly
1	0.3173	one pixel in 3
2	0.0455	one in 22
3	0.0027	one in 370
3.42	0.000626	one in 1,600

(`tools/compute_rx.py` computes every number in this chapter.) If the thermal scene really were Gaussian, a pixel at $z = 3.42$ or beyond would occur once in about 1,600 pixels; a Landsat scene has millions of pixels, so you would still expect thousands of them. That is the first lesson of detection: *unusual* is relative to how many pixels you look at.

Turn the table around and you have a *threshold*. Decide what fraction of normal pixels you are willing to flag by mistake, the *false-alarm rate* α, and choose t so that $P(|z| > t) = \alpha$. For $\alpha = 1\%$, $t = 2.576$; for $\alpha = 0.1\%$, $t = 3.291$. In terms of the score z^2 the thresholds are $2.576^2 = 6.635$ and $3.291^2 = 10.83$.

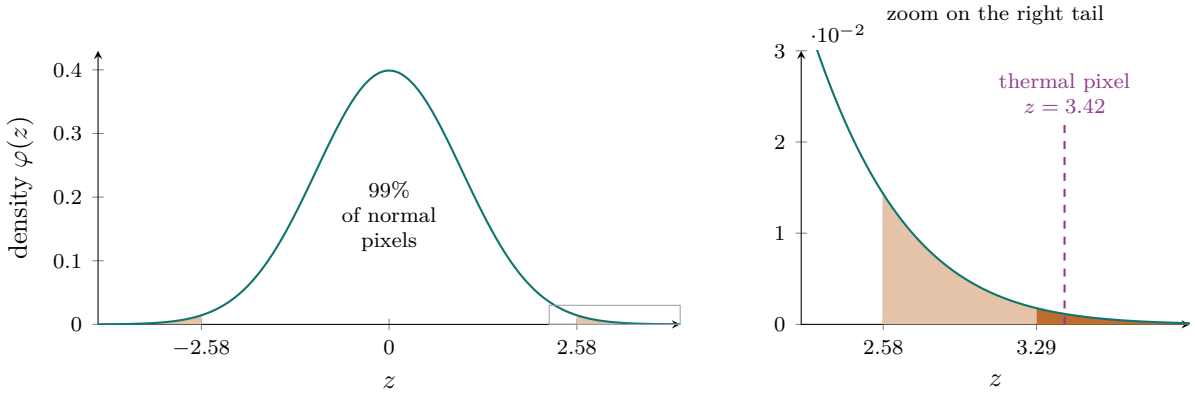


Figure 7.1: The standard normal density. The light and dark tails together hold 1% of the area (beyond ± 2.576); the dark tails alone hold 0.1% (beyond ± 3.291). They are thin, so the right panel magnifies the grey box on the right tail. The thermal pixel of Section 7.1 sits at $z = 3.42$, beyond both thresholds.

7.3 Why many bands need more than many z-scores

The obvious way to score a 165-band pixel is to compute a z-score per band and add the squares. That fails because **bands are not independent**. Vegetation is dark in red and bright in the near infrared, and across a scene the two bands move together: more vegetation, darker red and brighter infrared. A pixel slightly bright in both is ordinary. A pixel bright in red *and* dark in the infrared is bizarre, even if neither value alone is extreme. Per-band z-scores look at one band at a time and cannot see the relationship.

Worked example 7.1: Per-band z-scores miss the pattern

Use the two-band covariance of worked example 6.9, $\mathbf{\Sigma} = \left(\begin{smallmatrix} 2 & 3 \\ 3 & 10 \end{smallmatrix}\right)$ (spreads $\sqrt{2}$ and $\sqrt{10}$, positive covariance). Two test pixels sit these distances from the mean:

$$\mathbf{d}_A = (2, 6) \qquad \text{(both bands up, the way the scene moves)},$$
$$\mathbf{d}_B = (2, -3) \qquad \text{(band 1 up, band 2 down)}.$$

Per-band squared z-scores, $d_1^2/2 + d_2^2/10$:

$$A: \quad \frac{2^2}{2} + \frac{6^2}{10} = \frac{4}{2} + \frac{36}{10} = 2 + 3.6 = 5.6$$
$$B: \quad \frac{2^2}{2} + \frac{(-3)^2}{10} = \frac{4}{2} + \frac{9}{10} = 2 + 0.9 = 2.9.$$

By this score A is nearly twice as unusual as B. Now use the covariance properly. With $\mathbf{\Sigma}^{-1} = \frac{1}{11}\left(\begin{smallmatrix} 10 & -3 \\ -3 & 2 \end{smallmatrix}\right)$ (worked example 6.8):

$$\mathbf{\Sigma}^{-1}\mathbf{d}_A = \tfrac{1}{11}(10 \times 2 - 3 \times 6,\ -3 \times 2 + 2 \times 6) = \tfrac{1}{11}(20 - 18,\ -6 + 12) = \tfrac{1}{11}(2, 6)$$
$$\mathbf{d}_A^\top \mathbf{\Sigma}^{-1}\mathbf{d}_A = \tfrac{1}{11}(2 \times 2 + 6 \times 6) = \tfrac{1}{11}(4 + 36) = \tfrac{40}{11} = 3.64$$
$$\mathbf{\Sigma}^{-1}\mathbf{d}_B = \tfrac{1}{11}(10 \times 2 - 3 \times (-3),\ -3 \times 2 + 2 \times (-3)) = \tfrac{1}{11}(20 + 9,\ -6 - 6) = \tfrac{1}{11}(29, -12)$$
$$\mathbf{d}_B^\top \mathbf{\Sigma}^{-1}\mathbf{d}_B = \tfrac{1}{11}(2 \times 29 + (-3) \times (-12)) = \tfrac{1}{11}(58 + 36) = \tfrac{94}{11} = 8.55.$$

The ranking flips. A moves along the direction the scene always moves, and scores 3.64; B breaks the relationship between the bands, and scores 8.55. The quantity $\mathbf{d}^\top \mathbf{\Sigma}^{-1}\mathbf{d}$ is the Mahalanobis distance, and the rest of the chapter explains why it is the right score.

7.4 The multivariate Gaussian

To score many bands the way z^2 scores one, you need a probability model for a whole spectrum: a density over $\mathbb{R}^C$ that has a mean vector, a covariance matrix, and a bell shape. Build it from the one-dimensional case in three steps.

Step 1: independent standard normals. Let $\mathbf{z} = (z_1, \ldots, z_C)$ have entries that are independent standard normals. For independent quantities the joint density is the product of the individual densities, and products of exponentials add their exponents:

$$p(\mathbf{z}) = \prod_{k=1}^{C} \frac{1}{\sqrt{2\pi}} e^{-z_k^2/2} = \frac{1}{(2\pi)^{C/2}} \exp\Big(-\tfrac{1}{2}\left(z_1^2 + \cdots + z_C^2\right)\Big)$$
$$= \frac{1}{(2\pi)^{C/2}} \exp\Big(-\tfrac{1}{2}\left\|\mathbf{z}\right\|^2\Big). \tag{7.2}$$

The density depends on $\mathbf{z}$ only through its length: the contours of equal density are circles (spheres in more dimensions). Its mean is $\mathbf{0}$ and its covariance is $\mathbf{I}$ (each entry has variance 1, and independent entries have covariance 0). It is exactly the round, whitened cloud of Figure 6.2(b).

Step 2: stretch and turn it. Take a mean $\boldsymbol{\mu}$ and a covariance $\mathbf{\Sigma} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$ with all eigenvalues positive. Define

$$\mathbf{x} = \boldsymbol{\mu} + \mathbf{A}\mathbf{z}, \qquad \mathbf{A} = \mathbf{U}\mathbf{\Lambda}^{1/2}.$$

In words: stretch the round cloud by $\sqrt{\lambda_k}$ along axis k, rotate the axes onto the eigenvectors, and move the centre to $\boldsymbol{\mu}$. By Equation (6.13) the covariance of $\mathbf{x}$ is $\mathbf{A}\mathbf{I}\mathbf{A}^\top = \mathbf{U}\boldsymbol{\Lambda}^{1/2}\boldsymbol{\Lambda}^{1/2}\mathbf{U}^\top = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top = \boldsymbol{\Sigma}$, as wanted.

Step 3: change variables. What is the density of $\mathbf{x}$? First the one-dimensional version. If $x = \mu + \sigma z$ then $z = (x-\mu)/\sigma$, and an interval of width Δx in x corresponds to an interval of width $\Delta x/\sigma$ in z. The same probability is spread over an interval σ times wider, so the density is σ times lower: $p_x(x) = \frac{1}{\sigma}\,\varphi\Big(\frac{x-\mu}{\sigma}\Big)$, which is exactly Equation (7.1). In C dimensions the same argument uses volume instead of width: the map $\mathbf{z} \mapsto \mathbf{A}\mathbf{z}$ multiplies volumes by $|\det \mathbf{A}|$, so

$$p_x(\mathbf{x}) = \frac{1}{|\det \mathbf{A}|}\, p_z\Big(\mathbf{A}^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big)\,.$$

Now identify the pieces. The inverse of $\mathbf{A} = \mathbf{U}\boldsymbol{\Lambda}^{1/2}$ is $\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top$ (check: $\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top\mathbf{U}\boldsymbol{\Lambda}^{1/2} = \boldsymbol{\Lambda}^{-1/2}\boldsymbol{\Lambda}^{1/2} = \mathbf{I}$), and that is the whitening matrix $\mathbf{W}$ of Equation (6.22). The determinant of a product is the product of determinants, an orthogonal matrix has determinant ± 1, and a diagonal matrix's determinant is the product of its diagonal, so $|\det \mathbf{A}| = \sqrt{\lambda_1 \cdots \lambda_C} = \sqrt{\det \boldsymbol{\Sigma}}$. Substituting into Equation (7.2):

Definition 7.3 (Multivariate Gaussian)**.** A spectrum $\mathbf{x} \in \mathbb{R}^C$ has a multivariate Gaussian distribution with mean $\boldsymbol{\mu}$ and positive definite covariance $\boldsymbol{\Sigma}$, written $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$, if

$$\begin{aligned} p(\mathbf{x}) &= \frac{1}{(2\pi)^{C/2}\sqrt{\det \boldsymbol{\Sigma}}} \exp\Big(-\tfrac{1}{2}\,\|\mathbf{W}(\mathbf{x}-\boldsymbol{\mu})\|^2\Big) \\ &= \frac{1}{(2\pi)^{C/2}\sqrt{\det \boldsymbol{\Sigma}}} \exp\Big(-\tfrac{1}{2}\,(\mathbf{x}-\boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big)\,. \end{aligned} \tag{7.3}$$

The second form uses the identity $\|\mathbf{W}\mathbf{d}\|^2 = \mathbf{d}^\top\boldsymbol{\Sigma}^{-1}\mathbf{d}$, which the next section proves. With $C = 1$, $\boldsymbol{\Sigma} = \sigma^2$, and Equation (7.3) is Equation (7.1). As in one dimension, everything that depends on the pixel sits in the exponent, and the exponent is minus one half of a squared distance.

7.5 Mahalanobis distance

Definition 7.4 (Mahalanobis distance)**.** The (squared) *Mahalanobis distance* of a vector $\mathbf{x}$ from a distribution with mean $\boldsymbol{\mu}$ and covariance $\boldsymbol{\Sigma}$ is

$$D^2(\mathbf{x}) = (\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}). \tag{7.4}$$

It is distance from the mean measured in units of how much the data actually vary in that direction. Ten units along a direction in which the scene varies a lot are unremarkable; ten units along a direction in which it hardly varies at all are extraordinary. The next result makes that sentence exact.

7.5.1 Mahalanobis distance is Euclidean distance after whitening

Claim. With $\boldsymbol{\Sigma} = \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$ and $\mathbf{W} = \boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top$,

$$(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu}) = \|\mathbf{W}(\mathbf{x}-\boldsymbol{\mu})\|^2 = \sum_{k=1}^{C} \frac{\Big(\mathbf{u}_k^\top(\mathbf{x}-\boldsymbol{\mu})\Big)^2}{\lambda_k}. \tag{7.5}$$

Proof. Three steps.

(i) The inverse of the covariance. Guess $\boldsymbol{\Sigma}^{-1} = \mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top$, with $\boldsymbol{\Lambda}^{-1} = \mathrm{diag}(1/\lambda_1, \ldots, 1/\lambda_C)$, and check by multiplying:

$$\begin{aligned}
\boldsymbol{\Sigma}\,\mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top &= \mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top\mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top && \text{spectral theorem} \\
&= \mathbf{U}\boldsymbol{\Lambda}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top && \mathbf{U}^\top\mathbf{U} = \mathbf{I} \\
&= \mathbf{U}\mathbf{U}^\top = \mathbf{I} && \text{diagonal entries } \lambda_k/\lambda_k = 1;\ \mathbf{U} \text{ is square and orthogonal.}
\end{aligned}$$

So the inverse has the same eigenvectors and the reciprocal eigenvalues. Remember this: *a small eigenvalue of $\boldsymbol{\Sigma}$ is a large eigenvalue of $\boldsymbol{\Sigma}^{-1}$.*

(ii) $\mathbf{W}^\top\mathbf{W} = \boldsymbol{\Sigma}^{-1}$. Using Equation (6.8),

$$\mathbf{W}^\top\mathbf{W} = (\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top)^\top\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top = \mathbf{U}\boldsymbol{\Lambda}^{-1/2}\boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top = \mathbf{U}\boldsymbol{\Lambda}^{-1}\mathbf{U}^\top = \boldsymbol{\Sigma}^{-1}.$$

(iii) Put them together. Write $\mathbf{d} = \mathbf{x} - \boldsymbol{\mu}$. Then

$$\|\mathbf{W}\mathbf{d}\|^2 = (\mathbf{W}\mathbf{d})^\top(\mathbf{W}\mathbf{d}) = \mathbf{d}^\top\mathbf{W}^\top\mathbf{W}\mathbf{d} = \mathbf{d}^\top\boldsymbol{\Sigma}^{-1}\mathbf{d}.$$

For the sum form, entry k of $\mathbf{W}\mathbf{d}$ is row k of $\mathbf{W}$ times $\mathbf{d}$, which is $\mathbf{u}_k^\top\mathbf{d}/\sqrt{\lambda_k}$; squaring and adding gives the right-hand side of Equation (7.5). □

Mahalanobis distance = turn the pixel onto the principal directions, divide each coordinate by its standard deviation, and take the ordinary squared length.

The sum form is the one to hold in your head. Each principal direction contributes its squared coordinate divided by its eigenvalue. A direction with a large eigenvalue (the scene varies a lot that way) contributes little; a direction with a small eigenvalue contributes a lot. The contours of equal Mahalanobis distance are the ellipses of Figure 6.2(a), long along the large eigenvectors and short along the small ones.

Worked example 7.2: Two ways to the same Mahalanobis distance

Take pixel B of worked example 7.1, $\mathbf{d}_B = (2, -3)$, with $\mathbf{u}_1 = (1, 3)/\sqrt{10}$, $\lambda_1 = 11$, $\mathbf{u}_2 = (3, -1)/\sqrt{10}$, $\lambda_2 = 1$ (worked example 6.10). Coordinates:

$$\begin{aligned}
\mathbf{u}_1^\top\mathbf{d}_B &= \frac{1\times 2 + 3\times(-3)}{\sqrt{10}} = \frac{2-9}{\sqrt{10}} = \frac{-7}{\sqrt{10}} \\
\mathbf{u}_2^\top\mathbf{d}_B &= \frac{3\times 2 + (-1)\times(-3)}{\sqrt{10}} = \frac{6+3}{\sqrt{10}} = \frac{9}{\sqrt{10}}.
\end{aligned}$$

Squared, divided by the eigenvalues, added:

$$D^2 = \frac{49/10}{11} + \frac{81/10}{1} = \frac{49}{110} + \frac{81}{10} = 0.445 + 8.100 = 8.545 = \frac{94}{11}.$$

The same 8.55 as worked example 7.1, and now you can see where it comes from: 8.1 of it is the pixel's step of $9/\sqrt{10} = 2.85$ along $\mathbf{u}_2$, the direction in which the scene's spread is only 1.

7.6 The RX detector

The *RX detector*, named after Reed and Xiaoli Yu, who derived it in 1990 [36], scores every pixel by its Mahalanobis distance from the scene's background statistics:

$$\mathrm{RX}(\mathbf{x}) = (\mathbf{x} - \hat{\boldsymbol{\mu}})^\top\hat{\boldsymbol{\Sigma}}^{-1}(\mathbf{x} - \hat{\boldsymbol{\mu}}), \tag{7.6}$$

where the hats mean "estimated from the pixels": $\hat{\boldsymbol{\mu}}$ is the scene's mean spectrum and $\hat{\boldsymbol{\Sigma}}$ its covariance matrix (Equations (6.10) and (6.11)). *Global RX* estimates them once from all valid pixels of the scene; *local RX* (Chapter 8) estimates them separately for each pixel from a ring of its neighbours.

7.6.1 Where the formula comes from: a likelihood ratio

Why this score and not some other? Because it is what a likelihood-ratio test gives. Frame the decision at one pixel as two hypotheses:

- H_0 (background): $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$;
- H_1 (target): $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu} + \mathbf{s}, \boldsymbol{\Sigma})$ for some unknown signature shift $\mathbf{s}$.

A *likelihood ratio* compares how well each hypothesis explains the observed pixel: $p(\mathbf{x} \mid H_1)/p(\mathbf{x} \mid H_0)$. We do not know $\mathbf{s}$, so the *generalised* likelihood ratio uses the $\mathbf{s}$ that explains the pixel best. Under H_1 the exponent is $-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu} - \mathbf{s})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu} - \mathbf{s})$, which is at most 0 and equals 0 when $\mathbf{s} = \mathbf{x} - \boldsymbol{\mu}$. So the best H_1 density is the peak value $1/((2\pi)^{C/2}\sqrt{\det \boldsymbol{\Sigma}})$, and taking the logarithm of the ratio:

$$
\begin{aligned}
\log \frac{\max_{\mathbf{s}} p(\mathbf{x} \mid H_1)}{p(\mathbf{x} \mid H_0)} &= \log \frac{\frac{1}{(2\pi)^{C/2}\sqrt{\det \boldsymbol{\Sigma}}}\, e^{0}}{\frac{1}{(2\pi)^{C/2}\sqrt{\det \boldsymbol{\Sigma}}}\, e^{-\frac{1}{2}D^2(\mathbf{x})}} && \text{Equation (7.3) for both} \\
&= \log e^{\frac{1}{2}D^2(\mathbf{x})} && \text{the normalising factors cancel} \\
&= \tfrac{1}{2}\, D^2(\mathbf{x}).
\end{aligned}
$$

The log likelihood ratio is half the Mahalanobis distance, and a larger ratio means stronger evidence for a target. Ranking by the ratio and ranking by D^2 are the same ranking.

Source check: the original RX statistic

Reed and Yu derived their test with the background mean and covariance themselves estimated from a finite set of N background pixels [36], which adds terms involving N to the statistic; it is still an increasing function of the Mahalanobis distance computed with the estimates, so it ranks pixels the same way, and with the large backgrounds used here it reduces to Equation (7.6). The code computes Equation (7.6) directly: `GlobalRXDetector.detect` (`app/detectors/global_rx_detector.py`, lines 227–229) calls `spectral.rx` on the valid pixels, which estimates $\hat{\boldsymbol{\mu}}$ and $\hat{\boldsymbol{\Sigma}}$ from those same pixels (the `spectral` library's `calc_stats`, divisor $N - 1$) and forms $\hat{\boldsymbol{\Sigma}}^{-1}$ with `np.linalg.inv`.

Notice what the global version implies: the statistics are estimated from *all* valid pixels, including any anomalies. A handful of anomalous pixels among millions barely moves the mean and covariance, so for global RX this contamination is mild. For local RX, where the background is a few thousand pixels around the target, it is not mild at all, and Chapter 8 builds a guard window against it.

7.7 How large is large? The chi-square distribution

If the background really is Gaussian, the RX score of a background pixel has a known distribution, and that tells you what a "large" score is.

By Equation (7.5), $D^2 = \|\mathbf{z}\|^2 = z_1^2 + \cdots + z_C^2$ with $\mathbf{z} = \mathbf{W}(\mathbf{x} - \boldsymbol{\mu})$. If $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ then, running the construction of Section 7.4 backwards, $\mathbf{z}$ has independent standard normal entries.

So under the Gaussian background,

$$D^2 = z_1^2 + z_2^2 + \cdots + z_C^2, \qquad z_k \text{ independent } \mathcal{N}(0,1). \tag{7.7}$$

Definition 7.5 (Chi-square distribution)**.** The distribution of a sum of C squared independent standard normals is the *chi-square distribution with C degrees of freedom*, written χ^2_C. Its density, for $t > 0$, is

$$f_C(t) = \frac{t^{C/2-1}\, e^{-t/2}}{2^{C/2}\,\Gamma(C/2)}, \tag{7.8}$$

where Γ is the gamma function ($\Gamma(n) = (n-1)!$ for whole numbers n, $\Gamma(1/2) = \sqrt{\pi}$, $\Gamma(3/2) = \sqrt{\pi}/2$).

You will not need to derive Equation (7.8) in general, but three facts about it you should be able to derive yourself.

The mean is C. Each z_k^2 has mean $\mathbb{E}[z_k^2] = \mathrm{Var}(z_k) + (\mathbb{E}z_k)^2 = 1 + 0 = 1$, and means add, so $\mathbb{E}[D^2] = C$. A background pixel in a 10-dimensional space has an average RX score of 10, not 1. Scores grow with the number of bands even when nothing is anomalous, which is why you can never compare RX scores computed in different dimensions directly.

The spread is $\sqrt{2C}$. For a standard normal $\mathbb{E}[z^4] = 3$ (a standard fact), so $\mathrm{Var}(z^2) = \mathbb{E}[z^4] - (\mathbb{E}[z^2])^2 = 3 - 1 = 2$. Variances of independent quantities add, so $\mathrm{Var}(D^2) = 2C$ and the standard deviation is $\sqrt{2C}$: 4.47 for $C = 10$, 18.2 for $C = 165$.

Two cases by hand. For $C = 1$, $D^2 = z^2$ and $P(z^2 > t) = P(|z| > \sqrt{t})$: the one-dimensional tail table of Section 7.2, with t squared. For $C = 2$ the tail has a closed form. The density of (z_1, z_2) is $\frac{1}{2\pi}e^{-r^2/2}$ with $r^2 = z_1^2 + z_2^2$; integrate over the region outside the circle of radius $\sqrt{t}$ in polar coordinates (the ring at radius r has circumference $2\pi r$):

$$P(D^2 > t) = \int_{\sqrt{t}}^{\infty} \frac{1}{2\pi} e^{-r^2/2}\, 2\pi r\, dr = \Big[-e^{-r^2/2} \Big]_{\sqrt{t}}^{\infty} = 0 - (-e^{-t/2}) = e^{-t/2}.$$

So for two bands the threshold with false-alarm rate α solves $e^{-t/2} = \alpha$: $t = 2\ln(1/\alpha)$. For $\alpha = 1\%$, $t = 2\ln 100 = 2 \times 4.605 = 9.21$.

7.7.1 Thresholds by false-alarm rate

The χ^2_C tail gives a threshold for any false-alarm rate. The values below come from `scipy.stats.chi2.isf` [C]:

C	mean	standard deviation	threshold at $\alpha = 1\%$	at $\alpha = 0.1\%$
1	1	1.41	6.63	10.83
2	2	2.00	9.21	13.82
3	3	2.45	11.34	16.27
10	10	4.47	23.21	29.59
30	30	7.75	50.89	59.70
165	165	18.17	210.18	226.88

The $C = 1$ row matches the squared normal thresholds of Section 7.2 (6.635 and 10.83), and the $C = 2$ row matches $2\ln 100$ and $2\ln 1000$. The $C = 10$ row is the one that would apply to MNF-RX with its 10 components (Chapter 8), and $C = 30$ to the 30-component scores of Chapters 8 and 31.

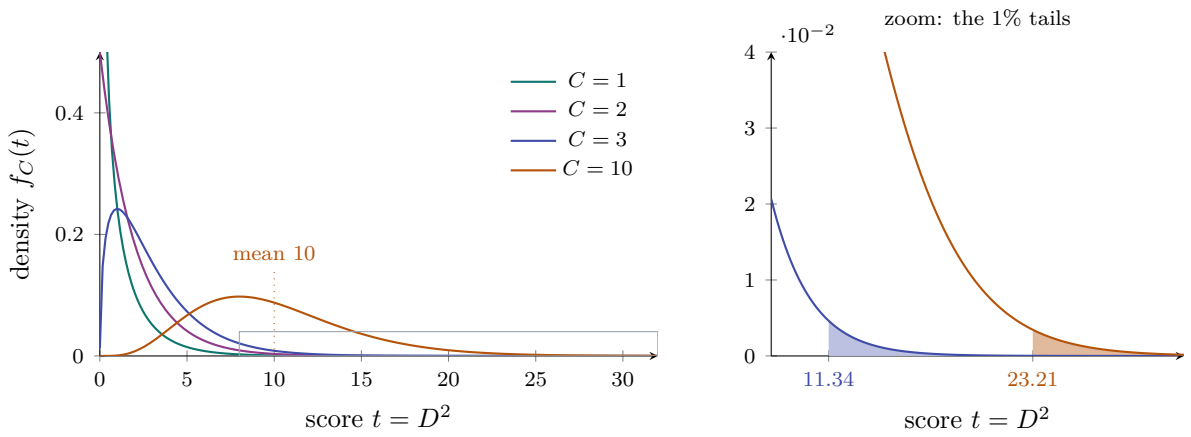


Figure 7.2: Chi-square densities for $C = 1, 2, 3, 10$ degrees of freedom. The right panel magnifies the grey box: its shaded tails hold 1% of the area for $C = 3$ (beyond 11.34) and for $C = 10$ (beyond 23.21). As C grows the whole distribution moves right (mean C) and spreads (standard deviation $\sqrt{2C}$), so a score of 15 is alarming in 3 dimensions and ordinary in 10.

Under a Gaussian background the RX score of a normal pixel is χ^2_C: average C, and a threshold for any false-alarm rate can be read from a table.

Why the product does not use these thresholds. Real scenes are not one Gaussian. They are mixtures of surfaces, with heavy tails from edges, glint and stripes, so the fraction of real pixels above the χ^2 threshold is usually far more than α. Every score in the product is therefore thresholded by *percentile* of its own map (the human threshold step of Chapter 4), and drawn with a square-root stretch because squared distances have a long right tail. The chi-square result is still worth knowing: it tells you what scale to expect (scores near C are ordinary), it explains why scores computed in different numbers of dimensions are not comparable, and it is the reason the residual score's docstring calls its output "chi-squared-like".

7.7.2 A fact that holds for any data

One property of RX scores does not need the Gaussian at all. When the mean and covariance are estimated from the same N pixels that are scored (as `spectral.rx` does), the scores always average to exactly $C(N-1)/N$. The proof uses the *trace*, the sum of a square matrix's diagonal, and two of its properties: it is linear, and for vectors $\mathbf{a}, \mathbf{b}$, $\mathbf{a}^\top\mathbf{b} = \mathrm{tr}(\mathbf{b}\mathbf{a}^\top)$ (both are $\sum_i a_i b_i$; the second is the sum of the diagonal of the outer product). Then

$$
\begin{aligned}
\sum_{n=1}^{N} \mathbf{d}_n^\top \hat{\boldsymbol{\Sigma}}^{-1} \mathbf{d}_n &= \sum_n \mathrm{tr}\left(\hat{\boldsymbol{\Sigma}}^{-1} \mathbf{d}_n \mathbf{d}_n^\top\right) && \mathbf{a} = \mathbf{d}_n,\ \mathbf{b} = \hat{\boldsymbol{\Sigma}}^{-1}\mathbf{d}_n \\
&= \mathrm{tr}\left(\hat{\boldsymbol{\Sigma}}^{-1} \sum_n \mathbf{d}_n \mathbf{d}_n^\top\right) && \text{the trace and the matrix product are linear} \\
&= \mathrm{tr}\left(\hat{\boldsymbol{\Sigma}}^{-1} (N-1)\hat{\boldsymbol{\Sigma}}\right) && \text{Equation (6.11)} \\
&= (N-1)\,\mathrm{tr}(\mathbf{I}_C) = (N-1)\,C.
\end{aligned}
$$

Dividing by N gives the average. This is a useful sanity check on any implementation: the mean in-sample score of a correct RX is $C(N-1)/N$, whatever the data look like. You will verify it on the worked example below.

7.8 A full worked example in three bands

Source check: the course's three-band table cannot be scored

The anomaly course builds a covariance from four pixels in three bands, $(0.10, 0.20, 0.30)$, $(0.12, 0.24, 0.28)$, $(0.08, 0.16, 0.32)$ and $(0.10, 0.20, 0.30)$ (`docs / training / anomaly - detection/08-rx-detection.md`, "Covariance, in three bands"). Its deviations are $\mathbf{0}$, $(0.02, 0.04, -0.02)$, $-(0.02, 0.04, -0.02)$ and $\mathbf{0}$: every one is a multiple of the same vector. Their covariance is therefore $\frac{0.0008}{3}\,\mathbf{v}\mathbf{v}^\top$ with $\mathbf{v} = (1, 2, -1)$, a matrix of rank 1 with eigenvalues 0.0016, 0 and 0 (`tools/compute_rx_output.txt`). Its determinant is 0 and it has no inverse, so RX cannot score anything against it. The table is a fine illustration of covariance signs, which is what the course uses it for, and an accidental illustration of the singular covariance of Section 7.9.2. The example below uses five pixels that span all three directions.

Worked example 7.3: RX by hand on three bands

Five background pixels, three bands, reflectance in percent. This is the longest calculation in the chapter; do it on paper, one step at a time.

pixel	band 1	band 2	band 3
1	8	18	30
2	9	21	30
3	11	19	28
4	11	21	30
5	11	21	32

Step 1, the mean.

$$\mu_1 = (8 + 9 + 11 + 11 + 11)/5 = 50/5 = 10$$
$$\mu_2 = (18 + 21 + 19 + 21 + 21)/5 = 100/5 = 20$$
$$\mu_3 = (30 + 30 + 28 + 30 + 32)/5 = 150/5 = 30.$$

Step 2, the deviations $\mathbf{d}_n = \mathbf{x}_n - (10, 20, 30)$:

$$\mathbf{d}_1 = (-2, -2, 0), \quad \mathbf{d}_2 = (-1, 1, 0), \quad \mathbf{d}_3 = (1, -1, -2), \quad \mathbf{d}_4 = (1, 1, 0), \quad \mathbf{d}_5 = (1, 1, 2).$$

Each band's deviations add to zero: $-2 - 1 + 1 + 1 + 1 = 0$; $-2 + 1 - 1 + 1 + 1 = 0$; $0 + 0 - 2 + 0 + 2 = 0$.

Step 3, the covariance. The six distinct entries are sums of products of deviations, divided by $N - 1 = 4$:

$$\Sigma_{11} = \tfrac{1}{4}\left[(-2)^2 + (-1)^2 + 1^2 + 1^2 + 1^2\right] = \tfrac{1}{4}(4 + 1 + 1 + 1 + 1) = \tfrac{8}{4} = 2$$
$$\Sigma_{22} = \tfrac{1}{4}\left[(-2)^2 + 1^2 + (-1)^2 + 1^2 + 1^2\right] = \tfrac{8}{4} = 2$$
$$\Sigma_{33} = \tfrac{1}{4}\left[0 + 0 + (-2)^2 + 0 + 2^2\right] = \tfrac{8}{4} = 2$$
$$\Sigma_{12} = \tfrac{1}{4}\left[(-2)(-2) + (-1)(1) + (1)(-1) + (1)(1) + (1)(1)\right] = \tfrac{1}{4}(4 - 1 - 1 + 1 + 1) = \tfrac{4}{4} = 1$$
$$\Sigma_{23} = \tfrac{1}{4}\left[(-2)(0) + (1)(0) + (-1)(-2) + (1)(0) + (1)(2)\right] = \tfrac{1}{4}(0 + 0 + 2 + 0 + 2) = 1$$
$$\Sigma_{13} = \tfrac{1}{4}\left[(-2)(0) + (-1)(0) + (1)(-2) + (1)(0) + (1)(2)\right] = \tfrac{1}{4}(0 + 0 - 2 + 0 + 2) = 0.$$

So

$$\boldsymbol{\Sigma} = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 1 \\ 0 & 1 & 2 \end{pmatrix}.$$

Each band has variance 2; neighbouring bands have covariance 1 (correlation 1/2); bands 1 and 3 are uncorrelated. This is the pattern of a real spectrum in miniature: neighbours move together, distant bands less so.

Step 4, the determinant by expansion along the first row (each entry times the determinant of the 2×2 block left when you delete its row and column, with signs $+, -, +$):

$$\begin{aligned}\det \boldsymbol{\Sigma} &= 2 \det \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} - 1 \det \begin{pmatrix} 1 & 1 \\ 0 & 2 \end{pmatrix} + 0 \det \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix} \\ &= 2(4-1) - 1(2-0) + 0 = 6 - 2 = 4.\end{aligned}$$

Step 5, the inverse. For a 3×3 matrix the recipe that generalises Equation (6.9) is: for each entry, compute its *cofactor* (the determinant of the block left after deleting its row and column, with sign $(-1)^{i+j}$), arrange the cofactors as a matrix, transpose it, and divide by the determinant. Because $\boldsymbol{\Sigma}$ is symmetric, the cofactor matrix is symmetric and the transpose changes nothing. The six distinct cofactors:

$$\begin{aligned} C_{11} &= +\det\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right) = 4 - 1 = 3, & C_{12} &= -\det\left(\begin{smallmatrix} 1 & 1 \\ 0 & 2 \end{smallmatrix}\right) = -(2-0) = -2, \\ C_{13} &= +\det\left(\begin{smallmatrix} 1 & 2 \\ 0 & 1 \end{smallmatrix}\right) = 1 - 0 = 1, & C_{22} &= +\det\left(\begin{smallmatrix} 2 & 0 \\ 0 & 2 \end{smallmatrix}\right) = 4 - 0 = 4, \\ C_{23} &= -\det\left(\begin{smallmatrix} 2 & 1 \\ 0 & 1 \end{smallmatrix}\right) = -(2-0) = -2, & C_{33} &= +\det\left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right) = 4 - 1 = 3. \end{aligned}$$

So

$$\boldsymbol{\Sigma}^{-1} = \frac{1}{4} \begin{pmatrix} 3 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 3 \end{pmatrix}.$$

Check one row: row 1 of $\boldsymbol{\Sigma}$ times column 1 of the bracket is $2 \times 3 + 1 \times (-2) + 0 \times 1 = 6 - 2 = 4$, and $4/4 = 1$; row 1 times column 2 is $2 \times (-2) + 1 \times 4 + 0 \times (-2) = 0$. Check the rest yourself.

Step 6, score a test pixel $\mathbf{x} = (12, 19, 32)$. Its deviation is $\mathbf{d} = (12-10, 19-20, 32-30) = (2, -1, 2)$. Multiply by the inverse, row by row:

$$\begin{aligned} \text{row 1:} &\quad \tfrac{1}{4}\left[3 \times 2 + (-2) \times (-1) + 1 \times 2\right] = \tfrac{1}{4}(6 + 2 + 2) = \tfrac{10}{4} = 2.5 \\ \text{row 2:} &\quad \tfrac{1}{4}\left[(-2) \times 2 + 4 \times (-1) + (-2) \times 2\right] = \tfrac{1}{4}(-4 - 4 - 4) = \tfrac{-12}{4} = -3 \\ \text{row 3:} &\quad \tfrac{1}{4}\left[1 \times 2 + (-2) \times (-1) + 3 \times 2\right] = \tfrac{1}{4}(2 + 2 + 6) = \tfrac{10}{4} = 2.5. \end{aligned}$$

Then the dot product with $\mathbf{d}$:

$$D^2 = 2 \times 2.5 + (-1) \times (-3) + 2 \times 2.5 = 5 + 3 + 5 = 13.$$

Step 7, compare. The per-band score would be $\frac{2^2}{2} + \frac{(-1)^2}{2} + \frac{2^2}{2} = 2 + 0.5 + 2 = 4.5$, about the mean of χ^2_3 (3): unremarkable. The Mahalanobis score is 13, beyond the 1% threshold of 11.34 and with a χ^2_3 tail probability of 0.0046. The pixel is high in bands 1 and 3 and low in band 2, between them, while the background's neighbouring bands move together.

Now a second test pixel, $\mathbf{x}' = (12, 23, 32)$, with $\mathbf{d}' = (2, 3, 2)$: further from the mean in every band, but in step with the background's correlations.

$$\begin{aligned} \boldsymbol{\Sigma}^{-1}\mathbf{d}' &= \tfrac{1}{4}\left(6 - 6 + 2,\ -4 + 12 - 4,\ 2 - 6 + 6\right) = \tfrac{1}{4}(2, 4, 2) = (0.5, 1, 0.5) \\ D^2 &= 2 \times 0.5 + 3 \times 1 + 2 \times 0.5 = 1 + 3 + 1 = 5. \end{aligned}$$

Per-band it scores $2 + 4.5 + 2 = 8.5$, nearly twice the first pixel; by Mahalanobis it scores 5, an ordinary value for χ^2_3 (tail probability 0.17). The `spectral` library, given the five background pixels' statistics, returns exactly $[13, 5]$ [C].

Step 8, check the in-sample identity. Scoring the five background pixels against their own statistics (`spectral.rx` on them) gives 3, 2.75, 2.75, 0.75 and 2.75 [C]. They add to $3 + 2.75 + 2.75 + 0.75 + 2.75 = 12 = (N-1)C = 4 \times 3$, as Section 7.7 promised.

Worked example 7.4: The same score through the eigenvectors

The eigenvalues of this $\mathbf{\Sigma}$ are $2+\sqrt{2}=3.414$, 2 and $2-\sqrt{2}=0.586$, with unit eigenvectors $\mathbf{u}_1=(\frac{1}{2},\frac{1}{\sqrt{2}},\frac{1}{2})$, $\mathbf{u}_2=(\frac{1}{\sqrt{2}},0,-\frac{1}{\sqrt{2}})$, $\mathbf{u}_3=(\frac{1}{2},-\frac{1}{\sqrt{2}},\frac{1}{2})$. (Check one: $\mathbf{\Sigma}\mathbf{u}_3=(1-0.7071,\ 0.5-1.4142+0.5,\ -0.7071+1)=(0.2929,-0.4142,0.2929)$, and $0.586\times(0.5,-0.7071,0.5)=(0.2929,-0.4142,0.2929)$.) The first direction is "all bands up together"; the third is "middle band against the outer two".
For $\mathbf{d}=(2,-1,2)$ the coordinates are

$$\mathbf{u}_1^\top\mathbf{d}=1-0.7071+1=1.2929,$$
$$\mathbf{u}_2^\top\mathbf{d}=1.4142-1.4142=0,$$
$$\mathbf{u}_3^\top\mathbf{d}=1+0.7071+1=2.7071,$$

and Equation (7.5) gives

$$D^2=\frac{1.2929^2}{3.414}+\frac{0^2}{2}+\frac{2.7071^2}{0.586}=\frac{1.6716}{3.414}+0+\frac{7.3284}{0.586}=0.490+0+12.51=13.0.$$

Almost all of the score, 12.5 of 13, comes from the direction with the smallest eigenvalue. That is the mechanism of RX in one line, and in the next section it becomes the problem.

7.9 Why global RX fails on real scenes

Global RX makes two assumptions that a real hyperspectral scene breaks: that the background is one Gaussian cloud, and that its covariance can be inverted safely.

7.9.1 Mixed backgrounds

A scene is half cool forest and half warm bare ground. A hot patch of machinery sits on the bare ground. Its temperature is high, but the scene-wide mean sits between forest and bare ground, and the standard deviation is inflated by the split.

Worked example 7.5: A mixture hides an anomaly from the global mean

Assume for illustration [A] that the forest is at 20 °C with standard deviation 2, the bare ground at 40 °C with standard deviation 2, half the scene each. The scene mean is $0.5\times20+0.5\times40=30$. The scene variance is the average squared value minus the squared mean; for each half the average squared value is its variance plus its squared mean:

$$\text{average of } x^2=0.5\,(2^2+20^2)+0.5\,(2^2+40^2)=0.5\times404+0.5\times1604=202+802=1004$$
$$\text{variance}=1004-30^2=1004-900=104,\qquad \text{standard deviation}=\sqrt{104}=10.2.$$

The machinery is at 46 °C. Against the whole scene, $z=(46-30)/10.2=1.57$: about one pixel in nine of a Gaussian scene is that far out. Against the bare ground around it, $z=(46-40)/2=3.0$. The global statistics hide it; the local ones reveal it.

The global mean and covariance describe a cloud that no pixel belongs to: the average of two separate clouds, stretched along the line between them. Chapter 8 answers this with a local background per pixel. Notice that the reconstruction family answers it differently: a network that has learnt what each surface looks like leaves residuals that are much closer to one cloud, which is why the residual score of Chapter 31 can use global statistics with less harm.

7.9.2 The near-singular covariance

This is the reason plain RX was taken out of the product.

Source check: why plain RX is not shipped

`backend/allotrope/foundation_models/resolver.py`, lines 185–189: "Plain RX on hyperspectral is intentionally not shipped (dropped 2026-05-11). With 150+ surviving bands the covariance comes out near-singular and Mahalanobis distances blow up to 1e11. Use MNF-RX instead – same machinery, but RX runs in a 10-dim noise-whitened subspace so the covariance is well-conditioned." The product's classical options are `mnf_rx` and `thermal_grx`. The class `GlobalRXDetector` still exists and is registered under `ADModel.RX` in `app/utils/anomaly_detection/detector_factory.py`, for notebooks and thermal use, but no product model resolves to it. Do not reintroduce it on hyperspectral data.

A matrix is *singular* when its determinant is zero, which for a covariance means some eigenvalue is zero: the pixels do not vary at all along that direction. It is *near-singular* when some eigenvalue is tiny compared with the largest. The size of the problem is measured by the *condition number*

$$\kappa(\mathbf{\Sigma}) = \frac{\lambda_{\max}}{\lambda_{\min}}. \tag{7.9}$$

By step (i) of the proof in Section 7.5, the eigenvalues of $\mathbf{\Sigma}^{-1}$ are $1/\lambda_k$, so the largest entry-scale of $\mathbf{\Sigma}^{-1}$ is $1/\lambda_{\min}$: the inverse amplifies the least-varying direction the most.

Worked example 7.6: Two highly correlated bands

Two neighbouring bands, each with variance 1, correlation $\rho = 0.999$: $\mathbf{\Sigma} = \left(\begin{smallmatrix} 1 & 0.999 \\ 0.999 & 1 \end{smallmatrix}\right)$.
Eigenvalues. Trace 2, determinant $1 - 0.999^2 = 1 - 0.998001 = 0.001999$. With $a = c = 1$ and $b = \rho$, Equation (6.17) gives $\lambda = 1 \pm \rho$:

$$\lambda_1 = 1.999 \text{ along } \mathbf{u}_1 = (1,1)/\sqrt{2}, \qquad \lambda_2 = 0.001 \text{ along } \mathbf{u}_2 = (1,-1)/\sqrt{2}.$$

The bands nearly always move together; the direction "band 1 against band 2" has almost no variance. The condition number is $1.999/0.001 = 1999$.
The inverse. By Equation (6.9),

$$\mathbf{\Sigma}^{-1} = \frac{1}{0.001999}\begin{pmatrix} 1 & -0.999 \\ -0.999 & 1 \end{pmatrix} = \begin{pmatrix} 500.25 & -499.75 \\ -499.75 & 500.25 \end{pmatrix}.$$

Entries of 500, from a matrix whose entries were at most 1.
Two pixels. A pixel one unit up in both bands, $\mathbf{d} = (1,1)$, has coordinate $2/\sqrt{2} = 1.414$ along $\mathbf{u}_1$ and 0 along $\mathbf{u}_2$: $D^2 = 1.414^2/1.999 = 2/1.999 = 1.0005$. A pixel that differs by a tenth of a unit in opposite directions, $\mathbf{d} = (0.1, -0.1)$, has coordinate 0 along $\mathbf{u}_1$ and $0.2/\sqrt{2} = 0.1414$ along $\mathbf{u}_2$: $D^2 = 0.1414^2/0.001 = 0.02/0.001 = 20$. A difference ten times smaller in Euclidean length scores twenty times higher.
That is RX working as designed, *if* the eigenvalue 0.001 is right. Now suppose it was estimated slightly wrong. If the true correlation is 0.999 but the estimate says 0.9995, the small eigenvalue is $1 - 0.9995 = 0.0005$ and the same pixel scores $0.02/0.0005 = 40$; if the estimate says 0.9999, it scores $0.02/0.0001 = 200$. A change in the fourth decimal place of one covariance entry changed the score tenfold.

Three things make small eigenvalues unreliable in practice, and all three bite hardest exactly where RX looks hardest.

Sampling error. A covariance estimated from a finite sample has errors in every entry. The script draws 200 pixels from the $\rho = 0.999$ distribution of worked example 7.6 five times [C]; the estimated small eigenvalue comes out as 0.00102, 0.00080, 0.00107, 0.00097 and 0.00093. Scores along that direction therefore vary by ±25% from one sample to the next, while the large eigenvalue's relative error is similar but harmless, because it divides a direction that contributes little.

Rounding error. A computer stores about 7 significant digits in `float32` (machine epsilon 1.19×10^{-7}) and about 16 in `float64` (2.22×10^{-16}). An eigenvalue solver's absolute error is roughly epsilon times the largest eigenvalue, so any eigenvalue smaller than about $\varepsilon\,\lambda_{\max}$ is rounding noise. A rule of thumb follows: inverting a matrix with condition number κ loses about $\log_{10}\kappa$ significant digits. For the example, $\log_{10} 1999 = 3.3$ digits. The script shows the extreme case: with $\rho = 1 - 10^{-8}$ the true small eigenvalue is 10^{-8}; `float64` recovers $1.000000005 \times 10^{-8}$, but in `float32` the number $1 - 10^{-8}$ rounds to exactly 1, the matrix becomes exactly singular, and the small eigenvalue comes out as 0 [C].

Dimension. With 165 bands there are 165 eigenvalues, and neighbouring bands of a reflectance spectrum are close to copies of each other, so many of the smallest eigenvalues sit at the level of noise and estimation error rather than real variation. Each one is a direction like $\mathbf{u}_2$ above, multiplied by its reciprocal. One such direction with an eigenvalue of 10^{-10} and a pixel that differs along it by 10^{-3} contributes $(10^{-3})^2/10^{-10} = 10^{-6}/10^{-10} = 10^4$ to the score, more than every meaningful direction together (Chapter 31 works the same arithmetic). The ranking is then decided by which pixels happen to have noise along the noisiest directions. That is what "distances blow up to 1e11" [M: `backend/allotrope/foundation_models/resolver.py`] means: the numbers are not merely large, they are meaningless.

A careful reader will object using Section 7.7: in exact arithmetic the in-sample scores average exactly $C(N-1)/N$, about 165, so how can any be 10^{11}? The identity holds only when the covariance that is inverted is exactly the covariance of the pixels that are scored. Three things break that link in practice, each through the small eigenvalues: rounding in the inverse (a direction whose true eigenvalue is below $\varepsilon\,\lambda_{\max}$ gets an arbitrary tiny or zero estimate, as in the `float32` example), statistics estimated on different pixels from those scored (a ring, a keep mask, a sample), and values substituted into the cube (the detectors fill a pixel's missing bands with the band mean, which puts it off the scene's correlation structure in exactly the low-variance directions). The anomaly score is then dominated by the mismatch, not by the anomaly.

> Inverting a covariance turns its smallest, least reliable eigenvalues into its largest weights; on 165 correlated bands those directions are noise, and RX ranks pixels by noise.

7.10 Regularisation

The standard repair is to make the small eigenvalues less small before inverting.

Definition 7.6 (Diagonal loading)**.** *Diagonal loading* (also called ridge regularisation) replaces $\boldsymbol{\Sigma}$ by $\boldsymbol{\Sigma} + \epsilon\mathbf{I}$ for a small $\epsilon > 0$ before inverting.

Its effect on the eigen-decomposition is exact and short. If $\boldsymbol{\Sigma}\mathbf{u}_k = \lambda_k\mathbf{u}_k$, then

$$(\boldsymbol{\Sigma} + \epsilon\mathbf{I})\mathbf{u}_k = \boldsymbol{\Sigma}\mathbf{u}_k + \epsilon\mathbf{u}_k = \lambda_k\mathbf{u}_k + \epsilon\mathbf{u}_k = (\lambda_k + \epsilon)\mathbf{u}_k.$$

The eigenvectors are unchanged and every eigenvalue moves up by ϵ. Three consequences follow.

1. The matrix is invertible even when $\boldsymbol{\Sigma}$ is singular: the smallest eigenvalue is now at least ϵ.
2. The condition number falls to $(\lambda_{\max} + \epsilon)/(\lambda_{\min} + \epsilon)$.
3. In the score, each direction contributes $(\mathbf{u}_k^\top \mathbf{d})^2/(\lambda_k + \epsilon)$ instead of $(\mathbf{u}_k^\top \mathbf{d})^2/\lambda_k$. Directions with $\lambda_k \gg \epsilon$ are almost untouched; directions with $\lambda_k \ll \epsilon$ are capped at $(\mathbf{u}_k^\top \mathbf{d})^2/\epsilon$.

Worked example 7.7: Loading the correlated pair

For $\boldsymbol{\Sigma} = \left(\begin{smallmatrix} 1 & 0.999 \\ 0.999 & 1 \end{smallmatrix}\right)$ (eigenvalues 1.999 and 0.001) and the pixel $\mathbf{d} = (0.1, -0.1)$, whose only coordinate is 0.1414 along $\mathbf{u}_2$ (squared: 0.02):

ϵ	eigenvalues	condition number	$D^2 = 0.02/(\lambda_2 + \epsilon)$
0	1.999, 0.001	$1.999/0.001 = 1999$	$0.02/0.001 = 20$
10^{-4}	1.9991, 0.0011	$1.9991/0.0011 = 1817$	$0.02/0.0011 = 18.2$
10^{-3}	2.000, 0.002	$2.000/0.002 = 1000$	$0.02/0.002 = 10$
10^{-2}	2.009, 0.011	$2.009/0.011 = 183$	$0.02/0.011 = 1.82$

The large eigenvalue barely moves; the small one is rescued. The choice of ϵ decides how much of the "bands disagreeing" direction you still believe: at 10^{-4} nearly all of it, at 10^{-2} almost none.

Two warnings come with diagonal loading.

The right ϵ depends on the units. ϵ is added to variances, so it is in squared data units. 10^{-4} is huge for reflectances whose variances are around 10^{-4}, and negligible for MNF components whose variances are 1 or more (Chapter 8). A fixed default is only meaningful for data on a known scale.

It keeps the solve from failing; it does not restore meaning. With 165 near-degenerate directions, a small ϵ leaves them nearly as amplified as before, and a large one throws away the very directions RX is meant to use. The anomaly course puts it this way: the ridge "keeps the solve from failing but does not restore meaning to 165 near-degenerate dimensions". The product's fix (Chapter 8) is to choose a small number of well-estimated directions first, and only then to whiten.

Shrinkage. A more general family, *shrinkage*, blends the covariance with a multiple of the identity: $(1 - \alpha)\boldsymbol{\Sigma} + \alpha\,\bar{\lambda}\,\mathbf{I}$, with $\bar{\lambda} = \operatorname{tr}\boldsymbol{\Sigma}/C$ the average eigenvalue and $0 \le \alpha \le 1$. By the same argument the eigenvectors are unchanged and each eigenvalue becomes $(1 - \alpha)\lambda_k + \alpha\bar{\lambda}$: pulled towards the average, small ones up and large ones down. For the three-band $\boldsymbol{\Sigma}$ of worked example 7.3 (average eigenvalue $6/3 = 2$) with $\alpha = 0.1$, the eigenvalues $3.414, 2, 0.586$ become $0.9 \times 3.414 + 0.2 = 3.273$, $0.9 \times 2 + 0.2 = 2$ and $0.9 \times 0.586 + 0.2 = 0.727$. Unlike diagonal loading, shrinkage scales with the data automatically. No detector in this repository uses it; it is here so that you recognise it in the literature.

Source check: what the code actually does to small eigenvalues

Four different safeguards appear in the code, and only one of them is diagonal loading.

- Global RX (`GlobalRXDetector`, via `spectral.rx`): none. It forms $\hat{\boldsymbol{\Sigma}}^{-1}$ with `np.linalg.inv`. It logs a warning when there are fewer than 10 pixels per band (`global_rx_detector.py`, lines 145–150), which addresses sample size, not conditioning.
- Local RX and MNF local RX: diagonal loading with $\epsilon = 10^{-4}$ (`DEFAULT_REGULARIZATION=1e-4`, `local_rx_detector.py` line 55 and `mnf_compression_lrx_detector.py` line 39; added in `app/detectors/_local_background.py` line 71),

and a linear solve instead of an explicit inverse (line 78).

- MNF: a floor of 10^{-10} on the *noise* eigenvalues (`mnf_compression_detector.py`, line 321), then truncation to 10 components.
- Residual Mahalanobis and PCA-RX: truncation to the top 30 principal directions and a floor of 10^{-12} on their eigenvalues (`residual_mahalanobis.py`, line 46).

The anomaly course says a ridge term "is used, in the local variants"; that matches the code.

Solving instead of inverting. The local detectors never form $\mathbf{\Sigma}^{-1}$. They solve the linear system $(\mathbf{\Sigma} + \epsilon\mathbf{I})\mathbf{s} = \mathbf{d}$ for $\mathbf{s}$ with `torch.linalg.solve`, then take $D^2 = \mathbf{d}^\top\mathbf{s}$. The two are equal in exact arithmetic, because $\mathbf{s} = \mathbf{\Sigma}^{-1}\mathbf{d}$; in floating point, solving is faster and loses fewer digits than computing every entry of the inverse and then multiplying. The source comment says it plainly: "Solving beats forming the inverse explicitly - better conditioned and faster."

7.11 The thermal case: RX in one band

With one band, $\mathbf{\Sigma}$ is the single number σ^2, its inverse is $1/\sigma^2$, and Equation (7.6) collapses to

$$\mathrm{RX}(x) = \frac{(x-\mu)^2}{\sigma^2} = z^2,$$

the squared z-score you started the chapter with. For the thermal pixel, $3.42^2 = 11.7$, with a χ^2_1 tail probability of 0.000625 [C], the same as the two-sided normal tail of 3.42. Single-band covariance cannot be near-singular (unless the scene is perfectly uniform), which is why RX is safe on thermal data and is shipped there as `thermal_grx`.

Source check: the thermal detector computes z^2 directly

`ThermalGRXDetector.detect` (`app/detectors/thermal_grx_detector.py`, lines 95–104) special-cases one band, because `spectral.rx` needs a covariance of at least 2×2: it computes `((vals-mu)**2)/var` with `var=vals.var()`, numpy's divisor N, and returns zeros if the variance is 0. Its mean and variance are those of the scene being scored, not the shipped constants in `thermal_pixel_consts.json` (those normalise inputs for the thermal neural model, Chakshu); the course's worked numbers use the shipped constants as a stand-in for a typical scene.

7.12 Common confusions

Is RX machine learning?
No. It is descriptive statistics computed from the scene in front of it: a mean, a covariance, a quadratic form. No training, no checkpoint, no labels. Hand it a new sensor and it works immediately.

Is a high RX score a large difference from the mean?
Not necessarily. Pixel B of worked example 7.1 and pixel (12, 19, 32) of worked example 7.3 are smaller in Euclidean terms than pixels that score lower. RX rewards differences in directions the background rarely takes.

Why is the Mahalanobis distance squared?
Strictly, the Mahalanobis distance is the square root of $\mathbf{d}^\top\mathbf{\Sigma}^{-1}\mathbf{d}$. The code and this book

work with the square, D^2, because that is what appears in the Gaussian's exponent and what follows the chi-square distribution. The ranking is the same either way.

Can I threshold RX at the chi-square value for 1%?
Only if the background really is one Gaussian, which a real scene is not; you would flag far more than 1% of pixels. The product thresholds every score by percentile of its own map.

Why not regularise the 165-band covariance and keep plain RX?
A ridge keeps the solve from failing but cannot tell real low-variance directions from estimation noise; it either keeps the noise (small ϵ) or discards the real directions too (large ϵ). MNF-RX and the top-30 PCA scores choose well-estimated directions first.

Does a larger number of bands make RX more sensitive?
It makes scores larger (the mean is C) and the covariance harder to estimate (about $C^2/2$ numbers). More bands help only while each new direction is estimated well; on 165 correlated bands they do not.

7.13 Check yourself

Exercise 7.1. A thermal pixel reads 61 °C; use the scene mean 24.58 and standard deviation 13.57. Compute its z-score and its one-band RX score. Is it beyond the χ^2_1 threshold for a 1% false-alarm rate?

Exercise 7.2. For $\mathbf{\Sigma} = \left(\begin{smallmatrix} 5 & 2 \\ 2 & 2 \end{smallmatrix}\right)$ (the matrix of Exercise 6.3), compute $\mathbf{\Sigma}^{-1}$ and the Mahalanobis distances of $\mathbf{d} = (1, -2)$ and $\mathbf{d} = (2, 1)$. Both have Euclidean length $\sqrt{5}$; explain the difference using the eigenvectors.

Exercise 7.3. With the three-band $\mathbf{\Sigma}$ and $\mathbf{\Sigma}^{-1}$ of worked example 7.3, score the deviation $\mathbf{d} = (0, 2, 0)$ (only the middle band is high). Compare with its per-band score.

Exercise 7.4. Two bands with variance 1 and correlation 0.99. Find the eigenvalues and the condition number, the Mahalanobis distance of $\mathbf{d} = (0.1, -0.1)$, and both again after diagonal loading with $\epsilon = 0.01$.

Exercise 7.5. An MNF-RX score map has $C = 10$ and is computed on a 1000×1000 scene. If the background were exactly Gaussian, how many pixels would exceed 23.21? Name two reasons a real scene exceeds it far more often.

Exercise 7.6. Show that for $C = 2$ the threshold with false-alarm rate α is $2\ln(1/\alpha)$, and compute it for $\alpha = 0.1\%$.

Next: Chapter 8, Local RX, regularisation and MNF-RX.

Chapter 8

Local RX, regularisation and MNF-RX

The one thing this chapter teaches

Global RX fails in two ways: one Gaussian cannot describe a scene of many surfaces, and a 165-band covariance cannot be inverted safely. Local RX fixes the first by comparing each pixel with a ring of its own neighbours; MNF-RX fixes the second by keeping only the few directions in which signal beats noise, so the covariance it inverts has no small eigenvalues to explode. Both are the Mahalanobis distance of Chapter 7, applied to a better-chosen background or a better-chosen space.

8.1 Two failures, two repairs

Chapter 7 ended with global RX failing for two separate reasons.

The background is not one cloud. In worked example 7.5 a hot patch of machinery on warm bare ground scored $z = 1.57$ against a scene that was half forest, and $z = 3.0$ against the bare ground around it. Global RX asks "is this pixel unusual for the scene?"; the question you often mean is "is this pixel unusual for its surroundings?". Both are real questions with different answers, and *local RX* answers the second by estimating a separate mean and covariance for every pixel from the pixels around it.

The covariance is nearly singular. On 165 correlated bands the smallest eigenvalues are estimation noise, and inverting turns them into the largest weights (Section 7.9.2). Diagonal loading keeps the arithmetic from failing but cannot tell real low-variance directions from noise (Section 7.10). The repair the product ships is to reduce the dimension first, keeping only directions that are well estimated and carry signal, and to run RX there. The obvious way to reduce dimension is PCA; the right way, for a sensor whose noisiest bands are also its most variable, is the *maximum noise fraction* transform, MNF.

Analogy

A security guard deciding whether a car is out of place could compare it with every car in the city (global) or with the cars parked on the same street (local). A sports car is unremarkable in the city-wide statistics and alarming on a street of delivery vans. But the guard must not include the suspicious car's own parking space, or the spaces right next to it where its passengers' other cars may be, in "the street": those would make the street look like the suspect. That margin left out of the comparison is the guard window of this chapter.

8.2 The dual window

For each pixel under test, local RX takes its background from a square ring: a square *outer window* centred on the pixel, with a smaller square *inner window* (the *guard window*) removed from its middle. This shape is called a *dual window* or *annulus*.

Definition 8.1 (Ring of half-sizes o and i)**.** With outer half-size o and inner half-size $i < o$, the outer window is the $(2o+1) \times (2o+1)$ square of pixels within o rows and o columns of the test pixel, the guard is the $(2i+1) \times (2i+1)$ square within i, and the background ring is the outer window minus the guard.

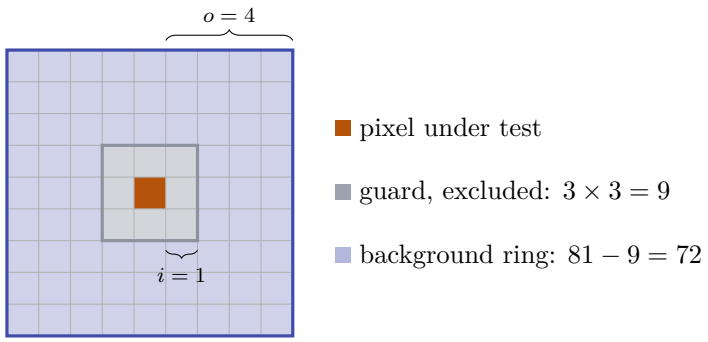


Figure 8.1: A dual window with outer half-size $o = 4$ and inner half-size $i = 1$. The background is the 9×9 outer square minus the 3×3 guard: $81 - 9 = 72$ pixels. The product's defaults, $o = 25$ and $i = 5$, give a 51×51 square and an 11×11 guard.

8.2.1 Counting the ring

The number of background pixels is the outer area minus the guard area. Factor it with the difference of two squares, $a^2 - b^2 = (a-b)(a+b)$:

$$\begin{aligned} n_{\text{ring}} &= (2o+1)^2 - (2i+1)^2 \\ &= \big((2o+1) - (2i+1)\big)\big((2o+1) + (2i+1)\big) \\ &= (2o - 2i)(2o + 2i + 2) \\ &= 4\,(o-i)(o+i+1). \end{aligned} \tag{8.1}$$

Worked example 8.1: The default ring, and the ring at an edge

`LocalRXDetector` defaults to `outer_window=25` and `inner_window=5`, which its docstring defines as half-sizes (`app/detectors/local_rx_detector.py`, lines 53–54 and 155–156). Directly:

$$\begin{aligned} \text{outer square} &= (2 \times 25 + 1)^2 = 51^2 = 2601 \\ \text{guard} &= (2 \times 5 + 1)^2 = 11^2 = 121 \\ n_{\text{ring}} &= 2601 - 121 = 2480. \end{aligned}$$

By Equation (8.1): $4 \times (25-5) \times (25+5+1) = 4 \times 20 \times 31 = 2480$. The code computes the same number as `max_bg=(2*outer+1)**2-(2*inner+1)**2` (line 240).
Near the edge of the image the windows are cut off. At a corner pixel the outer window keeps only the rows and columns inside the image, 0 to 25, so it is 26×26, and the guard is 6×6:

$$n_{\text{corner}} = 26 \times 26 - 6 \times 6 = 676 - 36 = 640.$$

On the middle of the top edge the outer window is 26×51 and the guard 6×11: $1326 - 66 = 1260$. A corner pixel has about a quarter of the full background, and pixels beside cloud or nodata lose more.

8.3 Why the guard window exists

The guard is the subtle part, and leaving it out quietly ruins the detector. A real target is usually larger than one pixel, and the sensor blurs it further into its neighbours. Without a guard, those neighbours are *also* target, and they go into the background statistics. The background mean shifts towards the target, the background covariance inflates along exactly the direction that makes the target unusual, and the Mahalanobis distance shrinks. The target partly whitens itself out of detection.

Worked example 8.2: A 3×3 target with and without a guard

One band, a 5×5 patch, outer half-size $o = 2$. The target is the 3×3 block of 25s in the middle; the 16 pixels around it are background, eight at 20 and eight at 22:

$$\begin{array}{ccccc} 20 & 22 & 20 & 22 & 20 \\ 22 & 25 & 25 & 25 & 22 \\ 20 & 25 & \mathbf{25} & 25 & 20 \\ 22 & 25 & 25 & 25 & 22 \\ 20 & 22 & 20 & 22 & 20 \end{array}$$

We score the centre pixel (bold). Do both cases on paper.

With a guard of half-size $i = 1$. The guard is the whole 3×3 target, and the ring is the 16 border pixels: $4 \times (2-1) \times (2+1+1) = 16$. Their mean and variance:

$$\mu = \frac{8 \times 20 + 8 \times 22}{16} = \frac{160 + 176}{16} = \frac{336}{16} = 21$$

$$\sum (x-\mu)^2 = 8 \times (20-21)^2 + 8 \times (22-21)^2 = 8 + 8 = 16$$

$$\sigma^2 = \frac{16}{16-1} = \frac{16}{15} = 1.0667.$$

With one band RX is the squared z-score:

$$\mathrm{RX} = \frac{(25-21)^2}{16/15} = \frac{16 \times 15}{16} = 15.$$

That is far beyond the χ^2_1 threshold for 0.1% (10.83).

Without a guard ($i = 0$, only the centre pixel excluded). The ring is now 24 pixels: the 16 border pixels and the 8 other target pixels at 25.

$$\mu = \frac{336 + 8 \times 25}{24} = \frac{336 + 200}{24} = \frac{536}{24} = 22.333$$

$$\begin{aligned} \sum (x-\mu)^2 &= 8 \times (20-22.333)^2 + 8 \times (22-22.333)^2 + 8 \times (25-22.333)^2 \\ &= 8 \times 5.444 + 8 \times 0.111 + 8 \times 7.111 = 43.56 + 0.89 + 56.89 = 101.33 \end{aligned}$$

$$\sigma^2 = \frac{101.33}{23} = 4.406$$

$$\mathrm{RX} = \frac{(25-22.333)^2}{4.406} = \frac{7.111}{4.406} = 1.61.$$

The same pixel drops from 15 to 1.61, an ordinary value. The target's own neighbours raised the background mean from 21 to 22.3 and quadrupled its variance. The repository's `batch_mahalanobis` returns 15.0000 and 1.6140 on this patch [C] (with the default ridge of 10^{-4}, 14.9986 and 1.6140).

Whatever the background window contains is, by definition, normal; the guard keeps the target out of its own definition of normal.

The guard is a compromise, not a guarantee. A target larger than the guard contaminates its ring anyway; a guard so large that it holds every possible target leaves a ring far from the pixel, which is a worse description of its surroundings. A target that fills the whole outer window becomes its own background and vanishes. That is one reason to run global and local detectors side by side: they fail on different targets.

8.4 The local RX algorithm

For every valid pixel $\mathbf{x}$ at position (r, c), `LocalRXDetector.detect` does the following.

1. Collect the valid pixels of the ring around (r, c): $\mathbf{b}_1, \ldots, \mathbf{b}_n$.
2. If n is below `min_bg_pixels`, leave the pixel unscored (NaN).
3. Compute the ring mean $\hat{\boldsymbol{\mu}} = \frac{1}{n}\sum_j \mathbf{b}_j$ and covariance $\hat{\boldsymbol{\Sigma}} = \frac{1}{n-1}\sum_j (\mathbf{b}_j - \hat{\boldsymbol{\mu}})(\mathbf{b}_j - \hat{\boldsymbol{\mu}})^\top$.
4. Score

$$\mathrm{LRX}(\mathbf{x}) = (\mathbf{x} - \hat{\boldsymbol{\mu}})^\top (\hat{\boldsymbol{\Sigma}} + \epsilon \mathbf{I})^{-1} (\mathbf{x} - \hat{\boldsymbol{\mu}}), \qquad \epsilon = 10^{-4}. \tag{8.2}$$

The source docstring states the same formula, "Score = (x - μ)$^\top$ ($\Sigma + \lambda$I)$^{-1}$ (x - μ)" (`local_rx_detector.py`, line 10), with the ridge called λ there; this book calls it ϵ to keep λ for eigenvalues.

8.4.1 Why at least $B + 1$ background pixels

The default `min_bg_pixels` is the number of bands plus one, $B + 1$ (`local_rx_detector.py`, line 187; $10 + 1 = 11$ for the 10-component MNF version). The reason is a fact about sample covariances.

Claim. A covariance estimated from n points has rank at most $n - 1$: at most $n - 1$ of its eigenvalues are non-zero.

Proof. By Equation (6.12), $\hat{\boldsymbol{\Sigma}}$ is a sum of n outer products $\mathbf{d}_j \mathbf{d}_j^\top$. For any vector $\mathbf{v}$, $\hat{\boldsymbol{\Sigma}}\mathbf{v} = \frac{1}{n-1}\sum_j \mathbf{d}_j(\mathbf{d}_j^\top \mathbf{v})$ is a combination of the deviations $\mathbf{d}_j$, so every output of $\hat{\boldsymbol{\Sigma}}$ lies in the space the deviations span. The deviations add to zero (Section 6.5), so the last one is minus the sum of the others, and they span at most $n - 1$ dimensions. A matrix whose outputs fill at most $n - 1$ dimensions has at least $B - (n - 1)$ zero eigenvalues. □

So with $n \leq B$ background pixels the covariance is exactly singular and has no inverse; $n = B + 1$ is the least that can give a full-rank matrix. The script confirms it on random 10-dimensional points: 2 points give rank 1, 5 give rank 4, 10 give rank 9, 11 give rank 10 [C]. Full rank is necessary, not sufficient: a covariance from 11 points in 10 dimensions is invertible but its small eigenvalues are very poorly estimated, which is where the ridge earns its place. On 177 raw PRISMA bands the corner pixel's 640 background pixels are enough for full rank but not for a trustworthy inverse.

8.4.2 Batching, padding and the masked mean

Local RX needs one covariance, one solve and one quadratic form *per pixel*. Doing this one pixel at a time in Python is hopeless (Section 8.5), so the detector stacks a batch of pixels' rings into one array of shape `(N, max_bg, B)` and computes all their statistics in a few tensor operations (`app/detectors/_local_background.py`, `batch_mahalanobis`). Arrays must be rectangular,

but rings are not all the same size, so each row is padded to the full ring size `max_bg` and a count array `n` says how many entries are real. A boolean mask `arange(max_bg)<n[:,None]` marks the real entries.

Worked example 8.3: The masked mean must divide by the real count

Two pixels, one band, `max_bg=4`. Pixel 0 has four real background values; pixel 1 has two, 0.30 and 0.34, padded with two zeros.

$$\begin{aligned} &\text{mask, pixel 0:} \quad [0,1,2,3] < 4 \;\rightarrow\; [\mathrm{T},\mathrm{T},\mathrm{T},\mathrm{T}] \\ &\text{mask, pixel 1:} \quad [0,1,2,3] < 2 \;\rightarrow\; [\mathrm{T},\mathrm{T},\mathrm{F},\mathrm{F}] \end{aligned}$$

The correct mean of pixel 1 divides the masked sum by its real count, 2:

$$\frac{0.30 + 0.34 + 0 + 0}{2} = \frac{0.64}{2} = 0.32.$$

Dividing by `max_bg` instead would give $0.64/4 = 0.16$, half the true value, because two of the four slots are padding. The code divides by `mask.sum(dim=1)` (line 60–61), and it multiplies the centred values by the mask again (line 64) so that padding contributes nothing to the covariance either. With the correct statistics, a test value of 0.40 at pixel 1 scores

$$\frac{(0.40 - 0.32)^2}{\sigma^2 + \epsilon}, \quad \sigma^2 = \frac{(0.30 - 0.32)^2 + (0.34 - 0.32)^2}{2 - 1} = \frac{0.0004 + 0.0004}{1} = 0.0008,$$

so $0.0064/(0.0008 + 0.0001) = 0.0064/0.0009 = 7.11$, which is what `batch_mahalanobis` returns [C]. Edge pixels are exactly the ones with padding, and exactly where you least want spurious detections.

8.4.3 Three practical details

Precision depends on the device. `batch_mahalanobis` computes in `float32` on a GPU and `float64` on the CPU (line 47). The same scene can therefore give scores differing in the last digits on different hardware, far below any threshold anyone sets, but enough to break a test that demands exact equality. Test with a tolerance, or test the ranking.

Two masks. The result object carries a `spatial_mask` (pixels *eligible* for scoring: valid and kept) and a `computed_mask` (pixels that actually *got* a score, `local_rx_detector.py` line 368). They differ wherever a ring held fewer than `min_bg_pixels` valid neighbours. Always reduce statistics over `computed_mask`. Treating an unscored pixel as score zero biases every statistic, and since unscored pixels cluster at edges and beside cloud, the bias is systematic rather than random.

Stride. A `stride` greater than 1 scores only every `stride`-th row and column and fills the rest by bilinear interpolation (`_interpolate_strided`). A stride of 2 scores a quarter of the pixels, about four times faster, at the price of smoothing small targets.

8.5 The cost of local RX

Worked example 8.4: How much arithmetic one PRISMA scene costs

A PRISMA scene is about 1210×1219 pixels [M: `docs/lld/local-background.md`]:

$$1210 \times 1219 = 1{,}474{,}990 \text{ pixels.}$$

For each, building a $B \times B$ covariance from 2480 ring pixels takes about $2480 \times B^2$ multiply-adds (each ring pixel adds its B^2-entry outer product), and solving one $B \times B$ system about $B^3/3$. Two cases:

B	covariance per pixel	solve per pixel	whole scene
10	$2480 \times 100 = 248{,}000$	≈ 333	$\approx 3.7 \times 10^{11}$
177	$2480 \times 31{,}329 = 77{,}695{,}920$	$\approx 1.85 \times 10^6$	$\approx 1.2 \times 10^{14}$

For comparison, global RX at $B = 10$ builds one covariance from all the pixels: $1{,}474{,}990 \times 100 \approx 1.5 \times 10^8$ multiply-adds, more than two thousand times less than local RX at the same B. Running local RX on 177 raw bands is about 300 times more expensive again than running it in 10 MNF components [C].

Memory matters as much as arithmetic. The detector sizes its GPU batch from `bytes_per_elem=(2*max_bg*B+B*B)*4` (`local_rx_detector.py`, lines 198–199): for $B = 10$ that is $(2 \times 2480 \times 10 + 100) \times 4 = 198{,}800$ bytes per pixel in the batch, and for $B = 177$ it is 3,636,996 bytes, so a batch that fits a GPU at $B = 10$ must be about 18 times smaller at $B = 177$.

The two repairs of this chapter therefore support each other. Reducing the dimension first (MNF) makes local RX affordable, and makes each local covariance well estimated from 2480 pixels.

8.6 Choosing a better space: why not PCA

The resolver's comment says the fix for the near-singular covariance is to run RX "in a 10-dim noise-whitened subspace". Why noise-whitened, and not the obvious choice, the top 10 principal components of Section 6.7?

PCA keeps the directions of greatest variance. Variance is signal plus noise, and on an imaging spectrometer the noise is not the same in every band: bands at the ends of the spectrum and in absorption regions have little light and a lot of noise. If a noisy band's noise variance is large, PCA ranks it high and keeps it, while a clean band with real but modest variation is ranked low and thrown away. What you want to rank by is not variance but *signal-to-noise ratio*.

8.6.1 Signal plus noise

Model every pixel as $\mathbf{x} = \mathbf{s} + \mathbf{n}$: a signal $\mathbf{s}$ (what the ground looks like) plus sensor noise $\mathbf{n}$, with the noise independent of the signal and of mean zero. The covariance of a sum of independent parts is the sum of their covariances. (Expand $(\mathbf{d}_s+\mathbf{d}_n)(\mathbf{d}_s+\mathbf{d}_n)^\top = \mathbf{d}_s\mathbf{d}_s^\top + \mathbf{d}_s\mathbf{d}_n^\top + \mathbf{d}_n\mathbf{d}_s^\top + \mathbf{d}_n\mathbf{d}_n^\top$ and average: the cross terms average to zero for independent parts.) So

$$\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_s + \boldsymbol{\Sigma}_n. \tag{8.3}$$

Along a unit direction $\mathbf{a}$, by Equation (6.14), the signal variance is $\mathbf{a}^\top\boldsymbol{\Sigma}_s\mathbf{a}$ and the noise variance $\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a}$.

Definition 8.2 (Noise fraction and signal-to-noise ratio)**.** The *noise fraction* of a direction $\mathbf{a}$ is the share of its variance that is noise, $\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a}/\mathbf{a}^\top\boldsymbol{\Sigma}\mathbf{a}$. Its *signal-to-noise ratio* (SNR) is $\mathbf{a}^\top\boldsymbol{\Sigma}_s\mathbf{a}/\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a}$.

The *maximum noise fraction* transform of Green et al. [8] finds the directions in order of noise fraction; ordered from least noisy to most, they are the directions of highest SNR first. The same transform is often called the *minimum noise fraction* transform, which is the name in the course and in the repository's docstrings; both names refer to the same directions, listed from opposite ends.

8.7 Deriving MNF

8.7.1 The generalised eigenproblem

Maximise the ratio of total variance to noise variance along $\mathbf{a}$:

$$\max_{\mathbf{a}} \; R(\mathbf{a}) = \frac{\mathbf{a}^\top\boldsymbol{\Sigma}\mathbf{a}}{\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a}}. \tag{8.4}$$

Because $\boldsymbol{\Sigma} = \boldsymbol{\Sigma}_s + \boldsymbol{\Sigma}_n$, $R(\mathbf{a}) = 1 + \mathrm{SNR}(\mathbf{a})$, so maximising R maximises SNR and minimises the noise fraction $1/R$. Scaling $\mathbf{a}$ does not change R (numerator and denominator both scale by the square), so fix the scale by requiring $\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a} = 1$ and maximise the numerator, exactly as PCA fixed $\mathbf{u}^\top\mathbf{u} = 1$ in Section 6.7. The Lagrangian is

$$L(\mathbf{a}, \lambda) = \mathbf{a}^\top\boldsymbol{\Sigma}\mathbf{a} - \lambda(\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a} - 1).$$

The gradient of a quadratic form $\mathbf{a}^\top\mathbf{S}\mathbf{a}$ with symmetric $\mathbf{S}$ is $2\mathbf{S}\mathbf{a}$ (Section 6.7), so setting the gradient of L to zero gives

$$\begin{aligned} 2\boldsymbol{\Sigma}\mathbf{a} - 2\lambda\boldsymbol{\Sigma}_n\mathbf{a} &= \mathbf{0} \\ \boldsymbol{\Sigma}\mathbf{a} &= \lambda\,\boldsymbol{\Sigma}_n\mathbf{a}. \end{aligned} \tag{8.5}$$

This is a *generalised eigenproblem*: λ and $\mathbf{a}$ are a generalised eigenvalue and eigenvector of the pair $(\boldsymbol{\Sigma}, \boldsymbol{\Sigma}_n)$. If $\boldsymbol{\Sigma}_n$ is invertible, multiplying both sides on the left by $\boldsymbol{\Sigma}_n^{-1}$ gives the ordinary eigenproblem $\boldsymbol{\Sigma}_n^{-1}\boldsymbol{\Sigma}\,\mathbf{a} = \lambda\mathbf{a}$. Multiplying Equation (8.5) on the left by $\mathbf{a}^\top$ shows what λ means: $\mathbf{a}^\top\boldsymbol{\Sigma}\mathbf{a} = \lambda\,\mathbf{a}^\top\boldsymbol{\Sigma}_n\mathbf{a}$, so $\lambda = R(\mathbf{a}) = 1 + \mathrm{SNR}(\mathbf{a})$. The best direction is the generalised eigenvector with the largest λ.

8.7.2 Solving it: noise-whiten, then PCA

$\boldsymbol{\Sigma}_n^{-1}\boldsymbol{\Sigma}$ is not symmetric, so the spectral theorem does not apply to it directly. The trick is to change variables so that it does. Let $\mathbf{W}_n = \boldsymbol{\Sigma}_n^{-1/2}$ be a whitening matrix for the *noise* (Section 6.8, symmetric form), so that $\mathbf{W}_n\boldsymbol{\Sigma}_n\mathbf{W}_n = \mathbf{I}$. Write $\mathbf{a} = \mathbf{W}_n\mathbf{v}$ and substitute into Equation (8.5):

$$\begin{aligned} \boldsymbol{\Sigma}\mathbf{W}_n\mathbf{v} &= \lambda\,\boldsymbol{\Sigma}_n\mathbf{W}_n\mathbf{v} && \\ \mathbf{W}_n\boldsymbol{\Sigma}\mathbf{W}_n\mathbf{v} &= \lambda\,\mathbf{W}_n\boldsymbol{\Sigma}_n\mathbf{W}_n\mathbf{v} && \text{multiply on the left by } \mathbf{W}_n \\ (\mathbf{W}_n\boldsymbol{\Sigma}\mathbf{W}_n)\,\mathbf{v} &= \lambda\,\mathbf{v} && \mathbf{W}_n\boldsymbol{\Sigma}_n\mathbf{W}_n = \mathbf{I}. \end{aligned}$$

The matrix $\mathbf{W}_n\boldsymbol{\Sigma}\mathbf{W}_n$ is symmetric: it is the covariance of the noise-whitened data $\mathbf{W}_n\mathbf{x}$ (Equation (6.13)). So MNF is two steps you already know:

1. **Noise-whiten**: transform the data so that the noise has identity covariance, equal in every direction.

2. **PCA** in the whitened space. Now that every direction has noise variance 1, the direction of greatest variance is the direction of greatest $1 + \text{SNR}$.

The MNF directions in the original band space are $\mathbf{a}_k = \mathbf{W}_n \mathbf{v}_k$, and the k-th *MNF component* of a pixel is $y_k = \mathbf{a}_k^\top (\mathbf{x} - \boldsymbol{\mu}) = \mathbf{v}_k^\top \mathbf{W}_n (\mathbf{x} - \boldsymbol{\mu})$. This is exactly the code:

```
# app/detectors/mnf_compression_detector.py, _compute_mnf_transform (lines 315-344)
centered = pixels - self._mnf_mean                      # (N, B)
noise_eigvals, noise_eigvecs = np.linalg.eigh(noise_cov)
noise_eigvals = np.maximum(noise_eigvals, 1e-10)        # floor the noise eigenvalues
# W_n, (B, B):
whiten = noise_eigvecs @ np.diag(1.0 / np.sqrt(noise_eigvals)) @ noise_eigvecs.T
whitened = centered @ whiten.T                          # step 1, (N, B)
# step 2, PCA of the noise-whitened data:
eigvals, eigvecs = np.linalg.eigh(np.cov(whitened, rowvar=False))
order = np.argsort(eigvals)[::-1]                       # largest 1 + SNR first
combined = (eigvecs[:, :n_components].T) @ whiten       # rows a_k^T, (k, B)
```

The symmetric form $\mathbf{V}\boldsymbol{\Lambda}^{-1/2}\mathbf{V}^\top$ used here is one of many noise-whitening matrices; any other is a rotation of it, $\mathbf{R}\mathbf{W}_n$ with $\mathbf{R}$ orthogonal. A rotation before PCA rotates the eigenvectors $\mathbf{v}_k$ by the same $\mathbf{R}$ and leaves the eigenvalues unchanged, so the final directions $\mathbf{a}_k$ and components y_k are the same whichever whitening you pick.

Worked example 8.5: MNF against PCA on two bands

Band 1 is variable but noisy; band 2 is quiet but clean. Suppose the total and noise covariances are

$$\boldsymbol{\Sigma} = \begin{pmatrix} 8 & 2 \\ 2 & 1.25 \end{pmatrix}, \qquad \boldsymbol{\Sigma}_n = \begin{pmatrix} 4 & 0 \\ 0 & 0.25 \end{pmatrix}, \qquad \text{so} \qquad \boldsymbol{\Sigma}_s = \boldsymbol{\Sigma} - \boldsymbol{\Sigma}_n = \begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix}.$$

Half of band 1's variance is noise (4 of 8); a fifth of band 2's is (0.25 of 1.25). The signal covariance has determinant $4 \times 1 - 2 \times 2 = 0$: all the signal lies along one direction, $(2, 1)$, as if one material's abundance varied across the scene. Do this on paper.

Step 1, noise-whiten. $\boldsymbol{\Sigma}_n$ is diagonal, so $\mathbf{W}_n = \boldsymbol{\Sigma}_n^{-1/2} = \text{diag}(1/\sqrt{4}, 1/\sqrt{0.25}) = \text{diag}(0.5, 2)$. The whitened covariance is $\mathbf{W}_n \boldsymbol{\Sigma} \mathbf{W}_n$; entry (i, j) is Σ_{ij} times the i-th and j-th diagonal entries of $\mathbf{W}_n$:

$$\begin{aligned} (1,1): &\quad 8 \times 0.5 \times 0.5 = 2 \\ (1,2): &\quad 2 \times 0.5 \times 2 = 2 \\ (2,2): &\quad 1.25 \times 2 \times 2 = 5 \end{aligned}$$

so $\mathbf{W}_n \boldsymbol{\Sigma} \mathbf{W}_n = \left(\begin{smallmatrix} 2 & 2 \\ 2 & 5 \end{smallmatrix}\right)$.

Step 2, PCA. Trace $2 + 5 = 7$, determinant $2 \times 5 - 2 \times 2 = 6$, characteristic polynomial $\lambda^2 - 7\lambda + 6 = (\lambda - 6)(\lambda - 1)$: $\lambda_1 = 6$, $\lambda_2 = 1$. For $\lambda_1 = 6$: $(2 - 6)v_1 + 2v_2 = 0$, so $v_2 = 2v_1$ and $\mathbf{v}_1 = (1, 2)/\sqrt{5}$. For $\lambda_2 = 1$: $(2 - 1)v_1 + 2v_2 = 0$, so $\mathbf{v}_2 = (2, -1)/\sqrt{5}$.

Step 3, back to bands. $\mathbf{a}_1 = \mathbf{W}_n \mathbf{v}_1 = (0.5 \times 1, 2 \times 2)/\sqrt{5} = (0.5, 4)/\sqrt{5} = (0.224, 1.789)$. The first MNF direction weights the clean band 8 times as heavily as the noisy one.

Step 4, read the eigenvalues. $\lambda_1 = 6 = 1 + \text{SNR}$, so the first component has SNR 5. Check directly: $\boldsymbol{\Sigma}_s \mathbf{a}_1 = (4 \times 0.5 + 2 \times 4,\ 2 \times 0.5 + 1 \times 4)/\sqrt{5} = (10, 5)/\sqrt{5}$, so the signal variance is $\mathbf{a}_1^\top \boldsymbol{\Sigma}_s \mathbf{a}_1 = (0.5 \times 10 + 4 \times 5)/5 = 25/5 = 5$; the noise variance is $(4 \times 0.25 + 0.25 \times 16)/5 = (1 + 4)/5 = 1$; SNR $= 5/1 = 5$. The second component has $\lambda_2 = 1$: SNR 0, pure noise, as it must be when all the signal lies along one direction.

Compare PCA on the raw bands. $\boldsymbol{\Sigma}$ has trace 9.25 and determinant $8 \times 1.25 - 4 = 6$, so its eigenvalues are $(9.25 \pm \sqrt{9.25^2 - 24})/2 = (9.25 \pm 7.846)/2$: 8.548 and 0.702. The first

principal direction is $(0.964, 0.264)$, mostly the noisy band 1. Its signal variance is 4.81 and its noise variance 3.74 [C]: SNR 1.29. PCA's best direction is a quarter as clean as MNF's.

8.7.3 Estimating the noise from neighbouring pixels

MNF needs $\mathbf{\Sigma}_n$, and nobody hands you the sensor noise. The *shift-difference* method of Green et al. [8] estimates it from the image itself. Adjacent pixels usually show nearly the same ground, so their *difference* is mostly noise. Write two horizontal neighbours as $\mathbf{x}(p) = \mathbf{s}(p) + \mathbf{n}(p)$ and $\mathbf{x}(p') = \mathbf{s}(p') + \mathbf{n}(p')$. If $\mathbf{s}(p') \approx \mathbf{s}(p)$,

$$\mathbf{x}(p') - \mathbf{x}(p) \approx \mathbf{n}(p') - \mathbf{n}(p).$$

If the noise at the two pixels is independent with the same covariance $\mathbf{\Sigma}_n$, the covariance of the difference is

$$\begin{aligned}\mathbf{\Sigma}(\mathbf{n}(p') - \mathbf{n}(p)) &= \mathbf{\Sigma}(\mathbf{n}(p')) + \mathbf{\Sigma}(\mathbf{n}(p)) - (\text{cross terms, zero for independent noise}) \\ &= \mathbf{\Sigma}_n + \mathbf{\Sigma}_n = 2\,\mathbf{\Sigma}_n,\end{aligned}$$

using Equation (6.13) with $\mathbf{A} = (\mathbf{I} \;\; -\mathbf{I})$ applied to the stacked pair. So $\mathbf{\Sigma}_n \approx \frac{1}{2}\mathbf{\Sigma}(\text{differences})$: the line `noise_cov=np.cov(diff_pixels,rowvar=False)/2.0` (`mnf_compression_detector.py`, line 291), with differences taken between each valid pixel and its right-hand neighbour (lines 283–285).

Worked example 8.6: A noise covariance from a strip of five pixels

Two bands, one row of five pixels:

	p_1	p_2	p_3	p_4	p_5
band 1	10.0	10.2	9.9	10.1	10.0
band 2	20.0	20.1	20.1	20.0	20.0

Horizontal differences, right minus left:

$$\begin{aligned}&\text{band 1:} \quad 0.2,\ -0.3,\ 0.2,\ -0.1 \qquad (\text{sum } 0)\\ &\text{band 2:} \quad 0.1,\ 0.0,\ -0.1,\ 0.0 \qquad (\text{sum } 0).\end{aligned}$$

Their means are 0, so the covariance of the four difference vectors (divisor $4 - 1 = 3$) is

$$\begin{aligned}\text{var}_1 &= \tfrac{1}{3}(0.04 + 0.09 + 0.04 + 0.01) = \tfrac{1}{3} \times 0.18 = 0.06\\ \text{var}_2 &= \tfrac{1}{3}(0.01 + 0 + 0.01 + 0) = \tfrac{1}{3} \times 0.02 = 0.00667\\ \text{cov}_{12} &= \tfrac{1}{3}(0.2 \times 0.1 + (-0.3) \times 0 + 0.2 \times (-0.1) + (-0.1) \times 0) = \tfrac{1}{3}(0.02 - 0.02) = 0.\end{aligned}$$

Halving: $\hat{\mathbf{\Sigma}}_n = \text{diag}(0.03, 0.00333)$, noise standard deviations $\sqrt{0.03} = 0.173$ and $\sqrt{0.00333} = 0.058$. Band 1 is three times as noisy. The repository's `_estimate_noise_covariance`, called on this strip, returns the same matrix [C].

The assumption behind the estimate fails wherever neighbours really differ: along a coastline, a road edge, a field boundary. Those differences are counted as noise, inflating $\hat{\mathbf{\Sigma}}_n$ in the directions of the edges' contrast. Over a whole scene, edges are a small fraction of neighbour pairs and the estimate is useful; and it needs no calibration data. One consequence is worth noticing: an imaging spectrometer such as PRISMA records each image column with its own detector element, so a column-to-column calibration offset (a stripe) appears in exactly these horizontal

differences. The estimate counts striping as noise, which is why the MNF local detector's docstring credits it with "suppressing sensor noise and striping artefacts" (`mnf_compression_lrx_detector.py`, lines 6–8).

8.7.4 The eigenvalue diagnostic

Because each MNF eigenvalue is $1 + \text{SNR}$ of its component, the list of eigenvalues says how much signal the scene has and in how many directions. Signal components have eigenvalues well above 1; pure noise components sit near 1. The anomaly course states the practical rule: the eigenvalues should *fall away sharply*, a few components carrying the structure; nearly flat eigenvalues mean the compression found no dominant signal and the scores deserve suspicion. That is why `MNFCompressionRXResult` carries `mnf_eigenvalues` and why the detector logs the ratio of the first to the last retained one (`mnf_compression_detector.py`, lines 346–349).

Worked example 8.7: Eigenvalues of PCA and MNF on a synthetic cube

The script builds a 12-band, 50×50 synthetic cube from three smooth spatial patterns mixed by three random spectra, plus independent noise whose standard deviation rises from 0.05 in band 1 to 0.6 in band 12, and runs the repository's own MNF code on it [C]. The noise estimate is close: for band 12 the true noise variance is $0.6^2 = 0.36$ and the estimate is 0.350; for band 4, 0.040 against 0.042. The two eigenvalue lists:

PCA of the cube	1.26, 0.53, 0.31, 0.26, 0.20, 0.17, 0.14, 0.085, 0.059, 0.035, 0.024, 0.010
MNF	65.4, 4.02, 2.18, 1.07, 1.06, 1.02, 1.00, 0.99, 0.98, 0.97, 0.95, 0.93

PCA's list falls smoothly: you cannot see that the cube has exactly three signals, because the noisy bands supply variance in every direction. MNF's list has three eigenvalues well above 1 and nine within 7% of 1: three signal components and nine noise components, read straight off the table.

8.8 MNF-RX

MNF-RX projects every valid pixel onto the first k MNF directions and runs global RX on the k components. The product's default is $k = 10$ (`DEFAULT_N_COMPONENTS=10`, `mnf_compression_detector.py` line 33; the worker calls `fit()` with no arguments). To see what this buys, you need one more fact about the Mahalanobis distance.

8.8.1 Mahalanobis distance ignores invertible changes of coordinates

Claim. Let $\mathbf{y} = \mathbf{A}\mathbf{x}$ with $\mathbf{A}$ square and invertible. Then the Mahalanobis distance of $\mathbf{y}$ in its own statistics equals that of $\mathbf{x}$ in its own.

Proof. The mean of $\mathbf{y}$ is $\mathbf{A}\boldsymbol{\mu}$ and its covariance $\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$ (Equation (6.13)). The inverse of a product of invertible matrices is the product of inverses in reverse order, $(\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)^{-1} = (\mathbf{A}^\top)^{-1}\boldsymbol{\Sigma}^{-1}\mathbf{A}^{-1}$ (check: multiplying by $\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$ cancels the factors pair by pair from the middle). With $\mathbf{d} = \mathbf{x} - \boldsymbol{\mu}$, use the transpose of a product and then the inverse of a product:

$$\begin{aligned}(\mathbf{A}\mathbf{d})^\top(\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)^{-1}(\mathbf{A}\mathbf{d}) &= \mathbf{d}^\top\mathbf{A}^\top(\mathbf{A}^\top)^{-1}\,\boldsymbol{\Sigma}^{-1}\,\mathbf{A}^{-1}\mathbf{A}\mathbf{d} \\ &= \mathbf{d}^\top\,\mathbf{I}\,\boldsymbol{\Sigma}^{-1}\,\mathbf{I}\,\mathbf{d} = \mathbf{d}^\top\boldsymbol{\Sigma}^{-1}\mathbf{d}.\end{aligned}$$

□

The MNF transform with all B components kept is an invertible matrix, so **MNF-RX with every component kept is exactly plain RX**. The script confirms it on the synthetic cube: the two score maps differ by at most 7×10^{-14} [C], rounding error. Every benefit of MNF-RX therefore comes from *discarding* components, not from the transform itself. MNF's job is to sort the directions so that the ones you discard are the noise.

8.8.2 What MNF-RX computes

The MNF components are uncorrelated over the pixels used to fit them, with variances equal to the MNF eigenvalues: their covariance is $\mathbf{V}_k^\top \mathbf{W}_n \mathbf{\Sigma} \mathbf{W}_n \mathbf{V}_k = \mathrm{diag}(\lambda_1, \ldots, \lambda_k)$, by the same argument as worked example 6.13. When `spectral.rx` re-estimates the covariance of the k components on those same pixels, it finds this diagonal matrix, so

$$\text{MNF-RX}(\mathbf{x}) = \sum_{j=1}^{k} \frac{y_j^2}{\lambda_j}, \qquad y_j = \mathbf{a}_j^\top (\mathbf{x} - \boldsymbol{\mu}). \tag{8.6}$$

The script checks this on the synthetic cube with $k = 3$: the library's scores and the sum agree to 5×10^{-14}, and the mean in-sample score is $2.9988 = 3 \times (N-1)/N$ [C], as Section 7.7 predicts.

Now the numerical point. The denominators in Equation (8.6) are the k largest MNF eigenvalues, each $1 + \text{SNR} \geq$ roughly 1. No denominator can be tiny, so nothing explodes; the condition number of the 10×10 covariance is λ_1/λ_{10}, the ratio the detector logs, and with λ_{10} near or above 1 it is bounded by the strongest component's SNR. Compare plain RX, whose denominators include the smallest of 165 eigenvalues of the raw covariance. Under a Gaussian background the scores follow χ^2_{10}, mean 10 (Section 7.7).

There is a statistical point as well, and it explains why discarding helps even when nothing explodes. A discarded MNF component with eigenvalue near 1 is almost pure noise; kept, it adds a squared standard normal to every pixel's score and nothing to a target's. Under a Gaussian background a score in C dimensions has standard deviation $\sqrt{2C}$ (Section 7.7), and a target must stand out against that spread. Suppose a target adds 30 to the score along one signal component. In $C = 10$ dimensions the background spread is $\sqrt{20} = 4.5$, and 30 is $30/4.5 = 6.7$ spreads; in $C = 165$ dimensions the spread is $\sqrt{330} = 18.2$, and the same 30 is $30/18.2 = 1.6$ spreads. Every noise component kept dilutes the signal components.

Source check: MNF-RX in the product

The worker builds the classical detector, calls `detector.fit()`, which estimates the MNF transform from all spatially valid pixels, and only then narrows `detector._spatial_mask` to the upstream keep mask (segmentation or cloud mask) before `detect()` (`backend/allotrope/action_types/_anomaly_scoring_run.py`, lines 361–392). So the MNF directions and the noise estimate come from all valid pixels, cloud included, while the RX mean and covariance in MNF space come from the kept pixels only. In that case the components are no longer exactly uncorrelated over the pixels RX sees, and `spectral.rx`'s full 10×10 inverse, not the diagonal sum of Equation (8.6), is what is computed; the two agree only when no pixels are masked. The course describes MNF-RX without this detail.

8.8.3 MNF local RX

`MNFCompressionLRXDetector` combines both repairs: project onto the top $k = 10$ MNF components, then run the local RX of Section 8.4 on the k-band cube, with the same defaults (outer half-size 25, guard half-size 5, ridge 10^{-4}, `min_bg_pixels` $= k + 1 = 11$; `mnf_compression_lrx_detector.py`, lines 36–39 and 215–219). Two consequences of the earlier sections:

- Each ring has 2480 pixels to estimate a 10×10 covariance with $10 \times 11/2 = 55$ distinct entries: about 45 pixels per number, well determined.
- The ridge $\epsilon = 10^{-4}$ is negligible here. The MNF components have variances of order 1 or more, so adding 10^{-4} changes each eigenvalue by a fraction of 10^{-4} or less. In this space the ridge does what it should: it only matters when a ring is so small that its covariance is singular.

It is registered (`ADModel.MNF_LRX`) but, like plain local RX, it is not one of the product's resolved classical models (`mnf_rx` and `thermal_grx`).

8.9 PCA-RX, the evaluation baseline

One more Mahalanobis detector appears throughout Part VII as the classical yardstick. In the evaluation framework (`app/evaluation/scorers.py`, lines 39–40) the scorer `pca_rx` is the residual score of Chapter 31 applied to the cube with a reconstruction of zeros: the "residual" is then the cube itself.

Definition 8.3 (PCA-RX)**.** Fit PCA to the valid pixels of the cube, keep the top $d = 30$ principal directions, and whiten them (Section 6.8): $\mathbf{z}(p) = \mathbf{\Lambda}_d^{-1/2}\mathbf{U}_d^\top(\mathbf{x}(p) - \boldsymbol{\mu})$. The score is the squared distance of a pixel's whitened components from the mean of the whitened components over a ring around it:

$$\text{PCA-RX}(p) = \|\mathbf{z}(p) - \bar{\mathbf{z}}_{\text{ring}}(p)\|^2 . \tag{8.7}$$

It differs from `LocalRXDetector` in four ways, each worth knowing:

1. **PCA, not MNF.** The 30 kept directions are those of greatest variance. Truncating at 30 rather than 10 keeps more structure; the price is that noise-dominated high-variance directions, if any, are kept too.
2. **Local mean, global covariance.** Only the mean comes from the ring; the whitening uses one covariance for the scene, fitted on at most 200,000 valid pixels drawn with seed 0 (`residual_mahalanobis.py`, lines 35 and 38–46). A local mean costs a few sums per pixel instead of a covariance and a solve, and Chapter 31 computes it for every pixel at once with integral images.
3. **Full window widths.** `LOCAL_WINDOW=(21,61)` gives the guard and outer *full* widths (line 33: "inner (guard) and outer full widths"), not half-sizes. The ring has $61^2 - 21^2 = 3721 - 441 = 3280$ pixels. In `LocalRXDetector`'s half-size convention this would be $o = 30$, $i = 10$: $4 \times 20 \times 41 = 3280$.
4. **A fallback instead of NaN.** A pixel whose ring holds fewer than $2d = 60$ valid pixels (`MIN_RING_FACTOR=2`) is scored against the global mean, $\|\mathbf{z}(p)\|^2$, and pixels outside the valid mask score 0, not NaN.

Worked example 8.8: PCA-RX on a tiny whitened patch

Take $d = 1$ and suppose the whitened component z is 1 everywhere on a 7×7 patch except 4 at the centre, with windows $(1, 5)$: a guard of full width 1 (the pixel itself) and an outer square of full width 5. The ring around the centre has $5^2 - 1^2 = 24$ pixels, all with $z = 1$, so its mean is 1 and

$$\text{PCA-RX(centre)} = (4 - 1)^2 = 9.$$

`local_mean_score` returns 9.0 on this patch [C]. Note what is *not* divided: there is no ring variance in the denominator, because z is already in units of the scene's global spread.

Values that will drift

PRISMA labelled bench, mean pixel ROC-AUC over four scenes, report generated 2026-10-15, production Indradhanu v0.2.0 (`research/experiments/dual-window-mahalanobis/README.md`): MNF-RX with 10 components 0.752, local RX with a dual window on the cube 0.743, PCA-RX with 30 components on the cube 0.831, residual Mahalanobis on Indradhanu's residual 0.839 (global mean) and 0.854 (local mean, 21/61) [M: `research/experiments/dual-window-mahalanobis/README.md`]. On implant bench-v1 (evaluation stage 3, 2026-10-17) the model-free PCA-RX caught 30.2% of objects in the top 1% against 26.9% for FT03 with residual Mahalanobis, while on the real objects its pAUC was 0.128 against FT03's 0.158 [M: `docs/training/anomaly-detection/19-running-an-evaluation.md`]. Chapters 34 and 35 explain what that reversal says about the bench.

The README draws the lesson that matters for this chapter: PCA30-RX on the cube beats MNF-RX on both datasets it was tried on, "which suggests MNF-RX's 10 components are too few. That is a separate, untested hypothesis." Read it as a caution, not a verdict: MNF-RX is the product's classical hyperspectral detector because it is numerically safe, not because it was shown to be the best classical detector.

8.10 Global and local together

Global and local detectors fail differently. Global RX misses a target that is ordinary for the scene but strange for its street; local RX misses a target larger than its guard, and a target that sits in a region it shares with nothing else. The repository has a detector that fuses the two, `StatisticalEnsembler` (`app/detectors/statistical_ensembler.py`): it runs global and local RX, turns each score map into ranks between 0 and 1 (its empirical cumulative distribution), and combines them by product (the default: a pixel must look anomalous to both), maximum or mean. It is not registered in the detector factory and is used only from notebooks. The same rank-fusion idea returns in the scoring experiments of Chapter 40.

8.11 Common confusions

Why not always use local RX, if it is more sensitive?
It is far more expensive (one covariance and solve per pixel), it misses targets larger than its guard, and it is blind to a target that fills its ring. Global and local answer different questions.

Is the ring square or circular?
Square: two nested squares, the inner one excluded. A circle would be marginally more principled and slower to index.

Are outer_window = 25 and LOCAL_WINDOW = (21, 61) the same kind of number?
No. `LocalRXDetector`'s windows are half-sizes, so 25 means a 51×51 square; the residual score's and PCA-RX's are full widths, so 61 means a 61×61 square. The `spectral` library's own `rx(window=(inner, outer))` also takes full widths.

MNF is PCA, isn't it?
PCA after noise-whitening. On the raw bands PCA maximises variance; MNF maximises signal-to-noise. Where the noisiest bands are also the most variable, PCA keeps exactly the wrong directions (worked example 8.5).

Does MNF-RX change the RX score of a pixel?
Only by discarding components. With all components kept it equals plain RX exactly; the

Mahalanobis distance does not care about invertible changes of coordinates.

Estimating noise from neighbour differences seems crude.
It assumes adjacent pixels show the same ground, which fails at edges, and counts column stripes as noise. It needs no calibration data and is good enough over a whole scene, which is why it has been standard since 1988.

What is min_bg_pixels for?
A covariance from n points has rank at most $n-1$, so with B bands you need at least $B+1$ background pixels for an invertible covariance. Pixels with fewer are left unscored (NaN), and that is why `computed_mask` exists.

8.12 Check yourself

Exercise 8.1. Compute the number of background pixels for half-sizes $o = 15$, $i = 3$, both directly and with Equation (8.1). How many does a corner pixel have?

Exercise 8.2. In the 5×5 patch of worked example 8.2, replace the eight target neighbours by background values (four 20s and four 22s) so that only the centre is 25. Score the centre with $i = 0$. What does this tell you about when a guard is needed?

Exercise 8.3. A pixel has 3 real background values padded to 6: 0.20, 0.22, 0.24. Compute the correct mean and the mean you would get by dividing by 6, and the correct (unregularised) one-band score of a test value 0.30.

Exercise 8.4. For $\mathbf{\Sigma} = \left(\begin{smallmatrix} 3 & 1 \\ 1 & 1.5 \end{smallmatrix}\right)$ and $\mathbf{\Sigma}_n = \left(\begin{smallmatrix} 2 & 0 \\ 0 & 0.5 \end{smallmatrix}\right)$, compute the noise-whitened covariance, its eigenvalues, and the SNR of the first MNF component. Which band does the first MNF direction weight more heavily?

Exercise 8.5. Prove that MNF-RX with all components kept equals plain RX, starting from $\mathbf{y} = \mathbf{A}(\mathbf{x} - \boldsymbol{\mu})$ with $\mathbf{A}$ invertible. Why, then, does MNF-RX help?

Exercise 8.6. How many distinct numbers does a 30×30 covariance hold? How many valid pixels per number does PCA-RX's 200,000-pixel sample give, and how many per number would a local 30-dimensional covariance from a 3280-pixel ring give?

Next: Chapter 9, Reconstruct, then subtract: why a pixel must never help reconstruct itself.

Part III

Reconstruction, the Indradhanu architecture and Chakshu

Chapter 9

Reconstruct, then subtract: why a pixel must never help reconstruct itself

The one thing this chapter teaches

A reconstruction model predicts each pixel's spectrum and the size of its mistake is the anomaly score; for that mistake to mean anything, every pixel must be predicted while it is hidden from the model, because a model that can see a pixel will copy it.

Part II of this book found anomalies with statistics: you modelled the background as a Gaussian cloud and asked how far each pixel sat from its centre (Chapters 7 and 8). That works, but it compresses everything the background can look like into one mean vector and one covariance matrix. A field next to a river next to a road is not one Gaussian cloud. This chapter starts Part III, which replaces the Gaussian with a neural network that has learned, from tens of thousands of ordinary scenes, what terrain looks like. The detection principle changes from "how far from the mean" to "how badly could the model redraw it".

The principle has one trap, and the trap is total: done naively, the model learns to copy and finds nothing. Most of this chapter is about that trap and the technique that closes it, masking. By the end you will be able to say, with a formula, why a pixel that helps reconstruct itself hides its own anomaly, and by how much.

9.1 The problem: no labels, and a background too varied for one Gaussian

Recall from Chapter 1 that nobody gives you labelled anomalies. You do not have a pile of spectra marked "anomaly" to train a classifier on, because anomalies are, by definition, the things nobody expected. What you do have is an enormous amount of *ordinary* imagery: every scene ever captured is mostly fields, forest, water, roads, soil and rooftops.

So the question becomes: can you use ordinary imagery, and only ordinary imagery, to build a detector for the extraordinary? The answer this chapter develops is yes, in three moves.

1. Train a model to *redraw* ordinary images.
2. At detection time, ask it to redraw the scene you care about.
3. Wherever the redrawing disagrees badly with what the satellite recorded, flag the pixel.

The obvious fix for "the background is too varied for one Gaussian" would be many Gaussians, one per land-cover type. That needs you to know the land-cover types and to draw their boundaries, which is another labelling problem. A reconstruction model learns the variety for itself, without labels, because the only thing it is ever asked to do is reproduce what it was shown.

Analogy

A countryside painter has spent twenty years painting the same valley: hedgerows, wheat, a river, a church. You cover part of a photograph of the valley with a card and ask the painter to paint what is under the card. Where the card covers wheat, the painter paints wheat, and the painting matches the photograph. Where the card covers a crashed aeroplane, the painter, who has never seen it, paints wheat, and the painting is badly wrong. The size of the painter's mistake tells you where something unusual lies, and the painter needed no list of "unusual things" to do it.

The analogy has one condition you must not drop: *the painter must not see what is under the card.* A painter allowed to look at the aeroplane will paint the aeroplane, the painting will match, and you will learn nothing. This whole chapter is about enforcing that condition. The same idea is the *cloze test* of language teaching: "The cat sat on the ___". You fill the blank from context; if the word were visible, the exercise would measure nothing; and if the hidden word was "helicopter", your confident "mat" is wrong in exactly the place where the text was surprising.

9.2 An autoencoder is a function from a cube to a cube

9.2.1 Tensors and the cube

A *tensor* is a rectangular block of numbers with any number of axes (a single number has none, a list has one, a grid has two). Chapter 10 treats tensors properly; for now you need one fact. A hyperspectral patch is a tensor with three axes,

$$\mathbf{x} \in \mathbb{R}^{C \times H \times W}, \tag{9.1}$$

read as "C layers, each H rows by W columns". Here $C = 165$ is the number of spectral bands on the common grid (Chapter 3) and $H = W = 128$ is the patch size used in training. One number $x_{c,i,j}$ is the reflectance in band c at row i, column j. The whole spectrum of one pixel is the column

$$\mathbf{x}_{:,i,j} = \left(x_{1,i,j},\ x_{2,i,j},\ \ldots,\ x_{C,i,j}\right)^{\top} \in \mathbb{R}^{C}, \tag{9.2}$$

where the colon means "every value along this axis".

9.2.2 The model as a function

Definition 9.1 (Reconstruction model)**.** A *reconstruction model* is a function

$$f_\theta \colon \mathbb{R}^{C \times H \times W} \to \mathbb{R}^{C \times H \times W}, \qquad \hat{\mathbf{x}} = f_\theta(\mathbf{x}), \tag{9.3}$$

whose output has exactly the same shape as its input. The vector θ collects the model's *parameters*: the numbers inside the model that training adjusts. The output $\hat{\mathbf{x}}$ ("x hat") is the model's *reconstruction* of $\mathbf{x}$.

The hat is standard mathematical notation for "our estimate of". Whenever you see `x_hat` in the code, it is the model's output.

An *autoencoder* is a reconstruction model built in two halves: an *encoder* that turns the input into an internal summary, and a *decoder* that rebuilds the input from that summary. The internal summary is usually smaller than the input in some sense, and is called the *bottleneck*. You will meet a precise version of "smaller" in Section 9.5 and again in Chapter 12.

The shape requirement in Equation (9.3) is not decoration. You are going to subtract $\hat{\mathbf{x}}$ from $\mathbf{x}$, and you can only subtract two tensors of the same shape. Every reconstruction model in this repository starts at `(B, 165, 128, 128)` and ends at `(B, 165, 128, 128)`, where B is the number of patches processed together (Chapter 10).

9.2.3 The residual and the score

Definition 9.2 (Residual)**.** The *residual* is the difference between what was recorded and what was predicted,

$$\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}} \in \mathbb{R}^{C \times H \times W}, \qquad r_{c,i,j} = x_{c,i,j} - \hat{x}_{c,i,j}. \tag{9.4}$$

The residual is itself a cube: one number per band per pixel. To get one number per pixel you need a rule that turns a residual spectrum $\mathbf{r}_{:,i,j}$ into a single *anomaly score*. The simplest such rule is the mean absolute residual across bands,

$$s(i,j) = \frac{1}{C} \sum_{c=1}^{C} |r_{c,i,j}| \,. \tag{9.5}$$

The product uses better rules: a spectral-angle term (Chapter 25) and, since October 2026, a Mahalanobis distance computed on the residual (Chapter 31). Every one of them has the same shape, "turn the residual spectrum into a number", and every one of them is zero when the residual is zero. Keep that last fact in mind; it is the root of the trap in Section 9.3.

Worked example 9.1: A three-band residual and its score

Take one pixel with $C = 3$ bands. The satellite recorded and the model predicted

$$\mathbf{x}_{:,i,j} = (0.30,\ 0.25,\ 0.40)^\top, \qquad \hat{\mathbf{x}}_{:,i,j} = (0.28,\ 0.27,\ 0.31)^\top.$$

Work it on paper. First the residual, one band at a time:

$$\begin{aligned} r_1 &= 0.30 - 0.28 = 0.02, \\ r_2 &= 0.25 - 0.27 = -0.02, \\ r_3 &= 0.40 - 0.31 = 0.09. \end{aligned}$$

Next take absolute values: $|r_1| = 0.02$, $|r_2| = 0.02$, $|r_3| = 0.09$. Add them:

$$\begin{aligned} 0.02 + 0.02 &= 0.04, \\ 0.04 + 0.09 &= 0.13. \end{aligned}$$

Divide by $C = 3$:

$$s = \frac{0.13}{3} = 0.0433.$$

Band 3 contributes most of the score. If band 3 is where a material has an absorption feature that the surrounding terrain lacks, this is exactly the kind of disagreement you want to see.

9.2.4 Training: choosing the parameters

Training means choosing θ so that $\hat{\mathbf{x}}$ is close to $\mathbf{x}$ on ordinary data. "Close" is measured by a *loss*: a single number that is small when the reconstruction is good. The reconstruction losses in this repository are averages of per-value errors, for example the *L1 loss*

$$\mathcal{L}(\theta) = \frac{1}{CHW} \sum_{c,i,j} \left| x_{c,i,j} - f_\theta(\mathbf{x})_{c,i,j} \right|, \tag{9.6}$$

averaged again over every training patch. Training repeatedly nudges θ in the direction that lowers $\mathcal{L}$; Chapter 26 shows how. For this chapter you only need: *training finds parameters with a small loss.* If some choice of θ gives a loss of exactly zero, a successful training run will head towards it.

9.3 The trap: the identity is the perfect answer to the wrong question

9.3.1 The naive plan

Here is the plan a beginner builds. Show the model a patch, ask it to output the same patch, and train it to minimise Equation (9.6). Then score pixels by Equation (9.5).

Consider one particular function, the *identity*

$$\mathrm{id}(\mathbf{x}) = \mathbf{x}. \tag{9.7}$$

It copies its input to its output and does nothing else. Put it into the loss:

$$\begin{aligned}
\mathcal{L}(\mathrm{id}) &= \frac{1}{CHW} \sum_{c,i,j} \left| x_{c,i,j} - \mathrm{id}(\mathbf{x})_{c,i,j} \right| && \text{substitute } f = \mathrm{id} \\
&= \frac{1}{CHW} \sum_{c,i,j} \left| x_{c,i,j} - x_{c,i,j} \right| && \text{the identity returns its input} \\
&= \frac{1}{CHW} \sum_{c,i,j} 0 = 0. && \text{every term vanishes}
\end{aligned}$$

A loss built from absolute values can never be negative, because every term $|\cdot|$ is at least zero. So $\mathcal{L} \geq 0$ for every model, and the identity reaches the floor. **The identity is a global minimiser of the naive training objective.** Nothing in the objective prefers "understand the terrain" over "copy the pixels"; both give zero loss on training data, and copying is far easier to learn.

Now look at what the identity does to detection. On *any* input, ordinary or anomalous,

$$\mathbf{r} = \mathbf{x} - \mathrm{id}(\mathbf{x}) = \mathbf{0} \quad \Longrightarrow \quad s(i,j) = 0 \text{ for every pixel.} \tag{9.8}$$

The score map is uniformly zero. The detector reports no anomalies and no error. That is what makes the failure dangerous: it is silent. There is no crash and no warning, only an empty result that looks like a clean scene.

> Copying is not a subtle failure of a reconstruction model; it is the optimal solution to "reproduce your input". Any model that can copy will drift towards copying, because copying is easier than understanding.

9.3.2 Partial copying is partial blindness

Real models rarely become the exact identity. A more realistic failure is a model that lets each pixel contribute *some* weight to its own reconstruction. You can measure the damage exactly.

Suppose the model reconstructs pixel value x as a blend of the pixel itself and a prediction c made from its neighbours (its *context*):

$$\hat{x} = w\,x + (1-w)\,c, \qquad 0 \le w \le 1. \tag{9.9}$$

The number w is the self-weight: how much the pixel helps reconstruct itself. Compute the residual step by step.

$$\begin{aligned} r &= x - \hat{x} && \text{definition of the residual} \\ &= x - w\,x - (1-w)\,c && \text{substitute Equation (9.9)} \\ &= (1-w)\,x - (1-w)\,c && \text{collect the } x \text{ terms: } x - wx = (1-w)x \\ &= (1-w)\,(x-c). && \text{take out the common factor } (1-w) \end{aligned} \tag{9.10}$$

Read Equation (9.10) carefully. The quantity $x - c$ is the honest surprise: how far the pixel is from what its surroundings predict. The model reports only a fraction $(1 - w)$ of it.

- $w = 0$: the pixel takes no part in its own reconstruction and the full surprise $x - c$ survives.
- $w = 1$: the pixel is copied, $(1 - w) = 0$, and every residual is zero, anomalies included.
- In between, every anomaly is shrunk by the same factor $(1 - w)$.

Since the anomaly score is built from the residual, a self-weight of w costs you a fraction w of every anomaly's evidence. A pixel must never help reconstruct itself, and Equation (9.10) is the reason in one line.

9.4 A one-dimensional example you can do by hand

Everything so far fits into six numbers. Take a one-dimensional "image" of six pixels with one band each:

$$\mathbf{x} = (x_1, \ldots, x_6) = (1.0,\ 1.1,\ 1.2,\ 5.0,\ 1.4,\ 1.5). \tag{9.11}$$

The background rises gently by 0.1 per pixel. Pixel 4 is an anomaly: it should be about 1.3 and it reads 5.0.

9.4.1 Three predictors

Compare three ways of predicting each pixel.

1. **Copy.** $\hat{x}_k = x_k$. This is the identity.
2. **Leave-one-out neighbour average.** Hide pixel k and predict it from the two pixels either side, $\hat{x}_k = \frac{1}{2}(x_{k-1} + x_{k+1})$. At the two ends there is only one neighbour, so $\hat{x}_1 = x_2$ and $\hat{x}_6 = x_5$. This predictor never sees the pixel it predicts: $w = 0$ in Equation (9.9).
3. **Self-including average.** Average the pixel with its two neighbours, $\hat{x}_k = \frac{1}{3}(x_{k-1} + x_k + x_{k+1})$. The pixel helps reconstruct itself with $w = \frac{1}{3}$.

Worked example 9.2: Leave-one-out prediction of six pixels

Do every line on paper. Predictions first:

$$\begin{aligned}
\hat{x}_1 &= x_2 = 1.1, \\
\hat{x}_2 &= \tfrac{1}{2}(x_1 + x_3) = \tfrac{1}{2}(1.0 + 1.2) = \tfrac{1}{2}(2.2) = 1.1, \\
\hat{x}_3 &= \tfrac{1}{2}(x_2 + x_4) = \tfrac{1}{2}(1.1 + 5.0) = \tfrac{1}{2}(6.1) = 3.05, \\
\hat{x}_4 &= \tfrac{1}{2}(x_3 + x_5) = \tfrac{1}{2}(1.2 + 1.4) = \tfrac{1}{2}(2.6) = 1.3, \\
\hat{x}_5 &= \tfrac{1}{2}(x_4 + x_6) = \tfrac{1}{2}(5.0 + 1.5) = \tfrac{1}{2}(6.5) = 3.25, \\
\hat{x}_6 &= x_5 = 1.4.
\end{aligned}$$

Residuals $r_k = x_k - \hat{x}_k$:

$$\begin{aligned}
r_1 &= 1.0 - 1.1 = -0.1, & r_4 &= 5.0 - 1.3 = 3.7, \\
r_2 &= 1.1 - 1.1 = 0.0, & r_5 &= 1.4 - 3.25 = -1.85, \\
r_3 &= 1.2 - 3.05 = -1.85, & r_6 &= 1.5 - 1.4 = 0.1.
\end{aligned}$$

Scores $|r_k|$: 0.1, 0.0, 1.85, 3.7, 1.85, 0.1.
The largest score, 3.7, is at pixel 4, the anomaly. Its prediction, 1.3, is exactly what the gentle background said should be there.

Two things in worked example 9.2 deserve a second look.

First, pixels 3 and 5 also score high (1.85). They are ordinary, but their predictions used pixel 4 as a neighbour, and pixel 4's value 5.0 dragged their predictions upwards. An anomaly contaminates the reconstruction of its neighbours and leaves a *halo* of moderate scores around itself. The real model shows the same effect around bright objects, and Chapter 31 deals with it by excluding a ring of pixels around the one being scored.

Second, the copy predictor gives $r_k = 0$ for all six pixels: you can check that without arithmetic.

Worked example 9.3: The self-including average hides a third of the anomaly

Now let pixel 4 help predict itself:

$$\begin{aligned}
\hat{x}_4 &= \tfrac{1}{3}(x_3 + x_4 + x_5) \\
&= \tfrac{1}{3}(1.2 + 5.0 + 1.4) \\
&= \tfrac{1}{3}(7.6) \\
&= 2.5333.
\end{aligned}$$

The residual is

$$r_4 = 5.0 - 2.5333 = 2.4667.$$

Check it against Equation (9.10). The self-weight is $w = \frac{1}{3}$, and the honest context prediction is the average of the two neighbours, $c = 1.3$ from worked example 9.2. So

$$\begin{aligned}
(1 - w)(x_4 - c) &= \tfrac{2}{3}\,(5.0 - 1.3) \\
&= \tfrac{2}{3}\,(3.7) \\
&= 2.4667.
\end{aligned}$$

The two routes agree. Letting the pixel see itself cost a third of the anomaly's score: 3.7 became 2.4667.

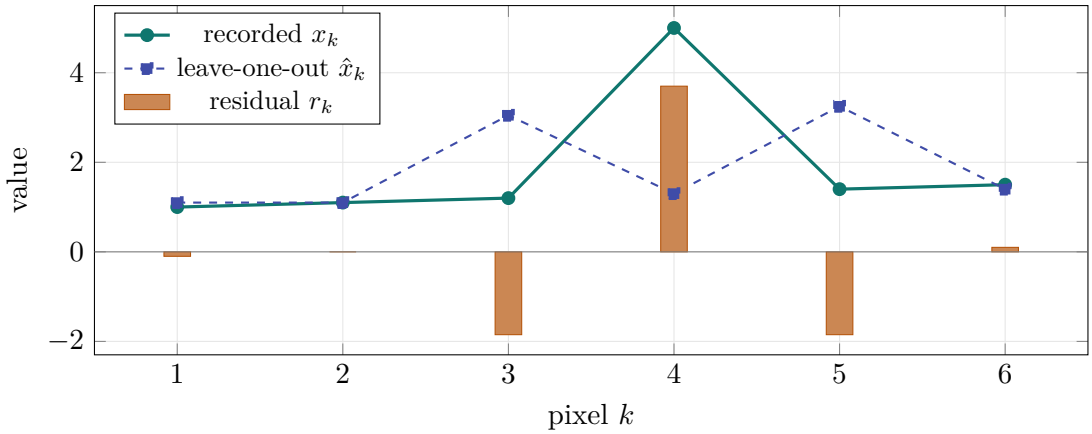


Figure 9.1: The six-pixel signal of Equation (9.11), its leave-one-out predictions and the residuals, drawn to scale. The anomaly at pixel 4 gets the largest residual; its two neighbours get a halo of residual because their predictions used pixel 4.

9.4.2 Leave-one-out with two passes

There is a neat way to get the leave-one-out predictions for every pixel using only two runs of a predictor. In pass 1, hide the odd pixels $\{1, 3, 5\}$ and predict them from the visible even pixels. In pass 2, hide the even pixels $\{2, 4, 6\}$ and predict them from the visible odd pixels. Because every neighbour of an odd pixel is even, and every neighbour of an even pixel is odd, each prediction in worked example 9.2 uses only visible pixels in the pass where its own pixel is hidden. Keep from each pass only the predictions at the pixels it hid. Every pixel then has a prediction made without seeing itself, after exactly two passes. Hold on to this idea: Section 9.8 uses it on the real model.

9.5 The bottleneck makes copying hard, not impossible

The first line of defence that people reach for is the bottleneck: if the internal summary is too small to hold the input, the model cannot copy it. That is partly true, and you should understand exactly how far it goes.

9.5.1 Counting the values in a convolutional autoencoder

The repository's oldest models (codenames Pratibimba, Antardhana and Tirohita; see Table 9.1) are convolutional autoencoders for one-band thermal patches. Their encoder is built in `app/foundation_models/components/spatial_encoder.py` as

```
SpatialEncoder(in_channels=1, base_channels=32, num_stages=3)
```

It halves the height and width three times and doubles the number of channels each time:

$$\texttt{(B, 1, 128, 128)} \to \texttt{(B, 32, 64, 64)} \to \texttt{(B, 64, 32, 32)} \to \texttt{(B, 128, 16, 16)}.$$

Count the values in one patch before and after:

$$\text{input} = 1 \times 128 \times 128 = 16{,}384,$$
$$\text{bottleneck} = 128 \times 16 \times 16 = 128 \times 256 = 32{,}768.$$

The bottleneck holds *twice* as many numbers as the input. Counted in values, nothing is compressed at all. So whatever stops this network from copying, it is not a shortage of storage.

What does constrain it is that the same parameters serve every patch the network ever sees: 165,024 of them in the encoder [C] (Chapter 10 derives that number by hand), against millions of training pixels. The network cannot afford a separate trick for each patch, so it is pushed towards transformations that work for terrain in general. That is a pressure, not a guarantee. A network with enough capacity and a convolution path that carries pixels straight through can still learn to copy, partially, and Equation (9.10) tells you what partial copying costs.

9.5.2 Why not a bigger model?

It is tempting to think a bigger model would reconstruct better and therefore detect better. The first half is true and the second is false. A model with more capacity reconstructs *everything* better, anomalies included. Lower error everywhere means smaller residuals at the anomalies and less contrast between anomaly and background. Capacity in a reconstruction detector is deliberately limited; the decision to grow Indradhanu into the three-times-wider "Heavy" variant (Chapter 43) had to be justified by measured detection scores, not by lower reconstruction loss.

9.6 L2 teaches the model to reconstruct anomalies; L1 does not

The roster of models moved from the squared-error (L2) loss to the absolute-error (L1) loss. The reason is a small piece of mathematics that you can do with three numbers, and it sharpens what "the model should fail on anomalies" means.

9.6.1 The best constant guess under each loss

Suppose the model, from context, must make one guess $\hat{x}$ for a hidden pixel, and across the training data the true values behind that context are $x^{(1)}, \dots, x^{(n)}$. Which guess minimises the average loss?

Under L2. The average squared error is

$$J_2(\hat{x}) = \frac{1}{n} \sum_{k=1}^{n} \big(x^{(k)} - \hat{x}\big)^2.$$

Differentiate with respect to $\hat{x}$. The derivative of $(a - \hat{x})^2$ with respect to $\hat{x}$ is $-2(a - \hat{x})$ (chain rule: the outer square gives $2(a - \hat{x})$, the inner $a - \hat{x}$ has derivative -1). So

$$\begin{aligned} \frac{dJ_2}{d\hat{x}} &= \frac{1}{n} \sum_{k=1}^{n} -2\big(x^{(k)} - \hat{x}\big) && \text{differentiate term by term} \\ &= -\frac{2}{n}\Big(\sum_{k} x^{(k)} - n\hat{x}\Big). && \hat{x} \text{ appears } n \text{ times in the sum} \end{aligned}$$

Set the derivative to zero and solve:

$$\begin{aligned} \sum_{k} x^{(k)} - n\hat{x} = 0 && \text{the factor } -2/n \text{ is not zero} \\ \hat{x} = \frac{1}{n} \sum_{k} x^{(k)}. && \text{divide both sides by } n \end{aligned}$$

The L2-optimal guess is the *mean* of the values.

Under L1. The average absolute error is $J_1(\hat{x}) = \frac{1}{n}\sum_k \left|x^{(k)} - \hat{x}\right|$. Its slope with respect to $\hat{x}$ is -1 for every value above $\hat{x}$ and $+1$ for every value below it, so the slope is (number below) − (number above). It is zero when as many values lie above $\hat{x}$ as below. The L1-optimal guess is the *median*.

Worked example 9.4: Mean versus median when one value is an anomaly

Three training pixels share the same context. Two are ordinary and one is an anomaly: $x^{(1)} = 1.0$, $x^{(2)} = 1.2$, $x^{(3)} = 5.0$.
The mean:

$$1.0 + 1.2 + 5.0 = 7.2,$$
$$7.2/3 = 2.4.$$

The median, after sorting 1.0, 1.2, 5.0, is the middle value 1.2.
Compare the average L1 loss of the two guesses. For $\hat{x} = 1.2$:

$$|1.0 - 1.2| + |1.2 - 1.2| + |5.0 - 1.2| = 0.2 + 0 + 3.8 = 4.0,$$
$$4.0/3 = 1.333.$$

For $\hat{x} = 2.4$:

$$|1.0 - 2.4| + |1.2 - 2.4| + |5.0 - 2.4| = 1.4 + 1.2 + 2.6 = 5.2,$$
$$5.2/3 = 1.733.$$

Under L1 the median wins. Now the average L2 loss. For $\hat{x} = 1.2$:

$$0.2^2 + 0^2 + 3.8^2 = 0.04 + 0 + 14.44 = 14.48,$$
$$14.48/3 = 4.827.$$

For $\hat{x} = 2.4$:

$$1.4^2 + 1.2^2 + 2.6^2 = 1.96 + 1.44 + 6.76 = 10.16,$$
$$10.16/3 = 3.387.$$

Under L2 the mean wins, and the mean, 2.4, has been dragged halfway towards the anomaly.

The lesson of worked example 9.4: a model trained with L2 is rewarded for moving its predictions towards rare extreme values, so it partly learns to reconstruct the very things you want it to fail on. A model trained with L1 settles on the typical value and leaves the extreme ones badly reconstructed.

9.6.2 The same story in the gradients

Training moves $\hat{x}$ by an amount proportional to the derivative of the loss. For one pixel with residual $r = x - \hat{x}$:

$$\frac{d}{d\hat{x}}(x - \hat{x})^2 = -2r, \qquad \text{L2: proportional to the error}$$
$$\frac{d}{d\hat{x}}\,|x - \hat{x}| = -\operatorname{sign}(r), \qquad \text{L1: size 1 whatever the error}$$

(for $r \neq 0$). Take an ordinary pixel with $r = 0.1$ and an anomalous one with $r = 3.8$. Under L2 the pull is $2 \times 0.1 = 0.2$ against $2 \times 3.8 = 7.6$: the anomaly pulls 38 times harder. Under L1

both pull with size 1. L2 spends its effort on the worst-fitting pixels, which are the anomalies; L1 spends it evenly. Every later model in the roster uses L1. Chapter 25 develops the L1 loss and its spectral-angle partner fully.

9.7 Masking: make the answer absent

The bottleneck makes copying hard. Masking makes it impossible, because it removes the thing to be copied.

9.7.1 The masked objective

Choose a *mask* $\mathbf{m}$: an array of zeros and ones with one entry per position, where $m = 1$ means "visible" and $m = 0$ means "hidden". Feed the model only the visible part, and grade it only on the hidden part:

$$\mathcal{L}_{\text{mask}}(\theta) = \frac{\sum_p (1 - m_p)\,|x_p - f_\theta(\mathbf{x} \odot \mathbf{m})_p|}{\sum_p (1 - m_p)}. \tag{9.12}$$

Here p runs over positions, $\odot$ means multiply entry by entry (so $\mathbf{x} \odot \mathbf{m}$ zeroes the hidden entries), and the factor $(1 - m_p)$ is 1 at hidden positions and 0 at visible ones, so only hidden positions count. The denominator counts the hidden positions, which turns the sum into an average.

Try the identity again on this objective. At a hidden position p the identity outputs $(\mathbf{x} \odot \mathbf{m})_p = x_p \cdot 0 = 0$, so its error there is $|x_p - 0| = |x_p|$. Its loss is the average of $|x_p|$ over hidden positions, which is not zero for any real image. Copying no longer wins. The model only scores well by predicting x_p from the visible positions, which is exactly the context prediction c of Equation (9.9) with $w = 0$.

> Masking changes the question from "reproduce what you see" to "predict what you cannot see", and the identity has no answer to the second question.

9.7.2 What the model learns to predict

Section 9.6 showed that the best single guess under L1 is the median of the values that occur behind a given context, and under L2 the mean. A masked model trained on ordinary terrain therefore learns to output, at each hidden position, the *typical* value given the visible surroundings. At an ordinary pixel that typical value is close to the truth and the residual is small. At an anomaly it is the background that "ought" to be there, and the residual is the honest surprise $x - c$ of Equation (9.10), at full size because $w = 0$.

9.7.3 Hide blocks, not single pixels

What exactly gets hidden matters. Hide a single pixel and its eight immediate neighbours almost give it away: on smooth terrain the average of the neighbours is nearly exact. Look back at worked example 9.2: on the ramp $1.0, 1.1, 1.2$ the neighbour average predicts pixel 2 perfectly with no understanding of anything. A model trained that way learns interpolation and nothing more.

Indradhanu hides *tokens* instead: square blocks of 4×4 pixels (Chapter 13 builds them). A hidden pixel in the middle of a hidden block has its nearest visible pixel up to two pixels away, and the model must reason from further out: what this field's texture is like, which way the river bends. That is the understanding you want it to acquire.

9.7.4 The masked autoencoder in five steps

The technique is called a *masked autoencoder*, MAE [11]. It is the "mae" in the model name `hyperspectral_segformer_mae`. One training step:

1. Cut the patch into tokens (4×4 blocks).
2. Choose a random subset of tokens to hide; in Indradhanu's training configuration about 65% [M: `configs/hyperspectral_segformer_exp_2.json`].
3. Delete the hidden tokens, so the network processes only the survivors.
4. Ask the network for the whole patch, hidden parts included.
5. Grade it only at the hidden positions, as in Equation (9.12).

Step 5 is what makes the exam honest: the model is marked only on things it could not see.

Values that will drift

Mask ratio: `"mask_ratio":0.65` in `configs/hyperspectral_segformer_exp_2.json` (the v0.2.0 training config, batch size 128, 200 epochs), while the code default in `app/models/training/training_config.py` is 0.5. Checked 2026-10-29.

Figure 9.2 shows the difference between the naive and the masked scheme on one small grid.

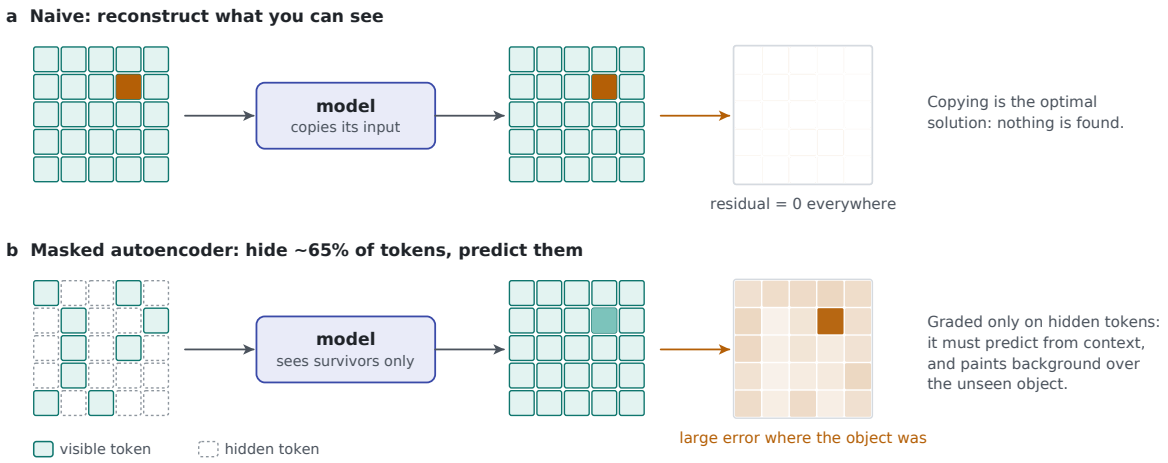


Figure 9.2: Why the model must not see what it predicts. (a) Asked to reproduce what it can see, a model learns to copy; the residual is zero everywhere and nothing is detected. (b) A masked autoencoder hides about 65% of the tokens (dashed) and is graded only on them; it must predict from context, so at the hidden anomaly (amber) it paints ordinary background and leaves a large error. (Course figure IW 6.1.)

9.7.5 Deleting is stronger than zeroing

There are two ways to hide a token. You can set its values to zero and leave it in place, or you can delete it from the list of tokens the network processes. The older convolutional masked models zero; the SegFormer models (Chakshu, Indradhanu) delete. A zeroed token still occupies a slot and still takes part in every computation that follows; a deleted token does not exist for the encoder, costs no computation, and leaks nothing. This choice forces a design decision on how tokens are made: if two neighbouring tokens shared pixels, deleting one would leave some of its pixels alive inside the other. Chapter 13 shows how stage 1 of the encoder is built so that no pixel belongs to two tokens, and Chapter 18 covers the deletion and restoration machinery in full.

9.7.6 Three channels for the convolutional masked models

The convolutional masked models solve a problem that zeroing creates. A zero in the pixel channel is ambiguous: it might be off-swath (the sensor never looked), cloud-masked (the ground is hidden), or a deliberately hidden target. The first two want no prediction; the third wants one. So `app/foundation_models/components/normalized_masked_spatial_auto_encoder.py` feeds three channels:

validity	input mask	meaning
1	1	valid and visible: use it
1	0	valid but hidden: **predict this**
0	0	invalid: ignore, no prediction expected
0	1	cannot occur

Channel 0 is the pixel value (zero where hidden or invalid), channel 1 is the validity mask, and channel 2 is the input mask. Comparing the two masks tells the network which zeros are holes in the data and which are questions. The SegFormer models carry the same distinction through token-level masks instead (Chapter 18).

9.8 Inference: every pixel predicted while hidden

Training only needs *some* hidden positions per step: over thousands of patches, every kind of position gets hidden eventually. Inference is stricter. You need a score at *every* pixel of one particular scene, and each must come from a prediction made while that pixel was hidden. One pass cannot do it: hide half and you score half.

The answer is the two-pass idea of Section 9.4.2, in two dimensions. Choose a token mask $\mathbf{m}^{(1)}$ for pass 1 and its complement $\mathbf{m}^{(2)} = 1 - \mathbf{m}^{(1)}$ for pass 2. Let $\mathbf{p}^{(1)} = 1 - \mathbf{m}^{(1)}$ mark the pixels hidden in pass 1 and $\mathbf{p}^{(2)} = 1 - \mathbf{m}^{(2)}$ the pixels hidden in pass 2. Run the model twice and combine:

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^{(1)} \odot \mathbf{p}^{(1)} + \hat{\mathbf{x}}^{(2)} \odot \mathbf{p}^{(2)}. \tag{9.13}$$

This is a *selection*, not an average. To see it, compute the two masks' sum and product at any pixel:

$$\begin{aligned} p^{(1)} + p^{(2)} &= (1 - m^{(1)}) + (1 - m^{(2)}) && \text{definitions} \\ &= (1 - m^{(1)}) + m^{(1)} = 1, && \text{since } 1 - m^{(2)} = m^{(1)} \\ p^{(1)}\, p^{(2)} &= (1 - m^{(1)})\, m^{(1)} = 0. && \text{one factor is always 0 for a 0/1 value} \end{aligned}$$

So at each pixel exactly one of $p^{(1)}, p^{(2)}$ is 1 and the other is 0, and Equation (9.13) picks $\hat{x}^{(1)}$ or $\hat{x}^{(2)}$ whole. The line in the source is

```
reconstruction = x_hat_1 * pass1_pixels + x_hat_2 * pass2_pixels
```

in method `infer` of `app/foundation_models/inferencers/segformer_mae_inferencer.py` (line 228 on 2026-10-29 [M: `app/foundation_models/inferencers/segformer_mae_inferencer.py`]).

Worked example 9.5: Tracing a checkerboard through two passes

Take a 4×4 corner of the token grid with a checkerboard mask. 1 means visible and 0

means hidden; position (i, j) is row i, column j, counted from 0.

$$\mathbf{m}^{(1)} = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}, \qquad \mathbf{m}^{(2)} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

Position $(0, 0)$: $m^{(1)} = 1$ (visible in pass 1), $m^{(2)} = 0$ (hidden in pass 2). So $p^{(1)} = 0$, $p^{(2)} = 1$, and the final value comes from pass 2.
Position $(0, 1)$: $m^{(1)} = 0$, so $p^{(1)} = 1$ and the value comes from pass 1.
Position $(1, 1)$: $m^{(1)} = 1$, $m^{(2)} = 0$, so pass 2.
Position $(2, 3)$: $m^{(1)} = 0$, $m^{(2)} = 1$, so pass 1.
A pattern appears: the value comes from pass 1 exactly when $i + j$ is odd. Every position is hidden in exactly one pass.

Source check: inference masking and a comment that disagrees with training

Two details in the inference path differ from training and are recorded in the known-issues register. (1) `InferenceConfig.masking_strategy` defaults to `"checkerboard"` (`app/models/training/inference_config.py`, line 96), a deterministic token-level checkerboard, while training and validation hide a random subset. A `"random"` option with a complementary second pass exists. (2) In `app/foundation_models/inferencers/segformer_mae_inferencer.py`, `MIN_VALID_FRACTION=0.1` (line 284) sits under a comment saying it "matches training: patches with $< 40\%$ valid pixels were discarded". Training uses `MIN_VALID_PIXEL_FRACTION=0.4` (`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, line 34). The source wins: the comment is wrong about the number.

9.9 Where the rule is enforced, and the whole model on one page

The rule "a pixel never helps reconstruct itself" is enforced in three separate places in Indradhanu. Learn all three; Equation (9.10) says what happens if any one leaks.

where	mechanism	chapter
encoder input	hidden tokens are deleted from the sequence	Chapter 18
inference	two complementary passes, Equation (9.13)	Chapter 30
the grade	the loss counts only hidden positions, Equation (9.12)	Chapter 25

Indradhanu is a *SegFormer* [46]: a hierarchical vision transformer that looks at the patch at four scales ($\frac{1}{4}$, $\frac{1}{8}$, $\frac{1}{16}$ and $\frac{1}{32}$ of the input size) and a light decoder that fuses the four back into a full-resolution picture. Transformers use *attention*, which lets any token consult any other (Chapter 14); that matters when a whole region is hidden and the nearest evidence is far away. The hyperspectral version adds exactly two things to its thermal sibling Chakshu: a learned spectral compressor that squeezes 165 bands into 32 and expands them back (Chapter 12), and a spectral-angle term in the loss (Chapter 25). Figure 9.3 transcribes its `forward` method from `app/foundation_models/components/hyperspectral_seg_former_mae.py`.

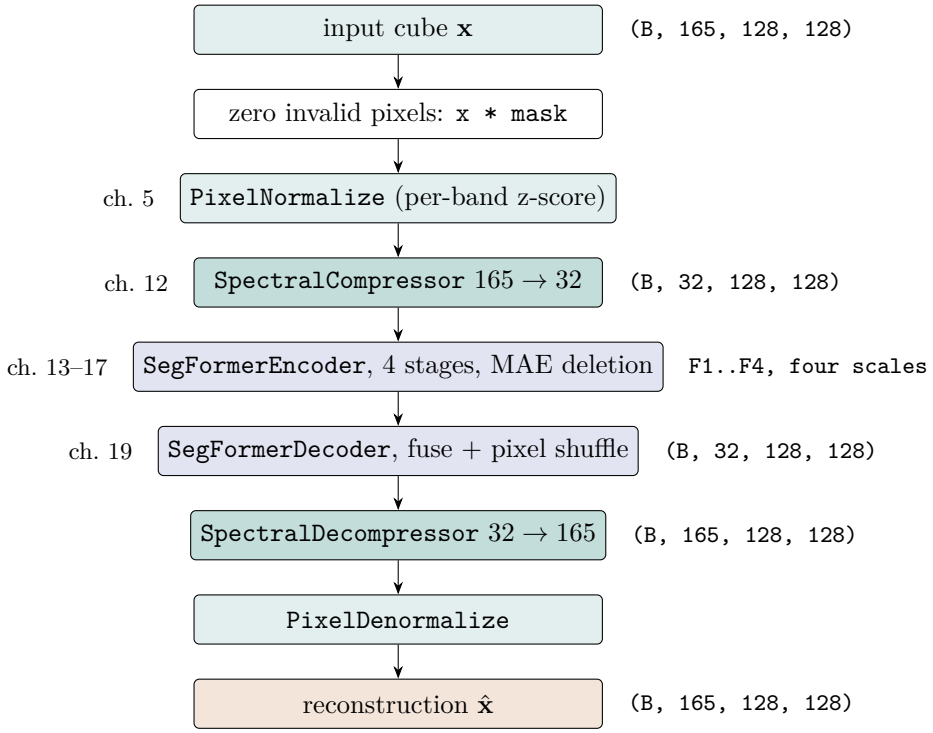


Figure 9.3: The Indradhanu forward pass, transcribed from `HyperspectralSegFormerMAE.forward`. It starts and ends at the same shape, so the reconstruction can be subtracted from the input. The rest of Part III zooms into one box at a time.

Table 9.1: The reconstruction-model roster. The backend selects models by codename, not by class name; the first three share one class and differ only in training (from course part AD10, slugs checked against `FoundationModelName` in `app/models/training/training_config.py`).

codename	slug	notes
Pratibimba	`spatial_autoencoder`	the convolutional baseline
Antardhana	`spatial_masked_autoencoder`	masked, L2 loss
Tirohita	`spatial_masked_autoencoder_l1`	masked, L1 loss
Asanskrita	`spatial_masked_autoencoder_l1_unnormalized`	works in raw Celsius
Drashta	`normalized_masked_autoencoder`	Asanskrita plus normalisation
Chakshu	`segformer_mae`	transformer, thermal
Indradhanu	`hyperspectral_segformer_mae`	transformer, 165 bands

Values that will drift

Shipped figures quoted by the anomaly-detection course (AD10): Chakshu, 406,500 parameters, validation loss 0.2565 at epoch 495 [M: docs / training / anomaly - detection / 10 - reconstruction-models.md]; Indradhanu v0.2.0, "5,507,354 parameters", validation loss 0.0435 at epoch 200 [M: research/model_break_down/05_hyperspectral_segformer_mae.md]. The current product model is Indradhanu V2 (Chapter 39).

Source check: 5,507,354 is not the parameter count

Building `HyperspectralSegFormerMAE(compressed_channels=32)` and summing `p.numel()` over `parameters()` gives **5,506,629** [C], which is also the `torchinfo` total in `research/model_break_down/05_hyperspectral_segformer_mae.md`. Summing over `state_dict()` instead gives 5,507,354 [C]. The difference, 725, is the non-trainable *buffers*: the normalisation mean and standard deviation ($2 \times 165 \times 2 = 660$, once each in `PixelNormalize` and `PixelDenormalize`), the compressor BatchNorm's running mean and variance ($2 \times 32 = 64$) and its batch counter (1): $660 + 64 + 1 = 725$. Chapter 12 explains the BatchNorm buffers. Checked 2026-10-29 with torch 2.5.1 by `tools/compute_recon.py` (output saved in `tools/compute_recon_output.txt`).

A last remark about what a healthy reconstruction looks like. Indradhanu's reconstructions are somewhat blurry. That is expected: they are predictions made with most of the evidence removed. A crisp, perfect reconstruction would be the warning sign, because it would mean the model had found a way to copy.

9.10 Common confusions

Is a masked autoencoder a different thing from an autoencoder?
It is an autoencoder with one extra rule: part of the input is removed before the encoder and the grade is taken only there. The bottleneck alone makes copying hard; the mask makes it impossible.

The bottleneck has more values than the input. Is it compression at all?
Not in value count: the convolutional autoencoder's bottleneck holds 32,768 values for a 16,384-value input. The constraint is that one set of parameters must serve every patch, which pushes the network towards reusable structure. That is a pressure, not a guarantee against copying.

Does the model know which pixels were hidden?
It knows that something is missing at those positions: after the deleted tokens are put back, their slots hold zeros, and the convolutional models carry an explicit input-mask channel. It does not know what was there, which is the point.

If the reconstruction is blurry, is the model broken?
No. A prediction made with about two thirds of the evidence removed is expected to be smooth. A perfectly sharp reconstruction would suggest leakage.

Why not train the model to output an anomaly map directly?
That needs labelled anomalies, which do not exist in useful numbers (Chapter 1). Reconstruction needs only ordinary imagery.

Is the two-pass combination an average of two reconstructions?
No. The two pixel masks are complements, so at each pixel one weight is 1 and the other 0: it selects the pass in which the pixel was hidden.

9.11 Check yourself

Exercise 9.1. Explain in two sentences why a model trained to reproduce its visible input can never detect an anomaly, and why that failure is dangerous rather than merely unhelpful.

Exercise 9.2. A model reconstructs each pixel as $\hat{x} = 0.25\,x + 0.75\,c$, where c is a prediction from the neighbours. For a pixel with $x = 0.80$ and $c = 0.20$, compute the residual directly, then again with Equation (9.10). What fraction of the honest surprise is reported?

Exercise 9.3. Repeat worked example 9.2 for the signal $\mathbf{x} = (2.0,\ 2.0,\ 2.0,\ 2.0,\ 0.4,\ 2.0)$ with the leave-one-out neighbour average. Which pixel scores highest, and which pixels form the halo?

Exercise 9.4. Four training values share a context: 0.9, 1.0, 1.1 and 6.0. Find the guess that minimises the average L2 loss and the guess that minimises the average L1 loss (any value between the two middle values minimises L1; give the midpoint). Which one is pulled towards the anomaly, and by how much compared with the other?

Exercise 9.5. In the 4×4 checkerboard of worked example 9.5, which pass supplies the final value at positions $(3, 0)$ and $(3, 3)$? Then show, using the masks' sum and product, that Equation (9.13) cannot mix the two passes at any pixel.

Exercise 9.6. Name the three places Indradhanu enforces "never see yourself", and say for each what would leak if it were removed.

Next: Chapter 10, Tensors, shapes, layers and the arithmetic of convolutions.

Chapter 10

Tensors, shapes, layers and the arithmetic of convolutions

The one thing this chapter teaches

A shape such as `(B, N, C)` is a precise statement about how numbers are arranged, and the three layer types this model is built from (the linear layer, the convolution and the 1×1 convolution) each change shapes and own parameters by formulas you can work out by hand before the computer tells you.

You are about to read a model whose every line changes the shape of a tensor. Most bugs in tensor code are shape bugs, and the worst of them do not crash: a wrong reshape produces a tensor of the right size with its numbers in the wrong places, and training carries on with scrambled data. The defence is to know, for every operation, what shape goes in, what shape comes out, and in what order the numbers sit.

This chapter gives you that knowledge in four pieces. First the vocabulary: tensors, axes, the batch, and the two layouts the model keeps switching between. Second the linear layer. Third the convolution, worked by hand, with the formula for its output size derived by counting and then checked on every stage of Indradhanu's encoder. Fourth the 1×1 convolution, which ties the other two together and is the subject of Chapter 12. Throughout, you will predict parameter counts by hand and check them against the real model.

Analogy

Think of a tensor as a warehouse. The numbers are boxes laid out along one very long aisle (computer memory is one-dimensional). The *shape* is the warehouse's filing system: "aisle positions 0–15 are channel 0, 16–31 are channel 1", and within a channel "four rows of four". Two warehouses can hold exactly the same boxes in exactly the same aisle order and still mean different things if their filing systems differ. A *reshape* changes the filing system without moving a box. A *transpose* changes which boxes the filing system points at. If you relabel the aisle when you needed to move the boxes, every box is filed under the wrong name, and nothing tells you.

10.1 Tensors and how to read a shape

Definition 10.1 (Tensor, axis, shape)**.** A *tensor* is a rectangular block of numbers. Each direction along which you can index it is an *axis* (also called a dimension), and the list of the lengths of its axes is its *shape*. The number of axes is the tensor's *rank*.

rank	example shape	what it is
0	`()`	a single number
1	`(5,)`	a list of five numbers
2	`(3, 4)`	a grid, three rows by four columns
3	`(165, 128, 128)`	a stack of 165 grids: one hyperspectral patch
4	`(8, 165, 128, 128)`	eight such patches

Read a shape left to right as nested containers. `(8, 165, 128, 128)` is "eight items, each containing 165 layers, each of which is 128 rows by 128 columns". The number of values in a tensor is the product of its shape: $8 \times 165 \times 128 \times 128 = 21{,}626{,}880$.

10.1.1 The batch

Neural networks process several examples at once, because a graphics processor is far more efficient on 128 patches together than on one patch 128 times. That group is a *batch*, and its size is written B. Almost every tensor in the model has B as its first axis, and almost every operation treats the B items independently. When you read the code you can usually ignore B and think about one patch.

10.1.2 The symbols used in this book

symbol	meaning	typical value
B	batch size	128 in training*
C	input spectral bands	165
D	channels after the spectral compressor	32 (24 in v0.1.0)
H, W	height and width in pixels	128, 128
N	number of tokens at a stage	1024 at stage 1
C_i	token width at encoder stage i	32, 64, 160, 256
k, s, p	kernel, stride, padding of a convolution	Section 10.4

*Batch size 128 is the v0.2.0 training configuration [M: `configs/hyperspectral_segformer_exp_2.json`]; the code default is 32.

Watch the two meanings of "channels". $C = 165$ is the number of input bands. C_i is the width of the vectors inside encoder stage i. They are unrelated numbers that both get called channels.

10.1.3 Where the numbers live: row-major order

Computer memory is a single line of numbers. A tensor of shape `(B, C, H, W)` is stored in *row-major order*: the last axis changes fastest. Walking along memory, you pass W values of row 0 of channel 0, then row 1 of channel 0, and so on through all H rows; then the same for channel 1; and after all C channels, the next batch item.

You can derive the memory position (the *offset*) of element (b, c, h, w), counting everything from 0, by counting how many values come before it:

$$\begin{aligned} \text{offset}(b,c,h,w) &= b \cdot (CHW) + c \cdot (HW) + h \cdot W + w && \text{items, channels, rows, columns before it} \\ &= \big((b\,C + c)\,H + h\big)\,W + w. && \text{factor out } W\text{, then } H \end{aligned}$$

The numbers CHW, HW, W and 1 multiplying the indices are called the tensor's *strides* (not to be confused with a convolution's stride in Section 10.4). They say how far apart in memory two neighbours along each axis are.

Worked example 10.1: Finding a value in memory

Take a tensor of shape `(1, 2, 2, 2)`: one item, two channels, each a 2×2 grid. Where is element $(b, c, h, w) = (0, 1, 0, 1)$, that is channel 1, row 0, column 1? The strides are $CHW = 2 \cdot 2 \cdot 2 = 8$, $HW = 2 \cdot 2 = 4$, $W = 2$ and 1. So

$$\begin{aligned} \text{offset} &= 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 \\ &= 0 + 4 + 0 + 1 \\ &= 5. \end{aligned}$$

It is the sixth value in memory (offset 5, counting from 0): four values of channel 0 come first, then channel 1's row 0, column 0 at offset 4, then our element.

10.2 Two layouts of the same numbers

The model keeps its data in one of two arrangements.

layout	shape	used by
spatial	`(B, C, H, W)`	convolutions, pixel shuffle, the loss
sequence	`(B, N, C)`	attention, LayerNorm, linear layers

Spatial says "a picture with C layers". Sequence says "a list of N *tokens*, each described by C numbers". A token here is one spatial position's vector of channel values; when you convert spatial to sequence, $N = H \times W$. The conversion is written in the code as

```
x = x.flatten(2)         # (B, C, H, W) -> (B, C, H*W) = (B, C, N)
x = x.transpose(1, 2)    # (B, C, N)    -> (B, N, C)
```

`flatten(2)` merges every axis from axis 2 onwards into one: the two spatial axes become a single list of N positions. `transpose(1,2)` swaps axes 1 and 2. The way back is

```
x = x.transpose(1, 2).reshape(B, C, H, W)   # (B, N, C) -> (B, C, N) -> (B, C, H, W)
```

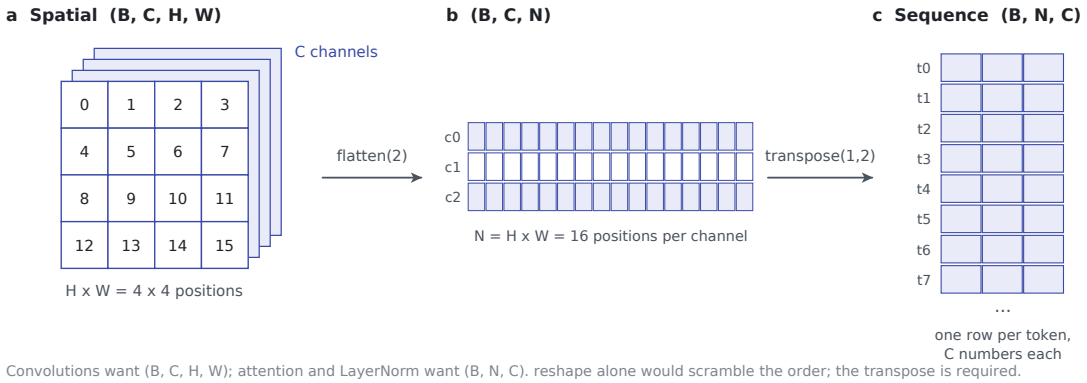


Figure 10.1: The two layouts of the same numbers. (a) Spatial `(B, C, H, W)`: a grid of positions per channel, as convolutions need. (b) `flatten(2)` joins the two spatial axes into $N = H \times W$ positions. (c) `transpose(1, 2)` makes each position a row of C numbers, a token, as attention and LayerNorm need. A reshape alone would scramble the order. (Course figure IW 7.1.)

10.2.1 Why a reshape alone scrambles the data

A `reshape` changes the shape but keeps the numbers in the same memory order. The two layouts need *different* memory orders: spatial wants all of channel 0's pixels, then all of channel 1's; sequence wants token 0's channel values, then token 1's. So an axis swap is unavoidable. The smallest example that shows this has two channels and four pixels.

Worked example 10.2: A $2 \times 2 \times 2$ cube through flatten, transpose and a wrong reshape

Take $B = 1$, $C = 2$, $H = W = 2$, holding the numbers 1 to 8 in memory order:

$$\text{channel } 0 = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \qquad \text{channel } 1 = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}, \qquad \text{memory: } 1\ 2\ 3\ 4\ 5\ 6\ 7\ 8.$$

The four pixels are: top-left (channel values 1 and 5), top-right (2, 6), bottom-left (3, 7) and bottom-right (4, 8). The correct tokens are therefore $[1, 5]$, $[2, 6]$, $[3, 7]$, $[4, 8]$.
Step 1, flatten(2). Shape `(1, 2, 2, 2)` becomes `(1, 2, 4)`. Each channel's 2×2 grid is read row by row into a list of four:

$$\begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \end{pmatrix}.$$

Memory has not moved: still 1 2 3 4 5 6 7 8.
Step 2, transpose(1, 2). Shape `(1, 2, 4)` becomes `(1, 4, 2)`: rows and columns swap.

$$\begin{pmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{pmatrix}.$$

Row n is token n, and each token holds its pixel's two channel values. Correct. Laid out contiguously, memory now reads 1 5 2 6 3 7 4 8: the boxes have moved.
The wrong way, reshape(1, 4, 2) straight from the cube. A reshape reads memory in order, 1 2 3 4 5 6 7 8, and cuts it into four rows of two:

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \\ 7 & 8 \end{pmatrix}.$$

The "token" $[1, 2]$ is two different pixels' channel-0 values. Every token is wrong, the shape is right, and no error is raised.

The source carries the same warning, in `app/foundation_models/components/overlap_patch_embedding.py`: "Cannot use view/reshape directly because the axes need to swap".

10.2.2 Broadcasting

When two tensors of different shapes are multiplied, numpy and torch stretch any axis of length 1 to match the other tensor. This is *broadcasting*:

$$\underbrace{\texttt{(B, 165, H, W)}}_{\text{the cube}} \times \underbrace{\texttt{(B, 1, H, W)}}_{\text{a pixel mask}} \to \texttt{(B, 165, H, W)}.$$

The mask's single channel is reused for all 165 bands. That is why pixel masks in this code have shape `(B, 1, H, W)` and not `(B, H, W)`: the 1 is a placeholder in the channel slot, so the axes line up. In the same way the per-band normalisation statistics are stored with shape `(1, 165, 1, 1)` (`app/foundation_models/components/pixel_normalization.py`) so that one mean per band is reused at every pixel of every patch.

10.3 The linear layer

10.3.1 One neuron

A *neuron* takes a vector $\mathbf{x} \in \mathbb{R}^n$, multiplies each entry by its own *weight*, adds the products, and adds a *bias*:

$$y = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b = \mathbf{w}^\top \mathbf{x} + b. \tag{10.1}$$

The sum of products $\mathbf{w}^\top \mathbf{x}$ is the dot product (Chapter 6). A neuron has n weights and one bias: $n + 1$ parameters. In most networks a nonlinear function (an *activation*, such as GELU in Chapter 16) is applied to y afterwards; here we count only the linear part, because that is where the parameters are.

10.3.2 A layer of neurons is a matrix

Put m neurons side by side, all reading the same $\mathbf{x}$. Stack their weight vectors as the rows of a matrix $\mathbf{W} \in \mathbb{R}^{m \times n}$ and their biases into $\mathbf{b} \in \mathbb{R}^m$:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad y_i = \sum_{j=1}^{n} W_{ij}\, x_j + b_i \quad (i = 1, \ldots, m). \tag{10.2}$$

This is a *linear layer* (PyTorch `nn.Linear(n,m)`). Output i is a weighted mixture of all the inputs, with its own recipe of weights, row i of $\mathbf{W}$. Counting:

$$\text{parameters} = \underbrace{m \cdot n}_{\text{entries of } \mathbf{W}} + \underbrace{m}_{\text{entries of } \mathbf{b}} = \text{in} \times \text{out} + \text{out}. \tag{10.3}$$

Worked example 10.3: A linear layer from three inputs to two outputs

Let $n = 3$, $m = 2$,

$$\mathbf{W} = \begin{pmatrix} 1.0 & 0.0 & 0.0 \\ 0.0 & 0.5 & 0.5 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 0.0 \\ 0.1 \end{pmatrix}, \quad \mathbf{x} = \begin{pmatrix} 0.10 \\ 0.20 \\ 0.30 \end{pmatrix}.$$

Row 1 of $\mathbf{W}$ is $(1.0, 0.0, 0.0)$:

$$\begin{aligned} y_1 &= 1.0 \times 0.10 + 0.0 \times 0.20 + 0.0 \times 0.30 + 0.0 \\ &= 0.10 + 0 + 0 + 0 = 0.10. \end{aligned}$$

Row 2 is $(0.0, 0.5, 0.5)$:

$$\begin{aligned} y_2 &= 0.0 \times 0.10 + 0.5 \times 0.20 + 0.5 \times 0.30 + 0.1 \\ &= 0 + 0.10 + 0.15 + 0.1 = 0.35. \end{aligned}$$

So $\mathbf{y} = (0.10,\ 0.35)^\top$. Output 1 copied input 1; output 2 averaged inputs 2 and 3 and added an offset. Parameter count: $3 \times 2 + 2 = 8$, which is what PyTorch reports for `nn.Linear(3,2)` [C].

10.3.3 Applied to a sequence

`nn.Linear` acts on the *last* axis of its input and treats all other axes as a batch. Given tokens of shape `(B, N, C_in)` it applies the same $\mathbf{W}$ and $\mathbf{b}$ to every one of the $B \cdot N$ tokens and returns `(B, N, C_out)`. That is why linear layers live in the sequence layout: the last axis must be the feature axis.

10.3.4 Why not one linear layer for the whole image?

You could flatten an entire patch into one vector and apply one linear layer to it. Count the cost of mapping a patch to a patch of the same size: the input has $165 \times 128 \times 128 = 2{,}703{,}360$ values, so

$$\text{weights} = 2{,}703{,}360 \times 2{,}703{,}360 \approx 7.3 \times 10^{12}\,[C].$$

Seven trillion parameters, for one layer. Worse, such a layer treats a field in the top-left corner and the same field in the bottom-right as unrelated problems. The convolution fixes both faults.

10.4 The convolution

10.4.1 Two ideas: locality and sharing

A convolution builds each output value from a small neighbourhood of the input (*locality*), and it uses the same weights at every position (*weight sharing*). The small block of weights is the *kernel*; its side length is the *kernel size* k. The kernel slides over the input; at each position it multiplies the values under it by its weights and adds them up.

10.4.2 Single channel, written as a sum

Definition 10.2 (2-D cross-correlation)**.** For a single-channel input x (an $H \times W$ grid) and a $k \times k$ kernel K with bias b, the output at row i, column j (counting from 0) is

$$y_{i,j} = b + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v}\; x_{i+u,\; j+v}. \tag{10.4}$$

Deep-learning libraries call this a *convolution*. Strictly it is a *cross-correlation*: the mathematical convolution flips the kernel first ($K_{k-1-u,\,k-1-v}$). Because the kernel is learned, the flip makes no difference to what a network can represent, and this book follows the library usage.

Read Equation (10.4) as "place the kernel's top-left corner on input position (i, j), multiply the overlapping pairs, add". With an $H \times W$ input and no padding the corner can go at rows 0 to $H - k$ and columns 0 to $W - k$, so the output is $(H - k + 1) \times (W - k + 1)$.

Worked example 10.4: A 3×3 input and a 2×2 kernel, by hand

Do this on paper. Input and kernel, bias $b = 0$:

$$x = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{pmatrix}, \qquad K = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}.$$

The output is $(3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$. Each entry pairs K with the 2×2 block of x whose top-left corner is at (i, j).

$(i, j) = (0, 0)$, block $\begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}$:

$$\begin{aligned} y_{0,0} &= 1 \times 1 + 0 \times 2 + (-1) \times 0 + 2 \times 1 \\ &= 1 + 0 + 0 + 2 = 3. \end{aligned}$$

$(0, 1)$, block $\begin{pmatrix} 2 & 0 \\ 1 & 3 \end{pmatrix}$:

$$\begin{aligned} y_{0,1} &= 1 \times 2 + 0 \times 0 + (-1) \times 1 + 2 \times 3 \\ &= 2 + 0 - 1 + 6 = 7. \end{aligned}$$

$(1, 0)$, block $\begin{pmatrix} 0 & 1 \\ 2 & 1 \end{pmatrix}$:

$$\begin{aligned} y_{1,0} &= 1 \times 0 + 0 \times 1 + (-1) \times 2 + 2 \times 1 \\ &= 0 + 0 - 2 + 2 = 0. \end{aligned}$$

$(1, 1)$, block $\begin{pmatrix} 1 & 3 \\ 1 & 1 \end{pmatrix}$:

$$\begin{aligned} y_{1,1} &= 1 \times 1 + 0 \times 3 + (-1) \times 1 + 2 \times 1 \\ &= 1 + 0 - 1 + 2 = 2. \end{aligned}$$

So

$$y = \begin{pmatrix} 3 & 7 \\ 0 & 2 \end{pmatrix}.$$

PyTorch's `F.conv2d` gives the same four numbers [C]. This kernel responds most to "top-left and bottom-right bright, bottom-left dark", and its four weights are used at every position.

10.4.3 Stride and padding

Two more settings control where the kernel goes.

Definition 10.3 (Stride and padding)**.** The *stride* s is how far the kernel jumps between positions: with $s = 2$ it is placed at every second row and every second column. The *padding* p is a border of p zeros added on every side of the input before the kernel slides.

With stride and padding, Equation (10.4) becomes

$$y_{i,j} = b + \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{u,v}\, \tilde{x}_{si+u,\; sj+v}, \tag{10.5}$$

where $\tilde{x}$ is the zero-padded input (so $\tilde{x}_{a,b'} = x_{a-p,\, b'-p}$ inside the original picture and 0 in the border). The kernel's top-left corner sits at padded position (si, sj).

10.4.4 Many channels in, many channels out

Real inputs have C_{in} channels, and a layer produces C_{out} output channels. Each output channel o has its own kernel for every input channel, so the weights form a four-axis tensor

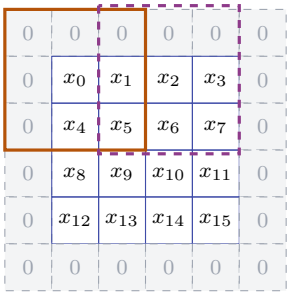


Figure 10.2: A 3×3 kernel sliding with stride 2 over a 4×4 input padded by one ring of zeros, the setting of encoder stages 2 to 4. Consecutive windows share a column (the overlap is $k - s = 1$). The corner windows see only 4 real pixels; the bottom-right one sees 9.

$K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$, and each output channel has one bias:

$$y_{o,i,j} = b_o + \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{k-1} \sum_{v=0}^{k-1} K_{o,c,u,v}\, \tilde{x}_{c,\ si+u,\ sj+v}. \tag{10.6}$$

In words: output channel o at position (i, j) looks at a $k \times k$ window in *every* input channel, and mixes all $C_{\text{in}} \cdot k \cdot k$ values with its own weights. The parameter count is

$$\text{parameters} = C_{\text{in}} \cdot C_{\text{out}} \cdot k \cdot k + C_{\text{out}}. \tag{10.7}$$

For example `nn.Conv2d(3,2,kernel_size=2)` has $3 \cdot 2 \cdot 2 \cdot 2 + 2 = 24 + 2 = 26$ parameters [C]. Notice what is *absent* from Equation (10.7): H and W. Weight sharing makes the cost independent of the picture size.

10.5 The output-size formula, derived by counting

You will use this formula more than any other in Part III, so derive it rather than memorise it.

Work along one axis; the other is the same. After padding, the axis has length

$$L = H + 2p.$$

The kernel's first cell sits at positions $0, s, 2s, 3s, \ldots$, that is at ns for $n = 0, 1, 2, \ldots$. Placed at ns, the kernel covers positions ns to $ns + k - 1$. It must fit inside the padded axis, so its last cell must be at most $L - 1$:

$$\begin{aligned} ns + k - 1 &\le L - 1 && \text{the last covered cell is inside} \\ ns &\le L - k && \text{add } 1 - k \text{ to both sides} \\ n &\le \frac{L - k}{s}. && \text{divide by } s > 0 \end{aligned}$$

Since n is a whole number, the largest allowed n is $\lfloor (L - k)/s \rfloor$, where $\lfloor \cdot \rfloor$ (the *floor*) means "round down to a whole number". The allowed values are $n = 0, 1, \ldots, \lfloor (L - k)/s \rfloor$; counting the 0 as well, there are

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1 \tag{10.8}$$

of them. Each kernel position gives one output value, so this is the output size.

Worked example 10.5: Every stage of Indradhanu's encoder

The encoder builds its four patch embeddings in `app/foundation_models/components/seg_former_encoder.py` (lines 110–114): stage 1 has $k = 4$, $s = 4$, $p = 0$; stages 2 to 4 have $k = 3$, $s = 2$, $p = 1$. Input $H = 128$.
Stage 1:

$$\begin{aligned} H + 2p - k &= 128 + 0 - 4 = 124, \\ 124/4 &= 31, \quad \lfloor 31 \rfloor = 31, \\ H_{\text{out}} &= 31 + 1 = 32. \end{aligned}$$

Stage 2 ($H = 32$):

$$\begin{aligned} 32 + 2 - 3 &= 31, \\ 31/2 &= 15.5, \quad \lfloor 15.5 \rfloor = 15, \\ H_{\text{out}} &= 15 + 1 = 16. \end{aligned}$$

Stage 3 ($H = 16$):

$$\begin{aligned} 16 + 2 - 3 &= 15, \\ 15/2 &= 7.5, \quad \lfloor 7.5 \rfloor = 7, \\ H_{\text{out}} &= 7 + 1 = 8. \end{aligned}$$

Stage 4 ($H = 8$):

$$\begin{aligned} 8 + 2 - 3 &= 7, \\ 7/2 &= 3.5, \quad \lfloor 3.5 \rfloor = 3, \\ H_{\text{out}} &= 3 + 1 = 4. \end{aligned}$$

The grids are $32, 16, 8, 4$ on a side, so the token counts are $32^2 = 1024$, $16^2 = 256$, $8^2 = 64$ and $4^2 = 16$. Running the real encoder on a `(1, 32, 128, 128)` input returns feature maps `(1, 32, 32, 32)`, `(1, 64, 16, 16)`, `(1, 160, 8, 8)` and `(1, 256, 4, 4)` [C].

Two small facts drop out of Equation (10.8). When $k = s$ and $p = 0$, it gives $\lfloor (H-k)/k \rfloor + 1 = \lfloor H/k \rfloor$: the windows tile the axis. When $k = 3$, $s = 2$, $p = 1$, it gives $\lfloor (H-1)/2 \rfloor + 1$, which equals $H/2$ when H is even.

Worked example 10.6: Why stage 1 must set its padding explicitly

`OverlapPatchEmbedding` defaults to padding $p = \lfloor k/2 \rfloor$ when none is given. Had stage 1 used that default with $k = 4$, the padding would be 2:

$$\begin{aligned} 128 + 2 \times 2 - 4 &= 128 + 4 - 4 = 128, \\ 128/4 &= 32, \\ H_{\text{out}} &= 32 + 1 = 33. \end{aligned}$$

A 33×33 grid of 1089 tokens, windows hanging two pixels over the edge, and token boundaries no longer aligned with the 4×4 blocks that the masking code assumes. The encoder therefore passes `padding = 0 if i == 0 else None`.

Source check: "H_out = H_in // stride" holds only for even sizes

The docstring of `OverlapPatchEmbedding` (`app / foundation _ models / components / overlap_patch_embedding.py`, lines 87–105) states "`H'=H//desired_compression`" and "with padding = patch_size // 2: `H_out=H_in//stride`". Equation (10.8) says otherwise for odd inputs. With $k = 3$, $s = 2$, $p = 1$ and $H = 33$: $\lfloor (33 + 2 - 3)/2 \rfloor + 1 = \lfloor 16 \rfloor + 1 = 17$, while $33//2 = 16$. PyTorch returns 17 [C]. The docstring line 87 also says the convolution uses `padding=patch_size//2`, which is false for stage 1 ($p = 0$). Neither matters for the 128×128 patches the model is trained on, because every stage then receives an even size; the formula is the authority.

Worked example 10.7: The convolutional autoencoder and a 100×100 patch

The old `SpatialEncoder` (Chapter 9) uses $k = 4$, $s = 2$, $p = 1$ at each of three stages. On $H = 128$:

$$\lfloor (128 + 2 - 4)/2 \rfloor + 1 = \lfloor 126/2 \rfloor + 1 = 63 + 1 = 64,$$

an exact halving; then 32, then 16. On $H = 100$:

$$\begin{aligned} \text{stage 1: } & \lfloor (100 + 2 - 4)/2 \rfloor + 1 = \lfloor 98/2 \rfloor + 1 = 49 + 1 = 50, \\ \text{stage 2: } & \lfloor (50 + 2 - 4)/2 \rfloor + 1 = \lfloor 48/2 \rfloor + 1 = 24 + 1 = 25, \\ \text{stage 3: } & \lfloor (25 + 2 - 4)/2 \rfloor + 1 = \lfloor 23/2 \rfloor + 1 = 11 + 1 = 12. \end{aligned}$$

The decoder doubles three times, $12 \to 24 \to 48 \to 96$, and $96 \neq 100$: the output cannot be subtracted from the input. Height and width must be divisible by $2^3 = 8$.

10.6 Receptive field

Definition 10.4 (Receptive field)**.** The *receptive field* of an output value is the set of input pixels that can affect it. For a stack of convolutions it is a square; its side is what this book calls the receptive field size.

One convolution with kernel k has a receptive field of k pixels. Stack layers and it grows. Keep two numbers per layer ℓ: the receptive field r_ℓ and the *jump* J_ℓ, the distance in input pixels between two neighbouring outputs of layer ℓ. Start from the input itself: $r_0 = 1$, $J_0 = 1$.

Layer ℓ has kernel k_ℓ and stride s_ℓ. Its window spans k_ℓ neighbouring outputs of layer $\ell - 1$. Those are $J_{\ell-1}$ input pixels apart, so the window's first and last positions are $(k_\ell - 1)J_{\ell-1}$ input pixels apart, and each of them brings its own receptive field of $r_{\ell-1}$:

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, J_{\ell-1}, \tag{10.9}$$

$$J_\ell = J_{\ell-1}\, s_\ell. \tag{10.10}$$

The jump multiplies because moving one output of layer ℓ moves s_ℓ outputs of layer $\ell - 1$, each $J_{\ell-1}$ pixels.

Worked example 10.8: Receptive field of the four patch embeddings

Stage 1, $k = 4$, $s = 4$:

$$r_1 = 1 + (4 - 1) \times 1 = 1 + 3 = 4, \qquad J_1 = 1 \times 4 = 4.$$

Stage 2, $k = 3$, $s = 2$:

$$r_2 = 4 + (3 - 1) \times 4 = 4 + 8 = 12, \qquad J_2 = 4 \times 2 = 8.$$

Stage 3:

$$r_3 = 12 + 2 \times 8 = 12 + 16 = 28, \qquad J_3 = 8 \times 2 = 16.$$

Stage 4:

$$r_4 = 28 + 2 \times 16 = 28 + 32 = 60, \qquad J_4 = 16 \times 2 = 32.$$

A stage-4 token sits on a 32-pixel grid (J_4) but its patch-embedding path sees a 60×60 window of the input (r_4). Back-propagating from one centre output through four such convolutions with all-one weights touches exactly 4, 12, 28 and 60 input rows [C].

Two cautions. First, these numbers count only the patch-embedding convolutions. The transformer blocks in between contain a 3×3 depthwise convolution (Chapter 16) that widens the field further, and attention (Chapter 14) lets a token read every other token at its stage, which makes the effective receptive field the whole patch. Second, "a stage-4 token covers a 32×32 region" (as the Indradhanu course puts it) describes the jump: the grid cell each token stands for. Its receptive field is wider, because the stride-2 windows of stages 2 to 4 overlap.

10.7 The 1×1 convolution is a linear layer at every pixel

Set $k = 1$, $s = 1$, $p = 0$ in Equation (10.6). The sums over u and v have a single term ($u = v = 0$), the padded input is the input, and the kernel tensor $K_{o,c,0,0}$ is a matrix $W_{o,c}$:

$$\begin{aligned} y_{o,i,j} &= b_o + \sum_c K_{o,c,0,0}\, x_{c,\,i,\,j} && \text{only } u = v = 0 \text{ survives} \\ &= b_o + \sum_c W_{o,c}\, x_{c,i,j}. && \text{rename } K_{o,c,0,0} \text{ as } W_{o,c} \end{aligned} \tag{10.11}$$

Compare with Equation (10.2). At each pixel (i, j), the vector of channel values $\mathbf{x}_{:,i,j}$ is multiplied by the same matrix $\mathbf{W}$ and shifted by the same $\mathbf{b}$:

$$\mathbf{y}_{:,i,j} = \mathbf{W}\,\mathbf{x}_{:,i,j} + \mathbf{b} \qquad \text{for every pixel } (i, j). \tag{10.12}$$

A 1×1 *convolution* is a linear layer applied independently at every pixel. It mixes channels and does no spatial mixing at all: pixel $(3, 7)$ never looks at pixel $(3, 8)$. Its parameter count, from Equation (10.7) with $k = 1$, is $C_{\text{in}} \cdot C_{\text{out}} + C_{\text{out}}$, the same as Equation (10.3).

The equivalence is exact in code too: a `nn.Conv2d(3,2,1)` and a `nn.Linear(3,2)` with the same weights give identical results, one in the spatial layout and one on the flattened tokens (checked numerically [C]).

Chapter 12 uses exactly this: Indradhanu's spectral compressor is a 1×1 convolution from 165 channels to 32.

10.8 Counting parameters: a convolutional encoder by hand

The best test of understanding is to predict a real model's parameter count before asking the computer. Take the convolutional autoencoder's encoder of Chapter 9, `SpatialEncoder(in_channels=1,base_channels=32,num_stages=3)`. Each stage is a `Conv2d` with $k = 4$, $s = 2$, $p = 1$, followed by a `BatchNorm2d`, which has 2 parameters per channel (Chapter 12 explains why). Channels go $1 \to 32 \to 64 \to 128$.

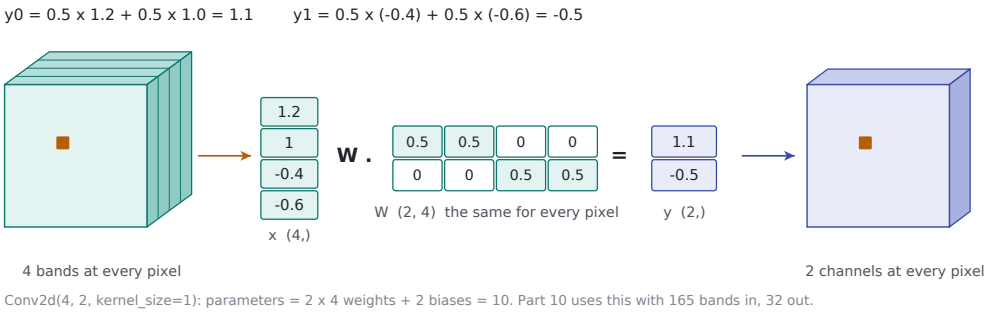


Figure 10.3: A 1×1 convolution is a matrix-vector product per pixel. At every pixel the 4-band vector $\mathbf{x}$ is multiplied by the same 2×4 matrix $\mathbf{W}$, giving two output channels, $y_0 = 1.1$ and $y_1 = -0.5$ (the numbers of worked example 12.1). The spatial grid is untouched; only the channel axis changes. (Course figure IW 8.1.)

Worked example 10.9: The SpatialEncoder's 165,024 parameters

Stage 0, $1 \to 32$:

$$\begin{aligned} \text{conv weights} &= 1 \times 32 \times 4 \times 4 = 32 \times 16 = 512, \\ \text{conv biases} &= 32, \\ \text{BatchNorm} &= 2 \times 32 = 64, \\ \text{stage total} &= 512 + 32 + 64 = 608. \end{aligned}$$

Stage 1, $32 \to 64$:

$$\begin{aligned} \text{conv weights} &= 32 \times 64 \times 16 = 2048 \times 16 = 32{,}768, \\ \text{conv biases} &= 64, \\ \text{BatchNorm} &= 2 \times 64 = 128, \\ \text{stage total} &= 32{,}768 + 64 + 128 = 32{,}960. \end{aligned}$$

Stage 2, $64 \to 128$:

$$\begin{aligned} \text{conv weights} &= 64 \times 128 \times 16 = 8192 \times 16 = 131{,}072, \\ \text{conv biases} &= 128, \\ \text{BatchNorm} &= 2 \times 128 = 256, \\ \text{stage total} &= 131{,}072 + 128 + 256 = 131{,}456. \end{aligned}$$

Total:

$$\begin{aligned} 608 + 32{,}960 &= 33{,}568, \\ 33{,}568 + 131{,}456 &= 165{,}024. \end{aligned}$$

`sum(p.numel() for p in SpatialEncoder(1, 32, 3).parameters())` returns 165,024 [C]. The whole autoencoder with its mirrored decoder has 329,665 [M: `docs/training/anomaly-detection/10-reconstruction-models.md`].

10.9 The whole forward pass as a table of shapes

Table 10.1 lists every shape a training patch passes through in Indradhanu, with $C = 165$, $D = 32$ and $H = W = 128$. You are not expected to understand every row yet. You are expected to know the table exists, to see that the model starts and ends at `(B, 165, 128, 128)`, and to check each patch-embedding row against worked example 10.5.

Table 10.1: Shapes through one Indradhanu forward pass (course part IW07, with stage rows checked against the real encoder by `tools/compute_shapes.py`). "∼358" is the number of visible stage-1 tokens at mask ratio 0.65 ($1024 \times 0.35 \approx 358$).

step	module	output shape	chapter
input		`(B, 165, 128, 128)`	
normalise	`PixelNormalize`	`(B, 165, 128, 128)`	5
compress	`SpectralCompressor`	`(B, 32, 128, 128)`	12
S1 patch embed	$k4, s4, p0$	`(B, 1024, 32)`	13
S1 delete hidden	`remove_tokens`	`(B, ∼358, 32)`	18
S1 blocks ×2	`SegFormerBlock`	`(B, ∼358, 32)`	16
S1 restore, to 2-D		`(B, 32, 32, 32)` = F1	17
S2 embed + blocks	$k3, s2, p1$	`(B, 256, 64)` → F2 `(B, 64, 16, 16)`	17
S3 embed + blocks	$k3, s2, p1$	`(B, 64, 160)` → F3 `(B, 160, 8, 8)`	17
S4 embed + blocks	$k3, s2, p1$	`(B, 16, 256)` → F4 `(B, 256, 4, 4)`	17
decoder fuse	1×1 convs, upsample	`(B, 256, 32, 32)`	19
sub-pixel conv	3×3 conv	`(B, 512, 32, 32)`	19
pixel shuffle	factor 4	`(B, 32, 128, 128)`	19
decompress	`SpectralDecompressor`	`(B, 165, 128, 128)`	12
denormalise	`PixelDenormalize`	`(B, 165, 128, 128)`	5

A habit worth adopting: annotate every line of tensor code with its shape, as the repository does (`docs/10-code-style.md`):

```
x = x.flatten(2)          # x: (B, c_out, N)
x = x.transpose(1, 2)     # x: (B, N, c_out)
```

10.10 Common confusions

Is N the number of pixels?
Only when you flatten a full-resolution map. After a patch embedding N counts tokens: at stage 1 each token covers a 4×4 block, so $N = (H/4)(W/4)$, one sixteenth of the pixel count.

Is a reshape the same as a transpose?
No. A reshape keeps memory order and changes only the filing system; a transpose changes which number sits where. Converting `(B, C, H, W)` to `(B, N, C)` needs the transpose (worked example 10.2).

Is a deep-learning convolution the mathematician's convolution?
It is a cross-correlation: the kernel is not flipped. Since the kernel is learned, nothing is lost.

Does the stride of a tensor have anything to do with the stride of a convolution?
No. A tensor's strides are memory distances between neighbours along each axis; a convolution's stride is how far the kernel jumps. Same word, two meanings.

Does a bigger image need more convolution parameters?
No. Equation (10.7) has no H or W in it: the same kernel is reused at every position.

Is a 1×1 convolution really a convolution?
Formally yes, with a window of one pixel. In practice it is a per-pixel matrix multiply that mixes channels and never looks at neighbours.

10.11 Check yourself

Exercise 10.1. Read `(8, 165, 128, 128)` aloud in plain words. Then convert `(4, 64, 16, 16)` to the sequence layout: what are N and the feature width?

Exercise 10.2. Compute by hand the convolution of $x = \begin{pmatrix} 2 & 0 & 1 \\ 1 & 3 & 0 \\ 0 & 1 & 2 \end{pmatrix}$ with $K = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}$, bias 0, stride 1, no padding.

Exercise 10.3. Use Equation (10.8) to find the output size of a convolution with $k = 7$, $s = 4$, $p = 3$ on $H = 128$ (the stale "kernel 7" of Chapter 13), and of $k = 3$, $s = 2$, $p = 1$ on $H = 31$. Is the second equal to $31//2$?

Exercise 10.4. How many parameters does `nn.Conv2d(64,160,kernel_size=3)` have? And `nn.Conv2d(32,165,kernel_size=1)`?

Exercise 10.5. A stack of three convolutions has $(k, s) = (3, 1)$, $(3, 2)$, $(3, 1)$. Compute the receptive field and jump after each layer.

Exercise 10.6. Starting from the tensor of worked example 10.2 in sequence layout $\begin{pmatrix} 1 & 5 \\ 2 & 6 \\ 3 & 7 \\ 4 & 8 \end{pmatrix}$, apply `transpose(1,2)` then `reshape(1,2,2,2)` and confirm you recover the two channels. What would `reshape(1,2,2,2)` without the transpose give?

Next: Chapter 11, The five convolutional autoencoders.

Chapter 11

The five convolutional autoencoders

The one thing this chapter teaches

Before the transformers, the project built five convolutional autoencoders for one thermal band, Pratibimba, Antardhana, Tirohita, Asanskrita and Drashta: two classes, one stack of stride-2 convolutions and its mirror, between 264,961 and 1,379,393 trainable numbers, which differ in how they hide pixels, which loss they learn from and whether they normalise; each step answered a fault of the one before, and you can count every one of them by hand.

Chapter 9 listed seven codenames; Chapter 10 counted one convolutional encoder. This chapter is the condensed tour of the five convolutional models, one section each: the class, the shapes of one forward pass, the parameter count, why the model exists, and where it sits in the product. It is shorter than its neighbours on purpose. These models are not the product's default for anything (Chakshu is, for thermal scenes, Chapter 21), but they are in the model volume, the scoring dialog offers them, and their history explains the design of everything after them.

Every count and shape tagged [C] comes from `research/textbook/tools/compute_convae.py`, which loads each model's served checkpoint (copied from the stack's `/models/` volume on 2026-10-31) into the real class with a strict load and records shapes with forward hooks; its output is `compute_convae_output.txt`. Paths abbreviate: `components/` is `app/foundation_models/components/`, `trainers/` and `inferencers/` likewise.

Analogy

Five apprentices learn to restore damaged photographs. The first is only ever shown undamaged photographs and asked to copy them. The second is shown photographs with a few holes punched in them, the third with most of the picture punched out. The fourth gets the punched photograph and, on a separate sheet, a stencil showing where the holes are, so a hole is never mistaken for a black object. The fifth gets the stencil too, and every photograph is first adjusted to the same brightness. On the job all five are handed photographs with every other dot punched out. Only the last two were trained for anything like that.

11.1 Two building blocks: the stride-2 convolution and its transpose

All five use `SpatialEncoder` and `SpatialDecoder` (`components/spatial_encoder.py`, `components/spatial_decoder.py`). An encoder stage is a convolution with kernel $k = 4$, stride $s = 2$, padding $p = 1$, then BatchNorm, GELU and a channel dropout of 0.3; by the

output-size formula of Chapter 10, a side of H becomes $\lfloor (H+2-4)/2 \rfloor + 1 = H/2$ for even H. The channel count starts at a *base* width c and doubles each stage: $c, 2c, 4c, \ldots$

Definition 11.1 (BatchNorm, briefly)**.** A BatchNorm layer over c channels standardises each channel with the batch's mean and variance in training (and with stored running averages at inference), then applies a learned scale and shift per channel: $2c$ parameters, plus $2c+1$ buffers (Chapter 12 works through it).

Definition 11.2 (Transposed convolution)**.** A *transposed convolution* with stride s spreads each input value over the output: input position i adds x_i times the kernel into the output starting at position si, overlapping contributions add, and p values are cropped from each end. Its output size is

$$H_{\text{out}} = (H_{\text{in}} - 1)\, s - 2p + k, \tag{11.1}$$

so with $k=4$, $s=2$, $p=1$: $(H-1)\cdot 2 - 2 + 4 = 2H$, an exact doubling. Its parameters count like a convolution's, $c_{\text{in}} c_{\text{out}} k^2 + c_{\text{out}}$.

Worked example 11.1: A transposed convolution in one dimension

Input $(1, 2)$, kernel $(1, 1, 1, 1)$, stride 2, padding 1, no bias.

$$\begin{aligned} x_0 = 1 \text{ at } 0 &: \text{adds } (1,1,1,1) \text{ to positions 0–3} \\ x_1 = 2 \text{ at } 2 &: \text{adds } (2,2,2,2) \text{ to positions 2–5} \\ \text{sum} &: (1,\ 1,\ 1+2,\ 1+2,\ 2,\ 2) = (1,1,3,3,2,2) \\ \text{crop } p=1 \text{ each end} &: (1,3,3,2) \end{aligned}$$

Two values became four, as Equation (11.1) says: $(2-1)\times 2 - 2 + 4 = 4$.

A decoder stage is the transpose, $k=4$, $s=2$, $p=1$, halving channels, with BatchNorm, GELU and dropout, except the last, which is the bare transposed convolution. The code's comment explains why: the last layer must emit real temperatures, which BatchNorm would force to zero mean and GELU would clip below zero (`components/spatial_decoder.py`).

The counting rule for one stage. With the rules of Chapter 10,

$$P_{\text{enc stage}}(a \to b) = 16ab + b + 2b = 16ab + 3b, \qquad P_{\text{dec stage}}(a \to b) = 16ab + 3b \ \ (\text{last: } 16ab + b). \tag{11.2}$$

11.2 The five side by side

codename	class	encoder channels	hides	loss	norm.	trainable
Pratibimba	A	$1 \to 32 \to 64 \to 128 \to 256$	nothing	MSE	yes	1,379,393
Antardhana	A	$1 \to 64 \to 128$	13–25%	MSE	yes	264,961
Tirohita	A	$1 \to 32 \to 64 \to 128$	50–75%	L1	yes	329,665
Asanskrita	B	$3 \to 64 \to 128$	35–55%	L1	no	267,009
Drashta	C	$3 \to 64 \to 128$	50–75%	L1	yes	267,009

Table 11.1: The five convolutional models as served. Classes: A `SpatialAutoencoder`, B `UnNormalizedSpatialAutoencoder`, C `NormalizedMaskedSpatialAutoencoder`. "Hides" is the share of valid pixels hidden per training patch, drawn uniformly in the range for each patch. Counts from strict loads of the served checkpoints [C]; masking ranges and losses from `trainers/`. Asanskrita's and Drashta's encoders read three channels: the picture, a validity mask and an input mask.

All five were trained on Landsat 9 thermal patches in °C at sizes 128, 256 and 512 within each epoch, with Adam at 10^{-3}, and all five discard patches with under 40% valid pixels (`trainers/`, `MIN_VALID_PIXEL_FRACTION`). The four normalising models carry one mean and one spread for Landsat temperatures, 24.5756 ± 13.5744 °C (`app/constants/thermal_pixel_consts.json`); Chapter 21, which uses the same two numbers, works with them in detail. Every one is served with patch 128, stride 64, and two-pass *pixel* checkerboard inference: one pass hides every pixel whose row plus column is odd, the other those where it is even (`inferencers/spatial_autoencoder_inferencer.py` and its two siblings).

11.3 Pratibimba: the plain autoencoder

What it is. Class A with one input channel, base 32 and 4 stages (`components/spatial_auto_encoder.py`), checkpoint v8.1.0, epoch 29. Codename: "reflection".

step	layer	shape ($B = 1$, 128×128)
input, normalised		`(1, 1, 128, 128)`
encoder 0–3	conv $k4\,s2\,p1$, BN, GELU, dropout	`(1, 32, 64, 64)`, `(1, 64, 32, 32)`, `(1, 128, 16, 16)`, `(1, 256, 8, 8)`
decoder 0–2	transposed conv, BN, GELU, dropout	`(1, 128, 16, 16)`, `(1, 64, 32, 32)`, `(1, 32, 64, 64)`
decoder 3	transposed conv only	`(1, 1, 128, 128)`, then denormalised

Count. Encoder, by Equation (11.2): $(16 \cdot 32 + 96) + (16 \cdot 32 \cdot 64 + 192) + (16 \cdot 64 \cdot 128 + 384) + (16 \cdot 128 \cdot 256 + 768) = 608 + 32{,}960 + 131{,}456 + 525{,}056 = 690{,}080$. Decoder: $(16 \cdot 256 \cdot 128 + 384) + (16 \cdot 128 \cdot 64 + 192) + (16 \cdot 64 \cdot 32 + 96) + (16 \cdot 32 + 1) = 524{,}672 + 131{,}264 + 32{,}864 + 513 = 689{,}313$. Total 1,379,393 [C]. The manifest's 1,380,812 adds 1,419 buffers: a running mean and variance for each of the $480 + 224 = 704$ BatchNorm channels ($2 \times 704 = 1{,}408$), 7 batch counters, and 4 normalisation values; $1{,}408 + 7 + 4 = 1{,}419$.

Why it exists, and its fault. It is the baseline: reproduce the patch through a bottleneck. Its trainer (`trainers/spatial_autoencoder_trainer.py`) hides nothing; the loss is the MSE over all valid pixels. That is the identity trap of Chapter 9: the only thing stopping a copy is the bottleneck, $256 \times 8 \times 8 = 16{,}384$ values for a 16,384-pixel patch, exactly as many numbers as the input. At inference the product then hides half the pixels in a checkerboard, a regime the model never saw.

11.4 Antardhana and Tirohita: hide pixels, keep the class

What they are. The same `SpatialAutoencoder` class. Antardhana ("concealment"), v0.1.0 epoch 121, is base 64 with 2 stages; Tirohita ("the hidden one"), v0.4.0 epoch 77, base 32 with 3 stages. Their trainers (`trainers/spatial_masked_autoencoder_trainer.py`, `..._trainer_l1_loss.py`) draw a hiding share per patch, uniformly in $[0.13, 0.25]$ for Antardhana and $[0.50, 0.75]$ for Tirohita, hide that share of the valid pixels by setting them to 0, and grade only the hidden ones, with MSE and L1 respectively.

	Antardhana	Tirohita

encoder outputs	`(1, 64, 64, 64)`
	`(1, 128, 32, 32)`
		`(1, 32, 64, 64)`
		`(1, 64, 32, 32)`
		`(1, 128, 16, 16)`

decoder outputs	`(1, 64, 64, 64)`
	`(1, 1, 128, 128)`
		`(1, 64, 32, 32)`
		`(1, 32, 64, 64)`
		`(1, 1, 128, 128)`

bottleneck values	$128 \times 32 \times 32 = 131{,}072$	$128 \times 16 \times 16 = 32{,}768$
trainable	264,961	329,665

Antardhana's stages: $16{\cdot}64{+}192 = 1{,}216$; $16{\cdot}64{\cdot}128{+}384 = 131{,}456$; decoder $16{\cdot}128{\cdot}64{+}192 = 131{,}264$ and the last $16{\cdot}64{+}1 = 1{,}025$. Tirohita's encoder is the 165,024 of worked example 10.9; its decoder is $131{,}264 + 32{,}864 + 513 = 164{,}641$ [C].

Why they exist, and their fault. Hiding pixels removes the identity answer for the hidden ones (Chapter 9); Tirohita adds the L1 loss, which does not pull predictions towards anomalies (Section 9.6), and much heavier hiding. The fault both keep: a hidden pixel is set to 0, then normalised to $(0 - 24.5756)/13.5744 = -1.81$, which is exactly what a real pixel at 0 °C looks like. With one input channel the network cannot tell "hidden" from "freezing".

Source check: two siblings that differ in more than the loss, and their validation losses

The manifest and `components/spatial_auto_encoder.py` present Antardhana and Tirohita as one model with two losses. They also differ in size (base 64, 2 stages against base 32, 3 stages) and in hiding share (13–25% against 50–75%) (Chapter 25 noted the size difference). The manifest's note that Tirohita has "Lower val loss than the MSE sibling" compares 1.79 (an L1, in °C) with 17.29 (an MSE, in °C^2), and both trainers validate with *nothing hidden* (`model(pixels, mask=mask)`), so neither number measures the hidden-pixel task they were trained on. The two numbers cannot be compared. Also, both manifests name `MaskedSpatialAutoencoderInferencer`; the resolver routes both through `SpatialAutoencoderInferencer` (their `foundation_model_name` is `spatial_autoencoder`), and `docs/04-models.md`'s "two architectures can be trained but not run" is true of the enum names only: the product runs both models under Pratibimba's inferencer.

11.5 Asanskrita: tell the network where the holes are

What it is. Class B, base 64, 2 stages (`components/unnormalized_spatial_auto_encoder.py`), v0.3.0 epoch 82. Codename: "unrefined", a pun on *unnormalised*: it is the only model with no normalisation buffers. Its encoder reads three channels:

$$\left[\, T \cdot m_{\text{valid}} \cdot m_{\text{input}},\quad m_{\text{valid}},\quad m_{\text{input}} \,\right], \tag{11.3}$$

the temperature in °C with hidden and invalid pixels zeroed, the validity mask, and the input mask (1 where a pixel is valid and visible). A hidden pixel has $m_{\text{valid}} = 1$, $m_{\text{input}} = 0$; an invalid one has both 0; so the network can tell "hidden" from "nothing here" from "a real 0 °C".

step	layer	shape
concatenate	(11.3)	`(1, 3, 128, 128)`
encoder 0, 1	conv $3 \to 64$, $64 \to 128$	`(1, 64, 64, 64)`, `(1, 128, 32, 32)`
decoder 0, 1	transposed $128 \to 64$, $64 \to 1$	`(1, 64, 64, 64)`, `(1, 1, 128, 128)`, in °C

Count: the first stage is $16 \cdot 3 \cdot 64 + 64 + 128 = 3{,}072 + 192 = 3{,}264$; the rest is Antardhana's $(131{,}456 + 131{,}264 + 1{,}025)$, so $3{,}264 + 131{,}456 + 131{,}264 + 1{,}025 = 267{,}009$ [C]. The two extra input channels cost $3{,}264 - 1{,}216 = 2{,}048 = 2 \times 16 \times 64$ weights.

Why, and the trade-off. The mask channels fix the "hidden or freezing" ambiguity. Dropping normalisation was the experiment: raw °C goes straight into the first convolution, followed by BatchNorm, which re-centres the channels anyway. Its trainer hides 35–55% of valid pixels with L1; the configuration's `masking_range` is ignored by this trainer, whose function defaults happen to be the same $[0.35, 0.55]$.

11.6 Drashta: the masks and the normalisation

What it is. Class C, base 64, 2 stages, in `components/normalized_masked_spatial_auto_encoder.py`; v1.2.0 epoch 69. Codename: "the seer". Asanskrita's architecture with `PixelNormalize` on the way in and `PixelDenormalize` on the way out; its trainer reads `masking_range` $[0.5, 0.75]$ from the configuration, with L1. Shapes and count are Asanskrita's: 267,009 trainable, and 519 buffers against Asanskrita's 515 (the four normalisation values) [C].

Source check: a hidden pixel is -1.81, not 0

The class's docstring says channel 0 holds "normalized thermal pixels (zeroed where input_mask == 0)". The forward pass multiplies by the masks *before* normalising, so a hidden pixel enters as $(0 - 24.5756)/13.5744 = -1.81$, the same value as in Antardhana, and only the mask channels say it is hidden. Harmless, because the mask channels are there; but read the docstring as describing the raw picture, not channel 0.

11.7 What the product does with them

codename	inferencer (as routed)	default score	normalisation file
Pratibimba	`SpatialAutoencoderInferencer`	MSE	thermal
Antardhana	`SpatialAutoencoderInferencer`	MSE	thermal
Tirohita	`SpatialAutoencoderInferencer`	L1	thermal
Asanskrita	`MaskedSpatialAutoencoderInferencer`	L1	none
Drashta	`NormalizedMaskedAutoencoderInferencer`	L1	thermal

All five: scoring methods L1 and MSE, patch 128, stride 64, batch 8, sensor family thermal (`backend/allotrope/foundation_models/resolver.py`). The resolver rebuilds each configuration from the manifest's `encoder_dims` string (`"1→32→64→128"`: base 32, three stages; `"3→64→128"`: input 3, so constructor `in_channels` $3 - 2 = 1$), and every rebuilt configuration matches its checkpoint [C].

Three behaviours differ from Chakshu's inferencer (Chapter 21): the masking is per pixel, not per token; the validity mask is not eroded; and a pixel no tile covers is set to 0, not to its input value, so it scores its whole temperature as residual (`docs/09-known-issues.md`, item 4).

A synthetic comparison. The script runs each model, through the product's own inferencer and with its served weights, on a smooth synthetic field, $T = 25 + 3\sin(x/40) + 2\cos(y/30)$ °C for column x and row y of a 128×128 patch, and on copies with one hot pixel or a 3×3 hot spot at 45 °C planted near the centre [A]:

model	background (median)	one hot pixel	3×3 spot, centre
Pratibimba	10.87 °C	19.20	17.60
Antardhana	6.59	24.23	22.05
Tirohita	2.32	20.21	14.78
Asanskrita	0.75	19.37	16.57
Drashta	0.58	18.14	16.42
Chakshu (256×256)	0.019	20.96	7.59

All values are $|T - \hat{T}|$ in °C [C]; Chakshu's row is from Chapter 21, which runs the same field at its own patch size. On a field this smooth the background error is the number to read: a detector's hot pixel must stand above it. Pratibimba's 10.9 °C, half the planted anomaly, is what happens when a model trained on whole pictures meets a checkerboard of holes it has never seen; the three single-channel models improve in step with how much hiding they trained on; the two mask-channel models are an order of magnitude better again; Chakshu is thirty times better than the best of them. One synthetic field is not a benchmark: nothing here was measured on a labelled thermal scene, and none exists in the repository.

Source check: the per-scene override and these models

Four of the five carry `normalize.*` buffers in their checkpoints. Section 21.9, later, shows that loading a checkpoint writes those buffers back over a per-scene override, so the HotSat-1 plan fails for these four exactly as for Chakshu. Asanskrita has no normalisation to override: it would read HotSat's digital numbers as degrees.

Values that will drift

Read 2026-10-31 from the `/models/` volume's manifests: Pratibimba v8.1.0 (epoch 29, validation 13.77, MSE), Antardhana v0.1.0 (epoch 121, 17.29, MSE), Tirohita v0.4.0 (epoch 77, 1.79, L1), Asanskrita v0.3.0 (epoch 82, 2.15, L1), Drashta v1.2.0 (epoch 69, 5.53, L1). Manifest parameter figures (1,380,812; 265,480; 330,314; 267,524; 267,528) include buffers. Validation for every masked model is computed with nothing hidden.

11.8 Common confusions

Antardhana and Tirohita differ only in the loss.
They also differ in width, depth and how much they hide (13–25% against 50–75%).

Tirohita's validation loss is ten times lower, so it is ten times better.
One is an L1 in °C, the other an MSE in °C^2, and both are measured with nothing hidden. They are not comparable.

The masked models cannot be run: no inferencer is registered for them.
The resolver routes Antardhana and Tirohita through Pratibimba's inferencer; only the enum names lack an entry.

Asanskrita is unnormalised, so its network sees raw temperatures throughout.
Only its first convolution does; the BatchNorm after it standardises every channel, with learned scale and shift.

A hidden pixel in Drashta is a zero.
It is -1.81 after normalisation; the mask channels mark it.

More parameters, better model.
Pratibimba has five times Drashta's parameters and an error nineteen times larger on the synthetic field ($10.87/0.58 = 18.7$). What it was trained to do matters more.

11.9 Check yourself

Exercise 11.1. Use Equation (11.1) to find the output length of a transposed convolution with $k = 4$, $s = 2$, $p = 1$ on an input of 16, and with $k = 3$, $s = 2$, $p = 1$ on 16. Which one doubles?

Exercise 11.2. Count by hand a `SpatialAutoencoder` with base 16 and 3 stages (one input channel), encoder and decoder separately.

Exercise 11.3. How many values does Pratibimba's bottleneck hold for a 256×256 patch, and how does that compare with the input?

Exercise 11.4. A pixel is valid and hidden. Write its three input channels for Asanskrita and for Drashta, if its true temperature is 30 °C.

Exercise 11.5. Explain in two sentences why Pratibimba's background error on the synthetic field is so large, using the words "trained" and "checkerboard".

Exercise 11.6. Repeat worked example 11.1 with input $(3, -1)$ and kernel $(1, 2, 2, 1)$.

Next: Chapter 12, The spectral compressor.

Chapter 12

The spectral compressor

The one thing this chapter teaches

Before the transformer runs, a 1×1 convolution with BatchNorm replaces each pixel's 165 normalised bands with 32 learned mixtures, and afterwards a second 1×1 convolution, deliberately without any normalisation, expands 32 back to 165; the pair is a linear bottleneck, so every reconstructed spectrum lies in a 32-dimensional space.

The spectral compressor is the only structural difference between Indradhanu and its thermal sibling Chakshu. It is a short module, 59 lines in `app/foundation_models/components/spectral_compressor.py`, most of them explanation. It is also the module whose mathematics sets a limit on everything downstream, and the reason Part IV of this book (Chapter 22) exists. In this chapter you will compute what it does to one pixel by hand, learn BatchNorm properly, count its parameters, and prove that the pair of convolutions can never reproduce more than 32 independent spectral directions.

12.1 The problem: 165 channels into a 32-wide transformer

Indradhanu's transformer is a small SegFormer configuration ("B0"), whose first stage works with token vectors of 32 numbers (Chapter 13). Hand it the 165 normalised bands directly and three things go wrong.

The first layer becomes bloated. Stage 1 begins with a 4×4 convolution from the input channels to 32 (Chapter 10). Its weights number $C_{\text{in}} \times 32 \times 4 \times 4$:

$$\begin{aligned} \text{with } C_{\text{in}} = 165: &\quad 165 \times 32 \times 16 = 5{,}280 \times 16 = 84{,}480, \\ \text{with } C_{\text{in}} = 32: &\quad 32 \times 32 \times 16 = 1{,}024 \times 16 = 16{,}384. \end{aligned}$$

The ratio is $84{,}480/16{,}384 \approx 5.16$: five times the weights, on the layer that runs at the finest resolution.

Most of the 165 numbers are near-duplicates. Neighbouring bands on the common grid are 10 nm apart (Chapter 3). Over 10 nm almost nothing changes in a natural spectrum, so bands 62 and 63 carry nearly the same value in nearly every pixel. You pay full price for information you already have.

Every intermediate tensor is five times bigger. A `(B, 165, 128, 128)` activation holds $165/32 \approx 5.2$ times as many numbers as `(B, 32, 128, 128)`. Memory limits the batch size,

and the batch size limits training speed.

The obvious fix, "pick 32 of the 165 bands", throws away whatever the unpicked bands knew. The fix the model uses is to *mix*: build each of the 32 channels as a weighted blend of all 165 bands, with the blend learned.

Analogy

A sound engineer mixes a 48-track recording down to a stereo pair. They do not keep two tracks and throw away 46; each output channel is a blend of every track, set so that what matters survives. A listener's speakers later turn the stereo pair back into a room full of sound. The compressor is the mixdown, the decompressor is the playback, and here the mixing-desk settings are learned. The analogy also tells you what is lost: two instruments panned identically can never be separated again from the stereo pair. Section 12.5 makes that precise.

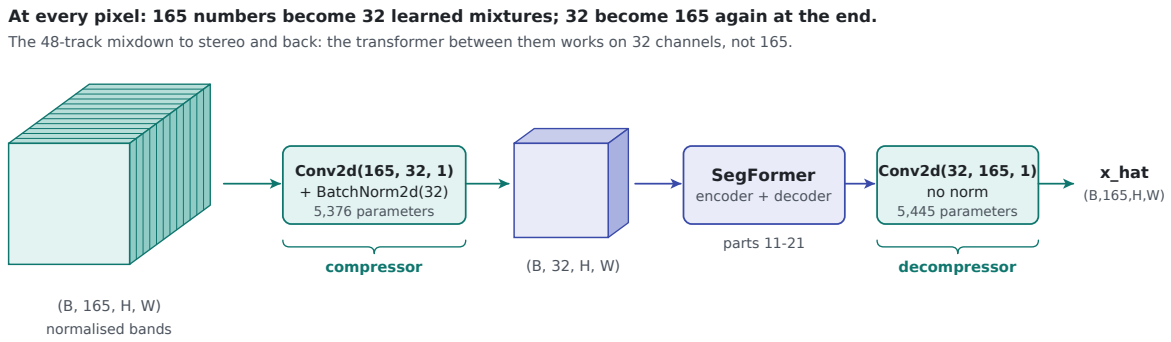


Figure 12.1: The spectral bottleneck around the SegFormer. A 1×1 convolution with BatchNorm mixes each pixel's 165 normalised bands into 32 channels (5,376 parameters); the spatial model runs on those 32; a second 1×1 convolution, without normalisation, expands them back to 165 (5,445 parameters). (Course figure IW 10.1.)

12.2 The compressor is a matrix applied at every pixel

The code is short enough to quote whole:

```
class SpectralCompressor(nn.Module):
    def __init__(self, in_channels, compressed_channels):
        super().__init__()
        self.compress = nn.Conv2d(in_channels, compressed_channels, kernel_size=1)
        self.norm = nn.BatchNorm2d(compressed_channels)

    def forward(self, x):           # x: (B, 165, H, W)
        x = self.compress(x)        # x: (B, 32, H, W)
        x = self.norm(x)            # x: (B, 32, H, W)
        return x

class SpectralDecompressor(nn.Module):
    def __init__(self, compressed_channels, out_channels):
        super().__init__()
        self.decompress = nn.Conv2d(compressed_channels, out_channels, kernel_size=1)

    def forward(self, x):           # x: (B, 32, H, W)
        return self.decompress(x)   # -> (B, 165, H, W)
```

Chapter 10 showed that a 1×1 convolution is a linear layer applied independently at every pixel (Equation (10.11)). So the convolution part of the compressor is, for every pixel (i, j),

$$\mathbf{z}_{:,i,j} = \mathbf{W}\,\mathbf{x}_{:,i,j} + \mathbf{b}, \qquad \mathbf{W} \in \mathbb{R}^{32\times 165},\ \mathbf{b} \in \mathbb{R}^{32}, \tag{12.1}$$

where $\mathbf{x}_{:,i,j}$ is the pixel's 165 normalised band values (Chapter 5). Row d of $\mathbf{W}$ is the recipe for compressed channel d: how much of each band goes into it. The same $\mathbf{W}$ is used at every pixel of every patch; the compressor never looks at a neighbour.

Worked example 12.1: Four bands into two

Pretend there are 4 bands and 2 compressed channels. One pixel, already normalised:

$$\mathbf{x} = (1.2,\ 1.0,\ -0.4,\ -0.6)^\top.$$

Suppose training arrived at these recipes, with zero biases:

$$\mathbf{W} = \begin{pmatrix} 0.5 & 0.5 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.5 & 0.5 \end{pmatrix}.$$

Channel 0 uses row 0:

$$\begin{aligned} z_0 &= 0.5 \times 1.2 + 0.5 \times 1.0 + 0.0 \times (-0.4) + 0.0 \times (-0.6) \\ &= 0.60 + 0.50 + 0 + 0 = 1.10. \end{aligned}$$

Channel 1 uses row 1:

$$\begin{aligned} z_1 &= 0.0 \times 1.2 + 0.0 \times 1.0 + 0.5 \times (-0.4) + 0.5 \times (-0.6) \\ &= 0 + 0 - 0.20 - 0.30 = -0.50. \end{aligned}$$

So $\mathbf{z} = (1.10,\ -0.50)^\top$. Four numbers became two, and the two still describe the shape of the spectrum: bright at the short-wavelength end, dark at the long. The real compressor does the same with 165 inputs and 32 outputs, with recipes found by gradient descent (Chapter 26).

Figure 10.3 in Chapter 10 draws this example.

12.3 BatchNorm

The compressor's second line is `nn.BatchNorm2d(32)`. Batch normalisation [17] appears throughout the convolutional parts of this repository, and it behaves differently in training and in use, which has caused real trouble in this project (Chapter 29). It deserves a careful treatment.

12.3.1 What it computes in training

Definition 12.1 (Batch normalisation, training mode)**.** Let $z_{b,d,i,j}$ be channel d of the input to `BatchNorm2d`, for patch b of the batch at pixel (i, j). For each channel d separately, *batch normalisation*

1. computes the mean and variance of that channel over the whole batch and every pixel, that is over $M = B \cdot H \cdot W$ values:

$$\mu_d = \frac{1}{M}\sum_{b,i,j} z_{b,d,i,j}, \qquad \sigma_d^2 = \frac{1}{M}\sum_{b,i,j}\left(z_{b,d,i,j} - \mu_d\right)^2; \tag{12.2}$$

2. standardises every value of the channel with them, adding a small ε (here 10^{-5}) so that a constant channel does not divide by zero:

$$\hat{z}_{b,d,i,j} = \frac{z_{b,d,i,j} - \mu_d}{\sqrt{\sigma_d^2 + \varepsilon}}; \tag{12.3}$$

3. rescales and shifts with two learned parameters per channel, the *scale* γ_d and the *shift* β_d:

$$u_{b,d,i,j} = \gamma_d \, \hat{z}_{b,d,i,j} + \beta_d. \tag{12.4}$$

After step 2 each channel has mean 0 and variance (almost exactly) 1 over the batch. Step 3 lets training choose a different mean (β_d) and spread (γ_d) if that suits the next layer better; they start at $\gamma_d = 1$ and $\beta_d = 0$. The variance in Equation (12.2) divides by M, not $M - 1$: it is the *biased* variance.

Worked example 12.2: BatchNorm on four values

One channel; the batch and pixels together give four values $z = 1,\ 2,\ 3,\ 6$. Take $\gamma = 1$, $\beta = 0$. Work on paper.
Mean:

$$1 + 2 + 3 + 6 = 12,$$
$$\mu = 12/4 = 3.$$

Deviations from the mean: $1-3 = -2$, $2-3 = -1$, $3-3 = 0$, $6-3 = 3$. Squares: $4,\ 1,\ 0,\ 9$. Their sum:

$$4 + 1 + 0 + 9 = 14.$$

Biased variance:

$$\sigma^2 = 14/4 = 3.5.$$

Denominator:

$$\sqrt{3.5 + 0.00001} = \sqrt{3.50001} = 1.87083.$$

Standardised values, one per deviation:

$$-2/1.87083 = -1.0690,$$
$$-1/1.87083 = -0.5345,$$
$$0/1.87083 = 0,$$
$$3/1.87083 = 1.6036.$$

With $\gamma = 1$, $\beta = 0$ these are the outputs. With $\gamma = 2$, $\beta = 0.5$ instead, each is doubled and shifted: $2 \times (-1.0690) + 0.5 = -1.6381$, $2 \times (-0.5345) + 0.5 = -0.5690$, $2 \times 0 + 0.5 = 0.5$, $2 \times 1.6036 + 0.5 = 3.7071$. PyTorch gives the same eight numbers [C].

12.3.2 Running statistics and evaluation mode

At inference there is often one patch, or a batch whose make-up is accidental; normalising each scene by its own batch statistics would make a pixel's output depend on which other patches happened to be processed with it. So during training BatchNorm also keeps *running statistics*, exponential moving averages of the batch statistics, stored as buffers (not parameters; they are not trained by gradient descent):

$$\text{running_mean}_d \leftarrow (1 - m)\,\text{running_mean}_d + m\,\mu_d, \tag{12.5}$$
$$\text{running_var}_d \leftarrow (1 - m)\,\text{running_var}_d + m\,s_d^2, \tag{12.6}$$

with momentum $m = 0.1$ (PyTorch's default, used here) and starting values 0 and 1. Note a PyTorch detail: the running variance is fed the *unbiased* variance $s_d^2 = \frac{1}{M-1}\sum(z-\mu_d)^2$, while the normalisation in training uses the biased one.

In *evaluation mode* (`model.eval()`), BatchNorm stops computing batch statistics and uses the running ones:

$$u_{b,d,i,j} = \gamma_d \frac{z_{b,d,i,j} - \text{running_mean}_d}{\sqrt{\text{running_var}_d + \varepsilon}} + \beta_d. \tag{12.7}$$

Each output now depends only on its own input. Training mode and evaluation mode give the same answer only when the running statistics match the statistics of the data you feed.

Worked example 12.3: One training step of running statistics, then evaluation

Continue worked example 12.2 with a fresh layer (running mean 0, running variance 1). The batch mean was $\mu = 3$. The unbiased variance divides the same sum of squares by $M - 1 = 3$:

$$s^2 = 14/3 = 4.66667.$$

Update with $m = 0.1$:

$$\text{running_mean} = 0.9 \times 0 + 0.1 \times 3 = 0 + 0.3 = 0.3,$$
$$\text{running_var} = 0.9 \times 1 + 0.1 \times 4.66667 = 0.9 + 0.46667 = 1.36667.$$

Now switch to evaluation mode and feed the same four values ($\gamma = 1$, $\beta = 0$). The denominator is $\sqrt{1.36667 + 0.00001} = 1.16905$, and

$$(1 - 0.3)/1.16905 = 0.7/1.16905 = 0.5988,$$
$$(2 - 0.3)/1.16905 = 1.7/1.16905 = 1.4542,$$
$$(3 - 0.3)/1.16905 = 2.7/1.16905 = 2.3096,$$
$$(6 - 0.3)/1.16905 = 5.7/1.16905 = 4.8758.$$

In training mode the value 6 became 1.6036; in evaluation mode, after one step, it becomes 4.8758. After thousands of steps on similar data the running statistics converge and the two modes agree. When the data in use differ from the training data, they do not. PyTorch reproduces all of these numbers [C].

12.3.3 The buffers travel inside the checkpoint

`BatchNorm2d(32)` has $2 \times 32 = 64$ trainable parameters (γ and β) and three buffers: `running_mean` (32 values), `running_var` (32) and `num_batches_tracked` (one counter) [C]. That is 65 values that are saved in the checkpoint but are not parameters. Together with the 660 values of normalisation statistics they account for the gap between Indradhanu's 5,506,629 parameters and the 5,507,354 values in its state dictionary (Chapter 9).

12.3.4 The convolution's bias is redundant in front of BatchNorm

Here is a small fact worth deriving, because it surprises people. Add a constant b_d to every value of channel d before BatchNorm. In training mode,

$$\text{mean of } (z + b_d) = \mu_d + b_d \qquad \text{a constant shifts the mean by itself}$$
$$(z + b_d) - (\mu_d + b_d) = z - \mu_d, \qquad \text{the shift cancels in the deviation}$$

and the variance, built from deviations, is unchanged too. So the standardised value Equation (12.3) does not depend on b_d at all. The 32 biases of the compressor's convolution have

no effect on its training-mode output, receive zero gradient, and stay at their initial values; the running mean absorbs them, so they cancel in evaluation mode as well. Adding 10 to every bias leaves the training-mode output unchanged, as the compute script confirms [C]. They are harmless, and they are still counted as parameters.

12.3.5 Why the compressor has a BatchNorm

The compressor's job is to hand the encoder a stable, well-scaled input. Whatever mixtures $\mathbf{W}$ learns, their scale can drift during training; BatchNorm resets every compressed channel to a controlled mean and spread, so the first patch embedding always sees inputs of a predictable size. The module's own docstring: "The compressor's job is to hand the encoder a stable distribution".

12.4 The decompressor, and why it must have no normalisation

The decompressor maps the decoder's 32-channel output back to 165 bands at every pixel:

$$\hat{\mathbf{x}}_{:,i,j} = \mathbf{V}\,\hat{\mathbf{z}}_{:,i,j} + \mathbf{c}, \qquad \mathbf{V} \in \mathbb{R}^{165\times 32},\ \mathbf{c} \in \mathbb{R}^{165}. \tag{12.8}$$

It is *not* the inverse of the compressor. $\mathbf{W}$ is 32×165 and has no inverse; $\mathbf{V}$ is a separate set of learned weights, trained together with everything else so that the whole chain reconstructs well. Nothing ties $\mathbf{V}$ to $\mathbf{W}$.

It has no BatchNorm and no activation function, and the docstring warns against "tidying" that: "the decompressor has NO norm and NO activation …the decompressor's output feeds denormalisation and must stay free to represent any value". Here is what each would break.

A BatchNorm would erase what the batch has in common. Training-mode BatchNorm forces every output band to mean β_d over the batch. But the batch's true mean in a band is information: a batch of patches over open water has normalised near-infrared values well below zero in every pixel. Take two water pixels whose true normalised value in some band is -1.5, and suppose the decoder predicts exactly that, -1.5 and -1.5. BatchNorm computes mean -1.5, deviations 0 and 0, and outputs β for both. The reconstruction now says "average brightness" where the truth was "dark water", and the residual is large at every pixel of the batch, ordinary or not. The answer would also depend on which other patches shared the batch.

An activation would clip. A ReLU sets every negative value to zero. In normalised space a negative value means "below this band's average"; after a ReLU no band could ever be reconstructed darker than its average.

The same reasoning applies to the final layer of the old convolutional decoder, whose docstring (`app/foundation_models/components/spatial_decoder.py`) explains that BatchNorm there "would force the output to zero-mean unit-variance per channel - but real thermal data is NOT zero-mean or unit-variance".

The asymmetry is the design: the compressor normalises because the encoder needs a stable input; the decompressor does not, because its output must be free to take any value.

12.5 A linear bottleneck: everything passes through 32 directions

Now the mathematics that matters most. Whatever happens between them, the compressor and decompressor impose a hard limit on what the model can output.

12.5.1 Rank, briefly

The *rank* of a matrix is the number of linearly independent columns it has, equivalently the dimension of the set of all vectors $\mathbf{Au}$ it can produce (Chapter 6). A 165×32 matrix has only 32 columns, so its rank is at most 32.

12.5.2 The output lies in a 32-dimensional space

Look again at Equation (12.8). Whatever the encoder and decoder do, and they are nonlinear and spatial, their output at pixel (i, j) is some vector $\hat{\mathbf{z}}_{:,i,j} \in \mathbb{R}^{32}$. Write its entries $\hat{z}_1, \ldots, \hat{z}_{32}$ and the columns of $\mathbf{V}$ as $\mathbf{v}_1, \ldots, \mathbf{v}_{32}$. Then

$$\begin{aligned} \hat{\mathbf{x}}_{:,i,j} &= \mathbf{V}\hat{\mathbf{z}}_{:,i,j} + \mathbf{c} && \text{Equation (12.8)} \\ &= \hat{z}_1\mathbf{v}_1 + \hat{z}_2\mathbf{v}_2 + \cdots + \hat{z}_{32}\mathbf{v}_{32} + \mathbf{c}. && \text{a matrix times a vector is a mix of its columns} \quad (12.9) \end{aligned}$$

Every reconstructed spectrum, at every pixel of every scene, is the fixed vector $\mathbf{c}$ plus a mixture of the same 32 fixed spectra $\mathbf{v}_1, \ldots, \mathbf{v}_{32}$. In the language of Chapter 6: it lies in a 32-dimensional affine subspace of $\mathbb{R}^{165}$. Any part of a real spectrum that points outside that subspace can never be reconstructed, however good the transformer is.

There is a matching limit on the way in. $\mathbf{W}$ has 32 rows, so at most 32 independent combinations of the bands reach the transformer; any change to a pixel's spectrum along a direction $\mathbf{n}$ with $\mathbf{Wn} = \mathbf{0}$ is invisible to everything after the compressor.

12.5.3 The chain without the transformer

If the transformer were removed and BatchNorm ignored, the chain would be

$$\begin{aligned} \hat{\mathbf{x}} &= \mathbf{V}(\mathbf{Wx} + \mathbf{b}) + \mathbf{c} && \text{compress, then decompress} \\ &= \mathbf{VW}\,\mathbf{x} + (\mathbf{Vb} + \mathbf{c}). && \text{multiply out the bracket} \end{aligned}$$

The 165×165 matrix $\mathbf{VW}$ has rank at most 32. The best such matrix for reconstructing data, in the least-squares sense, projects onto the data's 32 leading principal components (Chapter 6); the trained compressor is not constrained to find that, because it is trained jointly with a network that also uses spatial context.

Worked example 12.4: A 3-to-2-to-3 bottleneck and what it loses

Three bands, two compressed channels, zero biases:

$$\mathbf{W} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0.5 \end{pmatrix}, \qquad \mathbf{V} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}.$$

Pixel $\mathbf{x} = (0.2,\ 0.4,\ 0.6)^\top$. Compress:

$$\begin{aligned} z_1 &= 1 \times 0.2 + 0 \times 0.4 + 0 \times 0.6 = 0.2, \\ z_2 &= 0 \times 0.2 + 0.5 \times 0.4 + 0.5 \times 0.6 = 0 + 0.2 + 0.3 = 0.5. \end{aligned}$$

Decompress, row by row of $\mathbf{V}$:

$$\begin{aligned} \hat{x}_1 &= 1 \times 0.2 + 0 \times 0.5 = 0.2, \\ \hat{x}_2 &= 0 \times 0.2 + 1 \times 0.5 = 0.5, \\ \hat{x}_3 &= 0 \times 0.2 + 1 \times 0.5 = 0.5. \end{aligned}$$

Residual:

$$\mathbf{x} - \hat{\mathbf{x}} = (0.2 - 0.2,\ 0.4 - 0.5,\ 0.6 - 0.5) = (0,\ -0.1,\ 0.1).$$

The product is

$$\mathbf{VW} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0.5 & 0.5 \\ 0 & 0.5 & 0.5 \end{pmatrix},$$

whose last two rows are equal, so its rank is 2 [C]. What was lost? The difference between bands 2 and 3. Check that the direction $\mathbf{n} = (0,\ 1,\ -1)^\top$ is invisible to the compressor:

$$\begin{aligned} \mathbf{Wn} &= \big(1 \times 0 + 0 \times 1 + 0 \times (-1),\ \ 0 \times 0 + 0.5 \times 1 + 0.5 \times (-1)\big)^\top \\ &= (0,\ 0)^\top. \end{aligned}$$

The pixel's residual $(0, -0.1, 0.1)$ is exactly $-0.1\,\mathbf{n}$. Every pixel whose bands 2 and 3 differ leaves that difference in its residual, ordinary or anomalous. If an anomaly's signature were a band-2-versus-band-3 contrast, this model would see it in the residual, but so would it see the ordinary variation along $\mathbf{n}$, at every pixel.

Worked example 12.4 is the whole story in miniature. The bottleneck is lossy; the bet is that what it loses is mostly redundancy and noise. Training decides which 32 directions to keep, and keeps the ones that matter for reconstructing ordinary terrain. A narrow absorption feature that ordinary terrain rarely shows may lie largely outside them. Chapter 22 measures that cost and is the reason the project later built a cross-attention stem in place of the compressor.

12.6 Counting the parameters

Use in × out + out for a 1 × 1 convolution (Chapter 10) and 2 × channels for a BatchNorm.

Worked example 12.5: The spectral bottleneck's 10,821 parameters

Compressor convolution, 165 → 32:

$$\begin{aligned} 165 \times 32 &= 5{,}280, \\ 5{,}280 + 32 &= 5{,}312. \end{aligned}$$

Compressor BatchNorm:

$$2 \times 32 = 64.$$

Compressor total:

$$5{,}312 + 64 = 5{,}376.$$

Decompressor convolution, 32 → 165:

$$\begin{aligned} 32 \times 165 &= 5{,}280, \\ 5{,}280 + 165 &= 5{,}445. \end{aligned}$$

Both:

$$5{,}376 + 5{,}445 = 10{,}821.$$

The `torchinfo` table in `research/model_break_down/05_hyperspectral_segformer_mae.md` lists `Conv2d:2-1` with 5,312, `BatchNorm2d:2-2` with 64 and `Conv2d:2-19` with 5,445 [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`], and summing

`parameters()` of the real modules gives 5,376 and 5,445 [C]. Out of 5,506,629 parameters, the bottleneck is

$$10{,}821/5{,}506{,}629 = 0.00197,$$

under 0.2% of the model.

It is almost free, and it saves the $84{,}480 - 16{,}384 = 68{,}096$ weights of Section 12.1 in stage 1 alone, plus a factor of five in memory at full resolution.

12.7 It is MNF-shaped, but it is not MNF

Chapter 8 met the Minimum Noise Fraction transform, MNF [8]: a classical linear projection from many bands to few, found by an eigen-decomposition that ranks directions by signal-to-noise ratio. Structurally it is the same kind of object as $\mathbf{W}$: a matrix of shape (few, many). The differences are in how the matrix is found and what it serves.

	MNF	`SpectralCompressor`
matrix found by	eigen-decomposition	gradient descent
optimises	signal-to-noise ratio	reconstruction loss
runs	offline, per scene	learned once, all scenes
learned with the network	no	yes

The last row is the real advantage: MNF optimises a proxy, while the compressor optimises the thing the model is graded on, and adapts while the rest of the network adapts to it. The module docstring calls it "a trainable analogue of MNF".

12.8 Why D is 32, not 8 and not 128

Not much larger: stage 1 of the encoder embeds tokens at 32 numbers, so feeding it far more than 32 channels inflates the first convolution only to compress again immediately. **Not much smaller:** squeeze too hard and real spectral structure is destroyed before the transformer sees it; by Equation (12.9) the model then cannot get spectral shape right however long it trains.

The project's training configuration (the file `configs/hyperspectral_segformer_exp_1.json`) records this as a diagnosis rule:

```
"SAM_stuck_above_0.3_after_epoch_30":
    "Compressor bottleneck too narrow (try compressed_channels=32)"
```

SAM is the spectral-angle error (Chapter 25). That is exactly the change made between versions.

Values that will drift

	v0.1.0	v0.2.0
compressed channels D	24[a]	32[b]
final validation loss	0.07694	0.04349

[a] [M: `configs/hyperspectral_segformer_exp_1.json`]
[b] [M: `configs/hyperspectral_segformer_exp_2.json`]
Validation losses from course part IW10 (quoting a checkpoint manifest that is not in the repository tree) and, for v0.2.0, `research/model_break_down/05_hyperspectral_segformer_mae.md`. Total parameters: 5.21 M at $D = 24$, 5.51 M at $D = 32$ [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`]. The shipped model since 2026-10 is

Indradhanu V2 (Chapter 39).

Source check: the code default is 24, the shipped model is 32

Both `HyperspectralSegFormerMAE.__init__` (`app/foundation_models/components/hyperspectral_seg_former_mae.py`, line 55) and `HyperspectralSegFormerMAEConfig` (`app/models/training/training_config.py`, line 89) default `compressed_channels` to 24, and the class docstring's example says "(e.g. 24)". The v0.2.0 model and its successors are built with 32 from the configuration file. If you construct the class without arguments you get the old, narrower bottleneck, with $(165 \times 24 + 24 + 48) + (24 \times 165 + 165) = 4{,}032 + 4{,}125 = 8{,}157$ compressor and decompressor parameters instead of 10,821 [C].

12.9 Common confusions

Is the decompressor the inverse of the compressor?
No. A 32×165 matrix has no inverse. The decompressor is an independent set of learned weights; nothing ties it to the compressor except that both are trained to make the chain reconstruct well.

Does the compressor look at neighbouring pixels?
No. Its kernel is 1×1, so every pixel is processed alone. All spatial reasoning happens in the transformer.

Is the compression lossy?
Yes. By Equation (12.9) every output spectrum lies in a 32-dimensional space. The bet is that what is lost is mostly redundancy and noise; validation loss and detection scores test the bet.

Why is it called spectral compression when the picture stays 128×128?
Because only the band axis is compressed. Height and width are untouched.

Does BatchNorm normalise each pixel's spectrum?
No. It normalises each *channel* across the batch and all pixels. Normalising each token across its own features is LayerNorm (Chapter 13).

Are BatchNorm's running mean and variance parameters?
No. They are buffers, updated by Equations (12.5) and (12.6), not by gradients. They are saved in the checkpoint and used in evaluation mode.

12.10 Check yourself

Exercise 12.1. Compute the parameter count of `Conv2d(165,24,kernel_size=1)` and of the whole v0.1.0 compressor (with its `BatchNorm2d(24)`).

Exercise 12.2. A BatchNorm channel receives the batch values 2, 4, 4, 6 with $\gamma = 1$, $\beta = 0$, ε negligible. Compute the training-mode outputs. Then, starting from running mean 0 and running variance 1 with momentum 0.1, compute the running statistics after this one step.

Exercise 12.3. Give the two reasons the decompressor has no BatchNorm and no activation, each with a one-line example.

Exercise 12.4. In worked example 12.4, find the reconstruction of the pixel $\mathbf{x} = (0.3,\ 0.5,\ 0.5)^\top$ and its residual. Explain the result using $\mathbf{n}$.

Exercise 12.5. Show that adding a constant b to every input value of a BatchNorm channel leaves its training-mode output unchanged, using the values of worked example 12.2 with $b = 10$.

Exercise 12.6. How is the compressor like MNF, and how is it different? What symptom suggests D is too small, and what was done about it?

Next: Chapter 13, Patch embedding.

Chapter 13

Patch embedding

The one thing this chapter teaches

A transformer reads a list of vectors, not a picture; patch embedding makes that list with a strided convolution that summarises each block of pixels as one vector (a token), followed by a LayerNorm that standardises each token on its own, and at stage 1 the blocks must not overlap so that a hidden token leaves no trace in its neighbours.

Chapter 12 left you with a `(B, 32, 128, 128)` tensor: 32 learned spectral mixtures at every pixel. The next module is the entrance to the transformer. Transformers were invented for text, where a sentence is a list of tokens (words or word pieces), each represented by a vector of numbers. To use one on an image you must decide what a token is, and turn the picture into a list of them. This chapter does that for all four stages of Indradhanu's encoder, works a tiny example by hand, derives LayerNorm on a four-number token, and counts every parameter.

13.1 The problem: pixels are too many tokens

The simplest choice would make every pixel a token: $128 \times 128 = 16{,}384$ tokens of 32 numbers. The transformer's central operation, attention (Chapter 14), compares every token with every other, so its cost grows with the square of the number of tokens:

$$\begin{aligned} \text{pixel tokens:} \quad & 16{,}384^2 = 268{,}435{,}456 \text{ pairs}, \\ 4 \times 4 \text{ blocks:} \quad & 1{,}024^2 = 1{,}048{,}576 \text{ pairs}. \end{aligned}$$

Grouping 4×4 pixels into one token divides the token count by 16 and the number of pairs by $16^2 = 256$ [C]. There is a second reason, from Chapter 9: a single hidden pixel is too easy to predict from its eight neighbours, while a hidden 4×4 block forces the model to reason from further away. So the token is a block.

Definition 13.1 (Token, patch embedding)**.** At stage 1 of Indradhanu, a *token* is a 4×4 block of the compressed patch, summarised as a vector of 32 numbers. *Patch embedding* is the operation that cuts a feature map into such blocks and computes one vector per block; the vector is the block's *embedding*.

The word "embedding" is borrowed from language models, where a word is embedded into a vector. Here a block of 4×4 pixels with 32 channels, $4 \times 4 \times 32 = 512$ numbers, becomes a

token of 32 numbers: a 16-to-1 summary. Where does 1024 come from?

$$128/4 = 32 \text{ blocks across},$$
$$128/4 = 32 \text{ blocks down},$$
$$32 \times 32 = 1{,}024 \text{ tokens}.$$

Analogy

Patch embedding is the executive summary of a long report. Each page (a 4×4 block) is replaced by a short paragraph (32 numbers) written to a fixed template (the learned kernel), and the board (the transformer) reads the paragraphs, not the pages. The template is the same for every page, so a page about wheat gets the same kind of summary wherever it sits in the report. And, as Section 13.4.1 shows, if you want to test the board on a page whose summary you withheld, no other summary may quote from that page.

13.2 A strided convolution makes the tokens

The module is `OverlapPatchEmbedding` in `app/foundation_models/components/overlap_patch_embedding.py`. Its core:

```
self.proj = nn.Conv2d(c_in, c_out, kernel_size=patch_size,
                      stride=desired_compression, padding=padding)
self.norm = nn.LayerNorm(c_out)

def forward(self, x):                    # x: (B, C_in, H, W)
    x = self.proj(x)                     # x: (B, c_out, H', W')
    B, _, H_dash, W_dash = x.shape
    x = x.flatten(2)                     # x: (B, c_out, N),  N = H' * W'
    x = x.transpose(1, 2)                # x: (B, N, c_out)
    out = self.norm(x)                   # LayerNorm over each token's c_out features
    return out, H_dash, W_dash
```

At stage 1 the convolution has kernel $k = 4$, stride $s = 4$ and padding $p = 0$. From Chapter 10 you know what that means: the kernel's windows sit at rows and columns $0, 4, 8, \ldots$, each covering exactly one 4×4 block, with no gap and no overlap. Each window position produces one output vector of $C_{\text{out}} = 32$ numbers: one token.

13.2.1 The convolution is a matrix applied to each flattened block

Write out Equation (10.6) for this case. Token (i, j) of the 32×32 grid, output channel o:

$$t_o(i,j) = b_o + \sum_{c=0}^{31} \sum_{u=0}^{3} \sum_{v=0}^{3} K_{o,c,u,v}\, x_{c,\,4i+u,\,4j+v}. \tag{13.1}$$

The triple sum runs over the $32 \times 4 \times 4 = 512$ numbers of block (i, j), each paired with its own weight. Line up those 512 numbers as one vector $\mathbf{p}_{ij} \in \mathbb{R}^{512}$ (the flattened block), and line up the matching weights of output channel o as row o of a matrix $\mathbf{E} \in \mathbb{R}^{32 \times 512}$. Then Equation (13.1) is

$$\mathbf{t}_{ij} = \mathbf{E}\,\mathbf{p}_{ij} + \mathbf{b}. \tag{13.2}$$

A non-overlapping patch embedding is a linear layer applied to every flattened block, with the same $\mathbf{E}$ for all 1024 blocks. Its parameter count is that of a linear layer from 512 to 32, $512 \times 32 + 32 = 16{,}416$, identical to the convolution count $32 \times 32 \times 4 \times 4 + 32$. The original

Vision Transformer [5] described patch embedding in exactly this form, "flatten each patch and apply a linear projection"; the convolution is the efficient way to compute it.

Worked example 13.1: Four tokens from a 4×4 image

Take one channel, a 4×4 image, kernel 2, stride 2, no padding, and two output channels:

$$x = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 3 & 4 & 1 & 0 \\ 0 & 0 & 2 & 2 \\ 1 & 1 & 2 & 2 \end{pmatrix}, \quad K_0 = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}, \quad K_1 = \begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix},$$

biases zero. K_0 averages a block; K_1 is "left column minus right column". The output grid is $\lfloor (4-2)/2 \rfloor + 1 = 2$ on a side: four tokens of two numbers each.

Top-left block $\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$:

$$t_0 = 0.25 \times (1+2+3+4) = 0.25 \times 10 = 2.5,$$
$$t_1 = (1+3) - (2+4) = 4 - 6 = -2.$$

Top-right block $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$:

$$t_0 = 0.25 \times (0+1+1+0) = 0.25 \times 2 = 0.5,$$
$$t_1 = (0+1) - (1+0) = 0.$$

Bottom-left block $\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix}$:

$$t_0 = 0.25 \times 2 = 0.5, \qquad t_1 = (0+1) - (0+1) = 0.$$

Bottom-right block $\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$:

$$t_0 = 0.25 \times 8 = 2, \qquad t_1 = (2+2) - (2+2) = 0.$$

After `flatten(2)` and `transpose(1,2)`, in row-major order (top-left, top-right, bottom-left, bottom-right), the token sequence of shape `(1, 4, 2)` is

$$\begin{pmatrix} 2.5 & -2 \\ 0.5 & 0 \\ 0.5 & 0 \\ 2 & 0 \end{pmatrix}.$$

Each row is one token: "how bright" and "how much brighter on the left". PyTorch gives the same matrix [C]. In the real model the 32 recipes are learned, not chosen.

13.2.2 Why the module returns the grid size

After `flatten(2)`, the fact that the 1024 tokens came from a 32×32 grid is gone from the tensor's shape. Later modules need it: attention's spatial reduction (Chapter 15) and the feed-forward network's depthwise convolution (Chapter 16) both reshape the token list back into a grid, do something spatial, and flatten again. So the module returns `H_dash` and `W_dash`

alongside the tokens, and you will see `forward(self,x,H,W)` in every later signature.

13.3 LayerNorm: each token standardised on its own

The last line of the module is a LayerNorm [1]. It looks like BatchNorm (Chapter 12) and differs from it in one decisive way: the axis it averages over.

Definition 13.2 (Layer normalisation)**.** For one token $\mathbf{t} = (t_1, \ldots, t_C)$, *layer normalisation* computes the mean and the (biased) variance of that token's own C features,

$$\mu = \frac{1}{C}\sum_{f=1}^{C} t_f, \qquad \sigma^2 = \frac{1}{C}\sum_{f=1}^{C}(t_f - \mu)^2, \tag{13.3}$$

standardises, and applies a learned scale γ_f and shift β_f *per feature*:

$$\mathrm{LN}(\mathbf{t})_f = \gamma_f \, \frac{t_f - \mu}{\sqrt{\sigma^2 + \varepsilon}} + \beta_f, \qquad \varepsilon = 10^{-5}. \tag{13.4}$$

	BatchNorm (Chapter 12)	LayerNorm
statistics computed over	the batch and all pixels, per channel	one token's features
one output depends on	other examples in the batch	its own token only
train and eval behave	differently (running statistics)	identically
learned parameters	$2\times$ channels	$2\times$ features

Worked example 13.2: LayerNorm on a four-number token

Token $\mathbf{t} = (2,\ 4,\ 4,\ 6)$, with $\gamma = (1,1,1,1)$ and $\beta = (0,0,0,0)$, the initial values. Do this on paper.
Mean of the token's own features:

$$2 + 4 + 4 + 6 = 16,$$
$$\mu = 16/4 = 4.$$

Deviations: $2-4=-2$, $4-4=0$, $4-4=0$, $6-4=2$. Squares: $4,\ 0,\ 0,\ 4$. Sum:

$$4 + 0 + 0 + 4 = 8.$$

Variance:

$$\sigma^2 = 8/4 = 2.$$

Denominator (the ε changes only the sixth decimal):

$$\sqrt{2 + 0.00001} = 1.41422.$$

Outputs:

$$-2/1.41422 = -1.4142, \qquad 0/1.41422 = 0,$$
$$0/1.41422 = 0, \qquad 2/1.41422 = 1.4142.$$

Now let training have moved the parameters to $\gamma = (1,1,2,2)$ and $\beta = (0,0,0,1)$. Apply Equation (13.4) feature by feature:

$$1 \times (-1.4142) + 0 = -1.4142,$$
$$1 \times 0 + 0 = 0,$$
$$2 \times 0 + 0 = 0,$$
$$2 \times 1.4142 + 1 = 2.8284 + 1 = 3.8284.$$

PyTorch gives $(-1.4142, 0, 0, 1.4142)$ and $(-1.4142, 0, 0, 3.8284)$ [C].

13.3.1 What LayerNorm keeps and what it discards

LayerNorm keeps the *pattern* across a token's features and discards the token's overall level and spread. Two tokens that differ only by a constant shift or a positive scale come out identical: for $\mathbf{t}' = a\mathbf{t} + c$ with $a > 0$, the mean becomes $a\mu + c$, the deviations become $a(t_f - \mu)$, the standard deviation becomes $a\sigma$, and the ratio is unchanged (up to ε). The tiny tokens of worked example 13.1 show the extreme case. With only two features, every token whose two values differ normalises to $(1, -1)$ or $(-1, 1)$:

$$\begin{aligned} (2.5,\ -2): &\quad \mu = 0.25,\ \text{deviations } \pm 2.25,\ \sigma = 2.25 \ \Rightarrow\ (1,\ -1), \\ (0.5,\ 0): &\quad \mu = 0.25,\ \text{deviations } \pm 0.25,\ \sigma = 0.25 \ \Rightarrow\ (1,\ -1), \end{aligned}$$

up to the fifth decimal (the ε matters slightly when σ is small: PyTorch gives 0.99992 for the second [C]). A bright block with a strong left-right edge and a faint flat block become indistinguishable. With 32 features the loss is far milder, since only 2 of 32 degrees of freedom are removed, but the lesson stands: after LayerNorm, a token's absolute brightness lives only in how its features differ from one another. The learned γ and β can restore a fixed scale and offset per feature, not the per-token ones.

13.3.2 Why LayerNorm and not BatchNorm here

The code's own comment: "BatchNorm normalizes across the batch per-channel – wrong for sequences". Two reasons make it concrete for this model. First, BatchNorm couples each token's output to whatever else is in the batch, so a scene would be encoded differently depending on its batch-mates. Second, masking (Chapter 18) deletes a different random set of tokens in every patch, so the set of tokens a batch statistic would be computed over changes from step to step; LayerNorm, which never looks beyond one token, is indifferent to how many tokens survive.

13.4 Four stages, two kinds of window

Every encoder stage begins with its own patch embedding. Stages 2 to 4 do not cut up the original image; they cut up the previous stage's output grid, after it has been reshaped back to `(B, C, H, W)`. The encoder builds them in `app/foundation_models/components/seg_former_encoder.py`, lines 108–123:

```
for i in range(self.num_stages):
    c_in = in_channels if i == 0 else embed_dims[i - 1]
    patch_size = 4 if i == 0 else 3
    stride = 4 if i == 0 else 2
    padding = 0 if i == 0 else None      # None means patch_size // 2
```

with `embed_dims=[32,64,160,256]` and, for Indradhanu, `in_channels` $= D = 32$.

Each stage halves the grid in each direction, so the token count falls by four each time: 1024, 256, 64, 16. Meanwhile the token width grows: 32, 64, 160, 256. Fewer, richer tokens at coarser scales.

13.4.1 Overlapping and non-overlapping windows

When $k = s$ the windows tile the input: each pixel belongs to exactly one window. When $k > s$ they overlap by $k - s$ pixels, and a pixel can feed several tokens. Plain Vision Trans-

Table 13.1: The four patch embeddings for a 128×128 patch. Sizes from Equation (10.8); every shape checked by running the real encoder (`tools/compute_shapes.py`).

stage	k	s	p	in	grid	input shape	tokens out
1	4	4	0	32	$128 \to 32$	`(B, 32, 128, 128)`	`(B, 1024, 32)`
2	3	2	1	32	$32 \to 16$	`(B, 32, 32, 32)`	`(B, 256, 64)`
3	3	2	1	64	$16 \to 8$	`(B, 64, 16, 16)`	`(B, 64, 160)`
4	3	2	1	160	$8 \to 4$	`(B, 160, 8, 8)`	`(B, 16, 256)`

formers use non-overlapping windows; SegFormer [46] normally overlaps them, which preserves continuity across token boundaries and avoids visible seams. That is where the module's name, *Overlap*PatchEmbedding, comes from.

Indradhanu uses both, on purpose. Stages 2 to 4 overlap ($k = 3$, $s = 2$, overlap 1). Stage 1 does not ($k = s = 4$, $p = 0$). The encoder's comment gives the reason in one line: "NON-OVERLAPPING - zero information leakage for MAE token removal. Each token sees exactly its own 4x4 block."

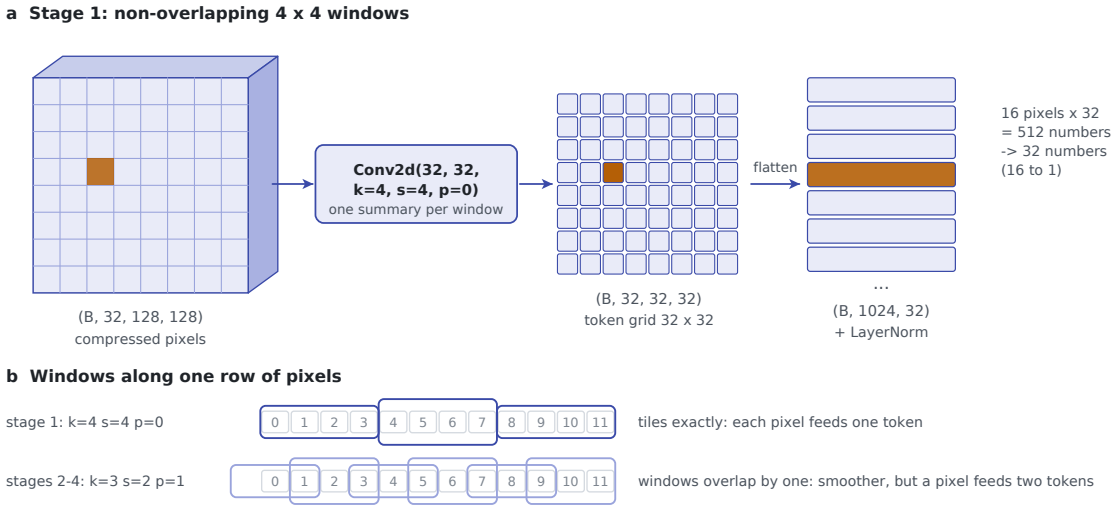


Figure 13.1: Turning pixels into tokens. (a) At stage 1 a `Conv2d` with kernel 4 and stride 4 summarises each 4×4 block of the 32-channel input as one 32-number token (amber: one block and its token), giving a 32×32 token grid that is flattened to 1,024 tokens. (b) Stage 1's windows tile exactly, so each pixel feeds one token, which hiding requires; stages 2 to 4 use kernel 3, stride 2, padding 1, whose windows overlap by one. (Course figure IW 11.1.)

13.4.2 Why stage 1 must not overlap: counting the leak

Stage 1 is the only place where tokens are hidden (Chapters 9 and 18). Suppose stage 1's windows overlapped. Hide token B, keep its neighbour A; any pixel of B's block that also lies in A's window still reaches the encoder through A. The model could partly see what it is graded on, and by Equation (9.10) every such leak shrinks the anomaly evidence.

You can count the leak along one axis. A window of token n starts at padded position ns, that is at original pixel $ns - p$, and covers k pixels. Count how many windows cover each pixel.

Worked example 13.3: How many windows see each pixel

Pixels 0 to 11 of one row.
The real stage 1, $k = 4$, $s = 4$, $p = 0$. Windows: $n = 0$ covers 0..3, $n = 1$ covers 4..7, $n = 2$ covers 8..11. Every pixel is in exactly one window:

1 1 1 1 1 1 1 1 1 1 1 1.

The stale design, $k = 7$, $s = 4$, $p = 3$. Window n covers $4n-3$ to $4n+3$: $n = 0$ covers -3..3 (only 0..3 are real), $n = 1$ covers 1..7, $n = 2$ covers 5..11, $n = 3$ covers 9..15. Counting:

1 2 2 2 1 2 2 2 1 2 2 2.

Take token $n = 1$, whose own block is pixels 4..7. Pixels 5, 6 and 7 also lie in window $n = 2$. If token 1 is hidden and token 2 is visible, three of the hidden block's four pixels per row still reach the encoder. In two dimensions the same happens along both axes.
Stages 2 to 4, $k = 3$, $s = 2$, $p = 1$. Window n covers $2n - 1$ to $2n + 1$, and the counts alternate:

1 2 1 2 1 2 1 2 1 2 1 2.

Half the pixels feed two tokens. That is harmless there, because nothing is hidden after stage 1. The compute script reproduces all three rows [C].

The average number of windows covering a pixel is k/s per axis: $4/4 = 1$ for the real stage 1, $7/4 = 1.75$ for the stale design, $3/2 = 1.5$ for stages 2 to 4. In two dimensions square it: 1, 3.06 and 2.25.

There is a second, quieter reason for $p = 0$ at stage 1: the masking code turns a token mask into a pixel mask by blowing each token up into a 4×4 block of pixels (Chapter 18). That is exact only when token (i, j) is computed from pixels $4i..4i + 3$ by $4j..4j + 3$ and nothing else, which is what $k = s = 4$, $p = 0$ guarantees. Worked example 10.6 showed that the default padding $p = 2$ would even change the grid to 33×33.

Source check: stale "kernel 7" descriptions of stage 1

The encoder builds stage 1 with kernel 4, stride 4, padding 0 (`app/foundation_models/components/seg_former_encoder.py`, lines 110–114; confirmed by printing the built module: `Conv2d(32, 32, kernel_size=(4, 4), stride=(4, 4))` [C]). Three pieces of prose disagree:

- the header of `app/foundation_models/components/overlap_patch_embedding.py` (lines 8–10): "Example for Stage 1 … kernel=7, stride=4 -> overlap = 7 - 4 = 3 pixels shared between neighbors, (B, 1, 128, 128) -> (B, 1024, 64)". Kernel, overlap, input channels and token width are all stale; the real stage 1 maps `(B, 32, 128, 128)` to `(B, 1024, 32)` with no overlap;
- the class docstring of the same file (line 27) describes the convolution as using `padding=patch_size//2`, which is false for stage 1;
- the docstring of `erode_mask` in `app/foundation_models/components/token_masking.py` (lines 47–62): "Should match OPE kernel_size (7 for Stage 1)", with a worked example built on a 7×7 kernel.

The prose is left over from an earlier design that used overlapping stage-1 patches; it was changed, presumably to close exactly the leak of worked example 13.3, and the comments were not updated. The code is right and the comments are stale. Do not "fix" the code to match them.

13.5 Parameter counts, derived and checked

Two formulas from earlier chapters:

$$\begin{aligned} \text{Conv2d}(c_{\text{in}}, c_{\text{out}}, k): &\quad c_{\text{in}} \cdot c_{\text{out}} \cdot k \cdot k + c_{\text{out}}, \\ \text{LayerNorm}(c): &\quad 2c \quad \text{(one } \gamma \text{ and one } \beta \text{ per feature).} \end{aligned}$$

Worked example 13.4: All four patch embeddings

Stage 1, $32 \to 32$, $k = 4$:

$$\begin{aligned} \text{weights} &= 32 \times 32 \times 4 \times 4 = 1{,}024 \times 16 = 16{,}384, \\ \text{biases} &= 32, \\ \text{conv} &= 16{,}384 + 32 = 16{,}416, \\ \text{LayerNorm} &= 2 \times 32 = 64, \\ \text{stage 1} &= 16{,}416 + 64 = 16{,}480. \end{aligned}$$

Stage 2, $32 \to 64$, $k = 3$:

$$\begin{aligned} \text{weights} &= 32 \times 64 \times 9 = 2{,}048 \times 9 = 18{,}432, \\ \text{conv} &= 18{,}432 + 64 = 18{,}496, \\ \text{stage 2} &= 18{,}496 + 2 \times 64 = 18{,}496 + 128 = 18{,}624. \end{aligned}$$

Stage 3, $64 \to 160$, $k = 3$:

$$\begin{aligned} \text{weights} &= 64 \times 160 \times 9 = 10{,}240 \times 9 = 92{,}160, \\ \text{conv} &= 92{,}160 + 160 = 92{,}320, \\ \text{stage 3} &= 92{,}320 + 2 \times 160 = 92{,}320 + 320 = 92{,}640. \end{aligned}$$

Stage 4, $160 \to 256$, $k = 3$:

$$\begin{aligned} \text{weights} &= 160 \times 256 \times 9 = 40{,}960 \times 9 = 368{,}640, \\ \text{conv} &= 368{,}640 + 256 = 368{,}896, \\ \text{stage 4} &= 368{,}896 + 2 \times 256 = 368{,}896 + 512 = 369{,}408. \end{aligned}$$

All four:

$$\begin{aligned} 16{,}480 + 18{,}624 &= 35{,}104, \\ 35{,}104 + 92{,}640 &= 127{,}744, \\ 127{,}744 + 369{,}408 &= 497{,}152. \end{aligned}$$

All four match the `torchinfo` table and the built model [C]. The four patch embeddings are 497,152/5,506,629 $\approx$ 9.0% of the model, and stage 4's alone is three quarters of that: its kernel sees $160 \times 9 = 1{,}440$ inputs per output, and it has 256 outputs. Do not confuse these LayerNorms with the separate LayerNorm at the *end* of each stage (64, 128, 320 and 512 parameters, `self.norms` in the encoder), which Chapter 17 covers.

Table 13.2: Patch-embedding parameters: by hand, in the `torchinfo` table of `research/model_break_down/05_hyperspectral_segformer_mae.md`, and from the built model.

stage	conv	conv params	LN	total	torchinfo
1	$32 \to 32$, $k = 4$	16,416	64	16,480	16,480
2	$32 \to 64$, $k = 3$	18,496	128	18,624	18,624
3	$64 \to 160$, $k = 3$	92,320	320	92,640	92,640
4	$160 \to 256$, $k = 3$	368,896	512	369,408	369,408
				497,152	

torchinfo column [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`]

13.5.1 What the compressor bought, in this layer

Without the spectral compressor, stage 1 would read 165 channels:

$$\begin{aligned} 165 \times 32 \times 16 &= 84{,}480, \\ 84{,}480 + 32 &= 84{,}512 \text{ (conv)}, \\ 84{,}512 + 64 &= 84{,}576 \text{ (with LayerNorm)}. \end{aligned}$$

Against 16,480, that is $84{,}576/16{,}480 \approx 5.1$ times as many parameters, on the stage that runs at the finest resolution [C]. The compressor itself costs 5,376 (Chapter 12), so the trade saves $84{,}576 - 16{,}480 - 5{,}376 = 62{,}720$ parameters here, besides the memory.

13.6 Common confusions

Is a token the same as a pixel?
No. At stage 1 one token summarises a 4×4 block, 16 pixels. At stage 4 the grid spacing is 32 pixels (Chapter 10).

Does every stage cut the original image into blocks?
No. Only stage 1 reads the (compressed) image. Stages 2 to 4 cut up the previous stage's output grid.

Why "Overlap" patch embedding when stage 1 does not overlap?
The module is SegFormer's, whose default is to overlap. Indradhanu passes $k = s = 4$, $p = 0$ at stage 1 to switch the overlap off there, and keeps it at stages 2 to 4.

Could a plain reshape replace the convolution?
A reshape can cut the blocks out, and the original Vision Transformer then applied a linear layer to each, which is the same computation as the non-overlapping convolution (Equation (13.2)). A reshape alone would give 512-number tokens with no learned summary.

Does LayerNorm mix information between tokens?
No. Each token is standardised using only its own features; the transformer blocks do the mixing.

13.7 Check yourself

Exercise 13.1. How many stage-1 tokens does a 96×96 patch produce? How many tokens at each of stages 2 to 4?

Exercise 13.2. Compute the parameters of `Conv2d(64,160,kernel_size=3)` plus its LayerNorm, and of a hypothetical stage 1 that read 24 compressed channels ($24 \to 32$, $k = 4$).

Exercise 13.3. Apply LayerNorm (with $\gamma = 1$, $\beta = 0$, ε negligible) to the token $(1,\ 3,\ 5,\ 7)$. Then to $(10,\ 30,\ 50,\ 70)$. Explain the result.

Exercise 13.4. Repeat the tiny patch embedding of worked example 13.1 for the image

$$\begin{pmatrix} 4 & 0 & 1 & 1 \\ 4 & 0 & 1 & 1 \\ 2 & 2 & 0 & 3 \\ 2 & 2 & 0 & 3 \end{pmatrix},$$

giving the token sequence.

Exercise 13.5. For a 1-D stage-1 design with $k = 6$, $s = 4$, $p = 1$, list which pixels 0..11 each window covers and count the windows per pixel. If token 1 is hidden and its neighbours are visible, how many of its block's four pixels leak?

Exercise 13.6. Why does `OverlapPatchEmbedding` return `H_dash` and `W_dash`, and why does it use LayerNorm rather than BatchNorm?

Next: Chapter 14, Attention, from one query to many heads.

Chapter 14

Attention, from one query to many heads

The one thing this chapter teaches

Attention is a similarity-weighted average: every token scores every other token with a dot product, turns the scores into proportions that add up to one with a softmax, and replaces itself with the matching blend of the other tokens' values.

This chapter builds ordinary self-attention from nothing. Chapter 15 then shows the one change SegFormer makes to it, and Chapter 16 wraps it into the repeating unit of the encoder. Every formula here is followed by a calculation small enough to do on paper; please do them on paper, with a pencil, because the arithmetic is the understanding. All the numbers in this chapter were also checked by a small script, `research/textbook/tools/compute_13-15.py`, whose output is saved next to it.

14.1 The problem: a token needs its surroundings

After the patch embedding of Chapter 13, a 128×128 patch has become a grid of $32 \times 32 = 1024$ *tokens*. A token is a short vector of numbers (32 of them at stage 1) that summarises one 4×4 block of pixels. Recall from Chapter 10 that the encoder holds all the tokens of a batch in a tensor of shape `(B, N, C)`: B patches in the batch, N tokens per patch, C numbers per token.

So far each token knows only its own block. But the meaning of a block depends on what surrounds it. A dark block in the middle of a lake is ordinary water. The identical dark block in the middle of a bright desert is exactly the kind of thing an anomaly detector exists to notice. If the model is to reconstruct a block from its context (the whole idea of Chapter 9), each token must be able to look at other tokens and revise itself.

The obvious fixes, and why they fail.

1. *Average the neighbours with fixed weights.* That is a convolution (Chapter 10). Its weights are learned once and then applied identically everywhere, whatever the content. A convolution cannot decide that, for *this* dark block, the relevant neighbours are the other dark blocks two rows away rather than the bright ones next door.

2. *Learn one weight for every pair of tokens.* With $N = 1024$ tokens that is $1024 \times 1024 = 1{,}048{,}576$ weights, and they would still be fixed: pair (3, 517) would get the same weight in a lake patch and in a desert patch.

What we want is weights that are *computed from the content*, freshly for every patch. That is what attention provides.

Analogy

You walk into a library with a question. You do not read every book with equal care, and you do not consult a fixed list of books regardless of the question. You read the titles on the spines (keys), compare each with your question (query), and spend your time on the books whose titles match best, taking from each a share of its contents (values) in proportion to how well it matched. Change the question and you read a different blend of books. Every token in the encoder is, at the same time, a reader with a question and a book on the shelf.

The mechanism has three ingredients: a way to measure similarity (the dot product, Section 14.2), a way to turn similarities into proportions (the softmax, Section 14.3), and a way to blend (the weighted average, Section 14.4). Then we assemble them in matrix form, explain the scaling by $\sqrt{d}$, and split the work across several heads.

14.2 Similarity: the dot product

Definition 14.1 (Dot product)**.** For two vectors $\mathbf{a} = (a_1, \ldots, a_d)$ and $\mathbf{b} = (b_1, \ldots, b_d)$ of the same length d, the dot product is

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^\top \mathbf{b} = \sum_{i=1}^{d} a_i b_i. \tag{14.1}$$

Chapter 6 met it already; here is the property attention relies on. Writing $\|\mathbf{a}\| = \sqrt{\mathbf{a} \cdot \mathbf{a}}$ for the length of a vector and θ for the angle between the two vectors,

$$\mathbf{a} \cdot \mathbf{b} = \|\mathbf{a}\| \, \|\mathbf{b}\| \cos\theta. \tag{14.2}$$

So the dot product is large and positive when the vectors point the same way ($\cos\theta$ near 1), near zero when they are at right angles, and negative when they point in opposite directions. It is a similarity score, and it grows with the lengths of the vectors too, which will matter in Section 14.7.

Worked example 14.1: Three dot products

Take the query $\mathbf{q} = (1, 0)$ and three keys $\mathbf{k}_1 = (1, 0)$, $\mathbf{k}_2 = (0, 1)$, $\mathbf{k}_3 = (1, 1)$. Multiply component by component and add.

$$\begin{aligned} \mathbf{q} \cdot \mathbf{k}_1 &= 1 \cdot 1 + 0 \cdot 0 = 1 + 0 = 1, \\ \mathbf{q} \cdot \mathbf{k}_2 &= 1 \cdot 0 + 0 \cdot 1 = 0 + 0 = 0, \\ \mathbf{q} \cdot \mathbf{k}_3 &= 1 \cdot 1 + 0 \cdot 1 = 1 + 0 = 1. \end{aligned}$$

The query "asks about" the first feature. Keys 1 and 3 both have a first feature of 1, so they score 1; key 2 has none, so it scores 0. Note that $\mathbf{k}_3$ is longer ($\|\mathbf{k}_3\| = \sqrt{2}$) but the score depends only on the part of $\mathbf{k}_3$ that lies along $\mathbf{q}$.

14.3 From scores to proportions: the softmax

14.3.1 What we need, and the formula that delivers it

Scores can be any real numbers, negative included. To take a weighted average we need weights that are

1. *positive*, so that no token is subtracted;
2. *summing to one*, so that the result is an average and not a sum that grows with the number of tokens;
3. *order-preserving*, so that a higher score always gets a higher weight;
4. *smooth*, so that a small change in a score makes a small change in the weights (training needs derivatives, Chapter 26).

Step 1, make every score positive while keeping the order: apply the exponential function e^s, which is positive for every real s and strictly increasing. Step 2, make them sum to one: divide each by their total. The result is the softmax.

Definition 14.2 (Softmax)**.** For scores $\mathbf{s} = (s_1, \ldots, s_n)$,

$$\operatorname{softmax}(\mathbf{s})_i = \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}}, \qquad i = 1, \ldots, n. \tag{14.3}$$

Check the four requirements. Each numerator is positive and so is the denominator, so every weight is positive. Adding all n weights gives $\sum_i e^{s_i} / \sum_j e^{s_j} = 1$. Because e^s is increasing, $s_a > s_b$ gives $e^{s_a} > e^{s_b}$ and so a larger weight. And e^s is smooth.

The name says what it does: it is a "soft" version of picking the maximum. The largest score gets the largest share, but every score gets something.

Worked example 14.2: Softmax of three scores

Take $\mathbf{s} = (2, 1, 0)$. Do this on paper with a calculator for e^x.
Exponentiate each score.

$$e^2 = 7.3891, \qquad e^1 = 2.7183, \qquad e^0 = 1.0000.$$

Add them.

$$7.3891 + 2.7183 + 1.0000 = 11.1074.$$

(To four decimals the exact total is 11.1073; the last digit differs because each term was rounded first.)
Divide each by the total.

$$7.3891/11.1073 = 0.6652,$$
$$2.7183/11.1073 = 0.2447,$$
$$1.0000/11.1073 = 0.0900.$$

Check the sum. $0.6652 + 0.2447 + 0.0900 = 0.9999$, which is 1 up to rounding.
The scores were evenly spaced (2, 1, 0), but the weights are not: each step down in score divides the weight by $e \approx 2.718$. Softmax turns *differences* of scores into *ratios* of weights.

That last remark is worth making exact. For any two positions a and b,

$$\begin{aligned} \frac{\operatorname{softmax}(\mathbf{s})_a}{\operatorname{softmax}(\mathbf{s})_b} &= \frac{e^{s_a} / \sum_j e^{s_j}}{e^{s_b} / \sum_j e^{s_j}} && \text{write out both weights} \\ &= \frac{e^{s_a}}{e^{s_b}} && \text{the common denominator cancels} \\ &= e^{s_a - s_b} && \text{rule } e^x/e^y = e^{x-y}. \end{aligned} \tag{14.4}$$

A score lead of 1 means a weight ratio of $e = 2.718$; a lead of 2 means $e^2 = 7.389$; a lead of 11 means about 60,000.

14.3.2 Shifting all scores changes nothing

Adding the same constant to every score leaves the softmax unchanged; only the differences between scores matter.

Proof. Let c be any number and add it to every score.

$$\begin{aligned}
\text{softmax}(\mathbf{s}+c)_i &= \frac{e^{s_i+c}}{\sum_j e^{s_j+c}} && \text{definition, with } s_j+c \text{ in place of } s_j \\
&= \frac{e^{s_i}e^{c}}{\sum_j e^{s_j}e^{c}} && \text{rule } e^{x+y}=e^x e^y \\
&= \frac{e^{c}\,e^{s_i}}{e^{c}\sum_j e^{s_j}} && e^c \text{ does not depend on } j, \text{ so it comes out of the sum} \\
&= \frac{e^{s_i}}{\sum_j e^{s_j}} = \text{softmax}(\mathbf{s})_i && \text{cancel } e^c \neq 0. \quad \blacksquare
\end{aligned}$$

This agrees with Equation (14.4): the ratios depend only on differences.

14.3.3 Numerical stability: subtract the maximum

Computers store these numbers as 32-bit floating-point values (*float32*). The largest float32 is about 3.4028×10^{38}, and $\ln(3.4028 \times 10^{38}) = 88.72$. So e^s for any score above 88.72 cannot be stored; it becomes "infinity". In PyTorch, `torch.exp(torch.tensor(89.0))` returns `inf`. Two infinities divided give "not a number", and one such value poisons a whole training run.

The cure follows from Section 14.3.2. Choose $c = -\max_j s_j$: subtract the largest score from all of them. The softmax is unchanged, but now the largest shifted score is exactly 0, so the largest exponential is $e^0 = 1$ and nothing can overflow. Every other exponential lies between 0 and 1.

Worked example 14.3: The stable softmax

First redo worked example 14.2 the stable way. The maximum of $(2, 1, 0)$ is 2.
Subtract the maximum. $(2-2,\ 1-2,\ 0-2) = (0, -1, -2)$.
Exponentiate. $e^0 = 1.0000$, $e^{-1} = 0.3679$, $e^{-2} = 0.1353$.
Add. $1.0000 + 0.3679 + 0.1353 = 1.5032$.
Divide. $1.0000/1.5032 = 0.6652$, $0.3679/1.5032 = 0.2447$, $0.1353/1.5032 = 0.0900$.
The same weights as before, as the proof promised. Now a case where it matters: scores $(1000, 999)$. Directly, e^{1000} overflows. Stably:

$$\begin{aligned}
(1000-1000,\ 999-1000) &= (0, -1), \\
e^0 = 1.0000, \quad e^{-1} &= 0.3679, \\
1.0000 + 0.3679 &= 1.3679, \\
1.0000/1.3679 = 0.7311, \quad 0.3679&/1.3679 = 0.2689.
\end{aligned}$$

PyTorch's `F.softmax`, which the model calls, does this subtraction internally.

14.3.4 Saturation: when one score takes everything

Stability stops the computer from failing. It does not stop softmax from becoming useless when the scores are spread far apart. Consider scores $(20, 0, -15)$:

$$e^{20} = 485{,}165{,}195.4, \qquad e^{0} = 1, \qquad e^{-15} = 0.000\,000\,306.$$

The total is 485,165,196.4 (the third term does not even reach the first decimal), so the weights are

$$\begin{aligned} 485{,}165{,}195.4/485{,}165{,}196.4 &= 0.999\,999\,998, \\ 1/485{,}165{,}196.4 &= 0.000\,000\,002\,1, \\ 0.000\,000\,306/485{,}165{,}196.4 &= 0.000\,000\,000\,000\,000\,63. \end{aligned}$$

The first token takes the whole average. We say the softmax has *saturated*. Two things go wrong.

1. The output is a copy of one value; no blending happens.
2. Training cannot fix it, because, as we now show, the softmax's derivatives are essentially zero when it is saturated.

The derivative of the softmax. Training adjusts every number in the model in the direction that lowers the loss (Chapter 9 introduced the loss; Chapter 26 does the adjusting). To know that direction it needs derivatives. Write $p_i = \text{softmax}(\mathbf{s})_i$ and $Z = \sum_j e^{s_j}$, so $p_i = e^{s_i}/Z$. We want $\partial p_i/\partial s_k$, how much weight i changes when score k moves.

First, $\partial Z/\partial s_k = e^{s_k}$, because only the k-th term of the sum contains s_k, and the derivative of e^s is e^s.

Case $k = i$. Use the quotient rule $(u/v)' = (u'v - uv')/v^2$ with $u = e^{s_i}$, $v = Z$:

$$\begin{aligned} \frac{\partial p_i}{\partial s_i} &= \frac{e^{s_i}Z - e^{s_i}e^{s_i}}{Z^2} && \text{quotient rule} \\ &= \frac{e^{s_i}}{Z}\cdot\frac{Z - e^{s_i}}{Z} && \text{take out the factor } e^{s_i}/Z \\ &= p_i(1 - p_i) && \text{since } e^{s_i}/Z = p_i. \end{aligned}$$

Case $k \neq i$. Now $u = e^{s_i}$ does not depend on s_k, so $u' = 0$:

$$\frac{\partial p_i}{\partial s_k} = \frac{0\cdot Z - e^{s_i}e^{s_k}}{Z^2} = -\frac{e^{s_i}}{Z}\cdot\frac{e^{s_k}}{Z} = -p_i p_k.$$

Both cases in one line, with δ_{ik} equal to 1 when $i = k$ and 0 otherwise:

$$\frac{\partial p_i}{\partial s_k} = p_i(\delta_{ik} - p_k). \tag{14.5}$$

Every derivative contains a product of two weights. If one weight is nearly 1 and the rest nearly 0, every such product is nearly 0.

Worked example 14.4: Gradients of a healthy and a saturated softmax

Use $\partial p_i/\partial s_i = p_i(1 - p_i)$.
Healthy scores $(1, 0, -1)$. By the stable recipe the weights are $(0.6652, 0.2447, 0.0900)$ (the same as for $(2, 1, 0)$, by Section 14.3.2). Then

$$\begin{aligned} 0.6652 \times (1 - 0.6652) &= 0.6652 \times 0.3348 = 0.2227, \\ 0.2447 \times (1 - 0.2447) &= 0.2447 \times 0.7553 = 0.1848, \\ 0.0900 \times (1 - 0.0900) &= 0.0900 \times 0.9100 = 0.0819. \end{aligned}$$

Saturated scores $(20, 0, -15)$. The weights are $(0.999\,999\,998,\ 2.1\times10^{-9},\ 6.3\times10^{-16})$. Then

$$0.999\,999\,998 \times 0.000\,000\,002 \approx 2.1 \times 10^{-9},$$
$$2.1 \times 10^{-9} \times (1 - 2.1 \times 10^{-9}) \approx 2.1 \times 10^{-9},$$
$$6.3 \times 10^{-16} \times (1 - 6.3 \times 10^{-16}) \approx 6.3 \times 10^{-16}.$$

The healthy softmax passes about a fifth of any error signal back to its scores; the saturated one passes about two parts in a billion. The losing tokens never get the signal that would let them improve, and learning stalls.

So we want scores whose spread is of order 1, not of order 20. Section 14.7 shows where large spreads come from and how one division prevents them.

14.4 One query against many keys

We now have all three ingredients. Give one token a *query* vector $\mathbf{q}$ and give every token $j = 1, \ldots, n$ a *key* $\mathbf{k}_j$ and a *value* $\mathbf{v}_j$, all of length d. Then

$$s_j = \mathbf{q} \cdot \mathbf{k}_j \qquad \text{score: how relevant is token } j? \tag{14.6}$$
$$p_j = \mathrm{softmax}(s_1, \ldots, s_n)_j \qquad \text{weight: its share of attention} \tag{14.7}$$
$$\mathbf{o} = \sum_{j=1}^{n} p_j \mathbf{v}_j \qquad \text{output: the weighted average of values.} \tag{14.8}$$

(The full model also divides each score by $\sqrt{d}$; we add that in Section 14.7, and the worked example below already includes it so that its numbers match the course.)

Worked example 14.5: One query, three tokens of dimension 2

Three tokens, each with two features:

$$\mathbf{x}_1 = (1, 0), \qquad \mathbf{x}_2 = (0, 1), \qquad \mathbf{x}_3 = (1, 1).$$

For now let the query, key and value of every token be the token itself (in the next section they become learned transformations of it). Token 1 asks: $\mathbf{q} = \mathbf{x}_1 = (1, 0)$.
Step 1, scores. From worked example 14.1: $(1, 0, 1)$.
Step 1b, scale by $1/\sqrt{d}$ with $d = 2$. $1/\sqrt{2} = 0.7071$, so the scaled scores are $(0.7071,\ 0,\ 0.7071)$.
Step 2, softmax. Exponentiate:

$$e^{0.7071} = 2.0281, \qquad e^{0} = 1.0000, \qquad e^{0.7071} = 2.0281.$$

Add: $2.0281 + 1.0000 + 2.0281 = 5.0562$. Divide:

$$2.0281/5.0562 = 0.4011, \qquad 1.0000/5.0562 = 0.1978, \qquad 2.0281/5.0562 = 0.4011.$$

Check: $0.4011 + 0.1978 + 0.4011 = 1.0000$.
Step 3, weighted average of the values, one feature at a time. First feature (the first number of each value):

$$0.4011 \times 1 = 0.4011$$
$$0.1978 \times 0 = 0.0000$$
$$0.4011 \times 1 = 0.4011$$
$$\text{sum} = 0.8022.$$

Second feature:

$$\begin{aligned} 0.4011 \times 0 &= 0.0000 \\ 0.1978 \times 1 &= 0.1978 \\ 0.4011 \times 1 &= 0.4011 \\ \text{sum} &= 0.5989. \end{aligned}$$

So $\mathbf{o}_1 = (0.8022,\ 0.5989)$.
Token 1 went in as $(1, 0)$ and came out as $(0.80, 0.60)$. Its second feature was zero and is now 0.60, because it judged tokens 2 and 3, which have a second feature, worth listening to. It has absorbed context.

14.5 Queries, keys and values are learned projections

In worked example 14.5 the query, key and value were all the token itself. That forces a token to search for exactly what it advertises, and to hand over exactly what it is. Real attention separates the three roles with three learned matrices.

Definition 14.3 (Query, key, value)**.** Let $\mathbf{x} \in \mathbb{R}^C$ be a token. With three learned matrices $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{C\times C}$ and bias vectors $\mathbf{b}_Q, \mathbf{b}_K, \mathbf{b}_V \in \mathbb{R}^C$,

$$\mathbf{q} = \mathbf{x}^\top \mathbf{W}_Q + \mathbf{b}_Q^\top, \qquad \mathbf{k} = \mathbf{x}^\top \mathbf{W}_K + \mathbf{b}_K^\top, \qquad \mathbf{v} = \mathbf{x}^\top \mathbf{W}_V + \mathbf{b}_V^\top. \tag{14.9}$$

The query is what the token is looking for, the key is what it advertises about itself, and the value is what it hands over when consulted.

Each is a *linear layer* (Chapter 10): a matrix product plus a bias. The learned numbers in $\mathbf{W}$ and $\mathbf{b}$ are *parameters*: values the training loop adjusts. In code (`app/foundation_models/components/efficient_self_attention.py`, lines 87–94):

```
self.q = nn.Linear(embed_dim, embed_dim)      # (*, C) -> (*, C)
self.k = nn.Linear(embed_dim, embed_dim)
self.v = nn.Linear(embed_dim, embed_dim)
self.proj = nn.Linear(embed_dim, embed_dim)  # output projection, section 7
```

Source check: which way round PyTorch stores the matrix

PyTorch's `nn.Linear(in,out)` stores its weight with shape `(out, in)` and computes $\mathbf{y} = \mathbf{x}\mathbf{W}^\top + \mathbf{b}$, as the source comment at `efficient_self_attention.py` line 89 says. This book writes $\mathbf{x}^\top\mathbf{W}$ with $\mathbf{W}$ of shape (in, out); the two are the same map with $\mathbf{W}$ stored transposed. The parameter count, $\text{in} \times \text{out} + \text{out}$, is the same either way.

Why three separate matrices and not one? Because "what I am looking for" and "what I advertise" are usually different. A token over a road edge might want to find other road pixels (its query), while advertising itself as "bright, linear" (its key) and contributing its exact spectrum-derived features (its value). Separate projections let a token search for one thing while offering another.

14.6 Attention in matrix form

Doing worked example 14.5 once per token is slow to write and slow to compute. Matrices do all tokens at once.

14.6.1 Stacking tokens

Stack the N tokens as the rows of a matrix $\mathbf{X}$ of shape (N, C); row i is token i. Then all queries at once are

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \qquad (N, C)\,(C, C) \to (N, C), \tag{14.10}$$

because row i of a product $\mathbf{XW}$ is (row i of $\mathbf{X}$) times $\mathbf{W}$. (We drop the biases in this section to keep the algebra short; they add the same vector to every row.) Likewise $\mathbf{K} = \mathbf{X}\mathbf{W}_K$ and $\mathbf{V} = \mathbf{X}\mathbf{W}_V$, each (N, C).

14.6.2 All scores at once: $\mathbf{QK}^\top$

The transpose $\mathbf{K}^\top$ swaps rows and columns, so it has shape (C, N) and its column j is key j. Recall that entry (i, j) of a product $\mathbf{AB}$ is the dot product of row i of $\mathbf{A}$ with column j of $\mathbf{B}$. Therefore

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^\top, \qquad (N, C)\,(C, N) \to (N, N), \qquad S_{ij} = \mathbf{q}_i \cdot \mathbf{k}_j. \tag{14.11}$$

Entry (i, j) is "how relevant token j is to token i". Row i holds all of token i's scores.

14.6.3 The whole layer

Definition 14.4 (Scaled dot-product attention)**.**

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}_{\text{row}}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \tag{14.12}$$

where $\text{softmax}_{\text{row}}$ applies Equation (14.3) to each row separately and d is the length of each query and key.

Shapes, step by step:

$\mathbf{QK}^\top/\sqrt{d} : (N, d)(d, N) \to (N, N)$	one score per (query, key) pair
$\mathbf{A} = \text{softmax}_{\text{row}}(\cdot) : (N, N)$	each row sums to 1
$\mathbf{AV} : (N, N)(N, d) \to (N, d)$	one blended value per token.

Row i of $\mathbf{AV}$ is $\sum_j A_{ij}\mathbf{v}_j$, exactly Equation (14.8) for query i. The matrix $\mathbf{A}$ is the *attention matrix*.

This is the operation of Vaswani et al. [44], and it is what the four lines of the model's forward pass compute (`efficient_self_attention.py`, lines 182–195):

```
scores  = Q_x @ K_x.transpose(-2, -1) * self.scale   # (B, h, N, N_red)
weights = F.softmax(scores, dim=-1)                   # rows sum to 1
out     = weights @ V_x                               # (B, h, N, d)
out     = self.proj(out_concatenated)                 # section 7
```

(`@` is matrix multiplication in Python; `dim=-1` means "along the last axis", that is, along each row; `N_red` becomes N until Chapter 15.)

Worked example 14.6: Full attention on three tokens, identity projections

Use the tokens of worked example 14.5 with $\mathbf{W}_Q = \mathbf{W}_K = \mathbf{W}_V = \mathbf{I}$, so

$$\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{pmatrix}.$$

Transpose. $\mathbf{K}^\top = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$.

Scores $\mathbf{QK}^\top$, one entry at a time (row of $\mathbf{Q}$ dotted with column of $\mathbf{K}^\top$):

$$\begin{aligned} S_{11} &= 1\cdot 1 + 0\cdot 0 = 1, & S_{12} &= 1\cdot 0 + 0\cdot 1 = 0, & S_{13} &= 1\cdot 1 + 0\cdot 1 = 1, \\ S_{21} &= 0\cdot 1 + 1\cdot 0 = 0, & S_{22} &= 0\cdot 0 + 1\cdot 1 = 1, & S_{23} &= 0\cdot 1 + 1\cdot 1 = 1, \\ S_{31} &= 1\cdot 1 + 1\cdot 0 = 1, & S_{32} &= 1\cdot 0 + 1\cdot 1 = 1, & S_{33} &= 1\cdot 1 + 1\cdot 1 = 2. \end{aligned}$$

Scale by 0.7071:

$$\frac{\mathbf{S}}{\sqrt{2}} = \begin{pmatrix} 0.7071 & 0 & 0.7071 \\ 0 & 0.7071 & 0.7071 \\ 0.7071 & 0.7071 & 1.4142 \end{pmatrix}.$$

Softmax each row. Row 1 was done in worked example 14.5: $(0.4011, 0.1978, 0.4011)$. Row 2 has the same numbers in a different order, so its weights are $(0.1978, 0.4011, 0.4011)$. Row 3 is new:

$$\begin{gathered} e^{0.7071} = 2.0281, \quad e^{0.7071} = 2.0281, \quad e^{1.4142} = 4.1133, \\ 2.0281 + 2.0281 + 4.1133 = 8.1695, \\ 2.0281/8.1695 = 0.2483, \quad 2.0281/8.1695 = 0.2483, \quad 4.1133/8.1695 = 0.5035. \end{gathered}$$

Check: $0.2483 + 0.2483 + 0.5035 = 1.0001$, one up to rounding. So

$$\mathbf{A} = \begin{pmatrix} 0.4011 & 0.1978 & 0.4011 \\ 0.1978 & 0.4011 & 0.4011 \\ 0.2483 & 0.2483 & 0.5035 \end{pmatrix}.$$

Multiply by $\mathbf{V}$. Row 1 gave $(0.8022, 0.5989)$. Row 2:

$$\begin{aligned} &\text{feature 1: } 0.1978\cdot 1 + 0.4011\cdot 0 + 0.4011\cdot 1 = 0.1978 + 0 + 0.4011 = 0.5989, \\ &\text{feature 2: } 0.1978\cdot 0 + 0.4011\cdot 1 + 0.4011\cdot 1 = 0 + 0.4011 + 0.4011 = 0.8022. \end{aligned}$$

Row 3:

$$\begin{aligned} &\text{feature 1: } 0.2483\cdot 1 + 0.2483\cdot 0 + 0.5035\cdot 1 = 0.2483 + 0 + 0.5035 = 0.7518, \\ &\text{feature 2: } 0.2483\cdot 0 + 0.2483\cdot 1 + 0.5035\cdot 1 = 0 + 0.2483 + 0.5035 = 0.7518. \end{aligned}$$

(The script gives 0.7517 from unrounded weights.) The output is

$$\mathbf{AV} = \begin{pmatrix} 0.8022 & 0.5989 \\ 0.5989 & 0.8022 \\ 0.7518 & 0.7518 \end{pmatrix}.$$

Tokens 1 and 2 are mirror images, as the symmetry of the input demands, and token 3, which resembles both, has become an even blend.

Worked example 14.7: Full attention with real projections

Now let the three roles differ. Keep $\mathbf{X}$ as before and take

$$\mathbf{W}_Q = \begin{pmatrix} 1 & 0 \\ 1 & 1 \end{pmatrix}, \qquad \mathbf{W}_K = \mathbf{I}, \qquad \mathbf{W}_V = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

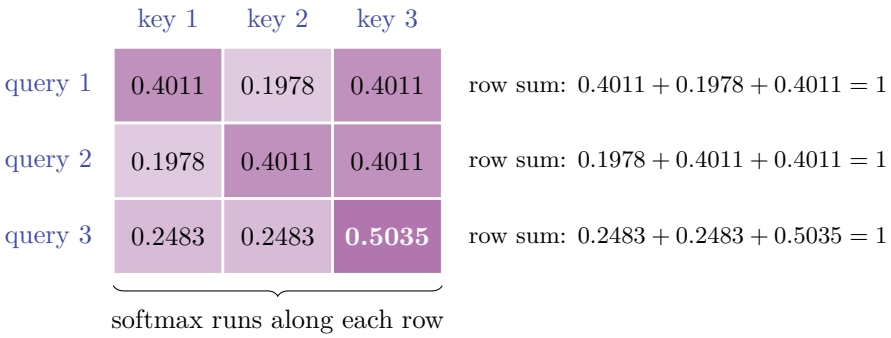


Figure 14.1: The attention matrix $\mathbf{A}$ of worked example 14.6, drawn as a heat map (darker plum = larger weight). Row i is how query i shares its attention among the three keys; every row sums to 1, columns need not. Token 3, which resembles both others, puts half its attention on itself.

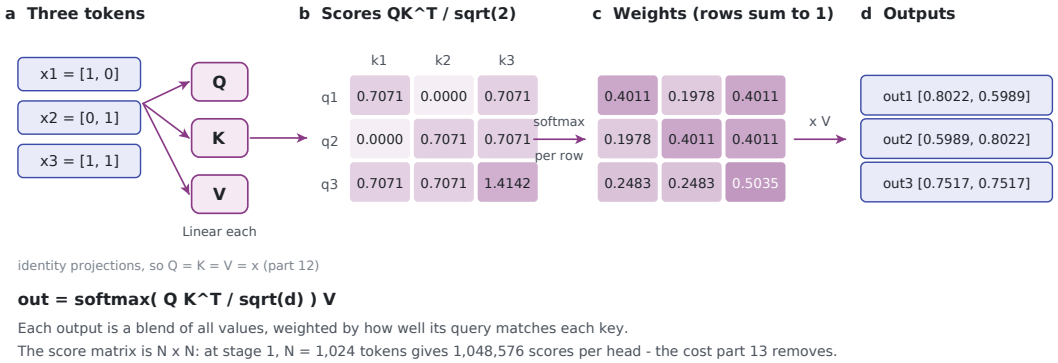


Figure 14.2: The same example as the course draws it: (a) three 2-number tokens with $\mathbf{Q} = \mathbf{K} = \mathbf{V} = \mathbf{X}$; (b) scaled scores $\mathbf{q} \cdot \mathbf{k}/\sqrt{2}$; (c) softmax along each row; (d) each output is the weighted average of the values. Every number matches worked example 14.6.

$\mathbf{W}_V$ swaps the two features: what a token hands over is its features in reverse order.
Queries $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$, row by row (row of $\mathbf{X}$ times each column of $\mathbf{W}_Q$):

$$\begin{aligned}(1,0)\mathbf{W}_Q &= (1\cdot 1 + 0\cdot 1,\ 1\cdot 0 + 0\cdot 1) = (1,0),\\ (0,1)\mathbf{W}_Q &= (0\cdot 1 + 1\cdot 1,\ 0\cdot 0 + 1\cdot 1) = (1,1),\\ (1,1)\mathbf{W}_Q &= (1\cdot 1 + 1\cdot 1,\ 1\cdot 0 + 1\cdot 1) = (2,1).\end{aligned}$$

Keys $\mathbf{K} = \mathbf{X}$. *Values* $\mathbf{V} = \mathbf{X}\mathbf{W}_V$*:* $(0,1)$, $(1,0)$, $(1,1)$.
Scores $\mathbf{Q}\mathbf{K}^\top$*:*

$$\begin{aligned}&\text{row 1, } \mathbf{q} = (1,0): && 1,\ 0,\ 1\\ &\text{row 2, } \mathbf{q} = (1,1): && 1\cdot 1 + 1\cdot 0 = 1,\quad 1\cdot 0 + 1\cdot 1 = 1,\quad 1\cdot 1 + 1\cdot 1 = 2\\ &\text{row 3, } \mathbf{q} = (2,1): && 2\cdot 1 + 1\cdot 0 = 2,\quad 2\cdot 0 + 1\cdot 1 = 1,\quad 2\cdot 1 + 1\cdot 1 = 3.\end{aligned}$$

Scale by 0.7071 and softmax. Rows 1 and 2 are rows we have met: $(0.4011, 0.1978, 0.4011)$ and $(0.2483, 0.2483, 0.5035)$. Row 3, scaled, is $(1.4142, 0.7071, 2.1213)$:

$$\begin{aligned}e^{1.4142} = 4.1133,\quad e^{0.7071} = 2.0281,\quad e^{2.1213} = 8.3421,\\ 4.1133 + 2.0281 + 8.3421 = 14.4835,\\ 4.1133/14.4835 = 0.2840,\quad 2.0281/14.4835 = 0.1400,\quad 8.3421/14.4835 = 0.5760.\end{aligned}$$

Blend the values. Row 1: $0.4011(0,1) + 0.1978(1,0) + 0.4011(1,1)$:

$$\text{feature 1: } 0 + 0.1978 + 0.4011 = 0.5989, \qquad \text{feature 2: } 0.4011 + 0 + 0.4011 = 0.8022.$$

Row 2: $0.2483(0,1) + 0.2483(1,0) + 0.5035(1,1)$ gives $(0.2483 + 0.5035,\ 0.2483 + 0.5035) = (0.7518, 0.7518)$. Row 3: $0.2840(0,1) + 0.1400(1,0) + 0.5760(1,1)$:

$$\text{feature 1: } 0 + 0.1400 + 0.5760 = 0.7160, \qquad \text{feature 2: } 0.2840 + 0 + 0.5760 = 0.8600.$$

Compare with worked example 14.6. Token 2 used to attend mostly to itself and token 3; with its new query $(1,1)$ it now prefers token 3, the one that has both features. The projections changed *who* is listened to (through $\mathbf{W}_Q$) and *what* is heard (through $\mathbf{W}_V$). Training learns these matrices.

14.7 Why divide by $\sqrt{d}$

Section 14.3.4 showed that softmax saturates when the scores are spread over tens of units. Where would such spreads come from? From the length of the vectors.

14.7.1 The variance of a dot product

Recall from Chapter 6 that the *variance* of a random number X is $\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$, the average squared distance from its mean; its square root is the *standard deviation*, the typical size of a deviation.

Assumptions. Model the components of a query and a key as random numbers: $q_1, \ldots, q_d$ and $k_1, \ldots, k_d$, all independent, each with mean 0 and variance 1. (This is roughly what LayerNorm, Chapter 16, and standard weight initialisation produce at the start of training.)

Step 1: one term $q_i k_i$ has mean 0. For independent numbers the mean of a product is the product of the means:

$$\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\,\mathbb{E}[k_i] = 0 \cdot 0 = 0.$$

Step 2: one term has variance 1. Using the definition of variance and then independence again (now for q_i^2 and k_i^2):

$$\begin{aligned}
\operatorname{Var}(q_i k_i) &= \mathbb{E}[(q_i k_i)^2] - (\mathbb{E}[q_i k_i])^2 && \text{definition} \\
&= \mathbb{E}[q_i^2 k_i^2] - 0^2 && \text{step 1} \\
&= \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] && \text{independence} \\
&= 1 \cdot 1 = 1 && \mathbb{E}[q_i^2] = \operatorname{Var}(q_i) + (\mathbb{E}[q_i])^2 = 1 + 0.
\end{aligned}$$

Step 3: add d independent terms. The variance of a sum of independent random numbers is the sum of their variances. The terms $q_1 k_1, \ldots, q_d k_d$ are independent of each other, so

$$\operatorname{Var}(\mathbf{q} \cdot \mathbf{k}) = \sum_{i=1}^{d} \operatorname{Var}(q_i k_i) = \underbrace{1 + 1 + \cdots + 1}_{d \text{ terms}} = d. \tag{14.13}$$

The standard deviation of a raw score is therefore $\sqrt{d}$: it grows with the head dimension.

Step 4: divide by $\sqrt{d}$. For any constant a, $\operatorname{Var}(aX) = a^2 \operatorname{Var}(X)$ (every deviation is multiplied by a, every squared deviation by a^2). With $a = 1/\sqrt{d}$:

$$\operatorname{Var}\left(\frac{\mathbf{q} \cdot \mathbf{k}}{\sqrt{d}}\right) = \frac{1}{d} \cdot d = 1. \tag{14.14}$$

Whatever the head dimension, the scaled scores have a standard deviation of about 1, which keeps the softmax in its gradual, trainable range.

Worked example 14.8: The variance of a 2-dimensional dot product, exactly

Let each of q_1, q_2, k_1, k_2 be $+1$ or -1 with equal probability. Each has mean $\frac{1}{2}(1)+\frac{1}{2}(-1) = 0$ and variance $\frac{1}{2}(1)^2 + \frac{1}{2}(-1)^2 = 1$, so the assumptions hold. There are $2^4 = 16$ equally likely combinations.

Each product $q_i k_i$ is $+1$ (when the signs agree) or -1 (when they differ), each with probability $\frac{1}{2}$. The score $q_1 k_1 + q_2 k_2$ is therefore

$$\begin{aligned}
(+1) + (+1) = +2 \qquad & \text{probability } \tfrac{1}{2} \cdot \tfrac{1}{2} = \tfrac{1}{4} \quad (4 \text{ of the } 16) \\
(+1) + (-1) \text{ or } (-1) + (+1) = 0 \qquad & \text{probability } \tfrac{1}{4} + \tfrac{1}{4} = \tfrac{1}{2} \quad (8 \text{ of the } 16) \\
(-1) + (-1) = -2 \qquad & \text{probability } \tfrac{1}{4} \quad (4 \text{ of the } 16).
\end{aligned}$$

The mean is $\frac{1}{4}(2) + \frac{1}{2}(0) + \frac{1}{4}(-2) = 0$. The variance is the mean of the squares:

$$\tfrac{1}{4}(2)^2 + \tfrac{1}{2}(0)^2 + \tfrac{1}{4}(-2)^2 = \tfrac{1}{4} \cdot 4 + 0 + \tfrac{1}{4} \cdot 4 = 1 + 0 + 1 = 2 = d.$$

Dividing every score by $\sqrt{2}$ gives scores ± 1.4142 and 0, with variance $2/2 = 1$.

The script repeats the experiment with 200,000 random Gaussian queries and keys for larger d [C]:

d	sample variance of $\mathbf{q} \cdot \mathbf{k}$	sample std	$\sqrt{d}$	variance after $\div\sqrt{d}$
2	2.002	1.415	1.414	1.001
16	15.985	3.998	4.000	0.999
32	31.746	5.634	5.657	0.992
64	64.029	8.002	8.000	1.000

What this means in this model. The head dimension at stages 2–4 is 32 (next section). Unscaled, a typical score would be ± 5.66, and a score two standard deviations above another ($2 \times 5.657 = 11.31$ higher) would get $e^{11.31} \approx 81{,}937$ times the weight, by Equation (14.4). Scaled, the same two-standard-deviation lead is 2, a weight ratio of $e^2 = 7.39$. The first is a single-token copy; the second is a blend. The code (`efficient_self_attention.py`, line 82) stores the factor once:

```
self.scale = self.head_dim ** -0.5      # 1 / sqrt(d); d = 16 gives 0.25
```

(`**-0.5` is "to the power minus one half", which is one over the square root.)

14.8 Attention ignores order: permutation equivariance

Here is a property that surprises most readers. Shuffle the tokens before attention, and the outputs come out shuffled in exactly the same way, with the same values. Attention has no idea where any token sits.

Why. Let $\mathbf{P}$ be a *permutation matrix*: the identity matrix with its rows reordered, so that $\mathbf{PX}$ is $\mathbf{X}$ with its rows reordered. Feed $\mathbf{PX}$ to attention.

$$\mathbf{Q}' = \mathbf{PXW}_Q = \mathbf{PQ}, \quad \mathbf{K}' = \mathbf{PK}, \quad \mathbf{V}' = \mathbf{PV} \qquad \text{rows reordered}$$

The scores use the rule for the transpose of a product, $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$ (for example, a 1×2 row times a 2×1 column is a number, and transposing swaps the order of the factors):

$$\mathbf{Q}'\mathbf{K}'^\top = \mathbf{PQ}(\mathbf{PK})^\top = \mathbf{PQK}^\top\mathbf{P}^\top = \mathbf{PSP}^\top$$

The softmax works row by row, and reordering the rows of $\mathbf{S}$ and the entries within each row only reorders the softmax's output the same way:

$$\begin{aligned} \text{softmax}_{\text{row}}(\mathbf{PSP}^\top) &= \mathbf{P}\ \text{softmax}_{\text{row}}(\mathbf{S})\,\mathbf{P}^\top = \mathbf{PAP}^\top \\ \mathbf{A}'\mathbf{V}' &= \mathbf{PAP}^\top\mathbf{PV} = \mathbf{PAV} \qquad\qquad \mathbf{P}^\top\mathbf{P} = \mathbf{I}. \end{aligned}$$

The output is the old output with its rows reordered by the same $\mathbf{P}$. A function with this property is called *permutation equivariant*.

Worked example 14.9: Swapping two tokens

Swap tokens 1 and 2 of worked example 14.6: the input becomes $(0,1),(1,0),(1,1)$. Redo row 1: query $(0,1)$ against keys $(0,1),(1,0),(1,1)$ scores $1,0,1$, exactly the old row 1 of $\mathbf{S}$, so the weights are $(0.4011, 0.1978, 0.4011)$ and the output is

$$\begin{aligned} 0.4011(0,1) + 0.1978(1,0) + 0.4011(1,1) &= (0 + 0.1978 + 0.4011,\ 0.4011 + 0 + 0.4011) \\ &= (0.5989,\ 0.8022), \end{aligned}$$

which is the old output of token 2. The script confirms the whole output is the old one with rows 1 and 2 swapped: $(0.5989, 0.8022), (0.8022, 0.5989), (0.7517, 0.7517)$ [C].

For a picture this is a problem: "next to" and "far from" are among the most useful facts about a token, and attention cannot see them. Most transformers add a learned *positional encoding* vector to every token to say where it is. This codebase has none: a search of `app/` for `pos_embed` finds nothing. SegFormer gets its sense of position from a 3×3 depthwise convolution inside the feed-forward network [46], which Chapter 16 explains.

14.9 Many heads

14.9.1 Split, attend, concatenate, project

One attention gives one blend per token: a single compromise between everything the token might want to know. Multi-head attention runs h smaller attentions side by side, each on its own slice of the features, so that different heads can learn different notions of relevance (one might attend to near neighbours, another to anything of similar brightness anywhere in the patch).

Definition 14.5 (Multi-head attention)**.** Let C be divisible by h and let $d = C/h$, the *head dimension*. Compute $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ of shape (N, C) as before and cut each into h blocks of d columns: $\mathbf{Q} = [\mathbf{Q}^{(1)} \mid \cdots \mid \mathbf{Q}^{(h)}]$, and likewise for $\mathbf{K}$ and $\mathbf{V}$. Then

$$\mathbf{O}^{(m)} = \operatorname{softmax}_{\text{row}}\left(\frac{\mathbf{Q}^{(m)}\mathbf{K}^{(m)\top}}{\sqrt{d}}\right)\mathbf{V}^{(m)}, \qquad m = 1, \ldots, h, \tag{14.15}$$

$$\operatorname{MHA}(\mathbf{X}) = [\mathbf{O}^{(1)} \mid \cdots \mid \mathbf{O}^{(h)}]\,\mathbf{W}_O + \mathbf{b}_O^{\top}. \tag{14.16}$$

Each head has its own (N, N) attention matrix. The outputs, each (N, d), are concatenated back to (N, C) and passed through an output projection $\mathbf{W}_O \in \mathbb{R}^{C \times C}$.

The output projection matters. Without it the h heads would never interact; the model would be h separate narrow models stapled side by side. $\mathbf{W}_O$ lets every output feature draw on every head.

Analogy

The library again. One reader forms one impression. Five specialists reading the same shelf (a lawyer, an accountant, an engineer, a historian, an editor) each notice different things; the output projection is the meeting where their notes are merged into one report.

14.9.2 The reshaping, with shapes

With a batch, $\mathbf{Q}$ is a tensor `(B, N, C)`. The code splits the last axis into (head, feature within head) and moves the head axis forward so each head's data is a contiguous `(N, d)` matrix (`efficient_self_attention.py`, lines 128–134 and 190–195):

```
Q_x = self.q(x)                                          # (B, N, C)
Q_x = torch.reshape(Q_x, (B, N, self.num_heads, self.head_dim))  # (B, N, h, d)
Q_x = Q_x.transpose(1, 2)                                # (B, h, N, d)
# ... K_x, V_x likewise; attention runs on the last two axes for every (b, head)
out = out.transpose(1, 2)                                # (B, N, h, d)
out = torch.reshape(out, (B, N, self.head_dim * self.num_heads))  # (B, N, C)
out = self.proj(out)                                     # (B, N, C)
```

Matrix multiplication with `@` on 4-axis tensors treats the first two axes as a batch: for every patch b and head m it multiplies the `(N, d)` and `(d, N)` matrices on the last two axes. So `(B, h, N, d) @ (B, h, d, N)` gives `(B, h, N, N)`.

Worked example 14.10: Splitting a 4-number token into two heads

Let $C = 4$, $h = 2$, so $d = 4/2 = 2$. A token's query $(0.5, -1, 2, 0)$ is cut into head 1's query $(0.5, -1)$ and head 2's query $(2, 0)$. If a key is $(1, 1, 1, -3)$, head 1 scores $0.5{\cdot}1 + (-1){\cdot}1 = -0.5$ and head 2 scores $2 \cdot 1 + 0 \cdot (-3) = 2$, each then divided by $\sqrt{2}$. The full 4-number dot product would be $-0.5 + 2 = 1.5$: one head sees the key as irrelevant, the other as highly

relevant, and a single head would have averaged the two views away.

14.9.3 The heads in this model

stage	C (`embed_dim`)	h (`num_heads`)	$d = C/h$	whole number?
1	32	2	$32/2 = 16$	yes
2	64	2	$64/2 = 32$	yes
3	160	5	$160/5 = 32$	yes
4	256	8	$256/8 = 32$	yes

Widths and heads are those of the shipped fine-tune FT04: `embed_dims` $= [32, 64, 160, 256]$ and `num_heads` $= [2, 2, 5, 8]$

[M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`].

Version 0.1.0 used $[1, 2, 5, 8]$ [M: `configs/hyperspectral_segformer_exp_1.json`], which gave stage 1 a head dimension of 32.

Source check: the code's default head count is not the trained one

The constructors `app/foundation_models/components/hyperspectral_seg_former_mae.py` (line 71) and `spectral_stem_seg_former_mae.py` (line 54) default to `num_heads=[1,2,5,8]`, and the encoder's docstring table (`seg_former_encoder.py`, line 78) lists 1 head at stage 1. Every Indradhanu run since v0.2.0 passes $[2, 2, 5, 8]$ explicitly (`configs/hyperspectral_segformer_exp_2.json` line 33, the FT04 config above, and the backend's `backend/allotrope/foundation_models/resolver.py` line 489). The course table follows the configs and is right for the trained models; a model built with no arguments would have one head at stage 1. As the next subsection shows, the parameter count is the same either way.

14.9.4 Parameter count: $4C^2 + 4C$

The learned parts of plain multi-head attention are four linear layers, each $C \to C$: the matrices $\mathbf{W}_Q$, $\mathbf{W}_K$, $\mathbf{W}_V$ and $\mathbf{W}_O$, and their biases. A linear layer from C_{in} to C_{out} has $C_{\text{in}}C_{\text{out}}$ weights and C_{out} biases (Chapter 10). So

$$\begin{aligned} \text{one projection} &= C \cdot C + C = C^2 + C, \\ \text{four projections} &= 4(C^2 + C) = 4C^2 + 4C. \end{aligned} \tag{14.17}$$

The number of heads does not appear. Splitting into heads reshapes the outputs of the same four matrices; it adds no weights. The script builds the stage-1 attention module with one head and with two and gets 69,856 parameters both times [C] (that total includes the spatial-reduction convolution of Chapter 15).

Worked example 14.11: The projection parameters at every stage

Apply Equation (14.17) stage by stage, squaring first, then multiplying, then adding.

$$
\begin{aligned}
C = 32: &\quad 32^2 = 1{,}024;\quad 4 \times 1{,}024 = 4{,}096;\quad 4 \times 32 = 128;\quad 4{,}096 + 128 = 4{,}224\\
C = 64: &\quad 64^2 = 4{,}096;\quad 4 \times 4{,}096 = 16{,}384;\quad 4 \times 64 = 256;\quad 16{,}384 + 256 = 16{,}640\\
C = 160: &\quad 160^2 = 25{,}600;\quad 4 \times 25{,}600 = 102{,}400;\quad 4 \times 160 = 640;\\
&\quad 102{,}400 + 640 = 103{,}040\\
C = 256: &\quad 256^2 = 65{,}536;\quad 4 \times 65{,}536 = 262{,}144;\quad 4 \times 256 = 1{,}024;\\
&\quad 262{,}144 + 1{,}024 = 263{,}168
\end{aligned}
$$

Check against the model. Building the real `EfficientSelfAttention` modules, the script reports `q`, `k`, `v`, `proj` at 1,056 each for $C = 32$ ($4 \times 1{,}056 = 4{,}224$), 4,160 each for $C = 64$, 25,760 each for $C = 160$, and 65,792 each for $C = 256$ [C]. At stage 4, where there is no reduction convolution, the whole attention module is 263,168, exactly our number.

14.9.5 Cost

Per head, $\mathbf{QK}^\top$ takes $N \cdot N \cdot d$ multiply-adds (each of N^2 entries is a d-term dot product) and $\mathbf{AV}$ another $N \cdot N \cdot d$. Over h heads that is $2hN^2d = 2N^2C$: the work grows with the *square* of the number of tokens.

14.10 Self-attention and cross-attention

Everything in this chapter is *self*-attention: the queries, keys and values all come from the same set of tokens. In *cross*-attention the queries come from somewhere else (a small set of learned "askers") while the keys and values still come from the tokens. The four steps are identical; only the source of $\mathbf{Q}$ changes, and with it the shape of the score matrix, which becomes (number of askers)$\times N$. Part IV of this book uses exactly that to compress a spectrum (Chapters 22 and 23).

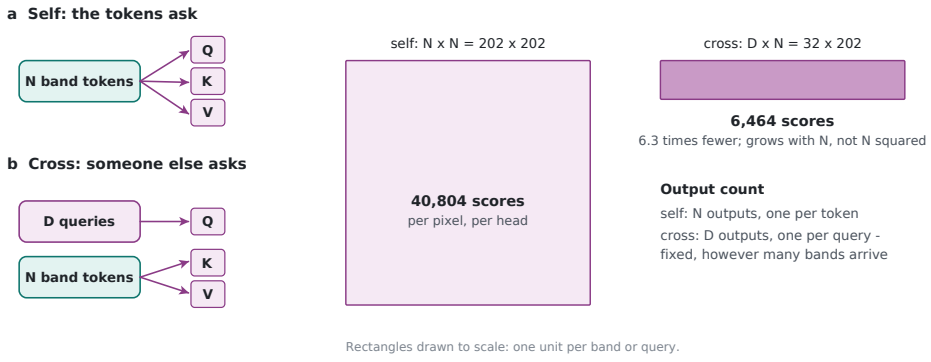


Figure 14.3: Where the questions come from. (a) In self-attention the tokens produce queries, keys and values. (b) In cross-attention an outside set of queries reads the tokens' keys and values. Right: the two score matrices for the spectral case of Chapter 22, drawn to scale.

14.11 The cost problem

The score matrix has one entry per (query, key) pair: $N \times N$ per head, per patch, per block. For a 128×128 patch:

stage	grid	N	$N \times N$
1	32×32	1024	$1024 \times 1024 = 1{,}048{,}576$
2	16×16	256	$256 \times 256 = 65{,}536$
3	8×8	64	$64 \times 64 = 4{,}096$
4	4×4	16	$16 \times 16 = 256$

Stage 1 has over a million scores for one head of one block of one patch: 4,096 times stage 4. And it is the stage where every other tensor is largest. Chapter 15 is the fix.

14.12 Common confusions

Is attention the same as correlation?
Related but not identical. Correlation is a fixed statistic of two signals. Attention compares learned projections of the tokens, so training decides what "similar" means for this task.

Do the rows or the columns of the attention matrix sum to one?
The rows. Softmax runs along the last axis (`dim=-1`), which is the key axis, so each query distributes exactly one unit of attention. A column sum says how popular a key is and can be anything from 0 to N (in Figure 14.1 column 3 sums to 1.3057).

Why three separate matrices instead of comparing tokens directly?
Because what a token looks for, what it advertises, and what it hands over are different things. Worked example 14.7 shows one projection changing who is listened to and another changing what is heard.

Do more heads mean more parameters?
No. The head count only decides how the C features are cut up; the four $C \times C$ matrices are the same size. It changes the head dimension $d = C/h$, the scale $1/\sqrt{d}$, and how many separate attention patterns are formed.

Does the scale $1/\sqrt{d}$ change which token wins?
No. Dividing every score by the same positive number keeps their order. It changes how decisively the winner wins: the gaps between scores shrink, so the weights are less extreme.

What does "self" mean in self-attention?
That queries, keys and values all come from the same token set. Cross-attention takes its queries from elsewhere (Section 14.10).

14.13 Check yourself

Exercise 14.1. Compute $\text{softmax}(3, 1, 0)$ by hand, first directly and then by the stable recipe (subtract the maximum). Show every exponential, the sum and each division, and check that the weights sum to 1.

Exercise 14.2. Redo worked example 14.5 for token 2, $\mathbf{q} = (0, 1)$, with identity projections. Give the scores, scaled scores, exponentials, weights and output.

Exercise 14.3. Show that for two scores the softmax weight of the first is $1/(1+e^{-(s_1-s_2)})$, the logistic sigmoid of the score difference. What weight does a lead of $s_1 - s_2 = 2$ give?

Exercise 14.4. Suppose the query and key components are independent with mean 0 but variance $\sigma^2 = 4$ instead of 1. Rederive $\mathrm{Var}(\mathbf{q} \cdot \mathbf{k})$ step by step for head dimension $d = 16$. What divisor would bring the variance back to 1?

Exercise 14.5. Stage 3 has $C = 160$ and $h = 5$. (a) What is the head dimension and the scale factor? (b) Count the parameters of its four projections by hand. (c) The whole stage-3 attention module has 205,920 parameters; how many belong to something other than the four projections?

Exercise 14.6. Stage 4 has $C = 256$, $h = 8$ and $N = 16$. Give the shapes of $\mathbf{Q}$ after the head split, of the score tensor for a batch of B patches, and of the output after concatenation. How many score entries are there per patch, summed over all heads?

Next: Chapter 15, Efficient self-attention with spatial reduction.

Chapter 15

Efficient self-attention with spatial reduction

The one thing this chapter teaches

Keep the questions detailed but summarise the answers: queries stay at full resolution, while keys and values are first shrunk by a strided convolution, so that every stage of the encoder attends over the same 4×4 summary of 16 tokens instead of over every token.

Chapter 14 built attention and ended on its cost: an $N \times N$ score matrix, over a million entries at stage 1. This chapter derives that cost exactly, in multiply-adds and in bytes, then shows the single modification SegFormer makes [46], borrowed from the Pyramid Vision Transformer [45]: spatial-reduction attention. The code is `app/foundation_models/components/efficient_self_attention.py`. Every number in the chapter is printed by `research/textbook/tools/compute_13-15.py`.

15.1 The cost of plain attention, counted

15.1.1 Multiply-adds

We count work in *multiply-adds* (MACs): one multiplication followed by one addition, the basic step of every dot product. A dot product of two length-d vectors is d MACs.

Take one attention module on one patch with N tokens of width C, split into h heads of width $d = C/h$ (Section 14.9). Its steps cost:

$$\begin{aligned}
\text{each of } \mathbf{Q}, \mathbf{K}, \mathbf{V} &: N \text{ tokens} \times C \cdot C \text{ MACs} = NC^2 && \text{a } C \times C \text{ matrix per token} \\
\mathbf{QK}^\top \text{ per head} &: N \cdot N \text{ entries} \times d \text{ MACs} = N^2 d \\
\mathbf{AV} \text{ per head} &: N \text{ rows} \times d \text{ columns} \times N \text{ terms} = N^2 d \\
\text{output projection} &: NC^2.
\end{aligned}$$

Add them, using $h \cdot d = C$ to sum the two per-head steps over the h heads:

$$\text{MACs}_{\text{plain}} = \underbrace{4NC^2}_{\text{four projections}} + \underbrace{2N^2C}_{\text{scores and blend}} . \tag{15.1}$$

The first term grows in proportion to N; the second with N^2. At stage 1 the second wins heavily.

Worked example 15.1: The cost of plain attention at stage 1

Stage 1 has $N = 1024$ tokens of width $C = 32$. Work each term out.

$$\begin{aligned} C^2 &= 32 \times 32 = 1{,}024 \\ 4NC^2 &= 4 \times 1024 \times 1{,}024 = 4 \times 1{,}048{,}576 = 4{,}194{,}304 \\ N^2 &= 1024 \times 1024 = 1{,}048{,}576 \\ 2N^2C &= 2 \times 1{,}048{,}576 \times 32 = 2 \times 33{,}554{,}432 = 67{,}108{,}864 \\ \text{total} &= 4{,}194{,}304 + 67{,}108{,}864 = 71{,}303{,}168. \end{aligned}$$

About 94% of the work (67.1/71.3) is the score matrix and the blend.

15.1.2 Memory

The score tensor has shape `(B, h, N, N)` and the softmax weights another tensor of the same shape. Training must keep the weights in memory until the backward pass (Chapter 26) needs them. Each float32 number is 4 bytes.

Worked example 15.2: The memory of one stage-1 score tensor

Per patch and per head, the scores are $1024 \times 1024 = 1{,}048{,}576$ numbers, and

$$1{,}048{,}576 \times 4 \text{ bytes} = 4{,}194{,}304 \text{ bytes} = 4 \text{ MiB}$$

(a MiB is $2^{20} = 1{,}048{,}576$ bytes). The v0.2.0 pretraining configuration uses a batch of $B = 128$ patches [M: `configs/hyperspectral_segformer_exp_2.json`] and stage 1 has $h = 2$ heads, so one block's weights tensor would be

$$\begin{aligned} 128 \times 2 \times 4{,}194{,}304 \text{ bytes} &= 256 \times 4{,}194{,}304 \text{ bytes} \\ &= 1{,}073{,}741{,}824 \text{ bytes} = 1{,}024 \text{ MiB} = 1 \text{ GiB}, \end{aligned}$$

and stage 1 has two blocks, each storing scores and weights. Several gigabytes for one stage of one batch, before counting anything else.

15.1.3 Why the obvious fixes fail

1. *Use fewer, bigger tokens.* Stage 1 could cut the patch into 8×8 blocks instead of 4×4 ($N = 256$). But then every output token describes 64 pixels instead of 16, and the fine detail that small anomalies live in is gone from the whole encoder.

2. *Attend only to nearby tokens.* Restrict each token to a window of neighbours. The cost falls, but a token in one corner can no longer see the other corner, and the context that makes a dark block surprising ("this is a desert") may be outside the window.

3. *Average the keys and values over blocks.* Closer, but the averaging is fixed: it cannot learn that, inside a block, the edge pixels matter more than the middle ones.

Spatial-reduction attention keeps every query (so every token still gets its own update), keeps global reach (every query sees a summary of the whole patch), and makes the summarising *learned*.

Analogy

You have a precise question about a thousand-page report. Rather than reading all thousand pages you read a sixteen-paragraph executive summary, one paragraph per chapter. Your question is as specific as it was; only the material you consult has been condensed. And the summary was written by someone who learned, over many reports, which parts of each chapter are worth keeping: that is the learned convolution. Where the analogy stops: a person reads one summary once, while here each of the 1,024 questions consults the same 16-paragraph summary, and every block writes a fresh summary of its own.

15.2 The strided convolution that summarises

15.2.1 Kernel equal to stride

Recall from Chapter 10 that a 2-D convolution with kernel size k and stride s slides a $k \times k$ window across the grid in steps of s, and that its output side length is

$$H_{\text{out}} = \left\lfloor \frac{H + 2p - k}{s} \right\rfloor + 1 \tag{15.2}$$

for padding p. The reduction uses $k = s = R$ and no padding ($p = 0$). Then

$$\begin{aligned} H_{\text{out}} &= \left\lfloor \frac{H - R}{R} \right\rfloor + 1 && p = 0,\ k = s = R \\ &= \left\lfloor \frac{H}{R} - 1 \right\rfloor + 1 && \text{split the fraction} \\ &= \frac{H}{R} && \text{when } R \text{ divides } H. \end{aligned}$$

With kernel equal to stride the windows tile the grid exactly, with no overlap and no gap: each output position summarises one $R \times R$ block of input tokens. (If R did not divide H, the floor would drop the last partial block: $H = 30$, $R = 8$ gives $\lfloor 22/8 \rfloor + 1 = 3$, and the last six rows would be ignored. For this model it always divides; see Section 15.4.)

Definition 15.1 (Spatial-reduction convolution)**.** For token features $x_{c,i,j}$ (channel c, grid row i, column j) the reduced token at summary position (u, v) has channels

$$y_{c',u,v} = b_{c'} + \sum_{c=1}^{C} \sum_{a=0}^{R-1} \sum_{b=0}^{R-1} W_{c',c,a,b}\, x_{c,\, Ru+a,\, Rv+b}, \qquad c' = 1, \ldots, C. \tag{15.3}$$

Every output channel looks at every input channel at every one of the R^2 positions of its block. That is $C \cdot C \cdot R^2$ weights plus C biases, a count we use in Section 15.6.

Average pooling is one special case. Choose $W_{c',c,a,b} = 1/R^2$ when $c' = c$ and 0 otherwise, and every bias $b_{c'} = 0$. Then Equation (15.3) becomes $y_{c,u,v} = \frac{1}{R^2} \sum_{a,b} x_{c,Ru+a,Rv+b}$, the plain average of the block, channel by channel. The learned convolution can do this, or anything else linear: weight the centre more, pick one corner, or mix channels.

Worked example 15.3: Summarising a 4×4 grid with $R = 2$

Take a grid of 4×4 tokens with a single channel ($C = 1$):

$$\begin{pmatrix} 1 & 2 & 0 & 1 \\ 3 & 2 & 1 & 0 \\ 0 & 0 & 4 & 4 \\ 0 & 0 & 4 & 4 \end{pmatrix}.$$

With $R = 2$ the output is $4/2 = 2$ by 2. The four blocks are the four 2×2 quarters.

Averaging kernel $\begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}$, bias 0:

$$\begin{aligned} \text{top-left:} \quad & 0.25(1+2+3+2) = 0.25 \times 8 = 2.0 \\ \text{top-right:} \quad & 0.25(0+1+1+0) = 0.25 \times 2 = 0.5 \\ \text{bottom-left:} \quad & 0.25(0+0+0+0) = 0.0 \\ \text{bottom-right:} \quad & 0.25(4+4+4+4) = 0.25 \times 16 = 4.0 \end{aligned}$$

"Pick the top-left" kernel $\begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$, bias 0: the outputs are the top-left entries of the blocks, $1, 0, 0, 4$.

Two different summaries of the same grid. Training chooses the kernel, and with C channels in and out it chooses a different mix for each output channel. Sixteen tokens became four; with $R = 8$ at stage 1, 1024 become 16.

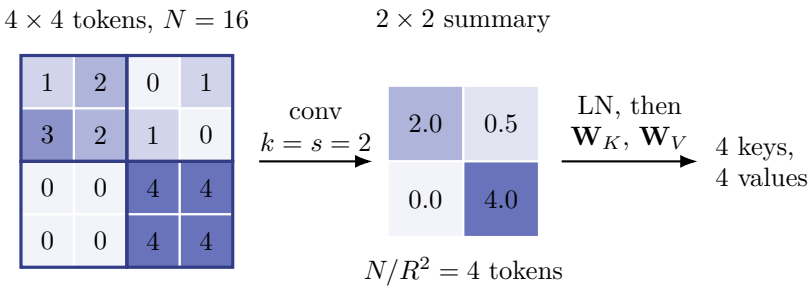


Figure 15.1: The toy reduction of worked example 15.3 with the averaging kernel. Each output token summarises one $R \times R$ block of the input (outlined); the queries are still made from all 16 input tokens.

15.2.2 LayerNorm after the reduction

The summarised tokens pass through a LayerNorm before they are projected into keys and values (`efficient_self_attention.py`, line 111). Chapter 16 derives LayerNorm in full; for now, it rescales each token so that its C numbers have mean 0 and variance 1, then applies a learned per-channel scale and shift. Here it keeps the summary tokens on the same scale as the full-resolution tokens the queries come from, which matters because Section 14.7 assumed unit-variance components when it chose the divisor $\sqrt{d}$.

15.3 The data flow, with shapes at every step

```
# x: (B, N, C), N = H * W    (efficient_self_attention.py, lines 125-195)
Q = self.q(x)                                  # (B, N, C)
```

```
Q = Q.reshape(B, N, h, d).transpose(1, 2)      # (B, h, N, d): ALL tokens
r = x.transpose(1, 2)                          # (B, C, N)
r = r.reshape(B, C, H, W)                      # (B, C, H, W): a grid
r = self.reduction(r)                          # (B, C, H/R, W/R): conv
r = r.flatten(2).transpose(1, 2)               # (B, N/R^2, C): sequence
r = self.reduction_norm(r)                     # (B, N/R^2, C): LayerNorm
K = self.k(r).reshape(B, -1, h, d).transpose(1, 2)  # (B, h, N/R^2, d)
V = self.v(r).reshape(B, -1, h, d).transpose(1, 2)  # (B, h, N/R^2, d)
S = Q @ K.transpose(-2, -1) * d ** -0.5        # (B, h, N, N/R^2)
A = softmax(S, dim=-1)                         # (B, h, N, N/R^2)
O = A @ V                                      # (B, h, N, d)
O = O.transpose(1, 2).reshape(B, N, C)         # (B, N, C)
O = self.proj(O)                               # (B, N, C)
```

(This is a condensed transcription; the source spells each step out on its own line with the same shapes in its comments.) Why the `transpose` before the `reshape`? Memory order: a `(B, N, C)` tensor stores token 0's C numbers, then token 1's; a grid `(B, C, H, W)` must store channel 0's $H \cdot W$ numbers first. A reshape alone would scramble them (Chapter 10).

The key sentence for reading the scores: *entry* (b, m, i, j) *of* $\mathbf{S}$ *is how much full-resolution token* i *should attend, in head* m*, to summary token* j*, which stands for an* $R \times R$ *block of the grid.* Fine-grained questions, spatially aggregated answers. The output still has one vector per original token: nothing was lost on the query side.

15.3.1 Every stage

For a 128×128 patch, the stage widths, heads and ratios $C = (32, 64, 160, 256)$, $h = (2, 2, 5, 8)$, $R = (8, 4, 2, 1)$ give these shapes

[M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]:

step	stage 1 C=32, h=2, R=8	stage 2 C=64, h=2, R=4	stage 3 C=160, h=5, R=2	stage 4 C=256, h=8, R=1
input $\mathbf{x}$	$(B, 1024, 32)$	$(B, 256, 64)$	$(B, 64, 160)$	$(B, 16, 256)$
$\mathbf{Q}$ split	$(B, 2, 1024, 16)$	$(B, 2, 256, 32)$	$(B, 5, 64, 32)$	$(B, 8, 16, 32)$
grid	$(B, 32, 32, 32)$	$(B, 64, 16, 16)$	$(B, 160, 8, 8)$	not built
after conv	$(B, 32, 4, 4)$	$(B, 64, 4, 4)$	$(B, 160, 4, 4)$	no conv
sequence, LN	$(B, 16, 32)$	$(B, 16, 64)$	$(B, 16, 160)$	$(B, 16, 256)$
$\mathbf{K}, \mathbf{V}$ split	$(B, 2, 16, 16)$	$(B, 2, 16, 32)$	$(B, 5, 16, 32)$	$(B, 8, 16, 32)$
scores $\mathbf{S}$	$(B, 2, 1024, 16)$	$(B, 2, 256, 16)$	$(B, 5, 64, 16)$	$(B, 8, 16, 16)$
$\mathbf{AV}$	$(B, 2, 1024, 16)$	$(B, 2, 256, 32)$	$(B, 5, 64, 32)$	$(B, 8, 16, 32)$
output	$(B, 1024, 32)$	$(B, 256, 64)$	$(B, 64, 160)$	$(B, 16, 256)$

At stage 4, $R = 1$ and the module has no reduction layer at all (the `if` at line 103 only builds it for $R > 1$); keys and values come straight from the 16 tokens, which is ordinary full attention. The script runs a random input through each stage's module and gets the output shapes in the last row [C].

Worked example 15.4: One query against the four summary tokens

Continue worked example 15.3 with the averaging summary $(2.0, 0.5, 0.0, 4.0)$. Keep $C = 1$, one head ($d = 1$, so the scale is $1/\sqrt{1} = 1$), and identity projections. (We skip the LayerNorm here: with a single channel it would map every token to the same number, which is Exercise 15.5.) Take the query of the top-left token, whose value is 1.
Scores, query times each key: $1 \times 2.0 = 2.0$, $1 \times 0.5 = 0.5$, $1 \times 0.0 = 0.0$, $1 \times 4.0 = 4.0$.

Stable softmax. Subtract the maximum 4.0: $(-2.0, -3.5, -4.0, 0.0)$. Exponentiate:

$$e^{-2.0} = 0.1353, \quad e^{-3.5} = 0.0302, \quad e^{-4.0} = 0.0183, \quad e^{0} = 1.0000.$$

Add: $0.1353 + 0.0302 + 0.0183 + 1.0000 = 1.1838$. Divide:

$$0.1353/1.1838 = 0.1143, \quad 0.0302/1.1838 = 0.0255,$$
$$0.0183/1.1838 = 0.0155, \quad 1.0000/1.1838 = 0.8447.$$

Blend the values:

$$\begin{aligned} 0.1143 \times 2.0 &= 0.2286 \\ 0.0255 \times 0.5 &= 0.0128 \\ 0.0155 \times 0.0 &= 0.0000 \\ 0.8447 \times 4.0 &= 3.3788 \\ \text{sum} &= 3.6202. \end{aligned}$$

The top-left token is pulled strongly toward the bright bottom-right block, which it can see only through its one summary token. The full layer does this for all 16 queries, so the score matrix is $16 \times 4 = 64$ entries instead of $16 \times 16 = 256$: a factor of $R^2 = 4$.

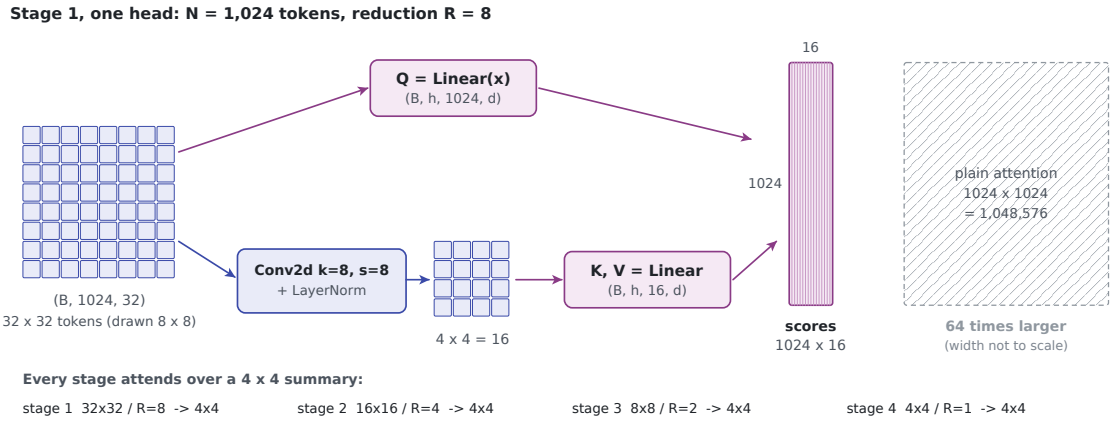


Figure 15.2: Spatial reduction at stage 1, as the course draws it. Queries keep all 1,024 tokens; keys and values are first shrunk by a strided convolution (kernel = stride = $R = 8$) to a 4×4 grid of 16 summary tokens. The score matrix is 1024×16 instead of 1024×1024, 64 times smaller (the hatched square is not drawn to scale). With $R = 8, 4, 2, 1$ every stage attends over the same 4×4 summary.

15.4 The 16-token summary

The ratios $R = (8, 4, 2, 1)$ were chosen so that every stage, however fine its own grid, attends over the same 4×4 summary of the whole patch.

The token grid halves in side at each stage ($128/4 = 32$, then 16, 8, 4; Chapter 17) and the

ratio halves too, so the summary side never changes:

$$\begin{aligned}
&\text{stage 1: } 32/8 = 4 && 4 \times 4 = 16 \text{ summary tokens}\\
&\text{stage 2: } 16/4 = 4 && 4 \times 4 = 16\\
&\text{stage 3: } 8/2 = 4 && 4 \times 4 = 16\\
&\text{stage 4: } 4/1 = 4 && 4 \times 4 = 16.
\end{aligned}$$

In symbols: the stage-s grid side is $128/(4 \cdot 2^{s-1})$ and $R_s = 8/2^{s-1}$, so the summary side is $\frac{128}{4 \cdot 2^{s-1}} \cdot \frac{2^{s-1}}{8} = \frac{128}{32} = 4$, independent of s. For a patch of side P it would be $P/32$: a 256×256 patch would give an 8×8 summary of 64 tokens at every stage.

stage	grid	N	R	summary	N/R^2	score entries	plain would be
1	32×32	1024	8	4×4	16	$1024 \times 16 = 16{,}384$	1,048,576
2	16×16	256	4	4×4	16	$256 \times 16 = 4{,}096$	65,536
3	8×8	64	2	4×4	16	$64 \times 16 = 1{,}024$	4,096
4	4×4	16	1	4×4	16	$16 \times 16 = 256$	256

The score matrix shrinks by exactly R^2: 64× at stage 1, 16× at stage 2, 4× at stage 3, and not at all at stage 4, where 16 tokens is already the target.

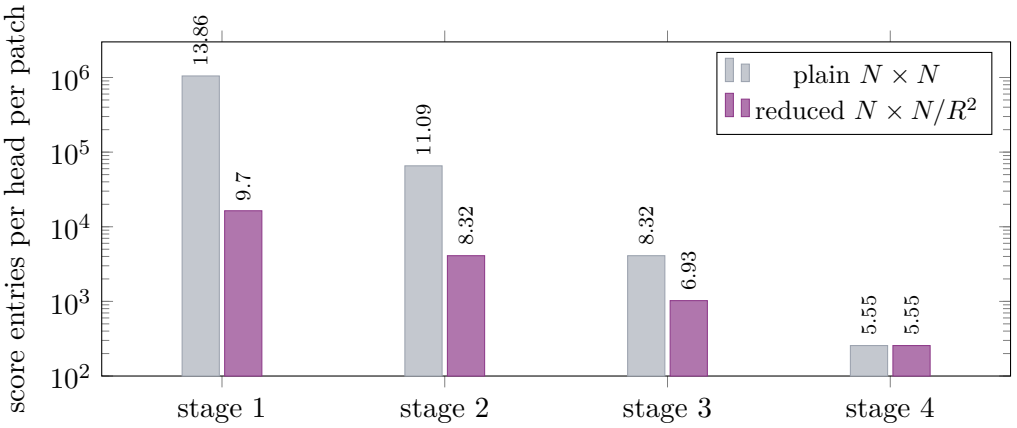


Figure 15.3: Score entries per head per patch, plain against spatially reduced, on a logarithmic axis (each gridline is ten times the one below). The reduction factor R^2 is 64, 16, 4 and 1.

15.5 What the reduction saves, in work and memory

15.5.1 Multiply-adds with the reduction

Write $N' = N/R^2$ for the number of summary tokens. Redo Equation (15.1) step by step:

$$\begin{aligned}
\mathbf{Q} &: NC^2 && \text{all } N \text{ tokens, unchanged}\\
\text{reduction conv} &: N' \text{ outputs} \times C \text{ channels} \times CR^2 \text{ MACs} &&\\
&= N'R^2C^2 = NC^2 && \text{since } N'R^2 = N\\
\mathbf{K}, \mathbf{V} &: 2N'C^2 && \text{only } N' \text{ summary tokens}\\
\mathbf{QK}^\top,\ \mathbf{AV} &: 2NN'C && N' \text{ keys instead of } N\\
\text{output projection} &: NC^2.
\end{aligned}$$

So

$$\text{MACs}_{\text{reduced}} = 3NC^2 + 2N'C^2 + 2NN'C \qquad (R > 1). \tag{15.4}$$

A curiosity: the reduction convolution costs exactly NC^2, the same as one projection, at every stage. The LayerNorm adds a few operations per number and is left out.

Worked example 15.5: The cost of reduced attention at stage 1

$N = 1024$, $N' = 16$, $C = 32$, $C^2 = 1{,}024$.

$$\begin{aligned} 3NC^2 &= 3 \times 1024 \times 1{,}024 = 3 \times 1{,}048{,}576 = 3{,}145{,}728 \\ 2N'C^2 &= 2 \times 16 \times 1{,}024 = 32{,}768 \\ 2NN'C &= 2 \times 1024 \times 16 \times 32 = 2 \times 524{,}288 = 1{,}048{,}576 \\ \text{total} &= 3{,}145{,}728 + 32{,}768 + 1{,}048{,}576 = 4{,}227{,}072. \end{aligned}$$

Against the plain 71,303,168 of worked example 15.1: 71,303,168/4,227,072 = 16.9 times less work. The score-and-blend part alone fell from 67,108,864 to 1,048,576, the full factor of 64; the projections, which do not shrink, now dominate.

The script evaluates both formulas at every stage [C]:

stage	plain MACs	reduced MACs	of which conv	ratio
1	71,303,168	4,227,072	1,048,576	16.9
2	12,582,912	3,801,088	1,048,576	3.3
3	7,864,320	6,062,080	1,638,400	1.3
4	4,325,376	4,325,376	0	1.0

After the reduction the four stages cost roughly the same. Without it, stage 1 alone would cost nearly three times the other three stages together (71.3 million MACs against $12.6 + 7.9 + 4.3 =$ 24.8 million, all plain).

15.5.2 Memory with the reduction

Worked example 15.6: The stage-1 score tensor, reduced

Per patch and head: $1024 \times 16 = 16{,}384$ numbers, times 4 bytes = 65,536 bytes = 64 KiB, against 4 MiB plain. For the batch of 128 with 2 heads:

$$128 \times 2 \times 65{,}536 \text{ bytes} = 256 \times 65{,}536 \text{ bytes} = 16{,}777{,}216 \text{ bytes} = 16 \text{ MiB},$$

against 1,024 MiB plain. The same factor of 64.

15.6 What the reduction costs in parameters

The convolution of Equation (15.3) has one weight for every (output channel, input channel, kernel row, kernel column) and one bias per output channel; the LayerNorm has a scale and a shift per channel:

$$\text{reduction conv} = C \cdot C \cdot R \cdot R + C, \qquad \text{reduction LN} = 2C. \tag{15.5}$$

Add the four projections of Equation (14.17):

$$\text{ESA}(C, R) = \underbrace{4C^2 + 4C}_{\text{q, k, v, proj}} + \underbrace{C^2R^2 + C + 2C}_{\text{only if } R>1}. \tag{15.6}$$

Worked example 15.7: The stage-1 attention module, parameter by parameter

$C = 32$, $R = 8$.

$$\begin{aligned}
\text{one projection: } & 32 \times 32 + 32 = 1{,}024 + 32 = 1{,}056 \\
\text{four projections: } & 4 \times 1{,}056 = 4{,}224 \\
\text{reduction conv: } & 32 \times 32 \times 8 \times 8 + 32 = 1{,}024 \times 64 + 32 = 65{,}536 + 32 = 65{,}568 \\
\text{reduction LN: } & 2 \times 32 = 64 \\
\text{total: } & 4{,}224 + 65{,}568 + 64 = 69{,}856.
\end{aligned}$$

Check. Building `EfficientSelfAttention(32,2,8)` the script reports 69,856, with `q`, `k`, `v`, `proj` at 1,056 each, `reduction` at 65,568 and `reduction_norm` at 64 [C].

The same arithmetic at every stage, checked against the built modules [C]:

stage	C	R	projections	conv $C^2R^2 + C$	LN $2C$	ESA total
1	32	8	4,224	$65{,}536 + 32 = 65{,}568$	64	69,856
2	64	4	16,640	$65{,}536 + 64 = 65{,}600$	128	82,368
3	160	2	103,040	$102{,}400 + 160 = 102{,}560$	320	205,920
4	256	1	263,168	none	none	263,168

The surprise. At stage 1 the reduction convolution is $65{,}568/69{,}856 = 93.9\%$ of the attention module, about fifteen times all four projections together ($65{,}568/4{,}224 = 15.5$). The reason is in Equation (15.5): $C^2R^2 = (CR)^2$. At stage 1, $CR = 32 \times 8 = 256$; at stage 2, $CR = 64 \times 4 = 256$ again, which is why the two convolutions are almost the same size ($256^2 = 65{,}536$ weights each). Stage 1 has the narrowest tokens and yet one of the larger attention modules. The 64-fold saving in compute is bought with parameters; it is a trade, not a free lunch.

Values that will drift

The widths, heads and ratios above are those of the FT04 configuration (`research/experiments/indradhanu-finetune/FT04-final-joint/config.json`, read 2026-10-29). Indradhanu Heavy triples the widths to $(96, 192, 480, 768)$ with heads $(3, 6, 10, 12)$ [M: `indradhanuworkings/68-indradhanu-heavy-pretraining.md`]; its stage-1 reduction convolution is $96 \times 96 \times 64 + 96 = 589{,}920$ parameters [C], nine times FT04's (Chapter 43).

15.7 The fallback: when the grid is incomplete

The reduction rebuilds a full 2-D grid from the token sequence, which requires $N = H \cdot W$ exactly. During masked training (Chapter 18) stage 1 removes the tokens it will be asked to predict before the blocks run. With the mask ratio 0.65

[M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`],

about

$$1024 \times (1 - 0.65) = 1024 \times 0.35 = 358.4 \approx 358$$

tokens survive, and 358 tokens cannot be arranged into a 32×32 grid. The code checks first (`efficient_self_attention.py`, line 144):

```
can_reduce = (self.reduction_ratio > 1) and (N == H * W)
if can_reduce:
    ...                    # grid, conv, flatten, LayerNorm: K/V over N / R^2 tokens
```

```
else:
    reduced_x = x          # (B, N, C): K/V over all surviving tokens
```

So *whenever tokens have been removed, stage 1 runs plain full attention over the survivors.*

Worked example 15.8: Is the fallback affordable?

Per head and patch, with 358 survivors:

$$358 \times 358 = 128{,}164 \text{ score entries.}$$

Compare: plain attention over all 1024 tokens, 1,048,576 (the fallback is 128,164/1,048,576 = 12.2% of it, about one eighth); reduced attention over all 1024 tokens, 16,384 (the fallback is 128,164/16,384 = 7.8 times more). Removing 65% of the tokens shrinks N^2 by $0.35^2 = 0.1225$, which is why it is affordable.

Source check: the fallback comment uses an old mask ratio

The comment above the check (`efficient_self_attention.py`, lines 142–143) says "With ~512 tokens after 50% masking, full attention costs 512*512 = 262K". That was the mask ratio when the code was written. The FT04 configuration uses 0.65, giving about 358 survivors and $358^2 = 128{,}164$ entries per head. The reasoning in the comment is right; its numbers are dated. The course (IW13) already notes this.

Two consequences follow from reading the code, and both are worth knowing.

1. *The model is not the same machine with and without a mask.* A forward pass with a `keep_mask` at stage 1 uses full attention there; one without uses the reduction. This is intended and documented, but it means masked and unmasked outputs are produced by slightly different computations (Chapter 30 shows how inference uses both).

2. *In a masked pass the stage-1 reduction layers are idle.* The `else` branch never touches `self.reduction` or `self.reduction_norm`, so a masked forward pass gives them no gradient. Their 65,632 parameters (65,568 + 64) are trained only by passes in which all 1024 stage-1 tokens are present. Stages 2–4 never remove tokens, so their reductions always run.

15.8 Common confusions

Which of query, key and value stay at full resolution?
Only the queries. Every token must receive its own updated vector, so there must be one query per token. Keys and values are what is consulted, and those are summarised.

Does the reduction lose information?
On the answer side, yes: fine spatial detail within each $R \times R$ block is blended. The bet, validated by the SegFormer authors across many tasks, is that queries need to be precise while answers can be summarised. The next stage of the encoder works at a coarser grain anyway.

Is the reduction the same as pooling?
It is learned pooling. Average pooling is one setting of its weights (worked example 15.3); training is free to find a better one, and to mix channels while it summarises.

Does $R = 1$ at stage 4 mean attention is switched off?
No. $R = 1$ means no reduction, which is ordinary full attention over the 16 tokens. The

module has no reduction layer at that stage.

Does the fallback make masked training less accurate?
It makes stage 1 less compressed, if anything: full attention over the survivors sees more detail than a summary would. The cost is compute, and that is small because most tokens are gone.

Why does ESA have more parameters at stage 1 than its width suggests?
Because the reduction convolution has $(CR)^2$ weights and $CR = 256$ at stage 1. The projections grow as C^2; the convolution grows as C^2R^2.

15.9 Check yourself

Exercise 15.1. Stage 2 has $N = 256$ tokens on a 16×16 grid, $C = 64$, $h = 2$, $R = 4$. Write the shape after each step of the data flow for a batch of B patches, from the input to the output projection.

Exercise 15.2. Count the parameters of the stage-3 reduction convolution and LayerNorm by hand ($C = 160$, $R = 2$), and then the whole stage-3 attention module. Compare with 205,920.

Exercise 15.3. For a 256×256 patch, give the token grid, the number of summary tokens and the score-matrix size per head at each of the four stages. By what factor has stage 1's score matrix grown compared with a 128×128 patch?

Exercise 15.4. Evaluate Equations (15.1) and (15.4) by hand for stage 2 ($N = 256$, $N' = 16$, $C = 64$) and check them against the table in Section 15.5.

Exercise 15.5. Show that LayerNorm on a token with a single channel ($C = 1$) always returns its learned shift β, whatever the input. Why does this not matter for the real model?

Exercise 15.6. With the older mask ratio 0.5, how many stage-1 tokens survive, how many score entries does the fallback compute per head, and what fraction is that of plain attention over all 1024 tokens?

Next: Chapter 16, Mix-FFN, LayerNorm, residuals and the transformer block.

Chapter 16

Mix-FFN, LayerNorm, residuals and the transformer block

The one thing this chapter teaches

A SegFormer block is two sublayers, attention and Mix-FFN, each wrapped the same way: normalise the input with LayerNorm, transform it, drop a random part of the result during training, and add it back to what came in. Two lines of code, and every piece has a reason you can calculate.

Chapters 14 and 15 built the attention half of the block. This chapter builds the other half, the Mix-FFN, and the three pieces of wiring that hold the two together: LayerNorm, the residual connection and dropout. It ends by counting the parameters of a whole block by hand and matching the real model's number exactly. The code is `app/foundation_models/components/mix_ffn.py` and `app/foundation_models/components/segformer_block.py`; the numbers are printed by `research/textbook/tools/compute_13-15.py`.

16.1 Why attention alone is not enough

Three problems remain after Chapter 15.

1. Attention gathers but hardly computes. Look at what attention does to the values: it multiplies them by the weights matrix $\mathbf{A}$ and then by $\mathbf{W}_O$. If the weights were held fixed, the output $\mathbf{A}\mathbf{X}\mathbf{W}_V\mathbf{W}_O$ would be a *linear* function of the input: doubling every token would double every output. The only nonlinearity is the softmax, and it acts on the weights, deciding *who* is listened to, not on the content that is passed along. A stack of linear maps is still a linear map ($\mathbf{W}_1\mathbf{W}_2$ is one matrix), and a linear map cannot represent "bright in band 40 *and* dark in band 90". Each token needs some private, nonlinear processing. That is the feed-forward network.

2. Deep stacks are hard to train. The encoder has eight blocks, each with two sublayers. Training sends an error signal backwards through all of them, and every sublayer multiplies it by its own derivative. Sixteen multiplications can shrink the signal to nothing or blow it up. The residual connection fixes this.

3. Scales drift. Each sublayer adds its output to the stream of tokens, so the typical size of a token's numbers changes from block to block. Section 14.7 chose the divisor $\sqrt{d}$ assuming

unit-variance inputs; if the inputs drift, the softmax saturates after all. LayerNorm puts every token back on a standard scale before each sublayer sees it.

Analogy

Think of a document passed around an editorial office. Each editor receives a clean photocopy (LayerNorm: everyone works from a standardised copy, whatever state the original is in), writes suggested changes in the margin (the sublayer), and some suggestions are struck out at random during training so that nobody relies on one editor (dropout). The suggestions are then applied to the *original* (the residual addition): the document is never replaced by an editor's rewrite, only amended. Even if one editor does nothing useful, the document reaches the next desk intact.

16.2 LayerNorm

16.2.1 The derivation

LayerNorm [1] standardises each token on its own, across its C features. Take one token $\mathbf{x} = (x_1, \ldots, x_C)$.

Step 1, the mean of its features:

$$\mu = \frac{1}{C}\sum_{i=1}^{C} x_i. \tag{16.1}$$

Step 2, the variance, the average squared distance from that mean (divided by C, not $C-1$: PyTorch's LayerNorm uses this "biased" form):

$$\sigma^2 = \frac{1}{C}\sum_{i=1}^{C}(x_i - \mu)^2. \tag{16.2}$$

Step 3, normalise: subtract the mean, so the features centre on zero, and divide by the standard deviation, so their spread becomes one. A small ϵ (PyTorch's default is 10^{-5}) prevents division by zero when all features are equal:

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}. \tag{16.3}$$

Step 4, the learned affine map. Forcing every token to mean 0 and spread 1 may throw away something useful, so each feature gets a learned scale γ_i and shift β_i:

$$\mathrm{LN}(\mathbf{x})_i = \gamma_i\,\hat{x}_i + \beta_i. \tag{16.4}$$

The vectors $\boldsymbol{\gamma}$ and $\boldsymbol{\beta}$ are the only parameters, C each, so

$$\text{LayerNorm}(C) \text{ has } 2C \text{ parameters.} \tag{16.5}$$

They start at $\gamma_i = 1$, $\beta_i = 0$, which makes LayerNorm a pure standardisation at initialisation.

Worked example 16.1: LayerNorm of a 4-number token

Take $\mathbf{x} = (1.0, 2.0, 3.0, 4.0)$ with $\boldsymbol{\gamma} = (1, 1, 1, 1)$ and $\boldsymbol{\beta} = (0, 0, 0, 0)$. Do it on paper.

Mean.

$$\mu = \frac{1.0 + 2.0 + 3.0 + 4.0}{4} = \frac{10.0}{4} = 2.5.$$

Deviations and their squares.

$$\begin{aligned} 1.0 - 2.5 &= -1.5, & (-1.5)^2 &= 2.25 \\ 2.0 - 2.5 &= -0.5, & (-0.5)^2 &= 0.25 \\ 3.0 - 2.5 &= 0.5, & (0.5)^2 &= 0.25 \\ 4.0 - 2.5 &= 1.5, & (1.5)^2 &= 2.25 \end{aligned}$$

Variance. Sum $2.25 + 0.25 + 0.25 + 2.25 = 5.00$; divide by 4: $\sigma^2 = 1.25$.
Standard deviation. $\sqrt{1.25} = 1.1180$. (With ϵ: $\sqrt{1.25 + 0.00001} = 1.118038$ against 1.118034; the difference is in the sixth digit.)
Normalise.

$$\begin{aligned} -1.5/1.1180 &= -1.3416 \\ -0.5/1.1180 &= -0.4472 \\ 0.5/1.1180 &= 0.4472 \\ 1.5/1.1180 &= 1.3416 \end{aligned}$$

So $\mathrm{LN}(\mathbf{x}) = (-1.3416, -0.4472, 0.4472, 1.3416)$, which is what PyTorch's `nn.LayerNorm(4)` returns [C]. The pattern (evenly spaced, increasing) is kept; the level and the spread are standardised. Check: the outputs sum to 0, and their squares, $1.8 + 0.2 + 0.2 + 1.8 = 4.0$, average to 1.
Now with an affine map, $\boldsymbol{\gamma} = (2, 1, 1, 0.5)$ and $\boldsymbol{\beta} = (0, 0, 1, 0)$:

$$\begin{aligned} 2 \times (-1.3416) + 0 &= -2.6833 \\ 1 \times (-0.4472) + 0 &= -0.4472 \\ 1 \times 0.4472 + 1 &= 1.4472 \\ 0.5 \times 1.3416 + 0 &= 0.6708 \end{aligned}$$

(using the unrounded -1.34164 for the first line). Training can stretch, shrink and shift each feature after the standardisation.

16.2.2 What LayerNorm forgets

LayerNorm is blind to the overall level and overall scale of a token: $\mathrm{LN}(a\mathbf{x} + b) = \mathrm{LN}(\mathbf{x})$ for any number $a > 0$ and any number b.

Proof (ignoring the tiny ϵ). Let $\mathbf{y} = a\mathbf{x} + b$, meaning $y_i = ax_i + b$ for every i.

$$\begin{aligned} \mu_y &= \frac{1}{C}\sum_i (ax_i + b) = a\,\frac{1}{C}\sum_i x_i + b = a\mu_x + b && \text{the mean follows the map} \\ y_i - \mu_y &= ax_i + b - a\mu_x - b = a(x_i - \mu_x) && \text{the shift } b \text{ cancels} \\ \sigma_y^2 &= \frac{1}{C}\sum_i a^2(x_i - \mu_x)^2 = a^2\sigma_x^2 && \text{square the line above} \\ \hat{y}_i &= \frac{a(x_i - \mu_x)}{\sqrt{a^2\sigma_x^2}} = \frac{a(x_i - \mu_x)}{a\,\sigma_x} = \hat{x}_i && \sqrt{a^2} = a \text{ since } a > 0. \end{aligned}$$ ■

The script confirms it: for the token of worked example 16.1,

$$\mathrm{LN}(3\mathbf{x} + 10) = (-1.3416, -0.4472, 0.4472, 1.3416) = \mathrm{LN}(\mathbf{x})\ [C].$$

Whatever a previous block did to the level and scale of a token, the next sublayer sees a standardised copy. (The same proof with $C = 1$ shows that a single number is always mapped to β; Exercise 15.5.)

16.2.3 LayerNorm against BatchNorm

Chapter 12 met BatchNorm, which normalises each channel using statistics gathered over the whole batch. The difference is which numbers are averaged.

	LayerNorm	BatchNorm
averages over	one token's C features	one channel, across the batch and positions
depends on other patches in the batch?	no	yes
same in training and evaluation?	yes	no (running statistics)
suits token sequences?	yes	poorly

With BatchNorm a token's output would depend on which other patches happened to share its batch, and small batches would give noisy statistics. LayerNorm keeps every token self-contained. (Chapter 29 tells what BatchNorm's batch dependence cost this project in fine-tuning.)

16.3 Residual connections

16.3.1 The chain rule, on a tiny example

Training needs the derivative of the loss with respect to every parameter (Chapter 26). When functions are composed, $y = g(f(x))$, the derivative is the *chain rule*: multiply the derivatives along the way,

$$\frac{dy}{dx} = g'(f(x)) \cdot f'(x). \tag{16.6}$$

Check it on numbers. Let $f(x) = 3x$ and $g(u) = u^2$, so $y = (3x)^2 = 9x^2$ and directly $dy/dx = 18x$, which is 36 at $x = 2$. By the chain rule: $f(2) = 6$, $g'(u) = 2u$ so $g'(6) = 12$, $f'(x) = 3$; then $12 \times 3 = 36$. The same.

For a stack of L layers $x_\ell = f_\ell(x_{\ell-1})$, the chain rule multiplies L factors:

$$\frac{dx_L}{dx_0} = f'_L \cdot f'_{L-1} \cdots f'_1. \tag{16.7}$$

16.3.2 What the residual changes

A *residual connection* (or skip connection) computes

$$x_\ell = x_{\ell-1} + f_\ell(x_{\ell-1}) \tag{16.8}$$

instead of $x_\ell = f_\ell(x_{\ell-1})$ [10]. Differentiate: the derivative of x with respect to itself is 1, and the derivative of $f_\ell(x)$ is f'_ℓ, so

$$\frac{dx_\ell}{dx_{\ell-1}} = 1 + f'_\ell. \tag{16.9}$$

Chained over L layers:

$$\frac{dx_L}{dx_0} = (1 + f'_L)(1 + f'_{L-1}) \cdots (1 + f'_1). \tag{16.10}$$

Multiply out the brackets and the first term is $1 \cdot 1 \cdots 1 = 1$: there is always a path along which the signal passes untouched, whatever the layers do. (For vectors the 1 becomes the identity matrix $\mathbf{I}$ and f' becomes the matrix of partial derivatives, the *Jacobian*; the conclusion is the same.)

Worked example 16.2: Twenty layers with and without residuals

Suppose every layer's derivative is the same number and there are 20 layers.
Without residuals, Equation (16.7) gives $(f')^{20}$:

$$f' = 0.9: \quad 0.9^{20} = 0.1216$$
$$f' = 0.5: \quad 0.5^{20} = 0.000\,000\,954.$$

Even a mild 0.9 per layer loses 88% of the signal; 0.5 per layer loses all but one part in a million. This is the *vanishing gradient*.
With residuals, Equation (16.10) gives $(1 + f')^{20}$. Sublayers start near zero output, so their derivatives start small:

$$f' = +0.05: \quad 1.05^{20} = 2.6533$$
$$f' = -0.05: \quad 0.95^{20} = 0.3585.$$

Both are of order 1. The signal reaching the first layer is neither lost nor exploded.

The "learn a correction" view. Unroll Equation (16.8) from the input:

$$\begin{aligned} x_1 &= x_0 + f_1(x_0) \\ x_2 &= x_1 + f_2(x_1) = x_0 + f_1(x_0) + f_2(x_1) \\ &\vdots \\ x_L &= x_0 + \sum_{\ell=1}^{L} f_\ell(x_{\ell-1}). \end{aligned}$$

The output is the input plus a sum of corrections. A sublayer that has nothing useful to add can learn $f_\ell = 0$ (all weights near zero), which is easy. Without the residual, "do nothing" would mean learning to be exactly the identity function, which is surprisingly hard for a stack of matrices and nonlinearities.

16.4 Pre-norm, not post-norm

Where the LayerNorm sits matters.

	post-norm (the original 2017 transformer)	pre-norm (this model)
formula	$x \leftarrow \mathrm{LN}(x + f(x))$	$x \leftarrow x + f(\mathrm{LN}(x))$
the residual path	passes through a LayerNorm every sublayer	untouched

In post-norm the derivative of each step is $\mathrm{LN}' \cdot (1 + f')$: the clean "1" of Equation (16.9) is multiplied by LayerNorm's own derivative at every one of the 16 sublayers, so the motorway passes through a toll booth every block. In pre-norm the step is $x + f(\mathrm{LN}(x))$, whose derivative is $1 + f' \cdot \mathrm{LN}'$: the 1 survives, and the unrolled sum above holds exactly. Modern transformers (GPT-2, ViT, SegFormer and this code) use pre-norm. The source says so in its docstring (`segformer_block.py`, lines 4–13).

16.5 Dropout

16.5.1 What it does

Dropout [42], during training, sets each number of a tensor to zero independently with probability p, and multiplies the survivors by $1/(1-p)$. At evaluation it does nothing.

Why the factor $1/(1-p)$. Write the dropped value as $m\,x/(1-p)$ where m is 1 with probability $1-p$ and 0 with probability p. Its expected value is

$$\mathbb{E}\left[\frac{m\,x}{1-p}\right] = \frac{x}{1-p}\,\mathbb{E}[m] = \frac{x}{1-p}\,\big((1-p)\cdot 1 + p\cdot 0\big) = \frac{x}{1-p}(1-p) = x.$$

On average the output equals the input, so switching dropout off at evaluation changes nothing on average.

Worked example 16.3: Dropout with $p = 0.4$

Scale factor $1/(1-0.4) = 1/0.6 = 1.6667$. Take $x = (0.5, -1.0, 2.0, 0.3, 1.2)$ and suppose the random draw keeps positions 1, 3 and 4 ($m = (1, 0, 1, 1, 0)$):

$$\begin{aligned} 0.5 \times 1 \times 1.6667 &= 0.8333 \\ -1.0 \times 0 &= 0 \\ 2.0 \times 1 \times 1.6667 &= 3.3333 \\ 0.3 \times 1 \times 1.6667 &= 0.5000 \\ 1.2 \times 0 &= 0. \end{aligned}$$

Two of five values vanished; the survivors grew by two thirds.

16.5.2 Where it sits

The block applies dropout to the sublayer's output *before* the residual addition,

$$x + \text{Dropout}\,(f(\text{LN}(x))).$$

Only the new contribution can be dropped; the information travelling along the skip connection is never touched. Dropping the residual stream itself would destroy accumulated information at random, which is damage, not regularisation.

Values that will drift

Dropout rates (`drop_rate`): 0.3 in v0.1.0 [M: `configs/hyperspectral_segformer_exp_1.json`] and 0.4 from v0.2.0 on [M: `configs/hyperspectral_segformer_exp_2.json`], also 0.4 in FT04 [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]. At 0.4 nearly half of every sublayer's output is discarded in training; it was raised to fight overfitting. At inference dropout is off twice over: the model is built with `drop_rate=0.0` and put in `model.eval()` mode (IW15).

Source check: one dropout module, and no drop path

`segformer_block.py` builds a single `self.drop=nn.Dropout(drop_ratio)` (line 79) and uses it after both sublayers (lines 96 and 103). That is harmless: dropout has no parameters and draws a fresh random mask on every call. The block has no *drop path* (stochastic depth, which drops a whole sublayer's contribution for a whole sample and which many transformer implementations add): a search of `app/` for `DropPath` or `drop_path` finds nothing. The only

regularising randomness inside the block is this element-wise dropout.

16.6 Mix-FFN

16.6.1 The feed-forward network, and why it widens

The standard transformer feed-forward network processes each token on its own:

$$\text{Linear}(C \to EC) \;\to\; \text{nonlinearity} \;\to\; \text{Linear}(EC \to C),$$

with an *expansion ratio* $E = 4$ here

[M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`].

The middle is four times wider than the ends. A wider middle gives the nonlinearity more room: the token is unfolded into $4C$ numbers, each of which can be bent independently, and then folded back. Going from 32 to 32 through one nonlinearity is far more restrictive than from 32 to 128 and back.

16.6.2 SegFormer's twist: a depthwise convolution in the middle

SegFormer's Mix-FFN [46] inserts a 3×3 convolution on the token grid between the expansion and the nonlinearity:

$$\begin{aligned}(B, N, C) &\xrightarrow{\text{fc1: Linear}(C \to 4C)} (B, N, 4C) \xrightarrow{\text{transpose, reshape}} (B, 4C, H, W) \\ &\xrightarrow{\text{dwconv } 3\times3} (B, 4C, H, W) \xrightarrow{\text{reshape, transpose}} (B, N, 4C) \\ &\xrightarrow{\text{GELU}} (B, N, 4C) \xrightarrow{\text{fc2: Linear}(4C \to C)} (B, N, C).\end{aligned}$$

The convolution has stride 1 and padding 1, so by Equation (15.2) the grid keeps its size: $\lfloor (H + 2 - 3)/1 \rfloor + 1 = H$. *Mix-FFN never changes the number of tokens.*

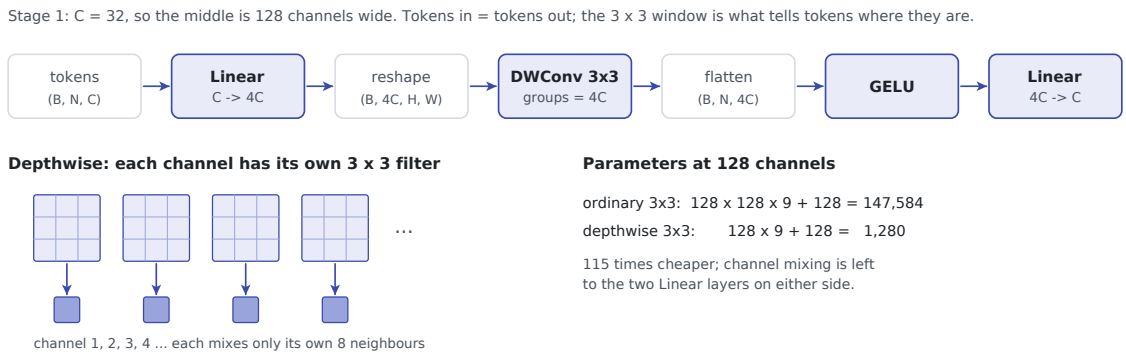


Figure 16.1: The feed-forward half of a block. Tokens are expanded fourfold, reshaped back to their grid, mixed with their eight neighbours by a depthwise 3×3 convolution (one private filter per channel), passed through GELU and projected back. The depthwise convolution is what gives the model a sense of position.

16.6.3 Depthwise convolution

Definition 16.1 (Depthwise convolution)**.** A convolution with `groups` equal to the number of channels: each channel has its own private 3×3 filter and never looks at any other channel,

$$y_{c,i,j} = b_c + \sum_{a=-1}^{1} \sum_{b=-1}^{1} w_{c,a,b}\, x_{c,\, i+a,\, j+b}, \tag{16.11}$$

with $x = 0$ outside the grid (zero padding).

In code (`mix_ffn.py`, line 79):

```
self.dwconv = nn.Conv2d(hidden_dim, hidden_dim, kernel_size=3,
                        padding=1, groups=hidden_dim)   # (B, 4C, H, W) -> (B, 4C, H,
    W)
```

Parameters. An ordinary 3×3 convolution from $4C$ channels to $4C$ channels has $4C \cdot 4C \cdot 9 + 4C$; the depthwise one has 9 weights per channel and a bias:

$$\text{dwconv} = 4C \cdot 9 + 4C = 40C. \tag{16.12}$$

Worked example 16.4: Ordinary against depthwise at stage 1

Stage 1 has $C = 32$, so $4C = 128$ hidden channels.

$$\begin{aligned} \text{ordinary: } & 128 \times 128 \times 9 + 128 = 16{,}384 \times 9 + 128 = 147{,}456 + 128 = 147{,}584 \\ \text{depthwise: } & 128 \times 9 + 128 = 1{,}152 + 128 = 1{,}280. \end{aligned}$$

$147{,}584/1{,}280 = 115$ times fewer. Nothing is lost, because `fc1` and `fc2` on either side already mix channels; the convolution's job is purely spatial.

16.6.4 Where position comes from

Section 14.8 showed that attention cannot tell where a token is. A convolution can: its output at (i, j) depends on the tokens at $(i \pm 1, j \pm 1)$, so "next to" is built into it. More than that, the zero padding makes the border different from the interior, and over several layers that difference spreads inward, so the network can tell how far a token is from the edge. SegFormer's authors argue that this 3×3 convolution provides enough positional information to do without positional encodings [46]; this codebase has none.

Worked example 16.5: A depthwise filter sees the border

One channel on a 3×3 grid, values rising evenly:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \qquad \text{filter } \begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \text{ bias } 0.$$

The filter adds the four direct neighbours and subtracts four times the centre.
Centre $(2, 2)$. Neighbours 2 (above), 4 (left), 6 (right), 8 (below):

$$2 + 4 + 6 + 8 - 4 \times 5 = 20 - 20 = 0.$$

Top-left corner $(1, 1)$. Above and left are padding zeros; right is 2, below is 4:

$$0 + 0 + 2 + 4 - 4 \times 1 = 6 - 4 = 2.$$

Top-middle $(1, 2)$. Above is padding; left 1, right 3, below 5:

$$0 + 1 + 3 + 5 - 4 \times 2 = 9 - 8 = 1.$$

The whole output, from the script [C]:

$$\begin{pmatrix} 2 & 1 & -4 \\ -3 & 0 & -7 \\ -16 & -11 & -22 \end{pmatrix}.$$

On a perfectly even ramp the interior gives 0, but every border position gives something else, and each corner something different. The output carries a trace of *where* each token is, which no amount of attention could produce.

16.6.5 GELU

The nonlinearity is the Gaussian Error Linear Unit [13]:

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \Phi(x) = \tfrac{1}{2}\left(1 + \mathrm{erf}\left(\tfrac{x}{\sqrt{2}}\right)\right), \tag{16.13}$$

where $\Phi(x)$ is the probability that a standard Gaussian random number is below x (the Gaussian's cumulative distribution function; erf is the error function that computes it). Read it as "let x through in proportion to how likely a Gaussian is to be below it": large positive x passes almost whole ($\Phi \approx 1$), large negative x is almost blocked ($\Phi \approx 0$), and in between it bends smoothly. PyTorch's `nn.GELU()` (used at `mix_ffn.py`, line 86) computes this exact form; a common approximation is $\frac{1}{2}x(1 + \tanh(\sqrt{2/\pi}\,(x + 0.044715x^3)))$.

Worked example 16.6: GELU at -1, 0 and 1

From tables of the Gaussian, $\Phi(1) = 0.8413$, and by symmetry $\Phi(-1) = 1 - 0.8413 = 0.1587$; $\Phi(0) = 0.5$.

$$\begin{aligned} \mathrm{GELU}(-1) &= -1 \times 0.1587 = -0.1587 \\ \mathrm{GELU}(0) &= 0 \times 0.5 = 0 \\ \mathrm{GELU}(1) &= 1 \times 0.8413 = 0.8413. \end{aligned}$$

PyTorch gives $(-0.1587, 0, 0.8413)$ [C]; the tanh approximation gives $(-0.1588, 0, 0.8412)$.
The slope. By the product rule, $\mathrm{GELU}'(x) = \Phi(x) + x\,\varphi(x)$, where $\varphi(x) = e^{-x^2/2}/\sqrt{2\pi}$ is the Gaussian density and $\varphi(1) = \varphi(-1) = 0.2420$:

$$\begin{aligned} \mathrm{GELU}'(-1) &= 0.1587 + (-1)(0.2420) = -0.0833 \\ \mathrm{GELU}'(0) &= 0.5 + 0 = 0.5 \\ \mathrm{GELU}'(1) &= 0.8413 + 0.2420 = 1.0833. \end{aligned}$$

Unlike ReLU, whose slope jumps from 0 to 1 at the origin, GELU's slope changes smoothly; it even dips slightly below zero for negative inputs, so GELU is not monotonic near $x = -0.75$.

GELU sits *after* the depthwise convolution and *before* `fc2`, that is, in the wide $4C$-dimensional space, where the nonlinearity has the most numbers to bend.

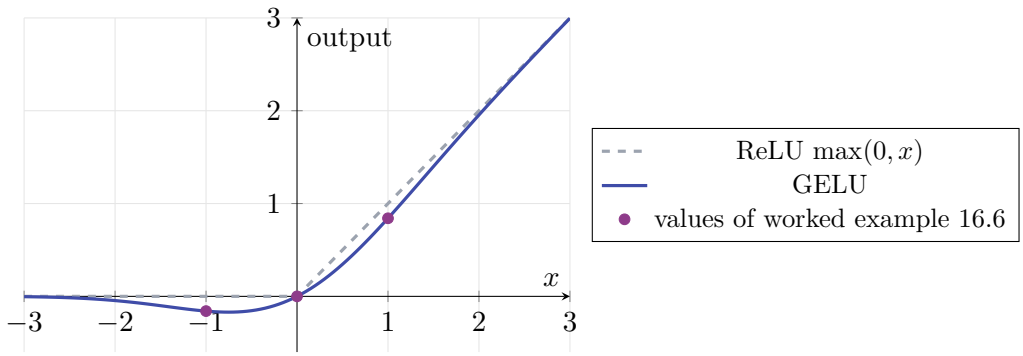


Figure 16.2: GELU against ReLU. GELU is a smooth ReLU: near zero for very negative inputs, a soft bend through the origin, near-linear for large inputs. The curve is drawn with the tanh approximation, which differs from the exact form by less than 0.001 on this range.

16.6.6 The sparse fallback

Like the reduction of Section 15.7, the depthwise convolution needs a full grid, $N = H \cdot W$. When stage-1 tokens have been removed it is skipped (`mix_ffn.py`, line 119):

```
if N == H * W:
    ...                          # reshape to grid, dwconv, reshape back: (B, N, 4C)
else:
    sequence_x = expanded_x      # (B, N, 4C): no spatial mixing
```

In a masked pass, stage 1's Mix-FFN is a plain two-layer network with a GELU in the middle. The docstring calls this "a minor quality tradeoff for correct handling of sparse token sets", since the attention still supplies spatial context. As with the reduction convolution, the stage-1 depthwise filters receive no gradient from a masked pass.

16.6.7 Mix-FFN parameter count

Three layers: `fc1` has $C \cdot 4C + 4C$, `dwconv` has $40C$ (Equation (16.12)), `fc2` has $4C \cdot C + C$. In total

$$\text{Mix-FFN}(C) = (4C^2 + 4C) + 40C + (4C^2 + C) = 8C^2 + 45C. \tag{16.14}$$

Worked example 16.7: Mix-FFN at stage 1

$C = 32$, hidden width $4 \times 32 = 128$.

$$\begin{aligned} \text{fc1:}\ & 32 \times 128 + 128 = 4{,}096 + 128 = 4{,}224 \\ \text{dwconv:}\ & 128 \times 9 + 128 = 1{,}152 + 128 = 1{,}280 \\ \text{fc2:}\ & 128 \times 32 + 32 = 4{,}096 + 32 = 4{,}128 \\ \text{total:}\ & 4{,}224 + 1{,}280 + 4{,}128 = 9{,}632. \end{aligned}$$

Check with Equation (16.14): $8 \times 32^2 + 45 \times 32 = 8 \times 1{,}024 + 1{,}440 = 8{,}192 + 1{,}440 = 9{,}632$.

16.7 The block

16.7.1 Two lines

Putting the pieces together (`segformer_block.py`, lines 95–104):

$$\mathbf{a} = \mathbf{x} + \text{Dropout}\left(\text{ESA}(\text{LN}_1(\mathbf{x}), H, W)\right), \tag{16.15}$$
$$\mathbf{y} = \mathbf{a} + \text{Dropout}\left(\text{MixFFN}(\text{LN}_2(\mathbf{a}), H, W)\right). \tag{16.16}$$

```
attn = self.drop(self.attn(self.norm1(x), H, W))   # (B, N, C)
attn = x + attn                                    # (B, N, C) residual 1
ffn  = self.drop(self.ffn(self.norm2(attn), H, W)) # (B, N, C)
ffn  = attn + ffn                                  # (B, N, C) residual 2
```

Read it aloud: *normalise, attend, maybe drop some, add it to what we had; then normalise, think, maybe drop some, add it to what we had.* The second residual adds to $\mathbf{a}$, the output of the first sublayer, not to the original $\mathbf{x}$; information accumulates through both. The shape `(B, N, C)` never changes inside a block.

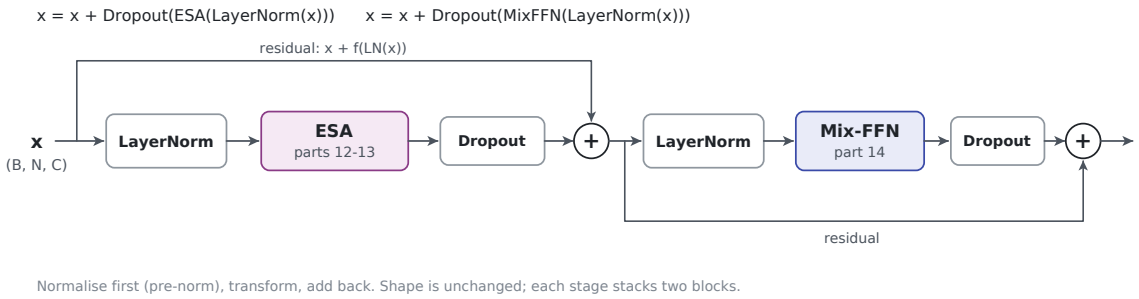


Figure 16.3: One SegFormer block. Each of the two sublayers, efficient self-attention (plum) and Mix-FFN (indigo), normalises its input first (pre-norm), transforms it, applies dropout and adds the result back to its input through a residual connection. The shape (B, N, C) is unchanged; each encoder stage stacks two blocks.

Each stage stacks `num_blocks` $= [2, 2, 2, 2]$ blocks, the SegFormer-B0 setting [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`].

Within a stage all blocks share C, h and R; width and resolution change only at the patch embeddings between stages (Chapter 17).

16.7.2 The parameter count, by hand

A block is two LayerNorms, one ESA and one Mix-FFN. With Equations (15.6), (16.5) and (16.14):

$$\text{block}(C, R) = \underbrace{2 \cdot 2C}_{\text{norm1, norm2}} + \underbrace{4C^2 + 4C + [R > 1](C^2R^2 + 3C)}_{\text{ESA}} + \underbrace{8C^2 + 45C}_{\text{Mix-FFN}}, \tag{16.17}$$

where $[R > 1]$ is 1 if $R > 1$ and 0 otherwise.

Worked example 16.8: A stage-1 block: 79,616 parameters

$C = 32$, $R = 8$, $h = 2$ (the head count does not matter, Section 14.9).

$$\begin{aligned}
\text{norm1: } & 2 \times 32 = 64 \\
\text{q, k, v, proj: } & 4 \times (32 \times 32 + 32) = 4 \times 1{,}056 = 4{,}224 \\
\text{reduction conv: } & 32 \times 32 \times 8 \times 8 + 32 = 65{,}536 + 32 = 65{,}568 \\
\text{reduction norm: } & 2 \times 32 = 64 \\
\text{norm2: } & 2 \times 32 = 64 \\
\text{fc1: } & 32 \times 128 + 128 = 4{,}224 \\
\text{dwconv: } & 128 \times 9 + 128 = 1{,}280 \\
\text{fc2: } & 128 \times 32 + 32 = 4{,}128
\end{aligned}$$

Add, in groups: ESA $= 4{,}224 + 65{,}568 + 64 = 69{,}856$; Mix-FFN $= 4{,}224 + 1{,}280 + 4{,}128 = 9{,}632$; the two block norms $= 64 + 64 = 128$. One block:

$$69{,}856 + 9{,}632 + 128 = 79{,}616.$$

Two blocks per stage: $2 \times 79{,}616 = 159{,}232$.
Check against the real model. The `torchinfo` summary of the hyperspectral SegFormer MAE lists stage 1's block list (`ModuleList:3-2`) at 159,232 parameters [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`], and the course on Indradhanu Heavy quotes FT04's stage-1 block as 79,616 [M: `indradhanuworkings/68-indradhanu-heavy-pretraining.md`]. Building `SegFormerBlock(32,2,8)` in the script gives 79,616 with exactly the eight per-layer counts above [C]. You have predicted a number from a real model by adding up pieces derived by hand in three chapters.

Worked example 16.9: A stage-4 block: 800,000 parameters

$C = 256$, $R = 1$ (no reduction layers), hidden width $4 \times 256 = 1{,}024$.

$$\begin{aligned}
\text{norm1: } & 2 \times 256 = 512 \\
\text{q, k, v, proj: } & 4 \times (256 \times 256 + 256) = 4 \times (65{,}536 + 256) = 4 \times 65{,}792 = 263{,}168 \\
\text{norm2: } & 2 \times 256 = 512 \\
\text{fc1: } & 256 \times 1{,}024 + 1{,}024 = 262{,}144 + 1{,}024 = 263{,}168 \\
\text{dwconv: } & 1{,}024 \times 9 + 1{,}024 = 9{,}216 + 1{,}024 = 10{,}240 \\
\text{fc2: } & 1{,}024 \times 256 + 256 = 262{,}144 + 256 = 262{,}400
\end{aligned}$$

ESA $= 263{,}168$; Mix-FFN $= 263{,}168 + 10{,}240 + 262{,}400 = 535{,}808$; norms $= 512 + 512 = 1{,}024$. One block:

$$263{,}168 + 535{,}808 + 1{,}024 = 800{,}000.$$

A round number by coincidence. Two blocks: 1,600,000, which is the `torchinfo` figure for stage 4's block list [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`], and the script's [C].

All four stages, checked against the built blocks [C] and the `torchinfo` table [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`]:

stage	C	ESA	Mix-FFN	2 LNs	one block	two blocks
1	32	69,856	9,632	128	79,616	159,232
2	64	82,368	35,648	256	118,272	236,544
3	160	205,920	212,000	640	418,560	837,120
4	256	263,168	535,808	1,024	800,000	1,600,000
all eight blocks						2,832,896

The balance flips. At stage 1 attention is 88% of the block (69,856/79,616), almost all of it the 8×8 reduction convolution. At stage 4 Mix-FFN is 67% (535,808/800,000): with no reduction and $C = 256$, the two $C \times 4C$ matrices dominate. Chapter 20 puts these blocks into the budget of the whole model.

Values that will drift

Indradhanu Heavy (Chapter 43) uses the same block with $C = 96$ at stage 1; its stage-1 block is 705,792 parameters [M: `indradhanuworkings/68-indradhanu-heavy-pretraining.md`], which the script reproduces [C]. You count it in Exercise 16.5.

16.8 Common confusions

Why two LayerNorms per block?
One before attention, one before Mix-FFN. Each sublayer gets a freshly standardised copy of its input. The ESA's reduction has a third LayerNorm of its own, on the summary tokens.

Is the plus sign ordinary addition?
Yes: element-by-element addition of two tensors of the same shape `(B, N, C)`. Its power is in the derivative, $1 + f'$.

Is Mix-FFN mixing tokens or channels?
Both, in different places. `fc1` and `fc2` mix channels within each token; the depthwise convolution mixes neighbouring tokens within each channel. Between them everything is mixed, hence the name.

Does Mix-FFN change the number of tokens?
No. Stride 1 and padding 1 keep the grid size. Only the patch embeddings change the token count.

LayerNorm or BatchNorm: which normalises over the batch?
BatchNorm. LayerNorm uses one token's own C features and nothing else, and so behaves the same in training and evaluation.

Does dropout run at inference?
No. `drop_rate=0.0` and `model.eval()` both switch it off. Run-to-run variation at inference comes from the masking strategy (Chapter 30), not from dropout.

Is GELU the same as ReLU?
It is a smooth relative of ReLU. It equals ReLU far from zero, but it is curved near zero, its slope changes continuously, and it lets small negative values through ($\text{GELU}(-1) = -0.1587$).

16.9 Check yourself

Exercise 16.1. LayerNorm the token $(2.0, 4.0, 4.0, 6.0)$ with $\gamma = 1$, $\beta = 0$ (ignore ϵ). Show the mean, every deviation and square, the variance, the standard deviation and the four outputs.

Then predict, without calculating, $\mathrm{LN}(10, 20, 20, 30)$.

Exercise 16.2. Compute $\mathrm{GELU}(2)$ and $\mathrm{GELU}(-2)$ given $\Phi(2) = 0.9772$. What fraction of the input does GELU pass at $x = 2$ and at $x = -2$?

Exercise 16.3. A stack of 16 sublayers (the encoder's eight blocks) each has derivative $f' = -0.1$. Compute the end-to-end derivative without residuals and with pre-norm residuals (assume LayerNorm's derivative is absorbed into f'). Which one still lets the first layer learn?

Exercise 16.4. Count the Mix-FFN parameters for stage 2 ($C = 64$) layer by layer, and check your answer with $8C^2 + 45C$.

Exercise 16.5. Indradhanu Heavy's stage-1 block has $C = 96$, $R = 8$, expansion 4. Count its parameters layer by layer and confirm 705,792. Why is it close to nine times FT04's 79,616 but not exactly nine?

Exercise 16.6. With dropout probability $p = 0.4$, what is the expected output and the variance of the dropped value $m\,x/(1-p)$ for an input $x = 1$? (Use $\mathrm{Var} = \mathbb{E}[\cdot^2] - (\mathbb{E}[\cdot])^2$.)

Next: Chapter 17, The four-stage encoder.

Chapter 17

The four-stage encoder

The one thing this chapter teaches

The encoder looks at the same 128 × 128 patch at four zoom levels, from a 32 × 32 grid of narrow tokens down to a 4 × 4 grid of wide ones, and hands all four to the decoder; it hides tokens at the first zoom level only, and the pyramid itself blurs those holes away.

You now own every part of one encoder stage: the patch embedding that turns a picture into tokens (Chapter 13), efficient self-attention (Chapter 15), and the transformer block that wraps attention and Mix-FFN with LayerNorms and residuals (Chapter 16). This chapter stacks four of those stages into a pyramid. The new mathematics is modest: one output-size formula applied four times, a receptive-field recurrence, and an expected-value calculation that tells you how many stage-2 tokens see no visible data at all. The new *ideas* are bigger, because the pyramid is what makes masked training work.

The source is `app/foundation_models/components/seg_former_encoder.py`. It is 261 lines [C], over half of them comments. Read it end to end after this chapter.

17.1 Why one resolution is not enough

17.1.1 The plain Vision Transformer

A plain Vision Transformer (ViT) [5] cuts the image into patches once, turns each patch into a token, and then keeps that one grid of tokens, at one width, through every layer. If you did that here with 4 × 4 pixel patches you would have $N = 32 \times 32 = 1024$ tokens in every layer. Full attention compares every token with every other token, so every layer would build a score table of

$$\begin{aligned} N \times N &= 1024 \times 1024 \\ &= 1{,}048{,}576 \end{aligned}$$

entries per head. Efficient self-attention (Chapter 15) cuts that down, but the deeper problem is not cost. It is that one grid fixes one scale of question.

17.1.2 Anomalies come in every size

What the model must learn is "what normally goes here". The answer depends on the size of the thing you are asking about:

Table 17.1: The scale of an anomaly decides which zoom level sees it best.

Anomaly	Typical extent	Best seen by
one odd pixel (a flare, a small vessel)	1 pixel	F1, the only map with that resolution
a field of unusual material	about 20 pixels	F2 and F3
a whole region that does not belong	about 200 pixels	F4

A single-resolution model must pick one of those rows. A hierarchical encoder returns four feature maps at four scales and lets the decoder (Chapter 19) fuse them, so nothing has to be chosen in advance.

Analogy

You are surveying unfamiliar terrain with four maps open at once: a street plan, a city map, a regional map and a country map. The street plan tells you exactly *where* each building stands but nothing about the region; the country map tells you *what kind* of region you are in but cannot locate a house. A surveyor who wants to spot a building that does not belong uses all four together. The encoder's four stages are those four maps, printed on the same ground: F1 is the street plan, F4 the country map. The analogy will keep working in Section 17.6: if somebody tears holes in the street plan, the city map still covers the ground underneath.

17.2 The pyramid, stage by stage

17.2.1 The output-size formula

Every stage starts with a patch embedding, which is a convolution (Chapter 10, Chapter 13). A reminder of what you need: a convolution with kernel size k, stride s and padding p turns an input grid of side H_{in} into an output grid of side

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2p - k}{s} \right\rfloor + 1. \tag{17.1}$$

In words: pad both sides by p, slide a window of width k in steps of s, and count the positions at which the window still fits. The floor $\lfloor \cdot \rfloor$ rounds down, because a window that would hang over the edge is not computed.

The encoder uses two settings, read from the constructor (`seg_former_encoder.py`, lines 108–123):

- stage 1: $k = 4$, $s = 4$, $p = 0$ (non-overlapping 4×4 blocks);
- stages 2–4: $k = 3$, $s = 2$, $p = \lfloor 3/2 \rfloor = 1$ (overlapping windows).

Worked example 17.1: The four grid sizes from the formula

Do this on paper. The input to stage 1 is the compressor's output, 128×128.

Stage 1, $k = 4$, $s = 4$, $p = 0$:

$$\begin{aligned} H_1 &= \left\lfloor \frac{128 + 2 \cdot 0 - 4}{4} \right\rfloor + 1 && \text{substitute} \\ &= \left\lfloor \frac{124}{4} \right\rfloor + 1 && 128 + 0 - 4 = 124 \\ &= \lfloor 31 \rfloor + 1 && 124/4 = 31 \text{ exactly} \\ &= 32. \end{aligned}$$

Stage 2, $k = 3$, $s = 2$, $p = 1$, input 32:

$$H_2 = \left\lfloor \frac{32 + 2 \cdot 1 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{31}{2} \right\rfloor + 1 = \lfloor 15.5 \rfloor + 1 = 15 + 1 = 16.$$

Stage 3, input 16:

$$H_3 = \left\lfloor \frac{16 + 2 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{15}{2} \right\rfloor + 1 = 7 + 1 = 8.$$

Stage 4, input 8:

$$H_4 = \left\lfloor \frac{8 + 2 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{7}{2} \right\rfloor + 1 = 3 + 1 = 4.$$

So the grids are $128 \to 32 \to 16 \to 8 \to 4$. Notice that in stages 2–4 the floor throws away a half every time: the padded input has an odd length (31, 15, 7), and the last half-step does not fit a whole window. That is why $p = 1$ with $k = 3$, $s = 2$ halves an even side exactly.

Every grid is square, so the token count of stage i is $N_i = H_i \times W_i = H_i^2$:

$$N_1 = 32 \times 32 = 1024, \qquad N_2 = 16 \times 16 = 256,$$
$$N_3 = 8 \times 8 = 64, \qquad N_4 = 4 \times 4 = 16.$$

Each stage has a quarter as many tokens as the one before: halving each side quarters the area.

17.2.2 Widths go up as grids go down

Each patch embedding also sets the *embedding width* C_i, the number of features every token carries. The configuration is the SegFormer-B0 one [46]: $C = (32, 64, 160, 256)$. Two quantities move in opposite directions:

- **spatial detail falls**, from a 32×32 grid to a 4×4 grid;
- **channel width rises**, from 32 to 256 numbers per token.

Early stages know *where* things are but little about *what* they are; late stages know a great deal about *what* but almost nothing about *where*. A pyramid like this, with grid size falling and width rising stage by stage, is called a *feature pyramid*.

How much information does each stage hold? Count the numbers in one feature map, $N_i \times C_i$:

$$\begin{aligned} \text{F1}: \ & 1024 \times 32 = 32{,}768, \\ \text{F2}: \ & 256 \times 64 = 16{,}384, \\ \text{F3}: \ & 64 \times 160 = 10{,}240, \\ \text{F4}: \ & 16 \times 256 = 4{,}096. \end{aligned}$$

The width grows by ×2, ×2.5, ×1.6 while the token count falls by ×4 each time, so each map holds fewer numbers than the last. The pyramid *summarises*: it is a compression, not a reshuffle. Compare the input to stage 1: $32 \times 128 \times 128 = 524{,}288$ numbers. F1 already holds sixteen times fewer.

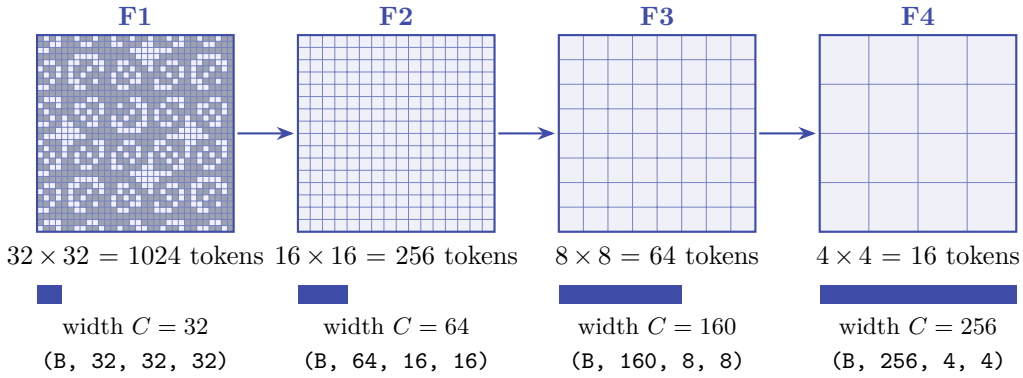


Figure 17.1: The encoder pyramid drawn to scale. Each square covers the same 128 × 128 pixels of ground; its grid lines are the stage's tokens (stage 1: one token per 4 × 4 pixels, stage 4: one per 32 × 32). The bar under each square is the token width, drawn at one scale (256 channels = the full square width). Grey cells in F1 are hidden tokens at a 65% hiding rate (an illustrative pattern); only F1 has them. Arrows are patch embeddings (stride 2, kernel 3).

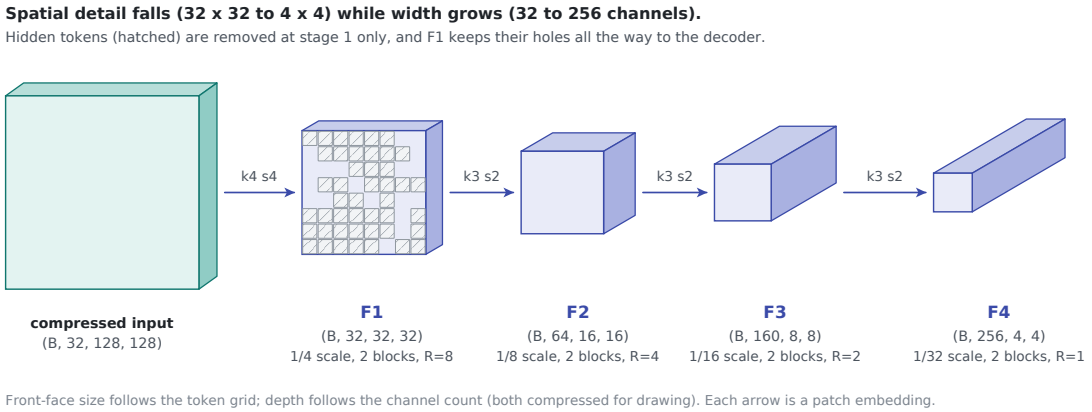


Figure 17.2: The course's view of the same pyramid: each patch embedding (arrow) shrinks the grid, by 4 at stage 1 and by 2 after, and widens the channels, producing F1 to F4 at 1/4 to 1/32 of the input resolution. Hidden tokens (hatched) exist at stage 1 only. Front-face size and depth are compressed for drawing; Figure 17.1 is to scale.

17.2.3 The full configuration

The head dimension is C_i divided by the number of heads (Chapter 14): $32/2 = 16$, $64/2 = 32$, $160/5 = 32$, $256/8 = 32$.

Source check: the number of heads

The model class's default is `num_heads = [1, 2, 5, 8]` (`app/foundation_models/components/hyperspectral_seg_former_mae.py`, line 71), and the encoder's docstring table shows 1 head at stage 1 (`seg_former_encoder.py`, line 80). The configurations that trained the shipped models override it to `[2, 2, 5, 8]` (`configs/hyperspectral_`

Table 17.2: The encoder configuration. The last two columns are derived in Section 17.3.

Stage	C_i	heads	head dim	R_i	blocks	grid	N_i	keys N_i/R_i^2
1	32	2	16	8	2	32×32	1024	16
2	64	2	32	4	2	16×16	256	16
3	160	5	32	2	2	8×8	64	16
4	256	8	32	1	2	4×4	16	16

`segformer_exp_2.json`, line 33, and `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`). The v0.1.0 configuration used `[1, 2, 5, 8]`. Heads change no parameter count (Chapter 20), so a checkpoint from either loads into either shape, but they change what the model computes. Trust the config, not the default.

Values that will drift

Configuration values as of 2026-10-29: `num_heads = [2, 2, 5, 8]` [M: `configs/hyperspectral_segformer_exp_2.json`], `embed_dims = [32, 64, 160, 256]`, `reduction_ratios = [8, 4, 2, 1]`, `num_blocks = [2, 2, 2, 2]` [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]. Indradhanu Heavy (Chapter 43) widens the stages; everything in this chapter is for the standard widths.

17.3 The attention budget per stage

Efficient self-attention shrinks the keys and values by a reduction ratio R in each direction (Chapter 15). A grid of $H_i \times W_i$ tokens becomes $(H_i/R_i) \times (W_i/R_i)$ keys, so the key count is N_i/R_i^2:

$$\begin{aligned} \text{stage 1:} \quad & \frac{1024}{8^2} = \frac{1024}{64} = 16, \\ \text{stage 2:} \quad & \frac{256}{4^2} = \frac{256}{16} = 16, \\ \text{stage 3:} \quad & \frac{64}{2^2} = \frac{64}{4} = 16, \\ \text{stage 4:} \quad & \frac{16}{1^2} = 16. \end{aligned}$$

Every stage attends over exactly 16 keys. That is not a coincidence: the reduction ratios $(8, 4, 2, 1)$ halve as the grid halves, so each stage's keys form the same 4×4 summary of the patch. The score table per head is queries times keys, $N_i \times 16$:

$$1024 \times 16 = 16{,}384, \quad 256 \times 16 = 4{,}096, \quad 64 \times 16 = 1{,}024, \quad 16 \times 16 = 256.$$

Stage 1 has the most queries and therefore the largest table, but the table is 64 times smaller than the 1,048,576 entries of full attention from Section 17.1.

Source check: masked stage 1 falls back to full attention

When tokens have been removed, stage 1 holds fewer than $H \times W$ tokens, so ESA cannot fold them back into a grid for its reduction convolution. It tests `can_reduce = (self.reduction_ratio > 1) and (N == H * W)` (`app/foundation_models/components/efficient_self_attention.py`, line 144) and, when that fails, uses the kept tokens themselves as keys: full attention among the survivors. Mix-FFN likewise skips its depth-

wise convolution when $N \neq H \times W$ (`mix_ffn.py`, lines 118–137). Two comments in `seg_former_encoder.py` describe something else: lines 203–206 say "ESA's reshape will produce a grid with gaps, which is fine: the reduction conv pools over them", and lines 219–223 say "we pass the actual token grid dims instead". Neither happens: the encoder passes the *full* H, W, and ESA takes its full-attention fallback. The source code wins over its comments.
The consequence for cost: with 359 kept tokens (Section 17.7) the masked score table is $359 \times 359 = 128{,}881$ entries per head [C], about eight times the unmasked reduced table of 16,384. Masking makes the linear layers of stage 1 about three times cheaper, but makes its attention scores more expensive. The ESA comment (line 142) estimates 262K entries "very cheap" for 50% masking; at 65% it is 129K.

17.4 How far each token sees: the receptive field

Definition 17.1 (Receptive field)**.** The *receptive field* of a token is the set of input pixels that can change its value through the convolutions that built it. For a square window we quote its side, in pixels.

Attention lets every token read every other token, so in a transformer the receptive field is in principle the whole patch. But the *patch embeddings* alone already fix how many pixels go into a token *before* attention runs, and that is what decides how fine a stage's picture is. Two numbers track it:

- r_i, the receptive field side, in input pixels, of one stage-i token;
- j_i, the *jump*: how many input pixels apart two neighbouring stage-i tokens are.

A convolution with kernel k and stride s applied to a grid with jump j and receptive field r gives

$$r_{\text{out}} = r + (k-1)\, j, \qquad j_{\text{out}} = j\, s. \tag{17.2}$$

Derive it in words. The output window covers k neighbouring input tokens. The first of them sees r pixels. Each further token is j pixels along, so it adds j new pixels at the far edge. There are $k-1$ further tokens, giving $r + (k-1)j$. Neighbouring outputs are s input tokens apart, each j pixels apart, so the new jump is $s \cdot j$.

Worked example 17.2: Receptive fields of the four stages

Start from the input pixels: $r_0 = 1$ (a pixel sees itself), $j_0 = 1$. Do each line on paper.
Stage 1 ($k = 4$, $s = 4$):

$$r_1 = r_0 + (4-1)\, j_0 = 1 + 3 \cdot 1 = 4, \qquad j_1 = j_0 \cdot 4 = 4.$$

Stage 2 ($k = 3$, $s = 2$):

$$r_2 = r_1 + (3-1)\, j_1 = 4 + 2 \cdot 4 = 12, \qquad j_2 = 4 \cdot 2 = 8.$$

Stage 3:

$$r_3 = 12 + 2 \cdot 8 = 28, \qquad j_3 = 8 \cdot 2 = 16.$$

Stage 4:

$$r_4 = 28 + 2 \cdot 16 = 60, \qquad j_4 = 16 \cdot 2 = 32.$$

A stage-1 token sees exactly its own 4×4 block: no overlap with its neighbours, because $r_1 = j_1$. A stage-4 token is built from a 60×60 window, and its neighbours are 32 pixels apart: neighbouring windows overlap by $60 - 32 = 28$ pixels. The jumps $4, 8, 16, 32$ are the "1/4 to 1/32 scale" of the stage names.

That $r_1 = j_1$ at stage 1 is a deliberate design choice, and the masking chapter (Chapter 18) depends on it: a stage-1 token is built only from its own 16 pixels, so hiding a token hides exactly those pixels and nothing leaks across token borders.

Source check: kernel 4, not 7

The patch-embedding module's docstring describes stage 1 as "kernel=7, stride=4 -> overlap = 7 - 4 = 3 pixels" (`app/foundation_models/components/overlap_patch_embedding.py`, line 10), as in the original SegFormer. The encoder builds stage 1 with `patch_size = 4 if i == 0 else 3` (`seg_former_encoder.py`, line 110) and `padding = 0` (line 114). The source wins: kernel 4, no overlap. With kernel 7 the receptive field would be $1+6 = 7$ pixels and a hidden token's pixels would leak into its visible neighbours.

17.5 One stage, in code

The encoder's `forward` is one loop that runs four times:

```
for i in range(self.num_stages):
    x, H, W = self.patch_embeds[i](x)        # -> (B, N, C_i)
    B, N, C = x.shape                        # remember the FULL token count

    gather_indices = None
    if i == 0 and keep_mask is not None:     # hide tokens, stage 1 only
        x, gather_indices = TokenMasking.remove_tokens(x, keep_mask)
                                             # (B, N_kept, C_1)

    for block in self.blocks[i]:             # 2 x SegFormerBlock, shape unchanged
        x = block(x, H, W)

    x = self.norms[i](x)                     # stage-output LayerNorm

    if i == 0 and gather_indices is not None:
        x = TokenMasking.restore_tokens(x, gather_indices, N, C)  # (B, N, C_1)

    B_cur, N_cur, C_cur = x.shape
    x = x.transpose(1, 2).reshape(B_cur, C_cur, H, W)  # (B, C_i, H_i, W_i)
    features.append(x)
```

Six steps: **embed, (maybe hide), blocks, normalise, (maybe restore), reshape**. The two "maybe" steps are the subject of Chapter 18.

The last line does double duty. The reshaped tensor is both this stage's feature map, saved in `features` for the decoder, and the next stage's input, because the loop variable `x` carries it into the next iteration. Stages 2–4 therefore never see the compressed picture: stage 2 reads F1, stage 3 reads F2, stage 4 reads F3.

The reshape at the end is the sequence-to-spatial conversion of Chapter 10: `transpose(1, 2)` swaps the token and channel axes, then `reshape` folds the N tokens back into $H \times W$ rows and columns. The transpose is necessary because the memory order differs between the layouts.

17.6 Hiding happens at stage 1 only

Masked training (Chapter 9) hides some tokens from the encoder so it must predict them. A natural first guess is that every stage should hide tokens. It does not, and the source says why:

> "Token removal only happens at Stage 1. After scattering back and reshaping to 2D, Stage 2's OPE (Conv2d stride=2) pools over 3x3 regions, so most Stage 2 tokens contain at least some real information even with 50% masking. Stages 2-4 process ALL tokens – the information loss from masking naturally dilutes through the spatial reduction." (`seg_former_encoder.py`, lines 22–26)

In plainer terms: **the holes are punched once, at the finest resolution, and then the pyramid blurs them out by itself.** Hiding again at stage 2 would starve the coarse stages, which are exactly where the model learns its "what usually goes here" knowledge.

"Most stage-2 tokens" is a claim you can check with numbers.

17.6.1 Which stage-1 tokens a stage-2 token reads

A stage-2 token at row h is computed by a 3×3 window, stride 2, padding 1, over the 32×32 grid of F1. Its window covers F1 rows $2h-1$, $2h$, $2h+1$. Row -1 is padding (a zero that is not a token).

- For $h = 0$: rows $-1, 0, 1$, so only rows 0 and 1 are real: 2 rows.
- For $h = 15$: rows $29, 30, 31$, all real: 3 rows. The padding at the bottom is never reached, because $2 \cdot 15 + 1 = 31$ is the last row.

The same holds for columns. So in the 16×16 grid of stage 2:

- 1 token (row 0, column 0) reads $2 \times 2 = 4$ stage-1 tokens;
- $15 + 15 = 30$ tokens (row 0 or column 0, not both) read $2 \times 3 = 6$;
- $15 \times 15 = 225$ tokens read $3 \times 3 = 9$.

Check: $1 + 30 + 225 = 256$. Every stage-2 token is accounted for.

17.6.2 The expected number of empty stage-2 tokens

Definition 17.2 (Expected value, linearity)**.** The *expected value* $\mathbb{E}[X]$ of a random count X is its long-run average over many repetitions. If $X = X_1 + X_2 + \cdots + X_n$ is a sum of counts, then $\mathbb{E}[X] = \mathbb{E}[X_1] + \cdots + \mathbb{E}[X_n]$, whether or not the X_k are independent (*linearity of expectation*). For an indicator X_k (1 if an event happens, 0 if not), $\mathbb{E}[X_k]$ is the probability of the event.

Call a stage-2 token *empty* if every stage-1 token in its window was hidden. Treat each stage-1 token as hidden with probability $\rho = 0.65$, independently (an approximation; the exact draw hides a fixed count, see below). A window of m tokens is empty with probability ρ^m:

$$\begin{aligned}
\rho^4 &= 0.65^4 = 0.65^2 \cdot 0.65^2 = 0.4225 \times 0.4225 \approx 0.1785, \\
\rho^6 &= 0.65^4 \cdot 0.65^2 \approx 0.1785 \times 0.4225 \approx 0.0754, \\
\rho^9 &= 0.65^6 \cdot 0.65^3 \approx 0.0754 \times 0.2746 \approx 0.0207.
\end{aligned}$$

By linearity, the expected number of empty stage-2 tokens is the sum of the 256 probabilities:

$$\begin{aligned}
\mathbb{E}[\text{empty}] &= 1 \times 0.1785 + 30 \times 0.0754 + 225 \times 0.0207 \\
&\approx 0.1785 + 2.262 + 4.66 \\
&\approx 7.1.
\end{aligned}$$

About 7 of the 256 stage-2 tokens [C], under 3%, have no visible data in their window. The other 97% see at least one survivor. The exact draw (665 of 1024 hidden, without replacement) gives 6.94; the script `research/textbook/tools/compute_masking.py` computes both [C]. By stage 3, whose tokens see 3×3 windows of stage-2 tokens, an empty token would need nine empty stage-2 tokens in a row and column block, which essentially never happens.

Masking at stage 1 at 65% leaves only about 7 of 256 stage-2 tokens without any visible input; the pyramid fills the holes for free.

17.7 The complete masked walk-through

Take two fully valid patches ($B = 2$) of 128×128 and the training ratio $\rho = 0.65$. The ratio comes from the training configuration [M: `configs/hyperspectral_segformer_exp_2.json`]. How many tokens survive? The hiding count is $\lfloor 1024 \times 0.65 \rfloor = \lfloor 665.6 \rfloor = 665$ (Chapter 18 explains the rounding down), so $1024 - 665 = 359$ tokens are kept.

Table 17.3: One masked pass through the encoder, $B = 2$, fully valid patches, $\rho = 0.65$. Shapes were confirmed by running the real encoder on CPU (`research/textbook/tools/compute_params.py`).

Stage	Step	Shape after the step	Note
1	patch embed $k4\,s4\,p0$	`(2, 1024, 32)`	all tokens
	remove tokens	`(2, 359, 32)`	665 hidden tokens deleted
	2 blocks	`(2, 359, 32)`	full-attention fallback
	LayerNorm	`(2, 359, 32)`	
	restore tokens	`(2, 1024, 32)`	zeros at hidden slots
	reshape	`(2, 32, 32, 32)` = F1	a grid with holes
2	patch embed $k3\,s2\,p1$	`(2, 256, 64)`	pools over the holes
	2 blocks, LayerNorm	`(2, 256, 64)`	$R = 4$
	reshape	`(2, 64, 16, 16)` = F2	no holes
3	patch embed, blocks, norm	`(2, 64, 160)`	$R = 2$
	reshape	`(2, 160, 8, 8)` = F3	
4	patch embed, blocks, norm	`(2, 16, 256)`	$R = 1$, true full attention
	reshape	`(2, 256, 4, 4)` = F4	

Source check: 359, not 358

The course writes "~358" kept tokens at $\rho = 0.65$, which is 35% of 1024 (358.4). The code rounds the *hidden* count down, `(num_valid * mask_ratio).long()` (`app/foundation_models/components/token_masking.py`, line 186), so a fully valid patch hides 665 and keeps 359. Running the encoder confirms 359 per item (`research/textbook/tools/compute_params_output.txt`).

17.8 Three things that look like bugs and are not

17.8.1 H and W keep describing the full grid

After removal there are 359 tokens, but the loop still passes $H = W = 32$ to the blocks. That is correct and necessary. The restore step needs the full size to rebuild the grid, and the final

reshape needs it too. Meanwhile the mismatch $N \neq H \times W$ is *how* ESA and Mix-FFN detect that masking is active and take their fallback paths.

17.8.2 Hidden positions become exactly zero

The masked-autoencoder paper [11] inserts a learned *mask token* at hidden positions: one trained vector that means "something was here". This encoder writes zeros (`restore_tokens` starts from `torch.zeros`, `token_masking.py`, line 402). It works because the decoder's job here is spatial fusion of four maps rather than sequence modelling: it fills the gaps from the surviving stage-1 tokens around them and from the fully populated F2–F4.

17.8.3 F1 keeps its holes all the way to the decoder

Nothing fills them at stage 1. F1 arrives at the decoder with zero vectors at every hidden token. That is fine: the decoder receives four maps covering the same ground and only one of them has holes.

17.9 What the encoder returns

```
return features        # [F1, F2, F3, F4]
# F1: (B, 32, 32, 32)   F2: (B, 64, 16, 16)   F3: (B, 160, 8, 8)   F4: (B, 256, 4, 4)
```

A Python list of four tensors at four resolutions and four widths: the complete interface between encoder and decoder. Any patch side that divides by 32 works, because the side is divided by 4 once and by 2 three times ($4 \times 2 \times 2 \times 2 = 32$); for a 256×256 patch the grids are $64, 32, 16, 8$.

17.10 Common confusions

Do stages 2–4 see the original image?
No. Stage 2 reads F1, stage 3 reads F2, stage 4 reads F3. Only stage 1 touches the compressed picture.

Is the encoder a bottleneck like an ordinary autoencoder's?
A hierarchical one. There is no single narrow layer; there are four maps, and the coarsest (16 tokens of 256 numbers, 4,096 values) is very compressed. All four go to the decoder, so the fine map is not squeezed through the coarse one.

Why does the width go up as the grid goes down?
To shift capacity from "where" to "what" while each map still shrinks overall (32,768 to 4,096 numbers). It is the standard convolutional backbone pattern, inherited by SegFormer.

Stage 1 has R = 8, so its attention is always reduced?
Only without masking. In a masked pass the token count is not $H \times W$, ESA cannot rebuild the grid, and stage 1 runs full attention among the kept tokens (Section 17.3).

Does every stage-2 token mix hidden and visible tokens?
Most do. About 7 of 256 are expected to see only hidden tokens at $\rho = 0.65$; they carry only bias, LayerNorm and padding values, and the decoder must rely on F3 and F4 there.

Could I use a patch size other than 128?
Yes, if it divides by 32. The token count, the key count and the run time change; the parameter count does not (Chapter 20).

17.11 Check yourself

Exercise 17.1. Use Equation (17.1) to find the four grid sizes for a 256×256 patch, and the token count of each stage.

Exercise 17.2. For a 128×128 patch, how many keys does each stage's attention use in an unmasked pass, and how many score entries per head does stage 2 compute?

Exercise 17.3. Using Equation (17.2), find the receptive field and jump of a stage-2 token if stage 1 were built with kernel 7, stride 4 (the original SegFormer). Why would that break masked training?

Exercise 17.4. A batch is masked at $\rho = 0.5$ instead of 0.65. Using the independence approximation, compute the expected number of empty stage-2 tokens for a 128×128 patch.

Exercise 17.5. Name the six steps of one stage, and say which two only happen at stage 1 and why the reshape at the end is used twice.

Next: Chapter 18, Masked autoencoding: validity, targets, remove-and-restore, erosion.

Chapter 18

Masked autoencoding: validity, targets, remove-and-restore, erosion

The one thing this chapter teaches

Masked training is four small index computations done in order: decide which tokens hold real data (average-pool the pixel mask, keep a token when more than half its pixels are valid), hide a fixed fraction of the valid ones by sorting random numbers, physically delete the hidden tokens with a gather and put them back as zeros with a scatter, and grade the model only on hidden, valid pixels that are not too close to invalid ones.

Chapter 9 established the rule that makes reconstruction useful for anomaly detection: a pixel must never help reconstruct itself. The encoder (Chapter 17) showed *where* hiding happens: at stage 1, between the patch embedding and the blocks. This chapter is the machinery behind that one line of the encoder, and the masks that the trainer builds around it. All of it lives in one file, `app/foundation_models/components/token_masking.py`, a class of static methods with no learnable parameters, plus a few lines of the trainer, `app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`.

18.1 The problem, and why the obvious fixes fail

You want the model to learn "what normally goes here" by predicting parts of a patch it cannot see. Three obvious approaches each fail in a specific way.

1. **Hide pixels by setting them to zero.** A zeroed pixel still enters the patch embedding and still becomes part of a token that attention can read. Worse, the patch already contains genuine zeros: invalid pixels (cloud, no-data, swath edge) were zeroed before normalisation (Chapter 5). The model could not tell "hidden" from "missing".
2. **Hide random pixels, not tokens.** A token is built from a 4×4 block. If you hide half of a block's pixels, the token still contains the other half, and the model can interpolate from inside the very token it is graded on.
3. **Grade the model everywhere.** Then it is graded on invalid pixels, where the "truth" is a fill value, and on pixels it could see, where copying is a perfect answer.

The design that survives is: hide whole tokens, choose them only among tokens that hold real data, delete them from the encoder's input so nothing can attend to them, and grade only the pixels that were both hidden and valid.

Table 18.1: The four steps of masking and the masks each produces. B is the batch size, $N = 1024$ tokens for a 128×128 patch.

Step	Question	Output	Section
1	Which tokens hold real data?	token mask v, `(B, N)`	18.2
2	Of those, which are hidden?	keep mask k, prediction mask q, `(B, N)`	18.3
3	How are they deleted and put back?	gather indices `(B, N_kept)`	18.4
4	Which pixels are graded?	loss mask `(B, 1, H, W)`	18.5, 18.6

Analogy

Think of a cloze test, the reading exercise in which words are blanked out of a paragraph and you must fill them in. A good examiner blanks whole words, not half a word (a token, not a pixel). She never blanks a word that was already illegible on the photocopy (an invalid token), because there is no right answer to mark. She removes the blanked words from the sheet entirely rather than writing them in faint ink (deletion, not zeroing). And she does not mark a filled-in word when the smudge from the illegible word next to it covers half of it (erosion). Each of those rules is one section of this chapter.

18.2 Step 1: which tokens hold real data

18.2.1 The mismatch

Validity arrives per *pixel*: a tensor m of shape `(B, 1, H, W)` with $m_{y,x} = 1$ where the pixel is a real measurement and 0 where it is not. Masking works per *token*: one decision for each of the N tokens. One stage-1 token covers a 4×4 block of 16 pixels (Chapter 17, $r_1 = j_1 = 4$). When a block is partly valid, somebody has to make a ruling.

18.2.2 The rule

Definition 18.1 (Token validity)**.** Let P_t be the set of 16 pixels of token t. The *valid fraction* of t is

$$f_t = \frac{1}{|P_t|} \sum_{(y,x) \in P_t} m_{y,x} = \frac{1}{16} \sum_{(y,x) \in P_t} m_{y,x}. \tag{18.1}$$

Token t is *valid* when $f_t > \frac{1}{2}$, strictly:

$$v_t = \begin{cases} 1 & \text{if } f_t > 0.5, \\ 0 & \text{otherwise.} \end{cases} \tag{18.2}$$

Why is Equation (18.1) a fraction? Because m contains only 0s and 1s, the sum counts the valid pixels, and dividing by 16 turns the count into a proportion. Sixteen valid pixels give $16/16 = 1$; eight give $8/16 = 0.5$; none gives 0.

The code computes all f_t at once with an *average pool*, an operation that slides a window over a grid and outputs the mean of each window (`token_masking.py`, lines 112–127):

```
token_fractions = F.avg_pool2d(pixel_mask.float(),     # (B, 1, 128, 128)
                               kernel_size=4, stride=4, padding=0)   # -> (B, 1, 32,
    32)
token_valid = (token_fractions > 0.5).float()          # strict: 0.5 itself is
    invalid
token_mask  = token_valid.squeeze(1).flatten(1)        # (B, 32, 32) -> (B, 1024)
```

`squeeze(1)` removes the size-1 channel axis; `flatten(1)` joins the two grid axes into one list of 1024 in *row-major order* (row 0 left to right, then row 1, and so on), the same order the tokens themselves use (Chapter 10).

Worked example 18.1: Token validity on a 4 × 4 pixel grid with 2 × 2 tokens

To keep the arithmetic small, shrink the token to 2 × 2 pixels (pooling kernel 2, stride 2; the real model uses 4 and 4). The pixel mask is

$$m = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}.$$

The 2 × 2 token grid has four tokens, numbered row-major: t_0 top-left, t_1 top-right, t_2 bottom-left, t_3 bottom-right. Do each on paper.
Token t_0, rows 0–1, columns 0–1: pixels $1, 1, 1, 1$.

$$f_0 = \tfrac{1}{4}(1+1+1+1) = \tfrac{4}{4} = 1.00, \qquad 1.00 > 0.5 \;\Rightarrow\; v_0 = 1.$$

Token t_1, rows 0–1, columns 2–3: pixels $1, 0, 0, 0$.

$$f_1 = \tfrac{1}{4}(1+0+0+0) = \tfrac{1}{4} = 0.25, \qquad 0.25 \not> 0.5 \;\Rightarrow\; v_1 = 0.$$

Token t_2, rows 2–3, columns 0–1: pixels $1, 0, 1, 0$.

$$f_2 = \tfrac{1}{4}(1+0+1+0) = \tfrac{2}{4} = 0.50, \qquad 0.50 \not> 0.5 \;\Rightarrow\; v_2 = 0.$$

Token t_3, rows 2–3, columns 2–3: pixels $1, 1, 1, 1$.

$$f_3 = \tfrac{4}{4} = 1.00 \;\Rightarrow\; v_3 = 1.$$

Flattened row-major: $v = [1, 0, 0, 1]$. Token t_2 is exactly half valid and is rejected: **ties go to "invalid"**. The same computation with the real 4 × 4 tokens on the course's 8 × 8 grid is in Figure 18.1; it ends in $[1, 0, 1, 0]$, again with one token rejected at exactly 8/16.

The tie rule is the conservative choice: a token built half from no-data is not something you want to grade the model on.

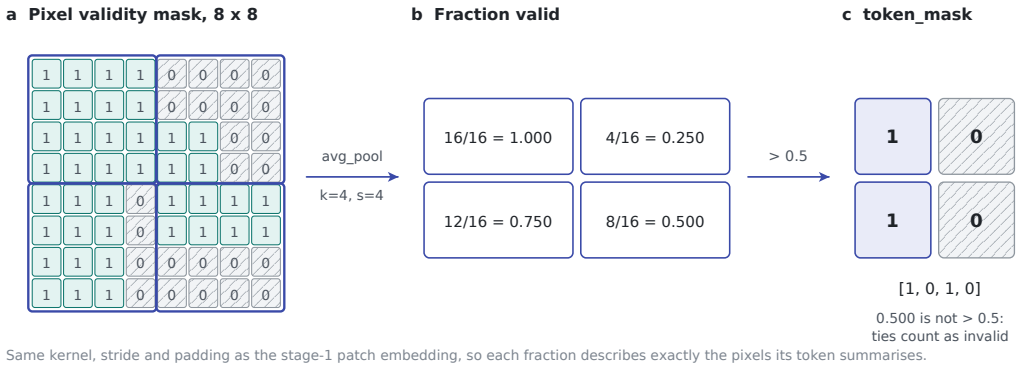


Figure 18.1: Token validity on an 8 × 8 pixel mask with the real stage-1 geometry (kernel 4, stride 4). (a) The pixel mask split into four 4 × 4 blocks. (b) Average pooling gives each block's valid fraction: 16/16, 4/16, 12/16, 8/16. (c) The strict threshold > 0.5 gives the token mask $[1, 0, 1, 0]$; the exactly-half block is invalid.

18.2.3 Exactly the same geometry as the patch embedding

The pooling uses kernel 4, stride 4, padding 0: the stage-1 patch embedding's numbers (Chapter 13). That is a requirement, not a coincidence. With a different window, a token's *validity* would be judged from a different set of pixels than the token's *value* was computed from, and you could grade the model on a token built entirely from no-data. The trainer and the inferencer both declare the constants at the top of their files (`STAGE1_KERNEL_SIZE=4`, `STAGE1_STRIDE=4`, `STAGE1_PADDING=0`; trainer lines 36–38, inferencer `app/foundation_models/inferencers/segformer_mae_inferencer.py` lines 41–43) and pass them at every call site.

Source check: padding defaults

`pixel_mask_to_token_mask` defaults `padding=None` to `kernel_size//2` (`token_masking.py`, lines 107–108), which for kernel 4 would be 2 and would *not* match stage 1. Every caller passes `padding=0` explicitly, so the default is never used for stage 1. If you add a caller, pass the constant.

18.2.4 What happens to invalid tokens

This is the part people get wrong. An invalid token ($v_t = 0$) is

- **not** deleted from the encoder's input,
- **not** eligible to be hidden,
- **not** graded.

It stays in the sequence carrying its zero-filled, normalised values (an invalid pixel sits near -1.22 after normalisation, Chapter 5), so the encoder is told where the holes in the data are rather than being asked to invent them.

The token mask v says "is this real data?"; it is a fact about the sensor. The keep mask of the next section says "shall the encoder see it?"; it is a choice. An invalid token has $v = 0$ and keep $= 1$.

18.3 Step 2: choosing which valid tokens to hide

18.3.1 How many: the hiding count and its rounding

Let $n_{\text{valid}} = \sum_t v_t$ be the number of valid tokens and ρ the *mask ratio*, the fraction of valid tokens to hide. The number hidden is

$$n_{\text{hide}} = \lfloor \rho \, n_{\text{valid}} \rfloor, \tag{18.3}$$

because the code converts with `(num_valid*mask_ratio).long()` (`token_masking.py`, line 186), and converting a positive float to an integer in PyTorch truncates toward zero, which for positive numbers is the floor. The ratio is a fraction of the *valid* tokens, not of all tokens.

The count is deterministic: for a given patch the number hidden is fixed; only *which* tokens are hidden is random. Because every valid token is equally likely to be any of the hidden ones (shown below), each valid token is hidden with probability

$$\Pr[\text{token } t \text{ hidden}] = \frac{n_{\text{hide}}}{n_{\text{valid}}} = \frac{\lfloor \rho \, n_{\text{valid}} \rfloor}{n_{\text{valid}}} \leq \rho, \tag{18.4}$$

the *effective* mask ratio. The floor can only lower it.

Worked example 18.2: Six tokens at $\rho = 0.65$: what the rounding does

Six valid tokens, $\rho = 0.65$ (the training value). On paper:

$$\begin{aligned} \rho\, n_{\text{valid}} &= 0.65 \times 6 && \text{multiply} \\ &= 3.9, \\ n_{\text{hide}} &= \lfloor 3.9 \rfloor = 3 && \text{truncate, do not round to 4} \\ n_{\text{keep}} &= 6 - 3 = 3. \end{aligned}$$

The effective ratio is $3/6 = 0.50$, well below 0.65. Rounding to nearest would have given 4 and an effective $4/6 \approx 0.67$. For the real patch the effect is tiny:

$$0.65 \times 1024 = 665.6, \qquad \lfloor 665.6 \rfloor = 665, \qquad \frac{665}{1024} \approx 0.6494,$$

so a fully valid patch hides 665 tokens and keeps $1024 - 665 = 359$. The lesson: on tiny examples the floor dominates; on real patches it costs a fraction of a token.

Two edge cases follow from Equation (18.3). A patch with one valid token hides $\lfloor 0.65 \rfloor = 0$ tokens, and so does a patch with none: the loss mask is then empty and the trainer returns a zero loss with zero counted pixels, which the training loop skips (`hyperspectral_segformer_mae_trainer.py`, lines 249–250). In practice the trainer has already dropped every patch with less than 40% valid pixels (`MIN_VALID_PIXEL_FRACTION=0.4`, line 34).

Values that will drift

$\rho = 0.65$ in v0.2.0 (`configs/hyperspectral_segformer_exp_2.json`) and in every spectral-stem run up to FT04 [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]; v0.1.0 used 0.50 (`configs/hyperspectral_segformer_exp_1.json`). The config class default is 0.5 (`app/models/training/training_config.py`, line 97), which no shipped run uses. As of 2026-10-29.

18.3.2 Which ones: a random permutation by sorting noise

The obvious way to choose is a Python loop: for each patch, list the valid tokens, shuffle, take the first n_{hide}. On a GPU processing 128 patches at once, with a different list length per patch, that loop would stall the device on every batch. So the choice is written with array operations only, identical work for every patch. That is why it looks indirect.

Definition 18.2 (Permutation, argsort)**.** A *permutation* of $\{0, \ldots, N-1\}$ is a reordering: a list π in which each index appears exactly once. The *argsort* of a list of numbers a is the permutation that sorts it: $\pi = \text{argsort}(a)$ means $a_{\pi_0} \le a_{\pi_1} \le \cdots \le a_{\pi_{N-1}}$. Read π_k as "the index of the k-th smallest value".

The recipe (`token_masking.py`, lines 169–223):

```
noise = torch.rand(B, N)                          # uniform in [0, 1), one per token
noise = noise + (1 - token_mask) * 2              # invalid tokens move to [2, 3)
num_to_mask = (token_mask.sum(1) * mask_ratio).long()          # (B,)
sorted_indices = noise.argsort(dim=1)             # (B, N): smallest noise first
positions = torch.arange(N).float().expand(B, -1)              # 0, 1, ..., N-1
pred_mask_sorted = (positions < num_to_mask.unsqueeze(1)).float()   # first n_hide
    places
pred_mask = torch.zeros(B, N)
pred_mask.scatter_(1, sorted_indices, pred_mask_sorted)        # back to token order
```

```
keep_mask = 1.0 - pred_mask
```

In words: give every token a random priority; push every invalid token's priority up by 2; sort by priority; flag the first n_{hide} places in the queue; carry the flags back to the tokens they belong to.

Why every hidden set is equally likely. The noise values are independent draws from the same continuous distribution. Any particular ordering of N such values is exactly as likely as any other (relabelling the tokens does not change the joint distribution), and ties have probability zero. So argsort(noise) is a *uniformly random permutation*: each of the $N!$ orders has probability $1/N!$. Taking its first n_{hide} entries therefore picks a uniformly random subset of size n_{hide}, which is what Equation (18.4) assumed.

Why the +2 protects invalid tokens. Valid tokens keep noise in $[0, 1)$; invalid tokens get noise in $[2, 3)$. Every valid value is below every invalid one, so all valid tokens sort ahead of all invalid tokens. Since $n_{\text{hide}} \le n_{\text{valid}}$, the first n_{hide} places are always valid tokens. Any shift of at least 1 would work; 2 is a clear margin. The valid tokens among themselves are still in uniformly random order, because adding the same constant to every invalid token does not touch their relative order.

Worked example 18.3: Hiding three of six tokens by hand

Six tokens, all valid, $\rho = 0.65$, so $n_{\text{hide}} = 3$ (worked example 18.2). Suppose the noise is

$$\text{noise} = [0.42,\ 0.91,\ 0.07,\ 0.66,\ 0.18,\ 0.55] \quad (\text{tokens } t_0, \ldots, t_5).$$

No token is invalid, so the +2 adds nothing. Sort the values ascending and write under each value the token it came from:

sorted value	0.07	0.18	0.42	0.55	0.66	0.91
came from token	2	4	0	5	3	1

So $\pi = \text{argsort}(\text{noise}) = [2, 4, 0, 5, 3, 1]$. Flag the first three places:

$$\text{pred_mask_sorted} = [\,0 < 3,\ 1 < 3,\ 2 < 3,\ 3 < 3,\ 4 < 3,\ 5 < 3\,] = [1, 1, 1, 0, 0, 0].$$

Carry the flags back. `scatter_` writes `pred_mask[pi[k]]=pred_mask_sorted[k]`:

$$\begin{aligned}
k = 0: &\quad \pi_0 = 2, && \text{pred_mask}[2] = 1 \\
k = 1: &\quad \pi_1 = 4, && \text{pred_mask}[4] = 1 \\
k = 2: &\quad \pi_2 = 0, && \text{pred_mask}[0] = 1 \\
k = 3: &\quad \pi_3 = 5, && \text{pred_mask}[5] = 0 \\
k = 4: &\quad \pi_4 = 3, && \text{pred_mask}[3] = 0 \\
k = 5: &\quad \pi_5 = 1, && \text{pred_mask}[1] = 0
\end{aligned}$$

Result: $q = \text{pred_mask} = [1, 0, 1, 0, 1, 0]$ and $k = \text{keep_mask} = 1 - q = [0, 1, 0, 1, 0, 1]$. Tokens 0, 2 and 4, the three with the smallest noise, are hidden.

18.3.3 The inverse permutation

The original masked autoencoder [11] writes the same idea with two index lists: `ids_shuffle`, the argsort of the noise (our π), and `ids_restore`, its *inverse*. This code base never builds

`ids_restore`; the `scatter_` does its job. Seeing why is worth the effort, because it is the same fact that makes remove-and-restore work in Section 18.4.

Definition 18.3 (Inverse permutation)**.** The inverse of a permutation π is the permutation π^{-1} with $\pi^{-1}_{\pi_k} = k$ for every k. If π answers "which token sits in place k?", then π^{-1} answers "in which place does token t sit?". That is the *rank* of token t.

Worked example 18.4: Building ids_restore for six tokens

Take $\pi = [2, 4, 0, 5, 3, 1]$ from worked example 18.3. Apply the definition one place at a time: place k holds token π_k, so token π_k has rank k.

$$\begin{aligned} &\pi_0 = 2 \;\Rightarrow\; \pi_2^{-1} = 0, && \pi_1 = 4 \;\Rightarrow\; \pi_4^{-1} = 1, && \pi_2 = 0 \;\Rightarrow\; \pi_0^{-1} = 2, \\ &\pi_3 = 5 \;\Rightarrow\; \pi_5^{-1} = 3, && \pi_4 = 3 \;\Rightarrow\; \pi_3^{-1} = 4, && \pi_5 = 1 \;\Rightarrow\; \pi_1^{-1} = 5. \end{aligned}$$

Read off in token order: $\pi^{-1} = [2, 5, 0, 4, 1, 3]$. Check: token 2 had the smallest noise (0.07) and has rank 0; token 1 had the largest (0.91) and has rank 5.
There is a second way to get it: $\pi^{-1} = \text{argsort}(\pi)$. Sort $\pi = [2, 4, 0, 5, 3, 1]$ ascending: the value 0 is at place 2, value 1 at place 5, value 2 at place 0, value 3 at place 4, value 4 at place 1, value 5 at place 3. So $\text{argsort}(\pi) = [2, 5, 0, 4, 1, 3]$, the same list. (The MAE code computes `ids_restore=argsort(ids_shuffle)` for this reason.)
Now unshuffle the flags with a *gather* instead of a scatter:

$$\begin{aligned} q_t &= \text{pred_mask_sorted}[\pi_t^{-1}] && \text{look up token } t\text{'s place} \\ q &= [\,s_2,\ s_5,\ s_0,\ s_4,\ s_1,\ s_3\,] && \text{with } s = [1, 1, 1, 0, 0, 0] \\ &= [\,1,\ 0,\ 1,\ 0,\ 1,\ 0\,]. \end{aligned}$$

The same q as the scatter produced. Scattering through π and gathering through π^{-1} are the same operation.

Why are they the same? A scatter through π writes $q_{\pi_k} = s_k$ for every place k. Substitute $t = \pi_k$, so $k = \pi_t^{-1}$: then $q_t = s_{\pi_t^{-1}}$ for every token t, which is the gather through π^{-1}. One line of algebra, and it is the reason this code needs no `ids_restore`.

Worked example 18.5: The course example: two invalid tokens and the +2

This is the example in the source's own docstring (`token_masking.py`, lines 147–163). Six tokens with $v = [1, 1, 0, 1, 1, 0]$ (tokens 2 and 5 invalid) and $\rho = 0.5$.
Noise: $[0.30, 0.70, 0.10, 0.50, 0.20, 0.80]$. Token 2, which is invalid, drew the smallest value; left alone it would be hidden first.
Add 2 where $v = 0$:

$$\begin{aligned} &[0.30,\ 0.70,\ 0.10,\ 0.50,\ 0.20,\ 0.80] \\ +\,&[0,\ 0,\ 2,\ 0,\ 0,\ 2] \\ =\,&[0.30,\ 0.70,\ 2.10,\ 0.50,\ 0.20,\ 2.80]. \end{aligned}$$

Count: $n_{\text{valid}} = 1+1+0+1+1+0 = 4$, $n_{\text{hide}} = \lfloor 4 \times 0.5 \rfloor = 2$. Sort: $0.20(t_4) < 0.30(t_0) < 0.50(t_3) < 0.70(t_1) < 2.10(t_2) < 2.80(t_5)$, so $\pi = [4, 0, 3, 1, 2, 5]$. Flags $[1, 1, 0, 0, 0, 0]$ scatter to $q[4] = 1$, $q[0] = 1$, all others 0:

$$q = [1, 0, 0, 0, 1, 0], \qquad k = [0, 1, 1, 1, 0, 1].$$

Two of the four valid tokens are hidden; neither invalid token was touched; both invalid tokens have $k = 1$ and stay in the sequence.



Figure 18.2: The worked example of worked example 18.5 as one picture. Invalid tokens (hatched) get $+2$ so they sort to the back. After an ascending sort the first $\lfloor 4 \times 0.5 \rfloor = 2$ places (amber) are flagged and scattered back: tokens 0 and 4 are hidden, and the keep mask is the exact complement of the prediction mask.

There is no third state: every token is either hidden-and-graded ($q = 1$, $k = 0$) or shown-and-not-graded ($q = 0$, $k = 1$), and $k = 1 - q$ always.

18.3.4 Choosing the ratio

The ratio sets the difficulty of the exam. At $\rho = 0.1$ nine tokens in ten are visible and filling the tenth is interpolation; the model learns smoothing and, at inference, reconstructs anomalies well from their surroundings, so they stop standing out. At $\rho = 0.9$ there is too little evidence, the best strategy is a blurry average, and every pixel has a large residual. The masked-autoencoder literature on photographs settled near 0.75 [11]; this project uses 0.65.

Source check: inference does not use this function

At inference every pixel needs a score, so random hiding is replaced by an exact complementary split (a checkerboard, or a random half and its complement; Chapter 30). The invalid-token guarantee is kept by multiplication instead of the $+2$: `pred_mask=token_validity*(1.0-checker)` (`segformer_mae_inferencer.py`). Only valid tokens can become targets either way.

18.4 Step 3: gather and scatter, remove and restore

18.4.1 The two index operations

Definition 18.4 (Gather and scatter)**.** Given data x of length N and a list of indices $g = (g_0, \ldots, g_{M-1})$:

- a *gather* reads from the named positions: $y_k = x_{g_k}$ for $k = 0, \ldots, M-1$. The output is as long as the index list.
- a *scatter* writes into the named positions: starting from a given z (here all zeros), set $z_{g_k} = y_k$. Positions not named keep their starting value.

For tokens, each x_t is a vector of C numbers and the whole vector moves.

A tiny example. With $x = [a, b, c, d, e]$ and $g = [3, 0, 4]$, the gather gives $[d, a, e]$. Scattering $[d, a, e]$ through the same g into $[0, 0, 0, 0, 0]$ writes d at 3, a at 0, e at 4: $[a, 0, 0, d, e]$. Gather

then scatter with the same indices puts every gathered value back where it came from and leaves zeros elsewhere.

The same thing as matrices. Build the *selection matrix* $\mathbf{S}$ of size $M \times N$ whose row k is all zeros except a 1 in column g_k. Then

$$\text{gather:} \quad \mathbf{y} = \mathbf{S}\mathbf{x}, \qquad\qquad \text{scatter into zeros:} \quad \mathbf{z} = \mathbf{S}^\top \mathbf{y}.$$

Check the first on the tiny example: row 0 of $\mathbf{S}$ has its 1 in column 3, so $y_0 = 0 \cdot a + 0 \cdot b + 0 \cdot c + 1 \cdot d + 0 \cdot e = d$. Two products tell the whole story:

- $\mathbf{S}\mathbf{S}^\top = \mathbf{I}_M$: entry (k, l) is the dot product of rows k and l of $\mathbf{S}$, which is 1 when they point at the same column ($k = l$, since the g_k are distinct) and 0 otherwise. Scatter then gather returns exactly what you scattered.
- $\mathbf{S}^\top\mathbf{S}$ = diag(keep): an $N \times N$ diagonal matrix with 1 at the gathered positions and 0 elsewhere. Gather then scatter keeps the gathered tokens and zeroes the rest.

The second line *is* remove-then-restore, with the encoder's blocks in between.

Worked example 18.6: The selection matrix for three kept tokens of six

From worked example 18.3, $k = [0, 1, 0, 1, 0, 1]$: tokens 1, 3, 5 are kept. With $g = [1, 3, 5]$,

$$\mathbf{S} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \end{pmatrix}.$$

Multiply $\mathbf{S}^\top\mathbf{S}$ entry by entry: entry (i, j) sums $S_{ki}S_{kj}$ over the three rows k. It is nonzero only when column i and column j both hold a 1 in the same row, which happens only for $i = j \in \{1, 3, 5\}$. So $\mathbf{S}^\top\mathbf{S} = \text{diag}(0, 1, 0, 1, 0, 1) = \text{diag}(k)$.

18.4.2 remove_tokens, line by line

```
num_kept = keep_mask.sum(dim=1).long()                          # (B,)
max_kept = num_kept.max().item()                                # one number for the
    batch
sorted_indices = keep_mask.argsort(dim=1, descending=True)      # (B, N): 1s first
gather_indices = sorted_indices[:, :max_kept]                   # (B, max_kept)
idx = gather_indices.unsqueeze(-1).expand(-1, -1, C)            # (B, max_kept, C)
kept_tokens = torch.gather(tokens, dim=1, index=idx)            # (B, max_kept, C)
```

Sorting the keep mask descending puts every 1 before every 0, so the first `max_kept` indices are the kept positions. `torch.gather` needs an index tensor of the output's shape, so each index is repeated C times by `expand` (a view, no copy): for $g = [1, 3, 5]$ and $C = 2$ the expanded index is $[[1, 1], [3, 3], [5, 5]]$, which pulls whole token vectors.

Worked example 18.7: Remove and restore on six tokens

Continue with $k = [0, 1, 0, 1, 0, 1]$, tokens $t_0, \ldots, t_5$, each of $C = 2$ numbers.

$$\begin{aligned} \text{num_kept} &= 0 + 1 + 0 + 1 + 0 + 1 = 3, \qquad \text{max_kept} = 3, \\ \text{argsort descending of } k &= [1, 3, 5, 0, 2, 4] \quad \text{(the three 1s, then the three 0s)}, \\ g &= [1, 3, 5], \\ \text{kept} &= [t_1, t_3, t_5], \quad \text{shape } (1, 3, 2). \end{aligned}$$

The blocks turn these into t'_1, t'_3, t'_5. Restore starts from zeros of shape $(1, 6, 2)$ and scatters through g:

$$\text{position } 1 \leftarrow t'_1, \qquad \text{position } 3 \leftarrow t'_3, \qquad \text{position } 5 \leftarrow t'_5,$$

giving $[\,\mathbf{0},\ t'_1,\ \mathbf{0},\ t'_3,\ \mathbf{0},\ t'_5\,]$. The hidden positions 0, 2, 4 are exactly zero vectors.

Source check: the order of the kept tokens is not guaranteed, and does not matter

The course says the descending sort keeps ties "in index order". `torch.argsort` is not stable by default (`stable=False`), so among equal keys the order is implementation-defined. On this machine (CPU, PyTorch 2.5.1) the small cases happen to come out in index order, $[1, 3, 5, 0, 2, 4]$ for the keep mask above (`research/textbook/tools/compute_masking_output.txt`), but on a GPU and for $N = 1024$ nothing promises it. It does not matter, for two reasons. First, restore scatters through the *same* g, so every token returns to its own slot whatever order it travelled in. Second, nothing in the stage-1 blocks depends on order when tokens have been removed: the linear layers and LayerNorms act on each token separately, attention takes its full-attention fallback, which treats the keys as a set, and Mix-FFN skips its depthwise convolution (Chapter 17). There is no positional encoding to disturb. In the language of Chapter 14, a masked stage 1 is *permutation-equivariant*: shuffling the input tokens shuffles the outputs the same way.

18.4.3 The documented wrinkle: max_kept

`max_kept` is the largest kept count in the batch. If patch A keeps 400 tokens and patch B keeps 380, both gather 400 indices, and B's last 20 come from the tail of its sort, which is its *hidden* tokens. Those 20 slip back into the encoder. The source says so: "shorter items get a few extra tokens from the sort tail (minor leakage)" (`token_masking.py`, lines 301–304).

Worked example 18.8: Leakage in a batch of two

Patch A is fully valid: $n_{\text{valid}} = 1024$, $n_{\text{hide}} = 665$, kept 359. Patch B has $n_{\text{valid}} = 1000$ and 24 invalid tokens:

$$n_{\text{hide}}^{B} = \lfloor 0.65 \times 1000 \rfloor = \lfloor 650 \rfloor = 650,$$
$$\text{kept}^{B} = 1024 - 650 = 374 \quad \text{(350 visible valid plus 24 invalid).}$$

Then max_kept $= \max(359, 374) = 374$, and patch A gathers $374 - 359 = 15$ indices from the tail of its sort: 15 of its 665 hidden tokens, about $15/665 \approx 2.3\%$, are seen by the encoder and still graded. Notice the direction: the patch with *fewer* invalid tokens keeps fewer tokens (its hiding count is larger) and is the one that leaks.

The source calls it tolerable because counts differ only when validity differs, and the 40%-valid filter keeps batches fairly homogeneous. It is a real, small leak: on mixed batches with very different cloud cover the fully valid patches leak most.

18.4.4 restore_tokens, and why zeros

```
full_tokens = torch.zeros(B, N, C)                                  # (B, 1024, 32)
scatter_indices = gather_indices.unsqueeze(-1).expand(-1, -1, C)    # (B, N_kept, C)
```

```
full_tokens.scatter_(1, scatter_indices, kept_tokens)              # kept slots
    written
```

Positions not written stay zero (`token_masking.py`, lines 402–417). The original masked autoencoder fills them with a learned *mask token* [11]; this model does not, and the encoder chapter gives the reason (the decoder fuses four maps and reads the gaps from F2–F4). The round trip for one patch at $\rho = 0.65$:

$$\texttt{(B, 1024, 32)} \xrightarrow{\text{remove}} \texttt{(B, 359, 32)} \xrightarrow{\text{2 blocks, LN}} \texttt{(B, 359, 32)}$$
$$\xrightarrow{\text{restore}} \texttt{(B, 1024, 32)} \xrightarrow{\text{reshape}} \texttt{(B, 32, 32, 32)}.$$

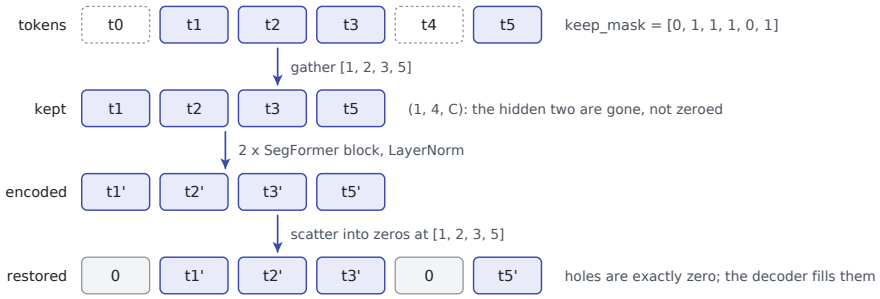


Figure 18.3: Hidden tokens are removed, not zeroed. The gather keeps only the visible tokens (here 4 of 6), so the stage-1 blocks never see the hidden ones; after the blocks the scatter writes the encoded tokens back into a zero tensor at their original positions.

Why delete rather than zero? Two reasons. *Zero leakage*: a zeroed token is still a key; after the pre-norm LayerNorm and the projections' biases it is a perfectly ordinary key and value, and it takes a share of every softmax. In the unmasked path it would also enter the depthwise convolution and the reduction convolution of its neighbours. A deleted token cannot influence anything. *Cost*: the linear layers of stage 1 run on 359 tokens instead of 1024. (The attention scores do not get cheaper, because a masked stage 1 falls back to full attention: Chapter 17.)

18.5 Step 4a: prediction targets and the loss mask

The model returns a full reconstruction $\hat{\mathbf{x}}$ of shape `(B, 165, H, W)`; the trainer decides which pixels count. It builds three pixel-level masks (`hyperspectral_segformer_mae_trainer.py`, lines 186–238):

1. The **pixel prediction mask** Q: each token's flag q_t copied to its 16 pixels by nearest-neighbour upsampling (`F.interpolate(...,mode="nearest")` from 32 × 32 to 128 × 128). Pixel (y, x) belongs to token $(\lfloor y/4 \rfloor, \lfloor x/4 \rfloor)$, so $Q_{y,x} = q_{\lfloor y/4 \rfloor, \lfloor x/4 \rfloor}$.
2. The **eroded validity mask** $E = \text{erode}(m)$ of Section 18.6. In training the erosion kernel is 1, which leaves $E = m$.
3. The **loss mask**, their product:

$$L_{y,x} = Q_{y,x} \cdot E_{y,x}. \tag{18.5}$$

A pixel is graded when it was hidden *and* is valid (and, with erosion on, far enough from an invalid pixel). The L1 part of the loss (Chapter 25) averages the per-pixel band-mean error over

the set $\Omega = \{(y, x) : L_{y,x} = 1\}$:

$$\mathcal{L}_1 = \frac{1}{|\Omega|} \sum_{(y,x)\in\Omega} \frac{1}{165} \sum_{c=1}^{165} |\hat{x}_{c,y,x} - x_{c,y,x}| \,. \tag{18.6}$$

Worked example 18.9: The loss mask on the 4 × 4 example

Return to worked example 18.1 with 2 × 2 tokens, $v = [1, 0, 0, 1]$. Two tokens are valid; say $\rho = 0.65$ gives $\lfloor 0.65 \times 2 \rfloor = \lfloor 1.3 \rfloor = 1$ hidden token, and the noise picks t_3 (bottom-right). So $q = [0, 0, 0, 1]$. Upsample to pixels, then multiply by m (erosion off):

$$Q = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}, \quad m = \begin{pmatrix} 1 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 0 & 1 & 1 \end{pmatrix}, \quad L = Q \odot m = \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \end{pmatrix}.$$

($\odot$ is the entry-by-entry product.) $|\Omega| = 4$. Now suppose instead the course's 8 × 8 grid of Figure 18.1 hid its token 2, which has 12 valid pixels of 16. Then Q has 16 ones in that block but L has only 12: **a hidden token is graded only on its valid pixels**. A token can be valid (more than half) and still contain invalid pixels, and those are never graded.

18.6 Step 4b: erosion of the mask border

18.6.1 The problem

Take a pixel that is perfectly valid but sits two pixels from a cloud edge. Its token's 4 × 4 block may include cloud pixels, which were zeroed and sit near −1.22 in normalised space. The token's value is built partly from ground and partly from "no data", and the reconstruction there is poor for a purely technical reason. In a scene this shows up as a bright ring around every cloud, every water edge and every ragged swath edge in the anomaly map: the most common false-positive pattern in this system. The fix is to shrink the valid region by a border strip and not count that strip in the loss (training) or the score (inference). The pixels are still reconstructed; they do not count.

18.6.2 Binary erosion and dilation

Definition 18.5 (Binary erosion and dilation)**.** Let A be a set of pixels (here the valid ones) and K_r the square of offsets $\{(\delta_y, \delta_x) : |\delta_y| \le r,\ |\delta_x| \le r\}$, of side $k = 2r + 1$. The *erosion* of A keeps a pixel only if its whole square neighbourhood lies in A:

$$A \ominus K_r = \{\, p : p + \delta \in A \text{ for every } \delta \in K_r \,\}.$$

The *dilation* of a set D adds every pixel within reach of it:

$$D \oplus K_r = \{\, p : p + \delta \in D \text{ for some } \delta \in K_r \,\}.$$

Erosion and dilation are linked by complements. Write A^c for the pixels not in A (the invalid ones). Then

$$(A \ominus K_r)^c = A^c \oplus K_r. \tag{18.7}$$

Derive it in words, one step at a time:

$$\begin{aligned}
p \notin A \ominus K_r &\iff \text{not } (p+\delta \in A \text{ for every } \delta) && \text{definition of erosion}\\
&\iff p+\delta \notin A \text{ for some } \delta && \text{“not every” is “some not”}\\
&\iff p+\delta \in A^c \text{ for some } \delta && \text{definition of complement}\\
&\iff p \in A^c \oplus K_r && \text{definition of dilation.}
\end{aligned}$$

So to erode the valid region you may *grow* the invalid region and flip back. Growing a 0/1 mask is a *max pool* with stride 1: the window's maximum is 1 exactly when some pixel in the window is 1. That is the whole implementation (`token_masking.py`, lines 64–74):

```
padding = kernel_size // 2                          # radius r
invalid = 1.0 - mask                                # (B, 1, H, W): invalid -> 1
dilated_invalid = F.max_pool2d(invalid, kernel_size=kernel_size, stride=1, padding=
    padding)
eroded_mask = 1.0 - dilated_invalid                 # flip back
```

The erosion radius is $r = \lfloor k/2 \rfloor$: kernel 5 removes 2 pixels, kernel 15 removes 7, and kernel 1 removes nothing, because a 1×1 max pool returns its input. That is how “off” is expressed without a special case.

Worked example 18.10: Erosion in one dimension, kernel 5

The mask $[0,0,1,1,1,1,1,1,1,1,0,0]$ (indices 0–11), kernel 5, radius $\lfloor 5/2 \rfloor = 2$. Flip: invalid $= [1,1,0,0,0,0,0,0,0,0,1,1]$. Take the maximum over $[i-2, i+2]$, clipped at the ends:

$$\begin{aligned}
i = 0,1,2,3 &: \text{window contains index 0 or 1, max 1}\\
i = 4 &: \text{indices 2..6, all 0, max 0}\\
i = 5,6,7 &: \text{indices 3..9 at most, all 0, max 0}\\
i = 8,9,10,11 &: \text{window contains index 10 or 11, max 1}
\end{aligned}$$

Dilated invalid: $[1,1,1,1,0,0,0,0,1,1,1,1]$. Flip back: $[0,0,0,0,1,1,1,1,0,0,0,0]$. The valid run was indices 2–9 (8 pixels) and is now 4–7 (4 pixels): two removed from each end, as the radius promised.

Worked example 18.11: Erosion in two dimensions, kernel 3

A 5×5 mask whose top row is invalid (a swath edge) and whose centre pixel is invalid (one bad pixel). Kernel 3, radius 1: a pixel survives only if all 8 neighbours and itself are valid.

$$m = \begin{pmatrix} 0&0&0&0&0\\ 1&1&1&1&1\\ 1&1&0&1&1\\ 1&1&1&1&1\\ 1&1&1&1&1 \end{pmatrix} \quad\Longrightarrow\quad \operatorname{erode}(m) = \begin{pmatrix} 0&0&0&0&0\\ 0&0&0&0&0\\ 1&0&0&0&1\\ 1&0&0&0&1\\ 1&1&1&1&1 \end{pmatrix}.$$

Check three pixels by hand. Pixel $(1,0)$: its window covers rows 0–2, and row 0 is invalid, so it is eroded. Pixel $(2,0)$: its window covers rows 1–3 and columns 0–1 (column -1 is outside the image); none of those is invalid, so it survives. Pixel $(3,1)$: its window covers rows 2–4 and columns 0–2, which includes the bad centre $(2,2)$, so it is eroded. The single bad pixel grew a 3×3 hole; the invalid row grew a second invalid row beneath it.

Source check: the patch edge itself is not eroded

`F.max_pool2d` pads with $-\infty$, not with zeros, so positions outside the image never count as invalid. Erosion therefore grows only around *invalid pixels*; the edge of a 128-pixel patch or of the scene array is not trimmed unless the pixels there are invalid. Real swath and scene borders are no-data pixels, so they are trimmed. The script `research/textbook/tools/compute_masking.py` checks both cases: an all-valid 4×4 mask comes back unchanged with kernel 3.

Source check: the kernel must be odd

Nothing validates oddness: the training field has `ge=1` and the inference field `gt=0` (`app/models/training/training_config.py`, lines 101–106; `app/models/training/inference_config.py`, lines 109–116). An even kernel breaks the shape. By the output-size formula of Chapter 17 with stride 1 and padding $k/2$, a side H becomes $H + 2(k/2) - k + 1 = H + 1$, so the eroded mask would be one pixel larger than the image and the loss mask product would fail. Both docstrings also say the kernel "should match OPE kernel size (7 for Stage 1)" (`token_masking.py`, line 48; `inference_config.py`, line 115); stage 1's kernel is 4 (Chapter 17). Use odd kernels.

18.6.3 Why throw the border strip away, and how wide

A valid pixel within r pixels of an invalid one shares its token, or a neighbouring token whose patch-embedding window it feeds, with no-data. The stage-1 token is 4 pixels wide, so a radius of 3 is enough to cover any token that touches an invalid pixel; larger radii also cover the wider windows of stages 2–4 ($r_2 = 12$ pixels, Chapter 17). Training uses radius 0 because training patches come from scene interiors and are mostly fully valid: eroding every patch would throw away supervision for little gain. Inference covers whole scenes with real cloud and swath edges, where a false ring costs more than a few missing pixels, so it erodes by 7.

Table 18.2: The three erosions in the pipeline. A pixel must survive every one that applies to contribute a score.

Erodes	Where	Field	Default kernel
sensor validity	training loss	`erosion_kernel_size`	1 (off)
sensor validity	inference	`erosion_kernel_size`	15
analyst's region	scoring	`keep_mask_erosion_kernel_size`	1 (off)

Values that will drift

Defaults as of 2026-10-29: training erosion kernel 1 [M: `app/models/training/training_config.py`], inference 15 [M: `app/models/training/inference_config.py`]; FT04 trained with 1 [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]. The scoring erosion uses SciPy's `binary_erosion` in `_anomaly_scoring_run.py` rather than a max pool.

18.7 All four masks on one patch

Figure 18.5 puts the four steps on one small patch of 16×16 pixels with 4×4 tokens (16 tokens). A cloud covers rows 10–15, columns 11–15.

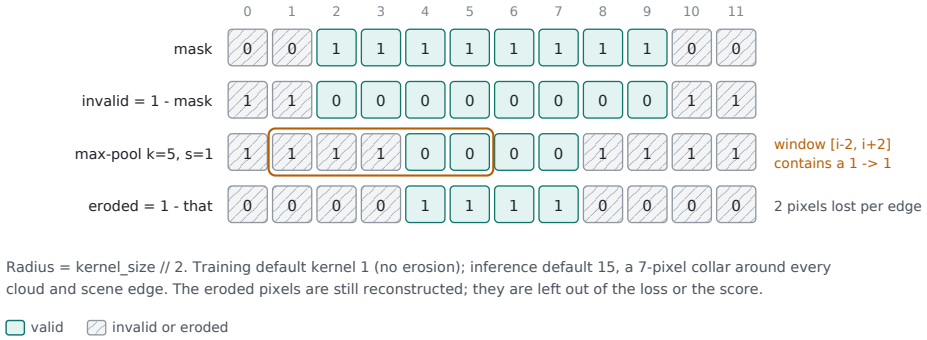


Figure 18.4: Erosion as a max pool, with the 1-D example of worked example 18.10 (kernel 5). Flipping makes invalid pixels 1; a stride-1 max pool grows that region by $\lfloor 5/2 \rfloor = 2$ pixels on each side; flipping back leaves the valid region shrunk by 2 pixels per edge.

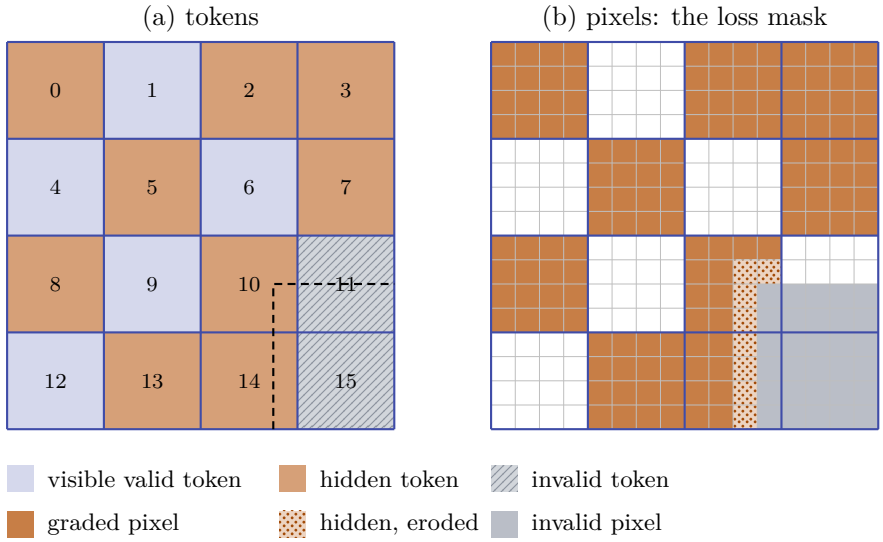


Figure 18.5: The four masks on one 16×16 patch with 4×4 tokens, in the book's colours (indigo spatial tokens, amber loss, grey invalid). A cloud (dashed) covers rows 10–15, columns 11–15. (a) Tokens 11 and 15 are invalid (valid fractions 8/16 and 0/16); the other 14 are valid and $\lfloor 0.65 \times 14 \rfloor = 9$ of them are hidden (an illustrative draw). (b) The loss mask with erosion radius 1 (kernel 3): hidden pixels count only when valid and not next to the cloud. Worked in worked example 18.12.

Worked example 18.12: Counting the graded pixels in Figure 18.5

Token t covers rows $4\lfloor t/4 \rfloor$ to $+3$ and columns $4(t \bmod 4)$ to $+3$. The cloud is rows ≥ 10 and columns ≥ 11.
Validity. Token 10 (rows 8–11, columns 8–11) contains cloud at rows 10–11, column 11: 2 pixels, so $f_{10} = 14/16 = 0.875$, valid. Token 11 (rows 8–11, columns 12–15) has cloud in rows 10–11, all four columns: 8 pixels, so $f_{11} = 8/16 = 0.5$, *invalid* by the strict rule. Token 14 (rows 12–15, columns 8–11) has cloud in column 11: 4 pixels, $f_{14} = 12/16 = 0.75$, valid. Token 15 is all cloud, $f_{15} = 0$. So $n_{\text{valid}} = 16 - 2 = 14$.
Hiding. $n_{\text{hide}} = \lfloor 0.65 \times 14 \rfloor = \lfloor 9.1 \rfloor = 9$; the draw shown hides tokens $0, 2, 3, 5, 7, 8, 10, 13, 14$. That is $9 \times 16 = 144$ hidden pixels.
Without erosion, the loss mask drops the hidden pixels that are invalid: 2 in token 10 and 4 in token 14, so $144 - 2 - 4 = 138$ graded pixels.
With kernel 3 (radius 1), the invalid region grows by one pixel: rows ≥ 9, columns ≥ 10. In token 10 that removes rows 9–11, columns 10–11: $3 \times 2 = 6$ pixels (the 2 invalid ones included). In token 14, rows 12–15, columns 10–11: $4 \times 2 = 8$ pixels (the 4 invalid ones included). Graded pixels: $144 - 6 - 8 = 130$. The 8 extra pixels lost to erosion are the border strip next to the cloud.

18.8 Common confusions

Is the token mask the same as the keep mask?
No, and this is the most common mix-up. The token mask says whether a token holds real data; the keep mask says whether the encoder is shown it. Invalid tokens have token mask 0 and keep mask 1.

Is the mask ratio a fraction of all tokens?
Of the valid ones: $\lfloor \rho \, n_{\text{valid}} \rfloor$. A half-cloudy patch hides about half as many tokens in absolute terms.

Does 0.65 mean 65% are hidden?
Slightly fewer: the count is rounded down. A fully valid patch hides 665 of 1024 (64.9%); six valid tokens would hide 3 (50%).

Why add exactly 2?
Any shift of at least 1 puts invalid noise at or above 1, above every valid value in $[0, 1)$. Two is a comfortable margin.

Where is ids_restore?
Nowhere. The scatter through the sort order is the same operation as a gather through the inverse permutation (worked example 18.4), so it is never built.

Does a hidden token get graded on all 16 of its pixels?
Only on the valid ones (and, with erosion on, only those away from invalid pixels). A token can be valid with up to 7 invalid pixels.

Does the same mask get used every batch?
No. `torch.rand` is called afresh each time, so every batch of every epoch hides a different set.

Does erosion shrink the output image?
No. The image keeps its size; eroded pixels are reconstructed but not counted in the loss or score.

18.9 Check yourself

Exercise 18.1. A token's 4×4 block has 9 valid pixels. Compute f_t and decide whether it is valid. Then do the same for 8 valid pixels.

Exercise 18.2. A patch has 37 valid tokens and $\rho = 0.65$. How many are hidden, how many tokens does the encoder see if the patch has 1024 tokens in all, and what is the effective mask ratio?

Exercise 18.3. Six tokens, $v = [1, 1, 1, 0, 1, 1]$, $\rho = 0.65$, noise $[0.9, 0.1, 0.05, 0.4, 0.6, 0.02]$. Work through the $+2$, the count, the argsort, the scatter and the keep mask. Then compute the inverse permutation and check that gathering the sorted flags through it gives the same prediction mask.

Exercise 18.4. `keep_mask=[1,0,1,1]` for tokens $t_0..t_3$. Write the gather indices, the kept tokens, the selection matrix $\mathbf{S}$, and the restored sequence after the blocks produce t'_0, t'_2, t'_3.

Exercise 18.5. Erode $m = [0, 1, 1, 1, 1, 0]$ with kernel 3, by hand, using the dilate-then-flip method. Then erode $[1, 1, 1, 1, 0, 1, 1, 1, 1]$ with kernel 5.

Exercise 18.6. In a batch of two, patch A has 1024 valid tokens and patch B 960. With $\rho = 0.65$, compute each patch's kept count, `max_kept`, and how many hidden tokens leak into which patch.

Next: Chapter 19, The decoder, pixel shuffle and one full forward trace.

Chapter 19

The decoder, pixel shuffle and one full forward trace

The one thing this chapter teaches

The decoder brings the four encoder maps to one width (256 channels, by 1×1 convolutions) and one size (32×32, by bilinear interpolation), stacks and fuses them, refines at quarter resolution, and then reaches full resolution by *rearranging* 512 channels into 4×4 blocks of 32 channels, so that every output pixel gets its own learned prediction.

Chapter 17 ended with four feature maps of the same ground: F1 `(B, 32, 32, 32)`, F2 `(B, 64, 16, 16)`, F3 `(B, 160, 8, 8)` and F4 `(B, 256, 4, 4)`. The model must return a reconstruction of the input's shape, `(B, 165, 128, 128)`, because a residual is a subtraction and you can only subtract tensors of the same shape (Chapter 9). This chapter builds that last stretch: the decoder (`app/foundation_models/components/seg_former_decoder.py`, 176 lines), the decompressor and the denormalisation. It ends by walking one batch through the whole model.

19.1 The problem, and why the obvious fixes fail

Three things must happen: the four maps have to be combined, the result has to reach 128×128, and the 32 compressed channels have to become 165 bands again. The obvious ways to do the first two each fail.

- **Use only F4 and upsample it 32 times.** Sixteen tokens cannot say where a single odd pixel is; all fine detail is lost.
- **Run convolutions at full resolution** (a U-Net style decoder). It works, but a 3×3 convolution at 128×128 costs sixteen times as much as at 32×32, because there are $128^2/32^2 = 16$ times as many positions.
- **Work at 32×32 and finish with bilinear upsampling.** Cheap, but bilinear interpolation *averages* neighbours, so a single anomalous pixel's prediction is blended with its fifteen ordinary neighbours, and the residual that should flag it is diluted. That is exactly the signal this system measures.

The SegFormer decoder [46], as modified here, takes a fourth route: do all the expensive thinking at 32×32, and jump to full size with a free rearrangement, *pixel shuffle* [39], that gives each output pixel its own channel and therefore its own weights.

Analogy

A mixing desk combines a 48-track recording into one stereo master. Each track first goes through its own fader and equaliser so that all tracks sit at a common level (the 1×1 projections to 256 channels). Tracks recorded at different sample rates are resampled to one rate (the bilinear upsampling). The desk sums them into a mix bus (the concatenation and the fuse convolution) and a mastering stage polishes the result (the refine convolutions). The last step is the one the analogy makes vivid: a surround-sound master is not "stretched" to more speakers; each speaker gets its own dedicated channel. Pixel shuffle is that: 16 channels per position, one for each of the 16 output pixels in the 4×4 block.

19.2 Step 1: one width for all four maps

You cannot stack maps of 32, 64, 160 and 256 channels into one tensor and treat every channel alike, so each map first passes through its own 1×1 convolution to the common *decoder width* $D_{\text{dec}} = 256$:

```
self.linear_projections = nn.ModuleList([
    nn.Conv2d(embed_dim, decoder_dim, kernel_size=1)    # 32/64/160/256 -> 256
    for embed_dim in embed_dims])
```

A 1×1 convolution looks at one position at a time and mixes only channels: at every position (h, w) it computes

$$\mathbf{p}_{h,w} = \mathbf{W}_i \, \mathbf{f}_{h,w} + \mathbf{b}_i, \qquad \mathbf{W}_i \in \mathbb{R}^{256 \times C_i}, \ \mathbf{b}_i \in \mathbb{R}^{256}, \tag{19.1}$$

with the same $\mathbf{W}_i$ and $\mathbf{b}_i$ at every position (Chapter 10). It is a linear layer applied per pixel.

Worked example 19.1: A 1×1 projection at one position

Shrink the sizes: 2 input channels, 3 output channels. At one position the feature vector is $\mathbf{f} = (2, -1)^\top$. Let

$$\mathbf{W} = \begin{pmatrix} 1 & 0 \\ 0.5 & 2 \\ -1 & 1 \end{pmatrix}, \qquad \mathbf{b} = \begin{pmatrix} 0 \\ 1 \\ 0.5 \end{pmatrix}$$

(illustrative weights). Multiply row by row, then add the bias. Do it on paper:

$$\begin{aligned} p_0 &= 1 \cdot 2 + 0 \cdot (-1) + 0 = 2 + 0 + 0 = 2, \\ p_1 &= 0.5 \cdot 2 + 2 \cdot (-1) + 1 = 1 - 2 + 1 = 0, \\ p_2 &= -1 \cdot 2 + 1 \cdot (-1) + 0.5 = -2 - 1 + 0.5 = -2.5. \end{aligned}$$

So $\mathbf{p} = (2, 0, -2.5)^\top$. The decoder does the same with $\mathbf{W}_1 \in \mathbb{R}^{256 \times 32}$ at each of F1's $32 \times 32 = 1024$ positions, and similarly for F2–F4.

Shapes after step 1:

$$\begin{aligned} \text{F1} : (B, 32, 32, 32) &\ \to\ (B, 256, 32, 32) \\ \text{F2} : (B, 64, 16, 16) &\ \to\ (B, 256, 16, 16) \\ \text{F3} : (B, 160, 8, 8) &\ \to\ (B, 256, 8, 8) \\ \text{F4} : (B, 256, 4, 4) &\ \to\ (B, 256, 4, 4) \end{aligned}$$

F4 already has 256 channels but still gets its own projection: the four maps live in different learned spaces and each needs its own mapping into the shared one.

19.3 Step 2: one size for all four maps, by bilinear interpolation

Every projected map is resized to F1's 32×32:

```
if proj.shape[2] != target_h or proj.shape[3] != target_w:          # F1 is skipped
    proj = F.interpolate(proj, size=(target_h, target_w),
                         mode='bilinear', align_corners=False)      # (B, 256, 32,
    32)
```

F2 grows by a factor 2, F3 by 4, F4 by 8. What does *bilinear* compute?

19.3.1 Linear interpolation on a line

Definition 19.1 (Linear interpolation)**.** Given values f_0 at position 0 and f_1 at position 1, the *linear interpolation* at a position $u \in [0, 1]$ between them is

$$f(u) = (1 - u)\, f_0 + u\, f_1. \tag{19.2}$$

Derive it: the straight line through $(0, f_0)$ and $(1, f_1)$ has slope $(f_1 - f_0)/(1 - 0) = f_1 - f_0$ and starts at f_0, so $f(u) = f_0 + u(f_1 - f_0)$. Expand the bracket: $f_0 + uf_1 - uf_0$. Collect the f_0 terms: $(1 - u)f_0 + uf_1$. The two weights $1 - u$ and u are non-negative and sum to 1, so $f(u)$ is a weighted average of the two neighbours, weighted by closeness. At $u = 0$ you get f_0, at $u = 1$ you get f_1, at $u = 0.25$ you get three parts f_0 to one part f_1.

19.3.2 Where does an output pixel sit on the input grid?

To resize a row of n_{in} values to n_{out} values you must decide where each output pixel falls among the inputs. With `align_corners=False`, PyTorch treats every pixel as a cell of width 1 whose *centre* is the sample, and lines up the outer edges of the two grids. Output pixel d has its centre at $d + \frac{1}{2}$ in output units; multiplying by the scale $s = n_{\text{in}}/n_{\text{out}}$ converts to input units; subtracting $\frac{1}{2}$ converts from "centre" back to an index:

$$u_d = \left(d + \tfrac{1}{2}\right) \frac{n_{\text{in}}}{n_{\text{out}}} - \tfrac{1}{2}, \qquad \text{clamped below at 0.} \tag{19.3}$$

The output value is then the linear interpolation between input indices $\lfloor u_d \rfloor$ and $\lfloor u_d \rfloor + 1$ with weight $u_d - \lfloor u_d \rfloor$; when $\lfloor u_d \rfloor + 1$ falls off the end, the last input value is used.

Worked example 19.2: Upsampling a row of two values to four

Input $[0, 4]$, so $f_0 = 0$, $f_1 = 4$; $n_{\text{in}} = 2$, $n_{\text{out}} = 4$, scale $s = 2/4 = 0.5$. On paper, one output at a time:

$d = 0$:	$u = (0.5)(0.5) - 0.5 = -0.25 \to$ clamp to 0,	value $= f_0 = 0$
$d = 1$:	$u = (1.5)(0.5) - 0.5 = 0.25$,	value $= 0.75 \cdot 0 + 0.25 \cdot 4 = 1$
$d = 2$:	$u = (2.5)(0.5) - 0.5 = 0.75$,	value $= 0.25 \cdot 0 + 0.75 \cdot 4 = 3$
$d = 3$:	$u = (3.5)(0.5) - 0.5 = 1.25$,	value $= f_1 = 4$

(At $d = 3$ there is no input index 2 to interpolate towards, so the last value is used.) Output $[0, 1, 3, 4]$. `F.interpolate` returns exactly this (`research/textbook/tools/compute_masking_output.txt`).

19.3.3 Bilinear: two linear interpolations

On a grid, a point at fractional position (u, v) (row, column) sits inside a square of four samples f_{00}, f_{01} (upper row) and f_{10}, f_{11} (lower row), with fractional offsets $\alpha = u - \lfloor u \rfloor$ down and $\beta = v - \lfloor v \rfloor$ across. *Bilinear* interpolation interpolates along each row first, then between the two results:

$$\begin{aligned} \text{upper row:} \quad & g_0 = (1-\beta) f_{00} + \beta f_{01}, \\ \text{lower row:} \quad & g_1 = (1-\beta) f_{10} + \beta f_{11}, \\ \text{between rows:} \quad & f = (1-\alpha)\, g_0 + \alpha\, g_1. \end{aligned} \tag{19.4}$$

Substitute g_0 and g_1 and expand to see all four weights at once:

$$f = (1-\alpha)(1-\beta) f_{00} + (1-\alpha)\beta f_{01} + \alpha(1-\beta) f_{10} + \alpha\beta f_{11}.$$

The four weights are products of the two one-dimensional weights and sum to $\big((1-\alpha)+\alpha\big)\big((1-\beta)+\beta\big) = 1$. The expansion is symmetric in the two directions, so interpolating the columns first gives the same answer.

Worked example 19.3: Bilinear upsampling of a 2 × 2 grid to 4 × 4

Input $\begin{pmatrix} 1 & 3 \\ 5 & 7 \end{pmatrix}$. Rows and columns each go from 2 to 4, so by worked example 19.2 the fractional positions are $0, 0.25, 0.75, 1$ (after clamping) in both directions. First interpolate along each input row, using the weights of worked example 19.2:

$$\begin{aligned} \text{row } [1,3]: \quad & [\,1,\ 0.75\cdot 1 + 0.25\cdot 3,\ 0.25\cdot 1 + 0.75\cdot 3,\ 3\,] = [1,\ 1.5,\ 2.5,\ 3], \\ \text{row } [5,7]: \quad & [\,5,\ 0.75\cdot 5 + 0.25\cdot 7,\ 0.25\cdot 5 + 0.75\cdot 7,\ 7\,] = [5,\ 5.5,\ 6.5,\ 7]. \end{aligned}$$

Now interpolate down each column between these two rows, with the same weights $0, 0.25, 0.75, 1$. For the second output column (values 1.5 above, 5.5 below):

$$\begin{aligned} &\text{out row } 0 : 1.5, \\ &\text{out row } 1 : 0.75\cdot 1.5 + 0.25\cdot 5.5 = 1.125 + 1.375 = 2.5, \\ &\text{out row } 2 : 0.25\cdot 1.5 + 0.75\cdot 5.5 = 0.375 + 4.125 = 4.5, \\ &\text{out row } 3 : 5.5. \end{aligned}$$

Doing every column the same way:

$$\begin{pmatrix} 1 & 1.5 & 2.5 & 3 \\ 2 & 2.5 & 3.5 & 4 \\ 4 & 4.5 & 5.5 & 6 \\ 5 & 5.5 & 6.5 & 7 \end{pmatrix}.$$

This is exactly what `F.interpolate(...,mode='bilinear',align_corners=False)` returns (checked in `research/textbook/tools/compute_masking.py`). Notice that no output value is outside the range $[1, 7]$ of the inputs, and that no output differs from its neighbour by more than 2: bilinear interpolation can only smooth.

That last remark is why bilinear interpolation is fine *here*, on smooth coarse context from F2–F4, and wrong as the final step to full resolution (Section 19.5).

19.4 Steps 3 and 4: stack, then fuse

```
fused = torch.cat(unified, dim=1)      # 4 x (B, 256, 32, 32) -> (B, 1024, 32, 32)
fused = self.fuse_conv(fused)          # Conv2d(1024, 256, k=1) -> (B, 256, 32, 32)
fused = self.fuse_act(fused)           # GELU
```

`torch.cat(...,dim=1)` glues along the channel axis: $4 \times 256 = 1024$ channels. At every one of the 1024 positions there are now 1024 numbers: 256 describing fine texture there (from F1), 256 medium structure (F2), 256 coarse (F3) and 256 global context (F4). The fuse convolution is another 1×1 convolution, (19.1) with $\mathbf{W} \in \mathbb{R}^{256\times1024}$: at each position it learns how much of each scale to use. Splitting $\mathbf{W}$ into four 256×256 blocks $[\mathbf{W}^{(1)}\ \mathbf{W}^{(2)}\ \mathbf{W}^{(3)}\ \mathbf{W}^{(4)}]$ shows what it does:

$$\mathbf{W}\begin{pmatrix}\mathbf{p}^{(1)}\\ \mathbf{p}^{(2)}\\ \mathbf{p}^{(3)}\\ \mathbf{p}^{(4)}\end{pmatrix} = \mathbf{W}^{(1)}\mathbf{p}^{(1)} + \mathbf{W}^{(2)}\mathbf{p}^{(2)} + \mathbf{W}^{(3)}\mathbf{p}^{(3)} + \mathbf{W}^{(4)}\mathbf{p}^{(4)}\ :$$

a learned sum of the four scales, one linear map per scale. The nonlinearity GELU (Chapter 16) follows. The result: one 256-channel map at 32×32 carrying all four scales. This projection, upsample, concatenate and fuse pattern is what the SegFormer paper calls an *all-MLP decoder*, because 1×1 convolutions are per-pixel MLP layers.

> Up to the fuse, the decoder is a per-pixel linear mix of four interpolated maps; everything spatial in it so far is the smoothing of bilinear interpolation.

19.5 Step 5: refine at quarter resolution, then pixel shuffle

```
self.refine = nn.Sequential(
    nn.Conv2d(decoder_dim, decoder_dim, kernel_size=3, padding=1),          # 256 ->
    256
    nn.GELU(),
    nn.Conv2d(decoder_dim, out_channels * 4 * 4, kernel_size=3, padding=1)) # 256 ->
    512
...
out = F.pixel_shuffle(refined, 4)          # (B, 512, 32, 32) -> (B, 32, 128, 128)
```

19.5.1 The two 3×3 convolutions

Both keep the grid at 32×32: by the output-size formula of Chapter 17 with $k = 3$, $s = 1$, $p = 1$, a side of 32 becomes $\lfloor(32 + 2 - 3)/1\rfloor + 1 = 31 + 1 = 32$. They are the decoder's only spatial layers. A 3×3 window at quarter resolution spans 3 cells of 4 pixels, $3 \times 4 = 12$ original pixels across; two in a row reach $1 + 2 \cdot 2 = 5$ cells, 20 pixels (the receptive-field recurrence of Chapter 17 with $j = 4$). That is local context around every point at one sixteenth of the cost of full resolution.

The second convolution, a *sub-pixel convolution*, outputs $D \times 4 \times 4 = 32 \times 16 = 512$ channels, where $D = 32$ is the compressed band count, and has **no activation after it**. Its output flows through the decompressor into denormalisation and must be free to take any value, including large negatives; a GELU or sigmoid would make whole ranges of reflectance unreachable. The same reasoning keeps the decompressor bare (Chapter 12).

19.5.2 Pixel shuffle: the index map

Definition 19.2 (Pixel shuffle)**.** For an upscale factor r, `pixel_shuffle` maps a tensor of shape (B, Cr^2, H, W) to (B, C, rH, rW) by

$$\text{out}[b, c,\ rh + i,\ rw + j] = \text{in}[b,\ cr^2 + ri + j,\ h, w], \tag{19.5}$$

for $0 \le c < C$, $0 \le h < H$, $0 \le w < W$ and $0 \le i, j < r$.

Read it from right to left. Input channels come in C consecutive groups of r^2; group c becomes output channel c. Inside a group, channel offset $ri + j$ is a row-major position (i, j) in an $r \times r$ block (row $i = \lfloor \text{offset}/r \rfloor$, column $j = \text{offset} \bmod r$). That block is placed at rows $rh..rh + r - 1$ and columns $rw..rw + r - 1$, the output block that sits over input position (h, w).

Inverting it. Given an output pixel (c, Y, X), which input number made it? Divide each coordinate by r with remainder: $h = \lfloor Y/r \rfloor$, $i = Y \bmod r$, $w = \lfloor X/r \rfloor$, $j = X \bmod r$. Then the source is channel $cr^2 + ri + j$ at position (h, w). Every output pixel has exactly one source and every input number has exactly one destination: pixel shuffle is a permutation of the numbers, with no arithmetic and no parameters.

Worked example 19.4: Pixel shuffle by hand, and PyTorch's order

Take $C = 2$, $r = 2$, $H = W = 1$: an input of $Cr^2 = 8$ channels, each a single number. Let channel k hold the value k, so $\text{in} = [0, 1, 2, 3, 4, 5, 6, 7]$. Apply (19.5) with $h = w = 0$, so the output rows are i and columns j. On paper:

$$\begin{aligned}
c = 0: \quad & (i, j) = (0, 0) \leftarrow 0 \cdot 4 + 0 + 0 = 0, && (0, 1) \leftarrow 0 + 0 + 1 = 1, \\
& (1, 0) \leftarrow 0 + 2 + 0 = 2, && (1, 1) \leftarrow 0 + 2 + 1 = 3, \\
c = 1: \quad & (0, 0) \leftarrow 1 \cdot 4 + 0 + 0 = 4, && (0, 1) \leftarrow 4 + 0 + 1 = 5, \\
& (1, 0) \leftarrow 4 + 2 + 0 = 6, && (1, 1) \leftarrow 4 + 2 + 1 = 7.
\end{aligned}$$

$$\text{out}[0] = \begin{pmatrix} 0 & 1 \\ 2 & 3 \end{pmatrix}, \qquad \text{out}[1] = \begin{pmatrix} 4 & 5 \\ 6 & 7 \end{pmatrix}.$$

Running `F.pixel_shuffle` on this input in PyTorch 2.5.1 gives exactly these two blocks (`research/textbook/tools/compute_masking_output.txt`), which confirms the ordering: channels are grouped by output channel first, and within a group the offset runs along a row before moving down.

Now the real case, $r = 4$, $C = 32$. Which input number produces output pixel $(c, Y, X) = (1, 5, 6)$?

$$\begin{aligned}
h = \lfloor 5/4 \rfloor = 1, && i = 5 \bmod 4 = 1, \\
w = \lfloor 6/4 \rfloor = 1, && j = 6 \bmod 4 = 2, \\
\text{channel} = 1 \cdot 16 + 4 \cdot 1 + 2 = 22. &&
\end{aligned}$$

So the source is input channel 22 at position $(h, w) = (1, 1)$.

The count of numbers is unchanged, as it must be for a permutation:

$$\begin{aligned}
\text{before:} \quad & 512 \times 32 \times 32 = 512 \times 1024 = 524{,}288, \\
\text{after:} \quad & 32 \times 128 \times 128 = 32 \times 16{,}384 = 524{,}288.
\end{aligned}$$

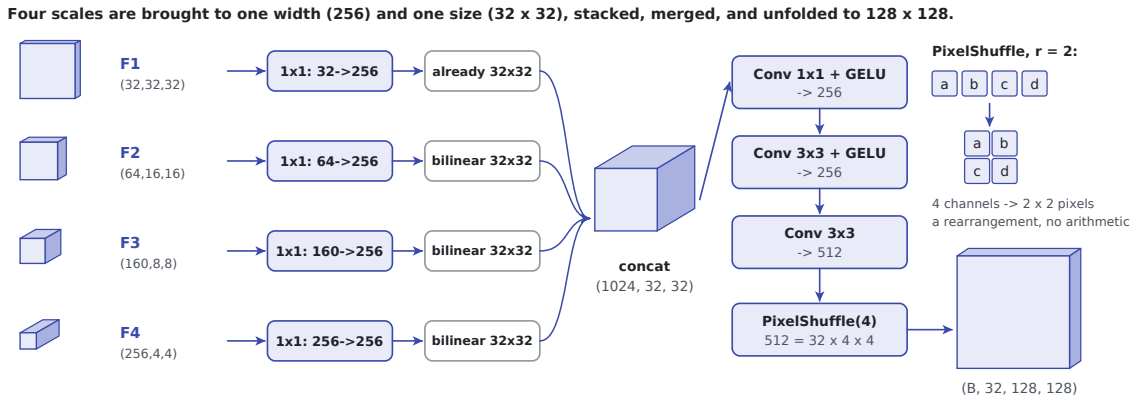


Figure 19.1: The decoder. The four encoder maps are projected to 256 channels by 1×1 convolutions, resized to F1's 32×32, concatenated to 1,024 channels, fused and refined, and expanded to $512 = 32 \times 4 \times 4$ channels. PixelShuffle(4) rearranges each group of 16 channels into a 4×4 block of pixels (inset, with $r = 2$), returning `(B, 32, 128, 128)` without any arithmetic.

19.5.3 Why pixel shuffle and not a final bilinear upsampling

With pixel shuffle, output pixel (i, j) of a 4×4 block comes from its own channel $16c + 4i + j$, and that channel is produced by its own row of weights in the last convolution. So the 16 pixels of a block can take 16 unrelated values. With bilinear upsampling all 16 would be weighted averages of the same few coarse values.

Worked example 19.5: A one-pixel anomaly, two ways

One row of a 4-pixel block, true values $[0, 0, 8, 0]$ (one hot pixel on a cold background, in illustrative units). A bilinear upsampler at factor 4 sees one coarse value per block, the block mean

$$\tfrac{1}{4}(0 + 0 + 8 + 0) = 2,$$

and neighbouring blocks with value 0. Whatever it does, it outputs a smooth bump between neighbouring coarse values: every pixel of the block gets a value between 0 and 2. The prediction at the hot pixel is at most 2, the residual there at least $8 - 2 = 6$, and the three cold pixels pick up spurious residuals of up to 2 each. A pixel-shuffle head has four channels for this row, one per pixel, and can output $[0, 0, 8, 0]$ exactly if its weights have learned that pattern.

The decoder's own comment puts it in the terms of the thermal model it was written for: "a 1-pixel anomaly at 45°C surrounded by 30°C background gets its own predicted value, not a smoothed average of the 4x4 block" (`seg_former_decoder.py`, lines 107–109). The important subtlety: you do *not* want the model to reconstruct an anomaly well. You want it to be *able* to give each pixel its own prediction, so that what it predicts at an anomalous pixel is "what normally goes here", unblurred, and the difference is large and sharp.

19.5.4 What it costs

A convolution's work is its multiply-adds per position (the weight count without biases) times the number of positions. For the two refine convolutions at 32×32:

$$\begin{aligned} &\text{refine 1:} && 256 \times 256 \times 9 = 589{,}824 \text{ per position}, && 589{,}824 \times 1024 = 603{,}979{,}776, \\ &\text{refine 2:} && 256 \times 512 \times 9 = 1{,}179{,}648 \text{ per position}, && 1{,}179{,}648 \times 1024 = 1{,}207{,}959{,}552, \end{aligned}$$

about 1.8 billion multiply-adds per patch [C]. At 128 × 128 the same two convolutions would do sixteen times as much, about 29 billion. Pixel shuffle itself costs nothing but memory movement.

Source check: thermal comments in a hyperspectral decoder

`SegFormerDecoder` was written for the thermal model (Chakshu) and is reused unchanged: its default is `out_channels=1` and its comments speak of temperatures (lines 49–51, 107–112). The hyperspectral model builds it with `out_channels=compressed_channels` (`hyperspectral_seg_former_mae.py`, lines 107–111), 32 in every shipped configuration, so the last convolution has $32 \times 16 = 512$ outputs. The same class, built with width 128 and 5 output channels, is the segmentation head of Chapter 28; a different, progressive decoder is the subject of Chapter 42.

19.6 Decompress and denormalise

The decoder's output is still in the compressed 32-channel space. Two last steps bring it back to bands and to physical units.

The decompressor (Chapter 12) is a 1 × 1 convolution from 32 to 165 channels, with no normalisation and no activation:

$$\texttt{(B, 32, 128, 128)} \xrightarrow{\text{Conv2d(32, 165, k=1)}} \texttt{(B, 165, 128, 128)}.$$

At each pixel it computes $\hat{\mathbf{z}} = \mathbf{W}_{\text{dec}}\,\mathbf{y} + \mathbf{b}_{\text{dec}}$ with $\mathbf{W}_{\text{dec}} \in \mathbb{R}^{165\times 32}$: each of the 165 bands is a learned mixture of the 32 compressed channels.

Denormalisation undoes the per-band z-score of Chapter 5. With the band's stored mean μ_c and standard deviation σ_c (buffers baked into the model):

$$\hat{x}_{c,y,x} = \sigma_c\,\hat{z}_{c,y,x} + \mu_c. \tag{19.6}$$

It is the normalisation $z = (x - \mu)/\sigma$ solved for x: multiply both sides by σ, then add μ.

Worked example 19.6: Denormalising one value

Suppose a band has $\mu = 0.10$ and $\sigma = 0.08$ [A] (illustrative values of the right size for reflectance), and the decompressor outputs $\hat{z} = -0.5$ at some pixel. Then

$$\begin{aligned} \hat{x} &= 0.08 \times (-0.5) + 0.10 && \text{multiply by } \sigma, \text{ then add } \mu \\ &= -0.04 + 0.10 \\ &= 0.06. \end{aligned}$$

A predicted reflectance of 0.06, half a standard deviation below the band mean.

19.7 One patch, all the way through

Every component of the model's `forward` method is now yours (`app/foundation_models/components/hyperspectral_seg_former_mae.py`, lines 116–156). Here is the whole walk for a batch of $B = 4$ patches of 128 × 128 with 165 bands, compressed to $D = 32$, at $\rho = 0.65$.

```
def forward(self, x, mask=None, keep_mask=None):   # x (4,165,128,128), mask
    (4,1,128,128), keep_mask (4,1024)
```

```
    if mask is not None:
        x = x * mask                                    # invalid pixels -> 0.0 in every
    band
    if self.normalize is not None:
        x = self.normalize(x)                           # per-band z-score
    x = self.compressor(x)                              # (4, 32, 128, 128)
    features = self.encoder(x, keep_mask=keep_mask)     # [F1, F2, F3, F4]
    x_hat = self.decoder(features)                      # (4, 32, 128, 128)
    x_hat = self.decompressor(x_hat)                    # (4, 165, 128, 128)
    if self.denormalize is not None:
        x_hat = self.denormalize(x_hat)                 # back to reflectance
    return x_hat
```

Notice what is *not* here: no loss, no score, no masking strategy. The model reconstructs whatever it is given with whatever keep mask it is handed. The trainer chooses random masks (Chapter 18), the inferencer chooses complementary ones (Chapter 30); the same class serves both.

Table 19.1: One forward pass, every shape. $B = 4$, 165 bands, $D = 32$, 128×128, $\rho = 0.65$ on fully valid patches. Shapes confirmed by running the real model on CPU (`research/textbook/tools/compute_params.py`).

#	Operation	Output shape	Note
	input x	`(4, 165, 128, 128)`	reflectance
1	`x * mask`	`(4, 165, 128, 128)`	mask broadcasts over bands
2	normalise	`(4, 165, 128, 128)`	z-scores; invalid $\approx -\mu/\sigma$
3	compressor	`(4, 32, 128, 128)`	1×1 conv + BatchNorm
4	S1 patch embed	`(4, 1024, 32)`	$k4\,s4\,p0$, grid 32×32
	S1 remove tokens	`(4, 359, 32)`	665 hidden per patch
	S1 2 blocks + LN	`(4, 359, 32)`	full-attention fallback
	S1 restore + reshape	`(4, 32, 32, 32)`	F1, zeros at hidden tokens
	S2 embed, blocks, reshape	`(4, 64, 16, 16)`	F2, $R = 4$
	S3 embed, blocks, reshape	`(4, 160, 8, 8)`	F3, $R = 2$
	S4 embed, blocks, reshape	`(4, 256, 4, 4)`	F4, $R = 1$
5	4 projections	`(4, 256, H_i, W_i)`	one 1×1 conv per map
	bilinear to 32×32	$4\times$ `(4, 256, 32, 32)`	F1 skipped
	concatenate	`(4, 1024, 32, 32)`	4×256
	fuse + GELU	`(4, 256, 32, 32)`	1×1 conv
	refine 1 + GELU	`(4, 256, 32, 32)`	3×3 conv
	refine 2	`(4, 512, 32, 32)`	$512 = 32 \times 16$, no activation
	pixel shuffle(4)	`(4, 32, 128, 128)`	a permutation
6	decompressor	`(4, 165, 128, 128)`	1×1 conv, bare
7	denormalise	`(4, 165, 128, 128)`	$\sigma_c \hat{z} + \mu_c$

The shape story on one line:

$$(4, 165, 128, 128) \to (4, 32, 128, 128) \to \text{tokens} \to \text{F1..F4} \to (4, 32, 128, 128) \to (4, 165, 128, 128).$$

Input and output shapes are identical. Reconstruction models are shape-preserving; they have to be, or you could not subtract one from the other.

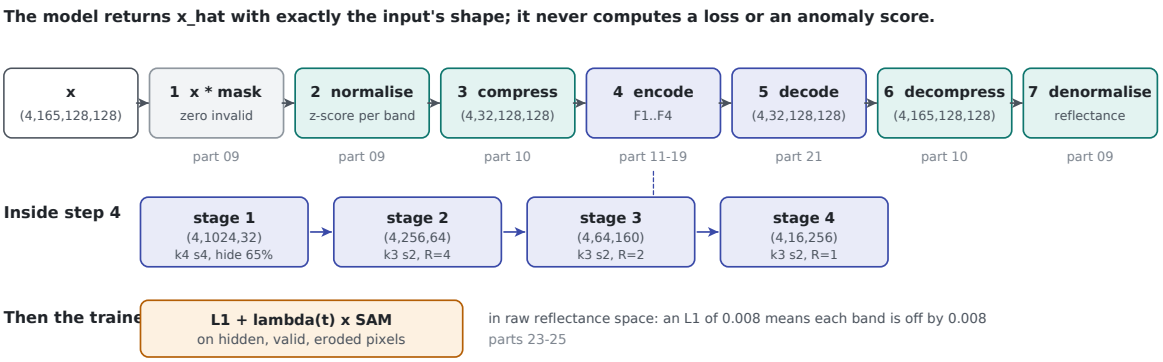


Figure 19.2: The seven steps of `forward()` for a batch of four 128 × 128 patches. Spectral steps are teal, spatial steps indigo. Step 4 expands into the four encoder stages (hiding at stage 1 only). The model stops at the reconstruction; the trainer computes the loss (amber) in raw reflectance.

19.7.1 What the caller does next, and in which units

The trainer takes the output and builds the loss mask of Chapter 18:

```
x_hat = model(pixels, mask=mask, keep_mask=keep_mask)          # (B, 165, H, W),
    reflectance
loss_mask = pixel_pred_mask * eroded_mask                       # (B, 1, H, W)
per_pixel_l1_mean = (x_hat - pixels).abs().mean(dim=1, keepdim=True)   # (B, 1, H, W)
l1_loss = per_pixel_l1_mean[loss_mask == 1].mean()
```

It compares the *denormalised* output with the *raw* input: reflectance against reflectance. So an L1 of 0.008 means that, averaged over graded pixels, each band's predicted reflectance is off by 0.008, a few percent of typical values between 0.02 and 0.30. In normalised space the same number would mean "0.008 standard deviations" and would need translating before it meant anything. The losses themselves are Chapter 25.

Values that will drift

The L1 of 0.008 is the upper end of the expectation "~0.003–0.008" at epoch 500 written into the v0.2.0 configuration's notes [M: `configs/hyperspectral_segformer_exp_2.json`] (line 15), not a measured result of the shipped model; the same file states that the loss is computed in raw reflectance space (line 6). Measured losses for each run are in Chapter 27 and Chapter 38.

19.8 Common confusions

Why upsample to F1's size rather than to full resolution?
Cost. F1's grid is a quarter of the patch side, so convolutions there are sixteen times cheaper. The final jump is a free rearrangement.

Is pixel shuffle learned?
No. It has no parameters; it moves numbers. What is learned is the convolution before it, which produces the 16 sub-pixel channels per output channel.

Does the decoder know which positions were hidden?
Not explicitly. F1 arrives with zero vectors there, an implicit signal; no mask is passed to the decoder.

Why is it fine that F1 has holes?
F2, F3 and F4 cover the same ground without holes, and the fuse convolution learns how much to take from each scale at each position.

Then why use bilinear interpolation at all?
For the coarse maps it is the right tool: their content is smooth context, and smoothing it onto F1's grid loses nothing. Only the last jump to individual pixels must not smooth.

Compress, then decompress: does it undo itself?
No. The encoder and decoder sit between them and change the representation. The compressor makes the transformer affordable; the decompressor turns its answer back into bands.

19.9 Check yourself

Exercise 19.1. Upsample the row $[2, 10]$ to four values with `align_corners=False`, showing u_d for each output. Then upsample the 2×2 grid $\begin{pmatrix} 0 & 4 \\ 8 & 12 \end{pmatrix}$ to 4×4.

Exercise 19.2. Apply `pixel_shuffle` with $r = 2$ to a 1×1 input with 4 channels holding $[9, 8, 7, 6]$.

Exercise 19.3. With $r = 4$ and $C = 32$, which input channel and position produce output pixel $(c, Y, X) = (31, 127, 0)$? And which output pixel does input channel 37 at position $(2, 3)$ go to?

Exercise 19.4. If the compressed width were $D = 24$, what would the last refine convolution's output channel count be, and how many numbers would the tensor before pixel shuffle hold for one patch? Check it against the decoder's output size.

Exercise 19.5. Give the shape after each of the seven steps of `forward` for a batch of 2 patches of 256×256 with 165 bands and $D = 32$, including F1–F4.

Exercise 19.6. A band has $\mu = 0.25$, $\sigma = 0.1$. The decompressor outputs $\hat{z} = 1.3$. What reflectance does the model predict? If the true reflectance is 0.36, what is that pixel's absolute error in this band?

Next: Chapter 20, The parameter budget, counted by hand.

Chapter 20

The parameter budget, counted by hand

The one thing this chapter teaches

With three counting rules (a convolution has $c_{\text{in}}c_{\text{out}}k^2 + c_{\text{out}}$ numbers, a depthwise one $ck^2 + c$, a LayerNorm $2c$) you can derive the size of Indradhanu layer by layer, 5,506,629 trainable numbers for the 165-band model, and land on the real model's figure to the last digit; then the same rules tell you where the parameters live (blocks and decoder), where the compute lives (the decoder), and which parameters a masked pass never trains.

Every chapter of this part ended with a shape. This one ends with a count. Counting parameters is the strongest test of understanding an architecture: if any layer in your head is the wrong size, the total will not match, and you will know. The totals are checked against two independent sources: the `torchinfo` table in `research/model_break_down/05_hyperspectral_segformer_mae.md`, and a build of the real classes on CPU by `research/textbook/tools/compute_params.py`, whose output is saved beside it as `compute_params_output.txt`.

20.1 What a parameter is, and what it is not

Definition 20.1 (Parameter, buffer)**.** A *parameter* is one number that training adjusts: a single weight or bias. A *buffer* is a number stored in the model file that training does not adjust by gradient: the normalisation means and standard deviations, and BatchNorm's running statistics.

Each parameter is a 32-bit float, 4 bytes, so the parameter count fixes the file size:

$$5{,}506{,}629 \times 4 = 22{,}026{,}516 \text{ bytes} \approx 22.03 \text{ MB},$$

which is the "Params size (MB): 22.03" that `torchinfo` reports. The figure is measured: [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`].

Why bother counting? Three reasons. A count that matches proves you know every layer's shape. A count tells you where capacity is spent, which matters when you decide what to widen or shrink (Chapter 43). And a count is *not* a measure of cost, which is a lesson worth learning with real numbers (Section 20.9).

20.2 Three counting rules

A convolution. `Conv2d(c_in,c_out,k)` has, for each of its c_{out} output channels, one kernel of $c_{\text{in}} \times k \times k$ weights (it reads every input channel over a $k \times k$ window) and one bias:

$$P_{\text{conv}} = c_{\text{out}}\,(c_{\text{in}}k^2 + 1) = c_{\text{in}}c_{\text{out}}k^2 + c_{\text{out}}. \tag{20.1}$$

A depthwise convolution. With `groups=c`, each channel has its own $k \times k$ kernel and reads only itself, so the factor c_{in} drops to 1:

$$P_{\text{dw}} = c\,(k^2 + 1) = ck^2 + c. \tag{20.2}$$

A linear layer and a LayerNorm. `Linear(a,b)` is a $b \times a$ weight matrix plus b biases; it is a convolution with $k = 1$. A LayerNorm over c features learns one scale and one shift per feature (Chapter 16):

$$P_{\text{lin}} = ab + b, \qquad P_{\text{LN}} = 2c. \tag{20.3}$$

BatchNorm also learns $2c$ (scale and shift) and in addition keeps $2c+1$ buffers: a running mean and a running variance per channel and one counter of batches seen.

Worked example 20.1: The rules on a tiny convolution

`Conv2d(2,3,k=2)`. Each of the 3 output channels has a kernel of $2 \times 2 \times 2 = 8$ weights and one bias. Count on paper:

$$\begin{aligned} \text{weights} &= 2 \times 3 \times 2^2 = 2 \times 3 \times 4 = 24, \\ \text{biases} &= 3, \\ P &= 24 + 3 = 27. \end{aligned}$$

The depthwise version with 3 channels, `Conv2d(3,3,k=2,groups=3)`: $3{\times}4{+}3 = 12{+}3 = 15$. The number of heads in an attention layer, the patch size and the batch size never appear in any rule: they change shapes, not parameters.

20.3 The configuration being counted

Table 20.1: The configuration of the 165-band Indradhanu (v0.2.0).

Input bands	165
Compressed width D	32
Stage widths $C_1..C_4$	32, 64, 160, 256
Reduction ratios $R_1..R_4$	8, 4, 2, 1
Blocks per stage	2, 2, 2, 2
Mix-FFN expansion E	4
Decoder width	256
Heads	2, 2, 5, 8 (no effect on the count)

20.4 The front end: the spectral compressor

Chapter 12: a `Conv2d(165,32,k=1)` and a `BatchNorm2d(32)`.

$$\begin{aligned} \text{conv} &= 165 \times 32 \times 1 \times 1 + 32 = 5{,}280 + 32 = 5{,}312, \\ \text{BatchNorm} &= 2 \times 32 = 64, \\ \text{compressor} &= 5{,}312 + 64 = 5{,}376. \end{aligned}$$

20.5 The encoder

20.5.1 The four patch embeddings

Each is a convolution followed by a LayerNorm over its output width (Chapter 13). Stage 1 reads $D = 32$ channels with $k = 4$; stages 2–4 read the previous width with $k = 3$ (Chapter 17).

stage 1:	$32 \times 32 \times 16 + 32 = 16{,}384 + 32 = 16{,}416;$	$+\ 2 \times 32 = 16{,}480$
stage 2:	$32 \times 64 \times 9 + 64 = 18{,}432 + 64 = 18{,}496;$	$+\ 2 \times 64 = 18{,}624$
stage 3:	$64 \times 160 \times 9 + 160 = 92{,}160 + 160 = 92{,}320;$	$+\ 2 \times 160 = 92{,}640$
stage 4:	$160 \times 256 \times 9 + 256 = 368{,}640 + 256 = 368{,}896;$	$+\ 2 \times 256 = 369{,}408$

$$16{,}480 + 18{,}624 + 92{,}640 + 369{,}408 = 497{,}152.$$

20.5.2 One block, as a formula

A block (Chapter 16) of width C with reduction ratio R and expansion 4 contains:

- **ESA** (Chapter 15): four `Linear(C,C)` (query, key, value, output projection), and, only when $R > 1$, a reduction `Conv2d(C,C,k=R,s=R)` and a `LayerNorm(C)`;
- **Mix-FFN**: `Linear(C,4C)`, a depthwise `Conv2d(4C,4C,k=3)`, `Linear(4C,C)`;
- **two LayerNorms** (pre-norm before each sub-layer).

Count each piece with the rules, in symbols:

$$\begin{aligned} \text{q, k, v, out} &= 4\,(C^2 + C) = 4C^2 + 4C, \\ \text{reduction conv + norm} &= (C \cdot C \cdot R^2 + C) + 2C = R^2C^2 + 3C \quad (\text{if } R > 1), \\ \text{fc1} &= C \cdot 4C + 4C = 4C^2 + 4C, \\ \text{depthwise} &= 4C \cdot 9 + 4C = 40C, \\ \text{fc2} &= 4C \cdot C + C = 4C^2 + C, \\ \text{two LayerNorms} &= 2 \times 2C = 4C. \end{aligned}$$

Add them, collecting the C^2 terms ($4 + 4 + 4 = 12$) and the C terms ($4 + 4 + 40 + 1 + 4 = 53$):

$$P_{\text{block}}(C, R) = 12C^2 + 53C + [R > 1]\,(R^2C^2 + 3C), \tag{20.4}$$

where $[R > 1]$ is 1 when $R > 1$ and 0 otherwise. A closed form for the whole block.

Worked example 20.2: The four stages from (20.4)

Do each on paper and compare with the column-by-column count of the course in Table 20.2.

Stage 1, $C = 32$, $R = 8$:

$$12 \times 32^2 = 12 \times 1{,}024 = 12{,}288, \qquad 53 \times 32 = 1{,}696,$$
$$8^2 \times 32^2 = 64 \times 1{,}024 = 65{,}536, \qquad 3 \times 32 = 96,$$
$$P = 12{,}288 + 1{,}696 + 65{,}536 + 96 = 79{,}616; \qquad \times 2 \text{ blocks} = 159{,}232.$$

Stage 2, $C = 64$, $R = 4$:

$$12 \times 4{,}096 = 49{,}152, \qquad 53 \times 64 = 3{,}392,$$
$$16 \times 4{,}096 = 65{,}536, \qquad 3 \times 64 = 192,$$
$$P = 49{,}152 + 3{,}392 + 65{,}536 + 192 = 118{,}272; \qquad \times 2 = 236{,}544.$$

Stage 3, $C = 160$, $R = 2$:

$$12 \times 25{,}600 = 307{,}200, \qquad 53 \times 160 = 8{,}480,$$
$$4 \times 25{,}600 = 102{,}400, \qquad 3 \times 160 = 480,$$
$$P = 307{,}200 + 8{,}480 + 102{,}400 + 480 = 418{,}560; \qquad \times 2 = 837{,}120.$$

Stage 4, $C = 256$, $R = 1$ (no reduction):

$$12 \times 65{,}536 = 786{,}432, \qquad 53 \times 256 = 13{,}568,$$
$$P = 786{,}432 + 13{,}568 = 800{,}000; \qquad \times 2 = 1{,}600{,}000.$$

The round 800,000 is a coincidence of these widths. Blocks in total:

$$159{,}232 + 236{,}544 + 837{,}120 + 1{,}600{,}000 = 2{,}832{,}896.$$

Every stage figure matches `torchinfo` and the CPU build.

Table 20.2: One block per stage, piece by piece (two blocks per stage in the model). These per-module figures are exactly what the CPU build reports for stage 1 (q, k, v and proj 1,056 each, reduction 65,568, reduction norm 64, fc1 4,224, dwconv 1,280, fc2 4,128).

Piece	Stage 1	Stage 2	Stage 3	Stage 4
q, k, v, out	4,224	16,640	103,040	263,168
reduction conv	65,568	65,600	102,560	–
reduction norm	64	128	320	–
fc1	4,224	16,640	103,040	263,168
depthwise conv	1,280	2,560	6,400	10,240
fc2	4,128	16,448	102,560	262,400
two LayerNorms	128	256	640	1,024
one block	79,616	118,272	418,560	800,000

Two things stand out in (20.4). The parameters grow with C^2: doubling the width quadruples the block. And the reduction convolution R^2C^2 is as large as all the rest of the block at stage 1 (65,536 against 14,080): a stride-8 kernel over 32 channels has $8 \times 8 \times 32 \times 32$ weights. Stages 1 and 2 have the same reduction weight count, $64 \times 32^2 = 16 \times 64^2 = 65{,}536$, because RC is 256 in both.

20.5.3 The stage-output norms, and the encoder total

One LayerNorm per stage: $2 \times (32 + 64 + 160 + 256) = 2 \times 512 = 1{,}024$. The encoder:

$$497{,}152 + 2{,}832{,}896 + 1{,}024 = 3{,}331{,}072.$$

20.6 The decoder and the back end

From Chapter 19, with the rule (20.1):

$$\begin{aligned}
\text{projections:}\quad & 32 \times 256 + 256 = 8{,}448 \\
& 64 \times 256 + 256 = 16{,}640 \\
& 160 \times 256 + 256 = 41{,}216 \\
& 256 \times 256 + 256 = 65{,}792 \\
\text{fuse:}\quad & 1{,}024 \times 256 + 256 = 262{,}144 + 256 = 262{,}400 \\
\text{refine 1:}\quad & 256 \times 256 \times 9 + 256 = 589{,}824 + 256 = 590{,}080 \\
\text{refine 2:}\quad & 256 \times 512 \times 9 + 512 = 1{,}179{,}648 + 512 = 1{,}180{,}160
\end{aligned}$$

$$8{,}448 + 16{,}640 + 41{,}216 + 65{,}792 + 262{,}400 + 590{,}080 + 1{,}180{,}160 = 2{,}164{,}736.$$

Pixel shuffle, bilinear interpolation and GELU have no parameters. `Conv2d(32,165,k=1)` is the decompressor:

$$32 \times 165 + 165 = 5{,}280 + 165 = 5{,}445.$$

20.7 The grand total, and the buffers

Table 20.3: Indradhanu (165 bands, $D = 32$), every parameter. Measured by `torchinfo` [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`] and by the CPU build [M: `research/textbook/tools/compute_params_output.txt`]; both report 5,506,629.

Component	Parameters	Share
Spectral compressor	5,376	0.10%
Patch embeddings	497,152	9.03%
Transformer blocks	2,832,896	51.44%
Stage-output norms	1,024	0.02%
Decoder	2,164,736	39.31%
Spectral decompressor	5,445	0.10%
Total	**5,506,629**	100%

Add the column on paper:

$$\begin{aligned}
5{,}376 + 497{,}152 &= 502{,}528 \\
502{,}528 + 2{,}832{,}896 &= 3{,}335{,}424 \\
3{,}335{,}424 + 1{,}024 &= 3{,}336{,}448 \\
3{,}336{,}448 + 2{,}164{,}736 &= 5{,}501{,}184 \\
5{,}501{,}184 + 5{,}445 &= 5{,}506{,}629.
\end{aligned}$$

`torchinfo` reports "Total params: 5,506,629". Exact to the last digit. (Shares: each component divided by 5,506,629; for the blocks $2{,}832{,}896/5{,}506{,}629 \approx 0.5144$.)

Worked example 20.3: Where the manifest's 5,507,354 comes from

The checkpoint manifest says `"params" : 5507354`, as the model breakdown records [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`]. The difference is

$$5{,}507{,}354 - 5{,}506{,}629 = 725.$$

Count the buffers. The normaliser and denormaliser each store a mean and a standard deviation per band:

$$4 \times 165 = 660.$$

The compressor's BatchNorm keeps a running mean and running variance for each of its 32 channels and one batch counter:

$$32 + 32 + 1 = 65.$$

Together $660 + 65 = 725$, and $5{,}506{,}629 + 725 = 5{,}507{,}354$. The CPU build reports 725 buffer numbers. Those 660 normalisation values live inside the checkpoint, which is why you cannot swap normalisation statistics without rebuilding the model (Chapter 5).

20.8 Where the parameters live

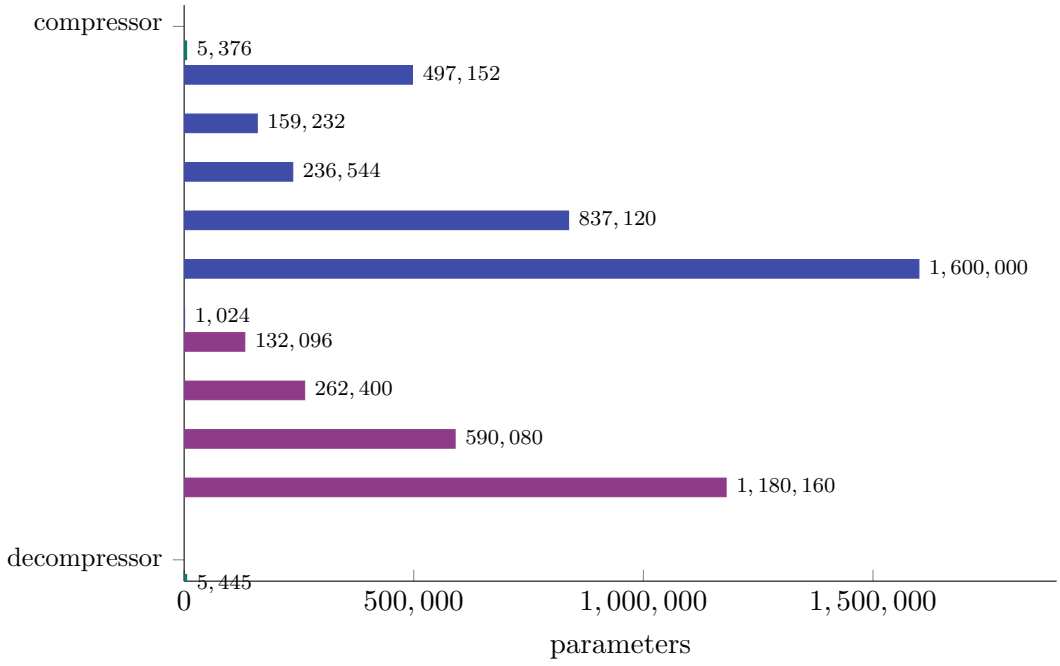


Figure 20.1: Where Indradhanu's 5,506,629 parameters live, drawn to scale from the counts of this chapter. Spectral layers teal, encoder indigo, decoder plum. Stage 4's two blocks (16 tokens) hold more than any other component; the decoder's two refine convolutions hold 1,770,240 between them; the spectral compressor and decompressor are invisible at this scale (0.2% together).

Five lessons follow from the table and the chart.

1. **Stage 4 is the most expensive stage**: 1.6 million parameters, 29% of the model, while processing 16 tokens. Parameters follow width squared, not token count.
2. **The decoder is 39% of the model.** "Lightweight" means light compared with a segmentation decoder running at full resolution with skip connections, not small.

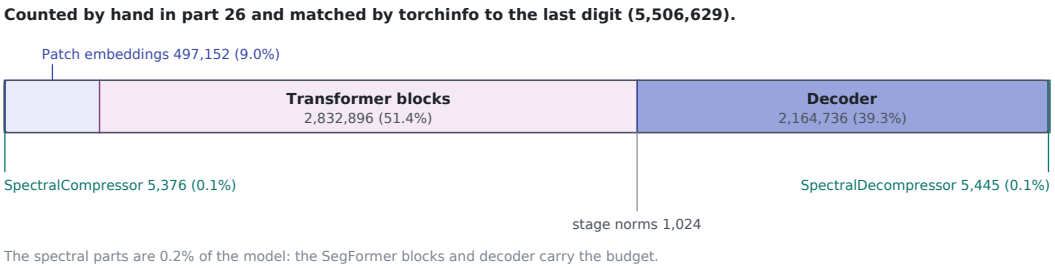


Figure 20.2: The course's view of the same budget by component, bar widths to scale: the blocks (51.4%) and decoder (39.3%) carry almost all of it.

3. **Two layers are 32% of everything.** The refine convolutions hold $590{,}080 + 1{,}180{,}160 = 1{,}770{,}240$, and $1{,}770{,}240/5{,}506{,}629 \approx 0.321$.
4. **The spectral bottleneck costs 0.2% and saves far more.** Without it, stage 1's patch embedding would read 165 channels: $165 \times 32 \times 16 + 32 = 84{,}480 + 32 = 84{,}512$ instead of 16,416, and every activation in the encoder's first convolution would be five times larger.
5. **Parameters are not compute**, which needs its own section.

20.9 Parameters are not compute

A layer's work is its multiply-adds per output position times the number of positions. A layer with few parameters applied at many positions can cost more than a large layer applied at few.

Worked example 20.4: Multiply-adds of three layers

For one 128×128 patch, weights per output position (no biases) times positions:

compressor:	$165 \times 32 = 5{,}280$ per pixel;	$5{,}280 \times 16{,}384 = 86{,}507{,}520$
stage-4 fc1:	$256 \times 1{,}024 = 262{,}144$ per token;	$262{,}144 \times 16 = 4{,}194{,}304$
decoder refine 2:	$256 \times 512 \times 9 = 1{,}179{,}648$;	$1{,}179{,}648 \times 1{,}024 = 1{,}207{,}959{,}552$

The compressor has 50 times fewer parameters than stage 4's fc1 (5,312 against 263,168) but does about 20 times the work, because it runs at 16,384 pixels instead of 16 tokens.

Hooking every convolution and linear layer of the real model during one unmasked forward pass of a 128×128 patch gives the totals in Table 20.4 [C] (attention's own matrix products, at most a few million per block with 16 keys, are not included).

Source check: where the compute goes

The course says of `torchinfo`'s 2.32 G multiply-adds that "most of that happens at stage 1, where the parameters are fewest but the tensors are largest". Counting layer by layer contradicts it: the whole encoder does about 147 million multiply-adds (6%), and the *decoder* does 2.10 billion (87%), because its fuse and refine convolutions run at all 1,024 positions with 256 to 1,024 channels. The two refine convolutions alone are 1.81 billion (Chapter 19). The lesson the course wanted (few parameters at many positions can dominate) is right, and the compressor and decompressor show it; but the layer that dominates is the decoder. The count here is 2.42 billion against `torchinfo`'s 2.32 (`research/model_break_down/05_hyperspectral_segformer_mae.md`, line 293); the 4% gap is in how the two tools count, not in the model, and does not change the ranking.

Table 20.4: Multiply-adds of all convolution and linear layers, one unmasked 128 × 128 patch, counted by `research/textbook/tools/compute_params.py`.

Component	Multiply-adds	Share of work	Share of parameters
compressor	86,507,520	3.58%	0.10%
stage 1 (embed, blocks, norm)	42,270,720	1.75%	3.19%
stage 2	29,229,056	1.21%	4.64%
stage 3	44,318,720	1.83%	16.89%
stage 4	31,358,976	1.30%	35.77%
decoder	2,096,627,712	86.75%	39.31%
decompressor	86,507,520	3.58%	0.10%
total	2,416,820,224	100%	

20.10 Parameters a masked pass never trains

There is one more thing a count reveals, and it follows from Chapter 17. In a masked pass stage 1 has fewer than $H \times W$ tokens, so ESA skips its reduction convolution and norm, and Mix-FFN skips its depthwise convolution. Layers that are skipped get no gradient. Per stage-1 block:

$$\begin{aligned} \text{reduction conv} + \text{reduction norm} + \text{depthwise} &= 65{,}568 + 64 + 1{,}280 = 66{,}912, \\ \text{two blocks} &= 2 \times 66{,}912 = 133{,}824. \end{aligned}$$

That is 133,824/5,506,629 ≈ 2.4% of the model [C]. A backward pass through the real model with a 65% keep mask confirms it: exactly these twelve tensors, 133,824 numbers, have zero gradient (`research/textbook/tools/compute_params_output.txt`).

Source check: idle weights during masked training

Masked pretraining always passes a keep mask (`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, line 231), and masked inference does too, so these 133,824 weights keep their initial values through all of reconstruction training. They are used, untrained, by any *unmasked* pass: `keep_mask=None` in `forward`, and the segmentation head's pass in fine-tuning, which encodes without a keep mask (`spectral_stem_seg_former_mae.py`, `segment`), where they first receive a gradient (Chapter 28). The course does not mention this. It is not a bug in the masked path, which never uses them, but it is a trap for anyone who evaluates a pretrained model unmasked.

20.11 Other models: which figure belongs to which

The courses quote several parameter figures. They belong to different models, and mixing them up is easy.

20.11.1 The v0.1.0 model, $D = 24$

Shrinking D from 32 to 24 changes four layers: the compressor, stage 1's patch embedding (its input width), the decompressor, and the last refine convolution (its output is $16D$).

$$\begin{aligned}
\text{compressor:} &\quad 165 \times 24 + 24 + 2 \times 24 = 3{,}960 + 24 + 48 = 4{,}032 && \text{(was 5,376)}\\
\text{patch embed 1:} &\quad 24 \times 32 \times 16 + 32 + 64 = 12{,}288 + 96 = 12{,}384 && \text{(was 16,480)}\\
\text{decompressor:} &\quad 24 \times 165 + 165 = 3{,}960 + 165 = 4{,}125 && \text{(was 5,445)}\\
\text{refine 2:} &\quad 256 \times 384 \times 9 + 384 = 884{,}736 + 384 = 885{,}120 && \text{(was 1,180,160)}
\end{aligned}$$

The differences: $-1{,}344$, $-4{,}096$, $-1{,}320$, $-295{,}040$, together $-301{,}800$. So

$$5{,}506{,}629 - 301{,}800 = 5{,}204{,}829$$

parameters, plus $660 + (24 + 24 + 1) = 709$ buffers, 5,205,538 in all: the v0.1.0 manifest's figure, and the CPU build's. Ninety-eight percent of the saving is the decoder's last convolution: shrinking D mostly shrinks the decoder.

20.11.2 The spectral-stem model (Indradhanu V2, FT04)

From Chapter 23 on, the fixed 165-band compressor is replaced by a wavelength-aware stem, and fine-tuning adds a segmentation head. Table 20.5 gives that model's split; its encoder and reconstruction decoder are *the same classes at the same widths* as above, which is why 3,331,072 and 2,164,736 reappear.

Table 20.5: The spectral-stem model with the reduced-attention stem and a 5-class segmentation head, as configured for FT04 [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]. Counts from the CPU build [M: `research/textbook/tools/compute_params_output.txt`]; the course quotes the same split (IW67, IW40, IW47).

Component	Where it is taught	Parameters
reduced-attention compressor (`spectral_in`)	Chapter 24	14,721
BatchNorm after the stem		64
encoder	Chapter 17, this chapter	3,331,072
reconstruction decoder	Chapter 19, this chapter	2,164,736
wavelength decoder (`spectral_out`)	Chapter 24	4,257
segmentation head (width 128, 5 classes)	Chapter 28	371,536
total		5,886,386

Two checks. Without the segmentation head the reconstruction model has $5{,}886{,}386 - 371{,}536 = 5{,}514{,}850$ parameters, only 8,221 more than the 165-band model: the new stem and decoder (18,978 together, plus the BatchNorm's 64) replace a compressor and decompressor of 10,821. And the head is the decoder class of Section 20.6 at width 128 with $5 \times 16 = 80$ output channels:

$$\begin{aligned}
\text{projections:} &\quad (32 + 64 + 160 + 256) \times 128 + 4 \times 128 = 65{,}536 + 512 = 66{,}048\\
\text{fuse:} &\quad 512 \times 128 + 128 = 65{,}536 + 128 = 65{,}664\\
\text{refine 1:} &\quad 128 \times 128 \times 9 + 128 = 147{,}456 + 128 = 147{,}584\\
\text{refine 2:} &\quad 128 \times 80 \times 9 + 80 = 92{,}160 + 80 = 92{,}240\\
\text{head:} &\quad 66{,}048 + 65{,}664 + 147{,}584 + 92{,}240 = 371{,}536.
\end{aligned}$$

None of these counts contains the band count 202: the stem and the wavelength decoder work for any band list, which is their point (Chapter 22).

Values that will drift

As of 2026-10-29 the shipped Indradhanu V2 (v2.1.0) is *Indradhanu Heavy* (FTH3 epoch 5), a wider model with a progressive decoder: stage widths 96, 192, 480, 768, decoder width 512, $D = 64$, segmentation width 256 [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/config.json`]. Building it on CPU gives 35,315,314 parameters [C]: encoder 29,580,288, progressive decoder 4,279,616, segmentation head 1,431,120, stem 17,825 + 128 + 6,337 (`research/textbook/tools/compute_params_output.txt`). Without the segmentation head the total is $35{,}315{,}314 - 1{,}431{,}120 = 33{,}884{,}194$, the figure Chapter 43 and the pretraining README quote. Chapter 43 tells that story; every count in this chapter is for the standard widths.

20.12 Common confusions

Does the number of heads change the count?
No. Multi-head attention splits the same four $C \times C$ matrices into groups; no weights are added. The rules never use it.

Why is the count independent of patch size?
Convolutions and linear layers reuse the same weights at every position. A bigger patch means more positions, more work and more memory, not more parameters.

Are the normalisation statistics parameters?
No, buffers: saved in the file, never updated by gradient. They explain the 725 between 5,506,629 and the manifest's 5,507,354.

The decoder is called lightweight; is it small?
It is 39% of the parameters and 87% of the convolution work. Lightweight compared with full-resolution segmentation decoders, not small.

Is 5.5 million big?
Small by modern standards (ResNet-50 has about 25 million). It is deliberately compact: it runs in a worker container, sometimes on a CPU.

Which total is "Indradhanu's"?
It depends on the model: 5,506,629 for the 165-band v0.2.0, 5,204,829 for v0.1.0, 5,886,386 for the spectral-stem FT04 model with its head, 35,315,314 for Heavy. Quote the model with the number.

20.13 Check yourself

Exercise 20.1. How many parameters does `Conv2d(64,128,k=3)` have? And the depthwise `Conv2d(64,64,k=3,groups=64)`? How many times fewer?

Exercise 20.2. Use (20.4) to count one block of width $C = 128$ with $R = 4$, and check it against a piece-by-piece count.

Exercise 20.3. A colleague proposes three blocks at stage 4 instead of two. How many parameters does that add, and what fraction of the 165-band model is that?

Exercise 20.4. Compute the 165-band model's total if the decoder width were 128 instead of 256 (everything else unchanged). Which decoder layer shrinks most?

Exercise 20.5. Reconcile 5,506,629 with 5,507,354 from memory of the model's structure, without looking: which buffers, how many of each?

Exercise 20.6. The compressor has 5,376 parameters and the decoder's fuse convolution 262,400. For one 128×128 patch, which does more multiply-adds, and by what factor?

Next: Chapter 21, Chakshu: the same machine for one thermal band.

Chapter 21

Chakshu: the same machine for one thermal band

The one thing this chapter teaches

Chakshu, the product's thermal model, is the SegFormer masked autoencoder of the last nine chapters with one input band of surface temperature in degrees Celsius, widths shrunk to 16, 32, 64 and 96, one block per stage and a feed-forward expansion of 2: 406,496 trainable numbers that you can count by hand, trained to fill in 70% hidden tokens of 256-pixel Landsat patches, and scored by the absolute temperature error; its one serious weakness is outside the network, in the normalisation path that was meant to let it read HotSat-1 and does not.

You have now built Indradhanu from its parts. This chapter uses every one of those parts again, on a simpler input, and asks you to redo the arithmetic at new sizes. Nothing here is new machinery; what is new is the thermal domain, the training recipe and three places where the code and its documentation part company. It is also the chapter where you can check yourself: if a shape or a count below surprises you, the chapter that explains it is named beside it.

Provenance tags abbreviate paths. `components/` is `app/foundation_models/components/`; the shipped checkpoint is `segformer_mae_v0.8.0_epoch495.pt` in the stack's `/models/segformer_mae/` volume, which is not in the repository. Its configuration is also in the repository as `configs/seg_former_exp_8.json`, which matches the checkpoint's saved configuration key for key. Every count and shape tagged [C] comes from `research/textbook/tools/compute_chakshu.py`, which loads that checkpoint (copied from the volume on 2026-10-31) into the real classes; its output is saved beside it as `compute_chakshu_output.txt`.

Analogy

A town has a thermometer on every street corner. One morning you cover seven out of every ten and ask an apprentice to say what each covered one reads, from the uncovered ones around it. On an ordinary morning the apprentice is right to a fraction of a degree, because temperature varies smoothly across a town. If a warehouse is burning, the covered thermometer beside it reads 20 degrees more than anything its neighbours suggest, and the apprentice's guess is 20 degrees wrong. That error is the alarm. Two details of the analogy will matter later: the apprentice must never see the covered thermometer itself, and if the fire is large enough to reach an uncovered thermometer too, the apprentice sees part of the fire and the alarm is quieter.

21.1 Why a thermal model, and why this one

Two of the product's five sensors are *thermal* (Chapters 2 and 3): Landsat 9's band 10 and SatVu's HotSat-1. Each records one number per pixel, related to how much heat the ground radiates, near 10.9 μm (the product files thermal scenes under the wavelength `10900.0` nm, `backend/allotrope/action_types/_anomaly_scoring_run.py`). A thermal anomaly is something hotter or colder than its surroundings say it should be: a fire, a furnace, a cooling-water plume, a vessel's engine.

Definition 21.1 (Land surface temperature)**.** The *land surface temperature* of a pixel is the temperature of the ground's skin, as inferred from the radiation it emits. The product stores it in degrees Celsius for Landsat 9.

The product has two ways to score a thermal scene. The classical `thermal_grx` detector (Chapter 7) measures how far each pixel's temperature sits from the scene's mean, in standard deviations: a hot pixel in a cold field scores high, but so does every ordinary hot roof in a city. A reconstruction model asks a sharper question, "is this pixel hot *for where it is*?", because its prediction comes from the pixel's own neighbourhood (Chapter 9).

Which reconstruction model? Chapter 11 ran the five convolutional autoencoders over a smooth synthetic field with the product's own inference: their median background error ran from 0.58 °C (Drashta) to 10.9 °C (Pratibimba, which never trained with hidden pixels). Three of them cannot tell a hidden pixel from a real one at 0 °C. The SegFormer design avoids that: hidden tokens are *deleted* before the encoder (Chapter 18), so there is no stand-in value to misread, and the decoder's pixel shuffle gives every pixel its own output (Chapter 19), so a one-pixel hot spot is not averaged into its neighbours. On the same synthetic field Chakshu's background error is 0.019 °C (Section 21.8).

Why so small. Indradhanu's encoder is 32, 64, 160, 256 channels wide, with two blocks per stage. Chakshu's is 16, 32, 64, 96 with one. A thermal pixel carries one number, not 165, so there is far less to model; and the project's own record says the small model won. The manifest lists the larger SegFormer-B0 run (v0.1.0, 3,714,086 numbers) as an alternative with validation loss 6.59877, against 0.2565 for the small one, and notes that the small one "currently outperforms it 25× on val loss" [M: the `/models/segformer_mae/current.json` manifest]. Read that comparison with care: the large run had trained for 10 epochs, the small one for 495, and the two losses were measured on different patch sizes (128 and 256 pixels). It shows that the large run was unfinished, not that the large architecture is worse.

21.2 The input: one band, in Celsius

21.2.1 From Landsat's numbers to degrees

Landsat 9's Level-2 surface-temperature product stores band 10 as a 16-bit integer, a *digital number* (DN). The builder (`app/utils/dataset_builder/landsat_dataset_builder.py`) converts it with the published scale and offset (`app/utils/data_transformations/l2sp_dn_to_temperature_transformer.py`):

$$T_{\mathrm{K}} = 0.00341802 \cdot \mathrm{DN} + 149.0, \qquad T = T_{\mathrm{K}} - 273.15. \tag{21.1}$$

The unit is fixed to Celsius in the builder; it is not an option.

Worked example 21.1: Two Landsat pixels into degrees

An ordinary pixel, DN = 43,500:

$$0.00341802 \times 43{,}500 = 148.68387 \quad \text{scale}$$
$$148.68387 + 149.0 = 297.68387\ \text{K} \quad \text{offset}$$
$$297.68387 - 273.15 = 24.53\ ^\circ\text{C} \quad \text{Kelvin to Celsius}$$

A hot pixel, DN = 52,000: $0.00341802 \times 52{,}000 = 177.73704$; $+149.0 = 326.73704\,\text{K}$; $-273.15 = 53.59\,^\circ\text{C}$.

21.2.2 The normalisation the checkpoint carries

Chakshu normalises its input with one mean and one spread, stored in the checkpoint as buffers (Chapter 5): $\mu_T = 24.5756$ and $\sigma_T = 13.5744$ [M: checkpoint buffers `normalize.mean`, `normalize.std`]. The same two numbers are in `app/constants/thermal_pixel_consts.json`, measured over 10,000 Landsat patches of 64 pixels (25,413,600 valid pixels) [M: `app/constants/thermal_pixel_consts.json`].

$$z = \frac{T - \mu_T}{\sigma_T}. \tag{21.2}$$

Worked example 21.2: Normalising the two pixels, and an invalid one

$$\text{ordinary} : (24.5339 - 24.5756)/13.5744 = -0.0417/13.5744 = -0.0031$$
$$\text{hot} : (53.5870 - 24.5756)/13.5744 = 29.0114/13.5744 = 2.1372$$
$$\text{invalid, zeroed to 0 °C} : (0 - 24.5756)/13.5744 = -24.5756/13.5744 = -1.8104$$

The model zeroes invalid pixels *before* normalising (`x = x * mask` in `components/seg_former_mae.py`), so a cloud pixel arrives as -1.81: a consistent value that the network learns to read as "no measurement" (Chapter 5) [C].

Source check: Celsius near 25, not Kelvin near 290

Three places describe Chakshu's input wrongly. The comment above the per-scene override in `backend/allotrope/action_types/_anomaly_scoring_run.py` says the models assume "Kelvin / reflectance" (the same comment, ten lines later, says Landsat is "already in Celsius"). `PixelStatsOverride`'s docstring (`app/models/training/inference_config.py`) and `docs/lld/pixel-stats.md` say a model "fitted on Celsius (~290 +/- 10)", and the LLD and the course (part AD10) work the HotSat example with a mean of 290 and a spread of 10. 290 is a Kelvin-sized number. The code and the checkpoint agree on Celsius: the vendable's field is documented as "surface temperatures in celsius" (`app/models/dataset/vendables.py`), the builder converts to Celsius, and the buffers are 24.58 ± 13.57. In Kelvin the mean would be $24.5756 + 273.15 = 297.73$. The comment in `components/spatial_decoder.py` ("e.g. surface temperatures in Kelvin") is the same slip in the convolutional family.

setting	Chakshu (v0.8.0, served)	class default (SegFormer-B0)
`in_channels`	1	1
`embed_dims`	16, 32, 64, 96	32, 64, 160, 256
`num_heads`	1, 2, 4, 8	1, 2, 5, 8
channels per head	16, 16, 16, 12	32, 32, 32, 32
`reduction_ratios`	8, 4, 2, 1	8, 4, 2, 1
`num_blocks`	1, 1, 1, 1	2, 2, 2, 2
`expansion_ratio` E	2	4
`decoder_dim`	96	256
`drop_rate`	0.3 (training)	0.3
`mask_ratio` ρ	0.7	0.5
`trim_fraction` τ	0.05	0.0
patch size in training	256	–

Table 21.1: Chakshu against the defaults of `SegFormerMAEConfig` (`app/models/training/training_config.py`). Source: [M: `configs / seg _ former _ exp _ 8 . json`], identical to the checkpoint's saved configuration [C].

21.3 The configuration

Source check: the served model is not SegFormer-B0

`research/model_break_down/04_segformer_mae_thermal.md` draws its architecture diagram with stage widths 32, 64, 160, 256 and two blocks per stage, and says "The default config (used by both checkpoints)" is SegFormer-B0. Only the unused alternative, v0.1.0, has that configuration. The served v0.8.0 is the shrunk one of Table 21.1; the same document's checkpoint table, further down, says so correctly. The book's earlier remark that Chakshu "shares almost all its code with Indradhanu" (Chapter 2) is true of the classes, not of the sizes.

How does the product build this model? The worker never opens the checkpoint's configuration. The resolver rebuilds it from the manifest (`_model_config_from_manifest` in `backend/allotrope/foundation_models/resolver.py`): widths from the manifest's `encoder_dims` string `"[16, 32, 64, 96]"`, decoder width from `decoder_dim`, and *fixed* heads $[1, 2, 4, 8]$, ratios $[8, 4, 2, 1]$, one block per stage and expansion 2, hard-coded for `segformer_mae`. Those happen to equal the checkpoint's, and the strict load succeeds [C]. A future thermal checkpoint with two blocks per stage would build the wrong model and fail to load.

21.4 One forward pass at full size

Follow one masked pass, as in training, with patches of 256×256 pixels, all valid, and the training mask ratio 0.7. Training batches hold 64 patches; take $B = 8$, the product's inference batch, since the batch size changes no shape except the first axis.

21.4.1 Stage 1: the patch embedding and the deletion

The stage-1 patch embedding is a convolution with kernel 4, stride 4 and no padding (Chapter 13) from 1 channel to 16. A 256×256 patch becomes a 64×64 grid:

$$\left\lfloor \frac{256 + 2 \cdot 0 - 4}{4} \right\rfloor + 1 = \lfloor 63 \rfloor + 1 = 64 \qquad \text{output-size formula, Chapter 10}$$

$64 \times 64 = 4{,}096$ tokens of 16 numbers.

Each token is a weighted sum of its own $4 \times 4 = 16$ temperatures and nothing else: the weights form a 16×16 matrix, one row per output channel. With one input band, stage 1 is exactly a linear map from a block's 16 temperatures to 16 features, followed by LayerNorm.

Worked example 21.3: How many tokens a training pass hides

The hiding count rounds down (Chapter 18):

$4{,}096 \times 0.7 = 2{,}867.2$	tokens to hide, unrounded
$\lfloor 2{,}867.2 \rfloor = 2{,}867$	the code's `.long()` rounds down
$4{,}096 - 2{,}867 = 1{,}229$	tokens the encoder sees
$2{,}867 \times 16 = 45{,}872$	hidden pixels, the loss's positions

Running the real encoder gives 2,867 hidden and 1,229 kept per item [C].

The encoder sees 1,229 of 4,096 tokens, 30%. Indradhanu hides 65% (Chapter 18); Chakshu hides more because a temperature field is smoother than a reflectance cube, so the same context predicts more of it. That is a design judgement recorded only in the configuration's value; no document gives the reason.

21.4.2 The four stages

stage	step	shape after the step	note
input	normalise, $z = (T - \mu_T)/\sigma_T$	`(8, 1, 256, 256)`	worked example 21.2
1	patch embed $k4\,s4\,p0$, $1 \to 16$	`(8, 4096, 16)`	64×64 grid
	remove hidden tokens	`(8, 1229, 16)`	2,867 deleted
	1 block: attention, Mix-FFN ($16 \to 32 \to 16$)	`(8, 1229, 16)`	full attention; no depthwise conv
	LayerNorm, restore, reshape	`(8, 16, 64, 64)` = F1	zeros at hidden tokens
2	patch embed $k3\,s2\,p1$, $16 \to 32$	`(8, 1024, 32)`	32×32 grid
	1 block, $R = 4$, 2 heads; FFN $32 \to 64 \to 32$	`(8, 1024, 32)`	64 keys
	LayerNorm, reshape	`(8, 32, 32, 32)` = F2	
3	patch embed, $32 \to 64$; 1 block, $R = 2$, 4 heads	`(8, 256, 64)`	64 keys
	reshape	`(8, 64, 16, 16)` = F3	
4	patch embed, $64 \to 96$; 1 block, $R = 1$, 8 heads	`(8, 64, 96)`	64 keys
	reshape	`(8, 96, 8, 8)` = F4	

Table 21.2: Chakshu's encoder on a masked training pass, $B = 8$, 256×256, $\rho = 0.7$. Forward hooks on the real modules record every shape [C].

Two rows deserve a second look. At stage 1 the token count after deletion, 1,229, is not 64×64, so efficient self-attention cannot rebuild its grid and attends over all kept tokens with no reduction, and Mix-FFN skips its depthwise convolution (Chapters 15 and 16, the `can_reduce` test). And at stage 4 each head is $96/8 = 12$ channels wide, not a power of two: harmless, since nothing in attention needs one.

Worked example 21.4: Every stage attends to 64 keys

Efficient self-attention shrinks the key grid by R on each side (Chapter 15). At 256×256:

$$\text{stage } 1 : (64/8)^2 = 8^2 = 64 \text{ keys for } 4{,}096 \text{ queries}$$
$$\text{stage } 2 : (32/4)^2 = 8^2 = 64 \text{ keys for } 1{,}024 \text{ queries}$$
$$\text{stage } 3 : (16/2)^2 = 8^2 = 64 \text{ keys for } 256 \text{ queries}$$
$$\text{stage } 4 : (8/1)^2 = 64 \text{ keys for } 64 \text{ queries}$$

Unmasked, stage 1's score table has $4{,}096 \times 64 = 262{,}144$ entries per head. Masked, it falls back to full attention among the kept tokens: in training $1{,}229^2 = 1{,}510{,}441$, and at inference, where half the tokens are kept, $2{,}048^2 = 4{,}194{,}304$, sixteen times the unmasked table [C]. The fallback is the expensive case.

21.4.3 The decoder

The decoder is `SegFormerDecoder` (Chapter 19) at width 96, writing $1 \times 4 \times 4 = 16$ channels for the pixel shuffle:

step	operation	shape
project	1×1 conv of each F_i to 96 channels	F1 `(8, 96, 64, 64)`, …, F4 `(8, 96, 8, 8)`
upsample	bilinear, to 64×64	four of `(8, 96, 64, 64)`
concatenate	along channels	`(8, 384, 64, 64)`
fuse	1×1 conv $384 \to 96$, GELU	`(8, 96, 64, 64)`
refine	3×3 conv $96 \to 96$, GELU	`(8, 96, 64, 64)`
sub-pixel	3×3 conv $96 \to 16$, no activation	`(8, 16, 64, 64)`
pixel shuffle	factor 4	`(8, 1, 256, 256)`
denormalise	$T = z\,\sigma_T + \mu_T$	`(8, 1, 256, 256)`, in °C

Every shape was recorded by forward hooks [C]. Figure 21.1 draws the whole pass.

21.5 The parameters, counted by hand

Chapter 20 counted a block with expansion 4. Chakshu's expansion is 2, so first generalise the block formula to any expansion E, then use it. The counting rules are those of Section 20.2: a linear layer from a to b has $ab+b$, a convolution $c_{\text{in}}c_{\text{out}}k^2 + c_{\text{out}}$, a depthwise convolution $ck^2 + c$, a LayerNorm $2c$.

21.5.1 A block of width C, reduction R, expansion E

$$\begin{aligned}
\text{norm1} &: 2C \\
\text{q, k, v, proj} &: 4\,(C \cdot C + C) = 4C^2 + 4C \\
\text{reduction conv } (R > 1) &: C \cdot C \cdot R^2 + C = R^2C^2 + C \\
\text{reduction norm } (R > 1) &: 2C \\
\text{norm2} &: 2C \\
\text{fc1 } (C \to EC) &: C \cdot EC + EC = EC^2 + EC \\
\text{depthwise } 3 \times 3 \text{ on } EC &: EC \cdot 9 + EC = 10EC \\
\text{fc2 } (EC \to C) &: EC \cdot C + C = EC^2 + C
\end{aligned}$$

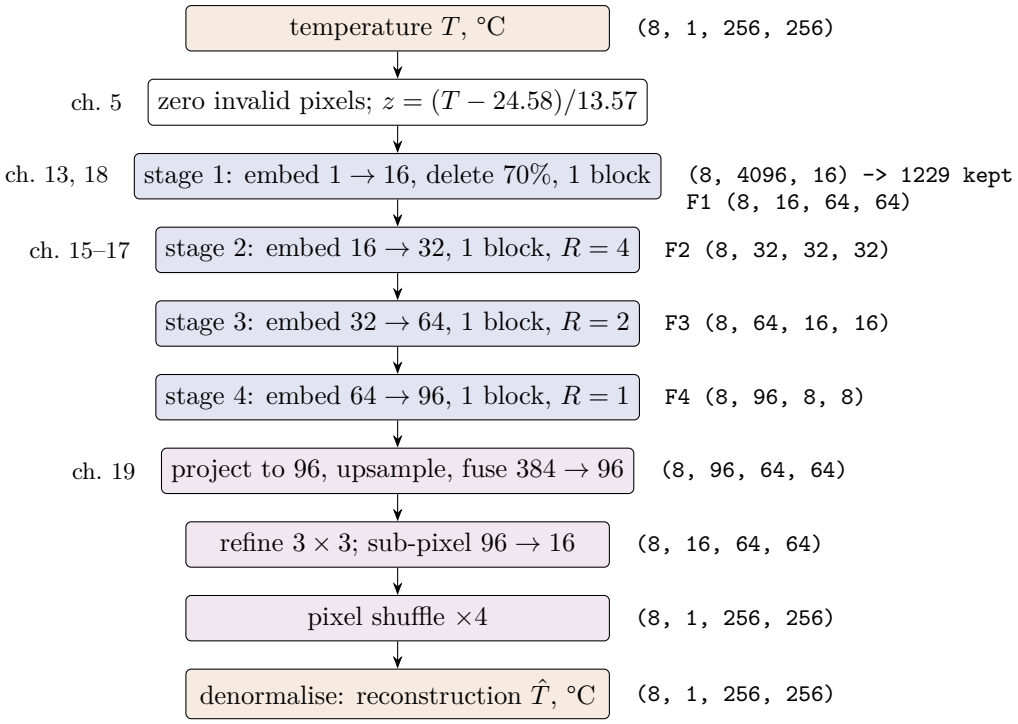


Figure 21.1: Chakshu's forward pass on a masked batch of eight 256×256 Landsat patches, with the shape after each box (recorded by forward hooks on the real model [C]). Blue: encoder; plum: decoder; amber: the two ends, in degrees Celsius. The chapters on the left explain each box.

Collect the C^2 terms: $4 + R^2 + E + E = 4 + R^2 + 2E$. Collect the C terms: $2 + 4 + 1 + 2 + 2 + E + 10E + 1 = 12 + 11E$. So

$$P_{\text{block}}(C, R, E) = (4 + 2E + R^2)\, C^2 + (12 + 11E)\, C \quad (R > 1), \tag{21.3}$$

and for $R = 1$ drop the reduction convolution and its norm: $(4 + 2E)\, C^2 + (9 + 11E)\, C$.

Check against Chapter 20. With $E = 4$ the coefficients are $4 + 8 + R^2 = 12 + R^2$ and $12 + 44 = 56$. Equation (20.4) gives $12C^2 + 53C + R^2C^2 + 3C = (12 + R^2)C^2 + 56C$ for $R > 1$: the two agree. With $E = 2$ the coefficients are $8 + R^2$ and 34 (31 for $R = 1$).

Worked example 21.5: Chakshu's four blocks

$C = 16, R = 8 : (8 + 64) \times 256 + 34 \times 16 = 72 \times 256 + 544 = 18{,}432 + 544 = 18{,}976$
$C = 32, R = 4 : (8 + 16) \times 1{,}024 + 34 \times 32 = 24{,}576 + 1{,}088 = 25{,}664$
$C = 64, R = 2 : (8 + 4) \times 4{,}096 + 34 \times 64 = 49{,}152 + 2{,}176 = 51{,}328$
$C = 96, R = 1 : 8 \times 9{,}216 + 31 \times 96 = 73{,}728 + 2{,}976 = 76{,}704$

Line by line for stage 1, to see where the weight is: norm1 32; q, k, v, proj $4 \times 272 = 1{,}088$; reduction $16 \times 16 \times 64 + 16 = 16{,}400$; reduction norm 32; norm2 32; fc1 $16 \times 32 + 32 = 544$; depthwise $32 \times 9 + 32 = 320$; fc2 $32 \times 16 + 16 = 528$. Sum: $32 + 1{,}088 + 16{,}400 + 32 + 32 + 544 + 320 + 528 = 18{,}976$. The 8×8 reduction convolution is $16{,}400/18{,}976 = 86\%$ of the block. All four match the built model [C].

21.5.2 The rest of the encoder

Patch embeddings, each a convolution plus a LayerNorm of $2c_{\text{out}}$:

$$\begin{aligned}
\text{stage 1 } (k=4) &: 1\times 16\times 16 + 16 + 32 = 256 + 48 = 304\\
\text{stage 2 } (k=3) &: 16\times 32\times 9 + 32 + 64 = 4{,}608 + 96 = 4{,}704\\
\text{stage 3} &: 32\times 64\times 9 + 64 + 128 = 18{,}432 + 192 = 18{,}624\\
\text{stage 4} &: 64\times 96\times 9 + 96 + 192 = 55{,}296 + 288 = 55{,}584\\
\text{sum} &: 304 + 4{,}704 = 5{,}008;\ +18{,}624 = 23{,}632;\ +55{,}584 = 79{,}216
\end{aligned}$$

Stage-end LayerNorms: $2\times(16+32+64+96) = 2\times 208 = 416$. Blocks: $18{,}976+25{,}664 = 44{,}640$; $+51{,}328 = 95{,}968$; $+76{,}704 = 172{,}672$. Encoder:

$$172{,}672 + 79{,}216 + 416 = 252{,}304.$$

21.5.3 The decoder and the total

$$\begin{aligned}
\text{projections} &: (16+32+64+96)\times 96 + 4\times 96 = 208\times 96 + 384 = 19{,}968 + 384 = 20{,}352\\
\text{fuse} &: 384\times 96 + 96 = 36{,}864 + 96 = 36{,}960\\
\text{refine } 3\times 3 &: 96\times 96\times 9 + 96 = 82{,}944 + 96 = 83{,}040\\
\text{sub-pixel } 3\times 3 &: 96\times 16\times 9 + 16 = 13{,}824 + 16 = 13{,}840\\
\text{decoder} &: 20{,}352 + 36{,}960 = 57{,}312;\ +83{,}040 = 140{,}352;\ +13{,}840 = 154{,}192
\end{aligned}$$

$$252{,}304 + 154{,}192 = 406{,}496 \text{ trainable numbers.} \tag{21.4}$$

`sum(p.numel() for p in model.parameters())` on the loaded checkpoint returns 406,496 [C].

Source check: 406,500 counts four buffers

The manifest, `docs/04-models.md` ("406 k params") and the course (AD10: "406,500 parameters") quote 406,500. That is the number of values in the checkpoint's `state_dict`, which also holds four buffers: the normalisation mean and spread, once in `PixelNormalize` and once in `PixelDenormalize`. $406{,}496 + 4 = 406{,}500$. The same distinction explained 5,506,629 against 5,507,354 for Indradhanu (Chapter 9).

21.5.4 The unused alternative, and why it cannot load

The manifest's alternative, v0.1.0, is SegFormer-B0 with 3,714,086 values in its `state_dict`. Today's code cannot load it. Its stage-1 kernel is 7×7 (`(32, 1, 7, 7)`), from before the change to non-overlapping 4×4 tokens (Chapter 13), and its decoder has a BatchNorm after the fuse and a single final projection where today's has the two refine convolutions. A strict load reports four missing keys, seven unexpected ones and one size mismatch [C]. Selecting it in the product would fail at the load.

21.6 Training: 70% hidden, the trimmed L1

The trainer is `app/foundation_models/trainers/segformer_mae_trainer.py`. Each step:

1. builds the validity mask as `pure_validity * predicted_cloud_mask`: the provider's valid pixels and the cloud mask of the scene's own temperature mixture model (Section 4.9);

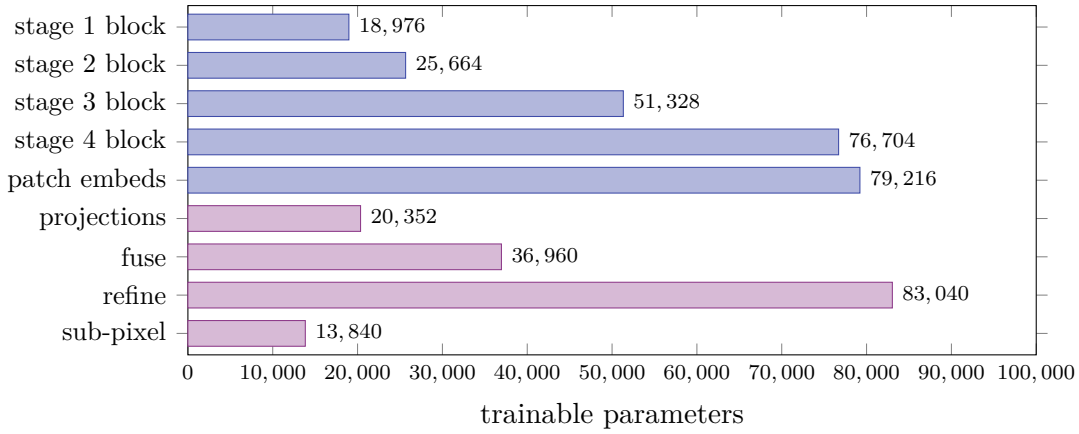


Figure 21.2: Where Chakshu's 406,496 parameters live, drawn to scale from worked example 21.5 and the sums above (blue encoder, plum decoder). The largest single layer is the decoder's refine convolution, 20% of the model; the stage-4 patch embedding (55,584) alone outweighs the first three blocks together (stage-end norms, 416, not drawn).

2. drops patches with less than 40% valid pixels, the rule Indradhanu's trainers share (Section 25.4 covers it later);
3. turns pixel validity into token validity (a token is valid if more than half of its 16 pixels are) and hides $\lfloor 0.7 \times \text{valid tokens} \rfloor$ of the valid ones at random (Chapter 18);
4. runs the model with that keep mask and takes the absolute error at every hidden, valid pixel;
5. trims, then averages.

21.6.1 The trimmed L1

Definition 21.2 (Trimmed mean)**.** The *trimmed mean* of n numbers at trim fraction τ sorts them, keeps the smallest $\lfloor n(1-\tau) \rfloor$ and averages those. Chakshu's loss is the trimmed mean of the per-pixel absolute errors $|\hat{T} - T|$ over the hidden, valid pixels of the whole batch, with $\tau = 0.05$.

The code says why: "The highest-error pixels are assumed to be anomalies - dropping them from the training signal teaches the model to reconstruct only normal background patterns." The training archive has no anomaly labels, so any fire in it would otherwise be a target the model is graded on reconstructing, which is exactly what you do not want it to learn (Section 9.6).

Worked example 21.6: Trimming 20 errors at 5%

Twenty hidden pixels have errors (°C) of 0.10, 0.20, 0.15, 0.30, 0.05, 0.25, 0.20, 0.10, 0.15, 0.10, 0.20, 0.30, 0.25, 0.10, 0.15, 0.20, 0.05, 0.10, 0.20, and one fire pixel at 9.00 [A].

$$\begin{aligned} \text{keep} &: \lfloor 20 \times (1 - 0.05) \rfloor = \lfloor 19.0 \rfloor = 19 \text{ smallest} \\ \text{sum of all 20} &: 3.15 + 9.00 = 12.15 \quad \text{(the nineteen small ones sum to 3.15)} \\ \text{plain mean} &: 12.15/20 = 0.6075 \\ \text{trimmed mean} &: 3.15/19 = 0.1658 \end{aligned}$$

To get 3.15: $2 \times 0.05 = 0.10$; $5 \times 0.10 = 0.50$; $3 \times 0.15 = 0.45$; $5 \times 0.20 = 1.00$; $2 \times 0.25 = 0.50$; $2 \times 0.30 = 0.60$; and $0.10 + 0.50 + 0.45 + 1.00 + 0.50 + 0.60 = 3.15$. One fire pixel had made the loss almost four times larger. After the trim it contributes nothing, and its gradient is

zero: the model is not pushed to reconstruct it. The script gives the same two means [C].

At full size the trim is large in absolute terms. A batch of 64 fully valid patches has $64 \times 45{,}872 = 2{,}935{,}808$ hidden pixels; the trim keeps $\lfloor 2{,}935{,}808 \times 0.95 \rfloor = 2{,}789{,}017$ and drops 146,791 [C]. The 5% is taken over the whole batch, not per patch, so a patch with a large fire can lose more than 5% of its pixels and a quiet one none.

21.6.2 Optimiser, schedule, data

The optimiser is Adam at a learning rate of 10^{-4}, lowered along half a cosine wave over 500 epochs to 10^{-6} (both are derived in Chapter 26); the configuration's `warmup_epochs: 5` is read by nothing in `app/abstract_classes/foundation_trainer.py`, which builds `CosineAnnealingLR` with only `T_max`, `eta_min` and `last_epoch`. An epoch is a budget of 30,000 patches of 256×256 from the Landsat archive, $30{,}000/64 = 468.75$ batches, and validation draws 1,000 [M: `configs/seg_former_exp_8.json`]. By the cosine formula, epoch 495 runs at $10^{-6} + (10^{-4} - 10^{-6}) \frac{1+\cos(495\pi/500)}{2} = 1.02 \times 10^{-6}$ [C]: the shipped checkpoint was taken when the learning rate was within 2.5% of its floor.

21.6.3 Validation, and why train loss is lower

Validation (`compute_validation_loss`) masks a random half of the valid tokens, then the complementary half, keeps each pixel's prediction from the pass that hid it, and averages $|\hat{T} - T|$ over every valid pixel with no trim. So the two numbers in the checkpoint measure different things [M: checkpoint `train_loss`, `avg_val_loss`]:

train loss 0.1730 °C (70% hidden, trimmed) validation loss 0.2565 °C (50% hidden twice, untrimmed).

The validation number is the more honest one: a mean absolute temperature error of about a quarter of a degree on held-out patches.

21.7 Inference on a scene

The inferencer is `app/foundation_models/inferencers/segformer_mae_inferencer.py`; Chapter 30 covers the same machinery for Indradhanu. The product's settings for Chakshu come from the resolver's capability entry and `InferenceConfig`'s defaults:

setting	value
patch, stride	256, 128 (half-overlapping tiles)
batch	8 patches
masking	checkerboard of single stage-1 tokens, two complementary passes
erosion of the validity mask	kernel 15: a 7-pixel margin at every invalid edge
skip a patch if valid share is below	0.1
uncovered pixels	take the input value, so their residual is exactly 0

Worked example 21.7: Tiles over two scene sizes

Along an axis of length W with patch $P = 256$ and stride $S = 128$, tiles start every S pixels while a whole tile fits, and one more is snapped back to end exactly at the edge, so the count is $\lceil (W - P)/S \rceil + 1$ when $(W - P)/S$ is not whole (Equation (30.4) derives it). For

$W = 1{,}000$:

$$(1{,}000 - 256)/128 = 744/128 = 5.8125, \qquad \lceil 5.8125 \rceil + 1 = 6 + 1 = 7,$$

starts 0, 128, …, 640, and the snapped last one at $1{,}000 - 256 = 744$. A $1{,}000 \times 1{,}000$ scene needs $7 \times 7 = 49$ tiles and $2 \times 49 = 98$ forward passes. A full Landsat scene of about $7{,}801 \times 7{,}681$ pixels [A] needs $60 \times 60 = 3{,}600$ tiles and 7,200 passes. The patch planner returns 49 and 3,600 [C].

Source check: two training-inference mismatches, already known

`docs/09-known-issues.md` (item 3) records both, and the source confirms them. Inference defaults to the checkerboard while training and validation use random masks; and the inferencer's `MIN_VALID_FRACTION = 0.1` sits under a comment saying it "Matches training", where training discards below 0.4. A patch between 10% and 40% valid is therefore reconstructed by a model that never trained on one.

21.8 From reconstruction to score

The resolver offers Chakshu two scoring methods, `L1` (the default) and `MSE`. With one band both are simple (`app/utils/anomaly_detection/scoring.py`, `compute_score`):

$$s_{\mathrm{L1}}(p) = |T(p) - \hat{T}(p)|, \qquad s_{\mathrm{MSE}}(p) = (T(p) - \hat{T}(p))^2, \tag{21.5}$$

in °C and °C^2. There is no spectral angle (one number has no direction) and no residual Mahalanobis; there is no segmentation head, so nothing is excluded after scoring. A `cloud_mask` action upstream supplies a keep mask, and pixels outside it score 0.

21.8.1 A synthetic test, with the real weights

To see the score at work, the script builds a smooth 256×256 field, $T = 25 + 3\sin(x/40) + 2\cos(y/30)$ °C, and plants hot spots at 45 °C in copies of it [A]. It runs the product's inferencer on each with the shipped weights and the product's checkerboard. These are synthetic numbers, not a measurement on Landsat scenes.

field	median $\lvert T - \hat{T} \rvert$, background	at the spot	largest
plain	0.019 °C	–	0.092 °C
1 pixel at 45 °C	0.019 °C	20.96 °C	20.96 °C
4×4 spot inside one token	0.019 °C	21.05 °C (centre)	21.17 °C
3×3 spot across four tokens	0.019 °C	7.59 °C (centre)	12.60 °C

[C]. The background is reconstructed to two hundredths of a degree. A one-pixel or token-sized hot spot is reconstructed as background: at the token-sized spot the model predicts 23.95 °C where the field reads 45, and the residual, 21.05 °C, is the whole of the anomaly. The 3×3 spot is the surprise.

21.8.2 Why a smaller spot can score lower

Stage-1 tokens are 4×4 blocks starting at multiples of 4. The 3×3 spot occupies rows and columns 126 to 128: rows 126 and 127 lie in token row 31 (pixels 124–127), row 128 in token

row 32 (128–131), and the same for columns. So the spot straddles a 2×2 group of tokens. The checkerboard colours a token by (row + column) mod 2:

$$(31, 31) : 62 \bmod 2 = 0, \qquad (31, 32) : 63 \bmod 2 = 1,$$
$$(32, 31) : 63 \bmod 2 = 1, \qquad (32, 32) : 64 \bmod 2 = 0.$$

In the pass that hides colour 0, tokens $(31, 32)$ and $(32, 31)$ are visible, and each holds two of the spot's hot pixels. The model sees part of the spot beside the hidden part and predicts warm. In the other pass the roles swap. So every hot pixel is predicted from context that contains other pixels of the same spot: the spot helps reconstruct itself, and the residual drops from about 21 °C to 7.6 °C at the centre. Figure 21.3 draws it.

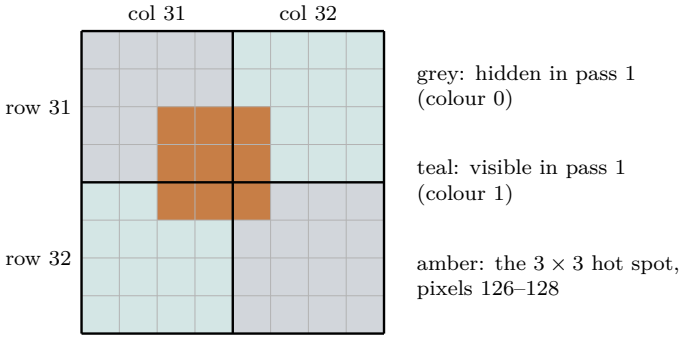


Figure 21.3: An 8×8-pixel window around the 3×3 spot of Section 21.8.2. Thick lines are stage-1 token borders. In pass 1 the grey tokens are hidden, but the two teal tokens still show four of the spot's nine hot pixels to the encoder, so the hidden hot pixels are predicted warm; pass 2 swaps the colours.

A hot object larger than one pixel can reconstruct itself through the visible tokens it reaches into; Chakshu's residual is largest for objects that fit inside one 4×4 token.

This is the analogy's large fire, and it is the same leakage Indradhanu has. It is a property of the two-pass design, not a bug in the code, but it means a residual's size depends on where an object falls on the token grid. Larger checkerboard cells (`checkerboard_cell_size`, Chapter 30) would hide 2×2 groups of tokens together; the product does not use them for Chakshu, and no measurement of that option on thermal scenes exists.

21.9 The HotSat-1 units caveat

HotSat-1's L2 Visual product carries uncalibrated digital numbers, "relative radiance differences within a scene", around 5,000 with a spread of about 400 (the builder's and the override's docstrings, `app/utils/dataset_builder/hotsat_dataset_builder.py`, `app/models/training/inference_config.py`). The product scores HotSat-1 with Chakshu by default (`backend/allotrope/action_types/anomaly_scoring.py`), and its plan is a *per-scene override*: replace μ_T and σ_T by the scene's own mean and spread, so the input arrives centred with unit spread and scores become comparable within the scene, not across scenes.

Worked example 21.8: A HotSat number under the baked statistics

$$z = (5{,}000 - 24.5756)/13.5744 = 4{,}975.4244/13.5744 = 366.53$$

Training inputs sat within a few units of 0. The course and `docs/lld/pixel-stats.md` give 471.0 for this, from a mean of 290 and spread of 10 (Section 21.2); with the real buffers it is 366.5 [C]. Under the override with mean 5,000 and spread 400, the same pixel gives

$(5{,}000 - 5{,}000)/400 = 0$ and a pixel at 5,400 gives $400/400 = 1$.

That is the plan. The code defeats it twice, and the second time is new.

First: the units never arrive. The override is built only when the vendable's `units` begins with `"DN_"`. `HotSatDatasetBuilder` passes `units="DN_14bit_relative"` to `VendableThermalDataset`, which declares no such field, so Pydantic discards it, and `getattr(vendable, "units", None)` is `None` (`docs/09-known-issues.md`, item 7; Chapters 4 and 5).

Second: the checkpoint overwrites the override. Suppose the units did arrive. The handler builds `PixelStatsOverride(mean=[5000], std=[...])`, and the inferencer constructs `SegFormerMAE` with those numbers in its `PixelNormalize` buffers. Then `FoundationInferencer._load_weights` calls `load_state_dict(ckpt["model_state_dict"])`, and the checkpoint's state dictionary contains `normalize.mean`, `normalize.std`, `denormalize.mean` and `denormalize.std`: it writes 24.5756 and 13.5744 back over the override. The script shows it directly: a model built with mean 5,000.03 has mean 24.5756 after the load, and an inferencer constructed with the override holds [24.5756], [13.5744] [C]. `docs/lld/pixel-stats.md`'s invariant, "The override wins outright", holds in `resolve_pixel_stats` and is undone two calls later.

Source check: the override cannot work for any checkpoint with normalisation buffers

Measured on CPU with the shipped weights (`tools/compute_chakshu.py`): a synthetic HotSat-like field (5,000 DN, spread 193) reconstructs to a mean of 100.3 DN, a median residual of 4,899.5 DN everywhere, with and without the override; the two runs are identical [C]. Every pixel looks equally anomalous, which is the failure the LLD describes as the reason the override exists. The same load-after-build order applies to every model whose checkpoint carries `normalize.*` buffers: Pratibimba, Antardhana, Tirohita and Drashta (Chapter 11). A third, smaller fault: the handler's check that the override's length matches `in_channels` reads `getattr(m.model_config, "in_channels", None)`, but `model_config` is a dictionary, so the attribute is never found and the check never runs. Fixing the second fault means applying the override after the weights load, or dropping the `normalize.*` keys from the state dictionary before loading; this book does not change code.

There is no HotSat-1 scene in the local database (`docs/proposals/aigc-evaluation-submission.md`, row R3), so none of this has been seen on real data. The diagnostics would still say `normalization_mode: "baked"`, which is true, and nothing would warn that the input was 366 standard deviations from anything the model learned.

21.10 Where Chakshu sits in the product

resolver field (`_CAPABILITIES`, key `segformer_mae`)	value
`foundation_model_name`	`segformer_mae` → `SegFormerMAEInferencer`
`scoring_methods`, default	L1, MSE; L1
`default_patch_size`, `default_stride`	256, 128
`default_batch_size`	8
`valid_patch_sizes`	64, 128, 256 (each divisible by 32)
`pixel_stats_relpath`	`constants/thermal_pixel_consts.json`
`band_grid`, `exclude_classes`	common165 (not checked for thermal), none

The scoring action's sensor defaults pick Chakshu for `landsat9` and `hotsat1` (`backend/allotrope/action_types/anomaly_scoring.py`). For a thermal scene the action reads the onboarding vendable directly (no band filter), takes `normalized_thermal_cube` as one

band, attaches a `cloud_mask` output's keep mask if there is one, and writes `models/Chakshu/anomaly_score.tif`, the reconstruction and their previews. The thermal chain is `cloud_mask` → `anomaly_scoring` → `anomaly_detection_prep` → export, with the georeference read back from the Landsat GeoTIFF at export time (Chapter 32). That chain has not been run end to end on the local stack: no thermal scene is loaded (`docs/proposals/aigc-evaluation-submission.md`).

Values that will drift

Read 2026-10-31 from the `/models/segformer_mae/` volume: current checkpoint `segformer_mae_v0.8.0_epoch495.pt`, epoch 495, train loss 0.1730, validation loss 0.2565 (256-pixel patches), 406,496 trainable numbers (406,500 with buffers); normalisation 24.5756 ± 13.5744 °C; alternative v0.1.0 (epoch 10, validation 6.59877 on 128-pixel patches), not loadable by the current code. Synthetic-field residuals in Section 21.8 were computed with these weights [C].

21.11 Common confusions

Chakshu reads Kelvin.
It reads Celsius. The Landsat builder converts to Celsius and the buffers are 24.58 and 13.57. "Kelvin" and "290" in comments and in the pixel-stats LLD are wrong.

Chakshu is SegFormer-B0 with one input channel.
The classes are, the sizes are not: widths 16 to 96, one block per stage, expansion 2, decoder width 96. Only the unused v0.1.0 run is B0.

406,500 parameters.
406,496 parameters and 4 buffers. The 406,500 is the state dictionary's size.

The small model beat the large one 25 times over.
The large run had 10 epochs against 495, and their losses were on different patch sizes. It shows the large run was unfinished.

Train loss 0.173 below validation 0.257 means overfitting.
The two measure different things: training trims the worst 5% and hides 70%; validation hides half twice and trims nothing.

The per-scene override makes Chakshu safe for HotSat-1.
It is never built (the units field is dropped), and if it were built the checkpoint's buffers would overwrite it when the weights load.

A bigger hot spot always scores higher.
Not with two-pass masking: a spot that straddles token borders shows part of itself to the encoder in both passes and reconstructs itself partly; a spot inside one token is hidden whole.

With one band, L1 and MSE rank pixels differently.
They cannot: the square is increasing for non-negative numbers, so $|r_1| < |r_2|$ exactly when $r_1^2 < r_2^2$, and every threshold on one is a threshold on the other. They differ only in how the map looks.

21.12 Check yourself

Exercise 21.1. Use Equation (21.3) to count a Chakshu stage-2 block ($C = 32$, $R = 4$, $E = 2$), and the same block with $E = 4$. How much does doubling the expansion add, and why is it exactly $2EC^2$-shaped?

Exercise 21.2. At a patch size of 128 instead of 256, give the token grid, the number of tokens and the number of attention keys at each stage, and the number of tokens a training pass hides at $\rho = 0.7$. Does the parameter count change?

Exercise 21.3. A Landsat pixel has DN = 48,000. Convert it to °C with Equation (21.1) and normalise it with Equation (21.2).

Exercise 21.4. Ten hidden pixels have errors 0.1, 0.3, 0.2, 0.1, 0.4, 0.2, 0.1, 0.3, 0.2 and 5.0 °C. Compute the plain mean and the trimmed mean at $\tau = 0.05$. Then at $\tau = 0.1$. What does the floor do at $\tau = 0.05$ with only ten numbers?

Exercise 21.5. A scene is $1{,}000 \times 600$ pixels. How many Chakshu tiles does the product run, and how many forward passes?

Exercise 21.6. A HotSat-1 pixel reads 4,600 DN. Compute its z under the baked statistics, and under a per-scene override of mean 5,000 and spread 400. Which z does the product actually use today, and why?

Exercise 21.7. A 2×2 hot spot covers pixels 127–128 in both rows and columns. Which stage-1 tokens does it touch, what are their checkerboard colours, and would you expect its residual to be closer to the one-token spot's or the 3×3 spot's?

Exercise 21.8. The resolver hard-codes heads, ratios, blocks and expansion for `segformer_mae`. Name the one manifest field that changes the model's width and the one that changes the decoder, and say what would happen if a two-blocks-per-stage thermal checkpoint were published with the same manifest format.

Next: Chapter 22, Why the compressor is a ceiling: cross-attention and a spectrum as tokens.

Part IV

The spectral stem

Chapter 22

Why the compressor is a ceiling: cross-attention and a spectrum as tokens

The one thing this chapter teaches

Today's spectral compressor mixes the bands of every pixel on Earth with one fixed matrix, so it cannot adapt to a pixel, cannot leave a broken band out, and only accepts exactly 165 band positions; cross-attention over *band tokens that carry their wavelength* removes all three limits at once.

This chapter opens Part IV, the spectral stem. Everything in Parts I to III still holds: the SegFormer encoder of Chapter 17, the masking of Chapter 18 and the decoder of Chapter 19 are untouched. What changes is the thin layer at each end of the model that turns a pixel's spectrum into 32 numbers and back. The chapter leans on three earlier ones: Chapter 6 for matrices, rank and dot products, Chapter 12 for the compressor itself, and Chapter 14 for attention.

The chapter has three movements. Sections 22.2 to 22.4 take the compressor apart and prove, with a rank argument and two pieces of hand arithmetic, that its limits are structural and not a matter of training longer. Section 22.5 introduces cross-attention, the one mechanism that answers all three limits. Section 22.6 turns a single pixel's spectrum into the tokens that cross-attention reads, and derives the wavelength encoding the repository actually uses, worked for two wavelengths.

22.1 The compressor you already know

Recall from Chapter 12 what the compressor does to one pixel. A pixel arrives as a vector $\mathbf{x} \in \mathbb{R}^N$ of N normalised band values (on the 165-band grid of Chapter 3, $N = 165$). A *parameter* is a number the model learns during training; the compressor's parameters are a weight matrix $\mathbf{W} \in \mathbb{R}^{D \times N}$ and a bias vector $\mathbf{b} \in \mathbb{R}^D$. Its output for that pixel is

$$\mathbf{z} = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \mathbf{z} \in \mathbb{R}^D, \; D = 32, \tag{22.1}$$

followed by a BatchNorm that rescales each of the 32 outputs. In the code this is `nn.Conv2d(165, 32, kernel_size=1)` in `app/foundation_models/components/spectral_compressor.py`. A *1×1 convolution* is a convolution whose kernel covers one pixel only, so at every position of the image it computes exactly Equation (22.1) with the same $\mathbf{W}$ and $\mathbf{b}$. The source says so in its own docstring:

> "A learnable 1x1 Convolution to reduce spectral channels – analogous to MNF but trained end-to-end for the reconstruction objective."

Chapter 12 used a four-band pixel that we keep using here, because it is small enough to do on paper:

$$\mathbf{x} = \begin{bmatrix} 1.2 \\ 1.0 \\ -0.4 \\ -0.6 \end{bmatrix}, \qquad \mathbf{W} = \begin{bmatrix} 0.5 & 0.5 & 0 & 0 \\ 0 & 0 & 0.5 & 0.5 \end{bmatrix}, \qquad \mathbf{b} = \mathbf{0}. \tag{22.2}$$

Row 0 of $\mathbf{W}$ is output 0's *recipe*: half of band 0 plus half of band 1. Row 1 is output 1's recipe: half of band 2 plus half of band 3. Multiply row by column:

$$\begin{aligned} z_0 &= 0.5 \times 1.2 + 0.5 \times 1.0 + 0 \times (-0.4) + 0 \times (-0.6) \\ &= 0.60 + 0.50 + 0 + 0 \\ &= 1.10, \\ z_1 &= 0 \times 1.2 + 0 \times 1.0 + 0.5 \times (-0.4) + 0.5 \times (-0.6) \\ &= 0 + 0 - 0.20 - 0.30 \\ &= -0.50. \end{aligned}$$

So $\mathbf{z} = [1.10, -0.50]$. Hold on to one fact about $\mathbf{W}$: **it is the same matrix for every pixel.** The forest pixel, the desert pixel, the cloud-shadowed pixel and the gas-flare pixel are all mixed by the same two recipes. Training chooses $\mathbf{W}$ once; after that, nothing about an individual pixel changes it.

Analogy

Think of a mixing desk with 48 input tracks and a stereo output. The compressor is a desk whose faders were set once, at the factory, and then glued in place. Every song that will ever be played through it gets the same mix. A pop song and a string quartet come out balanced by the same settings. This chapter is about unglueing the faders: letting each song (each pixel) set its own mix, letting a broken track be muted completely, and letting the desk accept a different number of tracks. The analogy returns in Chapter 24, where a cheaper rival only adds a row of adjustable faders in front of the glued desk.

22.2 Consequence 1: one recipe for every pixel, proved with rank

"Fixed" sounds like a description. It is in fact a theorem with a measurable consequence: a fixed linear map must throw away the same spectral directions for every pixel. To see it you need two facts about matrices.

22.2.1 Linearity: the recipe cannot depend on the pixel

A map f is *linear* if $f(a\mathbf{x} + c\mathbf{x}') = af(\mathbf{x}) + cf(\mathbf{x}')$ for all vectors and numbers a, c. Ignoring the bias, Equation (22.1) is linear: multiplying a matrix by a sum is the sum of the products. A practical way to say what "fixed recipe" means is to ask how the output changes when one input band changes slightly. That rate of change is the derivative $\partial z_c / \partial x_b$. Differentiate $z_c = \sum_b W_{cb} x_b + b_c$ with respect to x_b: every term except the b-th is a constant, so

$$\frac{\partial z_c}{\partial x_b} = W_{cb}. \tag{22.3}$$

The right-hand side contains no $\mathbf{x}$. Whatever pixel you are at, nudging band b by 0.01 moves output c by $0.01\,W_{cb}$. **That is the precise meaning of "one recipe for every pixel": the sensitivity of every output to every band is a constant.** An adaptive compressor would have a derivative that changes with $\mathbf{x}$; Chapter 23 shows attention has exactly that property.

22.2.2 Rank and the null space

The *rank* of a matrix is the number of linearly independent rows it has (equivalently, of columns; Chapter 6). The *null space* of $\mathbf{W}$ is the set of vectors $\mathbf{n}$ with $\mathbf{Wn} = \mathbf{0}$: directions in band space that the matrix sends to nothing. The two are tied by the *rank–nullity theorem*: for a matrix with N columns,

$$\operatorname{rank}(\mathbf{W}) + \dim(\text{null space of } \mathbf{W}) = N. \tag{22.4}$$

Worked example 22.1: Rank and null space of the toy compressor

Take $\mathbf{W}$ from Equation (22.2). Do this on paper.
Rank. The two rows are $[0.5, 0.5, 0, 0]$ and $[0, 0, 0.5, 0.5]$. Neither is a multiple of the other (the first has zeros where the second does not), so they are independent and $\operatorname{rank}(\mathbf{W}) = 2$.
Null space. Look for $\mathbf{n} = [n_0, n_1, n_2, n_3]$ with $\mathbf{Wn} = \mathbf{0}$. Row 0 gives

$$\begin{aligned} 0.5n_0 + 0.5n_1 &= 0 \\ n_1 &= -n_0. \end{aligned}$$

Row 1 gives

$$\begin{aligned} 0.5n_2 + 0.5n_3 &= 0 \\ n_3 &= -n_2. \end{aligned}$$

So every null vector has the form $n_0[1, -1, 0, 0] + n_2[0, 0, 1, -1]$: two free numbers, dimension 2. Check Equation (22.4): $2 + 2 = 4 = N$.
What the null space hides. Add 0.2 times the first null vector to the pixel:

$$\begin{aligned} \mathbf{x}' &= \mathbf{x} + 0.2\,[1, -1, 0, 0] \\ &= [1.2 + 0.2,\ 1.0 - 0.2,\ -0.4,\ -0.6] \\ &= [1.4,\ 0.8,\ -0.4,\ -0.6]. \end{aligned}$$

Compress it:

$$\begin{aligned} z'_0 &= 0.5 \times 1.4 + 0.5 \times 0.8 = 0.70 + 0.40 = 1.10, \\ z'_1 &= 0.5 \times (-0.4) + 0.5 \times (-0.6) = -0.20 - 0.30 = -0.50. \end{aligned}$$

The output is identical to the original pixel's. The difference between $\mathbf{x}$ and $\mathbf{x}'$ (band 0 up, band 1 down: a change of *slope* between the two bands) is invisible to the compressor.

Now scale up. The real compressor has $N = 165$ columns and $D = 32$ rows, so its rank is at most 32 (a matrix cannot have more independent rows than it has rows). By Equation (22.4),

$$\begin{aligned} \dim(\text{null space}) &= 165 - \operatorname{rank}(\mathbf{W}) \\ &\geq 165 - 32 \\ &= 133. \end{aligned}$$

There are at least 133 independent spectral directions that the compressor maps to exactly zero, and they are the same 133 directions for every pixel. If the interesting

signal of a mineral survey (say a narrow dip near 2200 nm) happens to lie largely in that null space, every pixel loses it, whether the pixel is bare rock (where the dip matters) or forest (where it does not). Training can rotate which 133 directions are discarded, to suit the training set as a whole. It cannot choose differently for different pixels.

A fixed 32×165 matrix discards at least 133 spectral directions, and it discards the same ones everywhere.

The repository's configuration history shows the symptom. `configs/hyperspectral_segformer_exp_2.json` carries the note "SAM stuck above 0.3 after epoch 30 – Compressor bottleneck too narrow (try compressed_channels=32)", (the same note sits in `exp_1`), and version 0.2.0 raised D from 24 to 32 [M: `configs/hyperspectral_segformer_exp_2.json`]. In the language above, that reduced the null space from at least $165 - 24 = 141$ dimensions to at least 133. It widened the pipe. It did not make the pipe smarter.

22.3 Consequence 2: a broken band cannot be left out

Real sensors lose bands: a dead detector column, a saturated pixel, a band that is no-data in one scene. The band pipeline of Chapter 5 fills such holes with a substitute value before the model sees them. The compressor then mixes the substitute in as if it were a measurement.

22.3.1 The error of one bad band, in general

Suppose band b should hold the true value x_b but holds a fill value f instead. Write the broken pixel as the true pixel plus an error that lives only in slot b:

$$\mathbf{x}_{\text{broken}} = \mathbf{x} + (f - x_b)\,\mathbf{e}_b,$$

where $\mathbf{e}_b$ is the vector with a 1 in slot b and 0 elsewhere. Compress, using linearity to split the product:

$$\begin{aligned} \mathbf{W}\mathbf{x}_{\text{broken}} &= \mathbf{W}\mathbf{x} + (f - x_b)\,\mathbf{W}\mathbf{e}_b \\ &= \mathbf{z} + (f - x_b)\,\mathbf{w}_{:,b}, \end{aligned} \tag{22.5}$$

because multiplying a matrix by $\mathbf{e}_b$ picks out its b-th column, written $\mathbf{w}_{:,b}$. So the error in the output is the band's column of weights times the size of the bad value. Every output c whose recipe gives band b a non-zero weight W_{cb} is contaminated, by exactly $W_{cb}(f - x_b)$.

Worked example 22.2: A dead band filled with zero

Band 2 of the toy pixel is dead and arrives filled with $f = 0.0$. Its true value was $x_2 = -0.4$. Do this on paper.
Directly:

$$\begin{aligned} \mathbf{x}_{\text{broken}} &= [1.2,\ 1.0,\ 0.0,\ -0.6], \\ z_1 &= 0 \times 1.2 + 0 \times 1.0 + 0.5 \times 0.0 + 0.5 \times (-0.6) \\ &= 0 + 0 + 0 - 0.30 \\ &= -0.30. \end{aligned}$$

By Equation (22.5): column 2 of $\mathbf{W}$ is $[0, 0.5]$, and $f - x_2 = 0.0 - (-0.4) = 0.4$, so the output error is

$$0.4 \times [0, 0.5] = [0,\ 0.20],$$

and z_1 moves from -0.50 to $-0.50 + 0.20 = -0.30$, agreeing with the direct calculation. The error, 0.20, is

$$\frac{0.20}{|-0.50|} = 0.40 = 40\%$$

of the true output. Output 0 is untouched only because its recipe happens to give band 2 a zero weight.

A fixed weighted sum cannot leave an input out: whatever sits in that slot gets multiplied by its weight and added. Setting the fill value to zero is not "no information"; it is a claim that the band measured zero. In a trained model almost every one of the 32 recipes gives every band a non-zero weight, so one dead band contaminates all 32 outputs. The cost is small for one band in one pixel and systematic across a scene with a striped detector.

22.4 Consequence 3: tied to 165 positions

`Conv2d(165,32,kernel_size=1)` has a weight for "input 0", "input 1", …, "input 164". It does not have a weight for "460 nm" or "470 nm". **The layer knows positions, not wavelengths.** The matrix **W** has exactly 165 columns, and a vector with 202 entries cannot be multiplied by it at all: the shapes do not match.

That is the real reason the 165-band common grid of Chapter 3 exists. Every sensor must be resampled onto exactly those 165 positions before this layer can accept it, and the sensors do not agree on their own:

Table 22.1: Bands as delivered by each source (counts from the anomaly-detection course part 04 and `docs/03-data-pipeline.md`).

source	bands as delivered
SpectralEarth (Chapter 27)	202 [M: `app/constants/spectralearth_bands.json`]
EnMAP	224 [M: `docs/03-data-pipeline.md`]
PRISMA	239 [M: `docs/03-data-pipeline.md`]
AVIRIS-NG	about 425 [M: `docs/training/anomaly-detection/04-the-sensors.md`]

Resampling is not free. Interpolating onto a 10 nm grid blurs narrow features and invents values between real measurements. Worse, the grid's gaps (920–970, 1140–1150, 1350–1450 and 1800–1950 nm, Chapter 3) are invisible to the layer: input 45 and input 46 might be 10 nm apart or 100 nm apart, and the layer cannot tell, because nothing in it refers to a wavelength.

22.4.1 From three problems to three requirements

Turn each consequence into a requirement on its replacement:

Chapter 14 already showed you a mechanism whose mixing weights depend on the data: attention. The weights there came out of a softmax of dot products between queries and keys, so they change whenever the keys change. The rest of this chapter adapts attention to the job.

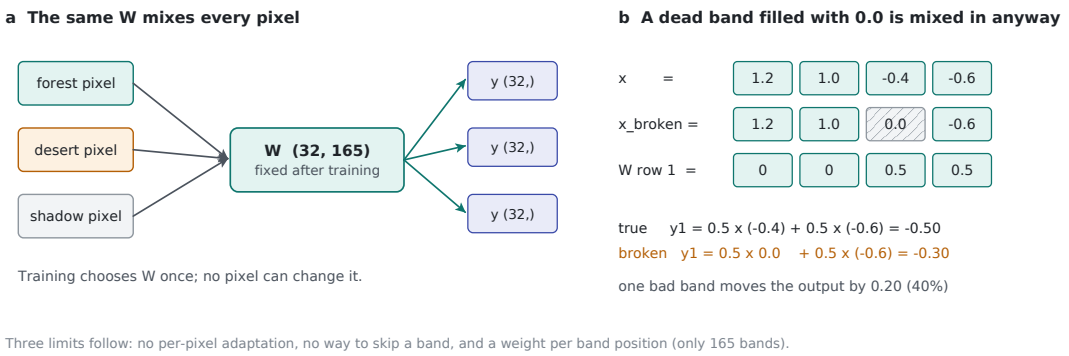


Figure 22.1: The ceiling of a fixed compressor. (a) Every pixel – forest, desert or shadow – is mixed by the same trained matrix $\mathbf{W}$. (b) A dead band filled with 0.0 still receives its weight: with the toy numbers the output moves from -0.50 to -0.30 (worked example 22.2).

Table 22.2: What a replacement for the compressor has to do.

problem	requirement
one recipe for every pixel (constant derivative, Equation (22.3))	the mixing weights must **depend on the pixel's own spectrum**
a broken band is mixed in (Equation (22.5))	a band must be **skippable**, per pixel, with weight exactly zero
tied to 165 positions	bands must be described by **wavelength**, not position, so any count works

22.5 Cross-attention: the same four steps, someone else asking

22.5.1 Attention in one paragraph

In Chapter 14, every *token* (a vector standing for one item, there a patch of the image) produced three vectors from itself: a *query* (what am I looking for?), a *key* (what do I offer?) and a *value* (what do I hand over if chosen?). Each query was scored against every key by a dot product, the scores were scaled by $1/\sqrt{d_k}$ where d_k is the length of a key, a softmax turned each row of scores into weights that sum to 1, and each token's output was the weighted average of the values. Because queries, keys and values all came from the same set of tokens, it is called *self-attention*. `EfficientSelfAttention` in `app/foundation_models/components/efficient_self_attention.py` is exactly that, over spatial tokens (Chapter 15).

22.5.2 The one change

Definition 22.1 (Cross-attention)**.** Let $\mathbf{Q} \in \mathbb{R}^{M \times d_k}$ hold M queries, one per row, and let $\mathbf{K} \in \mathbb{R}^{N \times d_k}$ and $\mathbf{V} \in \mathbb{R}^{N \times d_v}$ hold the keys and values of N tokens. *Cross-attention* computes

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}, \tag{22.6}$$

with the softmax applied to each row separately, where the queries come from a *different source* than the keys and values.

Everything in Equation (22.6) is the formula of Chapter 14. Only the origin of $\mathbf{Q}$ is new:

	queries from	keys and values from
self-attention	the N tokens	the same N tokens
cross-attention	M outside askers	the N tokens

Analogy

Self-attention is a meeting where everyone listens to everyone and updates their own view: N people in, N revised opinions out. Cross-attention is a questionnaire: a fixed list of M questions is put to a crowd of N people, and you get back exactly M answers, each a blend of what the crowd said. Add more people to the crowd and you still get M answers.

That last sentence is the property the spectral stem needs. A pixel's spectrum will be the crowd (one token per band); the compressor's 32 outputs will be the answers. Change the number of bands and the number of answers stays fixed.

22.5.3 Shapes, step by step

Work through Equation (22.6) one product at a time, using the rule for a matrix product: an $(m \times n)$ matrix times an $(n \times p)$ matrix is defined only when the inner sizes agree ($n = n$), and gives an $(m \times p)$ matrix.

1. *Transpose the keys.* $\mathbf{K}$ is $(N \times d_k)$, so $\mathbf{K}^\top$ is $(d_k \times N)$.
2. *Scores.* $\mathbf{QK}^\top$ is $(M \times d_k)(d_k \times N)$: the inner sizes are both d_k, so the product exists and is $(M \times N)$. Entry (i, j) is the dot product of query i with key j.
3. *Scale.* Dividing by $\sqrt{d_k}$ changes no shape: still $(M \times N)$.
4. *Softmax per row.* Still $(M \times N)$, now each row sums to 1. Call this matrix $\mathbf{A}$, the *attention weights*.
5. *Blend the values.* $\mathbf{AV}$ is $(M \times N)(N \times d_v)$: inner sizes both N, result $(M \times d_v)$.

N appears only as an inner size, and inner sizes vanish from a product. The output has M rows, one per query, whatever N is. In self-attention $M = N$, so the output count follows the input count; in cross-attention M is chosen by the designer and stays fixed.

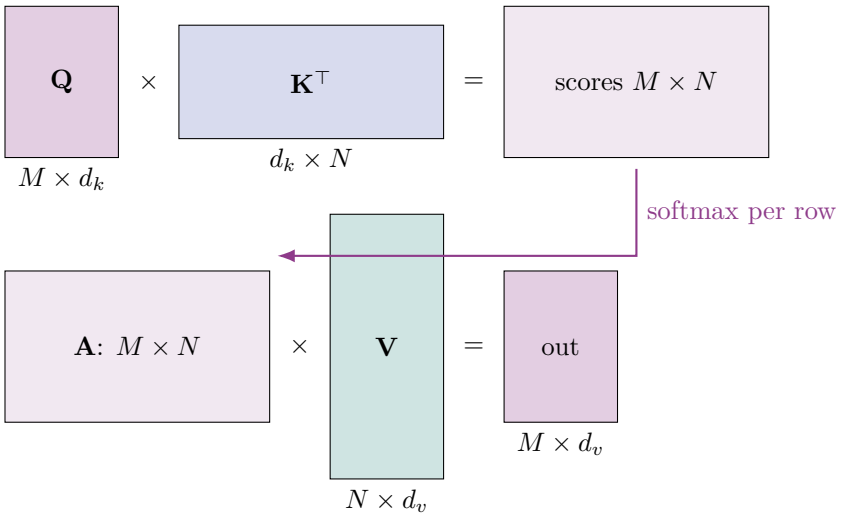


Figure 22.2: Shapes in cross-attention. The token count N is an inner size in both products, so it never reaches the output: M queries always give M answers.

22.5.4 Cross-attention by hand

Before going further, do one cross-attention completely by hand. The example is tiny on purpose: one query, three tokens, two features. Every number below fits on a page of paper.

Worked example 22.3: One query reading three tokens

One query, three key/value tokens, $d_k = 2$, one head. As in Chapter 14, pretend the projection matrices are the identity, so the vectors below are used as they stand. Do this on paper.

query (from the asker):	$\mathbf{q} = [1, 0]$	
keys (from the tokens):	$\mathbf{k}_1 = [2, 0]$	value $v_1 = 10$
	$\mathbf{k}_2 = [0, 1]$	value $v_2 = 20$
	$\mathbf{k}_3 = [1, 1]$	value $v_3 = 30$

The scale is $1/\sqrt{2} = 0.7071$.
Step 1, raw scores (dot products).

$$\mathbf{q} \cdot \mathbf{k}_1 = 1 \times 2 + 0 \times 0 = 2,$$
$$\mathbf{q} \cdot \mathbf{k}_2 = 1 \times 0 + 0 \times 1 = 0,$$
$$\mathbf{q} \cdot \mathbf{k}_3 = 1 \times 1 + 0 \times 1 = 1.$$

Step 2, scale by 0.7071.

$$2 \times 0.7071 = 1.4142, \qquad 0 \times 0.7071 = 0, \qquad 1 \times 0.7071 = 0.7071.$$

Step 3, softmax. Exponentiate each scaled score:

$$e^{1.4142} = 4.1132,$$
$$e^{0} = 1.0000,$$
$$e^{0.7071} = 2.0281.$$

Add them:

$$4.1132 + 1.0000 + 2.0281 = 7.1413.$$

Divide each by the sum:

$$a_1 = 4.1132/7.1413 = 0.5760,$$
$$a_2 = 1.0000/7.1413 = 0.1400,$$
$$a_3 = 2.0281/7.1413 = 0.2840.$$

Check: $0.5760 + 0.1400 + 0.2840 = 1.0000$.
Step 4, weighted average of the values.

$$0.5760 \times 10 = 5.760,$$
$$0.1400 \times 20 = 2.800,$$
$$0.2840 \times 30 = 8.520,$$
$$5.760 + 2.800 + 8.520 = 17.080.$$

The query asked about "feature 0". Token 1 has the most of it, so it received the largest weight and pulled the answer towards its value of 10; token 3 has some, token 2 none. The answer, 17.08, is neither the plain average of the values (20) nor a copy of the best match (10): it is a *relevance-weighted blend*. Running the same numbers through PyTorch gives weights $[0.576, 0.140, 0.284]$ and answer 17.0802 [C] (`research/textbook/tools/compute_ceiling.py`).

22.5.5 Two properties you will rely on

The answer is a convex combination. The weights a_j are positive and sum to 1. A sum $\sum_j a_j v_j$ with such weights is called a *convex combination*, and it always lies between the smallest and the largest value: here between 10 and 30. Proof for scalars: replace every v_j by the largest value $v_{\max}$; the sum can only grow, and becomes $v_{\max} \sum_j a_j = v_{\max}$. The same argument with $v_{\min}$ gives the lower bound. This is why attention "blends" and never extrapolates beyond what the values offer.

The order of the tokens does not matter. Suppose you list the three tokens in a different order, say $(3, 1, 2)$. Each token's score is computed from its own key alone, so the three scores are the same numbers, listed in the new order. The softmax denominator is a sum, and a sum does not depend on the order of its terms, so each token keeps its weight. The final weighted sum is again a sum. The answer is unchanged. In symbols: if π is any reordering,

$$\sum_{j=1}^{N} a_{\pi(j)}\, v_{\pi(j)} = \sum_{j=1}^{N} a_j v_j. \tag{22.7}$$

Attention treats its keys and values as an unordered *set*. This is a strength (Section 22.6 uses it to accept bands in any order) and a danger: unless the token itself says which band it is, attention cannot tell band 10 from band 150. Keep that in mind; it is why tokens need a wavelength tag.

22.5.6 Where do the outside queries come from?

Two options, and the next Indradhanu uses both:

Table 22.3: The two sources of cross-attention queries in the spectral stem.

source of queries	example	used in
learned parameters: a table of M vectors trained like any weight	"32 questions the model has learned to ask of every spectrum"	the compressor, Chapter 23
computed from something else	"what is the reflectance at 1650 nm?", built from the number 1650	the decoder, Chapter 24

Learned queries are an `nn.Parameter` of shape `(D, d)`: D rows, each a vector of d numbers. They are the same for every pixel, like **W** in Section 22.2, but they are *questions*, not *recipes*. What comes back depends on the pixel, because the keys and values do. This difference is the heart of Part IV, so state it twice: in the compressor, the weight band b receives in output c is the constant W_{cb}; in cross-attention it is a_{cb}, a softmax of scores between a fixed query and a key computed from the pixel.

22.5.7 Self against cross, side by side

The score matrix is worth a number. With 202 band tokens per pixel and 32 queries:

$$\begin{aligned} \text{self:} &\quad 202 \times 202 = 40{,}804 \text{ scores per pixel, per head,} \\ \text{cross:} &\quad 32 \times 202 = 6{,}464 \text{ scores per pixel, per head.} \end{aligned}$$

Self-attention's count grows with the *square* of the band count; cross-attention's grows *linearly*. Chapter 23 turns that difference into gigabytes.

Table 22.4: Self-attention against cross-attention.

	self-attention (Chapter 14)	cross-attention
queries from	the tokens themselves	outside: learned table or another input
number of outputs	N (one per token)	M (one per query)
score matrix	$N \times N$	$M \times N$
changing N changes the output count?	yes	**no**
in Indradhanu	spatial tokens, `EfficientSelfAttention`	the spectral stem, Chapter 23

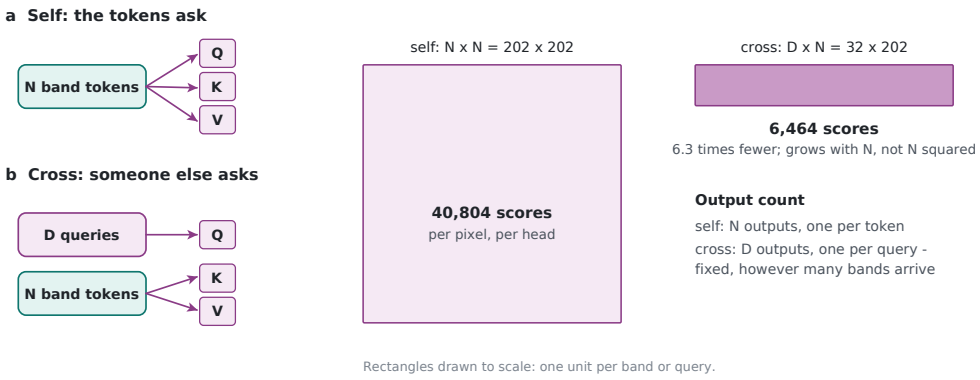


Figure 22.3: Where the questions come from, and what it costs. (a) In self-attention the band tokens produce queries, keys and values. (b) In cross-attention an outside set of D queries reads the tokens' keys and values. Right: score matrices for one pixel and head with 202 bands, drawn to scale: 40,804 against 6,464 scores.

22.6 A spectrum as tokens that know their wavelength

22.6.1 Where the tokens live

In Chapter 13 a token was a small square of the image. Here a token is **one band of one pixel**. A SpectralEarth pixel has 202 bands, so it becomes 202 tokens; an EnMAP pixel 224; a PRISMA pixel 239.

one pixel's spectrum:	$[0.084, 0.091, \ldots, 0.183]$	`(202,)` reflectance
as tokens:	202 vectors of d numbers	`(202, d)`

d is the *token width*. The design uses $d = 32$, matching the $D = 32$ channels of the current compressor so that parameter budgets compare cleanly (Chapter 23). Every pixel is handled separately: nothing in this section mixes one pixel with another.

A token has two added parts: the band's measured value, stretched into a vector, and a fixed fingerprint of its wavelength.

22.6.2 Part 1 of a token: the value, stretched

A single reflectance number cannot take part in dot products with 32-number queries. It is stretched by a tiny linear layer, `nn.Linear(1,d)`: multiply by a learned vector $\mathbf{w} \in \mathbb{R}^d$ and add a learned bias $\beta \in \mathbb{R}^d$,

$$\text{stretch}(v) = v\,\mathbf{w} + \beta. \tag{22.8}$$

Worked example 22.4: Stretching one band value

Take $d = 2$ so it fits on paper, and suppose training gave $\mathbf{w} = [0.5, -1.0]$, $\beta = [0.1, 0.2]$. A band measuring $v = 0.3$:

$$\begin{aligned} \text{component 0:} \quad & 0.3 \times 0.5 + 0.1 = 0.15 + 0.10 = 0.25, \\ \text{component 1:} \quad & 0.3 \times (-1.0) + 0.2 = -0.30 + 0.20 = -0.10. \end{aligned}$$

So $\text{stretch}(0.3) = [0.25, -0.10]$.

The layer has d weights and d biases: at $d = 32$, $32 + 32 = 64$ parameters. Notice that the stretch is the same for every band: the vector $\mathbf{w}$ does not know which band it is stretching. Two bands with the same value get the same value part. Something else must tell them apart.

22.6.3 Part 2 of a token: why a wavelength tag is required

Chapter 14 noted that Indradhanu's spatial attention needs no positional encoding, because the overlapping convolutions of the patch embedding already supply position. Here there is no grid and no convolution. By Equation (22.7), 202 band tokens built from values alone are, to attention, an unordered crowd: swap the values of bands 10 and 150 and every query gets the same answer. A spectrum with a dip at 700 nm and one with the same dip at 2200 nm would be indistinguishable if their values were the same multiset. So every token must carry its wavelength.

Definition 22.2 (Sinusoidal wavelength encoding)**.** For a wavelength λ in nanometres and an even width $d = 2H$, choose H *periods* $P_0, \ldots, P_{H-1}$ (a period is the distance in nm after which a wave repeats). The encoding is the vector

$$\mathbf{e}(\lambda) = \big[\sin\theta_0, \ldots, \sin\theta_{H-1},\ \cos\theta_0, \ldots, \cos\theta_{H-1}\big], \qquad \theta_i = \frac{2\pi\lambda}{P_i}. \tag{22.9}$$

It has no learned parameters: it is a formula of λ, identical in every run.

This is the same family of encodings as the positional encoding of the original transformer [44], with the token's position replaced by its physical wavelength.

Source check: the encoding as the code builds it

`app/foundation_models/components/spectral_tokens.py`, lines 25–37, defines `wavelength_encoding`. Three facts the course states loosely are exact in the code:

- *Sinusoidal, not learned.* No `nn.Parameter` appears; the function is pure arithmetic.
- *The periods are geometric.* With $H = d/2 = 16$ and `MIN_PERIOD_NM` $= 10$ and `MAX_PERIOD_NM` $= 10{,}000$, line 35 computes $P_i = 10 \cdot (10{,}000/10)^{i/(H-1)} = 10 \cdot 1000^{i/15}$ for $i = 0, \ldots, 15$.
- *Sines first, then cosines.* Line 37 is `torch.cat([torch.sin(angle),torch.cos(angle)],dim=-1)`. The course writes the tag as [sin, cos] pairs per period; the code puts all 16 sines in components 0–15 and all 16 cosines in 16–31. Dot products and cosine similarities do not depend on this order (a dot product is a sum), so every number in this chapter is the same either way, but if you read a component index off the tensor, use the code's layout.

The function raises `ValueError` unless d is even and at least 4.

22.6.4 The sixteen periods

Work out the periods. The ratio between neighbouring periods is

$$\begin{aligned} r &= 1000^{1/15} \\ &= \left(10^3\right)^{1/15} \\ &= 10^{3/15} \\ &= 10^{0.2} \\ &= 1.5849. \end{aligned}$$

So each period is 1.585 times the last, and five steps make a factor of $10^{0.2 \times 5} = 10$: **five periods per decade**. Starting at 10 nm:

Table 22.5: The sixteen periods of `wavelength_encoding` at $d = 32$, in nm (computed by `research/textbook/tools/compute_ceiling.py`).

i	0	1	2	3	4	5	6	7
P_i	10.00	15.85	25.12	39.81	63.10	100.0	158.5	251.2
i	8	9	10	11	12	13	14	15
P_i	398.1	631.0	1000	1585	2512	3981	6310	10,000

The source comment explains the two ends: "10 nm resolves neighbouring bands (EnMAP and PRISMA sample every ~6–12 nm); 10,000 nm exceeds the whole 400–2500 nm range, so the slowest pair varies monotonically across it and acts as a coarse 'where in the spectrum' coordinate." Short periods separate neighbours; long periods separate regions.

22.6.5 Two periods worked by hand

Two of the sixteen periods are round numbers, $P_{10} = 1000\,\text{nm}$ and $P_{15} = 10{,}000\,\text{nm}$, which makes them good for hand work.

Worked example 22.5: The tag of two wavelengths at one period

Take $P = 1000\,\text{nm}$ and the wavelengths 660 nm and 670 nm (red light, 10 nm apart). Use a calculator in radian mode; do it on paper.
660 nm.

$$\begin{aligned} \theta &= 2\pi \times 660/1000 \\ &= 6.2832 \times 0.660 \\ &= 4.1469 \text{ rad}, \\ \sin(4.1469) &= -0.8443, \\ \cos(4.1469) &= -0.5358. \end{aligned}$$

670 nm.

$$\begin{aligned} \theta &= 6.2832 \times 0.670 = 4.2097 \text{ rad}, \\ \sin(4.2097) &= -0.8763, \\ \cos(4.2097) &= -0.4818. \end{aligned}$$

The two pairs, $[-0.8443, -0.5358]$ and $[-0.8763, -0.4818]$, are close: nearby wavelengths get nearby tags. Now a third wavelength far away, 1650 nm:

$$\begin{aligned} \theta &= 6.2832 \times 1.650 = 10.3673 \text{ rad}, \\ \sin(10.3673) &= -0.8090, \\ \cos(10.3673) &= -0.5878. \end{aligned}$$

That is almost the same pair as 660 nm's. With this one period, 1650 nm and 660 nm are nearly indistinguishable: 1650 is 990 nm past 660, almost exactly one full period, so the wave has gone round once and come back. **One period aliases.**
Add the long period, $P = 10{,}000\,\text{nm}$:

$$\begin{aligned} 660: &\quad \theta = 6.2832 \times 0.066 = 0.4147, && \sin = 0.4029, && \cos = 0.9152, \\ 670: &\quad \theta = 6.2832 \times 0.067 = 0.4210, && \sin = 0.4086, && \cos = 0.9127, \\ 1650: &\quad \theta = 6.2832 \times 0.165 = 1.0367, && \sin = 0.8607, && \cos = 0.5090. \end{aligned}$$

Now 1650 is far from 660, while 660 and 670 stay close. The short period separates neighbours, the long one separates regions, and several periods together do not alias.

22.6.6 The similarity of two tags depends only on their distance

How similar are two full 32-number tags? Use the dot product. Write $\theta_i = 2\pi\lambda_1/P_i$ and $\phi_i = 2\pi\lambda_2/P_i$. By Equation (22.9) the dot product pairs sines with sines and cosines with cosines:

$$\mathbf{e}(\lambda_1) \cdot \mathbf{e}(\lambda_2) = \sum_{i=0}^{H-1} \sin\theta_i \sin\phi_i + \sum_{i=0}^{H-1} \cos\theta_i \cos\phi_i$$

Collect the two sums term by term:

$$= \sum_{i=0}^{H-1} \left(\cos\theta_i \cos\phi_i + \sin\theta_i \sin\phi_i\right)$$

and use the cosine of a difference, $\cos(\theta - \phi) = \cos\theta\cos\phi + \sin\theta\sin\phi$:

$$= \sum_{i=0}^{H-1} \cos(\theta_i - \phi_i)$$
$$= \sum_{i=0}^{H-1} \cos\left(\frac{2\pi(\lambda_1 - \lambda_2)}{P_i}\right). \tag{22.10}$$

Two consequences fall out.

Every tag has the same length. Put $\lambda_1 = \lambda_2$: every cosine is $\cos 0 = 1$, so $\mathbf{e}(\lambda)\cdot\mathbf{e}(\lambda) = H = 16$ and $\|\mathbf{e}(\lambda)\| = \sqrt{16} = 4$. (Running the module confirms a norm of 4.0 for every wavelength.)

The cosine similarity depends only on the distance $\Delta = \lambda_1 - \lambda_2$. The *cosine similarity* of two vectors is their dot product divided by the product of their lengths (Chapter 6): 1 means the same direction, 0 unrelated. With both lengths 4,

$$s(\Delta) = \frac{1}{4 \times 4} \sum_{i=0}^{15} \cos\left(\frac{2\pi\Delta}{P_i}\right) = \frac{1}{16} \sum_{i=0}^{15} \cos\left(\frac{2\pi\Delta}{P_i}\right). \tag{22.11}$$

The encoding is *shift invariant*: 660 against 670 nm is exactly as similar as 2000 against 2010 nm.

Worked example 22.6: Cosine similarity of 660 and 670 nm, all sixteen periods

Here $\Delta = 10$ nm. Compute $2\pi \times 10/P_i$ and its cosine for each period of Table 22.5. Do the first few on paper; the rest follow the same pattern.

$$\begin{aligned}
P_0 = 10: &\quad 2\pi \times 10/10 = 6.2832, &&\cos = 1.0000,\\
P_1 = 15.85: &\quad 62.832/15.85 = 3.9644, &&\cos = -0.6801,\\
P_2 = 25.12: &\quad 62.832/25.12 = 2.5013, &&\cos = -0.8020,\\
P_3 = 39.81: &\quad 62.832/39.81 = 1.5783, &&\cos = -0.0075,\\
P_4 = 63.10: &\quad 62.832/63.10 = 0.9958, &&\cos = 0.5438,\\
P_5 = 100: &\quad 62.832/100 = 0.6283, &&\cos = 0.8090.
\end{aligned}$$

For the ten longer periods the angle is small and the cosine is close to 1: 0.9224, 0.9689, 0.9876, 0.9950, 0.9980, 0.9992, 0.9997, 0.9999, 1.0000, 1.0000. Add all sixteen:

$$\begin{aligned}
&1.0000 - 0.6801 - 0.8020 - 0.0075 + 0.5438 + 0.8090\\
&\quad + 0.9224 + 0.9689 + 0.9876 + 0.9950 + 0.9980\\
&\quad + 0.9992 + 0.9997 + 0.9999 + 1.0000 + 1.0000\\
&= 0.8632 + 9.8707\\
&= 10.7339.
\end{aligned}$$

Divide by 16:

$$s(10) = 10.7339/16 = 0.6709.$$

The same sum for $\Delta = 990$ (660 against 1650 nm) comes to 0.2153, so $s(990) = 0.2153/16 = 0.0135$. Running `wavelength_encoding` and `cosine_similarity` on the real module gives 0.6709 and 0.0135 [C] (`research/textbook/tools/compute_ceiling_output.txt`), matching the course's 0.671 and 0.013.

Look at the first line of that table. At $P_0 = 10\,\text{nm}$, a 10 nm step is exactly one period, so the cosine is 1: **the shortest period cannot tell apart two wavelengths exactly 10 nm apart** (nor any two wavelengths that are whole multiples of 10 nm, such as 660 and 670, which both get sin = 0, cos = 1 there). This is the aliasing of worked example 22.5 again, at the short end. The other fifteen periods break the tie, and real sensors' band centres are not round numbers (the first SpectralEarth band sits at 418.416 nm [M: `app/constants/spectralearth_bands.json`]), so it does no harm, but it is a good illustration of why one period is never enough.

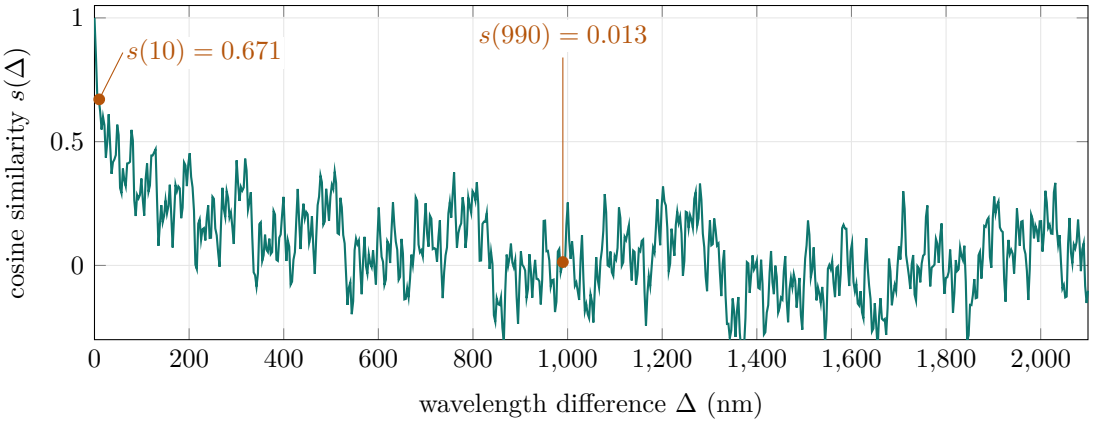


Figure 22.4: The cosine similarity of two wavelength tags, Equation (22.11), plotted exactly from the sixteen periods of the source. It starts at 1 for $\Delta = 0$, is still about 0.6 at 20–30 nm, and beyond about 100 nm it fluctuates around a mean of about 0.06, never above 0.49 nor below −0.35: tags of distant wavelengths are close to orthogonal. The two marked points are worked example 22.6.

22.6.7 The token is the sum of the two parts

The two parts are added, component by component:

$$\mathbf{t}_b = v_b\,\mathbf{w} + \beta + \mathbf{e}(\lambda_b) \;\in \mathbb{R}^d. \tag{22.12}$$

Worked example 22.7: Building one band token

Continue worked example 22.4 with $d = 2$. For this illustration let the tag be the period-1000 pair only (the real tag has 32 numbers). The band measures 0.3 at 660 nm:

$$\begin{aligned} \text{value part:} &\quad [\,0.25,\ -0.10\,] \\ \text{wavelength tag:} &\quad [-0.8443,\ -0.5358] \\ \text{token:} &\quad [\,0.25 - 0.8443,\ -0.10 - 0.5358\,] \\ &\quad = [-0.5943,\ -0.6358]. \end{aligned}$$

Adding rather than placing side by side (*concatenating*) keeps the token at width d. Concatenating would double the width to 64 and every later layer would grow with it. Adding works because training learns a value stretch whose direction $\mathbf{w}$ is usefully different from the directions the tags occupy, so later layers can still read both. In `spectral_attention.py`, line 77, the token is built exactly so: `self.value_embed(rows)+wavelength_encoding(wl,self.d)`.

22.6.8 Why wavelength beats position

The "any order" row matters in practice. SpectralEarth's band list is not sorted by wavelength: EnMAP's two detectors overlap near 900–1000 nm, and the list interleaves them

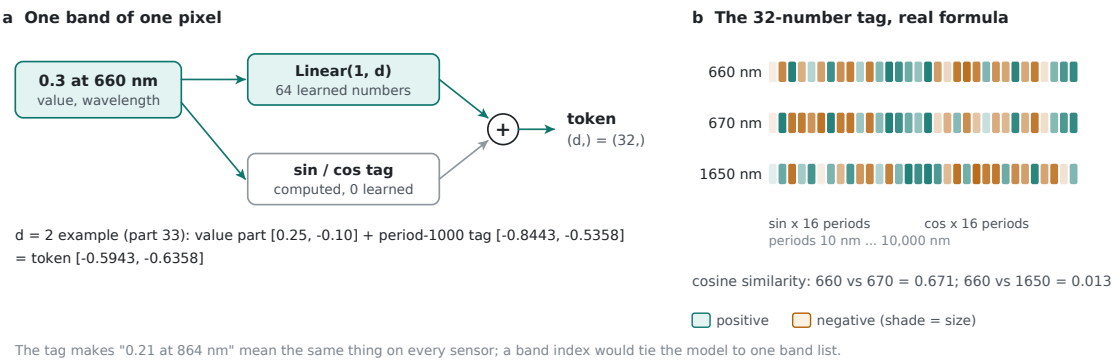


Figure 22.5: Building one band token. (a) The band's value is stretched to d numbers by `Linear(1, d)` and added to a fixed sinusoidal tag of its wavelength. (b) The real 32-number tag for three wavelengths (teal positive, amber negative, shade = magnitude): 660 and 670 nm share most of their pattern (cosine similarity 0.671), 660 and 1650 nm almost none (0.013).

Table 22.6: A band described by its position against a band described by its wavelength.

	position (today)	wavelength (the spectral stem)
what slot 87 means	depends on the sensor	slot numbers are not used
the gap 1350–1450 nm	invisible	1340 and 1460 get distinct tags
a new sensor	resample onto 165 first	read its native bands directly
band order	fixed	any order (Equation (22.7))
band width (FWHM)	ignored	could be encoded too (a refinement for later)

[M: `app/foundation_models/components/pixel_normalization.py`]. Because the tag carries the wavelength and attention ignores order, the spectral stem needs no special sorting. The dropped water-vapour bands are absent: SpectralEarth sends 202 tokens, not 224, and nothing marks the gap except the wavelengths themselves, which is enough.

A band token is "value stretched, plus a fixed fingerprint of the wavelength", so the model reads "0.21 at 864 nm" rather than "0.21 in slot 87".

With tokens defined and cross-attention in hand, all three requirements of Table 22.2 have a candidate answer: softmax weights that depend on the pixel (adaptivity), weights that can be forced to zero (skipping; Chapter 23), and tokens indexed by wavelength (any band list). The next chapter assembles them into a compressor and counts what it costs.

22.7 Common confusions

Isn't the compressor trained? Then it adapts.
It adapts to the *training set as a whole*, once. Inside a trained model it is frozen: every pixel of every scene meets the same $\mathbf{W}$, and the derivative of Equation (22.3) is the same constant everywhere. "Adaptive" in this Part means changing per pixel, at run time.

Interpolation fills the dead band with a good guess, so where is the harm?
A guess is not a measurement, and the compressor cannot tell them apart. By Equation (22.5) the error is the band's column of weights times the size of the guess's error. The cost is small for one band in one pixel and systematic across a scene with a striped detector.

Could we give the convolution more outputs instead?
That is what raising D from 24 to 32 did. The null space shrank from at least 141 to at least 133 dimensions; it is still the same null space for every pixel, and the layer is still tied to 165 inputs.

If the queries are fixed, isn't this the fixed recipe again?
No. In the compressor the *weights* are fixed. In cross-attention the *questions* are fixed, and the weights come from a softmax over scores that depend on the pixel's keys. Same questions, different answers per pixel.

Does cross-attention let the tokens talk to each other?
Not directly. Each query reads all tokens, but the tokens are not updated. Information flows from the tokens into the M answers only.

Which side is "cross"?
The word means queries and keys come from different sources. It says nothing about direction; the decoder of Chapter 24 is also cross-attention, pointed the other way.

Is the wavelength tag learned?
No. It is Equation (22.9), a formula identical in every run, with periods fixed in the encoding's source file. Only the value stretch (64 parameters) is learned. That is deliberate: a learned table indexed by band number would bring back the position problem.

The course writes the tag as sine-cosine pairs. Is that wrong?
It is a different layout of the same 32 numbers. The code stores all sines, then all cosines. Dot products, and so attention, do not care; component indices do.

Does this mean the current Indradhanu is wrong?
No. A fixed linear bottleneck is a sensible, cheap choice that works; Chapter 12 shows it costs a tiny share of the parameters. It is a ceiling, not a bug.

22.8 Check yourself

Exercise 22.1. Using the toy $\mathbf{W}$ of Equation (22.2), band 3 of $\mathbf{x} = [1.2, 1.0, -0.4, -0.6]$ arrives broken and filled with 0.0. Compute z_1 directly, then again with Equation (22.5), and give the error as a percentage of the true z_1. Do it on paper.

Exercise 22.2. The current compressor is a 1×1 convolution from 165 to 32 channels, followed by `BatchNorm2d(32)` (which has a scale and a shift per channel). Compute its parameter count. Then give a lower bound on the dimension of its null space, and say what that bound would be at $D = 24$.

Exercise 22.3. Redo worked example 22.3 with the query $\mathbf{q} = [0, 1]$. Compute the three scaled scores, the softmax weights and the answer.

Exercise 22.4. Using $\mathbf{w} = [0.5, -1.0]$, $\beta = [0.1, 0.2]$, compute the value part of a band measuring 0.6. Then compute the period-1000 tag $[\sin, \cos]$ for 500 nm and add the two to get a $d = 2$ token.

Exercise 22.5. Using Equation (22.11), explain without a calculator why $s(\Delta)$ can never exceed 1, and why $s(10\,000)$ is not close to 1 even though the longest period is 10,000 nm. (Hint: what is $\cos(2\pi \times 10{,}000/P_i)$ for $P_0 = 10$?)

Exercise 22.6. A crowd of 224 band tokens is read by 32 learned queries. How many answers come back, and how many scores are computed per head? For 425 bands (AVIRIS-NG),

how many scores would self-attention compute per pixel per head, and how many would cross-attention with 32 queries compute?

Next: Chapter 23, The cross-attention compressor, dead bands and what it costs.

Chapter 23

The cross-attention compressor, dead bands and what it costs

The one thing this chapter teaches

Thirty-two learned queries attend over a pixel's band tokens and each answer becomes one output channel; the result has almost exactly the parameters of the 1×1 convolution it replaces (5,345 against 5,376), skips a dead band exactly by setting its score to $-\infty$, and costs about 163 times the arithmetic and hundreds of mebibytes of memory per patch, which is where the design meets its wall.

Chapter 22 set three requirements for a replacement of the spectral compressor (per-pixel adaptivity, exact skipping of a broken band, any band list) and supplied two ingredients: cross-attention and band tokens that carry their wavelength. This chapter assembles the ingredients into a working module, `SpectralCrossAttentionCompressor` in `app/foundation_models/components/spectral_attention.py`, and then does three things to it with a pencil. It counts the parameters by hand and checks the count against the real module (Section 23.2). It proves the masking trick that removes a dead band exactly (Section 23.4). And it counts the arithmetic and the bytes (Sections 23.5 and 23.6), which turn out to be the real story: the module has the parameter budget of the old compressor and the appetite of something a hundred times larger.

One design rule runs through the chapter: **change one thing, measure it.** The new compressor hands the SegFormer encoder exactly the tensor the old one did, `(B, 32, H, W)`, so everything downstream (Chapters 17 and 19) stays as it is.

23.1 The data flow for one pixel

Follow one SpectralEarth pixel through the module. Its spectrum is 202 normalised reflectances.

1. *Tokenise* (Chapter 22, Equation (22.12)). Each band b becomes $\mathbf{t}_b = v_b\mathbf{w} + \beta + \mathbf{e}(\lambda_b)$, a vector of $d = 32$ numbers. Shape `(202, 32)`.
2. *Keys and values.* Two linear layers map every token to a key $\mathbf{k}_b = \mathbf{W}_K\mathbf{t}_b + \mathbf{b}_K$ and a value $\mathbf{v}_b = \mathbf{W}_V\mathbf{t}_b + \mathbf{b}_V$. Shapes `(202, 32)` each.
3. *Queries.* The module stores a table $\mathbf{Q}_0$ of 32 learned queries, shape `(32, 32)`. A third linear layer projects them, $\mathbf{Q} = \mathbf{Q}_0\mathbf{W}_Q^\top + \mathbf{b}_Q$, still `(32, 32)`. The table does not depend on the pixel, so this projection is the same for every pixel.
4. *Attend.* The 32 queries attend over the 202 keys with 4 heads (Section 23.1.2): answers of shape `(32, 32)`, one 32-number answer per query.

5. *Output projection.* A fourth linear layer $\mathbf{W}_O$ mixes the heads back together: still `(32, 32)`.
6. *To channels.* A final `Linear(32,1)`, shared by all 32 answers, turns each answer vector into one number: shape `(32,)`. These are the 32 channels of the compressed pixel.

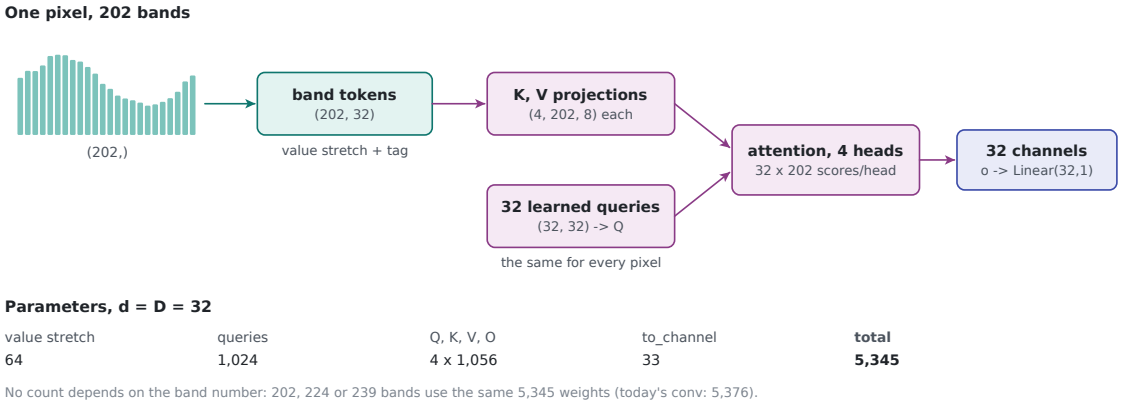


Figure 23.1: The per-band cross-attention compressor for one pixel. Each of 202 bands becomes a token; keys and values are projected from the tokens; 32 learned queries, shared by every pixel, attend over them with 4 heads, and each answer becomes one output channel. None of the 5,345 parameters depends on the band count.

23.1.1 A whole patch as a batch of pixels

A *batch* is a set of examples processed together in one tensor (Chapter 10). Here the trick is that the *pixels* of a patch are the batch: nothing in the module moves information between pixels, so the $128 \times 128 = 16{,}384$ pixels of a patch are treated as 16,384 independent rows. In the code this is one `permute` and one `reshape`:

Table 23.1: Tensor shapes in the cross-attention compressor for B patches of 128×128 pixels and $N = 202$ bands ($P = 16{,}384B$ pixel rows; from `spectral_attention.py`, lines 73–94).

tensor	shape	note
input	`(B, 202, 128, 128)`	B patches
pixels as rows	`(P, 202, 1)`	each row one spectrum
tokens	`(P, 202, 32)`	value stretch + wavelength tag
keys, values	`(P, 4, 202, 8)`	4 heads $\times$ 8 = 32
queries	`(P, 4, 32, 8)`	the same learned queries for every pixel
answers	`(P, 4, 32, 8)`	
after $\mathbf{W}_O$	`(P, 32, 32)`	heads joined again
channels	`(P, 32)` $\rightarrow$ `(B, 32, 128, 128)`	back to an image

23.1.2 Heads, by reshaping

Multi-head attention (Chapter 14) splits each 32-number key, value and query into 4 groups of 8 and runs attention separately on each group. In code no new weights appear: the 32 numbers are *viewed* as a `(4, 8)` block and the head axis is moved forward. For keys, line 78 reads

```
k = self.k(tokens).view(p, n, self.heads, hd).transpose(1, 2)   # (P, N, 32) -> (P, N
    , 4, 8) -> (P, 4, N, 8)
```

`view` reinterprets the last axis of 32 as 4×8 without copying; `transpose(1,2)` swaps the band axis and the head axis, so each head sees its own `(N, 8)` key matrix. The scale used inside each head is $1/\sqrt{8} = 0.3536$, because d_k in Equation (22.6) is now the per-head width 8. The queries are projected once (shape `(4, 32, 8)` after the same reshaping) and then `expand`ed to every pixel, which creates a view, not 16,384 copies.

23.1.3 The code

Here is the forward pass of the real module, lightly trimmed (the line numbers are those of `spectral_attention.py`):

```
def forward(self, x, wavelengths_nm, band_valid=None):
    b, n, h, w = x.shape                                    # x: (B, N, H, W)
    p, hd = b * h * w, self.d // self.heads                 # P pixels, head width 8
    rows = x.permute(0, 2, 3, 1).reshape(p, n, 1)           # (P, N, 1)
    wl = _per_pixel_wavelengths(wavelengths_nm.to(x.device), b, h * w)
    tokens = self.value_embed(rows) + wavelength_encoding(wl, self.d).to(x.dtype)  #
    (P, N, d)
    k = self.k(tokens).view(p, n, self.heads, hd).transpose(1, 2)    # (P, h, N, hd)
    v = self.v(tokens).view(p, n, self.heads, hd).transpose(1, 2)
    q = self.q(self.queries).view(self.D, self.heads, hd).transpose(0, 1)  # (h, D,
    hd)
    q = q.unsqueeze(0).expand(p, -1, -1, -1)                          # (P, h, D, hd)
    mask, empty = None, None
    if band_valid is not None:
        mask = band_valid.permute(0, 2, 3, 1).reshape(p, 1, 1, n)    # True = attend
        empty = ~mask.any(dim=-1, keepdim=True)                       # (P, 1, 1, 1)
        mask = mask | empty
    out = _attend(q, k, v, mask)                                      # (P, h, D, hd)
    out = self.to_channel(self.o(out.transpose(1, 2).reshape(p, self.D, self.d)))  #
    (P, D, 1)
    if empty is not None:
        out = out.masked_fill(empty.view(p, 1, 1), 0.0)
    return out.view(b, h, w, self.D).permute(0, 3, 1, 2)              # (B, D, H, W)
```

The two lines about `empty` are explained in Section 23.4.3, and `_attend` in Section 23.6.4.

23.2 Predict the parameter count

Do this on paper before reading the code. A *linear layer* `Linear(n_in,n_out)` stores a weight matrix of $n_{\text{out}} \times n_{\text{in}}$ numbers and one bias per output, so

$$\text{parameters} = n_{\text{out}} \times n_{\text{in}} + n_{\text{out}}. \tag{23.1}$$

A table of learned vectors (`nn.Parameter`) has no bias: its count is rows times columns. With $d = 32$ and $D = 32$:

Worked example 23.1: The cross-attention compressor's parameters by hand

Apply Equation (23.1) to every piece. Do it on paper, one line at a time.

$$\begin{aligned}
\text{value stretch, } \texttt{Linear(1, 32)}: &\quad 32 \times 1 + 32 = 64 \\
\text{learned queries, table } (32, 32): &\quad 32 \times 32 = 1{,}024 \\
\text{Q projection, } \texttt{Linear(32, 32)}: &\quad 32 \times 32 + 32 = 1{,}024 + 32 = 1{,}056 \\
\text{K projection, } \texttt{Linear(32, 32)}: &\quad 1{,}056 \\
\text{V projection, } \texttt{Linear(32, 32)}: &\quad 1{,}056 \\
\text{output projection, } \texttt{Linear(32, 32)}: &\quad 1{,}056 \\
\text{to one channel, } \texttt{Linear(32, 1)}: &\quad 1 \times 32 + 1 = 33
\end{aligned}$$

Add them in two steps. The four projections:

$$4 \times 1{,}056 = 4{,}224.$$

Everything:

$$\begin{aligned}
64 + 1{,}024 + 4{,}224 + 33 &= 1{,}088 + 4{,}224 + 33 \\
&= 5{,}312 + 33 \\
&= 5{,}345.
\end{aligned}$$

The wavelength tag contributes nothing: it is a formula (Chapter 22).

Instantiating the real module on a CPU and counting its parameters gives [C]

```
compressor params 5345
{'queries': 1024, 'value_embed.weight': 32, 'value_embed.bias': 32, 'q.weight': 1024,
     'q.bias': 32,
 'k.weight': 1024, 'k.bias': 32, 'v.weight': 1024, 'v.bias': 32, 'o.weight': 1024, 'o
    .bias': 32,
 'to_channel.weight': 32, 'to_channel.bias': 1}
```

(`research/textbook/tools/compute_crossstem.py`, output saved beside it). Your hand count matches line for line, and so does the module's own docstring: "Parameters at D = d = 32: 5,345, whatever N is." The repository pins the figure in a test (`docs/lld/spectral-stem.md`, invariant 5).

23.2.1 The general formula

Redo the count with symbols to see what it depends on:

$$\text{params}(d, D) = \underbrace{(d + d)}_{\text{stretch}} + \underbrace{Dd}_{\text{queries}} + \underbrace{4(d^2 + d)}_{\text{Q, K, V, O}} + \underbrace{(d + 1)}_{\text{to channel}}$$

collect the powers of d:

$$\begin{aligned}
&= 4d^2 + Dd + (2 + 4 + 1)d + 1 \\
&= 4d^2 + Dd + 7d + 1. \qquad (23.2)
\end{aligned}$$

Check at $d = D = 32$:

$$4 \times 1{,}024 + 32 \times 32 + 7 \times 32 + 1 = 4{,}096 + 1{,}024 + 224 + 1 = 5{,}345.$$

Two observations. First, **the band count N appears nowhere in Equation (23.2).** Every layer acts on one token at a time with shared weights, or on the fixed query table; no layer has a weight per band. The band count sets how many tokens there are, not how many weights. That is the sensor independence of Chapter 22 showing up in the arithmetic. Second, the count is dominated by the $4d^2$ of the projections: halving d roughly quarters the module.

23.2.2 Against the old compressor

Table 23.2: Old and new spectral compressors, parameters.

compressor	parameters
`Conv2d(165, 32, 1)` + `BatchNorm2d(32)`: $5{,}280 + 32 + 64$	5,376
cross-attention compressor, worked example 23.1	5,345

Thirty-one fewer. The new compressor is *not bigger*; it spends the same budget differently, on questions and projections instead of one fixed recipe per output.

Source check: where the BatchNorm went

The old count includes a `BatchNorm2d(32)`; the new module has none. That is deliberate and documented: `spectral_attention.py`, module docstring, "No BatchNorm inside; the model adds the BatchNorm2d all variants share." In `spectral_stem_seg_former_mae.py`, line 75, one `nn.BatchNorm2d(D)` sits after *every* stem (design decision 8 of `docs/proposals/spectral-attention-bottleneck.md`), so the encoder sees the same kind of input whichever stem produced it. The course's guess that the normalisation "will come back as a LayerNorm" did not happen; the shared BatchNorm is what the code does.

23.3 The 1x1 convolution is a special case

Chapter 22 claimed that cross-attention is "the compressor with data-dependent weights". Here is the derivation, for one output channel c and, to keep the algebra short, one head (the four-head case follows the same steps, Section 23.3.1).

Write the pieces with the names of Section 23.1: a_{cb} is the attention weight that query c gives band b; $\mathbf{W}_V, \mathbf{b}_V$ the value projection; $\mathbf{W}_O, \mathbf{b}_O$ the output projection; $\mathbf{u}, \gamma$ the weights and bias of `to_channel`. Then

$$y_c = \mathbf{u}^\top \Big(\mathbf{W}_O \sum_b a_{cb} \, (\mathbf{W}_V \mathbf{t}_b + \mathbf{b}_V) + \mathbf{b}_O \Big) + \gamma .$$

Move $\mathbf{u}^\top \mathbf{W}_O$ inside the sum (a matrix times a sum is the sum of the products), and collect the terms that do not depend on b into one constant $C = \mathbf{u}^\top \mathbf{b}_O + \gamma$:

$$= \sum_b a_{cb} \, \mathbf{r}^\top (\mathbf{W}_V \mathbf{t}_b + \mathbf{b}_V) + C, \qquad \mathbf{r} = \mathbf{W}_O^\top \mathbf{u}.$$

Substitute the token $\mathbf{t}_b = v_b \mathbf{w} + \beta + \mathbf{e}(\lambda_b)$ and split it into the part that carries the measured value and the part that does not:

$$= \sum_b a_{cb} \, \big(v_b \underbrace{\mathbf{r}^\top \mathbf{W}_V \mathbf{w}}_{\kappa} + \underbrace{\mathbf{r}^\top (\mathbf{W}_V (\beta + \mathbf{e}(\lambda_b)) + \mathbf{b}_V)}_{\phi(\lambda_b)} \big) + C$$

$$= \kappa \sum_b a_{cb} \, v_b + \sum_b a_{cb} \, \phi(\lambda_b) + C. \tag{23.3}$$

Here κ is one number (a row vector times a matrix times a column vector) and $\phi(\lambda_b)$ is one number per wavelength. Put Equation (23.3) beside the 1×1 convolution:

$$\begin{aligned} \text{1×1 convolution:} \quad & y_c = \sum_b W_{cb}\, v_b + b_c, \\ \text{cross-attention, one head:} \quad & y_c = \sum_b \kappa\, a_{cb}(\mathbf{v})\, v_b + (\text{a wavelength-dependent offset}). \end{aligned}$$

In the convolution the weight W_{cb} is a fixed number. In attention the weight $\kappa\, a_{cb}(\mathbf{v})$ is a softmax of scores that depend on the whole spectrum $\mathbf{v}$, because the keys are computed from the tokens. Freeze the a_{cb} to constants and attention collapses back into a fixed weighted sum. **The cross-attention compressor is the 1×1 convolution with data-dependent weights**: that is the entire idea, and why it answers the first requirement of Chapter 22.

Worked example 23.2: One channel of a one-head compressor

Take three bands with values $\mathbf{v} = [0.1, 0.4, 0.2]$, a channel with $\kappa = 2$, and suppose the softmax gave $a_{c\cdot} = [0.5, 0.3, 0.2]$ for this pixel. Ignore the offset terms ($\phi = 0$, $C = 0$). Do it on paper.

$$\begin{aligned} \sum_b a_{cb} v_b &= 0.5 \times 0.1 + 0.3 \times 0.4 + 0.2 \times 0.2 \\ &= 0.05 + 0.12 + 0.04 \\ &= 0.21, \\ y_c &= 2 \times 0.21 = 0.42. \end{aligned}$$

The effective recipe for this pixel is $\kappa a_{c\cdot} = [1.0, 0.6, 0.4]$. A different pixel produces different keys, so different scores and a different softmax, say $[0.2, 0.2, 0.6]$, and therefore a different effective recipe $[0.4, 0.4, 1.2]$. The convolution would have used one recipe for both.

23.3.1 A limit hidden in the derivation, and why there are four heads

Look again at the effective recipe κa_{cb} of one head. The softmax weights a_{cb} are all positive and sum to 1, so the effective weights all have the *sign of* κ, and they sum to κ. A single head therefore computes "κ times a weighted average of the bands", and cannot compute a *difference* such as "band 40 minus band 41", which is what a slope or the depth of an absorption dip needs. A 1×1 convolution can: $W_{c,40} = 1$, $W_{c,41} = -1$.

With 4 heads the derivation goes through head by head. The output projection reads the concatenation of the four 8-number head outputs, so $\mathbf{r} = \mathbf{W}_O^\top \mathbf{u}$ splits into four blocks $\mathbf{r}_1, \ldots, \mathbf{r}_4$ of 8 numbers, each head h has its own value matrix $\mathbf{W}_V^{(h)}$ (the rows of $\mathbf{W}_V$ that feed head h) and its own weights $a_{cb}^{(h)}$, and

$$y_c = \sum_{h=1}^{4} \kappa_h \sum_b a_{cb}^{(h)}\, v_b + (\text{offsets}), \qquad \kappa_h = \mathbf{r}_h^\top \mathbf{W}_V^{(h)} \mathbf{w}. \tag{23.4}$$

Now the four κ_h can have different signs, so one channel can be "head 1's average around 700 nm minus head 2's average around 680 nm": a difference of two data-chosen averages. This is one concrete reason (this book's derivation, not a statement from the course or the code) why the module uses several heads even though, as Section 23.2 showed, the head count changes no parameter count.

23.4 Skipping a dead band, exactly

Chapter 22, Equation (22.5), showed that a fixed weighted sum cannot leave an input out: a dead band filled with 0.0 moved the toy output from -0.50 to -0.30. Attention has a lever the convolution lacks: the weights are *computed*, so one can be forced to zero.

23.4.1 Minus infinity in softmax

Recall the softmax of a row of scores $s_1, \ldots, s_N$ (Chapter 14):

$$a_j = \frac{e^{s_j}}{\sum_{k=1}^{N} e^{s_k}}. \tag{23.5}$$

Now replace a score by $-\infty$. The exponential of a very negative number is a very small positive number, and it shrinks towards zero without limit: $e^{-10} = 0.0000454$, $e^{-20} = 0.000000002$, and in the limit

$$\lim_{s \to -\infty} e^{s} = 0, \qquad \text{so by convention } e^{-\infty} = 0. \tag{23.6}$$

Floating-point arithmetic follows the convention: `torch.exp(torch.tensor(float('-inf')))` is exactly `0.`, not a tiny number.

Definition 23.1 (Masked softmax)**.** Given scores $s_1, \ldots, s_N$ and a set $\mathcal{V}$ of *valid* positions, the masked softmax sets $s_j = -\infty$ for every $j \notin \mathcal{V}$ and then applies Equation (23.5).

What does that give? Every masked position contributes $e^{-\infty} = 0$ to both its own numerator and to the sum in the denominator. So for a valid position j,

$$a_j = \frac{e^{s_j}}{\sum_{k \in \mathcal{V}} e^{s_k} + \sum_{k \notin \mathcal{V}} 0} = \frac{e^{s_j}}{\sum_{k \in \mathcal{V}} e^{s_k}}, \tag{23.7}$$

and for a masked position $a_j = 0/(\ldots) = 0$ *exactly*. Read Equation (23.7) as a theorem: **the masked softmax is the ordinary softmax computed over the valid positions only**, as if the masked bands had never existed. Two consequences:

- The surviving weights still sum to 1, because the valid terms are divided by their own sum. This is called *renormalisation*.
- The ratio of any two surviving weights is unchanged: $a_i / a_j = e^{s_i} / e^{s_j} = e^{s_i - s_j}$ both before and after masking, because the common denominator cancels. Masking takes the dead band's share and hands it out to the others in proportion to what they already had.

Worked example 23.3: Masking a dead band among four

Four bands with scores $[2, 1, 0, -1]$. Band 2 (the third, score 0) is dead. Do this on paper. *Without the mask.* Exponentiate:

$$e^{2} = 7.3891, \qquad e^{1} = 2.7183, \qquad e^{0} = 1.0000, \qquad e^{-1} = 0.3679.$$

Sum:

$$7.3891 + 2.7183 + 1.0000 + 0.3679 = 11.4753.$$

Weights:

$$7.3891/11.4753 = 0.6439, \qquad 2.7183/11.4753 = 0.2369,$$
$$1.0000/11.4753 = 0.0871, \qquad 0.3679/11.4753 = 0.0321.$$

The dead band receives 8.71% of the attention, and whatever garbage it holds is mixed in with that weight.
With the mask. Its score becomes $-\infty$ and its exponential becomes 0:

$$e^{2} = 7.3891, \qquad e^{1} = 2.7183, \qquad e^{-\infty} = 0, \qquad e^{-1} = 0.3679.$$

Sum:

$$7.3891 + 2.7183 + 0 + 0.3679 = 10.4753.$$

Weights:

$$7.3891/10.4753 = 0.7054, \qquad 2.7183/10.4753 = 0.2595,$$
$$0/10.4753 = 0, \qquad 0.3679/10.4753 = 0.0351.$$

Check the sum: $0.7054+0.2595+0+0.0351 = 1.0000$. Check a ratio: before, $0.6439/0.2369 = 2.718$; after, $0.7054/0.2595 = 2.718$, both equal to $e^{2-1} = e$. PyTorch's `softmax` on the same rows gives the same weights [C].

The dead band's weight is *exactly* zero, so its value is multiplied by zero, whatever that value is. The answer is built only from bands that were really measured. Compare the fill-with-zero approach of Chapter 22: a zero *value* is still a claim that the reflectance was 0, and it still receives a weight. Only a zero *weight* removes the band.

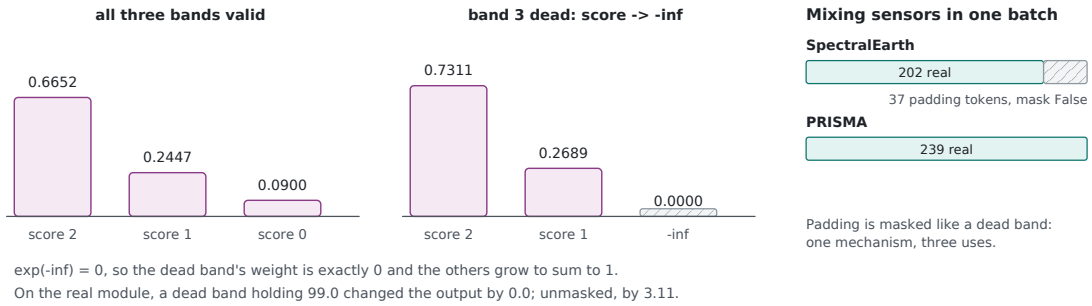


Figure 23.2: Minus infinity before softmax. Left: three valid scores $[2, 1, 0]$ give weights $[0.6652, 0.2447, 0.0900]$. Middle: marking band 3 dead sets its score to $-\infty$, its weight to exactly 0, and the other two renormalise to $[0.7311, 0.2689]$. Right: the same mask lets a 202-band SpectralEarth pixel share a batch with 239-band PRISMA pixels, its 37 padding tokens masked out.

23.4.2 In code: a boolean mask

PyTorch's `F.scaled_dot_product_attention` takes an `attn_mask`. A boolean mask means: **True where attention is allowed**, False where the score must become $-\infty$. The compressor takes `band_valid`, shape `(B, N, H, W)`, one boolean per band per pixel, and reshapes it to `(P, 1, 1, N)` (line 84) so that the same mask applies to every head (the first 1) and every query (the second 1). Broadcasting fills those two size-1 axes.

Getting the polarity backwards masks every valid band and keeps only the dead ones. The cheapest guard is a test that cannot be fooled.

Values that will drift

The 99 test. Take 16 pixels with SpectralEarth's real 202 wavelengths, mark band 100 dead, then change band 100's value to a wild 99.0. If masking works, the output must not move at all.

- Course run (IW35, the study prototype): masked, maximum change 0.0; unmasked, 3.1075 [M: `indradhanuworkings/35-skipping-a-dead-band.md`].
- This book's run (2026-10-29, real module, random untrained weights, seed 0, `research/textbook/tools/compute_crossstem.py`): masked, maximum change 0.0 [C]; unmasked, 2.1322 [C].

The unmasked figure depends on the random initial weights and will differ from run to run. The masked figure is 0.0 in every run, and that is the point: not "very little", nothing.

Chapter 22's contamination is gone *by construction*, not by hoping an interpolation guessed well.

23.4.3 A pixel with no valid band

What if every band of a pixel is masked? Then every term in the denominator of Equation (23.7) is zero, and each weight is 0/0, which is undefined (`NaN`, "not a number", in floating point). One NaN in a batch spreads through the loss to every weight in the model during the backward pass.

Source check: what the code does with an empty pixel

The course (IW35) says such pixels "must be dropped before [they reach] attention". The code does something different and more robust, `spectral_attention.py`, lines 85–93: it finds the empty pixels (`empty=~mask.any(dim=-1,keepdim=True)`), lets them attend to *every* band (`mask=mask|empty`) so the softmax is well defined, and then overwrites their output with exactly 0 (`out.masked_fill(empty,0.0)`). The comment explains why: a plain softmax gives NaN, "zeros in torch 2.5's fused CPU kernel, unspecified elsewhere". Invariant 2 of `docs/lld/spectral-stem.md` pins the result: "A pixel with no valid band outputs exactly 0."
A second line of defence sits one level up. `SpectralStemSegFormerMAE._stem` (`spectral_stem_seg_former_mae.py`, lines 142–145) normalises the input and then sets every dead voxel to 0 *before* any stem sees it, so even an unmasked dead band arrives as the mean of the normalisation curve, never as 99.0. The 99 test above calls the compressor directly, bypassing that step, which is what a test of the mask should do.

The existing pixel-level rule still makes sense on top: a pixel with more than 40% invalid voxels is invalid (`BandFilterConfig.max_invalid_voxel_fraction`, Chapter 5). Masking 150 of 202 bands leaves an answer built from 52; honest, but thin.

23.4.4 The same trick, a second use: mixing sensors

A tensor is rectangular: every row of a batch must have the same number of band tokens. SpectralEarth pixels have 202; PRISMA pixels 239. To put both in one batch, *pad* the shorter rows up to the longest with dummy tokens and mark the padding invalid:

pixel	tokens	mask
SpectralEarth	202 real + 37 padding = 239	202 True, 37 False
PRISMA	239 real	239 True

The padding count is $239 - 202 = 37$. The padding gets weight zero exactly as a dead band does, so whatever value and wavelength the padding tokens carry is irrelevant. One mechanism handles broken bands, missing bands and different sensors. The price is wasted arithmetic: the SpectralEarth rows compute 37 scores per query that are then thrown away, $37/239 = 15.5\%$ of their attention work.

The compressor supports this directly: `wavelengths_nm` may be `(B, N)`, one wavelength list per sample, and `_per_pixel_wavelengths` (lines 48–52) repeats each sample's list for its $H \times W$ pixels.

Source check: how the model really mixes sensors

Padding is supported by the compressor, but the model does not use it to mix sensors. `SpectralStemSegFormerMAE.forward_mixed` (`spectral_stem_seg_former_mae.py`, lines 159–180) takes a list of groups, one per band list, runs the stem *separately* on each group with its own `(N,)` wavelengths, and concatenates the 32-channel outputs before the shared BatchNorm, encoder and decoder. No padding tokens are built at all, and the waste computed above does not arise. The reduced stem of Chapter 24 cannot take per-sample lists at all, which is one reason this group-wise design was chosen.

The loss must skip padded and dead bands too: there is no truth to compare a reconstruction against. Chapter 24 returns to this.

23.5 What it costs in arithmetic

Section 23.2 found 5,345 parameters against today's 5,376. That says how much the model must *store and learn*. It says nothing about how much *work* each pixel needs, or how much *memory* the work occupies while it runs. Those are different numbers, and here they differ enormously.

A *multiply-add* (MAC) is one multiplication followed by one addition, the basic unit of a dot product. A dot product of two 32-number vectors is 32 multiply-adds. A matrix-vector product with an $(m \times n)$ matrix is m dot products of length n: mn multiply-adds.

23.5.1 Today: 5,280 multiply-adds per pixel

`Conv2d(165,32,kernel_size=1)` computes 32 dot products of length 165 per pixel:

$$165 \times 32 = 5{,}280 \text{ multiply-adds per pixel.}$$

23.5.2 Cross-attention: 861,184 multiply-adds per pixel

Worked example 23.4: Counting the compressor's multiply-adds

202 band tokens, $d = 32$, $D = 32$. Count each stage as (number of dot products) $\times$ (their length). Do it on paper.
K and V projections. Each of the 202 tokens is multiplied by two 32×32 matrices; one 32×32 matrix-vector product is 1,024 multiply-adds:

$$2 \times 202 \times 1{,}024 = 404 \times 1{,}024 = 413{,}696.$$

Scores. 32 queries times 202 keys, each a dot product of length 32 (split over 4 heads of 8, which is the same total):

$$32 \times 202 \times 32 = 6{,}464 \times 32 = 206{,}848.$$

Weighted values. Each of the 32 answers is a weighted sum of 202 value vectors of 32 numbers:

$$32 \times 202 \times 32 = 206{,}848.$$

Output projection and to-channel. 32 answers through a 32×32 matrix, then 32 dot products of length 32:

$$32 \times 1{,}024 + 32 \times 32 = 32{,}768 + 1{,}024 = 33{,}792.$$

Total.

$$\begin{aligned} 413{,}696 + 206{,}848 + 206{,}848 + 33{,}792 &= 620{,}544 + 206{,}848 + 33{,}792 \\ &= 827{,}392 + 33{,}792 \\ &= 861{,}184. \end{aligned}$$

The query projection is the same for every pixel, so it is computed once per batch, not per pixel, and is left out.

The course's count also leaves out the value stretch, `Linear(1,32)` on 202 tokens, $202 \times 32 = 6{,}464$ multiplications, and the addition of the wavelength tags. Including the stretch gives $861{,}184 + 6{,}464 = 867{,}648$, a 0.75% difference. This book keeps the course's 861,184 so that the ratios below agree with the course and with `research/experiments/indradhanu-se-pretrain/S01-laptop-smoke/README.md`.

Compare:

$$\frac{861{,}184}{5{,}280} = 163.1.$$

About **163 times** the arithmetic of today's 165-band compressor. Against a 1×1 compressor reading SpectralEarth's 202 bands, $202 \times 32 = 6{,}464$ multiply-adds, the ratio is

$$\frac{861{,}184}{6{,}464} = 133.2,$$

which is the "133x" quoted in the S01 smoke-test notes and in Chapter 24. Both numbers are right; they answer different questions ("against the shipped model" and "against variant A on the same data").

Per 128×128 patch:

$$861{,}184 \times 16{,}384 = 14{,}109{,}638{,}656 \approx 14.1 \text{ billion multiply-adds.}$$

At batch 128 that is $14{,}109{,}638{,}656 \times 128 = 1{,}806{,}033{,}747{,}968$, about 1.8 trillion per step for the compressor alone [C]. A modern training GPU does tens to hundreds of trillions of multiply-adds per second in half precision, so this is significant but affordable. **Arithmetic is not what breaks first.**

23.6 The memory wall

To compute attention, the scores, the softmax weights, the keys and the values must exist in memory at the same time. Count the bytes. A 32-bit float (*fp32*) takes 4 bytes; a 16-bit *bfloat16* (*bf16*) takes 2. A *mebibyte* (MiB) is $2^{20} = 1{,}048{,}576$ bytes and a gibibyte (GiB) is $2^{30} = 1{,}073{,}741{,}824$ bytes.

Worked example 23.5: Bytes of attention for one patch

One 128×128 patch, 16,384 pixels, 202 bands, 32 queries, 4 heads, fp32. Do it on paper; multiply left to right.
Attention weights, if stored. One weight per (pixel, head, query, key):

$$\begin{aligned} 16{,}384 \times 4 &= 65{,}536, \\ 65{,}536 \times 32 &= 2{,}097{,}152, \\ 2{,}097{,}152 \times 202 &= 423{,}624{,}704 \text{ weights}, \\ 423{,}624{,}704 \times 4 \text{ bytes} &= 1{,}694{,}498{,}816 \text{ bytes}, \\ 1{,}694{,}498{,}816/1{,}048{,}576 &= 1{,}616 \text{ MiB}. \end{aligned}$$

Keys and values, which must exist. Two tensors of (pixel, band, 32):

$$\begin{aligned} 16{,}384 \times 2 &= 32{,}768, \\ 32{,}768 \times 202 &= 6{,}619{,}136, \\ 6{,}619{,}136 \times 32 &= 211{,}812{,}352 \text{ numbers}, \\ 211{,}812{,}352 \times 4 &= 847{,}249{,}408 \text{ bytes} = 808 \text{ MiB}. \end{aligned}$$

One patch: 1,616 MiB of weights and 808 MiB of keys and values. A batch of 128 would need over 200 GB for the weights alone. That is not a tuning problem; it is impossible. (A version of this estimate in an early design note came out near 400 MB: it left out the 4 heads. Count heads.)

23.6.1 And self-attention would be far worse

Chapter 22 noted that self-attention compares every band with every band. Its weights, per patch, fp32:

$$\begin{aligned} 16{,}384 \times 4 \times 202 \times 202 \times 4 &= 65{,}536 \times 40{,}804 \times 4 \\ &= 2{,}674{,}130{,}944 \times 4 \\ &= 10{,}696{,}523{,}776 \text{ bytes} \\ &= 10{,}696{,}523{,}776/1{,}073{,}741{,}824 = 9.96 \text{ GiB per patch}. \end{aligned}$$

Ten gigabytes per patch, for the weights only. **This is the concrete reason the design is cross-attention**: 32 questions against 202 bands, not 202 against 202. Decision 1 of `docs/proposals/spectral-attention-bottleneck.md` records exactly this rejection.

23.6.2 Three ways the cost comes down

A *fused* kernel is one GPU routine that computes scores, softmax and the weighted sum together, tile by tile, so the full 32×202 weight matrix of a pixel never exists in memory at once. It is not an approximation: it computes the same numbers in a different order (and the 99 test of Section 23.4 ran through it).

Band grouping, worked:

$$16{,}384 \times 2 \times 51 \times 32 \times 4 = 213{,}909{,}504 \text{ bytes} = 204 \text{ MiB per patch},$$

and $204/2 = 102$ MiB in bf16, against $808 + 1{,}616 = 2{,}424$ MiB naively. Grouping is a genuine trade-off: four neighbouring bands become one token, so detail finer than about 4 bands (roughly 25–40 nm on SpectralEarth) is mixed before attention sees it. Chapter 24 shows how the shipped design took this lever and repaired its weakness.

Table 23.3: Ways to cut the compressor's memory.

technique	what it removes	effect per patch
fused attention (`F.scaled_dot_product_attention`)	never stores the full weight matrix; computes the softmax in small tiles	1,616 MiB of weights → near zero
half precision (bf16, 2 bytes)	halves every tensor	K+V 808 → 404 MiB
band grouping	one token per about 4 adjacent bands: 202 → 51 tokens	K+V and scores shrink about 4×

23.6.3 Training memory is not inference memory

The arithmetic so far is about one forward pass. Training is worse. The backward pass (Chapter 26) needs every intermediate the forward pass made: the tokens, the stretched values, the keys, the values, the attention output and more. Many of them have the shape `(pixels, 202, 32)`. One such tensor, one patch, bf16:

$$\begin{aligned} 16{,}384 \times 202 \times 32 \times 2 &= 3{,}309{,}568 \times 32 \times 2 \\ &= 105{,}906{,}176 \times 2 \\ &= 211{,}812{,}352 \text{ bytes} \\ &= 211{,}812{,}352/1{,}048{,}576 = 202 \text{ MiB}. \end{aligned}$$

The measured training memory of about 3.7 GiB per patch (below) is about

$$\frac{3.7 \times 1{,}024}{202} = \frac{3{,}789}{202} = 18.8 \approx 19$$

such tensors, and $19 \times 202 = 3{,}838$ MiB $= 3{,}838/1{,}024 = 3.75$ GiB. **Inference-time memory is not training-time memory**: count every tensor the backward pass must keep, not only the ones a forward pass holds at its peak.

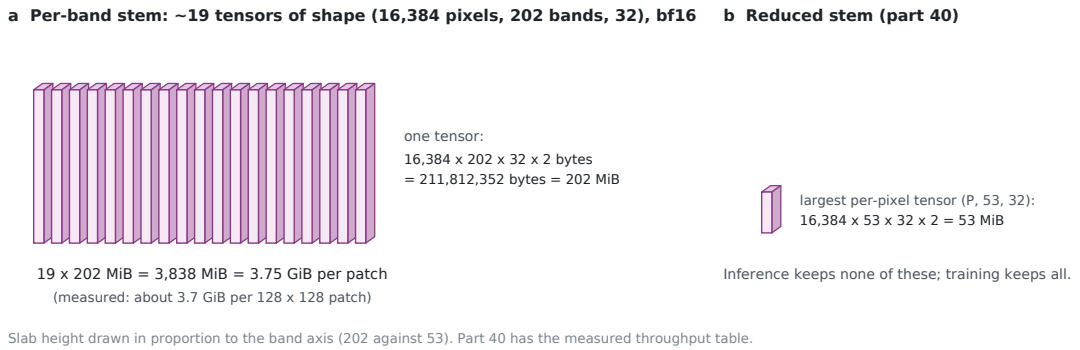


Figure 23.3: Why training, not inference, hits the wall. (a) For backpropagation the per-band stem keeps about 19 intermediates of shape (16,384 pixels, 202 bands, 32), 202 MiB each in bf16: about 3.75 GiB per 128 × 128 patch, matching the measured 3.7 GiB. (b) The reduced stem of Chapter 24 never builds a band-by-32 tensor; its largest per-pixel tensor has 53 bin slots, 53 MiB.

23.6.4 A hardware limit nobody mentions

The GPU's fused attention kernel puts the batch on one axis of its launch grid, and that axis is capped at $2^{16} - 1 = 65{,}535$ blocks. Here the "batch" is pixels, so 4 patches, $4 \times 16{,}384 = 65{,}536$

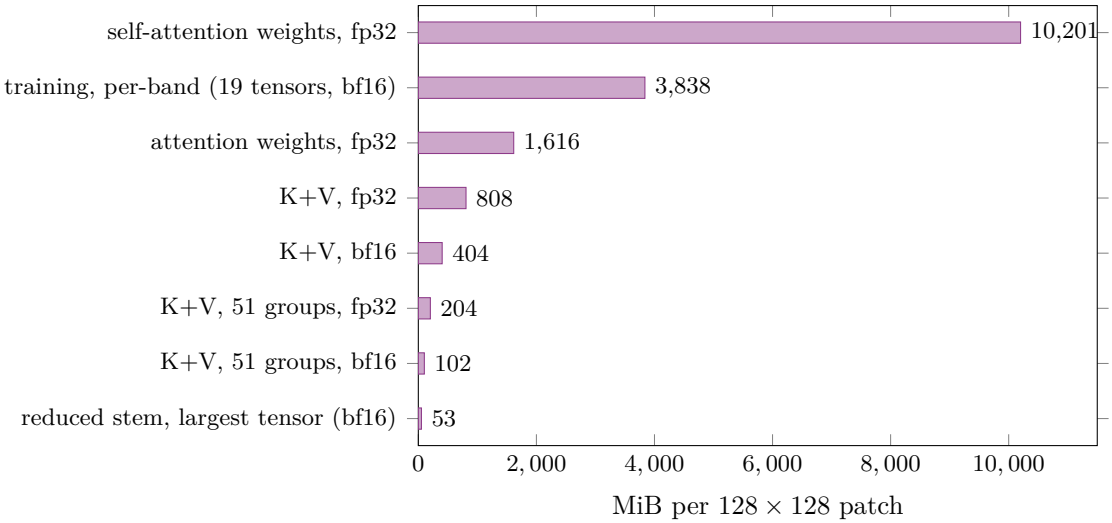


Figure 23.4: Memory per patch of the quantities computed in this section, to scale (linear axis). All are this book's arithmetic from the shapes; the training bar is 19 × 202 MiB, matching the measured 3.7 GiB. The self-attention bar, 9.96 GiB, is why the design is cross-attention.

rows, failed with `invalid configuration argument`: one row over. The module now splits the call:

```
SDPA_MAX_ROWS = 32_768   # spectral_attention.py, line 35

def _attend(q, k, v, mask=None):
    """scaled_dot_product_attention over (P, h, L, hd), at most SDPA_MAX_ROWS pixels
    per call."""
    if q.shape[0] <= SDPA_MAX_ROWS:
        return F.scaled_dot_product_attention(q, k, v, attn_mask=mask)
    return torch.cat([F.scaled_dot_product_attention(q[i:i + SDPA_MAX_ROWS], k[i:i +
    SDPA_MAX_ROWS],
                                                     v[i:i + SDPA_MAX_ROWS], attn_mask=None if mask is None else
     mask[i:i + SDPA_MAX_ROWS])
                      for i in range(0, q.shape[0], SDPA_MAX_ROWS)])
```

Because pixels are independent, cutting the rows into chunks and concatenating the results gives *identical* numbers. 4 patches become 65,536/32,768 = 2 calls. The general count is $\lceil P/32{,}768 \rceil$, where $\lceil \cdot \rceil$ rounds up.

23.6.5 Measured on a real GPU

Values that will drift

S01, 2026-10-14, laptop RTX 4000 SFF Ada 20 GB, one training step, real Indradhanu encoder (embedding widths 32, 64, 160, 256), 128 × 128 patches, bf16. Best batch and throughput [M: `research/experiments/indradhanu-se-pretrain/S01-laptop-smoke/README.md`]:

stem	best batch	patches/s	memory per patch
today's 1×1 compressor (A)	64	391	about 0.08 GiB
per-pixel gate (B, Chapter 24)	64	282	about 0.11 GiB
cross-attention (this chapter)	4	5.0	**about 3.7 GiB**

At batch 8 and above the cross-attention stem exceeded the 20 GB card and Windows spilled to shared system memory, collapsing throughput (batch 16: 0.8 patches/s). End

to end the cross-attention model was 391/5 = 78 times slower than today's, not 163: the shared SegFormer costs the same in all three and dilutes the difference.
L40S, 2026-10-14, AWS 48 GB: the per-band stem trained at 13.5 patches/s at batch 4 (11.3 GiB peak), 12.2 at batch 8, 10.2 at batch 16 (32.4 GiB) and ran out of memory at batch 32 [M: `docs/lld/spectral-stem.md`]. Chapter 24 puts the full table on a chart.

Why did the toy run of 16 pixels never hint at this? **Costs that scale with the pixel count do not show up in a toy run.** Always redo the byte arithmetic at the real patch and batch size before trusting a design.

23.7 Common confusions

Each query gives a 32-number answer; why is `to_channel` needed?
The encoder expects 32 *numbers* per pixel, not 32 vectors. `Linear(32, 1)` turns each answer vector into one number, and it is shared by all 32 queries: 33 parameters, not 32×33.

Why 4 heads?
Four groups of 8 numbers can learn four different ways of scoring relevance, and, by Equation (23.4), one channel can then combine averages with different signs, which a single head cannot. The head count changes no parameter count: the projections are 32×32 either way.

Does every pixel get its own queries?
No. All 16,384 pixels of a patch share the same 1,024-number query table. Only the keys and values differ per pixel.

Why not set the dead band's value to zero instead of its score?
A zero value still receives a weight (8.71% in worked example 23.3), and a zero is a claim that the reflectance is 0. Only a zero weight removes the band.

Is the boolean mask True for valid or invalid?
In `scaled_dot_product_attention` a boolean `attn_mask` is **True where attention is allowed**. Getting it backwards masks everything valid. The 99 test catches a flipped mask immediately.

Same parameters, so same speed?
No. Parameters are stored once; arithmetic and activations are paid per pixel, per patch, per step. Here 5,345 parameters cost 861,184 multiply-adds per pixel.

Is fused attention an approximation?
No. It computes exactly the same result in a different order, never holding the full weight matrix at once.

Is 163× going to make training 163× slower?
No. The compressor is one part of the model and the SegFormer dominates the arithmetic; the measured end-to-end slowdown was 78×. The real limit was memory: 4 patches per batch on a 20 GB card.

23.8 Check yourself

Exercise 23.1. Recompute the compressor's parameter count for $d = 16$, $D = 32$ (16 numbers per token, still 32 output channels), line by line. Check your total against Equation (23.2).

Exercise 23.2. Scores $[3, 1, 1]$, with the second band dead. Compute the masked weights by hand, and check that the ratio of the two surviving weights is the same as before masking.

Exercise 23.3. Compute the multiply-adds per pixel for an EnMAP pixel (224 bands), $d = D = 32$, using the four lines of worked example 23.4.

Exercise 23.4. Compute the K+V memory per patch for 224 bands in bf16 (2 bytes), in MiB.

Exercise 23.5. Training needs about 3.7 GiB per patch. On a 48 GB card, leaving 4 GB for everything else, how many patches fit in one batch? How many pixel rows is that, and how many calls to the fused kernel does `_attend` make?

Exercise 23.6. Using Equation (23.3), show that a *single* head with frozen weights a_{cb} cannot compute $y_c = v_0 - v_1$ (ignore the offsets). Then give κ_1, κ_2 and head weights for two heads over two bands that do compute it with Equation (23.4).

Next: Chapter 24, The spectral decoder, the traps, per-pixel gating and spectral reduction.

Chapter 24

The spectral decoder, the traps, per-pixel gating and spectral reduction

The one thing this chapter teaches

Reconstruction runs the cross-attention idea backwards: each wavelength you want back becomes a query, so one set of weights rebuilds any sensor's bands; the design is only safe if it hides pixels (not bands), normalises by wavelength, and is tested against a cheap per-pixel gate; and because per-band attention hits a memory wall, the shipped stem attends over 40 nm wavelength bins while three per-band paths keep the detail a bin average would lose.

Chapter 23 built the way *in*: 32 learned queries read a pixel's band tokens and hand the SegFormer a 32-channel image. This chapter builds the way *out*, and then follows the design from paper to production. It has four movements.

1. Section 24.1: the cross-attention decoder, whose queries are *target wavelengths*, counted by hand (6,401 parameters).
2. Section 24.2: one pixel traced from file to loss, and the three traps the design must avoid.
3. Section 24.3: the cheap rival, a per-pixel sigmoid gate (13,162 parameters), and what it can and cannot do.
4. Section 24.4: spectral reduction, the stem that actually trains, with its bin arithmetic, its three per-band fixes, its parameter counts (14,721 and 4,257) and the measured throughput that justified it.

24.1 The way back out: a decoder that answers for any wavelength

24.1.1 What has to come back out

The model's job (Chapter 9) is to redraw its input and compare. So whatever went in must come back out at the *same wavelengths*: a SpectralEarth pixel's 202 bands, a PRISMA pixel's 239. Today's `SpectralDecompressor` (Chapter 12) is `Conv2d(32,165,kernel_size=1)`: one fixed recipe per output band, exactly 165 of them, with

$$32 \times 165 + 165 = 5{,}280 + 165 = 5{,}445 \text{ parameters.}$$

It has the three problems Chapter 22 found in the compressor, mirrored: one recipe for every pixel, no notion of a band that should not be produced, and exactly 165 output *positions*. It cannot produce a 202nd band, or any band not on the grid, because its weight matrix has exactly 165 rows.

24.1.2 Turn it around: the wavelength asks the question

In the compressor, 32 learned questions read many band tokens. In the decoder the roles swap:

	queries	keys and values	answers
compressor	32 learned questions	N band tokens	32 channels
decoder	N wavelength questions	32 channel tokens	N reflectances

Each output band is one question, built from its wavelength alone: "given everything this pixel's 32 channels say, what is the reflectance at 1650 nm?" Ask 202 questions, get 202 answers; ask 239, get 239. Chapter 22 called this kind of query "computed from something else".

Analogy

The compressor is a reporter who interviews 202 witnesses and files a fixed 32-point summary. The decoder is a reader of that summary who may ask it any question that has a date on it, and gets an answer for any date, not only the dates the original witnesses covered. The answers are only as good as the summary: a question about a date nobody witnessed gets a confident-sounding guess, which is the extrapolation trap of Section 24.1.8.

24.1.3 The pieces

Per pixel, after the SegFormer encoder and decoder (Chapters 17 and 19) hand back 32 channels $\mathbf{z} \in \mathbb{R}^{32}$:

- *Channel tokens.* Each channel value z_c is stretched by `Linear(1,32)` and added to a *learned* identity vector ι_c for that channel, giving $\mathbf{s}_c = z_c\mathbf{w}' + \beta' + \iota_c$. Why learned, when band tokens used a computed wavelength tag? **Channels have no wavelength.** Channel 7 is "whatever the compressor's 7th query measured"; its meaning is whatever training made it, so its tag must be learned too. The source says the same (`spectral_attention.py`, lines 110–112).
- *Wavelength queries.* For each requested wavelength, the Chapter 22 tag $\mathbf{e}(\lambda)$ passes through `Linear(32,32)`, then through the Q projection.
- *Attention.* Q, K, V and output projections, 4 heads: N queries against 32 keys.
- *To reflectance.* `Linear(32,1)` per answer.

The forward pass, trimmed from `SpectralCrossAttentionDecoder.forward` (lines 116–130):

```
rows = z.permute(0, 2, 3, 1).reshape(p, self.D, 1)                    #
     (P, 32, 1)
tokens = self.channel_value_embed(rows) + self.channel_identity       #
     (P, 32, d)
k = self.k(tokens).view(p, self.D, self.heads, hd).transpose(1, 2)    #
     (P, 4, 32, 8)
v = self.v(tokens).view(p, self.D, self.heads, hd).transpose(1, 2)
q = self.q(self.query_from_wavelength(wavelength_encoding(wl, self.d)))  #
     (N, d)
```

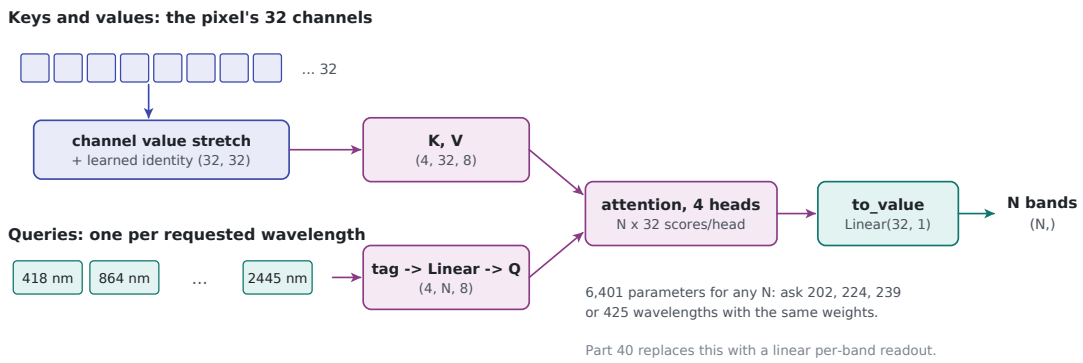


Figure 24.1: The decoder turned around. The pixel's 32 latent channels, each stretched and given a learned identity, supply keys and values; every requested wavelength becomes a query through its tag. Attention and a final Linear(32, 1) produce one reflectance per requested wavelength, so the same 6,401 parameters reconstruct any band list.

```
q = q.view(-1, n, self.heads, hd).transpose(1, 2).expand(p, -1, -1, -1)
    # (P, 4, N, 8)
out = _attend(q, k, v)                                                    #
     (P, 4, N, 8)
out = self.to_value(self.o(out.transpose(1, 2).reshape(p, n, self.d)))    #
     (P, N, 1)
```

Note what is absent: no mask. The decoder's keys are the 32 channels, which are never dead; dead input bands are handled at the compressor (Chapter 23) and in the loss (Section 24.1.7).

24.1.4 Predict the parameter count

Worked example 24.1: The cross-attention decoder's parameters by hand

Use $n_{\text{out}} \times n_{\text{in}} + n_{\text{out}}$ for each linear layer and rows × columns for a table, with $d = D = 32$. Do it on paper.

$$\begin{aligned}
\text{query from wavelength, Linear(32, 32)}: &\quad 32 \times 32 + 32 = 1{,}056 \\
\text{channel value stretch, Linear(1, 32)}: &\quad 1 \times 32 + 32 = 64 \\
\text{channel identities, table (32, 32)}: &\quad 32 \times 32 = 1{,}024 \\
\text{Q, K, V, O, 4} \times \text{Linear(32, 32)}: &\quad 4 \times 1{,}056 = 4{,}224 \\
\text{to reflectance, Linear(32, 1)}: &\quad 32 + 1 = 33
\end{aligned}$$

Total:

$$\begin{aligned}
1{,}056 + 64 + 1{,}024 + 4{,}224 + 33 &= 1{,}120 + 1{,}024 + 4{,}224 + 33 \\
&= 2{,}144 + 4{,}224 + 33 \\
&= 6{,}368 + 33 \\
&= 6{,}401.
\end{aligned}$$

The real module, instantiated on a CPU, reports 6,401 parameters [C]: 1,024 in `channel_identity`, 1,056 in `query_from_wavelength` (weight 1,024 + bias 32), 64 in `channel_value_embed`, 1,056 in each of `q`, `k`, `v`, `o`, and 33 in `to_value` (`research/textbook/tools/compute_specdecoder_output.txt`). The docstring says "Parameters at D = d = 32: 6,401, whatever N is", and `docs/lld/spectral-stem.md` invariant 5 pins it in a test.

In symbols, the count is

$$(d^2 + d) + 2d + Dd + 4(d^2 + d) + (d + 1) = 5d^2 + Dd + 8d + 1, \tag{24.1}$$

and at $d = D = 32$: $5{,}120 + 1{,}024 + 256 + 1 = 6{,}401$. Again **no band count appears**: today's decompressor (5,445) has one row per output band; this decoder has none.

24.1.5 One set of weights, any sensor

The same trained decoder, asked for four wavelength lists:

sensor	requested wavelengths	output shape
SpectralEarth	202	`(pixels, 202)`
EnMAP	224	`(pixels, 224)`
PRISMA	239	`(pixels, 239)`
AVIRIS-NG	425	`(pixels, 425)`

This is the property that makes "pretrain on SpectralEarth, fine-tune on PRISMA" work without resampling. The model never has a weight tied to a band position, on the way in or the way out. In `SpectralStemSegFormerMAE.forward` the requested list can even differ from the input list (`out_wavelengths_nm`), which band-list augmentation uses: read a random subset of bands, reconstruct all of them (`docs/lld/spectral-stem.md`, "Band-list robustness").

24.1.6 The cost, with roles swapped

The score matrix is N queries against 32 keys, the same $32 \times N$ size per head as the compressor's. But keys and values are now 32 per pixel instead of N, which is lighter. Counting the decoder's multiply-adds per pixel for $N = 202$, line by line:

$$\begin{aligned}
\text{channel tokens}: &\quad 32 \times 32 = 1{,}024 \\
\text{K, V}: &\quad 2 \times 32 \times 1{,}024 = 65{,}536 \\
\text{scores}: &\quad 202 \times 32 \times 32 = 206{,}848 \\
\text{weighted values}: &\quad 202 \times 32 \times 32 = 206{,}848 \\
\text{output projection}: &\quad 202 \times 1{,}024 = 206{,}848 \\
\text{to reflectance}: &\quad 202 \times 32 = 6{,}464 \\
\text{total}: &\quad 693{,}568.
\end{aligned}$$

(The wavelength queries are the same for every pixel and are computed once.) The output projection now runs once per *band*, not once per channel, so the decoder is nearly as expensive as the compressor's 861,184: per-band attention costs on both sides, a fact Section 24.4 will exploit.

24.1.7 The loss, on native bands

Chapter 25 built the loss as $L_1 + \lambda$ SAM, computed per pixel over the bands. With native bands nothing about that changes except which bands count:

- compare each reconstructed band with the measured band at the **same wavelength**; there is no grid in between;
- **skip dead and padded bands**: there is no truth to compare against;
- the spectral angle is computed over the bands that remain.

Worked example 24.2: A per-pixel L1 over valid bands only

Four bands, truth $[0.20, 0.30, \text{dead}, 0.10]$, reconstruction $[0.22, 0.27, 0.90, 0.10]$. Do it on paper.
Absolute errors on the valid bands only:

$$|0.22 - 0.20| = 0.02,$$
$$|0.27 - 0.30| = 0.03,$$
$$|0.10 - 0.10| = 0.00.$$

Sum and divide by the number of *valid* bands, 3:

$$(0.02 + 0.03 + 0.00)/3 = 0.05/3 = 0.0167.$$

The reconstruction's 0.90 in the dead band never enters. Dividing by 4 instead would wrongly report $0.05/4 = 0.0125$, rewarding a pixel for having a dead band.

The source does exactly this: invariant 3 of `docs/lld/spectral-stem.md` reads "A dead band never enters the loss – zeroed in prediction and target before SAM, and excluded from the per-pixel L1 mean." Zeroing both prediction and target in a band adds 0×0 to every dot product and every norm of the SAM formula, so the angle is computed over the remaining bands.

The spatial masking of Chapter 18 is untouched: which *pixels* are hidden and must be predicted from neighbours is the encoder's business, and still decides where the loss is taken.

24.1.8 Asking for wavelengths nobody measured

Any wavelength can be turned into a question, so the decoder will produce numbers for 2600 nm or for the middle of a water-vapour gap. Whether those numbers mean anything depends on whether training ever saw that region. Asking SpectralEarth-trained weights for 1400 nm is extrapolation, not measurement.

Source check: the guard against untrained wavelengths

The code turns this caution into a rule. `SpectralStemSegFormerMAE.trained_band_mask` (`spectral_stem_seg_former_mae.py`, lines 126–131) keeps a band only if it lies within 15 nm of a wavelength the model has trained on (the buffer `trained_wl`, which starts as the pretraining list and grows as fine-tuning reads other sensors). The evaluator and the product's inferencer drop the rest; for PRISMA set1 this removes 20 of 185 bands, those inside SpectralEarth's water-vapour gaps and past 2445 nm [M: `docs/lld/spectral-stem.md`].

24.2 One pixel start to finish, and the three traps

24.2.1 The trace

Shapes are for one patch ($B = 1$); the pixel we follow is one of its 16,384.

The new parts run strictly inside each pixel; everything that moves information between pixels is the old, tested machinery. That separation is what makes the first trap answerable.

Table 24.1: One SpectralEarth patch through the model with a wavelength stem. Rows 8–10 are the unchanged spatial model.

	step	shape	where
1	raw patch file	`(202, 128, 128)` int16	shard sample
2	reflectance, validity	`(202, 128, 128)` float	$\times 0.0001$; $-32768 \to$ invalid
3	normalise by wavelength	`(202, 128, 128)`	trap 2
4	pixels as rows	`(16384, 202)`	
5	band tokens	`(16384, 202, 32)`	Chapter 22
6	32 queries read them	`(16384, 32)`	Chapter 23
7	back to an image, BatchNorm	`(32, 128, 128)`	same as today
8	hide about 65% of tokens		Chapter 18
9	SegFormer encoder	four stages	Chapter 17
10	SegFormer decoder	`(32, 128, 128)`	Chapter 19
11	pixels as rows	`(16384, 32)`	
12	202 wavelength queries	`(16384, 202)`	Section 24.1
13	back to an image, denormalise	`(202, 128, 128)`	native bands
14	loss	$L_1 + \lambda\,\mathrm{SAM}$	hidden pixels, valid bands

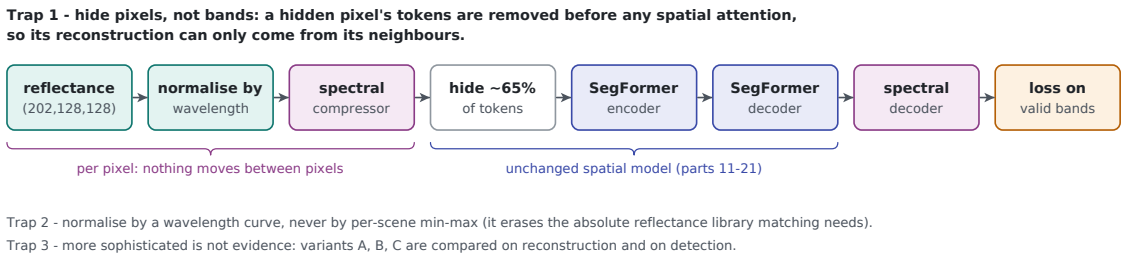


Figure 24.2: The trace in one row. The new spectral parts (plum) and wavelength normalisation work strictly inside each pixel; hiding, encoder and decoder are the unchanged spatial model; the loss counts valid bands only. Because hidden pixels are removed before any spatial attention, a pixel cannot rebuild its own spectrum.

24.2.2 Trap 1: a pixel must never rebuild its own spectrum

Chapter 9 and Chapter 18 rest on one rule: a hidden pixel is reconstructed from its neighbours, never from itself. Does spectral attention break it?

Not as designed. The compressor (step 6) mixes bands *within* a pixel and nothing between pixels. A pixel hidden at step 8 has its token removed before any spatial attention runs, so its compressed channels cannot reach any other pixel's reconstruction, and its own reconstruction (step 12) is built from what the encoder inferred from its neighbours. The rule holds.

It would break under a second, tempting kind of masking: hiding some *bands* of a visible pixel and asking the model to fill them from that pixel's other bands. That trains *spectral self-completion*. A model good at it can reconstruct an anomalous spectrum from the anomalous spectrum itself: the painter of Chapter 1 who has never seen a crashed aeroplane, painting it back in because half of it is still on the canvas. **Hide pixels, not bands.** Decision 7 of `docs/proposals/spectral-attention-bottleneck.md` states it, and the design LLD lists band masking for pretraining under "Rejected".

Source check: band-list augmentation is not trap 1

The code does train on random band subsets (`band_augment_prob`, `sample_band_subset`): a batch reads, say, 60% of its bands and reconstructs all of them. `docs/lld/spectral-stem.md` explains why this is not self-completion: "The loss is taken only on pixels hidden by the token mask … and a hidden pixel's own bands are never an input; it is rebuilt from its neighbours' band subsets. Scoring never augments." The pixel whose bands are being predicted contributes nothing to its own prediction. The LLD adds the condition under which this would stop being true: if the loss is ever extended to visible pixels, augmentation must be switched off for it.

24.2.3 Trap 2: normalise by wavelength, never by scene

Chapter 5 z-scores each of the 165 bands with a mean and standard deviation per band *position*. With native bands there are no fixed positions. The fix is statistics as a *function of wavelength*: one mean curve and one standard-deviation curve, measured once over SpectralEarth's training split, read off at any sensor's own wavelengths by linear interpolation.

Linear interpolation between two known points (λ_0, m_0) and (λ_1, m_1) estimates the value at λ between them as

$$m(\lambda) = m_0 + t\,(m_1 - m_0), \qquad t = \frac{\lambda - \lambda_0}{\lambda_1 - \lambda_0} \in [0, 1]. \tag{24.2}$$

t is the fraction of the way from λ_0 to λ_1.

Worked example 24.3: Reading the mean curve at 864 nm

The curve has mean 0.20 at 860 nm and 0.24 at 870 nm (illustrative numbers [A]). A sensor has a band at 864 nm. Do it on paper.

$$\begin{aligned} t &= \frac{864 - 860}{870 - 860} = \frac{4}{10} = 0.4, \\ m(864) &= 0.20 + 0.4 \times (0.24 - 0.20) \\ &= 0.20 + 0.4 \times 0.04 \\ &= 0.20 + 0.016 \\ &= 0.216. \end{aligned}$$

The standard deviation is read off the same way, and the band is normalised as $(r -$

$0.216)/\sigma(864)$.

Source check: `WavelengthStats`, as built

`app/foundation_models/components/pixel_normalization.py`, lines 44–81. Three details the course leaves out:

- The curve is *sorted* at load (lines 56–60), because SpectralEarth's band order is not monotonic in wavelength (the two EnMAP detectors overlap near 900–1000 nm) and interpolation needs a sorted curve.
- Outside the measured range the end values are held (`clamp`, line 64): a band at 2480 nm reads the curve's value at 2445.3 nm.
- The interpolation is `torch.lerp(mean[lo],mean[hi],t)`, exactly Equation (24.2).

Invariant 6 of `docs/lld/spectral-stem.md`: the curve is fixed at pretraining and travels inside the checkpoint; fine-tuning and inference never recompute it, because changing it would shift every input the pretrained weights learned to read.

The tempting shortcut is to normalise each scene by its own minimum and maximum. Don't. It erases absolute reflectance, and absolute reflectance is what spectral library matching against USGS splib07 depends on (Chapter 32, and the companion volume on spectral matching). An external model reviewed for this project, HyperSL, does exactly that per-scene min-max scaling, and it is the reason its outputs cannot be matched against the library.

24.2.4 Trap 3: "more sophisticated" is not evidence

Chapters 22 and 23 make a strong case. None of it shows that the new model finds anomalies better, and a harder-to-optimise bottleneck can lose to a simple one. So the design was judged by a controlled comparison, written down before any run: same SpectralEarth subset, same training budget, same spatial model, same normalisation.

Table 24.2: The three variants of the stem experiment, from the design proposal on the spectral attention bottleneck.

variant	spectral stem	input bands
A	today's 1×1 convolution, in and out	202 bands as fixed positions
B	per-pixel gate, then A (Section 24.3)	202 bands as fixed positions
C	cross-attention compressor and decoder (later C′, the reduced stem of Section 24.4)	202 wavelength tokens

Two scores per variant: reconstruction on the held-out SpectralEarth sites (L1 and SAM), and anomaly detection on the labelled PRISMA scenes, the job the model exists for. The decision rule was fixed in advance: C proceeds only if it beats A on detection in mean ROC-AUC across the four scenes; if B matches C within the seed spread of A, B is preferred on cost; if neither beats A, the spectral stem is not what limits detection. Chapter 27 tells what the runs found.

24.3 Per-pixel gating: the cheap rival

24.3.1 Why a rival at all

If variant C beats A, you still do not know *why*, because two different things changed at once:

1. **adaptivity**: each pixel gets its own recipe instead of one for all;
2. **wavelength awareness and masking**: bands become tokens that know their wavelength, and dead bands are skipped exactly.

Variant B adds only the first. If B gets most of C's gain, the gain came from adaptivity, and B delivers it for a fraction of the cost. In the mixing-desk picture of Chapter 22, B keeps the glued desk and adds a row of faders in front of it that each song sets for itself.

24.3.2 A new function: the sigmoid

Definition 24.1 (Sigmoid)**.** The *sigmoid* squashes any real number into the open interval $(0, 1)$:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{24.3}$$

Worked example 24.4: Four values of the sigmoid

Do each on paper.

$$\begin{aligned}
z = 0: &\quad e^{0} = 1, &&\sigma = 1/(1+1) = 0.5,\\
z = 2: &\quad e^{-2} = 0.1353, &&\sigma = 1/(1+0.1353) = 1/1.1353 = 0.8808,\\
z = -2: &\quad e^{2} = 7.3891, &&\sigma = 1/(1+7.3891) = 1/8.3891 = 0.1192,\\
z = 5: &\quad e^{-5} = 0.0067, &&\sigma = 1/(1+0.0067) = 1/1.0067 = 0.9933.
\end{aligned}$$

Large positive inputs land near 1, large negative ones near 0, zero lands at exactly 0.5. Notice that $\sigma(2) + \sigma(-2) = 0.8808 + 0.1192 = 1$. That is general: multiply the numerator and denominator of $\sigma(-z) = 1/(1 + e^{z})$ by e^{-z} to get $e^{-z}/(e^{-z} + 1)$, and add $\sigma(z) = 1/(1 + e^{-z})$:

$$\sigma(z) + \sigma(-z) = \frac{1 + e^{-z}}{1 + e^{-z}} = 1.$$

The sigmoid never reaches 0 or 1 exactly: e^{-z} is positive for every finite z, so the denominator in Equation (24.3) is always strictly greater than 1 (giving $\sigma < 1$) and always finite (giving $\sigma > 0$). Keep that in mind; it matters in Section 24.3.7.

24.3.3 The gate

Today (Chapter 12), one recipe $\mathbf{W}$ for every pixel: $\mathbf{y} = \mathbf{W}\mathbf{x}$. With a gate, each pixel first computes a volume setting per band from *its own spectrum*:

$$\begin{aligned}
\mathbf{s} &= \mathbf{W}_2\ \mathrm{GELU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2, && \mathbf{s} \in \mathbb{R}^N \text{ (one score per band)}\\
\mathbf{g} &= \sigma(\mathbf{s}), && \text{each } g_b \in (0, 1)\\
\mathbf{x}' &= \mathbf{g} \odot \mathbf{x}, && \text{each band scaled by its gate}\\
\mathbf{y} &= \mathbf{W}\mathbf{x}', && \text{today's compressor, unchanged.}
\end{aligned} \tag{24.4}$$

Here $\odot$ is element-by-element multiplication, $\mathbf{W}_1$ is $32 \times N$ and $\mathbf{W}_2$ is $N \times 32$, and GELU is the smooth activation of Chapter 16. The small network in front squeezes the spectrum to 32 numbers and expands it back to one score per band: the "MLP" in $\sigma(\mathrm{MLP}(\mathbf{x}))$.

Source check: the gate is built from 1×1 convolutions

`app/foundation_models/components/spectral_gate.py`, lines 26–32, builds the network as `nn.Conv2d(bands,hidden,kernel_size=1),nn.GELU(),nn.Conv2d(hidden,bands,kernel_size=1)` and returns `x*torch.sigmoid(self.score(x))`. A 1×1 convolution is the same linear map at every pixel, so this is Equation (24.4) applied pixel by pixel, with exactly the parameters of two `Linear` layers. In the model, the gate stem is `nn.Sequential(SpectralGate(bands,gate_hidden),Conv2d(bands,D,1))` followed by the shared BatchNorm (`spectral_stem_seg_former_mae.py`, line 71).

Why sigmoid and not softmax? A softmax over the bands would make the gates compete: they would sum to 1, so with 202 bands the average gate would be $1/202 = 0.005$, and turning one band up would force the others down. Compare the two on scores $[2, 2, -2, -2]$. The sigmoid gives $[0.8808, 0.8808, 0.1192, 0.1192]$, each band its own volume. The softmax: exponentials $7.3891, 7.3891, 0.1353, 0.1353$, sum 15.0488, weights $[0.4910, 0.4910, 0.0090, 0.0090]$, a share of a fixed total. A volume knob should not flip a band's sign, should not amplify it without limit, and should not depend on how many other knobs exist: the sigmoid satisfies all three.

Worked example 24.5: Gating the toy pixel

The toy pixel and recipe of Chapter 22: $\mathbf{x} = [1.2, 1.0, -0.4, -0.6]$, $\mathbf{W}$ with rows $[0.5, 0.5, 0, 0]$ and $[0, 0, 0.5, 0.5]$, so today $\mathbf{y} = [1.1, -0.5]$. Suppose the gate network decides that the first two bands are unreliable (a shadowed pixel, say) and outputs scores $\mathbf{s} = [-2, -2, 2, 2]$. Do it on paper.

Gates, from worked example 24.4, rounded to two places:

$$\mathbf{g} = [\sigma(-2), \sigma(-2), \sigma(2), \sigma(2)] = [0.12, 0.12, 0.88, 0.88].$$

Scale each band:

$$\begin{aligned}\mathbf{x}' &= [0.12 \times 1.2,\ 0.12 \times 1.0,\ 0.88 \times (-0.4),\ 0.88 \times (-0.6)] \\ &= [0.144,\ 0.120,\ -0.352,\ -0.528].\end{aligned}$$

Compress:

$$\begin{aligned}y_0 &= 0.5 \times 0.144 + 0.5 \times 0.120 = 0.072 + 0.060 = 0.132, \\ y_1 &= 0.5 \times (-0.352) + 0.5 \times (-0.528) = -0.176 - 0.264 = -0.440.\end{aligned}$$

y_0 fell from 1.1 to 0.132: for this pixel, the first output barely listens to bands 0 and 1.

24.3.4 Every pixel gets W times a diagonal

Look at the gate the other way. Write $\mathrm{diag}(\mathbf{g})$ for the square matrix with $\mathbf{g}$ on its diagonal and zeros elsewhere; multiplying a vector by it scales each component, so $\mathbf{g} \odot \mathbf{x} = \mathrm{diag}(\mathbf{g})\mathbf{x}$. Then

$$\mathbf{y} = \mathbf{W}(\mathrm{diag}(\mathbf{g})\mathbf{x}) = (\mathbf{W}\,\mathrm{diag}(\mathbf{g}))\,\mathbf{x},$$

by associativity of the matrix product. Multiplying $\mathbf{W}$ on the right by a diagonal matrix scales its *columns*: entry (c, b) of $\mathbf{W}\,\mathrm{diag}(\mathbf{g})$ is $\sum_k W_{ck}\,[\mathrm{diag}(\mathbf{g})]_{kb} = W_{cb}\,g_b$, since only $k = b$ survives. For the toy pixel:

$$\mathbf{W}\,\mathrm{diag}(\mathbf{g}) = \begin{bmatrix} 0.5 \times 0.12 & 0.5 \times 0.12 & 0 & 0 \\ 0 & 0 & 0.5 \times 0.88 & 0.5 \times 0.88 \end{bmatrix} = \begin{bmatrix} 0.06 & 0.06 & 0 & 0 \\ 0 & 0 & 0.44 & 0.44 \end{bmatrix}.$$

Every pixel gets its own recipe, but every recipe is today's W with its columns rescaled. That sentence is both B's strength and its limit: the gate can turn a band down for all 32 outputs at once, but it cannot make band 3 matter more to output 0 than to output 1 in one pixel and the reverse in the next. Cross-attention's recipe, $\kappa_h a_{cb}^{(h)}$ in Chapter 23, has a separate weight per (channel, band) pair for every pixel.

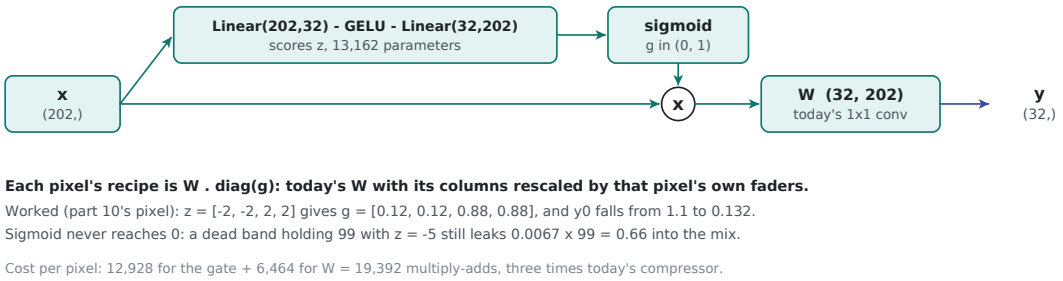


Figure 24.3: Variant B. A small network reads each pixel's own spectrum and emits one score per band; sigmoid turns the scores into gates between 0 and 1, the bands are scaled by them, and today's fixed 1×1 compressor mixes the result. Each pixel therefore gets the recipe $\mathbf{W}\,\mathrm{diag}(\mathbf{g})$, today's $\mathbf{W}$ with rescaled columns, at three times today's arithmetic.

24.3.5 Predict the parameter count

Worked example 24.6: The gate's parameters at 202 bands

Hidden width 32, $N = 202$. Do it on paper.

$$\begin{aligned} \text{first layer, } 202 \to 32: &\quad 202 \times 32 + 32 = 6{,}464 + 32 = 6{,}496 \\ \text{GELU}: &\quad 0 \\ \text{second layer, } 32 \to 202: &\quad 32 \times 202 + 202 = 6{,}464 + 202 = 6{,}666 \\ \text{gate total}: &\quad 6{,}496 + 6{,}666 = 13{,}162. \end{aligned}$$

The real module reports 13,162 (6,464 + 32 in the first layer's weight and bias, 6,464 + 202 in the second's) [C], matching the docstring of `SpectralGate` and invariant 5 of the LLD; at 165 bands it reports 10,757. Variant A's compressor on 202 bands is $202 \times 32 + 32 = 6{,}496$, plus the BatchNorm's 64, so B's stem has $13{,}162 + 6{,}496 + 64 = 19{,}722$ parameters, against C's 5,345. **B is the bigger model and the cheaper one**: parameters and arithmetic are different things, as Chapter 23 showed.

24.3.6 What it costs

Multiply-adds per pixel, 202 bands:

$$\begin{aligned} \text{A, compressor}: &\quad 202 \times 32 = 6{,}464 \\ \text{B, gate}: &\quad 2 \times 202 \times 32 = 12{,}928, \text{ plus the compressor } 6{,}464, \\ &\quad 12{,}928 + 6{,}464 = 19{,}392 \\ \text{C, cross-attention (Chapter 23)}: &\quad 861{,}184 \end{aligned}$$

Ratios: $19{,}392/6{,}464 = 3.0$, so B costs three times A; $861{,}184/19{,}392 = 44.4$, so C costs 44 times B. (The 163 of Chapter 23 compared C with the shipped 165-band compressor, 5,280.)

24.3.7 What a gate cannot do

Table 24.3: Gate against cross-attention.

	B: gate	C: cross-attention
each pixel its own recipe	yes, $\mathbf{W}\,\mathrm{diag}(\mathbf{g})$	yes
bands influence each other	through one 32-number squeeze, and only by setting volumes	yes, each output channel is its own weighted mix
exact skip of a dead band	no: the gate never reaches 0	yes: $-\infty$ gives weight exactly 0
knows a band's wavelength	no: it knows "input 87"	yes
works on another band list	no: retrain the input layers	yes: pass the new wavelengths

The dead-band row deserves arithmetic. A dead band holding the value 99 (the 99 test of Chapter 23) and gated as hard as $z = -5$ still leaks

$$\sigma(-5) \times 99 = 0.0067 \times 99 = 0.66$$

into the mix, more than most real normalised values. Even $z = -10$, $\sigma(-10) = 1/(1+22{,}026) = 0.0000454$, leaks $0.0000454 \times 99 = 0.0045$. In the full model the problem is softened because `_stem` sets dead voxels to 0 before any stem sees them (Chapter 23), but a zero is still a claim, not an absence.

How to read the experiment's outcome:

outcome	conclusion
B about equal to C, both beat A	adaptivity was the win; prefer B on cost
C beats B by a margin, B beats A	adaptivity helps, and wavelength awareness and masking add more
neither beats A	the spectral stem is not what limits detection

24.4 Spectral reduction: attention over wavelength bins

24.4.1 Why: the per-band stem hits the memory wall

Chapter 23 measured about 3.7 GiB of training memory per 128×128 patch for the per-band stem: about 19 kept tensors of shape (16,384 pixels, 202 bands, 32), 202 MiB each in bf16. A 48 GB card fits only a handful of patches per step, and the measured speed was 13.5 patches per second. A pretraining run of tens of thousands of patches per epoch for dozens of epochs becomes days per variant. On 2026-10-14 the experiment moved its attention variant from `cross_attention` to `reduced_attention`, the stem of this section (`docs/proposals/spectral-attention-bottleneck.md`, status line).

The model's `stem` argument takes four values: `linear` (A), `gate` (B), `cross_attention` (C, Chapter 23 and Section 24.1) and `reduced_attention` (C′).

24.4.2 The spatial trick, turned sideways

Chapter 15 met the same kind of wall in space. Attention over the 1,024 tokens of a 32×32 grid would compare every token with every token, so SegFormer's efficient self-attention [46]

Table 24.4: The spectral modules as they exist in `app/foundation_models/components/`.

what	file
per-band cross-attention compressor and decoder	`spectral_attention.py`
wavelength encoding	`spectral_tokens.py`
per-pixel gate	`spectral_gate.py`
reduced stem (this section)	`spectral_reduced.py`
the model that swaps between them	`spectral_stem_seg_former_mae.py`

shrinks the keys and values to a small summary grid before attending, and keeps the queries at full detail. The reduced stem does the same along the wavelength axis:

Table 24.5: Spatial reduction (Chapter 15) and spectral reduction side by side.

	spatial reduction	spectral reduction
what is too many	1,024 tokens on a 32×32 grid	202 band tokens per pixel
how it shrinks	strided convolution, $R \times R$ tokens $\to 1$	fixed 40 nm bins, the bands inside $\to 1$
what attention reads	16 summary tokens	about 42 bin tokens
who asks	every full-resolution token	the 32 learned queries

24.4.3 The bins

The bins are a fixed grid, the same for every sensor, starting at 400 nm. From `spectral_reduced.py`, lines 38, 78 and 87:

```
BIN_ORIGIN_NM, BIN_END_NM = 400.0, 2500.0
self.n_bins = int(math.ceil((BIN_END_NM - BIN_ORIGIN_NM) / bin_nm))      # bin_nm =
    40
return ((wavelengths_nm - BIN_ORIGIN_NM) / self.bin_nm).floor().long().clamp(0, self.
    n_bins - 1)
```

Two functions appear. The *floor* $\lfloor u \rfloor$ is the largest whole number not above u ($\lfloor 11.6 \rfloor = 11$); the *ceiling* $\lceil u \rceil$ is the smallest whole number not below u ($\lceil 52.5 \rceil = 53$). The bin of a wavelength λ is

$$g(\lambda) = \left\lfloor \frac{\lambda - 400}{40} \right\rfloor, \quad \text{clamped to } 0, \ldots, G-1, \qquad G = \left\lceil \frac{2500 - 400}{40} \right\rceil. \tag{24.5}$$

Worked example 24.7: Counting the bins and placing three bands

How many bins? Do it on paper.

$$\frac{2500 - 400}{40} = \frac{2100}{40} = 52.5,$$
$$\lceil 52.5 \rceil = 53 \text{ bins.}$$

The last bin, number 52, covers 2480–2500 nm, only half the usual width, because 2100 is not a whole multiple of 40.

Place three real SpectralEarth bands (`app/constants/spectralearth_bands.json`):

$$\begin{aligned}
418.416 \text{ nm}: &\quad (418.416 - 400)/40 = 18.416/40 = 0.46, &\quad \lfloor 0.46 \rfloor = 0 &\quad (400\text{–}440 \text{ nm}) \\
864 \text{ nm}: &\quad (864 - 400)/40 = 464/40 = 11.6, &\quad \lfloor 11.6 \rfloor = 11 &\quad (840\text{–}880 \text{ nm}) \\
2445.3 \text{ nm}: &\quad (2445.3 - 400)/40 = 2045.3/40 = 51.13, &\quad \lfloor 51.13 \rfloor = 51 &\quad (2440\text{–}2480 \text{ nm})
\end{aligned}$$

Bin g covers $400 + 40g$ to $400 + 40(g + 1)$ nm, and its centre, used for its tag below, is $400 + 40(g + 0.5)$: 420, 460, …, 2500 nm.

Doing that for all 202 bands (`research/textbook/tools/compute_specdecoder.py`) gives:

- bins holding at least one band: 42 of 53 [C];
- bands per occupied bin: 1 to 9, average $202/42 = 4.8$;
- empty bins: 23–26 (1320–1480 nm), 33–38 (1720–1960 nm) and 52 (2480–2500 nm) [C].

The empty bins sit in the water-vapour gaps SpectralEarth already removed (EnMAP bands 127–141 and 161–167, Chapter 3) and past its last band at 2445.3 nm. An empty bin is masked out of attention exactly as Chapter 23 masked a dead band, so attention effectively reads 42 tokens. **Because the grid starts at 400 nm on every sensor, bin 11 means 840–880 nm on SpectralEarth, EnMAP, PRISMA and AVIRIS-NG alike.** A bin is a wavelength range, not a slot.

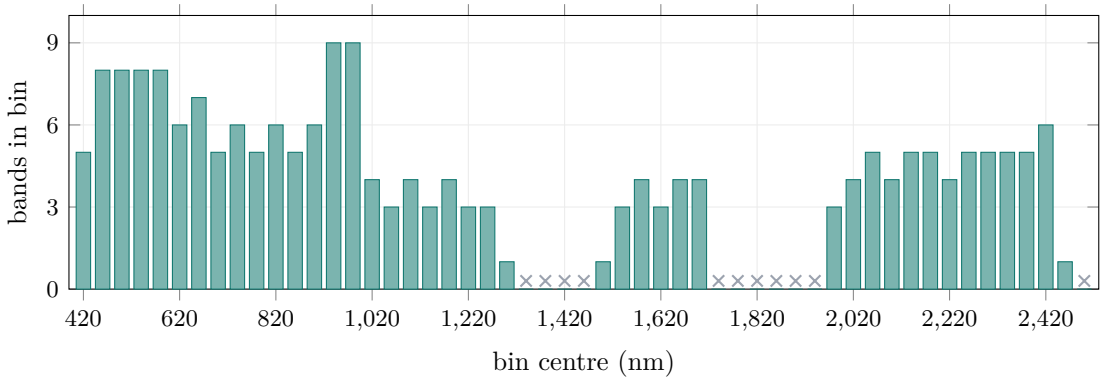


Figure 24.4: SpectralEarth's 202 bands counted into the 53 fixed 40 nm bins, to scale, from the real wavelength table (this book's script). Grey crosses mark the 11 empty bins, masked out of attention. The two bins of 9 (920–1000 nm) are where EnMAP's two detectors overlap.

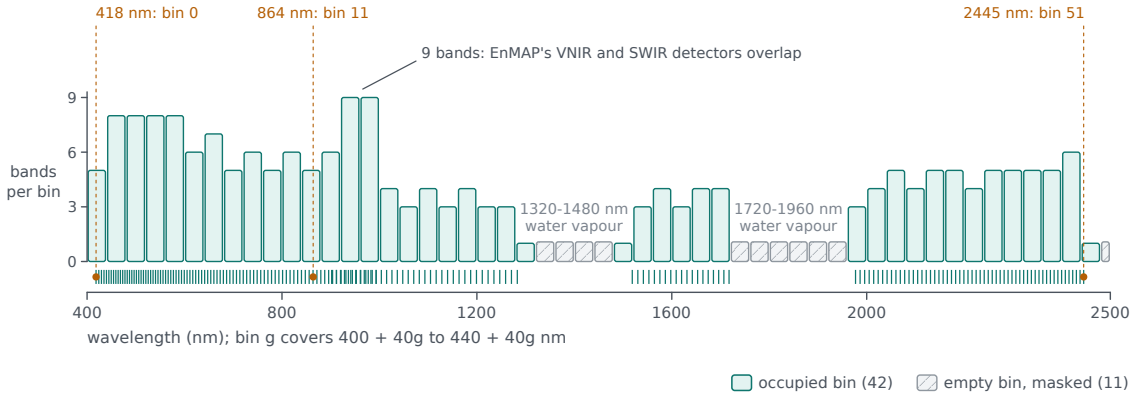


Figure 24.5: The fixed bin grid as the course draws it. The 400 to 2500 nm axis is cut into 53 bins of 40 nm (top); each tick below is one of SpectralEarth's 202 band centres, and the bar height is the number of bands in that bin (1 to 9). 42 bins hold at least one band; the 11 empty bins (hatched) fall in the two water-vapour gaps and at the long end.

24.4.4 Why a plain bin average would lose detail

The obvious way to make one token per bin is to average the bin's band tokens. Use the per-band stem's tokens, $\mathbf{t}_b = v_b\mathbf{w} + \beta + \mathbf{e}(\lambda_b)$, where $\mathbf{w}, \beta$ are the shared `Linear(1,32)` of Chapter 22. Sum two bands of one bin:

$$\begin{aligned}\mathbf{t}_1 + \mathbf{t}_2 &= v_1\mathbf{w} + \beta + \mathbf{e}(\lambda_1) + v_2\mathbf{w} + \beta + \mathbf{e}(\lambda_2)\\ &= (v_1 + v_2)\,\mathbf{w} + 2\beta + \mathbf{e}(\lambda_1) + \mathbf{e}(\lambda_2).\end{aligned}$$

The only part that depends on the measurements is $(v_1 + v_2)\mathbf{w}$. **Everything about the individual values is gone except their sum.**

Worked example 24.8: A dip and its mirror image collide

Spectrum A has a dip then its shoulder in one bin, $\mathbf{v} = [0.2, 0.6]$; spectrum B the reverse, $[0.6, 0.2]$.

$$\begin{aligned}\text{sum A} &= 0.2 + 0.6 = 0.8,\\ \text{sum B} &= 0.6 + 0.2 = 0.8.\end{aligned}$$

Identical, so the averaged tokens are identical. A narrow absorption dip and its mirror image become indistinguishable, and narrow dips are exactly what mineral and gas signatures look like.

24.4.5 Fix 1: every band gets its own weight vector inside its bin

Instead of the shared $\mathbf{w}$, each band's value is multiplied by a vector *computed from its wavelength*, $\mathbf{k}(\lambda_b) \in \mathbb{R}^{32}$, and the bin token is the mean over the bin's valid bands:

$$\tau_g = \frac{1}{n_g} \sum_{b \in g,\ b \text{ valid}} v_b\,\mathbf{k}(\lambda_b), \tag{24.6}$$

where n_g is the number of valid bands in bin g. The function $\mathbf{k}$ is a small network, `_hyper` in the source, applied to the Chapter 22 wavelength tag:

```
def _hyper(out_dim, enc_dim=32, hidden=64):          # spectral_reduced.py, lines
    56-58
    return nn.Sequential(nn.Linear(enc_dim, hidden), nn.GELU(), nn.Linear(hidden,
    out_dim))
```

A network that outputs the weights of another computation is called a *hypernetwork*, hence the name.

Worked example 24.9: Fix 1 separates the dip from its mirror

Use two-number tokens and suppose training gave $\mathbf{k}(\lambda_1) = [1, 0]$ and $\mathbf{k}(\lambda_2) = [0, 1]$. Do it on paper.
Spectrum A, $[0.2, 0.6]$:

$$\begin{aligned}\tfrac{1}{2}(0.2 \times [1,0] + 0.6 \times [0,1]) &= \tfrac{1}{2}([0.2, 0] + [0, 0.6])\\ &= \tfrac{1}{2}[0.2, 0.6] = [0.1, 0.3].\end{aligned}$$

Spectrum B, $[0.6, 0.2]$:

$$\begin{aligned}\tfrac{1}{2}(0.6 \times [1,0] + 0.2 \times [0,1]) &= \tfrac{1}{2}([0.6, 0] + [0, 0.2])\\ &= \tfrac{1}{2}[0.6, 0.2] = [0.3, 0.1].\end{aligned}$$

$[0.1, 0.3]$ and $[0.3, 0.1]$: different tokens.

Why does this work in general? Equation (24.6) is a linear map from the bin's n_g values to $\mathbb{R}^{32}$, with matrix columns $\mathbf{k}(\lambda_b)/n_g$. A linear map is one-to-one exactly when its columns are linearly independent (if two inputs gave the same output, their difference would be a non-zero vector sent to zero, a null-space vector in the sense of Chapter 22). A bin holds at most 9 bands and the token has 32 numbers, so 9 or fewer columns in $\mathbb{R}^{32}$ *can* be independent. **Nothing forces two in-bin spectra to collide**, while the shared $\mathbf{w}$ of the plain average gives every column the same direction, rank 1, and a null space of dimension $n_g - 1$.

The token then gets a second term saying where the bin sits, `bin_bias`, a `Linear(32,32)` of the tag of the bin's centre wavelength (line 108):

```
tok = tok + self.bin_bias(wavelength_encoding(centres, self.d))[None]   # centres:
    420, 460, ... nm
```

24.4.6 Attention over the bins

From here it is Chapter 23's compressor with bin tokens in place of band tokens: the 32 learned queries are projected to Q, the bin tokens to K and V, 4 heads attend with empty bins masked, and the output projection and `Linear(32,1)` turn each of the 32 answers into one channel.

tensor	shape	note
bin tokens	`(P, 53, 32)`	53 slots, the empty ones masked
K, V	`(P, 4, 53, 8)`	4 heads $\times$ 8 = 32
Q	`(P, 4, 32, 8)`	the same 32 learned queries for every pixel
mask	`(P, 1, 1, 53)`	count > 0: bin has a valid band
channels	`(P, 32)`	the attention half of $\mathbf{z}$

Attention still decides, per pixel, how much each region of the spectrum matters: the first requirement of Chapter 22. What it no longer does is choose between individual bands; that choice is now made at bin granularity.

24.4.7 Fix 2: a per-band skip path straight to the channels

Attention over bins adds adaptivity, but it is a narrow route. So every valid band also reaches the 32 channels directly:

$$\text{skip} = \frac{1}{n} \sum_{b \text{ valid}} v_b \, \mathbf{W}(\lambda_b), \qquad \mathbf{z} = \text{attention output} + \text{skip}, \tag{24.7}$$

where $\mathbf{W}(\lambda) \in \mathbb{R}^{32}$ comes from a second `_hyper` and n is the number of valid bands of the pixel. Look at what that is. The 1×1 compressor was $\mathbf{y} = \mathbf{W}\mathbf{x} = \sum_b x_b \, \mathbf{w}_{:,b}$: each band's value times its column of $\mathbf{W}$, summed. Here each band's column is *computed from its wavelength* instead of stored per slot, and the sum is divided by the band count. **The skip path is today's 1×1 compressor, made to work for any band list.**

Worked example 24.10: The skip path on the toy pixel

Take the computed columns to be the toy compressor's columns (two channels instead of 32): $\mathbf{W}(\lambda_0) = [0.5, 0]$, $\mathbf{W}(\lambda_1) = [0.5, 0]$, $\mathbf{W}(\lambda_2) = [0, 0.5]$, $\mathbf{W}(\lambda_3) = [0, 0.5]$, and $\mathbf{x} =$

$[1.2, 1.0, -0.4, -0.6]$. Do it on paper.

$$\begin{aligned}\text{sum} &= 1.2[0.5, 0] + 1.0[0.5, 0] + (-0.4)[0, 0.5] + (-0.6)[0, 0.5] \\ &= [0.6, 0] + [0.5, 0] + [0, -0.2] + [0, -0.3] \\ &= [1.1, -0.5] \qquad \text{(the toy compressor's } \mathbf{y}\text{, as promised)}, \\ \text{mean} &= [1.1, -0.5]/4 = [0.275, -0.125].\end{aligned}$$

Why divide by the count? Without it, a 239-band PRISMA pixel would sum more terms than a 202-band SpectralEarth pixel and land on a bigger scale. The mean keeps the scale independent of the sensor; the BatchNorm after the stem absorbs the remaining constant factor.

24.4.8 Dead bands drop out of every mean exactly

The source multiplies by validity *before* anything is summed, and counts only valid bands (lines 97–103):

```
valid    = torch.ones_like(x) if band_valid is None else band_valid.to(x.dtype)
vals     = _rows(x * valid)          # (P, N), a dead band becomes 0
cnt_band = _rows(valid)              # (P, N), 1 per valid band
onehot   = F.one_hot(self.bin_index(wl), G).to(x.dtype)   # (N, G) which bin each
    band is in
cnt      = cnt_band @ onehot         # (P, G) valid bands per bin
```

Worked example 24.11: The skip path with a dead band holding 99

Redo worked example 24.10 with band 2 dead and holding the garbage value 99.

$$\begin{aligned}\text{vals} &= [1.2,\ 1.0,\ 99 \times 0,\ -0.6] = [1.2,\ 1.0,\ 0,\ -0.6], \\ \text{sum} &= [0.6 + 0.5,\ 0 + (-0.3)] = [1.1,\ -0.3], \\ \text{mean} &= [1.1, -0.3]/3 = [0.3667, -0.1] \qquad \text{(3 valid bands, not 4)}.\end{aligned}$$

The 99 never appears.

The 99 test of Chapter 23 on the real reduced module (202 SpectralEarth wavelengths, band 100 dead, its value changed to 99.0) gives a largest output change of 0.0, both in the course's run and in this book's (`research/textbook/tools/compute_specdecoder_output.txt`) [C]. The bin tokens get the same treatment through `cnt`, and a bin whose count falls to 0 is masked out of attention.

Source check: `band_weighting = "width"`, a later fix the course does not show

`SpectralReducedCompressor` takes a `band_weighting` argument, `"count"` (the default, everything above) or `"width"` (`spectral_reduced.py`, lines 69–99). With `"width"`, each band counts by the nanometres of spectrum it stands for, computed by `band_widths` (lines 43–53): half the gap to each neighbour, the single gap at either end, gaps capped at `MAX_GAP_NM=20`. Worked for wavelengths $[400, 410, 430, 470]$: the gaps are $10, 20, 40$, capped to $10, 20, 20$; the widths are

$$[10,\ (10+20)/2,\ (20+20)/2,\ 20] = [10,\ 15,\ 20,\ 20].$$

The validity mask is multiplied by these widths, so every "mean" above becomes a width-weighted mean, an approximate integral over wavelength. On an evenly spaced list all widths are equal and the result equals `"count"` exactly. The fix was adopted on 2026-10-14

after E004 read PRISMA's native bands badly: with a plain $1/n$ mean every band has one vote, so a sensor's band *density* changed the mix (`docs/lld/spectral-stem.md`, "Band-list robustness").

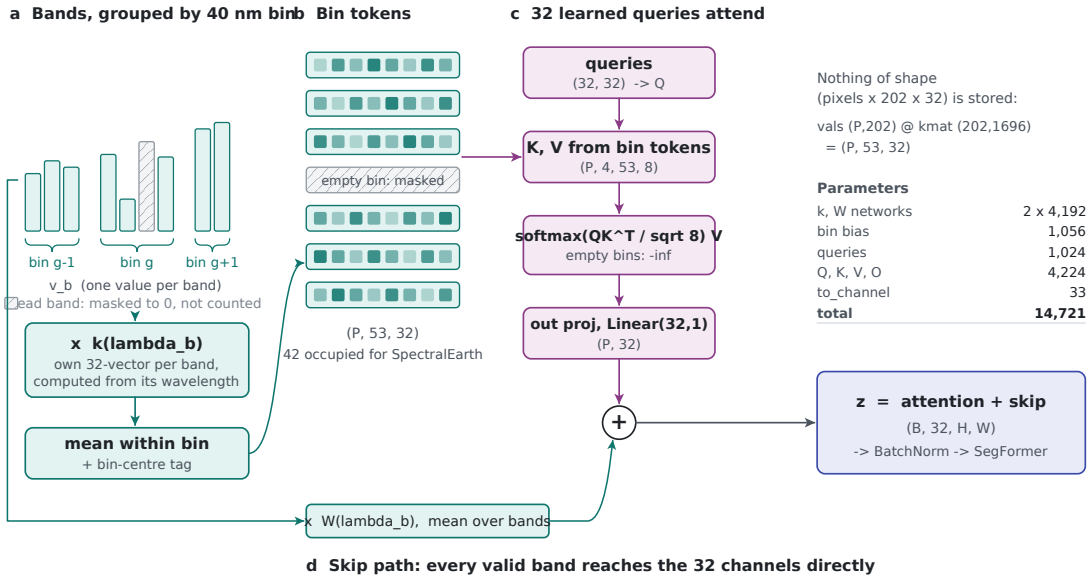


Figure 24.6: The reduced compressor for one pixel. Left: band values (teal) are masked by validity (grey hatching marks a dead band) and multiplied by a per-band weight vector $k(\lambda)$ computed from each band's wavelength; the products are averaged within each 40 nm bin and given the bin centre's tag, giving about 42 bin tokens. Centre: 32 learned queries attend over the bin tokens (plum), empty bins masked, and each answer becomes one channel. Bottom: the skip path multiplies every valid band by $W(\lambda)$ and averages, the 1×1 compressor with wavelength-computed columns. The two halves add into the 32-channel latent z.

24.4.9 Fix 3: a per-band decoder without attention

On the way out there is no attention at all. Each requested wavelength computes its own output weights:

$$\hat{x}_n = \mathbf{U}(\lambda_n) \cdot \mathbf{z} + c(\lambda_n), \qquad \mathbf{U}(\lambda) \in \mathbb{R}^{32},\ c(\lambda) \in \mathbb{R}. \tag{24.8}$$

One `_hyper` network with 33 outputs produces both: the first 32 are $\mathbf{U}$, the last is c (`WavelengthLinearDecoder`, lines 122–136). That is the 1×1 decompressor with each output row computed from its wavelength, so it reconstructs 202, 224 or 239 bands from the same weights, like the attention decoder of Section 24.1.

Worked example 24.12: Decoding one band

Two channels, one wavelength. Take $\mathbf{z} = [1.1, -0.5]$, and suppose the network gives $\mathbf{U}(660) = [0.8, 0.1]$ and $c(660) = 0.05$. Do it on paper.

$$\begin{aligned}\hat{x} &= 0.8 \times 1.1 + 0.1 \times (-0.5) + 0.05 \\ &= 0.88 + (-0.05) + 0.05 \\ &= 0.88.\end{aligned}$$

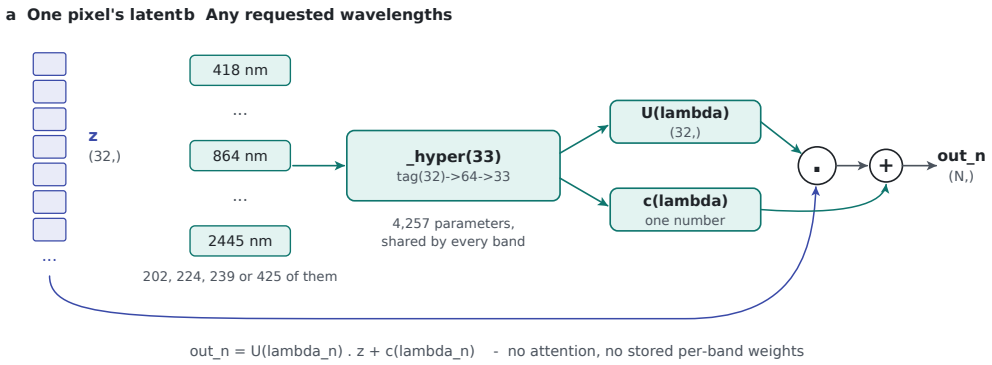


Figure 24.7: The wavelength-linear decoder. Each requested wavelength (any list: 202 SpectralEarth, 224 EnMAP, 239 PRISMA bands) passes through one small network that emits a 32-number weight row $U(\lambda)$ and a bias $c(\lambda)$; the band's value is the dot product of that row with the pixel's latent z, plus the bias. No attention and no per-band stored weights: the same 4,257 parameters serve every band list.

24.4.10 Nothing of shape (pixels, bands, 32) is stored

Fix 1 reads as "multiply every band by a 32-number vector", which sounds like it builds the very `(P, 202, 32)` tensor the stem exists to avoid. It does not. The per-band vectors $\mathbf{k}(\lambda_b)$ do not depend on the pixel, so the source folds them into one matrix first (lines 105–106):

```
kmat = (onehot[:, :, None] * self.band_weight(enc)[:, None, :]).reshape(n, G * self.d
    )
tok  = (vals @ kmat).view(p, G, self.d) / cnt.clamp(min=1)[..., None]
```

Table 24.6: The tensors of the folded computation, 202 bands, 53 bins.

tensor	shape	pixel axis?
`onehot`: bin of each band	`(202, 53)`	no
`band_weight(enc)`: $\mathbf{k}(\lambda_b)$ per band	`(202, 32)`	no
`kmat`: $\mathbf{k}(\lambda_b)$ in its bin's slot	`(202, 1696)`	no
`vals`	`(P, 202)`	yes, 1 number per band
`vals @ kmat`	`(P, 1696)` → `(P, 53, 32)`	yes, but 53 slots, not 202

Row b of `kmat` is zero everywhere except the 32 entries of bin $g(\lambda_b)$'s slot, which hold $\mathbf{k}(\lambda_b)$; $53 \times 32 = 1{,}696$. So entry (p, g, j) of `vals@kmat` is $\sum_b v_{pb}\,[g(\lambda_b) = g]\,k_j(\lambda_b)$: exactly the sum in Equation (24.6). One matrix product over the band axis does the multiply, the placement into bins and the sum, in one step. The largest per-pixel tensor is now `(P, 53, 32)`. For one patch in bf16:

$$16{,}384 \times 53 \times 32 \times 2 = 55{,}574{,}528 \text{ bytes},$$
$$55{,}574{,}528/1{,}048{,}576 = 53 \text{ MiB},$$

against 202 MiB for one `(P, 202, 32)` tensor. The skip path is `vals@W` with `W` of shape `(202, 32)`, and the decoder is one `einsum` against **U** of shape `(202, 32)`: neither has a band axis and a 32 axis on the same pixel.

24.4.11 Predict the parameter counts

Worked example 24.13: The reduced compressor, 14,721 parameters

Start with `_hyper(32)`, the network behind $\mathbf{k}$ and $\mathbf{W}$. Do it on paper.

$$\begin{aligned}
\text{Linear}(32,\,64): &\quad 64 \times 32 + 64 = 2{,}048 + 64 = 2{,}112\\
\text{GELU}: &\quad 0\\
\text{Linear}(64,\,32): &\quad 32 \times 64 + 32 = 2{,}048 + 32 = 2{,}080\\
\texttt{_hyper(32)}: &\quad 2{,}112 + 2{,}080 = 4{,}192.
\end{aligned}$$

Now the compressor, line by line:

$$\begin{aligned}
\texttt{band_weight} = \texttt{_hyper(32)},\ \mathbf{k}(\lambda): &\quad 4{,}192\\
\texttt{skip_weight} = \texttt{_hyper(32)},\ \mathbf{W}(\lambda): &\quad 4{,}192\\
\texttt{bin_bias},\ \text{Linear}(32,\,32): &\quad 32 \times 32 + 32 = 1{,}056\\
\text{queries, table } (32,\,32): &\quad 1{,}024\\
\text{q, k, v, o}, 4 \times \text{Linear}(32,\,32): &\quad 4 \times 1{,}056 = 4{,}224\\
\texttt{to_channel},\ \text{Linear}(32,\,1): &\quad 33
\end{aligned}$$

Total:

$$\begin{aligned}
4{,}192 + 4{,}192 + 1{,}056 + 1{,}024 + 4{,}224 + 33 &= 8{,}384 + 1{,}056 + 1{,}024 + 4{,}224 + 33\\
&= 9{,}440 + 1{,}024 + 4{,}224 + 33\\
&= 10{,}464 + 4{,}224 + 33\\
&= 14{,}688 + 33\\
&= 14{,}721.
\end{aligned}$$

Worked example 24.14: The wavelength-linear decoder, 4,257 parameters

It is one `_hyper` with 33 outputs.

$$\begin{aligned}
\text{Linear}(32,\,64): &\quad 64 \times 32 + 64 = 2{,}112\\
\text{Linear}(64,\,33): &\quad 33 \times 64 + 33 = 2{,}112 + 33 = 2{,}145\\
\text{decoder}: &\quad 2{,}112 + 2{,}145 = 4{,}257.
\end{aligned}$$

The real modules report 14,721 and 4,257, with every line as above (`band_weight` $2{,}048 + 64 + 2{,}048 + 32$; in the decoder's last layer a weight of 2,112 numbers and a bias of 33; and so on) [C]. As in Chapter 23, **202 appears nowhere in either count**.

Table 24.7: Parameters of the four stems' spectral parts, all counted by hand in this Part and confirmed by instantiation.

stem	compressor	decoder	depends on band count?
linear (A), 202 bands	6,496	6,666	yes
gate (B), 202 bands	13,162 + 6,496	6,666	yes
per-band cross-attention (C)	5,345	6,401	no
reduced attention (C′)	14,721	4,257	no

(The linear row: `Conv2d(202,32,1)` has $202 \times 32 + 32 = 6{,}496$ and `Conv2d(32,202,1)` has

$32 \times 202 + 202 = 6{,}666$. All four share one `BatchNorm2d(32)`, 64 more, not shown.)

24.4.12 Where the saving did not come from

You might guess the reduced stem is faster because it does less arithmetic. Count it, per pixel, 202 bands, $d = D = 32$, 53 bin slots (the dense matrix product computes all 53, empty or not):

per-band stem: 861,184 (compressor) + 693,568 (decoder) = 1,554,752,

reduced stem:
- bin tokens, `vals @ kmat` : $202 \times 1{,}696 = 342{,}592$
- K, V : $2 \times 53 \times 1{,}024 = 108{,}544$
- scores : $32 \times 53 \times 32 = 54{,}272$
- weighted values : $32 \times 53 \times 32 = 54{,}272$
- output + to channel : $32{,}768 + 1{,}024 = 33{,}792$
- skip path : $202 \times 32 = 6{,}464$
- decoder : $202 \times 32 = 6{,}464$
- total : 606,400.

Add the reduced stem's lines in pairs to check: $342{,}592 + 108{,}544 = 451{,}136$; $+\,54{,}272 + 54{,}272 = 559{,}680$; $+\,33{,}792 = 593{,}472$; $+\,6{,}464 + 6{,}464 = 606{,}400$. The ratio is

$$1{,}554{,}752/606{,}400 = 2.56 :$$

less arithmetic, but not six times less. Most of `vals@kmat` is multiplying by zero (each row of `kmat` has 32 non-zeros out of 1,696), which the code accepts in exchange for one fast dense kernel and no per-band intermediate. **The speed-up comes mostly from what is no longer written to and read back from memory**, which is what Chapter 23 said would bind first.

24.4.13 Measured on a real GPU

Values that will drift

L40S, 2026-10-14. One full training step (forward, backward, Adam update), bf16, the real Indradhanu encoder (embedding widths 32, 64, 160, 256), 128×128 patches. Patches per second, with peak memory in GiB [M: `docs/lld/spectral-stem.md`]:

stem	batch 4	batch 8	batch 16	batch 32
linear (A)	140.8 (0.4)	270.6 (0.8)	506.7 (1.4)	780.5 (2.8)
gate (B)	138.1 (0.5)	260.2 (1.0)	449.3 (1.9)	612.2 (3.7)
per-band cross-attention (C)	13.5 (11.3)	12.2 (18.3)	10.2 (32.4)	out of memory
reduced attention (C′)	70.6 (3.8)	**80.0** (5.4)	69.9 (8.5)	48.6 (14.6)

In a full epoch (E004, batch 8, 6 loader workers, two-pass validation) the reduced stem ran at about 55 patches/s, 9.2 minutes per 30,000 patches [M: `docs/lld/spectral-stem.md`].

Source check: the gate row

The course's table (IW40) shows the gate only at batches 8 and 32, with dashes at 4 and 16. The design record `docs/lld/spectral-stem.md`, section "The reduced stem (C′)", has all four: 138.1 (0.5 GiB) at batch 4 and 449.3 (1.9 GiB) at batch 16. This book uses the fuller record. The E004 README (`research/experiments/indradhanu-se-pretrain/E004-reduced-attention/README.md`, line 23) quotes the same probe as 2.8 GiB per patch for

the per-band stem and 0.68 GiB for the reduced stem, which are 11.3/4 and 5.4/8 from this table.

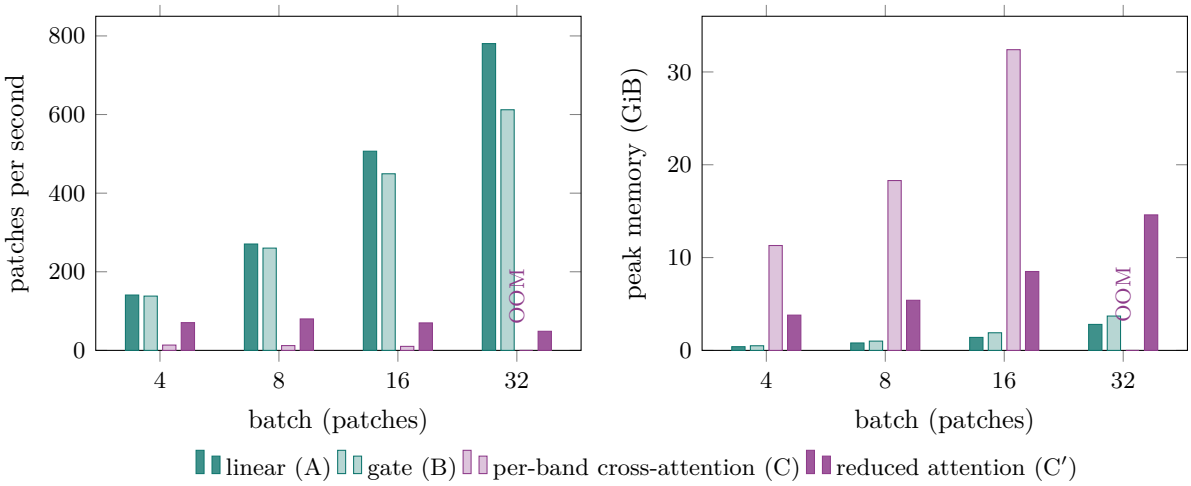


Figure 24.8: The L40S measurements of the drift box, as bars on linear axes (to scale). Left: training throughput; the per-band stem is the short plum bars, the reduced stem the dark plum. Right: peak memory. "OOM": the per-band stem ran out of memory at batch 32.

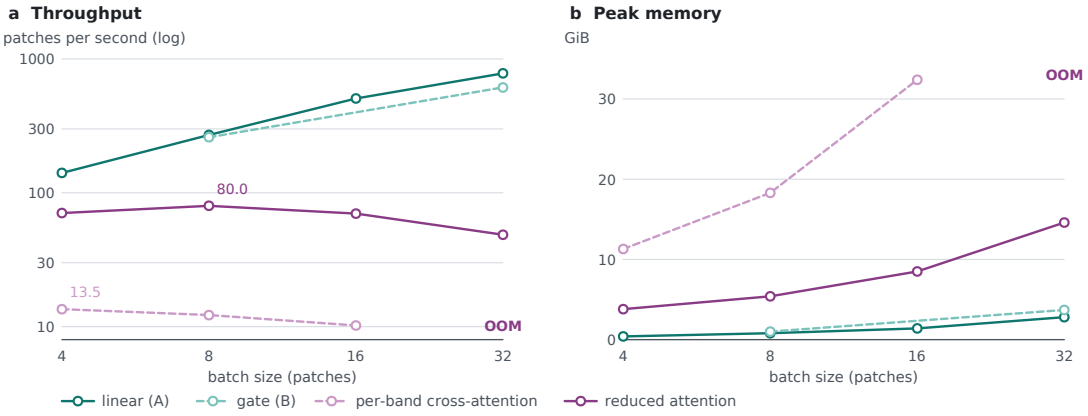


Figure 24.9: The same measurements as the course draws them. Left: patches per second against batch size (log scale). Right: peak memory. The per-band cross-attention stem runs out of memory at batch 32; the reduced stem is about six times faster at each stem's best batch and needs about a quarter of the memory per added patch.

Read the table with arithmetic, not by eye.

Worked example 24.15: Reading the throughput table

Speed at each stem's best batch.

$$\text{reduced best} : 80.0 \text{ patches/s (batch 8)},$$
$$\text{per-band best} : 13.5 \text{ patches/s (batch 4)},$$
$$80.0/13.5 = 5.93 \quad \text{(about 6 times faster)}.$$

Memory per patch at batch 16, where both run.

$$\begin{aligned} \text{per-band} &: 32.4/16 = 2.03 \text{ GiB per patch}, \\ \text{reduced} &: 8.5/16 = 0.53 \text{ GiB per patch}, \\ 2.03/0.53 &= 3.8. \end{aligned}$$

Memory per extra patch. The peak includes fixed costs (weights, optimiser state, the CUDA workspace) that do not grow with the batch. Remove them by asking what each *extra* patch costs between batch 4 and batch 16:

$$\begin{aligned} \text{per-band} &: (32.4 - 11.3)/(16 - 4) = 21.1/12 = 1.76 \text{ GiB per extra patch}, \\ \text{reduced} &: (8.5 - 3.8)/(16 - 4) = 4.7/12 = 0.39 \text{ GiB per extra patch}, \\ 1.76/0.39 &= 4.5. \end{aligned}$$

Three more honest observations. The reduced stem is still about ten times slower than the linear stem (780.5/80.0 = 9.8, each at its best batch): attention is not free. Its speed *falls* at batch 32 (48.6 against 80.0 at batch 8); the cause was not investigated, so batch 8, the fastest measured, is its operating point. And the gate costs little more than the linear stem, as its 3× stem arithmetic, diluted by the shared SegFormer, predicted.

24.4.14 What the reduction gives up

This is a genuine trade-off, not a free win.

Table 24.8: Per-band against reduced stem.

	per-band stem	reduced stem
per-pixel choice between individual bands	yes	no: between bins; within a bin the weights are fixed per wavelength
narrow dip inside one bin preserved	yes	yes, through fix 1's one-to-one map and fix 2
feature straddling a bin edge	one token per band	split across two bin tokens
dead bands skipped exactly	yes	yes
any band list	yes	yes
different band lists inside one batch	yes, wavelengths `(B, N)`	**no**: `(N,)` only, else `ValueError`
measured training speed (L40S, best batch)	13.5 patches/s	80.0 patches/s

The structural row is worth remembering: the source rejects a per-sample wavelength table with a `ValueError` whose message reads "reduced stem takes one band list per batch: wavelengths (N,)" (line 92). SpectralEarth always has one band list, so pretraining is unaffected; mixed-sensor fine-tuning uses the group-wise `forward_mixed` of Chapter 23, one stem pass per band list.

The reduced stem attends over about 42 fixed wavelength bins instead of 202 bands, keeps per-band detail through a per-band weight inside each bin, a per-band skip path and a per-band decoder, and trains about six times faster because it never stores a pixels-by-bands-by-32 tensor.

24.5 Common confusions

Can the decoder predict wavelengths the sensor never measured?
It will produce numbers for any wavelength. Whether they mean anything depends on whether training saw that region; the code drops bands more than 15 nm from any trained wavelength (`trained_band_mask`).

Why not reuse the compressor's 32 queries in the decoder?
They are questions about spectra; the decoder needs questions about wavelengths. Different jobs, different queries.

Does each output band get its own weights in the decoder?
No; that is exactly what was removed. Every band uses the same projections (or the same hypernetwork, in the reduced stem); only its wavelength differs.

Does the spatial model need retraining for a new stem?
Yes: its inputs change meaning, so the whole model is trained from scratch or from a SpectralEarth pretraining run. Its *code* does not change.

The gate starts at 0.5 everywhere. Doesn't that halve the signal?
At the start of training the scores are near 0, so every band is scaled by about 0.5. The BatchNorm after the compressor rescales its output to a standard spread, so the encoder never sees the halving (`spectral_gate.py`, docstring).

Is the gate per patch or per pixel?
Per pixel. A per-patch version (pool the patch to one spectrum, compute one gate) exists, squeeze-and-excitation [14], but it gives a shadowed pixel and a bright pixel in the same patch the same recipe, which is the problem being fixed.

Is a bin token a plain average of its bands?
No. It is the mean of $v_b\,\mathbf{k}(\lambda_b)$, each band times its own 32-number vector. A plain average keeps only the bin's sum.

Are the bins learned?
No. They are a fixed 40 nm grid from 400 nm, the same on every sensor and every run. What is learned is $\mathbf{k}$, $\mathbf{W}$, $\mathbf{U}$, the bin bias, the queries and the projections.

53 or 42 bin tokens?
The tensor has 53 slots, because the grid has 53 bins. For SpectralEarth 11 are empty and masked, so attention effectively reads 42. Another sensor fills a different set.

Does the skip path make the attention pointless?
It makes it optional. The skip path alone is a wavelength-aware 1×1 compressor, variant A that can read any sensor. Attention adds per-pixel adaptivity on top; whether that earns its time is what the experiment measures (trap 3).

24.6 Check yourself

Exercise 24.1. Recompute the cross-attention decoder's parameters for $d = 16$, $D = 32$, line by line, and check the total with Equation (24.1).

Exercise 24.2. Compute $\sigma(1)$ and $\sigma(-1)$, showing each step. Then, for the toy pixel with gate scores $\mathbf{s} = [2, 2, -2, -2]$ (gates rounded to 0.88 and 0.12), compute y_0 and y_1.

Exercise 24.3. Count the gate's parameters for the 165-band grid with hidden width 32. A dead band holds 50 and its gate score is -4: how much leaks into $\mathbf{x}'$, and how much would the cross-attention compressor let through?

Exercise 24.4. How many bins would the grid from 400 to 2500 nm have with a bin width of 50 nm? Which bin would 864 nm fall in, and what range does that bin cover?

Exercise 24.5. A bin holds two bands. With $\mathbf{k}(\lambda_1) = [1, 0]$ and $\mathbf{k}(\lambda_2) = [1, 1]$, compute the bin tokens of the spectra $[0.3, 0.5]$ and $[0.5, 0.3]$ with Equation (24.6). Are they different? What would the plain average of worked example 24.8 have said?

Exercise 24.6. Skip path, toy pixel and columns of worked example 24.10, band 0 dead (holding 99). Compute the skip output.

Exercise 24.7. From the measured table, what does each extra patch cost the linear stem in memory between batch 4 and batch 32? How many times less is that than the reduced stem's 0.39 GiB?

Next: Chapter 25, Losses: L1, the spectral angle and the ramp.

Part V

Training

Chapter 25

Losses: L1, the spectral angle and the ramp

The one thing this chapter teaches

Indradhanu is trained to minimise one number, $L = L_1 + \lambda(t)\,\mathrm{SAM}$: the masked mean absolute error (how far each band is off, in reflectance) plus a weighted spectral angle (how far the *shape* of each spectrum is off, in radians), with the weight $\lambda(t)$ rising from zero over the first epochs so that brightness is learned before shape.

Every chapter so far has described what the network *computes*. None has said how it comes to compute anything useful. A freshly built Indradhanu has about five and a half million parameters set to random values (Chapter 20); its reconstructions are noise. Training is the process that turns those random numbers into useful ones, and training needs a single number that says how wrong the network currently is. That number is the *loss*. This chapter builds Indradhanu's loss piece by piece, and computes its derivatives by hand, because the derivatives are what training actually uses (Chapter 26).

25.1 What a loss is, and what it must do

Definition 25.1 (Loss)**.** A *loss* (or *loss function*, or *objective*) is a function that takes the network's output and the truth it should have produced, and returns one non-negative real number. Smaller means better. Training changes the network's parameters so as to make this number smaller.

Recall from Chapter 10 that a *parameter* is one adjustable number inside the network (one entry of a weight matrix or a bias vector), and from Chapter 18 that the network is shown a patch with most of its tokens hidden and must reconstruct the hidden part. Here the truth is the patch the satellite recorded, $\mathbf{x}$, and the output is the reconstruction, $\hat{\mathbf{x}}$. Both have shape `(B, 165, 128, 128)`: a batch of B patches, 165 bands, 128×128 pixels.

A loss must have two properties.

1. **Smaller must mean better, always.** If two reconstructions differ and one is closer to the truth in the sense you care about, it must get the smaller loss. Otherwise training improves the wrong thing.
2. **It must be differentiable** (almost everywhere). Training does not search blindly; it asks, for every parameter, "if I nudge you up a little, does the loss go up or down, and

how fast?" That rate is a derivative. Section 25.6 shows how it is computed through the whole network; this chapter computes it for the loss itself.

The obvious first choice, "count the pixels that are wrong", fails property 2: a count does not change when a reflectance moves from 0.2001 to 0.2002, so its derivative is zero almost everywhere and gives no direction. The losses below all change smoothly (or at least continuously) as $\hat{\mathbf{x}}$ moves.

Analogy

The loss is the marking scheme of an exam the network sits millions of times. L1 marks every band separately, like a spelling test: each wrong letter costs marks in proportion to how wrong it is. The spectral angle marks the *handwriting*: it ignores how large you wrote and asks only whether the letters have the right shape. A good marking scheme for a hyperspectral model needs both, and, like a teacher, it asks for legibility before style (the ramp of Section 25.10).

25.2 The masked mean absolute error

25.2.1 Which pixels are graded

The loss is not computed over every pixel of the patch. A pixel is *graded* only if all three of the following hold.

1. It was a **prediction target**: its token was hidden from the encoder (Chapter 18). Grading pixels the model could see would reward copying.
2. It is **valid**: a real measurement, not nodata or cloud-filled.
3. It is **interior**: not inside the erosion collar around an invalid region, where the overlapping patch embedding has mixed invalid neighbours into it (Chapter 18).

Conditions 2 and 3 together are the *eroded mask*; condition 1 is the pixel prediction mask. Both are arrays of 0s and 1s of shape `(B, 1, H, W)`, and the loss mask is their product:

```
pixel_pred_mask = self._pred_mask_to_pixel_mask(pred_mask, H, W)            # (B, 1,
    H, W)
eroded_mask     = TokenMasking.erode_mask(mask, kernel_size=cfg.erosion_kernel_size)
loss_mask       = pixel_pred_mask * eroded_mask                             # (B, 1,
    H, W)
```

Definition 25.2 (Loss mask)**.** The *loss mask* M is the set of pixels p whose mask value is 1, that is, pixels that were hidden, valid and interior. Its size $|M|$ is the number of graded pixels.

Multiplying two 0/1 numbers is a logical AND: $1 \times 1 = 1$, and any product with a 0 in it is 0. Four pixels show every case.

pixel	hidden?	valid and interior?	product	graded?
A	1	1	$1 \times 1 = 1$	yes
B	1	0	$1 \times 0 = 0$	no: nodata, or on a border
C	0	1	$0 \times 1 = 0$	no: the model could see it
D	0	0	$0 \times 0 = 0$	no

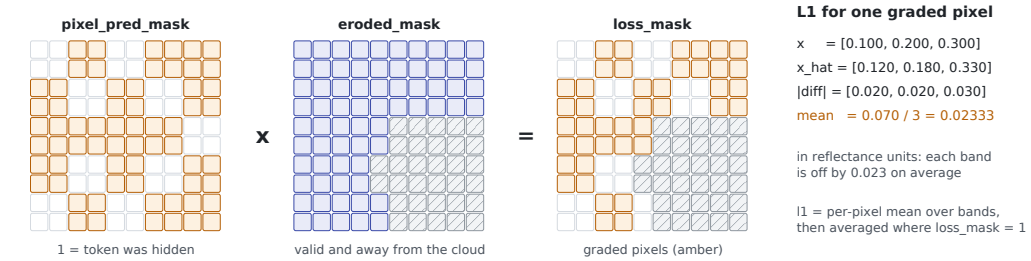


Figure 25.1: Which pixels are graded. The pixel prediction mask (hidden tokens, amber) is multiplied by the eroded validity mask (indigo; the cloud and its collar hatched). Multiplying 0/1 masks is a logical AND, so only pixels that were hidden, valid and away from invalid regions enter the loss. Right: the L1 term for one graded pixel.

25.2.2 From the token grid to the pixel grid

The prediction mask is born on the token grid: one flag per stage-1 token, shape `(B, N)` with $N = 32 \times 32 = 1024$ for a 128-pixel patch (stage 1 has stride 4, Chapter 13). The loss lives on the 128×128 pixel grid, so each flag must be copied to the $4 \times 4 = 16$ pixels its token covers:

```
token_grid = pred_mask.reshape(B, 1, H_tokens, W_tokens)                    # (B, 1,
    32, 32)
return F.interpolate(token_grid, size=(H, W), mode="nearest")               # (B, 1,
    128, 128)
```

`mode="nearest"` copies values; it never averages. Bilinear interpolation would produce 0.25, 0.5 and 0.75 at block edges, and the later test `loss_mask==1` would silently reject those pixels, leaving a lattice of ungraded gaps. Use nearest-neighbour for masks, always.

25.2.3 The definition, and the order of averaging

Write C for the number of bands (165), $e_{c,p} = \hat{x}_{c,p} - x_{c,p}$ for the *error* (or *residual*) of band c at pixel p, and M for the graded pixels.

Definition 25.3 (Masked mean absolute error)**.**

$$L_1 \;=\; \frac{1}{|M|}\sum_{p\in M}\left(\frac{1}{C}\sum_{c=1}^{C}|e_{c,p}|\right) \;=\; \frac{1}{C\,|M|}\sum_{p\in M}\sum_{c=1}^{C}|\hat{x}_{c,p}-x_{c,p}|\,. \tag{25.1}$$

The name comes from the *L1 norm*, $\|\mathbf{v}\|_1 = \sum_i |v_i|$, the sum of absolute values. Its sibling, the L2 norm $\|\mathbf{v}\|_2 = \sqrt{\sum_i v_i^2}$, gives the mean squared error when you square it and average.

The two forms in Equation (25.1) are equal. To see it, move the constant $1/C$ out of the inner bracket:

$$\begin{aligned}\frac{1}{|M|}\sum_{p\in M}\frac{1}{C}\sum_{c}|e_{c,p}| &= \frac{1}{|M|}\cdot\frac{1}{C}\sum_{p\in M}\sum_{c}|e_{c,p}| && (1/C \text{ is the same for every } p)\\ &= \frac{1}{C\,|M|}\sum_{p\in M}\sum_{c}|e_{c,p}|\,. && (\text{multiply the two fractions})\end{aligned}$$

The step works only because every pixel has the same number of bands C. A "mean of means" equals the overall mean when the groups are the same size; it would not if some pixels had

fewer bands. (That is exactly the situation in the SpectralEarth trainer, where dead bands are skipped per pixel; there each pixel divides by its own count of valid bands, Chapter 27.)

The code follows the first form, band mean first:

```
per_pixel_l1      = (x_hat - pixels).abs()                    # (B, 165, H, W)
per_pixel_l1_mean = per_pixel_l1.mean(dim=1, keepdim=True)    # (B, 1, H, W)  one
    value per pixel
valid_l1          = per_pixel_l1_mean[loss_mask == 1]         # (|M|,)  graded pixels
     only
l1_loss           = valid_l1.mean()                           # ()  a single number
```

Line 3 is *boolean indexing*: it pulls the graded pixels out into a flat list, so the mean on line 4 automatically divides by $|M|$ and never counts a masked zero as if it were a perfect prediction.

Worked example 25.1: Masked L1 on three bands and two graded pixels

Do this one on paper. Use three bands instead of 165, and three pixels, of which A and B are graded and C was visible to the model (so its loss-mask value is 0).

	band 1	band 2	band 3
$\mathbf{x}_A$	0.100	0.200	0.300
$\hat{\mathbf{x}}_A$	0.120	0.180	0.330
$\mathbf{x}_B$	0.050	0.100	0.080
$\hat{\mathbf{x}}_B$	0.060	0.070	0.080
$\mathbf{x}_C$	0.200	0.200	0.200
$\hat{\mathbf{x}}_C$	0.500	0.500	0.500

Step 1, errors at A.

$$e_{1,A} = 0.120 - 0.100 = +0.020$$
$$e_{2,A} = 0.180 - 0.200 = -0.020$$
$$e_{3,A} = 0.330 - 0.300 = +0.030$$

Step 2, absolute values at A (an error of -0.02 is as wrong as $+0.02$): $0.020,\ 0.020,\ 0.030$.
Step 3, band mean at A.

$$0.020 + 0.020 + 0.030 = 0.070$$
$$0.070/3 = 0.023333$$

Step 4, the same at B.

$$e_{1,B} = 0.060 - 0.050 = +0.010$$
$$e_{2,B} = 0.070 - 0.100 = -0.030$$
$$e_{3,B} = 0.080 - 0.080 = \ \ 0.000$$
$$0.010 + 0.030 + 0.000 = 0.040$$
$$0.040/3 = 0.013333$$

Step 5, pixel C. Its band mean is 0.300, a terrible reconstruction, but its mask value is 0, so it is not in M and contributes nothing.
Step 6, mean over the graded pixels, $|M| = 2$:

$$0.023333 + 0.013333 = 0.036667$$
$$0.036667/2 = 0.018333$$

Check with the second form of Equation (25.1): the six absolute errors sum to $0.070 + 0.040 = 0.110$, and $C\,|M| = 3 \times 2 = 6$, so $0.110/6 = 0.018333$. The same.
The real code, run on these numbers, prints $L_1 = 0.018333$ [C] (script `research/textbook/tools/compute_23-losses.py`).

Is 0.018 good? Reflectance sits roughly between 0.02 and 0.30, so an average error of 0.018 is large; you could see it in a plot. The training configuration's own baseline table expects L_1 of about 0.08–0.10 at the first epoch and 0.003–0.008 after 500 epochs [M: `configs/hyperspectral_segformer_exp_2.json`]. So these made-up pixels are an early-training reconstruction.

25.2.4 The derivative of L1

Training needs $\partial L_1/\partial \hat{x}_{c,p}$ for every band and pixel: how fast the loss changes when one predicted value moves.

First, the derivative of the absolute value. For $e > 0$, $|e| = e$, whose derivative is $+1$. For $e < 0$, $|e| = -e$, whose derivative is -1. At $e = 0$ the graph has a corner and no derivative exists; PyTorch uses 0 there. All three cases together are the *sign function*:

$$\frac{d\,|e|}{de} = \operatorname{sign}(e) = \begin{cases} +1 & e > 0 \\ 0 & e = 0 \\ -1 & e < 0. \end{cases}$$

Now differentiate Equation (25.1) with respect to one predicted value $\hat{x}_{c,p}$ of a graded pixel. Only one term of the double sum contains it, the term $|\hat{x}_{c,p} - x_{c,p}|$; every other term is a constant as far as $\hat{x}_{c,p}$ is concerned and has derivative zero. The derivative of $e_{c,p} = \hat{x}_{c,p} - x_{c,p}$ with respect to $\hat{x}_{c,p}$ is 1. So

$$\frac{\partial L_1}{\partial \hat{x}_{c,p}} = \begin{cases} \dfrac{\operatorname{sign}(e_{c,p})}{C\,|M|} & p \in M, \\ 0 & p \notin M. \end{cases} \tag{25.2}$$

Write $n = C\,|M|$ for the number of graded (band, pixel) entries. Every graded entry gets a push of the same size, $1/n$; only its direction depends on the error.

Worked example 25.2: The gradient of L1 for the two-pixel example

In worked example 25.1, $n = 3 \times 2 = 6$, so $1/n = 0.16667$. Read off the signs of the errors:

	band 1	band 2	band 3
signs at A	+	−	+
gradient at A	+0.16667	−0.16667	+0.16667
signs at B	+	−	0
gradient at B	+0.16667	−0.16667	0
gradient at C	0	0	0

PyTorch's autograd on the same tensors returns exactly this table [C]. Two things to notice. Band 3 at B is already exact, so it gets no push. Pixel C, which has the largest errors of all, gets no push either, because it was not graded.

25.3 Why L1 and not squared error

The most common default loss is the *mean squared error* (MSE), $\frac{1}{n}\sum e^2$, whose derivative is $\frac{2e}{n}$: proportional to the error. L1's derivative is $\frac{\pm 1}{n}$: the same size however large the error. That one difference decides what each loss learns.

25.3.1 L1 learns a median, L2 learns a mean

Strip the problem to its bones. You must predict one number c for five observations $y_1, \ldots, y_5$, and you are graded by either

$$S_2(c) = \sum_i (y_i - c)^2 \qquad \text{or} \qquad S_1(c) = \sum_i |y_i - c| \,.$$

The best c under S_2 is the mean. Differentiate S_2 with respect to c. By the chain rule (stated properly in Section 25.6; here you need only that the derivative of $(y_i - c)^2$ is $2(y_i - c) \cdot (-1)$):

$$\frac{dS_2}{dc} = \sum_i 2(y_i - c)(-1) = -2\sum_i (y_i - c).$$

Set it to zero, the condition for the bottom of the bowl:

$$0 = \sum_i y_i - 5c$$
$$c = \frac{1}{5}\sum_i y_i.$$

The best c under S_1 is the median. Differentiate S_1:

$$\frac{dS_1}{dc} = \sum_i \operatorname{sign}(c - y_i) = (\text{number of } y_i \text{ below } c) - (\text{number of } y_i \text{ above } c).$$

This is negative while more observations lie above c than below (so moving c up lowers S_1), positive once more lie below, and it changes sign at the middle observation: the median.

Worked example 25.3: A glitch pulls the mean, not the median

Take five reflectances of the same field, one of which is a sensor glitch: $y = (0.10,\ 0.11,\ 0.12,\ 0.13,\ 0.90)$.
Mean.

$$0.10 + 0.11 + 0.12 + 0.13 + 0.90 = 1.36$$
$$1.36/5 = 0.272$$

Median. Sorted, the middle value is 0.12.
The mean, 0.272, is larger than four of the five observations; it describes none of them. The median, 0.12, describes the field.

The pull of each observation at $c = 0.12$. Under S_2 each observation pulls with $2(c - y_i)$:

$$\begin{aligned} 2(0.12 - 0.10) &= +0.04 \\ 2(0.12 - 0.11) &= +0.02 \\ 2(0.12 - 0.12) &= 0.00 \\ 2(0.12 - 0.13) &= -0.02 \\ 2(0.12 - 0.90) &= -1.56 \end{aligned}$$

The glitch pulls 39 times harder than the next largest term ($1.56/0.04 = 39$). Under S_1 each pull is $\operatorname{sign}(c - y_i)$: $+1, +1, 0, -1, -1$. The glitch is one vote among five.

25.3.2 Why that matters for this training set

The training corpus is real satellite imagery, not curated. It contains genuine anomalies (the very things the model should find surprising), dead detectors and stripes, and cloud edges the masks missed. Under MSE, those few pixels with huge errors would dominate every gradient, and the model would spend its capacity learning to reconstruct exactly what it should fail on. Under L1, each pixel's pull has the same size; outliers vote, but they do not take over. The model learns the bulk of the data, which is normality, and that is what an anomaly detector needs (Chapter 9).

	L1	MSE
penalty for error e	$\lvert e \rvert$	e^2
a 2 × larger error costs	2 ×	4 ×
a 10 × larger error costs	10 ×	100 ×
pull (derivative) of one entry	$\pm 1/n$	$2e/n$
best constant prediction	median	mean
dominated by outliers?	no	yes

The term for a loss whose pull is bounded is *robust*; the classical reference for robust estimation of a location is Huber [15].

Source check: Antardhana and Tirohita are not identical apart from the loss

The course (IW23) says the repo holds two models that differ *only* in loss, Antardhana (MSE) and Tirohita (L1). The model register `research/model_break_down/README.md` (rows 3 and 7) lists them as the same family, the spatial masked autoencoder, but with different parameter counts, 330,314 for Tirohita and 265,480 for Antardhana [M: `research/model_break_down/README.md`], and different versions. So they are siblings with the loss as the headline difference, not a controlled one-variable comparison. The robustness argument above stands on its own; treat the pair as a hint, not as proof.

25.4 The 40% rule and sample counting

Before any loss is computed, whole patches are thrown away:

```
MIN_VALID_PIXEL_FRACTION = 0.4
valid_fractions = mask.flatten(1).float().mean(dim=1)     # (B,)  mean of 0/1 =
    fraction valid
keep = valid_fractions >= MIN_VALID_PIXEL_FRACTION        # (B,)  bool
```

```
num_kept = keep.sum().item()                                    # int
```

(`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, lines 34 and 162–170.) A patch less than 40% valid is mostly nodata: training on it teaches the model to reconstruct holes, and its few real pixels all lie near a border. The mean of a 0/1 array is the fraction of 1s, the same trick as in Chapter 18.

`compute_loss` returns the loss together with `num_kept`, and the training loop weights by it (Chapter 26). If no patch survives, the loss is a zero tensor and the loop skips the batch. The function also returns zero when the loss mask is empty, so a batch with nothing graded cannot divide by zero.

Source check: training and inference use different thresholds

Training drops patches below 40% valid. The scene-scale inference loop in `app/foundation_models/inferencers/segformer_mae_inferencer.py` (lines 281–284), which the hyperspectral inferencer inherits, uses `MIN_VALID_FRACTION=0.1` under a comment that says "Matches training: patches with $<$ 40% valid pixels were discarded." The comment is wrong; the constant is 0.1. The looser inference threshold is defensible (you would rather have an imperfect score near a scene edge than none), but if you debug edge artefacts, know that these two numbers differ. Chapter 30 returns to this.

25.5 The spectral angle

25.5.1 Why L1 is not enough

L1 measures error in reflectance units, and so it is dominated by bright pixels. A bright pixel at reflectance 0.30 that is 10% off costs 0.03 per band; a dark pixel at 0.03 that is 10% off costs 0.003. Same proportional error, ten times the penalty, so gradient descent spends its effort on bright surfaces and neglects shadow, dark water and dark soil.

Yet the diagnostic content of a spectrum is its *shape*, not its level (Chapter 2). Sunlit grass and shadowed grass are the same material with the same shape at different brightness; a gas plume changes the shape at a few wavelengths without changing the overall level much. You need a second measurement that sees shape and ignores brightness.

25.5.2 The definition

Treat each pixel's C values as a vector in $\mathbb{R}^C$ (Chapter 6). The angle between two vectors measures how differently they point, regardless of length.

Definition 25.4 (Spectral angle)**.** For nonzero spectra $\mathbf{x}, \hat{\mathbf{x}} \in \mathbb{R}^C$,

$$u = \cos\theta = \frac{\langle \mathbf{x}, \hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, \|\hat{\mathbf{x}}\|}, \qquad \mathrm{SAM}(\mathbf{x}, \hat{\mathbf{x}}) = \theta = \arccos u \in [0, \pi], \tag{25.3}$$

where $\langle \mathbf{x}, \hat{\mathbf{x}} \rangle = \sum_c x_c \hat{x}_c$ is the dot product and $\|\mathbf{x}\| = \sqrt{\langle \mathbf{x}, \mathbf{x} \rangle}$ the length. The angle is in radians.

The name is *Spectral Angle Mapper*, from the remote-sensing classifier of Kruse et al. [22], which matches an unknown spectrum against a library by this angle. (The companion volume on spectral matching uses it that way; Chapter 32 uses it at the end of the chain.) Here it is repurposed as a training loss between the observed and the reconstructed spectrum.

That u really is the cosine of an angle, and lies in $[-1, 1]$, is the Cauchy–Schwarz inequality $|\langle \mathbf{x}, \hat{\mathbf{x}} \rangle| \le \|\mathbf{x}\| \, \|\hat{\mathbf{x}}\|$ (Chapter 6).

angle	meaning
0	identical shape, whatever the brightness
0.01 to 0.10 rad (0.6° to 5.7°)	a good reconstruction here
$\pi/2 \approx 1.571$	at right angles: unrelated
$\pi \approx 3.142$	exactly opposite

Worked example 25.4: The spectral angle of pixel A, by hand

Do this on paper. Pixel A of worked example 25.1: $\mathbf{x} = (0.100, 0.200, 0.300)$, $\hat{\mathbf{x}} = (0.120, 0.180, 0.330)$.

Step 1, the dot product.

$$0.100 \times 0.120 = 0.0120$$
$$0.200 \times 0.180 = 0.0360$$
$$0.300 \times 0.330 = 0.0990$$
$$0.0120 + 0.0360 + 0.0990 = 0.1470$$

Step 2, the two lengths.

$$\|\mathbf{x}\|^2 = 0.100^2 + 0.200^2 + 0.300^2 = 0.0100 + 0.0400 + 0.0900 = 0.1400$$
$$\|\mathbf{x}\| = \sqrt{0.1400} = 0.374166$$
$$\|\hat{\mathbf{x}}\|^2 = 0.120^2 + 0.180^2 + 0.330^2 = 0.0144 + 0.0324 + 0.1089 = 0.1557$$
$$\|\hat{\mathbf{x}}\| = \sqrt{0.1557} = 0.394588$$

Step 3, the cosine.

$$\|\mathbf{x}\|\,\|\hat{\mathbf{x}}\| = 0.374166 \times 0.394588 = 0.147641$$
$$u = 0.1470/0.147641 = 0.995655$$

Step 4, the angle.

$$\theta = \arccos(0.995655) = 0.09325 \text{ rad}$$
$$\theta = 0.09325 \times 180/\pi = 5.34°$$

So pixel A has L_1 contribution 0.0233 (its brightness is off by 0.023 reflectance on average) and spectral angle 0.0933 rad (its shape is off by about 5.3°). They measure different things; neither implies the other.

Source check: the course's angle for this pixel

The course (IW24) gets $\|\mathbf{x}\|\,\|\hat{\mathbf{x}}\| = 0.147644$, $u = 0.995637$ and $\theta = 0.0935$ rad $= 5.35°$. The product is 0.147641 (more digits: 0.1476415), which gives $u = 0.995655$ and $\theta = 0.09325$ rad $= 5.34°$. A float64 evaluation and the real `SAMLoss` (in float32, eps included) both give 0.09325 [C] (`research/textbook/tools/compute_23-losses_output.txt`). The difference is a rounding slip in the course's third step; it changes no conclusion. The same slip propagates into the course's atan2 check, which gives sin-term 0.013784 instead of 0.013748.

25.5.3 Scale invariance

Worked example 25.5: A reconstruction twice as bright

Suppose the model predicted $\hat{\mathbf{x}} = (0.200, 0.400, 0.600)$, exactly twice the truth in every band.

L1.

$$|0.200 - 0.100| = 0.100$$
$$|0.400 - 0.200| = 0.200$$
$$|0.600 - 0.300| = 0.300$$
$$(0.100 + 0.200 + 0.300)/3 = 0.600/3 = 0.200$$

Enormous: almost nine times pixel A's 0.0233.

Spectral angle.

$$\langle \mathbf{x}, \hat{\mathbf{x}} \rangle = 0.100 \times 0.200 + 0.200 \times 0.400 + 0.300 \times 0.600 = 0.02 + 0.08 + 0.18 = 0.28$$
$$\|\hat{\mathbf{x}}\| = \sqrt{0.04 + 0.16 + 0.36} = \sqrt{0.56} = 0.748331$$
$$\|\mathbf{x}\| \, \|\hat{\mathbf{x}}\| = 0.374166 \times 0.748331 = 0.28$$
$$u = 0.28/0.28 = 1, \qquad \theta = \arccos 1 = 0.$$

The angle reports a perfect match.

This is not a coincidence of the numbers. Here is the general statement and its proof.

Claim. For any $\alpha > 0$, $\mathrm{SAM}(\mathbf{x}, \alpha\hat{\mathbf{x}}) = \mathrm{SAM}(\mathbf{x}, \hat{\mathbf{x}})$.

Proof. Compute the cosine for $\alpha\hat{\mathbf{x}}$ and simplify.

$$\begin{aligned}
\frac{\langle \mathbf{x}, \alpha\hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, \|\alpha\hat{\mathbf{x}}\|} &= \frac{\alpha \langle \mathbf{x}, \hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, \|\alpha\hat{\mathbf{x}}\|} && \text{(a constant factor comes out of a dot product)} \\
&= \frac{\alpha \langle \mathbf{x}, \hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, \sqrt{\textstyle\sum_c \alpha^2 \hat{x}_c^2}} && \text{(definition of the length)} \\
&= \frac{\alpha \langle \mathbf{x}, \hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, |\alpha| \, \|\hat{\mathbf{x}}\|} && (\sqrt{\alpha^2} = |\alpha|) \\
&= \frac{\langle \mathbf{x}, \hat{\mathbf{x}} \rangle}{\|\mathbf{x}\| \, \|\hat{\mathbf{x}}\|} && (\alpha > 0, \text{ so } |\alpha| = \alpha \text{ cancels}).
\end{aligned}$$

The cosine is unchanged, so the angle is unchanged. □

If $\alpha < 0$, $|\alpha| = -\alpha$ and the cosine changes sign: the angle becomes $\pi - \theta$. Reflectances are non-negative, so this case does not arise in practice.

So the model needs both terms:

kind of error	caught by L1	caught by the angle
brightness only (right shape, wrong scale)	yes	no
shape only (right level, wrong colour)	weakly	yes
both	yes	yes

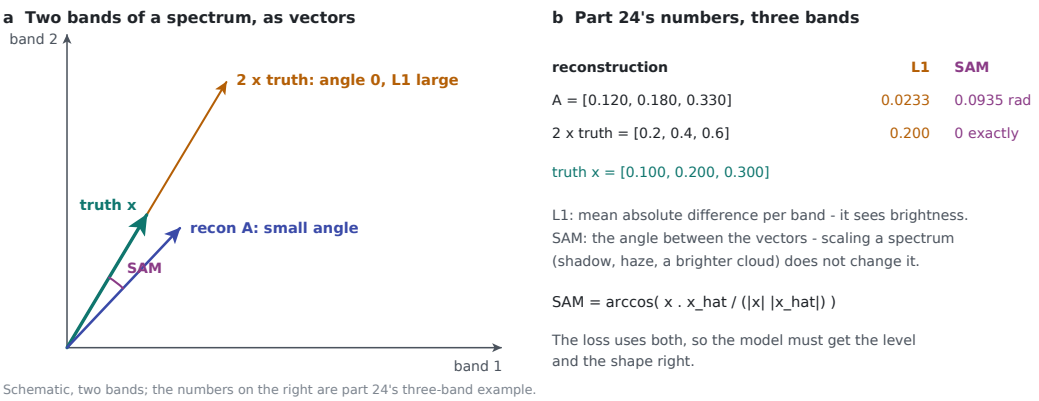


Figure 25.2: Two complementary errors. (a) Treating a spectrum as a vector, the spectral angle is the angle between reconstruction and truth, and L1 the average per-band distance. A reconstruction that is the truth scaled by 2 (amber) has an angle of exactly 0 but a large L1; a reconstruction close in level but slightly rotated (indigo) has a small L1 and a small angle. (b) The three-band numbers of worked example 25.4.

25.6 The chain rule

To train with the spectral angle you need its derivative with respect to every predicted value, and the angle is a function of a function of a function of $\hat{\mathbf{x}}$. The tool for that is the chain rule. It is used everywhere in the rest of this book, so it is stated carefully here and shown on a tiny example first.

Definition 25.5 (Chain rule, one variable)**.** If $y = f(g(w))$, write $z = g(w)$. Then

$$\frac{dy}{dw} = \frac{dy}{dz} \cdot \frac{dz}{dw} = f'(g(w))\, g'(w).$$

In words: the rate at which the outer function changes with the input is the rate at which it changes with its own input, times the rate at which that input changes with w.

Worked example 25.6: The chain rule on a tiny function

Let $y = (3w + 1)^2$ and find dy/dw at $w = 1$.
Name the inner function. $z = g(w) = 3w + 1$, so $y = f(z) = z^2$.
Differentiate each piece. $dz/dw = 3$, and $dy/dz = 2z$.
Evaluate at $w = 1$**.** $z = 3 \times 1 + 1 = 4$, so $dy/dz = 2 \times 4 = 8$.
Multiply. $dy/dw = 8 \times 3 = 24$.
Check numerically. Move w a little either side and divide the change in y by the change in w:

$$w = 1.001: \quad y = (3.003 + 1)^2 = 4.003^2 = 16.024009$$
$$w = 0.999: \quad y = (2.997 + 1)^2 = 3.997^2 = 15.976009$$
$$\frac{16.024009 - 15.976009}{1.001 - 0.999} = \frac{0.048}{0.002} = 24.$$

The chain rule agrees with the measured slope.

With several variables the rule is the same, summed over every path. If y depends on w through intermediate quantities $z_1, \ldots, z_k$, then

$$\frac{\partial y}{\partial w} = \sum_{j=1}^{k} \frac{\partial y}{\partial z_j} \frac{\partial z_j}{\partial w}.$$

Training a network is this rule applied from the loss backwards through every layer, which is why the procedure is called *backpropagation* (Chapter 26).

Two more rules are used below.

The quotient rule. For $q = a/b$ where a and b both depend on w: $\dfrac{dq}{dw} = \dfrac{a'b - ab'}{b^2}$. Tiny check: $q = w/(w+1)$ at $w = 1$ has $a' = 1$, $b' = 1$, so $q' = (1 \cdot 2 - 1 \cdot 1)/2^2 = 1/4$. Numerically, $q(1.001) = 1.001/2.001 = 0.500250$ and $q(0.999) = 0.999/1.999 = 0.499750$, so the slope is $0.000500/0.002 = 0.25$.

The derivative of a length. For $b = \|\hat{\mathbf{x}}\| = \big(\sum_c \hat{x}_c^2\big)^{1/2}$, the chain rule with outer function $(\cdot)^{1/2}$ gives

$$\frac{\partial b}{\partial \hat{x}_c} = \frac{1}{2}\Big(\sum_k \hat{x}_k^2\Big)^{-1/2} \cdot 2\hat{x}_c = \frac{\hat{x}_c}{\|\hat{\mathbf{x}}\|}.$$

The gradient of a length is the unit vector pointing along it.

25.7 The derivative of arccos, and where it blows up

Derivation. Let $y = \arccos u$, so $\cos y = u$, with $y \in [0, \pi]$.

$$\begin{aligned}
\cos y &= u && \text{(definition of arccos)}\\
-\sin y \,\frac{dy}{du} &= 1 && \text{(differentiate both sides with respect to } u\text{; chain rule on the left)}\\
\frac{dy}{du} &= -\frac{1}{\sin y} && \text{(divide by } -\sin y\text{)}\\
&= -\frac{1}{\sqrt{1-\cos^2 y}} && (\sin^2 + \cos^2 = 1\text{, and } \sin y \geq 0 \text{ on } [0,\pi])\\
&= -\frac{1}{\sqrt{1-u^2}}. && \text{(substitute } \cos y = u\text{)}
\end{aligned}$$

Numbers. Here is the size of that derivative as u approaches 1, with the angle that each u corresponds to:

u	$1-u^2$	$\sqrt{1-u^2}$	$d\arccos/du$	angle
0.9	0.19	0.4359	−2.29	0.451 rad (25.8°)
0.99	0.0199	0.1411	−7.09	0.142 rad (8.1°)
0.999	0.001999	0.04471	−22.4	0.0447 rad (2.6°)
0.9999	0.00019999	0.01414	−70.7	0.0141 rad (0.8°)
1	0	0	$-\infty$	0

Work the third row on paper: $0.999^2 = 0.998001$; $1 - 0.998001 = 0.001999$; $\sqrt{0.001999} = 0.04471$; $1/0.04471 = 22.37$.

Now set beside this table where a *good* model lives. The configuration's baseline expects an angle of 0.02–0.05 rad after long training [M: `configs/hyperspectral_segformer_exp_2.json`], that is u between $\cos 0.05 = 0.99875$ and $\cos 0.02 = 0.99980$. A well-trained model sits at $u = 0.999$ and above, in the bottom rows, where the factor $d\arccos/du$ is in the tens and heading for infinity. The better the model gets, the closer it sits to the singularity.

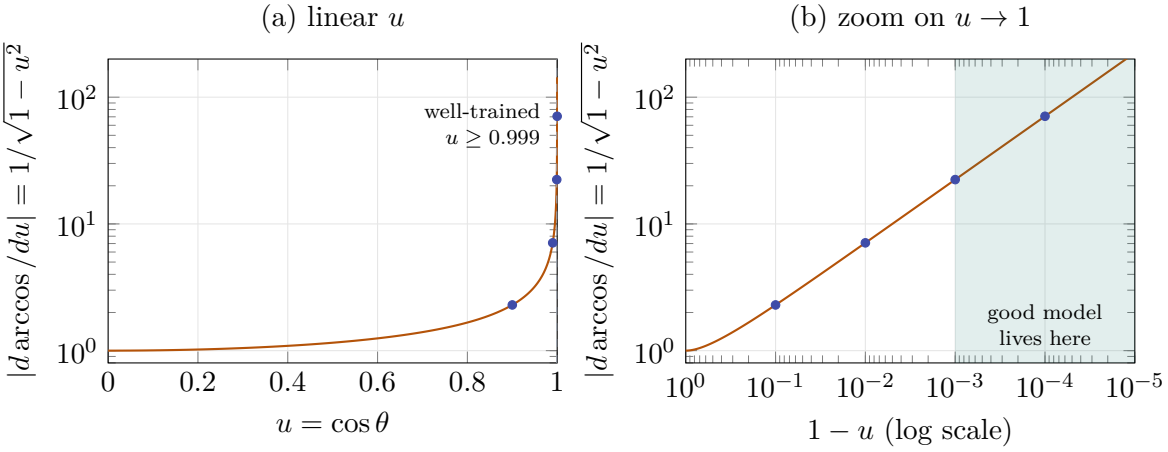


Figure 25.3: The size of the derivative of arccos, drawn to scale from $1/\sqrt{1-u^2}$, on a logarithmic vertical axis. Dots are the four table rows. (a) Against u itself: flat until about $u = 0.9$, then a wall. (b) Against $1 - u$ on a log axis, reading right to left as $u \to 1$: a straight line of slope $-\frac{1}{2}$, because $1/\sqrt{1-u^2} \approx 1/\sqrt{2(1-u)}$ there. The shaded band, $u \geq 0.999$, is where a well-trained model's pixels sit.

25.8 The gradient of the spectral angle

Now combine the pieces. Write $a = \|\mathbf{x}\|$ (a constant, since the truth does not depend on the parameters), $b = \|\hat{\mathbf{x}}\|$, $s = \langle \mathbf{x}, \hat{\mathbf{x}} \rangle$, so $u = s/(ab)$ and $\theta = \arccos u$.

Step 1: the derivative of the cosine. Use the quotient rule with numerator s and denominator ab. The pieces: $\partial s/\partial \hat{x}_c = x_c$ (only one term of the dot product contains $\hat{x}_c$), and $\partial(ab)/\partial \hat{x}_c = a\,\hat{x}_c/b$ (the derivative of a length, times the constant a).

$$\begin{aligned}
\frac{\partial u}{\partial \hat{x}_c} &= \frac{x_c \cdot ab - s \cdot a\hat{x}_c/b}{(ab)^2} && \text{(quotient rule)} \\
&= \frac{x_c}{ab} - \frac{s\,\hat{x}_c}{ab^3} && \text{(split the fraction; cancel } ab \text{ in the first, } a \text{ in the second)} \\
&= \frac{1}{b}\left(\frac{x_c}{a} - u\,\frac{\hat{x}_c}{b}\right). && \text{(factor out } 1/b\text{; } s/(ab) = u\text{)} \qquad (25.4)
\end{aligned}$$

Step 2: the chain rule through arccos.

$$\frac{\partial \theta}{\partial \hat{x}_c} = \frac{d\theta}{du} \cdot \frac{\partial u}{\partial \hat{x}_c} = -\frac{1}{\sqrt{1-u^2}} \cdot \frac{1}{b}\left(\frac{x_c}{a} - u\,\frac{\hat{x}_c}{b}\right). \qquad (25.5)$$

Reading the formula. The bracket is the unit truth $\mathbf{x}/a$ minus u times the unit reconstruction $\hat{\mathbf{x}}/b$: the part of the truth's direction that is perpendicular to the reconstruction. Two facts follow.

1. **The gradient is perpendicular to $\hat{\mathbf{x}}$.** Take the dot product of the bracket with $\hat{\mathbf{x}}$: $\langle \mathbf{x}, \hat{\mathbf{x}} \rangle / a - u \langle \hat{\mathbf{x}}, \hat{\mathbf{x}} \rangle / b = s/a - ub = ub - ub = 0$. So the angle's gradient never asks the reconstruction to get brighter or darker, only to rotate. That is scale invariance seen through the derivative.

2. **The product is finite, even though one factor is not.** The squared length of the bracket is

$$\left\| \tfrac{\mathbf{x}}{a} - u\tfrac{\hat{\mathbf{x}}}{b} \right\|^2 = 1 - 2u \cdot u + u^2 = 1 - u^2,$$

(expand the square; the two unit vectors have length 1 and dot product u). So $\|\partial u/\partial \hat{\mathbf{x}}\| = \sqrt{1-u^2}/b$, and

$$\left\|\frac{\partial \theta}{\partial \hat{\mathbf{x}}}\right\| = \frac{1}{\sqrt{1-u^2}} \cdot \frac{\sqrt{1-u^2}}{b} = \frac{1}{\|\hat{\mathbf{x}}\|}.$$

The $\sqrt{1-u^2}$ blowing up in the arccos factor is cancelled exactly by the same quantity shrinking in the cosine's gradient. In exact arithmetic the spectral angle's gradient has the fixed length $1/\|\hat{\mathbf{x}}\|$ at every angle $\theta > 0$. Only at $\theta = 0$ is its direction undefined: the angle has a corner there, as $|e|$ has at $e = 0$.

Worked example 25.7: The gradient of the angle for pixel A

Continue worked example 25.4: $a = 0.374166$, $b = 0.394588$, $s = 0.147$, $u = 0.995655$.
The first term of Equation (25.4), $x_c/(ab)$ with $ab = 0.147641$:

$$0.100/0.147641 = 0.677317$$
$$0.200/0.147641 = 1.354633$$
$$0.300/0.147641 = 2.031950$$

The second term, $s\,\hat{x}_c/(ab^3)$. First $b^3 = 0.394588^3 = 0.061437$, then $ab^3 = 0.374166 \times 0.061437 = 0.022988$, and $s/(ab^3) = 0.147/0.022988 = 6.39470$:

$$6.39470 \times 0.120 = 0.767364$$
$$6.39470 \times 0.180 = 1.151047$$
$$6.39470 \times 0.330 = 2.110252$$

Subtract to get $\partial u/\partial \hat{\mathbf{x}}$:

$$0.677317 - 0.767364 = -0.090048$$
$$1.354633 - 1.151047 = +0.203586$$
$$2.031950 - 2.110252 = -0.078302$$

The arccos factor. $1 - u^2 = 1 - 0.991329 = 0.008671$; $\sqrt{0.008671} = 0.093116$; $-1/0.093116 = -10.7393$.
Multiply to get $\partial\theta/\partial \hat{\mathbf{x}}$:

$$-10.7393 \times (-0.090048) = +0.96705$$
$$-10.7393 \times (+0.203586) = -2.18638$$
$$-10.7393 \times (-0.078302) = +0.84092$$

Interpret. Gradient descent moves $\hat{\mathbf{x}}$ *against* the gradient: down in band 1, up in band 2, down in band 3. Compare the unit vectors: $\mathbf{x}/a = (0.267, 0.535, 0.802)$ and $\hat{\mathbf{x}}/b = (0.304, 0.456, 0.836)$. The reconstruction leans too far towards bands 1 and 3 and not enough towards band 2; the gradient corrects exactly that.
Check the two facts. The dot product with $\hat{\mathbf{x}}$: $0.120(-0.090048) + 0.180(0.203586) + 0.330(-0.078302) = -0.010806 + 0.036645 - 0.025840 = -0.000001 \approx 0$. The length: $\sqrt{0.96705^2 + 2.18638^2 + 0.84092^2} = \sqrt{0.93519 + 4.78026 + 0.70715} = \sqrt{6.42260} = 2.5343$, and $1/b = 1/0.394588 = 2.5343$.
PyTorch's autograd through `torch.acos` in float64 returns $(0.96705, -2.18638, 0.84092)$ [C]; the real `SAMLoss` in float32 returns $(0.96701, -2.18630, 0.84087)$ [C] (`research/textbook/tools/compute_23-losses_output.txt`).

25.9 Why the code does not call arccos

If the gradient's length is a tame $1/\|\hat{\mathbf{x}}\|$, what is the danger? The danger is in how a computer evaluates Equation (25.5). Automatic differentiation computes the two factors separately and then multiplies them. The arccos factor is evaluated from u as computed in floating point, and close to $u = 1$ floating point cannot tell nearby angles apart.

How coarse is float32 near 1? A float32 number has 24 significant bits, so the gap between 1 and the next smaller representable number is $2^{-24} = 5.96 \times 10^{-8}$. For a small angle, $\cos\theta \approx 1 - \theta^2/2$. When $\theta^2/2$ is less than half the gap, 2.98×10^{-8}, the cosine rounds to exactly 1:

$$\frac{\theta^2}{2} < 2.98 \times 10^{-8} \iff \theta < \sqrt{5.96 \times 10^{-8}} = 2.44 \times 10^{-4} \text{ rad.}$$

Indeed, in float32 $\cos(3 \times 10^{-4}) = 0.99999994$ but $\cos(2 \times 10^{-4}) = 1.0$ exactly [C]. Below about 2.4×10^{-4} rad (0.014°) the cosine carries no information, and the computed u is rounding noise that can even exceed 1.

Worked example 25.8: The naive formula producing NaN

Feed a perfect reconstruction, $\hat{\mathbf{x}} = \mathbf{x}$, through `torch.acos(dot/(norm_x*norm_hat))` in float32 and ask for the gradient (`research/textbook/tools/compute_23-losses.py`).

- $\mathbf{x} = (0.2, 0.2, 0.2)$: the computed u is 1.0000001, above 1. arccos of it is NaN ("not a number"), and so is the gradient. With the usual guard `u.clamp(-1,1)` the angle becomes 0 and the gradient becomes exactly 0 (a clamp passes no gradient once it is active), so training silently stops pushing.
- $\mathbf{x} = (0.25, 0.5, 0.125)$: u comes out exactly 1.0. arccos gives 0, but the factor $-1/\sqrt{1-u^2}$ is $-\infty$ and $\partial u/\partial\hat{\mathbf{x}}$ is 0, and $\infty \times 0$ is NaN. The clamp does not help here.

One NaN in one pixel's gradient spreads through the optimiser into every parameter at the next step. And it appears precisely when the model has become good: late in a long, expensive run.

25.9.1 The atan2 formulation

`SAMLoss` (`app/foundation_models/components/sam_loss.py`) rewrites the angle with a sine as well as a cosine. With $N = \|\mathbf{x}\|\,\|\hat{\mathbf{x}}\|$:

$$\begin{aligned} N\cos\theta &= \langle \mathbf{x}, \hat{\mathbf{x}} \rangle && \text{(the dot product)} \\ N\sin\theta &= \sqrt{N^2 - \langle \mathbf{x}, \hat{\mathbf{x}} \rangle^2} && \text{(from } \sin^2\theta = 1 - \cos^2\theta\text{, times } N^2\text{)} \\ \theta &= \operatorname{atan2}(N\sin\theta,\ N\cos\theta), \end{aligned}$$

where $\operatorname{atan2}(y, x)$ is the angle of the point (x, y) measured from the positive x-axis, in $[0, \pi]$ when $y \geq 0$. Its derivatives, $\partial/\partial y = x/(x^2+y^2)$ and $\partial/\partial x = -y/(x^2+y^2)$, are finite everywhere except at the origin.

The code, with its guards:

```
x_m  = x * mask                                                    # (B, 165, H,
    W)
xh_m = x_hat * mask
dot      = (xh_m * x_m).sum(dim=1, keepdim=True)                   # (B, 1, H, W)
norm_hat = (xh_m * xh_m).sum(dim=1, keepdim=True).add(self.eps).sqrt() # guard 0
```

```
norm_x   = (x_m * x_m).sum(dim=1, keepdim=True).add(self.eps).sqrt()
cross_norm  = (norm_hat * norm_x).clamp(min=self.eps)                    # guard 1
cos_term    = dot.clamp(-cross_norm, cross_norm)                         # guard 2
sin_term_sq = (cross_norm ** 2 - cos_term ** 2).clamp(min=0)
sin_term    = (sin_term_sq + self.eps).sqrt()                            # guard 3
angles      = torch.atan2(sin_term, cos_term)                            # (B, 1, H, W)
    , in [0, pi]
```

with $\varepsilon = 10^{-8}$ (`eps`, line 30).

Guard 0, $+\varepsilon$ inside each length.
A masked pixel is a zero vector; its length would be $\sqrt{0}$, whose derivative $1/(2\sqrt{0})$ is infinite.

Guard 1, `clamp(min=eps)` on the product of lengths.
Prevents a zero (or vanishing) N.

Guard 2, clamp the dot product to $[-N, N]$.
Mathematically $|\langle \mathbf{x}, \hat{\mathbf{x}} \rangle| \le N$ (Cauchy–Schwarz); in floating point it can exceed it by a hair, as in worked example 25.8, and then $N^2 - \text{dot}^2$ would be negative.

Guard 3, $+\varepsilon$ inside the sine's square root.
For a perfect reconstruction $N^2 - \text{dot}^2 = 0$, and $d\sqrt{v}/dv = 1/(2\sqrt{v})$ is infinite at $v = 0$: the same disease as arccos, one level down. Adding ε keeps the sine term at least $\sqrt{10^{-8}} = 10^{-4}$, so the derivative of the square root is at most $1/(2 \times 10^{-4}) = 5000$.

Source check: the module docstring, and the guard the course leaves out

The module docstring of `app/foundation_models/components/sam_loss.py` (line 9) still writes the loss as $\arccos\left(\langle a, b\rangle / (\|a\|\,\|b\| + \varepsilon)\right)$; the code (lines 58–70) uses the atan2 form above, with the comment that it "avoids arccos which has infinite gradient at ± 1". The code wins. The course (IW24) lists three guards; the code has a fourth, the $+\varepsilon$ inside each length on lines 55–56 (guard 0 above).

Worked example 25.9: The atan2 form on pixel A, and on a perfect reconstruction

Pixel A. $N = 0.147641$, dot $= 0.147$.

$$
\begin{aligned}
N^2 &= 0.147641^2 = 0.021798 \\
\text{dot}^2 &= 0.147^2 = 0.021609 \\
N^2 - \text{dot}^2 &= 0.000189 \\
\sqrt{0.000189} &= 0.013748 \\
\theta &= \text{atan2}(0.013748,\ 0.147) = \arctan(0.013748/0.147) \\
&= \arctan(0.093524) = 0.09325
\end{aligned}
$$

The same angle as arccos gave, with a safe derivative. (The $\varepsilon = 10^{-8}$ under the root changes the sine term only in the seventh decimal place here.)
A perfect reconstruction. With $\hat{\mathbf{x}} = \mathbf{x} = (0.1, 0.2, 0.3)$, $N = \|\mathbf{x}\|^2 = 0.14$ and $N^2 - \text{dot}^2 = 0$, so the sine term is $\sqrt{0 + 10^{-8}} = 10^{-4}$ and

$$\theta = \text{atan2}(10^{-4},\ 0.14) = \arctan(0.000714) = 0.000714 \text{ rad}.$$

The loss never quite reaches zero: guard 3 sets a floor. The real `SAMLoss` returns 0.00084 here [C] (the floor plus float32 rounding) and a finite gradient; for $(0.2, 0.2, 0.2)$ and $(0.25, 0.5, 0.125)$, the two vectors that broke the naive formula in worked example 25.8, it returns finite angles below 0.001 rad and finite gradients [C].

A side effect: the floor depends on brightness. The floor is $\arctan(10^{-4}/\|\mathbf{x}\|^2)$. For a real 165-band pixel at reflectance about 0.2 [A] (an assumed typical level), $\|\mathbf{x}\|^2 \approx 165 \times 0.2^2 = 6.6$ and the floor is $10^{-4}/6.6 = 1.5 \times 10^{-5}$ rad, a thousandth of the 0.02–0.05 rad a good model reaches. A very dark pixel has a higher floor. This is a tiny breach of scale invariance, irrelevant to training, and the price of the guard.

25.9.2 Masking and averaging the angle

```
spatial_mask = mask[:, 0:1, :, :]                          # (B, 1, H, W)
num_valid    = spatial_mask.sum().clamp(min=1)             # number of graded pixels,
    at least 1
sam_loss     = (angles * spatial_mask).sum() / num_valid  # ()  mean angle over
    graded pixels
```

So the spectral-angle term is

$$L_{\text{SAM}} = \frac{1}{|M|} \sum_{p \in M} \theta_p, \tag{25.6}$$

the mean angle over the graded pixels. Unlike L1 it keeps the spatial shape and divides by an explicit count, which `clamp(min=1)` keeps from being zero.

25.9.3 Where arccos is fine

Two other places compute the spectral angle with plain arccos: the inferencer's `compute_anomaly_scores` and `_sam()` in `app/utils/anomaly_detection/scoring.py` (lines 118–128), the numpy version the product calls. Both run at inference, where no gradient is taken, so the infinite derivative never arises; only the domain clamp `np.clip(cos, -1, 1)` is needed. Numerical-stability requirements differ between the forward pass and the backward pass. The rounding floor near $u = 1$ still applies to those scores: an angle below about 2.4×10^{-4} rad reads as 0 in float32.

25.10 The combined loss and the ramp

25.10.1 The objective

Definition 25.6 (Combined reconstruction loss)**.**

$$L(t) = L_1 + \lambda(t)\, L_{\text{SAM}}, \tag{25.7}$$

with L_1 from Equation (25.1), L_{SAM} from Equation (25.6), both over the same loss mask, and $\lambda(t)$ the *SAM weight* at epoch index t (`sam_weight` in the code).

Its gradient is the sum of the two gradients, the second scaled by $\lambda(t)$, because differentiation is linear: $\partial L/\partial \hat{x} = \partial L_1/\partial \hat{x} + \lambda(t)\, \partial L_{\text{SAM}}/\partial \hat{x}$.

25.10.2 The ramp

λ is not constant. From `app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, lines 197–204:

```
def _get_sam_weight(self, epoch: int) -> float:
    sam_max     = cfg.sam_weight
    ramp_epochs = cfg.sam_ramp_epochs
```

```
    if ramp_epochs <= 0:
        return sam_max
    return sam_max * min(1.0, epoch / ramp_epochs)
```

Definition 25.7 (Linear ramp)**.** With final weight $\lambda_{\max}$ (`sam_weight`) and ramp length R (`sam_ramp_epochs`),

$$\lambda(t) = \lambda_{\max} \cdot \min\left(1, \frac{t}{R}\right), \qquad t = 0, 1, 2, \ldots \tag{25.8}$$

and $\lambda(t) = \lambda_{\max}$ for all t if $R \leq 0$.

A straight line from 0 to $\lambda_{\max}$ over R epochs, then flat forever. Three details from the source matter.

- t is the zero-based *epoch index*. The trainer sets `self._current_epoch=epoch` at the start of each epoch (line 53) from `range(start_epoch,num_epochs)`, and the log line prints `epoch+1`. So the logged "Epoch 1" trains with $\lambda = 0$, and full strength arrives at logged epoch $R + 1$.
- λ is updated once per epoch, not per step. Within an epoch it is constant, so the ramp is really a staircase of R steps (Figure 25.4).
- The fine-tuning configurations set `sam_ramp_epochs` to 0 (FT02c onwards), so they run at $\lambda_{\max} = 0.5$ from the first epoch: a model that already reconstructs well does not need the ramp. (FT01, FT02 and FT02b still had $R = 10$.)
  [M: `research/experiments/indradhanu-finetune/FT04-final-joint/config.json`]

25.10.3 Why ramp at all

At epoch 0 the parameters are random and the reconstructions have random shape, so the angle is large, about 0.3–0.4 rad [M: `configs/hyperspectral_segformer_exp_2.json`], and its gradient points in a direction that has little to do with the more basic job of getting the levels roughly right. Optimising both from a random start is unstable; one tends to sabotage the other. The ramp says: learn the easy thing first. Early, $\lambda \approx 0$ and L1 leads; later, the shape constraint tightens on a model that already produces sensible magnitudes.

The configuration names the failure mode of turning shape on too fast:

```
"L1_increases_when_SAM_ramp_starts": "SAM weight too high (try sam_weight=0.5)"
```

If the shape penalty is too strong, the model sacrifices brightness accuracy to satisfy it and L1 goes *up*.

25.10.4 The two shipped configurations

setting	v0.1.0	v0.2.0
`sam_weight` ($\lambda_{\max}$)	1.0	0.5
`sam_ramp_epochs` (R)	20	10
`mask_ratio`	0.50	0.65
`compressed_channels` (D)	24	32
`drop_rate`	0.3	0.4
`learning_rate`	10^{-4}	10^{-3}
`num_epochs`	500	200

Values from `configs/hyperspectral_segformer_exp_1.json` and `configs/hyperspectral_segformer_exp_2.json` [M: `configs/hyperspectral_segformer_exp_2.json`]. v0.2.0 halved both the final weight and the ramp length: it reaches full strength twice as fast, but full strength is half as strong.

Worked example 25.10: The v0.2.0 ramp, epoch by epoch

With $\lambda_{\max} = 0.5$ and $R = 10$, $\lambda(t) = 0.5 \min(1, t/10)$.

epoch index t	logged epoch	$t/10$	$\min(1, t/10)$	λ
0	1	0.0	0.0	0.00
1	2	0.1	0.1	0.05
2	3	0.2	0.2	0.10
5	6	0.5	0.5	0.25
9	10	0.9	0.9	0.45
10	11	1.0	1.0	0.50
50	51	5.0	1.0	0.50
199	200	19.9	1.0	0.50

Row $t = 5$ in full: $5/10 = 0.5$; $\min(1, 0.5) = 0.5$; $0.5 \times 0.5 = 0.25$.

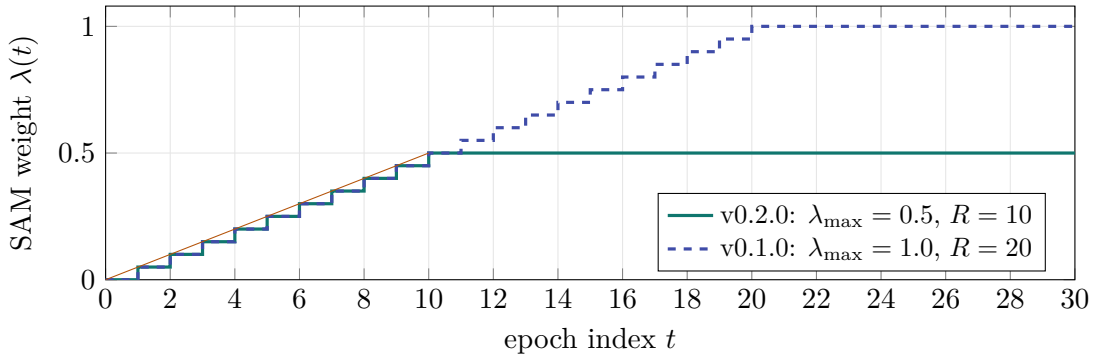


Figure 25.4: The SAM-weight ramp of both shipped configurations, drawn to scale from Equation (25.8). The weight is held for a whole epoch, so each ramp is a staircase; the thin amber line is the formula $\lambda = 0.05\,t$ that the stairs follow. Both configurations happen to rise at the same rate, 0.05 per epoch; v0.2.0 stops at half the height. After the ramp both stay flat for the rest of training (to $t = 199$ or 499).

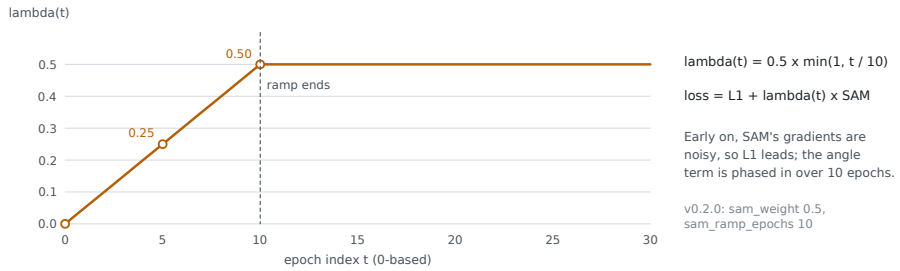


Figure 25.5: The course's drawing of the v0.2.0 ramp: the weight on the angle rises linearly from 0 at epoch index 0 to 0.5 at epoch index 10 and then stays flat, so L1 leads early training and the angle is phased in.

25.10.5 After the ramp, the angle dominates

Worked example 25.11: The combined loss at the configuration's epoch-100 baseline

The configuration's baseline at epoch 100 is about $L_1 = 0.010$ and $L_{\text{SAM}} = 0.060$ rad [M: configs/hyperspectral_segformer_exp_2.json] (midpoints of its ranges). At $t = 100$, $\lambda = 0.5$.

$$\begin{aligned} L &= 0.010 + 0.5 \times 0.060 \\ &= 0.010 + 0.030 \\ &= 0.040 \end{aligned}$$

Shares: $0.010/0.040 = 25\%$ from L1, $0.030/0.040 = 75\%$ from the angle.

So after the ramp the shape term carries three times the weight of the brightness term. The reason is units. L1 is in reflectance and a good value is about 0.01; the angle is in radians and a good value is about 0.06, six times larger. Multiplying by $\lambda = 0.5$ leaves it three times larger. λ is doing unit conversion as much as weighting: for a hyperspectral model, spectral shape is meant to be the dominant signal, and it is.

25.10.6 Watch the components, not the sum

During the ramp, λ grows, so the combined number can rise while both components fall.

t	L_1	L_{SAM}	λ	combined
0	0.090	0.35	0.00	$0.090 + 0.00 \times 0.35 = 0.090$
10	0.040	0.20	0.50	$0.040 + 0.50 \times 0.20 = 0.040 + 0.100 = 0.140$

These are illustrative values inside the configuration's baseline ranges [A]. L1 more than halved, the angle nearly halved, and the combined loss rose by 56% (0.140/0.090 = 1.56). The trainer logs the components separately for this reason (`L1:...|SAM:...rad(...deg)|lambda:...`, lines 65–68) and mirrors them to Weights and Biases as `train/l1_loss`, `train/sam_loss`, `train/sam_loss_deg` and `train/sam_weight`.

Values that will drift

The configuration's baseline table for v0.1.0's schedule (from `configs/hyperspectral_segformer_exp_2.json`, `_loss_reference`; its λ column reflects v0.1.0's 20-epoch ramp to 1.0) [M: configs/hyperspectral_segformer_exp_2.json]:

epoch	L_1	angle	combined
0	0.08–0.10	0.3–0.4 rad (20–25°)	0.08–0.10
10	0.03–0.05	0.15–0.25 rad (10–15°)	0.10–0.15
20	0.02–0.03	0.08–0.15 rad (5–8°)	0.10–0.15
100	0.005–0.015	0.04–0.08 rad (2–5°)	0.04–0.08
500	0.003–0.008	0.02–0.05 rad (1–3°)	0.02–0.05

The combined column rises from epoch 0 to epoch 10 and then falls: the ramp artefact, in the project's own documented baselines. The same file's red flags: L1 not below 0.05 by epoch 10 (learning rate too low, or data loading); angle stuck above 0.3 after epoch 30 (bottleneck D too narrow, try 32); L1 rising when the ramp starts (`sam_weight` too high, try 0.5); training loss falling but validation not (overfitting). Two of these were hit and fixed between v0.1 and v0.2: D went 24 to 32 and `sam_weight` 1.0 to 0.5.

The v0.2.0 checkpoint chosen for the product had validation loss 0.04349, and the v0.1.0 family's best validation L1 was 0.077 at epoch 92 [M: research/model_break_down/05_hyperspectral_segformer_mae.md].

25.10.7 What `val_loss` means, and why it cannot be compared across models

A checkpoint's `val_loss` of 0.04349 is $L_1 + 0.5\,L_{\mathrm{SAM}}$ on the validation patches. It is not an accuracy, not a percentage and not an L1. A thermal model's `val_loss` (Chakshu) is pure L1 on one band in degrees Celsius: different terms, different units. Comparing the two numbers is meaningless; models are compared on detection scores (Chapter 33).

25.10.8 Validation grades every pixel exactly once

Training hides a random 65% of the valid tokens, so the training loss depends on which tokens happened to be hidden. Validation removes that luck: `compute_validation_loss` (lines 286–342) draws one random half-split of the tokens and runs the model twice, hiding each half once:

```
rand_mask   = (torch.rand(1, N, device=self.device) > 0.5).float()   # (1, N)
pred_mask_1 = token_validity * (1.0 - rand_mask)    # hide where rand_mask = 0
pred_mask_2 = token_validity * rand_mask            # hide the exact complement
x_hat = x_hat_1 * pass1_pixels + x_hat_2 * pass2_pixels   # each pixel from the pass
    that hid it
```

Every valid pixel is then graded once, from a prediction that did not see it, over the eroded mask alone (no prediction-mask factor). The number is comparable across epochs, which is what choosing a checkpoint requires.

Source check: the ramped weight in validation

In `app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py` (line 337), the validation loss also uses the *ramped* weight $\lambda(t)$. That makes validation numbers from different epochs of the ramp incomparable for the same reason the training sum is: at $t = 0$ the score is pure L1 and looks best of all. For v0.2.0 the ramp ends at $t = 10$, long before the chosen checkpoint, so it did not bite. In the SpectralEarth trainer, whose validation score drives early stopping and `best.pt`, it did bite, and that trainer now scores validation at the full weight from the first epoch (`app/foundation_models/trainers/spectral_stem_mae_trainer.py`, lines 337–341). The story is told in Chapter 27.

A loss is a marking scheme: L1 marks levels, the angle marks shape, and $\lambda(t)$ both schedules when shape starts to count and converts radians into the same currency as reflectance.

25.11 Common confusions

Why average bands first and then pixels? Is averaging an average not wrong?
It would be wrong if the groups had different sizes. Every pixel has the same 165 bands, so the mean of per-pixel means equals the overall mean (Section 25.2). It is done this way so that a per-pixel value exists for the mask to select.

Does the loss include pixels the model could see?
No. The pixel prediction mask excludes them; grading them would reward copying the input.

Is the spectral angle a distance?
Not in the usual sense. It is an angle, bounded in $[0, \pi]$, and two spectra that differ enormously in level can have angle 0 (worked example 25.5).

The derivative of arccos is infinite at 1. Does the angle's gradient really blow up?
In exact arithmetic, no: the gradient has length $1/\|\hat{\mathbf{x}}\|$ at every nonzero angle (Section 25.8). The trouble is computing it as a product of an exploding factor and a vanishing factor in float32, where u near 1 is rounding noise (worked example 25.8).

Would normalising each spectrum and then taking L1 give the same thing as the angle?
Close, but not identical, and you would lose the brightness information entirely. Keeping two terms lets you weight them separately.

Why radians and not degrees?
Radians are what the trigonometric functions produce. The trainer converts to degrees only when it logs.

Is a lower combined loss always better?
Only between epochs with the same λ. During the ramp the weighting changes, so the combined number is not comparable epoch to epoch.

Is the L1 loss the same as the L1 anomaly score?
Same formula, different job. Here it is averaged into one number to train with; at scoring time it is kept per pixel as a map (Chapter 30).

25.12 Check yourself

Exercise 25.1. By hand: compute the masked L1 for two graded pixels with two bands each, $\mathbf{x}_A = (0.10, 0.20)$, $\hat{\mathbf{x}}_A = (0.13, 0.16)$ and $\mathbf{x}_B = (0.30, 0.05)$, $\hat{\mathbf{x}}_B = (0.30, 0.09)$, and give the gradient $\partial L_1/\partial \hat{\mathbf{x}}$ for all four entries.

Exercise 25.2. By hand: compute the spectral angle between $\mathbf{x} = (1, 0)$ and $\hat{\mathbf{x}} = (1, 1)$ in radians and degrees, then compute $\partial\theta/\partial\hat{\mathbf{x}}$ from Equation (25.5), and check that it is perpendicular to $\hat{\mathbf{x}}$ and has length $1/\|\hat{\mathbf{x}}\|$.

Exercise 25.3. Explain why the derivative of arccos near $u = 1$ matters *more* as training succeeds, and why, nonetheless, the true gradient of the angle does not blow up. What does go wrong in float32, and which guard in `SAMLoss` prevents it?

Exercise 25.4. For v0.2.0 ($\lambda_{\max} = 0.5$, $R = 10$): compute λ at epoch index 4, and the combined loss at epoch index 50 if $L_1 = 0.02$ and $L_{\mathrm{SAM}} = 0.12$. Which term contributes more, and by what factor?

Exercise 25.5. The five observations $(0.10, 0.11, 0.12, 0.13, 0.90)$ of worked example 25.3: compute $S_1(c)$ and $S_2(c)$ at $c = 0.12$ and at $c = 0.272$, and say which c each loss prefers.

Exercise 25.6. Why must the token-to-pixel upsampling of the prediction mask be nearest-neighbour? What exactly goes wrong with bilinear?

Next: Chapter 26, Training data and the training loop: gradients, AdamW, warmup and cosine schedules.

Chapter 26

Training data and the training loop: gradients, AdamW, warmup and cosine schedules

The one thing this chapter teaches

Training repeats one small move millions of times: take a batch of patches, compute the loss of Chapter 25, compute its gradient with respect to every parameter, and let the optimiser (here Adam) step each parameter a little against its gradient, with a step size that a schedule (here cosine annealing) shrinks over the run. Everything else in the loop, from shards to checkpoints, exists to keep that move fed, honest and resumable.

Chapter 25 ended with a number, the loss, and its derivative with respect to the reconstruction. Two gaps remain between that and a trained model. First, the network has about 5.5 million parameters [M:`research/model_break_down/05_hyperspectral_segformer_mae.md`], not one reconstruction; you need the derivative with respect to each of them and a rule for what to do with it. Second, the loss is computed on one batch of patches, and the satellite archive holds millions; you need a pipeline that turns scenes into a steady stream of batches, and a loop that consumes it. This chapter builds both: the mathematics of the update rule first (Sections 26.2 to 26.8), then the data (Section 26.10), then the loop that ties them together (Section 26.11).

The obvious approach, "try a change to each parameter and keep it if the loss went down", fails on arithmetic alone: one trial per parameter is 5.5 million forward passes for a single improvement. The gradient gives you the best local direction for all 5.5 million parameters at the cost of about one extra pass.

Analogy

Training is walking down a fog-bound hillside to the valley floor. You cannot see the valley; you can feel the slope under your feet (the gradient). Plain gradient descent takes a step straight downhill each time. Momentum is a heavy ball that keeps rolling through small bumps. Adam is a walker who shortens their stride on ground that has been steep and jagged and lengthens it on ground that has been gentle and consistent. Warmup is taking small steps while your eyes adjust to the fog; cosine annealing is slowing down as you near the bottom so you stop in the valley instead of striding across it.

26.1 Vocabulary: parameter, gradient, batch, step, epoch

Definition 26.1 (Gradient)**.** For a loss L that depends on parameters $\mathbf{w} = (w_1, \ldots, w_P)$, the *gradient* is the vector of partial derivatives,

$$\nabla L = \nabla_{\mathbf{w}} L = \left(\frac{\partial L}{\partial w_1}, \ldots, \frac{\partial L}{\partial w_P} \right).$$

Entry j says how fast the loss changes when w_j alone is nudged up. The gradient points in the direction of steepest increase, so $-\nabla L$ points steepest downhill.

Definition 26.2 (Batch, step, epoch)**.** A *batch* (or *mini-batch*) is the group of patches processed together in one forward pass; its size B is `batch_size`. A *step* (or *iteration*) is one update of the parameters. In most textbooks an *epoch* is one full pass over the training set; in this project an epoch is a fixed *sample budget* instead (Section 26.10.3), the interval at which validation, the learning-rate schedule, the SAM ramp and checkpointing advance.

The loss over a batch is the mean of the per-patch losses, and the gradient of a mean is the mean of the gradients (differentiation is linear). So the batch gradient is an *estimate* of the gradient you would get from the whole training set: noisy, but cheap. Training with such estimates is *stochastic gradient descent*. A larger batch gives a less noisy estimate; it does not give a different expected direction.

Backpropagation, in one paragraph. Chapter 25 computed $\partial L/\partial \hat{\mathbf{x}}$. The reconstruction $\hat{\mathbf{x}}$ is the output of the decoder, which is a function of its weights and of the encoder's output, and so on back to the compressor. The chain rule (Section 25.6) carries the derivative backwards layer by layer: each layer receives $\partial L/\partial$(its output), and computes from it $\partial L/\partial$(its weights) and $\partial L/\partial$(its input), which it passes on. PyTorch does this automatically when you call `loss.backward()`; it stores each parameter's derivative in the parameter's `.grad` field, *adding* to whatever is already there. That adding is why the loop calls `optimizer.zero_grad()` every step, and it is what gradient accumulation exploits (Section 26.9).

26.2 Gradient descent, by hand

Definition 26.3 (Gradient descent)**.** With *learning rate* (step size) $\eta > 0$, each step replaces every parameter by

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \, \nabla L(\mathbf{w}_t). \tag{26.1}$$

To see the rule work, use a toy loss with one parameter whose answer you know: $L(w) = (w-3)^2$, smallest at $w = 3$. Its derivative, by the chain rule, is $L'(w) = 2(w-3)$.

Worked example 26.1: Three steps of gradient descent

Do this on paper. Start at $w_0 = 0$ (loss $(0-3)^2 = 9$), with $\eta = 0.1$.
Step 1.

$$g_0 = 2(0-3) = -6$$
$$w_1 = 0 - 0.1 \times (-6) = 0 + 0.6 = 0.6$$
$$L(w_1) = (0.6-3)^2 = (-2.4)^2 = 5.76$$

Step 2.

$$g_1 = 2(0.6 - 3) = 2 \times (-2.4) = -4.8$$
$$w_2 = 0.6 - 0.1 \times (-4.8) = 0.6 + 0.48 = 1.08$$
$$L(w_2) = (1.08 - 3)^2 = (-1.92)^2 = 3.6864$$

Step 3.

$$g_2 = 2(1.08 - 3) = 2 \times (-1.92) = -3.84$$
$$w_3 = 1.08 - 0.1 \times (-3.84) = 1.08 + 0.384 = 1.464$$
$$L(w_3) = (1.464 - 3)^2 = (-1.536)^2 = 2.359296$$

The loss falls $9 \to 5.76 \to 3.69 \to 2.36$. The negative gradient pointed right (towards 3) every time, and the steps shrink as the slope flattens.

Why the steps shrink, and when they explode. Substitute the derivative into Equation (26.1) and subtract 3 from both sides:

$$\begin{aligned} w_{t+1} &= w_t - 2\eta(w_t - 3) && \text{(the update with } L' = 2(w-3)\text{)} \\ w_{t+1} - 3 &= (w_t - 3) - 2\eta(w_t - 3) && \text{(subtract 3 from both sides)} \\ w_{t+1} - 3 &= (1 - 2\eta)(w_t - 3). && \text{(factor out } w_t - 3\text{)} \end{aligned}$$

The distance to the answer is multiplied by $1 - 2\eta$ every step. With $\eta = 0.1$ the factor is 0.8: $-3 \to -2.4 \to -1.92 \to -1.536$, exactly the distances above. The method converges when $|1 - 2\eta| < 1$, that is $0 < \eta < 1$. With $\eta = 1.1$ the factor is -1.2: each step overshoots to the other side and lands further away, $w = 6.6, -1.32, 8.184$, and the loss grows $12.96, 18.66, 26.87$ [C]. A learning rate too large does not merely slow training; it makes it diverge. Real networks have millions of such directions with different curvatures, and the learning rate must suit the steepest of them.

26.3 Momentum

Plain gradient descent forgets everything at every step. On a long, gently sloping valley it crawls; on a narrow one it zig-zags across. *Momentum* [35] keeps a running velocity:

$$\mathbf{v}_t = \beta\,\mathbf{v}_{t-1} + \nabla L(\mathbf{w}_t), \qquad \mathbf{w}_{t+1} = \mathbf{w}_t - \eta\,\mathbf{v}_t, \qquad \mathbf{v}_{-1} = \mathbf{0}, \tag{26.2}$$

with β typically 0.9. (This is PyTorch's `SGD(momentum=0.9)` form.) Gradients that keep pointing the same way add up; gradients that alternate cancel.

Worked example 26.2: Three steps with momentum

Same toy, $w_0 = 0$, $\eta = 0.1$, $\beta = 0.9$.
Step 1. $g = 2(0 - 3) = -6$. $v = 0.9 \times 0 + (-6) = -6$. $w = 0 - 0.1 \times (-6) = 0.6$.
Step 2. $g = 2(0.6 - 3) = -4.8$.

$$v = 0.9 \times (-6) + (-4.8) = -5.4 - 4.8 = -10.2$$
$$w = 0.6 - 0.1 \times (-10.2) = 0.6 + 1.02 = 1.62$$

Step 3. $g = 2(1.62 - 3) = -2.76$.

$$v = 0.9 \times (-10.2) + (-2.76) = -9.18 - 2.76 = -11.94$$
$$w = 1.62 - 0.1 \times (-11.94) = 1.62 + 1.194 = 2.814$$

After three steps $w = 2.814$ (loss 0.0346), against plain descent's 1.464 (loss 2.36). PyTorch's `SGD` with momentum gives 2.814 [C]. The velocity is still large, so step 4 will overshoot past 3; on this toy, momentum oscillates around the answer before settling.

26.4 Adam

26.4.1 The problem Adam solves

Different parameters see gradients of wildly different sizes. A bias in the final layer may see gradients of order 1; a weight deep in the encoder, of order 10^{-4}. One learning rate cannot suit both: large enough for the second makes the first diverge. Adam [19] gives every parameter its own effective step size, by dividing its gradient by a running estimate of the gradient's typical magnitude.

26.4.2 Exponential moving averages

Definition 26.4 (Exponential moving average)**.** An *exponential moving average* (EMA) of a sequence $g_1, g_2, \ldots$ with decay β is

$$m_t = \beta\, m_{t-1} + (1 - \beta)\, g_t, \qquad m_0 = 0.$$

Unroll it to see what it averages:

$$m_1 = (1 - \beta) g_1$$
$$m_2 = \beta(1 - \beta) g_1 + (1 - \beta) g_2$$
$$m_t = (1 - \beta) \sum_{k=1}^{t} \beta^{t-k} g_k.$$

Each older gradient's weight shrinks by a factor β per step: with $\beta = 0.9$ a gradient ten steps old has weight $0.9^{10} = 0.35$ of the newest; with $\beta = 0.999$ the memory is about a thousand steps long.

26.4.3 The algorithm

Definition 26.5 (Adam)**.** For each parameter w separately, with gradient g_t at step $t = 1, 2, \ldots$, decays β_1, β_2, learning rate η and a small ϵ:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t \qquad \text{(EMA of the gradient: first moment)} \tag{26.3}$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2 \qquad \text{(EMA of the squared gradient: second moment)} \tag{26.4}$$
$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \qquad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \qquad \text{(bias correction)} \tag{26.5}$$
$$w_t = w_{t-1} - \eta\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}. \qquad \text{(the step)} \tag{26.6}$$

PyTorch's defaults, which this project uses unchanged, are $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$ and weight decay 0.

$\hat{m}_t$ estimates the mean gradient; $\sqrt{\hat{v}_t}$ estimates its root-mean-square size. Their ratio is near ± 1 when the gradient is consistent and near 0 when it flips sign from step to step. So each parameter moves about η per step when its gradient is steady, whatever the gradient's scale, and less when it is noisy.

26.4.4 Why bias correction

Both EMAs start at 0, so early on they are too small. Suppose the gradient's distribution does not change, with mean $\mathbb{E}[g]$. Take expectations of the unrolled form:

$$\begin{aligned}
\mathbb{E}[m_t] &= (1-\beta_1)\sum_{k=1}^{t}\beta_1^{t-k}\,\mathbb{E}[g] && \text{(expectation is linear)}\\
&= (1-\beta_1)\,\mathbb{E}[g]\,(1+\beta_1+\cdots+\beta_1^{t-1}) && \text{(write out the powers)}\\
&= (1-\beta_1)\,\mathbb{E}[g]\,\frac{1-\beta_1^t}{1-\beta_1} && \text{(sum of a geometric series)}\\
&= (1-\beta_1^t)\,\mathbb{E}[g]. && \text{(cancel } 1-\beta_1\text{)}
\end{aligned}$$

The EMA is short by the factor $1-\beta_1^t$; dividing by it, as in Equation (26.5), removes the bias. The same argument with g^2 gives the factor $1-\beta_2^t$ for v_t.

Worked example 26.3: The bias-correction factors for t = 1, 2, 3

With $\beta_1 = 0.9$:

$$\begin{aligned}
t=1:&\quad 1-0.9 = 0.1\\
t=2:&\quad 1-0.9^2 = 1-0.81 = 0.19\\
t=3:&\quad 1-0.9^3 = 1-0.729 = 0.271
\end{aligned}$$

With $\beta_2 = 0.999$:

$$\begin{aligned}
t=1:&\quad 1-0.999 = 0.001\\
t=2:&\quad 1-0.999^2 = 1-0.998001 = 0.001999\\
t=3:&\quad 1-0.999^3 = 1-0.997003 = 0.002997
\end{aligned}$$

So at $t=1$ the raw m_1 is divided by 0.1 (multiplied by 10) and the raw v_1 by 0.001 (multiplied by 1000). Without correction, the first step would divide a gradient estimate that is 10 times too small by a size estimate that is $\sqrt{1000} = 31.6$ times too small, and the step would be about 3 times too large. The second-moment correction matters for much longer: $1-0.999^t$ is still only 0.095 at $t=100$ and 0.632 at $t=1000$ [C].

Worked example 26.4: Three steps of Adam on the toy loss

Toy $L(w) = (w-3)^2$, $w_0 = 0$, $\eta = 0.1$, default βs; ignore ϵ (it is 10^{-8}).
Step $t=1$. $g_1 = 2(0-3) = -6$.

$$\begin{aligned}
m_1 &= 0.9\times 0 + 0.1\times(-6) = -0.6\\
v_1 &= 0.999\times 0 + 0.001\times 36 = 0.036\\
\hat{m}_1 &= -0.6/0.1 = -6\\
\hat{v}_1 &= 0.036/0.001 = 36, \qquad \sqrt{36} = 6\\
w_1 &= 0 - 0.1\times(-6)/6 = 0 + 0.1 = 0.1
\end{aligned}$$

Step $t = 2$. $g_2 = 2(0.1 - 3) = -5.8$, $g_2^2 = 33.64$.

$$
\begin{aligned}
m_2 &= 0.9 \times (-0.6) + 0.1 \times (-5.8) = -0.54 - 0.58 = -1.12 \\
v_2 &= 0.999 \times 0.036 + 0.001 \times 33.64 = 0.035964 + 0.03364 = 0.069604 \\
\hat{m}_2 &= -1.12/0.19 = -5.894737 \\
\hat{v}_2 &= 0.069604/0.001999 = 34.81941, \qquad \sqrt{34.81941} = 5.900797 \\
w_2 &= 0.1 - 0.1 \times (-5.894737)/5.900797 = 0.1 + 0.099897 = 0.199897
\end{aligned}
$$

Step $t = 3$. $g_3 = 2(0.199897 - 3) = -5.600205$, $g_3^2 = 31.36230$.

$$
\begin{aligned}
m_3 &= 0.9 \times (-1.12) + 0.1 \times (-5.600205) = -1.008 - 0.560021 = -1.568021 \\
v_3 &= 0.999 \times 0.069604 + 0.001 \times 31.36230 = 0.069534 + 0.031362 = 0.100897 \\
\hat{m}_3 &= -1.568021/0.271 = -5.786054 \\
\hat{v}_3 &= 0.100897/0.002997 = 33.66589, \qquad \sqrt{33.66589} = 5.802231 \\
w_3 &= 0.199897 - 0.1 \times (-5.786054)/5.802231 = 0.199897 + 0.099721 = 0.299618
\end{aligned}
$$

`torch.optim.Adam(lr=0.1)` on the same toy gives $w = 0.100000, 0.199897, 0.299618$ [C](`research/textbook/tools/compute_24-trainloop_output.txt`).
Notice the steps: $0.1000, 0.0999, 0.0997$. Each is almost exactly η, although the gradient was -6. Plain descent with the same η took a first step of 0.6.

The first Adam step is $\eta\,\hat{m}_1/\sqrt{\hat{v}_1} = \eta\, g_1/|g_1| = \pm\eta$ for every parameter, whatever the size of its gradient: for $g_1 = 10^{-4}$, 1 or 10^4 the step is 0.1 [C]. That is Adam's strength (no parameter is starved) and the reason it needs care at the start: on step 1, every one of the 5.5 million parameters moves by the full learning rate, in a direction estimated from a single batch. Section 26.6.1 returns to this.

Source check: the optimiser actually used

`app/abstract_classes/foundation_trainer.py`, lines 68–70: `self.optimizer=torch.optim.Adam(self.model.parameters(),lr=config.learning_rate)`. Plain Adam with PyTorch's defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, `weight_decay=0`. The fine-tuning trainer builds the same optimiser over the trainable parameters only (`app/foundation_models/trainers/finetune_trainer.py`, line 254). There is no AdamW anywhere in `app/`, `scripts/` or `research/` (a grep for `AdamW` finds nothing), and no weight decay. The course (IW28) says so and is right.

26.5 Weight decay, L2 regularisation and AdamW

The project does not use weight decay, but the chapter title promises AdamW, and the distinction is one of the most common confusions in practice, so it is worth deriving.

Definition 26.6 (Weight decay)**.** *Weight decay* with coefficient λ_w shrinks every weight towards zero a little at each step: $w \leftarrow w - \eta\lambda_w w$. It discourages large weights, a form of *regularisation* (a preference for simpler models that is not part of fitting the data).

With plain gradient descent, weight decay equals an L2 penalty. Add $\frac{\lambda_w}{2}\|\mathbf{w}\|^2$ to the loss. Its gradient is $\lambda_w\mathbf{w}$, so the update becomes

$$\mathbf{w} \leftarrow \mathbf{w} - \eta(\nabla L + \lambda_w \mathbf{w}) = \mathbf{w} - \eta\nabla L - \eta\lambda_w\mathbf{w},$$

gradient descent followed by weight decay. The two ideas are the same thing here.

With Adam they are not. If you add $\lambda_w \mathbf{w}$ to the gradient ("L2 in Adam"), the decay term enters m_t and v_t and is then divided by $\sqrt{\hat{v}_t}$ like everything else. A parameter whose gradients have been large gets a large $\sqrt{\hat{v}_t}$, and therefore *less* decay; a parameter with small gradients gets *more*. AdamW [29] *decouples* the decay: it applies Adam to the loss gradient alone and subtracts $\eta\lambda_w w$ separately,

$$w_t = w_{t-1} - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda_w \, w_{t-1} \right). \tag{26.7}$$

Worked example 26.5: The same decay, two different shrinks

Two weights, both $w = 1.0$, with $\lambda_w = 0.01$, $\eta = 10^{-3}$. Weight A has had large gradients, $\sqrt{\hat{v}} = 10$; weight B small ones, $\sqrt{\hat{v}} = 0.01$. Look only at the decay part of one step (take the loss gradient to be zero at this moment, and treat the decay term as passing straight through the normaliser, the steady-state picture).
L2 in Adam: the decay $\lambda_w w = 0.01$ is divided by $\sqrt{\hat{v}}$.

$$\text{A: } 10^{-3} \times 0.01/10 = 10^{-6}$$
$$\text{B: } 10^{-3} \times 0.01/0.01 = 10^{-3}$$

AdamW: the decay is not normalised.

$$\text{A: } 10^{-3} \times 0.01 \times 1.0 = 10^{-5}$$
$$\text{B: } 10^{-3} \times 0.01 \times 1.0 = 10^{-5}$$

Under L2-in-Adam, B shrinks a thousand times faster than A, for no reason connected to the regularisation you asked for. Under AdamW both shrink equally, as the coefficient λ_w intends.

26.6 Learning-rate schedules

A fixed learning rate is a compromise: large steps cover ground fast but cannot settle into a narrow minimum; small steps settle but crawl. A *schedule* changes η over the run. Two shapes matter here.

26.6.1 Linear warmup

Definition 26.7 (Linear warmup)**.** Over the first W epochs (or steps), raise the learning rate linearly from a small $\eta_{\min}$ to the target $\eta_{\max}$:

$$\eta(t) = \eta_{\min} + (\eta_{\max} - \eta_{\min}) \, \frac{t}{W}, \qquad 0 \le t \le W.$$

With $W = 5$, $\eta_{\min} = 10^{-6}$ and $\eta_{\max} = 10^{-3}$ the five warmup epochs would use 10^{-6}, 2.008×10^{-4}, 4.006×10^{-4}, 6.004×10^{-4} and 8.002×10^{-4}, reaching 10^{-3} at $t = 5$. (Row $t = 1$: $10^{-6} + 0.000999 \times 1/5 = 0.000001 + 0.0001998 = 0.0002008$.)

Why warm up? Worked example 26.4 showed that Adam's first steps move every parameter by about η, steered by moment estimates built from one or two batches, on a network whose parameters are random. Large, poorly aimed early steps can push the network into a bad region it never recovers from. Warmup keeps those steps small until the moment estimates have

seen enough batches to be trustworthy. Goyal et al. [7] made warmup standard for large-batch training.

Source check: `warmup_epochs` is documented, defaulted, set in every config, and never read

`app/models/training/training_config.py`, lines 418–422, defines

```
warmup_epochs: int = Field(default=5, ge=0,
    description="Linear warmup from min_lr to learning_rate")
```

All 22 training configurations in `configs/` set it (both Indradhanu configurations to 5), the fine-tuning configurations set it to 0 or 1 and the SpectralEarth pretraining configurations to 0. A search of every Python file in the repository,

```
grep -rn -i "warmup" --include="*.py" .
```

returns three lines, all in `training_config.py`: the field (418), its description (421) and a docstring example (505). `_build_scheduler` (`app/abstract_classes/foundation_trainer.py`, lines 517–553) builds `cosine`, `step` or `plateau` and consults none of it. Checked 2026-10-29; the teaching skill's note is still true. Every run in this repository, including the v0.2.0 run, started at the full learning rate on step 1.
The course (IW28) links this to the v0.2.0 history: "v0.2.0 epoch 2 hits 0.035 (lr=1e-3 was too aggressive; later epochs regress to ~0.04)" [M: `research/model_break_down/05_hyperspectral_segformer_mae.md`]. An unusually good early epoch followed by a regression that never fully recovers is consistent with early instability, the thing warmup exists to prevent; it is not proof. The lesson the repository states as policy: verify a config field against the source before assuming it does anything.

26.6.2 Cosine annealing

Definition 26.8 (Cosine annealing)**.** Over T epochs, from $\eta_{\max}$ down to $\eta_{\min}$:

$$\eta(t) = \eta_{\min} + \frac{1}{2}\left(\eta_{\max} - \eta_{\min}\right)\left(1 + \cos\frac{\pi t}{T}\right), \qquad 0 \le t \le T. \tag{26.8}$$

This is the schedule of Loshchilov and Hutter [28] without restarts, and the PyTorch class `CosineAnnealingLR` implements it. Check the endpoints: at $t = 0$, $\cos 0 = 1$ and the bracket is 2, so $\eta = \eta_{\min} + (\eta_{\max} - \eta_{\min}) = \eta_{\max}$. At $t = T$, $\cos\pi = -1$, the bracket is 0, and $\eta = \eta_{\min}$.

The shape. Differentiate with respect to t (chain rule: the derivative of $\cos(\pi t/T)$ is $-\sin(\pi t/T)\cdot\pi/T$):

$$\frac{d\eta}{dt} = -\frac{\pi}{2T}\left(\eta_{\max} - \eta_{\min}\right)\sin\frac{\pi t}{T}.$$

The slope is zero at $t = 0$ and $t = T$ (where sin is 0) and steepest at $t = T/2$. So the learning rate stays high and nearly flat at the start, falls fastest in the middle, and flattens out near the floor at the end.

Worked example 26.6: The v0.2.0 schedule at five epochs

v0.2.0 uses $\eta_{\max} = 10^{-3}$, $\eta_{\min} = 10^{-6}$ and $T = 200$ [M: `configs/hyperspectral_segformer_exp_2.json`]. So $\frac{1}{2}(\eta_{\max} - \eta_{\min}) = \frac{1}{2} \times 0.000999 = 0.0004995$.

t	$\pi t/200$	cos	$1+\cos$	$\eta(t)$
0	0	1.000000	2.000000	$10^{-6}+0.0004995\times 2 = 1.000\times 10^{-3}$
50	$\pi/4$	0.707107	1.707107	$10^{-6}+0.0004995\times 1.707107 = 8.537\times 10^{-4}$
100	$\pi/2$	0.000000	1.000000	$10^{-6}+0.0004995\times 1 = 5.005\times 10^{-4}$
150	$3\pi/4$	−0.707107	0.292893	$10^{-6}+0.0004995\times 0.292893 = 1.473\times 10^{-4}$
200	π	−1.000000	0.000000	$10^{-6}+0 = 1.000\times 10^{-6}$

Row $t = 100$ in full: $0.0004995 \times 1 = 0.0004995$; $0.000001 + 0.0004995 = 0.0005005$. PyTorch's `CosineAnnealingLR`, stepped as the trainer steps it, gives the same five values to the digits shown [C].

When the schedule steps. The trainer calls `self.scheduler.step()` once, at the end of each epoch, after validation (`app/abstract_classes/foundation_trainer.py`, lines 237–243). So epoch index e trains at $\eta(e)$: index 0 at 10^{-3}, the last index, 199, at $\eta(199) = 1.06\times 10^{-6}$ [C], and $\eta(200) = 10^{-6}$ is reached only after training ends. The log line is written *after* the step, so the "lr" it prints for logged epoch k (index $k-1$) is $\eta(k)$, the rate the *next* epoch will use.

Source check: the example log line's learning rate

The course (IW28) shows a log line "Epoch 101/200 … lr: 4.94e-04". Logged epoch 101 prints $\eta(101)$: $\cos(101\pi/200) = -0.015707$, so $\eta = 10^{-6} + 0.0004995 \times 0.984293 = 10^{-6} + 0.00049165 = 4.93\times 10^{-4}$ (PyTorch: 4.9265×10^{-4} [C]). The course's 4.94e-4 is an illustrative line, off in the third digit.

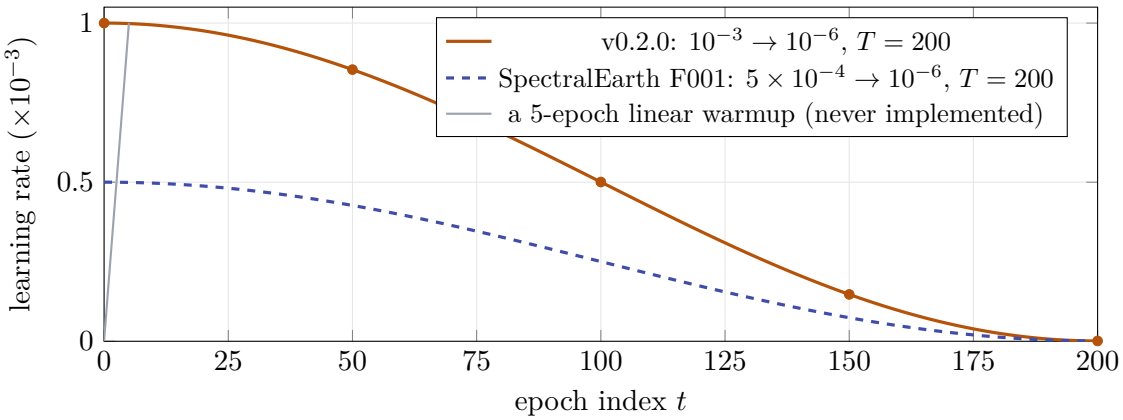


Figure 26.1: The learning-rate schedules of two real configurations, drawn to scale from Equation (26.8): v0.2.0 (`configs/hyperspectral_segformer_exp_2.json`) and the SpectralEarth final pretraining F001 (`research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`). Dots are the worked values of worked example 26.6. The grey ramp at the left is what `warmup_epochs = 5` describes and what no code implements.

Values that will drift

Learning-rate settings as of 2026-10-29: v0.2.0, 10^{-3} over 200 epochs; v0.1.0, 10^{-4} over 500; SpectralEarth pretraining F001, 5×10^{-4} over 200 [M: `research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`]; fine-tuning runs between 3×10^{-5} and 10^{-3} over 3 to 20 epochs (FTH3, the run shipped as Indradhanu V2 v2.1.0, used 3×10^{-5} over 6 epochs [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/config.json`]). All of them use `min_lr` $= 10^{-6}$ and the cosine schedule.

A resume bug, told as a lesson. When a run is resumed, the scheduler must continue from where it stopped. The original resume code rebuilt the scheduler with `last_epoch=k`, but `CosineAnnealingLR` computes each rate recursively from the optimiser's current rate, so the rebuild took one extra step: a run killed after epoch 2 continued at 10^{-6} and then 1.47×10^{-4} instead of 1.47×10^{-4} and then 10^{-6} (measured 2026-10-14, `app/abstract_classes/foundation_trainer.py`, lines 580–584). The fix saves and restores the scheduler's own state. A schedule is part of the model's history, and resuming it is as delicate as resuming the weights.

26.7 Gradient clipping

Definition 26.9 (Gradient clipping by norm)**.** If the gradient's length (L2 norm over all parameters) exceeds a threshold c, scale the whole gradient down to length c; otherwise leave it:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right).$$

Clipping by norm keeps the gradient's direction and bounds only the step's size, a guard against a single bad batch (a glitch, a NaN-adjacent value) throwing the parameters far away [34].

Worked example 26.7: Clipping a 3-vector

$\mathbf{g} = (3, 4, 12)$, threshold $c = 1$.

$$\begin{aligned} \|\mathbf{g}\|^2 &= 3^2 + 4^2 + 12^2 = 9 + 16 + 144 = 169 \\ \|\mathbf{g}\| &= \sqrt{169} = 13 \\ \min(1, 1/13) &= 1/13 = 0.076923 \\ \mathbf{g}_{\text{clipped}} &= (3/13,\ 4/13,\ 12/13) = (0.230769,\ 0.307692,\ 0.923077) \end{aligned}$$

Check the length: $0.230769^2 + 0.307692^2 + 0.923077^2 = 0.053254 + 0.094675 + 0.852071 = 1.000000$. The direction is unchanged: the ratios $3 : 4 : 12$ survive. PyTorch's `clip_grad_norm_` returns $(0.230769, 0.307692, 0.923077)$ [C] (it divides by $\|\mathbf{g}\| + 10^{-6}$, which changes the seventh digit).

Source check: no clipping

A grep for `clip_grad` across `app/`, `scripts/` and `research/` finds nothing: no trainer in this repository clips gradients. The loss's own guards (Section 25.9) and the 40% patch filter are the only protections against a wild batch. The course (IW28) says "no gradient clipping"; correct.

26.8 Mixed precision

A floating-point number stores a sign, an exponent (the range) and a mantissa (the precision). Three formats matter:

format	exponent bits	mantissa bits	relative spacing	range
float32	8	23	$2^{-23} \approx 1.2 \times 10^{-7}$	$\sim 10^{\pm 38}$
bfloat16	8	7	$2^{-7} \approx 0.0078$	$\sim 10^{\pm 38}$
float16	5	10	$2^{-10} \approx 0.00098$	$\sim 6.5 \times 10^4$ max

Mixed precision runs the expensive matrix multiplications in a 16-bit format (half the memory, much faster on modern GPUs) and keeps precision-critical parts in float32 [31]. bfloat16 keeps float32's exponent, so it has the same range and never underflows where float32 would not; float16 needs *loss scaling* to keep small gradients from flushing to zero, bfloat16 does not.

Worked example 26.8: Why the loss stays in float32

A reflectance near 0.2 lies in $[0.125, 0.25) = [2^{-3}, 2^{-2})$, where bfloat16's spacing is $2^{-3} \times 2^{-7} = 2^{-10} = 0.000977$. So

$$0.2000 \to 0.2001953 \text{ in bfloat16}$$
$$0.2003 \to 0.2001953$$
$$0.2005 \to 0.2001953$$

(PyTorch's conversions [C]). A reconstruction error of 0.0003 or 0.0005, the size a good model's L1 is made of (Chapter 25), vanishes completely. In float32 the three values stay distinct.

The SpectralEarth and fine-tuning trainers wrap only the model's forward pass in bfloat16 autocast and convert its output back before the loss:

```
bf16 = self.config.model_config_.amp_bf16 and self.device.type == "cuda"
with torch.autocast("cuda", dtype=torch.bfloat16, enabled=bf16):
    out = model(pixels, wl, band_valid, keep_mask, out_wl)
return out.float()          # the loss - small differences of reflectance - in full
    precision
```

(`app/foundation_models/trainers/spectral_stem_mae_trainer.py`, lines 269–275). Every SpectralEarth configuration except the laptop smoke tests, and every fine-tuning configuration, sets `amp_bf16:true`.

[M:`research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`]

The configuration field's description says it roughly halves the cross-attention stem's activation memory. Be precise about what the cast buys. The prediction is still computed by bfloat16 arithmetic inside the network, so it carries bfloat16-sized rounding in the network's own (normalised) units; what `out.float()` guarantees is that the truth $\mathbf{x}$ is never rounded and that the subtraction, the absolute values, the angle and the averaging over millions of entries are done in float32. There is no gradient scaler (`GradScaler`) anywhere, which is correct for bfloat16. The original v0.2.0 trainer (`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`) has no autocast: it trained entirely in float32.

26.9 Gradient accumulation

When the batch you want does not fit in GPU memory, process K smaller batches and step once:

```
scaled_loss = loss / accum_steps          # divide so the sum of K gradients is their
    mean
scaled_loss.backward()                    # .grad += d(scaled_loss)/dw
if accum_count % accum_steps == 0:
    self.optimizer.step()
    self.optimizer.zero_grad()
```

(`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, lines 100–148.) Because `backward()` adds into `.grad`, after K calls the field holds $\sum_{k=1}^{K} \nabla(L_k/K) =$

$\frac{1}{K}\sum_k \nabla L_k$, the mean of the K batch gradients. If each batch has B patches and L_k is the mean over them, this is

$$\frac{1}{K}\sum_{k=1}^{K}\frac{1}{B}\sum_{i\in\text{batch }k}\nabla\ell_i = \frac{1}{KB}\sum_{\text{all }KB}\nabla\ell_i,$$

exactly the gradient of one batch of KB patches. Without the division by K the gradient would be K times larger; with Adam that barely matters (it divides by the gradient's size), but with plain descent it would multiply the learning rate by K. If the loader runs out mid-group, the leftover gradients are applied ("the flush", lines 143–146), not discarded.

Two cautions. The equality needs equal batch sizes; after the 40% filter, batches carry different numbers of surviving patches, so the accumulated gradient weights each batch equally rather than each patch. And accumulation does not reproduce a large batch's BatchNorm statistics, which are still computed per small batch (Chapter 29).

Values that will drift

v0.2.0 set `gradient_accumulation_steps:1`, so the override returns straight to the base loop. The SpectralEarth pretraining F001 uses `batch_size` 8 and accumulation 4, an effective batch of $8 \times 4 = 32$ and at most $30{,}000/32 = 937.5$, so 938, optimiser steps per epoch [M: `research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`].

26.10 The training data

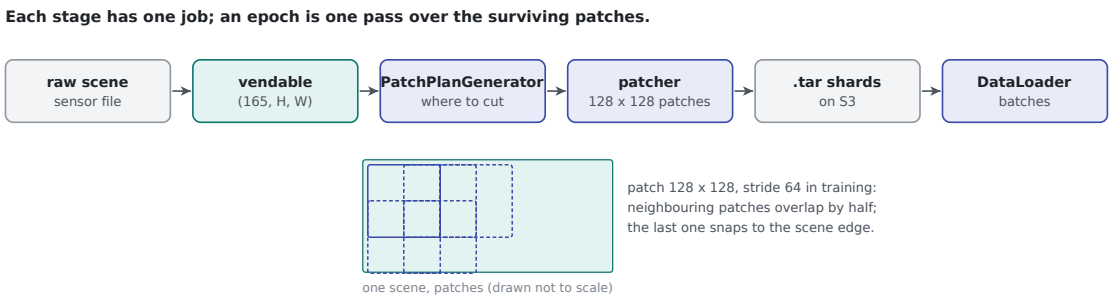


Figure 26.2: The training-data pipeline, one job per stage. A raw scene becomes a 165-band vendable; PatchPlanGenerator decides where to cut 128 × 128 patches (stride 64 in training, so neighbours overlap by half, the last snapping to the scene edge); the patcher cuts them; they are packed into .tar shards on S3 and streamed by the DataLoader as batches.

26.10.1 Deciding where to cut

`PatchPlanGenerator` (`app/utils/patch_generation/generate_patch_plan.py`) returns coordinates only; no pixels are touched. Given the cube's height and width, a patch size and a *stride* (how far to move between patches), it lists every top-left corner. The training convention is stride = size/2, so 128-pixel patches step by 64 and neighbours overlap by half: every interior pixel appears in several patches, sometimes centred, sometimes near an edge, which evens out the poorer reconstruction near patch borders.

When the next step would run off the edge, the last patch is snapped back to end flush with the boundary (lines 66–79):

```
if row + request.height > cube_height:
    row_coords.append(cube_height - request.height)   # last patch ends exactly at
    the edge
```

```
    break
```

Worked example 26.9: The row coordinates of a 300-pixel scene

Height 300, size 128, stride 64. Walk the loop.

$$\begin{aligned} \text{row } 0: &\quad 0 + 128 = 128 \le 300, \text{ keep } 0 \\ \text{row } 64: &\quad 64 + 128 = 192 \le 300, \text{ keep } 64 \\ \text{row } 128: &\quad 128 + 128 = 256 \le 300, \text{ keep } 128 \\ \text{row } 192: &\quad 192 + 128 = 320 > 300, \text{ snap to } 300 - 128 = 172, \text{ stop} \end{aligned}$$

Rows $[0, 64, 128, 172]$: four, and the last gap is $172 - 128 = 44$, not 64. The patch count is not $\lceil 300/64 \rceil = 5$; the module's docstring warns about exactly this. The real generator returns $[0, 64, 128, 172]$ [C].

Source check: a duplicated last patch

Run the same loop on a height of 256: rows 0, 64, 128 are kept ($128 + 128 = 256 \le 256$); at row 192, $192 + 128 = 320 > 256$, so it snaps to $256 - 128 = 128$, a coordinate it already has. The real generator returns rows $[0, 64, 128, 128]$, and for a dimension of exactly 128 it returns $[0, 0]$ [C] (`research/textbook/tools/compute_24-trainloop_output.txt`). The duplicate appears whenever (extent − size) is a multiple of the stride, because the loop continues while `row<cube_height` rather than stopping once a patch has reached the edge. The patcher (`app/utils/patch_generation/hyperspectral_patcher.py`, line 96) iterates the plan without removing duplicates, so such a scene emits the same patch twice under the same key. That is consistent with the "duplicate file name in tar file" webdataset warning that the hyperspectral trainer suppresses (`app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, line 21, whose comment attributes it to resampled shards); whether the shards actually contain such duplicates was not checked here. The effect on training is a slight over-weighting of edge patches; at inference, an overlapping patch predicted twice is harmless if overlaps are averaged.

26.10.2 Doing the cutting, and the shard contract

For each coordinate the patcher emits one dictionary, written as named members of a tar file:

```
{"__key__":            f"{scene_id}#row_coord:{r}#col_coord:{c}",
 "meta.json":          {... "patch_height": 128, "patch_stride": 64, "band_count": 165
     ...},
 "pixels.npy":         cube[:, r:r+h, c:c+w].astype(np.float32),      # (165, 128,
    128)
 "validity_cube.npy": validity[:, r:r+h, c:c+w].astype(np.int8),     # (165, 128,
    128) 0/1
 "wavelengths.npy":    wavelengths}                                   # (165,)
```

The key names are a contract: the trainer reads `batch["pixels.npy"]` and `batch["validity_cube.npy"]` literally. The mask is int8 because it only ever holds 0 or 1, a quarter of float32's size.

Definition 26.10 (Shard)**.** A *shard* is a `.tar` archive packing many patches, read sequentially. Millions of tiny files would each cost a network request; a few hundred large shards stream efficiently. This is the *webdataset* convention, which is why the keys look like file names.

The S3 location is a template,

```
patches/{provider}/{split}/{stage}/w{size}_h{size}_s{stride}/
```

filled for Indradhanu as `patches/hyperspectral/train/final/w128_h128_s64/` in bucket `allotrope-raw-data-india` [M: `configs/hyperspectral_segformer_exp_2.json`]. The provider is "hyperspectral", not "prisma" or "enmap": PRISMA and EnMAP patches share the same shards, interleaved, because on the 165-band common grid (Chapter 3) they are indistinguishable at the tensor level. The schema's default template points at Landsat thermal data, and the source says so (`app/models/training/training_config.py`, line 296):

```
# Important to set this properly The default is landsat which is dangerous.
```

Hot storage. Shards are copied once to the training machine's local disk and kept (`hot_storage`: 500 training and 100 test shards for v0.2.0, `skip_sync_if_exists:true`). There is no rotation in the base trainer: those 500 shards are the training set for the whole run, and variety across epochs comes from `shardshuffle` (30 in v0.2.0), which reorders shards and samples within a buffer.

26.10.3 What an epoch means here

The corpus is too large to pass over even once in reasonable time, so an epoch is a sample budget:

```
"train_samples_per_epoch": { "128": 40000 },
"test_samples_per_epoch":  { "128":  5000 }
```

"Consume 40,000 training patches of size 128, then 5,000 validation patches, and call that an epoch." The dictionary is keyed by patch size because the thermal models train on several sizes at once; Indradhanu uses only 128. Only patches that survive the 40% filter (Section 25.4) count:

```
for batch in loader:
    if valid_samples >= sample_cap:      # 40,000
        break
    loss, num_kept = self.compute_loss(batch, self.model)
    if num_kept == 0:
        continue                         # whole batch filtered out: no step, no
    budget used
    ...
    valid_samples += num_kept
```

With $B = 128$ and no filtering, $40{,}000/128 = 312.5$, so the loop runs 313 batches and stops at $313 \times 128 = 40{,}064$ samples [C]: the cap is checked before each batch, so the last batch overshoots by up to $B - 1$. Over 200 epochs that is about $40{,}000 \times 200 = 8{,}000{,}000$ patch presentations [C], drawn from the same 500 cached shards.

Feeding it is an I/O problem. One patch is $165 \times 128 \times 128$ float32 values,

$$165 \times 128 = 21{,}120$$
$$21{,}120 \times 128 = 2{,}703{,}360 \text{ values}$$
$$2{,}703{,}360 \times 4 \text{ bytes} = 10{,}813{,}440 \text{ bytes} \approx 10.8 \text{ MB},$$

and a batch of 128 is 1,384,120,320 bytes, about 1.4 GB, before any activation [C]. That is why v0.2.0 ran 40 loader workers (`num_workers`). If the data pipeline stalls, an expensive GPU sits idle.

26.10.4 Randomness and augmentation

The v0.2.0 trainer applies no geometric augmentation: no flips, rotations or crops (a search of the trainers for `flip` and `rot90` finds none). The variety each epoch comes from three random sources: the shard and sample order (`shardshuffle`); the token mask, redrawn for every batch so that a different 65% of each patch's valid tokens is hidden each time it is seen (Chapter 18); and, in validation, the random half-split of Chapter 25.

The later trainers add two augmentations, each taught where it belongs:

- **Band-list augmentation** (SpectralEarth pretraining and fine-tuning; `sample_band_subset` in `app/foundation_models/trainers/spectral_stem_mae_trainer.py`, lines 75–92). With probability `band_augment_prob` (0.8 in F001) a batch's input is a random subset of its bands, in one of three shapes: thinning (keep 50–95%), one to three gaps of 20–150 nm, or the bands nearest an even grid of 6–15 nm spacing, never fewer than 40 bands; the stated wavelengths are jittered by up to ±`band_jitter_nm` (3 nm in F001), and the loss still covers every band. It teaches a wavelength-aware stem to cope with sensors it was not trained on (Chapters 23 and 27).
- **Implant augmentation** (fine-tuning run FT05): synthetic objects implanted in the input, the clean scene as the target (Chapter 38).

The SpectralEarth shards also change the data contract: `pixels.npy` is int16, reflectance ×10,000, with −32768 marking a dead voxel, and validity is per band rather than per pixel (Chapter 27).

26.11 The loop

26.11.1 From the command line to the epoch loop

```
python scripts/train_foundation_model.py configs/hyperspectral_segformer_exp_2.json
```

The script validates the JSON into a pydantic `TrainingConfig` (a typo fails at once, not forty minutes in), looks up the trainer class by model name in `app/foundation_models/trainers/trainer_factory.py`, and calls `trainer.train()`.

`FoundationTrainer` (`app/abstract_classes/foundation_trainer.py`) owns everything generic: device, optimiser, scheduler, data loaders, hot storage, the epoch loop, checkpoints and logging. A model's trainer implements three methods, `build_model`, `compute_loss` (returns the loss and the number of surviving samples) and `validation_step`; Indradhanu's also overrides `_run_epoch` (to record the epoch index for the SAM ramp and log the loss components) and `_run_train_pass` (to add gradient accumulation).

26.11.2 What one epoch does

```
def _run_epoch(self, epoch, train_loaders, test_loaders):
    self.model.train()
    for size in patch_sizes:                                    # Indradhanu: only 128
        size_loss, size_samples = self._run_train_pass(train_loaders[size], cap)
    avg_train_loss = epoch_train_loss / max(epoch_train_samples, 1)
    self.model.eval()
    for size in patch_sizes:
        val_losses[size] = self._run_val_pass(test_loaders[size], cap)   # under
    torch.no_grad()
    avg_val_loss = sum(val_losses.values()) / len(val_losses)
```

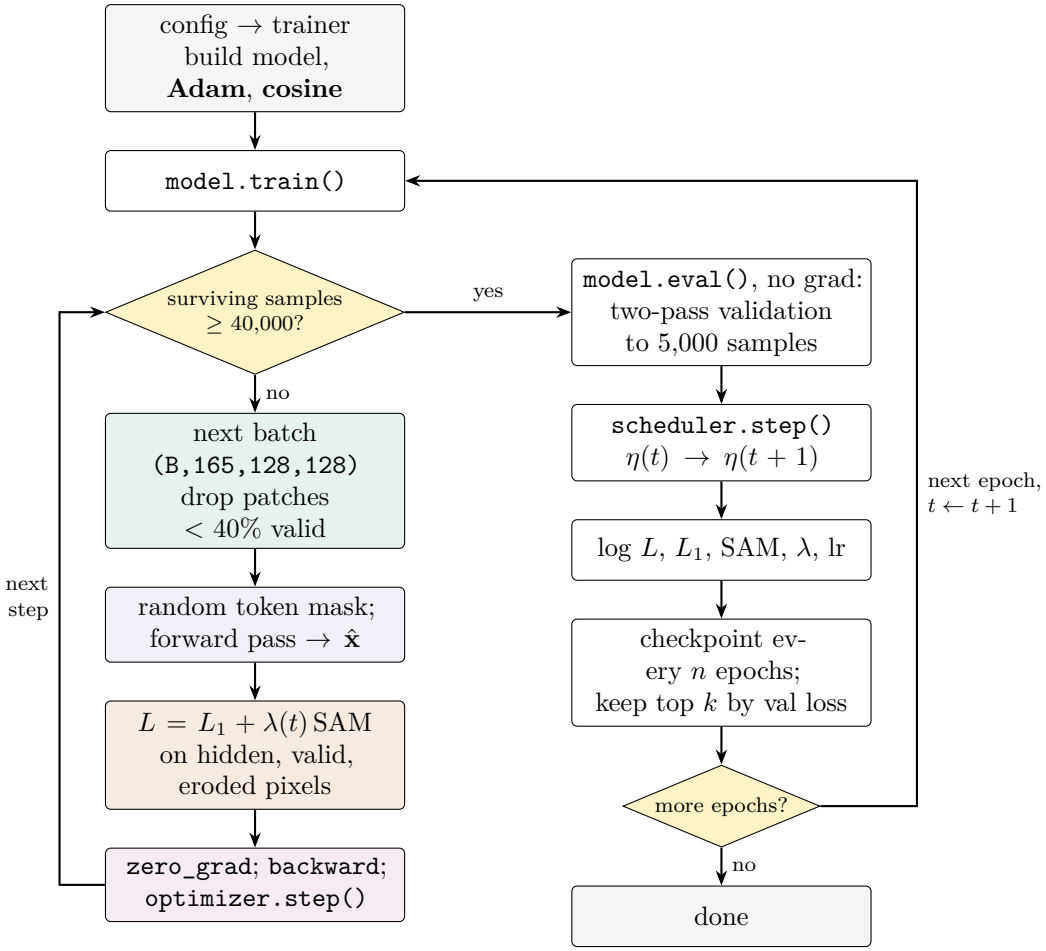


Figure 26.3: The training loop of `FoundationTrainer` with Indradhanu's loss, as the source runs it. The left column is one training step, repeated until the epoch's sample budget is spent; the right column runs once per epoch: validation, the scheduler step, logging and the periodic checkpoint. The SAM weight $\lambda(t)$ and the learning rate $\eta(t)$ both change only at the epoch boundary.

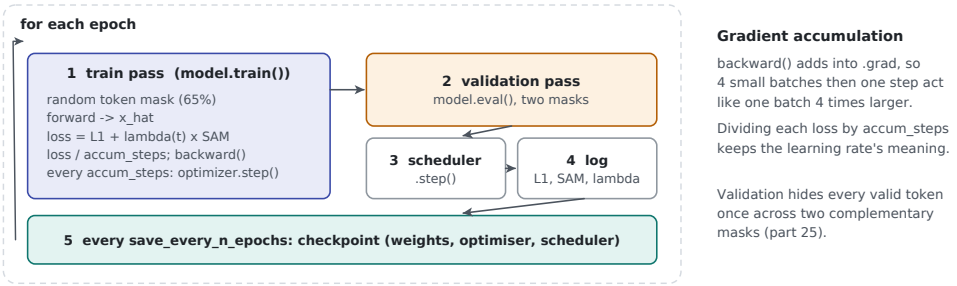


Figure 26.4: The course's drawing of one epoch: a training pass with random masking and gradient accumulation, a validation pass, a scheduler step, logging, and a periodic checkpoint. The dashed frame repeats once per epoch.

```
    self.scheduler.step()
    ...log...
    if (epoch + 1) % ckpt.save_every_n_epochs == 0:
        self._save_checkpoint(epoch + 1, avg_train_loss, val_losses)
        self._cleanup_checkpoints()
```

Five phases: train, validate, step the scheduler, log, maybe checkpoint (`app/abstract_classes/foundation_trainer.py`, lines 202–274).

`model.train()` and `model.eval()` do not train or evaluate anything. They flip a flag that changes two kinds of layer. Dropout zeroes random activations in train mode and does nothing in eval mode; BatchNorm uses the current batch's statistics and updates its running averages in train mode, and uses the stored averages in eval mode. Indradhanu has both (`drop_rate` 0.4 and a BatchNorm in the compressor), so forgetting `eval()` before validation would make validation noisy and slightly optimistic without any error. Do not confuse `model.train()` (a mode flag on the network) with `trainer.train()` (the method that runs the whole loop).

The training step (lines 276–307) is the move of the opening box: `optimizer.zero_grad()`, `compute_loss`, `loss.backward()`, `optimizer.step()`.

26.11.3 Sample-weighted averaging

`compute_loss` returns a per-sample mean, so the pass multiplies it back by the number of surviving samples before summing:

```
total_loss += loss.item() * num_kept
valid_samples += num_kept
...
return total_loss / max(valid_samples, 1)        # max(.., 1): an all-filtered epoch
    gives 0, not a crash
```

Worked example 26.10: Why the averaging is weighted

Two batches: in the first only 3 patches survived the filter, with mean loss 0.05; in the second all 128 survived, with mean loss 0.03.
Weighted (what the code does):

$$0.05 \times 3 = 0.15$$
$$0.03 \times 128 = 3.84$$
$$(0.15 + 3.84)/(3 + 128) = 3.99/131 = 0.030458$$

Unweighted (averaging the two batch means): $(0.05 + 0.03)/2 = 0.04$.
The unweighted answer lets 3 patches count as much as 128. The code carries a comment explaining the multiplication, because it is the kind of line a reader might "simplify" away.

26.11.4 Checkpoints and resuming

Each checkpoint stores the model's parameters and buffers, the optimiser's state (Adam's m and v for every parameter, needed to resume cleanly), the scheduler's state, the epoch, the training and per-size validation losses, and the whole configuration. Files are named `{model_name}_v{version}_epoch{epoch}.pt`, which is why the shipped v0.2.0 file is

`hyperspectral_segformer_mae_v0.2.0_epoch200.pt`; `keep_top_k` (20 in v0.2.0) prunes all but the best by validation loss.

`resume_mode`	restores	use it when
`"resume"`	weights, optimiser state, epoch counter, scheduler state	a run crashed or was stopped
`"finetune"`	weights only; fresh optimiser, scheduler, epoch 0	adapting a trained model to new data

Resuming a 200-epoch cosine run at epoch 150 with a fresh scheduler would restart the learning rate at 10^{-3} and undo the decay; `"resume"` exists to prevent that. The SpectralEarth trainer adds `last.pt` and `best.pt`, mirrored to S3, automatic resumption, and early stopping on the validation score (Chapter 27).

Source check: which number ranks the checkpoints

`keep_top_k` and `best.pt` rank epochs by validation loss. In the base Indradhanu trainer that loss uses the ramped SAM weight (Chapter 25), so epochs inside the ramp are on a different scale from later ones. The SpectralEarth trainer found this the hard way (an early-stop rule that would have fired whatever the model did) and now scores validation at the full weight from the first epoch; Chapter 27 tells that story.

26.11.5 Reading a log line

```
Epoch 101/200 | train_loss: 0.041203 | val_loss: [128px: 0.043490] | avg_val:
    0.043490 | lr: 4.93e-04
  L1: 0.009812 | SAM: 0.062776 rad (3.60 deg) | lambda: 0.500
```

(An illustrative line in the format of lines 245–256 of the base trainer and 65–68 of Indradhanu's, with the learning rate corrected as in the source check above.) `train_loss` and `val_loss` are the combined $L_1 + \lambda\,\mathrm{SAM}$; `lr` is the rate the next epoch will use; the second line gives the components. Watch the second line, for the reasons in Chapter 25.

The loop is one move repeated: batch, loss, gradient, Adam step; the epoch boundary is where the learning rate, the SAM weight, validation and checkpoints change, and an epoch here is a sample budget, not a pass over the data.

26.12 Common confusions

Does model.train() do the training?
No. It sets a mode flag on the network (dropout on, BatchNorm in batch-statistics mode). `trainer.train()`, a different method on a different object, runs the loop.

Is an epoch one pass over the training data?
Not here. It is a budget of 40,000 surviving training patches and 5,000 validation patches (v0.2.0), drawn from 500 cached shards. Coverage is statistical, not exhaustive.

Does warmup_epochs = 5 mean the run warmed up?
No. The field is never read; every run started at the full learning rate (Section 26.6.1).

Is Adam with weight decay the same as AdamW?
No. Adding an L2 term to the gradient sends the decay through Adam's normaliser, so weights with large gradients decay less; AdamW subtracts the decay separately (worked example 26.5). This project uses neither: weight decay is 0.

Why is the learning rate in the log not the one the epoch trained at?
The scheduler steps before the log line is written, so the printed rate is the next epoch's.

Can I change the learning rate mid-run?
Edit the config and resume with `"finetune"`, but you lose the optimiser's moments and the scheduler's position. Consider whether that is what you want.

Why is the validation loss averaged over patch sizes?
Because the thermal models train on several sizes. Indradhanu has one, so the average is over one value.

26.13 Check yourself

Exercise 26.1. By hand: run three steps of gradient descent on $L(w) = (w - 3)^2$ from $w_0 = 0$ with $\eta = 0.25$. What factor multiplies the distance to 3 at each step? For which η does one step land exactly on 3?

Exercise 26.2. By hand: a parameter's gradient is the constant $g = 2$ at every step. Compute Adam's m_t, v_t, $\hat{m}_t$, $\hat{v}_t$ for $t = 1, 2$ with the default βs, and the size of the step with $\eta = 10^{-3}$. What would the step have been at $t = 1$ without bias correction?

Exercise 26.3. By hand: compute the v0.2.0 learning rate at epoch indices 25 and 175 ($\cos(\pi/8) = 0.923880$). Which of the two epochs is the rate changing faster in, and why?

Exercise 26.4. Clip $\mathbf{g} = (1, 2, 2)$ to norm 1.5, and then to norm 5. Then explain in two sentences why the repository's training would or would not be changed by adding gradient clipping at norm 1.0, given what Adam does to a gradient's scale.

Exercise 26.5. List the row and column patch coordinates that `PatchPlanGenerator` produces for a 400-row, 300-column scene with size 128 and stride 64, and the number of patches. Then give a height for which it emits a duplicate row.

Exercise 26.6. Explain why a reconstruction error of 0.0004 at reflectance 0.2 cannot be measured in bfloat16, and what the SpectralEarth trainer does about it.

Next: Chapter 27, Pretraining on SpectralEarth.

Chapter 27

Pretraining on SpectralEarth

The one thing this chapter teaches

A long pretraining run is only as good as three quiet decisions: the number that ranks its epochs must be measured on the same scale every epoch; a stem that *can* read any band list must be *trained* on many band lists, or it reads only the one it saw; and data that does not fit on the disk is walked through in blocks on one unbroken learning-rate schedule. This chapter derives each decision from numbers, then follows the pretrained model onto the three sensors it will be fine-tuned on.

27.1 What this chapter is about

Chapter 26 built a training loop: batches of patches go through the model, a loss is computed, the gradient of the loss nudges every parameter, and after each *epoch* (one pass over the epoch's training patches) the loop measures a *validation loss* on patches it never trains on. Chapter 25 built the loss itself, $L1 + \lambda(t)$ SAM, with the weight λ of the spectral angle [22] ramping up from 0 over the first epochs. Chapter 24 built the reduced-attention spectral stem, which reads a pixel's bands as tokens that carry their own wavelengths, so that in principle it can read any sensor's band list.

This chapter takes that model and trains it for real, on *SpectralEarth*: about half a million hyperspectral patches that the German space agency DLR cut from EnMAP scenes and published for exactly this purpose [2]. Three things went wrong, or nearly wrong, on the way, and each teaches a piece of mathematics:

1. **The ruler moved.** The validation loss was computed with the training loss's ramped weight. A score whose formula changes from epoch to epoch cannot rank epochs (Section 27.3).
2. **One band list is not any band list.** A stem trained on SpectralEarth's single list of 202 bands read PRISMA's own 185 bands badly, worst over water. The cause is an average that gives every band one vote, and a set of small wavelength networks that were only ever evaluated at one list (Section 27.4).
3. **The data did not fit.** The final run, F001, needed ten times more patches than the disk could hold. It walked through them in blocks of 30,000 patches, with arithmetic that decides how much it sees, how much it stores and when it rotates (Section 27.5).

The chapter ends with the three real sensors that fine-tuning (Chapter 29) will read at their own bands: EnMAP, PRISMA and AVIRIS-NG, each with quirks that one rule handles (Section 27.6).

27.2 SpectralEarth in numbers

Before training anything, count what you have. The data layer lives in `app/utils/spectralearth/` and is documented in `docs/lld/spectralearth.md`.

Patches. SpectralEarth holds 539,755 [M: `docs/lld/spectralearth.md`] patches. Each patch is a small square cut from a scene: 128×128 pixels, each pixel a spectrum of 202 bands. The values are stored as 16-bit integers (*int16*, 2 bytes each): reflectance multiplied by 10,000, with -32768 meaning "no data". So reflectance 0.1934 is stored as 1934.

Why 202 bands. EnMAP records 224 bands. SpectralEarth drops two runs of them, the bands where water vapour in the air absorbs almost all sunlight so that the ground is invisible: bands 127 to 141 and 161 to 167 (1-based). [M: `docs/lld/spectralearth.md`] Count each run. A run from a to b inclusive has $b - a + 1$ members:

$$\begin{aligned} \text{run 1:} \quad & 141 - 127 + 1 = 15 \text{ bands} \\ \text{run 2:} \quad & 167 - 161 + 1 = 7 \text{ bands} \\ \text{kept:} \quad & 224 - 15 - 7 = 202 \text{ bands.} \end{aligned}$$

The data layer derives this list from a table and refuses anything else; the LLD's first invariant is "never hard-code a SpectralEarth wavelength; read the table".

Worked example 27.1: The size of one patch

Do this on paper. A patch is 202 bands of 128×128 pixels at 2 bytes per number.

$$\begin{aligned} \text{pixels per band:} \quad & 128 \times 128 = 16{,}384 \\ \text{numbers per patch:} \quad & 202 \times 16{,}384 = 3{,}309{,}568 \\ \text{bytes per patch:} \quad & 3{,}309{,}568 \times 2 = 6{,}619{,}136 \text{ bytes} \\ \text{in megabytes } (10^6): \quad & 6{,}619{,}136/1{,}000{,}000 \approx 6.62 \text{ MB.} \end{aligned}$$

A *shard* is a file that bundles 150 patches (a tar archive, read in order). One shard is therefore

$$150 \times 6{,}619{,}136 = 992{,}870{,}400 \text{ bytes} \approx 0.99 \text{ GB},$$

which matches the LLD's "150-patch shards (~0.99 GB)". [M: `docs/lld/spectralearth.md`]

The split. Patches from one place on two dates look alike. If one date were in training and the other in validation, the validation score would flatter the model. So the split is *by site*: a site goes to the test side when a hash of its name, read as a number between 0 and 1, falls below the test fraction f. Every date of a site lands on the same side, and the same seed gives the same split on any machine. The `se-v1` split at $f = 0.05$ gave 512,849 training and 26,906 test patches. [M: `docs/lld/spectralearth.md`] Check that it adds up and that the test share is close to 5%:

$$\begin{aligned} 512{,}849 + 26{,}906 &= 539{,}755 \quad \text{(every patch, once)} \\ 26{,}906/539{,}755 &= 0.0498 \approx 5\%. \end{aligned}$$

The share is not exactly 5% because sites hold different numbers of patches; the hash chooses sites, not patches.

The normalisation curve. The sharder also writes `stats.json`: for every one of the 202 wavelengths, the mean and standard deviation of valid reflectance over the training split. This is the *normalisation curve* of Chapter 5: inside the model each band value v at wavelength λ becomes $(v - \mu(\lambda))/\sigma(\lambda)$, "standard deviations from that band's mean". Keep this curve in mind; Chapter 29 shows why it must never be recomputed after pretraining.

27.3 A ruler that does not move

27.3.1 Two jobs for one number

During pretraining the validation loss has a second job besides reporting progress. It decides two things:

- **`best.pt`**: which epoch's weights are kept as "the best". A *checkpoint* is a file holding the model's weights (and the optimiser's state) at the end of an epoch; `best.pt` is the one with the lowest validation score so far.
- **Early stopping**: whether to stop the run before its last epoch.

Definition 27.1 (Early stopping)**.** Keep the best validation score seen so far, S^*, and a counter of *bad epochs*. After each epoch with score S_e: if $S_e < S^*(1 - \delta)$, the epoch is an improvement, so set $S^* \leftarrow S_e$ and reset the counter to 0; otherwise add 1 to the counter. Stop when the counter reaches the *patience* P, but never before a minimum epoch $E_{\min}$. The number δ is the *minimum relative improvement*.

The rule in `app/foundation_models/trainers/spectral_stem_mae_trainer.py` is exactly this:

```
if self._best_val == math.inf or avg_val < self._best_val * (1 - cfg.
    early_stop_min_delta):
    self._best_val, self._bad_epochs = avg_val, 0   # improvement: new best, count
    reset
else:
    self._bad_epochs += 1                           # not better by delta: one more
    bad epoch
self._stop = bool(cfg.early_stop_patience) and epoch >= cfg.early_stop_min_epochs \
    and self._bad_epochs >= cfg.early_stop_patience
```

The experiment configs (`research/experiments/indradhanu-se-pretrain/E00*/config.json`) set $\delta = 0.01$, $P = 10$, $E_{\min} = 20$, a final SAM weight of 0.5 and a ramp of 10 epochs. [M: `research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`] With $\delta = 0.01$ a score counts as an improvement only if it is below $0.99\,S^*$: one percent better than the best.

27.3.2 The trap, as algebra

The training loss of Chapter 25 is

$$\mathcal{L}_e = L1_e + \lambda_e \,\mathrm{SAM}_e, \qquad \lambda_e = \lambda_{\max} \min\!\left(1, \frac{e-1}{R}\right), \tag{27.1}$$

where $e = 1, 2, \ldots$ is the epoch as a person counts it, $\lambda_{\max} = 0.5$ and $R = 10$. The trainer counts epochs from 0 internally (`_get_sam_weight(epoch)` returns $\lambda_{\max} \min(1, \texttt{epoch}/R)$ with

`epoch` $= e - 1$), which is why $e - 1$ appears. Work four values:

$$\begin{aligned} e = 1: &\quad 0.5 \times \min(1, 0/10) = 0.5 \times 0.0 = 0.00 \\ e = 3: &\quad 0.5 \times \min(1, 2/10) = 0.5 \times 0.2 = 0.10 \\ e = 6: &\quad 0.5 \times \min(1, 5/10) = 0.5 \times 0.5 = 0.25 \\ e = 11: &\quad 0.5 \times \min(1, 10/10) = 0.5 \times 1.0 = 0.50. \end{aligned}$$

The first validation code scored epochs with the same formula, ramp included. Ask when that score can rise from one epoch to the next even though both measurements fall. Write the change in score:

$$S_{e+1} - S_e = (L1_{e+1} + \lambda_{e+1}\,\mathrm{SAM}_{e+1}) - (L1_e + \lambda_e\,\mathrm{SAM}_e).$$

Add and subtract $\lambda_e\,\mathrm{SAM}_{e+1}$, which changes nothing, so that each difference appears on its own:

$$= (L1_{e+1} - L1_e) + \lambda_e(\mathrm{SAM}_{e+1} - \mathrm{SAM}_e) + (\lambda_{e+1} - \lambda_e)\,\mathrm{SAM}_{e+1}\,.$$

Name the three differences $\Delta L1$, $\Delta\,\mathrm{SAM}$ and $\Delta\lambda$:

$$S_{e+1} - S_e = \underbrace{\Delta L1}_{\leq 0 \text{ if L1 improves}} + \underbrace{\lambda_e\,\Delta\,\mathrm{SAM}}_{\leq 0 \text{ if SAM improves}} + \underbrace{\Delta\lambda\;\mathrm{SAM}_{e+1}}_{>0 \text{ while the ramp rises}}\,.$$

While λ is still rising, the third term is positive and grows with the size of SAM; if it outweighs the first two, a model that improved on *both* measurements scores *worse*.

SAM is measured in radians and is several times larger than L1 in reflectance (a SAM of 0.2 rad against an L1 of 0.05), so the third term easily wins.

Worked example 27.2: The same model scored two ways

These values are illustrative [A], chosen to match what the live run E003 showed: a first-epoch score of about 0.055 and every later epoch about 0.1. L1 and SAM both improve every epoch.

epoch e	L1	SAM (rad)	λ_e
1	0.055	0.20	0.00
3	0.045	0.17	0.10
6	0.040	0.15	0.25
11	0.034	0.13	0.50

Scored with the ramp, $S_e = L1_e + \lambda_e\,\mathrm{SAM}_e$:

$$\begin{aligned} e = 1: &\quad 0.055 + 0.00 \times 0.20 = 0.055 + 0.000 = 0.055 \\ e = 3: &\quad 0.045 + 0.10 \times 0.17 = 0.045 + 0.017 = 0.062 \\ e = 6: &\quad 0.040 + 0.25 \times 0.15 = 0.040 + 0.0375 = 0.0775 \\ e = 11: &\quad 0.034 + 0.50 \times 0.13 = 0.034 + 0.065 = 0.099. \end{aligned}$$

Scored at the full weight, $S_e = L1_e + 0.5\;\mathrm{SAM}_e$ every epoch:

$$\begin{aligned} e = 1: &\quad 0.055 + 0.5 \times 0.20 = 0.055 + 0.100 = 0.155 \\ e = 3: &\quad 0.045 + 0.5 \times 0.17 = 0.045 + 0.085 = 0.130 \\ e = 6: &\quad 0.040 + 0.5 \times 0.15 = 0.040 + 0.075 = 0.115 \\ e = 11: &\quad 0.034 + 0.5 \times 0.13 = 0.034 + 0.065 = 0.099. \end{aligned}$$

Now check the decomposition on the step from epoch 1 to epoch 3 with the ramp. The three terms are

$$\begin{aligned} \Delta L1 &= 0.045 - 0.055 = -0.010 \\ \lambda_1 \, \Delta \,\mathrm{SAM} &= 0.00 \times (0.17 - 0.20) = 0.00 \times (-0.03) = 0.000 \\ \Delta\lambda \; \mathrm{SAM}_3 &= (0.10 - 0.00) \times 0.17 = 0.017 \\ \mathrm{sum} &= -0.010 + 0.000 + 0.017 = +0.007, \end{aligned}$$

and indeed $0.062 - 0.055 = 0.007$. The ramp term ($+0.017$) is larger than the whole improvement in L1 (-0.010), so the score rises. The model is identical in both columns; only the ruler differs.

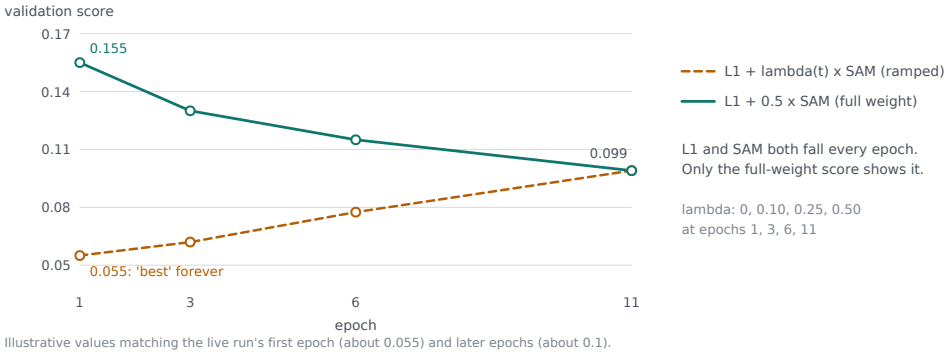


Figure 27.1: The four epochs of worked example 27.2 scored two ways. With the training ramp (amber, dashed) the validation score climbs from 0.055 to 0.099 although L1 and SAM both fall; epoch 1 stays "best" for ever. At the full SAM weight (teal) the score falls from 0.155 to 0.099. The two meet at epoch 11, where the ramp ends.

27.3.3 What early stopping would have done

Run the rule on the ramped scores. Epoch 1 sets $S^* = 0.055$, so a later epoch must score below

$$0.055 \times (1 - 0.01) = 0.055 \times 0.99 = 0.05445.$$

Every later epoch scores near 0.1, so none qualifies:

epoch	below 0.05445?	bad epochs, and the stop test
2	no	1
3	no	2
⋮		
11	no	10: patience reached, but $11 < 20$, so no stop
⋮		
20	no	19: $20 \geq 20$ and $19 \geq 10$, **stop**

The run would have stopped at epoch 20, the first epoch the rule allows, and kept epoch 1's weights as `best.pt`, whatever the model had learned. The minimum of 20 epochs delayed the damage but did not prevent it: the comparison itself was broken.

Analogy

A golf club records each member's best round of the season. The first round is scored on putts only; from the second round on, every stroke counts. The first round becomes everyone's "best round" for the rest of the year, however much their game improves. Early stopping is the club secretary who, seeing no new best for ten rounds, cancels the member's season.

27.3.4 The fix, and making it safe to resume into

One line in `compute_validation_loss`: training keeps the ramp, validation uses the final weight from the first epoch.

```
return l1 + cfg.sam_weight * sam, n          # validation: lambda = 0.5 at every
    epoch
```

The source records why in its own comment: with the ramp, "epoch 1 (lambda 0) scored ~0.055 and every later epoch ~0.1 - early stop would have fired at the first allowed epoch whatever the model did" (found in E003 on 2026-10-14). Two smaller pieces make the fix safe:

- **A checkpoint remembers how it was scored.** The early-stop state saved inside each checkpoint carries `"score":"l1+full_sam"` (the constant `VAL_SCORE`). On resume, a stored state without that tag is ignored rather than compared with numbers on another scale.
- **The parts are logged separately.** `metrics.csv` records `val_l1`, `val_sam_deg` and `sam_weight` each epoch, so you can recompute any score by hand. Convert the angle first: radians = degrees $\times \pi/180$, so $4.90° = 4.90 \times 3.1416/180 = 0.0855$ rad.

Source check: the validation score and the resume check

Verified in `app/foundation_models/trainers/spectral_stem_mae_trainer.py`: `VAL_SCORE="l1+full_sam"` (line 44); `compute_validation_loss` returns `l1+cfg.sam_weight*sam` (line 341) while `compute_loss` returns `l1+self._get_sam_weight(self._current_epoch)*sam` (line 315); a loaded early-stop state whose `score` differs from `VAL_SCORE` is discarded with a log line (lines 148–151). The ramp itself is `sam_max*min(1.0,epoch/ramp_epochs)` with a 0-based epoch (`hyperspectral_segformer_mae_trainer.py`, `_get_sam_weight`).

27.4 One band list is not any band list

27.4.1 The symptom

E004, the reduced-attention stem of Chapter 24, was scored on the four labelled PRISMA scenes. A *ROC-AUC* (Chapter 33) of 0.5 means "no better than a coin". Scene set1 scored 0.217: worse than a coin, so the model ranked real anomalies *below* ordinary background.
[M: `indradhanuworkings/42-one-band-list.md`] Set1 is 75% sea. Splitting its mean reconstruction error by surface showed where the problem sat:

	mean L1 on water	on land
E004, PRISMA's native bands	0.0462	0.0282
v0.2.0 (production)	0.0046	0.0113

E004's error on water was 0.0462/0.0046 ≈ 10 times production's, and 93% of its highest-scoring pixels were sea.

The cheap explanation, measured first. "SpectralEarth is land, so the model never learned water." A census counted water: 6.8% of EnMAP training tiles had more than 30% water, against 6–12% of a 3,000-patch SpectralEarth sample.
[M: `research/experiments/indradhanu-se-pretrain/E004-reduced-attention/`] SpectralEarth has as much water as EnMAP. Measuring the cheap explanation before building its fix saved a training run that would have changed nothing.

The clue. The PRISMA bench scores each scene two ways: on PRISMA's *native* bands (its own list, 185 bands after the product's default band filter) and *resampled* onto SpectralEarth's 202 wavelengths. At epoch 18, set1 scored 0.205 native and 0.725 resampled. Same scene, same model; only the band list changed.

27.4.2 One vote per band

Chapter 24 gave the reduced stem two routes from a spectrum to its 32 channels, added together. Route 1 averages the bands inside each 40 nm *bin* and attends across the bins. Route 2, the *skip path*, sends every band straight to the channels:

$$\mathbf{s} = \frac{1}{N}\sum_{b=1}^{N} v_b\,\mathbf{W}(\lambda_b), \tag{27.2}$$

where v_b is the normalised value of band b, λ_b its wavelength, and $\mathbf{W}(\lambda)$ a weight vector produced by a small network from the wavelength. Because $\mathbf{W}$ is computed from λ, the formula works for any list of bands. But look at the factor $1/N$: every band gets one equal vote.

To see what that does, set every weight to 1 (one channel, $\mathbf{W} = 1$). Then (27.2) is the plain mean of the band values. Suppose a spectrum takes value a in the visible and c elsewhere, and a fraction f of the sensor's bands lie in the visible:

$$s = \frac{1}{N}\Big(\underbrace{a + \cdots + a}_{fN \text{ times}} + \underbrace{c + \cdots + c}_{(1-f)N \text{ times}}\Big)$$

Add the repeated terms:

$$= \frac{1}{N}\big(fNa + (1-f)Nc\big)$$

Cancel N:

$$= fa + (1-f)c.$$

The skip path's output depends on f, the share of bands in the visible: a property of the *sensor*, not of the ground.

Worked example 27.3: One spectrum, two sensors, two answers

Do this on paper. The spectrum is the same for both sensors: value 2 in the visible, −1 in the shortwave infrared. Sensor A has bands at 500, 505 and 2000 nm; sensor B at 500 and

2000 nm.

$$\text{A:}\quad s = \frac{2+2+(-1)}{3} = \frac{3}{3} = 1.0$$
$$\text{B:}\quad s = \frac{2+(-1)}{2} = \frac{1}{2} = 0.5.$$

Check with the formula $s = fa + (1-f)c$. For A, $f = 2/3$: $\frac{2}{3}\times 2 + \frac{1}{3}\times(-1) = \frac{4}{3} - \frac{1}{3} = 1.0$. For B, $f = 1/2$: $\frac{1}{2}\times 2 + \frac{1}{2}\times(-1) = 1 - 0.5 = 0.5$. Route 1 does not have this problem: it averages *inside* a bin first, so 500 and 505 nm become one bin value of 2 for both sensors.

The real lists differ by more than the toy: [M: `indradhanuworkings/42-one-band-list.md`]

	bands	visible (<700 nm)	NIR (700–1000)	SWIR (>1000)
SpectralEarth (trained on)	202	53 (26%)	48 (24%)	101 (50%)
PRISMA native (set1)	185	30 (16%)	23 (12%)	132 (71%)

On PRISMA's list the skip path hears the visible at about $16/26 = 0.62$ of the volume it was trained with. Water's signal is almost all in the visible (beyond about 750 nm water is nearly black), so water pixels are distorted most.

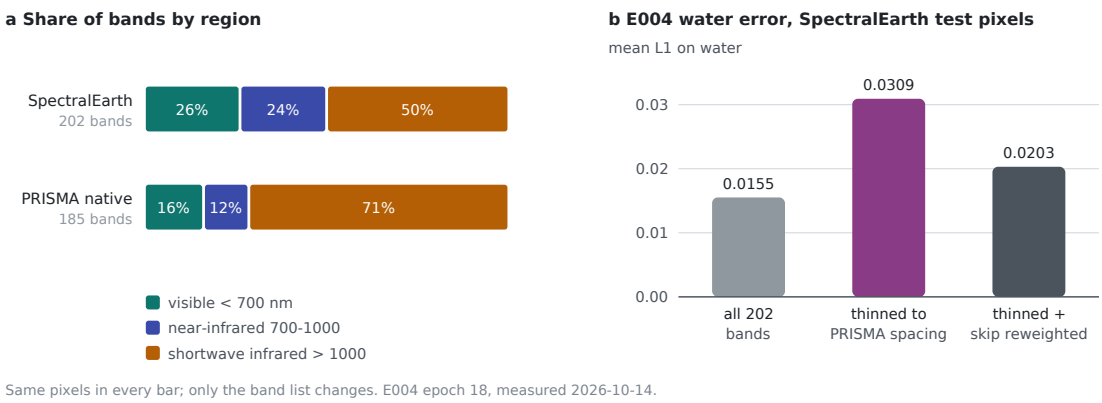


Figure 27.2: Left: how each sensor's bands divide between the visible, near-infrared and shortwave infrared. PRISMA's native list gives the visible 16% of its bands against SpectralEarth's 26%. Right: E004's error on water, on SpectralEarth test patches with all 202 bands, on the same pixels thinned to PRISMA's spacing, and with the skip path reweighted at inference.

27.4.3 Proving it one variable at a time

Two experiments isolated the band list from everything else, on E004's epoch-18 checkpoint (2026-10-14):

- **Thinning.** Take SpectralEarth test patches (the model's own sensor and wavelengths) and keep only the 151 of 202 bands nearest PRISMA's. No new sensor, no new wavelength: only fewer bands, spaced like PRISMA.
- **Reweighting.** At inference, scale each band's vote in the skip path by $n_{\text{train}}(\text{bin})/n_{\text{now}}(\text{bin})$. A bin that had 8 bands in training and has 5 now gives each of its 5 bands weight $8/5 = 1.6$.

condition (mean L1)	water	land
SpectralEarth, all 202 bands	0.0155	0.0157
same pixels, thinned to PRISMA spacing	0.0309	0.0226
thinned + skip reweighted	0.0203	0.0205
set1 native	0.0462	0.0282
set1 native + skip reweighted	0.0352	0.0200
set1 resampled to SpectralEarth's wavelengths	0.0138	0.0160

Source:
[M: `research/experiments/indradhanu-se-pretrain/E004-reduced-attention/`] (as quoted in IW42).

Read it in order. Thinning doubles the water error: $0.0309/0.0155 = 1.99$. Reweighting closes a fraction of that gap,

$$\begin{aligned} \text{gap} &= 0.0309 - 0.0155 = 0.0154 \\ \text{closed} &= 0.0309 - 0.0203 = 0.0106 \\ \text{fraction} &= 0.0106/0.0154 = 0.69, \end{aligned}$$

about two thirds, so the one-vote-per-band average is a real cause. It is not the only one: no fix brings set1 down to the resampled 0.0138.

27.4.4 The root cause and the three fixes

Every wavelength-dependent piece of the stem (the in-bin kernel $k(\lambda)$, the skip weights $\mathbf{W}(\lambda)$, the decoder's $\mathbf{U}(\lambda)$) is a small network fed the band's wavelength. In pretraining those networks were only ever evaluated at one list: the same 202 wavelengths, every batch, for 18 epochs. "Reads any band list" was a property of the *architecture*, never of the *training*.

> A function learned from examples is only trustworthy near the examples. A network of the wavelength that has only seen one list of 202 wavelengths has learned nothing about what a missing neighbour means.

The fixes adopted on 2026-10-14:

Fix 1: train on many band lists. With probability `band_augment_prob` (0.8 in F001) a batch reads a random subset of its bands, in one of three shapes chosen with equal probability (`sample_band_subset` in `spectral_stem_mae_trainer.py`):

1. **thinning**: keep each band with a probability drawn uniformly between 0.5 and 0.95;
2. **gaps**: remove one to three windows, each 20–150 nm wide, at random centres;
3. **even grid**: keep the band nearest each point of a grid with a random step of 6–15 nm (PRISMA-like at 10 nm, EnMAP-like at 6.5 nm).

If fewer than 40 bands survive, the batch keeps all of them. The kept wavelengths are also jittered by up to ±3 nm (`band_jitter_nm`). The model still reconstructs *all* the bands, so it learns to fill in the ones it was not given. Work the expected size of a thinned list on SpectralEarth: the keep probability averages $(0.5 + 0.95)/2 = 0.725$, so on average $202 \times 0.725 = 146.45$, about 146 bands are read. An even grid of 10 nm over SpectralEarth's range (about 418 to 2445 nm, [A] rounded) has about $(2445 - 418)/10 = 202.7$ points, but the bands inside the dropped water-vapour regions are missing, so several grid points share a nearest band and the kept list is shorter than the grid.

Fix 2: one vote per nanometre. Weight each band by the width of spectrum it stands for, w_b, and divide by the total width:

$$s_w = \frac{\sum_b w_b\, v_b}{\sum_b w_b}. \tag{27.3}$$

Why this is the right average: if the spectrum is a function $v(\lambda)$ and band b covers an interval of width w_b around λ_b, then $w_b v_b$ approximates the area under v over that interval. The sum is a *Riemann sum*, an approximation of the integral:

$$\sum_b w_b v_b \approx \int_{\lambda_{\min}}^{\lambda_{\max}} v(\lambda)\, d\lambda, \qquad \sum_b w_b \approx \lambda_{\max} - \lambda_{\min},$$

so

$$s_w \approx \frac{1}{\lambda_{\max} - \lambda_{\min}} \int_{\lambda_{\min}}^{\lambda_{\max}} v(\lambda)\, d\lambda,$$

the mean value of the spectrum over wavelength. That quantity belongs to the ground, not to the sensor: two sensors that cover the same range at different densities approximate the same integral.

Worked example 27.4: The toy again, with widths

Use idealised widths [A]: in A each visible band covers 5 nm; in B the one visible band covers 10 nm; each infrared band covers 10 nm.

$$\text{A:}\quad s_w = \frac{2\times 5 + 2\times 5 + (-1)\times 10}{5+5+10} = \frac{10+10-10}{20} = \frac{10}{20} = 0.5$$

$$\text{B:}\quad s_w = \frac{2\times 10 + (-1)\times 10}{10+10} = \frac{20-10}{20} = \frac{10}{20} = 0.5.$$

Same spectrum, same answer, and no new parameters.

Source check: how the real widths are computed

The course's toy uses idealised widths. The code (`band_widths` in `app/foundation_models/components/spectral_reduced.py`, lines 39–53) sorts the wavelengths, takes the gaps between neighbours, *caps each gap at 20 nm* (`MAX_GAP_NM`, "a band beside an excluded region would otherwise claim the whole gap"), gives an interior band the mean of its two gaps, and gives each end band its one gap in full. On the toy's three-band list A (500, 505, 2000 nm) that gives gaps 5 and $\min(1495, 20) = 20$, so widths 5, $(5+20)/2 = 12.5$ and 20, and

$$s_w = \frac{2\times 5 + 2\times 12.5 + (-1)\times 20}{5+12.5+20} = \frac{10+25-20}{37.5} = \frac{15}{37.5} = 0.4,$$

while B (gap capped at 20, widths 20 and 20) gives $(40-20)/40 = 0.5$. On a three-band toy the end rule and the cap dominate; on real lists of 185–372 bands only the two end bands and the bands beside the water-vapour gaps are affected, and the capped widths are the point: a band next to a 200 nm hole must not claim to represent it.

Fix 3: ignore wavelengths the model never trained on. The checkpoint carries its training wavelengths, so a band farther than 15 nm from all of them is dropped when a scene is loaded (`trained_band_mask`). The rule comes from the checkpoint, not from a fixed grid. On the bench's 185 PRISMA bands, 165 pass and 20 are dropped: those inside SpectralEarth's water-vapour gaps (1306–1502 and 1727–1959 nm) and past its last band. Chapter 29 shows how fine-tuning grows that record.

Did it work? E005 carried the three fixes. Extended to 20 epochs it read PRISMA's own bands about as well as E004 had read the resampled ones: mean ROC-AUC 0.752 against 0.754. [M: `research/experiments/indradhanu-se-pretrain/E005-band-robust/README.md`] That made E005's configuration the one to train properly.

27.5 F001: two hundred epochs on rotating data

27.5.1 Why more data, not only more epochs

F001 is the final SpectralEarth pretraining of the next Indradhanu, with E005's configuration exactly, and the checkpoint all fine-tuning starts from. Its plan was 200 epochs. An epoch here is `train_samples_per_epoch` = 30,000 patches.
[M: `research/experiments/indradhanu-se-pretrain/F001-final-pretrain/config.json`]

The cheap way to train 200 epochs is to show the same 30,000 patches 200 times. A model with millions of parameters can then *memorise*: improve on those exact patches without improving on patches it has never seen. F001 instead walks through ten *blocks*. A block is 200 consecutive training shards; at 150 patches per shard that is $200 \times 150 = 30{,}000$ patches, exactly one epoch's worth. Block k is shards $200k$ to $200k + 199$ and trains epochs $20k + 1$ to $20(k + 1)$:

block k	shards	epochs
0	0–199	1–20
1	200–399	21–40
2	400–599	41–60
⋮	⋮	⋮
9	1800–1999	181–200

Worked example 27.5: Coverage per epoch and per run

Do this on paper. How often is each patch seen, how many different patches does the run see, and what share of the training split is that?

epochs per block:	$200/10 = 20$
times each patch is seen:	20 (one per epoch of its block)
distinct patches:	$10 \times 30{,}000 = 300{,}000$
share of the split:	$300{,}000/512{,}849 = 0.585 \approx 58\%$
patch presentations in total:	$200 \times 30{,}000 = 6{,}000{,}000$.

Compare the E-runs (E001–E005), capped at 20 epochs on one block: each patch also seen 20 times, but only 30,000 distinct patches, $30{,}000/512{,}849 = 0.058$, under 6% of the split. F001 repeats each patch exactly as often as the E-runs did, and sees ten times as many of them.

27.5.2 Why not put all ten blocks on the disk

Because they do not fit. From worked example 27.1, one shard is about 0.99 GB, so

one block:	200×0.99 GB $= 198$ GB
all ten:	10×198 GB $= 1{,}980$ GB ≈ 1.98 TB.

The L40S instance's local NVMe disk holds about 412 GB. It fits two blocks, the one training and the one being downloaded for next time: $2 \times 198 = 396\,\text{GB} < 412\,\text{GB}$. Three blocks (594 GB) do not.

27.5.3 One schedule across all ten blocks

The learning rate follows the cosine schedule of Chapter 26 [28], computed over all 200 epochs, with $\eta_0 = 0.0005$ and $\eta_{\min} = 10^{-6}$:

$$\eta(t) = \eta_{\min} + \tfrac{1}{2}(\eta_0 - \eta_{\min})\Big(1 + \cos\frac{\pi t}{200}\Big). \tag{27.4}$$

Note $\eta_0 - \eta_{\min} = 0.0005 - 0.000001 = 0.000499$. At the first three rotations:

$$\begin{aligned}
t = 20: &\quad \cos(0.1\pi) = 0.9511 \\
&\quad \eta = 0.000001 + 0.5 \times 0.000499 \times 1.9511 = 0.000001 + 0.0004868 = 0.000488 \\
t = 40: &\quad \cos(0.2\pi) = 0.8090 \\
&\quad \eta = 0.000001 + 0.5 \times 0.000499 \times 1.8090 = 0.000001 + 0.0004513 = 0.000452 \\
t = 60: &\quad \cos(0.3\pi) = 0.5878 \\
&\quad \eta = 0.000001 + 0.5 \times 0.000499 \times 1.5878 = 0.000001 + 0.0003962 = 0.000397.
\end{aligned}$$

The data changes at each rotation; the schedule does not. A rotation resumes the scheduler from the checkpoint, so epoch 21 continues from epoch 20's learning rate rather than jumping back to 0.0005.

Source check: no warm-up

The F001 README once described "one cosine schedule over 200 epochs (5 warm-up)". The run's `config.json` sets `warmup_epochs:0`, and `FoundationTrainer._build_scheduler` (`app/abstract_classes/foundation_trainer.py`) builds a plain `CosineAnnealingLR` that never reads that field. The source wins: F001 has no warm-up, and its first epoch ran at the full 0.0005. The README has since been corrected ("no warm-up: `warmup_epochs` is 0 in the config").

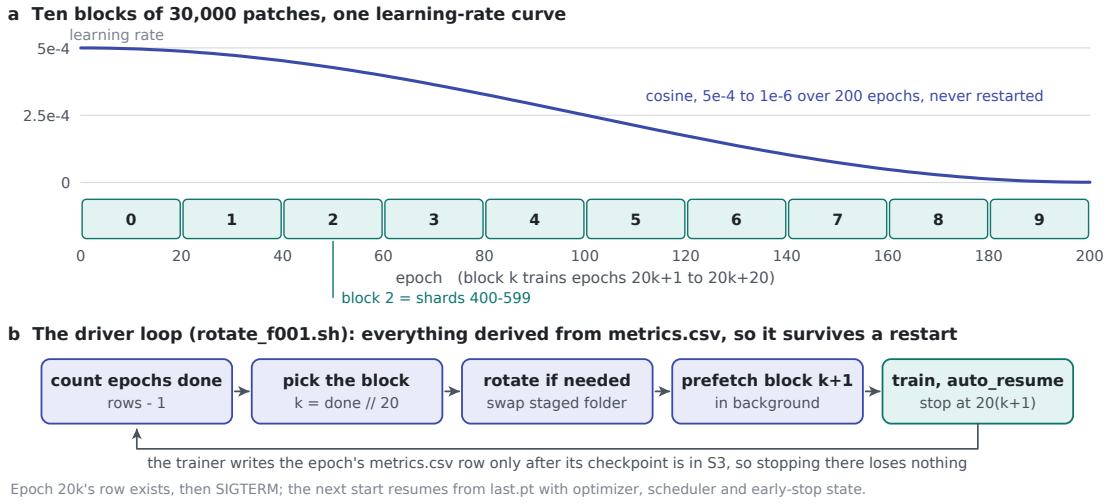


Figure 27.3: (a) The ten 30,000-patch blocks along F001's 200 epochs, under the one cosine learning-rate curve that runs through all of them. (b) The loop in `rotate_f001.sh`: every decision is computed from `metrics.csv` and two marker files, so the script can be killed and restarted at any point.

27.5.4 Early stopping is off

F001 sets `early_stop_patience:null`, which turns early stopping off (the stop test begins with `bool(cfg.early_stop_patience)`, which is false for `null`). The reason is the rotation. Within a block the model sees the same 30,000 patches 20 times, so it can flatten out near the end of a block and improve again when fresh data arrives. Suppose patience 10 and a best score at epoch 30:

epochs 31–40	no 1% improvement; bad epochs climb 1, …, 10
epoch 40	patience reached, $40 \geq 20$: **stop**
epoch 41	block 2 would have arrived; never trained

`best.pt` still tracks the lowest validation score, so the best epoch is kept either way.

27.5.5 The driver: state read from the run's own records

`research/experiments/indradhanu-se-pretrain/F001-final-pretrain/rotate_f001.sh` runs on the instance. The trainer trains on whatever shard files sit in its local train folder, and `auto_resume` continues from `last.pt` with the optimiser, scheduler and early-stop state. So a rotation is: stop, swap the folder's contents, start again. Each pass of the driver's loop does five things:

1. **Count the epochs done.** `metrics.csv` has a header plus one row per finished epoch, so done = lines − 1.
2. **Pick the block.** $k = \lfloor \text{done}/20 \rfloor$ (integer division) and stop_at $= 20(k+1)$.
3. **Rotate if needed.** A marker file `BLOCK` says which block the train folder holds. If it is not k, swap in the staged copy (only if a second marker says the stage holds block k and all 200 files are there); otherwise download block k now.
4. **Prefetch** block $k+1$ into a staging folder in the background.
5. **Train**: start the trainer, poll every 30 seconds, and stop it once done $\geq$ stop_at.

Worked example 27.6: One pass of the driver after a crash

The instance restarts after a crash and `metrics.csv` has 41 lines. Do each step:

$$\text{done} = 41 - 1 = 40$$
$$k = \lfloor 40/20 \rfloor = 2$$
$$\text{stop_at} = 20 \times (2+1) = 60.$$

`BLOCK` says 1, not 2, so the driver swaps in block 2 (shards $200 \times 2 = 400$ to $400 + 199 = 599$), starts prefetching block 3 (shards 600–799), and resumes from `last.pt` (epoch 40): epochs 41 to 60, stopped at 60.

Why stopping is safe. The trainer saves the checkpoint and uploads it before it appends the epoch's row to `metrics.csv`. So by the time the driver sees row 60, epoch 60's checkpoint is already safe. A state machine whose only state is "how many rows are in this file" can be killed and restarted at any moment.

What it costs. About 9.2 minutes per 30,000-patch epoch on the L40S at $2.69 per hour:

$$200 \times 9.2 \text{ min} = 1{,}840 \text{ min}$$
$$1{,}840/60 = 30.7 \text{ hours}$$
$$30.7 \times \$2.69 = \$82.5,$$

matching the README's pre-launch estimate of "roughly 30 hours, ~$82".
[M: research/experiments/indradhanu-se-pretrain/F001-final-pretrain/README.md]

27.5.6 What the curves showed

Two scores answer two questions. Validation L1 and SAM ask how well the model redraws SpectralEarth patches it never trained on. The PRISMA bench asks how well it *finds anomalies*: four labelled PRISMA scenes on PRISMA's own bands, scored by mean ROC-AUC under the old `combined` score and, from epoch 52, under residual Mahalanobis (Chapter 31).

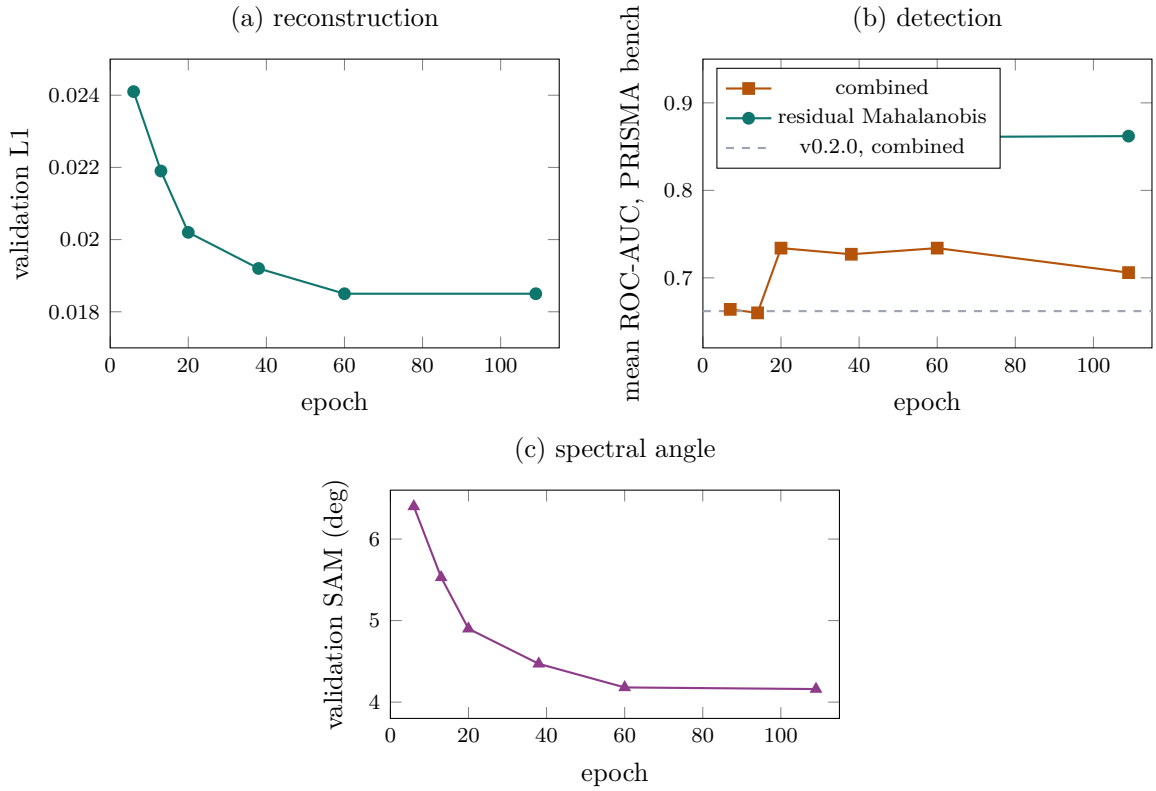


Figure 27.4: F001's records drawn to scale from its README (validation rows at epochs 6, 13, 20, 38, 60, 109; bench at 7, 14, 20, 38, 60, 109). (a) Validation L1 falls at every checkpoint until epoch 60 and then flattens. (b) The bench under the `combined` score jumps above production's 0.662 by epoch 20 and then stays flat; residual Mahalanobis, measured at epochs 60 and 109, sits about 0.13 higher. (c) Validation SAM follows L1.

Source for Figure 27.4:
[M: research/experiments/indradhanu-se-pretrain/F001-final-pretrain/README.md]. From epoch 20 to 60 reconstruction improved:

$$\text{L1:}\quad 0.0202 - 0.0185 = 0.0017; \quad 0.0017/0.0202 = 0.084 \approx 8\% \text{ better}$$
$$\text{SAM:}\quad 4.90 - 4.18 = 0.72^\circ; \quad 0.72/4.90 = 0.147 \approx 15\% \text{ better.}$$

Detection under `combined` did not move: 0.734, 0.727, 0.734. There are two ways to read a stall like that: the model has stopped learning what matters, or the ruler cannot see the improvement.

The same reconstructions scored with residual Mahalanobis gave 0.861 at epoch 60, far above `combined`'s 0.734. The stall was the ruler: `combined` was dominated by errors every pixel shares (detector stripes, haze, dark sea), so reducing the model's error on ordinary ground barely moved it. Chapter 31 derives why.

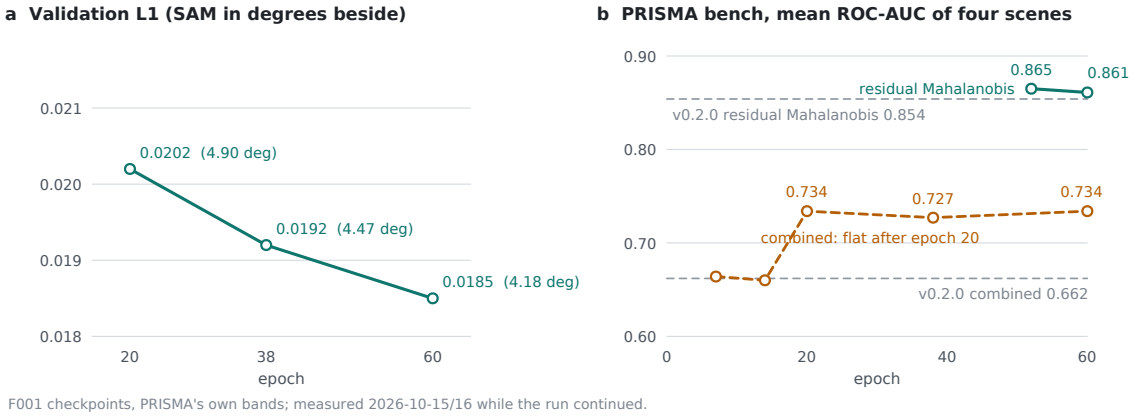


Figure 27.5: The course's version of the same records, including the residual-Mahalanobis measurement at epoch 52 (0.865) reported in the scoring comparison of 2026-10-15.

Values that will drift

F001 was planned for 200 epochs and stopped at epoch 113 by decision, because the bench under residual Mahalanobis was flat: 0.861 at epoch 60, 0.862 at epoch 109. [M: `docs/proposals/fine-tuning-and-segmentation.md`] Fine-tuning starts from the epoch-109 checkpoint, `F001_ep109.pt`. Under residual Mahalanobis, production's v0.2.0 scored 0.854, only 0.011 behind F001's epoch 52 (0.865), and caught more objects (403 against 383 of 909) (IW43, 2026-10-15). One scene can swing a mean: set1 scored 0.430 at epoch 14 and 0.702 at epoch 20 under `combined`, a swing of 0.27 from one block of training. Four scenes cannot give a reliable mean when one of them moves that much, which is why every later gate compares runs on the same scenes with the same score (Chapter 36).

27.6 Three sensors at their own bands

27.6.1 Why new data at all

F001 has only seen SpectralEarth: EnMAP data on one fixed list of 202 bands. The product scores three hyperspectral sensors, PRISMA, EnMAP and AVIRIS-NG (Chapter 3), each on its own band list and at its own pixel size. Fine-tuning (Chapter 29) trains on them directly, for three reasons given in `docs/proposals/fine-tuning-and-segmentation.md`: real sensors; the wavelengths SpectralEarth dropped (including the hydrocarbon band near 1730 nm); and EnMAP's cloud and water labels for a segmentation head (Chapter 28).

Definition 27.2 (Native and resampled)**.** A *native* band list is the sensor's own list of band centres and its own pixel grid, untouched. A *resampled* list holds values interpolated onto some other list, such as the 165-band grid of Chapter 3 or SpectralEarth's 202.

All three sets share SpectralEarth's sample layout: `pixels.npy` (or compressed `pixels.i16dz`), int16 `(bands, 128, 128)`, multiplied by 0.0001 for reflectance, −32768 for nodata; `wavelengths.npy`, float32 `(bands,)`, the centre of each band in the same order; `meta.json` with scene facts; and for EnMAP the provider's quality layers `label_*.npy`. A tile is kept only if at least 10% of its pixels are valid.

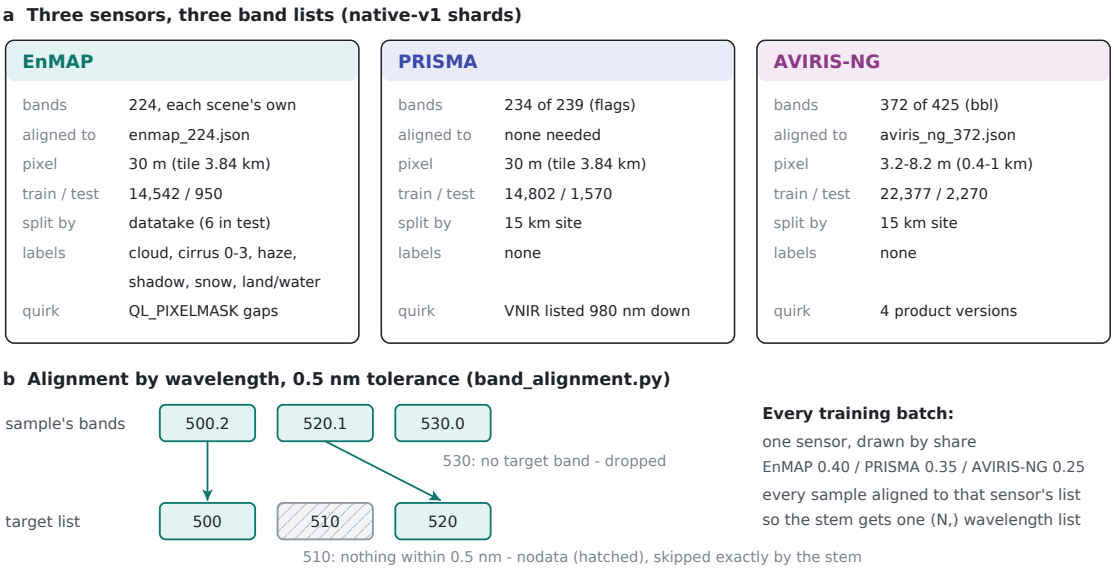


Figure 27.6: (a) The three shard sets as they stood on 2026-10-16 (colours identify the sensor only). (b) The one rule that lets mixed band lists share a trainer: match by wavelength within 0.5 nm; a target band with no match becomes nodata, a source band with no target is dropped.

27.6.2 EnMAP: 224 bands, labels and a defect map

210 EnMAP L2A scenes became 14,542 training and 950 test tiles, each with all 224 bands at the scene's own wavelengths. [M: `docs/lld/enmap-shards.md`]

A renumbered band. Five early-2023 scenes (processor version V010502) number their shortwave bands one off: there band 142 is 1530.29 nm, elsewhere 1518.87 nm. Reading "band 142" by position would silently mix two wavelengths. So every sample carries its wavelengths, and everything downstream matches *by wavelength, never by band number*. Across all scenes the stats list 227 distinct wavelengths: 224 per scene plus the renumbered scenes' shifted ones.

The split by datatake. A *datatake* is one continuous recording; consecutive scenes from it border each other. Whole datatakes go to one side of the split: 6 of the 66 went to test (13 scenes), with cloudy and clear ones represented. A random split by scene would put ground a few pixels from a training scene into the test set.

The pixel mask. EnMAP's `QL_PIXELMASK` flags individual voxels (one band of one pixel); the tiler turns each flagged voxel into nodata. A probe of three scenes found bands 131 to 135 (about 1343–1390 nm, the L2A water-vapour blank) flagged everywhere, and about 200 other bands with scattered flags whose share is the same in all three scenes to 0.01% (band 187: 5.31%). A share that does not change with the scene follows the *detector*, not the ground: a fixed defect map. Net effect: 85–95% of pixels lose 2–4 of their 224 bands. That is harmless: a dead band drops out of the wavelength stem exactly (Chapter 23) and out of the loss too. A pixel with 221 of 224 bands is a slightly thinner spectrum, not a hole.

Labels. EnMAP's L2A product ships quality layers: cloud, cirrus (levels 0–3), haze, cloud shadow, snow, and a class map. They are stored exactly as delivered and never applied to the pixels: a cloudy tile stays in, because cloudy tiles are what a segmentation head learns from. Chapter 28 audits them.

27.6.3 PRISMA: reading the file without the product pipeline

223 PRISMA L2D scenes became 14,802 training and 1,570 test tiles. [M: `docs/lld/prisma-shards.md`] The product reads PRISMA through a builder that resamples onto the 165-band grid; the shards must keep the sensor's bands, so `read_scene` (`app/utils/prisma/scene.py`) opens the HDF5 file directly and applies three rules:

$$\text{reflectance} = \frac{\text{DN}}{65535}\,(\text{Max}_{\text{det}} - \text{Min}_{\text{det}}) + \text{Min}_{\text{det}}$$

$$\text{nodata} \iff \text{DN} = 0 \text{ or the detector's error flag} \neq 0$$

$$\text{dropped} \iff \text{the band's flag is 0 (5 of 239 bands).}$$

DN (digital number) is the stored integer, 0 to 65,535; the detector *det* is VNIR or SWIR, each with its own scale factors in the file. So $239 - 5 = 234$ bands remain.

Worked example 27.7: From a PRISMA digital number to a stored value

Take DN = 13,107 with illustrative factors [A] Min = 0.0, Max = 1.0.

$$13{,}107/65{,}535 = 0.2$$
$$\text{reflectance} = 0.2 \times (1.0 - 0.0) + 0.0 = 0.2$$
$$\text{stored} = \text{round}(0.2/0.0001) = \text{round}(2000) = 2{,}000 \text{ (int16)}.$$

The LLD's check: on set1 at 860 nm the result differs from the product builder's own read by at most 0.00005 reflectance, which is int16 rounding: one stored unit is 0.0001, so rounding moves a value by at most half of that.

Odd band order, one list everywhere. The file lists the VNIR bands from about 980 nm *downwards*, then the SWIR bands upwards, and the detectors overlap between about 920 and 980 nm. Nothing is sorted or de-duplicated: the stem takes bands in any order because each carries its wavelength. All 234 centres were identical to 0.000 nm across every scene checked, so PRISMA batches need no alignment.

234 or 185? The shards hold 234 PRISMA bands; the bench reads 185, PRISMA's own bands after the product's default band filter removes noisy and overlapping ones. Both are native; they are not the same list.

The split by site, single-link. PRISMA revisits places, so `app/utils/prisma/split.py` groups scenes into sites: scenes whose centres lie within 15 km of each other, *chained*. Single-link chaining means two scenes are in one site if a chain of steps, each at most 15 km, joins them:

pair	distance	
P–Q	10 km	≤ 15: same site
Q–R	12 km	≤ 15: same site
P–R	21 km	> 15, but P and R are already joined through Q

Result: one site $\{P, Q, R\}$. Any site within 30 km of a benchmark scene is excluded from both sides (on the real run, none was). Of the rest, 10% of sites went to test: 21 scenes against 202 for training. PRISMA L2D has no cloud or water mask, so PRISMA batches teach reconstruction only.

27.6.4 AVIRIS-NG: an airborne sensor in four product versions

AVIRIS-NG flies on an aircraft. The 79 flight lines of the ISRO–NASA India campaign became 22,377 training and 2,270 test tiles. [M: `docs/lld/aviris-shards.md`] Each line has 425 bands, of which the header's bad-band list marks 53 (the water-vapour windows): $425 - 53 = 372$ remain. The campaign ships four product versions that place the 372 centres up to 0.42 nm apart, so across all lines the stats list 1,116 distinct wavelengths in 372 tight groups ($3 \times 372 = 1{,}116$: the 372 bands appear at up to three positions). The fine-tuning band list `aviris_ng_372.json` takes the *median* of each group; every variant is within 0.5 nm of it.

Pixel size. 3.2 to 8.2 m, depending on the line, deliberately not resampled to 30 m. A tile is always 128 pixels across, so the ground it covers changes:

$$\begin{aligned} \text{EnMAP, PRISMA:} &\quad 128 \times 30 \text{ m} = 3{,}840 \text{ m} \\ \text{AVIRIS-NG:} &\quad 128 \times 3.2 \text{ m} = 409.6 \approx 410 \text{ m}, \qquad 128 \times 8.2 \text{ m} = 1{,}049.6 \approx 1{,}050 \text{ m}. \end{aligned}$$

An AVIRIS-NG tile shows a field; a satellite tile shows a district.

27.6.5 Making it fit: band-delta compression

Neighbouring bands of one pixel are similar, so storing each band as its *difference from the previous band* leaves small numbers, which a general-purpose compressor (zstd) packs far better (`app/utils/shard_codec.py`). The format is `.i16dz`.

Worked example 27.8: Delta-encoding one pixel

Four bands of one pixel, 1000, 1012, 1020, 1015. The encoder subtracts a zero band in front, so the first value is kept:

$$\begin{aligned} \text{deltas:} &\quad 1000 - 0 = 1000, \quad 1012 - 1000 = 12, \\ &\quad 1020 - 1012 = 8, \quad 1015 - 1020 = -5 \\ \text{decode (running sum):} &\quad 1000; \quad 1000 + 12 = 1012; \\ &\quad 1012 + 8 = 1020; \quad 1020 - 5 = 1015. \end{aligned}$$

Wrap-around is exact. The differences are taken in int16, which holds −32,768 to 32,767. Next to nodata a difference overflows, and the codec lets it wrap. Wrapping is arithmetic *modulo* $2^{16} = 65{,}536$: a result is replaced by the unique number in the int16 range that differs from it by a multiple of 65,536. Encoding and decoding both work modulo 65,536, and $(a - b) + b \equiv a \pmod{65{,}536}$, so the round trip returns a whenever a itself is in range:

$$\begin{aligned} \text{encode:} &\quad 1000 - (-32{,}768) = 33{,}768 \quad \text{(too big for int16)} \\ &\quad 33{,}768 - 65{,}536 = -31{,}768 \quad \text{(stored, wrapped)} \\ \text{decode:} &\quad -32{,}768 + (-31{,}768) = -64{,}536 \quad \text{(too small)} \\ &\quad -64{,}536 + 65{,}536 = 1000 \quad \text{(exact)}. \end{aligned}$$

The size arithmetic. AVIRIS-NG tiles were written compressed from the start; EnMAP and PRISMA were re-encoded on 2026-10-16, about 2.1 times smaller.
[M: `research/experiments/indradhanu-finetune/README.md`] From the sizes in the three shard LLDs

(GiB, train + test):

$$\begin{aligned}
\text{EnMAP raw:} &\quad 100.9 + 6.6 = 107.5; \quad \text{re-encoded } 107.5/2.1 = 51.2 \\
\text{PRISMA raw:} &\quad 105.8 + 11.2 = 117.0; \quad \text{re-encoded } 117.0/2.1 = 55.7 \\
\text{AVIRIS-NG:} &\quad 116.0 + 11.2 = 127.2 \quad \text{(already compressed)} \\
\text{total raw:} &\quad 107.5 + 117.0 + 127.2 = 351.7 \\
\text{total re-encoded:} &\quad 51.2 + 55.7 + 127.2 = 234.1.
\end{aligned}$$

351.7 GiB would leave no working room on a 412 GB disk ($351.7 \times 1.0737 = 377.6$ GB, since 1 GiB $= 2^{30}$ bytes $= 1.0737$ GB); 234.1 GiB (251.4 GB) fits, with room to spare.

Source check: two compression ratios

SpectralEarth's shards compress 1.89–1.91 times (`docs/lld/spectralearth.md`; the docstring of `app/utils/shard_codec.py` says 1.91x). The native-v1 EnMAP and PRISMA sets compress about 2.1 times (`research/experiments/indradhanu-finetune/README.md`). Comments in `recompress_shards.py` and `recompress_ft.sh` still quote SpectralEarth's ~1.9x for the fine-tuning sets. Both numbers are right for their own data; quote the one that belongs to the shards you are sizing.

27.6.6 One batch, one band list

The reduced stem takes *one wavelength list per call*: a single `(N,)` vector, not one per sample. A batch is a stack of samples, so every sample in it must carry the same bands in the same order. Two rules make that true:

1. **One sensor per batch** (relaxed in Chapter 29 to one band list per *stem pass*).
2. **Within a sensor, align by wavelength** (`app/utils/band_alignment.py`). Each target band takes the nearest source band within 0.5 nm; a target band with no match becomes nodata; a source band with no target is dropped. Nothing is interpolated.

The tolerance 0.5 nm is chosen so it "only absorbs rounding in the metadata and never pairs two neighbouring bands": EnMAP's bands are 6–12 nm apart, AVIRIS-NG's about 5 nm, so two real neighbours are always more than ten tolerances apart.

Worked example 27.9: Aligning a sample onto a band list

Target list 500, 510, 520 nm; a sample with bands at 500.2, 520.1 and 530.0 nm.

target	nearest source	distance	test	result
500	500.2	$\lvert 500 - 500.2 \rvert = 0.2$	≤ 0.5	take it
510	500.2	$\lvert 510 - 500.2 \rvert = 9.8$	> 0.5	nodata
520	520.1	$\lvert 520 - 520.1 \rvert = 0.1$	≤ 0.5	take it

Source 530.0 is near no target and is dropped. The aligned sample has three bands, one of them dead; the stem skips the dead one exactly.

Per sensor: EnMAP aligns onto `enmap_224.json`, the 224 centres of the standard scenes (the renumbered scenes' shifted bands are dropped, and standard bands they lack become nodata); PRISMA needs nothing; AVIRIS-NG aligns onto the medians in `aviris_ng_372.json`, and every variant pairs.

27.7 Common confusions

Should training use the full SAM weight from epoch 1 too?
No. The ramp exists for the gradients (Chapter 25): early SAM gradients are large and noisy. The fix changes only the number used to compare epochs, not what the model learns from.

Isn't a minimum of 20 epochs already protection against the trap?
It delays the stop until the ramp has finished, but epoch 1's 0.055 is still the best on record at epoch 20, so the stop fires there and `best.pt` is epoch 1.

So the wavelength-aware stem was a mistake?
No. A fixed 1×1 compressor cannot even be *asked* to read a different list. What was missing was training that exercised the ability. Band-list augmentation provides it.

Isn't band-list augmentation the "hiding bands" trap of an earlier chapter?
It hides bands from the *input*, but the loss is taken only on pixels the token mask hid completely, and a hidden pixel's own bands are never an input. A pixel never learns to fill in its own missing bands, so an anomaly cannot rebuild itself.

Does rotating the data restart the training?
No. Weights, optimiser state and learning rate all continue from `last.pt`. Only the files in the train folder change.

Why 20 epochs per block, not 1?
A block is one epoch's worth of patches, so one epoch per block would mean a 198 GB download every 9 minutes. Twenty epochs per block gives the prefetch about three hours and keeps each patch's repeat count equal to the E-runs'.

Does nodata mean zero?
No. Nodata is $-32{,}768$ in the file and becomes an invalid voxel in the trainer, which the stem skips exactly. A zero would be read as "black in this band".

PRISMA has 234 bands or 185?
Both. The shards hold all 234 real bands; the bench and the product read 185 after the default band filter.

27.8 Check yourself

Exercise 27.1. Using the table of worked example 27.2, a later epoch has L1 = 0.050 and SAM = 0.18 rad at $\lambda = 0.05$. Compute its score with the ramp and at the full weight, and use the decomposition $\Delta L1 + \lambda_e \Delta\,\mathrm{SAM} + \Delta\lambda\ \mathrm{SAM}_{e+1}$ to explain the change from epoch 1 under the ramp.

Exercise 27.2. Epoch 1 scores 0.155 at the full weight. What must a later epoch score to count as an improvement under $\delta = 0.01$? If every later epoch scores 0.1540, on which epoch does the run stop (patience 10, minimum 20)?

Exercise 27.3. A sensor C samples the visible three times (500, 505, 510 nm, value 2) and the infrared once (2000 nm, value -1). Compute the plain $1/N$ skip output, then the width-weighted output with idealised widths of 5 nm per visible band and 10 nm for the infrared band. Explain why the width-weighted answer is not 0.5.

Exercise 27.4. After a restart `metrics.csv` has 128 lines. Work out done, k, stop_at and the shards of the block, and compute the learning rate at $t = 100$ from (27.4).

Exercise 27.5. Delta-encode the bands 2000, 2010, 1990, 1995, then decode them. Then encode the pair (nodata −32,768, then 30,000) with int16 wrap-around and decode it again, showing each step.

Exercise 27.6. Align a sample with bands 500.3, 509.6, 525.0 nm onto the list 500, 510, 520 nm. Which bands are kept, which targets become nodata, which source bands are dropped?

Next: Chapter 28, The segmentation head, BCE plus Dice, and auditing labels.

Chapter 28

The segmentation head, BCE plus Dice, and auditing labels

The one thing this chapter teaches

Fine-tuning adds a small segmentation head to the shared encoder: five independent yes/no outputs per pixel (water, cloud, thin cloud, shadow, snow), each a sigmoid, trained with binary cross-entropy (the negative log-likelihood of a coin flip) plus soft Dice (an overlap score that a rare class cannot dodge), counted only over pixels whose label is trusted. Which labels to trust was decided by auditing the provider's labels without any ground truth.

28.1 Why a segmentation head

Segmentation means giving every pixel a label, "water" or "cloud", rather than one label per image. The product already segments: the `scene_segmentation` action (Chapter 4) computes spectral indices (NDVI for vegetation, NDWI for water, a brightness measure) and applies fixed thresholds to them (`backend/allotrope/action_types/scene_segmentation.py`). Its masks decide which pixels `anomaly_scoring` looks at.

Thresholds on indices are blunt. Cloud edges, thin cloud and shadows slip through and get scored as anomalies: a cloud is something the reconstruction model rarely redraws well, so its error is large, so it looks anomalous. The obvious fix is a better rule (a lower brightness threshold, a second index), and it fails the same way one notch later: a hand-written rule has no idea what thin cloud over a dark field looks like until someone writes that case in.

The fine-tuning plan (`docs/proposals/fine-tuning-and-segmentation.md`) instead teaches the model. Indradhanu's encoder has already spent a hundred epochs learning what hyperspectral scenes look like (Chapter 27); a small extra network on top of its features, a *head*, can learn to outline water and cloud from labelled examples at a fraction of the cost of a new model. The head is judged by detection: nothing ships that lowers the PRISMA bench.

The chapter has two halves. The first (Sections 28.2 to 28.5, 28.7 and 28.8) builds the head and its loss, deriving each formula. The second (Section 28.9) asks which of the labels it learns from can be trusted, when nobody has a correct set to compare with.

28.2 The head: the decoder of an earlier chapter, smaller

28.2.1 What it is

In `app/foundation_models/components/spectral_stem_seg_former_mae.py` the head is one line:

```
self.seg_head = (SegFormerDecoder(embed_dims=list(embed_dims), decoder_dim=
    seg_decoder_dim,
                                   out_channels=seg_classes) if seg_classes else None)
```

It is the same `SegFormerDecoder` class that reconstructs the spectrum (Chapter 19) [46], built with width `decoder_dim` $d = 128$ instead of 256 and `out_channels` $K = 5$ (one per class) instead of 32. Pretrained models have `seg_classes=0`, so no head; fine-tuning sets 5.

Recall the decoder's steps. The encoder (Chapter 17) produces four feature maps at strides 4, 8, 16 and 32 with 32, 64, 160 and 256 channels. The decoder

1. projects each map to d channels with a 1×1 convolution;
2. upsamples each to the stride-4 grid (bilinear) and stacks them: $4d$ channels;
3. fuses them back to d with a 1×1 convolution and a GELU;
4. refines with a 3×3 convolution and a GELU;
5. predicts $16K$ channels with a second 3×3 convolution: one value for each of the 16 sub-pixels of a 4×4 block, for each of the K classes;
6. rearranges those channels into a full-resolution map with *PixelShuffle*(4) [39].

Table 28.1: Shapes through the segmentation head for a batch of B tiles of 128×128 pixels.

step	shape
encoder stage 1, F_1	`(B, 32, 32, 32)`
encoder stage 2, F_2	`(B, 64, 16, 16)`
encoder stage 3, F_3	`(B, 160, 8, 8)`
encoder stage 4, F_4	`(B, 256, 4, 4)`
project each to 128, upsample to 32×32, stack	`(B, 512, 32, 32)`
fuse (1×1)	`(B, 128, 32, 32)`
refine 1 (3×3)	`(B, 128, 32, 32)`
refine 2 (3×3), $16 \times 5 = 80$ channels	`(B, 80, 32, 32)`
PixelShuffle(4): $80 = 5 \times 4 \times 4$	`(B, 5, 128, 128)`, logits

A *logit* is a raw score, any real number; Section 28.3 turns it into a probability.

Source check: design text against code

The proposal's first description of the head was "1x1 projections to a common width, upsample to 1/4, fuse, 1x1 to classes, bilinear to full size". The code reuses `SegFormerDecoder`, which ends in two 3×3 convolutions and a PixelShuffle, not a 1×1 and a bilinear upsample (`app/foundation_models/components/seg_former_decoder.py`, lines 113–117). The source wins: the head predicts each pixel of a 4×4 block separately, as the reconstruction does. The proposal now says "3x3 refine, PixelShuffle to full size", so the two agree today.

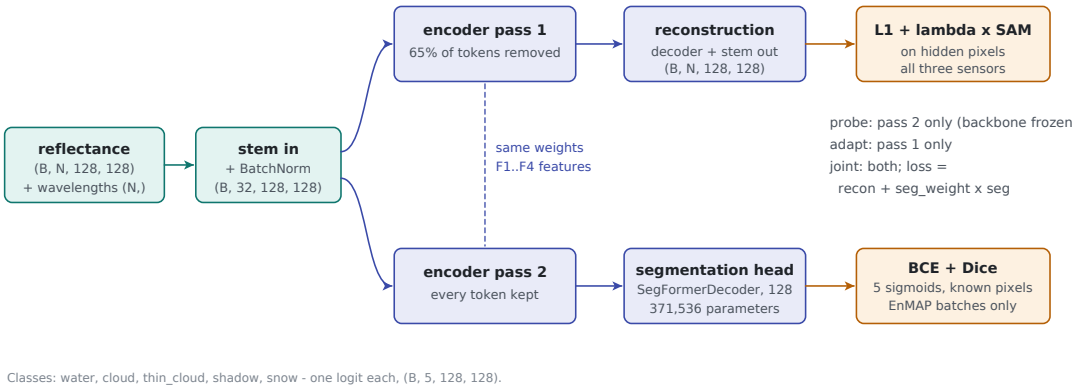


Figure 28.1: The head in context: the shared stem and encoder feed two heads. Pass 1 removes 65% of the tokens and reconstructs; pass 2 keeps every token and segments. Which passes run depends on the training mode (Chapter 29).

28.2.2 Counting its parameters before reading them

A convolution with C_{in} input channels, C_{out} output channels and a $k \times k$ kernel has $k^2 C_{\text{in}} C_{\text{out}}$ weights plus C_{out} biases (Chapter 10). Apply it to every layer, with $d = 128$ and $K = 5$:

Worked example 28.1: The head's parameter count

Do this on paper, one layer per line.

$$\begin{aligned}
\text{project } 32 \to 128: &\quad 32 \times 128 + 128 = 4{,}096 + 128 = 4{,}224 \\
\text{project } 64 \to 128: &\quad 64 \times 128 + 128 = 8{,}192 + 128 = 8{,}320 \\
\text{project } 160 \to 128: &\quad 160 \times 128 + 128 = 20{,}480 + 128 = 20{,}608 \\
\text{project } 256 \to 128: &\quad 256 \times 128 + 128 = 32{,}768 + 128 = 32{,}896 \\
\text{fuse } 512 \to 128\ (1 \times 1): &\quad 512 \times 128 + 128 = 65{,}536 + 128 = 65{,}664 \\
\text{refine 1 } 128 \to 128\ (3 \times 3): &\quad 9 \times 128 \times 128 + 128 = 147{,}456 + 128 = 147{,}584 \\
\text{refine 2 } 128 \to 80\ (3 \times 3): &\quad 9 \times 128 \times 80 + 80 = 92{,}160 + 80 = 92{,}240.
\end{aligned}$$

Add them in pairs:

$$\begin{aligned}
4{,}224 + 8{,}320 &= 12{,}544 \\
12{,}544 + 20{,}608 &= 33{,}152 \\
33{,}152 + 32{,}896 &= 66{,}048 \\
66{,}048 + 65{,}664 &= 131{,}712 \\
131{,}712 + 147{,}584 &= 279{,}296 \\
279{,}296 + 92{,}240 &= 371{,}536.
\end{aligned}$$

Building the head and counting its parameters gives 371,536 [M: `indradhanuworkings/47-the-segmentation-head.md`] exactly. The GELUs, the bilinear upsampling and the PixelShuffle have no parameters.

A closed form. Write the same count with symbols, d for the width and K for the classes. The four input widths sum to $32 + 64 + 160 + 256 = 512$.

$$\begin{aligned} \text{projections:} &\quad (32 + 64 + 160 + 256)\, d + 4d = 512d + 4d \\ \text{fuse:} &\quad 4d \cdot d + d = 4d^2 + d \\ \text{refine 1:} &\quad 9d^2 + d \\ \text{refine 2:} &\quad 9 \cdot d \cdot 16K + 16K = 144Kd + 16K. \end{aligned}$$

Collect the d^2 terms $(4 + 9 = 13)$ and the d terms $(512 + 4 + 1 + 1 = 518)$:

$$P(d, K) = 13d^2 + (518 + 144K)\, d + 16K.$$

Check it on the head, $d = 128$, $K = 5$: $13 \times 16{,}384 = 212{,}992$; $(518 + 720) \times 128 = 1{,}238 \times 128 = 158{,}464$; $16 \times 5 = 80$; total $212{,}992 + 158{,}464 + 80 = 371{,}536$. Check it on the reconstruction decoder, $d = 256$, $K = 32$: $13 \times 65{,}536 = 851{,}968$; $(518 + 4{,}608) \times 256 = 5{,}126 \times 256 = 1{,}312{,}256$; $16 \times 32 = 512$; total $851{,}968 + 1{,}312{,}256 + 512 = 2{,}164{,}736$, the figure Chapter 20 derived by hand. The head is $371{,}536/2{,}164{,}736 = 0.17$, about a sixth of the reconstruction decoder. The d^2 term explains the saving: halving d divides the dominant term by four.

28.3 Sigmoid or softmax: one yes/no per class

28.3.1 The sigmoid as log-odds

A probability p of "yes" can be rewritten as *odds*, $p/(1 - p)$: odds of 9 mean "nine to one on". The logarithm of the odds, the *log-odds*, runs over all real numbers: very unlikely events have large negative log-odds, near-certain ones large positive. So a network that outputs any real number z can be read as log-odds:

$$z = \ln \frac{p}{1 - p}.$$

Solve for p. Exponentiate both sides:

$$e^z = \frac{p}{1 - p}.$$

Multiply both sides by $1 - p$ and collect the p terms:

$$e^z - e^z p = p \quad \Longrightarrow \quad e^z = p\,(1 + e^z).$$

Divide by $1 + e^z$, then divide numerator and denominator by e^z:

$$p = \frac{e^z}{1 + e^z} = \frac{1}{1 + e^{-z}} = \sigma(z).$$

Definition 28.1 (Sigmoid)**.** $\sigma(z) = 1/(1 + e^{-z})$ maps a logit z to a probability in $(0, 1)$; $\sigma(0) = 0.5$, and $p > 0.5$ exactly when $z > 0$.

28.3.2 Multi-label is not multi-class

Definition 28.2 (Multi-class and multi-label)**.** A problem is *multi-class* when exactly one of K classes is true for each pixel; it is *multi-label* when any number of them, from none to all, can be true at once.

For multi-class problems the standard output is the *softmax*, which turns K logits into K probabilities that sum to 1 (Chapter 14):

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}.$$

The classes here overlap: thin cloud over water is both. So the head uses K independent sigmoids, one per class, with no link between them. It is five separate yes/no questions asked of the same features.

Worked example 28.2: Why the softmax cannot say "water and thin cloud"

A pixel is thin cloud over water. Give it logits $+2.197$ for water and for thin cloud, and -2.197 for cloud, shadow and snow. Use $e^{2.197} = 9.00$ and $e^{-2.197} = 1/9 = 0.111$ (because $\ln 9 = 2.197$).
Softmax:

$$\begin{aligned} \text{sum} &= 9 + 0.111 + 9 + 0.111 + 0.111 = 18.333 \\ \text{water} &= 9/18.333 = 0.491 \\ \text{thin cloud} &= 9/18.333 = 0.491. \end{aligned}$$

Each must give up half its probability to the other. Sigmoids:

$$\begin{aligned} \text{water} &= 1/(1 + e^{-2.197}) = 1/(1 + 0.111) = 1/1.111 = 0.900 \\ \text{thin cloud} &= 0.900, \text{ independently.} \end{aligned}$$

With a softmax the head could never say "certainly water, and certainly under thin cloud".

The link between the two. A softmax over two logits $(z, 0)$ gives the first class

$$\frac{e^z}{e^z + e^0} = \frac{e^z}{e^z + 1} = \sigma(z).$$

So one sigmoid is a two-class softmax with the second logit fixed at zero, and five sigmoids are five separate two-class problems (class or not-class). There is no "land" or "clear" output: land is water's "no" where the pixel is labelled, and clear sky is cloud's and thin cloud's "no".

28.3.3 The five classes

`app/utils/segmentation/targets.py` defines the classes and how each is read from EnMAP's quality layers:

index	class	target is 1 where	notes
0	water	`label_classes == 2`	1 = land; 3 = no-data
1	cloud	`label_cloud == 1` or `label_classes == 0`	code 0 is EnMAP's cloud class
2	thin_cloud	`label_cirrus >= 2` or `label_haze == 1`	cirrus levels 0–3
3	shadow	`label_cloud_shadow == 1`	
4	snow	`label_snow == 1`	

28.4 Binary cross-entropy from the coin flip

28.4.1 The likelihood of one label

Treat one pixel and one class as a coin flip. The head says the probability of "yes" is p; the label t is 1 (yes) or 0 (no). A *Bernoulli* variable with parameter p takes value 1 with probability p

and 0 with probability $1-p$. Both cases fit in one formula:

$$P(t \mid p) = p^{t}\,(1-p)^{1-t}. \tag{28.1}$$

Check it: for $t=1$, $p^1(1-p)^0 = p \times 1 = p$; for $t=0$, $p^0(1-p)^1 = 1 \times (1-p) = 1-p$.

28.4.2 From likelihood to loss

The *likelihood* of a set of labels is the probability the model gives to all of them at once. Treat the n pixels as independent, so the probabilities multiply:

$$\mathcal{P} = \prod_{i=1}^{n} p_i^{t_i}(1-p_i)^{1-t_i}.$$

A good model makes $\mathcal{P}$ large. Products of many numbers below 1 underflow, so take the logarithm, which turns the product into a sum (because $\ln(ab) = \ln a + \ln b$) and the powers into factors (because $\ln a^t = t \ln a$):

$$\ln \mathcal{P} = \sum_{i=1}^{n} \big[t_i \ln p_i + (1-t_i)\ln(1-p_i)\big].$$

The logarithm is increasing, so maximising $\ln \mathcal{P}$ maximises $\mathcal{P}$. Training minimises, so take the negative, and divide by n so the loss does not grow with the number of pixels:

$$\mathrm{BCE} = -\frac{1}{n}\sum_{i=1}^{n} \big[t_i \ln p_i + (1-t_i)\ln(1-p_i)\big].$$

Definition 28.3 (Binary cross-entropy)**.** For one pixel, $\ell(p,t) = -\big[t \ln p + (1-t)\ln(1-p)\big]$: $-\ln p$ when the label is 1 and $-\ln(1-p)$ when it is 0. BCE is the mean of ℓ over pixels: the negative log-likelihood of the labels under the model, per pixel.

Worked example 28.3: BCE on three pixels

Do this on paper with a calculator for the logarithms.

pixel	t	p	reading
1	1	0.9	confident and right
2	0	0.5	unsure
3	0	0.9	confident and wrong

$$\begin{aligned}
\ell_1 &= -\ln 0.9 = 0.1054\\
\ell_2 &= -\ln(1-0.5) = -\ln 0.5 = 0.6931\\
\ell_3 &= -\ln(1-0.9) = -\ln 0.1 = 2.3026\\
\text{sum} &= 0.1054 + 0.6931 + 2.3026 = 3.1011\\
\mathrm{BCE} &= 3.1011/3 = 1.0337.
\end{aligned}$$

The confident mistake costs 22 times the confident success ($2.3026/0.1054 = 21.8$). As $p \to 1$ on a pixel whose label is 0, $-\ln(1-p) \to \infty$: cross-entropy punishes confident errors without limit.

28.4.3 The gradient is $p - t$

The head outputs logits z, and $p = \sigma(z)$. Training needs $\partial \ell / \partial z$. Two steps and the chain rule (Chapter 26).

Step 1: the sigmoid's derivative. Write $\sigma(z) = (1 + e^{-z})^{-1}$ and differentiate the outer power, then the inside:

$$\frac{d\sigma}{dz} = -1 \cdot (1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Split the fraction into two factors:

$$\frac{d\sigma}{dz} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}}.$$

The first factor is σ. The second is $1 - \sigma$, because $1 - \frac{1}{1+e^{-z}} = \frac{1+e^{-z}-1}{1+e^{-z}} = \frac{e^{-z}}{1+e^{-z}}$. So

$$\frac{d\sigma}{dz} = \sigma(1 - \sigma) = p(1 - p).$$

Step 2: the loss's derivative in p. Differentiate $\ell = -t \ln p - (1 - t) \ln(1 - p)$ term by term (using $\frac{d}{dp} \ln p = 1/p$ and $\frac{d}{dp} \ln(1 - p) = -1/(1 - p)$):

$$\frac{\partial \ell}{\partial p} = -\frac{t}{p} + \frac{1 - t}{1 - p}.$$

Chain rule. Multiply the two:

$$\frac{\partial \ell}{\partial z} = \left(-\frac{t}{p} + \frac{1 - t}{1 - p}\right) p(1 - p)$$

Multiply each term out; the p cancels in the first and the $1 - p$ in the second:

$$= -t(1 - p) + (1 - t)\, p$$

Expand and cancel tp:

$$= -t + tp + p - tp = p - t.$$

> The gradient of binary cross-entropy with respect to the logit is the prediction minus the label, $p - t$: no larger than 1 in size, and zero only when the prediction equals the label.

For the three pixels of worked example 28.3: pixel 1, $0.9 - 1 = -0.1$ (push the logit up a little); pixel 2, $0.5 - 0 = 0.5$ (push down); pixel 3, $0.9 - 0 = 0.9$ (push down hard). The sigmoid's own derivative, $p(1 - p)$, becomes tiny as p nears 0 or 1 ($0.99 \times 0.01 = 0.0099$), yet the loss's gradient does not: the $1/p$ and $1/(1 - p)$ of the logarithm cancel it exactly. This is why sigmoid and cross-entropy are always used together.

Numerically safe form. Computing $\ln \sigma(z)$ for $z = -100$ underflows. PyTorch's `binary_cross_entropy_with_logits`, which `masked_bce_dice` calls, takes the logit directly and uses the equivalent form $\ell = \max(z, 0) - zt + \ln(1 + e^{-|z|})$, which never exponentiates a large positive number. Check it at $z = \ln 9 = 2.197$, $t = 0$: $\max(2.197, 0) - 0 + \ln(1 + e^{-2.197}) = 2.197 + \ln(1.111) = 2.197 + 0.105 = 2.303 = -\ln 0.1$, as in pixel 3.

28.5 Overlap: the Dice coefficient and soft Dice

28.5.1 Dice for sets

Let A be the set of pixels the head calls "cloud" and B the set the label calls "cloud".

Definition 28.4 (Dice coefficient)**.** $D(A,B) = \dfrac{2\,|A \cap B|}{|A| + |B|}$: twice the overlap divided by the two sizes added. $D = 1$ for identical sets, $D = 0$ for disjoint ones.

The factor 2 makes the scale right: if $A = B$ then $|A \cap B| = |A|$ and $D = 2\,|A| / (2\,|A|) = 1$.

28.5.2 Soft Dice: a differentiable version

Set membership is all-or-nothing, so its gradient is zero almost everywhere. Replace each pixel's membership of A by its probability p_i and of B by its label t_i [32]:

$$|A \cap B| \to \sum_i p_i t_i, \qquad |A| \to \sum_i p_i, \qquad |B| \to \sum_i t_i.$$

If the p_i are exactly 0 or 1 these are the set sizes again. The code adds 1 to the top and bottom ("smoothing") so the ratio is defined when a class is absent from both prediction and label, and turns the coefficient into a loss by subtracting it from 1:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{2\sum_i p_i t_i + 1}{\sum_i p_i + \sum_i t_i + 1}. \tag{28.2}$$

If a class is absent from the label and predicted absent, all sums are 0 and the loss is $1 - 1/1 = 0$: correct, with no division by zero.

Worked example 28.4: Soft Dice on a 2×2 mask

Do this on paper. Four pixels, labels and predicted probabilities:

	left	right
top	$t = 1,\ p = 0.8$	$t = 1,\ p = 0.4$
bottom	$t = 0,\ p = 0.3$	$t = 0,\ p = 0.1$

$$\begin{aligned}
\textstyle\sum p_i t_i &= 0.8 \times 1 + 0.4 \times 1 + 0.3 \times 0 + 0.1 \times 0 = 0.8 + 0.4 = 1.2 \\
\textstyle\sum p_i &= 0.8 + 0.4 + 0.3 + 0.1 = 1.6 \\
\textstyle\sum t_i &= 1 + 1 + 0 + 0 = 2 \\
\text{soft Dice, no smoothing} &= \frac{2 \times 1.2}{1.6 + 2} = \frac{2.4}{3.6} = 0.667 \\
\text{with smoothing} &= \frac{2.4 + 1}{3.6 + 1} = \frac{3.4}{4.6} = 0.739 \\
\mathcal{L}_{\text{Dice}} &= 1 - 0.739 = 0.261.
\end{aligned}$$

Compare the hard Dice at threshold 0.5. The head calls only the top-left pixel cloud: $A = \{\text{TL}\}$, $B = \{\text{TL}, \text{TR}\}$, so $|A \cap B| = 1$ and $D = 2 \times 1/(1 + 2) = 2/3 = 0.667$. Here the soft and hard values agree; in general they do not.

28.5.3 The gradient of soft Dice is global

Write $I = \sum p_i t_i$ and $S = \sum p_i + \sum t_i$, so the coefficient is $Q = (2I + 1)/(S + 1)$. Differentiate with respect to one pixel's p_j by the quotient rule ($\partial I/\partial p_j = t_j$, $\partial S/\partial p_j = 1$):

$$\frac{\partial Q}{\partial p_j} = \frac{2t_j\,(S+1) - (2I+1)\cdot 1}{(S+1)^2}.$$

In worked example 28.4, $S + 1 = 4.6$ and $2I + 1 = 3.4$, so $(S + 1)^2 = 21.16$:

$$\begin{aligned} t_j = 1: &\quad \frac{2\times 4.6 - 3.4}{21.16} = \frac{9.2 - 3.4}{21.16} = \frac{5.8}{21.16} = 0.274 \\ t_j = 0: &\quad \frac{0 - 3.4}{21.16} = -0.161. \end{aligned}$$

The loss is $1 - Q$, so its gradient has the opposite sign: raise p on labelled pixels, lower it on the others. Unlike BCE's $p_j - t_j$, each pixel's Dice gradient depends on I and S, sums over the *whole class*. That is what lets Dice see a rare class as a whole.

28.6 Why BCE plus Dice: class imbalance with numbers

28.6.1 The cheapest wrong answer for BCE

Suppose a class covers a fraction ρ of the pixels, and the head ignores the image and predicts one constant probability p everywhere. The BCE is then

$$\mathrm{BCE}(p) = -\big[\rho \ln p + (1-\rho)\ln(1-p)\big],$$

because a share ρ of the terms have $t = 1$ and the rest $t = 0$. Find the best constant: set the derivative to zero,

$$\frac{d\,\mathrm{BCE}}{dp} = -\frac{\rho}{p} + \frac{1-\rho}{1-p} = 0.$$

Multiply both sides by $p(1 - p)$, expand, and cancel ρp:

$$\rho(1-p) = (1-\rho)p \quad\Longrightarrow\quad \rho - \rho p = p - \rho p \quad\Longrightarrow\quad p = \rho.$$

The best constant is the class's base rate, and its loss is $-[\rho \ln \rho + (1-\rho)\ln(1-\rho)]$.

Worked example 28.5: A rare class and a common one under BCE and Dice

Take 1,000 pixels. Shadow covers 10 of them ($\rho = 0.01$), water 300 ($\rho = 0.3$); illustrative shares [A] close to the training data's 0.09% shadow and 6.9% water. Score the constant predictor $p = \rho$.

BCE.

$$\begin{aligned} \text{shadow:}\quad & -(0.01\ln 0.01 + 0.99\ln 0.99) = 0.01\times 4.6052 + 0.99\times 0.01005 \\ & = 0.04605 + 0.00995 = 0.0560 \\ \text{water:}\quad & -(0.3\ln 0.3 + 0.7\ln 0.7) = 0.3\times 1.2040 + 0.7\times 0.3567 \\ & = 0.3612 + 0.2497 = 0.6109. \end{aligned}$$

A head that never finds a single shadow pixel pays only 0.056 on shadow, eleven times less than the same laziness costs on water ($0.6109/0.0560 = 10.9$). Averaged over classes, ignoring shadow is nearly free.

Dice, same constant predictor:

$$\begin{aligned}
\text{shadow:}\quad & \textstyle\sum pt = 10 \times 0.01 = 0.1, \quad \sum p = 1000 \times 0.01 = 10, \quad \sum t = 10 \\
& \mathcal{L}_{\text{Dice}} = 1 - \frac{2 \times 0.1 + 1}{10 + 10 + 1} = 1 - \frac{1.2}{21} = 1 - 0.057 = 0.943 \\
\text{water:}\quad & \textstyle\sum pt = 300 \times 0.3 = 90, \quad \sum p = 300, \quad \sum t = 300 \\
& \mathcal{L}_{\text{Dice}} = 1 - \frac{180 + 1}{600 + 1} = 1 - 0.301 = 0.699.
\end{aligned}$$

Dice charges the lazy shadow predictor 0.943 out of a maximum of 1, more than it charges water. A perfect shadow prediction ($\sum pt = 10$, $\sum p = 10$) scores $1 - 21/21 = 0$. Under Dice, missing a rare class costs the whole term.

So the code adds the two, per class:

$$\mathcal{L}_k = \text{BCE}_k + \mathcal{L}_{\text{Dice},k}, \tag{28.3}$$

BCE for well-behaved per-pixel gradients ($p - t$, bounded), Dice so that every class, however rare, carries a full-sized term. The loss is the mean of $\mathcal{L}_k$ over the classes (Section 28.8 says which classes count).

28.7 Measuring it: intersection over union

Definition 28.5 (IoU)**.** The *intersection over union* (Jaccard index) of a predicted set A and a target set B is $J = |A \cap B| \,/\, |A \cup B|$, with $|A \cup B| = |A| + |B| - |A \cap B|$. A pixel counts as predicted when $p > 0.5$, which is $z > 0$.

IoU and Dice measure the same thing on different scales. Write $I = |A \cap B|$ and $U = |A \cup B| = |A| + |B| - I$. Then $|A| + |B| = U + I$, and

$$D = \frac{2I}{|A| + |B|} = \frac{2I}{U + I}.$$

Divide top and bottom by U:

$$D = \frac{2\,(I/U)}{1 + I/U} = \frac{2J}{1 + J}.$$

So $D \geq J$ always, with equality only at 0 and 1. In worked example 28.4, $J = 1/2$ and $D = 2 \times 0.5/1.5 = 0.667$.

Worked example 28.6: IoU of two pixel sets, and why the tally pools

Target water: pixels $\{1, 2, 3, 4\}$; predicted water: $\{2, 3, 4, 5, 6\}$.

$$\begin{aligned}
\text{intersection} &= \{2, 3, 4\}, \text{ size } 3 \\
\text{union} &= \{1, 2, 3, 4, 5, 6\}, \text{ size } 4 + 5 - 3 = 6 \\
J &= 3/6 = 0.50.
\end{aligned}$$

`IoUTally` adds intersections and unions over the whole validation set, known pixels only, and divides once at the end. Why not average each batch's IoU? Suppose batch 1 holds a

sliver of cloud, $I = 1$, $U = 2$, and batch 2 a large cloud, $I = 90$, $U = 100$:

$$\text{mean of ratios} = (1/2 + 90/100)/2 = (0.5 + 0.9)/2 = 0.70$$
$$\text{pooled} = (1 + 90)/(2 + 100) = 91/102 = 0.892.$$

The mean of ratios lets a two-pixel batch count as much as a hundred-pixel one; the pooled ratio weights every pixel equally.

28.8 Labels you cannot fully trust: the known mask

28.8.1 The problem

EnMAP's labels are the output of the provider's automatic tests, and they are not always right. A clear desert scene checked on 2026-09-30 was labelled 57% cirrus and 14% cloud. [M: `indradhanuworkings/48-checking-labels.md`] Train on that and the head learns "bare desert equals cloud". Dropping every doubtful scene would also throw away its water, shadow and snow labels, which may be fine.

So each target carries a second array: a *known mask*, one per class, saying which pixels' labels count. Unknown means *no opinion*, not *no*.

28.8.2 The rules, from the source

`enmap_targets` in `app/utils/segmentation/targets.py` builds targets and known masks, both `(5, H, W)`:

```
# class code 3 = provider no-data: unknown for every class
known = np.repeat((cls != 3)[None], len(CLASSES), axis=0)
# code 0 (EnMAP's cloud class): only "cloud" is known
known[[0, 2, 3, 4]] &= ~cloud_class
# cirrus level 1: unsure, not clear
known[2] &= ~((cirrus > 0) & (cirrus < policy.cirrus_min_level) & ~haze)
# snow wins over cloud
known[1] &= ~snow
# no water truth under cloud
known[0] &= ~(cloud | (cirrus >= policy.cirrus_min_level))
dry = (... elev >= policy.plateau_elevation_m) or (... wv < policy.low_water_vapour)
if dry:
    known[1] = known[2] = False      # cloud, thin_cloud unknown for the whole scene
```

In words:

1. **Provider no-data** (class code 3): unknown for every class.
2. **Under EnMAP's cloud class** (code 0): cloud is known and true; the other four are unknown.
3. **Cirrus level 1** is unknown for thin cloud, neither positive nor negative (unless haze is set).
4. **Overlaps**: where snow is set, cloud is unknown; where cloud or cirrus ≥ 2 is set, water is unknown.
5. **Dry scenes**: below 0.75 g/cm^2 of water vapour, cloud and thin cloud are unknown for the whole scene; water, shadow and snow still count. An elevation rule (3,500 m and above) is kept as a guard.

The trainer adds two more: a pixel that is nodata in the cube is unknown for every class (`seg_known&mask` in `FinetuneTrainer._seg_batch`), and a tile with less than 40% valid pixels is left out of the segmentation loss (`MIN_VALID_PIXEL_FRACTION=0.4`).

Source check: the cirrus rule

The course writes the cirrus rule as `known[2]&=~((cirrus==1)&~haze)`. The source writes `(cirrus>0)&(cirrus<policy.cirrus_min_level)&~haze` (`targets.py`, line 65). With the default `cirrus_min_level=2` the two are identical, since the only integer level with $0 < c < 2$ is 1; the source form also follows the policy if the threshold is ever changed.

28.8.3 The masked loss

Let $k_{ik} \in \{0, 1\}$ say whether pixel i's label for class k is known, and $n_k = \sum_i k_{ik}$. `masked_bce_dice` computes, for each class with $n_k > 0$,

$$\mathrm{BCE}_k = \frac{1}{n_k} \sum_i k_{ik}\, \ell(p_{ik}, t_{ik}), \tag{28.4}$$

$$\mathcal{L}_{\mathrm{Dice},k} = 1 - \frac{2 \sum_i k_{ik}\, p_{ik}\, t_{ik} + 1}{\sum_i k_{ik}\, p_{ik} + \sum_i k_{ik}\, t_{ik} + 1}, \tag{28.5}$$

and returns the mean of $\mathrm{BCE}_k + \mathcal{L}_{\mathrm{Dice},k}$ over the classes that have at least one known pixel in the batch. A class with no known pixel is *left out* of the average, not counted as zero.

Worked example 28.7: The known-mask loss on a 2×2 tile

A tile from a dry scene, three classes shown (snow and thin cloud have no known pixel here either). Pixel 4 is provider no-data, so it is unknown for every class; on a dry scene cloud is unknown everywhere. Allow five minutes.

		pixel 1	pixel 2	pixel 3	pixel 4
water	target	1	0	0	–
	p	0.9	0.1	0.5	(0.9, ignored)
shadow	target	0	0	0	–
	p	0.1	0.1	0.1	(0.5, ignored)
cloud		unknown in all four pixels			

Water, three known pixels:

$$\text{BCE terms:} \quad -\ln 0.9 = 0.1054, \quad -\ln(1-0.1) = 0.1054, \quad -\ln(1-0.5) = 0.6931$$

$$\mathrm{BCE} = (0.1054 + 0.1054 + 0.6931)/3 = 0.9039/3 = 0.3013$$

$$\textstyle\sum pt = 0.9 \times 1 = 0.9, \quad \sum p = 0.9 + 0.1 + 0.5 = 1.5, \quad \sum t = 1$$

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{2 \times 0.9 + 1}{1.5 + 1 + 1} = 1 - \frac{2.8}{3.5} = 1 - 0.8 = 0.2$$

$$\text{water term} = 0.3013 + 0.2 = 0.5013.$$

Shadow, three known pixels:

$$\mathrm{BCE} = (0.1054 + 0.1054 + 0.1054)/3 = 0.1054$$

$$\textstyle\sum pt = 0, \quad \sum p = 0.1 + 0.1 + 0.1 = 0.3, \quad \sum t = 0$$

$$\mathcal{L}_{\mathrm{Dice}} = 1 - \frac{0 + 1}{0.3 + 0 + 1} = 1 - \frac{1}{1.3} = 1 - 0.7692 = 0.2308$$

$$\text{shadow term} = 0.1054 + 0.2308 = 0.3362.$$

Cloud has no known pixel and drops out. The loss is the mean over the two classes present:

$$(0.5013 + 0.3362)/2 = 0.8375/2 = 0.419.$$

Calling `masked_bce_dice` on these numbers returns 0.4187.
[M: `indradhanuworkings/47-the-segmentation-head.md`] Notice pixel 4: its water probability of 0.9 would have cost $-\ln 0.1 = 2.30$ had it been treated as "not water".



Figure 28.2: The worked example. Hatched cells are unknown and contribute nothing. Cloud, unknown on a dry scene, drops out of the class average; the loss is the mean of water's and shadow's terms.

Source check: the code's Dice and class weights

In `masked_bce_dice` (`targets.py`, lines 76–95) the probabilities are multiplied by the known mask before the sums (`p=torch.sigmoid(logits)*k`), and the denominator is `(p+targets*k).sum(...)`: exactly (28.5). Since FT04 an optional `class_weights` vector turns the class mean into a weighted mean over the classes present, $\sum_{k\in\text{present}} w_k \mathcal{L}_k / \sum_{k\in\text{present}} w_k$ (Chapter 38).

Values that will drift

A smoke test of the probe stage on one EnMAP shard (dev laptop, 2026-10-16) took water IoU from 0 to 0.16, 0.38 and 0.42 over four small epochs: the first evidence that F001's frozen features already separate water. [M: `docs/lld/finetune.md`] The full probe (FT01) reached water 0.82 and cloud 0.80 (Chapter 38).

28.9 Checking labels when there is no ground truth

28.9.1 Why labels need auditing

The known-mask rules above did not come from a textbook. They come from a label audit run on 2026-10-16 over every EnMAP fine-tuning tile: 104 shards, 15,492 tiles, 210 scenes.
[M: `research/experiments/indradhanu-finetune/README.md`] The audit overturned the first draft of `targets.py` in three places: it treated class code 0 as no-data, switched cloud labels off above 3,500 m, and treated cirrus level 1 as "not thin cloud". All three were wrong.

EnMAP's labels are *quality layers*: masks the provider's processor computes for every scene with automatic tests (is this pixel bright, cold, high?). A head trained on them learns their mistakes along with their truths. Nobody has a second, correct set of labels to compare with. So the audit asks a weaker question that can still be answered: **does what is labelled "cloud" look like cloud?**

Analogy

You suspect a thermometer is faulty but own no reference thermometer. You can still check it against physics: if it reads 35 degrees while the pond outside is frozen, the reading is wrong. You cannot say what the right temperature was, but you can find *when* it goes wrong, and if that is always on frosty mornings, you know which readings to distrust. The audit's plausibility tests are the frozen pond; water vapour is the frosty morning.

28.9.2 Test 1: is this "cloud" bright and flat?

A cloud is white, which gives two measurable properties:

- **Bright**: the mean reflectance over 450–650 nm is above 0.2. Clear land in EnMAP averages 0.07 there.
- **Flat**: take the mean reflectance in four narrow windows, 460–500, 540–580, 640–680 and 840–880 nm, and divide the smallest by the largest. White means roughly equal in all four, so the ratio is at least 0.7.

A labelled cloud pixel that passes both is *plausible*.

Worked example 28.8: Four spectra through the cloud test

Illustrative values [A]; do each ratio by hand.

	460–500	540–580	640–680	840–880	min/max	verdict
cloud	0.31	0.30	0.30	0.37	$0.30/0.37 = 0.81$	bright, flat
vegetation	0.04	0.08	0.05	0.40	$0.04/0.40 = 0.10$	dark
bright dry soil	0.18	0.25	0.32	0.38	$0.18/0.38 = 0.47$	bright, not flat
snow	0.55	0.54	0.52	0.48	$0.48/0.55 = 0.87$	bright, flat

Snow passes, because snow is white too. The audit separates it with the *normalised difference snow index*: snow is bright in green and dark near 1600 nm, where cloud stays bright.

$$\mathrm{NDSI} = \frac{\mathrm{green} - \mathrm{SWIR}_{1550\text{–}1680}}{\mathrm{green} + \mathrm{SWIR}_{1550\text{–}1680}}$$

$$\text{snow (SWIR 0.086):}\quad \frac{0.54 - 0.086}{0.54 + 0.086} = \frac{0.454}{0.626} = 0.73 \quad (> 0.4\text{: snow-like})$$

$$\text{cloud (SWIR 0.34):}\quad \frac{0.30 - 0.34}{0.30 + 0.34} = \frac{-0.04}{0.64} = -0.06 \quad \text{(not snow-like).}$$

The SWIR values are the audit's group means: snow 0.086, EnMAP's cloud class 0.339.

These are plausibility tests, not truth. A thin cloud over dark sea can fail "bright" and still be real. The audit uses them to find *where* failures concentrate, not to relabel pixels.

28.9.3 Finding 1: class 0 is the cloud class

EnMAP's land/water map, `label_classes`, has four codes. The shard LLD first described codes 0 and 3 as no-data, and the first `targets.py` followed it. The audit looked at the pixels:

- every one of the 1,414,747 class-0 pixels has `label_cloud=1`;
- they hold 79% of all cloud-labelled pixels;

- their mean visible reflectance is 0.30, against 0.07 for clear land: bright, as cloud is.

Class 0 is EnMAP's cloud class; only class 3 is no-data. Treating class 0 as no-data would have thrown away most of the cloud the head is meant to learn:

$$\begin{aligned} \text{training tiles with at least 1\% cloud:} &\quad 755 \text{ counting class 0}, \quad 269 \text{ without} \\ \text{lost:} &\quad 1 - 269/755 = 1 - 0.356 = 0.644 \approx 64\%. \end{aligned}$$

Source check: the LLDs on class 0 and on cloud_plausible

`docs/lld/enmap-shards.md` (lines 43–44) now reads "classes 0 = cloud, 1 land, 2 water, 3 no-data - class 0 was first read as no-data"; the course's remark that the LLD "calls it no data" is out of date, and the data, the source and the LLD now agree. Separately, `docs/lld/finetune.md` lists `cloud_plausible(refl,wl)` among the interfaces of `app/utils/segmentation/targets.py` ("no caller since FT04"), but no function of that name exists anywhere under `app/` (grep `defcloud_plausible`): the audit's test lives in the dev-laptop scripts, not in the repository. The source wins.

28.9.4 Test 2: is this "cirrus" cloud, or the ground?

Cirrus is thin, high ice cloud. A thin cloud adds scattered light to whatever is below it, and that added light shows most where the ground is dark. Over land the visible is dark (0.07) and the shortwave infrared at 2100–2250 nm brighter (about 0.16), so real thin cloud brightens the visible *more* than the SWIR. Inside one tile the audit compares pixels labelled cirrus with clear-land pixels (at least 200 of each). If the cirrus pixels differ from clear land mainly at 2100–2250 nm, or hardly at all in the visible (less than 0.005), the label follows the ground (brighter, drier soil), not a cloud: it is *surface-driven*.

tile	band	clear land	"cirrus"	change
X	visible	0.070	0.074	+0.004
	SWIR	0.160	0.210	+0.050: SWIR rose more, surface-driven
Y	visible	0.070	0.110	+0.040
	SWIR	0.160	0.175	+0.015: visible rose more, cloud-like

28.9.5 Finding 2: dry air, not altitude

The draft rule switched off cloud labels on scenes above 3,500 m. No scene in the set reaches it: every scene is in India, and the highest scene-mean elevation is 2,776 m. What does predict surface-driven cirrus is the scene's *water vapour*: the amount of water in the air column above each square centimetre of ground.

The unit. If all of it condensed it would form a layer of liquid water. Liquid water has density $1\,\mathrm{g/cm^3}$, so a column of m grams over $1\,\mathrm{cm^2}$ is $m\,\mathrm{cm^3}$ of water, a layer m cm deep:

$$0.75\ \mathrm{g/cm^2} \div 1\ \mathrm{g/cm^3} = 0.75\ \mathrm{cm} = 7.5\ \mathrm{mm}.$$

water vapour (g/cm^2)	scenes	cirrus 1 surface-driven	cirrus $\geq$ 2 surface-driven
below 0.5	3	100%	100%
0.5–0.75	8	90%	100%
0.75–1.0	20	79%	42%
1.0–1.25	33	26%	10%
1.5–2.0	60	14%	8%
2.0–2.5	33	66%	32%
2.5 and above	32	42%	3%

Source: [M: `research/experiments/indradhanu-finetune/README.md`]. Below 0.75 g/cm^2, cirrus labels are almost all the ground. Above 1.0 most are not, though not all, and the 2.0–2.5 row shows the relationship is not tidy. A likely physical reason, consistent with the table: cirrus tests usually rely on a band near 1380 nm, where water vapour normally absorbs so strongly that the ground is invisible and only high cloud shows. In very dry air the ground shows through and looks like cirrus. The high scenes that prompted the draft rule were also dry; altitude was standing in for dryness.

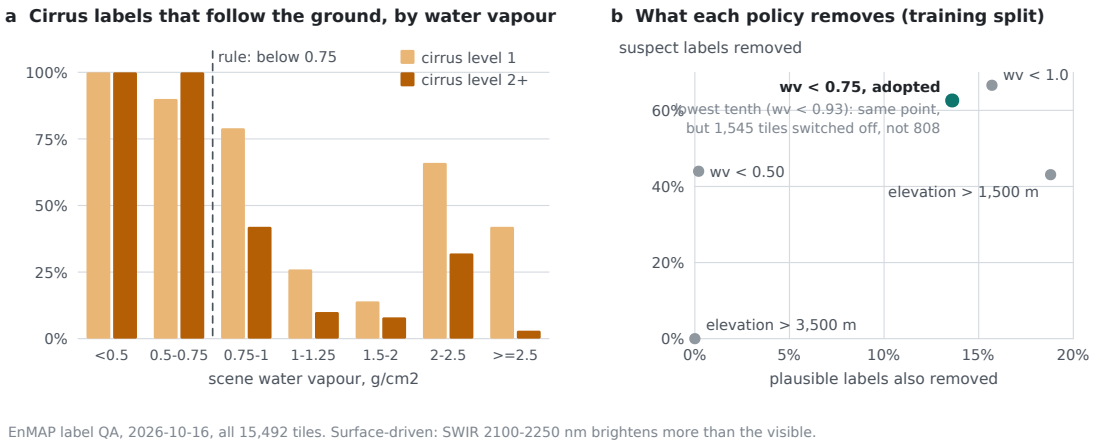


Figure 28.3: (a) Share of cirrus labels that are surface-driven, by the scene's water vapour, for cirrus level 1 and level 2 or more; below 0.75 g/cm^2 (dashed) nearly all are. (b) Candidate policies: share of suspect labels each removes against share of plausible labels it also removes. The adopted rule removes 62.6% of suspect labels for 13.6% of plausible ones.

28.9.6 Choosing the rule by its trade-off

Any rule that removes bad labels also removes some good ones. The audit scored each candidate on the training split by the share of *suspect* labels it removes (surface-driven cirrus, implausible cloud) and the share of *plausible* labels it removes too:

policy	scenes	tiles hit	suspect removed	plausible removed
water vapour below 0.50	3	1.5%	44.0%	0.2%
below 0.75 (adopted)	**11**	**5.6%**	**62.6%**	**13.6%**
below 0.93 (lowest tenth)	21	10.6%	62.6%	13.6%
water vapour below 1.0	30	15.2%	66.6%	15.7%
elevation above 1,500 m	13	6.6%	43.1%	18.8%
elevation above 3,500 m (draft)	0	0%	0%	0%

Read the adopted row as a ratio, suspect labels removed per plausible label removed:

$$\begin{aligned} \text{adopted:} \quad & 62.6/13.6 = 4.6 \\ \text{elevation} > 1{,}500\text{ m:} \quad & 43.1/18.8 = 2.3 \quad \text{(half as selective).} \end{aligned}$$

The lowest-tenth rule (below 0.93) removes exactly the same labels as the fixed 0.75, because the scenes between 0.75 and 0.93 hold no suspect labels to remove, but it switches cloud labels off on 1,545 tiles instead of 808, nearly twice as many. A fixed value wins. The adopted rule touches 11 training scenes and no test scene (the test split's driest scene has $0.94\,\mathrm{g/cm^2}$).

In the code `meta.json` stores `scene_wv` in $\mathrm{g/cm^2}$ times 1000, so the threshold is 750 (`LabelPolicy.low_water_vapour`). A scene with `scene_wv=698` has $698/1000 = 0.698\,\mathrm{g/cm^2} < 0.75$: dry.

> The rule was fixed from the audit before any training and is never tuned on results: a rule tuned on IoU would quietly drop whatever labels the head found hard, the opposite of an audit.

28.9.7 Findings 3 to 5: a cirrus level nobody can read, duplicates, rarity

Cirrus level 1 is neither yes nor no. Over water the audit measured the share of pixels brighter than 0.05 at 1550–1680 nm, where water is black and cloud is not: clear water 3.9%, cirrus level 1 1.45%, cirrus level ≥ 2 89%. Level 1 does not look like cloud, so it cannot be a positive; it is a label that says "maybe", so it cannot be a clean negative. It is unknown for thin cloud.

Snow wins over cloud; no water truth under cloud. 122,815 pixels are labelled both snow and cloud; snow is white, so the cloud test fires on it, and cloud is made unknown there. Pixels labelled water *and* cirrus ≥ 2 have a mean near-infrared reflectance of 0.179, against 0.024 for clear water: land-bright, not water. The water label there probably comes from a map of where water usually is, so water is unknown under cloud.

Four scenes counted twice. Four scenes appear under two processor versions (576 tiles, all in training). A skip list, `research/experiments/indradhanu-finetune/band_lists/enmap_drop_duplicate_scenes.json`, drops them in the loader workers by scene id.

Some classes are rare. After the policy, in the training split: water 6.9% of labelled pixels, thin cloud 1.97%, cloud 0.66%, snow 0.094%, shadow 0.086%. Shadow is about one pixel in $100/0.086 = 1{,}163$. Snow has one tile in the test split (639 pixels), too few to measure an IoU at all. And 53% of shadow pixels have no cloud within 900 m in the same tile: an upper bound, since the cloud may sit in the neighbouring tile, but a reason to treat shadow labels with care too.

28.10 Common confusions

Is there a "clear" or "land" output?
No. Five classes, each yes/no. Land is water's "no" where the pixel is labelled; clear sky is cloud's and thin cloud's "no".

Unknown pixels are counted as negatives, then?
No; that is the whole point of the known mask. They contribute nothing to BCE, to Dice or to IoU, and a class with no known pixel leaves the class average.

Why not use only Dice, since it handles rare classes?
Dice's gradient for each pixel depends on whole-class sums and is unstable when the sums are small; BCE gives every pixel a bounded, local gradient $p - t$. The two together give stable training and full weight to rare classes.

Does adding the head change reconstruction?
Not by itself: an untrained head leaves the reconstruction pass untouched (a test pins this). Training the encoder on the segmentation loss can, which is why every stage is gated on the detection bench (Chapter 29).

The audit says these labels are wrong; why not relabel them?
The tests are plausibility checks, not truth. They find *where* failures concentrate; they cannot say what each pixel really is. "Unknown" is the honest answer.

Does the dry-air rule throw away those scenes?
No. Only cloud and thin cloud become unknown; their water, shadow and snow labels still teach.

Why a whole-scene rule rather than a per-pixel one?
Water vapour is a scene property and the failures follow it. Per-pixel tests would bake the audit's own heuristics into the training targets.

28.11 Check yourself

Exercise 28.1. Use the closed form $P(d, K) = 13d^2 + (518 + 144K)d + 16K$ to compute the head's parameter count for $d = 64$, $K = 5$, and check it layer by layer.

Exercise 28.2. A pixel has logits $+1.0$ for water and thin cloud and -3.0 for the other three. What does the sigmoid give for water ($e^{-1} = 0.368$)? What would a softmax over the five logits give for water ($e^{-3} = 0.0498$, $e^{1} = 2.718$)?

Exercise 28.3. Compute the BCE and its gradient $p - t$ for $t = 1$, $p = 0.2$ and for $t = 0$, $p = 0.2$. Then verify the gradient for the first pixel numerically: its logit is $z = \ln(0.2/0.8) = -1.386$; compute ℓ at $z = -1.376$ and at $z = -1.396$ and divide the difference by 0.02.

Exercise 28.4. Redo the water term of worked example 28.7 with pixel 3's probability at 0.2 instead of 0.5. Then compute the soft Dice loss of worked example 28.4 if the top-right probability rises from 0.4 to 0.9.

Exercise 28.5. Target cloud covers 8 pixels, the prediction 10 pixels, 6 of them correct. Compute IoU and Dice, and check $D = 2J/(1 + J)$.

Exercise 28.6. A labelled cloud pixel has window means 0.22, 0.24, 0.25, 0.36 and visible mean 0.24. Is it plausible? A scene has `scene_wv=812`: are its cloud labels used? Which ratio of suspect to plausible labels does "water vapour below 1.0" achieve, and why was it not chosen?

Next: Chapter 29, Fine-tuning: the curriculum, the BatchNorm see-saw and mixed batches.

Chapter 29

Fine-tuning: the curriculum, the BatchNorm see-saw and mixed batches

The one thing this chapter teaches

Fine-tuning runs as stages (probe the frozen encoder, adapt it to three sensors, then train reconstruction and segmentation together), each opened only by a gate written down in advance. On the way, the model's single BatchNorm turned out to keep a running mean that is an exponential average dominated by the last couple of dozen batches, so sensor-pure batches made evaluation depend on which sensor came last. Mixing every sensor into every batch, one stem pass per band list and one encoder pass for all, fixed it.

29.1 Why a curriculum

F001 (Chapter 27) has only seen SpectralEarth. The product reads PRISMA, EnMAP and AVIRIS-NG at their own bands, and now has a segmentation head (Chapter 28) that has never been trained. The obvious plan is to switch everything on at once: load F001, add the head, feed all three sensors, train every parameter on reconstruction plus segmentation. It fails in three ways.

- **You cannot tell what helped.** If detection moves, was it the new sensors, the new loss, or the new head pulling the encoder?
- **A fresh head wrecks a trained encoder.** A randomly initialised head produces large, meaningless gradients in its first steps, and they flow back into an encoder that took a hundred epochs to train.
- **The new skill can cost the old one.** The product's job is detection, which reads the *reconstruction*. Training the shared encoder hard on segmentation can make it a worse reconstructor.

So the plan (`docs/proposals/fine-tuning-and-segmentation.md`, agreed 2026-10-16) is a *curriculum*: a sequence of stages, each changing one thing, each judged before the next starts. The code is `app/foundation_models/trainers/finetune_trainer.py` with its configuration `SpectralStemFinetuneConfig`; runs live in `research/experiments/indradhanu-finetune/`, one folder per run, each with its pass criteria written before launch. This chapter is about the machinery and its mathematics; what each run showed is told in Chapter 38.

29.2 The stages and their gates

Table 29.1: The fine-tuning curriculum. Stage 4 is optional.

stage	mode	trains	data	gate before the next
0 setup	–	nothing	–	everything exists and is tested; label policy fixed
1 probe (FT01)	`probe`	the segmentation head only	EnMAP	per-class IoU recorded
2 adapt (FT02)	`adapt`	stem, encoder, reconstruction decoder	all three, by share	bench $\geq$ 0.861; test reconstruction falls on every sensor
3 joint (FT03)	`joint`	everything	all three; segmentation loss on EnMAP	bench not below stage 2; IoU above stage 1
4 pseudo-labels	`joint`	as stage 3	plus confident PRISMA and AVIRIS-NG labels	IoU holds; bench holds
5 product	–	–	–	masks replace the threshold rules

Definition 29.1 (Bench and gate)**.** The *bench* is the mean ROC-AUC (Chapter 33) on the four labelled PRISMA scenes under the default score, residual Mahalanobis (Chapter 31). A *gate* is a pass criterion written in a run's README before launch and never moved afterwards. 0.861 is F001 at epoch 60.

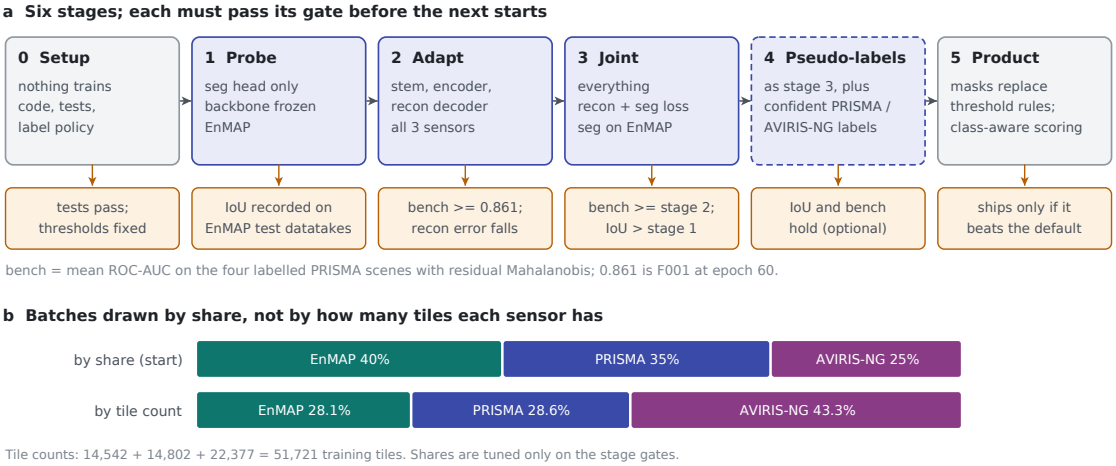


Figure 29.1: (a) The six stages, what each trains, and the gate (amber) it must pass; stage 4 is optional. (b) Batches are drawn by fixed shares (top), not in proportion to each sensor's tile count (bottom), which would give AVIRIS-NG 43% of training.

Why the gate is read at `best.pt`, chosen by another rule. A gate is read at the checkpoint that validation reconstruction picks (Chapter 27), never at whichever epoch scores best on the bench. Suppose a run's true detection ability does not change at all, and four checkpoints read 0.855, 0.858, 0.864 and 0.853 on the four bench scenes, the differences being noise [A]. Picking the best of the four reports 0.864, which passes a 0.861 gate, although nothing improved: the maximum of several noisy readings is biased upwards. Choosing by the bench turns the bench into training data. Chapter 36 measures how large that noise is.

29.3 Starting from F001

Loading with a check. `build_model` loads the pretrained checkpoint into a model that now has a head:

```
missing, unexpected = model.load_state_dict(state, strict=False)
# new_heads = ("seg_head.", "scale_head."): the only tensors allowed to be missing
if unexpected or any(not k.startswith(new_heads) for k in missing):
    raise ValueError(f"init_from does not match the model: ...")
```

Everything except the new heads must match exactly. A configuration that builds the wrong architecture fails here, loudly, instead of training half a model from scratch.

The normalisation curve comes from the checkpoint. Inside the model each band is normalised as $(v - \mu(\lambda))/\sigma(\lambda)$ with the curve computed on SpectralEarth (Chapter 27). The fine-tuning configuration takes that curve from `init_from`'s checkpoint and never recomputes it.

Worked example 29.1: Why the curve is frozen

One band with pretraining mean 0.20 and spread 0.10, and a pixel reading 0.25. Suppose the curve were recomputed on EnMAP and came out at mean 0.22, spread 0.12 (illustrative [A]).

$$\begin{aligned} \text{pretraining curve:} \quad & (0.25 - 0.20)/0.10 = 0.05/0.10 = 0.50 \\ \text{recomputed curve:} \quad & (0.25 - 0.22)/0.12 = 0.03/0.12 = 0.25. \end{aligned}$$

The same pixel would reach the weights as 0.25 instead of the 0.50 they learned to read, and every input in every band would shift at once.

29.4 Parameters, buffers and BatchNorm

29.4.1 Freezing parameters

A *probe* trains a small head on top of frozen features, to measure what the features already know. If water and cloud separate well with the encoder untouched, F001 learned them in pretraining. In probe mode the trainer marks every parameter outside the head as not trainable and gives the optimiser only the head's parameters:

```
for name, p in self.model.named_parameters():
    p.requires_grad = name.startswith("seg_head.")       # only the head learns
self.optimizer = torch.optim.Adam([p for p in self.model.parameters() if p.
    requires_grad],
                                  lr=config.learning_rate)
```

(Adam is the adaptive optimiser of Chapter 26 [19], here without decoupled weight decay.) That is not quite enough, because a model holds two kinds of tensors.

Definition 29.2 (Parameter and buffer)**.** A *parameter* is a tensor the optimiser changes through gradients. A *buffer* is a tensor the model keeps and may change itself, without gradients; `requires_grad` does not touch it.

29.4.2 BatchNorm: two modes

The model has one *batch normalisation* layer [17], `self.norm=nn.BatchNorm2d(D)` with $D = 32$, between the spectral stem and the encoder (`app/foundation_models/components/spectral_stem_seg_former_mae.py`, line 75). Every pixel's 32 stem channels pass through it. The encoder's blocks use LayerNorm (Chapter 16), which normalises each token on its own and keeps no statistics. So this is the model's only layer whose output depends on the *other* samples in a batch.

For one channel, with x the values of that channel over every pixel of every sample in the batch:

Training mode (`model.train()`)
Compute the batch's mean μ_B and variance σ_B^2 over all those values, and output

$$y = \gamma \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} + \beta, \tag{29.1}$$

with learned scale γ and shift β (parameters) and $\epsilon = 10^{-5}$. Also update two buffers, the running mean r and running variance, towards the batch's values:

$$r \leftarrow (1 - m)\, r + m\, \mu_B, \qquad m = 0.1. \tag{29.2}$$

Evaluation mode (`model.eval()`)
Use the stored running mean and variance in place of μ_B and σ_B^2, and update nothing.

`FoundationTrainer._run_epoch` (`app/abstract_classes/foundation_trainer.py`, lines 213 and 228) calls `model.train()` before the training pass and `model.eval()` before validation; inference also runs in evaluation mode. Training and evaluation therefore normalise by *different numbers*. That is harmless when every batch looks like the whole data set.

Source check: the momentum

`nn.BatchNorm2d(D)` is built with no `momentum` argument (line 75), so PyTorch's default $m = 0.1$ applies, with the convention of (29.2): the *new* batch gets weight m. (Some other libraries call $1 - m = 0.9$ the momentum; read the formula, not the name.) `docs/lld/finetune.md` records the same: "PyTorch default momentum 0.1". One detail the courses leave out: PyTorch updates the running *variance* with the batch's unbiased variance (divided by $n - 1$) while normalising the batch itself with the biased one (divided by n); with $8 \times 128 \times 128 = 131{,}072$ values per channel per micro-batch, the difference is a factor 131,072/131,071, negligible.

29.4.3 The running mean is an exponential moving average

Unroll (29.2). Write x_k for the k-th batch's mean and r_0 for the starting value; $0.9 = 1 - m$.

$$\begin{aligned} r_1 &= 0.9\, r_0 + 0.1\, x_1 \\ r_2 &= 0.9\, r_1 + 0.1\, x_2 = 0.9(0.9\, r_0 + 0.1\, x_1) + 0.1\, x_2 = 0.9^2 r_0 + 0.1(0.9\, x_1 + x_2) \\ r_3 &= 0.9\, r_2 + 0.1\, x_3 = 0.9^3 r_0 + 0.1(0.9^2 x_1 + 0.9\, x_2 + x_3). \end{aligned}$$

The pattern: each older batch's weight is multiplied by another 0.9. After n batches

$$r_n = 0.9^n\, r_0 + 0.1 \sum_{k=1}^{n} 0.9^{\,n-k}\, x_k.$$

This is an *exponential moving average*: the newest batch has weight 0.1, the one before 0.09, then 0.081, and so on.

How much weight do the last K batches carry? Add the weights of batches $n, n-1, \ldots, n-K+1$:

$$W_K = 0.1\,(1 + 0.9 + 0.9^2 + \cdots + 0.9^{K-1}).$$

The bracket is a *geometric series*. Check the formula $1 + q + \cdots + q^{K-1} = (1-q^K)/(1-q)$ on a tiny case, $q = 0.9$, $K = 3$: $1 + 0.9 + 0.81 = 2.71$ and $(1 - 0.729)/(1 - 0.9) = 0.271/0.1 = 2.71$. So

$$W_K = 0.1 \cdot \frac{1 - 0.9^K}{1 - 0.9} = 0.1 \cdot \frac{1 - 0.9^K}{0.1} = 1 - 0.9^K.$$

K	0.9^K	weight of the last K batches, $1 - 0.9^K$
1	0.9	0.10
5	0.590	0.41
10	0.349	0.651
22	0.098	0.902

How many batches make 90%? Solve $1 - 0.9^K = 0.9$, that is $0.9^K = 0.1$. Take logarithms: $K \ln 0.9 = \ln 0.1$, so $K = \ln 0.1 / \ln 0.9 = -2.3026/-0.1054 = 21.85$. About 22 batches carry 90% of the stored mean.

Worked example 29.2: A "frozen" encoder that drifts

After pretraining a channel's running mean is 0.00, and EnMAP batches have mean 0.50 (illustrative [A]). The probe freezes every parameter but leaves the model in training mode. With all $x_k = 0.5$ and $r_0 = 0$, the unrolled formula gives $r_n = 0.5\,(1 - 0.9^n)$:

$$\begin{aligned} \text{after 1 batch:} &\quad 0.9 \times 0.00 + 0.1 \times 0.50 = 0.050 \\ \text{after 2 batches:} &\quad 0.9 \times 0.050 + 0.1 \times 0.50 = 0.045 + 0.050 = 0.095 \\ \text{after 10 batches:} &\quad 0.50 \times (1 - 0.349) = 0.50 \times 0.651 = 0.326. \end{aligned}$$

Ten batches in, the "frozen" model normalises its input differently from the one that was pretrained, and the probe measures a drifted encoder.

So the probe puts the whole model in evaluation mode for the training pass and switches only the head back:

```
def _train_pass_mode(self) -> None:
    mode = self.config.model_config_.mode
    # frozen backbone: keep its BatchNorm statistics
    if mode in ("probe", "scale", "decoder"):
        self.model.eval()
        {"probe": self.model.seg_head, ...}[mode].train()
```

It runs after the parent's `model.train()` and before the training pass. Evaluation mode also turns off dropout (`drop_rate` 0.4 in F001), so the probe sees exactly the features inference sees. In probe mode the validation score that ranks epochs is the segmentation loss, since nothing else is trained, and a batch without labels contributes nothing, which is why the probe reads EnMAP alone.

29.5 Mixing sensors by share

FT01 and FT02 drew every batch from one sensor (the stem took one band list per call, Chapter 27). `_MixedLoader` chooses the sensor for each batch at random, weighted by its share:

```
name = self.rng.choices(self.names, self.weights)[0]      # weights e.g. 0.40, 0.35,
    0.25
```

It keeps one iterator per sensor, restarts a sensor whose shards run out, and seeds its generator with the starting epoch, so a resumed run draws a fresh but reproducible sequence.

Why shares, not one shuffled pool. Count the training tiles:
[M: research/experiments/indradhanu-finetune/README.md]

$$\begin{aligned} \text{total} &= 14{,}542 + 14{,}802 + 22{,}377 = 51{,}721 \\ \text{EnMAP} &= 14{,}542/51{,}721 = 0.281 \\ \text{PRISMA} &= 14{,}802/51{,}721 = 0.286 \\ \text{AVIRIS-NG} &= 22{,}377/51{,}721 = 0.433. \end{aligned}$$

A pool would give AVIRIS-NG 43% of training: one airborne campaign, about 200 tiles per flight line, against the product's two satellites. The starting shares are EnMAP 0.40, PRISMA 0.35, AVIRIS-NG 0.25.

How exact are random shares? Each batch is EnMAP with probability 0.40, independently, so over n batches the EnMAP count is a *binomial* variable with mean np and standard deviation $\sqrt{np(1-p)}$. Over 1,000 batches:

$$\begin{aligned} \text{mean} &= 1{,}000 \times 0.40 = 400 \\ \text{standard deviation} &= \sqrt{1{,}000 \times 0.40 \times 0.60} = \sqrt{240} = 15.5. \end{aligned}$$

So about 400 ± 15 EnMAP batches per thousand: shares are honoured to a few percent over an epoch of 3,750 micro-batches.

A validation trap. Validation runs every test source to the end, in order (`_SequentialLoader`), and `test_samples_per_epoch` caps the total. The test sets hold $950 + 1{,}570 + 2{,}270 = 4{,}790$ tiles. A cap below 4,790 cuts off whichever sensor comes last, so its per-sensor score would rest on part of its test set or none of it. FT02 and FT03 set the cap to 5,000. [M: research/experiments/indradhanu-finetune/FT03-joint/config.json]

29.6 The trained-wavelength record

Chapter 27's third band fix drops, at load time, any band more than 15 nm from every wavelength the model has trained on. For F001 that list is SpectralEarth's 202. Fine-tuning trains on PRISMA's own 234 bands, so after it the 20 PRISMA bench bands that F001 dropped are no longer untrained. The model keeps a buffer, `trained_wl`, that starts as the pretraining wavelengths and grows: each time a batch arrives with a band list the trainer has not seen, `extend_trained_wavelengths` adds every wavelength not within 0.5 nm of one already recorded, counting near-duplicates among the new ones once.

Worked example 29.3: Growing the record

Record 500, 510, 520 nm; a new list 500.3, 515.0, 515.2, 530.0.

new	nearest recorded	distance	result
500.3	500	$0.3 \leq 0.5$	already known, skipped
515.0	510 or 520	$5.0 > 0.5$	new
515.2	515.0 (new)	$0.2 \leq 0.5$	counted once with 515.0
530.0	520	$10.0 > 0.5$	new

The record becomes 500, 510, 515, 520, 530, and the call returns 2. `trained_band_mask` then accepts a band within 15 nm of anything recorded: 544 nm is $544 - 530 = 14$ nm away and accepted; 546 nm is 16 nm away and dropped.

The two tolerances do different jobs: 0.5 nm decides "is this the same wavelength?"; 15 nm decides "is this close enough to trained wavelengths for the wavelength networks to read it?". The record grows when a band list is *seen*, not when it is *learned*; "trained on" is the most the code can know.

Values that will drift

During FT02 the record grew from 202 to 733 wavelengths: AVIRIS-NG added 337, PRISMA 177, EnMAP 17 ($202 + 337 + 177 + 17 = 733$).
[M: `research / experiments / indradhanu - finetune / FT02 - adapt / README . md`] On the bench's 185 PRISMA bands, 165 were accepted before fine-tuning and all 185 after.

29.7 The see-saw and the one BatchNorm

29.7.1 The symptom

FT02 adapted F001 on all three sensors with one sensor per micro-batch (micro-batches of 8, gradients accumulated over 4). Its per-sensor validation did not fall smoothly: in epochs 11–12 and 15–17 PRISMA improved while EnMAP and AVIRIS-NG got worse, and then the reverse.

Values that will drift

FT02 (2026-10-16), validation SAM: at epoch 8 (`best.pt`) PRISMA 5.93°, AVIRIS-NG 6.07°; at epoch 17 PRISMA 5.04° (its best) and AVIRIS-NG 8.24° (worse than at epoch 1). [M: `research/experiments/indradhanu-finetune/FT02-adapt/README.md`] The gate was missed: bench 0.855 against 0.861 (Chapter 38).

Validation tiles are fixed and read in the same order every epoch, so the data did not change between epochs: the model did. Two explanations fit a see-saw between sensors: *gradient competition* (each optimiser step leans towards whichever sensors were in it) and *normalisation drift* (something that is not a weight changes with the most recent sensor). This section is about the second, because it was measured.

29.7.2 Training: each sensor normalised to itself

Give one channel three typical values after the stem (illustrative [A]): EnMAP around 0.2, PRISMA around 0.6, AVIRIS-NG around −0.4, each with spread 1. In a PRISMA-only batch, $\mu_B \approx 0.6$, so a typical PRISMA pixel becomes $(0.6 - 0.6)/1 = 0.0$. In an AVIRIS-NG-only batch, a typical AVIRIS-NG pixel becomes $(-0.4 - (-0.4))/1 = 0.0$. **In training the encoder never sees the sensors' offsets**: BatchNorm removes them batch by batch.

29.7.3 Evaluation: one blended estimate

At evaluation the same PRISMA pixel is normalised by the running mean. If that is 0.2, it becomes $(0.6 - 0.2)/1 = 0.4$: an offset the encoder never saw in training. And the running mean is not a fixed 0.2. From Section 29.4, 90% of it comes from the last 22 micro-batches. An FT02 epoch is 30,000 samples in micro-batches of 8, so $30{,}000/8 = 3{,}750$ micro-batches, and the last 22 are $22/3{,}750 = 0.6\%$ of the epoch. Which sensors those were is chance.

Worked example 29.4: The end of an epoch moves the stored mean

The running mean is 0.20 and the last four micro-batches are PRISMA, PRISMA, PRISMA, AVIRIS-NG. Apply $r \leftarrow 0.9\,r + 0.1\,\mu_B$ four times:

$$\begin{aligned}
\text{PRISMA:} \quad & 0.9 \times 0.20 + 0.1 \times 0.6 = 0.18 + 0.06 = 0.24 \\
\text{PRISMA:} \quad & 0.9 \times 0.24 + 0.1 \times 0.6 = 0.216 + 0.06 = 0.276 \\
\text{PRISMA:} \quad & 0.9 \times 0.276 + 0.1 \times 0.6 = 0.2484 + 0.06 = 0.3084 \\
\text{AVIRIS-NG:} \quad & 0.9 \times 0.3084 + 0.1 \times (-0.4) = 0.27756 - 0.04 = 0.23756.
\end{aligned}$$

Four batches moved it from 0.20 to 0.31 and back to 0.24. Validation runs with whatever the last batch left; the next epoch ends on a different sequence.

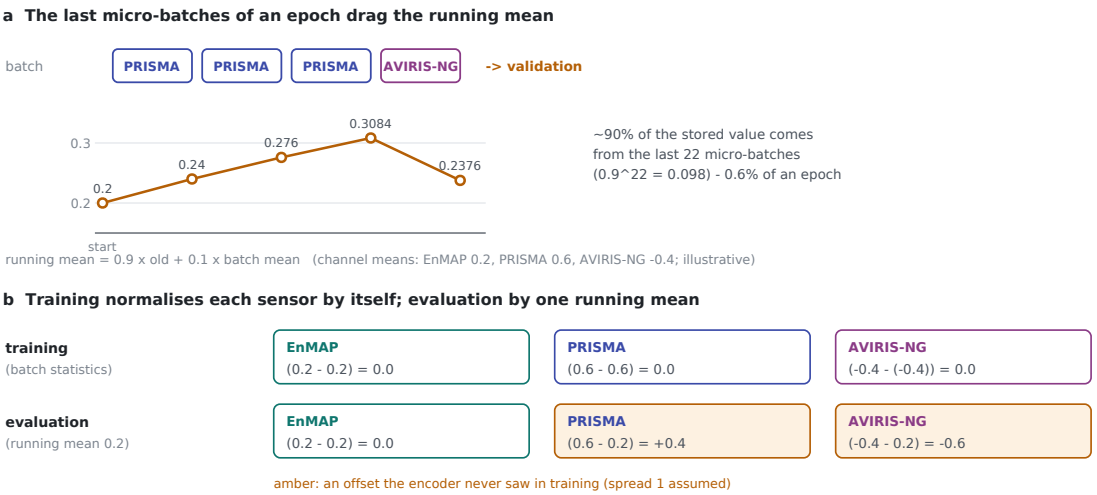


Figure 29.2: (a) The last micro-batches of an epoch, one sensor each, and the running mean they drag around; validation uses wherever it ends. (b) In training each sensor is normalised by its own batch mean (top); at evaluation every sensor is normalised by the one running mean (bottom), so each arrives with an offset the encoder never saw.

Analogy

A teacher marks three classes' exams and rescales each to a class average of 60% before grading, so the marks look comparable. Then the head of school rescales everything once more, using the average of whichever class happened to be marked last. Every class's marks shift, by an amount nobody chose, and the shift changes each term with the marking order. Marking the three classes' papers shuffled together fixes it: one average, used every time. The per-class rescaling is training-mode BatchNorm on sensor-pure batches; the head of school is the running mean; the shuffled pile is a mixed batch.

Gradient accumulation does not help. FT02 accumulated gradients over 4 micro-batches before each optimiser step. That mixes sensors in the *gradient*: a step is single-sensor only if all four micro-batches came from the same sensor. The draws are independent, so the probabilities multiply:

$$\begin{aligned} P(\text{all EnMAP}) &= 0.40^4 = 0.40 \times 0.40 \times 0.40 \times 0.40 = 0.0256 \\ P(\text{all PRISMA}) &= 0.35^4 = 0.1225 \times 0.1225 = 0.0150 \\ P(\text{all AVIRIS-NG}) &= 0.25^4 = 0.0625 \times 0.0625 = 0.0039 \\ P(\text{single-sensor step}) &= 0.0256 + 0.0150 + 0.0039 = 0.0445, \end{aligned}$$

about 4.5% of steps. But BatchNorm's statistics are computed per micro-batch, in the forward pass, before any accumulation. Accumulation cannot mix what BatchNorm sees.

29.7.4 The per-scene test

The FT02 README records a direct test on its epoch-20 checkpoint. The idea, known as *AdaBN* (adaptive batch normalisation) [23], is to replace the stored running statistics by statistics measured on the scene being scored, then score again.

Values that will drift

FT02 epoch 20, 2026-10-16: [M: `research/experiments/indradhanu-finetune/FT02-adapt/README.md`] re-estimating the statistics on each PRISMA bench scene moved them 0.35–0.76 standard deviations. Residual Mahalanobis barely moved (mean 0.853 → 0.858; objects caught 410 → 402); `combined` swung hard (set1 0.943 → 0.755, set4 0.780 → 0.642).

Two readings. The stored statistics are far from any one PRISMA scene's, by up to three quarters of a standard deviation, the size of the offsets in the worked example. And the raw-error score depends on them heavily while residual Mahalanobis barely does: a BatchNorm offset shifts the reconstruction error of a whole scene systematically, and residual Mahalanobis subtracts each pixel's local mean residual and whitens by the scene's covariance, which removes most of a systematic shift. So BatchNorm is consistent with the see-saw but does not explain the gate miss. Per-scene re-estimation is not a product option: `combined` fell, and the product scores one scene at a time.

The options. The FT02 README listed two: keep BatchNorm in evaluation mode during adapt, as the probe does (the statistics stay F001's, from SpectralEarth, matching none of the three sensors), or re-estimate them on a balanced mix before each validation (fixing what evaluation sees but not the training-time habit of normalising each sensor to itself). The user chose a third: **mixed-sensor batches**.

29.8 Mixed batches: three band lists, one encoder pass

29.8.1 The rule that had to bend

The stem reads a pixel's bands as tokens carrying their wavelengths, and its `forward` takes one `(N,)` wavelength list. EnMAP has 224 bands, PRISMA 234, AVIRIS-NG 372: a `(3, 224, H, W)` tensor and a `(3, 234, H, W)` tensor cannot be stacked. But the stem's output is `(B, 32, H, W)` whatever went in. From there on, nothing knows which sensor a pixel came from. So the rule only ever needed to be

> one band list per *stem pass*, not per batch (the invariant of `docs/lld/finetune.md`).

29.8.2 `forward_mixed`

In `app/foundation_models/components/spectral_stem_seg_former_mae.py`:

```
def forward_mixed(self, groups: list, with_scale: bool = False):
    # groups: [(x, wavelengths_nm, band_valid, keep_mask, out_wavelengths_nm), ...]
    if self.stem not in WAVELENGTH_STEMS:
        raise ValueError(f"{self.stem} stem reads one band list only")
    # one stem pass per sensor, outputs stacked: (8, 32, H, W)
    z = torch.cat([self._stem(x, wl, bv) for x, wl, bv, _, _ in groups])
    # the MAE keep masks, stacked in the same order
    keep = None if groups[0][3] is None else torch.cat([g[3] for g in groups])
    z = self.norm(z)                                    # ONE BatchNorm over all sensors
    features = self.encoder(z, keep_mask=keep)          # ONE encoder pass
    z_hat = self._decode(z, features, keep)             # (8, 32, H, W)
    out, start = [], 0
    for x, wl, _, _, out_wl in groups:                  # split by rows, decode at own
    bands
        wl_out = wl if out_wl is None else out_wl
        part = self.spectral_out(z_hat[start:start + len(x)], wl_out)   # (B_g, M_g,
    H, W)
        out.append(self.stats.denormalize(part, wl_out))
        start += len(x)
    return out
```

Source check: the listing is the current source

The course (IW52) shows the first version, `z_hat=self.decoder(self.encoder(self.norm(z),keep_mask=keep))`. The source has since gained `_decode` (for the progressive decoder of FT09, which also reads the normalised stem output) and an optional `with_scale` output (the uncertainty head of FT06); the listing above follows the source (lines 159–180), with the `with_scale` return omitted. For the SegFormer decoder, `_decode` is exactly `self.decoder(features)`, so the two versions compute the same thing.

Table 29.2: One mixed batch of 8 tiles of 128 × 128 through `forward_mixed`, without band augmentation.

step	EnMAP	PRISMA	AVIRIS-NG
input	`(3, 224, 128, 128)`	`(3, 234, 128, 128)`	`(2, 372, 128, 128)`
`_stem`	`(3, 32, 128, 128)`	`(3, 32, 128, 128)`	`(2, 32, 128, 128)`
`torch.cat`		`(8, 32, 128, 128)`	
norm → encoder → decoder		`(8, 32, 128, 128)`	
slice rows	0–2	3–5	6–7
`spectral_out`	`(3, 224, 128, 128)`	`(3, 234, 128, 128)`	`(2, 372, 128, 128)`

The fixed-band stems (`linear`, `gate`) are built for one band count, so `forward_mixed` refuses them. The keep masks are concatenated in the same order as the stem outputs, so row i of the batch is masked with sample i's own mask.

29.8.3 Where the batch comes from: rounded sub-batch sizes

With `mixed_batches: true`, each sensor gets its own data loader with batch size $\max(1, \text{round}(s \times 8))$ for share s, and `_AllSourcesLoader` yields one sub-batch from every

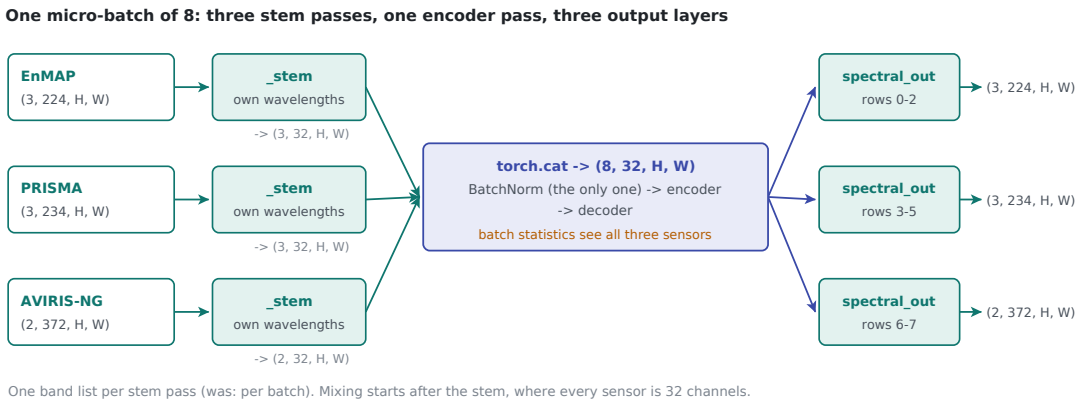


Figure 29.3: Three sensors, three stem passes (teal), one concatenated batch through the single BatchNorm, encoder and decoder (indigo), split by row ranges and decoded at each sensor's own wavelengths.

sensor per step:

$$\begin{aligned} \text{EnMAP:} &\quad \text{round}(0.40 \times 8) = \text{round}(3.2) = 3 \\ \text{PRISMA:} &\quad \text{round}(0.35 \times 8) = \text{round}(2.8) = 3 \\ \text{AVIRIS-NG:} &\quad \text{round}(0.25 \times 8) = \text{round}(2.0) = 2 \\ \text{total:} &\quad 3 + 3 + 2 = 8. \end{aligned}$$

The effective shares are $3/8, 3/8, 2/8 = 0.375, 0.375, 0.25$, not 0.40, 0.35, 0.25: PRISMA gains 2.5 points and EnMAP loses 2.5. Nothing forces the rounded sizes to sum to the batch size for other shares; at 8 with these shares they happen to.

29.8.4 The loss: every tile counts equally

`_mixed_loss` builds, per sensor, that sensor's MAE mask and its own band-augmentation draw, makes one `forward_mixed` call, and combines the per-sensor losses weighted by kept samples n_g:

```
total = total + (l1 + self._get_sam_weight(self._current_epoch) * sam + edge) * n
kept = kept + n                  # (one line in the source)
...
loss = total / kept
```

Why weight by n_g? If L_g is the mean loss over sensor g's n_g tiles, then $n_g L_g$ is the *sum* over those tiles, and

$$\frac{\sum_g n_g L_g}{\sum_g n_g} = \frac{\text{sum of every tile's loss}}{\text{number of tiles}},$$

the plain mean over all tiles in the batch: every tile counts equally, whichever sensor it is from. With $n = (3, 3, 2)$ and per-sensor losses 0.05, 0.06, 0.08 (illustrative [A]): $(3 \times 0.05 + 3 \times 0.06 + 2 \times 0.08)/8 = (0.15 + 0.18 + 0.16)/8 = 0.49/8 = 0.0613$, whereas an unweighted mean of the three would give $(0.05 + 0.06 + 0.08)/3 = 0.0633$.

29.8.5 Why this fixes two things at once

BatchNorm. Every micro-batch now holds all three sensors. Training normalises by the mix's statistics; the running statistics are averages of mixes; evaluation uses those. The offsets of Section 29.7 are no longer removed in training and reintroduced at evaluation: they are present in both.

Gradient balance. Every optimiser step now contains $3 \times 4 = 12$ EnMAP, 12 PRISMA and $2 \times 4 = 8$ AVIRIS-NG tiles.

Worked example 29.5: Alternating sensors never settle; mixed batches do

One weight w. Sensor A wants $w = 1$ (loss $(w-1)^2$, gradient $2(w-1)$); sensor B wants $w = -1$ (loss $(w+1)^2$, gradient $2(w+1)$). Learning rate 0.25.
Alternating, one sensor per step, from $w = 0$. Write each step as a map first:

$$\begin{aligned} \text{A step:} \quad & w \leftarrow w - 0.25 \times 2(w-1) = w - 0.5w + 0.5 = 0.5w + 0.5 \\ \text{B step:} \quad & w \leftarrow w - 0.25 \times 2(w+1) = w - 0.5w - 0.5 = 0.5w - 0.5. \end{aligned}$$

Then run it:

$$\begin{aligned} \text{A:} \quad & 0.5 \times 0 + 0.5 = 0.5 \\ \text{B:} \quad & 0.5 \times 0.5 - 0.5 = -0.25 \\ \text{A:} \quad & 0.5 \times (-0.25) + 0.5 = 0.375 \\ \text{B:} \quad & 0.5 \times 0.375 - 0.5 = -0.3125. \end{aligned}$$

Where does it end? Compose the two maps: an A step then a B step takes w to $0.5(0.5w + 0.5) - 0.5 = 0.25w + 0.25 - 0.5 = 0.25w - 0.25$. A value that this pair leaves unchanged satisfies $w = 0.25w - 0.25$, so $0.75w = -0.25$ and $w = -1/3$ after each B step; after each A step it is $0.5 \times (-1/3) + 0.5 = 1/3$. The weight swings for ever between $+1/3$ and $-1/3$, and where you read it depends on which sensor went last.
Mixed, both losses averaged in every step. The gradient of $\frac{1}{2}[(w-1)^2 + (w+1)^2]$ is $\frac{1}{2}[2(w-1) + 2(w+1)] = 2w$, so

$$w \leftarrow w - 0.25 \times 2w = 0.5w.$$

From $w = 0.5$: 0.25, 0.125, 0.0625, …→ 0. It settles at the compromise, 0, and stays there.

Real training is not this extreme (FT02's accumulation already mixed most steps), so the BatchNorm effect is expected to be the larger of the two.

29.8.6 What else changed when the next run started

FT02b was "FT02's config plus `mixed_batches: true`", from FT02's epoch-8 weights. But `init_from` loads *weights only*, and the epoch counter restarts at 0. So the optimiser state is fresh, the learning rate returns to 0.0001 on a new cosine, and the SAM ramp restarts. At FT02's epoch 8 (index 7 of a 20-epoch cosine):

$$\begin{aligned} \eta &= 0.0001 \times \frac{1 + \cos(7\pi/20)}{2} = 0.0001 \times \frac{1 + 0.454}{2} = 0.0001 \times 0.727 = 0.0000727, \\ \lambda &= 0.5 \times \min(1, 7/10) = 0.35, \end{aligned}$$

against FT02b's first epoch at $\eta = 0.0001$ and $\lambda = 0.5 \times 0/10 = 0$. If FT02b moves, mixed batches are not the only change.

Source check: warm-up is never read

FT02's and FT02b's configs set `lr_schedule.warmup_epochs: 1` and their READMEs say "1 warm-up epoch". `FoundationTrainer._build_scheduler` (`app/abstract_classes/foundation_trainer.py`, line 517) builds a plain `CosineAnnealingLR` stepped once per epoch, and no code reads `warmup_epochs`. There is no warm-up; the first epoch runs at the

full rate. `docs/lld/finetune.md` now says so.

Values that will drift

FT02b (2026-10-16, 2.2 hours, `best.pt` epoch 9): validation SAM 3.99 / 4.37 / 5.57° (EnMAP / PRISMA / AVIRIS-NG) against FT02's 4.17 / 5.93 / 6.07; most of PRISMA's drop arrived in the first epoch (4.70). The see-saw was gone by the README's account; the bench gate was missed again, 0.856 against 0.861.
[M:`research/experiments/indradhanu-finetune/FT02b-mixed/README.md`] Chapter 38 reads the run.

29.9 Joint training: reconstruction and segmentation together

29.9.1 The loss

Joint mode adds one term to the mixed reconstruction loss (`_mixed_loss`):

```
loss = total / kept                          # reconstruction: L1 + SAM weight x SAM, per
     tile
if cfg.mode == "joint":
    seg = self._mixed_seg_loss(groups, model)
    if seg is not None:
        loss = loss + cfg.seg_weight * seg   # seg_weight x masked BCE + Dice
```

In symbols, with w_s the segmentation weight:

$$\mathcal{L} = \underbrace{\frac{\sum_g n_g (L1_g + \lambda\ \mathrm{SAM}_g)}{\sum_g n_g}}_{\text{reconstruction, all tiles}} + w_s \underbrace{\frac{1}{|\mathcal{G}_\ell|} \sum_{g \in \mathcal{G}_\ell} \mathcal{L}_g^{\text{seg}}}_{\text{labelled groups only}}, \tag{29.3}$$

where $\mathcal{G}_\ell$ is the set of sensor groups that carry labels and $\mathcal{L}_g^{\text{seg}}$ is `masked_bce_dice` (Chapter 28) on group g. In FT03 only EnMAP carries labels, so the second term is EnMAP's segmentation loss. The code's `_mixed_seg_loss` returns `sum(losses)/len(losses)` over the labelled groups.

29.9.2 Choice 1: the segmentation pass sees the whole mixed batch

Segmentation needs a second forward pass: the reconstruction pass hides 65% of the tokens, and a mask predictor must see the whole tile. That second pass is a danger, because in training mode BatchNorm normalises with *and records* the statistics of every forward call. Suppose the segmentation pass held only the 3 EnMAP tiles. Every step would then run one mixed pass and one EnMAP-only pass.

Worked example 29.6: Where the running mean settles with an EnMAP-only second pass

Mixed batches have mean $a = 0.2$, EnMAP-only batches $b = 0.5$ (illustrative [A]); the running mean starts at 0.2.

$$\begin{aligned}
\text{step 1, mixed:} &\quad 0.9 \times 0.200 + 0.1 \times 0.2 = 0.180 + 0.020 = 0.200 \\
\text{step 1, EnMAP-only:} &\quad 0.9 \times 0.200 + 0.1 \times 0.5 = 0.180 + 0.050 = 0.230 \\
\text{step 2, mixed:} &\quad 0.9 \times 0.230 + 0.1 \times 0.2 = 0.207 + 0.020 = 0.227 \\
\text{step 2, EnMAP-only:} &\quad 0.9 \times 0.227 + 0.1 \times 0.5 = 0.2043 + 0.050 = 0.2543.
\end{aligned}$$

It keeps climbing. Call the settled value (right after an EnMAP-only pass) r^*; one full step must leave it unchanged:

$$\begin{aligned} \text{after the mixed pass:} \quad & 0.9\,r^* + 0.1\,a \\ \text{after the EnMAP-only pass:} \quad & 0.9\,(0.9\,r^* + 0.1\,a) + 0.1\,b = 0.81\,r^* + 0.09\,a + 0.1\,b. \end{aligned}$$

Set this equal to r^* and solve:

$$\begin{aligned} r^* &= 0.81\,r^* + 0.09\,a + 0.1\,b \\ 0.19\,r^* &= 0.09\,a + 0.1\,b \\ r^* &= \frac{0.09\,a + 0.1\,b}{0.19} = \frac{0.09 \times 0.2 + 0.1 \times 0.5}{0.19} \\ &= \frac{0.018 + 0.05}{0.19} = \frac{0.068}{0.19} = 0.358. \end{aligned}$$

The weights $0.09/0.19 = 0.47$ on the mix and $0.1/0.19 = 0.53$ on EnMAP alone show that the stored mean would settle about halfway between them, matching none of the sensors: the fault FT02b had removed one run earlier.

So the segmentation pass runs over all eight tiles, through `segment_mixed`:

```
def segment_mixed(self, groups: list) -> list:
    # groups: [(x, wavelengths_nm, band_valid), ...] - one per sensor
    z = torch.cat([self._stem(x, wl, bv) for x, wl, bv in groups])   # (8, 32, H, W)
    logits = self.seg_head(self.encoder(self.norm(z)))   # (8, 5, H, W), no keep mask
    # split back by sensor: (3, 5, H, W), (3, 5, H, W), (2, 5, H, W)
    return list(torch.split(logits, [len(x) for x, _, _ in groups]))
```

It is `forward_mixed` without a keep mask and with the segmentation head in place of the decoder. The PRISMA and AVIRIS-NG rows get logits that no loss reads; they are in the pass so that BatchNorm normalises, and records, the same mix as the reconstruction pass. With both passes at mean a, the fixed point is $r^* = (0.09a + 0.1a)/0.19 = a$: the stored mean is the mix's.

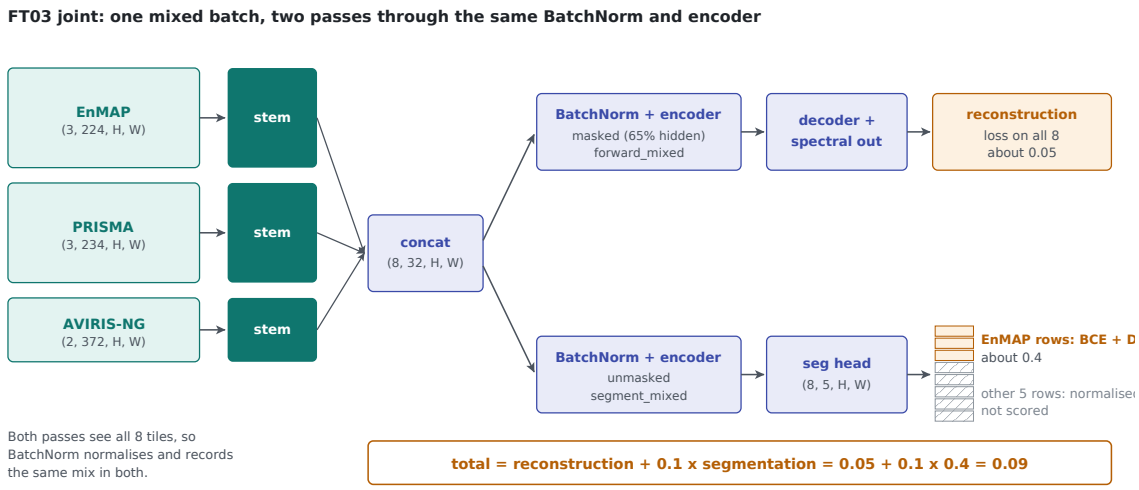


Figure 29.4: One mixed batch of 8 tiles goes through the shared BatchNorm and encoder twice: masked, to the decoder, for the reconstruction loss on all 8; unmasked, to the segmentation head, for the segmentation loss on the 3 EnMAP tiles only (the other 5 rows are normalised but not scored). Total loss = reconstruction $+0.1\times$ segmentation.

The argument-order trap. `_seg_inputs` returns `(pixels, band_valid, wavelengths, targets, known)`; `segment_mixed` wants `(x, wavelengths, band_valid)`. The call reorders them:

```
logits = model.segment_mixed([(p[0], p[2], p[1]) for p in parts])   # (x, wl,
    band_valid)
```

Pass `p[:3]` instead and the stem receives a `(B, N, H, W)` validity mask where it expects an `(N,)` list of wavelengths. Two neighbouring functions that order the same three arguments differently is a trap; a short smoke run on real batches is cheaper than a failed three-hour run.

Source check: the validator's stale name

`SpectralStemFinetuneConfig._mixed_batches_in_adapt_only` (`app/models/training/training_config.py`) now rejects only `probe`, with the message "mixed_batches is implemented for modes 'adapt' and 'joint'" (line 231); its name was not changed when joint mode gained `segment_mixed`. The course (IW55) adds that `docs/lld/finetune.md` "still says mode adapt only in four places"; the LLD has since been updated ("mixed_batches: adapt or joint"). The source wins, and today the LLD agrees with it.

29.9.3 Choice 2: `seg_weight` 0.1, not 1.0

FT03's README: "laptop smoke: segmentation loss ~0.4 against reconstruction ~0.05 (L1 + 0.5 SAM)". [M: `research/experiments/indradhanu-finetune/FT03-joint/README.md`] Where does 0.05 come from? Per-sensor validation in FT02b had L1 about 0.016 and SAM about 4.2°. `SAMLoss` returns radians:

$$4.2^\circ = 4.2 \times 3.1416/180 = 0.0733 \text{ rad}$$
$$\text{reconstruction} = 0.016 + 0.5 \times 0.0733 = 0.016 + 0.0367 = 0.0527.$$

Now the combined loss (29.3) at two weights, with segmentation 0.4:

$$w_s = 1.0: \quad 0.05 + 1.0 \times 0.4 = 0.45; \quad \text{segmentation share } 0.40/0.45 = 89\%$$
$$w_s = 0.1: \quad 0.05 + 0.1 \times 0.4 = 0.09; \quad \text{segmentation share } 0.04/0.09 = 44\%.$$

At 1.0 the shared encoder would be trained mostly to segment EnMAP, and reconstruction, which is what detection reads, would become a side task. At 0.1 the two are comparable, as the proposal's "MAE dominant" intends. A share of the loss *value* is a rough guide, not the share of the gradient (two terms of equal size can have gradients of very different size), but an order-of-magnitude imbalance is what it catches.

29.9.4 Choice 3: the probe's head as a warm start

`seg_head_from` names a checkpoint whose `seg_head.*` weights replace the fresh head after `init_from` has loaded the backbone. FT03 took FT01's probe head. The plan had said "the probe is a measurement, not a warm start"; it became both. The FT03 README is careful about what that buys: FT01 "learned on F001 features and the encoder has moved since, so it is a warm start, not a fit".

Values that will drift

FT03 (2026-10-16, 3.0 hours on the L40S, 10 epochs, `sam_ramp_epochs:0` so the restarted epoch counter does not train epoch 1 on L1 alone): EnMAP test IoU water 0.841 and cloud 0.815 at `best.pt` (epoch 10), above FT01's 0.82 / 0.80; per-sensor SAM 3.92 / 4.21 / 5.48°,

within 0.1° of its predecessor; detection unchanged within noise. All three criteria passed. [M: `research/experiments/indradhanu-finetune/FT03-joint/README.md`] Chapter 38 reads the run in full.

Why joint training did not lift detection, and was not expected to. The detection score is residual Mahalanobis on the *reconstruction*. The segmentation head is a separate output; nothing in the score reads it. Joint training could only change detection by changing the encoder, and at $w_s = 0.1$ the encoder kept reconstructing as before.

Source check: no gradual unfreezing

The proposal's first draft said stage 3 trains with the "encoder unfrozen from the top stage down" (gradual unfreezing). `finetune_trainer.py` has no such mechanism: `joint` trains every parameter from the first step. The source won, and the proposal was corrected ("gradual top-down unfreezing is not implemented").

29.10 How segmentation was meant to help detection

Stage 5 is where the head pays back. Its masks can replace `scene_segmentation`'s thresholds, removing cloud edges and shadows before scoring. A second idea was *class-aware* residual Mahalanobis: give each surface class its own normal. The mathematics behind it is the *law of total variance*.

Let a residual X (one whitened direction) come from land with probability π_1 and from sea with probability π_2, with class means μ_c and variances σ_c^2. The pooled mean is $\mu = \sum_c \pi_c \mu_c$. The pooled variance is the mean squared distance from μ; split each class's distances around its own mean:

$$\mathrm{Var}(X) = \sum_c \pi_c \, \mathbb{E}\big[(X-\mu)^2 \mid c\big] = \sum_c \pi_c \, \mathbb{E}\big[\big((X-\mu_c) + (\mu_c-\mu)\big)^2 \mid c\big].$$

Expand the square. The cross term $2(\mu_c - \mu)\mathbb{E}[X - \mu_c \mid c]$ is zero, because a class's values average to its own mean:

$$\mathrm{Var}(X) = \underbrace{\sum_c \pi_c \, \sigma_c^2}_{\text{within classes}} + \underbrace{\sum_c \pi_c \, (\mu_c-\mu)^2}_{\text{between classes}}.$$

Worked example 29.7: Pooled against per-class scoring on a coastal scene

A set1-like scene, 75% sea (illustrative [A]). Land residuals centre on 0 with variance 1; sea residuals centre on 2 with variance 0.25.

$$\begin{aligned}
\mu &= 0.25 \times 0 + 0.75 \times 2 = 1.5 \\
\text{within} &= 0.25 \times 1 + 0.75 \times 0.25 = 0.25 + 0.1875 = 0.4375 \\
\text{between} &= 0.25 \times (0-1.5)^2 + 0.75 \times (2-1.5)^2 \\
&= 0.25 \times 2.25 + 0.75 \times 0.25 = 0.5625 + 0.1875 = 0.75 \\
\text{total} &= 0.4375 + 0.75 = 1.1875.
\end{aligned}$$

Two pixels with residual 3.0: a land target, and an ordinary sea pixel two spreads above the

sea's mean. Score each by squared distance over variance:

$$\begin{aligned} \text{pooled, either pixel:} &\quad (3.0-1.5)^2/1.1875 = 2.25/1.1875 = 1.89 \\ \text{per class, land target:} &\quad (3.0-0)^2/1 = 9.0 \\ \text{per class, sea pixel:} &\quad (3.0-2)^2/0.25 = 1/0.25 = 4.0. \end{aligned}$$

Pooled, they are indistinguishable; per class, the target scores more than twice the sea pixel.

Values that will drift

Class-aware scoring was tested and not shipped: water's tight statistics made water noise dominate (set1 water share of false alarms 14% → 92%). Exclusion of cloud by the head's masks was chosen instead (water was kept, since vessels are in scope), lifting FT03's real-bench pAUC from 0.158 to 0.166 (2026-10-17/18, Chapters 38 and 39).
[M: `docs/proposals/fine-tuning-and-segmentation.md`]

29.11 Common confusions

Does `requires_grad = False` freeze a layer?
It freezes its *parameters*. BatchNorm's running statistics are buffers and keep updating in training mode; only `eval()` stops them.

Evaluation mode turns BatchNorm off?
It turns its *updating* off and switches it to the stored statistics. It still normalises.

BatchNorm is in every SegFormer block?
Not in this model. The blocks use LayerNorm. The one BatchNorm is `self.norm`, in front of the encoder.

Accumulating 4 micro-batches of 8 means BatchNorm sees 32?
It sees 8. Accumulation adds gradients after four separate forward passes.

Mixed batches mean the stem reads three band lists at once?
It is still called once per band list. Mixing starts after the stem.

Should validation be mixed too?
It does not need to be. Validation runs in evaluation mode, so BatchNorm uses its stored statistics, not the batch's; a sensor-pure validation batch is normalised exactly as a mixed one would be.

Is the normalisation curve updated for the new sensors?
No, never. The record of trained wavelengths grows; the curve that normalises inputs does not.

Joint means PRISMA and AVIRIS-NG are trained to segment too?
In FT03 they pass through the segmentation pass but no loss reads their logits; they are there for BatchNorm. (From FT04 they are taught from confident pseudo-labels, Chapter 38.)

The see-saw is proven to be BatchNorm?
It is consistent with it. Gradient competition would also produce a see-saw; mixed batches address both, so one run cannot tell them apart.

29.12 Check yourself

Exercise 29.1. A BatchNorm channel's running mean is 0.10 and training-mode batches all have mean 0.30. Compute it after each of 3 batches, then check the third value with the closed form $r_n = 0.9^n r_0 + (1 - 0.9^n)\, x$.

Exercise 29.2. What fraction of the stored mean comes from the last 5 micro-batches, and how many micro-batches carry 95% of it? ($\ln 0.05 = -2.996$, $\ln 0.9 = -0.1054$.)

Exercise 29.3. The record holds 1000 and 1010 nm. Extend it with 999.8, 1004.6, 1005.0, 1030.0. What is the record, what does the call return, and is a band at 1018 nm then accepted?

Exercise 29.4. With `batch_size` 4 and shares 0.40 / 0.35 / 0.25, what are the sub-batch sizes and the effective shares? In the toy problem of worked example 29.5, start at $w = 1$ and alternate B, A, B.

Exercise 29.5. Redo the running-mean fixed point of worked example 29.6 with mixed mean 0.1 and EnMAP-only mean 0.6. What is it with `segment_mixed`?

Exercise 29.6. A sensor's validation L1 is 0.015 and its SAM 4.5°. Compute the reconstruction loss at SAM weight 0.5. With segmentation loss 0.3 and reconstruction 0.06, what share of the loss is segmentation at $w_s = 1.0$, 0.1 and 0.2?

Next: Chapter 30, Inference on a whole scene.

Part VI

From model to answer

Chapter 30

Inference on a whole scene

The one thing this chapter teaches

To score a real scene, the model runs twice on every tile, with two complementary masks, so that no pixel is ever predicted while the model can see it; and it runs over hundreds of overlapping tiles, whose predictions are summed and divided by a count, because a scene is far larger than the 128-pixel patch the model was trained on.

Part V trained a model. You now have a checkpoint: a file of numbers that, loaded into the architecture of Part III, turns a masked 128×128 patch into a reconstruction. This chapter starts Part VI, which turns that ability into an answer an analyst can use. The route has three stages, and this book gives each its own chapter:

1. *Inference* (this chapter): run the model over a whole scene and obtain a full-size reconstruction in which every pixel was predicted without being seen.
2. *Scoring* (Chapter 31): turn the difference between the scene and its reconstruction into one number per pixel.
3. *Deciding and naming* (Chapter 32): a person picks a threshold, the flagged pixels are matched against a library of materials, and the result leaves the system as a georeferenced bundle.

At the end of this chapter you will also walk one real scene through the product's whole chain of actions, from upload to shortlist, so that you can see where each later chapter fits.

Two source files hold the machinery: `app/foundation_models/inferencers/segformer_mae_inferencer.py` (324 lines), which the hyperspectral model inherits unchanged, and `app/foundation_models/inferencers/hyperspectral_segformer_mae_inferencer.py` (105 lines), which only overrides how the model is built and adds one scoring method. Read them beside this chapter.

30.1 Two problems inference has that training does not

Problem 1: every pixel needs a prediction made while it was hidden. Recall the rule of Chapter 9: a pixel must never help reconstruct itself, or the model copies it and the residual is zero exactly where you hoped to see an anomaly. Training enforces the rule by hiding a random fraction of the tokens and grading the model only there (Chapter 18). In training that is enough: over thousands of patches, every position is hidden many times. At inference you have one particular scene and need a prediction at *every* pixel of it, each made while that pixel was hidden.

The obvious fix fails in both directions. Run the model once with no mask and every pixel is visible, so the model copies it. Run it once with a random mask and only the hidden fraction of pixels gets an honest prediction; the rest have none.

Problem 2: scenes are enormous. The model was trained on 128×128 patches. A PRISMA scene is about 1210×1219 pixels [M: `docs/lld/frequency-destriper.md`]. Again the obvious fixes fail:

- *Shrink the scene to* 128×128. You would average about 9.5×9.5 pixels into one, and a one-pixel target would vanish before the model ever saw it.
- *Feed the whole scene in at once.* The encoder's four stages were trained at one scale; a 1210×1219 input gives a token count at each stage about 90 times larger than anything seen in training, and the memory grows with it.
- *Cut the scene into non-overlapping tiles.* Every tile boundary becomes a seam: a pixel at the edge of a tile is predicted from context on one side only, so its residual is systematically worse, and the score map shows a grid.

The solutions are *two-pass masking* for the first problem and an *overlapping sliding window* for the second. Before either, the model has to be loaded.

30.2 Loading the model

The base class `FoundationInferencer` does three things in its constructor:

```
self.model = self.build_model().to(self.device)   # 1. build the architecture, empty
self._load_weights(config.checkpoint_path)         # 2. copy the checkpoint's numbers
    into it
self.model.eval()                                  # 3. switch to inference behaviour
```

and `_load_weights` reads

```
ckpt = torch.load(path, map_location=self.device, weights_only=False)
self.model.load_state_dict(ckpt["model_state_dict"])
```

Why the order matters. A *checkpoint* is a dictionary from parameter names (such as `encoder.stages.0.attn.q.weight`) to tensors. It holds numbers, not an architecture. PyTorch cannot rebuild the network from it; you must build the network first, with exactly the right sizes, and then copy the numbers into the slots. The copy, `load_state_dict`, is *strict* by default: every name in the file must have a slot of exactly the same shape, and every slot must be filled. That includes the 165-long normalisation buffers of Chapter 5 and the spectral compressor sized by D (Chapter 12), which is why the resolver reads D from the model's manifest. Build with $D = 24$ against a $D = 32$ checkpoint and you get an immediate shape error, not a silently wrong score. That strictness is a feature.

`model.eval()` switches off dropout (every unit is kept, Chapter 16) and makes BatchNorm layers use their stored running statistics instead of the batch's own (Chapter 29). Forgetting it gives reconstructions that change with the batch they happen to be computed in.

30.3 Two-pass masking

30.3.1 Tokens, keep masks and prediction masks

Recall three objects from Chapter 18. The first stage of the encoder cuts a 128×128 patch into non-overlapping 4×4 pixel blocks called *tokens*; there are

$$\begin{aligned} \frac{128}{4} \times \frac{128}{4} &= 32 \times 32 \\ &= 1024 \end{aligned}$$

of them, numbered $n = 0, \ldots, N-1$ with $N = 1024$, row by row. A *keep mask* is a list of N zeros and ones: 1 means "show this token to the encoder", 0 means "remove it". The *prediction mask* is its complement among valid tokens: 1 means "this token is hidden and its prediction will be used". The *token validity* v_n is 1 if the token's pixels are valid data and 0 if they are nodata (Chapter 18).

30.3.2 How many passes? The partition argument

You want each valid token to be hidden in *at least one* pass, and you want to use, for each token, the prediction from a pass in which it was hidden. Suppose you run K passes and pass k hides the set of tokens $\mathcal{H}_k$. The cheapest arrangement makes the hidden sets a *partition* of the tokens: they do not overlap, and together they cover everything,

$$\mathcal{H}_1 \cup \mathcal{H}_2 \cup \cdots \cup \mathcal{H}_K = \{0, 1, \ldots, N-1\}, \qquad \mathcal{H}_j \cap \mathcal{H}_k = \varnothing \text{ for } j \neq k. \tag{30.1}$$

Count the tokens on both sides. Because the sets do not overlap, the count of the union is the sum of the counts:

$$|\mathcal{H}_1| + |\mathcal{H}_2| + \cdots + |\mathcal{H}_K| = N.$$

If every pass hides the same fraction ρ (the *mask ratio*) of the tokens, each term is ρN:

$$K\rho N = N.$$

Divide both sides by N:

$$K\rho = 1,$$

and so

$$K = \frac{1}{\rho}.$$

With $\rho = \frac{1}{2}$ you need $K = 2$ passes: hide half, then hide the other half. With $\rho = \frac{1}{3}$ you would need $K = 3$ passes, each showing the model two thirds of its context. The product uses $K = 2$, $\rho = \frac{1}{2}$, always: both of its masking strategies build one mask and its exact complement (`segformer_mae_inferencer.py`, lines 186–216).

Source check: the inference mask ratio is not the training mask ratio

The module docstring says two-pass inference "matches the training regime (50% token masking)" (`app/foundation_models/inferencers/segformer_mae_inferencer.py`, line 15). The training-config default is indeed 0.5 (`app/models/training/training_config.py`, line 81), but the hyperspectral configurations the courses follow set `mask_ratio` to 0.65 (`configs/hyperspectral_segformer_exp_2.json`, line 39, and Indradhanu Heavy's

`research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/config.json`, line 41). So the shipped models were trained hiding 65% [M: `configs/hyperspectral_segformer_exp_2.json`] of the tokens and are run at inference hiding 50%. A ratio of 0.65 cannot be used in a two-pass partition at all: $1/0.65 = 1.54$ is not a whole number of passes. At inference the model therefore sees *more* context (50% visible) than in training (35% visible), which makes its task easier, not harder. The docstring is out of date; the code is what runs.

30.3.3 The selection is a sum of two masked predictions

Write $\hat{\mathbf{x}}^{(1)}$ and $\hat{\mathbf{x}}^{(2)}$ for the two passes' reconstructions of a patch, both of shape `(B, C, 128, 128)`. Let $M^{(1)}$ be a pixel mask that is 1 on the pixels whose token was hidden in pass 1, and $M^{(2)}$ the same for pass 2. The code combines them as

$$\hat{\mathbf{x}} = \hat{\mathbf{x}}^{(1)} \odot M^{(1)} + \hat{\mathbf{x}}^{(2)} \odot M^{(2)}, \tag{30.2}$$

where $\odot$ is element-wise multiplication (the masks broadcast over the band axis). Because the two hidden sets partition the tokens, at every pixel exactly one of the masks is 1:

$$\begin{aligned} \text{where } M^{(1)} = 1,\ M^{(2)} = 0: &\quad \hat{\mathbf{x}} = \hat{\mathbf{x}}^{(1)} \cdot 1 + \hat{\mathbf{x}}^{(2)} \cdot 0 = \hat{\mathbf{x}}^{(1)}, \\ \text{where } M^{(1)} = 0,\ M^{(2)} = 1: &\quad \hat{\mathbf{x}} = \hat{\mathbf{x}}^{(1)} \cdot 0 + \hat{\mathbf{x}}^{(2)} \cdot 1 = \hat{\mathbf{x}}^{(2)}. \end{aligned}$$

So Equation (30.2) is a *selection*, not a blend: each pixel takes its value from the one pass that hid it. Finally the code multiplies by the validity mask, so nodata pixels come out as zero.

Worked example 30.1: Selecting from two passes on four tokens

Take a 2×2 grid of tokens and one band. Pass 1 hides tokens $(0,0)$ and $(1,1)$; pass 2 hides $(0,1)$ and $(1,0)$. The two passes return these values (the true values do not matter here):

$$\hat{\mathbf{x}}^{(1)} = \begin{pmatrix} 0.21 & 0.30 \\ 0.28 & 0.19 \end{pmatrix}, \quad \hat{\mathbf{x}}^{(2)} = \begin{pmatrix} 0.25 & 0.33 \\ 0.26 & 0.22 \end{pmatrix}, \quad M^{(1)} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad M^{(2)} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Do it on paper, one entry at a time:

$$\begin{aligned} \hat{\mathbf{x}}_{0,0} &= 0.21 \times 1 + 0.25 \times 0 = 0.21 + 0 = 0.21, \\ \hat{\mathbf{x}}_{0,1} &= 0.30 \times 0 + 0.33 \times 1 = 0 + 0.33 = 0.33, \\ \hat{\mathbf{x}}_{1,0} &= 0.28 \times 0 + 0.26 \times 1 = 0 + 0.26 = 0.26, \\ \hat{\mathbf{x}}_{1,1} &= 0.19 \times 1 + 0.22 \times 0 = 0.19 + 0 = 0.19. \end{aligned}$$

The result $\begin{pmatrix} 0.21 & 0.33 \\ 0.26 & 0.19 \end{pmatrix}$ uses pass 1 on the diagonal and pass 2 off it. The value 0.30 that pass 1 produced at $(0,1)$ is thrown away: in pass 1 that token was visible, so its prediction may be a copy.

30.3.4 Strategy A: the checkerboard

The default strategy builds the mask from a checkerboard, in the function `checkerboard_token_mask` of `app/foundation_models/components/token_masking.py`:

```
rows = torch.arange(H_tokens) // cell_size          # (32,)
cols = torch.arange(W_tokens) // cell_size          # (32,)
```

```
grid = (rows[:, None] + cols[None, :]) % 2          # (32, 32): 1 = visible, 0 =
    hidden
if invert:
    grid = 1 - grid                                  # pass 2
return grid.flatten().unsqueeze(0).float()           # (1, N) = (1, 1024)
```

Broadcasting. `rows[:,None]` has shape `(4, 1)` on a 4×4 grid: a column. `cols[None,:]` has shape `(1, 4)`: a row. Adding a column to a row *broadcasts*: the column is copied across and the row is copied down until both are `(4, 4)`, and then they are added entry by entry. The result is the addition table.

Worked example 30.2: The checkerboard on a 4×4 token grid

With `cell_size=1`, build the table of row + col:

	col 0	col 1	col 2	col 3
row 0	$0+0=0$	$0+1=1$	$0+2=2$	$0+3=3$
row 1	$1+0=1$	$1+1=2$	$1+2=3$	$1+3=4$
row 2	$2+0=2$	$2+1=3$	$2+2=4$	$2+3=5$
row 3	$3+0=3$	$3+1=4$	$3+2=5$	$3+3=6$

Take each entry modulo 2 (the remainder after dividing by 2: 0 for even, 1 for odd):

$$\text{grid} = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix}.$$

In this grid 1 means *visible*. So pass 1 hides the tokens where row + col is *even*, starting with the top-left token, and pass 2 (`invert=True`) hides the odd ones. With `cell_size=2` the integer division first maps rows $0, 1, 2, 3$ to $0, 0, 1, 1$, and the same steps give 2×2 blocks of tokens.

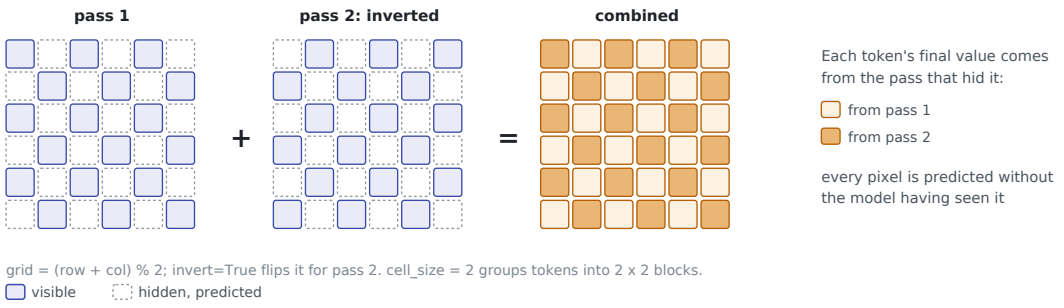


Figure 30.1: Two-pass checkerboard inference (course figure 29.1). Each token's final value is taken from the pass that hid it, so the combined reconstruction is built entirely from predictions. See the source check below for which pass hides the top-left token.

Source check: which pass hides the top-left token

The course figure and its caption draw pass 1 hiding the tokens where row+col is odd. In the code, `checkerboard_token_mask` returns 1 for visible and computes (row + col) mod 2, so with `invert=False` the even tokens are 0, hidden (`app/foundation_models/components/`

`token_masking.py`, lines 271–284; the comment there reads "row+col is even → 0 (masked)"). The difference is only a relabelling of the two passes and changes no reconstruction, but if you debug the masks, expect token $(0, 0)$ to be hidden in pass 1.

30.3.5 Invalid tokens are always kept

The keep mask of each pass is built in `_checkerboard_keep_mask` as

```
pred_mask = token_validity * (1.0 - checker)    # (B, N): hidden AND valid
keep_mask = 1.0 - pred_mask                     # (B, N): everything else is shown
```

Multiplying by `token_validity` means a token becomes a prediction target only if it is *both* valid and selected by the pattern. An invalid token is never removed: it stays in the sequence carrying the zero signal the model learned to treat as "no data", exactly as in training (Chapter 18).

Worked example 30.3: Keep mask for a row of four tokens with one invalid

Four tokens in one row, the third invalid: $v = (1, 1, 0, 1)$. In pass 1 the checker row is $(0, 1, 0, 1)$ (hide the first and third). Work entry by entry:

$$\begin{aligned} 1 - \text{checker} &= (1-0,\ 1-1,\ 1-0,\ 1-1) = (1, 0, 1, 0), \\ \text{pred} = v \odot (1 - \text{checker}) &= (1 \cdot 1,\ 1 \cdot 0,\ 0 \cdot 1,\ 1 \cdot 0) = (1, 0, 0, 0), \\ \text{keep} = 1 - \text{pred} &= (1-1,\ 1-0,\ 1-0,\ 1-0) = (0, 1, 1, 1). \end{aligned}$$

Only the first token is removed. The third would have been hidden by the pattern, but it is invalid, so it is kept and nothing is predicted for it.

30.3.6 Strategy B: random halves

The alternative, `masking_strategy="random"`, draws a random half and uses its complement:

```
rand_mask   = (torch.rand(1, N) > 0.5).float()   # (1, N): 1 = visible in pass 1
pred_mask_1 = token_validity * (1.0 - rand_mask)
pred_mask_2 = token_validity * rand_mask          # the exact complement
```

Why offer it? A checkerboard is regular. If the model is slightly weaker at token boundaries, and it is, that weakness lands at the same relative positions in every tile, and a faint grid is printed into the residual map that an analyst could mistake for structure on the ground. A random mask scatters the artefacts differently in every tile, and the heavy overlap of tiles (next section) averages them away. The default in `InferenceConfig` is `"checkerboard"` (`app/models/training/inference_config.py`, line 96), and the scoring action does not override it, so the product is deterministic: the same scene and checkpoint give the same score map every time.

30.3.7 Cells, blocks and a mismatch to know about

`checkerboard_cell_size` (default 1, `inference_config.py`, line 77) sets the size of the checkerboard cells in tokens. The native-band inferencer used by Indradhanu V2 is called `SpectralStemInferencer`. It overrides the method `_checkerboard_keep_mask` so that a checkpoint trained with block masking (`mask_block` > 1, Chapter 42) is hidden in blocks of the same size. The override is in `app/foundation_models/inferencers/spectral_stem_inferencer.py`, lines 68–77.

Source check: a block-masked checkpoint and the pixel selection

The override changes which tokens are *hidden*, but `infer()` in the parent class builds the *selection* masks $M^{(1)}, M^{(2)}$ of Equation (30.2) from `checkerboard_cell_size`, not from `mask_block` (`segformer_mae_inferencer.py`, lines 186–199). With `mask_block=2` and the default cell size 1 the two patterns disagree. The book's script `research/textbook/tools/compute_inference.py` reproduces both patterns on a 4×4 token grid: 8 of the 16 tokens are taken from a pass in which they were *visible*. The shipped Indradhanu V2 has `mask_block` $= 1$ (the default, `app/models/training/training_config.py`, line 199; not set in the FTH3 config), so the product is unaffected. The block-masked run FT08 (`research/experiments/indradhanu-finetune/FT08-gated-blocks/config.json`, line 107: `mask_block` $= 2$) was evaluated through the same inferencer with the default cell size (`app/evaluation/models.py`, lines 70–77), so its measured result may include this leak. This was not verified by rerunning; treat it as an open question for Chapter 42.

30.4 The sliding window

30.4.1 Patch size, stride and overlap

`predict_full_scene` handles a whole scene. Three numbers define its tiling:

- the *patch size* P, the side of a square tile: $P = 128$;
- the *stride* S, the distance between the starts of neighbouring tiles: the resolver's default for both Indradhanu models is $S = 32$ (`backend/allotrope/foundation_models/resolver.py`, lines 160 and 172);
- the *overlap* $P - S$, how many pixels two neighbouring tiles share: $128 - 32 = 96$.

If the stride is not given, `InferenceConfig` falls back to $P/2 = 64$ (`segformer_mae_inferencer.py`, line 257), which is also the stride used to cut training patches. The product passes 32: a quarter of the patch, giving heavy overlap. You run inference once per scene, so the extra compute buys a smoother map.

30.4.2 Counting tiles along one axis

The tile starts are decided by `PatchPlanGenerator` (`app/utils/patch_generation/generate_patch_plan.py`). Along an axis of length W, it places tiles at $0, S, 2S, \ldots$ while they fit, and then places one last tile flush against the far edge. Derive the count.

A tile starting at s covers pixels $s, \ldots, s + P - 1$. It fits if its last pixel is inside the axis:

$$s + P - 1 \leq W - 1.$$

Add 1 to both sides:

$$s + P \leq W.$$

The starts are $s = kS$ for $k = 0, 1, 2, \ldots$; substitute:

$$kS + P \leq W.$$

Subtract P and divide by $S > 0$:

$$k \leq \frac{W - P}{S}.$$

So the starts that fit are $k = 0, 1, \ldots, \lfloor (W - P)/S \rfloor$, where $\lfloor \cdot \rfloor$ (*floor*) rounds down to a whole number. Counting 0 as well, that is

$$K_{\text{fit}} = \left\lfloor \frac{W - P}{S} \right\rfloor + 1 \tag{30.3}$$

tiles. If $(W - P)/S$ is not a whole number, the last fitting tile ends before the edge, leaving a strip uncovered, so one more tile is placed at $s = W - P$, "snapped" back so that it ends exactly at the edge. Then

$$K = \left\lfloor \frac{W - P}{S} \right\rfloor + 2 = \left\lceil \frac{W - P}{S} \right\rceil + 1, \tag{30.4}$$

where $\lceil \cdot \rceil$ (*ceiling*) rounds up. The two forms agree because, for a number that is not whole, rounding up gives one more than rounding down.

Worked example 30.4: Tiles along one axis of a 1000-pixel scene

Take $W = 1000$, $P = 128$, $S = 32$. On paper:

$$\begin{aligned} W - P &= 1000 - 128 = 872, \\ \frac{W - P}{S} &= \frac{872}{32} = 27.25, \\ \lfloor 27.25 \rfloor &= 27, \\ K_{\text{fit}} &= 27 + 1 = 28 \quad (\text{starts } 0, 32, \ldots, 27 \times 32 = 864). \end{aligned}$$

The last fitting tile covers $864, \ldots, 864 + 127 = 991$. Pixels 992 to 999 are not yet covered, so a snapped tile starts at $1000 - 128 = 872$. In total

$$K = 28 + 1 = 29 = \lceil 27.25 \rceil + 1 = 28 + 1.$$

The scene has $29 \times 29 = 841$ tiles. The book's script, which calls the product's own `PatchPlanGenerator`, gives 29 per axis and 841 tiles [C].

Worked example 30.5: A real PRISMA frame, 1210×1219

A PRISMA L2D frame of 1210×1219 pixels [M: `docs/lld/frequency-destriper.md`]. Rows first:

$$\begin{aligned} \frac{1210 - 128}{32} &= \frac{1082}{32} = 33.8125, \\ K_{\text{rows}} &= \lceil 33.8125 \rceil + 1 = 34 + 1 = 35. \end{aligned}$$

Columns:

$$\begin{aligned} \frac{1219 - 128}{32} &= \frac{1091}{32} = 34.09375, \\ K_{\text{cols}} &= \lceil 34.09375 \rceil + 1 = 35 + 1 = 36. \end{aligned}$$

Tiles:

$$35 \times 36 = 1260.$$

The model takes `inference_batch_size` tiles at a time. The scoring action passes the resolver's default of 8 (`resolver.py`, line 62), so the number of batches is

$$\left\lceil \frac{1260}{8} \right\rceil = \lceil 157.5 \rceil = 158,$$

and since every batch runs the network twice (two passes),

$$158 \times 2 = 316 \text{ forward passes.}$$

The script confirms 35, 36, 1260 tiles and 316 forward passes [C].

Source check: when $(W-P)/S$ is a whole number, the code adds a duplicate tile

Equation (30.4) is what the generator's docstring intends, and it holds whenever $(W-P)/S$ is not a whole number. When it *is* whole, the last fitting tile already ends at the edge, yet the loop in `generate_patch_plan.py` (lines 66–80) appends it, steps on by S, finds the next tile overhanging and appends the snapped start $W-P$, which is the same start again. For $W = 1024$: $(1024-128)/32 = 28$, the formula gives 29 tiles, and the code produces 30 starts of which 29 are distinct; for $W = 128$ it produces the start 0 twice [C]. The duplicate tile wastes one tile's compute per row or column and counts that tile twice in the average below. With the deterministic checkerboard both copies are identical, so the reconstruction is unchanged; with random masking the last tile gets double weight. PRISMA frames are not multiples of 32 plus 128, so the product rarely meets the case.

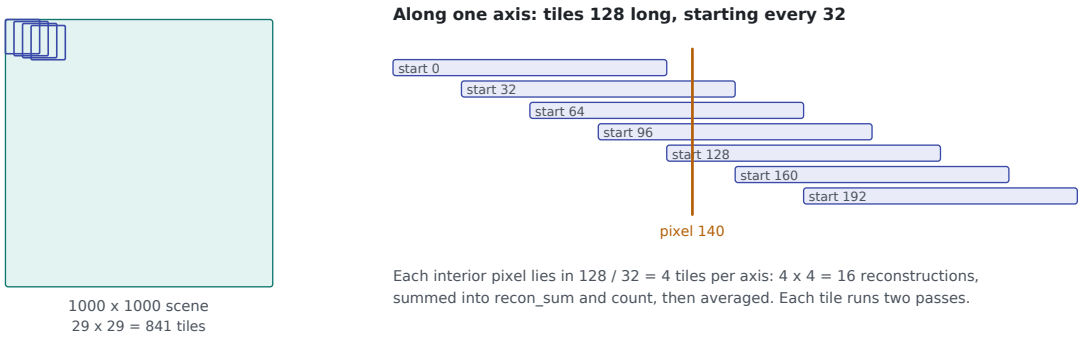


Figure 30.2: Sliding-window inference (course figure 29.2). 128×128 tiles start every 32 pixels, so a 1000×1000 scene needs $29 \times 29 = 841$ tiles; an interior pixel lies in $128/32 = 4$ tiles along each axis and so in $4 \times 4 = 16$ tiles in all.

30.4.3 How many tiles cover each pixel

Along one axis, a pixel at position x lies in every tile whose start s satisfies $s \le x \le s + P - 1$. Away from the edges the starts are S apart, so the number of such starts is the length P of the window divided by the gap S between starts:

$$\frac{P}{S} = \frac{128}{32} = 4.$$

In two dimensions a pixel lies in a tile only if both its row and its column do, so the counts multiply:

$$4 \times 4 = 16 \text{ tiles.}$$

Sixteen separate reconstructions of every interior pixel, each made with the tile in a different position and so with different context visible around it. Near the edges the count drops: a pixel in the first 32 columns is in only one tile along that axis. Near the far edge something less obvious happens: the snapped tile overlaps its neighbours by more than $P - S$, so a strip of pixels is covered *five* times. Figure 30.3 plots the count along one axis of the 1000-pixel scene, from the script's output [C].

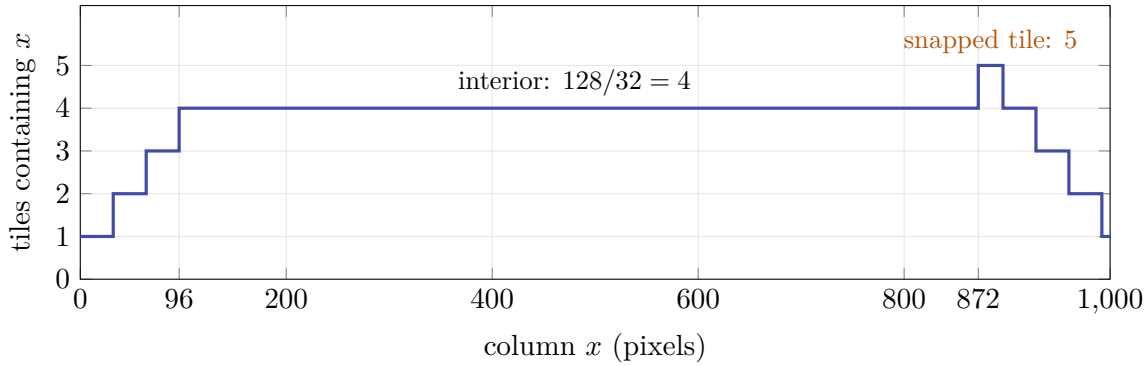


Figure 30.3: Number of tiles containing each column of a 1000-pixel axis with $P = 128$, $S = 32$, computed with the product's `PatchPlanGenerator` (`research/textbook/tools/compute_inference_output.txt`). The two-dimensional count at a pixel is the product of its row count and its column count: 16 in the interior, 1 at a corner, 25 where the two snapped strips cross.

30.5 Accumulate, then average

30.5.1 Two scene-sized accumulators

Storing 841 or 1260 reconstructed tiles and averaging them at the end would need hundreds of copies of the scene. Instead, `predict_full_scene` keeps two running totals the size of the scene:

```
recon_sum = torch.zeros(c, h, w)    # (C, H, W): sum of the predictions that reached
    each pixel
count     = torch.zeros(1, h, w)    # (1, H, W): how many predictions reached each
    pixel
...
for j, idx in enumerate(valid_indices):
    r, cs = batch_coords[idx]
    patch_eroded = eroded_mask[:, r:r+ps, cs:cs+ps]          # (1, 128, 128), 0 or 1
    recon_sum[:, r:r+ps, cs:cs+ps] += recon[j] * patch_eroded  # (C, 128, 128)
    count    [:, r:r+ps, cs:cs+ps] += patch_eroded
reconstruction = torch.where(count > 0, recon_sum / count, scene)   # (C, H, W)
```

In symbols, write $\hat{\mathbf{x}}_t(p)$ for tile t's prediction at pixel p and $w_t(p) \in \{0, 1\}$ for the eroded validity of p (defined below; 0 if p is not in tile t). Then

$$\hat{x}(p) = \begin{cases} \dfrac{\sum_t w_t(p)\, \hat{\mathbf{x}}_t(p)}{\sum_t w_t(p)} & \text{if } \sum_t w_t(p) > 0, \\ x(p) & \text{if } \sum_t w_t(p) = 0. \end{cases} \tag{30.5}$$

The top line is a *weighted average*: the weights are 0 or 1, so it is the plain mean of the predictions that count.

30.5.2 Three details

The weights come from an eroded mask. *Erosion* shrinks the valid region: a pixel survives only if every pixel in a $k \times k$ square around it is valid (Chapter 18). Inference uses $k = 15$ (`inference_config.py`, line 109), so any pixel within $\lfloor 15/2 \rfloor = 7$ pixels of nodata gets weight 0. Such pixels are predicted from context that is partly nodata, and their reconstructions are unreliable. With weight 0 they add nothing to the sum *and* nothing to the count: not a small amount, nothing at all.

Dividing by the count makes the average right everywhere. An interior pixel has count 16, a corner pixel count 1, a pixel in the snapped strip up to 25. Dividing by the actual count gives a true mean in every case with no special handling of edges.

Where nothing arrived, the scene stands in for itself. If no tile delivered a usable prediction (count 0), the reconstruction is set to the input. The residual there is $x(p) - x(p) = 0$, so the pixel scores zero and can never be flagged. Leaving zeros or NaN in such pixels would have produced huge or undefined residuals instead.

Worked example 30.6: Averaging three tiles, one of them eroded

A pixel p in one band lies in three tiles. Their predictions and weights are

$$\hat{\mathbf{x}}_1(p) = 0.24,\ w_1 = 1; \qquad \hat{\mathbf{x}}_2(p) = 0.28,\ w_2 = 1; \qquad \hat{\mathbf{x}}_3(p) = 0.90,\ w_3 = 0.$$

(Tile 3 is the one in which p sits within 7 pixels of a nodata hole, and its prediction is spoiled.) On paper:

$$\begin{aligned} \text{sum} &= 1 \times 0.24 + 1 \times 0.28 + 0 \times 0.90 \\ &= 0.24 + 0.28 + 0 \\ &= 0.52, \\ \text{count} &= 1 + 1 + 0 = 2, \\ \hat{x}(p) &= \frac{0.52}{2} = 0.26. \end{aligned}$$

Had the third tile been given a small weight such as 0.1 instead of 0, the sum would be $0.52 + 0.09 = 0.61$, the count 2.1, and the average $0.61/2.1 = 0.290$: a spoiled prediction still pulls the answer by 0.03.

Why averaging smooths the map. If the errors of the 16 predictions of an interior pixel were independent with equal variance σ^2, the variance of their mean would be

$$\operatorname{Var}\left(\frac{1}{16}\sum_{t=1}^{16}\hat{\mathbf{x}}_t\right) = \frac{1}{16^2}\sum_{t=1}^{16}\operatorname{Var}(\hat{\mathbf{x}}_t) = \frac{16\,\sigma^2}{256} = \frac{\sigma^2}{16},$$

a standard deviation four times smaller. In practice the predictions share most of their context and are far from independent [A], so the real gain is smaller, but the direction is right: overlapping tiles average away the noise that any one tile position adds. A smooth reconstruction is expected; a crisp one would suggest the model had found a way to copy.

30.5.3 The tile validity filter

A tile is only processed if at least 10% of its pixels are valid:

```
MIN_VALID_FRACTION = 0.1
valid_frac = m.float().mean().item()      # fraction of the tile's pixels that are
    valid
if valid_frac >= MIN_VALID_FRACTION:
    valid_indices.append(i)
```

Tiles that are more than 90% nodata are skipped: the model never saw such patches in training and its output on them is meaningless.

Source check: the comment says 0.4, the code says 0.1

The comment above this line says the threshold "matches training: patches with < 40% valid pixels were discarded" (`segformer_mae_inferencer.py`, lines 280–284). The training threshold is indeed 0.4 (`MIN_VALID_PIXEL_FRACTION`, `app/foundation_models/trainers/hyperspectral_segformer_mae_trainer.py`, line 34), but inference uses 0.1. The intent, skipping mostly empty tiles, is the same; the numbers differ. The course (IW29) already flags this.

30.5.4 Memory, roughly

One full cube of 165 bands at 1000 × 1000 pixels in 4-byte `float32` holds

$$165 \times 1000 \times 1000 = 165{,}000{,}000 \text{ numbers},$$
$$165{,}000{,}000 \times 4 = 660{,}000{,}000 \text{ bytes} = 660 \text{ MB}.$$

Inference keeps at least three such arrays: the scene, `recon_sum`, and the final reconstruction, about $3 \times 660 = 1980$ MB, about 2 GB, before any network activations. For the 1210 × 1219 frame each array is 973,493,400 bytes [C], close to 1 GB, so about 3 GB in total. That is why `inference_batch_size` is exposed as a per-action override with a conservative default of 8: the resolver's own comment calls it "the main memory dial when running on a CPU-only Mac VM" (`resolver.py`, lines 59–62).

Source check: two defaults for the batch size

`InferenceConfig.inference_batch_size` defaults to 16 (`inference_config.py`, line 104), but the scoring action always passes the resolver's `default_batch_size` of 8 unless the user overrides it (`backend/allotrope/action_types/_anomaly_scoring_run.py`, line 303). In the product the batch size is 8.

30.6 Shapes from scene to reconstruction

Every tensor on the inference path, for the 1210 × 1219 PRISMA frame and the 165-band Indradhanu:

```
scene            (165, 1210, 1219)  float32, from filtered_vendable.pkl
mask             (1, 1210, 1219)    1 = spatially valid (any band valid)
eroded_mask      (1, 1210, 1219)    after a 15 x 15 erosion
coords           1260 pairs (r, c)  35 row starts x 36 column starts
patches          (8, 165, 128, 128) one batch of tiles
patch_masks      (8, 1, 128, 128)
token_validity   (8, 1024)          32 x 32 tokens per tile
keep_mask_1/_2   (8, 1024)          1 = show, 0 = hide (complements among valid
    tokens)
pass1_pixels     (1, 1, 128, 128)   token mask expanded to pixels (nearest-neighbour)
x_hat_1, x_hat_2 (8, 165, 128, 128) the two passes
recon            (8, 165, 128, 128) x_hat_1*pass1_pixels + x_hat_2*pass2_pixels,
    times mask
recon_sum        (165, 1210, 1219)  running sum
count            (1, 1210, 1219)    running count, 0..25
reconstruction   (165, 1210, 1219)  recon_sum / count, or the scene where count = 0
```

30.7 From reconstruction to score map

30.7.1 The scoring call

The action does not use the inferencer's own scoring method. It calls one shared numpy function, `compute_score` in `app/utils/anomaly_detection/scoring.py`:

```
score = compute_score(cube, reconstruction, eroded_keep_mask,   # (C,H,W), (C,H,W), (H,W)
                      method=method, combined_weight=sam_l1_alpha)   # -> (H, W) float32
```

It returns one number per pixel, higher meaning more anomalous, and sets every pixel outside the keep mask to exactly 0 (`scoring.py`, line 114). Five methods exist. Four of them measure the *size* of the residual $\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}}$:

- `L1`: the mean of $|r_c|$ over the bands;
- `MSE`: the mean of r_c^2 (not offered for Indradhanu);
- `SAM`: the spectral angle between $\mathbf{x}$ and $\hat{\mathbf{x}}$ in radians (Chapter 25);
- `combined`: $w \cdot \mathrm{L1}_{\mathrm{norm}} + (1 - w) \cdot \mathrm{SAM}_{\mathrm{norm}}$, each divided by its maximum over valid pixels, $w = 0.5$ by default.

The fifth, `residual_mahalanobis`, measures how *unusual* the residual's direction is, and has been Indradhanu's default since 2026-10-15 (`resolver.py`, lines 152–158). Chapter 31 is devoted to it. The inferencer's own `compute_anomaly_scores`, which returns a dictionary of L1 and SAM maps, exists for notebooks; the product never calls it.

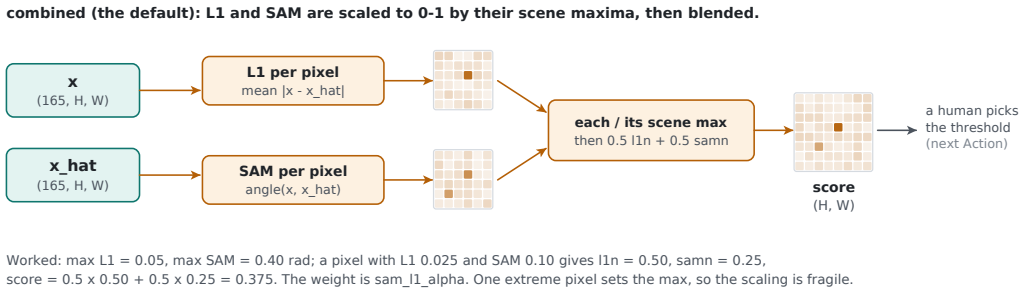


Figure 30.4: The `combined` scoring method (course figure 30.1): per-pixel L1 and SAM maps are each divided by their maximum over the valid pixels and blended with weight 0.5. Since 2026-10-15 this is no longer Indradhanu's default.

Worked example 30.7: The combined score and its fragile denominator

L1 is in reflectance units and SAM in radians; adding them directly would be adding metres to kilograms. So each is divided by its own maximum over the scene's valid pixels. Suppose the scene maxima are max L1 = 0.05 and max SAM = 0.40 rad, and one pixel has L1 = 0.025 and SAM = 0.10 rad [A]. Normalise:

$$\mathrm{L1}_{\mathrm{norm}} = \frac{0.025}{0.05} = 0.50,$$
$$\mathrm{SAM}_{\mathrm{norm}} = \frac{0.10}{0.40} = 0.25.$$

Blend with $w = 0.5$:

$$\begin{aligned} \text{score} &= 0.5 \times 0.50 + 0.5 \times 0.25 \\ &= 0.250 + 0.125 \\ &= 0.375. \end{aligned}$$

Now let one dead detector element produce a single pixel with L1 = 0.50, ten times the old maximum. The same pixel becomes

$$\begin{aligned} \text{L1}_{\text{norm}} &= \frac{0.025}{0.50} = 0.05, \\ \text{score} &= 0.5 \times 0.05 + 0.5 \times 0.25 = 0.025 + 0.125 = 0.150. \end{aligned}$$

One outlier has compressed every other pixel's L1 term by a factor of ten. The symptom is a dark score map with one bright dot; the check is the percentiles in `stats.json`, written for every model.

30.7.2 What the scoring action writes

For each model, in `models/<codename>/` under the action's output folder, where the folder name is the codename in lower case with spaces turned into underscores (`_safe_dirname`):

file	shape and type	contents
`anomaly_score.tif`	(H, W) float32	the score map
`anomaly_score.png`	image	preview; square-root stretch for squared distances
`reconstruction.tif`	(C, H, W) float32	the full-scene reconstruction
`reconstruction.png`	image	RGB preview of the reconstruction
`stats.json`	JSON	method, patch, stride, timings, score percentiles

The preview stretch is a square root for the classical detectors *and* for `residual_mahalanobis`, because both produce squared distances with a heavy right tail; linear otherwise (`_anomaly_scoring_run.py`, line 511). Every GeoTIFF is written with an identity transform (`_anomaly_scoring_render.py`, line 61): it carries no geography until export (Chapter 32).

30.8 One scene through the whole product

The components of this book stop being separate here. Follow one PRISMA scene from upload to a shortlist; each step names the chapter that explained it. The numbers are those of the anomaly course's end-to-end walk (AD13): a scene of 1210×1219 pixels, with the fractions lost at each step assumed for illustration [A].

1. **Upload and onboarding** (Chapter 4). The api streams the file to a staging folder and queues a `scene_onboard` job; it does no analysis itself. The worker builds the vendable with the eight-stage band pipeline (Chapter 5): a cube `(165, 1210, 1219)` of reflectance, a validity cube of the same shape, and 165 wavelengths. Note what is absent: no latitude, no longitude, no projection.
2. **Project** (Chapter 4). Bound to the scene. Nothing computes.
3. `band_filter_apply`. Re-vends the scene with explicit band settings and writes `filtered_vendable.pkl`, filling each invalid pixel with its nearest valid neighbour so the patch embedding never meets a cliff (Chapter 5).

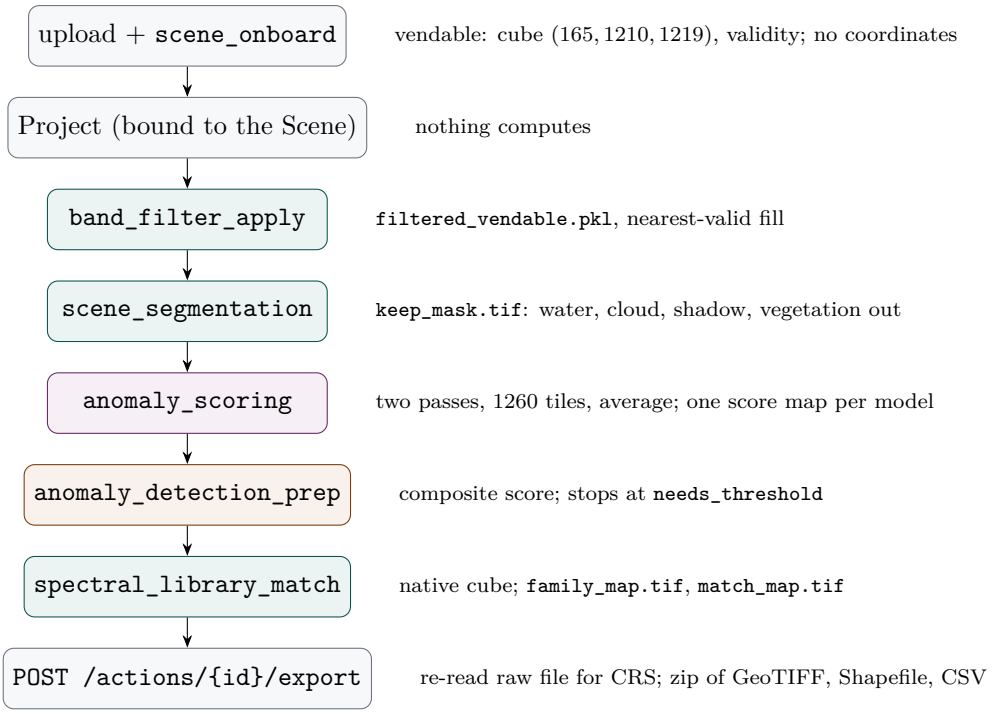


Figure 30.5: The product's action chain for a hyperspectral scene (AD13). Teal steps transform spectra, plum runs the model, amber is where a person decides.

4. **`scene_segmentation`**. Thresholds spectral indices into masks and writes `keep_mask.tif`, the region worth analysing.
5. **`anomaly_scoring`**. This chapter and the next: tile, two passes, average, score. A classical detector such as MNF-RX (Chapter 8) can run in the same action; its covariance is estimated only on the keep mask.
6. **`anomaly_detection_prep`**. Combines the models' maps into one composite and *stops* at the status `needs_threshold` until a person chooses a percentile (Chapter 32).
7. **`spectral_library_match`**. Hyperspectral only. Names the likely material family of each flagged pixel, reading the native onboarding cube, not the filtered one (Chapter 32).
8. **Export**. Coordinates finally appear: the raw file is re-read for its georeferencing, and a zip leaves the system (Chapter 32).

Worked example 30.8: The funnel from 1.47 million pixels to a shortlist

Assume onboarding loses 6% of the pixels to cloud and swath edges, segmentation keeps 69% of the rest, and the analyst flags the top 0.5% of the kept pixels [A]. On paper:

$$\begin{aligned} \text{pixels} &= 1210 \times 1219 = 1{,}474{,}990, \\ \text{valid} &= 1{,}474{,}990 \times 0.94 = 1{,}386{,}490.6 \approx 1{,}386{,}491, \\ \text{kept} &= 1{,}386{,}491 \times 0.69 = 956{,}678.8 \approx 956{,}679, \\ \text{flagged} &\approx 956{,}679 \times 0.005 = 4{,}783.4. \end{aligned}$$

The reduction factor is

$$\frac{1{,}474{,}990}{4{,}783} \approx 308.4.$$

From about 1.47 million pixels to about 4,800: the system narrows, and a person reviews the shortlist. Chapter 32 shows that the exact count the code flags is 4,784, and that with

a model-only chain the count can be quite different, because of which pixels the percentile is taken over.

What can go wrong at each step. The anomaly course lists the failures seen in practice: the wrong sensor selected (a `ValueError`, job failed); a class renamed in `vendables.py` (stored pickles unreadable); segmentation thresholds wrong for the terrain (the keep mask removes the target); the worker crash-looping (jobs sit in `queued` while the api reports healthy, which has happened twice, see CLAUDE.md); normalisation statistics wrong for the sensor (every pixel scores high); a threshold too tight (the real anomaly ranked a little below the cut); and georeferencing that cannot be resolved (`422crs_missing`).

30.9 Common confusions

Does the scene get resized to 128 by 128?
No. It is cut into overlapping 128×128 tiles, each reconstructed separately at full resolution, and the predictions are averaged back into a full-size cube.

Are the two passes twice as slow?
Yes, exactly twice the network calls. It is the price of the rule that no pixel predicts itself, and it is not negotiable.

Is inference masking the same as training masking?
No. Training hides a random 65% and grades only there; inference hides exactly half, twice, with complementary masks, and uses every hidden prediction. The partition argument forces $\rho = 1/K$.

Why does a stride of 32 cost sixteen times more than a stride of 128?
The number of tiles along each axis grows about four times ($P/S = 4$ instead of 1), and the two axes multiply: $4 \times 4 = 16$ times as many tiles, each giving every pixel one more prediction.

Pixels near a nodata hole score zero. Is the model ignoring them?
If no tile delivered a usable prediction for a pixel (all its weights eroded to 0), the reconstruction falls back to the input and the residual is exactly zero. If some did, the pixel is scored normally from those. Erosion of 15 removes only pixels within 7 of nodata from the average.

The score map has a faint grid.
Checkerboard artefacts at token boundaries. The random strategy or a different stride changes them; the overlap of 16 tiles already damps them.

30.10 Check yourself

Exercise 30.1. Explain in two sentences why one pass with a 50% mask is not enough, and why one pass with no mask is worse.

Exercise 30.2. A scene is 1200×900 pixels, $P = 128$, $S = 32$. Compute the number of tile starts along each axis with Equation (30.4), the number of tiles, the number of batches of 8 and the number of forward passes. Show every step.

Exercise 30.3. With `cell_size=2`, write out the 4×4 token grid that pass 1 hides, and the one pass 2 hides.

Exercise 30.4. A pixel lies in four tiles with predictions $0.31, 0.29, 0.30, 0.70$; the last tile's weight is 0 because of erosion. Compute the averaged reconstruction, and the residual if the observed value is 0.36. What would the residual be if all four weights were 0?

Exercise 30.5. Along a $W = 600$ axis with $P = 128$, $S = 32$, list the last three tile starts and give the coverage count (tiles containing it) of column 500.

Exercise 30.6. Why must a checkpoint be loaded into a model built with the same D, and what happens if it is not?

Next: Chapter 31, From residual to score: residual Mahalanobis, the ring and integral images.

Chapter 31

From residual to score: residual Mahalanobis, the ring and integral images

The one thing this chapter teaches

L1, SAM and `combined` measure how *big* a pixel's reconstruction error is, but in a real scene most big errors are shared by thousands of ordinary pixels. Residual Mahalanobis learns the directions those ordinary errors take, divides each by its ordinary spread, and measures what is left from the average of a ring of neighbours; an error then scores high only if it is *unusual*, however small it is, and integral images make the ring cost four look-ups per box instead of thousands of additions.

Chapter 30 ended with a reconstruction $\hat{\mathbf{x}}$ of the whole scene, every pixel predicted while it was hidden. Subtracting gives the *residual* at every pixel,

$$\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}} \in \mathbb{R}^C, \tag{31.1}$$

one number per band: $C = 165$ on the common grid, about 185 for a native-band model. A decision needs one number per pixel. How you collapse C numbers into one decides what you find. This chapter builds the collapse that has been Indradhanu's default since 2026-10-15, `residual_mahalanobis`, in `app/utils/anomaly_detection/residual_mahalanobis.py` (112 lines). It uses the Mahalanobis distance of Chapter 7, the covariance and eigenvectors of Chapter 6, and the local windows of Chapter 8, applied to a new input: the model's mistakes rather than the scene itself.

31.1 Size scores, and the big errors that are not anomalies

31.1.1 Four ways to measure a residual's size

Until 2026-10-15 every score the product offered for Indradhanu measured the *size* of $\mathbf{r}$ (`app/utils/anomaly_detection/scoring.py`, lines 95–107):

$$\mathrm{L1}(\mathbf{r}) = \frac{1}{C}\sum_{c=1}^{C}|r_c| \qquad \text{mean absolute error,}$$

$$\mathrm{MSE}(\mathbf{r}) = \frac{1}{C}\sum_{c=1}^{C} r_c^2 \qquad \text{mean squared error,}$$

$$\mathrm{SAM}(\mathbf{x},\hat{\mathbf{x}}) = \arccos\frac{\langle \mathbf{x},\hat{\mathbf{x}}\rangle}{\|\mathbf{x}\|\,\|\hat{\mathbf{x}}\|} \qquad \text{angle between spectra,}$$

$$\mathrm{combined} = 0.5\,\frac{\mathrm{L1}}{\max \mathrm{L1}} + 0.5\,\frac{\mathrm{SAM}}{\max \mathrm{SAM}} \qquad \text{both, maxima over valid pixels.}$$

Chapter 25 worked L1 and SAM by hand and showed that they find different things: L1 flags a pixel that is uniformly too bright, SAM one whose spectral *shape* is wrong. Both are sizes. Size is the right question only if every large error is rare.

31.1.2 Three kinds of large error that are not rare

On the four labelled PRISMA scenes the score maps showed three kinds of large error, each shared by thousands of ordinary pixels (IW45):

- **Detector stripes.** A push-broom spectrometer such as PRISMA records each image column with its own detector element. One element calibrated slightly off makes every pixel in its column err the same way, and the score map shows a stripe. On set 4 the `combined` map was dominated by them.
- **Haze.** A thin layer of atmosphere over part of a scene lifts the blue end of every spectrum beneath it. The model, trained mostly on clear ground, under-predicts all those pixels by a similar amount. Set 4 again.
- **Dark sea.** On set 1, three quarters sea, the map was dominated by the sea, through the trap worked next.

In each case thousands of pixels share one *kind* of error. A size score ranks all of them above a small real target. The model is not at fault; the score asks the wrong question.

31.1.3 The dark-sea trap, and why its patch is a patch

SAM is an angle, so it measures error *relative to* the signal. Give a dark sea pixel and a bright land pixel exactly the same residual, $(+0.01, -0.01, +0.01)$ (AD12):

	band 1	band 2	band 3	L1	SAM
sea, true	0.02	0.03	0.01		
sea, reconstruction	0.03	0.02	0.02	0.010	0.4335 rad
land, true	0.20	0.30	0.10		
land, reconstruction	0.21	0.29	0.11	0.010	0.0463 rad

L1 cannot tell them apart. SAM gives the sea pixel nine times the angle. Measured on set 1 with the F001 model, the median angle is 0.34 rad on sea, 0.13 on land and 0.18 on the labelled anomalies [M: `docs/05-detectors.md`]: ordinary sea outranks the targets.

`compute_score(...,sam_dark_fade=t)` multiplies the angle by $w = \text{clip}(\bar{x}/t, 0, 1)$, where $\bar{x}$ is the pixel's mean reflectance (`scoring.py`, line 131). With $t = 0.05$:

$$\begin{aligned} \text{sea, } \bar{x} = 0.02: &\quad w = \text{clip}(0.02/0.05, 0, 1) = \text{clip}(0.4, 0, 1) = 0.4, \\ \text{land, } \bar{x} = 0.20: &\quad w = \text{clip}(0.20/0.05, 0, 1) = \text{clip}(4.0, 0, 1) = 1.0. \end{aligned}$$

It works on set 1 (F001 at epoch 52: 0.731 → 0.887 [M: `indradhanuworkings/46-the-ring-and-the-comparison.md`]) and leaves the other scenes almost untouched. That is the problem. It is a patch: only set 1 has water, so four scenes cannot tell whether 0.05 is right, and the result is steep in t (F001 at epoch 14 scores 0.524 on set 1 with $t = 0.03$ and 0.810 with $t = 0.08$ [M: `indradhanuworkings/45-scoring-the-direction.md`]). It fixes darkness only; stripes and haze are untouched. And the next failure would need another special case. The fade is opt-in (`None` by default) and the product does not pass it.

31.2 A residual is a direction

Think of a residual as an arrow in a C-dimensional space, one axis per band. Its *length* is roughly what L1 measures. Its *direction* says which bands erred, and how they erred together.

Now look at what the shared errors have in common. Haze pushes many bands the same way at once: one direction. A miscalibrated detector column pushes its pixels another way: one direction. Darkness of the sea: another. A scene's ordinary errors are large along *a few shared directions* and tiny along all the others. A real anomaly, a material the model never learned to predict there, errs in a direction the background rarely takes, even if the arrow is short.

Analogy

A doctor reads twenty blood tests. A dehydrated patient has every concentration a little high, but they all rise *together*, because there is less water; the doctor reads them as one ordinary finding, not twenty alarms. One test far outside *its own* normal range while the rest are ordinary is the result that matters, even if its deviation is smaller in absolute units than the dehydration's. Residual Mahalanobis reads residuals the same way: movements many pixels share are one ordinary finding, and the scale of each test's "normal range" is learned from the scene itself. A movement nobody else makes is the alarm.

Definition 31.1 (Residual covariance and residual Mahalanobis distance)**.** Let $\mathbf{r}_1, \ldots, \mathbf{r}_N$ be the residuals of the scene's valid pixels. Their mean is $\boldsymbol{\mu}_r = \frac{1}{N}\sum_i \mathbf{r}_i$ and their *residual covariance* is the $C \times C$ matrix

$$\boldsymbol{\Sigma}_r = \frac{1}{N-1}\sum_{i=1}^{N}(\mathbf{r}_i - \boldsymbol{\mu}_r)(\mathbf{r}_i - \boldsymbol{\mu}_r)^\top. \tag{31.2}$$

The *residual Mahalanobis distance* of a pixel with residual $\mathbf{r}$ is

$$D^2(\mathbf{r}) = (\mathbf{r} - \boldsymbol{\mu}_r)^\top \boldsymbol{\Sigma}_r^{-1} (\mathbf{r} - \boldsymbol{\mu}_r). \tag{31.3}$$

The divisor $N - 1$ is the one `np.cov` uses, which the code calls; Chapter 6 explains why it is the unbiased choice. Equation (31.3) is exactly the RX detector of Chapter 7 with the cube replaced by the residual. The rest of this section computes it by hand.

31.2.1 Worked by hand: three bands, four ordinary pixels

Worked example 31.1: Residual Mahalanobis on three bands

Do this on paper; it takes about fifteen minutes. Three bands A, B and C. Four ordinary pixels have these residuals (in units of 0.001 reflectance, to keep the numbers whole):

pixel	r_A	r_B	r_C
1	1	2	−1
2	1	−1	−1
3	−2	−1	−1
4	0	0	3

Three test pixels: $\mathbf{P} = (3, 3, 0)$, $\mathbf{Q} = (2, -2, 0)$ and $\mathbf{R} = (0, 0, 4)$.

Step 1: the mean, one band at a time.

$$\mu_A = \frac{1 + 1 + (-2) + 0}{4} = \frac{0}{4} = 0,$$
$$\mu_B = \frac{2 + (-1) + (-1) + 0}{4} = \frac{0}{4} = 0,$$
$$\mu_C = \frac{(-1) + (-1) + (-1) + 3}{4} = \frac{0}{4} = 0.$$

The mean residual is $(0, 0, 0)$, so every deviation from the mean is the residual itself.

Step 2: the covariance, one entry at a time. Each entry is a sum of products over the four pixels, divided by $N - 1 = 4 - 1 = 3$. The three variances:

$$\sigma_{AA} = \frac{1^2 + 1^2 + (-2)^2 + 0^2}{3} = \frac{1 + 1 + 4 + 0}{3} = \frac{6}{3} = 2,$$
$$\sigma_{BB} = \frac{2^2 + (-1)^2 + (-1)^2 + 0^2}{3} = \frac{4 + 1 + 1 + 0}{3} = \frac{6}{3} = 2,$$
$$\sigma_{CC} = \frac{(-1)^2 + (-1)^2 + (-1)^2 + 3^2}{3} = \frac{1 + 1 + 1 + 9}{3} = \frac{12}{3} = 4.$$

The three covariances:

$$\sigma_{AB} = \frac{1 \cdot 2 + 1 \cdot (-1) + (-2)(-1) + 0 \cdot 0}{3} = \frac{2 - 1 + 2 + 0}{3} = \frac{3}{3} = 1,$$
$$\sigma_{AC} = \frac{1 \cdot (-1) + 1 \cdot (-1) + (-2)(-1) + 0 \cdot 3}{3} = \frac{-1 - 1 + 2 + 0}{3} = \frac{0}{3} = 0,$$
$$\sigma_{BC} = \frac{2 \cdot (-1) + (-1)(-1) + (-1)(-1) + 0 \cdot 3}{3} = \frac{-2 + 1 + 1 + 0}{3} = \frac{0}{3} = 0.$$

A covariance matrix is symmetric ($\sigma_{BA} = \sigma_{AB}$), so

$$\boldsymbol{\Sigma}_r = \begin{pmatrix} 2 & 1 & 0 \\ 1 & 2 & 0 \\ 0 & 0 & 4 \end{pmatrix}.$$

Read it: bands A and B each vary by 2 and move together ($\sigma_{AB} = 1 > 0$); band C varies twice as much and independently of the other two.

Step 3: the inverse. The matrix is *block diagonal*: an A–B block and a C block, with zeros between them. Its inverse is the inverse of each block, in place. The C block is the number 4, whose inverse is $\frac{1}{4}$. For the A–B block use the rule for a 2×2 matrix,

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}^{-1} = \frac{1}{ad - bc} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}, \tag{31.4}$$

which you can check by multiplying: the top-left entry of the product is $\frac{1}{ad-bc}(a\cdot d+b\cdot(-c)) = \frac{ad-bc}{ad-bc} = 1$, and the top-right is $\frac{1}{ad-bc}(a\cdot(-b)+b\cdot a) = 0$. Here $a=2$, $b=1$, $c=1$, $d=2$:

$$ad-bc = 2\cdot 2 - 1\cdot 1 = 4-1 = 3,$$
$$\begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}^{-1} = \frac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

So

$$\boldsymbol{\Sigma}_r^{-1} = \begin{pmatrix} \frac{2}{3} & -\frac{1}{3} & 0 \\ -\frac{1}{3} & \frac{2}{3} & 0 \\ 0 & 0 & \frac{1}{4} \end{pmatrix}.$$

Check one row of $\boldsymbol{\Sigma}_r\boldsymbol{\Sigma}_r^{-1}$ against the identity. Row 1 of $\boldsymbol{\Sigma}_r$ is $(2,1,0)$:

$$\begin{aligned} &\text{with column 1: } 2\cdot\tfrac{2}{3} + 1\cdot(-\tfrac{1}{3}) + 0\cdot 0 = \tfrac{4}{3} - \tfrac{1}{3} = 1, \\ &\text{with column 2: } 2\cdot(-\tfrac{1}{3}) + 1\cdot\tfrac{2}{3} + 0\cdot 0 = -\tfrac{2}{3} + \tfrac{2}{3} = 0, \\ &\text{with column 3: } 2\cdot 0 + 1\cdot 0 + 0\cdot\tfrac{1}{4} = 0. \end{aligned}$$

That is $(1,0,0)$, the first row of the identity.

Step 4: score each test pixel. First multiply $\boldsymbol{\Sigma}_r^{-1}$ by the residual, then take the dot product with the residual. For $\mathbf{P} = (3,3,0)$:

$$\begin{aligned} \boldsymbol{\Sigma}_r^{-1}\mathbf{P} &= \left(\tfrac{2}{3}\cdot 3 - \tfrac{1}{3}\cdot 3 + 0,\ -\tfrac{1}{3}\cdot 3 + \tfrac{2}{3}\cdot 3 + 0,\ \tfrac{1}{4}\cdot 0\right) \\ &= (2-1,\ -1+2,\ 0) = (1,1,0), \\ D^2(\mathbf{P}) &= 3\cdot 1 + 3\cdot 1 + 0\cdot 0 = 6. \end{aligned}$$

For $\mathbf{Q} = (2,-2,0)$:

$$\begin{aligned} \boldsymbol{\Sigma}_r^{-1}\mathbf{Q} &= \left(\tfrac{2}{3}\cdot 2 - \tfrac{1}{3}\cdot(-2),\ -\tfrac{1}{3}\cdot 2 + \tfrac{2}{3}\cdot(-2),\ 0\right) \\ &= \left(\tfrac{4}{3}+\tfrac{2}{3},\ -\tfrac{2}{3}-\tfrac{4}{3},\ 0\right) = (2,-2,0), \\ D^2(\mathbf{Q}) &= 2\cdot 2 + (-2)(-2) + 0 = 4+4 = 8. \end{aligned}$$

For $\mathbf{R} = (0,0,4)$:

$$\begin{aligned} \boldsymbol{\Sigma}_r^{-1}\mathbf{R} &= (0,\ 0,\ \tfrac{1}{4}\cdot 4) = (0,0,1), \\ D^2(\mathbf{R}) &= 0+0+4\cdot 1 = 4. \end{aligned}$$

And one ordinary pixel, pixel 1, $(1,2,-1)$:

$$\begin{aligned} \boldsymbol{\Sigma}_r^{-1}\mathbf{r}_1 &= \left(\tfrac{2}{3}\cdot 1 - \tfrac{1}{3}\cdot 2,\ -\tfrac{1}{3}\cdot 1 + \tfrac{2}{3}\cdot 2,\ \tfrac{1}{4}\cdot(-1)\right) = \left(0,\ 1,\ -\tfrac{1}{4}\right), \\ D^2(\mathbf{r}_1) &= 1\cdot 0 + 2\cdot 1 + (-1)(-\tfrac{1}{4}) = 0 + 2 + \tfrac{1}{4} = 2.25. \end{aligned}$$

Put the answers side by side (all computed above or in the book's script [C]):

pixel	residual	length $\|\mathbf{r}\|$	L1	D^2
ordinary 1–4		1.73–3.00	1.00–1.33	2.25 each
P	$(3,3,0)$	4.24	2.00	6
R	$(0,0,4)$	4.00	1.33	4
Q	$(2,-2,0)$	2.83	1.33	**8**

The ranking flips. By length, $\mathbf{P} > \mathbf{R} > \mathbf{Q}$; by L1, **P** first. By D^2, **Q** is the most anomalous and **P** only second. **P** errs along "A and B together", the direction the ordinary pixels share. **R** errs in band C, which is noisy anyway. **Q** is the shortest arrow, but it has A and B *disagreeing*, which the ordinary pixels rarely do.

A curiosity you may have noticed: all four ordinary pixels score exactly 2.25. That is not luck. With N pixels in $N-1$ dimensions, every pixel used to estimate the covariance scores $(N-1)^2/N = 3^2/4 = 2.25$ against it. The covariance has used up all its freedom fitting those few pixels. The code avoids this regime by estimating the covariance from up to 200,000 pixels.

31.3 Whitening: rotate, divide, measure

31.3.1 The eigen-decomposition turns the inverse into a sum

The code never forms $\mathbf{\Sigma}_r^{-1}$. It uses the eigenvectors (Chapter 6). A symmetric matrix has an *eigen-decomposition*

$$\mathbf{\Sigma}_r = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top, \tag{31.5}$$

where the columns $\mathbf{u}_k$ of $\mathbf{U}$ are unit-length, mutually perpendicular eigenvectors ($\mathbf{U}^\top\mathbf{U} = \mathbf{I}$) and $\mathbf{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_C)$ holds the eigenvalues, the variance of the residuals along each $\mathbf{u}_k$. Derive the inverse:

$$\mathbf{\Sigma}_r^{-1} = (\mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top)^{-1}$$

The inverse of a product is the product of the inverses in reverse order:

$$= (\mathbf{U}^\top)^{-1}\,\mathbf{\Lambda}^{-1}\,\mathbf{U}^{-1}$$

and since $\mathbf{U}^\top\mathbf{U} = \mathbf{I}$, the inverse of $\mathbf{U}$ is $\mathbf{U}^\top$ and the inverse of $\mathbf{U}^\top$ is $\mathbf{U}$:

$$= \mathbf{U}\,\mathbf{\Lambda}^{-1}\,\mathbf{U}^\top.$$

Substitute into Equation (31.3), writing $\mathbf{d} = \mathbf{r} - \boldsymbol{\mu}_r$:

$$D^2 = \mathbf{d}^\top\mathbf{U}\mathbf{\Lambda}^{-1}\mathbf{U}^\top\mathbf{d}$$

Group the product as $(\mathbf{U}^\top\mathbf{d})^\top\,\mathbf{\Lambda}^{-1}\,(\mathbf{U}^\top\mathbf{d})$, using $(\mathbf{U}^\top\mathbf{d})^\top = \mathbf{d}^\top\mathbf{U}$ (the transpose of a product reverses the order). The vector $\mathbf{y} = \mathbf{U}^\top\mathbf{d}$ has entries $y_k = \mathbf{u}_k^\top\mathbf{d}$, the coordinates of $\mathbf{d}$ along each eigenvector. $\mathbf{\Lambda}^{-1}$ is diagonal, so the product is a sum:

$$= \sum_{k=1}^{C} \frac{y_k^2}{\lambda_k} = \sum_{k=1}^{C} \left(\frac{\mathbf{u}_k^\top(\mathbf{r} - \boldsymbol{\mu}_r)}{\sqrt{\lambda_k}} \right)^2 .$$

Define the *whitened coordinates*

$$z_k = \frac{\mathbf{u}_k^\top(\mathbf{r} - \boldsymbol{\mu}_r)}{\sqrt{\lambda_k}}, \qquad D^2 = \sum_k z_k^2 = \|\mathbf{z}\|^2 . \tag{31.6}$$

Three operations: subtract the mean, *rotate* onto the eigenvectors, *divide* each coordinate by its direction's standard deviation $\sqrt{\lambda_k}$. After that the ordinary residuals spread equally in every direction, and the plain squared length is the Mahalanobis distance. Dividing by $\sqrt{\lambda_k}$ and not by λ_k matters: λ_k is a variance, in squared units, and a coordinate must be divided by a spread in its own units.

Worked example 31.2: The same scores by rotate and divide

The eigenvectors of $\mathbf{\Sigma}_r$ from worked example 31.1 can be found by inspection. Check each by multiplying:

$$\begin{aligned}
&\mathbf{u}_1 = \tfrac{1}{\sqrt{2}}(1,1,0): && \mathbf{\Sigma}_r\mathbf{u}_1 = \tfrac{1}{\sqrt{2}}(2+1,\ 1+2,\ 0) = 3\,\mathbf{u}_1, && \lambda_1 = 3,\\
&\mathbf{u}_2 = \tfrac{1}{\sqrt{2}}(1,-1,0): && \mathbf{\Sigma}_r\mathbf{u}_2 = \tfrac{1}{\sqrt{2}}(2-1,\ 1-2,\ 0) = 1\,\mathbf{u}_2, && \lambda_2 = 1,\\
&\mathbf{u}_3 = (0,0,1): && \mathbf{\Sigma}_r\mathbf{u}_3 = (0,\ 0,\ 4) = 4\,\mathbf{u}_3, && \lambda_3 = 4.
\end{aligned}$$

"A and B together" spreads with variance 3; "A and B disagreeing" with variance 1; band C with variance 4. Now rotate and divide, with $\frac{1}{\sqrt{2}} \approx 0.7071$. For $\mathbf{P} = (3,3,0)$:

$$\begin{aligned}
y_1 &= 0.7071\,(3+3) = 4.243, && z_1 = 4.243/\sqrt{3} = 4.243/1.732 = 2.449,\\
y_2 &= 0.7071\,(3-3) = 0, && z_2 = 0/1 = 0,\\
y_3 &= 0, && z_3 = 0/2 = 0,\\
D^2 &= 2.449^2 + 0 + 0 = 6.00.
\end{aligned}$$

For $\mathbf{Q} = (2,-2,0)$:

$$\begin{aligned}
y_1 &= 0.7071\,(2-2) = 0, && z_1 = 0,\\
y_2 &= 0.7071\,(2+2) = 2.828, && z_2 = 2.828/1 = 2.828,\\
y_3 &= 0, && z_3 = 0,\\
D^2 &= 0 + 2.828^2 + 0 = 8.00.
\end{aligned}$$

For $\mathbf{R} = (0,0,4)$: $y_3 = 4$, $z_3 = 4/\sqrt{4} = 4/2 = 2$, $D^2 = 2^2 = 4$. The same three numbers as with the inverse matrix, as the derivation promised. Running the product's own `whitened_pca` on the four ordinary pixels gives whitened coordinates whose squares sum to 2.25 for every pixel; it may return an eigenvector with the opposite sign, which changes nothing after squaring [C].

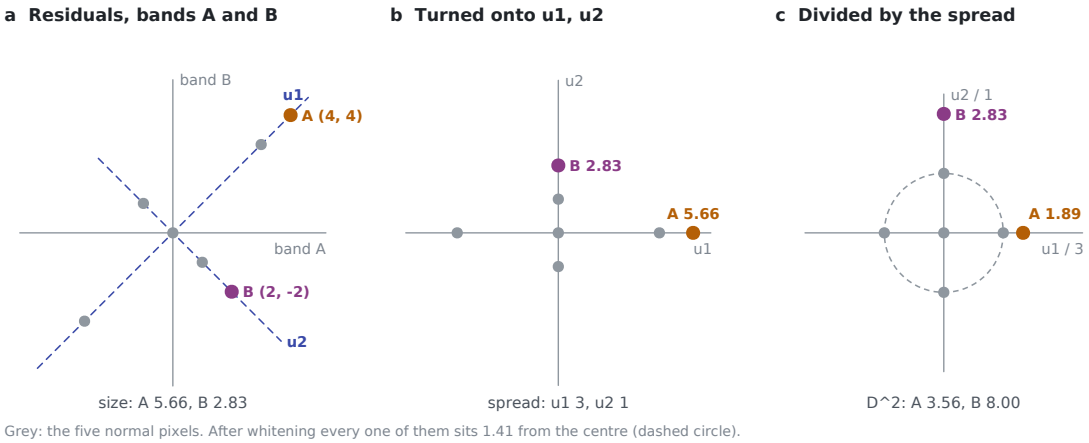


Figure 31.1: Rotate, divide, measure (course figure 45.1, the course's two-band version of the example above). (a) Ordinary residuals (grey) spread along the shared diagonal far more than across it; one test pixel lies further out along the diagonal, the other across it. (b) The same points turned onto the eigenvectors. (c) After dividing by each direction's spread the ordinary pixels sit on one circle and the across-the-diagonal pixel is far outside it.

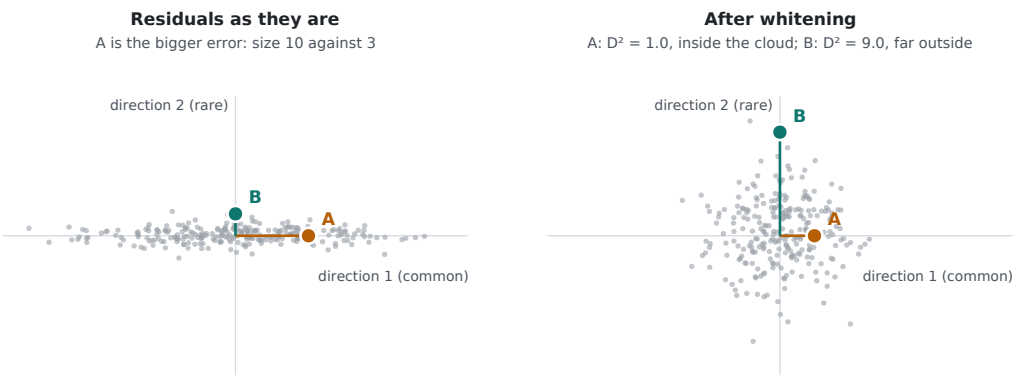


Figure 31.2: Size versus direction (anomaly course figure 12.3). Left: ordinary residuals spread widely along direction 1 and barely along direction 2. Right: after whitening the cloud is round; the pixel far out along the common direction is inside it, the one a short way along the rare direction is far outside.

31.3.2 The same steps in the code

`whitened_pca` (`residual_mahalanobis.py`, lines 38–49) does steps 1 to 4 for a whole scene:

```
X = residual[:, keep].T.astype(np.float64)          # (N, C): one row per valid pixel
mu = X.mean(0)                                       # (C,)   step 1: the mean
rng = np.random.default_rng(seed)                    # seed = 0: deterministic
sub = X[rng.choice(len(X), size=min(len(X), 200_000), replace=False)] - mu   # (<=200000, C)
evals, evecs = np.linalg.eigh(np.cov(sub.T))         # (C,), (C, C): steps 2-3 on a sample
order = np.argsort(evals)[::-1][:d]                  # (d,)   the d largest spreads, d = 30
P = ((X - mu) @ evecs[:, order]) / np.sqrt(np.maximum(evals[order], 1e-12))  # (N, d): rotate, divide
out = np.zeros((len(order),) + keep.shape, dtype=np.float32)                 # (d, H, W)
out[:, keep] = P.T                                   # whitened coordinates back on the image grid
```

and, with no local window, the score is `(X**2).sum(0)`: Equation (31.6). Three details:

- **A sample, with a fixed seed.** The covariance comes from at most 200,000 valid pixels (`_SAMPLE`, line 35) drawn with seed 0, so the score is deterministic and the cost of the eigen problem does not grow with the scene. The mean uses every valid pixel.
- **The mean is the scene's own.** Every scene gets its own directions and spreads. A stripe or a haze bank that dominates *this* scene is learned, and discounted, here.
- **The top $d = 30$ directions only** (`PCA_COMPONENTS`, line 32), with a floor of 10^{-12} on each eigenvalue. The next section explains both.

31.4 Regularisation: why only the top 30 directions

31.4.1 Small eigenvalues explode

Equation (31.6) divides by $\sqrt{\lambda_k}$. If an estimated eigenvalue is tiny, a tiny residual along it becomes huge. Watch it happen on a 2×2 covariance with unit variances and correlation c close

to 1, scoring the small residual $(0.01, -0.01)$, which lies exactly along the direction of the small eigenvalue $1 - c$ (computed by the book's script [C]):

c	eigenvalues	diagonal of $\mathbf{\Sigma}^{-1}$	D^2 of $(0.01, -0.01)$
0.9	0.1, 1.9	5.26	0.002
0.99	0.01, 1.99	50.3	0.02
0.999	0.001, 1.999	500	0.2

Every factor of ten in the smallest eigenvalue is a factor of ten in the score. Push it to $\lambda = 10^{-10}$ (a standard deviation of 0.00001) and a residual of 0.001 along it contributes

$$\frac{0.001^2}{10^{-10}} = \frac{0.000001}{0.0000000001} = 10{,}000.$$

One meaningless direction now outscores everything. With many bands the smallest eigenvalues of an *estimated* covariance are mostly estimation noise. This is why plain RX, Mahalanobis on the raw cube over all bands, was dropped from the product on 2026-05-11, with distances reaching about 10^{11} [M: `CLAUDE.md`].

31.4.2 Three remedies, and the one the code uses

- **Truncation** (what the code does): keep only the d directions with the largest eigenvalues. The smallest kept eigenvalue of 30 out of about 185 carries real variance; the explosive ones are discarded.
- **A floor**: `np.maximum(evals,1e-12)` only prevents a division by exactly zero; at 10^{-12} it does not tame anything else.
- **Shrinkage** (not used here): add a small ε to every eigenvalue, $\mathbf{\Sigma} + \varepsilon\mathbf{I}$ (Chapter 8). At $c = 0.999$ with $\varepsilon = 0.01$ the small eigenvalue becomes 0.011 and the score of $(0.01, -0.01)$ drops from 0.2 to $0.0002/0.011 = 0.018$.

How many numbers does the covariance estimate? A $d \times d$ symmetric matrix has $d(d+1)/2$ distinct entries: the diagonal plus the triangle above it. For $d = 30$:

$$\frac{30 \times 31}{2} = \frac{930}{2} = 465,$$

from up to 200,000 pixels: well determined. The full covariance over 185 bands would need $185 \times 186/2 = 17{,}205$ entries, and over 165 bands $165 \times 166/2 = 13{,}695$; the smallest directions of those are the ones that explode.

What truncation costs. Truncation throws away the directions in which the background barely moves, and those are exactly where a small anomaly like **Q** stands out. Repeat worked example 31.2 keeping only the top $d = 2$ directions, $\lambda_3 = 4$ and $\lambda_1 = 3$, and dropping $\lambda_2 = 1$:

$$\mathbf{P} : z = (0,\ 2.449) \to D^2 = 6.0,$$
$$\mathbf{R} : z = (2,\ 0) \to D^2 = 4.0,$$
$$\mathbf{Q} : z = (0,\ 0) \to D^2 = 0.$$

Q disappears. In a real scene the trade is better than in this toy, because the 155 discarded directions of a 185-band residual are mostly noise and a real material differs from its background in many bands at once, so it has a component along the kept directions too. But the trade is real: keeping 30 is a choice for stability over sensitivity in the quietest directions, made once on four labelled scenes.

31.4.3 Why on the residual, and how much the model adds

RX on the cube, Reed and Yu's 1990 detector [36], is the same distance with a different input. A raw scene mixes very different surfaces (sea, crops, towns), so one set of directions fits it badly. The model has already explained "what surface is this", and what it leaves behind is closer to one cloud of ordinary errors, which is what a single Mahalanobis distance assumes. Be honest about the size of that effect. On the four PRISMA scenes with the production model v0.2.0 (mean ROC-AUC, Chapter 33):

score	mean ROC-AUC
L1 on the residual	0.702
Mahalanobis, top 30, on the *cube*	0.831
Mahalanobis, top 30, on the *residual* (global mean)	0.839

Source: [M: `research/experiments/dual-window-mahalanobis/README.md`]

$$\begin{aligned} \text{gain from scoring the direction:} \quad & 0.839 - 0.702 = 0.137, \\ \text{gain from using the residual:} \quad & 0.839 - 0.831 = 0.008. \end{aligned}$$

Most of the improvement comes from whitening, not from the model. The README's own words: the residual "adds a small, consistent edge on top of a strong classical baseline".

31.5 The ring: a local mean with a global covariance

31.5.1 The global mean assumes one centre

Equation (31.3) measures every pixel from the scene's mean residual $\boldsymbol{\mu}_r$. That assumes every ordinary pixel's error scatters around one centre. It does not quite: the dual-window README puts it as "the model has a per-land-cover bias: its residual over water is not centred where its residual over crops is." Work it on one whitened coordinate. In a coastal scene the sea pixels sit around $+1.5$ along it and the land pixels around 0 [A]:

pixel	value	global score	ring mean	local score
ordinary sea	1.8	$1.8^2 = 3.24$	1.5	$(1.8 - 1.5)^2 = 0.3^2 = 0.09$
land target	1.6	$1.6^2 = 2.56$	0.0	$(1.6 - 0.0)^2 = 1.6^2 = 2.56$

Measured from the scene's centre the ordinary sea pixel outranks the target. Measured from its own neighbourhood it drops to almost nothing, and the target stands out.

31.5.2 The local score, derived

Let $\boldsymbol{\mu}_{\text{ring}}(p)$ be the mean residual over a ring of pixels around p. The *local-mean Mahalanobis distance* replaces the global mean with it but keeps the global covariance:

$$D^2_{\text{local}}(p) = (\mathbf{r}_p - \boldsymbol{\mu}_{\text{ring}}(p))^\top \boldsymbol{\Sigma}_r^{-1} (\mathbf{r}_p - \boldsymbol{\mu}_{\text{ring}}(p)). \tag{31.7}$$

The code computes it in whitened space. Whitening is a fixed linear map, $\mathbf{z} = \mathbf{W}(\mathbf{r} - \boldsymbol{\mu}_r)$ with $\mathbf{W} = \boldsymbol{\Lambda}^{-1/2}\mathbf{U}^\top$. The mean of a linear map over the ring is the linear map of the mean, because a sum can be taken inside a matrix product:

$$\bar{\mathbf{z}}_{\text{ring}} = \frac{1}{n} \sum_{q \in \text{ring}} \mathbf{W}(\mathbf{r}_q - \boldsymbol{\mu}_r) = \mathbf{W} \left(\frac{1}{n} \sum_{q \in \text{ring}} \mathbf{r}_q - \boldsymbol{\mu}_r \right) = \mathbf{W}(\boldsymbol{\mu}_{\text{ring}} - \boldsymbol{\mu}_r).$$

Subtract it from the pixel's own whitened vector; the global mean cancels:

$$\mathbf{z}_p - \bar{\mathbf{z}}_{\text{ring}} = \mathbf{W}(\mathbf{r}_p - \boldsymbol{\mu}_r) - \mathbf{W}(\boldsymbol{\mu}_{\text{ring}} - \boldsymbol{\mu}_r) = \mathbf{W}(\mathbf{r}_p - \boldsymbol{\mu}_{\text{ring}}).$$

Its squared length is Equation (31.7), restricted to the top d directions:

$$\|\mathbf{z}_p - \bar{\mathbf{z}}_{\text{ring}}\|^2 = (\mathbf{r}_p - \boldsymbol{\mu}_{\text{ring}})^\top \mathbf{W}^\top \mathbf{W}(\mathbf{r}_p - \boldsymbol{\mu}_{\text{ring}}).$$

So the code can whiten once, globally, and then average the whitened maps over rings. The covariance, the directions and spreads, still come from the whole scene; only the centre is local.

31.5.3 The ring and its guard

`LOCAL_WINDOW=(21,61)` (`residual_mahalanobis.py`, line 33) gives two full widths, both odd so that they centre on the pixel:

$$\begin{aligned} \text{outer box} &= 61 \times 61 = 3{,}721 \text{ pixels}, \\ \text{guard box} &= 21 \times 21 = 441 \text{ pixels}, \\ \text{ring} &= 3{,}721 - 441 = 3{,}280 \text{ pixels}. \end{aligned}$$

The *guard* is the inner box left out of the average. Without it, a target several pixels wide would sit partly in its own "background", pull the ring mean towards itself and shrink its own score. A guard 21 wide reaches $\lfloor 21/2 \rfloor = 10$ pixels from the centre, so anything within 10 pixels (in rows and in columns) is excluded; the outer box reaches $\lfloor 61/2 \rfloor = 30$.

Two edge rules from the code:

- Only *valid* pixels count: the ring mean is a sum of valid values divided by the number of valid pixels.
- If fewer than `MIN_RING_FACTOR` $\times d = 2 \times 30 = 60$ valid pixels remain in a ring (at a scene edge, or beside a nodata hole), that pixel falls back to its global-mean score (`residual_mahalanobis.py`, lines 34 and 67–69).

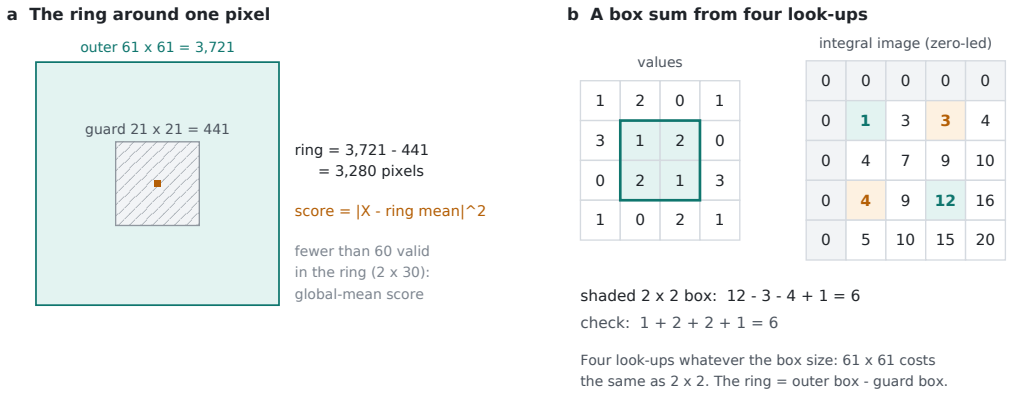


Figure 31.3: The ring, and a box sum from four look-ups (course figure 46.1). (a) The 61×61 window minus the hatched 21×21 guard leaves a ring of 3,280 pixels whose mean is subtracted from the centre pixel. (b) A 4×4 grid and its zero-led integral image: the shaded 2×2 box sums to 6 from four entries of the table.

31.5.4 Why not a local covariance too?

That is the classical dual-window RX of Chapter 8: mean *and* covariance from the ring. On the residual it scored worse, 0.814 against the local mean's 0.854 mean AUC with v0.2.0, in step 4 of the dual-window experiment log [M: `research/experiments/dual-window-mahalanobis/EXPERIMENT_LOG.md`]. A ring of a few hundred to a few thousand pixels, of which many are correlated neighbours, cannot estimate 465 covariance numbers reliably, and it soaks up nearby targets. The log's own conclusion: "The local covariance is the problem, not locality."

31.6 Summing a ring fast: integral images

31.6.1 The cost of doing it directly

The ring mean needs, at every pixel, the sum of 3,280 values in each of 30 whitened channels, plus the count of valid pixels: 31 channels. Done directly on the 1210×1219 PRISMA frame of Chapter 30:

$$\begin{aligned} \text{pixels} &= 1210 \times 1219 = 1{,}474{,}990, \\ \text{additions} &= 3{,}280 \times 31 \times 1{,}474{,}990 \\ &= 101{,}680 \times 1{,}474{,}990 \\ &\approx 1.50 \times 10^{11}. \end{aligned}$$

About 150 billion additions [C]. An integral image reduces each box sum to four look-ups.

31.6.2 Definition and construction

Definition 31.2 (Integral image)**.** The *integral image* (or *summed-area table*) of an $H \times W$ grid v is the $(H+1) \times (W+1)$ table

$$I[i][j] = \sum_{r=0}^{i-1} \sum_{c=0}^{j-1} v[r][c], \qquad 0 \le i \le H,\ 0 \le j \le W, \tag{31.8}$$

the sum of everything strictly above and to the left of position (i, j). Row 0 and column 0 are all zero (empty sums); the table is *zero-led*.

You can build it entry by entry with a recurrence. The block above-left of $(i+1, j+1)$ is the value $v[i][j]$ plus the block above it plus the block to its left; those two blocks share the block above-left of (i, j), which has then been counted twice, so subtract it once:

$$I[i+1][j+1] = v[i][j] + I[i][j+1] + I[i+1][j] - I[i][j]. \tag{31.9}$$

The code does something equivalent and faster: a running sum down each column, then a running sum along each row (`cumsum(1).cumsum(2)` on a `(channels, H, W)` array), then a row and column of zeros in front.

Worked example 31.3: Building an integral image on a 4×4 grid

Take the course's grid:

$$v = \begin{pmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 0 \\ 0 & 2 & 1 & 3 \\ 1 & 0 & 2 & 1 \end{pmatrix}.$$

Running sum down each column. Each entry is itself plus everything above it. Column 0: 1, $1+3=4$, $4+0=4$, $4+1=5$. Column 1: 2, $2+1=3$, $3+2=5$, $5+0=5$. Column 2: 0, $0+2=2$, $2+1=3$, $3+2=5$. Column 3: 1, $1+0=1$, $1+3=4$, $4+1=5$:

$$\begin{pmatrix} 1 & 2 & 0 & 1 \\ 4 & 3 & 2 & 1 \\ 4 & 5 & 3 & 4 \\ 5 & 5 & 5 & 5 \end{pmatrix}.$$

Running sum along each row of that table. Row 0: 1, $1+2=3$, $3+0=3$, $3+1=4$. Row 1: 4, $4+3=7$, $7+2=9$, $9+1=10$. Row 2: 4, $4+5=9$, $9+3=12$, $12+4=16$. Row 3: 5, $5+5=10$, $10+5=15$, $15+5=20$.
Add the zero row and column in front:

$$I = \begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 3 & 3 & 4 \\ 0 & 4 & 7 & 9 & 10 \\ 0 & 4 & 9 & 12 & 16 \\ 0 & 5 & 10 & 15 & 20 \end{pmatrix}.$$

Check one entry with the recurrence (31.9), the one for value row 1, column 1:

$$I[2][2] = v[1][1] + I[1][2] + I[2][1] - I[1][1] = 1 + 3 + 4 - 1 = 7,$$

and directly: the top-left 2×2 block is $1+2+3+1=7$. The last entry, 20, is the sum of the whole grid.

31.6.3 The four-term box sum

A box over rows $r_0, \ldots, r_1$ and columns $c_0, \ldots, c_1$ has sum

$$S = I[r_1+1][c_1+1] - I[r_0][c_1+1] - I[r_1+1][c_0] + I[r_0][c_0]. \tag{31.10}$$

Derive it from the definition. Call the box B and name three other regions, each a block that starts at the top-left corner of the grid:

- Big: rows $0..r_1$, columns $0..c_1$; its sum is $I[r_1+1][c_1+1]$.
- T (top strip): rows $0..r_0-1$, columns $0..c_1$; its sum is $I[r_0][c_1+1]$.
- L (left strip): rows $0..r_1$, columns $0..c_0-1$; its sum is $I[r_1+1][c_0]$.

Big is the box plus the parts of T and L:

$$\text{Big} = B + (T \cup L).$$

T and L overlap in the corner $T \cap L$: rows $0..r_0-1$, columns $0..c_0-1$, with sum $I[r_0][c_0]$. Adding T and L counts the corner twice, so

$$\text{sum}(T \cup L) = \text{sum}(T) + \text{sum}(L) - \text{sum}(T \cap L).$$

Substitute and solve for the box:

$$\text{sum}(B) = \text{sum}(\text{Big}) - \text{sum}(T) - \text{sum}(L) + \text{sum}(T \cap L),$$

which is Equation (31.10). Four look-ups, three additions or subtractions, whatever the size of the box: a 61×61 box costs the same as a 2×2.

Worked example 31.4: A box sum and a ring mean from the table

With I from worked example 31.3. The box of value rows 1–2 and columns 1–2 ($r_0 = 1$, $r_1 = 2$, $c_0 = 1$, $c_1 = 2$):

$$\begin{aligned} S &= I[3][3] - I[1][3] - I[3][1] + I[1][1] \\ &= 12 - 3 - 4 + 1 \\ &= 6. \end{aligned}$$

Check by adding the four values: $1 + 2 + 2 + 1 = 6$.

A ring is one box minus another. Shrink the windows to a toy: outer box 3×3, guard 1×1, around value row 1, column 1. The outer box is rows 0–2, columns 0–2:

$$S_{\text{outer}} = I[3][3] - I[0][3] - I[3][0] + I[0][0] = 12 - 0 - 0 + 0 = 12.$$

The guard is the single value at row 1, column 1:

$$S_{\text{guard}} = I[2][2] - I[1][2] - I[2][1] + I[1][1] = 7 - 3 - 4 + 1 = 1.$$

Ring:

$$\begin{aligned} \text{ring sum} &= 12 - 1 = 11, \\ \text{ring count} &= 3 \times 3 - 1 \times 1 = 9 - 1 = 8, \\ \text{ring mean} &= \frac{11}{8} = 1.375. \end{aligned}$$

The pixel's value is 1, so its local score in this one-channel toy is $(1 - 1.375)^2 = (-0.375)^2 = 0.140625$. Running the product's `local_mean_score` on this grid with `window=(1,3)` returns exactly 0.140625 at that pixel [C].

31.6.4 Counts, padding and the code

`local_mean_score` (`residual_mahalanobis.py`, lines 58–69), annotated:

```
d, H, W = X.shape                                    # X: (30, H, W) whitened, 0 outside
    keep
inner, outer = window                                # (21, 61)
r = outer // 2                                       # 30
F = np.concatenate([keep[None], X * keep], 0).astype(np.float64)   # (31, H, W):
    channel 0 = 1 if valid
I = np.pad(np.pad(F, ((0, 0), (r, r), (r, r))).cumsum(1).cumsum(2),
           ((0, 0), (1, 0), (1, 0)))                 # (31, H+2r+1, W+2r+1): zero-led
    integral image
A = _box(I, outer // 2, r, H, W) - _box(I, inner // 2, r, H, W)   # (31, H, W): ring
    count, ring sums
n = A[0]                                             # (H, W) valid pixels in each ring
ok = keep & (n >= MIN_RING_FACTOR * d)               # at least 60
s = ((X - A[1:] / np.maximum(n, 1)) ** 2).sum(0)   # (H, W): |z - ring mean|^2
return (np.where(ok, s, (X ** 2).sum(0)) * keep).astype(np.float32)   # fallback:
    global-mean score
```

and `_box` (lines 52–55) is Equation (31.10) for every pixel at once:

```
a, b = r - k, r + k + 1        # k = half-width of the box: 30 for the outer box, 10
    for the guard
```

```
return I[:, b:b+h, b:b+w] - I[:, a:a+h, b:b+w] - I[:, b:b+h, a:a+w] + I[:, a:a+h, a:a
    +w]
```

Three things make it exact.

- **Counts ride along as channel 0.** `keep` is 1 for valid pixels, so its box sum is the valid count. Invalid pixels were set to 0 in `X*keep`, so they add nothing to the sums either.
- **Padding by** r surrounds the scene with zeros, so a box hanging over the edge counts fewer pixels, and the count channel records how many.
- **The index arithmetic.** Pixel (i, j) of the scene sits at $(i + r, j + r)$ in the padded array. A box of half-width k around it spans padded rows $i + r - k, \ldots, i + r + k$, so by Equation (31.10) it needs I at rows $i + (r - k)$ and $i + (r + k + 1)$: the slices starting at $a = r - k$ and $b = r + k + 1$.

The whole score takes about 3–4 seconds per scene on a CPU, against about 46 seconds for F001's reconstruction [M: `docs/lld/residual-mahalanobis.md`].

31.6.5 Local covariances from the same trick, and why they are not used

Integral images give any local statistic that is a sum. A local covariance needs the sums of $\mathbf{x}$ and of $\mathbf{x}\mathbf{x}^\top$. Derive it from the definition with divisor n:

$$\boldsymbol{\Sigma}_{\text{loc}} = \frac{1}{n}\sum_q (\mathbf{x}_q - \mathbf{m})(\mathbf{x}_q - \mathbf{m})^\top$$

Expand the product of the brackets into four terms:

$$= \frac{1}{n}\sum_q \mathbf{x}_q\mathbf{x}_q^\top - \frac{1}{n}\sum_q \mathbf{x}_q\mathbf{m}^\top - \frac{1}{n}\sum_q \mathbf{m}\mathbf{x}_q^\top + \frac{1}{n}\sum_q \mathbf{m}\mathbf{m}^\top$$

and use $\frac{1}{n}\sum_q \mathbf{x}_q = \mathbf{m}$ in the middle terms and $\frac{1}{n}\sum_q 1 = 1$ in the last:

$$= \frac{1}{n}\sum_q \mathbf{x}_q\mathbf{x}_q^\top - \mathbf{m}\mathbf{m}^\top - \mathbf{m}\mathbf{m}^\top + \mathbf{m}\mathbf{m}^\top$$
$$= \frac{1}{n}\sum_q \mathbf{x}_q\mathbf{x}_q^\top - \mathbf{m}\mathbf{m}^\top.$$

Each entry $\sum_q x_{q,a}x_{q,b}$ is a box sum of the product image $x_a x_b$, so it too costs four look-ups.

Worked example 31.5: A local covariance from five box sums

Two channels a and b, one 3×3 box of $n = 9$ pixels (values from the book's script [C]):

$$a = \begin{pmatrix} 0 & 0 & 0 \\ 3 & 2 & 2 \\ 2 & 2 & 2 \end{pmatrix}, \qquad b = \begin{pmatrix} 0 & 0 & 3 \\ 0 & 1 & 1 \\ 0 & 0 & 0 \end{pmatrix}.$$

The five box sums:

$$\sum a = 0+0+0+3+2+2+2+2+2 = 13,$$
$$\sum b = 0+0+3+0+1+1+0+0+0 = 5,$$
$$\sum a^2 = 0+0+0+9+4+4+4+4+4 = 29,$$
$$\sum ab = 0+0+0+0+2+2+0+0+0 = 4,$$
$$\sum b^2 = 0+0+9+0+1+1+0+0+0 = 11.$$

Mean: $\mathbf{m} = (13/9,\ 5/9) = (1.444,\ 0.556)$. Covariance entries, in ninths and eighty-firsts:

$$\sigma_{aa} = \frac{29}{9} - \left(\frac{13}{9}\right)^2 = \frac{261}{81} - \frac{169}{81} = \frac{92}{81} = 1.136,$$
$$\sigma_{ab} = \frac{4}{9} - \frac{13}{9}\cdot\frac{5}{9} = \frac{36}{81} - \frac{65}{81} = -\frac{29}{81} = -0.358,$$
$$\sigma_{bb} = \frac{11}{9} - \left(\frac{5}{9}\right)^2 = \frac{99}{81} - \frac{25}{81} = \frac{74}{81} = 0.914.$$

These match `np.cov(...,bias=True)` on the nine pixels [C].

The cost argument. With integral images every pixel needs two boxes (outer and guard) of four look-ups per channel. For the 31 channels of the local mean on the PRISMA frame:

$$\begin{aligned} \text{direct:} &\quad 3{,}280 \times 31 \times 1{,}474{,}990 \approx 1.50 \times 10^{11}, \\ \text{integral:} &\quad 2 \times 4 \times 31 \times 1{,}474{,}990 = 365{,}797{,}520 \approx 3.7 \times 10^{8}, \\ \text{ratio:} &\quad \frac{3{,}280}{2 \times 4} = 410. \end{aligned}$$

Building the tables adds about two additions per entry per channel (two running sums) over the padded grid, $(1210 + 61) \times (1219 + 61) = 1271 \times 1280 = 1{,}626{,}880$ entries:

$$2 \times 31 \times 1{,}626{,}880 = 100{,}866{,}560 \approx 1.0 \times 10^{8},$$

small beside the direct method. A local covariance with $d = 30$ would need 465 product channels on top of the 30 sums and the count, 496 in all. In `float64` on the padded grid that is

$$496 \times 1{,}626{,}880 \times 8 \text{ bytes} = 6{,}455{,}459{,}840 \text{ bytes} \approx 6.5 \text{ GB},$$

against $31 \times 1{,}626{,}880 \times 8 = 403{,}466{,}240$ bytes, about 0.4 GB, for the local mean [C]; and every pixel would then need its own 30×30 inverse. Slower, sixteen times the memory, and, as measured, worse.

31.7 Eleven scores, fixed before the run

On 2026-10-15 eleven scoring methods were compared on the four labelled PRISMA scenes with the F001 model at epoch 52 (and on v0.2.0), with their parameters *fixed before the run*: nothing was tuned on the results. The comparison scripts are not in the repository; the methods are named in `docs/lld/residual-mahalanobis.md`, and the numbers below are the course's record of the measured results (IW46). A letter in brackets means "applied on top of": E(C) is method E run on C's score map.

Values that will drift

F001 at epoch 52, measured 2026-10-15 (`indradhanu workings/46-the-ring-and-the-comparison.md`; the means for C and D and the object counts 403 and 383 are confirmed by `docs/lld/residual-mahalanobis.md`). ROC-AUC per scene and the mean; "objects" is the number of labelled objects with at least one pixel in the top 1%.

method	set1	set2	set3	set4	mean	objects
A `combined` (old default)	0.731	0.755	0.677	0.698	0.715	208
A + SAM dark fade	0.887	0.755	0.685	0.698	0.756	
B `combined`, rank-normalised					0.709	
C residual Mahalanobis, global mean	0.921	0.848	0.751	0.876	0.849	
D residual Mahalanobis, ring 21/61	**0.929**	**0.859**	**0.787**	**0.885**	**0.865**	383
E(C) local z-score of the map					0.790	
F(C) difference-of-Gaussians boost					0.718	403
G(C) ±1 px shift suppression					0.845	

Eight of the eleven are shown; the other three were chained variants, none of which beat D. Blank cells were not recorded.

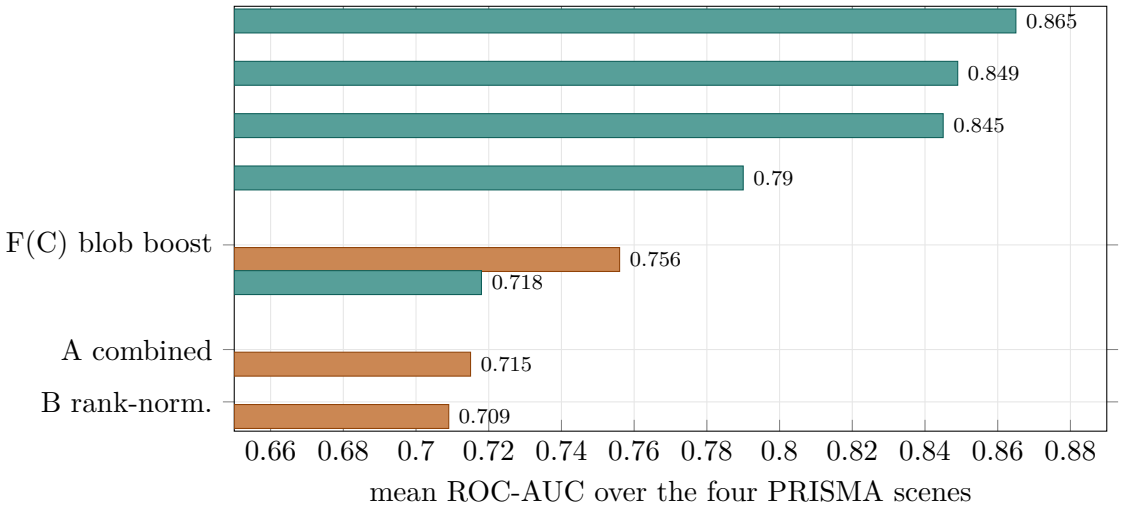


Figure 31.4: The comparison drawn to scale from the table (F001, epoch 52, 2026-10-15). Amber: scores built on the residual's size; teal: scores built on its direction (C, D and the filters applied to C). The axis starts at 0.65 to separate the bars; the gap from A to C is $0.849 - 0.715 = 0.134$.

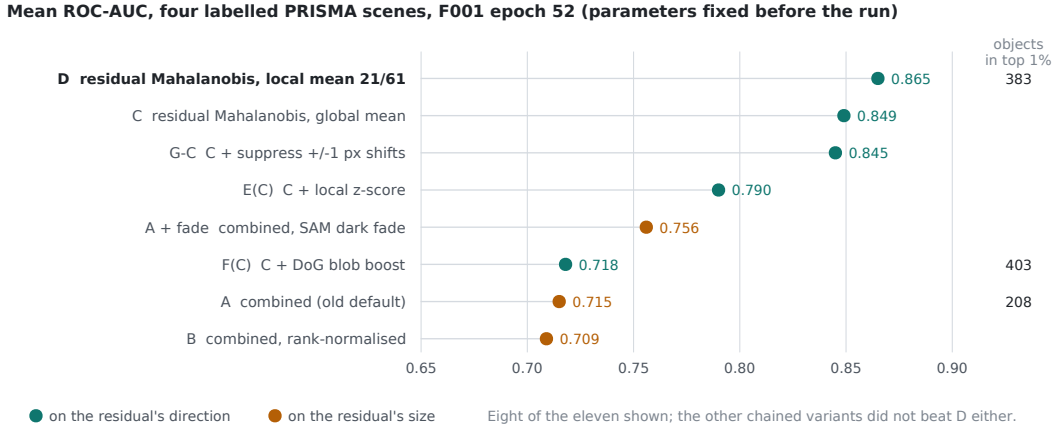


Figure 31.5: The same comparison as a dot plot (course figure 46.2), with object counts where they were recorded.

How to read it.

- **Size versus direction is the big step.** Every direction-based score except the blob

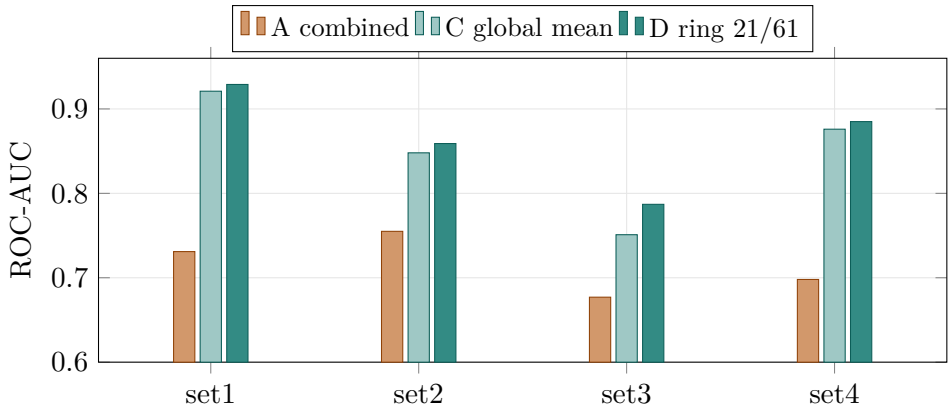


Figure 31.6: Per-scene ROC-AUC of A, C and D, to scale (F001, epoch 52). D beats C on every scene; the axis starts at 0.6.

boost is above every size-based score. A to C is $0.849 - 0.715 = 0.134$.

- **D beats C on every scene**, not only on average:

  set1: $0.929 - 0.921 = 0.008$, set2: $0.859 - 0.848 = 0.011$,
  set3: $0.787 - 0.751 = 0.036$, set4: $0.885 - 0.876 = 0.009$.

  A gain on every scene is more convincing than a gain in the mean.

- **Spatial post-filters did not help AUC.** A local z-score (each score re-expressed against its neighbourhood's spread), a blob boost (a difference of two blurs that favours compact blobs) and shift suppression (meant to discount errors that vanish if the reconstruction moves by one pixel) all scored below C.

- **AUC and object counts can disagree.** F(C) catches the most objects, 403 against D's 383, but its AUC is 0.718. AUC judges the ranking of *every* pixel; the object count looks only at the top 1% and gives one point per object touched (Chapter 33). A blob boost lifts compact objects into the top 1% while scrambling the ranking everywhere else. Against D it gave up $0.865 - 0.718 = 0.147$ of AUC for $403 - 383 = 20$ more objects. The product needs the ranking: an analyst moves the threshold, so the whole order matters.

D became Indradhanu's default on 2026-10-15 (roadmap row OB-13): the resolver lists `residual_mahalanobis` first for both Indradhanu models, and `combined`, `L1` and `SAM` stay selectable through the per-model override (`backend/allotrope/foundation_models/resolver.py`, lines 152–173).

Values that will drift

The same comparison with the production model v0.2.0 (`research/experiments/dual-window-mahalanobis/README.md`, section 3): `combined` 0.662, L1 0.702, SAM 0.570, MNF-RX 0.752, PCA30-RX on the cube 0.831, residual Mahalanobis global mean 0.839, ring 21/61 0.854. Set 3 is the exception there: PCA30-RX on the cube (0.766) beats residual Mahalanobis with the global mean (0.709).

31.7.1 Three honest caveats

The window was chosen on these scenes. The LLD says it plainly: "The window was the best of four in the original sweep on the same four scenes, so the margin is slightly flattering." When you pick the best of four settings on a test, part of its lead is luck that will not repeat

on new scenes. D's lead over C, $0.865 - 0.849 = 0.016$, is small enough that some of it may be that luck. That is why `local_window=None`, the global version, which has no window to choose, stays one argument away.

Four scenes are very few. If the two methods were truly equal, each scene would be a coin flip for which one wins:

$$P(\text{all 4 go to D}) = \left(\tfrac{1}{2}\right)^4 = \tfrac{1}{16},$$
$$P(\text{all 4 go one way or the other}) = 2 \times \tfrac{1}{16} = \tfrac{2}{16} = 0.125.$$

Even a clean sweep happens one time in eight by chance, so no paired test on four scenes can reach the usual 0.05. The evidence is the consistency and the size of the gap, not a significance test (Chapter 36 develops a better tool).

The ground truth may sit about one pixel off. AUC counts that against every method equally; the object count allows one pixel of slack.

Source check: the class-aware variant

The same module also defines `class_aware_residual_mahalanobis` (`residual_mahalanobis.py`, lines 75–102): a separate whitening per surface class from a segmentation, with same-class rings, and classes under 5,000 pixels merged into land. The product's scoring action does not call it; it is used by the evaluator's scorers (`app/evaluation/scorers.py`, lines 32–59) for experiments. When this book says "residual Mahalanobis" it means the default, class-blind version.

31.8 Common confusions

Is residual Mahalanobis a new model?
No. The model and its reconstruction are unchanged; only the step that turns a residual into a score differs. A checkpoint needs no retraining to use it.

Plain RX was dropped for exploding. Why is this allowed back?
Two changes: only the 30 largest directions are used, so the tiny, noisy eigenvalues that exploded are discarded; and it runs on the residual, which is closer to one cloud of ordinary errors than the raw cube.

Is a high score a big error?
Not necessarily, and that is the point. **Q** in worked example 31.1 has the smallest error and the highest score.

There is already a `mahalanobis` in the product.
That is the classical detector on the cube (MNF-RX, thermal GRX). The new method is deliberately named `residual_mahalanobis` so the two are never confused in configurations or the interface.

Local mean, so the covariance is local too?
No. Only the mean comes from the ring. The directions and spreads are the whole scene's.

Why does a ring pixel's validity matter if its whitened value is already 0?
A zero value would still be counted in the denominator. The count channel makes the mean divide by valid pixels only.

Can I threshold the score at, say, 10?

No. Scores are squared distances with a heavy right tail, and their scale depends on the scene. Like every score in the product, it is thresholded by percentile (Chapter 32) and drawn with a square-root stretch.

31.9 Check yourself

Exercise 31.1. Using the four ordinary pixels of worked example 31.1, score a new pixel with residual $(1, 1, 2)$ by $\mathbf{\Sigma}_r^{-1}$, and again by rotate-and-divide. Compare its L1 with that of $\mathbf{Q}$.

Exercise 31.2. Using the integral image of worked example 31.3, compute the sum of value rows 0–1, columns 2–3, and the toy ring mean (3×3 outer, 1×1 guard) around value row 1, column 2. Check both by adding values.

Exercise 31.3. Four pixels in two channels have $\sum a = 8$, $\sum b = 4$, $\sum a^2 = 20$, $\sum ab = 6$, $\sum b^2 = 6$. Compute the mean and the covariance (divide by n) from these sums alone.

Exercise 31.4. For a window `(11,31)`: how many pixels are in the ring, what is the fallback threshold if $d = 20$, and by what factor do integral images reduce the per-pixel cost of the ring sums?

Exercise 31.5. In the 3-band example keep only the top two directions. Score $(1, 1, 2)$ and $\mathbf{Q}$, and say in one sentence what truncation gave up.

Exercise 31.6. A fifth labelled scene arrives and D beats C there too. What is now the chance of a clean sweep in one direction or the other if the methods were equal?

Next: Chapter 32, Naming the material, export, and using Indradhanu V2 in a chain.

Chapter 32

Naming the material, export, and using Indradhanu V2 in a chain

The one thing this chapter teaches

A score map ranks pixels; a person turns the ranking into a decision by choosing a percentile, the flagged pixels are then compared with a library of laboratory spectra to propose what they are made of, and the result leaves the system as a bundle whose coordinates are recovered, at the last moment, from the raw file. Running the newer model, Indradhanu V2, in that chain needs a band filter that keeps native bands, the default since 2026-10-31. A redesign of the naming step, agreed and built on 2026-10-31 (Chapter 44), asks first whether nature could explain a pixel and only then which man-made class it is.

Chapter 30 reconstructed a whole scene and Chapter 31 turned the residual into a score per pixel. The product's chain does not stop there (Figure 30.5). Three steps remain, and this chapter takes them in order: the threshold, which only a person may set; the naming of materials, whose mathematics fills a companion volume and is only sketched here; and the export, where geography returns. The chapter ends with the analyst's view of Indradhanu V2: what to change in the chain to use it, what goes wrong if you do not, and what extra files it leaves behind.

32.1 Why a threshold is a person's job

You have a score map. Which pixels are anomalies? The obvious fix, a fixed cut on the score, cannot work, because typical scores differ by an order of magnitude between scenes (AD12; the numbers are illustrative [A]):

scene	typical score	fixed cut	result
calm lake	0.01	0.05	almost nothing flagged
burned area	0.10	0.05	most of the scene flagged

The same number is far too strict for one scene and far too loose for the other. Residual Mahalanobis makes this worse, not better: it is a squared distance whose scale depends on the scene's own covariance and on how many directions were kept.

So the convention is a *percentile*: "flag the top 1% of this scene". That adapts to each scene's own distribution, as a person looking at the map would. And *which* percentile is not a property of the data at all: it depends on how many candidates the team can review, how costly a miss is, and what the scene is. That is why the action that prepares the decision,

`anomaly_detection_prep`, ends in a status no other action uses, `needs_threshold`, and waits for a person (`backend/allotrope/action_types/anomaly_detection_prep.py`, line 53).

32.2 From score maps to an anomaly mask

32.2.1 Rescale, weight, combine

`anomaly_detection_prep` reads every model's `anomaly_score.tif` from one upstream `anomaly_scoring` output and builds one *composite score* (`backend/allotrope/action_types/_anomaly_detection_prep_run.py`, lines 88–191). Three steps:

1. **Min–max rescale each map** over its finite pixels,
$$\tilde{s}(p) = \frac{s(p) - s_{\min}}{s_{\max} - s_{\min}} \in [0, 1], \tag{32.1}$$
so that maps in different units (squared distances, angles, reflectance) become comparable. A flat map becomes all zeros.
2. **Normalise the weights**: the user's `algorithm_weights` (equal by default) are divided by their sum, $\omega_m = w_m / \sum_k w_k$, so the composite stays in $[0, 1]$.
3. **Combine**: $c(p) = \sum_m \omega_m \tilde{s}_m(p)$. A pixel is finite in the composite only if it is finite in every contributing map; otherwise it is NaN (not a number).

Note that this is a different normalisation from `combined`'s, which divides by the maximum only (worked example 30.7).

Worked example 32.1: A two-model composite at one pixel

Indradhanu's map runs from 0 to 36 and MNF-RX's from 2 to 82 over the scene; at one pixel they read 7.2 and 30 [A]. The user gives Indradhanu weight 2 and MNF-RX weight 1. On paper:

$$\begin{aligned}
\tilde{s}_{\text{Indradhanu}} &= \frac{7.2 - 0}{36 - 0} = \frac{7.2}{36} = 0.20, \\
\tilde{s}_{\text{MNF-RX}} &= \frac{30 - 2}{82 - 2} = \frac{28}{80} = 0.35, \\
\omega_{\text{Indradhanu}} &= \frac{2}{2+1} = \frac{2}{3}, \qquad \omega_{\text{MNF-RX}} = \frac{1}{2+1} = \frac{1}{3}, \\
c &= \tfrac{2}{3} \times 0.20 + \tfrac{1}{3} \times 0.35 \\
&= 0.1333 + 0.1167 \\
&= 0.25.
\end{aligned}$$

32.2.2 The percentile, as the code computes it

The viewer shows a slider. Each press of *Apply* calls `POST/actions/\{id\}/anomaly_detection_preview`, which converts the slider value q (a percentile, 0 to 100) into an absolute threshold and binarises the composite (`backend/allotrope/api/_anomaly_detection_preview.py`, lines 168–208):

```
absolute_threshold = float(np.percentile(finite_values, q))   # finite_values: sorted
    , (N,)
raw_mask = composite >= np.float32(absolute_threshold)         # (H, W) bool
raw_mask &= valid_mask                                         # NaN pixels are never
    flagged
```

An optional square dilation then grows the mask by a user-chosen odd kernel, and if a ground-truth annotation is attached the endpoint reports precision, recall and F1 (Chapter 33). Nothing is written until the user presses *Commit*, which calls `POST/actions/\{id\}/anomaly_detection_commit` and writes `anomaly_mask.tif` and `metrics.json` (`backend/allotrope/api/action_threshold.py`, lines 283–330).

Source check: commit is an endpoint, not an action

The prep action's docstring says finalisation "is deferred to a separate `anomaly_detection_commit` action" (`anomaly_detection_prep.py`, lines 21–25). In the source, commit is an api endpoint on the prep action itself, `anomaly_detection_commit` in `backend/allotrope/api/action_threshold.py`; there is no such action type in `backend/allotrope/action_types/`.

How many pixels does a percentile flag? `np.percentile` with its default (linear) interpolation places the q-th percentile of N sorted values at position

$$\text{pos} = \frac{q}{100}\,(N-1) \tag{32.2}$$

in the list, counting from 0, interpolating between neighbours when pos is not whole. If the values are all different, the ones at or above that threshold are those at positions $\lceil \text{pos} \rceil, \ldots, N-1$, so the number flagged is

$$n_{\text{flag}} = N - \lceil \text{pos} \rceil. \tag{32.3}$$

Worked example 32.2: Flagging the top 30% of ten pixels, and the top 0.5% of a scene

Ten composite values, sorted: $0.02, 0.05, 0.07, 0.10, 0.12, 0.15, 0.21, 0.33, 0.48, 0.90$ [A]. The slider is at $q = 70$:

$$\text{pos} = \frac{70}{100} \times (10-1) = 0.7 \times 9 = 6.3.$$

Position 6 holds 0.21 and position 7 holds 0.33; interpolate 0.3 of the way:

$$\text{threshold} = 0.21 + 0.3 \times (0.33 - 0.21) = 0.21 + 0.3 \times 0.12 = 0.21 + 0.036 = 0.246.$$

Flagged: the values ≥ 0.246, which are $0.33, 0.48, 0.90$: three pixels, and Equation (32.3) agrees, $10 - \lceil 6.3 \rceil = 10 - 7 = 3$.
Now the scene of worked example 30.8, with $N = 956{,}679$ kept pixels and $q = 99.5$:

$$\text{pos} = 0.995 \times (956{,}679 - 1) = 0.995 \times 956{,}678 = 951{,}894.61,$$
$$n_{\text{flag}} = 956{,}679 - \lceil 951{,}894.61 \rceil = 956{,}679 - 951{,}895 = 4{,}784.$$

The course's "$956{,}679 \times 0.005 = 4{,}783$" rounds the other way; the code flags 4,784 (checked by the book's script on distinct values [C]). Ties, which are common in maps with many exact zeros, can push the count higher, because every pixel equal to the threshold is flagged.

32.2.3 Which pixels the percentile counts

Whether "N" above is the kept pixels or the whole scene depends on the upstream models, because of how each kind marks the pixels it did not score.

- A *classical* detector writes NaN outside its mask (`_anomaly_scoring_run.py`, lines 535–541, the comment on classical NaNs). NaN pixels are not finite, so they are left out of the percentile population.
- A *foundation* model's score is set to exactly 0 outside the keep mask (`app/utils/anomaly_detection/scoring.py`, line 114), and the GeoTIFF is written with no nodata value. Zero is finite, so those pixels *are* in the population, at the bottom.

Source check: a model-only chain takes the percentile over the whole frame

The anomaly course's end-to-end walk (AD13) flags "the top 0.5%" of the 956,679 kept pixels. That holds when a classical detector (MNF-RX) is in the composite, because its NaNs remove the unkept pixels from the population. With Indradhanu alone, every one of the 1,474,990 pixels of the frame is finite, the 518,311 unkept ones being exact zeros, so $q = 99.5$ flags

$$1{,}474{,}990 - \lceil 0.995 \times 1{,}474{,}989 \rceil = 1{,}474{,}990 - 1{,}467{,}615 = 7{,}375 \text{ pixels},$$

which is $7{,}375/956{,}679 = 0.771\%$ of the kept pixels, not 0.5% [C]. To flag about 4,784 the slider would have to be at $q \approx 99.676$. The cut is still applied to a ranking the analyst can see, so nothing is wrong; but "top 0.5%" means 0.5% of the finite pixels, and which pixels are finite depends on the models chosen. Sources: `_anomaly_detection_preview.py`, line 84 (`valid_mask=np.isfinite`), and `_anomaly_detection_prep_run.py`, lines 179–191.

32.3 Naming the material

32.3.1 The question, and why only hyperspectral can answer it

A shortlist of unusual pixels is not yet useful. An analyst wants "probably a metal roof" or "probably calcite". Hyperspectral data can propose that, because a spectrum is close to a material's fingerprint (Chapter 2); a single thermal band cannot. The step that does it is the action `spectral_library_match`, implemented in `backend/allotrope/action_types/_spectral_library_match_run.py` on top of `app/spectral_match/`. Its full mathematics (resampling a laboratory spectrum through each band's response, the spectral angle, whitened target detection, cones of mixtures, non-negative least squares and the calibration against a scene's ordinary pixels) is the subject of *the companion volume on spectral matching* (`research/math/spectral-matching/`). This section gives the intuition and the outputs, so that you can read the chain end to end.

32.3.2 It reads the native cube, not the band-filtered one

The action loads the *native onboarding vendable*, not `band_filter_apply`'s output (`_spectral_library_match_run.py`, lines 155–164). Resampling onto the 165-band common grid (Chapter 5) is what makes sensors comparable for a model, but it blunts narrow absorption features, and those features are the evidence a material match relies on.

32.3.3 The spectral angle against a library, in one example

The library is USGS splib07 [20], curated to 519 laboratory spectra in `data/splib07_slim/` [M: `docs/training/anomaly-detection/14-material-and-export.md`]. Each library spectrum is first simulated through each band of the sensor: a band is not a point measurement but a weighted

average over a range of wavelengths, roughly Gaussian with full width at half maximum (FWHM) $\mathrm{FWHM} = 2\sqrt{2\ln 2}\,\sigma \approx 2.3548\,\sigma$. For a 10 nm band,

$$\sigma = \frac{10}{2.3548} = 4.247 \text{ nm}.$$

The first matcher, the *spectral angle mapper* [22], then ranks library entries by the angle between the pixel and each entry (Chapter 25). The angle ignores brightness, which is what you want for sun versus shadow, but it has a basic problem at 30-metre pixels: a roof, a truck or a tarpaulin rarely fills a pixel.

Worked example 32.3: SAM names the grass around a roof

The anomaly course's three-band example (AD15): grass $\mathbf{g} = (0.05, 0.40, 0.25)$, roof $\mathbf{r} = (0.30, 0.32, 0.35)$, and a pixel that is 80% grass and 20% roof:

$$\begin{aligned} x_1 &= 0.8 \times 0.05 + 0.2 \times 0.30 = 0.040 + 0.060 = 0.100, \\ x_2 &= 0.8 \times 0.40 + 0.2 \times 0.32 = 0.320 + 0.064 = 0.384, \\ x_3 &= 0.8 \times 0.25 + 0.2 \times 0.35 = 0.200 + 0.070 = 0.270. \end{aligned}$$

Angle to grass:

$$\begin{aligned} \mathbf{x}\cdot\mathbf{g} &= 0.100 \times 0.05 + 0.384 \times 0.40 + 0.270 \times 0.25 = 0.0050 + 0.1536 + 0.0675 = 0.2261, \\ \|\mathbf{x}\| &= \sqrt{0.0100 + 0.147456 + 0.0729} = \sqrt{0.230356} = 0.47995, \\ \|\mathbf{g}\| &= \sqrt{0.0025 + 0.1600 + 0.0625} = \sqrt{0.2250} = 0.47434, \\ \cos\theta_g &= \frac{0.2261}{0.47995 \times 0.47434} = \frac{0.2261}{0.22766} = 0.99315, \quad \theta_g = 6.7^\circ. \end{aligned}$$

Angle to roof:

$$\begin{aligned} \mathbf{x}\cdot\mathbf{r} &= 0.0300 + 0.12288 + 0.0945 = 0.24738, \\ \|\mathbf{r}\| &= \sqrt{0.09 + 0.1024 + 0.1225} = \sqrt{0.3149} = 0.56116, \\ \cos\theta_r &= \frac{0.24738}{0.47995 \times 0.56116} = \frac{0.24738}{0.26933} = 0.91851, \quad \theta_r = 23.3^\circ. \end{aligned}$$

SAM confidently names the surroundings. Subtracting the background first fixes it: with a ring of pure grass around the pixel, $\mathbf{x} - \mathbf{g} = (0.050, -0.016, 0.020)$, which is exactly $0.2 \times (\mathbf{r} - \mathbf{g})$. The difference points along "roof minus grass"; the 80% grass has vanished.

That subtraction is the first of five ideas in the *family matcher* (`app/spectral_match/family_match.py`) that has run after SAM since 2026-10-16 (AD15), and whose answer the viewer and the export now lead with. In one line each:

1. subtract the mean of a ring around the pixel (13×13 minus the central 5×5, 144 pixels; the same ring idea as Chapter 31, smaller);
2. measure how well the difference points along "library spectrum minus ring", after whitening, with the adaptive coherence estimator;
3. score a whole *family* of spectra (roofing, metal, plastic, …) as any non-negative mix of its members, found by non-negative least squares, for man-made families only;
4. calibrate each family score as a percentile against up to 10,000 ordinary pixels of the same scene;
5. give a man-made name only if that percentile reaches $p^* = 1 - \alpha/F$, where α is the share of

ordinary pixels you accept being given a man-made name and F the number of man-made families.

With the defaults, $\alpha = 0.05$ (`DEFAULT_ALPHA`, `family_match.py`, line 36) and $F = 10$ man-made families in the shipped library [M: `docs/training/anomaly-detection/15-naming-the-family.md`]:

$$\frac{\alpha}{F} = \frac{0.05}{10} = 0.005,$$
$$p^* = 1 - 0.005 = 0.995.$$

By the union bound, an ordinary pixel then gets *some* man-made name with probability at most $10 \times 0.005 = 0.05 = \alpha$. Why this works, and what it does not promise, is worked in full in the companion volume.

32.3.4 What the action writes

file	contents
`matches.parquet`	SAM: top-K library entries and angles per flagged pixel
`match_map.tif`	SAM: top-1 library index per pixel; -2 outside the mask, -1 no match
`family_matches.parquet`	family, group, calibrated confidence, named or not, best single entry
`family_map.tif`	int16 family code per pixel (below)
`family_legend.json`	code → family, group, colour; α and p^*
`summary.json`	counts per family, timings, `family.status`

The family codes (`_spectral_library_match_run.py`, lines 53–56 and 396–399):

code	meaning
-2	not an anomaly pixel
-1	not scored (nodata on a band the matcher uses)
0	best family is man-made but below p^*: "man-made, type unconfirmed"
$1, \ldots, 17$	that family, in sorted name order, plus one (natural families are shown at any confidence)

Family matching never fails the action: an exception is recorded as `family.status="failed"` in the summary, and the SAM outputs still make a complete result. It runs only in `mode="anomaly_pixels"`, because its per-pixel least squares cost about 2.5 ms per pixel (45 s for about 18,000 pixels, AD15): a million pixels would take about $1{,}000{,}000 \times 0.0025$ s $= 2{,}500$ s ≈ 42 minutes.

The output is a ranked hypothesis about a pixel the analyst already kept, not a filter that decides which pixels are real: false alarms outnumber targets about 50 to 1 on the PRISMA scenes, so most man-made names in a scene still land on false alarms (AD15).

32.4 A redesign: natural first, then the class

Source check: the design; the model is in Chapter 44

This section is the *why* of a redesign of the naming step agreed between the user and Claude on 2026-10-31. It was written up the same day as `docs/proposals/spectral-namer.md` and *built* the same day in `app/spectral_match/namer/`; its first run, N001, started training on an H100 at 11:30 UTC and has no results yet. Chapter 44 teaches the model as built, which departs from this section's design in places, most of all in how level 1 is trained (the

note after worked example 32.5). Nothing here runs in the product: the namer is wired into no action until it passes. The experiment README's heading "Design so far" predates the proposal. The one classical measurement, the separability check of Section 32.4.6, is `research/experiments/spectral-namer/separability.py`. The companion volume's Part VII derives the mathematics slowly.

32.4.1 Where the ten man-made families came from

`scripts/curate_family_library.py` builds `data/family_library/` from the USGS and ECOSTRESS libraries and gives every spectrum a group and a family by *regular expressions* (text patterns) on its file name, read top to bottom, first match wins. Four of the USGS chapter-A rules, in order:

```
(r"asphalt_shingle|asphalt_tar|tar_paper|roofing_felt|roofng|roofing|clear_roof|
    cedar_shake", "manmade", "roofing"),
(r"asphalt|concrete_gds375|brick.*pav", "manmade", "pavement"),
(r"brick|cinder|concrete", "manmade", "building"),
(r"plastic|polystyrene|tyvek|visqueen|fiberglass", "manmade", "plastic"),
```

So an asphalt shingle is roofing and an asphalt road is pavement; `concrete_gds375` is pavement and every other concrete is building. Three consequences:

1. **Families by use, not material.** Asphalt sits in two families and concrete in two, decided by a word in a file name. From orbit a sensor sees what a surface is made of, not what it is for.
2. **Family sizes reflect the libraries.** From `index.json`: building 11, glass 1, metal 13, oil 40, paint 8, pavement 15, plastic 39, roofing 27, textile 14, wood_paper 8; the 40s are `FAMILY_CAP=40` (`app/spectral_match/family_library.py`). Man-made $11 + 1 + 13 + 40 + 8 + 15 + 39 + 27 + 14 + 8 = 176$, natural 235, total 411.
3. **Some families cannot be told apart.** The smallest principal angle between the rank-3 subspaces of pavement and building is 0.51°; metal and plastic 2.04°; paint and plastic 5.92° (companion volume, `tab:angles`, computed there). At 0.51° the share of a unit target's energy outside the other family is $\sin^2(0.51°) = 0.0089^2 = 0.00008$: one material with two names.

32.4.2 What can be named from orbit

A car of $4.5 \times 1.8 = 8.1\,\mathrm{m}^2$ fills $8.1/900 = 0.009$ of a 30 m pixel and $8.1/25 = 0.32$ of a 5 m one. At PRISMA's and EnMAP's 30 m only large objects can be named; AVIRIS-NG, flown low, reaches vehicle scale (about 5 m in the discussion; it depends on flight height [A]). The discussion replaced the use-based families with a defence-oriented class list: *camouflage* and *military coatings* (no library spectra at all; they need field spectra), *painted metal*, *bare metal*, *vessels on water*, *synthetic fabric and tarpaulins*, *fuel and oil*, *concrete and asphalt* (one class, ending the split above), *disturbed earth*, *cut vegetation*, *glass and photovoltaic panels*, and *none / natural*. The two classes an analyst wants most are the two no library holds.

32.4.3 The two-level model

Naming becomes two questions in turn. Level 1: could nature explain this pixel? Level 2, only if not: which man-made class? There are three possible answers: *natural*, *man-made, class unknown*, and *man-made plus a class*. Write M for "man-made" and k for a class. Because a

class can only be man-made, the event "class k" is the event "man-made and class k", and the definition of conditional probability gives

$$P(k) = P(M)\,P(k \mid M). \tag{32.4}$$

Worked example 32.4: The product of the two levels

Level 1 gives $P(M) = 0.9$; level 2 gives painted metal 0.6, bare metal 0.25, tarp 0.15 [A].

$$\begin{aligned} P(\text{painted metal}) &= 0.9 \times 0.6 = 0.54 \\ P(\text{bare metal}) &= 0.9 \times 0.25 = 0.225 \\ P(\text{tarp}) &= 0.9 \times 0.15 = 0.135 \\ P(\text{natural}) &= 1 - 0.9 = 0.10 \\ \text{sum} &= 0.54 + 0.225 + 0.135 + 0.10 = 1.00. \end{aligned}$$

With $P(M) = 0.3$ instead, painted metal is $0.3 \times 0.6 = 0.18$. The same level-2 answer is worth 0.54 behind a confident level 1 and 0.18 behind a doubtful one: the product stops a confident class name from hiding a doubtful "is it man-made at all?".

Level 1 is a model of nature. The obvious level 1 is a classifier trained on library spectra of both kinds. Its man-made side would only know library materials, and camouflage is not in the library. The natural side is different: every scene is mostly natural pixels, and the project holds billions of them (SpectralEarth and the native shards). So level 1 is trained to say whether nature explains a pixel, and anything it cannot explain is called man-made, library or not. This is *open-set* recognition: the model may meet something it has never seen and say so.

Worked example 32.5: Near the natural average, off the natural line

In two dimensions, let nature lie on the line $x + y = 1$, sampled at $(0.1, 0.9)$, $(0.3, 0.7)$, $(0.5, 0.5)$, $(0.7, 0.3)$, $(0.9, 0.1)$; the library has one man-made spectrum at $(0.9, 0.9)$; a camouflage net sits at $(0.4, 0.4)$ [A].

$$\begin{aligned} \text{natural average} &= (0.5,\ 0.5) \\ \text{camouflage to it} &= \sqrt{0.1^2 + 0.1^2} = \sqrt{0.02} = 0.141 \\ \text{camouflage to the library} &= \sqrt{0.5^2 + 0.5^2} = \sqrt{0.5} = 0.707 \\ &\Rightarrow \text{ "natural" by nearest average;} \\ \text{residual } |x + y - 1| \text{ of each natural sample} &= 0 \\ \text{residual of the camouflage} &= |0.4 + 0.4 - 1| = 0.2 \\ &\Rightarrow \text{ not natural.} \end{aligned}$$

The camouflage is closer to the natural average than the two end samples are (each $\sqrt{0.4^2 + 0.4^2} = 0.566$ from it), so no rule built on distance to an average can catch it. It is off the line nature lives on, and a model of that line notices.

Source check: as built, level 1 is a two-class head

The built namer does not train level 1 as a model of nature. Its level-1 logit is trained with binary cross-entropy on planted samples, library *man-made* spectra mixed into real pixels against natural library spectra and plain pixels (`app/spectral_match/namer/train.py`, `loss`; `generator.py`, the three sample kinds). It does see man-made examples. Whether

it still flags a material it never saw is what run N002 measures, by holding the whole paint/coatings class out (Chapter 44).

32.4.4 How ACE matches, and why the fill drops out

Level 2 needs a match score. The separability check uses ACE in the form of Section 32.3: $\mathbf{d} = \mathbf{x} - \mathbf{m}$ (pixel minus ring mean), $\mathbf{s}_j = \mathbf{t}_j - \mathbf{m}$ (library spectrum j minus ring mean), both whitened, and

$$\mathrm{ACE}_j = \frac{(\mathbf{s}_j \cdot \mathbf{d})^2}{(\mathbf{s}_j \cdot \mathbf{s}_j)(\mathbf{d} \cdot \mathbf{d})}, \tag{32.5}$$

the squared cosine between the two whitened directions.

Worked example 32.6: ACE on three bands, at two fills

Ring mean grass $(0.05, 0.40, 0.25)$, roof $(0.30, 0.32, 0.35)$, soil $(0.20, 0.25, 0.30)$, band standard deviations $(0.02, 0.04, 0.01)$ [A]. Whitened directions (difference divided band by band):

$$\mathbf{s}_{\text{roof}} = (0.25/0.02,\ -0.08/0.04,\ 0.10/0.01) = (12.5,\ -2,\ 10), \qquad \mathbf{s}\cdot\mathbf{s} = 260.25$$
$$\mathbf{s}_{\text{soil}} = (0.15/0.02,\ -0.15/0.04,\ 0.05/0.01) = (7.5,\ -3.75,\ 5), \qquad \mathbf{s}\cdot\mathbf{s} = 95.3125.$$

A 20% roof: $\mathbf{d} = 0.2\,\mathbf{s}_{\text{roof}} = (2.5,\ -0.4,\ 2)$, $\mathbf{d}\cdot\mathbf{d} = 6.25 + 0.16 + 4 = 10.41$.

$$\text{soil: } \mathbf{s}\cdot\mathbf{d} = 18.75 + 1.5 + 10 = 30.25,$$
$$\mathrm{ACE} = 30.25^2/(95.3125 \times 10.41) = 915.06/992.20 = 0.9223.$$

Roof scores 1.00 (the directions are parallel). A 50% roof: $\mathbf{d} = (6.25,\ -1,\ 5)$, $\mathbf{d}\cdot\mathbf{d} = 65.0625$, soil $\mathbf{s}\cdot\mathbf{d} = 75.625$, ACE $= 5719.14/6201.27 = 0.9223$, the same. Scaling $\mathbf{d}$ by the fill a multiplies the numerator by a^2 and the denominator by a^2: **fill changes the length of d, not its direction**.
Noise does not scale with fill. Add one standard deviation per band, whitened $(1, 1, -1)$. At fill 0.2, $\mathbf{d} = (3.5,\ 0.6,\ 1)$, roof $\mathbf{s}\cdot\mathbf{d} = 52.55$, $\mathbf{d}\cdot\mathbf{d} = 13.61$, ACE $= 2761.50/3542.00 = 0.780$. At fill 0.5, $\mathbf{d} = (7.25,\ 0,\ 4)$, $\mathbf{s}\cdot\mathbf{d} = 130.625$, $\mathbf{d}\cdot\mathbf{d} = 68.5625$, ACE $= 17062.89/17843.39 = 0.956$ [C].

Leave one spectrum out. To measure naming, the check plants each library spectrum and matches it against the other 410, setting its own score to -1 (`ace[e]=-1`); matching a spectrum against itself would always win and measure nothing.

32.4.5 Matching versus a classifier

A *closed classifier* ends in one output per class; a new class needs a new output and retraining. A *matching model* (a dual encoder) has two networks: one embeds the pixel and its ring as a short vector, the other embeds a library spectrum as a vector of the same length; the score is their cosine, and naming takes the best library entry. A new library needs no retraining: embed its spectra and they are candidates. With a pixel embedded at $(0.8, 0.6)$ and library entries painted metal $(1, 0)$, tarp $(0, 1)$ and a newly added glass spectrum $(0.6, 0.8)$, the cosines are 0.80, 0.60 and $0.8 \times 0.6 + 0.6 \times 0.8 = 0.96$: the new entry wins without any training [A]. ACE is itself a matching model whose two "encoders" are fixed: subtract the ring and whiten. **Level 1 does not depend on any library**, because it models nature; a new library changes only level 2's candidates. The price is that a matching model generalises only to spectra like those its encoder

learned from, while a classifier trained on a fixed list can be sharper on that list. The companion volume's Part VII derives the contrastive loss that trains a dual encoder; Chapter 44 shows the class-level version that was built.

32.4.6 The separability check

Before training anything, `separability.py` bounds what a strong classical matcher can do: "No training - this bounds what the physics and the library allow before we design classes." For each scene (the four PRISMA bench scenes; three implant-bench host scenes each for EnMAP and AVIRIS-NG) it resamples the 411 spectra to the scene's bands, plants each into 12 real background pixels at least 8 pixels from any object as $\mathbf{x} = g\,(a\mathbf{t} + (1-a)\mathbf{b}) + \mathbf{n}$, with a smooth gain error $|g-1| \le 0.1$ and the scene's own per-band noise, and matches $\mathbf{d} = \mathbf{x} - g\,\mathbf{m}$ by ACE against every other spectrum, whitened with the scene's covariance (30 components plus a ridge). The best match's family is level 2, its group level 1. Plantings per fill: PRISMA $411 \times 12 \times 4 = 19{,}728$; EnMAP and AVIRIS-NG $411 \times 12 \times 3 = 14{,}796$.

	L1: man-made right / natural right			L2: class right		
fill	PRISMA	EnMAP	AVIRIS-NG	PRISMA	EnMAP	AVIRIS-NG
0.10	0.61 / 0.78	0.48 / 0.72	0.59 / 0.77	0.38	0.23	0.36
0.25	0.77 / 0.83	0.66 / 0.79	0.76 / 0.81	0.55	0.44	0.53
0.50	0.85 / 0.86	0.80 / 0.84	0.84 / 0.85	0.61	0.58	0.59
1.00	0.87 / 0.86	0.87 / 0.87	0.87 / 0.85	0.64	0.64	0.62

Table 32.1: The separability check, all three sensors, final on 2026-10-31. On PRISMA, level 2 right *given* level 1 right is 0.63 / 0.72 / 0.72 / 0.73. Source: [M: `research/experiments/spectral-namer/README.md`] and the three `separability_<sensor>.json` files.

A class can be right only if level 1 was, so Equation (32.4) holds on the counts: at PRISMA fill 0.10, $0.6096 \times 0.6309 = 0.3846$, the table's 0.38 [C].

Per family, PRISMA at fill 0.25 (from the JSON's confusion counts [C]): oil 1537/1920 = 80%, plastic 1319/1872 = 70%, textile 464/672 = 69%, wood and paper 239/384 = 62%, paint 174/384 = 45%, metal 255/624 = 41%, building 205/528 = 39% (16% called pavement), roofing 398/1296 = 31%, pavement 75/720 = 10% (15% called soil, 13% rock, 12% building), glass 0/48.

Source check: pavement is 10%, not 12%

The experiment README gives pavement at fill 0.25 as "∼12%". The JSON's pavement row has 75 of 720 right, 10.4%; the 12% is pavement called *building*, 87 of 720.

Four readings. Level 1 is the harder half at small fills: at 10% fill, 39% of man-made plantings are called natural on PRISMA and 52% on EnMAP, and most targets at 30 m are that small; at full fill all three sensors reach 0.87. Materials with their own absorption features (hydrocarbon, polymer C-H, cellulose) separate well: oil, plastic, textile, wood and paper. The use-based families fail as the regex section predicted: roofing is scattered, building and pavement confuse each other and natural rock and soil, and glass, with one spectrum, can never be matched once it is left out. And these are library spectra planted in real pixels, so the laboratory-to-satellite gap is absent: a real namer will do worse.

The README turns this into level 2's class list: *oil/fuel, plastic, textile/fabric, wood/paper, paint/coatings, metal, concrete/asphalt* (building and pavement merged, roofing dissolved into its materials), with glass/PV once spectra exist and camouflage and military coatings once field spectra arrive. "These classical numbers are the baseline the learned namer must beat, mainly at 10-25% fill." This list of *materials* is narrower than the analyst's list of *things* above: vessels,

disturbed earth and cut vegetation are contexts or mixtures, not single spectra.

Name what nature cannot explain before naming what it is, multiply the two answers, and match against spectra rather than classify into a fixed list, so that the unknown is flagged and a new library costs no retraining.

Values that will drift

The design (2026-10-31, now `docs/proposals/spectral-namer.md`), the class lists, the family counts (411 spectra, cap 40), and every separability number; a new library, class list or set of host scenes changes them.

32.5 Export: where geography comes back

32.5.1 Every action GeoTIFF has an identity transform

A GeoTIFF places its pixel grid on the Earth through an *affine transform*: six numbers (a, b, c, d, e, f) that map a column j and row i to map coordinates,

$$X = a\,j + b\,i + c, \qquad Y = d\,j + e\,i + f, \tag{32.6}$$

together with a *coordinate reference system* (CRS) that says what X and Y mean (degrees of longitude and latitude, metres in a UTM zone, …). Vendables carry no spatial reference (Chapter 4), so every raster an action writes uses the *identity* transform, $a = e = 1$ and $b = c = d = f = 0$, and no CRS (`_anomaly_scoring_render.py`, line 61):

$$X = j, \qquad Y = i.$$

Pixel coordinates pretending to be map coordinates. Analysis works in pixel grids and does not need more; only the deliverable does.

32.5.2 Recovering the CRS at export

The export endpoint, `POST/actions/\{id\}/export` in `backend/allotrope/api/action_export.py`, runs synchronously in the api. It re-opens the raw scene file through `app/georef/from_scene_file.py`, which knows each sensor:

- **EnMAP L2A, AVIRIS-NG, Landsat 9, HotSat**: the raw file is already a georeferenced raster; rasterio reads its CRS and transform. If the vendable's shape differs from the source's, the transform is scaled so that the outer corners stay pinned.
- **PRISMA L2D**: since 2026-10-31 the product's own UTM grid, from the HE5's root attributes `Epsg_Code` and `Product_{UL,UR,LL}corner_{easting,northing}`, which are the corner pixels' *centres* (`prisma_utm_grid`; worked for Set-1 in worked example 45.1). Only if those are missing, or the vendable's shape differs from the product's, does it fall back to the older method: the minimum and maximum of the per-pixel latitude and longitude arrays, as an axis-aligned EPSG:4326 transform built with rasterio's `from_bounds`.

If none can be recovered the export returns `422crs_missing` rather than a bundle that a downstream evaluation would disqualify for having no projection.

Source check: the bounding box was not a fit, and it was pixels off

The anomaly course used to say export "reads the per-pixel latitude and longitude arrays … and fits an affine transform" (AD13). The code never fitted: it took min and max of the

arrays and called `from_bounds(min_lon,min_lat,max_lon,max_lat,W,H)`, which is exact only for a north-aligned swath and treats pixel-centre coordinates as edges. Its docstring called that "the industry-standard simplification for L2D HCO data". Measured on Set-1, it put pixels a median 2.4 px (95% 4.1, max 5.6) from the product grid [M: `docs/proposals/aigc-evaluation-submission.md`]; the evaluation, which compares masks in pixel space, was never affected. Since 2026-10-31 it is only the fallback, and the docstring now says "approximate, can be pixels off".

The worked example below is the fallback, kept because it shows what an affine transform does; the product grid is worked in worked example 45.1.

Worked example 32.7: An axis-aligned transform from a bounding box (the fallback)

A PRISMA frame of $W = 1219$ columns and $H = 1210$ rows whose latitude and longitude arrays span 77.00° to 77.36° east and 12.80° to 13.13° north (an assumed box, for illustration [A]). `from_bounds` sets

$$a = \frac{\text{east} - \text{west}}{W} = \frac{77.36 - 77.00}{1219} = \frac{0.36}{1219} = 0.00029532^\circ,$$
$$e = -\frac{\text{north} - \text{south}}{H} = -\frac{13.13 - 12.80}{1210} = -\frac{0.33}{1210} = -0.00027273^\circ,$$
$$c = \text{west} = 77.00, \qquad f = \text{north} = 13.13, \qquad b = d = 0.$$

e is negative because rows count downwards while latitude grows northwards. The centre of the pixel in row 605, column 610 is at $j = 610.5$, $i = 605.5$:

$$X = 0.00029532 \times 610.5 + 77.00 = 0.18029 + 77.00 = 77.18029^\circ \text{ E},$$
$$Y = -0.00027273 \times 605.5 + 13.13 = -0.16514 + 13.13 = 12.96486^\circ \text{ N}.$$

How big is a pixel on the ground? A degree of latitude is about 110.6 km; a degree of longitude at 13° north is about $111.32 \times \cos 13^\circ = 111.32 \times 0.9744 = 108.47$ km. So

$$\text{width} \approx 0.00029532 \times 108{,}470 \text{ m} = 32.0 \text{ m},$$
$$\text{height} \approx 0.00027273 \times 110{,}574 \text{ m} = 30.2 \text{ m},$$

close to PRISMA's 30 m, as it should be. The book's script reproduces these with rasterio [C].

32.5.3 What the bundle holds

For hyperspectral scenes the zip is built by `app/spectral_match/export.py`. Every name starts with `hyper_`, as the submission rules require, and the manifest records the schema: `allotrope-export-hyper/2` for actions with family matching, `/1` for older SAM-only ones.

```
hyper_<scene8>_<action8>/
    hyper_anomaly_materials.tif      int16 family codes, nodata -2, real CRS and
    transform
    hyper_anomaly_materials.shp      one point per anomaly pixel (+ .shx .dbf .prj .
    cpg)
                                     DBF: family, group, fconf, named, best_ent, mat1
    ..3, scr1..3
    hyper_material_legend.json       code -> family, group, colour; alpha and p*
    hyper_sam_legend.json            SAM's library index -> material
```

```
    hyper_match_summary.csv          flat table, no geometry
    MANIFEST.json                    schema allotrope-export-hyper/2
```

Material identity travels three ways (a GeoTIFF metadata tag, the DBF columns, the legend file) because different GIS tools read different parts, and a shortlist nobody can open is worthless. A committed `anomaly_detection_prep` has its own bundle, from `app/anomaly_detection/export.py`: a binary 0/1 mask and one point per anomaly pixel. Until 2026-10-31 it was always named `thermal_`; it is now `thermal_` for Landsat 9 and HotSat-1 and `hyper_` otherwise, with no score attribute in its shapefile. That bundle, not the materials bundle above, is what an evaluation submission uses (Chapter 45).

32.6 Using Indradhanu V2 in an action chain

32.6.1 Two words, and what V2 is

A *codename* is the name you pick a model by in the scoring action: "Indradhanu", "MNF-RX", "Indradhanu V2". The resolver maps it to a checkpoint and its capabilities (`backend/allotrope/foundation_models/resolver.py`). *Native bands* are the wavelengths the sensor actually measured, PRISMA's own list or EnMAP's own list, as opposed to the 165-band common grid that `band_filter_apply` resampled to by default until 2026-10-31 (Chapter 5).

Indradhanu V2 is the spectral-stem model of Part IV, fine-tuned with a segmentation head (Chapter 29) and shipped as an opt-in codename (Chapter 39). It reads native bands, scores with residual Mahalanobis like Indradhanu, and sets to 0 the pixels its own head calls cloud.

Values that will drift

Indradhanu V2 was wired into `anomaly_scoring` on 2026-10-19 serving FT04 (`v2.0.0`); on 2026-10-29 it was switched to Indradhanu Heavy, FTH3 epoch 5 (`v2.1.0`, 35.3 M parameters) (`finaldesign/ROADMAP.md`, row OB-14; `docs/04-models.md`). On the real PRISMA bench with cloud exclusion: v0.2.0 pAUC 0.165, FT04 0.170 (a tie), Heavy 0.189 (+0.020 over FT04, resolved). CPU cost on the worker: 1819 s per PRISMA scene for Heavy against 1058 s for FT04 on 2 threads (ROADMAP OB-14); 454 s and 274 s since the compose file raised the worker to 16 threads on 2026-10-29 (worked example 43.10). The scoring codename defaults still pick the old "Indradhanu" (v0.2.0); since 2026-10-31 the band-filter defaults keep native bands (Chapter 45).

Analogy

A film lab with two developers. The old one only takes 35 mm film; the new one only takes medium format. You choose the format when you load the camera, long before you reach the lab. Hand the new developer a 35 mm roll and it is refused at the door with a note: "medium format only, reload the camera". And one trip to the lab cannot develop both, because one camera was loaded with one kind of film. The camera is `band_filter_apply`; the developers are the two codenames; the note is the error below.

32.6.2 The chain, setting by setting

Two settings matter; since 2026-10-31 the first is the sensor default, so in the UI only the second is a choice:

```
Scene -> Project
  band_filter_apply        use_common_wavelength_grid: false     <- change 1 (default
     since 2026-10-31)
```

```
anomaly_scoring          model_codenames: ["Indradhanu V2"]    <- change 2
anomaly_detection_prep   unchanged: reads models/indradhanu_v2/anomaly_score.tif
spectral_library_match   unchanged: reads the native onboarding cube anyway
```

The per-model folder is the codename in lower case with spaces turned into underscores (`_safe_dirname`):

"Indradhanu V2" → "indradhanu v2" → "indradhanu_v2".

32.6.3 The error you get otherwise

Give V2 a band filter on the 165-band grid (before 2026-10-31 the default; now `use_common_wavelength_grid:true`, or an API call that omits the field, whose pydantic default is still `true`) and the action is still accepted (the api cannot see the upstream band filter's settings when you submit). The worker starts and, before loading the model, runs `_check_band_grid` (`_anomaly_scoring_run.py`). The action fails with:

```
model 'Indradhanu V2' reads native bands: run band_filter_apply with
use_common_wavelength_grid=false (this one resampled to the 165-band grid)
```

The reverse, the old "Indradhanu" on a native band filter, used to fail with "needs the 165-band grid: run `band_filter_apply` with `use_common_wavelength_grid=true`". Since 2026-10-31 it runs: the worker resamples the cube to the grid for that model alone (`_to_common_grid`, "the same resampler `band_filter_apply` uses"), and a comment where the check was says common165 on native bands "is not an error". Before 2026-10-19 nothing checked any of this.

How does the worker know which grid it was given? Not by counting bands:

```
on_grid = len(wavelengths) == len(grid) and np.allclose(wavelengths, grid)   # grid:
    460, 470, ..., 2450 nm
```

A native list with exactly 165 bands would still not equal $460, 470, \ldots, 2450$ nm and would count as native. Classical detectors skip the check and run on either grid; so do thermal scenes.

32.6.4 Running both models

`anomaly_scoring` takes one upstream band filter (`input_band_filter_output_id`), and every model in `model_codenames` reads that one output. Since 2026-10-31, on a native band filter, one action can run both: V2 reads the native cube and v0.2.0 gets it resampled. Whether v0.2.0 on a resampled native cube matches v0.2.0 on a 165-grid band filter exactly has not been measured; the resampler is the same function. Before 2026-10-31, V2 needed native bands and v0.2.0 the grid, whichever did not match failed the whole action, and comparing them took two branches in the same Project:

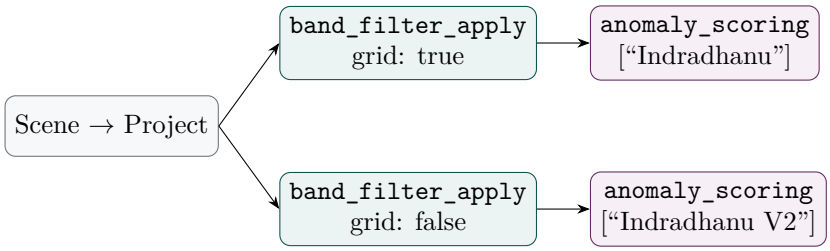


Figure 32.1: Before 2026-10-31, comparing the two Indradhanu models on one scene took two branches, each with its own band filter and scoring action. Since then one native band filter and one scoring action serve both. A classical detector can ride along in either.

One consequence of the old limit: `anomaly_detection_prep` reads a single scoring output (`input_anomaly_scoring_output_id`), so V2 and v0.2.0 could not be weighted into one composite. With both in one action on a native band filter, they can.

32.6.5 What comes out, and how to read it

Under the scoring action's output, in `models/indradhanu_v2/`, next to the usual files (`_anomaly_scoring_run.py`, lines 471–491 and 595–601):

output	contents
`mask_cloud.tif`	uint8, 1 where the score was set to 0 (V2 called the pixel cloud)
`class_map.tif`	uint8, V2's classes: 0 land, 1 water, 2 cloud, 3 shadow, 4 snow
`stats.json: bands_read`	how many bands V2 read (bands it never trained on are dropped first)
`stats.json: excluded_classes`	`["cloud"]`
`stats.json: cloud_rule`	`"thick"` or `"cloud_or_thin"`
`stats.json: excluded_percent`	100× cloud pixels / spatially valid pixels, to 3 places

The class map is built from the segmentation head's five sigmoid outputs, averaged over 128×128 windows at stride 64, by priority: cloud over snow over shadow over water over land (`app/foundation_models/inferencers/spectral_stem_inferencer.py`, lines 112–141). Only cloud is excluded; water is in the class map but *not* zeroed, because vessels on water are in scope.

Source check: since 2026-10-29 "cloud" means thick cloud only

The anomaly course (AD23, written for FT04) says `mask_cloud.tif` marks pixels V2 "called cloud or thin cloud". Since V2 became Indradhanu Heavy, the resolver sets `thin_cloud_is_cloud=False` (`backend/allotrope/foundation_models/resolver.py`, line 176): Heavy's thin-cloud output fires on haze, so only thick cloud is zeroed, and the new stats key `cloud_rule` records `"thick"` (`_anomaly_scoring_run.py`, lines 471–481).

The order matters. V2 scores every pixel in the keep mask first and sets cloud to 0 *afterwards*. A cloud pixel still takes part in fitting the residual Mahalanobis statistics of Chapter 31; only its own score becomes 0. That is the semantics the evaluator measured. A pixel scored 0 is not "unknown": it sinks to the bottom of the ranking. Read `excluded_percent` as the share of the scene in which you cannot find anything.

Worked example 32.8: The excluded percentage

A scene of 1000 × 1200 pixels, 72,000 invalid, 90,240 called cloud (AD23's example [A]):

$$\begin{aligned} \text{total} &= 1000 \times 1200 = 1{,}200{,}000, \\ \text{valid} &= 1{,}200{,}000 - 72{,}000 = 1{,}128{,}000, \\ \texttt{excluded_percent} &= \frac{100 \times 90{,}240}{1{,}128{,}000} = \frac{9{,}024{,}000}{1{,}128{,}000} = 8.000. \end{aligned}$$

Two caveats. EnMAP is cleaned twice: its default band filter applies the provider's quality masks (cloud, cloud shadow, haze) before scoring, and V2 then zeros its own cloud on top; this is the product's convention, but it is not how V2's PRISMA numbers were measured. And native PRISMA bands run downwards in wavelength (Chapter 37); `scene_segmentation` now picks

its visible bands by wavelength, not by position, so a native-band chain gets the right brightness too. The viewer does not show V2's masks yet; they are on disk only.

32.7 Common confusions

Why not a fixed threshold on residual Mahalanobis, which has a chi-squared-like scale?
Its scale depends on the scene's covariance, the number of kept directions and the local ring; the tail is heavy and scene-specific. A percentile adapts; a fixed number does not.

Top 0.5 percent means 0.5 percent of the kept pixels.
Only if a classical detector's NaNs remove the unkept pixels from the composite. With foundation models alone, the unkept pixels are finite zeros and the percentile is taken over the whole frame.

A confident family name means the pixel is that material.
No. The confidence is a percentile against the scene's ordinary pixels, and false alarms are unusual by definition. It is a lead for a person.

ACE ignores the fill, so the fill does not matter for naming.
ACE's formula ignores it; its accuracy does not, because noise stays the same size while the signal shrinks with the fill (worked example 32.6, Table 32.1).

The two-level namer and the companion volume's two-level covariance are the same idea.
No. The two-level covariance is a model of noise (a few loud directions, many quiet ones). The two-level namer is two questions asked in turn.

The export can be georeferenced because the score map is.
The score map is not: every action raster has an identity transform. The CRS is recovered from the raw file at export, or the export fails with 422.

The error from V2 means the model is broken.
It means the band filter is on the wrong grid. The message names the setting to change.

One scoring action cannot run both Indradhanus.
It could not before 2026-10-31. On a native band filter it now can: v0.2.0's cube is resampled inside scoring.

`mask_cloud.tif` is the keep mask.
No. The keep mask comes from `scene_segmentation`'s thresholds and decides which pixels the statistics are fitted on. `mask_cloud.tif` comes from V2's own head and is applied after scoring.

`spectral_library_match` needs the native band filter now.
It never used the band filter's output; it reads the native onboarding cube.

32.8 Check yourself

Exercise 32.1. A scene has 2,400,000 finite composite pixels with all values different. How many does $q = 99.75$ flag? Use Equation (32.3) and show every step.

Exercise 32.2. Two models with weights 1 and 3. At a pixel the first reads 0.4 on a range $[0.1, 1.6]$, the second 12 on $[0, 60]$. Compute the composite value.

Exercise 32.3. With $\alpha = 0.02$ and 15 man-made families, compute p^*, and the bound on the chance that an ordinary pixel gets some man-made name.

Exercise 32.4. A GeoTIFF has $a = 30$, $b = 0$, $c = 500{,}000$, $d = 0$, $e = -30$, $f = 1{,}400{,}000$ in a UTM zone (metres). Where is the centre of the pixel in row 10, column 20? What would the identity transform give for the same pixel?

Exercise 32.5. A PRISMA Project has a band filter created with `use_common_wavelength_grid:true` and an `anomaly_scoring` action with `model_codenames:["IndradhanuV2","MNF-RX"]`. What happens, and which single setting fixes it? What happens with today's default band filter?

Exercise 32.6. A scene is 800×900 pixels, 20,000 invalid, and V2 calls 35,800 valid pixels cloud. Compute `excluded_percent`. Is a cloud-shadowed vessel's score zero?

Exercise 32.7. Level 1 gives $P(M) = 0.7$ and level 2 gives bare metal 0.5, painted metal 0.3, tarp 0.2. Compute every class's probability and P(natural), and check that they add to 1.

Exercise 32.8. In worked example 32.6, make the pixel 100% roof with no noise. Compute the whitened $\mathbf{d}$ and ACE against soil, and say why the answer could have been written down without arithmetic. Then, from Table 32.1 at fill 0.25, compute $P(\text{L1 right}) \times P(\text{L2 right} \mid \text{L1 right})$ from the unrounded 0.7724 and 0.7151.

Next: Chapter 33, Detection measures: ROC, pAUC, TPR at a fixed FPR, objects caught and chance.

Part VII

Evaluation

Chapter 33

Detection measures: ROC, pAUC, TPR at a fixed FPR, objects caught and chance

The one thing this chapter teaches

When anomalies are 0.1–1% of a scene's pixels, the area under the ROC curve is decided almost entirely by how ordinary background pixels are ranked against each other far down the list, where no analyst ever looks; so the repository also measures the *top* of the ranking - partial AUC up to 1% false alarms, the detection rate at 0.1% false alarms, and how many objects land in a fixed review budget - and reads every one of them against what random scores would get.

33.1 Why a new set of measures was needed

Every chapter so far has ended with a *score map*: one number per pixel, high where the pixel looks anomalous (Chapter 31). To compare two detectors you need a way to turn a score map and a *ground-truth mask* - a map marking which pixels a human labelled as anomalous - into a number. Until 2026-10-16 the repository used one number for this, the AUC returned by `compute_roc` in `app/utils/anomaly_detection/scoring.py`. On that date a second family of measures arrived beside it. Since 2026-10-17 they live in `app/evaluation/metrics.py`, keyed `pauc_0.01`, `tpr_at_0.001`, `hit_1` and `hit_0.1`, and are tested by hand-worked numbers in `tests/test_evaluation/test_metrics.py`.

The problem the new measures solve is easy to state and surprisingly easy to miss. A PRISMA scene has about a million valid pixels. Suppose a thousand of them are targets [A] (0.1%, the order of the labelled bench scenes). An analyst can review perhaps the top 1% of the ranking: ten thousand pixels. Anything the detector does to the other 990,000 pixels is invisible to that analyst. A good measure must therefore be sensitive to the top of the list and indifferent to the bottom. The AUC is the opposite, as Section 33.6 proves with numbers.

Analogy

A search engine returns a million results for your query. You read the first page. A ranking metric that gives the same weight to the order of results 500,000 and 500,001 as to results 1 and 2 will happily prefer an engine whose first page is junk, as long as it sorts the junk at the back slightly better. The analyst is the reader of the first page; the measures of this chapter are page-one measures.

You need two ideas from earlier: that anomalies are rare (Chapter 1) and the idea of a threshold on a score (Chapter 31). Nothing else.

33.2 Thresholds and the confusion matrix

Definition 33.1 (Positive, negative, flagged)**.** A *positive* (or *target*) is a valid pixel the ground truth marks as anomalous. A *negative* (or *background* pixel) is every other valid pixel. Given a *threshold* τ, a pixel is *flagged* when its score $s \geq \tau$.

Every pixel then falls into exactly one of four boxes, and the table of their counts is called the *confusion matrix*:

	truth: target	truth: background
flagged ($s \geq \tau$)	true positive (TP)	false positive (FP)
not flagged ($s < \tau$)	false negative (FN)	true negative (TN)

Write P for the number of targets and N for the number of background pixels. Each column adds up to its total: $\mathrm{TP} + \mathrm{FN} = P$ and $\mathrm{FP} + \mathrm{TN} = N$. Two rates summarise the table.

Definition 33.2 (TPR and FPR)**.** The *true-positive rate* (TPR, also the detection rate) and the *false-positive rate* (FPR, also the false-alarm rate) at threshold τ are

$$\mathrm{TPR}(\tau) = \frac{\mathrm{TP}(\tau)}{P}, \qquad \mathrm{FPR}(\tau) = \frac{\mathrm{FP}(\tau)}{N}. \tag{33.1}$$

Both rates are divided by their own column total, which is the point: FPR does not care how many targets there are, and TPR does not care how much background there is. Both are *non-increasing* functions of τ: raise the threshold and you can only flag fewer pixels, never more.

33.3 Six pixels by hand: the ROC curve

Definition 33.3 (ROC curve)**.** The *receiver operating characteristic* (ROC) curve is the set of points $(\mathrm{FPR}(\tau), \mathrm{TPR}(\tau))$ as τ slides from above the highest score (nothing flagged, the point $(0, 0)$) down to the lowest score (everything flagged, the point $(1, 1)$), joined by straight segments.

You only need to try thresholds equal to the scores themselves: between two neighbouring scores nothing changes.

Worked example 33.1: The ROC of six pixels

Do this one on paper. Six valid pixels, three targets (T) and three background pixels (B), so $P = 3$, $N = 3$:

pixel	1	2	3	4	5	6
score	0.95	0.85	0.70	0.60	0.40	0.20
truth	T	B	T	T	B	B

Step 1 - sort by score, highest first. They are already sorted.
Step 2 - sweep the threshold down one pixel at a time, counting what has been flagged:

threshold τ	TP	FP	TPR = TP/3	FPR = FP/3
above 0.95	0	0	0	0
0.95 (T)	1	0	1/3	0
0.85 (B)	1	1	1/3	1/3
0.70 (T)	2	1	2/3	1/3
0.60 (T)	3	1	1	1/3
0.40 (B)	3	2	1	2/3
0.20 (B)	3	3	1	1

Step 3 - list the ROC points (FPR, TPR):

$$(0,0),\ (0,\tfrac{1}{3}),\ (\tfrac{1}{3},\tfrac{1}{3}),\ (\tfrac{1}{3},\tfrac{2}{3}),\ (\tfrac{1}{3},1),\ (\tfrac{2}{3},1),\ (1,1).$$

A target moves the curve *up* by $1/P = 1/3$; a background pixel moves it *right* by $1/N = 1/3$. The curve is a staircase. Figure 33.1 draws it.

The rule in the last sentence is general and worth stating on its own.

Walking down the sorted list, every target is a step up of $1/P$ and every background pixel a step right of $1/N$; the ROC curve is the path that walk traces.

33.4 The area under the curve, by the trapezoid rule

Definition 33.4 (AUC)**.** The *area under the ROC curve* (AUC, or ROC-AUC) is the area between the curve and the FPR axis, from FPR 0 to FPR 1. A perfect detector, which ranks every target above every background pixel, goes straight up to $(0,1)$ and then right: AUC 1. A detector that ranks at random follows the diagonal TPR = FPR: AUC 1/2.

Because the curve is made of straight segments, its area is a sum of trapezoids. A *trapezoid* between points (x_0, y_0) and (x_1, y_1) has area width times average height:

$$\text{area} = (x_1 - x_0)\,\frac{y_0 + y_1}{2}. \tag{33.2}$$

This is exactly what `auc_exact` in `app/evaluation/metrics.py` computes, one line:

```
def auc_exact(fpr, tpr):     # fpr, tpr: (K,) ROC points from exact_roc, starting at (0, 0)
    return float(np.sum((fpr[1:] - fpr[:-1]) * (tpr[1:] + tpr[:-1]) / 2))
```

Worked example 33.2: The AUC of the six pixels, strip by strip

Apply Equation (33.2) to each consecutive pair of points of worked example 33.1:

segment	width	heights	area
$(0,0) \to (0,\frac{1}{3})$	0	$0, \frac{1}{3}$	0
$(0,\frac{1}{3}) \to (\frac{1}{3},\frac{1}{3})$	$\frac{1}{3}$	$\frac{1}{3}, \frac{1}{3}$	$\frac{1}{3} \times \frac{1}{3} = \frac{1}{9}$
$(\frac{1}{3},\frac{1}{3}) \to (\frac{1}{3},\frac{2}{3})$	0		0
$(\frac{1}{3},\frac{2}{3}) \to (\frac{1}{3},1)$	0		0
$(\frac{1}{3},1) \to (\frac{2}{3},1)$	$\frac{1}{3}$	$1, 1$	$\frac{1}{3}$
$(\frac{2}{3},1) \to (1,1)$	$\frac{1}{3}$	$1, 1$	$\frac{1}{3}$

Add the column:

$$\begin{aligned}\text{AUC} &= \tfrac{1}{9} + \tfrac{1}{3} + \tfrac{1}{3} \\ &= \tfrac{1}{9} + \tfrac{3}{9} + \tfrac{3}{9} \\ &= \tfrac{7}{9} = 0.7778.\end{aligned}$$

The repository's `auc_exact` on these six pixels returns 0.7778 [C] (the book's check script `research/textbook/tools/compute_measures.py`).

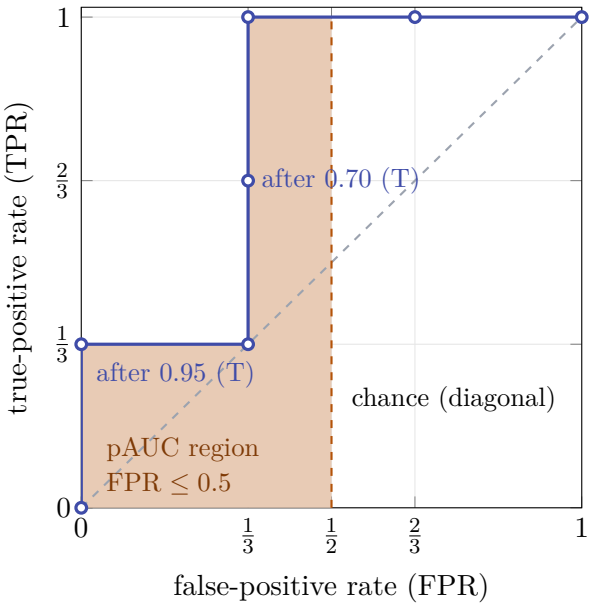


Figure 33.1: The ROC of the six pixels of worked example 33.1 (indigo staircase), the chance diagonal (grey, dashed), and the region counted by a partial AUC with limit FPR ≤ 0.5 (amber). Drawn to scale from the worked numbers. The full area under the staircase is 7/9; the amber area is 5/18, which normalised by the limit gives 5/9 (worked example 33.6).

33.5 AUC is a probability: the Mann–Whitney count

There is a second way to read the same number, and it explains everything that follows.

Definition 33.5 (Pair reading of AUC)**.** Pick one target at random and one background pixel at random. The *pair reading* of AUC is the probability that the target scores higher, counting a tie as half:

$$\text{AUC} = \frac{1}{PN} \sum_{\text{targets } t} \sum_{\text{background } b} \Big(\mathbf{1}[s_t > s_b] + \tfrac{1}{2}\,\mathbf{1}[s_t = s_b] \Big). \tag{33.3}$$

Here $\mathbf{1}[\cdot]$ is the *indicator*: 1 when the statement in brackets is true, 0 when false. The double sum counts winning pairs; it is the *Mann–Whitney U count*, and dividing by the PN pairs turns it into a probability [9].

Worked example 33.3: The six pixels, counted as pairs

There are $P \times N = 3 \times 3 = 9$ (target, background) pairs. For each target, count the background pixels it beats:

$$\begin{aligned}
&\text{target } 0.95 \text{ vs } 0.85, 0.40, 0.20: \text{ beats all three} && \to 3\\
&\text{target } 0.70 \text{ vs } 0.85, 0.40, 0.20: \text{ loses to } 0.85 && \to 2\\
&\text{target } 0.60 \text{ vs } 0.85, 0.40, 0.20: \text{ loses to } 0.85 && \to 2\\
&\quad \text{wins} = 3 + 2 + 2 = 7, && \text{AUC} = 7/9 = 0.7778.
\end{aligned}$$

The same 7/9 as the trapezoids of worked example 33.2.

That the two readings agree is not a coincidence. Here is the proof, in words first and then in symbols.

Proof that the area equals the pair count (no ties). By the key point of Section 33.3, the curve moves right only at background pixels, by $1/N$ each, and moves up only at targets. So the area under the staircase is a sum of rectangles, one per background pixel b: width $1/N$, and height equal to the TPR at the moment b is reached in the walk. That TPR is the share of targets already passed, which is the share of targets scoring above b:

$$\begin{aligned}
\text{area} &= \sum_b \frac{1}{N} \cdot \text{TPR when } b \text{ is reached} && \text{one rectangle per background pixel}\\
&= \sum_b \frac{1}{N} \cdot \frac{\#\{t : s_t > s_b\}}{P} && \text{TPR = share of targets above } b\\
&= \frac{1}{PN} \sum_b \sum_t \mathbf{1}[s_t > s_b] && \text{write the count as a sum of indicators}
\end{aligned}$$

which is Equation (33.3) with no ties. Check it on the six pixels: the background pixels are reached at heights $\frac{1}{3}$ (pixel 0.85), 1 (pixel 0.40) and 1 (pixel 0.20), so the area is $\frac{1}{3}(\frac{1}{3} + 1 + 1) = \frac{1}{3} \cdot \frac{7}{3} = \frac{7}{9}$.

With ties. When a target and a background pixel share a score, the threshold flags both at once, and the curve moves diagonally: right by $1/N$ and up by $1/P$ in one segment. A trapezoid under that diagonal step gets the average of the heights before and after - exactly half the target's contribution, the $\frac{1}{2}$ in Equation (33.3).

Worked example 33.4: A tie, both ways

Four pixels: 0.9 (T), 0.5 (T), 0.5 (B), 0.1 (B), so $P = N = 2$. The function `exact_roc` puts one ROC point at the end of each run of tied scores (Section 33.8), so the two 0.5s are flagged together.

points: $(0,0)$, $(0,\frac{1}{2})$, $(\frac{1}{2},1)$, $(1,1)$ (the middle segment is diagonal)

trapezoids: $0 + \frac{1}{2} \cdot \frac{\frac{1}{2}+1}{2} + \frac{1}{2} \cdot \frac{1+1}{2}$

$= 0 + \frac{1}{2} \cdot \frac{3}{4} + \frac{1}{2} = \frac{3}{8} + \frac{4}{8} = \frac{7}{8} = 0.875$

pairs: 0.9 beats $0.5, 0.1 \to 2$; 0.5 ties $0.5 \to \frac{1}{2}$

and beats $0.1 \to 1$; total $2 + 1.5 = 3.5$

AUC $= 3.5/4 = 0.875$.

33.6 Why AUC is decided by the bulk

Read Equation (33.3) again with the real scene in mind: $P = 1{,}000$ targets, $N = 999{,}000$ background pixels [A]. AUC averages over every one of the 999,000,000 pairs. A target ranked 500,000th still beats the 499,000 or so background pixels below it, and every one of those wins counts. **Almost all of the AUC is earned by putting targets above pixels nobody will ever look at.** The analyst cares about the region FPR $\in [0, 0.01]$, which is 1% of the curve's width.

Two invented detectors (from the course, part 16) make it concrete. Their ROC curves, as straight segments:

$$\text{A} : (0,0) \to (0.01, 0.40) \to (0.30, 0.80) \to (1,1)$$
$$\text{B} : (0,0) \to (0.01, 0.02) \to (0.10, 0.80) \to (1,1)$$

Worked example 33.5: Two detectors whose AUC and top-of-list disagree

Trapezoids for A:

$$0.01 \times \tfrac{0+0.40}{2} = 0.01 \times 0.20 = 0.002$$
$$0.29 \times \tfrac{0.40+0.80}{2} = 0.29 \times 0.60 = 0.174$$
$$0.70 \times \tfrac{0.80+1.00}{2} = 0.70 \times 0.90 = 0.630$$
$$\text{AUC}_A = 0.002 + 0.174 + 0.630 = 0.806.$$

Trapezoids for B:

$$0.01 \times \tfrac{0+0.02}{2} = 0.01 \times 0.01 = 0.0001$$
$$0.09 \times \tfrac{0.02+0.80}{2} = 0.09 \times 0.41 = 0.0369$$
$$0.90 \times \tfrac{0.80+1.00}{2} = 0.90 \times 0.90 = 0.810$$
$$\text{AUC}_B = 0.0001 + 0.0369 + 0.810 = 0.847.$$

B has the higher AUC. But at 1% false alarms A has found 40% of the targets and B 2%.

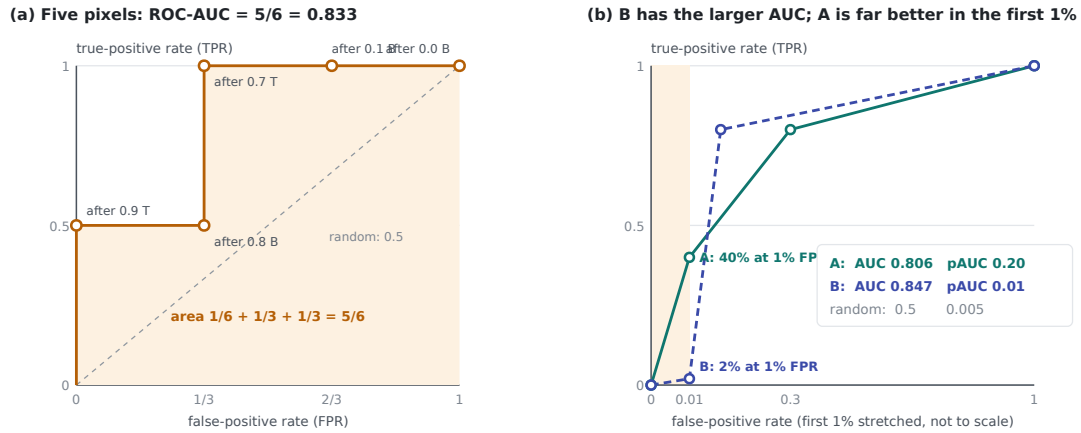


Figure 33.2: The course's figure for this chapter. (a) The five-pixel ROC of the course's own example (targets at 0.9 and 0.7 among five pixels), AUC 5/6. (b) Detectors A and B: B has the larger total area; inside the first 1% of false alarms (the amber strip, stretched, not to scale) A is twenty times higher.

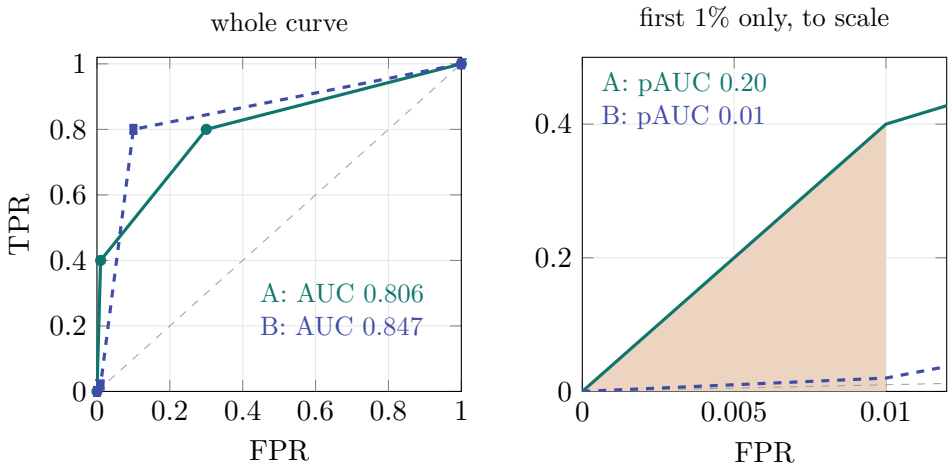


Figure 33.3: Detectors A (teal) and B (indigo, dashed) of worked example 33.5, drawn to scale. Left: the whole curve, where B's larger area comes from FPR 0.01–0.30. Right: the first 1.2% of false alarms, where A's area (amber, up to FPR 0.01) is twenty times B's. The chance diagonal (grey) is nearly flat on the right-hand scale.

A ranking measure averaged over the whole list can prefer the detector that is useless where it is used.

33.7 Partial AUC up to a false-alarm limit

Definition 33.6 (Normalised partial AUC)**.** The *partial AUC* up to an FPR limit m is the area under the ROC between FPR 0 and FPR m. The repository divides it by m so that a perfect detector scores 1:

$$\mathrm{pAUC}_m = \frac{1}{m}\int_0^m \mathrm{TPR}(f)\,df. \tag{33.4}$$

The repository's headline is $m = 0.01$, key `pauc_0.01`, configured in an evaluation config as `{"kind": "pauc", "max_fpr": 0.01}`.

Source check: what pAUC is normalised by

`app/evaluation/metrics.py`, module docstring and function `pauc` (lines 31–36): "area under the EXACT ROC over FPR 0..max_fpr, divided by max_fpr (1 perfect, max_fpr/2 random)". The default `max_fpr` is 0.01. The normalisation is a plain division by the width m. It is *not* the McClish standardisation used by some libraries (scikit-learn's `roc_auc_score(max_fpr=...)` applies McClish); derived below so you can tell the two apart when you compare with outside numbers.

Its chance value, derived. A random detector's ROC is the diagonal $\mathrm{TPR}(f) = f$. Put that into Equation (33.4):

$$\begin{aligned}
\mathrm{pAUC}_m^{\text{chance}} &= \frac{1}{m}\int_0^m f\,df && \text{TPR equals FPR on the diagonal}\\
&= \frac{1}{m}\cdot\left[\tfrac{1}{2}f^2\right]_0^m && \text{the integral of } f \text{ is } \tfrac{1}{2}f^2\\
&= \frac{1}{m}\cdot\tfrac{1}{2}m^2 && \text{evaluate at } m\text{, minus at } 0\\
&= \frac{m}{2}. && \text{cancel one } m
\end{aligned}$$

For $m = 0.01$: chance is $0.01/2 = 0.005$. Geometrically it is a triangle of base 0.01 and height 0.01, area $0.01 \times 0.01/2 = 0.00005$, divided by 0.01. **Random pAUC is 0.005, not 0.5.** The test `test_perfect_random_and_sklearn` checks that random scores stay below 0.05; on two million random pixels the book's script measured 0.0052, 0.0055 and 0.0057 in three draws [C].

The McClish standardisation, for comparison. McClish [30] rescales the raw partial area A (not divided by m) so that chance maps to $\frac{1}{2}$ and perfect to 1, like a full AUC. The smallest sensible raw area is the chance triangle $A_{\min} = m^2/2$; the largest is the full rectangle $A_{\max} = m$:

$$\mathrm{pAUC}^{\mathrm{McClish}} = \frac{1}{2}\left(1 + \frac{A - A_{\min}}{A_{\max} - A_{\min}}\right). \tag{33.5}$$

Check the ends: $A = A_{\min}$ gives $\frac{1}{2}(1+0) = \frac{1}{2}$, and $A = A_{\max}$ gives $\frac{1}{2}(1+1) = 1$. For a repository pAUC of 0.170 at $m = 0.01$ (FT04 with cloud excluded, Chapter 36):

$$\begin{aligned}
A &= 0.170 \times 0.01 = 0.0017 && \text{undo the division by } m\\
A_{\min} &= 0.01^2/2 = 0.00005, \quad A_{\max} = 0.01\\
\frac{A - A_{\min}}{A_{\max} - A_{\min}} &= \frac{0.0017 - 0.00005}{0.01 - 0.00005} = \frac{0.00165}{0.00995} = 0.1658\\
\mathrm{pAUC}^{\mathrm{McClish}} &= \tfrac{1}{2}(1 + 0.1658) = 0.5829.
\end{aligned}$$

The same detector reads 0.170 on the repository's scale and 0.583 on McClish's [C]. Never compare the two without converting.

Worked example 33.6: Partial AUC of the six pixels, with a limit that cuts a segment

Take the six pixels and a limit $m = 0.5$ (chosen so the cut falls inside the curve; the product uses 0.01). The segments left of FPR 0.5 are:

$$\begin{aligned}
(0, \tfrac{1}{3}) \to (\tfrac{1}{3}, \tfrac{1}{3}): &\quad \tfrac{1}{3} \times \tfrac{1}{3} = \tfrac{1}{9}\\
\text{vertical segments at } \tfrac{1}{3}: &\quad 0\\
(\tfrac{1}{3}, 1) \to (\tfrac{2}{3}, 1),\ \text{cut at } 0.5: &\quad (0.5 - \tfrac{1}{3}) \times 1 = \tfrac{1}{6}\\
\text{area} &= \tfrac{1}{9} + \tfrac{1}{6} = \tfrac{2}{18} + \tfrac{3}{18} = \tfrac{5}{18}\\
\mathrm{pAUC}_{0.5} &= \tfrac{5}{18}/0.5 = \tfrac{10}{18} = \tfrac{5}{9} = 0.556.
\end{aligned}$$

Chance at this limit is $0.5/2 = 0.25$. And at the product's limit $m = 0.01$: the curve sits at TPR $\frac{1}{3}$ from FPR 0 all the way to $\frac{1}{3}$, so the area up to 0.01 is $0.01 \times \frac{1}{3}$, and dividing by 0.01 gives $\mathrm{pAUC}_{0.01} = \frac{1}{3} = 0.333$. The code returns 0.5556 and 0.3333 [C].

Cutting a sloped segment. When the segment that crosses m is diagonal (a tie, or the straight lines of detectors A and B), `pauc` cuts it by linear interpolation. The segment from (x_0, y_0) to (x_1, y_1) with $x_0 < m < x_1$ has, at m, the height

$$y(m) = y_0 + (y_1 - y_0)\,\frac{m - x_0}{x_1 - x_0}, \tag{33.6}$$

and its piece left of m is the trapezoid of width $m - x_0$ and heights y_0, $y(m)$. For the tie of worked example 33.4 with $m = 0.25$: the diagonal runs $(0, \frac{1}{2}) \to (\frac{1}{2}, 1)$, so $y(0.25) = 0.5 + 0.5 \times \frac{0.25}{0.5} = 0.75$; the area is $0.25 \times \frac{0.5+0.75}{2} = 0.15625$; and $\mathrm{pAUC}_{0.25} = 0.15625/0.25 = 0.625$. For detector A at $m = 0.01$ the cut falls exactly at a point, so $\mathrm{pAUC}_A = 0.002/0.01 = 0.20$ and $\mathrm{pAUC}_B = 0.0001/0.01 = 0.01$: the order flips to the one the analyst would choose.

The code, annotated:

```
def pauc(fpr, tpr, max_fpr=0.01):            # fpr, tpr: (K,) exact ROC points
    x0, x1, y0, y1 = fpr[:-1], fpr[1:], tpr[:-1], tpr[1:]          # (K-1,) segments
    y1c = np.where(x1 > max_fpr,                                    # segment crosses the
    limit:
                   y0 + (y1 - y0) * (max_fpr - x0) / np.where(x1 > x0, x1 - x0, 1.0),  # eq.
    interp
                   y1)
    return float(np.sum(((np.minimum(x1, max_fpr) - x0) * (y0 + y1c) / 2)[x0 < max_fpr]) /
    max_fpr)
```

Segments that start at or beyond the limit (`x0>=max_fpr`) are dropped; vertical runs at one FPR (several targets above every background pixel) have zero width but do raise the height that later trapezoids use, so they "count fully", as the docstring says.

33.8 Exact ROC versus the product's sampled ROC

The product's `compute_roc` (`app/utils/anomaly_detection/scoring.py`, line 184) does not use every score as a threshold. It takes evenly spaced percentiles:

```
n_pts = min(max_thresholds, max(20, s.size // 10000 + 64))      # max_thresholds =
    256
qs = np.linspace(0.0, 100.0, n_pts)
```

For a scene of 1,000,000 valid pixels:

$$n_{\text{pts}} = \min(256,\ \max(20,\ 1{,}000{,}000 \,//\, 10{,}000 + 64)) = \min(256, \max(20, 100 + 64)) = 164$$
$$\text{spacing} = 100/(164 - 1) = 100/163 = 0.613 \text{ percentile.}$$

(Here // is integer division.) So between FPR 0 and 1% the sampled curve has about one point; its area there is a single triangle guessed from it. For the whole AUC the coarseness matters little; for the first 1% it is most of the answer.

Source check: the sampled ROC

Two descriptions disagree with `compute_roc`, and the code wins. "Samples 256 thresholds" is a cap: the count reaches 256 only from $(256 - 64) \times 10{,}000 = 1{,}920{,}000$ pixels. The function's own docstring says the thresholds are "evenly spaced over the score range plus the unique-value breakpoints sklearn would emit"; the code takes percentiles only (`app/utils/anomaly_detection/scoring.py`, lines 184–185).

`exact_roc` avoids sampling. It sorts every score, highest first, and makes one ROC point at the end of each run of tied scores:

```
def exact_roc(s, g):                          # s: (n,) scores, g: (n,) bool truth
    order = np.argsort(-s, kind="stable")
    s_sorted, g_sorted = s[order], g[order]
    last = np.r_[np.flatnonzero(np.diff(s_sorted)), len(s_sorted) - 1]   # end of each tie run
    fpr = np.r_[0.0, np.cumsum(~g_sorted)[last] / max((~g).sum(), 1)]
    tpr = np.r_[0.0, np.cumsum(g_sorted)[last] / max(g.sum(), 1)]
    return fpr, tpr
```

That is the walk of worked example 33.1, for a million pixels: the cumulative sums are the TP and FP columns. The evaluator reports the product's sampled AUC as `roc_auc` (for continuity with every older table) and uses the exact curve for everything else; the test checks `auc_exact` against scikit-learn's `roc_auc_score` to 10^{-9}.

33.9 Detection rate at 0.1% false alarms

Definition 33.7 (TPR at a fixed FPR)**.** The *TPR at FPR* f_0 is the best detection rate reached at any exact ROC point whose FPR is at most f_0:

$$\mathrm{TPR@}f_0 = \max\{\mathrm{TPR}_k : \mathrm{FPR}_k \le f_0\}. \tag{33.7}$$

In code, `float(tpr[fpr<=max_fpr].max())`; the repository's key is `tpr_at_0.001`, $f_0 = 0.001$.

There is no interpolation: only actual ROC points count. On a million-pixel scene 0.1% of the background is about 1,000 false alarms, roughly what an analyst can inspect. Its chance level is f_0 itself, 0.001, because on the diagonal TPR equals FPR. For the six pixels, the ROC points with FPR at most 0.001 are $(0, 0)$ and $(0, \frac{1}{3})$, so $\mathrm{TPR@}0.001 = \frac{1}{3}$.

33.10 Objects caught at a fixed budget

Pixel rates have a weakness. A 20-pixel roof and a 1-pixel vehicle are two objects, but in TPR the roof counts twenty times. An analyst needs *one* flagged pixel per object to go and look.

Definition 33.8 (Object)**.** An *object* is a *connected component* of the ground-truth mask: a maximal set of target pixels in which you can walk from any pixel to any other by steps up, down, left or right through target pixels. This is *4-connectivity*: two target pixels that touch only at a corner are two objects. `PrismaBench` labels objects with `ndimage.label(gt)`, whose default structure is the 4-neighbour cross (`app/evaluation/datasets.py`, line 160); implant benches store their object ids directly.

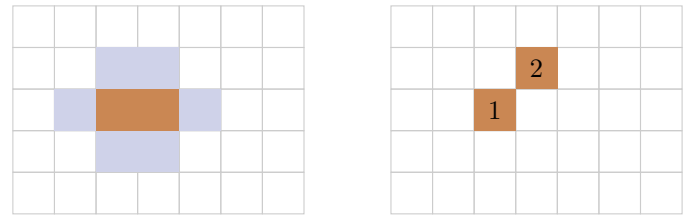
Definition 33.9 (Objects caught at the top $q\%$)**.** Flag the valid pixels scoring at or above the $(100-q)$th percentile of all valid scores. Grow the flagged set by one 4-neighbour step (a *binary dilation*). An object is *caught* when any of its pixels lies in the grown set. The measure is the share of objects caught.

The code is three lines (`app/evaluation/metrics.py`, `object_hits`):

```
k = valid & np.isfinite(score)                                  # (H, W) usable pixels
# flagged pixels, grown by one 4-neighbour step (the 1-pixel tolerance)
hits = ndimage.binary_dilation((score >= np.percentile(score[k], 100 - top_pct)) & k)
return set(np.unique(objects[hits & (objects > 0)]).tolist())    # ids of objects touched
```

Source check: the rule is "within one pixel", not "a pixel above the threshold"

You will read that an object is caught when "at least one of its pixels is above the threshold". The code is more generous: a flagged pixel *next to* the object (one 4-neighbour step) also catches it. This is the *1-pixel tolerance*; the module docstring of `app/evaluation/metrics.py` says "objects with a pixel within 1 px of the top top_pct% of scores". The reason is that the ground-truth masks may sit about one pixel off the scene grid. Note also that the percentile is taken over *all* valid pixels, targets included, so "top 1%" is not exactly "FPR 1%". Pixel measures (AUC, pAUC, TPR) have no tolerance.



domino: $m = 2 + 6 = 8$ corner touch: two objects

Figure 33.4: Left: a two-pixel object (amber) and the six pixels its 1-pixel tolerance adds (indigo); a flagged pixel anywhere in these $m = 8$ cells catches it. Right: two target pixels touching only at a corner are two objects under 4-connectivity (the book's script confirms `ndimage.label` returns 2).

33.11 The chance level of objects caught

Here is the trap the tolerance creates. Give every pixel an independent random score. At the top-q% budget, each pixel is flagged with probability $p = q/100$. Write m for the object's size *after* the 1-pixel dilation: its own pixels plus its 4-neighbours. The object is caught if any of those m cells is flagged.

Derivation. The chance of catching it is one minus the chance of missing all m cells:

$$\begin{aligned} P(\text{one cell not flagged}) &= 1 - p \\ P(\text{none of } m \text{ flagged}) &= (1-p)^m && \text{independent cells: multiply} \\ P(\text{caught}) &= 1 - (1-p)^m. && \text{the complement} \end{aligned}$$

This is `chance_hit(m,top_pct)` in `app/evaluation/metrics.py`, and the evaluator stores it beside every object's hit (`chance_1`, `chance_0.1`). It is a slight approximation: a percentile threshold flags exactly q% of pixels, so the cells are not perfectly independent; with a million pixels the difference is negligible.

Worked example 33.7: Chance for four object shapes at the top 1%

With $p = 0.01$:

1 pixel:	$m = 1 + 4 = 5,$	$0.99^5 = 0.9510,$	$1 - 0.9510 = 0.0490$
domino:	$m = 2 + 6 = 8,$	$0.99^8 = 0.9227,$	$1 - 0.9227 = 0.0773$
3 in a row:	$m = 3 + 8 = 11,$	$0.99^{11} = 0.8953,$	$1 - 0.8953 = 0.1047$
3×3 square:	$m = 9 + 12 = 21,$	$0.99^{21} = 0.8097,$	$1 - 0.8097 = 0.1903$

For the square, the 12 neighbours are 3 on each of its 4 sides. At the top 0.1% ($p = 0.001$) the one-pixel object has $1 - 0.999^5 = 1 - 0.99501 = 0.0050$. The script's values match:

0.0490, 0.0773, 0.1047, 0.1903 and 0.0050 [C].

Bigger objects are caught by chance more often, because they have more cells. So the chance level of a whole bench depends on its size mix. The evaluator averages `chance_hit` over the objects and prints it as the `*chance*` row of every table: **10.2%** at the top 1% on the PRISMA bench and **10.0%** on bench-v1 [M: `docs/training/anomaly-detection/19-running-an-evaluation.md`], about 1% at the top 0.1%.

Read every "objects caught" number next to its chance row: about 10% at the top 1% comes free.

33.12 The 3D-ROC terms, and why they are degenerate here

The hyperspectral literature also reports a "3D ROC", which plots TPR and FPR each against the threshold. Scores are first *min-max normalised*, $\tilde{s} = (s - s_{\min})/(s_{\max} - s_{\min})$, so the threshold τ runs from 0 to 1. Two areas follow: $\mathrm{AUC}(D, \tau)$ under detection rate against τ, and $\mathrm{AUC}(F, \tau)$ under false-alarm rate against τ (lower is better). From them, $\mathrm{AUC}_{\mathrm{BS}} = \mathrm{AUC} - \mathrm{AUC}(F, \tau)$ (background suppressibility) and $\mathrm{SNPR} = \mathrm{AUC}(D, \tau)/\mathrm{AUC}(F, \tau)$.

The identity the code uses. For scores in $[0, 1]$, the area under "share of pixels with $\tilde{s} \geq \tau$" as τ runs from 0 to 1 is the *mean* normalised score. Proof: a pixel with normalised score u is at or above every threshold from 0 to u, so it contributes $\int_0^1 \mathbf{1}[u \geq \tau]\, d\tau = u$ to the area; averaging over pixels gives the mean. So `three_d_roc` returns the target mean and the background mean of $\tilde{s}$. On the course's five pixels (0.9 T, 0.8 B, 0.7 T, 0.1 B, 0.0 B) the normalised scores are $1, 0.889, 0.778, 0.111, 0$; so $\mathrm{AUC}(D, \tau) = (1 + 0.778)/2 = 0.889$ and $\mathrm{AUC}(F, \tau) = (0.889 + 0.111 + 0)/3 = 0.333$, as the test asserts.

Why it does not help for residual Mahalanobis. That score is a squared distance with a heavy tail: a few pixels sit hundreds of times above the typical background. Min-max normalisation divides by the single highest score. With a typical background of 30 and one pixel at 30,000 [A], the normalised background is about $30/30{,}000 = 0.001$ for every model, so $\mathrm{AUC}_{\mathrm{BS}}$ is ROC-AUC minus 0.001 and SNPR is set by the one brightest pixel. The FT02b README measured exactly that on the PRISMA bench ("AUC_BS = ROC-AUC - 0.001"). The terms are computed for comparison with the literature; do not choose a model with them.

33.13 The measures side by side

measure	key	perfect	chance	question it answers
ROC-AUC	`roc_auc` (sampled), exact per group	1	0.5	are targets above background on average over everything?
pAUC to 1% FPR	`pauc_0.01`	1	0.005	how good is the top of the list?
TPR at 0.1% FPR	`tpr_at_0.001`	1	0.001	what share of target pixels is in about the top 1,000?
caught, top 1%	`hit_1`	100%	about 10%	how many objects in a 1% review budget?
caught, top 0.1%	`hit_0.1`	100%	about 1%	the same, in a 0.1% budget
3D-ROC terms	`auc_d_tau`, …	-	-	literature comparison only

Chapter 34 explains why the four labelled PRISMA scenes could not separate models even with these measures, and builds a bigger benchmark; Chapter 35 runs them all from one config; Chapter 36 asks how large a difference in them is real.

33.14 Common confusions

A pAUC of 0.15 is terrible - it is far below 0.5?
The chance level of the repository's pAUC is $m/2 = 0.005$, not 0.5. A pAUC of 0.15 is thirty times chance. Only McClish's standardisation (not used here) puts chance at 0.5.

`roc_auc` and the exact AUC should be the same number?
`roc_auc` is the product's sampled `compute_roc`, kept so results match what users see; the exact AUC uses every score. They differ slightly; quote which one you used.

Top 1% means exactly 1% of pixels are flagged?
It means "at or above the 99th percentile of valid scores". With many tied scores more can pass. A score map with a block of zeros (masked pixels left in) can put the 99th percentile at 0, so every pixel is "flagged" and every object "caught" whatever the scores.

Objects caught at 10% means the model found one object in ten?
It does, and so would random scores. Report it next to chance.

The 1-pixel tolerance applies to AUC too?
No. Only objects caught has it. AUC, pAUC and TPR at a fixed FPR are pixel measures with no tolerance.

A diagonal pair of target pixels is one object?
No. Objects are 4-connected; a corner touch makes two objects, each with its own chance level.

33.15 Check yourself

Exercise 33.1. Four pixels: scores 0.6 (T), 0.5 (B), 0.4 (B), 0.3 (T). List the ROC points, compute the AUC by trapezoids and by counting pairs, and compute $\text{pAUC}_{0.01}$ and TPR@0.001. What do the measures disagree about?

Exercise 33.2. Derive the chance level of the normalised pAUC at a limit $m = 0.05$, and the McClish value of a raw partial area $A = 0.02$ at that limit.

Exercise 33.3. With random scores at the top-1% budget, what is the chance of catching an L-shaped three-pixel object (cells (0,0), (1,0), (1,1))? Count its 1-pixel tolerance cells by hand.

Exercise 33.4. A scene has 600,000 valid pixels. How many thresholds does `compute_roc` use, and how many percentiles apart are they? Roughly how many sampled points fall between FPR 0 and 1%?

Exercise 33.5. A bench has equal numbers of one-pixel and two-pixel (domino) objects. A model catches 12% of them at the top 1%. How many times chance is that?

Exercise 33.6. Using the staircase proof of Section 33.5, compute the AUC of Exercise 33.1 a third way: one rectangle per background pixel.

Next: Chapter 34, Benchmarks: the public sets and the implant bench (contrast z, foreignness, PSF).

Chapter 34

Benchmarks: the public sets and the implant bench (contrast z, foreignness, PSF)

The one thing this chapter teaches

No public labelled anomaly set tests a model that reads satellite reflectance by wavelength, so the repository puts synthetic objects into its own held-out satellite scenes - mixed in linearly, blurred by a Gaussian point-spread function, and solved so that their contrast z and foreignness match real labelled objects *as they are realised* - and trusts the result only once it reproduces an ordering the real labels already showed.

34.1 Why a bigger benchmark was needed

Chapter 33 gave you measures that look at the top of the ranking. On the four labelled PRISMA scenes (909 objects [M: `research/experiments/implant-bench/README.md`]) they still could not separate the models: every measure tied the shipped model with the fine-tuned ones within scene-to-scene noise (`research/experiments/indradhanu-finetune/FT02b-mixed/README.md`). Four scenes and 909 objects are too few to see differences of the size being chased; Chapter 36 puts a number on "too few".

The obvious fix is to borrow a bigger public benchmark. On 2026-10-16 the two standard ones were downloaded and vetted, and the record is `research/experiments/benchmarks/README.md`. This chapter first shows why neither fits, then teaches the benchmark the repository built instead, and the mathematics of each of its parts: linear mixing, the ring and its contrast z, foreignness, the Gaussian point-spread function and the strength solve.

Analogy

To test proofreaders, a publisher takes real pages and plants typos of known difficulty, some glaring, some a single swapped letter. Real pages keep the surrounding text honest; planted errors make every answer known. The test is only fair if the planted typos look like the ones writers actually make - so you study a pile of real typos first, and you check that the planted test ranks last year's proofreaders in the order their real work did.

34.2 What a model needs from each scene

The Indradhanu models read *any band list by wavelength* and expect *reflectance* on a 0–1 scale (Chapter 2). So every benchmark scene must supply four things:

1. the cube;
2. the centre wavelength of every band;
3. a ground-truth mask;
4. the value scale: what a stored number means physically.

Miss any one and the model is either guessing or reading something it was never trained on. The evaluator enforces the fourth: `Evaluation.run` raises `ValueError` for any scene whose `units` are not `"reflectance"` (`app/evaluation/runner.py`).

34.3 HAD100

HAD100 was published with the AETNet paper [24] and is cut from the public AVIRIS-NG archive (the airborne sensor of Chapter 3). The numbers below are from the benchmarks README, section 2 [M: `research/experiments/benchmarks/README.md`].

item	value
what ships	94 test tiles, their ground truth, and 522 anomaly-free training tiles
protocol	`main.py` crops 100 test scenes of 64 × 64 from the 94 tiles (6 tiles give two crops)
labels	3,567 anomaly pixels in 460 objects (8-connected); per scene median 25.5 pixels (0.6%)
values	L1B at-sensor **radiance**, μW nm^{-1} cm^{-2} sr^{-1}, not reflectance
wavelengths	425 per tile, 376.7–2500.4 nm; they differ slightly between flight lines

Radiance is the light arriving at the sensor, atmosphere included. *Reflectance* is the fraction of sunlight the ground sends back, with the sun and atmosphere taken out. The crudest bridge is *top-of-atmosphere reflectance*:

$$\rho_{\mathrm{TOA}} = \frac{\pi L}{E_0 \cos\theta_s}, \tag{34.1}$$

with L the radiance, E_0 the sun's irradiance at that wavelength and θ_s the solar zenith angle. The π converts radiance (per steradian) to the flux leaving a surface that scatters equally in all directions, and $E_0 \cos\theta_s$ is the sunlight landing on a horizontal square metre when the sun is θ_s from overhead.

Worked example 34.1: Top-of-atmosphere reflectance

Take round numbers [A]: $L = 10$, $E_0 = 186$ (roughly the sun at 550 nm in the same units), $\theta_s = 30$ř.

$$\pi \times 10 = 31.42 \qquad \text{numerator}$$
$$\cos 30\text{ř} = 0.866$$
$$186 \times 0.866 = 161.1 \qquad \text{denominator}$$
$$\rho_{\mathrm{TOA}} = 31.42/161.1 = 0.195.$$

That needs the solar zenith of every flight line, which the tiles do not carry, and it leaves the atmosphere in. The preferred route - fetching AVIRIS-NG L2 reflectance for HAD100's ten flight lines - needs an Earthdata login and had not been done when this was written.

The leakage check. HAD100 comes from the same archive as the 79 AVIRIS-NG flight lines the models trained on. If a benchmark scene were cut from a training line, a model could score well by memory. The README's check [M: `research/experiments/benchmarks/README.md`]:

- **No exact line overlap.** None of HAD100's 10 lines is among the 79.
- **One near neighbour.** Line `ang20180227t082814`, near Kakinada in Andhra Pradesh, was flown on the same day as three training lines; one of them, 18 minutes later, ran on the parallel track immediately east. The outlines are about 630 m apart and do not overlap, and the four HAD100 tiles from that line sit 1,020–1,865 m from the nearest training footprint.
- Those 4 tiles become **6 of the 100 protocol scenes**: no pixel-level leakage, but near-domain proximity. **Report HAD100 results with and without those 6 scenes.**

A second trap: HAD100's own training and test tiles come from the same 10 flight lines, so training on its anomaly-free tiles would leak too.

34.4 ABU

The Airport–Beach–Urban set [18] has 13 small scenes and is used by essentially every hyperspectral anomaly paper. It has three problems for this product (README, section 4):

- twelve scenes are AVIRIS scaled at-sensor radiance with an unknown gain, not reflectance;
- one scene, `beach-4`, is ROSIS-03 over Pavia, 430–860 nm only, min-max normalised to 0–1 over the whole cube - not a physical quantity;
- **no wavelengths**: the files hold `data` and `map` and nothing else, and which AVIRIS bands were removed was never published.

Why "radiance", when nobody wrote it down? The median spectra peak in the blue at about 1,800 counts and fall to about 20–100 near 2.4 μm [M: `research/experiments/benchmarks/README.md`]:

$$20/1{,}800 = 0.011, \text{ about } 1\%$$
$$100/1{,}800 = 0.056, \text{ about } 6\%.$$

Airports and cities do not reflect only 1–6% as much at 2.4 μm as in the blue; sunlight itself is far weaker there. A spectrum with the sun's shape still contains the sunlight: radiance. A model that reads bands by wavelength cannot run on ABU without guessing them.

Protocols disagree too. "The first 50 bands" means the first 50 of HAD100's 276 "good" bands (452–697 nm) in one paper and the first 50 of the raw 425 (377–622 nm) in another. Both are visible-only; a model using the short-wave infrared is handicapped against either. Quote the protocol and do not mix them in a table.

And satellites? The implant-bench design (`docs/lld/implant-bench.md`) records the search: "no labelled PRISMA / EnMAP / Gaofen-5 anomaly set was found". The four PRISMA scenes are the only labelled satellite anomaly scenes the repository has.

What each benchmark supplies (amber = why it cannot test this product as it is)

	HAD100	ABU	PRISMA bench	implanted bench-v1
platform	airborne (AVIRIS-NG)	airborne (AVIRIS, ROSIS)	satellite (PRISMA)	2 satellites + AVIRIS-NG
values	**L1B radiance**	**radiance; 1 min-max**	reflectance	reflectance
wavelengths	per tile, vary by line	**none shipped**	yes	yes
labelled objects	460 in 100 crops	196 in 13 scenes	**909 in 4 scenes**	3,336 in 18 scenes
our sensors?	**no**	**no**	PRISMA	EnMAP, PRISMA, AVIRIS-NG

The PRISMA bench is the right kind of data, but 4 scenes could not separate the models (part 16).
Sources: research/experiments/benchmarks/README.md, research/experiments/implant-bench/README.md.

Figure 34.1: What each benchmark supplies: the four things a model needs, plus platform. Amber cells are the reason a benchmark cannot test this product as it is.

So the options were: convert HAD100 to reflectance (a login, more work, still airborne), guess ABU's wavelengths (a result resting on a guess), or put known targets into the repository's own held-out satellite scenes. The rest of this chapter is the third.

34.5 The implant idea, and what real objects look like

An *implant bench* takes real held-out scenes (*hosts*) and adds synthetic objects whose positions are known exactly. The pieces (`app/evaluation/implant.py` for the core, `scripts/build_implant_bench.py` to build, `tests/test_evaluation/test_implant.py` for the hand-worked tests). On 2026-10-17 the user asked for implanted targets "somewhat similar to the ones we do have", in "subtle, medium, easy" categories. "Similar" needs measurement first, so `build_implant_bench.pyreal` measures every labelled PRISMA object (4-connected; an object whose ring has fewer than 8 pixels is skipped) [M: `research/experiments/implant-bench/README.md`]:

	10th / 50th / 90th percentile
size, pixels	1 / 2 / 10
single-pixel objects	42%
background NDVI	−0.02 / 0.37 / 0.60

Half of all real targets are one or two pixels. NDVI (normalised difference vegetation index) measures greenness; the script's `ndvi_map` uses the mean of 450–650 nm as "visible" and of 800–900 nm as "near-infrared": NDVI $= (\text{nir} - \text{vis})/(\text{nir} + \text{vis})$.

34.6 The ring and the contrast z

"Subtle" and "easy" need a number. The repository's is the contrast z, defined in the module docstring of `app/evaluation/implant.py` (lines 5–11).

Definition 34.1 (Ring)**.** The *ring* of an object is the set of valid pixels 2 or 3 four-neighbour steps away from it, not inside another object:

```
# (H, W) bool: within 3 steps, minus within 1 step, valid only
ring = binary_dilation(obj, iterations=3) & ~binary_dilation(obj, iterations=1) & valid
```

Each dilation step goes only up, down, left or right, so "k steps" is *Manhattan distance* $|\Delta y| + |\Delta x| = k$, and the ring is a diamond, not a square.

Counting the ring of a one-pixel object. The pixels at Manhattan distance exactly $k \geq 1$ from a point number $4k$: on each of the diamond's four sides there are k pixels, counting from one corner up to (but not including) the next. So

$$\begin{aligned} \text{distance 1: } & 4 \times 1 = 4 && \text{removed, too close} \\ \text{distance 2: } & 4 \times 2 = 8 \\ \text{distance 3: } & 4 \times 3 = 12 \\ \text{ring: } & 8 + 12 = 20. \end{aligned}$$

A square ring at the same distances would be a 7×7 block minus a 3×3 one, $49 - 9 = 40$ pixels. The test `test_contrast_z_by_hand` pins 20, and the book's script agrees [C].

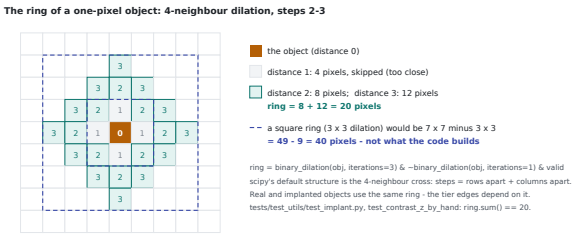


Figure 34.2: The diamond ring around a one-pixel object. Distance 1 (grey) is skipped; distances 2 (8 pixels) and 3 (12 pixels) form the 20-pixel ring (teal). The dashed square shows the 40-pixel ring a 3×3 dilation would have made.

Definition 34.2 (Contrast z)**.** Let o_b be the object's mean reflectance in band b, and r_b, σ_b the mean and the population standard deviation of the ring in band b. Over the B bands valid in both,

$$z = \frac{\frac{1}{B}\sum_b |o_b - r_b|}{\frac{1}{B}\sum_b \sigma_b}. \tag{34.2}$$

It is the standard-score idea of Chapter 7, averaged over bands: how far the object sits from its neighbourhood, in units of the neighbourhood's own variation. The code adds 10^{-6} to the denominator so a perfectly flat ring does not divide by zero.

Worked example 34.2: The test's z, by hand

Two bands, one-pixel object. Band 0 is 0.1 on the 8 distance-2 pixels and 0.3 on the 12 distance-3 pixels; band 1 is 0.5 everywhere. The object reads $(0.42, 0.5)$.

$$\begin{aligned} r_0 &= \frac{8 \times 0.1 + 12 \times 0.3}{20} = \frac{0.8 + 3.6}{20} = \frac{4.4}{20} = 0.22 \quad \text{(ring mean)} \\ \text{deviations} &: 0.1 - 0.22 = -0.12 \ (8\times), \quad 0.3 - 0.22 = +0.08 \ (12\times) \\ \sigma_0^2 &= \frac{8 \times 0.0144 + 12 \times 0.0064}{20} = \frac{0.1152 + 0.0768}{20} \quad \text{(divide by 20, not 19)} \\ &= \frac{0.192}{20} = 0.0096 \\ \sigma_0 &= \sqrt{0.0096} = 0.0980 \\ r_1 &= 0.5, \quad \sigma_1 = 0 \\ |o_0 - r_0| &= |0.42 - 0.22| = 0.20, \quad |o_1 - r_1| = 0 \\ z &= \frac{(0.20 + 0)/2}{(0.0980 + 0)/2} = \frac{0.10}{0.0490} = 2.04. \end{aligned}$$

The repository's `contrast_z` returns 2.0412 [C].

Tiers. On the 909 real objects z has median 0.68. Its terciles cut the tiers (`app/evaluation/implant.py`, line 25):

```
TIER_EDGES = (0.502, 0.934)      # z terciles of the 909 real PRISMA bench objects
    (2026-10-17)
def tier_of(z): return TIERS[int(z >= 0.502) + int(z >= 0.934)]   # subtle / medium / easy
```

By construction a third of real objects falls in each *tier*. "Subtle" means "as hard as the hardest third of what analysts labelled", not an absolute difficulty. The object of worked example 34.2, at $z = 2.04 \geq 0.934$, is easy.

Real and implanted objects must be measured with the same ring and the same bands, or their z values are not comparable and the tiers mean nothing.

34.7 Linear mixing: what an implant does to a pixel

Derivation. A sensor pixel records the average light from its ground footprint. Suppose a fraction a of that footprint is covered by a material of reflectance spectrum $\mathbf{t}$, and the rest by the original background of spectrum $\mathbf{b}$. The light from each part is proportional to its area times its reflectance, and the pixel reports the area-weighted average:

$$\begin{aligned} \mathbf{x} &= \frac{a \cdot \mathbf{t} + (1-a) \cdot \mathbf{b}}{a + (1-a)} && \text{area-weighted average} \\ &= (1-a)\,\mathbf{b} + a\,\mathbf{t}. && \text{the areas sum to 1} \end{aligned}$$

This is the *linear mixing model* (the sub-pixel mix of Chapter 32), and a is the *abundance* or fill fraction. bench-v1's material implant is exactly this, `implant_material`: `x'=(1-f)*x+f*t`.

The mixing line. Rearrange to see the geometry:

$$\begin{aligned} \mathbf{x} &= \mathbf{b} - a\,\mathbf{b} + a\,\mathbf{t} && \text{expand} \\ &= \mathbf{b} + a\,(\mathbf{t} - \mathbf{b}). && \text{collect the } a \text{ terms} \end{aligned}$$

As a runs from 0 to 1, $\mathbf{x}$ moves along the straight segment from $\mathbf{b}$ to $\mathbf{t}$: the *mixing line*. The pixel's difference from the background is $\mathbf{x} - \mathbf{b} = a(\mathbf{t} - \mathbf{b})$, linear in a, so a contrast measured against an unchanged background grows in proportion to a. That is why one number, the strength, is enough to hit a target z.

Worked example 34.3: A pixel on the mixing line

Two bands. Background $\mathbf{b} = (0.30, 0.20)$, material $\mathbf{t} = (0.10, 0.60)$ [A]. Then $\mathbf{t} - \mathbf{b} = (-0.20, +0.40)$ and:

$$\begin{aligned} a = 0.25: \quad \mathbf{x} &= (0.30, 0.20) + 0.25 \times (-0.20, 0.40) \\ &= (0.30 - 0.05,\ 0.20 + 0.10) = (0.25, 0.30) \\ a = 0.50: \quad \mathbf{x} &= (0.30, 0.20) + 0.50 \times (-0.20, 0.40) \\ &= (0.30 - 0.10,\ 0.20 + 0.20) = (0.20, 0.40) \end{aligned}$$

Check with the first form at $a = 0.25$: band 0 is $0.75 \times 0.30 + 0.25 \times 0.10 = 0.225 + 0.025 = 0.25$. If the ring around this pixel has $\sigma = (0.05, 0.05)$ and mean $\mathbf{b}$, then at $a = 0.25$: $z = \frac{(0.05+0.10)/2}{0.05} = \frac{0.075}{0.05} = 1.5$, and at $a = 0.5$, $z = 3.0$: doubling a doubles z.

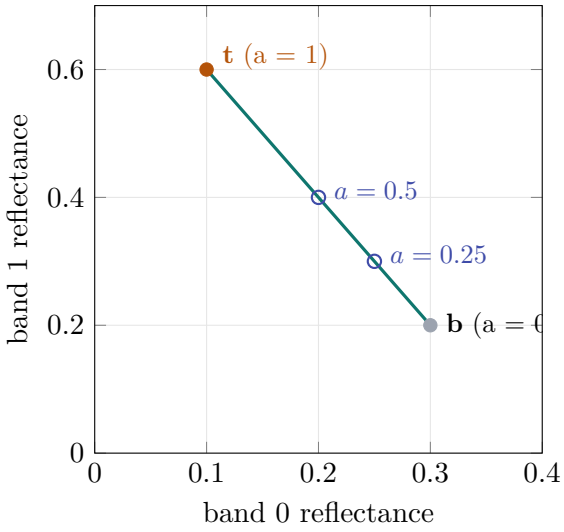


Figure 34.3: The mixing line of worked example 34.3 in the plane of two bands: every mixture of background **b** and material **t** lies on the straight segment between them, at a distance from **b** proportional to the abundance a.

The signature implant. bench-v1's second source is additive: `implant_signature` sets $\mathbf{x}' = \max(\mathbf{x} + a\,\mathbf{d}, 0)$, where $\mathbf{d} = \mathbf{o} - \mathbf{r}$ is a real object's difference from its ring, carried to the host's wavelengths by interpolation. The $\max(\cdot, 0)$ stops reflectance going negative. Both implants move the object's mean spectrum along a straight line: if $\mathbf{o}_0$ is the object's mean before implanting,

$$\mathbf{o}(s) = \mathbf{o}_0 + s\,\boldsymbol{\Delta}, \qquad \boldsymbol{\Delta} = \begin{cases} \mathbf{d} & \text{signature, } s = a \\ \mathbf{t} - \mathbf{o}_0 & \text{material, } s = f \le 1. \end{cases} \tag{34.3}$$

For the material, $(1-f)\mathbf{o}_0 + f\mathbf{t} = \mathbf{o}_0 + f(\mathbf{t} - \mathbf{o}_0)$, the mixing line again.

Source check: signatures and descending wavelengths

Until commit b94dbfd every signature implant was garbled: PRISMA lists its bands in descending wavelength, and `np.interp` needs ascending sample points. `interp_spectrum` now sorts first (`app/evaluation/implant.py`, lines 160–166). Chapter 37 tells the whole story; every signature-implant number before 2026-10-18 is void.

34.8 Solving for the strength on the realised contrast

Each object gets a target z, drawn from the real objects of its tier. Put Equation (34.3) into Equation (34.2), with the ring unchanged:

$$z(s) = \frac{\frac{1}{B}\sum_b |o_{0,b} + s\,\Delta_b - r_b|}{\bar{\sigma}}, \qquad \bar{\sigma} = \frac{1}{B}\sum_b \sigma_b. \tag{34.4}$$

A first version of the design gave the simpler $s = z\,\bar{\sigma}/\overline{|\Delta|}$. That assumes $\mathbf{o}_0 = \mathbf{r}$: that the spot looked exactly like its ring before anything was added. Real pixels never do.

Source check: `strength_for_z` never existed

The implant-bench LLD's first interface list named `strength_for_z` with the simple formula. That function was never in the source; the code has `solve_strength`, which solves Equation (34.4) (`app/evaluation/implant.py`, lines 80–104). The LLD now lists

`solve_strength`.

Worked example 34.4: One band: the simple formula misses, the realised solve hits

Ring mean $r = 0.20$, $\sigma = 0.05$; the spot reads $o_0 = 0.22$, already 0.02 above its ring; the implant darkens, $\Delta = -0.10$; target $z = 1.0$.

$$\begin{aligned} \text{simple: } & s = 1.0 \times 0.05/0.10 = 0.5 \\ \text{realised at } s = 0.5: \; & z = |0.22 + 0.5 \times (-0.10) - 0.20|/0.05 \\ & = |-0.03|/0.05 = 0.6 \quad \text{(medium, not easy)} \end{aligned}$$

The natural +0.02 cancelled part of the darkening. Solve Equation (34.4) instead:

$$\begin{aligned} z(s) &= |0.02 - 0.10\,s|/0.05 && o_0 - r = 0.02 \\ z(0) &= 0.02/0.05 = 0.4 && \text{the spot's own contrast} \\ z(s) &= 0 \text{ at } s = 0.2 && \text{implant cancels the offset} \\ 0.02 - 0.10\,s &= -0.05 && \text{rising branch, } |\cdot| = 0.05 \\ s &= 0.07/0.10 = 0.7 \end{aligned}$$

check: $z(0.7) = |0.22 - 0.07 - 0.20|/0.05 = 0.05/0.05 = 1.0$.

The repository's `solve_strength` returns $s = 0.7$, $z = 1.0$; with $\Delta = +0.10$ it returns $s = 0.3$ [C].

Why the first crossing after the minimum. Each term $|o_{0,b} + s\Delta_b - r_b|$ is the absolute value of a straight line in s: a "V" shape, which is *convex* (the segment between any two points of its graph lies on or above the graph). A sum of convex functions is convex, and dividing by the positive $\bar{\sigma}$ keeps it convex. A convex function of one variable falls (or stays flat) up to its minimum and rises from then on. So $z(s)$ can dip below $z(0)$ and then climb for good; the useful answer is the first s at or after the minimum where $z(s)$ reaches the target. The code evaluates z on a grid of 400 strengths, finds the minimum, takes the first grid point at or above the target to its right, and refines by linear interpolation between that point and the one before.

Two more rules follow from the shape:

1. If $z(0) > z_{\text{target}}$, the spot is already too contrasty; `solve_strength` returns `None` and the builder tries another spot (up to 200 times).
2. A material may not reach the target even at full fill $f = 1$; then f is capped and the lower realised z is recorded (2% of bench-v1's material implants [M: `research/experiments/implant-bench/README.md`]).

After implanting, z is measured again, and **the realised z and its tier are what the benchmark records**. One consequence: a subtle target of $z = 0.2$ can only go on a spot whose own $z(0) \leq 0.2$, so subtle implants sit on the most uniform spots of a scene.

34.9 bench-v1, and how it failed

bench-v1 was built on 2026-10-17 with seed 0: six held-out test scenes per sensor as hosts, objects placed on valid, cloud- and water-free land at the real object density, with centres weighted so their background NDVI mix matches the real objects' [M: `research/experiments/implant-bench/README.md`].

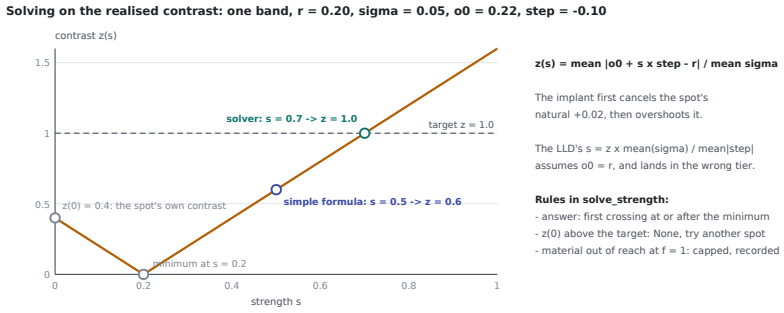


Figure 34.4: $z(s)$ for worked example 34.4. The simple formula's $s = 0.5$ lands at $z = 0.6$ (medium); the solver follows the curve through its minimum at $s = 0.2$ and crosses the target at $s = 0.7$.

sensor	hosts	objects	subtle / medium / easy
EnMAP (224 bands)	6	1,179	385 / 412 / 382
PRISMA (234 bands)	6	1,318	429 / 463 / 426
AVIRIS-NG (372 bands)	6	839	273 / 295 / 271
total	18	3,336	1,087 / 1,170 / 1,079

Size and contrast matched the real objects closely (size percentiles 1 / 2 / 9 against 1 / 2 / 10; per-tier z percentiles within a few hundredths); greenness did not (ring NDVI median 0.49 against 0.37, no host had the bright bare ground of real scene set4). 97% of objects landed in their targeted tier.

Then the stage-3 evaluation (Chapter 35) ran a model-free detector beside the models. PCA-RX, which is local RX on the cube's own top 30 principal components, caught 30.2% of bench-v1 objects at the top 1% against the best model's 26.9%, and lost by a wide margin on the real objects (pAUC 0.128 against 0.158)
[M: `docs/lld/evaluation.md`]. A benchmark on which a 1990s detector beats every model, while real labels say the opposite, is measuring something else. A diagnosis on the PRISMA hosts found two differences between implants and real objects: foreignness and halo.

34.10 Foreignness

Start from an object's difference from its ring, $\mathbf{d} = \mathbf{o} - \mathbf{r}$ (a vector with one entry per band).

Definition 34.3 (Background subspace and foreign fraction)**.** The *background subspace* of a scene is spanned by its top $K = 10$ principal directions (`FOREIGN_K`), the orthonormal rows of a $K \times B$ matrix $\mathbf{P}$ computed by `background_subspace` (Chapter 6 for principal directions). The part of $\mathbf{d}$ inside the subspace is its projection $\mathbf{d}_{\text{in}} = \mathbf{P}^\top\mathbf{P}\mathbf{d}$; the rest, $\mathbf{d}_{\text{out}} = \mathbf{d} - \mathbf{d}_{\text{in}}$, is outside. The *foreign fraction* is the share of $\|\mathbf{d}\|^2$ outside:

$$\phi(\mathbf{d}) = \frac{\|\mathbf{d}_{\text{out}}\|^2}{\|\mathbf{d}\|^2} = 1 - \frac{\|\mathbf{P}\mathbf{d}\|^2}{\|\mathbf{d}\|^2}. \tag{34.5}$$

The second form is what `foreign_fraction` computes. It holds because the inside and outside parts are perpendicular, so by Pythagoras $\|\mathbf{d}\|^2 = \|\mathbf{d}_{\text{in}}\|^2 + \|\mathbf{d}_{\text{out}}\|^2$, and because $\mathbf{P}$ has orthonormal rows, $\|\mathbf{d}_{\text{in}}\| = \left\|\mathbf{P}^\top\mathbf{P}\mathbf{d}\right\| = \|\mathbf{P}\mathbf{d}\|$.

Worked example 34.5: Foreignness of $(3, 4, 0)$

Three bands; the scene's single main direction is band 0, so $\mathbf{P} = (1, 0, 0)$. Take $\mathbf{d} = (3, 4, 0)$.

$$\begin{aligned} \|\mathbf{d}\|^2 &= 3^2 + 4^2 + 0^2 = 9 + 16 + 0 = 25 \\ \mathbf{Pd} &= 1 \times 3 + 0 \times 4 + 0 \times 0 = 3, \quad \|\mathbf{Pd}\|^2 = 9 && \text{inside: } (3, 0, 0) \\ \mathbf{d}_{\text{out}} &= (3, 4, 0) - (3, 0, 0) = (0, 4, 0), \quad \|\mathbf{d}_{\text{out}}\|^2 = 16 \\ \phi &= 16/25 = 0.64 = 1 - 9/25. \end{aligned}$$

The diagnosis medians on PRISMA [M: `docs/training/anomaly-detection/19-running-an-evaluation.md`]: real objects **0.006**, bench-v1 implants **0.042**, seven times as foreign. Real objects are unusual *mixtures of the scene's own materials*; bench-v1's implants pointed in directions the scene never takes. Whitening (Chapter 7) rewards exactly that: a difference of 0.01 along a direction where the scene varies with spread 0.05 is 0.2 spreads (squared, 0.04); along a direction of spread 0.002 it is 5 spreads (squared, 25), $25/0.04 = 625$ times the score.

Hitting a target foreignness. `match_foreignness` keeps $\mathbf{d}_{\text{in}}$ and rescales $\mathbf{d}_{\text{out}}$ by a factor β, so that $\mathbf{d}' = \mathbf{d}_{\text{in}} + \beta\,\mathbf{d}_{\text{out}}$ has foreign fraction t. Derive β. Write $A = \|\mathbf{d}_{\text{in}}\|^2$, $C = \|\mathbf{d}_{\text{out}}\|^2$:

$$\begin{aligned} \phi(\mathbf{d}') = \frac{\beta^2 C}{A + \beta^2 C} &= t && \text{parts stay perpendicular} \\ \beta^2 C &= tA + t\beta^2 C && \text{multiply both sides by } A + \beta^2 C \\ \beta^2 C(1 - t) &= tA && \text{collect the } \beta^2 \text{ terms} \\ \beta^2 &= \frac{t}{1 - t} \cdot \frac{A}{C}. && \text{divide by } C(1 - t) \end{aligned}$$

This is the formula in the docstring of `match_foreignness` (lines 137–148).

Worked example 34.6: Matching $(4, 0, 2)$ to foreignness 0.1

Same $\mathbf{P} = (1, 0, 0)$; $\mathbf{d} = (4, 0, 2)$, so $\mathbf{d}_{\text{in}} = (4, 0, 0)$, $A = 16$, and $\mathbf{d}_{\text{out}} = (0, 0, 2)$, $C = 4$.

$$\begin{aligned} t = 0.2: \quad \beta^2 &= \tfrac{0.2}{0.8} \times \tfrac{16}{4} = 0.25 \times 4 = 1, \quad \beta = 1 && \text{already at target} \\ t = 0.1: \quad \beta^2 &= \tfrac{0.1}{0.9} \times 4 = 0.4444, \quad \beta = 0.6667 \\ \mathbf{d}' &= (4, 0, 0) + 0.6667 \times (0, 0, 2) = (4, 0, 1.3333) \\ \text{check: } \phi &= \frac{1.3333^2}{16 + 1.3333^2} = \frac{1.7778}{17.7778} = 0.1. \end{aligned}$$

The repository's function returns $(4, 0, 1.3333)$ [C].

34.11 The halo and the Gaussian point-spread function

Definition 34.4 (Halo and halo ratio)**.** The *halo* of an object is the ring of valid pixels touching it (Manhattan distance 1). Its *halo ratio* is the halo's contrast z (against the same distance-2–3 ring) over the object's.

A ratio of 0 is a hard edge; near 1 the object bleeds fully into its neighbours. Real sensors blur - each pixel collects some light from next door - so real objects have halos: the diagnosis measured a median halo ratio of 0.74 for real objects against 0.60 for bench-v1 implants (diagnosis ruler; Section 34.12 explains the re-measured 0.643).

Definition 34.5 (Gaussian point-spread function)**.** A *point-spread function* (PSF) is the shape a single bright point takes in the image. A *Gaussian* PSF of standard deviation σ pixels spreads a point at the origin so that the pixel at offset $(\Delta y, \Delta x)$ receives a share proportional to

$$g(\Delta y, \Delta x) = \exp\Big(-\frac{\Delta y^2 + \Delta x^2}{2\sigma^2}\Big). \tag{34.6}$$

bench-v2 adds its signal through a blurred footprint. `psf_weights` (lines 151–157) blurs the object mask with `ndimage.gaussian_filter` and divides by the blurred mask's mean over the object, so the object keeps weight 1 on average and its neighbours get the tail:

```
w = ndimage.gaussian_filter(obj.astype(np.float64), sigma, mode="constant")   # (H, W)
return w / max(w[obj].mean(), 1e-12)                                          # mean 1 on obj
```

The build uses $\sigma = 1.4$ (`--psf-sigma 1.4`, in `research/experiments/implant-bench/README.md` and the LLD's build lines), chosen by calibration builds rather than from a formula. For a one-pixel object the blurred mask is the Gaussian kernel itself, so after dividing by the centre value the weight at $(\Delta y, \Delta x)$ is exactly Equation (34.6). With $\sigma = 1.4$, $2\sigma^2 = 2 \times 1.96 = 3.92$.

Worked example 34.7: The PSF weights and the abundance map on a 3×3 footprint

Do this one on paper with a calculator. One-pixel object at the centre of a 3×3 window.

$$\begin{aligned} \text{centre } (0,0): &\quad \exp(0) = 1 \\ \text{edge } (\pm1,0),(0,\pm1): &\quad \exp(-1/3.92) = \exp(-0.2551) = 0.7748 \\ \text{corner } (\pm1,\pm1): &\quad \exp(-2/3.92) = \exp(-0.5102) = 0.6004 \end{aligned}$$

Now the abundance. A bench-v2 material implant uses $\mathbf{d}_0 = \mathbf{t} - \mathbf{r}$ (library spectrum minus ring mean, `scripts/build_implant_bench.py`, line 238) and adds $s\,w(p)\,\mathbf{d}$ to each pixel p. Where the pixel looks like its ring, $\mathbf{x} \approx \mathbf{r}$:

$$\begin{aligned} \mathbf{x}' &= \mathbf{x} + s\,w(p)\,(\mathbf{t} - \mathbf{r}) && \text{the v2 implant} \\ &\approx \mathbf{x} + s\,w(p)\,(\mathbf{t} - \mathbf{x}) && \text{use } \mathbf{x} \approx \mathbf{r} \\ &= (1 - a(p))\,\mathbf{x} + a(p)\,\mathbf{t}, \quad a(p) = s\,w(p). && \text{the mixing model} \end{aligned}$$

So the PSF turns one strength into an *abundance map*. At strength $s = 0.1$ [A]:

$$a = 0.1 \times \begin{pmatrix} 0.6004 & 0.7748 & 0.6004 \\ 0.7748 & 1 & 0.7748 \\ 0.6004 & 0.7748 & 0.6004 \end{pmatrix} = \begin{pmatrix} 0.0600 & 0.0775 & 0.0600 \\ 0.0775 & 0.1000 & 0.0775 \\ 0.0600 & 0.0775 & 0.0600 \end{pmatrix}.$$

The centre pixel is 10% material; each edge neighbour 7.75%; each corner 6%. The script's `psf_weights` gives the same matrix [C].

The ring is lifted too: the shrink factor. The PSF tail reaches the ring. For a one-pixel object, average Equation (34.6) over the 20 ring cells (distance 2: 4 cells at $(\pm2,0),(0,\pm2)$ and

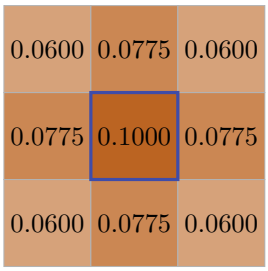


Figure 34.5: The abundance map of worked example 34.7: a one-pixel bench-v2 implant at strength 0.1 with a Gaussian PSF of $\sigma = 1.4$, shown on its 3×3 footprint (darker amber = more material). The blur continues beyond the window, weaker with distance.

4 at $(\pm1, \pm1)$; distance 3: 4 at $(\pm3, 0), (0, \pm3)$ and 8 at $(\pm2, \pm1), (\pm1, \pm2)$):

$$
\begin{aligned}
4 \times e^{-4/3.92} &= 4 \times 0.3604 = 1.4418 \\
4 \times e^{-2/3.92} &= 4 \times 0.6004 = 2.4015 \\
4 \times e^{-9/3.92} &= 4 \times 0.1007 = 0.4027 \\
8 \times e^{-5/3.92} &= 8 \times 0.2793 = 2.2343 \\
\bar{w}_{\text{ring}} &= (1.4418 + 2.4015 + 0.4027 + 2.2343)/20 = 6.4803/20 = 0.3240.
\end{aligned}
$$

The script's `psf_weights` gives 0.3240 [C]. The object mean moves by $s \times 1 \times \mathbf{d}$; the ring mean by $s \times 0.3240 \times \mathbf{d}$; so the object's difference from its ring moves by only

$$s\,(1 - \bar{w}_{\text{ring}})\,\mathbf{d} = 0.676\,s\,\mathbf{d}. \tag{34.7}$$

The builder passes this `shrink` $= 1 - \bar{w}_{\text{ring}}$ into the strength solve.

The halo ratio a PSF produces, on flat ground. With a perfectly flat background the halo and the object are measured against the same ring, so the ring's $\bar{\sigma}$ cancels in the ratio of their z values:

$$
\begin{aligned}
\text{halo ratio} &= \frac{|w_{\text{halo}} - \bar{w}_{\text{ring}}|\; s\,\overline{|\mathbf{d}|}/\bar{\sigma}}{|1 - \bar{w}_{\text{ring}}|\; s\,\overline{|\mathbf{d}|}/\bar{\sigma}} && \text{both differences scale } \mathbf{d} \\
&= \frac{0.7748 - 0.3240}{1 - 0.3240} = \frac{0.4508}{0.6760} = 0.667. && s,\ \mathbf{d},\ \bar{\sigma} \text{ cancel}
\end{aligned}
$$

Close to the real 0.643 measured on the bench's own ruler. Real halos also carry the background's texture, which is why σ was calibrated on builds, not fixed by this formula.

34.12 bench-v2: solve on the object as it ends up

bench-v2 (2026-10-17) copies three measured properties from real objects: z, foreign fraction and halo ratio, with z and foreignness drawn *as a pair from one real object* of the chosen tier. They are related: subtle real objects are about ten times as foreign as easy ones (median 0.0238 against 0.0024 on `real_v2.json`'s ruler [M: `docs/training/anomaly-detection/20-bench-v2.md`]), so drawing them independently would create objects that do not exist. A third source joined signature and material: a *host* spectrum taken from a land pixel at least 100 pixels away in the same scene. The first calibration build missed, and three bugs were found.

1. **Measured on the wrong bands.** Foreignness and z were computed over every host band, including the noisy water-absorption and detector-edge bands the evaluator never reads. Now `evaluation_bands` keeps the product's bands only (exclusions removed, and only bands at least 60% finite). Measure a benchmark with the ruler that will grade on it.

2. **One gap deleted a band.** `background_subspace` centres pixels by subtracting each band's mean. A plain mean with one missing value is missing: $(0.20{+}\text{NaN}{+}0.40)/3 = \text{NaN}$, and the whole band dropped out. `np.nanmean` gives $(0.20+0.40)/2 = 0.30$; centred values -0.10, gap, $+0.10$; the gap is then set to 0, "the band mean". 130–182 of 234 PRISMA host bands had at least one gap (docstring, lines 115–118).

3. **Solved on the signal, not on the object.** The first version matched the foreignness of the added signal. But the evaluator sees the realised difference

$$\mathbf{o} - \mathbf{r} = (\mathbf{o}_0 - \mathbf{r}) + s \cdot \text{shrink} \cdot \mathbf{d}', \tag{34.8}$$

and the spot's natural offset $\mathbf{o}_0 - \mathbf{r}$ is already foreign (0.009–0.045 on random un-implanted spots).

Worked example 34.8: An unforeign signal on a foreign spot

Two bands; the scene's main direction is band 0. Natural offset $\mathbf{o}_0 - \mathbf{r} = (0.00, 0.01)$, wholly outside. Signal $s \cdot \text{shrink} \cdot \mathbf{d}' = (0.05, 0.00)$, wholly inside: its foreignness is 0.

$$\begin{aligned} \mathbf{o} - \mathbf{r} &= (0.00 + 0.05,\ 0.01 + 0.00) = (0.05, 0.01) \\ \|\cdot\|^2 &= 0.05^2 + 0.01^2 = 0.0025 + 0.0001 = 0.0026, \quad \text{outside } 0.0001 \\ \phi &= 0.0001/0.0026 = 0.038. \end{aligned}$$

The signal was perfectly unforeign; the object the evaluator sees is five times the real median (0.0071).

`solve_realised` (lines 172–187) fixes it. It tries a ladder of signal foreignness levels, `FOREIGN_LADDER` $= (0, 0.0005, 0.001, \ldots, 0.25, 0.5)$; at each it solves the strength for the target z with `solve_strength` on `shrink*d`, measures the foreignness of the realised difference Equation (34.8), and keeps the level whose realised foreignness is closest to the target on a log scale, $|\ln(\phi_{\text{realised}}/\phi_{\text{target}})|$. A spot whose own offset is so foreign that no level comes within a factor `tol` of the target (1.5 in bench-v2, 1.2 in bench-v2.1) is rejected and another spot tried.

Match a benchmark property on the object the evaluator will see, never on the ingredient you added.

34.13 Accepting a benchmark: the ordering test

bench-v2 had two acceptance tests, written before the build.

Acceptance 1 - does it look like the real objects? Foreignness and halo medians within 25% of the real objects', overall and in every tier. The calibration build on 2 PRISMA hosts passed: foreignness 0.0071 real against 0.0081, halo 0.643 against 0.642, z 0.69 against 0.74, every tier within 0.90–1.15× [M: `research/experiments/implant-bench/README.md`]. The final bench, measured on all 6 PRISMA hosts, came out at 1.10 / 1.28 / 1.27× real foreignness (subtle / medium / easy) - marginally over the limit in two tiers - and halo 0.93–1.07×.

Acceptance 2 - does it reproduce the real ordering? The real bench says the fine-tuned FT03 beats PCA-RX (pAUC 0.158 against 0.128). bench-v2 may judge models only if it says the same.

Values that will drift

bench-v2, `configs/eval/bench_v2.json`, 18 hosts, 2,913 objects, residual Mahalanobis, 2026-10-17 [M:`research/experiments/implant-bench/README.md`]. Chance: 10.1% / 1.1% / 0.005.

	caught top 1% (0.1%)	pAUC	easy	medium	subtle
F001 ep109	29.2% (10.2%)	0.145	48.5%	23.1%	13.1%
FT02c ep10	32.8% (11.4%)	0.166	52.9%	27.6%	14.7%
FT03 ep10	32.3% (11.4%)	0.164	52.7%	26.9%	14.2%
PCA-RX (no model)	30.2% (9.1%)	0.154	47.7%	24.9%	15.2%

(v0.2.0's row, 22.1%, is void: a reading artefact of the sort bug, Chapter 37.) In objects at the top 1%: FT03 $0.323 \times 2{,}913 = 940.9$, about 941; PCA-RX $0.302 \times 2{,}913 = 879.7$, about 880.

Met: both fine-tuned models above PCA-RX on every headline measure (subtle is a tie). And the subtle tier, at chance on bench-v1, is now above it (13–15% against 10.1%).

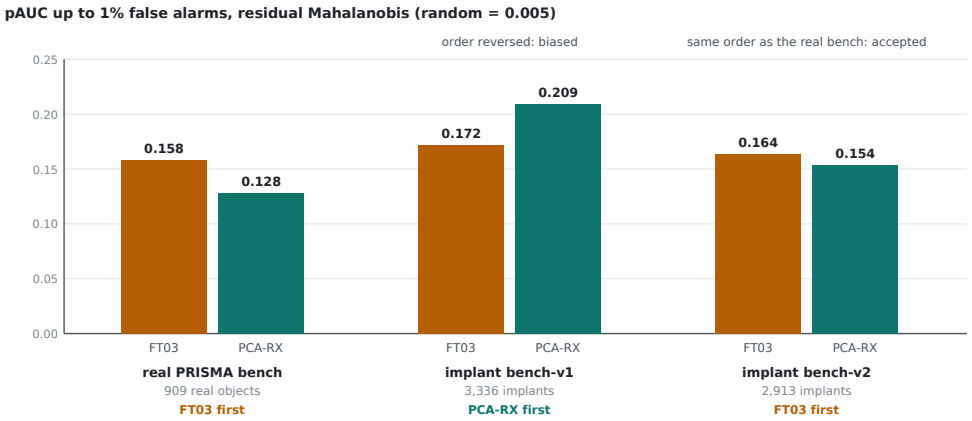


Figure 34.6: pAUC to 1% false alarms for FT03 (amber) and PCA-RX (teal) on three benchmarks. The real PRISMA bench puts FT03 first; bench-v1 reversed that order; bench-v2 restores it.

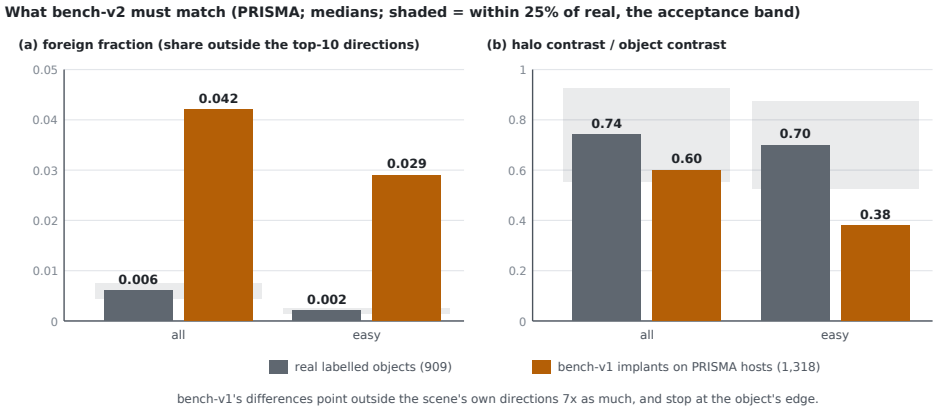


Figure 34.7: What bench-v2 had to match: medians on PRISMA, real objects (grey) against bench-v1 implants (amber), overall and in the easy tier, on the diagnosis ruler; the shaded band is within 25% of the real value.

Why the ordering test is the one that matters. Acceptance 1 checks that the implants *look* right on the properties somebody thought to measure. Acceptance 2 checks that the bench-

mark *behaves* right on a fact nobody built into it. bench-v1 would have passed a looser acceptance 1 - its sizes and contrasts matched - and still ranked a classical detector first. A benchmark is a model of reality; test it on something you already know before you use it to learn something you do not.

34.14 bench-v2.1

After the descending-wavelength fix (commit b94dbfd) every signature implant had to be rebuilt, and the foreignness tolerance was tightened from 1.5 to 1.2: bench-v2.1, built 2026-10-18 with `--version 2 --psf-sigma 1.4 --foreign-tol 1.2 --seed 0`, 18 hosts, 2,817 objects (signature 985, material 927, host 905).

Values that will drift

bench-v2.1, `configs/eval/bench_v2_1.json`, residual Mahalanobis, 2026-10-18 [M: `research/experiments/implant-bench/README.md`].

model	caught top 1% (0.1%)	pAUC	easy	medium	subtle
v0.2.0 (read correctly)	35.5% (12.6%)	0.168	53.7%	29.1%	19.5%
F001 ep109	33.1% (11.7%)	0.157	51.7%	26.7%	16.9%
FT02c ep10	35.1% (12.8%)	0.169	53.7%	28.7%	18.6%
FT03 ep10	34.9% (12.7%)	0.168	53.7%	28.3%	18.6%
FT05 ep5	35.4% (12.8%)	0.170	54.3%	28.9%	18.8%
PCA-RX	33.2% (9.8%)	0.154	50.4%	26.9%	18.4%

Per-tier foreignness against real (PRISMA hosts): 1.05 / 1.15 / **1.60**×; the tighter tolerance skips low-foreignness easy targets, so the survivors skew high. Halo 0.92–1.06×.

Read correctly, v0.2.0 ties or beats every new model here, as it ties them on the real bench; all models stay above PCA-RX at the top 0.1% (12.6–12.8% against 9.8%), as on the real bench; at the top 1% pretraining alone (F001, 33.1%) ties PCA-RX (33.2%). Since 2026-10-17 vessels on water are in scope (Chapter 35), but `place_and_implant` still removes water and cloud from the eligible area, so no bench-v2.x object sits on water.

34.15 Common confusions

HAD100 is AVIRIS-NG and we train on AVIRIS-NG, so it fits?
Same sensor, different product: the training data is L2 reflectance, HAD100 ships L1B radiance.

No exact flight-line overlap means no leakage worry?
Six scenes are 1–2 km from a training swath flown the same day. Not pixel leakage, but not unseen ground either; report both ways.

The ring is the 3×3 neighbourhood?
It is a 20-pixel diamond at Manhattan distance 2–3 for a one-pixel object; distance 1 (the halo) is skipped.

The tier is the one the builder aimed for?
It is the realised one, measured after implanting.

The foreignness of the added signal is the object's foreignness?
Only on a spot identical to its ring. The evaluator sees natural offset plus signal (worked example 34.8).

The PSF σ and the halo ratio are the same number?
The PSF sets the blur; the halo ratio is measured afterwards. $\sigma = 1.4$ gives 0.667 on flat ground and about 0.64 on real texture.

bench-v2 passed, so its absolute numbers are what the product achieves?
It passed a ranking test. Real objects are caught at about 45% at the top 1%, bench-v2's at about 32%. Compare models within one bench.

34.16 Check yourself

Exercise 34.1. Count the ring of a two-pixel horizontal object (pixels at Manhattan distance 2 or 3 from the nearer object pixel). Count row by row.

Exercise 34.2. A ring has mean 0.30 and σ 0.06 in each of two bands. An object reads 0.36 and 0.27. Compute z and its tier.

Exercise 34.3. One band: $r = 0.40$, $\sigma = 0.10$, $o_0 = 0.43$, $\Delta = -0.20$. What strength gives $z = 1.5$? What does the simple formula give, and what z does it realise?

Exercise 34.4. With $\mathbf{P} = (1, 0, 0)$, match $\mathbf{d} = (3, 0, 3)$ to foreignness 0.1: find β and $\mathbf{d}'$, and verify.

Exercise 34.5. Compute the PSF weight at offset $(2, 0)$ and $(1, 1)$ for $\sigma = 1.0$, the ring mean weight of a one-pixel object at $\sigma = 1.0$ (use $e^{-2} = 0.1353$, $e^{-1} = 0.3679$, $e^{-4.5} = 0.0111$, $e^{-2.5} = 0.0821$, $e^{-0.5} = 0.6065$), and the flat-ground halo ratio. Is the blur sharper or softer than the build's?

Exercise 34.6. A spot's natural offset is $(0.00, 0.02)$; the added signal is $(0.06, 0.00)$; band 0 is the scene's main direction. What is the realised foreignness, and what did the signal alone have?

Next: Chapter 35, Running an evaluation, and what a mask costs.

Chapter 35

Running an evaluation, and what a mask costs

The one thing this chapter teaches

An evaluation here is one JSON config - models, datasets, scorers, measures - run by one command that loops scene by scene, caches every score map under a key built from everything that could change it, and reports objects caught beside their chance level; its first real run showed that every benchmark needs a model-free baseline in it, and its second showed that excluding a surface class is a scope decision with a price you can count in objects.

35.1 Why one evaluator

Chapters 33 and 34 gave you the measures and the implant bench. Until 2026-10-17 each was run by its own script (`scripts/evaluate_prisma_bench.py`, `scripts/evaluate_implant_bench.py`, code in `app/utils/evaluation/`), and each read bands, reconstructed and counted slightly differently. Two scripts that disagree in the third decimal cannot tell you whether a model improved in the third decimal. On 2026-10-17 they were replaced by one subsystem (commit f6a444b), designed in `docs/lld/evaluation.md`:

file	what it holds
`app/evaluation/config.py`	`EvaluationConfig` and its four kinds of spec (model, dataset, scorer, metric)
`app/evaluation/datasets.py`	`PrismaBench`, `ImplantBench`, `EvalScene`
`app/evaluation/models.py`	`EvalModel`: reading, reconstruction, segmentation
`app/evaluation/scorers.py`	`score()`: every scorer kind
`app/evaluation/metrics.py`	the measures of Chapter 33, exact, and the analytic chance
`app/evaluation/runner.py`, `report.py`	the loop, the cache, the tables and `report.md`
`scripts/evaluate.py`	the command
`configs/eval/stage3.json`	the reference config

Analogy

A driving examiner builds a course of cones. A go-kart, low and twitchy, weaves through it faster than a rally driver in a real car; on a real road the rally driver wins by a mile. Nothing

was wrong with the stopwatch: the course rewarded what go-karts are good at. You only find that out if you put a go-kart on the course on purpose. In this chapter the go-kart is a model-free detector, PCA-RX, and the course is bench-v1.

35.2 One config

The command is

```
python scripts/evaluate.py configs/eval/stage3.json [--output <dir>]
```

and the config has five blocks (shortened from `configs/eval/stage3.json`):

```
{"name": "stage3: ...",
 "models": [
   {"name": "v0.2.0", "checkpoint": "C:/es/models/...epoch200.pt"},
   {"name": "FT03_ep10", "checkpoint": "s3://.../FT03-joint/checkpoints/best.pt"}, ...],
 "datasets": [
   {"kind": "prisma_bench", "path": "C:/Users/FITT/Documents/stage1_datasets"},
   {"kind": "implant", "path": "C:/es/implant/bench-v1"}],
 "scorers": [
   {"kind": "residual_mahalanobis"},
   {"kind": "combined"},
   {"kind": "exclude", "classes": ["water", "cloud"],
    "segmenter": "FT03_ep10", "fit_on_kept": false,
    "base": {"kind": "residual_mahalanobis"}}, ...,
   {"kind": "pca_rx"}],
 "metrics": [
   {"kind": "roc_auc"},
   {"kind": "pauc", "max_fpr": 0.01},
   {"kind": "tpr_at", "fpr": 0.001},
   {"kind": "objects_caught", "top_pct": [1.0, 0.1]}],
 "group_by": ["tier", "source", "sensor"],
 "output": "C:/es/eval/stage3"}
```

Four things in it need explaining.

A model's reading. Each model reads a scene its own way (`reading`, default `auto`): the wavelength-reading models read the scene's own bands within their trained-wavelength record; the shipped v0.2.0 reads the 165-band common grid (Chapter 5). PRISMA scenes reload through the product's band filter; implant hosts are linearly resampled, "close to, not identical with" the product.

Source check: the resample and the sort bug

On PRISMA hosts the resample was far from close until commit b94dbfd: PRISMA lists its bands descending and `np.interp` was called unsorted, so every grid band v0.2.0 read was a copy of one of two near-infrared bands. Invariant 2 of `docs/lld/evaluation.md` now requires `resample` to sort first, and the fixed generic path reproduces the product path on the real bench exactly. Every v0.2.0 implant number before 2026-10-18 is void (Chapter 37).

Two new scorers. `exclude` takes any base scorer, runs it, then sets chosen classes to 0 using a segmentation head's class map (`app/evaluation/scorers.py`, lines 60–63):

```
out = np.isin(classes, [CLASS_IDS[c] for c in spec.classes])   # (H, W) bool: excluded
s = score(spec.base, cube, recon, valid & ~out if spec.fit_on_kept else valid,
          classes, log_b)                                       # (H, W) base scores
```

```
return np.where(out, 0.0, s).astype(np.float32)                # excluded -> 0, still valid
```

`pca_rx` is the classical baseline: residual Mahalanobis (Chapter 31) handed a reconstruction of all zeros, so the "residual" is the cube itself. With the defaults `d=30`, `local_window=(21,61)` that is local RX (Chapter 8) on the scene's top 30 whitened principal components: no model, no training.

Labels. Each scorer gets a name from its settings, for example

```
residual_mahalanobis(d=30,w=21x61)
exclude(water+cloud;mask)[residual_mahalanobis(d=30,w=21x61)]
```

`mask` when `fit_on_kept` is false (score the whole scene, zero the classes afterwards), `fit` when the base scorer's statistics are fitted on the kept pixels only.

Validation. The config refuses to start if two models share a name, two scorers share a label, an `exclude` lacks its classes or segmenter, or a segmenter is not one of the configured models (`test_config_validation`).

35.3 The scene-major loop

`app/evaluation/runner.py` goes **scene first, then model**:

```
for each scene (one in memory):
  for each model:  reconstruct once -> every model-based scorer -> free the reconstruction
  model-free scorers (pca_rx) once, on the scene's own cube
  class maps (for exclude) once per scene per segmenter, cached
  each score map -> the measures -> per-object rows + per-scene rows
```

Why that order: a reconstruction is as large as the cube. One PRISMA scene of about 1,000 × 1,000 pixels and 185 bands [A] in 4-byte floats is

$$185 \times 1{,}000 \times 1{,}000 = 185{,}000{,}000 \text{ numbers}$$
$$185{,}000{,}000 \times 4 \text{ bytes} = 740{,}000{,}000 \text{ bytes} \approx 0.74 \text{ GB}.$$

The old scripts held several reconstructions at once and ran out of memory (LLD, "Decisions"). Now memory holds one scene and one reconstruction.

Worked example 35.1: Counting the work of stage 3

The stage-3 config has 4 models, 4 model-based scorers (residual Mahalanobis, combined, two exclusions), 1 model-free scorer, and 4 PRISMA scenes plus 18 implant hosts.

$$\text{score maps per scene} = 4 \text{ models} \times 4 \text{ scorers} + 1 = 16 + 1 = 17$$
$$\text{scenes} = 4 + 18 = 22$$
$$\text{score maps in all} = 22 \times 17 = 374$$
$$\text{reconstructions} = 22 \times 4 = 88 \quad \text{(one per scene per model)}$$

`test_end_to_end_and_cache` pins the "never more": a stand-in model counts its calls, and after a run with three model-based scorers it has been called once per scene.

35.4 The cache and its key

Every score map is saved, compressed, under `<output>/cache/scores/`. As 16-bit floats a 1,000 × 1,000 map is 1,000,000 × 2 = 2,000,000 bytes, about 2 MB before compression, against 740 MB for the reconstruction it came from. That ratio is why score maps are cached and reconstructions are not.

Float16, and its range. A 16-bit float stores numbers up to 65,504; anything larger becomes infinity. Residual Mahalanobis is a squared distance with a heavy tail, and the blurred and mixture variants of 2026-10-19 reached 10^5. A cached infinity would put that pixel at the top of every ranking for ever, so since then a map is stored as float16 only if all its values are finite and below 6×10^4, otherwise as float32 (`runner.py`, lines 115–118).

The key. The score map's path is

$$\texttt{scores/}\langle\text{dataset}\rangle\texttt{/}\langle\text{scene}\rangle\texttt{/}\langle\text{model id}\rangle\texttt{/}\text{sha1}(\text{scorer spec JSON} \,\|\, \texttt{"|"} \,\|\, \text{scene fingerprint})_{[:10]}\texttt{.npz}, \tag{35.1}$$

where ‖ means "joined to" and $_{[:10]}$ the first ten hexadecimal characters (`_score_path`, line 144). The *model id* is the model's name followed by the first ten characters of a SHA-1 of the checkpoint's resolved path, size, modification time and reading mode, plus adaptation settings and cloud rule when those are set (`_model_id`, line 130). The *scene fingerprint* is the PRISMA `.he5` file's size and modification time, or an implant host's `cube.npy` and `gt.npy` SHA-256s from `manifest.json`. A *hash* such as SHA-1 turns any text into a fixed-length code that changes completely when any character of the text changes, so equal keys mean equal inputs.

Change any part and the map is recomputed. Add a new measure or a new `group_by` and **no model runs at all**: the measures are recomputed from cached maps.

Source check: what the key did and did not cover

When the course part for this chapter was written, the scene's content was not in the key: a benchmark rebuilt in place under the same scene names, evaluated into the same output directory, reused the old score maps against the new ground truth with no error. Commit 229d423 (2026-10-18) added the fingerprint for score maps. *Class maps* for `exclude` are still keyed by scene name and segmenter only (`classes_for`, `runner.py` lines 80–90), so a host rebuilt in place keeps a class map computed on its old pixels. And because the key includes the scorer spec's full JSON, adding any field to `ScorerSpec` changes every key: the scoring-gains maps of 2026-10-19 could not be reused for the bootstrap of Chapter 36 after `scale_power` and `scale_floor_pct` were added (`docs/lld/evaluation.md`, "Decisions").

35.5 Chance, per object, and the report

Chapter 33 derived the chance level of "objects caught": an object with m cells after the 1-pixel dilation is caught by random scores at budget q% with probability $1 - (1 - q/100)^m$. The evaluator stores this for every object (`chance_1`, `chance_0.1` in `per_object.csv`) and the report averages it into a `*chance*` row at the top of every table: **10.2%** at the top 1% on the PRISMA bench and **10.0%** on bench-v1

[M: `docs/training/anomaly-detection/19-running-an-evaluation.md`].

Worked example 35.2: A bench's chance row

A toy bench [A] has 6 one-pixel objects ($m = 5$), 3 dominoes ($m = 8$) and 1 three-by-three square ($m = 21$). At the top 1%:

$$\begin{aligned} \text{one-pixel} &: 1 - 0.99^{5} = 0.0490 && \times 6 = 0.2941 \\ \text{domino} &: 1 - 0.99^{8} = 0.0773 && \times 3 = 0.2318 \\ \text{square} &: 1 - 0.99^{21} = 0.1903 && \times 1 = 0.1903 \end{aligned}$$

$$\text{chance row} = (0.2941 + 0.2318 + 0.1903)/10 = 0.7162/10 = 0.0716.$$

About 7.2%. A model catching 2 of these 10 objects (20%) is about 2.8 times chance.

`report.py` writes one section per dataset, **never pooled**, because their objects are different populations (LLD, invariant 5). In each: a main table with one row per `model | scorer` and the chance row first; "caught top q%" is the share of all objects in that dataset; pixel measures are *means over scenes*; one table per `group_by` attribute. Model `-` marks a model-free scorer. The run also writes `per_object.csv`, `per_scene.csv`, `summary.json` and `provenance.json` (commit, checkpoint SHA-256s).

35.6 Stage 3: the numbers held

A new framework that disagrees with the old one tells you nothing until you know which is wrong. So the reference run's first job was to reproduce every earlier figure. It did: on the PRISMA bench v0.2.0 scored pAUC 0.159, ROC-AUC 0.854 and 402 of 909 objects at the top 1%; F001 0.148 / 0.861 / 400; FT03 0.158 / 0.853 / 413 [M: `docs/lld/evaluation.md`].

35.7 The go-kart: PCA-RX wins on implants and loses on real objects

Values that will drift

Stage 3, `configs/eval/stage3.json`, 2026-10-17
[M: `research/experiments/implant-bench/README.md`].

	implant bench-v1		real PRISMA bench		
	caught top 1%	pAUC	pAUC	TPR at 0.1%	caught top 0.1%
FT03 + res. Mahal.	26.9%	0.172	0.158	0.058	14.3%
PCA-RX, no model	**30.2%**	**0.209**	0.128	0.029	8.1%

Count it in objects. On the implants PCA-RX catches $0.302 \times 3{,}336 = 1{,}007$ objects to FT03's $0.269 \times 3{,}336 = 897$. On the real objects it reaches half of FT03's detection rate at 0.1% false alarms ($0.029/0.058 = 0.5$) and catches $0.081 \times 909 = 74$ objects at the top 0.1% to FT03's $0.143 \times 909 = 130$. With bench-v1 as your only benchmark you would conclude that the models add nothing a 1990s detector [36] cannot do. The real objects say the opposite.

Why. Chapter 34 measured it: bench-v1 implants are seven times as foreign as real objects (foreign fraction 0.042 against 0.006), and whitening multiplies exactly foreignness. Work the multiplication once. Suppose the scene varies with standard deviation 0.05 along its main direction and 0.002 along a minor one [A], and a pixel differs by 0.01 along each in turn. Whitening

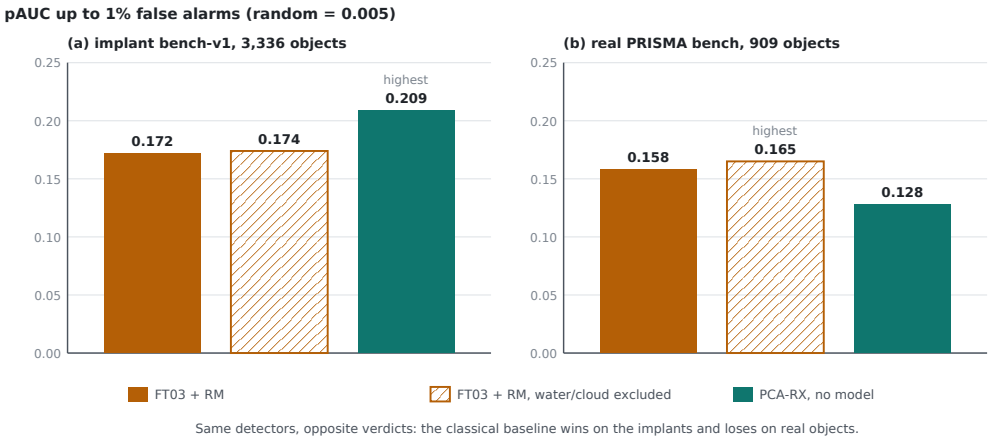


Figure 35.1: pAUC to 1% false alarms. On implant bench-v1 (a) the model-free PCA-RX (teal) is highest; on the real PRISMA bench (b) it is lowest, and excluding water and cloud (hatched) helps the model on both.

divides each component by its direction's standard deviation and squares:

$$\begin{aligned} \text{main direction} &: (0.01/0.05)^2 = 0.2^2 = 0.04 \\ \text{minor direction} &: (0.01/0.002)^2 = 5^2 = 25 \\ \text{ratio} &: 25/0.04 = 625. \end{aligned}$$

The same size of difference scores 625 times higher when it points where the scene does not vary. PCA-RX needs nothing more to light up a foreign implant. A model earns its keep on real objects, which are unusual mixtures of the scene's own materials, by having learnt what each place *should* look like from its surroundings.

Put a model-free baseline in every benchmark: if it wins there but loses on real labels, the benchmark is wrong.

That is why bench-v2 had to pass the ordering test (Section 34.13) before any model was ranked or trained on implants.

35.8 Exclusion, and why ROC-AUC falls while the top improves

The first product rule (user, 2026-10-17) was **"we don't detect anything on water and cloud"**. FT03's segmentation head marks them; `exclude` zeroes them. Invariant 7 of the LLD spells out the bookkeeping: **excluded pixels stay in the measures**, as negatives tied at 0 at the bottom of the ranking, and any labelled object inside them stays a positive that can no longer be found - "the product would not flag it either".

Two things happen when a large area is set to 0: its background pixels all tie at the bottom, which is harmless for the top of the list; and any target inside it is scored 0 too, which costs it. The AUC counts the second; the top-of-list measures mostly see the removal of bright false alarms.

Worked example 35.3: Exclusion in miniature

Two targets, A (land) and B (on water); four background pixels, one a bright water glint. ROC-AUC is the share of (target, background) pairs the target wins, a tie counting half

(Equation (33.3)).

before: A 0.9, B 0.6; background: glint 0.95 (water), 0.5, 0.2, 0.1 (water)
A beats $0.5, 0.2, 0.1 \to 3$ of 4; B beats $0.5, 0.2, 0.1 \to 3$ of 4
AUC $= (3+3)/8 = 0.75$; top pixel: the glint, a false alarm

exclude water: glint $\to 0$, $0.1 \to 0$, B $\to 0$
A beats $0, 0.5, 0.2, 0 \to 4$ of 4
B (0) loses to 0.5, 0.2; ties the two 0s $\to 0 + 0 + \frac{1}{2} + \frac{1}{2} = 1$
AUC $= (4+1)/8 = 0.625$; top pixel: A, a hit.

AUC fell and the top of the list got better: the same two directions as stage 3.

On the real bench, with FT03 [M: docs/lld/evaluation.md]: excluding water and cloud lifted objects caught at the top 0.1% from 14.3% to 16.5% ($0.143 \times 909 = 130$ to $0.165 \times 909 = 150$, 20 more objects in the same review budget) and pAUC from 0.158 to 0.165, while ROC-AUC fell from 0.853 to 0.830. Most of the fall is set1, which FT03 calls 55% water (ROC-AUC 0.932 to 0.851 there). Every model gained at the top 0.1% by about the same amount, including v0.2.0, which has no segmentation head: **the gain belongs to the mask, not to any model**.

35.9 Scope: cloud excluded, water kept

Analogy

A security guard is told to ignore the car park, because it is full of harmless reflections. Her shift gets quieter. But the day somebody breaks into a car, nobody sees it - the instruction did not make the car park safe, it made it invisible. And if her map of the car park is drawn too large, she also ignores half the loading bay.

So the question "should we exclude water?" is not a modelling question. It is: **are objects on water something the product must find?** For vessels, yes.

The claim that was wrong. Before the decision the record said "no real object sits on water". It came from a scan that called a pixel water when its visible reflectance (mean over roughly 450–650 nm) was below 0.04. Clear open water is that dark; turbid coastal water, with silt or a shallow bottom, is around 0.08.

Worked example 35.4: Two water rules on one turbid pixel

A pixel with visible 0.08, red 0.06, near-infrared 0.03 [A].

scan's rule: VIS < 0.04? $0.08 < 0.04$ is false $\to$ "land"

$$\text{NDVI} = \frac{\text{NIR} - \text{red}}{\text{NIR} + \text{red}} = \frac{0.03 - 0.06}{0.03 + 0.06} = \frac{-0.03}{0.09} = -0.33$$

builder's rule: NIR < 0.08 and NDVI < 0? yes and yes $\to$ water.

Water is dark in the near-infrared even when it is bright in the visible; the scan used the wrong band.

Re-checked on 2026-10-17 against FT03's water / cloud mask, **39 of the 909 real objects lie wholly inside it** [M: docs/lld/evaluation.md]:

where	objects	what they are
set1, on water, more than 3 px offshore	13	single pixels; likely vessels
set1, on water, within 3 px of shore	14	single pixels near the coast
set2 and set3, shorelines	8	objects on the land–water edge
set4, under haze FT03 calls cloud	4	a scope question at the haze edge
total	39	

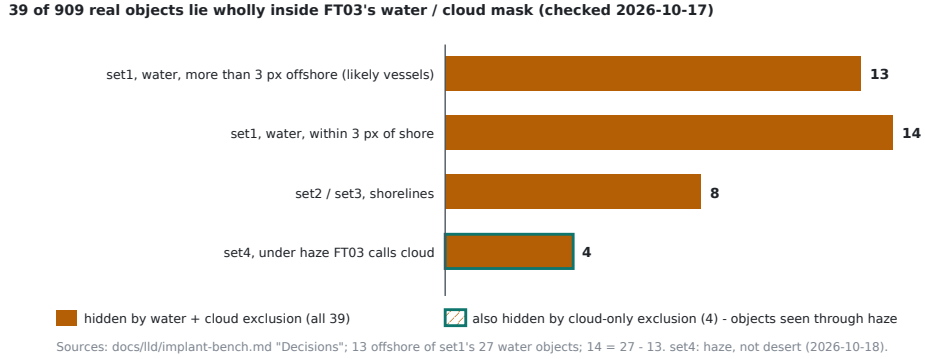


Figure 35.2: The 39 labelled objects lying wholly inside FT03's water / cloud mask, by kind. Excluding water and cloud hides all 39 (amber); excluding cloud only hides the 4 in set4 that sit under haze FT03 calls cloud.

The user's rule then changed: **vessels on water are in scope; only cloud is excluded.**

What each choice buys, counted in objects. FT03, residual Mahalanobis, PRISMA bench (`C:/es/eval/cloud\_only/report.md` on the dev laptop):

Values that will drift

`configs/eval/cloud_only.json`, 2026-10-17 [M: `docs/training/anomaly-detection/21-scope-and-masks.md`]. Objects caught at the top 0.1%, per scene, from `per_object.csv`:

	set1	set2	set3	set4	total
plain	56	33	7	34	130
cloud only	56	33	13	42	144
water + cloud	59	34	14	43	150

pAUC 0.158 / 0.166 / 0.165 and ROC-AUC 0.853 / 0.850 / 0.830 (plain / cloud only / water + cloud).

Work the shares: 130/909 = 14.3%, 144/909 = 15.8%, 150/909 = 16.5%. Cloud exclusion buys 14 objects (144 − 130: set3 +6, set4 +8) and costs almost no ROC-AUC. Adding water buys 6 more at the top 0.1% but none at the top 1% (both catch 419/909 = 46.1%), takes ROC-AUC from 0.850 to 0.830, and hides the vessels. pAUC, the measure the analyst's view tracks, is no better for it (0.165 against 0.166). **Water exclusion trades the vessels for a slightly tidier top of the list**; with vessels in scope, that is a bad trade. Chapter 36 shows that cloud exclusion's gain is one of the few on this bench whose interval clears zero.

The set4 haze, and a claim that was wrong twice. Cloud-only exclusion still hides 4 objects in set4. On 2026-10-17 that was read as a mask error, "FT03 calls bright desert cloud". A look at the true-colour picture on 2026-10-18 showed a bright cloud/smoke streak and a

large hazy patch that FT03's 11% follows. The colour test settles it: bright desert is red, its reflectance rising with wavelength, so near-infrared over blue is above 1; haze scatters blue most, so near-infrared over blue falls below 1.

$$\begin{aligned} \text{desert-like: blue } 0.20,\ \text{NIR } 0.35 &\rightarrow 0.35/0.20 = 1.75 \quad \text{(red)} \\ \text{set4 “cloud”: blue } 0.30,\ \text{NIR } 0.18 &\rightarrow 0.18/0.30 = 0.60 \quad \text{(bluish)} \end{aligned}$$

For 75% of set4's “cloud” pixels the ratio is 0.54–0.65 [M: `docs/training/anomaly-detection/21-scope-and-masks.md`]: haze. The 4 objects are things a labeller could see through haze, a scope question (should the product look under thin cloud?), not a mask fault. A negative claim about a picture deserves the same check as a positive one: look at the picture.

The arithmetic of a wrong mask. The next mask may be wrong, so know its price. Had 11% of set4 been called cloud wrongly, the analyst would lose 11% of the scene, holding 4 of set4's 230 objects [M: `docs/training/anomaly-detection/21-scope-and-masks.md`] ($4/230 = 1.7\%$). Small here; but on a scene full of targets a wrong mask loses them in proportion, *silently* - the score is 0, not “unknown”.

> Decide scope first, then count the labelled objects inside the mask, then audit the mask against the picture, and only then judge the result at the top of the list.

35.10 Common confusions

PCA-RX won on the implants, so the models are no better than classical detection?
It won on implants built in a way that suits it. On the 909 real objects it is last. The benchmark was wrong; the stopwatch was fine.

Exclusion lowered ROC-AUC, so it made detection worse?
It tied a large area at 0 and hid the objects inside it; the top of the list, where the analyst looks, improved for every model. Judge it by pAUC, TPR at 0.1% and objects caught, with ROC-AUC beside them.

I added a measure and re-ran; the numbers cannot be stale?
Score maps are keyed by scene fingerprint since 229d423; class maps are not. Give each benchmark version its own path and output directory.

`exclude` with segmenter FT03 only applies to FT03?
It applies to every model in the config; the segmenter only supplies the class map.

The PRISMA bench and the implant bench are averaged in the summary?
Never. Each dataset gets its own section and its own chance row.

Cloud-only exclusion also hides the 39 objects?
No: 35 of them are inside the water mask. Cloud-only hides the 4 in set4 under haze.

35.11 Check yourself

Exercise 35.1. A config has 3 models, 3 model-based scorers and 1 model-free scorer, over 5 scenes. How many score maps and reconstructions does a first run make? And a second run after adding one measure?

Exercise 35.2. In worked example 35.3, suppose B had been on land (not excluded) at 0.6. What are the AUC and the top pixel after excluding water?

Exercise 35.3. A pixel has visible 0.07, red 0.05, NIR 0.02. What do the scan's rule and the builder's rule call it?

Exercise 35.4. At the top 1%, plain FT03 catches 101 / 103 / 48 / 161 objects on set1–4, cloud-only 101 / 103 / 54 / 161, water + cloud 98 / 107 / 53 / 161. Compute each total and share of 909. What does water exclusion swap?

Exercise 35.5. A score map's largest value is 70,000. What happens if it is cached as float16, and what does the runner do instead? How many bytes does a $1{,}000 \times 1{,}000$ map take uncompressed in each format?

Exercise 35.6. Put the four questions of the last key point in order for a proposal to exclude snow.

Next: Chapter 36, How small a difference can we see: the paired object bootstrap.

Chapter 36

How small a difference can we see: the paired object bootstrap

The one thing this chapter teaches

The real PRISMA bench is 4 scenes and 909 objects, so a difference of 0.005 pAUC between two scorers can be luck in which objects happen to be on it; the paired object bootstrap redraws the objects with replacement, the same draw for both scorers, thousands of times, and since 2026-10-21 a gain counts only if the 95% interval of that difference clears zero.

36.1 The problem: 909 objects is not many

By October 2026 every verdict on a new model or score leaned on gaps like 0.170 against 0.163 pAUC (Chapter 33 for pAUC). Nobody had asked how big a gap the bench can tell apart from noise. On 2026-10-21 that question got its own small experiment and tool:

file	what it holds
`research / experiments / bench - resolution/README.md`	the question, the method, the results, the new rule
`app/evaluation/significance.py`	`object_pauc_parts` and `paired_bootstrap`: the method in 63 lines
`scripts/compare_scorers.py`	the command line: two (model, scorer) pairs from a finished run
`tests / test _ evaluation / test _ significance.py`	two tests: parts reproduce pAUC; identical scorers differ by 0

The gains being chased were about 0.005 to 0.01 pAUC. FT04's real-bench pAUC with cloud excluded is 0.170 [M: `research/experiments/bench-resolution/README.md`], so a gain of 0.005 is a change of $0.005/0.170 = 0.029$, about 3%. Would a bench of the same kind, with different objects on it, show the same 3%? You cannot collect another bench. You can ask the bench you have.

Analogy

Two teachers each mark the same class of 30 essays, and teacher B's average is 0.4 marks higher. Is B more generous, or did a few essays happen to suit B? You cannot hire a new class, but you can ask "if the class had been a slightly different 30 essays, drawn from the same kind of students, would B still come out ahead?" You redraw the class from the 30 you have, many times, and look at B minus A each time. Crucially, both teachers mark the

same redrawn class each time; otherwise you would compare one teacher's easy class with the other's hard one.

36.2 Sampling variation and the standard error

Any number computed from data - a mean, a pAUC - would come out differently on different data of the same kind. The spread of those hypothetical values is the number's *sampling distribution*, and its standard deviation is the *standard error*. For a plain mean there is a formula. You need two rules, each shown on a tiny example first.

Rule 1: the variance of a sum. For two random quantities X and Y with means μ_X, μ_Y, the *covariance* is $\mathrm{Cov}(X,Y) = \mathbb{E}[(X-\mu_X)(Y-\mu_Y)]$: positive when they tend to rise together. Then

$$\begin{aligned}
\mathrm{Var}(X+Y) &= \mathbb{E}\big[(X+Y-\mu_X-\mu_Y)^2\big] && \text{definition of variance}\\
&= \mathbb{E}\big[((X-\mu_X)+(Y-\mu_Y))^2\big] && \text{regroup}\\
&= \mathbb{E}[(X-\mu_X)^2] + 2\,\mathbb{E}[(X-\mu_X)(Y-\mu_Y)] + \mathbb{E}[(Y-\mu_Y)^2] && \text{expand the square}\\
&= \mathrm{Var}\,X + 2\,\mathrm{Cov}(X,Y) + \mathrm{Var}\,Y.
\end{aligned}$$

Tiny check: let X be a fair coin worth 0 or 1 and $Y = X$. Then $X+Y$ is 0 or 2 with equal chance, mean 1, variance $\frac{1}{2}(0-1)^2 + \frac{1}{2}(2-1)^2 = 1$. The rule: $\mathrm{Var}\,X = \frac{1}{4}$, $\mathrm{Cov}(X,X) = \frac{1}{4}$, so $\frac{1}{4} + 2\times\frac{1}{4} + \frac{1}{4} = 1$. Agreed. If X and Y are *independent*, the covariance is 0 and the variances add.

Rule 2: the variance of a mean. Take n independent values, each of variance σ^2. Their mean is the sum divided by n, and multiplying a quantity by c multiplies its variance by c^2:

$$\begin{aligned}
\mathrm{Var}\Big(\tfrac{1}{n}\textstyle\sum_i X_i\Big) &= \tfrac{1}{n^2}\,\mathrm{Var}\Big(\textstyle\sum_i X_i\Big) && \text{pull out } c = 1/n\\
&= \tfrac{1}{n^2}\cdot n\sigma^2 && \text{rule 1, independent: variances add}\\
&= \frac{\sigma^2}{n}, \qquad \text{standard error } = \frac{\sigma}{\sqrt{n}}.
\end{aligned}$$

For a pAUC built from thousands of correlated pixels there is no such clean formula. The bootstrap gets the standard error without one.

36.3 The bootstrap idea, by hand

Definition 36.1 (Bootstrap)**.** A *bootstrap* [6] estimates the sampling distribution of a number using only the data you have. From the n data items, draw n items *with replacement* - each draw picks any item, so some appear twice and some not at all - recompute the number on that *bootstrap sample*, and repeat many times. The spread of the recomputed numbers stands in for the spread you would have seen across fresh data.

The logic: the data are your best picture of the population they came from. Drawing from the data with replacement is drawing from that picture, so it mimics collecting a new dataset.

Worked example 36.1: Bootstrapping the mean of five numbers

Do this on paper. The data are $x = (2, 4, 5, 7, 12)$, with mean $(2{+}4{+}5{+}7{+}12)/5 = 30/5 = 6$. Six bootstrap draws are given [A] as positions 1–5 (a random number generator would supply them):

draw	positions	values	mean
1	4, 1, 5, 5, 2	7, 2, 12, 12, 4	$37/5 = 7.4$
2	2, 2, 3, 1, 4	4, 4, 5, 2, 7	$22/5 = 4.4$
3	3, 5, 1, 3, 2	5, 12, 2, 5, 4	$28/5 = 5.6$
4	1, 1, 2, 4, 3	2, 2, 4, 7, 5	$20/5 = 4.0$
5	5, 4, 3, 2, 5	12, 7, 5, 4, 12	$40/5 = 8.0$
6	2, 3, 4, 4, 1	4, 5, 7, 7, 2	$25/5 = 5.0$

Draw 1 contains 12 twice and 5 not at all: that is "with replacement". The mean of the six bootstrap means:

$$(7.4 + 4.4 + 5.6 + 4.0 + 8.0 + 5.0)/6 = 34.4/6 = 5.733.$$

Their spread (population standard deviation): deviations from 5.733 are $1.667, -1.333, -0.133, -1.733, 2.267, -0.733$; squares $2.778, 1.778, 0.018, 3.004, 5.138, 0.538$; sum 13.253; divided by 6, 2.209; square root

$$\text{bootstrap standard error} \approx 1.486.$$

Compare rule 2: the data's population standard deviation is $\sqrt{(16 + 4 + 1 + 1 + 36)/5} = \sqrt{58/5} = \sqrt{11.6} = 3.406$, so $\sigma/\sqrt{5} = 3.406/2.236 = 1.523$. Six draws already land near it; with 100,000 draws the book's script gets 1.522 [C].

36.4 The percentile interval

Definition 36.2 (Percentile 95% interval)**.** The *percentile 95% interval* from a bootstrap is the 2.5th and 97.5th percentiles of the recomputed numbers: the range holding the middle 95% of them.

`paired_bootstrap` uses `np.percentile(..., [2.5, 97.5])`, which interpolates linearly. With n values sorted from smallest (position 0) to largest (position $n - 1$), the q-th percentile sits at position $(n - 1)\, q/100$; if that falls between two positions, it takes the matching share of the gap.

Worked example 36.2: Percentiles of the six bootstrap means

Sorted: $4.0, 4.4, 5.0, 5.6, 7.4, 8.0$ at positions 0–5.

2.5th: position $(6 - 1) \times 0.025 = 0.125$ between positions 0 and 1

$$4.0 + 0.125 \times (4.4 - 4.0) = 4.0 + 0.05 = 4.05$$

97.5th: position $5 \times 0.975 = 4.875$ between positions 4 and 5

$$7.4 + 0.875 \times (8.0 - 7.4) = 7.4 + 0.525 = 7.925.$$

The interval from six draws is $[4.05, 7.925]$; `np.percentile` returns the same [C]. Six draws are far too few to trust; 100,000 give $[3.4, 9.2]$ [C]. With 1,000 draws the 2.5th percentile sits at position $999 \times 0.025 = 24.975$ (between the 25th and 26th smallest) and the 97.5th at 974.025; with 2,000 at 49.975 and 1,949.025.

The whole bootstrap distribution, exactly. Five items drawn five times with replacement give $5^5 = 5 \times 5 \times 5 \times 5 \times 5 = 3{,}125$ equally likely ordered draws, few enough to list them all. The book's script does, and Figure 36.1 plots how often each bootstrap mean occurs. Its standard deviation is 1.5232, exactly $\sigma/\sqrt{5}$, and its 2.5th and 97.5th percentiles are 3.4 and 9.2 [C]: the 100,000 random draws of worked example 36.2 had already found the exact answer. Real benches have far too many objects to enumerate, which is why the code draws at random, 1,000 or 2,000 times.

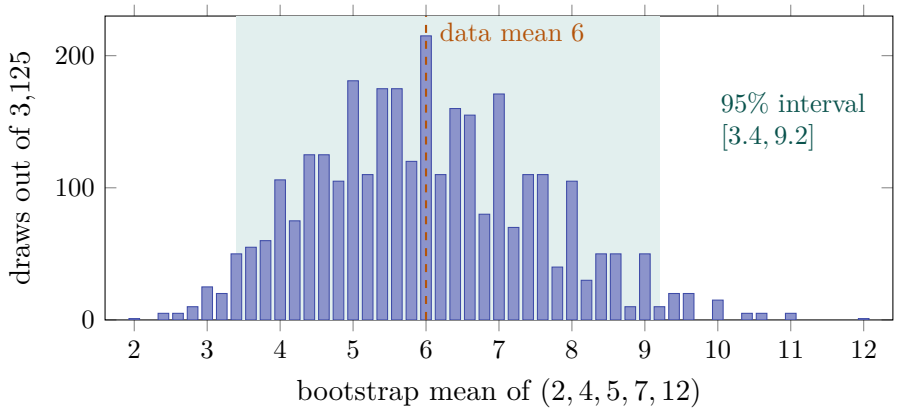


Figure 36.1: The exact bootstrap distribution of the mean of five numbers: every one of the $5^5 = 3{,}125$ draws with replacement, counted by the mean it gives (computed by `research/textbook/tools/compute_bootstrap.py`). The teal band is the percentile 95% interval. The distribution is lumpy because only 43 distinct means are possible, and skewed right because one value (12) is far from the rest.

36.5 Why resample objects, not pixels

pAUC averages over target pixels, but target pixels are not independent pieces of evidence. The pixels of one object rise and fall together: if the model reconstructs a rooftop well, all its pixels score low at once. Swap a dozen hard objects for a dozen easy ones and pAUC moves; swap a dozen pixels inside one object and it barely does.

How much dependence costs, derived. Model n objects of m pixels each. Each pixel's value (think of its contribution c below) has variance σ^2; two pixels of the same object have correlation ρ, so covariance $\rho\sigma^2$; pixels of different objects are independent. Apply rule 1 to one object's m pixels: m variance terms and $m(m-1)$ ordered pairs of distinct pixels, each a covariance:

$$\text{Var(object sum)} = m\sigma^2 + m(m-1)\rho\sigma^2 = m\sigma^2\,[1 + (m-1)\rho].$$

Objects are independent, so their sums' variances add, and the mean over all nm pixels divides by nm:

$$\text{Var(pixel mean)} = \frac{n \cdot m\sigma^2\,[1 + (m-1)\rho]}{(nm)^2} = \frac{\sigma^2}{nm}\,[1 + (m-1)\rho].$$

Compare rule 2 for N independent pixels, σ^2/N. The two agree when

$$N_{\text{eff}} = \frac{nm}{1 + (m-1)\rho}, \tag{36.1}$$

the *effective sample size*; the bracket $1 + (m-1)\rho$ is the *design effect*. At $\rho = 0$ every pixel counts; at $\rho = 1$, $N_{\text{eff}} = nm/m = n$: you have as much evidence as you have *objects*.

Worked example 36.3: An effective sample size

Suppose [A] 909 objects of 4 pixels each ($909 \times 4 = 3{,}636$ target pixels) with within-object correlation $\rho = 0.8$.

$$\text{design effect} = 1 + (4-1) \times 0.8 = 1 + 2.4 = 3.4$$
$$N_{\text{eff}} = 3{,}636/3.4 = 1{,}069.4$$
$$\text{standard error ratio} = \sqrt{3.4} = 1.84.$$

3,636 pixels carry the evidence of about 1,069 independent ones. A pixel bootstrap, which treats them as independent, would report a standard error 1.84 times too small.

The book's script checks this by simulation [A]: 300 objects of sizes 1–10 (967 pixels), each object's pixels sharing one value plus small noise. Resampling objects gave a standard error of 0.0235, resampling pixels 0.0098: the pixel bootstrap was 2.4 times too confident [C]. That is the reason in `significance.py`'s module docstring for drawing "each scene's objects with replacement".

More numbers is not more evidence: resample the unit that varies independently, here the object.

The background is held fixed. Each scene's background pixels are not resampled. The docstring's reason: "there are millions; their sampling error is small next to the objects"'. Only the target side of the measure is redrawn.

36.6 pAUC rebuilt from per-pixel contributions

To redraw objects you need pAUC as a sum over objects. The docstring of `app/evaluation/significance.py` gives the rearrangement.

Definition 36.3 (Per-pixel contribution)**.** For a target pixel p, let $\text{FPR}(p)$ be the share of the scene's background pixels scoring *at or above* it. With limit m (`max_fpr`, default 0.01), its *contribution* is

$$c(p) = \frac{\max(0,\ m - \text{FPR}(p))}{m}, \tag{36.2}$$

and the scene's pAUC is the mean of c over its P target pixels: $\text{pAUC}_m = \frac{1}{P}\sum_p c(p)$.

Read $c(p)$ as "the share of the first m of false alarms for which this pixel is caught". A target pixel above every background pixel has FPR $= 0$ and $c = 1$; one that beats the bottom 99.5% has FPR $= 0.005$ and is caught over the second half of the range, $c = 0.5$; one at or below the top 1% of background has $c = 0$.

Proof that the mean of c is the pAUC (no ties). In the ROC walk of Chapter 33, target p is passed when the false-alarm rate reaches $\text{FPR}(p)$, and from then on it counts in the TPR. So at false-alarm rate f,

$$\text{TPR}(f) = \frac{1}{P}\sum_p \mathbf{1}[\text{FPR}(p) \le f].$$

Put this into the definition of pAUC (Equation (33.4)):

$$\begin{aligned}
\mathrm{pAUC}_m &= \frac{1}{m}\int_0^m \frac{1}{P}\sum_p \mathbf{1}[\mathrm{FPR}(p) \le f]\, df && \text{substitute TPR} \\
&= \frac{1}{P}\sum_p \frac{1}{m}\int_0^m \mathbf{1}[\mathrm{FPR}(p) \le f]\, df && \text{swap the sum and the integral} \\
&= \frac{1}{P}\sum_p \frac{\max(0,\ m - \mathrm{FPR}(p))}{m} && \text{the indicator is 1 on } [\mathrm{FPR}(p), m] \\
&= \frac{1}{P}\sum_p c(p).
\end{aligned}$$

The third line: the indicator is 1 for f from $\mathrm{FPR}(p)$ up to m, an interval of length $m - \mathrm{FPR}(p)$, or of length 0 if $\mathrm{FPR}(p) \ge m$. With tied scores `exact_roc` draws a diagonal segment while $c(p)$ treats every pixel as its own step, so the two agree "up to ties"; with millions of distinct float scores the gap is negligible, and the README reports the bootstrap's point values reproducing the evaluator's (0.1621 and 0.1695).

So an object needs only two numbers: the *sum of c over its pixels* and its *pixel count*. `object_pauc_parts` returns them:

```
bg = np.sort(score[valid & (objects == 0)])                    # (N,) bg scores, ascending
t = valid & (objects > 0)                                       # (H, W) target pixels
fpr = 1.0 - np.searchsorted(bg, score[t], side="left") / max(len(bg), 1)   # share of bg >=
    score
c = np.maximum(0.0, max_fpr - fpr) / max_fpr                    # (P,) eq. c
ids = objects[t]
sums, counts = np.bincount(ids, weights=c), np.bincount(ids)    # per object: sum of c,
    count
```

`searchsorted(...,side="left")` counts the background scores strictly below each target score, so one minus that share is the share at or above it. A bootstrap draw adds up the drawn objects' pairs and divides; no score map is touched again, which is why thousands of draws are cheap.

Worked example 36.4: A tiny scene, both ways

One scene, 1,000 background pixels, three objects, four target pixels [A]:

obj 1, pixel a: 2 bg above,	$\mathrm{FPR} = 0.002$,	$c = (0.01 - 0.002)/0.01 = 0.8$
obj 1, pixel b: 6 bg above,	$\mathrm{FPR} = 0.006$,	$c = (0.01 - 0.006)/0.01 = 0.4$
obj 2, pixel c: 0 bg above,	$\mathrm{FPR} = 0.000$,	$c = 0.01/0.01 = 1.0$
obj 3, pixel d: 30 bg above,	$\mathrm{FPR} = 0.030$,	$c = \max(0, -0.02)/0.01 = 0$

Object parts: object 1 $(0.8 + 0.4, 2) = (1.2, 2)$; object 2 $(1.0, 1)$; object 3 $(0.0, 1)$.

$$\mathrm{pAUC} = \frac{1.2 + 1.0 + 0.0}{2 + 1 + 1} = \frac{2.2}{4} = 0.55.$$

Now the ROC area. Sort the target pixels by FPR: TPR is $\frac{1}{4}$ from FPR 0, $\frac{2}{4}$ from 0.002, $\frac{3}{4}$ from 0.006, and pixel d never arrives before 0.01:

$$\begin{aligned}
0 \text{ to } 0.002: &\quad 0.002 \times 0.25 = 0.0005 \\
0.002 \text{ to } 0.006: &\quad 0.004 \times 0.50 = 0.0020 \\
0.006 \text{ to } 0.010: &\quad 0.004 \times 0.75 = 0.0030 \\
\text{area} &= 0.0005 + 0.0020 + 0.0030 = 0.0055, \qquad 0.0055/0.01 = 0.55.
\end{aligned}$$

The same 0.55. The book's script builds this scene and gets 0.55 from `object_pauc_parts` and from `exact_roc` plus `pauc` [C].

36.7 The paired design

To compare scorers A and B, you want the interval of the *difference* $\text{pAUC}_B - \text{pAUC}_A$. You could bootstrap A and B separately, each with its own draws. That throws away the most useful fact: both scorers are judged on the same objects, and an object that is hard for A is usually hard for B.

The variance of a difference, derived. Apply rule 1 with Y replaced by $-B$ (and $\text{Var}(-B) = \text{Var}\, B$, $\text{Cov}(A, -B) = -\text{Cov}(A, B)$), or directly:

$$\begin{aligned}
\text{Var}(A - B) &= \mathbb{E}[((A - B) - (\mu_A - \mu_B))^2] && \text{definition}\\
&= \mathbb{E}[((A - \mu_A) - (B - \mu_B))^2] && \text{regroup}\\
&= \mathbb{E}[(A - \mu_A)^2] - 2\,\mathbb{E}[(A - \mu_A)(B - \mu_B)] + \mathbb{E}[(B - \mu_B)^2] && \text{expand the square}\\
&= \text{Var}\, A + \text{Var}\, B - 2\,\text{Cov}(A, B).
\end{aligned}$$

When A and B rise and fall together, the covariance is large and positive, and it cancels most of the variance. Drawing independently sets that covariance to zero by construction, and you get $\text{Var}\, A + \text{Var}\, B$.

Worked example 36.5: Five objects, paired and unpaired

Per-object scores [A] for two scorers on the same five objects: $A = (0.9, 0.2, 0.6, 0.1, 0.7)$, $B = (0.95, 0.25, 0.6, 0.15, 0.75)$.

$$\begin{aligned}
\mu_A &= (0.9 + 0.2 + 0.6 + 0.1 + 0.7)/5 = 2.5/5 = 0.50\\
A - \mu_A &= (0.40, -0.30, 0.10, -0.40, 0.20)\\
\text{Var}\, A &= (0.16 + 0.09 + 0.01 + 0.16 + 0.04)/5 = 0.46/5 = 0.0920\\
\mu_B &= (0.95 + 0.25 + 0.6 + 0.15 + 0.75)/5 = 2.7/5 = 0.54\\
B - \mu_B &= (0.41, -0.29, 0.06, -0.39, 0.21)\\
\text{Var}\, B &= (0.1681 + 0.0841 + 0.0036 + 0.1521 + 0.0441)/5 = 0.452/5 = 0.0904\\
\text{Cov}(A, B) &= (0.164 + 0.087 + 0.006 + 0.156 + 0.042)/5 = 0.455/5 = 0.0910\\
\text{Var}(A - B) &= 0.0920 + 0.0904 - 2 \times 0.0910 = 0.1824 - 0.1820 = 0.0004.
\end{aligned}$$

Check directly: $A - B = (-0.05, -0.05, 0, -0.05, -0.05)$, mean -0.04, deviations $(-0.01, -0.01, 0.04, -0.01, -0.01)$, variance $(4 \times 0.0001 + 0.0016)/5 = 0.002/5 = 0.0004$. Agreed. Unpaired, the variance would be $0.0920 + 0.0904 = 0.1824$, 456 times larger [C]: B's consistent $+0.05$ edge would vanish in the noise of which objects were drawn.

`paired_bootstrap` therefore draws one set of object positions per scene and applies it to both scorers.

Worked example 36.6: Two paired draws on the tiny scene

Give scorer B its own parts on the same objects of worked example 36.4: object 1 $(1.0, 2)$, object 2 $(0.9, 1)$, object 3 $(0.2, 1)$. Point values: $A = 2.2/4 = 0.550$, $B = (1.0+0.9+0.2)/4 = 2.1/4 = 0.525$, $B - A = -0.025$.

Draw objects 1, 1, 3 (with replacement), the same for both:

$$A = (1.2 + 1.2 + 0.0)/(2 + 2 + 1) = 2.4/5 = 0.48$$
$$B = (1.0 + 1.0 + 0.2)/(2 + 2 + 1) = 2.2/5 = 0.44, \qquad B - A = -0.04.$$

Draw objects 2, 2, 2:

$$A = (1.0 + 1.0 + 1.0)/3 = 1.00, \qquad B = (0.9 + 0.9 + 0.9)/3 = 0.90, \qquad B - A = -0.10.$$

A swung from 0.48 to 1.00 between the two draws; $B - A$ moved only from -0.04 to -0.10. The draw moves both scorers together, and the difference cancels most of it.
With only three objects the interval is still wide: `paired_bootstrap` with 1,000 draws on this toy gives $[-0.10, +0.20]$ and $P(B > A) = 0.28$ [C]. (A draw of object 3 three times gives $B - A = 0.2 - 0 = +0.2$.) Few objects, wide interval: that is the whole problem in miniature.

Scenes and the two measures. On the real bench each scene's objects are drawn separately, with `rng.integers(len(ids[s]),size=len(ids[s]))` per scene; each scene's pAUC is computed from its drawn parts, and the four are averaged - the same "mean over scenes" as the evaluator. If per-object hits are given (1 if caught at a cut-off, from the run's `per_object.csv`, column `hit_0.1` by default), objects caught is resampled too, but *pooled* across scenes: all caught drawn objects over all drawn objects. The function also returns $P(B > A)$, the share of draws in which B's pAUC beat A's.

36.8 What it does not cover: only four scenes

The bootstrap redraws objects *inside* each scene and never redraws the scenes. A model can be lucky in the scenes too. Why not bootstrap scenes as well? Count the distinct ways to draw 4 scenes from 4 with replacement, order not mattering. That is choosing 4 items from 4 kinds with repetition, $\binom{4+4-1}{4}$:

$$\binom{7}{4} = \frac{7 \times 6 \times 5 \times 4}{4 \times 3 \times 2 \times 1} = \frac{840}{24} = 35.$$

Thirty-five possible draws, one of which is "scene 2 four times": too few and too lumpy for an interval worth trusting. The README says it plainly: "Scene-level uncertainty (4 scenes) is not covered and is larger: this interval is a lower bound on the real uncertainty." Nor does it cover checkpoint-to-checkpoint variation: FT09c, five more epochs of the same run as FT09b, moved by as much as FT09b's resolved gain (Table 36.1).

36.9 The rule, and the intervals so far

A gain counts only if its paired-bootstrap 95% interval clears zero (rule adopted 2026-10-21).

Run it on a finished evaluation:

```
python scripts/compare_scorers.py configs/eval/scene_adapt.json "FT04_ep10|RM, cloud excluded"
    \
   "adapt50|RM, cloud excluded" [--top 0.1] [--boot 2000]
```

Each side is `<model name>|<scorer label>` as in the run's `per_scene.csv` (`-` as the model name for a model-free scorer). It reads the cached score maps and `per_object.csv` of the *finished* run, only for `prisma_bench` scenes, recomputes nothing, and prints both point values, $B - A$, the 95% interval and $P(B > A)$.

Source check: how many draws

The README states its results used 1,000 draws. The code's defaults are 2,000: `paired_bootstrap(...,n=2000,seed=0)` in `app/evaluation/significance.py` (line 32) and `--boot 2000` in `scripts/compare_scorers.py` (line 37). So the README's runs passed `--boot 1000`; re-running with the defaults will give slightly different interval ends. The generator is seeded (`seed=0`), so a given n is reproducible.

Values that will drift

Paired object bootstrap on the real PRISMA bench, pAUC $\leq 1\%$, $B - A$ [95% interval]. Rows 1–3 from `research/experiments/bench-resolution/README.md` (2026-10-21, 1,000 draws); rows 4–6 from the run table of `docs/lld/evaluation.md` (2026-10-22/23). FT04c is FT04 with cloud excluded; gated adaptation is $N = 50$.

	A → B	pAUC $B - A$ [95%]	caught 0.1%, pts [95%]
1	FT04 → + cloud excl.	+0.0074 [+0.0047, +0.0101]	+1.0 [+0.4, +1.7]
2	FT04c → gated adapt.	−0.0091 [−0.0176, −0.0028]	−0.7 [−1.7, +0.2]
3	FT04c → FT06 $/\sqrt{b}$	−0.0067 [−0.0174, +0.0037]	+0.8 [−0.4, +2.0]
4	FT04c → FT09b	+0.0060 [+0.0002, +0.0118]	unresolved
5	FT04c → FT09c	+0.0020 [−0.0026, +0.0064]	-
6	FT09b → FT09c	−0.0040 [−0.0076, −0.0010]	-

Table 36.1: Paired-bootstrap intervals on the real PRISMA bench.

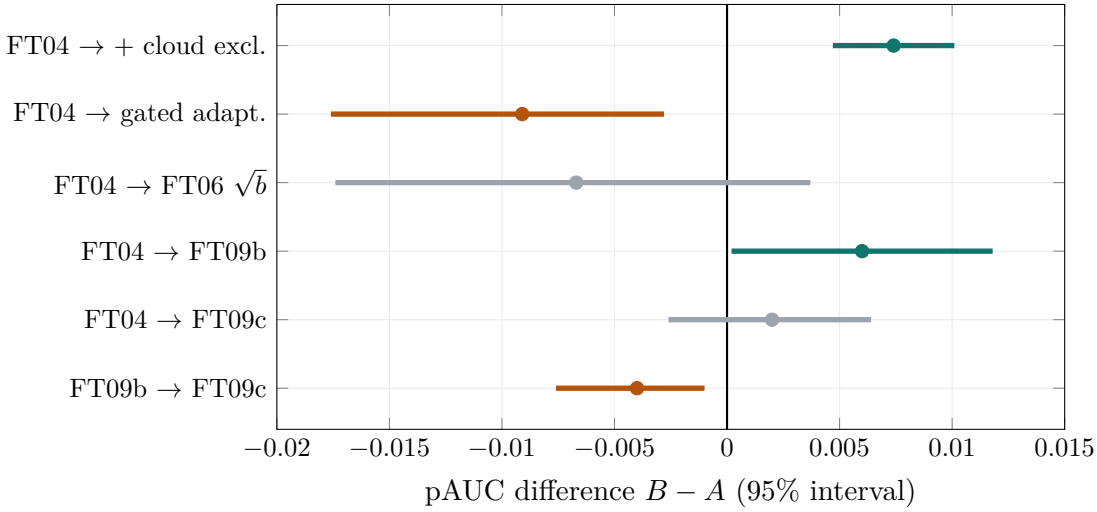


Figure 36.2: The paired-bootstrap intervals of Table 36.1, drawn to scale: point value (dot) and 95% interval (bar) of the pAUC difference $B - A$. Teal: interval wholly above zero (a resolved gain); amber: wholly below zero (a resolved loss); grey: crosses zero (a tie). FT09b's interval clears zero by 0.0002.

Reading the rows.

- **Cloud exclusion is a real gain** on both measures (row 1): the one change adopted since FT04 at the time. Its interval is the narrowest, half-width $(0.0101 - 0.0047)/2 = 0.0027$.

A likely reason, from the method rather than the README: excluding cloud leaves most objects' scores untouched, so most per-object differences are exactly zero and the paired difference barely wobbles - worked example 36.5's covariance at its largest.

- **Gated per-scene adaptation's loss is real** on pAUC (row 2), though its objects-caught interval straddles zero. Read each measure's interval on its own.
- **FT06's square-root variant is a tie** (row 3): both of its intervals cross zero, whatever the point values say.
- **FT09b was the first resolved gain since cloud exclusion** (row 4), by the narrowest of margins; and FT09c, five epochs later, is a tie with FT04 and a resolved loss against FT09b (rows 5, 6). The object bootstrap does not cover that checkpoint-to-checkpoint variation.

The README summarises the bench's resolution as interval half-widths of about ± 0.005 to ± 0.01 pAUC and ± 1 to ± 1.5 points of objects caught at the top 0.1%. So an old pass criterion of "+0.005 pAUC" sits at the edge of what this bench can resolve: a run that clears it on its point value may still be a tie.

36.10 Common confusions

The interval says there is a 95% chance the true gain lies inside it?
It says how far the difference wobbles when the *objects* are redrawn, scenes and checkpoints held fixed. Scene luck is extra, so the real uncertainty is wider: treat the interval as a lower bound.

Resampling pixels would be more precise, since there are far more of them?
More numbers, not more evidence. An object's pixels move together; counting them separately manufactures confidence (a design effect of 3.4 makes the standard error 1.84 times too small).

Why not bootstrap A and B separately and see if their intervals overlap?
Because the draw moves both together. Separate intervals would overlap almost always, and you would call real differences ties (worked example 36.5: variance 0.1824 unpaired against 0.0004 paired).

pAUC from $c(p)$ is an approximation of the real thing?
It is the same area, rearranged so each target pixel's share can be added up. It differs from the evaluator only at tied scores.

Objects caught is averaged per scene, like pAUC?
Not in the bootstrap: pAUC is a mean over the four scenes; objects caught is pooled over all drawn objects.

The gated adaptation loss straddles zero, so it is a tie?
On objects caught, yes. On pAUC its interval $[-0.0176, -0.0028]$ is wholly below zero, so the pAUC loss is real.

36.11 Check yourself

Exercise 36.1. In worked example 36.4, pixel d's score rises until 5 background pixels score at or above it. What is its c, and the scene's pAUC for A?

Exercise 36.2. Bootstrap the mean of $(1, 3, 8)$ by hand with the four given draws (positions) $(1, 1, 2)$, $(3, 2, 3)$, $(2, 2, 1)$, $(3, 3, 3)$. Give the four means, their mean, and the percentile interval `np.percentile` would return for $[2.5, 97.5]$.

Exercise 36.3. A result reads $+0.006$ $[-0.001, +0.013]$. Does it count as a gain under the rule? What is its half-width?

Exercise 36.4. Two scorers on four objects: $A = (0.8, 0.3, 0.5, 0.4)$, $B = (0.9, 0.3, 0.6, 0.6)$. Compute $\operatorname{Var} A$, $\operatorname{Var} B$, $\operatorname{Cov}(A, B)$ and $\operatorname{Var}(A - B)$ by the formula, and check $\operatorname{Var}(A - B)$ directly.

Exercise 36.5. 909 objects average 2 pixels each with within-object correlation 0.9. What are the design effect and the effective sample size? By what factor would a pixel bootstrap understate the standard error?

Exercise 36.6. In the paired example, draw objects 1, 2, 3 (each once). Compute A, B and $B - A$. Why is this the point value?

Exercise 36.7. With 2,000 bootstrap draws, at which 0-based positions do the 2.5th and 97.5th percentiles sit, and between which sorted draws (counting from 1) do they interpolate?

Next: Chapter 37, The sorting bug that rewrote the scoreboard.

Part VIII

The experiments, told as lessons

Chapter 37

The sorting bug that rewrote the scoreboard

The one thing this chapter teaches

`np.interp` assumes that its sample wavelengths go up and never checks; PRISMA lists its bands going down. For the implant benchmark's whole early life (2026-10-17 to 18) the evaluator therefore turned every PRISMA cube it resampled into copies of one or two bands, and flattened every transplanted signature - and the result everyone believed, "pretraining is the big step", was the bug. It was found only when one model was read two ways on the same real scenes and the two readings were compared.

This chapter opens the last part of the book, in which the experiments of the project are told as lessons. Each lesson has the same shape: what was believed, what was tried, what was measured, what it showed, and the mathematics behind it. The first lesson is the most uncomfortable one, because it is about a result that looked right, matched what everyone expected, and was wrong for a reason that fits on one line of code.

37.1 What was believed

You met the implant benchmark in Chapter 34: real held-out scenes, called *hosts*, into which synthetic objects are written with the size, contrast and spectral oddness of the 909 real labelled PRISMA objects. It was built because the real PRISMA bench (four scenes, 909 objects) could not separate the models: under residual Mahalanobis (Chapter 31) every fine-tuned model tied the shipped v0.2.0.

On 2026-10-17 the first version of the implant benchmark, bench-v1 (3,336 objects on 18 hosts [M: `research/experiments/implant-bench/README.md`]), seemed to give the answer the real bench could not. Figure 37.1 shows what it said. On the easy tier the pAUC (the area under the ROC curve up to 1% false alarms, divided by 0.01; Chapter 33) went from 0.257 for the shipped v0.2.0 to 0.342 for F001, the model pretrained on SpectralEarth, and then crept up to 0.371 through the fine-tuning runs [M: `research/experiments/indradhanu-finetune/README.md`]. The real bench wandered between 0.148 and 0.159 in no order. The reading written down that day was: *the largest step is pretraining.*

The second version, bench-v2, built later the same day to fix bench-v1's biases, agreed: v0.2.0 caught 22.1% of objects at the top 1% of scores, F001 29.2%, FT02c 32.8% [M: `research/experiments/implant-bench/README.md`]. Two benchmarks, built differently, telling the same story. And yet one fact did not fit. On the real bench v0.2.0 tied FT03 (pAUC 0.159 against 0.158), while on bench-v2 it was ten points behind (22.1% against 32.3%). Chapter 34 left two expla-

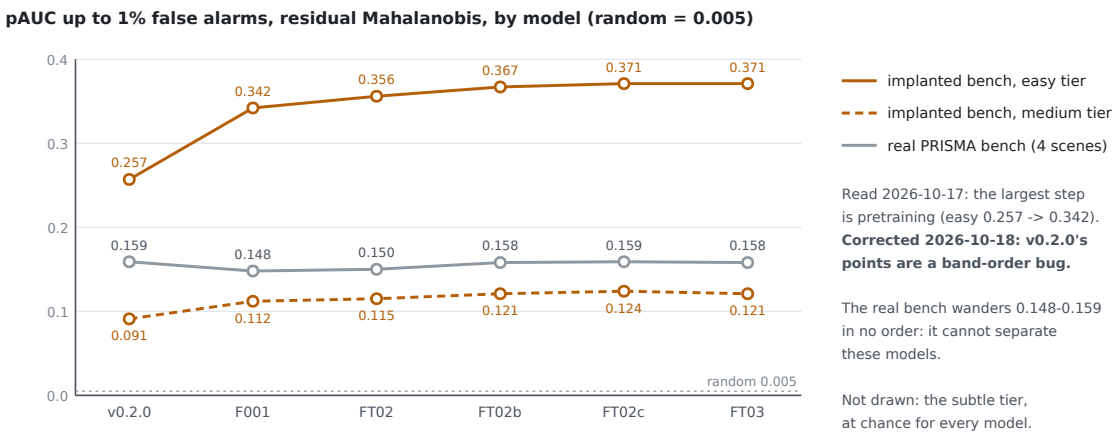


Figure 37.1: pAUC by training stage, as read on 2026-10-17: the implanted bench (amber, easy and medium tiers) rises from v0.2.0 to FT02c, the real bench (grey) wanders with no order. The v0.2.0 points are the bug this chapter dissects.

nations open: "a reading artefact" and "training overlap". On 2026-10-18 the reading artefact was confirmed, found to be a two-line bug, and fixed (commit `b94dbfd`).

Analogy

A bus timetable is printed with the last bus of the day at the top and the first at the bottom. A passenger who assumes the usual order checks one thing first: "is it later than the bottom row?" The bottom row is the 05:00 bus, so at 09:00, at noon and at 18:00 the answer is always yes - and she is told, every time, "you have missed the last bus". No single answer looks absurd. Only asking at several different times, and getting the same answer each time, gives it away. Keep this passenger in mind: in Section 37.3 you will see `np.interp` behave exactly like her.

37.2 Linear interpolation, derived

A sensor measures reflectance at a list of wavelengths, its *bands*. To compare sensors, or to move a spectrum from one sensor's bands to another's, you need values at wavelengths that were not measured. *Linear interpolation* estimates the value between two neighbouring measurements by the straight line through them.

Definition 37.1 (Linear interpolation)**.** Given two samples (x_0, y_0) and (x_1, y_1) with $x_0 < x_1$, the linearly interpolated value at a query point x with $x_0 \le x \le x_1$ is the height at x of the straight line through the two samples.

Derive the formula. A straight line has a constant slope, so the slope from (x_0, y_0) to (x, y) must equal the slope from (x_0, y_0) to (x_1, y_1):

$$\frac{y - y_0}{x - x_0} = \frac{y_1 - y_0}{x_1 - x_0}. \tag{37.1}$$

Multiply both sides by $(x - x_0)$ to isolate $y - y_0$:

$$y - y_0 = \frac{x - x_0}{x_1 - x_0}\,(y_1 - y_0). \tag{37.2}$$

Name the fraction t, the *fraction of the way* from x_0 to x_1, and add y_0 to both sides:

$$y = y_0 + t\,(y_1 - y_0), \qquad t = \frac{x - x_0}{x_1 - x_0}. \tag{37.3}$$

Expand the bracket and collect y_0 to see y as a weighted average of the two samples:

$$y = (1 - t)\,y_0 + t\,y_1. \tag{37.4}$$

Since $0 \le t \le 1$ inside the interval, the two weights $1 - t$ and t are non-negative and add to 1. Outside the sampled range a rule is needed; the usual one, and NumPy's default, is to hold the end value: below the smallest sample return its value, above the largest return its value.

Worked example 37.1: Interpolating the repo's own test spectrum

The repository's test (`tests/test_evaluation/test_implant.py`, `test_descending_band_lists`) uses samples at 500, 600 and 700 nm with values 1, 2 and 4. Do these on paper.
At 550 nm the neighbours are $(500, 1)$ and $(600, 2)$:

$$\begin{aligned} t &= \frac{550 - 500}{600 - 500} = \frac{50}{100} = 0.5 \\ y &= 1 + 0.5 \times (2 - 1) \\ &= 1 + 0.5 \\ &= 1.5 \end{aligned}$$

At 650 nm the neighbours are $(600, 2)$ and $(700, 4)$:

$$\begin{aligned} t &= \frac{650 - 600}{700 - 600} = \frac{50}{100} = 0.5 \\ y &= 2 + 0.5 \times (4 - 2) \\ &= 2 + 1.0 \\ &= 3.0 \end{aligned}$$

At 450 nm, below the smallest sample, the end value is held: $y = 1$. At 750 nm, above the largest: $y = 4$.

37.3 The contract of `np.interp`

NumPy's `np.interp(x,xp,fp)` takes the query points `x`, the sample positions `xp` (here wavelengths) and the sample values `fp` (here reflectances). Every function has a *contract*: what it promises to return, provided its inputs satisfy stated conditions. The conditions are called *preconditions*. If a precondition is violated, the contract promises nothing.

Definition 37.2 (Precondition of `np.interp`)**.** The sample positions `xp` must be increasing. In the words of NumPy's own docstring (section "Warnings", version 2.5.2): "The x-coordinate sequence is expected to be increasing, but this is not explicitly enforced. However, if the sequence `xp` is non-increasing, interpolation results are meaningless."

Read that carefully. Not an error. Not a warning at run time. *Meaningless* numbers, returned quietly. Behaviour on decreasing or unsorted `xp` is *undefined by contract*: a future NumPy may return different garbage, and no particular garbage can be relied on. What follows is what NumPy 2.5.2 on the dev laptop actually returned - an observation, not a promise.

Worked example 37.2: The same three samples, listed the other way round

Give `np.interp` the identical samples in descending order: `xp=(700,600,500)`, `fp=(4,2,1)`. Same physics, same pairs. The values below were produced by `research/textbook/tools/compute_sortbug.py` on NumPy 2.5.2 [C].

query (nm)	correct (worked example 37.1)	`np.interp`, descending `xp`
450	1 (held)	4
500	1	4
550	1.5	**1**
600	2	**1**
650	3.0	**1**
700	4	**1**
750	4 (held)	1

Every query above 500 nm got 1, and every query at or below 500 nm got 4. Only two values ever come out.

Why only two? The observed table is consistent with an implementation that, for each query x, asks two questions before doing any interpolation, in this order:

1. Is x beyond the *last* listed sample, $x >$ `xp`$[-1]$? If so, return the last listed value `fp`$[-1]$.
2. Otherwise, is x before the *first* listed sample, $x <$ `xp`$[0]$? If so, return the first listed value `fp`$[0]$.
3. Otherwise, search for the interval that contains x (assuming ascending order) and interpolate.

On an ascending list these checks are the "hold the end value" rule. On a descending list they are the passenger reading the timetable from the bottom.

On a strictly descending list, under the check order above, step 3 is never reached: every query is answered by one of the two edge checks, so the output is a step function with only two values.

Prove the key point in two lines. Let the list be descending, so its last entry is its smallest and its first is its largest: `xp`$[-1] <$ `xp`$[0]$. Take any query x.

- If $x >$ `xp`$[-1]$, check 1 fires and returns `fp`$[-1]$.
- If not, then $x \le$ `xp`$[-1]$. Chain the inequality with `xp`$[-1] <$ `xp`$[0]$ to get $x <$ `xp`$[0]$, so check 2 fires and returns `fp`$[0]$.

Every x falls in one of the two cases, so step 3 never runs. With `xp` $= (700, 600, 500)$: every $x > 500$ gets `fp`$[-1] = 1$ and every $x \le 500$ gets `fp`$[0] = 4$ - exactly the table. The proof also covers a list made of several descending runs, as long as its last entry is smaller than its first; you will need that for the PRISMA hosts in Section 37.5.

37.4 Resampling a cube is a matrix

The evaluator's `resample` in `app/evaluation/datasets.py` moves a whole cube of shape `(C, H, W)` (C bands, H rows, W columns) onto a fixed grid of G wavelengths. It does not call `np.interp` once per pixel. It builds one *weight matrix* $\mathbf{M}$ of shape `(G, C)` and multiplies every pixel's spectrum by it. Why is that allowed?

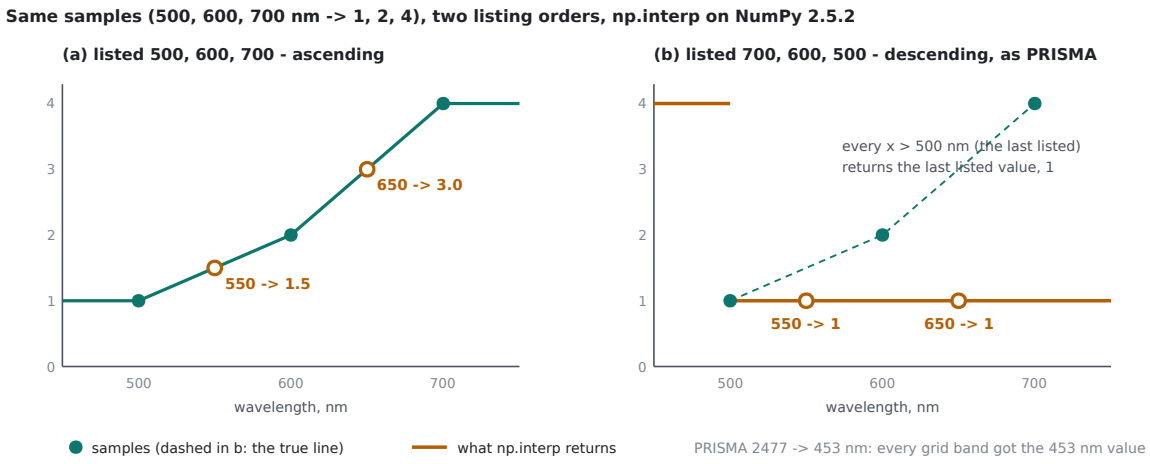


Figure 37.2: The same three samples (500, 600, 700 nm; values 1, 2, 4). (a) Listed ascending, `np.interp` draws the line: 1.5 at 550 nm and 3.0 at 650 nm. (b) Listed descending, it returns the last listed value, 1, for every wavelength above 500 nm: the spectrum goes flat.

Interpolation is linear in the values. Fix the sample positions and the query point. Equation (37.4) says the output is $(1-t)\,y_0 + t\,y_1$ - a weighted sum of the sample values, with weights that depend only on positions. So if you interpolate two value lists $\mathbf{f}$ and $\mathbf{g}$ and add the results, you get the same as interpolating $\mathbf{f}+\mathbf{g}$; and scaling the values by a number scales the output by that number. A map with those two properties is *linear*, and every linear map from C numbers to G numbers is a $G \times C$ matrix.

Finding the matrix column by column. Write a pixel's spectrum as a sum of *unit vectors*: $\mathbf{e}_b$ is the list with 1 in position b and 0 elsewhere, so a spectrum $\mathbf{y} = (y_1, \ldots, y_C)$ equals $\sum_b y_b\,\mathbf{e}_b$. By linearity,

$$\text{interp}(\mathbf{y}) = \text{interp}\left(\sum_b y_b\,\mathbf{e}_b\right) = \sum_b y_b\ \text{interp}(\mathbf{e}_b). \tag{37.5}$$

The right-hand side is a matrix-vector product whose b-th column is $\text{interp}(\mathbf{e}_b)$:

$$\text{interp}(\mathbf{y}) = \mathbf{M}\mathbf{y}, \qquad \mathbf{M}_{:,b} = \text{interp}(\text{grid};\ \texttt{wl},\ \mathbf{e}_b). \tag{37.6}$$

That is the code, line for line:

```
# app/evaluation/datasets.py, before b94dbfd
# M: (G, C); column i is the interpolation of unit vector e_i
M = np.stack([np.interp(grid, wl, np.eye(len(wl))[i]) for i in range(len(wl))], 1)
return np.einsum("gb,bhw->ghw", M, cube, optimize=True)   # (C, H, W) -> (G, H, W)
```

`np.eye(len(wl))[i]` is $\mathbf{e}_i$; `np.stack(...,1)` puts the results side by side as columns; `einsum("gb,bhw->ghw")` multiplies every pixel's C-vector by $\mathbf{M}$.

Worked example 37.3: One row of the resample matrix, right and wrong

Three scene bands listed 700, 600, 500 nm (descending, as PRISMA lists them); one grid wavelength, 550 nm. The pixel reads $(4, 2, 1)$ in that listed order.
The correct row puts weight on the two bands that bracket 550 nm. From worked example 37.1, $t = 0.5$ of the way from 500 to 600, so 600 nm gets $t = 0.5$ and 500 nm gets $1 - t = 0.5$:

band as listed	700	600	500
correct row for 550 nm	0.0	0.5	0.5
unsorted row for 550 nm	0.0	0.0	1.0
pixel value	4	2	1

Where does the unsorted row come from? Column b is interp(550; (700, 600, 500), $\mathbf{e}_b$). By the key point of Section 37.3, $550 > 500$ (the last listed), so each call returns the last entry of $\mathbf{e}_b$: 0 for $b = 700$, 0 for $b = 600$, 1 for $b = 500$.
Apply each row to the pixel:

$$\begin{aligned} \text{correct:} \quad & 0 \times 4 + 0.5 \times 2 + 0.5 \times 1 \\ & = 0 + 1.0 + 0.5 \\ & = 1.5 \\ \text{unsorted:} \quad & 0 \times 4 + 0 \times 2 + 1 \times 1 \\ & = 1 \end{aligned}$$

The fix: sort the pairs, keep them together. The fix sorts the wavelengths and builds the columns on the sorted list, then writes each column back to the position of the band it belongs to:

```
# app/evaluation/datasets.py, since b94dbfd
order = np.argsort(wl)                  # (C,) band positions in ascending order
M = np.zeros((len(grid), len(wl)))      # (G, C)
M[:, order] = np.stack([np.interp(grid, wl[order], np.eye(len(wl))[i])
                        for i in range(len(wl))], 1)
```

`np.argsort(wl)` returns the *permutation* that sorts `wl`: the list of positions such that `wl[order]` is ascending. Sorting does not change the data. Each wavelength keeps its own value; only the order in which the pairs are listed changes. A function sampled at a set of points is the same function however you list the points - the bug was never in the data, only in a precondition left unmet.

Worked example 37.4: The sorted resample, by hand

Bands listed `wl` = (700, 600, 500); grid (550, 650); pixel (4, 2, 1) in listed order.

1. Sort. The smallest wavelength, 500, sits at position 2; then 600 at position 1; then 700 at position 0. So `order` = (2, 1, 0) and `wl[order]` = (500, 600, 700), ascending.
2. Build columns on the sorted list. Sorted column 0 (500 nm): weight 0.5 at 550, 0 at 650. Sorted column 1 (600 nm): 0.5 at 550, 0.5 at 650. Sorted column 2 (700 nm): 0 at 550, 0.5 at 650.
3. Write back: sorted column i goes to original column `order`$[i]$. Sorted 0 → original 2, sorted 1 → original 1, sorted 2 → original 0:
$$\mathbf{M} = \begin{pmatrix} 0 & 0.5 & 0.5 \\ 0.5 & 0.5 & 0 \end{pmatrix} \quad \text{(columns: 700, 600, 500 nm).}$$
4. Multiply. Row 1: $0 \times 4 + 0.5 \times 2 + 0.5 \times 1 = 0 + 1 + 0.5 = 1.5$. Row 2: $0.5 \times 4 + 0.5 \times 2 + 0 \times 1 = 2 + 1 + 0 = 3.0$.

The old code gives $\mathbf{M} = \left(\begin{smallmatrix} 0 & 0 & 1 \\ 0 & 0 & 1 \end{smallmatrix}\right)$ and the output $(1, 1)$. Both matrices were confirmed by `research/textbook/tools/compute_sortbug.py` [C].

37.5 What it did to real PRISMA band lists

PRISMA has two detectors, a visible and near-infrared (VNIR) one and a short-wave infrared (SWIR) one, and its files list the bands of each detector from long to short wavelength. Two real lists met the bug.

The bench objects' list. The 185 bands on which the real PRISMA bench objects were measured (`C:/es/implant/real_v2.json`, field `wavelengths_nm`) start at 2477.05 nm and end at 453.39 nm [M: `C:/es/implant/real_v2.json`]. The 165-band common grid of Chapter 3 runs from 460 to 2450 nm in 10 nm steps. Every grid wavelength is longer than 453.39 nm, the last band listed. By the key point, every row of the old $\mathbf{M}$ has a single 1, in column 184 - the 453 nm band. Running the old line on the real list confirms it: only column 184 is ever used [C].

> The fixed-grid model was handed 165 copies of the same blue image: no spectrum to reconstruct, only one band's spatial texture.

The implant hosts' lists. A PRISMA host's `wavelengths.npy` (for example `C:/es/implant/bench-v2.1/prisma/PRS_L2D_STD_20230311055340/`) holds 234 bands in two descending runs: the VNIR detector from 977.37 down to 406.99 nm, then the SWIR detector from 2497.12 down to 943.36 nm [M: `C:/es/implant/bench-v2.1/prisma/PRS_L2D_STD_20230311055340/wavelengths.npy`]. The list is not descending overall, but its last entry (943.36) is smaller than its first (977.37), which is all the proof in Section 37.3 needs:

$$\begin{aligned} &\text{grid wavelength} > 943.36\,\text{nm} \;\rightarrow\; \text{the 943 nm band (last listed)},\\ &\text{grid wavelength} \le 943.36\,\text{nm} \;\rightarrow\; \text{the 977 nm band (first listed)}. \end{aligned}$$

Count them on the common grid. The grid skips the water-vapour window from 912 to 978 nm (`build_common_wavelength_grid` in `app/models/dataset/vendables.py`), so its wavelengths at or below 943.36 nm are 460, 470, ..., 910:

$$\frac{910-460}{10}+1=\frac{450}{10}+1=45+1=46 \text{ grid bands.}$$

So 46 grid bands became copies of the 977 nm image and the other $165-46=119$ copies of the 943 nm image (the script agrees: 46, and only columns 0 and 233 are used [C]). Every grid band was one of two near-infrared pictures.

Source check: which models the reading bug touched

Only fixed-grid models go through `resample`: the shipped v0.2.0 reads the 165-band grid. The wavelength-reading models (F001 and every FT run) read the host's own bands directly and never touched it. And only descending lists trip it: the EnMAP and AVIRIS-NG hosts were read correctly. The real PRISMA bench was also safe: for fixed-grid models the evaluator reloads its scenes through the product's `band_filter_apply`, whose resampler (`app/utils/data_transformations/spectral_resampler.py`, line 70) begins `# Sort source wavelengths ascending`. The product always sorted; the evaluator reimplemented a faster resample and left the sort out.

37.6 What it did to the signature implants

A *signature* implant (Chapter 34) takes a real object's difference spectrum $\mathbf{d}=\mathbf{o}-\mathbf{r}$ - the object's mean spectrum minus the mean of a ring of pixels around it - measured on the 185

bench bands, and carries it to the host's bands by interpolation. Before `b94dbfd` that was a bare `np.interp(wl,rw,d)`, in the builder's `place_and_implant` and in `implant_tile` alike, with `rw` the descending bench list. Same mechanism, same damage.

Worked example 37.5: A signature flattened

A difference spectrum listed descending: $+0.06$ at 700 nm, $+0.03$ at 600 nm, -0.02 at 500 nm. The host has bands at 520, 580 and 650 nm. Do it on paper.
Correct, by (37.3):

$$\begin{aligned}
520\,\text{nm}: \quad & t = \tfrac{520-500}{600-500} = 0.2, && y = -0.02 + 0.2 \times (0.03 - (-0.02)) \\
& && \quad = -0.02 + 0.01 = -0.01 \\
580\,\text{nm}: \quad & t = \tfrac{580-500}{600-500} = 0.8, && y = -0.02 + 0.8 \times 0.05 \\
& && \quad = -0.02 + 0.04 = 0.02 \\
650\,\text{nm}: \quad & t = \tfrac{650-600}{700-600} = 0.5, && y = 0.03 + 0.5 \times (0.06 - 0.03) \\
& && \quad = 0.03 + 0.015 = 0.045
\end{aligned}$$

Unsorted: all three host bands are above 500 nm (the last listed), so each gets the last listed value, -0.02. The real shape - dark in the blue, bright towards the red - arrives as a flat step $(-0.02, -0.02, -0.02)$. Both results match the script [C].

For the real bench list, every host band longer than 453.39 nm received **d** at 453 nm, one number; the few host bands at or below 453 nm received **d** at 2477 nm. bench-v2's later steps (`match_foreignness` and the strength solve of Chapter 34) then rescaled that step, so the garbled implants still hit their target contrast and foreignness - but not the shape of any real object.

The signature bug hit every signature implant in bench-v1 and bench-v2, for every model, and half of the implants FT05 trained on (Chapter 38).

The fix is a three-line function:

```
# app/evaluation/implant.py
def interp_spectrum(wl, src_wl, values):
    src_wl, values = np.asarray(src_wl, float), np.asarray(values, float)
    order = np.argsort(src_wl)                      # ascending, pairs kept together
    return np.interp(wl, src_wl[order], values[order])
```

37.7 How it hid in plain sight

Five things kept the bug invisible for two days. Each is a habit worth breaking.

1. **No error.** `np.interp` returned numbers of the right shape and range. Nothing crashed, nothing was NaN. A copy of a real band is a perfectly plausible-looking image.
2. **A warning that sounded like an excuse.** The resample's docstring said implant hosts are read "close to, not identical with" the product. A difference was expected, so a difference was not alarming.
3. **A story that fitted.** On bench-v1 the models lined up in training order with v0.2.0 at the bottom, and "pretraining helps" is what everyone expected. The Indradhanu course even noted that the model and the band reading changed together between v0.2.0 and F001 - and read the step as the model's.

4. **A cross-check that was run and misread.** The implant-bench README now says: "An earlier note here that the two readings were identical was a misread." The saved check said 15.8% against 44.2%.

5. **The tests used the wrong orderings.** Every test of the new resample used ascending lists, where sorted and unsorted code agree, until `test_descending_band_lists` was added with the fix. It feeds the same cube with the wavelengths reversed (`up[::-1]`) and asserts the result equals the ascending one, and that `interp_spectrum` returns 1.5 and 3.0 at 550 and 650 nm from a descending list.

37.8 The cross-check that found it

For a fixed-grid model the real PRISMA bench can be read two ways: through the product's `band_filter_apply` path (what the evaluator does for PRISMA bench scenes), or through the generic `resample` (what implant hosts get). If the evaluator is right, the two must agree on the same scenes. A one-off script, `C:/es/eval/resample_check.py`, swapped the reader and scored v0.2.0 (Table 37.1).

Table 37.1: v0.2.0 on the real PRISMA bench (909 objects), residual Mahalanobis, read three ways.

reading path	caught top 1%	top 0.1%	pAUC to 1%	ROC-AUC	TPR at 0.1%
product path	44.2%	14.1%	0.159	0.854	0.058
generic resample, before fix	15.8%	5.6%	0.094	0.752	0.023
generic resample, after fix	44.2%	14.1%	0.159	0.854	0.058

Sources: [M: `C:/es/eval/resample_check/report.md`]; [M: `research/experiments/implant-bench/README.md`] (Corrections, 2026-10-18).

After the fix the two paths give the same five numbers. In objects, count what the bug had cost v0.2.0.

Worked example 37.6: Objects the bug hid

At the top 1%:

$$0.442 - 0.158 = 0.284$$
$$0.284 \times 909 = 258.2 \approx 258 \text{ objects}$$

At the top 0.1%:

$$0.141 - 0.056 = 0.085$$
$$0.085 \times 909 = 77.3 \approx 77 \text{ objects}$$

(Script check: 258.156 and 77.265 [C].)

When a benchmark has its own data path, run one model through both paths on data where both exist, and treat any gap as a bug until it is explained.

37.9 The scoreboard, read correctly: bench-v2.1

bench-v2 was rebuilt with the fix as *bench-v2.1* (`--version 2 --psf-sigma 1.4 --foreign-tol 1.2 --seed 0`): 18 hosts, 2,817 objects (signature 985, material 927,

host 905) [M: research/experiments/implant-bench/README.md]. The foreignness tolerance was also tightened from 1.5 to 1.2, the candidate fix for bench-v2's marginal acceptance miss. Table 37.2 is the board.

Table 37.2: bench-v2.1, residual Mahalanobis. "Caught" is the share of the 2,817 objects with a pixel within one step of a top-scoring pixel (Chapter 33).

model	top 1%	top 0.1%	pAUC to 1%	TPR at 0.1%	EnMAP hosts	PRISMA hosts
chance	10.5%	1.1%				
v0.2.0 (read correctly)	**35.5%**	12.6%	0.168	0.082	21.6%	**42.2%**
F001 ep109	33.1%	11.7%	0.157	0.078	20.5%	37.7%
FT02c ep10	35.1%	12.8%	0.169	0.086	20.7%	40.3%
FT03 ep10	34.9%	12.7%	0.168	0.086	20.6%	40.1%
FT04 ep10	35.1%	12.7%	0.169	0.083	20.3%	40.4%
FT05 ep5	35.4%	12.8%	0.170	0.085	20.8%	40.7%
PCA-RX (no model)	33.2%	9.8%	0.154	0.069	19.8%	37.7%

Sources: [M: research/experiments/implant-bench/README.md] (bench-v2.1); FT04's row [M: research/experiments/indradhanu-finetune/FT04-final-joint/README.md] and [M: C:/es/eval/ft04/report.md].

Values that will drift

Table 37.2 and Figure 37.3 are the 2026-10-18 board (FT04 added 2026-10-19). Later runs (FT06 to FT09, Indradhanu Heavy; Chapters 41 to 43) are scored on the same bench and are not shown here. Any rebuild of the bench moves every row.

Read it against the old boards (Figure 37.3). v0.2.0 caught 19.0% on bench-v1, 22.1% on bench-v2, 35.5% on bench-v2.1. FT03 caught 26.9%, 32.3%, 34.9% [M: research/experiments/implant-bench/README.md].

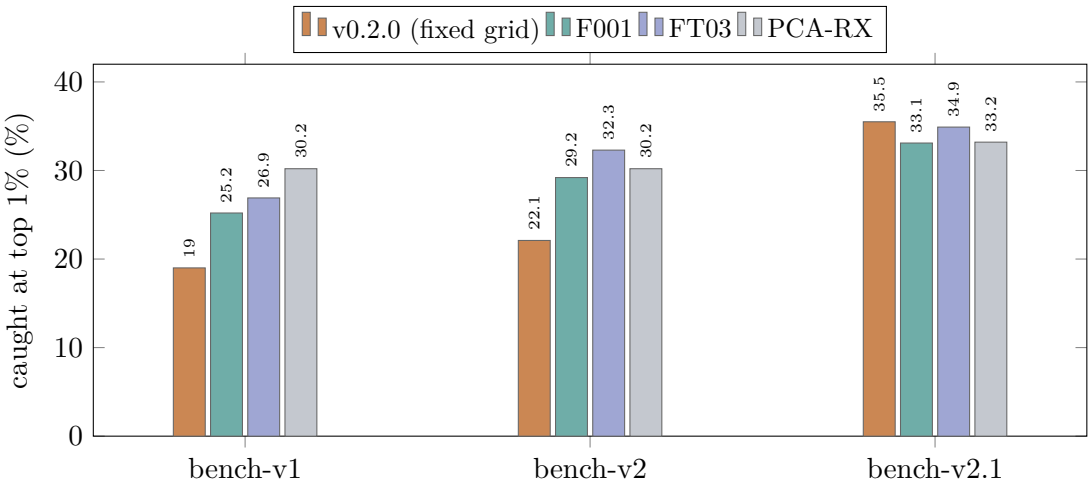


Figure 37.3: Objects caught at the top 1% on the three implant benches, drawn to scale. The wavelength-reading models (F001, FT03) and PCA-RX move a few points with each rebuild; v0.2.0, whose reading was the only one fixed, jumps 13.4 points from bench-v2 to bench-v2.1. Its bench-v1 and bench-v2 bars measured a broken resample.

Worked example 37.7: How much of v0.2.0's jump is the fix?

v0.2.0 moved

$$35.5 - 22.1 = 13.4 \text{ points}$$

from bench-v2 to bench-v2.1. FT03, which never went through the broken resample, moved

$$34.9 - 32.3 = 2.6 \text{ points}$$

with the rebuild alone (correct signatures, tighter tolerance). If the rebuild gave v0.2.0 the same 2.6 points, the fix accounts for

$$13.4 - 2.6 = 10.8 \text{ points.}$$

This split is rough: the rebuild changed the bench for every model, but not by exactly the same amount.

How big is noise? A rough ruler. Is v0.2.0's 35.5% really ahead of FT03's 34.9%? Treat each object as a coin that a model catches with some probability p. Write $X_i = 1$ if object i is caught and 0 if not; this is a *Bernoulli* variable. Derive its mean and variance:

$$\begin{aligned}
\mathbb{E}[X_i] &= 1 \cdot p + 0 \cdot (1-p) = p, \\
\mathbb{E}[X_i^2] &= 1^2 \cdot p + 0^2 \cdot (1-p) = p, \\
\mathrm{Var}[X_i] &= \mathbb{E}[X_i^2] - (\mathbb{E}[X_i])^2 = p - p^2 = p(1-p).
\end{aligned}$$

The share caught is the average $\bar{X} = \frac{1}{n}\sum_i X_i$. For independent objects the variance of a sum is the sum of variances, and dividing by n divides the variance by n^2:

$$\mathrm{Var}[\bar{X}] = \frac{1}{n^2} \cdot n\,p(1-p) = \frac{p(1-p)}{n}, \qquad \mathrm{SE} = \sqrt{\frac{p(1-p)}{n}}. \tag{37.7}$$

Worked example 37.8: Is 35.5% against 34.9% a difference?

With $p = 0.35$ and $n = 2{,}817$:

$$\begin{aligned}
p(1-p) &= 0.35 \times 0.65 = 0.2275 \\
0.2275/2{,}817 &= 0.0000808 \\
\sqrt{0.0000808} &= 0.0090 \quad \text{(about 0.9 points)} \\
\text{gap} &= 35.5 - 34.9 = 0.6 \text{ points} = 0.6/0.9 = 0.7 \text{ standard errors}
\end{aligned}$$

Under one standard error: a tie. (Script: SE = 0.00899, gap 0.67 SE [C].)

Two models scored on the *same* objects are correlated, so the honest uncertainty of their difference is smaller than this single-model ruler suggests. Chapter 36 builds the proper paired object bootstrap; use (37.7) only as a first, cautious check.

Three more readings of Table 37.2:

- **Every model stays above PCA-RX at the top 0.1%** (11.7–12.8% against 9.8%), as on the real bench - the ordering test of Chapter 34 still passes. At the top 1% F001 and PCA-RX tie (33.1% against 33.2%).
- **EnMAP hosts sit near 21% for every model and for PCA-RX.** Unexplained on bench-v1, v2 and v2.1.
- **The easy tier is too foreign.** Per-tier foreignness on PRISMA hosts is 1.05× / 1.15× / 1.60× real (subtle / medium / easy); halo 0.92–1.06× [M: `research/experiments/implant-bench/README.md`]. The easy tier is outside the 25% acceptance limit.

Worked example 37.9: The selection effect behind the easy tier

This is an illustration, not bench data [A]. Four easy targets want foreignness 0.001, 0.002, 0.004 and 0.008. With the tolerance at 1.2, suppose the realised object can only get within 1.2× of the two higher targets, so the two lower spots are skipped. The median of four numbers is the mean of the middle two:

$$\begin{aligned} \text{targets:} \quad & 0.001,\ 0.002,\ 0.004,\ 0.008 \qquad \text{median} = (0.002 + 0.004)/2 = 0.003 \\ \text{survivors:} \quad & 0.004,\ 0.008 \qquad \text{median} = (0.004 + 0.008)/2 = 0.006 \end{aligned}$$

Nothing was implanted wrongly; the population that remains is not the one aimed at.

Read correctly, v0.2.0 ties or beats every new model on bench-v2.1, as it ties them on the real bench: the "pretraining is the big step" result (v0.2.0 19–22% to F001 25–29%) was the reading bug.

37.10 Lessons for trusting a benchmark

Chapter 34's ordering test asked: does the benchmark reproduce something the real labels already told you? It was applied to PCA-RX, and passed. It was not applied to v0.2.0 - and v0.2.0 failed it on both bench-v1 and bench-v2 (a tie on real objects, far behind on implants). That failure was the bug announcing itself. So, before you trust a benchmark's ranking of models:

1. **Check every model's position against the real labels, not only the baseline's.** One model that the real bench and the benchmark disagree about is a flag on the benchmark's data path.
2. **Cross-check any re-implemented data path against the product's**, on data where both exist (Section 37.8).
3. **Test with real input orderings.** PRISMA's descending list is a property of real files; a test written with `[500,600,700]` cannot see it.
4. **Know your preconditions.** If a library function's contract says "meaningless" for some input, either enforce the precondition (sort) or assert it (`np.all(np.diff(xp)>0)`).
5. **Give a result you hoped for the scrutiny you would give one you feared.**

The same descending list surfaced once more, in the product: when Indradhanu V2 began reading native PRISMA bands (Chapter 39), `scene_segmentation`'s brightness - "the mean of band 0 up to the VNIR edge" - turned out to assume ascending order. On native PRISMA, band 0 is the far SWIR. That fix is part of the next two chapters' story.

37.11 Common confusions

The bug made every model's implant numbers wrong.
Two different effects. The *reading* bug hit fixed-grid models (v0.2.0) on PRISMA hosts only. The *signature* bug garbled half (bench-v1) or a third (bench-v2) of the implants for every model alike. The wavelength-reading models' rows moved with the rebuild, not with the reading fix.

The real bench was wrong too.
No. For fixed-grid models the evaluator reloads PRISMA bench scenes through the product's `band_filter_apply`, which always sorted. v0.2.0's real-bench 0.159 never changed.

Sorting the wavelengths reorders the data.
It reorders the pairs. Band 453 nm keeps its own values; it is only listed first instead of last. The function being sampled is unchanged.

NumPy guarantees the two-value step on descending input.
It guarantees nothing: the contract calls the result meaningless. The step is what NumPy 2.5.2 returned on the dev laptop, consistent with the check order of Section 37.3. Do not rely on it, and do not write a test that expects it.

bench-v2.1 says v0.2.0 is the best model.
It says v0.2.0 ties the best models: 0.6 points ahead of FT03 is under one standard error. No new model measurably beats it on detection.

bench-v2.1 failed acceptance, so it cannot be used.
The easy tier's foreignness misses the 25% limit, by a known selection effect. The ordering test still holds. Use it as a ranking with that caveat stated, and keep the real bench as the judge.

37.12 Check yourself

Exercise 37.1. By hand: samples at 800, 700, 600 nm with values 0.30, 0.20, 0.10. What is the correct value at 650 nm, and what does `np.interp` return (NumPy 2.5.2, the check order of Section 37.3) if the samples are listed 800, 700, 600? What does it return at 590 nm?

Exercise 37.2. Bands listed 700, 600, 500 nm. Write the correct resample row for a grid wavelength of 620 nm, then the row the old unsorted code built, and apply both to a pixel reading $(0.40, 0.30, 0.10)$.

Exercise 37.3. Bands listed `wl` $= (600, 800, 700)$ (unsorted, not descending). Give `np.argsort(wl)`, the sorted list, and the matrix $\mathbf{M}$ of the fixed `resample` for the grid $(650, 750)$. Apply it to the pixel $(1, 3, 2)$ in listed order.

Exercise 37.4. Through the old generic resample v0.2.0 caught 15.8% of the 909 real objects at the top 1%, through the product path 44.2%. If bench-v2.1's FT02c (35.1%) and F001 (33.1%) are compared with $p = 0.34$ over 2,817 objects, how many standard errors apart are they? Is that difference larger or smaller than the bug's effect on v0.2.0's real-bench catch, in standard errors of the real bench ($p = 0.30$, $n = 909$)?

Exercise 37.5. The repo's resample tests passed for weeks with the bug in place. Explain why, and write the one-line assertion you would add inside `resample` to make the old code fail loudly instead.

Next: Chapter 38, FT01 to FT05: what each fine-tuning run taught.

Chapter 38

FT01 to FT05: what each fine-tuning run taught

The one thing this chapter teaches

Six fine-tuning runs (FT01, FT02, FT02b, FT02c, FT03, FT05) and the release candidate FT04 bought better masks, native bands and better reconstruction on all three sensors - and no measurable gain in detection by the model alone. Each run was judged by criteria written before launch, and each taught one lesson: a frozen encoder's ceiling, a gate read at the wrong checkpoint, edges that are also targets, a target that hides the anomaly, and a score - residual Mahalanobis - that absorbs whatever better reconstruction changes.

38.1 Where the runs start

Chapter 29 laid out the curriculum: a *probe* (train only a small head on a frozen encoder), an *adapt* stage (continue masked reconstruction on the real sensors), a *joint* stage (reconstruction and segmentation together), optional pseudo-labels, and finally product use of the masks. It also taught the two mechanical fixes that the early runs forced - the BatchNorm see-saw and mixed batches. This chapter reads the runs themselves, in the order they ran on an L40S GPU (`g6e.2xlarge`) between 2026-10-16 and 2026-10-18, as a sequence of lessons. Table 38.1 is the map.

Why F001 stopped at epoch 113. Chapter 27 described F001 as a 200-epoch run. It did not finish. The PRISMA bench under residual Mahalanobis read 0.861 at epoch 60 and 0.862 at epoch 109 [M: `research/experiments/indradhanu-se-pretrain/F001-final-pretrain/README.md`]. Forty-nine epochs for 0.001: the run was stopped at epoch 113 by decision, and every fine-tuning run starts, directly or through its parent, from `F001_ep109.pt`.

Gates written before launch. Every run's README states, before launch, what it expects and what would count as passing. That habit is the spine of this chapter. A criterion written after seeing the numbers can always be made to pass; a criterion written before is a prediction that can be wrong, and several of the lessons below are exactly a prediction being wrong.

Analogy

A pianist moves from an upright to a concert grand. After a week of practice the scales are cleaner on every octave - that is reconstruction. In the recital, the only thing the audience

Table 38.1: The fine-tuning runs of 2026-10-16 to 18. "Verdict" is against the criteria each run's README fixed before launch.

run	stage	what changed	hours	verdict
FT01	probe	segmentation head on frozen F001 (epoch 109), EnMAP	1.4	passed (water 0.82)
FT02	adapt	reconstruction on three sensors, one sensor per batch	4.25	gate not met
FT02b	adapt	FT02 continued with mixed-sensor batches	2.2	gate not met
FT02c	adapt	FT02b continued with an edge term	2.2	not met
FT03	joint	reconstruction + segmentation, mixed batches	3.0	passed
FT05	joint	implants in the input, clean tile as target	–	not passed
FT04	joint	pseudo-labels on PRISMA / AVIRIS-NG, rare-class weights	–	passed

Source: [M: `research/experiments/indradhanu-finetune/README.md`] and the run READMEs beside it.

hears, one piece is very slightly worse - that is detection. Cleaner scales are evidence of progress, not a guarantee of a better recital; you still have to play the recital. Every run below improved its scales. Very few changed the recital.

38.2 FT01: the probe measures what the encoder already knows

A *probe* trains only the small segmentation head (Chapter 28) on top of a frozen encoder - frozen meaning its parameters, BatchNorm statistics included, do not change. Whatever the head learns must already be present in the frozen features, so the probe measures what the encoder knows. FT01 ran 15 epochs on EnMAP in 1.4 hours [M: `research/experiments/indradhanu-finetune/FT01-probe/README.md`].

Counting the head by hand. The FT01 README calls the head "about 372k parameters". Predict the exact figure. The head is a SegFormer-style decoder (Chapter 19) of width 128 over the encoder's four stages, whose channel counts are 32, 64, 160 and 256. Its layers, read from the shipped checkpoint's shapes:

Worked example 38.1: The segmentation head's parameters

Recall from Chapter 10 that a convolution with C_{in} inputs, C_{out} outputs and a $k \times k$ kernel has $C_{\text{out}} \cdot C_{\text{in}} \cdot k^2$ weights and C_{out} biases. Do each line on paper.

1. Four 1×1 projections to 128 channels. Weights:

$$128 \times (32 + 64 + 160 + 256) = 128 \times 512 = 65{,}536.$$

   Biases: $4 \times 128 = 512$. Total $65{,}536 + 512 = 66{,}048$.

2. A 1×1 fuse convolution from the four stacked projections, $4 \times 128 = 512$ channels, to 128:

$$128 \times 512 + 128 = 65{,}536 + 128 = 65{,}664.$$

3. A 3×3 refine convolution, 128 to 128:

$$128 \times 128 \times 9 + 128 = 147{,}456 + 128 = 147{,}584.$$

4. A 3×3 output convolution, 128 to 80. Why 80? Five classes, each needing a 4×4 block of pixels per quarter-resolution position for the pixel shuffle (Chapter 19): $5 \times 16 = 80$.

$$80 \times 128 \times 9 + 80 = 92{,}160 + 80 = 92{,}240.$$

Sum:

$$\begin{aligned} 66{,}048 + 65{,}664 &= 131{,}712 \\ 131{,}712 + 147{,}584 &= 279{,}296 \\ 279{,}296 + 92{,}240 &= 371{,}536 \end{aligned}$$

The shipped checkpoint's `seg_head.*` tensors hold exactly 371,536 numbers (`research/textbook/tools/compute_ftruns.py`) [C]. "About 372k" is right.

The measure. The head is judged by IoU, intersection over union (Chapter 28): for one class, the number of pixels both predicted and labelled, divided by the number predicted or labelled. If 6 pixels are labelled water, 5 are predicted water, and 4 are both,

$$\text{IoU} = \frac{4}{6 + 5 - 4} = \frac{4}{7} = 0.571.$$

(The union is $6 + 5 - 4$: adding the two sets counts the overlap twice, so subtract it once.)

Table 38.2: FT01: per-class IoU on EnMAP's 950 test tiles. best.pt is epoch 14 (lowest segmentation validation loss).

epoch	water	cloud	thin cloud	shadow	snow	mean
1	0.705	0.600	0.067	0.000	0.000	0.274
3	0.777	0.733	0.076	0.000	0.000	0.317
6	0.780	0.784	0.065	0.104	0.000	0.347
9	0.815	0.766	0.061	0.185	0.000	0.365
12	0.822	0.807	0.089	0.165	0.055	0.388
14	**0.822**	**0.801**	**0.091**	**0.199**	0.175	**0.418**
15	0.823	0.800	0.089	0.193	0.169	0.415

Source: [M: `research/experiments/indradhanu-finetune/FT01-probe/README.md`] (from `finetune_metrics.csv`).

The gate was "water at least 0.6, or the plan is revisited". Water reached 0.82 and cloud 0.80 (the README had expected cloud at least 0.5). The interesting row is thin cloud (Figure 38.1). Every other class climbed over the 15 epochs; thin cloud sat at 0.06 to 0.09 from start to end, against an expectation of 0.3 to 0.5. The label audit of Chapter 28 had already found that EnMAP's cirrus labels are often driven by the surface, not by cloud.

A head that cannot improve on one class while it improves on every other is usually hitting the labels' ceiling, not the encoder's: the encoder and head are shared by every class, so a feature limit would still let the class move a little.

So the FT01 README says: do not tune later runs on thin cloud. Snow's 0.17 rests on a single test tile and is not a measurement.

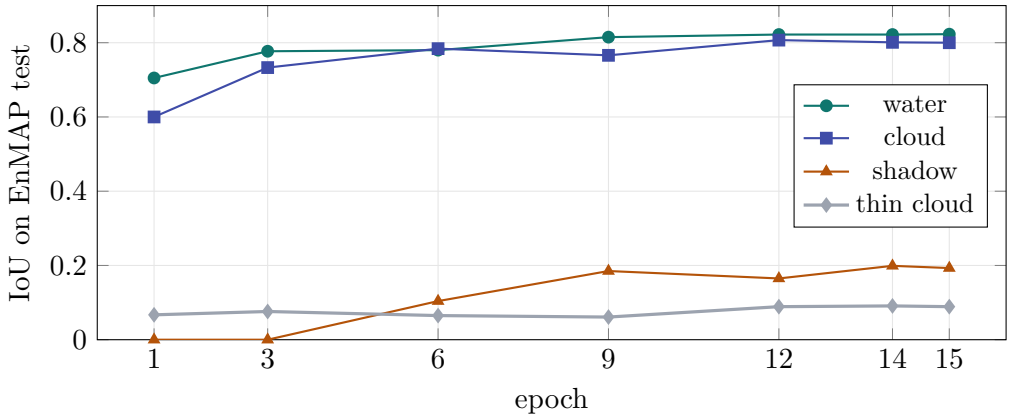


Figure 38.1: FT01's IoU by epoch, drawn to scale from Table 38.2. Water, cloud and shadow climb; thin cloud (grey) stays between 0.06 and 0.09 for the whole run.

38.3 FT02: better reconstruction, a gate missed by 0.006

FT02 continued F001's masked reconstruction on all three sensors - EnMAP, PRISMA and AVIRIS-NG at shares 0.40 / 0.35 / 0.25, one sensor per batch, 20 epochs of 30,000 samples, learning rate 10^{-4}, 4.25 hours. Its three pass criteria, written before launch:

1. Bench mean ROC-AUC (residual Mahalanobis) at least 0.861 at the best checkpoint.
2. No scene below F001's epoch-60 score by more than 0.03.
3. Each sensor's test reconstruction lower at the end than after epoch 1.

Table 38.3: FT02: validation SAM (degrees) per sensor and the PRISMA bench, ROC-AUC averaged over four scenes.

epoch	SAM EnMAP	SAM PRISMA	SAM AVIRIS-NG	res. Mahalanobis	combined
F001 ep109	–	–	–	0.862	0.706
1	4.81	7.30	7.94	**0.864**	0.760
8 (best.pt)	4.17	5.93	6.07	0.855	0.824
13	4.16	6.04	6.49	0.855	0.809
20 (last.pt)	4.07	6.31	6.19	0.853	0.814

Source: [M: `research/experiments/indradhanu-finetune/FT02-adapt/README.md`].

Worked example 38.2: Reading FT02's gate

best.pt (epoch 8) scored 0.931, 0.852, 0.757 and 0.879 on the four scenes. Add one at a time:

$$0.931 + 0.852 = 1.783$$
$$1.783 + 0.757 = 2.540$$
$$2.540 + 0.879 = 3.419$$
$$3.419/4 = 0.85475 \approx 0.855$$

Criterion 1 asks for 0.861: the miss is $0.861 - 0.855 = 0.006$. Criterion 2 compares each scene with F001 at epoch 60 (0.930, 0.860, 0.775, 0.881):

$$\text{set1: } 0.931 - 0.930 = +0.001 \qquad \text{set2: } 0.852 - 0.860 = -0.008$$
$$\text{set3: } 0.757 - 0.775 = -0.018 \qquad \text{set4: } 0.879 - 0.881 = -0.002$$

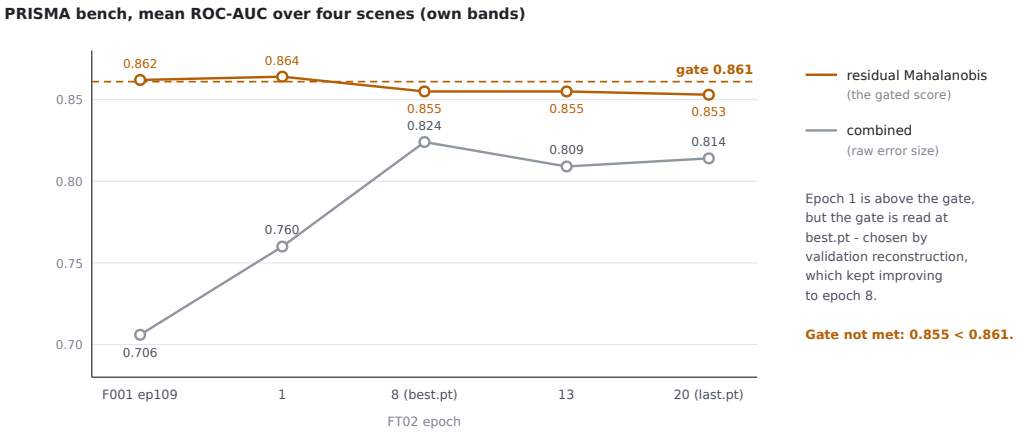


Figure 38.2: FT02's bench by epoch against the gate. Residual Mahalanobis (amber) starts above the 0.861 gate at epoch 1 and settles below it; `combined` (grey) climbs from F001's 0.706 to 0.824. best.pt is epoch 8, chosen by validation reconstruction, not by the bench.

The largest drop is 0.018, under 0.03: passed. Criterion 3 (SAM epoch 1 to 20): EnMAP $4.81 - 4.07 = 0.74$, PRISMA $7.30 - 6.31 = 0.99$, AVIRIS-NG $7.94 - 6.19 = 1.75$ degrees lower: passed. Gate: not met, by 0.006.

Lesson 1: a gate is read at one checkpoint, chosen by a different rule. The bench peaked at epoch 1, at 0.864 - above the gate. It does not count, because the gate is read at the best checkpoint, and best.pt is chosen by mean validation reconstruction (the fixed ruler of Chapter 27), which kept improving until epoch 8. Reconstruction and detection pulled apart after the first epoch. Writing "at the best checkpoint" before launch was deliberate: picking whichever epoch scores best on four labelled scenes would tune the run to those four scenes, turning the bench into training data.

Lesson 2: part of the change is more bands, not better weights. The trained-wavelength record of Chapter 29 grew during FT02 from 202 to 733 wavelengths: AVIRIS-NG added 337, PRISMA 177, EnMAP 17 [M: `research/experiments/indradhanu-finetune/README.md`]. Check the arithmetic: $202 + 337 = 539$, $539 + 177 = 716$, $716 + 17 = 733$. EnMAP added few because SpectralEarth *is* EnMAP, so most of its wavelengths were already recorded. From epoch 1 on, the bench reads all 185 bands of the PRISMA scenes, where F001 read 165; any comparison with F001 mixes the two effects.

Lesson 3: `combined` nearly caught up - and that says something about residual Mahalanobis. `combined` (half L1, half SAM, each divided by the scene's maximum; Chapter 31) measures how *big* a pixel's error is. F001 made large ordinary errors on PRISMA's background, which buried the targets. FT02 trained on PRISMA, PRISMA's validation SAM fell from 7.30 to 5.93 degrees, and the error floor dropped, so `combined` rose from 0.706 to 0.824. Residual Mahalanobis had already removed that floor - it subtracts the local mean residual and whitens by the scene's covariance - so it had little to gain. Keep this in mind: in Section 38.9.4 it becomes the central lesson of the whole series.

The see-saw and the decision. The per-sensor validation did not fall smoothly: in epochs 11–12 and 15–17 PRISMA improved while EnMAP and AVIRIS-NG got worse, then the reverse (epoch 17: PRISMA 5.04, AVIRIS-NG 8.24 degrees). Chapter 29 traced it to the model's one

BatchNorm and fixed it with mixed batches (FT02b: see-saw gone, SAM 3.99 / 4.37 / 5.57, bench 0.856, gate still not met). On 2026-10-16 the user decided to *build on* FT02 rather than discard it: reconstruction was better on every sensor and the miss was small.

Source check: the warm-up that does not exist

FT02's README lists "LR 1e-4 cosine (1 warm-up epoch)", and every config sets `warmup_epochs: 1`. The log of 2026-10-16 (`research / experiments / indradhanu - finetune/README.md`, FT02b launch entry) records: "The configured 1-epoch warm-up does not exist in code (`warmup_epochs` is never read)." The runs had no warm-up.

38.4 Edges: why correcting the score map fails

Look at a residual Mahalanobis score map of a PRISMA bench scene and you see lines: field boundaries, roads, coastlines. The model reconstructs a sharp boundary slightly blurred or shifted, the error on both sides is large, and those pixels rank among the top scores. On 2026-10-16 this was measured with rules fixed before the results (`C:/es/edges/edges.py`).

38.4.1 Measuring edge strength

An *edge* is a place where brightness changes quickly from one pixel to the next. For a smooth function $f(x, y)$, "how quickly" is the *gradient*, the vector of partial derivatives $(\partial f/\partial x, \partial f/\partial y)$. An image has no derivatives - only values on a grid - so you replace each derivative by a *finite difference*. Derive the central difference from the definition of the derivative:

$$\frac{\partial f}{\partial x}(x, y) = \lim_{h \to 0} \frac{f(x + h, y) - f(x - h, y)}{2h}. \tag{38.1}$$

On a pixel grid the smallest step is one pixel, $h = 1$, so drop the limit:

$$g_x(i, j) \approx \frac{f(i, j + 1) - f(i, j - 1)}{2}, \qquad g_y(i, j) \approx \frac{f(i + 1, j) - f(i - 1, j)}{2}. \tag{38.2}$$

The *gradient magnitude* is the length of the vector (g_x, g_y), by Pythagoras:

$$|\nabla f|\,(i, j) \approx \sqrt{g_x^2 + g_y^2}. \tag{38.3}$$

The diagnosis used a different, related measure: average a pixel's bands into one brightness number, then take the *largest absolute difference to any valid four-neighbour* (up, down, left, right):

$$e(i, j) = \max\left(\,|f_{i,j} - f_{i-1,j}|\,,\ |f_{i,j} - f_{i+1,j}|\,,\ |f_{i,j} - f_{i,j-1}|\,,\ |f_{i,j} - f_{i,j+1}|\,\right). \tag{38.4}$$

These are one-sided differences, and taking the maximum instead of a Pythagorean sum is the "L_∞" way of measuring a vector's size. A pixel is *strong* if its e is in the scene's top 10%.

Worked example 38.3: Two edge measures on a 3 × 3 patch

Band means of a 3×3 patch with a field boundary running down its right side:

$$\begin{matrix} 0.10 & 0.10 & 0.30 \\ 0.10 & 0.12 & 0.30 \\ 0.10 & 0.10 & 0.31 \end{matrix}$$

Central differences at the centre (0.12). Left 0.10, right 0.30, up 0.10, down 0.10:

$$g_x = (0.30 - 0.10)/2 = 0.20/2 = 0.10$$
$$g_y = (0.10 - 0.10)/2 = 0$$
$$|\nabla f| = \sqrt{0.10^2 + 0^2} = \sqrt{0.01} = 0.10$$

The diagnosis's measure (38.4) *at the centre.*

$$|0.12 - 0.10| = 0.02 \quad \text{(up)}$$
$$|0.12 - 0.10| = 0.02 \quad \text{(down)}$$
$$|0.12 - 0.10| = 0.02 \quad \text{(left)}$$
$$|0.12 - 0.30| = 0.18 \quad \text{(right)}$$
$$e = \max = 0.18$$

The top-left pixel (0.10) has neighbours right 0.10 and down 0.10 inside the patch, so $e = 0$. *Now a one-pixel target.* Replace the patch by a field of 0.10 with a single bright pixel of 0.30 at the centre. Central differences at the centre: $g_x = (0.10 - 0.10)/2 = 0$, $g_y = 0$, so $|\nabla f| = 0$ - the symmetric difference cannot see a one-pixel spike. The one-sided maximum: every neighbour differs by $|0.30 - 0.10| = 0.20$, so $e = 0.20$, a strong edge. (Script check of both [C].)

That last line is the whole trap. A small target, brighter or darker than its field, has a large difference to its neighbours on every side. By the diagnosis's definition it *is* an edge.

Table 38.4: Edges and false alarms on the four PRISMA bench scenes (F001 epoch 109 and FT02 epoch 13). By chance each share would be 10%, since "strong" is the top 10%.

measurement	range over the four scenes
top-1% false positives on strong edges	49–68%
ground-truth target pixels on strong edges	29–79%
detected target pixels (hits) on strong edges	39–91%

Source: [M: `research/experiments/indradhanu-finetune/FT02-adapt/README.md`] (Observations, edge diagnosis).

The first row says edges are the main source of false alarms - five to seven times chance. The second row is the trap.

38.4.2 Correcting the score map, and why it collapses

The obvious fix: compare each pixel only with pixels of similar edge strength. The measured version sorted the scene into 20 bins by edge strength and scored each pixel by *its rank minus the median rank of its bin*. A pixel on a strong edge then has to beat other strong-edge pixels, not the smooth field. It collapsed detection: bench mean 0.86 to 0.67. Work why on eight pixels.

Worked example 38.4: Rank minus the median rank of its bin

Eight pixels, two bins. A is a real target on an edge; B, C, D are edge false alarms; E, F, G, H are a smooth field with small noise. Scores and bins:

pixel	A	B	C	D	E	F	G	H
score	9	10	8	7	3	2	1	4
bin	strong	strong	strong	strong	smooth	smooth	smooth	smooth
rank (1 = lowest)	7	8	6	5	3	2	1	4

Medians of the ranks in each bin. Strong: ranks 5, 6, 7, 8; the median of four numbers is the mean of the middle two, $(6+7)/2 = 6.5$. Smooth: ranks 1, 2, 3, 4; median $(2+3)/2 = 2.5$. Corrected scores:

$$A:\ 7-6.5=0.5 \quad B:\ 8-6.5=1.5 \quad C:\ 6-6.5=-0.5 \quad D:\ 5-6.5=-1.5$$
$$E:\ 3-2.5=0.5 \quad F:\ 2-2.5=-0.5 \quad G:\ 1-2.5=-1.5 \quad H:\ 4-2.5=1.5$$

Before, the target A was second of eight. After, B and H share first place (1.5) and A ties with E, a noise pixel, for third. The correction promoted the top of the smooth field - pixels that were not anomalous at all - to the level of the strongest edges, and it pushed the target down with its edge neighbours. (Script check [C].)

Table 38.5: Earlier score-map sharpeners on F001 epoch 52; none beat plain residual Mahalanobis (D).

method, applied to the score map	bench AUC	objects caught of 909
D residual Mahalanobis, no post-processing	**0.865**	383
F(C) difference-of-Gaussians blob boost	0.718	403
E(C) local z-score of the score map (ring)	0.790	–
G-C suppress errors explained by a ±1 px shift	0.845	–

Source: [M: `research/experiments/indradhanu-finetune/FT02-adapt/README.md`] (`C:/es/postscore/`).

The blob boost found 20 more objects and gave up $0.865 - 0.718 = 0.147$ of AUC to do it: it lit up many more non-targets on the way. The FT02 README's conclusion: edge strength, measured on the score map or the image, cannot tell a field boundary from a target, because at the scale of a few pixels they look the same.

38.4.3 An edge term under the blind spot

Training is a different place to act. Recall inference (Chapter 30): two complementary checkerboard masks, so every pixel is reconstructed while its own token - a 4×4 pixel square, stage 1's patch embedding with kernel 4 and stride 4 - is hidden. The model predicts the hidden token's pixels from the visible tokens around it. Compare the two things that look alike on the score map:

- **A field boundary** that crosses the hidden token continues on both sides of it. Its direction and its contrast are visible in the neighbouring tokens. A model that has learned to continue straight boundaries sharply can predict it.
- **A small target** sitting inside the hidden token has no continuation. Nothing in the visible tokens says it is there. The model predicts the field, the error is large - and that error is the detection.

So a loss that rewards *sharp* reconstruction of hidden pixels can teach the first and cannot teach the second. The score map has already lost this information: it records only how large the error was, not whether the context could have predicted the structure.

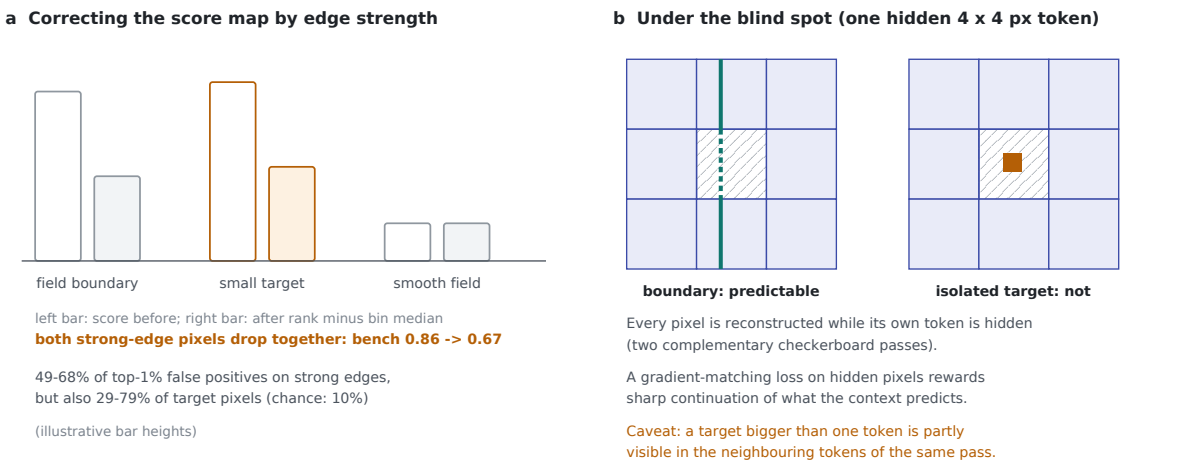


Figure 38.3: (a) Edge-strength correction (amber) pushes down everything on strong edges, the field boundary and the target alike. (b) Under the blind spot: a boundary passing through a hidden token (grey hatching) can be continued from its visible ends; a target wholly inside the hidden token leaves no trace in the context.

The gradient-matching loss. FT02c added `gradient_l1` (`app/foundation_models/trainers/finetune_trainer.py`) to the mixed-batch loss with weight `grad_loss_weight`. Its docstring: "mean $|d\,\hat{x} - d\,x|$ over horizontal and vertical neighbour differences, per band, on pairs whose two pixels are both in loss_mask … and valid in that band. Blur shows up as a gradient mismatch." In symbols, for one band, with P the set of neighbouring pairs (p, q) both hidden, valid and eroded (Chapter 18):

$$L_{\text{grad}} = \frac{1}{|P|} \sum_{(p,q)\in P} \Big| (\hat{x}_q - \hat{x}_p) - (x_q - x_p) \Big|. \tag{38.5}$$

L1 on the values (Chapter 25) punishes blur only mildly; a loss on the differences punishes it directly.

Worked example 38.5: The same blur, two losses

A row of four hidden pixels across a boundary:

input: 0.10, 0.10, 0.50, 0.50
reconstruction: 0.10, 0.20, 0.40, 0.50 (the boundary, blurred)

Value L1. Absolute differences: 0, 0.10, 0.10, 0. Sum 0.20; mean $0.20/4 = 0.05$.
Gradient L1. Input differences (next minus this): $0.10 - 0.10 = 0$, $0.50 - 0.10 = 0.40$, $0.50 - 0.50 = 0$. Reconstruction differences: $0.20 - 0.10 = 0.10$, $0.40 - 0.20 = 0.20$, $0.50 - 0.40 = 0.10$. Mismatches: $|0.10 - 0| = 0.10$, $|0.20 - 0.40| = 0.20$, $|0.10 - 0| = 0.10$. Sum 0.40; mean $0.40/3 = 0.133$.
The same blur costs 0.05 in value L1 and 0.133 in gradient L1: the edge term is what notices that a step became a ramp. (Script: 0.0500 and 0.1333 [C].)

Source check: where a hidden target could leak

It has been said that overlapping patch embeddings leak one or two pixels of a hidden token into its neighbours. In this model they do not, at the stage that hides tokens: stage 1's embedding is non-overlapping, kernel 4, stride 4, padding 0 (`app/foundation_`

`models/components/seg_former_encoder.py`: "Stage 1: kernel=4, stride=4 (NON-OVERLAPPING - zero information leakage"), hidden tokens are restored as zeros after stage 1, and the spectral stem in front works on one pixel at a time. The real leak is a target that spans several tokens: some of them are visible in each checkerboard pass, so an edge-continuer could "continue" it into the hidden part.

What FT02c showed. FT02c ran on 2026-10-16 (2.2 hours, code `9ff0484`) from FT02b's best.pt with `grad_loss_weight` 1.0 and the SAM weight held at 0.5 throughout. Its criterion: the share of top-1% false alarms on strong edges down on at least 3 of 4 scenes.

Table 38.6: FT02c against its parent FT02b.

	FT02b ep9	FT02c ep10
val. SAM, EnMAP / PRISMA / AVIRIS-NG	3.99 / 4.37 / 5.57	3.93 / 4.21 / 5.41
edge term `train_grad_l1`, ep 1 → 10	–	0.01163 → 0.01153
bench ROC-AUC, residual Mahalanobis	0.856	0.855
pAUC / caught top 1% / top 0.1%	0.158 / 411 / 128	0.159 / 413 / 131
top-1% false alarms on edges, set1–4	69 / 52 / 52 / 60%	69 / 52 / 52 / 61%

Source: [M: `research/experiments/indradhanu-finetune/FT02c-edges/README.md`].

The edge term barely moved: $0.01163 - 0.01153 = 0.00010$, and $0.00010/0.01163 = 0.0086$, about 1% in ten epochs, while L1 and SAM kept falling. The edge false-alarm share is unchanged on all four scenes. Verdict: not met.

Why the term could not be reduced. The decoder (Chapter 19) fuses all encoder stages at a quarter of the resolution, $H/4$, and the pixel shuffle of Shi et al. [39] turns *one* feature vector into a whole 4×4 block of pixels (`app/foundation_models/components/seg_former_decoder.py`). Where an edge falls inside that block has to be encoded in that one quarter-resolution vector; with the loss switched on, it was not learned. The likely fix is architectural - a higher-resolution path into the decoder, masked so it cannot see hidden tokens - not a loss term. (That path was built later, as FT09; Chapter 42.) The small reconstruction gains over FT02b are, in the README's words, "more plausibly from the constant SAM weight 0.5 (FT02b ramped it from 0) than from the edge term".

Edges are the main false alarm and a common shape of a true target; a correction applied to the finished score map cannot tell them apart, and a loss that could, in principle, was blocked by the decoder's 4×4 bottleneck.

38.5 The options not taken

After FT02, four other directions were weighed on 2026-10-16 and set aside, each for a stated reason. Knowing what each would cost is what lets you pick one up later without repeating the discussion.

1. A teacher ("oracle") head - deferred. A *teacher* is a copy of the model that sees more; a *student* is the model being trained, which sees less and is trained to match the teacher's output. Here the teacher would read a sensor's full band list and the student a degraded one. The teacher would be an *exponential moving average* (EMA) of the student's own weights: after

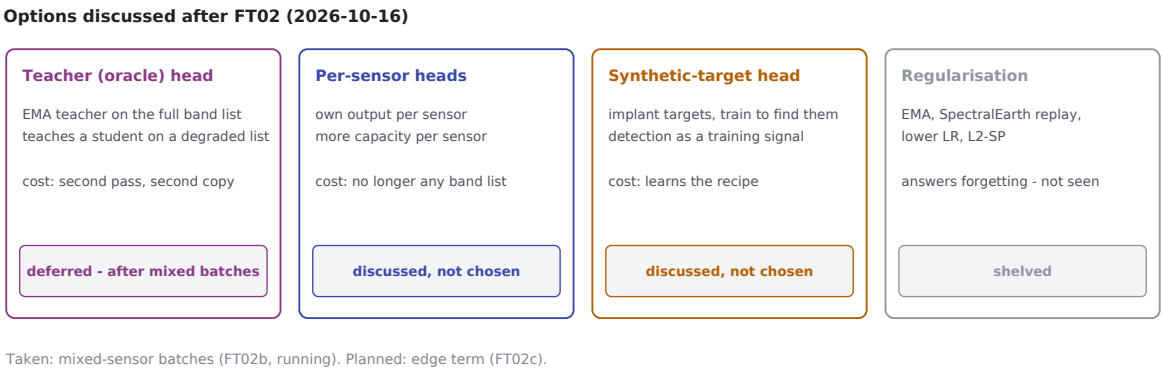


Figure 38.4: The four options discussed after FT02, what each would add, its main cost, and where it stood.

each step, $\theta_{\mathrm{T}} \leftarrow 0.999\,\theta_{\mathrm{T}} + 0.001\,\theta_{\mathrm{S}}$. How slow is that? Hold the student fixed at s and subtract s from both sides of the update:

$$\theta_{\mathrm{T}}^{(n+1)} - s = 0.999\,\theta_{\mathrm{T}}^{(n)} + 0.001\,s - s = 0.999\,\big(\theta_{\mathrm{T}}^{(n)} - s\big). \tag{38.6}$$

The gap shrinks by a factor 0.999 per step, so after n steps

$$\theta_{\mathrm{T}}^{(n)} - s = 0.999^{n}\,\big(\theta_{\mathrm{T}}^{(0)} - s\big). \tag{38.7}$$

The gap halves when $0.999^{n} = 1/2$. Take logarithms of both sides and divide:

$$n = \frac{\ln 2}{-\ln 0.999} = \frac{0.6931}{0.0010005} \approx 693 \text{ steps}. \tag{38.8}$$

One step from teacher 0.50 toward student 0.80: $0.999 \times 0.50 + 0.001 \times 0.80 = 0.4995 + 0.0008 = 0.5003$. The cost: a second forward pass per step, memory for a second copy of the weights, and one more thing that can drift. It was deferred because a teacher built on FT02's swings would inherit them.

2. Per-sensor heads - not chosen. A decoder per sensor would give each its own capacity, but it gives up the design's point: one set of wavelength-aware weights that reconstructs any band list, including sensors nobody has trained on. It also splits the data, and the effects mixed batches address (BatchNorm, gradient balance) sit *before* any head.

3. A synthetic-target detection head - not chosen (then). Implant artificial targets and train a head to find them. An implanted target is usually a *linear mixture*: at one band, background 0.20, target 0.50, fraction 0.30 gives $0.70 \times 0.20 + 0.30 \times 0.50 = 0.14 + 0.15 = 0.29$. The cost is in the word "synthetic": the head learns whatever the implanting recipe makes detectable - a perfectly linear mix, a sharp square outline - and it changes what the model is, from "fails to reconstruct the unexpected" to "knows what a target looks like".

4. Regularisation - shelved. Four standard ways to stop fine-tuning drifting from pretraining: an EMA of the weights, replaying SpectralEarth in each batch, a lower learning rate, and *L2-SP* ("L2 towards the starting point"), which adds a penalty

$$R(\mathbf{w}) = \frac{\alpha}{2} \sum_{i} \big(w_i - w_i^{0}\big)^2 \tag{38.9}$$

to the loss, with $\mathbf{w}^0$ the pretrained weights. Differentiate with respect to one weight to see what it does to training: the power rule brings the 2 down, which cancels the $\frac{1}{2}$,

$$\frac{\partial R}{\partial w_i} = \alpha\,(w_i - w_i^0),$$

a pull back toward the start, proportional to how far the weight has moved. With $\alpha = 0.01$ and two weights moved from $(0.5, -0.2)$ to $(0.6, -0.1)$: the penalty is $\frac{0.01}{2} \times (0.1^2 + 0.1^2) = 0.005 \times 0.02 = 0.0001$, and each weight is pulled back with gradient $0.01 \times 0.1 = 0.001$. All four answer one question - "is the model forgetting?" - and FT02 did not look like forgetting: reconstruction improved on every sensor, EnMAP included, the sensor closest to SpectralEarth.

Values that will drift

Status as of 2026-10-18: option 3 ran, in a different form - not as a head but as implant-denoising (FT05, Section 38.7). Option 1's teacher idea ran as a teacher for *masks* (FT04's pseudo-labels, Section 38.9); cross-sensor reconstruction distillation is still deferred. Regularisation and per-sensor heads have no code.

38.6 Stage 3: FT03, masks, and why the implants had to change first

FT03 passed. FT03 trained reconstruction and segmentation jointly on mixed batches, from FT02c with FT01's head and segmentation weight 0.1, 10 epochs, 3.0 hours. All three criteria were met: water / cloud IoU 0.841 / 0.815 (at or above FT01's 0.82 / 0.80), validation SAM within 0.1 degree of FT02c (3.92 / 4.21 / 5.48), and detection unchanged (PRISMA pAUC 0.158 against FT02c's 0.159) [M: `research/experiments/indradhanu-finetune/FT03-joint/README.md`]. Joint training changed detection by nothing measurable, as expected with a segmentation weight of 0.1.

The head's first job: exclusion. Class-aware scoring - a separate "normal" per surface class - was tried and not shipped: it removed cloud false alarms but water's tight statistics made water noise dominate (set1: water's share of false alarms 14% to 92%; PRISMA pAUC 0.158 to 0.134). The user's rule settled it: "we don't detect anything on water and cloud." So the head marks them and the scorer sets them to 0 after scoring (`exclude` in `app/evaluation/scorers.py`). The mask comes from FT03, but it can be applied to *every* model's scores.

Table 38.7: PRISMA bench, residual Mahalanobis, with and without water + cloud exclusion on FT03's masks.

model	caught top 0.1%, plain	excluded	pAUC, plain	excluded
v0.2.0	14.1%	15.3%	0.159	0.165
F001 ep109	13.2%	14.6%	0.148	0.155
FT02c ep10	14.4%	16.1%	0.159	0.167
FT03 ep10	14.3%	16.5%	0.158	0.165

Source: [M: `C:/es/eval/stage3/report.md`] (the `exclude(water+cloud;mask)` rows); summarised in [M: `docs/lld/evaluation.md`].

The pAUC gains are $0.165 - 0.159 = 0.006$, $0.155 - 0.148 = 0.007$, $0.167 - 0.159 = 0.008$ and $0.165 - 0.158 = 0.007$ - between 0.006 and 0.008 for every model, including v0.2.0, which has no segmentation head and was never fine-tuned. The gain belongs to the mask, not to any model's reconstruction. Two costs: ROC-AUC falls, and 39 of the 909 real objects lie wholly

inside FT03's water / cloud mask. Checked later: 13 of set1's 27 are single pixels more than 3 px offshore - likely vessels. The user then put *vessels on water in scope: only cloud is excluded* (Chapter 35). FT03 with cloud only: pAUC 0.158 to 0.166.

The model-free detector, and what was wrong with bench-v1. The stage-3 run also scored PCA-RX: classical local RX on the cube's top 30 principal components, no Indradhanu at all. On bench-v1 it caught 30.2% at the top 1% against FT03's 26.9%; on the real objects its TPR at 0.1% was 0.029 against FT03's 0.058 - half. bench-v1 rewarded something PCA-RX has more of than any model. A diagnosis compared the real objects with bench-v1's implants on two numbers, both starting from $\mathbf{d}$, an object's mean spectrum minus its ring's:

- **Foreign fraction**: the share of $\|\mathbf{d}\|^2$ lying outside the scene's 10 main spectral directions. Real 0.006, bench-v1 0.042 (easy tier: real 0.002, bench-v1 0.029).
- **Halo over object contrast**: the contrast of the one-pixel ring touching the object, over the object's own. Real 0.74, bench-v1 0.60 (easy: 0.70 against 0.38). Real objects bleed into their neighbours; implants stopped dead at their edge.

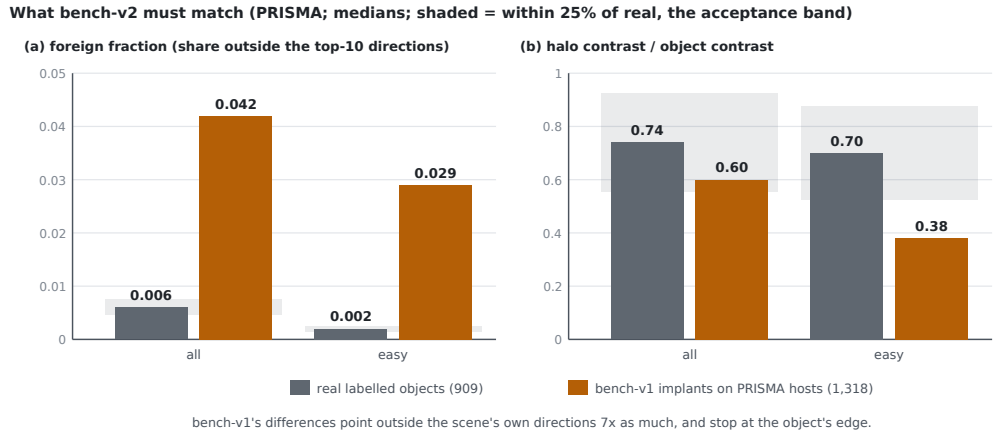


Figure 38.5: Medians on PRISMA, real objects (grey) against bench-v1 implants (amber), overall and in the easy tier; the shaded band is within 25% of the real value.

Matching foreignness, derived. bench-v2's fix rescales the part of $\mathbf{d}$ outside the scene's subspace so that the foreign fraction hits a target f, keeping the inside part (`match_foreignness` in `app/evaluation/implant.py`). Split $\mathbf{d} = \mathbf{i} + \mathbf{o}$ into its inside and outside parts; they are perpendicular, so by Pythagoras $\|\mathbf{d}\|^2 = \|\mathbf{i}\|^2 + \|\mathbf{o}\|^2$. Scale the outside part by β and demand the target share:

$$\frac{\beta^2 \|\mathbf{o}\|^2}{\|\mathbf{i}\|^2 + \beta^2 \|\mathbf{o}\|^2} = f. \tag{38.10}$$

Multiply both sides by the denominator:

$$\beta^2 \|\mathbf{o}\|^2 = f \, \|\mathbf{i}\|^2 + f \, \beta^2 \|\mathbf{o}\|^2 . \tag{38.11}$$

Collect the β^2 terms on the left and factor:

$$\beta^2 \|\mathbf{o}\|^2 (1 - f) = f \, \|\mathbf{i}\|^2 . \tag{38.12}$$

Divide by $\|\mathbf{o}\|^2 (1 - f)$:

$$\beta^2 = \frac{f}{1 - f} \cdot \frac{\|\mathbf{i}\|^2}{\|\mathbf{o}\|^2}. \tag{38.13}$$

Worked example 38.6: The repo's `match_foreignness` test case

Scene direction = band 0, $\mathbf{d} = (3, 4, 0)$, target $f = 0.1$. Inside part $\mathbf{i} = (3, 0, 0)$ with $\|\mathbf{i}\|^2 = 9$; outside part $\mathbf{o} = (0, 4, 0)$ with $\|\mathbf{o}\|^2 = 16$.

$$\beta^2 = \frac{0.1}{0.9} \times \frac{9}{16} = 0.1111 \times 0.5625 = 0.0625$$
$$\beta = \sqrt{0.0625} = 0.25$$
$$\mathbf{d}' = (3,\ 0.25 \times 4,\ 0) = (3, 1, 0)$$

Check: outside $1^2 = 1$; total $9 + 1 = 10$; share $1/10 = 0.1$. (Script: $\beta = 0.25$ [C].)

The acceptance rule, fixed before the build, asked that foreignness and halo medians lie within 25% of the real objects', overall and per tier. For the overall foreign fraction that band is $0.006 \times 0.75 = 0.0045$ to $0.006 \times 1.25 = 0.0075$; bench-v1's 0.042 fails. For the easy-tier halo, $0.70 \times 0.75 = 0.525$ to $0.70 \times 1.25 = 0.875$; bench-v1's 0.38 fails, even though its overall halo (0.60, against a band of 0.555 to 0.925) passes. That is why the test is per tier. bench-v2 was built and accepted the same day; Chapter 34 teaches it.

No model was trained on detection until the implants looked like real objects: the cheapest way to lower a loss that rewards finding bench-v1's implants is to become a better PCA-RX, which the real bench says is the wrong direction.

38.7 FT05: implant-denoising

With bench-v2 accepted, option 3 ran in a new form on 2026-10-17: keep the model and the score, put anomalies into the *input*, and keep the clean tile as the *target*.

38.7.1 Why the old target works against detection

Every earlier run used the input as the target: hide most tokens, reconstruct them, compare with what was there. Anomalies in the training tiles are in the target too. An object smaller than a token is invisible to its own reconstruction, but an object larger than a token, or one with a blurred halo, leaks into the visible neighbouring tokens. A model trained on input targets is rewarded for carrying that leak into its prediction.

Worked example 38.7: Which prediction does each target reward?

One band of one pixel. The background under it would be 0.20; an object makes it 0.50; the model sees part of the object in a neighbouring token and can predict either value. The loss is L1.

	predict 0.20	predict 0.50
input target 0.50	$\|0.20 - 0.50\| = 0.30$	$\|0.50 - 0.50\| = 0 \leftarrow$ rewarded
clean target 0.20	$\|0.20 - 0.20\| = 0 \leftarrow$ rewarded	$\|0.50 - 0.20\| = 0.30$
score	$\|0.50 - 0.20\| = 0.30$ (found)	$\|0.50 - 0.50\| = 0$ (missed)

With the input as target, the loss prefers the prediction that hides the object from the score.

With the clean tile as target, it prefers the prediction that exposes it. FT05 changes only the target, and that flips which prediction the loss rewards.

38.7.2 The implants and the weighted loss

`implant_sample` (`app/foundation_models/trainers/finetune_trainer.py`) runs in the data-loader workers on every *training* tile: with probability 0.5 it writes 1 to 4 bench-v2 implants (`implant_tile`, the per-tile twin of the bench builder), keeps the clean tile as `clean.npy`, and marks every pixel within 3 px of an implant as *near*. The generator is seeded from the tile's key, so a tile gets the same implants every epoch. Implants come from two sources, `signature` and `material`; bench-v2's third source, `host`, is held out so that its row of bench-v2 is an out-of-recipe check.

In `_mixed_loss` the model reads the implanted tile and is graded against the clean one, and the loss mask is re-weighted:

```
loss_mask = hidden_pixels * eroded_valid       # (B, 1, H, W), as in every run
# near: (B, 1, H, W), 1 within 3 px of an implant; implant_near_weight = 5
loss_mask = loss_mask * (1.0 + (implant_near_weight - 1.0) * near)
```

How many pixels are near? `binary_dilation(gt>0,iterations=3)` grows the implant by the default cross-shaped structuring element three times. Each pass adds the four direct neighbours of every marked pixel, so after r passes a single pixel has grown into a diamond: every offset $(\Delta y, \Delta x)$ with $|\Delta y| + |\Delta x| \le r$. Count it ring by ring: the ring at distance $k \ge 1$ has $4k$ pixels (one per unit of k along each of the four sides), so

$$|\text{diamond}_r| = 1 + \sum_{k=1}^{r} 4k = 1 + 4 \cdot \frac{r(r+1)}{2} = 1 + 2r(r+1). \tag{38.14}$$

For $r = 3$: $1 + 2 \times 3 \times 4 = 1 + 24 = 25$, which is $1 + 4 + 8 + 12$. SciPy agrees: 25 for a one-pixel implant and 32 for a two-pixel one [C].

Why weight them, and what the weight does. `_masked_l1_sam` divides by the sum of the mask, so the loss is a *weighted mean* over pixels. If N pixels count, n of them near an implant with weight w and the rest with weight 1, the near pixels' share of the loss's weight is

$$\text{share}(w) = \frac{w\,n}{(N-n) + w\,n}. \tag{38.15}$$

Worked example 38.8: Times five moves 0.15% to 0.76%

A 128×128 tile has 16,384 pixels; only the hidden ones count (mask ratio 0.65). One one-pixel implant has 25 near pixels.

$$\begin{aligned} N &= 0.65 \times 16{,}384 = 10{,}650 \quad \text{(hidden pixels)} \\ n &= 0.65 \times 25 = 16.25 \quad \text{(hidden near pixels, on average)} \\ \text{share}(1) &= \frac{16.25}{10{,}650} = 0.0015 \quad (0.15\%) \\ w\,n &= 5 \times 16.25 = 81.25 \\ (N-n) + w\,n &= 10{,}650 - 16.25 + 81.25 = 10{,}715 \\ \text{share}(5) &= \frac{81.25}{10{,}715} = 0.0076 \quad (0.76\%) \end{aligned}$$

The weight changes *shares*, not the loss's overall size, because the denominator grows too. The rest of the tile, and the clean half of the data, keep teaching plain reconstruction. (Script: 0.00153 and 0.00758 [C].)

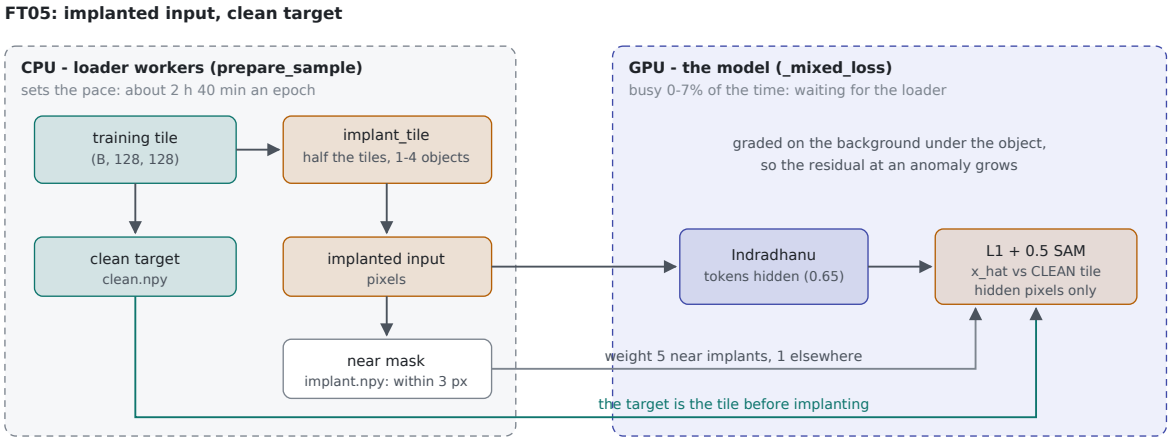


Figure 38.6: One FT05 step. The loader workers (CPU) implant half the tiles and keep the clean copy; the model (GPU) reads the implanted tile and is graded against the clean one, $\times 5$ near the implants. The CPU side sets the pace.

38.7.3 The bill: a data loader that could not keep up

FT03 ran 10 epochs in 3.0 hours, about $3.0 \times 60/10 = 18$ minutes an epoch. FT05 ran at about 2 h 40 min = 160 minutes an epoch, with the GPU 0–7% busy [M: `research/experiments/indradhanu-finetune/FT05-implant-denoise/README.md`]. The ratio is $160/18 = 8.9$; since the GPU's work per epoch did not change, it could be busy at most $18/160 = 0.11$ of the time. Ten epochs would take $10 \times 160 = 1{,}600$ minutes $= 26.7$ hours. All the implant work is NumPy on the CPU, inside the loader workers.

One speed fix went in before the relaunch: the background subspace of each tile was computed by an SVD of up to 100,000 pixels, and commit `558ff5e` replaced it by the eigenvectors of the bands-by-bands covariance - "the same subspace", 0.25 s to about 0.02 s per tile. Why the same?

Worked example 38.9: Right singular vectors are eigenvectors of $\mathbf{X}^\top\mathbf{X}$

Let $\mathbf{X}$ be the centred pixels, N rows (pixels) by B columns (bands), with singular value decomposition $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ ($\mathbf{U}^\top\mathbf{U} = \mathbf{I}$, $\mathbf{V}^\top\mathbf{V} = \mathbf{I}$, $\mathbf{S}$ diagonal). Form $\mathbf{X}^\top\mathbf{X}$, using the transpose of a product $(\mathbf{A}\mathbf{B})^\top = \mathbf{B}^\top\mathbf{A}^\top$:

$$\mathbf{X}^\top\mathbf{X} = (\mathbf{U}\mathbf{S}\mathbf{V}^\top)^\top(\mathbf{U}\mathbf{S}\mathbf{V}^\top) = \mathbf{V}\mathbf{S}\mathbf{U}^\top\mathbf{U}\mathbf{S}\mathbf{V}^\top = \mathbf{V}\mathbf{S}^2\mathbf{V}^\top.$$

That is an eigendecomposition: the columns of $\mathbf{V}$ are eigenvectors of the $B \times B$ matrix $\mathbf{X}^\top\mathbf{X}$, with eigenvalues the squared singular values. For a tile, $N = 16{,}384$ and $B = 224$, so the work moves from a $16{,}384 \times 224$ matrix to a 224×224 one.
Check on paper with $N = 4$, $B = 2$: rows $(1, 1)$, $(-1, -1)$, $(2, -2)$, $(-2, 2)$ (each column

sums to 0, so centred).

$$(\mathbf{X}^\top\mathbf{X})_{11} = 1^2 + (-1)^2 + 2^2 + (-2)^2 = 1 + 1 + 4 + 4 = 10$$
$$(\mathbf{X}^\top\mathbf{X})_{12} = 1\cdot 1 + (-1)(-1) + 2(-2) + (-2)\cdot 2 = 1 + 1 - 4 - 4 = -6$$
$$(\mathbf{X}^\top\mathbf{X})_{22} = 1 + 1 + 4 + 4 = 10$$

$\left(\begin{smallmatrix} 10 & -6 \\ -6 & 10 \end{smallmatrix}\right)$ times $(1,-1)$ gives $(16,-16) = 16\,(1,-1)$, and times $(1,1)$ gives $(4,4) = 4\,(1,1)$: eigenvalues 16 and 4. Their square roots, 4 and 2, are the singular values of $\mathbf{X}$ (script check [C]).

The loader was still CPU-bound after that fix. The cause, found on 2026-10-18 and recorded in `single_thread_worker`'s docstring: "18 workers (3 sources × 6) on 8 vCPUs oversubscribed the CPU about 8-fold; the implant transform ran 4.7 s per tile instead of 1.8 s" - every worker process had its own pool of maths-library (BLAS) threads. With one thread per worker and 2 workers per source, FT05 resumed at epoch 6 at 43 minutes an epoch: 161/43 = 3.7 times faster. FT04 inherited the setting.

38.7.4 The verdict

Pass criteria, fixed before launch: (1) the real PRISMA bench decides - not below FT03 on pAUC to 1%, TPR at 0.1% and caught top 0.1%, and above on at least two of the three; (2) bench-v2 above FT03, with the held-out `host` row not falling; (3) validation SAM within 0.1 degree of FT03 (3.92 / 4.21 / 5.48) and water / cloud IoU within 0.02.

Worked example 38.10: Criterion 1 at epoch 5

FT03 against FT05 epoch 5 on the real bench (`C:/es/eval/ft05_ep5/report.md`):

pAUC:	$0.15801 - 0.15805 = -0.00004$	(below, in the fifth decimal)
TPR at 0.1%:	$0.0594 - 0.0582 = +0.0012$	(above)
caught top 0.1%:	$130 - 130 = 0$	(equal)

Above on 1 of 3: criterion 1 not met. bench-v2 rose (32.3% to 32.8%; pAUC 0.164 to 0.167) and the held-out `host` row held (29.4% to 29.6%) - the pattern the pre-launch note had called "optimistic".

FT05 finished at epoch 10 on 2026-10-18; best.pt stayed at epoch 5 (lowest validation loss), whose detection is the tie above. Epoch 10 then failed criterion 3: AVIRIS-NG SAM 5.67 against FT03's 5.48, +0.19 against a limit of 0.1. On bench-v2.1 (Chapter 37) epoch 5 caught 35.4% against FT03's 34.9% - a tie. Verdict: **not passed**.

Source check: half of FT05's implants were garbled

Every `signature` implant FT05 trained on - half of them - carried the descending-wavelength `np.interp` bug of Chapter 37: each arrived as a flat step, though still at real contrast and foreignness. The bug was fixed (`b94dbfd`) after FT05's launch. The FT05 README calls a rerun with correct signatures "the fair test" and adds that "nothing here suggests it would flip the result". That rerun has not been made.

38.8 The scoreboard

Every model on every current benchmark fits in three tables. They read correctly only with chance and the model-free PCA-RX row beside them.

Table 38.8: The real PRISMA bench: 909 labelled objects in 4 scenes. Chance: 10.2% at the top 1%, 1.1% at the top 0.1%. "Cloud excluded" zeroes the model's own cloud mask after scoring.

model	caught top 1%	top 0.1%	pAUC to 1%	TPR at 0.1%	ROC-AUC
v0.2.0 (shipped)	44.2%	14.1%	0.159	0.058	0.854
F001 ep109	44.0%	13.2%	0.148	0.056	0.861
FT02c ep10	45.4%	14.4%	0.159	0.059	0.855
FT03 ep10	45.4%	14.3%	0.158	0.058	0.853
FT05 ep5	44.8%	14.3%	0.158	0.059	0.853
FT04 ep10	45.7%	14.9%	0.162	0.061	0.853
FT03, cloud excluded	46.1%	15.8%	0.166	0.064	0.850
FT04, cloud excluded	46.0%	15.8%	**0.170**	**0.065**	0.852
PCA-RX (no model)	39.7%	8.1%	0.128	0.029	0.839

Sources: [M: `C:/es/eval/stage3/report.md`], [M: `C:/es/eval/ft04/report.md`], [M: `research/experiments/indradhanu-finetune/FT04-final-joint/README.md`].

Table 38.9: Reconstruction (validation SAM, degrees; lower is better) and EnMAP test IoU for the fine-tuning runs. Earlier models were validated differently and are left out.

	SAM (deg)			IoU		
run	EnMAP	PRISMA	AVIRIS-NG	water	cloud	shadow
FT01 (head only)	–	–	–	0.82	0.80	0.20
FT02b ep9	3.99	4.37	5.57	–	–	–
FT02c ep10	3.93	4.21	5.41	–	–	–
FT03 ep10	3.92	4.21	5.48	0.841	0.815	0.205
FT05 ep5 (best.pt)	3.94	4.20	5.33	0.838	0.818	–
FT05 ep10 (last.pt)	3.90	4.20	**5.67**	0.844	0.814	–
FT04 ep10	**3.88**	**4.11**	5.46	0.839	0.810	**0.223**

Source: the run READMEs, [M: `research/experiments/indradhanu-finetune/`].

Values that will drift

These tables are the board of 2026-10-19 (FT04 added after its run on 2026-10-18). Later models - FT06 to FT09 and Indradhanu Heavy (Chapters 41 to 43) - are scored on the same benches and are not shown.

What each step bought, and how big noise is. Take bench-v2.1's caught-at-top-1% apart step by step, with the rough ruler of Chapter 37 (SE ≈ 0.9 points for $p = 0.35$, $n = 2{,}817$):

step	caught	change	reading
v0.2.0 → F001	35.5 → 33.1	−2.4 (2.7 SE)	pretraining alone: slightly worse
F001 → FT02c	33.1 → 35.1	+2.0 (2.2 SE)	three real sensors, mixed batches
FT02c → FT03	35.1 → 34.9	−0.2	joint training: a tie
FT03 → FT04	34.9 → 35.1	+0.2	pseudo-labels, rare-class weights: a tie
FT04 → + cloud	35.1 → 36.2	+1.1	a mask, not a model
FT03 → FT05 ep5	34.9 → 35.4	+0.5	implant-denoising: a tie

Read the two large steps together: pretraining alone cost a little on both benches (real-bench

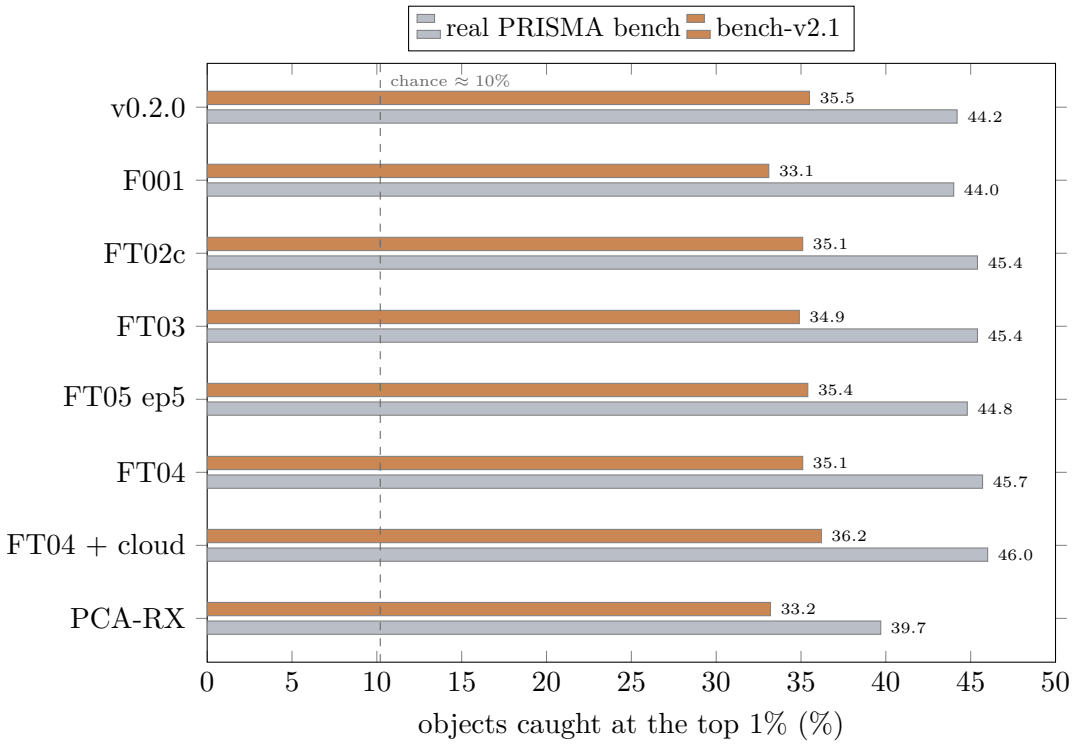


Figure 38.7: Share of objects caught at the top 1% on the real bench (grey) and bench-v2.1 (amber), drawn to scale from Table 38.8 and Table 37.2. Every model is far above chance and above PCA-RX; the models tie each other.

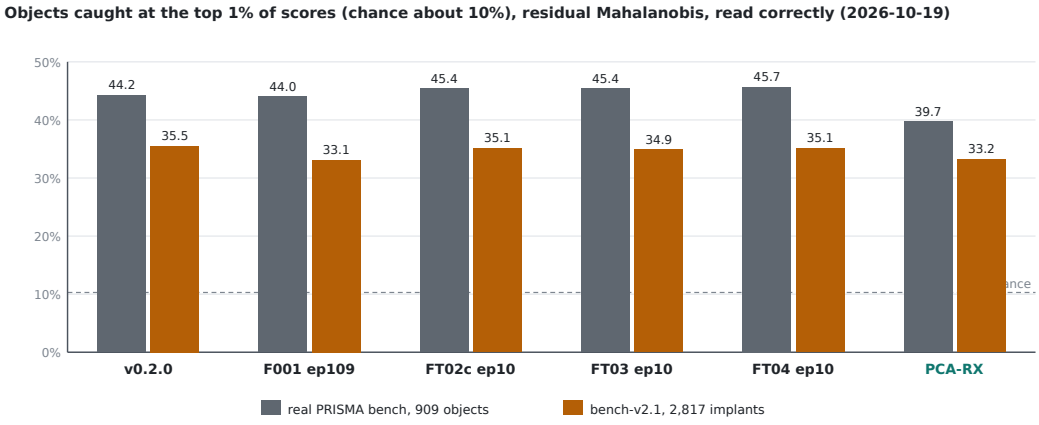


Figure 38.8: The course's version of the same board: share caught at the top 1% for each model and PCA-RX, on the real bench (grey) and bench-v2.1 (amber).

pAUC 0.159 to 0.148), and fine-tuning on the real sensors won it back - to where v0.2.0 already was.

38.9 FT04: pseudo-labels, rare classes and the release candidate

FT03's segmentation head learned only from EnMAP's labels. PRISMA and AVIRIS-NG tiles went through the segmentation pass, but no loss read their outputs. So the head that would mask clouds on a PRISMA scene had never been graded on one. FT04, the "final joint" run, fixed that with pseudo-labels and weighted the rare classes; it ran on 2026-10-18 from FT03's weights for 10 epochs and passed all three of its criteria.

38.9.1 Keep only the confident calls

A *pseudo-label* is a label made by a model rather than a person. The teacher is FT03; the student is FT04. `scripts/pseudo_label.py` runs FT03's head over every PRISMA and AVIRIS-NG training tile and stores, per tile, a `(5, 128, 128)` array of bytes, round$(255 \times \sigma(z))$ for each class's logit z. A probability of 0.93 is stored as round(255×0.93) = round$(237.15) = 237$. The raw probabilities are kept on purpose, so a threshold can change without re-running the teacher.

`pseudo_targets` (`app/utils/segmentation/targets.py`) turns them into *targets* (0 or 1 per class and pixel) and *known* (whether that pixel enters the loss):

```
PSEUDO_CONF = 0.9              # counts only where p > 0.9 or p < 0.1
p = probs.astype(np.float32) / 255.0                        # (5, 128, 128)
targets = (p > conf).astype(np.float32)
known = ((p > conf) | (p < 1 - conf)) & valid[None]   # valid: enough finite bands
known[2:] = False                             # thin cloud, shadow, snow: not taught
not_water = (targets[0] > 0) & (band_mean(refl, wl, 800, 900) > 0.15)
targets[0][not_water] = 0.0                # still known: a known NOT-water
```

In stored units, $0.9 \times 255 = 229.5$ and $0.1 \times 255 = 25.5$: "yes" needs a byte of 230 or more, "no" a byte of 25 or less. Only water and cloud are taught; the other classes stay unknown on PRISMA and AVIRIS-NG because FT01 to FT03 never got them good enough to copy.

The water guard. Water is dark in the near-infrared (NIR). A pixel the teacher calls water with a mean 800–900 nm reflectance above 0.15 is land-bright, so it becomes a *known not-water* - taught "no", not left out. The 0.15 is looser than the implant builder's water rule (NIR < 0.08): it vetoes only calls that are far too bright for water. The source records the value (the config's description: "water guarded by NIR ≤ 0.15") but not how it was chosen.

Worked example 38.11: Four pixels through `pseudo_targets`

Stored water bytes and NIR means for a 2×2 tile. Divide each byte by 255, compare with 0.9 and 0.1, then apply the guard.

pixel	byte	p = byte/255	NIR	decision
A	240	0.941	0.04	> 0.9, NIR $\leq 0.15 \rightarrow$ target 1, known
B	128	0.502	0.05	between 0.1 and 0.9 $\rightarrow$ unknown
C	13	0.051	0.30	$< 0.1 \rightarrow$ target 0, known
D	235	0.922	0.22	> 0.9 but NIR $> 0.15 \rightarrow$ target 0, known (not-water)

Three of the four pixels teach; one says "water", two say "not water"; one is left out. Pixel D is the guard at work. A check before launch, on 150 EnMAP tiles against EnMAP's own labels, gave the guarded pseudo-label water IoU 0.875, and the teacher's raw confident

cloud 0.82 [M: `research/experiments/indradhanu-finetune/FT04-final-joint/README.md`].

Why there is no cloud guard. A cloud guard, `cloud_plausible`, was built and dropped. It required a mean 450–650 nm reflectance above 0.2 *and* spectral flatness: the minimum of four window means (460–500, 540–580, 640–680, 840–880 nm) over their maximum at least 0.7. It rejected 69% of EnMAP's labelled clouds on a test shard, and the error it targeted - "bright desert called cloud" on set4 - was not there: set4's "11% cloud" is a cloud / smoke streak and haze. Haze is not flat: with window means 0.30, 0.27, 0.24, 0.18 (falling, because haze scatters blue most), min / max $= 0.18/0.30 = 0.60 < 0.7$, so it is rejected, though its visible mean $(0.30 + 0.27 + 0.24)/3 = 0.27$ is bright enough. A guard that throws out most of the true class costs more than the error it blocks.

38.9.2 Rare classes, weighted three times

`seg_class_weights` is $[1, 1, 1, 3, 3]$ for water, cloud, thin cloud, shadow and snow. Recall the loss of Chapter 28. For one class with predicted probability p and target $y \in \{0, 1\}$, the binary cross-entropy is the negative log of the probability the model gave to the right answer:

$$\ell(y, p) = -\big(y \ln p + (1 - y) \ln(1 - p)\big). \tag{38.16}$$

(If $y = 1$ the second term vanishes and $\ell = -\ln p$; if $y = 0$ the first vanishes and $\ell = -\ln(1-p)$.) `masked_bce_dice` averages each class's BCE over its known pixels, adds a soft Dice term, and then combines the classes with a weighted mean over the classes H that have any known pixel in the batch:

$$L = \frac{\sum_{c \in H} w_c \, L_c}{\sum_{c \in H} w_c}. \tag{38.17}$$

In the code: `return(w*per)[has].sum()/w[has].sum()`.

Worked example 38.12: Weighted BCE on a batch with no snow

To keep the arithmetic short, use the BCE part only (the Dice term is added to each L_c in the same way and is weighted identically). Each class has two known pixels; snow has none.

$$\begin{aligned}
\text{water:}\quad & y = 1, p = 0.9 : -\ln 0.9 = 0.1054; \quad y = 0, p = 0.2 : -\ln 0.8 = 0.2231; \\
& L_{\text{water}} = (0.1054 + 0.2231)/2 = 0.1643 \\
\text{cloud:}\quad & y = 1, p = 0.7 : -\ln 0.7 = 0.3567; \quad y = 0, p = 0.1 : -\ln 0.9 = 0.1054; \\
& L_{\text{cloud}} = (0.3567 + 0.1054)/2 = 0.2310 \\
\text{thin cloud:}\quad & y = 0, p = 0.4 : -\ln 0.6 = 0.5108; \quad y = 1, p = 0.6 : -\ln 0.6 = 0.5108; \\
& L_{\text{thin}} = 0.5108 \\
\text{shadow:}\quad & y = 1, p = 0.3 : -\ln 0.3 = 1.2040; \quad y = 0, p = 0.2 : -\ln 0.8 = 0.2231; \\
& L_{\text{shadow}} = (1.2040 + 0.2231)/2 = 0.7136
\end{aligned}$$

Snow has no known pixel, so it drops out of numerator and denominator.

$$\begin{aligned}
\text{unweighted:}\quad & (0.1643 + 0.2310 + 0.5108 + 0.7136)/4 = 1.6197/4 = 0.4049 \\
\text{weighted:}\quad & (0.1643 + 0.2310 + 0.5108 + 3 \times 0.7136)/(1 + 1 + 1 + 3) \\
& = (0.9061 + 2.1408)/6 = 3.0469/6 = 0.5078
\end{aligned}$$

(Script: 0.4049 and 0.5078 [C].) The total moved only from 0.40 to 0.51. What moved a lot is where the gradient goes: differentiate (38.17) with respect to one class's loss,

$$\frac{\partial L}{\partial L_c} = \frac{w_c}{\sum_{h \in H} w_h},$$

so shadow's share of every gradient step is $1/4 = 0.25$ unweighted and $3/6 = 0.50$ weighted - doubled.

On a PRISMA or AVIRIS-NG sub-batch only water and cloud are known, both weight 1, so the weights act on EnMAP's batches only. Result: shadow IoU 0.205 to 0.223.

38.9.3 Results, and the 0.159 to 0.170 decomposition

Worked example 38.13: FT04's criteria

1. Masks: EnMAP IoU not below FT03 by more than 0.01. Water $0.839 - 0.841 = -0.002$; cloud $0.810 - 0.815 = -0.005$. Both within 0.01: met.
3. Reconstruction: SAM within 0.1 degree of FT03. EnMAP $3.88 - 3.92 = -0.04$; PRISMA $4.11 - 4.21 = -0.10$; AVIRIS-NG $5.46 - 5.48 = -0.02$. All better: met.
2. Detection: default not below FT03, and cloud exclusion on FT04's masks at least as good as on FT03's. Default: pAUC 0.162 against 0.158, TPR at 0.1% 0.061 against 0.058, caught top 0.1% 14.9% against 14.3% - at or above on all three. Exclusion: 0.170 with FT04's masks against 0.166 with FT03's. Met.

The run also swapped the masks between the two models, which turns two numbers into a small experiment. A *two-by-two factorial* design varies two factors - here the reconstruction model and the mask source - in every combination, so each factor's effect can be read with the other held fixed.

Table 38.10: Real-bench pAUC to 1% with cloud exclusion: every combination of model and mask.

	FT03's masks	FT04's masks
FT03's reconstruction	0.166	0.165
FT04's reconstruction	0.170	0.170

Source: [M: `C:/es/eval/ft04/report.md`].

Worked example 38.14: Main effects of model and mask

The *main effect* of a factor is the average over the other factor of the change it causes. Model (FT03 to FT04), averaged over both masks:

$$\frac{0.170 + 0.170}{2} - \frac{0.166 + 0.165}{2} = 0.170 - 0.1655 = +0.0045.$$

Masks (FT03's to FT04's), averaged over both models:

$$\frac{0.165 + 0.170}{2} - \frac{0.166 + 0.170}{2} = 0.1675 - 0.168 = -0.0005.$$

The masks are interchangeable (FT04's exclude less: ROC-AUC 0.852 against 0.850, at no cost at the top). The step under exclusion is FT04's reconstruction, not its masks. (Script: $+0.0045$ and -0.0005 [C].)

Now take apart the real-bench gain of the release candidate over what shipped:

shipped v0.2.0, default	0.159	
FT04, default	0.162	model: $0.162 - 0.159 = +0.003$
FT04 + cloud exclusion	0.170	exclusion: $0.170 - 0.162 = +0.008$
total	$+0.011$	exclusion share $0.008/0.011 = 0.73$

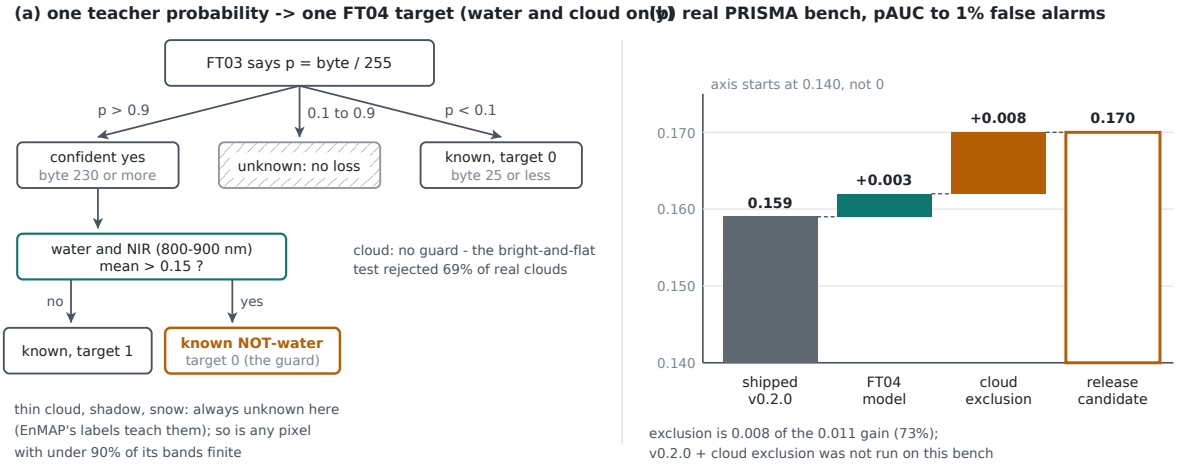


Figure 38.9: (a) How one teacher probability becomes a target: taught "yes" above 0.9, "no" below 0.1, nothing between, and a confident "water" over a bright NIR turned into a known not-water. (b) The real-bench pAUC from the shipped v0.2.0 to the release candidate: the model adds 0.003, cloud exclusion 0.008.

And the comparison is not like for like: v0.2.0 was scored without exclusion. The like-for-like run was made on 2026-10-19 (`configs/eval/release_like_for_like.json`): both with cloud exclusion on FT04's masks, v0.2.0 scored 0.165 and FT04 0.170 on the real bench; on bench-v2.1 v0.2.0 + exclusion is slightly ahead (36.7% against 36.2%) [M: `research/experiments/indradhanu-finetune/FT04-final-joint/README.md`]. A detection tie.

38.9.4 Why better reconstruction does not become better detection

The log's conclusion of 2026-10-18: "Residual Mahalanobis absorbs what better reconstruction changes." Here is the mechanism, as mathematics. Residual Mahalanobis scores a pixel's residual $\mathbf{r}$ by its squared Mahalanobis distance in units of the background residual's covariance $\mathbf{\Sigma}$ (Chapter 31), $s = \mathbf{r}^\top \mathbf{\Sigma}^{-1} \mathbf{r}$ (after subtracting the local mean). Suppose a better model shrinks *every* residual by the same factor c, so $\mathbf{r} \to c\,\mathbf{r}$. The covariance is fitted on those residuals, and scaling every vector by c scales every product of two components by c^2, so $\mathbf{\Sigma} \to c^2 \mathbf{\Sigma}$ and $\mathbf{\Sigma}^{-1} \to c^{-2} \mathbf{\Sigma}^{-1}$. Then

$$\begin{aligned} s' &= (c\,\mathbf{r})^\top (c^2 \mathbf{\Sigma})^{-1} (c\,\mathbf{r}) \\ &= c \cdot c^{-2} \cdot c\ \mathbf{r}^\top \mathbf{\Sigma}^{-1} \mathbf{r} \\ &= \mathbf{r}^\top \mathbf{\Sigma}^{-1} \mathbf{r} = s. \end{aligned}$$

The score does not move. In one dimension, with a background spread of 0.010 and a target residual of 0.050 the target sits $0.050/0.010 = 5.0$ spreads out; a better model with spread 0.008 and target residual 0.040 gives $0.040/0.008 = 5.0$ spreads - the same [A]. Only a model that improves on the background *and not* on the targets raises the score. That is exactly what FT05 tried to teach, without effect.

What the pretraining and FT01 to FT05 bought: masks (cloud exclusion, about a point for any model), native bands, better reconstruction and a benchmark you can trust - not detection by the model alone.

FT04 + cloud exclusion on its own masks became the release candidate, and shipped as Indradhanu V2 - the next chapter.

38.10 What the runs taught

1. **A class that stays flat while all others climb is a label ceiling** (FT01, thin cloud).
2. **Read a gate where you said you would**, and expect reconstruction and detection to part company (FT02).
3. **Count bands before crediting weights**: a growing trained-wavelength record changes what the model reads (FT02).
4. **The score map cannot tell a boundary from a target**; a loss under the blind spot could, but the decoder's 4×4 bottleneck blocked it (FT02c).
5. **A shelved option is shelved for a reason**; write the reason down so the next person need not re-argue it.
6. **Fix the benchmark before you train on it** (stage 3, bench-v1 against PCA-RX).
7. **The target decides what the loss rewards** - and a weight changes shares, not totals (FT05).
8. **The CPU can be the bottleneck of a GPU run** (FT05's loader).
9. **A mask can lift every model alike; credit the mask, not the model** (exclusion; FT04's swap).
10. **A scale-invariant score absorbs uniform improvements** (residual Mahalanobis).

38.11 Common confusions

FT02 forgot how to detect.
Not shown. Reconstruction improved on every sensor, and the bench moved 0.862 to 0.855, smaller than some single-scene swings. "Forgetting" would predict worse reconstruction on what F001 knew; EnMAP, the closest sensor to SpectralEarth, got better.

Epoch 1 passed the gate, so FT02 passed.
No. The gate names the checkpoint: the best by validation reconstruction. Choosing a checkpoint by the bench would turn the bench into training data.

Edges are false alarms, so suppress edges.
Edges are the main false alarm and a common shape of a true target. Suppressing them costs both (0.86 to 0.67).

With a clean target the model never sees anomalies.
It sees them in every implanted input. It is graded on what was underneath.

The x5 near weight makes the loss five times bigger.
The weighted mean divides by the weighted count. x5 shifts the near pixels' share from 0.15% to 0.76%; the total stays a mean.

Unknown means not-water.
No. Unknown pixels never enter the loss. A known not-water (pixel D) is taught as 0.

The 3x class weight makes the loss three times bigger.
It is a weighted mean: the denominator grows too (6 instead of 4). It doubles shadow's share of the gradient.

Cloud exclusion is FT04's improvement.
The mask is FT04's head, but the gain is the mask's: it lifts every model alike, v0.2.0 included, and FT03 scores the same with either head's masks.

FT04 is the best detector.
On the real bench by 0.003 pAUC without exclusion, and like for like with exclusion it ties v0.2.0. It is the release candidate because its masks, reconstruction and native bands are the best.

38.12 Check yourself

Exercise 38.1. The segmentation head of a smaller model has encoder stages of 16, 32, 80 and 128 channels, decoder width 64 and 5 classes, with the same layers as worked example 38.1. Count its parameters by hand.

Exercise 38.2. Compute the edge strength (38.4) and the central-difference gradient magnitude (38.3) of the right-hand centre pixel (0.30) of the patch in worked example 38.3. Its neighbours inside the patch are up 0.30, down 0.31, left 0.12 (treat the missing right neighbour as equal to the pixel itself for the central difference).

Exercise 38.3. Redo worked example 38.5 for a reconstruction $0.10, 0.10, 0.45, 0.50$. Give the value L1 mean and the gradient L1 mean.

Exercise 38.4. A two-pixel implant has 32 pixels within 3 px. With mask ratio 0.65, 10,650 hidden pixels and weight 5, what share of a tile's loss weight do its near pixels carry? What weight would give them a 2% share?

Exercise 38.5. Stored water bytes 199, 231, 24 and 26 for four pixels with NIR 0.05, 0.18, 0.30, 0.04. Give each pixel's water target and known flag.

Exercise 38.6. Per-class losses water 0.2, cloud 0.2, thin cloud 0.4, shadow 0.6, no known snow. Compute the unweighted and the $[1, 1, 1, 3, 3]$-weighted loss, and shadow's share of the gradient in each case.

Exercise 38.7. Show that the teacher of Section 38.5 closes 90% of its gap to a fixed student in about 2,300 steps.

Next: Chapter 39, Shipping Indradhanu V2.

Chapter 39

Shipping Indradhanu V2

The one thing this chapter teaches

Putting FT04 into the product took six small decisions, each forced by something the product already did: a new codename because codename routing is last-wins; a manifest that carries the whole model configuration because the api cannot load a checkpoint; a band-grid check because one band filter feeds every model in an action; `read_bands` because the model must be told which wavelengths it reads; cloud zeroed *after* scoring because that is the order the benchmark measured; and a parity run that proves the product path gives the evaluator's numbers.

39.1 From a checkpoint to a product

Chapter 38 ended with a release candidate: FT04, scored with residual Mahalanobis, with cloud zeroed on its own masks. On the real PRISMA bench that reads pAUC 0.170. But that number came from the *evaluator* (`app/evaluation/`, Chapter 35), a research harness that loads checkpoints directly, on a laptop. The *product* (Chapter 4) is something else: an api that must stay light and never imports torch, a worker that runs actions from a Postgres queue, a chain of actions in which one `band_filter_apply` output feeds an `anomaly_scoring` action, and defaults that every analyst inherits without choosing them.

The obvious way to ship a model - copy the checkpoint into the models folder and point the old code at it - fails in six different places, and the six sections below are those six places. On 2026-10-19 FT04 was served in `anomaly_scoring` under the codename **Indradhanu V2**, opt-in next to the shipped v0.2.0 "Indradhanu", which stayed the default. The design was agreed first (`docs/proposals/ship-ft04.md`) in two phases: Phase 1 makes V2 selectable without changing any default; Phase 2, later, would make it the hyperspectral default.

Analogy

A hospital takes on a new specialist who trained abroad. Before her first patient, the hospital gives her a name badge that no other doctor on the ward shares (or the switchboard puts calls through to whoever was added last), files her credentials in a form the front desk can read without phoning the overseas university, checks every referral is in the units she works in, and has her see a few patients the old consultant already examined - to confirm she reaches the same findings by the same route before anyone relies on her. Each of those four steps is a section of this chapter.

Table 39.1: What was shipped on 2026-10-19, from the manifest `current.json` on the dev laptop.

field	value
architecture	`spectral_stem_finetune`
codename	Indradhanu V2
version, epoch	`v2.0.0`, epoch 10 (of fine-tuning)
params	5,886,451
val_loss	0.05578
checkpoint file	`spectral_stem_finetune_v2.0.0_epoch10.pt`

Source: [M: `C:/es/ship/spectral_stem_finetune/current.json`].

Values that will drift

Since 2026-10-29 the codename Indradhanu V2 points to Indradhanu Heavy (FTH3 epoch 5, version `v2.1.0`, 35.3 M parameters; Chapter 43), behind the same architecture key, with FT04 (`v2.0.0`) kept as the manifest's first alternative (`docs/lld/spectral-stem.md`, "Serving"). Everything in this chapter describes the FT04 ship and still holds for the serving path, except where a box says otherwise.

39.2 Why the codename had to change

A *codename* is the name an analyst picks in `anomaly_scoring`: "Indradhanu", "MNF-RX". The worker turns it into a model with one dictionary:

```
# _anomaly_scoring_run.py (and anomaly_scoring.py's validate_config)
by_codename = {m.codename.lower(): m for m in catalog}
```

`catalog` comes from `list_catalog` in `backend/allotrope/foundation_models/resolver.py`, which walks the model directories in sorted order (`forpinsorted(models_root.iterdir())`). Two facts decide which model a repeated codename reaches.

Fact 1: a dictionary built this way keeps the last value. The comprehension assigns the entries one at a time, in catalog order. Assigning to a key that already exists overwrites its value. So if two manifests both say "Indradhanu", the dictionary ends up holding whichever came *last*.

Fact 2: strings sort by character codes, left to right. Python compares two strings *lexicographically*: compare the first characters; if they differ, the string with the smaller character code comes first; if they are equal, move to the second characters, and so on. Character codes are the Unicode code points: `a` is 97, `h` is 104, `s` is 115.

Worked example 39.1: Which manifest wins "Indradhanu"?

The two directories are `hyperspectral_segformer_mae` (v0.2.0) and `spectral_stem_finetune` (FT04). Compare first characters: `h` is 104, `s` is 115.

$104 < 115$

$\Rightarrow$ `hyperspectral_segformer_mae` sorts first, `spectral_stem_finetune` last

$\Rightarrow$ "indradhanu" $\rightarrow$ v0.2.0, then overwritten by FT04: FT04 wins.

The script confirms both the sort and the surviving value [C].

Every default hyperspectral action (`_DEFAULT_PRISMA`, `_DEFAULT_ENMAP` and `_DEFAULT_AVIRIS_NG` in `anomaly_scoring.py` all say `"model_codenames":["Indradhanu"]`) would have been re-routed to FT04 without anyone choosing it - and, with the grid check of Section 39.4, every one of them would then fail, because the default band filter resamples to the 165-band grid. `list_catalog` does log a warning on a collision ("codename collision … Rename one of them"), but a log line is not a guard. Hence a new name.

> When routing depends on the order of a directory listing, the only safe collision is no collision: give every model a name no other model shares.

39.3 A manifest that carries the whole configuration

The api lists models and validates actions; it never loads torch (the repo's lazy-import rule: action-type modules import torch only inside the worker's `run()`). So everything the resolver needs must be readable from `current.json`, a JSON file next to each checkpoint. For v0.2.0, `_model_config_from_manifest` rebuilds the model configuration from a few summary fields. For the spectral-stem models that is not possible:

```
# backend/allotrope/foundation_models/resolver.py, _model_config_from_manifest
if arch in ("spectral_stem_finetune", "spectral_stem_mae"):
    # The stem configs carry far more than a manifest summary can rebuild (sources,
    # band weighting, the stem variant), and the api cannot torch.load; the manifest
    # therefore embeds the checkpoint's full config.model_config.
    cfg = current.get("model_config")
    if not isinstance(cfg, dict) or cfg.get("model_type") != arch:
        raise KeyError(f"{arch}: current.json must embed the checkpoint's
    model_config")
    return cfg
```

`scripts/make_model_manifest.py` is the one place that does load the checkpoint: it copies `ck["config"]["model_config"]` into `current.model_config`, counts the parameters and copies the checkpoint next to it as `<architecture>_<version>_epoch<N>.pt`. If the embedded config is missing or names another architecture, the resolver raises `KeyError` - and `list_catalog` skips manifests that raise `KeyError`, so a bad manifest makes the model *disappear* from the picker rather than load wrongly. Failing closed is the right direction: a missing model is noticed; a wrongly configured one quietly scores.

The capabilities entry sets `pixel_stats_relpath=None`: there is no per-band statistics file, because the normalisation curve travels inside the checkpoint as buffers (`stats.wl`, `stats.mean`, `stats.std`; Chapter 29).

39.3.1 What "params" counts

A model's `state_dict` holds two kinds of tensors. *Parameters* are the numbers training changes by gradient descent. *Buffers* are stored numbers that are not trained by gradients - BatchNorm's running statistics, or here the normalisation curve and the trained-wavelength record. `make_model_manifest.py` counts

```
"params": int(sum(v.numel() for k, v in sd.items()
                  if not k.startswith(("stats.", "trained_wl"))))
```

that is, every tensor in the state dict except the normalisation and wavelength buffers. Predict the figure from the model's parts.

Worked example 39.2: Where 5,886,451 comes from

Trainable parameters of the shipped FT04, by top-level module (from the checkpoint [C]):

spectral stem in (`spectral_in`)	14,721
spectral decoder out (`spectral_out`)	4,257
BatchNorm scale and shift (`norm`, 2×32)	64
encoder	3,331,072
decoder	2,164,736
segmentation head (worked example 38.1)	371,536

Add them: $14{,}721 + 4{,}257 = 18{,}978$; $18{,}978 + 64 = 19{,}042$; $19{,}042 + 3{,}331{,}072 = 3{,}350{,}114$; $3{,}350{,}114 + 2{,}164{,}736 = 5{,}514{,}850$; $5{,}514{,}850 + 371{,}536 = 5{,}886{,}386$ parameters.
The manifest says 5,886,451. The difference, $5{,}886{,}451 - 5{,}886{,}386 = 65$, is the BatchNorm's buffers, which the manifest's filter does not exclude: running mean (32 numbers), running variance (32) and the step counter `num_batches_tracked` (1): $32 + 32 + 1 = 65$.

Source check: the manifest's "params" includes 65 buffers

`scripts/make_model_manifest.py` (line 59) excludes only `stats.*` and `trained_wl*` from its count, so the reported `params` of 5,886,451 includes the stem BatchNorm's 65 buffer entries. Building the model and summing `model.parameters()` gives 5,886,386 (`research/textbook/tools/compute_shipping.py`). The course quotes the manifest's 5,886,451 ("about 5.9 M"); both round to that. The excluded buffers are `trained_wl` (733 wavelengths in the checkpoint - the record of Chapter 38) and three 202-point normalisation curves.

39.4 One band filter, two kinds of model: the grid check

v0.2.0 reads the 165-band common grid (Chapter 3). V2 reads each sensor's *native* bands - the wavelengths the instrument actually measured. `band_filter_apply` can produce either (`use_common_wavelength_grid`, default `true`), but an `anomaly_scoring` action reads *one* upstream band-filter output for every model it runs. So the resolver now records which grid each model needs:

```
# resolver.py, ModelCapabilities
band_grid: str = "common165"          # v0.2.0 and every older model
# the spectral_stem_finetune entry:
band_grid="native", exclude_classes=("cloud",)
```

and the worker checks the vendable before running any foundation model on a hyperspectral scene:

```
# _anomaly_scoring_run.py, _check_band_grid
grid = np.asarray(DEFAULT_COMMON_WAVELENGTH_GRID, dtype=np.float64)     # (165,)
on_grid = len(wavelengths) == len(grid) and np.allclose(wavelengths, grid)
if band_grid == "native" and on_grid:
    raise ValueError("... reads native bands: run band_filter_apply with "
                     "use_common_wavelength_grid=false ...")
if band_grid == "common165" and not on_grid:
    raise ValueError("... needs the 165-band grid: run band_filter_apply with "
                     "use_common_wavelength_grid=true ...")
```

What `np.allclose` tests. `np.allclose(a,b)` is true when every pair of entries satisfies

$$|a_i - b_i| \le \text{atol} + \text{rtol} \cdot |b_i|\,, \qquad \text{rtol} = 10^{-5},\ \text{atol} = 10^{-8}\ \text{(the defaults)}. \tag{39.1}$$

The tolerance grows with the size of the number: a relative tolerance.

Worked example 39.3: How close is "on the grid"?

At the grid's first wavelength, 460 nm, the allowed difference is

$$10^{-8} + 10^{-5} \times 460 = 0.00000001 + 0.0046 = 0.0046\,\text{nm}.$$

At its last, 2450 nm: $10^{-8} + 10^{-5} \times 2450 = 0.0245\,\text{nm}$. So a list shifted by 0.001 nm everywhere still counts as the grid (0.001 is under 0.0046), while a shift of 0.01 nm does not (0.01 is over 0.0046 at the short end). The script agrees [C]. No real sensor lists its native bands within a few thousandths of a nanometre of a 10 nm grid, so the test separates "resampled" from "native" with a wide margin.

The test is "do these wavelengths equal the grid", not "are there 165 of them". A native list that happens to have 165 bands still fails `np.allclose` and counts as native. Classical detectors (MNF-RX and friends) skip the check: they take either grid.

Source check: the check runs in the worker, not at submit time

The design (`docs/proposals/ship-ft04.md`, Phase 1 step 4) placed the grid check in `anomaly_scoring.validate_config`. The code runs it in the worker (`_check_band_grid`, `backend/allotrope/action_types/_anomaly_scoring_run.py`, line 93, called at line 297), because the api cannot see the upstream band filter's settings when the action is submitted (`docs/09-known-issues.md`). The consequence: **one action cannot run V2 and v0.2.0 together** - whichever does not match the band filter makes the action fail at run time.

39.5 `read_bands`: tell the model what it is reading

A spectral stem reads a band list of any length, but only if it is told the wavelengths (Chapter 23). The product's older path never called `set_wavelengths`. The new helper does that and drops the bands the model has never trained on:

```
# app/foundation_models/inferencers/spectral_stem_inferencer.py
def read_bands(self, cube, wavelengths_nm):          # cube: (N, H, W)
    wl = np.asarray(wavelengths_nm, dtype=np.float64)
    keep = self.model.inner.trained_band_mask(wl).cpu().numpy()   # (N,) bool
    self.set_wavelengths(wl[keep])
    return cube[keep], wl[keep], np.flatnonzero(keep)  # (N_kept, H, W), (N_kept,),
    indices
```

`trained_band_mask` keeps a band when its distance to the nearest recorded wavelength is at most 15 nm:

$$\text{keep}(\lambda) = \Big[\min_{r \in R} |\lambda - r| \le 15\,\text{nm}\Big], \tag{39.2}$$

with R the trained-wavelength record (`trained_wl`), which grew during fine-tuning as FT04 read EnMAP, PRISMA and AVIRIS-NG. In code it is one line: `(wl[:,None]-self.trained_wl[None,:]).abs().min(dim=1).values<=tol_nm` - a matrix of all band-to-record distances, shape `(N, |R|)`, then the minimum along each row.

Worked example 39.4: Three bands against a record ending 2395, 2405, 2440

Build the distance matrix row by row (do it on paper).

band	to 2395	to 2405	to 2440	minimum	keep?
2400 nm	5	5	40	5	$5 \leq 15$: keep
2452 nm	57	47	12	12	$12 \leq 15$: keep
2470 nm	75	65	30	30	$30 > 15$: drop

(Script check [C].) A dropped band would meet wavelength networks that never saw anything there; dropping it is safer than letting them extrapolate.

The docstring states the trap in its second half: it "also sets the wavelengths for the next predict_full_scene / segment_full_scene". Skip it and the model keeps the *previous* scene's wavelengths - a mismatch that raises no error, the same family of silent bug as Chapter 37. In the worker, the branch is taken only for `band_grid=="native"`, and everything after it - reconstruction, scoring, segmentation, the reconstruction preview - uses the kept cube and wavelengths. The stats record how many bands the model read (`bands_read`). On the PRISMA bench V2 reads 185 bands of set1 and 186 of set4.

39.6 Score first, then zero the cloud

39.6.1 How the head turns a scene into classes

V2 has a segmentation head (Chapter 28). `segment_full_scene` runs it over the scene in 128×128 windows at stride 64, averages the sigmoid probabilities where windows overlap, and assigns each valid pixel one class. Three pieces of arithmetic hide in that sentence.

Padding so the windows fit. With window $T = 128$ and stride $S = 64$, a side of length $n > T$ is padded by $\text{pad} = (-(n - T)) \bmod S$, the smallest amount that makes $n + \text{pad} - T$ a multiple of S. For $n = 300$: $n - T = 172$; the next multiple of 64 at or above 172 is 192; $\text{pad} = 192 - 172 = 20$; padded side 320; window starts $0, 64, 128, 192$ (since $320 - 128 = 192$). For $n = 256$: $256 - 128 = 128 = 2 \times 64$, pad 0, starts $0, 64, 128$ [C].

Coverage and averaging. A row y is covered by every window whose start s satisfies $s \leq y < s+128$. With starts $0, 64, 128, 192$, row 100 is covered by the windows starting at 0 and 64 - two of them; row 10 only by the window at 0. In two dimensions an interior pixel is covered by $2 \times 2 = 4$ windows, and its probability for each class is the mean of four predictions. If the four windows say cloud 0.62, 0.48, 0.55 and 0.39, the pixel's cloud probability is $(0.62 + 0.48 + 0.55 + 0.39)/4 = 2.04/4 = 0.51$.

Threshold and priority. A class is "on" where its averaged probability exceeds 0.5. The code then assigns classes in a fixed order, each assignment overwriting the last:

```
cls = np.zeros((H, W), np.int64)                              # 0 land
cls[prob[0] > 0.5] = 1                                        # water
cls[prob[3] > 0.5] = 3                                        # shadow
cls[prob[4] > 0.5] = 4                                        # snow
cls[(np.maximum(prob[1], prob[2]) if thin_cloud else prob[1]) > 0.5] = 2   # cloud (
    or thin)
return cls * valid.astype(bool), prob
```

Because the last write wins (the same rule as Section 39.2, this time on an array), the effective priority is cloud > snow > shadow > water > land. `CLASS_NAMES=("land","water","cloud","shadow","snow")` gives the class-map ids 0 to 4.

Worked example 39.5: One pixel, two cloud rules

A pixel's averaged probabilities (water, cloud, thin cloud, shadow, snow) are $(0.70, 0.30, 0.60, 0.55, 0.10)$.

1. Water $0.70 > 0.5$: class 1.
2. Shadow $0.55 > 0.5$: overwrite, class 3.
3. Snow 0.10: no change.
4. Cloud, `thin_cloud=True`: $\max(0.30, 0.60) = 0.60 > 0.5$: overwrite, class 2 (cloud). Excluded.
5. Cloud, `thin_cloud=False`: 0.30, not above 0.5: stays class 3 (shadow). Not excluded.

Source check: thick cloud only, since 2026-10-29

At the FT04 ship, "cloud" meant cloud or thin cloud. Since 2026-10-29 the resolver sets `thin_cloud_is_cloud=False` for `spectral_stem_finetune` (`backend/allotrope/foundation_models/resolver.py`, the capabilities entry), and the worker passes it to `segment_full_scene(...,thin_cloud=m.thin_cloud_is_cloud)`: V2 excludes *thick* cloud only. The reason is Indradhanu Heavy's thin-cloud output, which fires on haze (75.4% of PRISMA set4's valid pixels and 209 of its 230 targets; `docs/lld/spectral-stem.md`). `stats.json` records the rule as `cloud_rule` (`"thick"` or `"cloud_or_thin"`). Chapter 43 tells that story.

39.6.2 Why zero after scoring

The worker then zeros the model's `exclude_classes` - for V2, cloud:

```
# _anomaly_scoring_run.py, after compute_score(...)
class_map, _ = inferencer.segment_full_scene(model_cube, spatial_valid.astype(bool),
                                             thin_cloud=m.thin_cloud_is_cloud)
excluded = np.isin(class_map, [inferencer.CLASS_NAMES.index(c) for c in m.
    exclude_classes])
score = np.where(excluded, 0.0, score).astype(np.float32)          # (H, W)
```

Why after, and not "remove the cloud from the mask, then score"? Residual Mahalanobis (Chapter 31) first fits its whitening - the spread of the residual - over every pixel in the mask, and measures each pixel in units of that spread. Which pixels are in the fit changes every score.

Worked example 39.6: The same pixel, twice the score

An illustration in one band, not a measurement [A]: four land residuals $0.02, -0.02, 0.02, -0.02$, two cloud residuals $0.06, -0.06$, and a target residual 0.08. All have mean 0, so the variance is the mean of the squares.

Fit on all six.

$$\begin{aligned} \text{squares:} &\quad 4 \times 0.0004 + 2 \times 0.0036 = 0.0016 + 0.0072 = 0.0088 \\ \text{variance:} &\quad 0.0088/6 = 0.001467 \\ \text{spread:} &\quad \sqrt{0.001467} = 0.0383 \\ \text{target:} &\quad 0.08/0.0383 = 2.09 \text{ spreads} \end{aligned}$$

Fit on land only.

$$\begin{aligned} \text{variance:} &\quad 0.0016/4 = 0.0004 \\ \text{spread:} &\quad \sqrt{0.0004} = 0.0200 \\ \text{target:} &\quad 0.08/0.0200 = 4.00 \text{ spreads} \end{aligned}$$

(Script: 2.089 and 4.000 [C].)

Same pixel, nearly twice the score. Neither order is wrong in principle, but **the benchmark measured "fit on everything valid, then zero the cloud"**, and the product must run what was measured. The evaluator's other order (`fit_on_kept:true` in `app/evaluation/scorers.py`) was tried on FT03 and was no better: 15.7% against 16.5% caught at the top 0.1% [M: `docs/lld/evaluation.md`].

A note on the keep mask: if the analyst also attaches an upstream `scene_segmentation` keep mask, that mask does feed the fit, as it always has. The parity run below used no upstream keep mask - only the valid pixels - which is the configuration the numbers describe.

39.6.3 What is written

Three things are written per model, under `models/<codename>/`:

output	contents
`mask_cloud.tif`	uint8, 1 where the score was zeroed
`class_map.tif`	uint8, the head's classes, 0 land to 4 snow
`stats.json` keys	`bands_read`, `excluded_classes`, `cloud_rule`, `excluded_percent`

`excluded_percent` is `round(100*excluded/valid,3)`.

Worked example 39.7: excluded_percent on a small tile

A 4×4 tile has 14 valid pixels, of which 3 are cloud:

$$\begin{aligned} 100 \times 3/14 = 300/14 &= 21.42857\ldots \\ \text{rounded to 3 places} &= 21.429 \end{aligned}$$

The keys are additive: the frontend reads stats by key, so new keys break nothing - and nothing in the viewer shows them yet (`docs/09-known-issues.md`).

39.7 The parity run: the guard

There are no behavioural backend tests in this repo. `tests/test_backend/test_native_band_models.py` checks the plumbing: the resolver reads an embedded config, refuses a missing or mismatched one, applies the thick-cloud rule, and the grid check accepts each grid for its own

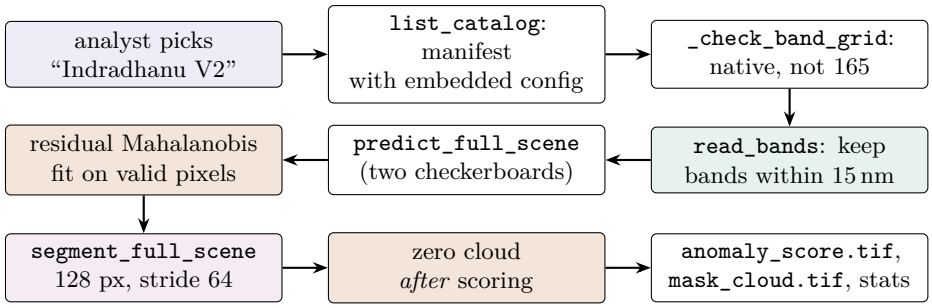


Figure 39.1: The worker's path for Indradhanu V2, in order. Every box is one of the decisions of this chapter; the parity run (Section 39.7) runs the same boxes outside the worker and compares with the evaluator.

kind of model and refuses the other. It does not check numbers. The guard for the numbers is `research/experiments/indradhanu-finetune/FT04-final-joint/product_parity.py`: it builds the inferencer *from the shipped manifest* through the resolver's `_from_manifest` and `get_inferencer`, then runs `read_bands`, `predict_full_scene`, `compute_score(method="residual_mahalanobis")`, `segment_full_scene` and the zeroing, exactly as the worker does, and compares against the evaluator's FT04 numbers on the PRISMA bench.

Table 39.2: The parity run, 2026-10-19: product path against the evaluator, PRISMA bench.

scene	bands read	pAUC plain	pAUC cloud-excluded	excluded
set1	185	0.2460	–	–
set4	186	0.2106	0.2199	11.83% of valid pixels

Source: [M: `docs/lld/spectral-stem.md`] (Serving), produced by `research/experiments/indradhanu-finetune/FT04-final-joint/product_parity.py`.

Identical to the evaluator's run; ROC-AUC within 0.0005. Two things made "identical" possible. The evaluator (`app/evaluation/models.py`) now calls the same `read_bands` and `segment_full_scene`, so there is one implementation of band reading and segmentation, not two that could drift. And Chapter 37's lesson - a re-implemented data path that quietly disagreed with the product's - is the reason a parity run exists at all.

These per-scene pAUCs are not comparable with the four-scene 0.170: they are single scenes. What matters is only that product and evaluator agree. If set4 has 1,000,000 valid pixels, 11.83% of them is $0.1183 \times 1{,}000{,}000 = 118{,}300$ pixels scored 0 by the exclusion.

Values that will drift

A second parity run was made when Heavy took over the codename (2026-10-29, thick-cloud rule): pAUC set1 0.2617, set2 0.1874, set3 0.0861, set4 0.2217 (mean 0.1892), matching the evaluator to four decimals (worked example 43.9). CPU cost inside the worker: on the image's 2 torch threads one PRISMA scene took 1,819 s with Heavy and 1,058 s with FT04, mostly `predict_full_scene` at stride 32; since 2026-10-29 the compose file raises the worker to 16 threads (`WORKER_THREADS`), and the same scene takes 454 s and 274 s, about four times faster (`docs/09-known-issues.md`; worked example 43.10).

Unit tests prove the plumbing; only a run of the real checkpoint on the real scenes, through the product's own path, proves the numbers.

39.8 What did not change

- **Defaults.** Every hyperspectral default still picks "Indradhanu" (v0.2.0); `band_filter_apply` still defaults to `use_common_wavelength_grid:true`. Phase 2 would change both.
- **EnMAP's band filter** still removes the provider's cloud, cloud-shadow and haze pixels (`quality_masks_to_apply`) before scoring, unlike the PRISMA evaluation behind V2's numbers. Kept as the product's convention (decision, 2026-10-19).
- **The detection case.** Like for like, both with cloud exclusion on FT04's masks, the real bench reads v0.2.0 0.165 against FT04 0.170 - a tie (Chapter 38). What V2 adds is the masks, native bands and better reconstruction, in one model.
- **One product fix rode along.** `scene_segmentation`'s brightness took the mean of "band 0 up to the VNIR edge". On native PRISMA, whose bands run downwards (Chapter 37), band 0 is the far SWIR. `app/utils/spectral_indices/scene_indices.py` now picks bands by wavelength (`find_band`: the index of the nearest wavelength, `np.argmin(np.abs(wavelengths-target_nm))`), which works in any order.

Still open (`docs/09-known-issues.md`): the viewer has no panel for the masks, and the CPU cost on the stack, measured since (drift box above), is several times v0.2.0's. Anything that runs the worker image outside compose is back on 2 threads, four times slower.

39.9 Common confusions

Indradhanu V2 replaced Indradhanu.
No. It is opt-in. "Indradhanu" is still v0.2.0 and still the default.

The codename collision would only matter if someone picked the name.
It matters for everyone: the defaults pick "Indradhanu" for you. A collision would have re-routed every default action to FT04 without a single config change.

The grid check counts bands.
It compares wavelengths with the grid using `np.allclose`. 165 native bands are still native.

Zeroing cloud before scoring is the same thing, earlier.
No. It changes which pixels the whitening is fitted on, so it changes every score, not only the cloud's.

mask_cloud.tif and the upstream keep_mask.tif are the same kind of mask.
`keep_mask.tif` comes from `scene_segmentation` (thresholds) and decides the fit. `mask_cloud.tif` comes from V2's own head and is applied after scoring. Both can be present.

The manifest's params is the number of trainable weights.
It is every state-dict entry except the normalisation and wavelength buffers, so it includes the BatchNorm's 65 running-statistics entries: 5,886,451 against 5,886,386 trainable.

The unit tests prove the product gives the benchmark's numbers.
They prove the plumbing (config, grid, cloud rule). The numbers are guarded by the parity run, which needs the checkpoint and the bench scenes and is not part of pytest.

39.10 Check yourself

Exercise 39.1. Suppose FT04's directory had been named `a_spectral_stem` and both manifests said "Indradhanu". Which model would the defaults route to, and why? Show the character codes.

Exercise 39.2. A trained record contains 1385 and 1460 nm and nothing between. Does `read_bands` keep a band at 1400 nm? At 1420 nm? At 1445 nm? Show the distances.

Exercise 39.3. A band filter output lists wavelengths equal to the 165-band grid plus 0.003 nm at every band. Does `_check_band_grid` treat it as on the grid? What about plus 0.03 nm? Use (39.1) at 460 and at 2450 nm.

Exercise 39.4. A 6×5 tile has 26 valid pixels; V2 calls 4 of them cloud. Compute `excluded_percent` by hand. Then a scene of side $n = 200$: what padding does `segment_full_scene` add, and where do its windows start?

Exercise 39.5. In worked example 39.6, add a third cloud residual of 0.06 (seven pixels in all, mean no longer exactly 0 - use the mean of the squares anyway, as an approximation). Recompute the target's score with the fit on all pixels. Does adding cloud to the fit raise or lower the target's score?

Exercise 39.6. An analyst picks both "Indradhanu" and "Indradhanu V2" in one `anomaly_scoring` action on a default band filter. What happens, and what should they do instead?

Next: Chapter 40, Improving the score on a fixed model, and gated scene adaptation.

Chapter 40

Improving the score on a fixed model, and gated scene adaptation

The one thing this chapter teaches

On 2026-10-19 and 20 four ways to find more targets without a better model were tried on FT04's fixed reconstruction - blurring the residual (a spatial matched filter), fusing residual Mahalanobis with PCA-RX by rank, whitening each background type separately, and briefly retraining on each scene with the likely anomalies gated out of the loss - and every one lost real objects to plain residual Mahalanobis. A pixel measure counts a big object's pixels many times and a one-pixel target once, so pass criteria must count objects caught; and a pretrained model has no early window for GT-HAD's early stopping to find.

40.1 The question: change the score, not the model

Chapter 38 closed on a hard fact. v0.2.0, FT03 and FT04 all tie on detection once you score them with residual Mahalanobis (RM, Chapter 31), and Section 38.9.4 showed why: if a better model shrinks every residual by the same factor, the whitening shrinks with it and every score stays put. The experiment record puts it in three words: RM "absorbs model gains". So the question on 2026-10-19 was the other half: **keep the reconstruction fixed and change only the score.** Can a better score find more?

These are evaluation scorers in `app/evaluation/scorers.py`, run by `configs/eval/scoring_gains.json` on one model (FT04 epoch 10) and two datasets. Nothing in the product's `anomaly_scoring` changed, and none of them was adopted. The baseline throughout is FT04 with plain RM and *no* cloud exclusion - pAUC 0.162, not the 0.170 of Indradhanu V2 with its cloud mask - so every row is on the same footing.

Analogy

A school wants better exam results without changing the teaching, so it tries changing the marking. Rule one: give credit for answers that are nearly right. The class average rises - but the students who got one hard question exactly right and nothing else now look like everyone else. Rule two: average two examiners' rankings. Rule three: grade each class against its own class average. Rule three makes the mock exam look great, because the mock spread its hard questions evenly over every class, while the real exam's hard cases all sat in one small class - who, graded against themselves, now look ordinary. Before trusting any new marking, count how many individual students it identifies correctly, not only the average mark. Each rule is one section of this chapter.

The pass criteria, fixed before the run. A scorer counted as a gain over RM on FT04 if:

1. real PRISMA bench: pAUC up to 1% false alarms rose by at least 0.005 *and* TPR at 0.1% false alarms did not fall;

2. bench-v2.1: objects caught at the top 1% fell by no more than 0.5 points.

Criterion 2 is only a guard. bench-v2.1's implants are blurred with $\sigma = 1.4$ by construction (Chapter 34), so a blur of 1.4 would partly be measuring its own reflection there. The real bench is the judge.

Table 40.1: The scorers on FT04's fixed reconstruction. Real bench: 4 PRISMA scenes, 909 objects; bench-v2.1: 18 hosts.

	real PRISMA bench			bench-v2.1	
scorer	pAUC	TPR @ 0.1%	caught 1% / 0.1%	caught 1%	criteria
RM (baseline)	0.162	0.061	**45.7% / 14.9%**	35.1%	–
A1 RM blur 1.0	0.177	0.064	40.7% / 12.0%	33.4%	1 met, 2 failed
A1 RM blur 1.4	0.177	0.069	36.3% / 10.2%	29.4%	1 met, 2 failed
A1+ RM + RM blur 1.4	0.180	0.064	43.0% / 12.8%	35.0%	both met
A2 rank mean	0.162	0.057	43.8% / 14.0%	34.9%	failed
A2 rank max	0.138	0.046	42.7% / 11.4%	34.8%	failed
A3 mixture $k = 6$	0.111	0.045	39.4% / 11.4%	41.2%	failed
A3 mixture $k = 10$	0.096	0.034	34.4% / 8.1%	41.9%	failed
PCA-RX (no model)	0.128	0.029	39.7% / 8.1%	33.2%	–

Source: [M: `research/experiments/scoring-gains/README.md`]; full tables in `research/experiments/scoring-gains/report.md`.

Read the bold column first. **No scorer catches as many real objects as plain RM, at either budget.** Figure 40.1 draws the puzzle to scale: the A1 points move right on pixel pAUC and down on objects caught.

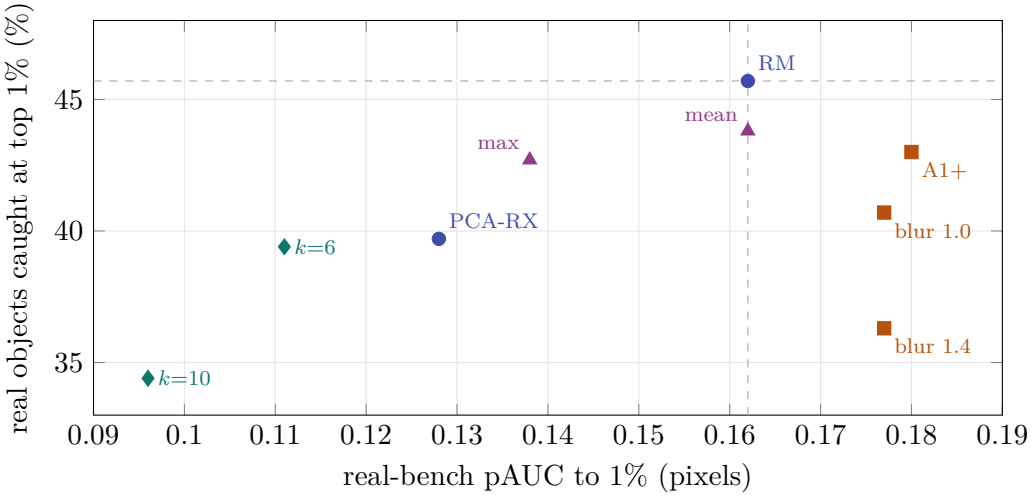


Figure 40.1: Pixel pAUC against objects caught, real PRISMA bench, drawn to scale from Table 40.1. The dashed lines cross at plain RM. Nothing lies above the horizontal line: the A1 blurs (amber) move right and down - more pixel pAUC, fewer objects.

40.2 A1: the spatial matched filter

40.2.1 The matched filter, derived

A *matched filter* is a weighting shaped like the signal you are looking for. Set it up in general. A measurement $\mathbf{x} \in \mathbb{R}^n$ is a known signal shape $\mathbf{t}$ with unknown amplitude a, plus noise $\mathbf{n}$ with mean $\mathbf{0}$ and covariance $\mathbf{\Sigma}$:

$$\mathbf{x} = a\,\mathbf{t} + \mathbf{n}.$$

A linear filter $\mathbf{w}$ turns it into one number, $y = \mathbf{w}^\top \mathbf{x} = a\,\mathbf{w}^\top \mathbf{t} + \mathbf{w}^\top \mathbf{n}$. The signal part is $a\,\mathbf{w}^\top \mathbf{t}$. The noise part has mean 0 and variance

$$\mathrm{Var}[\mathbf{w}^\top \mathbf{n}] = \mathbb{E}[(\mathbf{w}^\top \mathbf{n})^2] = \mathbb{E}[\mathbf{w}^\top \mathbf{n}\mathbf{n}^\top \mathbf{w}] = \mathbf{w}^\top \mathbb{E}[\mathbf{n}\mathbf{n}^\top]\mathbf{w} = \mathbf{w}^\top \mathbf{\Sigma}\mathbf{w}.$$

(The second step writes a square as a number times its own transpose; the third moves the constant $\mathbf{w}$ out of the expectation.) The *signal-to-noise ratio* per unit amplitude is

$$\mathrm{SNR}(\mathbf{w}) = \frac{\mathbf{w}^\top \mathbf{t}}{\sqrt{\mathbf{w}^\top \mathbf{\Sigma}\mathbf{w}}}. \tag{40.1}$$

Which $\mathbf{w}$ maximises it?

Step 1: the scale does not matter. Replace $\mathbf{w}$ by $c\,\mathbf{w}$ with $c > 0$: the numerator gains a factor c and the square root of the denominator gains $\sqrt{c^2} = c$. They cancel. So fix the scale by demanding $\mathbf{w}^\top \mathbf{t} = 1$, and maximising the SNR becomes *minimising the noise variance* $\mathbf{w}^\top \mathbf{\Sigma}\mathbf{w}$ subject to $\mathbf{w}^\top \mathbf{t} = 1$.

Step 2: a Lagrange multiplier. To minimise a function subject to a constraint, minimise the *Lagrangian*, the function plus a multiplier λ times the constraint written as "$= 0$":

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top \mathbf{\Sigma}\mathbf{w} - \lambda\,(\mathbf{w}^\top \mathbf{t} - 1). \tag{40.2}$$

Differentiate with respect to $\mathbf{w}$. For a symmetric $\mathbf{\Sigma}$, the gradient of $\mathbf{w}^\top \mathbf{\Sigma}\mathbf{w}$ is $2\mathbf{\Sigma}\mathbf{w}$ (in one dimension, $\frac{d}{dw}\sigma^2 w^2 = 2\sigma^2 w$), and the gradient of $\mathbf{w}^\top \mathbf{t}$ is $\mathbf{t}$. Set the gradient to zero:

$$2\mathbf{\Sigma}\mathbf{w} - \lambda\mathbf{t} = \mathbf{0}. \tag{40.3}$$

Multiply both sides on the left by $\frac{1}{2}\mathbf{\Sigma}^{-1}$:

$$\mathbf{w} = \tfrac{\lambda}{2}\,\mathbf{\Sigma}^{-1}\mathbf{t}. \tag{40.4}$$

Put this into the constraint $\mathbf{t}^\top \mathbf{w} = 1$ to find λ: $\frac{\lambda}{2}\,\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t} = 1$, so $\frac{\lambda}{2} = 1/(\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t})$. Therefore

$$\mathbf{w}^\star = \frac{\mathbf{\Sigma}^{-1}\mathbf{t}}{\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t}}. \tag{40.5}$$

Its SNR follows by substitution. The numerator is $\mathbf{w}^{\star\top}\mathbf{t} = 1$ by construction. The denominator: $\mathbf{w}^{\star\top}\mathbf{\Sigma}\mathbf{w}^\star = \frac{\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{\Sigma}\mathbf{\Sigma}^{-1}\mathbf{t}}{(\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t})^2} = \frac{1}{\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t}}$. So

$$\mathrm{SNR}(\mathbf{w}^\star) = \sqrt{\mathbf{t}^\top \mathbf{\Sigma}^{-1}\mathbf{t}}, \tag{40.6}$$

the Mahalanobis length of the signal shape. No linear filter does better: that is what "matched" means.

Worked example 40.1: A two-band matched filter

Two bands with correlated noise, $\mathbf{\Sigma} = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, and a signal only in band 1, $\mathbf{t} = (1, 0)$. Do it on paper.
Invert $\mathbf{\Sigma}$. For a 2×2 matrix $\left(\begin{smallmatrix} a & b \\ c & d \end{smallmatrix}\right)$ the inverse is $\frac{1}{ad-bc}\left(\begin{smallmatrix} d & -b \\ -c & a \end{smallmatrix}\right)$. Here $ad-bc = 2\times2-1\times1 = 3$:

$$\mathbf{\Sigma}^{-1} = \tfrac{1}{3}\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}.$$

Apply to $\mathbf{t}$. $\mathbf{\Sigma}^{-1}\mathbf{t} = \frac{1}{3}(2, -1) = (0.667, -0.333)$, and $\mathbf{t}^\top\mathbf{\Sigma}^{-1}\mathbf{t} = 1 \times 0.667 + 0 \times (-0.333) = 0.667$.
The filter. $\mathbf{w}^\star = (0.667, -0.333)/0.667 = (1, -0.5)$. It takes band 1 and *subtracts half of band 2*: band 2 holds no signal, but its noise is correlated with band 1's, so subtracting part of it cancels noise.
Check the SNR. $\mathbf{\Sigma}\mathbf{w}^\star = (2 \times 1 + 1 \times (-0.5),\ 1 \times 1 + 2 \times (-0.5)) = (1.5, 0)$; $\mathbf{w}^{\star\top}\mathbf{\Sigma}\mathbf{w}^\star = 1 \times 1.5 + (-0.5) \times 0 = 1.5$; SNR $= 1/\sqrt{1.5} = 0.816$, which equals $\sqrt{0.667} = 0.816$, as (40.6) says. The naive filter $\mathbf{w} = \mathbf{t} = (1, 0)$ gives $\mathbf{w}^\top\mathbf{\Sigma}\mathbf{w} = 2$ and SNR $= 1/\sqrt{2} = 0.707$. Matching gains about 15%. (Script check [C].)

The spatial version. Now let $\mathbf{x}$ be one band's residual over a small neighbourhood of pixels rather than several bands at one pixel. If the residual noise is independent from pixel to pixel with equal variance, $\mathbf{\Sigma} = \sigma^2\mathbf{I}$ and (40.5) gives $\mathbf{w}^\star \propto \mathbf{t}$: **weight the neighbours by the target's own footprint**. A target that always arrives smeared by the sensor's point-spread function (PSF, $\sigma \approx 1.4$ px on both benches; Chapter 34) is matched by a Gaussian blur of the same width. That is A1: blur the residual (`spatial_sigma` 1.0 or 1.4) before whitening.

Worked example 40.2: The promise and its price, on a 3-tap kernel

One band, already in units of the background's spread (noise variance 1, mean 0). Use the kernel $[0.25, 0.5, 0.25]$ as a stand-in for the Gaussian.
Noise after the blur. The variance of a weighted sum of independent values is the sum of squared weights times each variance:

$$0.25^2 + 0.5^2 + 0.25^2 = 0.0625 + 0.25 + 0.0625 = 0.375$$
$$\text{spread} = \sqrt{0.375} = 0.612$$

A footprint-shaped target $[1, 2, 1]$:

$$\text{blurred centre} = 0.25 \times 1 + 0.5 \times 2 + 0.25 \times 1 = 0.25 + 1 + 0.25 = 1.5$$
$$\text{SNR raw} = 2/1 = 2.00, \qquad \text{SNR blurred} = 1.5/0.612 = 2.45$$

The kernel is proportional to $[1, 2, 1]$, so it *is* this target's matched filter, and 2.45 equals $\sqrt{\mathbf{t}^\top\mathbf{t}} = \sqrt{1 + 4 + 1} = \sqrt{6} = 2.449$ from (40.6).
A one-pixel target $[0, 2, 0]$:

$$\text{blurred centre} = 0.25 \times 0 + 0.5 \times 2 + 0.25 \times 0 = 1.0$$
$$\text{SNR blurred} = 1.0/0.612 = 1.63 \qquad \text{(raw was 2.00)}$$

The same filter that helps a footprint-shaped target hurts a sharper one: a matched filter is matched to *one* shape. (Script: 2.449 and 1.633 [C].)

In the real code (`blurred_mahalanobis`) the whitening is refitted on the blurred residual -

the "divide by the new spread" step above. And `blur` uses *normalised convolution*, so nodata does not bleed into the scene's edge:

```
# app/evaluation/scorers.py, blur
k = valid.astype(np.float32)                                  # (H, W) 1 valid, 0 nodata
w = np.maximum(gaussian_filter(k, sigma), 1e-6)               # blurred validity
out[i] = gaussian_filter(x[i] * k, sigma) / w * k             # per band (H, W)
```

Blur the data with nodata set to 0, then divide by the blurred validity. At a pixel next to nodata, the blurred data counts fewer real neighbours, and dividing by the blurred validity renormalises the weights over the valid ones only. With the 3-tap kernel, values $(2, 4, \text{nodata})$ and the centre pixel: data blur $0.25\times2+0.5\times4+0.25\times0 = 2.5$; validity blur $0.25+0.5+0 = 0.75$; result $2.5/0.75 = 3.33$, the average of 2 and 4 with weights 0.25/0.75 and 0.5/0.75. Without the division you would get 2.5, dragged towards the zeros.

40.2.2 What a pixel measure rewards

pAUC and TPR are *pixel measures*: they count target pixels above a threshold. "Objects caught" is an *object measure*: an object counts once if any of its pixels is near a top-scoring pixel (Chapter 33). A filter that fills in more pixels of an object already found raises the first and not the second. You need a pen for the next example; it takes about five minutes.

Worked example 40.3: A strip of 12 pixels, plain and blurred

One band, background mean 0, score = residual squared (a one-band global RM). Pixels 5–8 are a big object, pixel 11 is a one-pixel target, pixels 2 and 3 are noise that flips sign from one pixel to the next.

pixel	1	2	3	4	5	6	7	8	9	10	11	12
residual	0	4.5	−4.2	0	3	4	5	3	0	0	4.8	0
score r^2	0	20.25	17.64	0	9	16	25	9	0	0	23.04	0

The analyst's budget is the top 4 pixels. Plain RM's top 4: pixel 7 (25), 11 (23.04), 2 (20.25), 3 (17.64).
Blur each pixel, $b_i = 0.25\,r_{i-1} + 0.5\,r_i + 0.25\,r_{i+1}$, zero outside the strip:

$$
\begin{aligned}
b_2 &= 0.25\times0 + 0.5\times4.5 + 0.25\times(-4.2) = 0 + 2.25 - 1.05 = 1.2 && \to 1.44\\
b_3 &= 0.25\times4.5 + 0.5\times(-4.2) + 0.25\times0 = 1.125 - 2.1 + 0 = -0.975 && \to 0.95\\
b_5 &= 0.25\times0 + 0.5\times3 + 0.25\times4 = 0 + 1.5 + 1 = 2.5 && \to 6.25\\
b_6 &= 0.25\times3 + 0.5\times4 + 0.25\times5 = 0.75 + 2 + 1.25 = 4.0 && \to 16\\
b_7 &= 0.25\times4 + 0.5\times5 + 0.25\times3 = 1 + 2.5 + 0.75 = 4.25 && \to 18.06\\
b_8 &= 0.25\times5 + 0.5\times3 + 0.25\times0 = 1.25 + 1.5 + 0 = 2.75 && \to 7.56\\
b_{11} &= 0.25\times0 + 0.5\times4.8 + 0.25\times0 = 2.4 && \to 5.76
\end{aligned}
$$

(Pixels 1, 4, 9, 10 and 12 come out at 1.27, 0.09, 0.56, 1.44 and 1.44 - all small.) Blurred top 4: pixel 7 (18.06), 6 (16), 8 (7.56), 5 (6.25). The target, at 5.76, is fifth.

	object pixels in top 4 (of 5)	false alarms in top 4	objects caught (of 2)
plain	2 (pixels 7, 11)	2 (pixels 2, 3)	2
blurred	4 (pixels 5–8)	0	1

A pixel measure such as TPR doubles, $2/5 = 0.4$ to $4/5 = 0.8$, and the false alarms vanish. Objects caught halves. (Every value matches the script [C].)

A pixel measure counts each pixel of an object, so filling in more of an object you had already found looks like progress, and losing a one-pixel target costs one pixel's worth.

The example simplifies three things - one band, a global mean instead of the 21/61 ring, a 3-tap kernel - but the real bench shows the same mechanism (Table 40.2). Blur loses objects at *every* size. Turned into counts: $0.365 \times 561 = 204.8 \approx 205$ small objects for RM against $0.333 \times 561 = 186.8 \approx 187$ blurred, 18 lost; $0.524 \times 170 = 89.1 \approx 89$ against $0.453 \times 170 = 77.0 \approx 77$ in the 3–5 px class, 12 lost. In the largest class the loss is one object ($0.80 \times 20 = 16$ against $0.75 \times 20 = 15$), too few to read. And 561 of 909 real objects, 62%, are 1–2 pixels: a filter that favours footprint-sized targets works against most of the bench.

Table 40.2: Real-bench objects caught at the top 1%, by object size (561 / 170 / 158 / 20 objects).

scorer	1–2 px	3–5 px	6–20 px	> 20 px
RM (baseline)	**36.5%**	**52.4%**	**66.5%**	80.0%
A1 RM blur 1.0	33.3%	45.3%	57.6%	75.0%
A1+ RM + RM blur 1.4	35.1%	48.2%	60.8%	80.0%
A3 mixture $k = 6$	32.6%	43.5%	54.4%	75.0%

Source: [M: `research/experiments/scoring-gains/README.md`].

A1+ passed its criteria and was still not adopted. A1+ (rank mean of RM and RM blurred at 1.4) met both criteria: pAUC $0.180 - 0.162 = +0.018$, at least $+0.005$; TPR 0.064, not below 0.061; bench-v2.1 $35.0 - 35.1 = -0.1$ points, within 0.5. It also lost $45.7 - 43.0 = 2.7$ points of real objects at the top 1% and $14.9 - 12.8 = 2.1$ at the top 0.1%. The README: "The criteria were pixel-level and missed this; not adopted." A criterion that was right to fix in advance and wrong in what it measured. From then on, every pass criterion in the record includes real objects caught.

40.3 A2: rank fusion with PCA-RX

PCA-RX (local RX on the cube's top principal components, no model; Reed and Yu [36]) wins where a target is foreign to the *scene*; RM wins where it is foreign to the *model*. The hope was that they fail on different objects, so combining them would catch the union. Their scores live on different scales, so they cannot be averaged directly. `rank_map` puts them on one footing:

$$\text{rank}(p) = \frac{\#\{\text{valid pixels with score} \le s(p)\}}{\#\{\text{valid pixels}\}} \quad \text{(ties averaged, 0 outside valid)}, \tag{40.7}$$

the empirical cumulative distribution of the scores, evaluated at each pixel. `fuse` stacks two such maps and takes the mean or the maximum.

Worked example 40.4: Rank mean and rank max on five pixels

P1 is a real target. P2 is RM's false alarm (a field edge); P4 is PCA-RX's (a roof that is spectrally unusual but ordinary to the model).

	P1	P2	P3	P4	P5
RM score	9	12	3	2	1
PCA-RX score	8	1	2	10	3
RM rank	4/5	5/5	3/5	2/5	1/5
PCA-RX rank	4/5	1/5	2/5	5/5	3/5
rank mean	0.8	0.6	0.5	0.7	0.4
rank max	0.8	1.0	0.6	1.0	0.6

Each rank counts the pixels at or below: RM's P1 (score 9) has 9, 3, 2, 1 at or below it, four of five, so 4/5. The mean for P1 is $(4/5 + 4/5)/2 = 0.8$; for P4, $(2/5 + 5/5)/2 = 7/10 = 0.7$. With a budget of one pixel: RM alone picks P2, PCA-RX alone picks P4 - both miss. Rank mean picks P1: a pixel both scorers think odd beats a pixel only one thinks odd. Rank max puts P2 and P4 first together (1.0) and P1 third. (Script check [C].)

Why does the maximum let false alarms through? Suppose, on background pixels, the two ranks behave like independent uniform numbers U, V on $[0, 1]$. At a threshold $1 - q$:

$$P(\max(U,V) > 1-q) = 1 - P(U \le 1-q)\,P(V \le 1-q) = 1 - (1-q)^2 = 2q - q^2,$$
$$P(\tfrac{U+V}{2} > 1-q) = P(U+V > 2-2q) = \tfrac{1}{2}(2q)^2 = 2q^2 \quad (q \le \tfrac{1}{2}).$$

(The second is the area of the corner triangle with legs $2q$ in the unit square.) At $q = 0.01$, the maximum passes $2 \times 0.01 - 0.0001 = 0.0199$ of the background - nearly twice either scorer alone - while the mean passes $2 \times 0.0001 = 0.0002$. The maximum's top is roughly the *union* of the two scorers' tops, false alarms included; the mean's top needs agreement. (In the evaluator the fused map is re-ranked and the budget is a fixed share of it, so these numbers describe the shape of the fused map, not the final false-alarm rate.)

The real bench gave the expected half and denied the hopeful half: rank max fell to pAUC 0.138, as predicted; rank mean tied pAUC (0.162) and *lost* objects (43.8% against 45.7%). Rank mean needs the scorers to fail on different objects, and on the real bench they did not. PCA-RX alone catches 39.7%; its wins on bench-v1 "do not carry to real targets", so averaging in its ranks mostly dilutes RM.

40.4 A3: a mixture of backgrounds

40.4.1 Gaussian mixtures and responsibilities

One covariance for a whole scene mixes land, water and roofs. A *Gaussian mixture* models the background instead as K Gaussian components, each a surface type: component k has a weight π_k (its share of the scene, $\sum_k \pi_k = 1$), a mean $\boldsymbol{\mu}_k$ and a covariance $\boldsymbol{\Sigma}_k$. The density of a pixel $\mathbf{x}$ is

$$p(\mathbf{x}) = \sum_{k=1}^{K} \pi_k \,\mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k). \tag{40.8}$$

Which component does a pixel come from? Bayes' rule turns "probability of the pixel given the component" into "probability of the component given the pixel" - the *responsibility*:

$$r_k(\mathbf{x}) = P(k \mid \mathbf{x}) = \frac{P(k)\,p(\mathbf{x} \mid k)}{p(\mathbf{x})} = \frac{\pi_k \,\mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \,\mathcal{N}(\mathbf{x};\, \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}. \tag{40.9}$$

A natural per-type anomaly score is then the *responsibility-weighted* squared Mahalanobis distance,

$$s(\mathbf{x}) = \sum_k r_k(\mathbf{x})\,(\mathbf{x} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x} - \boldsymbol{\mu}_k), \tag{40.10}$$

each type's distance counted in proportion to how likely the pixel belongs to it.

Worked example 40.5: Responsibilities and a weighted score in one band

Two types: a field ($\pi_1 = 0.8$, mean 0, variance 1) and roofs ($\pi_2 = 0.2$, mean 5, variance 4). A pixel at $x = 3$. The one-dimensional normal density is $\mathcal{N}(x; \mu, v) = \frac{1}{\sqrt{2\pi v}} e^{-(x-\mu)^2/(2v)}$.

$$\begin{aligned}
\mathcal{N}(3; 0, 1) &= \tfrac{1}{\sqrt{2\pi}}\, e^{-9/2} = 0.3989 \times 0.01111 = 0.004432 \\
\mathcal{N}(3; 5, 4) &= \tfrac{1}{\sqrt{8\pi}}\, e^{-4/8} = 0.1995 \times 0.6065 = 0.1210 \\
\pi_1 \mathcal{N}_1 &= 0.8 \times 0.004432 = 0.003545 \\
\pi_2 \mathcal{N}_2 &= 0.2 \times 0.1210 = 0.02420 \\
r_1 &= 0.003545/(0.003545 + 0.02420) = 0.003545/0.02774 = 0.128 \\
r_2 &= 0.02420/0.02774 = 0.872
\end{aligned}$$

Squared distances: to the field $(3-0)^2/1 = 9$; to the roofs $(3-5)^2/4 = 1$. Weighted score:

$$s = 0.128 \times 9 + 0.872 \times 1 = 1.15 + 0.87 = 2.02.$$

A single global Gaussian has mean $0.8 \times 0 + 0.2 \times 5 = 1$ and variance $\mathbb{E}[x^2] - 1^2 = 0.8 \times 1 + 0.2 \times (4 + 25) - 1 = 0.8 + 5.8 - 1 = 5.6$, giving $(3-1)^2/5.6 = 0.71$. (Script: $r_1 = 0.128$, $s = 2.022$ [C].)

40.4.2 What the code does: hard types by k-means

`background_types` does not fit a soft mixture. It runs *k-means* (Lloyd [27]'s algorithm) - the hard-assignment limit of a mixture, in which every responsibility is 0 or 1: pick k centres, give each pixel to its nearest centre, move each centre to the mean of its pixels, repeat.

Worked example 40.6: k-means on six numbers

Points $0, 1, 2, 8, 9, 10$; two centres starting at 0 and 2.

1. Assign. 0 is nearer 0; 1 is equally near both (distance 1) and the code's `argmin` takes the first, 0; 2, 8, 9, 10 are nearer 2. Update: centre $1 = (0+1)/2 = 0.5$; centre $2 = (2+8+9+10)/4 = 29/4 = 7.25$.
2. Assign. 2 is now $|2 - 0.5| = 1.5$ from centre 1 and $|2 - 7.25| = 5.25$ from centre 2: it moves. Update: centre $1 = (0+1+2)/3 = 1$; centre $2 = (8+9+10)/3 = 9$.
3. Assign: nothing moves. Converged at 1 and 9. (Script check [C].)

Source check: "mixture" is hard k-means plus class-aware RM

The README calls A3 a "mixture of backgrounds", but there is no soft Gaussian mixture in the code. `background_types` (`app/evaluation/scorers.py`) runs Lloyd's k-means, fixed seed, 25 iterations, fitted on a 50,000-pixel sample, on the *cube's* whitened top 10 principal components - so the types are surfaces, not residual patterns - and assigns every valid pixel to its nearest centre. `score` then hands those types to `class_aware_residual_mahalanobis` (`app/utils/anomaly_detection/residual_mahalanobis.py`), written for a segmentation head's classes: each type gets its own whitening and a ring of same-type neighbours, and types under 5,000 valid pixels (`MIN_CLASS_PX`) join type 0. In the language of (40.10), every r_k is 0 or 1.

40.4.3 Why it won the implant bench and lost the real one

A3 won bench-v2.1 by 6 points (35.1% to 41.2%) and cut the real bench's pAUC by a third (0.162 to 0.111). Here is why, by hand.

Worked example 40.7: Targets that become their type's normal

One band, every type's mean 0, score = residual squared / variance. An illustration [A]:

- field type (16 pixels): seven at $+1$, seven at -1, one edge at $+2.5$, one at -2.5;
- built-up type (6 pixels): $-2, -2, +2, +2$ (ordinary roofs), and two real targets at -6 and $+6$.

One global variance, all 22 pixels.

$$\text{sum of squares} = 14 \times 1 + 2 \times 6.25 + 4 \times 4 + 2 \times 36 = 14 + 12.5 + 16 + 72 = 114.5$$
$$\text{variance} = 114.5/22 = 5.20$$
$$\text{target score} = 36/5.20 = 6.92, \qquad \text{edge score} = 6.25/5.20 = 1.20$$

The targets top the ranking.
Each type on its own.

$$\text{field:} \quad (14 + 12.5)/16 = 26.5/16 = 1.656; \quad \text{edge score} = 6.25/1.656 = 3.77$$
$$\text{built-up:} \quad (16 + 72)/6 = 88/6 = 14.67; \quad \text{target score} = 36/14.67 = 2.45$$

Now the two field edges (3.77) outrank both targets (2.45). The targets are two of the built-up type's six pixels, so they *are* a large share of what that type's "normal" is fitted on. Graded against themselves, they look ordinary.

That is what the README found on the real scenes: 60–70% of target pixels sit in two or three small background types of about 5% of the scene each - the built-up surfaces where targets live. In set4, one type holds 60% of the target pixels on 5.2% of the scene [M: `research/experiments/scoring-gains/README.md`].

And the implants? bench-v2.1 spreads them over every surface. Put an implant with residual 4 into the field type above: globally $16/5.20 = 3.07$, per type $16/1.656 = 9.66$. A tighter normal makes an implant on an ordinary surface stand out more - so per-type whitening only sharpens the implants.

bench-v2.1 has a second bias (after bench-v1's cube-level oddity, Chapter 34): implant placement is not conditioned on where real targets occur. Judge scorers on the real bench and use bench-v2.1 to catch regressions.

`docs/09-known-issues.md` records the bias and says the fix (placement on target-bearing surfaces) is not built.

40.5 Expectations, a cache bug, and the verdict

The README wrote its expectations down before the run. Scored afterwards:

idea	expected	what happened
A1	a small gain on the real bench	right on pixels, wrong on objects
A2	rank mean a small gain, rank max a loss	rank max right; rank mean tied pAUC and lost objects
A3	a gain on multi-surface scenes (set2 / set4)	wrong in direction

A float16 overflow. The run also exposed a bug in the evaluator's cache (score maps are cached, Chapter 35). Maps were stored as float16, a 16-bit floating-point format with 1 sign bit, 5 exponent bits and 10 fraction bits. Its largest finite value has the largest normal exponent, 2^{15}, and all ten fraction bits set, a significand of $1 + (1 - 2^{-10}) = 2 - 2^{-10}$:

$$(2 - 2^{-10}) \times 2^{15} = 2^{16} - 2^{5} = 65{,}536 - 32 = 65{,}504.$$

Blurred and mixture squared distances went past it, so a reloaded map had `inf` at its top - exactly the pixels that matter. `runner.py` now keeps a map float32 if any value is non-finite or at least 6×10^4 in size; the 13 overflowed cache files were deleted. The README notes that this run's metrics were computed before the cast and are unaffected.

Verdict: RM stays the score. The lesson carried forward in writing: the next attempt's pass criteria include real objects caught at the top 1% and 0.1%. The next attempt was on the model side - an uncertainty head, FT06, which failed too (Chapter 41): it learned to expect large errors on the targets themselves, so dividing by that expectation hid them (pAUC 0.170 to 0.125).

40.6 Gated scene adaptation

40.6.1 What GT-HAD's gating and early stopping are for

Every failure so far had one shape: *every attempt to learn "normal" from a scene's own context also learned that scene's targets* - the mixture made targets their type's normal, and FT06's head learned to expect them. One method in the record seemed to have an answer. GT-HAD [25], a published hyperspectral anomaly detector, was reproduced on the four PRISMA scenes (`research/experiments/gt-had-prisma/summary.md`). It is a reconstruction detector with no pretraining: it trains from scratch on the very scene it scores, cut into 9×9 pixel blocks. Train long enough and it learns to reconstruct everything, targets included. It has two defences:

- **Gating.** A periodic search (the paper calls it CMM) decides which blocks look like background; those are routed to a background branch that learns from them, so its idea of normal is built without the likely anomalies.
- **Early stopping.** Stop training before the network has had time to learn the rare things: a network learns the common background before the rare targets.

The second defence is visible in the reproduction:

	5 epochs	25 epochs	150 epochs (paper's setting)
mean valid-pixel AUC	**0.873**	0.780	0.843
mean PD at 10^{-3} false alarms	**5.5%**	–	2.0%

Source: [M: `research/experiments/gt-had-prisma/summary.md`].

(PD, probability of detection, is the same idea as TPR.) The summary is careful: "choosing the epoch count from these GT AUCs would leak labels, so 5 epochs is a diagnostic, not a recommendation." After the GT-HAD discussion the user's comment - "Gating is something we can use" - started the experiment of 2026-10-20.

40.6.2 The transplant

GT-HAD starts from nothing; this version starts from FT04 and only adapts it. Per scene, starting from FT04's weights each time (`research/experiments/scene-adaptation/README.md`):

1. score the scene with FT04 + RM (the unadapted score);
2. gate = the top 1% of that score, dilated by 2 px (`gate_mask`);
3. adapt N steps on the scene's own tiles, gated pixels left out of the loss (`adapt`);
4. reconstruct, score RM with cloud excluded on *unadapted* FT04's masks; restore FT04's weights.

`adapt` (`app/foundation_models/scene_adaptation.py`) uses the fine-tuning loss - L1 plus 0.5× SAM on hidden pixels, 65% of tokens hidden - on batches of 8 random 128 px tiles from the scene, with an Adam optimiser. Three details matter:

- **The loss mask is hidden × valid × not-gated**: `learn=valid&~gate`, then `loss_mask=hidden*l[:,None].float()`. A gated pixel can still be *seen* as context; it never counts towards the loss.
- **Eval mode during training.** `model.eval()` is called before the loop, so BatchNorm keeps the statistics it learned on many scenes. Gradients still flow; only those stored averages are frozen.
- **The segmentation head is outside the loss**: parameters whose names start `seg_head.` or `scale_head.` are skipped - which is why cloud is excluded on the unadapted model's masks.

The evaluator copies the full `state_dict` before adapting and reloads it in a `finally` block, so a crash mid-scene cannot leave the next scene starting from adapted weights.

How far does a 1% gate reach? The probe printed "gated px 4.55%" on set1. Why that much, and why not more? `binary_dilation` with no structuring element uses the cross: each pass adds the four direct neighbours of every gated pixel. Two passes turn an $a \times b$ rectangle of gated pixels into the rectangle, plus four side strips of width 2, plus four corner triangles:

$$|\text{gate}| = ab + \underbrace{2 \cdot 2\,(a+b)}_{\text{side strips}} + \underbrace{4 \times 1}_{\text{corners}} = ab + 4(a+b) + 4. \tag{40.11}$$

(Each corner triangle at distance up to 2 holds the single diagonal pixel at offset $(1,1)$: $r(r-1)/2 = 1$ for $r = 2$.)

Worked example 40.8: Growth of a gate under two passes

1×1 :	$1 + 4 \times 2 + 4 = 1 + 8 + 4 = 13$	growth $13/1 = 13$
2×2 :	$4 + 4 \times 4 + 4 = 4 + 16 + 4 = 24$	growth $24/4 = 6.0$
3×3 :	$9 + 4 \times 6 + 4 = 9 + 24 + 4 = 37$	growth $37/9 = 4.1$

SciPy gives 13, 24 and 37 [C]. If set1's top 1% were all isolated pixels, the gate would cover about 13%. It covers 4.55%, a growth of 4.55, between the 3×3 and 2×2 cases: the top of the score comes in clumps, so a halo costs far less than it would for scattered single pixels. More than 95% of the scene still trains the network.

40.6.3 Choosing settings without ground truth

There are two settings: the learning rate and N, the number of steps. Choosing either by which detects the labelled targets best would be cheating - a setting tuned on the answers looks better than it will be on a new scene.

The learning rate, from the loss alone. `adapt_probe.py` ran 300 steps on set1 at three learning rates and recorded only the adaptation loss - no labels (1.6 steps per second on the laptop GPU):

lr	first 50 steps	steps 100–150	last 50 steps	relative fall
10^{-5}	0.0495	0.0409	0.0389	$0.0106/0.0495 = 21.4\%$
3×10^{-5}	0.0459	0.0382	0.0368	$0.0091/0.0459 = 19.8\%$
10^{-4}	0.0461	0.0365	0.0354	$0.0107/0.0461 = 23.2\%$

Losses: [M: `research/experiments/scene-adaptation/README.md`]; relative falls computed here [C].

The choice was 3×10^{-5}: "as fast as 1e-4 early, a third of the step size - the aim is to learn background without reaching the targets." Its first-50 average (0.0459) matches 10^{-4}'s (0.0461) with a third of the step. That is a judgement, not a measurement: the loss shows how fast the background is fitted, not whether the targets are being learned, because they are a tiny share of it (Section 40.6.4).

N, by leave-one-scene-out. Four variants were run: $N = 50$, 200 and 1000 gated, and $N = 200$ with the gate at 0% to isolate the gate's effect. *Cross-validation* estimates how a choice made on some data will do on new data by holding data out of the choice. *Leave-one-scene-out* is its version with scenes as the units: to report scene s, choose N using only the other scenes, then read s at that N; repeat for every scene and average.

Why bother? Pick N by the best mean over all four scenes and each reported scene has helped choose its own setting - the reported number is the maximum of several noisy numbers, which is biased upwards. Leave-one-scene-out guarantees that no scene's number comes from a setting tuned on that scene. It is not label-free - the other scenes' labels are used - but it is honest about how a setting chosen on old scenes would do on a new one, which is the product's situation.

Table 40.3: Real PRISMA bench, cloud excluded on unadapted FT04's masks.

	pAUC $\leq 1\%$	caught top 1%	caught top 0.1%	TPR @ 0.1%
FT04 (the bar)	**0.170**	46.0%	**15.8%**	**0.065**
gated, $N = 50$	0.160	46.8%	15.2%	0.059
gated, $N = 200$	0.159	46.9%	14.9%	0.059
gated, $N = 1000$	0.161	46.5%	15.0%	0.057
ungated, $N = 200$	0.159	46.9%	14.9%	0.059

Source: [M: `research/experiments/scene-adaptation/README.md`] (from `eval_report.md`).

The pass criteria, fixed before any detection result, asked for a leave-one-scene-out pAUC gain of at least +0.005 over FT04 with cloud excluded (0.170), objects caught at the top 1% and

0.1% not lower, and a bench-v2.1 guard. Part of the previous section's lesson is visible: objects caught are in the criteria now.

Worked example 40.9: Leave-one-scene-out on the real numbers

Per-scene pAUC, cloud excluded, from `eval_per_scene.csv`, rounded to three places:

	set1	set2	set3	set4
FT04	0.246	0.128	0.084	0.220
$N = 50$	0.223	0.142	0.084	0.192
$N = 200$	0.228	0.142	0.081	0.185
$N = 1000$	0.240	0.139	0.083	0.181

The baseline: FT04 mean $= (0.246 + 0.128 + 0.084 + 0.220)/4 = 0.678/4 = 0.1695$. You need a pen; it takes about ten minutes.
Hold out set1 (mean over set2, set3, set4):

$$\begin{aligned} N = 50: &\quad (0.142 + 0.084 + 0.192)/3 = 0.418/3 = 0.1393 \\ N = 200: &\quad (0.142 + 0.081 + 0.185)/3 = 0.408/3 = 0.1360 \\ N = 1000: &\quad (0.139 + 0.083 + 0.181)/3 = 0.403/3 = 0.1343 \end{aligned}$$

Best $N = 50$; set1 reports its $N = 50$ value, **0.223**.
Hold out set2 (set1, set3, set4):

$$\begin{aligned} N = 50: &\quad (0.223 + 0.084 + 0.192)/3 = 0.499/3 = 0.1663 \\ N = 200: &\quad (0.228 + 0.081 + 0.185)/3 = 0.494/3 = 0.1647 \\ N = 1000: &\quad (0.240 + 0.083 + 0.181)/3 = 0.504/3 = 0.1680 \end{aligned}$$

Best $N = 1000$; set2 reports **0.139**.
Hold out set3 (set1, set2, set4):

$$\begin{aligned} N = 50: &\quad (0.223 + 0.142 + 0.192)/3 = 0.557/3 = 0.1857 \\ N = 200: &\quad (0.228 + 0.142 + 0.185)/3 = 0.555/3 = 0.1850 \\ N = 1000: &\quad (0.240 + 0.139 + 0.181)/3 = 0.560/3 = 0.1867 \end{aligned}$$

Best $N = 1000$; set3 reports **0.083**.
Hold out set4 (set1, set2, set3):

$$\begin{aligned} N = 50: &\quad (0.223 + 0.142 + 0.084)/3 = 0.449/3 = 0.1497 \\ N = 200: &\quad (0.228 + 0.142 + 0.081)/3 = 0.451/3 = 0.1503 \\ N = 1000: &\quad (0.240 + 0.139 + 0.083)/3 = 0.462/3 = 0.1540 \end{aligned}$$

Best $N = 1000$; set4 reports **0.181**.
The headline.

$$\begin{aligned} \text{leave-one-scene-out} &= (0.223 + 0.139 + 0.083 + 0.181)/4 = 0.626/4 = 0.1565 \\ \text{gain over FT04} &= 0.1565 - 0.1695 = -0.013 \end{aligned}$$

The criterion asked for +0.005. On the unrounded CSV the same picks give 0.1567 [C]; the README records 0.157.

Notice two things. Set1 picked a different N from the others: three scenes are a small jury, and the choice is not stable. And the honest number, 0.1565, is a little worse than the best

single row's mean (0.161 for $N = 1000$). **That gap is the size of the flattery you would have reported by choosing N on all four scenes.**

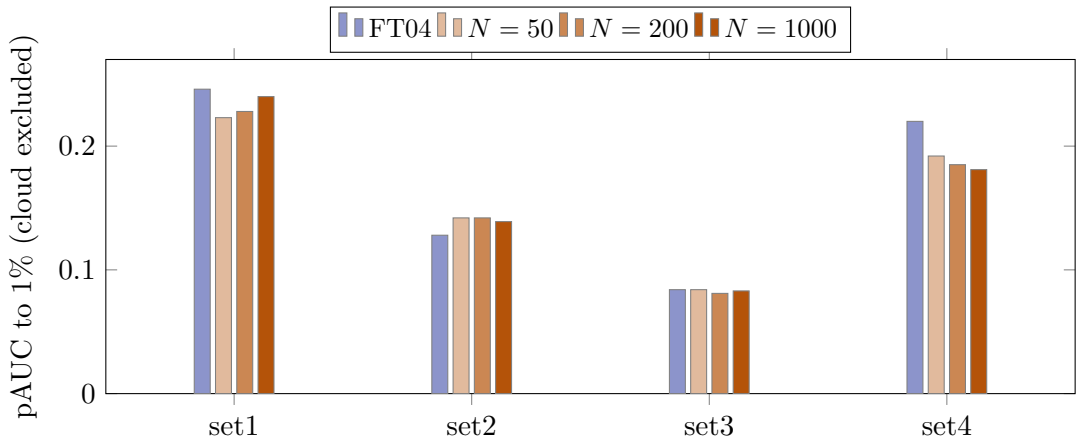


Figure 40.2: Per-scene pAUC of FT04 and its gated adaptations, drawn to scale. Set2 gains at every N; set1 and set4 lose at every N; nothing peaks at small N.

40.6.4 Results, and two lessons

Read Table 40.3 in three passes. Every N is below FT04 on pAUC (about -0.01) and on objects caught at the top 0.1% (-0.6 to -0.9 points): criteria 1 and 2 fail. Objects caught at the top 1% rose by 0.5 to 0.9 points; with the top 0.1% falling, the README reads this as "a shift of hits from the very top into the top 1%, not more finds." And gated and ungated $N = 200$ are the same row, to three decimals, on every measure. Both written expectations - a gain peaking at small N, and ungated worse than gated - were wrong. bench-v2.1 was not run; with criteria 1 and 2 failed it could not change the verdict. One scene, set2, gains at every N (0.128 to 0.139–0.142); "four scenes cannot say whether that is signal."

Lesson 1: the gate changed nothing, because adaptation never learned the targets. If gated and ungated agree to three decimals, the ungated network was not learning the targets either, so the gate had nothing to prevent. Why not? Look at their weight in the loss. GT-HAD's summary gives the scale: scenes of about 1200×1200 pixels with 600 to 1200 anomaly pixels.

$$\begin{aligned} \text{scene pixels} &= 1{,}200 \times 1{,}200 = 1{,}440{,}000 \\ \text{anomaly pixels} &\approx 1{,}000 \\ \text{share} &= 1{,}000/1{,}440{,}000 = 0.0007 = 0.07\% \end{aligned}$$

In a loss that is a mean over pixels, 0.07% of the pixels contribute about 0.07% of the gradient. The README puts it as the targets being "$<$ 0.1% of pixels" that "1000 steps of a 5.9 M-parameter model barely see". Be precise about "barely see": it is the *weight* that is small, not the number of visits. 1000 steps $\times$ 8 tiles $\times$ 128 $\times$ 128 $=$ 131,072,000 pixel-visits, about $131{,}072{,}000/1{,}440{,}000 = 91$ per pixel of the scene: each target is in a tile many times, but each time it is one voice in 16,384.

So what went wrong? Something that affects every pixel. Nudging the network towards this scene's background moved the residual of every pixel a little, and the careful ranking at the very top got slightly shuffled. No gate on 4.55% of the pixels can protect against a change to the other 95%.

Lesson 2: a pretrained model has no early window. GT-HAD's curve - better at 5 epochs than 150 - is a statement about a network that *started from nothing on this scene*. Early on it has learned the common background and not the targets; that moment is what early stopping finds. Place FT04 on the same ruler:

	detection at 10^{-3} false alarms
GT-HAD, 150 epochs (paper's setting)	2.0%
GT-HAD, 5 epochs (its best, a diagnostic)	5.5%
FT04, no adaptation	6.5%

(GT-HAD figures are PD on valid pixels; FT04's is TPR at 0.1% with cloud excluded. Close enough to compare, not identical: different band preparation, and GT-HAD did not exclude cloud. Without exclusion FT04's figure is 6.1%, still ahead.) FT04 already knows what ordinary land, water and roofs look like, from many scenes, and was never trained on this scene's targets. It is already where GT-HAD's early stopping is trying to get to, and beyond it. **A pretrained model starts past the point early stopping finds** - which is why the damage is the same at $N = 50$ as at $N = 1000$. Any adaptation moves it away from a good starting point, not towards one.

Gating and early stopping protect a network that is about to memorise its targets; a pretrained model adapted for a few hundred steps was never about to, so they have nothing to protect.

Values that will drift

All results in this chapter are dated 2026-10-19 (scoring gains) and 2026-10-20 (scene adaptation), on FT04. The next model-side attempts - the uncertainty head (FT06), gated training losses and block masking (FT07, FT08), the progressive decoder (FT09) and Indradhanu Heavy - are Chapters 41 to 43.

40.7 Common confusions

pAUC went up, so A1 is better.
pAUC and TPR count pixels. A1 filled in more pixels of objects already found and lost objects at every size. The analyst reviews objects, not pixels.

The blur in A1 is the same as bench-v2.1's PSF blur.
Same shape, different job. bench-v2.1 blurs the implant before scoring to imitate a real target; A1 blurs the residual of the whole scene. That overlap is why A1's bench-v2.1 numbers were treated as partly circular.

A1+ met the criteria, so it should have shipped.
The criteria were fixed in advance, which is right, but they checked only pixel measures. A1+ lost 2.7 points of real objects. The record changed the criteria rather than shipping a regression.

Rank max is the generous choice - a pixel only needs one scorer to like it.
That generosity is the problem: any pixel either scorer wrongly likes reaches the top too.

A3 won a benchmark, so it is a better scorer the real bench is too small to show.
The real bench lost a third of its pAUC, and the mechanism is understood: real targets cluster in small built-up types and become their types' normal. The benchmark's placement is what is wrong here.

The mixture scorer is a Gaussian mixture model.
It is hard k-means types handed to class-aware residual Mahalanobis: every responsibility is 0 or 1.

The gate made no difference, so gating is useless.
It made no difference here, because the network did not memorise the targets in 1000 steps. In a from-scratch method trained for 150 epochs, it does, and gating has a job.

Eval mode means the model is not training.
`model.eval()` only freezes BatchNorm's stored statistics and turns dropout off. The optimiser still updates the weights every step.

Objects caught at the top 1% went up, so adaptation found more.
The top 0.1% went down at the same time. Hits moved from the very top into the rest of the top 1%; the analyst looks at the very top first.

40.8 Check yourself

Exercise 40.1. Matched filter: $\mathbf{\Sigma} = \left(\begin{smallmatrix} 1 & 0.5 \\ 0.5 & 1 \end{smallmatrix}\right)$ and $\mathbf{t} = (1, 1)$. Compute $\mathbf{w}^\star$ by (40.5) and its SNR by (40.6). Compare with the naive $\mathbf{w} = \mathbf{t}$. Explain the result.

Exercise 40.2. Redo the strip of worked example 40.3 with the target at pixel 11 raised from 4.8 to 5.2. Compute its blurred score and say whether it enters the blurred top 4, and what that does to objects caught.

Exercise 40.3. In worked example 40.4, change P1's PCA-RX score from 8 to 11. Recompute both ranks of P1 and P4, the rank mean and the rank max. Which pixel does each rule put first?

Exercise 40.4. In worked example 40.5, compute the responsibilities and the weighted score for a pixel at $x = 2$.

Exercise 40.5. In worked example 40.7, the built-up type gains two more ordinary roofs at -2 and $+2$ (8 pixels, still two targets). Recompute the built-up variance and the target score. Do the targets now beat the field edges' 3.77?

Exercise 40.6. Redo the leave-one-scene-out of worked example 40.9 with only $N = 50$ and $N = 200$ as candidates. Which N does each scene pick, and what is the headline?

Exercise 40.7. Using (40.11), how many pixels does a gated 1×4 line cover after two passes? What share of a scene would the gate cover if its top 1% were made entirely of such lines?

Next: Chapter 41, The uncertainty head and the Laplace likelihood.

Chapter 41

The uncertainty head and the Laplace likelihood

The one thing this chapter teaches

A small extra head can learn, for every pixel, how large an error the model expects there, by minimising a Laplace negative log-likelihood whose best answer is exactly that expected error; but when the residual is divided by this prediction, real targets fade, because a head that predicts its own error well also predicts large errors on anomalies whenever it can see any trace of them.

This chapter is history told as a lesson. It covers one idea, the *uncertainty head*, from its proposal on 2026-10-19 to its failure on 2026-10-20 as the fine-tuning run FT06. Along the way you meet three pieces of mathematics that are worth owning for their own sake: the Laplace distribution, the method of maximum likelihood applied to a scale parameter, and Spearman's rank correlation. The chapter then shows, with arithmetic you can do on paper, why a head that passed its own sanity check still made detection worse.

A note on the provenance tags in this chapter and the next two. To keep lines short they abbreviate paths:

- run folders such as `FT06-uncertainty/`: in `research/experiments/indradhanu-finetune/`;
- `bench-resolution/`, `scoring-gains/` and `decoder-capacity/`: in `research/experiments/`;
- `finetune_trainer.py`: in `app/foundation_models/trainers/`;
- `scorers.py`: in `app/evaluation/`;
- `uncertainty-head.md` is in `docs/proposals/`, and `lld/finetune.md` in `docs/`.

41.1 Where we were, and the gap the score leaves

Chapter 39 shipped FT04 as Indradhanu V2. Its detection tied the older v0.2.0 model, and on the same day an experiment tried to win detection back on the *score* side while holding FT04's reconstruction fixed: blurring the residual, fusing residual Mahalanobis with PCA-RX, and whitening each background type separately (Chapter 40). None of them beat plain residual Mahalanobis on real objects caught. The experiment's verdict was that "the gains question goes to the model side" [M: `scoring-gains/README.md`].

Recall from Chapter 31 what residual Mahalanobis (RM) does. It takes the residual $\mathbf{r}(p) = \mathbf{x}(p) - \hat{\mathbf{x}}(p)$ at each pixel p, fits *one* mean and *one* covariance matrix for the whole scene, and scores each pixel by its Mahalanobis distance under that covariance. The covariance learns which *directions* of error the scene shares: a haze that lifts the blue bands everywhere, a detector stripe, dark water. What it cannot learn is which *places* are hard. The model's errors are not alike from place to place. Field edges, fine texture, sun glint, thin cloud and shorelines are hard to reconstruct everywhere, and their large residuals are exactly RM's false alarms. RM has no way to know that "the model always struggles here".

Analogy

A weather forecaster who says "20 mm of rain, give or take 2" and one who says "20 mm, give or take 15" can both see 35 mm fall. For the first it is a shock; for the second it is a wet day. What makes an observation surprising is the miss *divided by* how big a miss the forecaster expected. The uncertainty head is a forecaster of the reconstruction's own error. Later in the chapter the analogy acquires a twist: a forecaster who is never told about your town, only about the towns around it, will still see a storm parked over your town spilling rain into every neighbour, and will say "something big near there, give or take a lot". The storm then stops looking surprising.

The obvious fix is to give each pixel its own "give or take": a number $b(p) > 0$, the error size the model expects at p, and to score $\mathbf{r}(p)/b(p)$ instead of $\mathbf{r}(p)$. Two pixels with the same residual 0.3 then differ: at a field edge where the head expects $b = 0.3$ the ratio is $0.3/0.3 = 1.0$, ordinary; on a flat field where it expects $b = 0.05$ the ratio is $0.3/0.05 = 6.0$, a surprise. The rest of this chapter is about how to learn $b(p)$ honestly, and why it still did not work.

41.2 A probability model for errors: the Laplace distribution

To learn b by optimisation we need a loss whose minimum is "the right size of error". The cleanest way to build one is to write down a probability model for the error and ask which b makes the observed errors most probable. The model chosen is the Laplace distribution, because it is the distribution that matches the L1 reconstruction loss of Chapter 25, as you will see in Section 41.3.

Definition 41.1 (Probability density)**.** A *probability density* $f(r)$ for a real-valued quantity r is a function with $f(r) \geq 0$ everywhere and $\int_{-\infty}^{\infty} f(r)\,dr = 1$. The probability that r lands in a small interval of width Δ around r_0 is approximately $f(r_0)\,\Delta$.

Definition 41.2 (Laplace distribution)**.** The *Laplace distribution* with centre 0 and *scale* $b > 0$ has density

$$f(r \mid b) \;=\; \frac{1}{2b}\,\exp\!\left(-\frac{|r|}{b}\right). \tag{41.1}$$

Read Equation (41.1) in words: the density is highest at $r = 0$, where it equals $1/(2b)$, and falls away exponentially on both sides as the size of the error $|r|$ grows, at a rate set by b. A small b means a tall narrow peak (errors are expected to be small); a large b a low wide one.

41.2.1 It is a density: the integral is one

You should check that Equation (41.1) really integrates to one. The function is symmetric ($f(-r) = f(r)$, since $|-r| = |r|$), so the integral is twice the integral over $r \geq 0$, where $|r| = r$:

$$\int_{-\infty}^{\infty} \frac{1}{2b} e^{-|r|/b}\,dr = 2\int_{0}^{\infty} \frac{1}{2b} e^{-r/b}\,dr$$

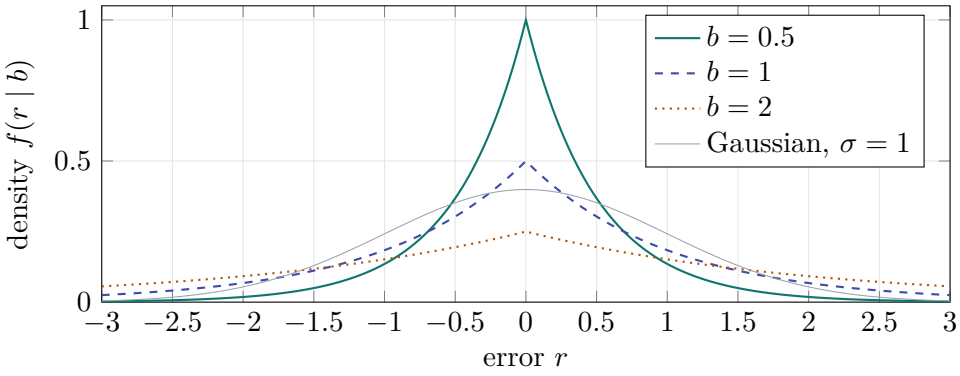


Figure 41.1: The Laplace density for three scales, drawn to scale from Equation (41.1), with a standard Gaussian in grey for comparison. The Laplace has a sharp peak at zero and heavier tails: a large error is less surprising under a Laplace than under a Gaussian of similar width.

cancel the 2 against the $\frac{1}{2}$:

$$= \frac{1}{b}\int_0^\infty e^{-r/b}\,dr$$

an antiderivative of $e^{-r/b}$ is $-b\,e^{-r/b}$ (differentiate it to check: $-b\cdot(-\frac{1}{b})e^{-r/b} = e^{-r/b}$):

$$= \frac{1}{b}\Big[-b\,e^{-r/b}\Big]_0^\infty$$

at $r \to \infty$ the exponential is 0; at $r = 0$ it is 1:

$$= \frac{1}{b}(0 - (-b)) \;=\; \frac{1}{b}\cdot b \;=\; 1.$$

41.2.2 The scale is the mean absolute error

The single most useful fact about the Laplace scale is that it equals the average size of the error, $\mathbb{E}\,|r| = b$. Derive it the same way, using the symmetry and then integration by parts (the rule $\int u\,dv = uv - \int v\,du$):

$$\mathbb{E}\,|r| = \int_{-\infty}^{\infty} |r|\;\frac{1}{2b}e^{-|r|/b}\,dr \;=\; \frac{1}{b}\int_0^\infty r\,e^{-r/b}\,dr$$

take $u = r$ (so $du = dr$) and $dv = e^{-r/b}dr$ (so $v = -b\,e^{-r/b}$):

$$= \frac{1}{b}\left(\Big[-b\,r\,e^{-r/b}\Big]_0^\infty + b\int_0^\infty e^{-r/b}\,dr\right)$$

the bracket is 0 at both ends ($r = 0$ kills it at the bottom; the exponential beats r at the top); the integral was found above to be b:

$$= \frac{1}{b}\,(0 + b\cdot b) \;=\; b.$$

So when the head outputs $b(p)$ it is literally predicting "the mean absolute error I expect at this pixel". That is why the proposal could describe b in plain words as the expected error size.

41.3 Likelihood, and the negative log-likelihood

Definition 41.3 (Likelihood)**.** Given an observed value r and a model with parameter b, the *likelihood* of b is the density of the observation viewed as a function of the parameter: $\mathcal{L}(b) = f(r \mid b)$. The *maximum-likelihood* estimate of b is the value that makes $\mathcal{L}(b)$ largest.

Maximising a product of small numbers is awkward, so we take logarithms (which turns products into sums and does not move the maximum, because log is increasing) and flip the sign (so that maximising becomes minimising, which is what an optimiser does). The result is the *negative log-likelihood* (NLL). For the Laplace:

$$-\log f(r \mid b) = -\log\left(\frac{1}{2b}\exp\left(-\frac{|r|}{b}\right)\right)$$

the log of a product is the sum of the logs:

$$= -\log\frac{1}{2b} \;-\; \log\exp\left(-\frac{|r|}{b}\right)$$

$\log(1/y) = -\log y$, and $\log\exp(z) = z$:

$$= \log(2b) + \frac{|r|}{b}$$

and $\log(2b) = \log 2 + \log b$:

$$= \frac{|r|}{b} + \log b + \log 2. \tag{41.2}$$

The last term, $\log 2 \approx 0.693$, does not depend on b, so it cannot change which b is best. The code drops it. What is left is the per-pixel loss the trainer minimises:

$$\ell(b;\,|r|) \;=\; \frac{|r|}{b} + \log b. \tag{41.3}$$

Two forces pull on b. The first term, $|r|/b$, punishes a b that is too small: claim to be confident and then miss, and you pay a lot. The second term, $\log b$, punishes a b that is too large: you cannot escape the first term by claiming to know nothing, because vagueness costs $\log b$.

The Laplace NLL is the L1 error divided by a predicted scale, plus the log of that scale; with b fixed it is the L1 loss of Chapter 25 up to a constant factor and offset, which is why the Laplace is the distribution that "matches" L1.

Worked example 41.1: The loss at three scales

Take one pixel whose error is $|r| = 0.2$ and try $b = 0.1$, 0.2 and 0.4. Do it on paper with a calculator for the natural logarithm.

$$b = 0.1: \quad \frac{0.2}{0.1} + \ln 0.1 = 2.0 + (-2.303) = -0.303$$
$$b = 0.2: \quad \frac{0.2}{0.2} + \ln 0.2 = 1.0 + (-1.609) = -0.609$$
$$b = 0.4: \quad \frac{0.2}{0.4} + \ln 0.4 = 0.5 + (-0.916) = -0.416$$

The lowest of the three is at $b = 0.2$, the error itself. At $b = 0.1$ the first term is large (2.0); at $b = 0.4$ the first term is small (0.5) but the second has risen from -1.609 to -0.916.

Negative losses are normal here: a density can exceed 1, so its log can be positive and the NLL negative.

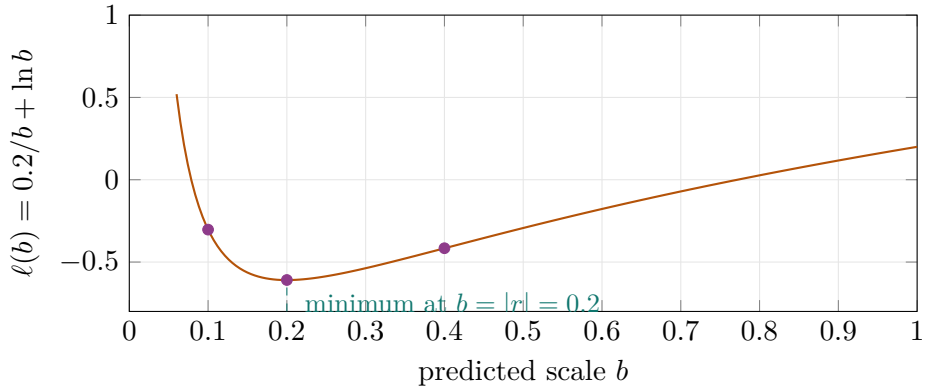


Figure 41.2: The per-pixel loss of Equation (41.3) for $|r| = 0.2$, drawn to scale, with the three points of worked example 41.1. The curve rises steeply to the left (overconfidence) and slowly to the right (vagueness).

41.4 The best scale is the error: minimising over b

The worked example suggests that the minimum sits at $b = |r|$. Prove it with calculus. At a minimum of a smooth function the derivative is zero.

$$\frac{d\ell}{db} = \frac{d}{db}\left(|r|\; b^{-1}\right) + \frac{d}{db}\log b$$

the power rule gives $\frac{d}{db}b^{-1} = -b^{-2}$; and $\frac{d}{db}\log b = \frac{1}{b}$:

$$= -\frac{|r|}{b^2} + \frac{1}{b}.$$

Set it to zero:

$$0 = -\frac{|r|}{b^2} + \frac{1}{b}$$

multiply both sides by b^2 (allowed, because $b > 0$):

$$0 = -|r| + b$$

so

$$b^\star = |r|\,.$$

Is it a minimum and not a maximum? Differentiate again:

$$\frac{d^2\ell}{db^2} = \frac{2\,|r|}{b^3} - \frac{1}{b^2},$$

and at $b = |r|$:

$$\frac{d^2\ell}{db^2}\Big|_{b=|r|} = \frac{2\,|r|}{|r|^3} - \frac{1}{|r|^2} = \frac{2}{|r|^2} - \frac{1}{|r|^2} = \frac{1}{|r|^2} > 0.$$

A positive second derivative means the curve bends upward there: a minimum.

Worked example 41.2: The derivative at three points

Keep $|r| = 0.2$. Evaluate $d\ell/db = -|r|/b^2 + 1/b$ at $b = 0.1,\ 0.2,\ 0.4$.

$$b = 0.1: \quad -\frac{0.2}{0.01} + \frac{1}{0.1} = -20 + 10 = -10 \quad \text{(negative: increasing } b \text{ lowers the loss)}$$
$$b = 0.2: \quad -\frac{0.2}{0.04} + \frac{1}{0.2} = -5 + 5 = 0 \quad \text{(flat: the minimum)}$$
$$b = 0.4: \quad -\frac{0.2}{0.16} + \frac{1}{0.4} = -1.25 + 2.5 = 1.25 \quad \text{(positive: decreasing } b \text{ lowers the loss)}$$

Notice the asymmetry: at half the right answer the slope is -10, at twice it only $+1.25$. Gradient descent pushes hard against overconfidence and gently against vagueness.

41.4.1 Many pixels sharing one scale: the mean absolute error

A single pixel's error says nothing about "how hard this kind of place is". The head sees context, not the pixel, so it cannot tell apart many pixels whose contexts look alike. Suppose N such pixels, with errors $|r_1|, \ldots, |r_N|$, must share one predicted b. Their average loss is

$$\bar{\ell}(b) = \frac{1}{N}\sum_{i=1}^{N}\left(\frac{|r_i|}{b} + \log b\right)$$

the $\log b$ is the same in every term, so it comes out of the average; the $1/b$ factors out of the sum:

$$= \frac{1}{b}\cdot\frac{1}{N}\sum_{i=1}^{N}|r_i| \;+\; \log b \;=\; \frac{m}{b} + \log b, \qquad m = \frac{1}{N}\sum_{i}|r_i|\,.$$

This is the single-pixel loss with $|r|$ replaced by m, so by the same derivative $b^\star = m$: **the best shared scale is the mean absolute error of the pixels that share it.** With infinitely many pixels, the mean becomes the expectation, $b^\star(p) = \mathbb{E}\,|r(p)|$, the typical error at that kind of place. This matches the result $\mathbb{E}\,|r| = b$ of Section 41.2: if errors really were Laplace with scale b, the maximum-likelihood fit recovers it.

Worked example 41.3: Four pixels, one scale

Four hidden pixels of a field edge have errors 0.10, 0.30, 0.20, 0.40. Find the best shared b and check that it beats a neighbour value. Do it on paper.

$$\text{sum of errors} = 0.10 + 0.30 + 0.20 + 0.40 = 1.00$$
$$m = 1.00/4 = 0.25$$
$$\bar{\ell}(0.25) = 0.25/0.25 + \ln 0.25 = 1.000 + (-1.386) = -0.386$$
$$\bar{\ell}(0.20) = 0.25/0.20 + \ln 0.20 = 1.250 + (-1.609) = -0.359$$
$$\bar{\ell}(0.30) = 0.25/0.30 + \ln 0.30 = 0.833 + (-1.204) = -0.371$$

-0.386 is the lowest: the mean absolute error wins against both neighbours.

41.5 Predicting $\log b$, and the detached residual

A network's last layer is a convolution, which can output any real number, including negative ones. A scale must be positive. The standard trick, used here, is to let the network output

$s = \log b$ and compute $b = e^s$, which is positive for every real s. Rewrite the loss in terms of s:

$$\ell(s) = \frac{|r|}{e^s} + s = |r|\ e^{-s} + s,$$

and differentiate with respect to s (the chain rule: $\frac{d}{ds}e^{-s} = e^{-s} \cdot (-1)$):

$$\frac{d\ell}{ds} = -\,|r|\ e^{-s} + 1.$$

Setting it to zero gives $e^{-s} = 1/\,|r|$, so

$$s^\star = \log |r|\,, \quad \text{that is } b^\star = e^{s^\star} = |r|\,.$$

The minimum is in the same place, as it must be: reparameterising does not move the best answer, only the path to it.

Worked example 41.4: Gradients with respect to b and to $\log b$

This is the chain rule on a tiny example. Keep $|r| = 0.2$ and stand at $b = 0.1$, so $s = \ln 0.1 = -2.303$.

$$\frac{d\ell}{ds} = -0.2\, e^{2.303} + 1 = -0.2 \times 10.0 + 1 = -2.0 + 1 = -1.0$$
$$\frac{d\ell}{db} = -10 \quad \text{(from worked example 41.2)}$$
$$\frac{db}{ds} = e^s = b = 0.1$$
$$\frac{d\ell}{ds} = \frac{d\ell}{db} \cdot \frac{db}{ds} = -10 \times 0.1 = -1.0 \quad \text{(the same, as the chain rule promises)}$$

In b the slope was -10; in $\log b$ it is -1. Near zero, where b is small, the $\log b$ parameterisation tames gradients that would otherwise explode as $1/b^2$.

The clamp. Before using the head's output, the trainer clamps it to the range $-10 \le s \le 5$ [M: `finetune_trainer.py`]. In terms of b that is $e^{-10} \approx 0.0000454$ to $e^5 \approx 148.4$: wide enough never to bind on a sensible error, narrow enough that one bad pixel cannot produce an infinite loss.

The detached residual. The error $|r|$ in the loss is computed inside `torch.no_grad()`: it is *detached*, meaning no gradient flows from the uncertainty loss back into the reconstruction. Without this, the network could lower $|r|\,/b + \log b$ by changing its *errors* to suit b, rather than changing b to fit its errors. The docstring of `_scale_loss` says it in one line: "detached: the target, not something to shrink".

The trainer's loss, from that docstring:

```
e(p) = mean over valid bands of |normalised residual|   # (B, H, W), computed under
    no_grad
loss = mean over hidden p of  e(p) / b(p) + log b(p)    # lowest where b(p) = e(p)
```

Note that $e(p)$ is a per-pixel mean over bands, so $b(p)$ is one number per pixel, not one per band. The proposal rejected per-band scales on purpose: the per-band shape is RM's job.

41.6 The head itself, counted by hand

The *uncertainty head* is a third decoder on the shared encoder features, beside the reconstruction decoder and the segmentation head (Chapter 28). It is a `SegFormerDecoder` (Chapter 19) built with width 64 and one output channel:

```
scale_head = SegFormerDecoder(embed_dims=[32, 64, 160, 256], decoder_dim=64,
    out_channels=1)
#   features F1..F4: (B,32,H/4,W/4) (B,64,H/8,W/8) (B,160,H/16,W/16) (B,256,H/32,W
    /32)
#   -> 1x1 projections to 64, bilinear up to H/4, concat     (B, 256, H/4, W/4)
#   -> 1x1 fuse + GELU                                       (B, 64,  H/4, W/4)
#   -> 3x3 refine + GELU                                     (B, 64,  H/4, W/4)
#   -> 3x3 conv to 1 x 16 sub-pixel channels                 (B, 16,  H/4, W/4)
#   -> PixelShuffle(4)                                       (B, 1,   H,   W)    =
    log b
```

Every convolution has $c_{\text{in}} \times c_{\text{out}} \times k \times k$ weights plus c_{out} biases (Chapter 10).

Worked example 41.5: The uncertainty head's 95,632 parameters

Count each layer on paper, then add.

$$\begin{aligned}
\text{projection from } 32 &: 32 \times 64 + 64 = 2{,}048 + 64 = 2{,}112 \\
\text{projection from } 64 &: 64 \times 64 + 64 = 4{,}096 + 64 = 4{,}160 \\
\text{projection from } 160 &: 160 \times 64 + 64 = 10{,}240 + 64 = 10{,}304 \\
\text{projection from } 256 &: 256 \times 64 + 64 = 16{,}384 + 64 = 16{,}448 \\
\text{fuse } (4 \times 64 = 256 \to 64) &: 256 \times 64 + 64 = 16{,}384 + 64 = 16{,}448 \\
\text{refine } 3 \times 3 &: 64 \times 64 \times 9 + 64 = 36{,}864 + 64 = 36{,}928 \\
\text{sub-pixel } 3 \times 3 &: 64 \times 16 \times 9 + 16 = 9{,}216 + 16 = 9{,}232 \\
\text{total} &: 2{,}112 + 4{,}160 = 6{,}272 \\
&\quad 6{,}272 + 10{,}304 = 16{,}576 \\
&\quad 16{,}576 + 16{,}448 = 33{,}024 \\
&\quad 33{,}024 + 16{,}448 = 49{,}472 \\
&\quad 49{,}472 + 36{,}928 = 86{,}400 \\
&\quad 86{,}400 + 9{,}232 = 95{,}632
\end{aligned}$$

Source check: the head's size, verified

Building `SegFormerDecoder([32,64,160,256],decoder_dim=64,out_channels=1)` on CPU and summing `p.numel()` gives 95,632 [C]; with `out_channels=2` it gives 104,864 [C]. Script and output: `research/textbook/tools/compute_39-41.py` and `compute_39-41_output.txt`. The proposal's "~0.1 M parameters" [M: `uncertainty-head.md`] is this number rounded. For comparison, the FT04 model it rides on has 5,886,451 parameters with its segmentation head (Chapter 39): the head adds under 2%.

41.7 The one rule that keeps it honest: hidden pixels only

The head must judge how hard a place is *from its context, never from the pixel itself.* If the head could see the pixel, it would learn "this pixel looks odd, so expect a big error", and dividing by that b would divide every anomaly away. So the loss is averaged only over pixels that were *hidden from the encoder* by the masked-autoencoder mask (Chapter 18), that are valid, and that lie inside the eroded valid region. The head reads the same encoder features as the reconstruction decoder, and those features were computed with the hidden tokens removed.

Inference matches training. The product reconstructs a scene in two complementary checkerboard passes (Chapter 30), and `predict_full_scene_with_scale` runs the same passes with the head switched on, taking each pixel's $\log b$ from the pass in which it was hidden.

Frozen means frozen. FT06 trained in a new trainer mode, `mode:"scale"`. In the trainer's constructor (`__init__` of `FineTuneTrainer`) that mode sets `requires_grad=True` only for parameter names starting with `scale_head.`, and builds the optimiser over those alone; the training pass puts the whole model in evaluation mode and switches only `scale_head` back to training mode, so FT04's BatchNorm statistics are not updated either [M: `lld/finetune.md`]. The reconstruction and the masks are therefore exactly FT04's.

41.8 The score: dividing by b, with a floor

The scorer `scaled_rm` in `app/evaluation/scorers.py` does three things:

```
b = np.exp(log_b)                                                    # (H, W)
b = np.maximum(b, np.percentile(b[valid], scale_floor_pct)) ** scale_power   # floor,
    then power
score = blurred_mahalanobis((cube - recon) / b[None], valid, spec)       # RM,
    whitening refitted
```

The default floor is the 5th percentile [M: `scorers.py`]. The floor exists because dividing by a very small b inflates noise wherever the model is usually right (deep water, flat desert): the proposal listed this as a risk and floored b per scene. `scale_power=1` is the design, residual/b; `scale_power=0.5` is residual/$\sqrt{b}$, a softer version tried later.

Worked example 41.6: The floor at work

Suppose a scene's 5th percentile of b is 0.04. Three pixels have residual 0.20 and predicted b of 0.01, 0.04 and 0.10. Compute each scaled residual with the floor, then without.

$$\begin{aligned} \text{with floor:} \quad & \max(0.01, 0.04) = 0.04, \quad 0.20/0.04 = 5.0 \\ & \max(0.04, 0.04) = 0.04, \quad 0.20/0.04 = 5.0 \\ & \max(0.10, 0.04) = 0.10, \quad 0.20/0.10 = 2.0 \\ \text{without:} \quad & 0.20/0.01 = 20.0, \quad 0.20/0.04 = 5.0, \quad 0.20/0.10 = 2.0 \end{aligned}$$

Without the floor the first pixel's score is four times larger, purely because the head was very confident there. The floor caps the reward for confidence.

41.9 Spearman's rank correlation

Before any detection test, the proposal set a sanity check: on held-out tiles, $b(p)$ must *rank* pixels by their actual error, with Spearman correlation at least 0.5. If the head cannot predict its own error, the score cannot use it.

Definition 41.4 (Rank and Spearman correlation)**.** The *rank* of a value in a list is its position after sorting from smallest to largest (the repo counts from 0). *Spearman's rank correlation* [41] between two equal-length lists a and e is the ordinary (Pearson) correlation of their ranks:

$$\rho = \frac{\sum_i (u_i - \bar{u})(v_i - \bar{v})}{\sqrt{\sum_i (u_i - \bar{u})^2}\,\sqrt{\sum_i (v_i - \bar{v})^2}}, \qquad u_i = \operatorname{rank}(a_i),\ v_i = \operatorname{rank}(e_i). \tag{41.4}$$

It is 1 when the two lists are in the same order, -1 when in reverse order, and near 0 when the orders are unrelated.

The repo's `spearman` in `app/foundation_models/trainers/finetune_trainer.py` does exactly this: two `argsort` calls turn values into ranks, the ranks are centred on their mean, and the result is the dot product divided by the product of the two vectors' lengths (with ties broken by order, and at most the first 50,000 values). Spearman asks "does a larger b go with a larger error?", not "is b the right size?". Doubling every b leaves ρ unchanged.

When there are no ties, Equation (41.4) simplifies to the textbook shortcut

$$\rho = 1 - \frac{6\sum_i d_i^2}{n(n^2-1)}, \qquad d_i = u_i - v_i, \tag{41.5}$$

which you can use to check your arithmetic.

Worked example 41.7: Spearman on five pairs

Five hidden pixels. The head predicts b and the actual mean error is e:

$$b = (0.12,\ 0.40,\ 0.08,\ 0.25,\ 0.30), \qquad e = (0.10,\ 0.35,\ 0.15,\ 0.20,\ 0.50).$$

Do every step on paper.
Step 1, ranks (0 = smallest). For b: the sorted order is $0.08 < 0.12 < 0.25 < 0.30 < 0.40$, so $u = (1, 4, 0, 2, 3)$. For e: $0.10 < 0.15 < 0.20 < 0.35 < 0.50$, so $v = (0, 3, 1, 2, 4)$.
Step 2, centre. The mean of the ranks $0, 1, 2, 3, 4$ is $10/5 = 2$ for both lists.

$$u - \bar{u} = (-1,\ 2,\ -2,\ 0,\ 1), \qquad v - \bar{v} = (-2,\ 1,\ -1,\ 0,\ 2).$$

Step 3, dot product.

$$(-1)(-2) + (2)(1) + (-2)(-1) + (0)(0) + (1)(2) = 2 + 2 + 2 + 0 + 2 = 8.$$

Step 4, lengths. Both centred vectors are permutations of $(-2, -1, 0, 1, 2)$:

$$\sqrt{4 + 1 + 0 + 1 + 4} = \sqrt{10} = 3.162 \text{ each}; \qquad 3.162 \times 3.162 = 10.$$

Step 5. $\rho = 8/10 = 0.80$.
Check with Equation (41.5). $d = u - v = (1, 1, -1, 0, -1)$, so $\sum d^2 = 1 + 1 + 1 + 0 + 1 = 4$, and

$$\rho = 1 - \frac{6 \times 4}{5 \times (25 - 1)} = 1 - \frac{24}{120} = 1 - 0.20 = 0.80. \checkmark$$

The head ranks the errors well but not perfectly: it put pixel 2 (error 0.35) at the top, where pixel 5 (error 0.50) belongs.

41.10 The run: FT06

FT06's configuration is FT04's (the same mixed batches of EnMAP, PRISMA and AVIRIS-NG at shares 0.4 / 0.35 / 0.25), with `mode:"scale"`, `scale_decoder_dim:64`, initialisation from FT04's `best.pt`, 3 epochs and learning rate 5×10^{-4} on a cosine schedule [M: `FT06-uncertainty/README.md`]. The pass criteria, fixed before launch, were:

1. the head predicts its own error: validation Spearman$(b, e) \geq 0.5$ on every sensor;
2. detection on the real PRISMA bench against FT04 + RM, both with cloud excluded on FT04's masks: pAUC up to 1% false alarms (Chapter 33) at least +0.010 (0.170 to 0.180),

TPR at 0.1% not lower, *and* objects caught at the top 1% and 0.1% not lower;

3. bench-v2.1 caught top 1% not lower by more than 0.5 points.

41.10.1 Result 1: the head learned its error

epoch	EnMAP	PRISMA	AVIRIS-NG	val NLL
1	0.584	0.620	0.607	−1.331
2	0.600	0.631	0.625	−1.338
3	0.604	0.636	0.640	−1.341 (best)

Table 41.1: FT06 validation Spearman(b, e) per sensor and the head's NLL, per epoch. Source: [M: `FT06-uncertainty/README.md`] (in `research/experiments/indradhanu-finetune/`).

Table 41.1 shows criterion 1 met on every sensor from the first epoch, and rising every epoch. The NLL falls each epoch too, and since mode `"scale"` ranks epochs by that NLL ("scale by its NLL", `docs/lld/finetune.md`), `best.pt` is epoch 3. Hold on to that: "best" here means best at predicting its own error.

41.10.2 Result 2: detection got worse

	pAUC ≤1%	TPR @ 0.1%	top 1%	top 0.1%	ROC-AUC
FT04 + RM (the bar)	**0.170**	0.065	**46.0%**	15.8%	**0.852**
FT06 + RM	0.170	0.065	46.0%	15.8%	0.852
FT06 + scaled RM, residual/b	0.125	0.064	37.8%	13.2%	0.708
FT06 + scaled RM, residual/$\sqrt{b}$	0.163	**0.072**	44.8%	**16.6%**	0.805
epoch 1 + residual/$\sqrt{b}$	0.176	0.079	46.1%	17.3%	0.822

Table 41.2: FT06 detection on the real PRISMA bench, cloud excluded on FT04's masks. "Top" columns are objects caught. Source: [M: `FT06-uncertainty/README.md`].

Values that will drift

FT06 results, run 2026-10-20 (`research/experiments/indradhanu-finetune/FT06-uncertainty/`): Tables 41.1 and 41.2. bench-v2.1 caught top 1%: FT04 36.2%, scaled 33.9%, sqrt 36.2%. Subtle tier caught top 1% with sqrt, cloud excluded: 40.3% to 37.7% [M: `FT06-uncertainty/eval_report.md`]. Set2 pAUC fell 0.128 to 0.097 under residual/b.

Read Table 41.2 row by row.

- **FT06 + RM equals FT04 + RM to every digit.** That is the frozen-backbone check working, not a bug: the reconstruction is FT04's, so plain RM on it is FT04's score.
- **As designed, residual/b loses about a quarter of its pAUC**: $0.170 - 0.125 = 0.045$, and $0.045/0.170 = 0.26$. It loses $46.0 - 37.8 = 8.2$ points of objects caught at the top 1%.
- **The square-root variant lifts the very top and loses overall**: TPR at 0.1% goes 0.065 to 0.072, but pAUC drops to 0.163.
- The proposal expected most of any gain on medium and subtle objects. The opposite happened.

No row meets criterion 2. FT06 was not adopted.

41.11 Why: the head expects large errors exactly where targets are

The diagnosis script `b_at_targets.py` asks where the head puts its expected error, on PRISMA sets 1, 2 and 4, for epochs 1 and 3. For each scene it takes the target pixels T and a *ring* around them,

```
ring = binary_dilation(t, iterations=8) & ~binary_dilation(t, iterations=3) & valid
```

and reports two numbers: the percentile of b at target pixels against every valid pixel in the scene, and the *target / ring ratio*

$$\text{ratio} = \exp\big(\,\text{median}_T \log b - \text{median}_{\text{ring}} \log b\big) = \frac{\text{median}_T\, b}{\text{median}_{\text{ring}}\, b}, \tag{41.6}$$

where the second form holds because exp is increasing (the median of $\log b$ is the log of the median of b) and $e^{x-y} = e^x/e^y$.

Source check: what "3–8 px" means

The FT06 README's observations call the ring "3-8 px", and the course repeats it. The code (`FT06-uncertainty/b_at_targets.py`) removes everything within 3 dilation steps and keeps up to 8, so the ring is pixels at distance 4 to 8 from the target. `binary_dilation`'s default structuring element is a cross, so the distance is the city-block distance $|\Delta x| + |\Delta y|$. The README's evidence line says this correctly: "the ring is 4-8 dilation steps, city-block distance".

	set1	set2	set4
target / ring b ratio, epoch 1	1.05	1.12	1.04
target / ring b ratio, epoch 3	1.07	1.19	1.13

Table 41.3: Median b on targets over median b on their own 4–8 px ring. Target pixels also sit at the 87th–90th percentile of b across each scene. Source: [M: `FT06-uncertainty/README.md`].

Two separate facts, and both hurt.

- **Across the scene**, targets sit on built-up surfaces, which are hard to reconstruct, so the head rightly expects large errors there (the 87th–90th percentile).
- **Locally**, b on the target is higher than on its own ring. That should not happen if the head only judged "how hard is this kind of place": the ring is the same kind of place.

41.11.1 Why the hidden-pixel rule did not stop it: halo arithmetic

The rule hides the pixel from the encoder. But the encoder hides whole 4×4-pixel tokens (stage 1 is a non-overlapping kernel-4, stride-4 convolution, Chapter 13), and a target is not confined to its token. Its *PSF halo* (the sensor's blur that spreads some of a target's light into neighbouring pixels, Chapter 34) and any part of a target larger than one token can sit in the *neighbouring* tokens, which are visible about 35% of the time.

Worked example 41.8: How often a halo leaks out of its token

Take a one-pixel target whose light reaches h pixels on either side. Its footprint along one axis is $2h + 1$ pixels wide. Assume (an illustrative assumption [A]) the target's position within its 4-pixel token is equally likely to be any of the 4 offsets $0, 1, 2, 3$. The footprint

stays inside the token along that axis only if it starts at offset ≥ 0 and ends at offset ≤ 3.

$$h = 0 : \text{width } 1;\ \text{fits at offsets } 0, 1, 2, 3 \to 4 \text{ of } 4 = 1$$
$$h = 1 : \text{width } 3;\ \text{the target must be at offset 1 or 2} \to 2 \text{ of } 4 = 1/2$$
$$h = 2 : \text{width } 5 > 4 \to 0 \text{ of } 4 = 0$$

Both axes must fit, so for $h = 1$ the footprint is inside one token with probability $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$: three times in four, some of the target's light lands in a neighbouring token. For $h = 2$ it always does. So for almost every target, a visible neighbour carries a trace of it.

The head learns "something odd is next to here, expect a big error". It is right about that: the odd spectrum next door genuinely predicts a large reconstruction error. And dividing by b then explains the anomaly away.

A head trained to predict its own error will predict a large error on an anomaly whenever it can see any trace of it, and then the anomaly is no longer surprising.

Worked example 41.9: Dividing by b, by hand

Use set2's epoch-3 ratio, 1.19. Keep to one whitened band, ignore that the whitening is refitted (it rescales every pixel alike, so it does not change the order), and score by squared distance from a ring mean of 0 (RM's score is a squared distance). Three pixels: a target with residual 3.0 and $b = 1.19$; a pixel in its ring with residual 1.0 and $b = 1.00$; a flat-field pixel elsewhere with residual 2.6 and $b = 1.00$. The residuals are illustrative [A]; only the ratio 1.19 is measured.

Plain RM:

$$\begin{aligned} \text{target} &: 3.0^2 = 9.00 \\ \text{ring pixel} &: 1.0^2 = 1.00 \\ \text{flat field} &: 2.6^2 = 6.76 \qquad \text{(target ranks first)} \end{aligned}$$

Residual/b:

$$\begin{aligned} \text{target} &: 3.0/1.19 = 2.521, \quad 2.521^2 = 6.36 \\ \text{ring pixel} &: 1.0/1.00 = 1.000, \quad 1.000^2 = 1.00 \\ \text{flat field} &: 2.6/1.00 = 2.600, \quad 2.600^2 = 6.76 \qquad \text{(the flat field now outranks the target)} \end{aligned}$$

The target's score fell from 9.00 to 6.36: divided by $1.19 \times 1.19 = 1.416$, about 29% lower. It fell below an ordinary bright pixel elsewhere, which is how objects drop out of the top 1%.

Residual/$\sqrt{b}$: $\sqrt{1.19} = 1.091$, so $3.0/1.091 = 2.750$, and $2.750^2 = 7.56$, still above 6.76, which is why the square-root variant hurts less.

41.11.2 Why epoch 1 looked better than epoch 3

The ratios in Table 41.3 grew with training: set1 1.05 to 1.07, set2 1.12 to 1.19, set4 1.04 to 1.13. As the head got better at its own job (Spearman up every epoch), it also learned more of the "something odd next door" signal, because that signal genuinely predicts large errors. More of it means more of each target divided away. So epoch 1 with the square root scored pAUC 0.176, and epoch 3, the `best.pt` by NLL, fell below FT04.

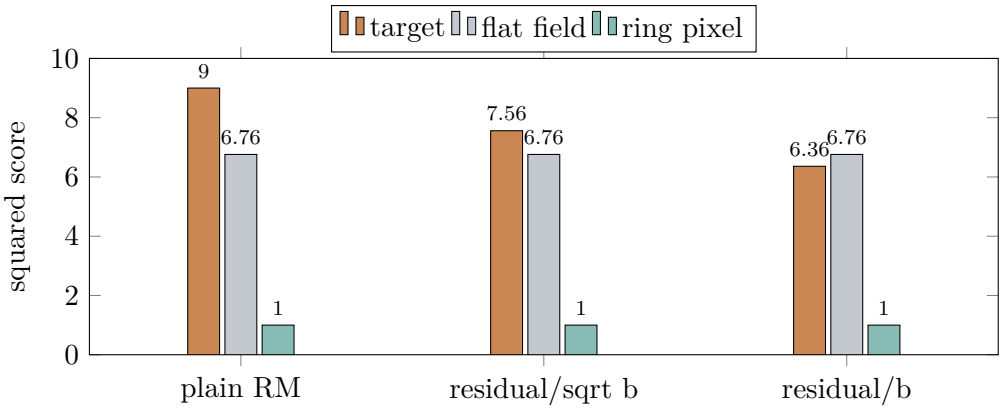


Figure 41.3: The three pixels of worked example 41.9, drawn to scale. Dividing by b with a target/ring ratio of 1.19 drops the target below an ordinary flat-field pixel; the square root keeps it a little above.

Predicting your own error well is not the same goal as detection: the checkpoint chosen by the head's own loss was the worst one for the product.

41.11.3 The selection-risk caveat

Do not read the epoch-1 row as a gain to ship. The square-root variant was *chosen after seeing epoch-1 results on the real bench*, and then judged on the same bench. When you try variants and keep the one that looks best on the data that will also grade it, part of the improvement is luck in that data, and you cannot tell how much. This is *selection risk*. Even at face value, epoch 1's pAUC of 0.176 is below the 0.180 bar. Chapter 36 later put a paired bootstrap interval on the epoch-3 square-root row: -0.0067 $[-0.0174, +0.0037]$ [M: `bench-resolution/README.md`], a tie.

41.12 What might rescue it, and what it taught

The FT06 README lists two ideas, both untried and both costing a retrain:

1. hide a wider context when training the head, for example 3×3 tokens around each predicted one, so the head never sees a target's halo or its other parts;
2. predict b from the reconstruction branch's features of a heavily masked pass only.

The README recommends neither before a cheaper idea. The halo problem did not go away: it is the motivation for the block masking of Chapter 42.

A practical note from the run: the evaluation took about 2.3 hours rather than the expected 1, because two models plus the scale pass filled the laptop GPU's 20 GB [M: `FT06-uncertainty/README.md`].

41.13 Common confusions

Spearman met its bar, so the head works.
It works at its own goal. Criterion 1 was a sanity check: a head that cannot predict its error is useless. Passing it was necessary, not sufficient; detection is the criterion that counts.

The hidden-pixel rule should have prevented this.
It prevents the head from seeing the pixel. It does not stop the head from seeing neighbouring

tokens, which carry the target's halo and any part larger than one 4×4 token (worked example 41.8).

FT06 + RM matching FT04 to every digit means the evaluation is broken.
It is the check that the backbone stayed frozen. Different numbers there would have been the bug.

The 1.19 is a percentile.
No. It is a ratio of medians, target b over ring b. The 87th–90th percentile is a separate measure, of target b against the whole scene.

The Laplace NLL has a log 2 that the code forgot.
It was dropped on purpose: a constant does not change where the minimum is, so it does not change any gradient.

Why predict log b instead of b?
A convolution can output negative numbers; e^s is always positive. It also turns the $1/b^2$ gradient near zero into a gentle one (worked example 41.4).

This is the same as RM's whitening.
RM divides by a per-scene spread along each error *direction*. $b(p)$ is a per-*pixel* scale. The design keeps per-band shape with RM on purpose.

best.pt is the best model.
It is the epoch with the lowest scale NLL, which mode `"scale"` uses to rank epochs. Here that was the worst epoch for detection.

41.14 Check yourself

Exercise 41.1. For $|r| = 0.5$, compute $\ell = |r|/b + \ln b$ at $b = 0.25$, 0.5 and 1.0. Which is lowest? Then compute $d\ell/db$ at each of the three.

Exercise 41.2. Five pixels have $b = (0.3, 0.1, 0.5, 0.2, 0.4)$ and errors $e = (0.25, 0.05, 0.30, 0.40, 0.45)$. Compute Spearman's ρ in two ways: with Equation (41.4), and with Equation (41.5).

Exercise 41.3. What would the head's parameter count be with decoder width 32 (and still one output channel)? Count every layer.

Exercise 41.4. Redo worked example 41.9 with set1's epoch-3 ratio, 1.07. What is the target's squared score under residual/b, and does it still outrank the flat field at 6.76? By what factor does residual/$\sqrt{b}$ shrink the squared score with ratio 1.19?

Exercise 41.5. A Gaussian error model gives the NLL $r^2/(2\sigma^2) + \log \sigma$ (constants dropped). Find the best σ for one error r, and for N errors sharing one σ. Compare with the Laplace result.

Exercise 41.6. Spearman rose every epoch while detection fell. Name the mechanism, and say why the epoch-1 square-root row is not evidence of a gain.

Next: Chapter 42, Gated losses, block masking, the decoder bottleneck and FT09.

Chapter 42

Gated losses, block masking, the decoder bottleneck and FT09

The one thing this chapter teaches

Changing what the training loss sees (gating the worst pixels, hiding tokens in blocks) cost a little detection, because most of what a model gets wrong is ordinary hard background; a diagnosis without training then showed that FT04's error is mostly detail its decoder cannot render, because each 4×4 token is squeezed into 32 numbers with no full-resolution path back, and the progressive decoder built to fix that (FT09) raised contrast and gave a gain that cleared the bootstrap by a hair and then did not survive five more epochs.

This chapter covers three days of experiments, 2026-10-21 to 2026-10-23, after the uncertainty head of Chapter 41 failed. It is three stories with one thread. The first two runs (FT07, FT08) changed the *training signal*. The diagnosis that followed ("is the decoder the bottleneck?") asked where the error actually comes from. The third run (FT09) changed the *shape of the network*. Each story carries a piece of mathematics: quantiles and what a gate removes from a gradient; expected numbers of visible neighbours under random masking; projection onto a span and the Pythagorean split of an error; a counting argument about information through a bottleneck; and a by-hand parameter count of a new decoder.

Provenance tags abbreviate paths as in Chapter 41: `FT07-gated/`, `FT09-decoder/` and the like are folders under `research/experiments/indradhanu-finetune/`; `decoder-capacity/` and `bench-resolution/` are under `research/experiments/`.

Analogy

A police sketch artist draws faces from a witness's description, and the sketches come out poor. Is the witness's memory the problem, or the artist's pencil? You could coach the witness to leave out the details they remember worst (that is FT07), or to describe the face in bigger pieces (FT08), and the sketches barely change. The decisive test is to put the real face in front of the artist and ask for a copy. If the copy is barely better than the sketch from memory, the memory was never the limit: the pencil is. That test is Section 42.5, and FT09 is a finer pencil.

42.1 The question behind FT07 and FT08

A masked autoencoder fine-tuned on shards that contain real targets can learn to reconstruct them, and a target it reconstructs well leaves no residual to score (Chapter 9). GT-HAD [25]

gates its training: it keeps likely anomalies out of what the network learns from. The anomaly course tried a gate per scene at inference time and found that a short run never learns the targets anyway, so the gate changed nothing (Chapter 40). Five epochs over the whole training mix is different: it sees every target-like pixel many times. So two runs, both from FT04's `best.pt`, 5 epochs, learning rate 5×10^{-5} on a cosine schedule, FT04's configuration otherwise [M: `FT07-gated/README.md`]:

run	change	configuration fields
FT07	gated loss	`gate_pct: 1.0`, `gate_halo: 2`
FT08	gated loss + 2×2-token block masking	as FT07, plus `mask_block: 2`

The expectation, written before launch: FT07 a small gain at the top 0.1% and slightly worse reconstruction; FT08 the larger detection change, in either direction.

42.2 Gating the worst 1%

42.2.1 Quantiles

Definition 42.1 (Quantile)**.** The *q-quantile* of a list of numbers, for $0 \le q \le 1$, is the value below which a fraction q of the list lies. With n sorted values $x_{(0)} \le x_{(1)} \le \cdots \le x_{(n-1)}$ (counting from 0), `torch.quantile` uses the position $q(n-1)$ and interpolates linearly between the two neighbouring sorted values: at position $k + f$ (k an integer, $0 \le f < 1$) the quantile is $x_{(k)} + f\,(x_{(k+1)} - x_{(k)})$.

The function `gate_pixels` in `app/foundation_models/trainers/finetune_trainer.py` does four things, for each source group (each sensor's tiles) in a mixed batch (Chapter 29):

```
err  = per-pixel L1, mean over valid bands                          # (B, 1, H, W)
cut  = torch.quantile(err[loss pixels][:1_000_000], 1 - pct/100)  # the group's top
    pct % threshold
gate = (err >= cut) & loss_pixel
gate = max_pool2d(gate, 2*halo + 1, stride=1, padding=halo) > 0   # dilate to a (2
    halo + 1)-wide square
```

and the reconstruction loss mask becomes `loss_mask*~gate`. Three details matter.

- `x_hat` is *detached* before the gate is computed: the gate is a selection made without gradient, so the model cannot learn to change which pixels get gated.
- The quantile is taken over *loss pixels only* (hidden, valid pixels), since only they carry loss.
- At most the first 1,000,000 loss pixels enter the quantile, because `torch.quantile` rejects very large inputs. Eight 128 px tiles are $8 \times 128 \times 128 = 131{,}072$ pixels, so at FT07's batch size of 8 the cap is never reached.

Worked example 42.1: The top 1% of 100 loss pixels, with its halo

Do this on paper. A group has 100 loss pixels laid out as a 10×10 patch, all hidden, with

distinct errors.

$$\begin{aligned}\text{quantile level} &= 1 - 1/100 = 0.99\\ \text{position} &= 0.99 \times (100-1) = 98.01 \quad (k = 98,\ f = 0.01)\\ \text{cut} &= x_{(98)} + 0.01\,(x_{(99)} - x_{(98)})\\ &\quad (1\% \text{ of the way from the 99th to the 100th smallest})\\ \text{err} \geq \text{cut} &: \text{only } x_{(99)}, \text{the single largest error}\\ \text{gated before the halo} &= 1 \text{ pixel} = 1\% \text{ of } 100\end{aligned}$$

Now the halo, `gate_halo=2`: `max_pool2d` with kernel $2 \times 2 + 1 = 5$ turns one pixel into a 5×5 square.

$$\begin{aligned}\text{gated pixel in the middle} &: 5 \times 5 = 25 \text{ pixels}; \quad 100 - 25 = 75 \text{ loss pixels left}\\ \text{gated pixel in a corner} &: 3 \times 3 = 9 \text{ pixels}; \quad 100 - 9 = 91 \text{ loss pixels left}\end{aligned}$$

A 1% gate removed a quarter of this group's loss pixels.

42.2.2 What fraction of the gradient a gate removes

"Removing 1% of the pixels" sounds small. Two different things could be meant by how much is removed: the share of the *loss value*, or the share of the *gradient*. For L1 they differ sharply.

The L1 loss over N loss pixels and C bands is $L = \frac{1}{NC}\sum_{p,c} |\hat{x}_{pc} - x_{pc}|$. Differentiate with respect to one reconstructed value $\hat{x}_{pc}$. Only one term contains it, and the derivative of $|u|$ is $\text{sign}(u)$ (for $u \neq 0$):

$$\frac{\partial L}{\partial \hat{x}_{pc}} = \frac{1}{NC}\,\text{sign}(\hat{x}_{pc} - x_{pc}). \tag{42.1}$$

Every included pixel-band pushes on the network with the *same strength*, $1/(NC)$, however large its error. So the share of gradient terms the gate removes is the share of loss pixels it removes, while the share of loss *value* it removes is larger, because the gated pixels are the large ones. (Under an L2 loss the gradient would be proportional to the error, and the worst pixels would dominate the gradient as well.)

Worked example 42.2: Loss share against gradient share

A group of 100 loss pixels: 99 have error 0.05 and one has error 0.50 (illustrative numbers [A]). Gate the worst one, without a halo.

$$\begin{aligned}\text{total loss value} &= 99 \times 0.05 + 0.50 = 4.95 + 0.50 = 5.45\\ \text{share of loss value removed} &= 0.50/5.45 = 0.092 \quad (9.2\%)\\ \text{share of L1 gradient terms removed} &= 1/100 = 0.01 \quad (1\%)\\ \text{under L2: } \textstyle\sum 2\,|r| &= 99 \times 0.10 + 1.00 = 9.90 + 1.00 = 10.90\\ \text{share of L2 gradient size removed} &= 1.00/10.90 = 0.092 \quad (9.2\%)\end{aligned}$$

With the halo of 25 pixels, the gradient share is 25%, and almost all of those 24 extra pixels are ordinary.

How large the gated share was in FT07 is not recorded: the run logged no gated share. You can bound it.

Worked example 42.3: An upper bound on a real batch

Use FT07's batch of 8 tiles of 128×128 and assume ([A]) about 65% of pixels are hidden and valid, and the gated pixels are isolated, far from edges and from each other.

$$\begin{aligned}
\text{pixels} &= 8 \times 128 \times 128 = 131{,}072 \\
\text{loss pixels} &\approx 0.65 \times 131{,}072 = 85{,}197 \\
\text{gated before halo} &\approx 0.01 \times 85{,}197 = 852 \\
\text{loss pixels per square} &\approx 0.65 \times 25 = 16.25 \quad \text{(the square also covers visible pixels)} \\
\text{removed} &\approx 852 \times 16.25 = 13{,}845 \\
\text{share} &\approx 13{,}845/85{,}197 = 0.16
\end{aligned}$$

At most about 16%. Real high-error pixels come in clumps whose squares overlap, so the true share is lower.

42.3 Block masking

42.3.1 Why blocks

Chapter 41 found that the head could see a target "through the neighbours": a target's halo reaches into neighbouring tokens, and a model filling a hidden token can copy the odd spectrum from next door. Hide bigger units and less of the target is left in view. FT08 hides tokens in 2×2 blocks, 8 px on a side.

Worked example 42.4: A 5 px footprint on a 4 px and an 8 px grid

A one-pixel target whose light reaches 2 px either side has a footprint $2 + 1 + 2 = 5$ px wide. Assume ([A]) its starting offset on the grid is uniform.

$$\begin{aligned}
&4\,\text{px token} : 5 > 4, \ \text{never fits} : 0 \text{ of } 4 \text{ offsets} \\
&8\,\text{px block} : \text{fits if it starts at offset } 0, 1, 2, 3 : \ 4 \text{ of } 8 = 1/2 \text{ per axis} \\
&\text{both axes} : 1/2 \times 1/2 = 1/4
\end{aligned}$$

So a block contains the whole footprint about a quarter of the time; otherwise it straddles a border and leaks unless the neighbouring block is hidden too.

42.3.2 How the block mask is drawn

`_build_prediction_mask` in the fine-tune trainer, with `mask_block=b>1`:

```
tokens = valid-token mask                                    # (B, N), N = Ht * Wt
grid   = tokens.reshape(B, 1, Ht, Wt)
blocks = max_pool2d(grid, b, ceil_mode=True) > 0          # (B, 1, Ht/b, Wt/b): valid
     if any token is
_, block_pred = generate_prediction_mask(blocks, mask_ratio)   # hide mask_ratio of
    the valid blocks
pred   = interpolate(block_pred, scale_factor=b, mode="nearest")[..., :Ht, :Wt] *
    tokens
```

Worked example 42.5: How much a 128 px tile hides

At `mask_ratio` 0.65:

$$\begin{aligned}\text{tokens per side} &= 128/4 = 32; \quad \text{tokens} = 32 \times 32 = 1{,}024\\ \text{blocks per side} &= 32/2 = 16; \quad \text{blocks} = 16 \times 16 = 256\\ \text{blocks hidden} &= 256 \times 0.65 = 166.4 \to 166 \text{ (truncated)}\\ \text{tokens hidden} &= 166 \times 4 = 664; \quad 664/1{,}024 = 0.648\\ \text{single tokens} &: 1{,}024 \times 0.65 = 665.6 \to 665\end{aligned}$$

FT08 hides almost exactly as much as FT04; it hides it in bigger pieces.

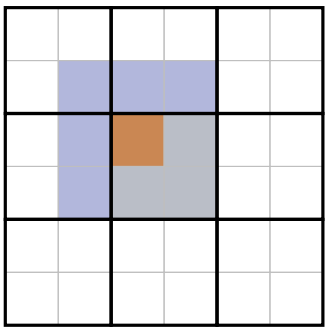
42.3.3 Expected visible neighbours

How much context does a hidden token keep? Count its *visible neighbours*: of the 8 tokens around it (sides and corners), how many are visible. Ignore tile borders ([A]).

Random single tokens. Given that our token is hidden, the remaining 1,023 tokens contain $665 - 1 = 664$ hidden and $1{,}024 - 665 = 359$ visible ones. Each neighbour is visible with probability 359/1,023. By *linearity of expectation* (the expected count is the sum of the probabilities, whether or not the events are independent):

$$\mathbb{E}[\text{visible neighbours}] = 8 \times \frac{359}{1{,}023} = 8 \times 0.3509 = 2.807. \tag{42.2}$$

2×2 blocks. Every token in a 2×2 block sits at a corner of its block. Take the top-left token. Of its 8 neighbours, 3 are in its own block (hidden for certain), 2 in the block above, 2 in the block to the left, 1 in the diagonal block.



amber: the hidden token
grey: its own block, hidden
indigo: the 5 neighbours in 3 other blocks

Figure 42.1: The top-left token of a hidden 2×2 block (thick outlines are blocks). Three of its 8 neighbours are in its own block and are certainly hidden; the other 5 lie in three neighbouring blocks.

Given our block is hidden, the other 255 blocks contain $166 - 1 = 165$ hidden and $256 - 166 = 90$ visible ones. Each of the 5 outside neighbours is visible with probability 90/255:

$$\begin{aligned}\mathbb{E}[\text{visible neighbours}] &= 3 \times 0 + 5 \times \frac{90}{255}\\ &= 5 \times 0.3529 = 1.765.\end{aligned}$$

Blocks cut the expected visible context from 2.807 to 1.765 neighbours, by a factor $1.765/2.807 = 0.63$. That is the intended effect: less of a target's halo in view. The cost is also visible in the arithmetic: ordinary texture is now rebuilt from neighbours further away.

42.3.4 Inference must match

The product reconstructs in two complementary checkerboard passes (Chapter 30). A model trained to fill 8 px holes but asked to fill 4 px ones sees more context than it trained with. So FT08 had to be inferred with checkerboard cells of `mask_block` tokens. On a 32×32-token tile with cell 2: 16×16 cells, and each pass hides 128 cells = 512 tokens = 50%.

The parent inferencer (`app/foundation_models/inferencers/segformer_mae_inferencer.py`) makes *two* checkerboard choices, both from `config.checkerboard_cell_size`: which tokens each pass hides (the keep masks, built by `_checkerboard_keep_mask`) and which pass each pixel's prediction is taken from (`pass1_pixels`, `pass2_pixels`). When FT08 was run, `SpectralStemInferencer` overrode only the first. The second still used cells of one token. Worked example 42.6 counts what that did, and Section 42.4.1 tells how it was found and fixed.

Worked example 42.6: Half of each block taken from a pass that saw it

A 4×4-token tile, block 2. Write $h(i,j) = 1$ if pass 1 hides token (i,j) (cells of 2 tokens, pass 1 hiding the cells whose block row plus block column is odd) and $s(i,j) = 1$ if the pixel's prediction is taken from pass 1 (cells of 1 token, $i + j$ odd). A token is predicted correctly when $s = h$: taken from pass 1 exactly when pass 1 hid it.

h				s				leak ($s \neq h$)			
0	0	1	1	0	1	0	1	.	x	x	.
0	0	1	1	1	0	1	0	x	.	.	x
1	1	0	0	0	1	0	1	x	.	.	x
1	1	0	0	1	0	1	0	.	x	x	.

Count the leaks: two per row, $2 + 2 + 2 + 2 = 8$ of 16 tokens, $8/16 = 0.5$. The reason is local: inside any 2×2 block h is constant, while s takes each value on exactly 2 of the 4 tokens. So in every block, whichever way the two patterns line up, 2 of 4 tokens agree and 2 disagree. **Half of each block's pixels were predicted by the pass in which their block was visible.**

Source check: checkerboard cell size is in tokens

`InferenceConfig.checkerboard_cell_size` (`app/models/training/inference_config.py`) was described as the "Size of each checkerboard cell in pixels". The code passes it to `checkerboard_token_mask` as `cell_size`, documented there as "checkerboard cell size in tokens", and the inferencer compares it with `mask_block`, also in tokens. It is a token count. The description was corrected to "stage-1 tokens (4 px each)" on 2026-10-21 after the course found it.

What validation measures. Both runs' per-epoch validation is ungated and uses single-token masks: `compute_validation_loss` in `spectral_stem_mae_trainer.py` builds two complementary random token masks itself, without `_build_prediction_mask` and without `gate_pixels`. So validation numbers compare with FT04's, and for FT08 they are measured under a masking different from its training and its inference.

42.4 FT07 and FT08: results

The pass criteria, fixed before launch, against FT04 + RM with cloud excluded on the real PRISMA bench: (1) pAUC $\leq$1% at least +0.005 *and* its paired-bootstrap 95% interval above zero (Chapter 36); (2) objects caught at the top 1% and 0.1% not lower; (3) bench-v2.1 caught

top 1% not lower by more than 0.5 points; (4) validation SAM within 0.1° of FT04 (3.88 / 4.11 / 5.46 for EnMAP / PRISMA / AVIRIS-NG) and EnMAP IoU water / cloud within 0.01 (0.839 / 0.810).

	pAUC	top 1% / 0.1%	TPR @ 0.1%	ROC-AUC	v2.1 top 1%
FT04	**0.170**	46.0% / 15.8%	0.065	0.852	36.2%
FT07	0.165	46.1% / 15.4%	0.062	–	36.4%
FT07 epoch 1	0.169	46.3% / 15.5%	0.063	–	–
FT08	0.164	46.4% / 16.1%	0.062	0.844	36.7%
Paired bootstrap against FT04 (1000 draws), pAUC: FT07 -0.0047 $[-0.0082, -0.0020]$; FT08 -0.0058 $[-0.0106, -0.0010]$					

Table 42.1: FT07 and FT08 detection, real PRISMA bench, cloud excluded on FT04's masks; "top" is objects caught. Both best checkpoints are epoch 4. Source: [M: `FT07-gated/README.md`].

Values that will drift

Run 2026-10-21. FT07 validation at epoch 4: SAM 3.90 / 4.16 / 5.47°, IoU 0.840 / 0.813. FT08 at epoch 4: SAM 3.92 / 4.17 / 5.33°, IoU 0.841 / 0.807; FT08's AVIRIS-NG SAM swung 5.78 (epoch 3), 5.33 (4), 5.60 (5), as large as the 0.1° guard [M: `FT07-gated/README.md`].

Both failed criterion 1: the intervals lie wholly below zero, a real loss, small. Objects caught were a tie. Reconstruction and masks held (criterion 4), so the cost is specific to detection. FT07's cost grew with training: epoch 1 was -0.001, epoch 4 -0.005.

Why gating hit the wrong pixels. Real targets are under 0.1% of pixels, and many are not the worst-reconstructed in a batch. The top 1% of training error is mostly hard *normal* pixels: field edges, texture, glint, shoreline. Gating them leaves the model slightly worse exactly where RM's false alarms come from, and does little to the targets.

Why blocks moved objects one way and pixels the other. Objects caught went up a little for FT08 (unresolved) while pAUC fell. Hiding 8 px blocks makes the model reconstruct each hidden pixel from further away (Section 42.3.3), so ordinary texture is rebuilt less exactly: more background residual everywhere, while targets gain little extra contrast.

Making the training ignore what it gets wrong does not make it ignore targets, because most of what it gets wrong is ordinary hard background.

42.4.1 The leak in FT08's evaluation, and the re-run

The numbers above for FT08 were measured with the pass selection of worked example 42.6. On 2026-10-31, while this book was being written, its authors found the mismatch, and it was fixed the same day (commit d025059). `SpectralStemInferencer.__init__` now copies the model's block into the inference configuration before the parent is built, so both checkerboard choices read the same number:

```
def __init__(self, config):
    block = getattr(config.model_config_, "mask_block", 1)
    if block > config.checkerboard_cell_size:
        config = config.model_copy(update={"checkerboard_cell_size": block})
    super().__init__(config)
```

The old `_checkerboard_keep_mask` override is gone. A test pins it: `test_block_inference_never_predicts_a_pixel_from_a_pass_that_saw_it` (`tests/test_foundation_models/test_gated_training.py`) builds a small model with `mask_block=2`, adds 0.4 to every band of one 8×8-pixel block and checks that the predictions inside that block do not change. If any of them came from a pass that saw the block, they would. The FT07 README records that the test fails without the fix. FT04 and Heavy have `mask_block` 1 (the configuration default), so the `if` is false and nothing changes for them.

FT08 was re-evaluated (`configs/eval/ft08_fixed.json`):

	pAUC	caught top 1% / 0.1%	paired difference [95% interval]
FT08, leaky (2026-10-21)	0.164	46.4% / 16.1%	−0.0058 [−0.0106, −0.0010] vs FT04
FT08, fixed (2026-10-31)	0.1638	46.1% / 15.4%	−0.0057 [−0.0101, −0.0013] vs FT04
			−0.0010 [−0.0047, +0.0028] vs FT07

Source: [M: `research/experiments/indradhanu-finetune/FT07-gated/README.md`] (last section).

Against FT04 the clean FT08 is still a resolved loss; against FT07, the same gated loss without blocks, it is a tie. Blocks added nothing on top of gating. **The verdict "block masking does not help" stands, now on a measurement without the leak.**

Why did predicting half the pixels from a pass that could see them move pAUC by only 0.0002? A likely reason, not tested for FT08: Section 42.5 measures that FT04 reproduces pixels it can see barely better than ones it must predict (set2 background 0.110 visible against 0.137 hidden). A model that cannot copy what it sees gains little from seeing it.

Lesson: one setting, read in two places

A configuration value was read in two places and overridden in one. Nothing crashed, the numbers were plausible, and the conclusion happened to survive; but nobody could know that without re-running the evaluation. Two habits follow. When a model needs a non-default inference setting, change it where *every* reader of the setting sees it (here, the configuration), not in one of the methods that reads it. And write the test that states the invariant directly ("no pixel is predicted by a pass that saw it"), so a future override cannot quietly break it again. The leak was found by people reading the code to explain it, which is one argument for writing explanations at all.

42.5 Is the decoder the bottleneck? A diagnosis without training

Every change since FT04 had altered the loss, the masking or the score, never the shape of the network. The user's prompt was: "Maybe a bigger model is the answer", "Or maybe more layers of decoding", "I feel the decoder maybe too un-expressive given the sophistication of the encoder." Before building anything, two measurements on the laptop, with no training, asked *where* FT04's error comes from [M: `decoder-capacity/README.md`].

42.5.1 Where the parameters are

component	parameters	role
encoder	3,331,072	four SegFormer stages (Chapter 17)
decoder (reconstruction)	2,164,736	fuse at 1/4, then PixelShuffle(4) (Chapter 19)
segmentation head	371,536	water / cloud masks (Chapter 28)
`spectral_in`	14,721	bands to 32 numbers per pixel (Chapter 24)
`spectral_out`	4,257	32 numbers per pixel to any band list

Building the FT04-sized model on CPU confirms the encoder, decoder and stem figures [C]; the script and its output are in `research/textbook/tools/`. The decoder is 39% of encoder plus decoder: 2,164,736/5,495,808 = 0.39. It is not small, so "make the decoder bigger" is not the obvious fix. The question is not how many parameters it has but *what it is able to draw.*

Worked example 42.7: FT04's decoder, 2,164,736 parameters

A `SegFormerDecoder` with widths $[32, 64, 160, 256]$, `decoder_dim` 256 and 32 output channels (the spectral latent), so the last convolution makes $32 \times 16 = 512$ sub-pixel channels.

$$\begin{aligned}
\text{projections} &: (32+64+160+256)\times 256 + 4\times 256 = 512\times 256 + 1{,}024 \\
&= 131{,}072 + 1{,}024 = 132{,}096 \\
\text{fuse} &: 1{,}024\times 256 + 256 = 262{,}144 + 256 = 262{,}400 \\
\text{refine } 3\times 3 &: 256\times 256\times 9 + 256 = 589{,}824 + 256 = 590{,}080 \\
\text{sub-pixel } 3\times 3 &: 256\times 512\times 9 + 512 = 1{,}179{,}648 + 512 = 1{,}180{,}160 \\
\text{total} &: 132{,}096 + 262{,}400 = 394{,}496; \quad 394{,}496 + 590{,}080 = 984{,}576 \\
&\quad 984{,}576 + 1{,}180{,}160 = 2{,}164{,}736
\end{aligned}$$

Over half the decoder (1,180,160) is the one layer that emits a whole 4×4 block at once.

42.5.2 Test 1: the spectral floor

`WavelengthLinearDecoder` (Chapter 24) makes band n of a pixel as

$$\hat{x}_n = \mathbf{U}(\lambda_n) \cdot \mathbf{z} + c(\lambda_n), \tag{42.3}$$

with $\mathbf{z}$ the 32 numbers the decoder predicts for the pixel and $\mathbf{U}(\lambda_n)$ a 32-vector. Stack the rows $\mathbf{U}(\lambda_n)$ into a matrix $\mathbf{U}$ of shape $N \times 32$: every output spectrum is the offset curve $\mathbf{c}$ plus $\mathbf{Uz}$, a mixture of the 32 fixed columns of $\mathbf{U}$.

Definition 42.2 (Span and spectral floor)**.** The *span* of a set of vectors is the set of all their weighted sums. The *spectral floor* of a pixel is the part of its (offset-removed) spectrum that lies outside the span of the columns of $\mathbf{U}$: error that no choice of $\mathbf{z}$, and so no change upstream of `spectral_out`, can remove.

The closest point of the span to a spectrum $\mathbf{x}$ is its *orthogonal projection* (Chapter 6). For one curve $\mathbf{u}$ the projection is $a^\star\mathbf{u}$ with $a^\star = (\mathbf{x}\cdot\mathbf{u})/(\mathbf{u}\cdot\mathbf{u})$.

Worked example 42.8: The floor with 3 bands and 1 curve

Do this on paper. The model has 3 bands, one curve $\mathbf{u} = (1, 1, 1)$ and no offset: every

reconstruction is a flat spectrum $a(1,1,1)$. The real pixel is $\mathbf{x} = (2,1,3)$.

$$\begin{aligned}
\mathbf{x}\cdot\mathbf{u} &= 2\times 1 + 1\times 1 + 3\times 1 = 2+1+3 = 6\\
\mathbf{u}\cdot\mathbf{u} &= 1+1+1 = 3\\
a^\star &= 6/3 = 2; \quad \text{projection } = (2,2,2)\\
\text{floor} &= \mathbf{x} - \text{projection} = (2-2,\ 1-2,\ 3-2) = (0,-1,1)\\
\text{floor energy} &= 0^2 + (-1)^2 + 1^2 = 0+1+1 = 2
\end{aligned}$$

Now the model predicts $a = 1.5$:

$$\begin{aligned}
\text{residual} &= (1.5-2,\ 1.5-1,\ 1.5-3) = (-0.5,\ 0.5,\ -1.5)\\
\text{energy} &= 0.25 + 0.25 + 2.25 = 2.75\\
\text{floor share} &= 2/2.75 = 0.73
\end{aligned}$$

73% of this residual's energy is floor; the other 27% is the model choosing 1.5 instead of 2.

Why the share can never exceed 1. Split the residual $\mathbf{e} = \hat{\mathbf{x}} - \mathbf{x}$ into its part in the span and its part at right angles to the span. Every reconstruction lies in the span, so the out-of-span part of $\mathbf{e}$ is exactly minus the out-of-span part of $\mathbf{x}$, the floor $\mathbf{f}$. Write $\mathbf{e} = \mathbf{s} - \mathbf{f}$ with $\mathbf{s}$ in the span and $\mathbf{f}$ orthogonal to it. Then

$$\|\mathbf{e}\|^2 = (\mathbf{s}-\mathbf{f})\cdot(\mathbf{s}-\mathbf{f})$$

expand the product:

$$= \mathbf{s}\cdot\mathbf{s} - 2\,\mathbf{s}\cdot\mathbf{f} + \mathbf{f}\cdot\mathbf{f}$$

$\mathbf{s}\cdot\mathbf{f} = 0$ because they are at right angles:

$$= \|\mathbf{s}\|^2 + \|\mathbf{f}\|^2 \ \geq\ \|\mathbf{f}\|^2 .$$

This is Pythagoras' theorem. Check it on worked example 42.8:

$$\mathbf{s} = (1.5,1.5,1.5) - (2,2,2) = (-0.5,-0.5,-0.5), \qquad \|\mathbf{s}\|^2 = 0.75, \qquad 0.75 + 2 = 2.75.\checkmark$$

The script `spectral_floor.py` does this with 32 curves, using a QR decomposition: `Q, _=np.linalg.qr(U)` gives an orthonormal basis $\mathbf{Q}$ of the same span, so the projection is $\mathbf{Q}(\mathbf{Q}^\top\mathbf{x})$. It runs over every target pixel and 200,000 random background pixels per scene.

scene	background: actual / floor	floor share	targets: actual / floor	floor share
set1	0.048 / 0.013	0.07	0.189 / 0.022	0.02
set2	0.122 / 0.017	0.02	0.204 / 0.025	0.02
set3	0.121 / 0.033	0.09	0.149 / 0.043	0.10
set4	0.115 / 0.015	0.03	0.177 / 0.025	0.03

Table 42.2: The spectral floor of FT04 on the PRISMA bench, normalised units. "Actual / floor" are mean absolute errors; the share is of squared error (energy). Source: [M: `decoder-capacity/README.md`].

The floor is 2–10% of the residual energy. Over 90% of FT04's error is in which $\mathbf{z}$ the decoder predicts. Test 1 clears `spectral_out`. (Mind the units: set1 background's floor is $0.013/0.048 = 0.27$ of the error in mean-absolute terms but 0.07 in energy terms; squaring shrinks small parts relative to large ones.)

42.5.3 Test 2: visible against hidden pixels

FT04's background error could come from two causes.

- **Unpredictability.** A hidden pixel's value is not fully fixed by its neighbours. No decoder can fix this, and it is also what makes targets detectable.
- **Capacity.** The network cannot *render* the detail even when it has it. A better architecture can fix this.

Run the model with *no mask*, every token visible: unpredictability is gone, so what error is left is capacity. Compare it with the error on hidden pixels from the product's two complementary passes. The script `visible_vs_hidden.py` cuts 24 random 128 px tiles per PRISMA scene, keeping only tiles at least 95% valid.

scene	background visible	background hidden	targets visible	targets hidden
set1	0.056	0.057	0.194	0.218
set2	0.110	0.137	0.161	0.220
set3	0.159	0.129	0.176	0.179
set4	0.135	0.139	0.169	0.191

Table 42.3: FT04's mean absolute error per band (normalised), with no mask (visible) and in the two-pass inference (hidden). Source: [M: `decoder-capacity/README.md`].

On set1 FT04 copies the background it can see (0.056) no better than the background it must predict (0.057). On set2 and set4 visible is only modestly better; on set3 it is *worse*, which neither the README nor this book can explain. A model limited by unpredictability would autoencode visible pixels almost exactly. **This one is limited by what its architecture can render.**

42.5.4 Why: the 4×4-token bottleneck, counted

Follow one stage-1 token. The stage-1 patch embedding is a convolution with kernel 4 and stride 4 (Chapter 13), so a token reads a 4×4 block of pixels with 32 channels each:

$$\begin{aligned} \text{numbers into the token} &= 4 \times 4 \times 32 = 512 \\ \text{numbers out (stage-1 width)} &= 32 \\ \text{compression} &= 512/32 = 16 \text{ to } 1; \quad \text{per pixel } 32/16 = 2 \text{ numbers.} \end{aligned}$$

A counting argument for a whole tile. The decoder must produce $32 \times 128 \times 128 = 524{,}288$ numbers for a 128 px tile (32 latent numbers per pixel). Everything it knows about the tile arrives through the four encoder stages:

$$\begin{aligned} \text{stage } 1 &: 32 \times 32 \text{ tokens} \times 32 = 32{,}768 \\ \text{stage } 2 &: 16 \times 16 \times 64 = 16{,}384 \\ \text{stage } 3 &: 8 \times 8 \times 160 = 10{,}240 \\ \text{stage } 4 &: 4 \times 4 \times 256 = 4{,}096 \\ \text{total} &: 32{,}768 + 16{,}384 + 10{,}240 + 4{,}096 = 63{,}488 \\ \text{per pixel} &: 63{,}488/16{,}384 = 3.875 \text{ numbers, against 32 to produce.} \end{aligned}$$

Only stage 1 is at the resolution of a 4×4 block, and it carries 2 numbers per pixel; the other stages are coarser still. This is a count, not a proof: the network is nonlinear, and natural images are compressible, so 3.875 numbers per pixel can carry a lot. But the detail that does not fit through has no other route.

The linear case: rank. If the path were linear, the argument would be exact. A linear map from 512 numbers to 32 and back to 512 is a product **DE** with **E** of shape 32×512 and **D** of shape 512×32; its *rank* (the dimension of the set of outputs it can produce) is at most 32. At least $512 - 32 = 480$ directions of variation inside a block are lost entirely.

Worked example 42.9: Two pixels through one number

Squeeze two one-channel pixels (a, b) through one number and back: encode $z = (a + b)/2$, decode (z, z).

$$(a,b) = (3,1): \; z = (3+1)/2 = 2, \text{ output } (2,2), \text{ error } (2-3,\; 2-1) = (-1,\; 1)$$
$$(a,b) = (1,3): \; z = (1+3)/2 = 2, \text{ output } (2,2), \text{ error } (1,\; -1)$$
$$(a,b) = (2,2): \; z = 2, \text{ output } (2,2), \text{ error } (0,0)$$

Three different inputs, one output. The difference $a - b$ never reaches the decoder, so no decoder, however large, can restore it, even though the input was *visible*. That is test 2's finding in miniature: visible pixels are copied no better than hidden ones.

FT04's decoder then fuses the four stages at 1/4 resolution and emits each 4×4 block *at once* from one 256-wide vector: a 3×3 convolution to 512 channels and PixelShuffle(4) (Chapter 19). Every pixel gets its own output channel and its own weights, but all 16 are computed from the same token-level input. **Nothing at full resolution ever comes back in.**

The decoder's limit is not its parameter count but its paths: sub-token detail that did not fit through 32 numbers per token is gone, for visible pixels as much as hidden ones.

42.6 FT09: the progressive decoder

The diagnosis asked for two things: upsampling in *stages*, so detail is rendered gradually, and a *full-resolution skip*: a second input that bypasses the encoder and arrives at the decoder at full size, in the style of U-Net [37]. This is not the residual connection inside a transformer block (Chapter 16); it runs from the start of the network to the end.

42.6.1 Shapes, stage by stage

The skip is **z**, the stem output after BatchNorm: 32 numbers per pixel. `ProgressiveDecoder` in `app/foundation_models/components/progressive_decoder.py`, with FT09's widths (`decoder_dim` 256, `skip_channels` 32, `stage_dims` (192, 96)), for a 128 px tile and batch B:

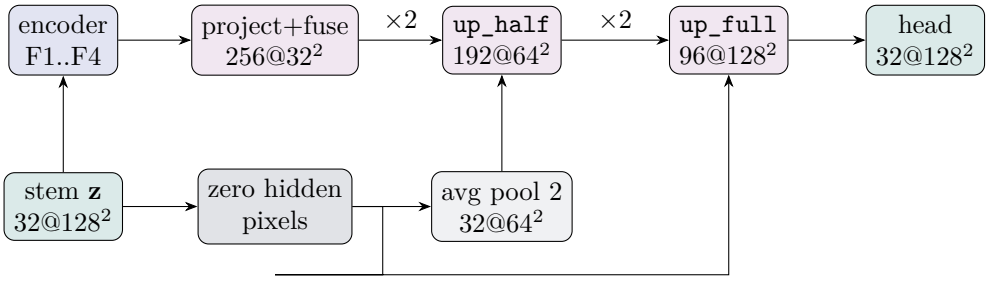


Figure 42.2: The progressive decoder. The stem output reaches the decoder twice: through the encoder (top) and through the masked skip (bottom), at half and full resolution. Channel counts are written as "channels @ size".

scale	operation	shape
in	encoder stages F1, F2	`(B,32,32,32)`, `(B,64,16,16)`
in	encoder stages F3, F4	`(B,160,8,8)`, `(B,256,4,4)`
in	skip **z**, hidden pixels zeroed	`(B,32,128,128)`
1/4	1×1 projections to 256, bilinear up, concat	`(B,1024,32,32)`
1/4	1×1 fuse + GELU (warm-started)	`(B,256,32,32)`
1/2	bilinear $\times 2$	`(B,256,64,64)`
1/2	concat `avg_pool2d(skip, 2)`	`(B,288,64,64)`
1/2	`up_half`: 3×3 conv, GELU, 3×3 conv, GELU	`(B,192,64,64)`
1	bilinear $\times 2$	`(B,192,128,128)`
1	concat skip	`(B,224,128,128)`
1	`up_full`: 3×3 conv, GELU, 3×3 conv, GELU	`(B,96,128,128)`
1	`head`: 1×1 conv	`(B,32,128,128)`
out	`spectral_out`, unchanged	`(B,N,128,128)`

Table 42.4: `ProgressiveDecoder.forward` for a 128 px tile with FT09's widths.

42.6.2 The parameter count by hand

Worked example 42.10: The progressive decoder's 1,504,096 parameters

Every convolution has $c_{\text{in}} \times c_{\text{out}} \times k \times k + c_{\text{out}}$ parameters. Do it on paper.

$$\begin{aligned}
\text{projections} &: (32+64+160+256) \times 256 + 4 \times 256 = 131{,}072 + 1{,}024 = 132{,}096 \\
\text{fuse} &: 1{,}024 \times 256 + 256 = 262{,}144 + 256 = 262{,}400 \\
\texttt{up_half}\ 1 &: 288 \times 192 \times 9 + 192 = 497{,}664 + 192 = 497{,}856 \\
\texttt{up_half}\ 2 &: 192 \times 192 \times 9 + 192 = 331{,}776 + 192 = 331{,}968 \\
\texttt{up_full}\ 1 &: 224 \times 96 \times 9 + 96 = 193{,}536 + 96 = 193{,}632 \\
\texttt{up_full}\ 2 &: 96 \times 96 \times 9 + 96 = 82{,}944 + 96 = 83{,}040 \\
\texttt{head} &: 96 \times 32 + 32 = 3{,}072 + 32 = 3{,}104
\end{aligned}$$

$$\begin{aligned}
\text{running total} &: 132{,}096 + 262{,}400 = 394{,}496 \\
&\quad 394{,}496 + 497{,}856 = 892{,}352 \\
&\quad 892{,}352 + 331{,}968 = 1{,}224{,}320 \\
&\quad 1{,}224{,}320 + 193{,}632 = 1{,}417{,}952 \\
&\quad 1{,}417{,}952 + 83{,}040 = 1{,}500{,}992 \\
&\quad 1{,}500{,}992 + 3{,}104 = 1{,}504{,}096
\end{aligned}$$

Source check: 1,504,096 verified

Building `ProgressiveDecoder([32,64,160,256])` on CPU gives 1,504,096 [C](`research/textbook/tools/compute_39-41.py`); the source comment reads "1.50 M with FT04 widths (FT04's decoder: 2.16 M)". **The new decoder has fewer parameters than the old one**: it is wired differently, not bigger.

It does far more arithmetic, though. `up_full`'s first convolution runs its 193,536 multiply-adds at each of $128 \times 128 = 16{,}384$ pixels: $193{,}536 \times 16{,}384 \approx 3.17 \times 10^9$ per tile. FT04's two refine convolutions together at 32×32: $(589{,}824 + 1{,}179{,}648) \times 1{,}024 = 1{,}769{,}472 \times 1{,}024 \approx 1.81 \times 10^9$. These are hand counts, not timings [C].

42.6.3 The skip must not carry hidden pixels

This is the trap in the design. The masked autoencoder works because a hidden pixel's value never reaches its own reconstruction (Chapter 9). A skip that bypasses the encoder would undo that: hand the decoder the real pixel and it will copy it, the residual vanishes, and every target with it. So `_decode` in `spectral_stem_seg_former_mae.py` zeroes them:

```
visible = keep_mask.reshape(B, 1, H // 4, W // 4).to(z.dtype)    # (B, 1, 32, 32): 1
    = visible token
z = z * F.interpolate(visible, scale_factor=4, mode="nearest")   # (B, 32, 128, 128):
     token's 16 pixels
return self.decoder(features, z)
```

Nearest-neighbour interpolation by 4 copies each token's 0 or 1 to its 16 pixels. The half-resolution skip is `avg_pool2d(skip,2)`: since 2 divides 4 and the 2×2 windows line up with token borders, each average stays inside one token, so a hidden token is 0 at both resolutions. Neighbouring *visible* pixels still reach a hidden pixel through the 3×3 convolutions; the docstring calls that context, as in the encoder.

The test `test_hidden_pixels_never_reach_the_output` builds a small model and a 64 px tile ($16 \times 16 = 256$ tokens), hides tokens 0, 17, 100 and 255, makes a second input in which *only those tokens' pixels* are replaced with fresh random values, and asserts the two reconstructions agree everywhere to 10^{-6}. Token t is at row $\lfloor t/16 \rfloor$, column t mod 16: token 17 is $(1, 1)$, pixels rows 4–7, columns 4–7; token 100 is $(6, 4)$, rows 24–27, columns 16–19. Its partner test shows the skip is live: adding 0.5 to one visible pixel moves that pixel's own reconstruction by more than 10^{-4}.

42.6.4 Warm start and two-phase training

FT09 starts from FT04's `best.pt`. The new decoder kept the old layer names `linear_projections` and `fuse_conv`, so those load; the old `refine.*` is dropped. How much of the decoder is warm? 132,096 + 262,400 = 394,496 of 1,504,096, that is 394,496/1,504,096 = 0.26. **Three quarters of the new decoder starts random.**

The first attempt trained everything jointly from the start (lr 10^{-4}, planned 8 epochs) and was stopped after epoch 3, still 0.5–1° SAM behind FT04: a mostly random decoder had $3 \times 30{,}000 = 90{,}000$ samples to learn from, against the millions behind FT04's, on a learning rate already decaying, and its early gradients pushed a good encoder around to suit a bad decoder. The fix split training in two [M: `FT09-decoder/README.md`]:

run	mode	what trains	lr	epochs	starts from
FT09a	`"decoder"`	`decoder.*` only	10^{-3}	3	FT04
FT09b	`"joint"`	everything	5×10^{-5}	5	FT09a
FT09c	`"joint"`	everything	3×10^{-5}	5	FT09b

With `spectral_out` frozen in FT09a, the decoder is in effect learning to emit FT04's own code `z`. FT09a reached FT04's reconstruction, within 0.1° on all three sensors (3.98 / 4.14 / 5.50° against 3.88 / 4.11 / 5.46°), in 3 epochs.

	set1	set2	set3	set4
hidden background FT04 → FT09a	0.057 → **0.051**	0.137 → **0.125**	0.129 → **0.117**	0.139 → **0.110**
hidden targets FT04 → FT09a	0.218 → 0.204	0.220 → 0.203	0.179 → 0.171	0.191 → 0.162
target / background	3.82 → **4.00**	1.61 → **1.62**	1.39 → **1.46**	1.37 → **1.47**
visible background	0.056 → 0.176	0.110 → 0.120	0.159 → 0.120	0.135 → 0.115

Table 42.5: The mechanism check on FT09a, mean absolute error per band, normalised. Source: [M: `FT09-decoder/README.md`].

42.6.5 Did it do what it was built for? Contrast

Worked example 42.11: Set1's contrast change

$$\begin{aligned} \text{background fall} &: 0.057 - 0.051 = 0.006; \quad 0.006/0.057 = 0.105\ (10.5\%) \\ \text{target fall} &: 0.218 - 0.204 = 0.014; \quad 0.014/0.218 = 0.064\ (6.4\%) \\ \text{new ratio} &: 0.204/0.051 = 4.00 \end{aligned}$$

Background fell by a larger fraction than targets, so the ratio rose.

Hidden background error fell more than target error on all four scenes, so contrast rose on all four: the intended effect, and the first model change in the series to move it.

Source check: "6–21%" or "9–21%"

Course part 67 records that the FT09 README said hidden background error "falls 6-21%", while its own table gives set1 10.5%, set2 0.012/0.137 = 8.8%, set3 0.012/0.129 = 9.3%, set4 0.029/0.139 = 20.9%: 9–21%. The README now reads "falls 9-21%" (`research/experiments/indradhanu-finetune/FT09-decoder/README.md`); the course note predates the correction.

Criterion 5, stated honestly. Before launch the README fixed a mechanism criterion: visible-pixel background error well below FT04's on all four scenes. The last row of Table 42.5 is that test, and it **failed**: set1 got much worse (0.056 to 0.176). Afterwards the README recorded a caveat: the progressive decoder only ever trained with about 65% of its skip zeroed, so a fully unmasked input is outside anything it has seen. That argument is reasonable, but the yardstick was judged wrong *after* the result came in, which is exactly what fixing criteria in advance is meant to prevent. Treat the mechanism as supported, not proven.

42.6.6 FT09b: detection, and the bootstrap

	pAUC	top 1% / 0.1%	TPR @ 0.1%	ROC-AUC	v2.1 top 1%
FT04	0.170	46.0% / **15.8%**	**0.065**	0.852	**36.2%**
FT09a	0.169	46.1% / 15.6%	0.061	0.856	35.8%
FT09b	**0.176**	**47.1%** / 15.4%	0.063	**0.854**	36.1%
FT09c	0.172	46.4% / 15.0%	–	–	36.3%

Table 42.6: FT09 detection, real PRISMA bench, cloud excluded on FT04's masks. Source: [M: `FT09-decoder/README.md`].

The bench pAUC is the mean over the four scenes. Check it from the unrounded per-scene

values in `eval_per_scene.csv`:

$$\begin{aligned} \text{FT04} &: 0.24603 + 0.12817 + 0.08428 + 0.21986 = 0.67834; \quad 0.67834/4 = 0.16959 \\ \text{FT09b} &: 0.26010 + 0.13061 + 0.08301 + 0.22873 = 0.70245; \quad 0.70245/4 = 0.17561 \\ \text{change} &: 0.17561 - 0.16959 = 0.00602 \end{aligned}$$

which is the bootstrap's point value +0.0060.

The paired object bootstrap of Chapter 36 resamples the bench's 909 objects with replacement 1000 times and recomputes the difference each time; the 95% interval runs from the 2.5th to the 97.5th percentile of those differences. For FT09b − FT04 [M: `FT09b-decoder-joint/bootstrap.txt`]:

$$\text{pAUC: } +0.0060\ [+0.0002,\ +0.0118], \quad P(B > A) = 0.976.$$

The interval clears zero, by 0.0002. Caught at the top 1%: +1.1 points $[-0.3, +2.4]$, a tie. Caught at the top 0.1%: −0.4 points $[-1.4, +0.6]$, a tie in the wrong direction: FT04 caught $57 + 33 + 13 + 41 = 144$ objects and FT09b $56 + 34 + 9 + 41 = 140$, and $4/909 = 0.44\%$; the whole loss is set3, 13 to 9.

Worked example 42.12: Reading an interval as a margin

The interval is centred near the point value: $0.0060 - 0.0002 = 0.0058$ below and $0.0118 - 0.0060 = 0.0058$ above. Its half-width 0.0058 is the bench's resolution for this pair. A gain of +0.0060 is only $0.0060/0.0058 = 1.03$ half-widths from zero: the smallest margin the rule ("interval above zero") allows. Compare cloud exclusion, +0.0074 $[+0.0047, +0.0101]$: half-width 0.0027, margin $0.0074/0.0027 = 2.7$.

So FT09b met criterion 1 by +0.0002, failed criterion 2 at the top 0.1% (unresolved), met 3 (−0.1 points) and 4, and failed 5 as written. A candidate, not a release. FT09a alone, with higher contrast already, *tied* FT04 (−0.0006 $[-0.0056, +0.0043]$): the joint phase, with the encoder adapting to the new decoder, did the work.

42.6.7 FT09c: the gain does not hold

FT09b's best checkpoint was its last epoch, so FT09c continued it for five epochs at 3×10^{-5}. Its reconstruction got better (AVIRIS-NG SAM 5.41 to 5.34°, better than FT04) and its detection worse: pAUC 0.172, a tie with FT04 (+0.0020 $[-0.0026, +0.0064]$), and a *resolved* loss against FT09b, −0.0040 $[-0.0076, -0.0010]$ [M: `FT09-decoder/README.md`].

A gain must hold across neighbouring checkpoints: the object bootstrap resamples objects, not training runs, so it cannot see checkpoint-to-checkpoint variation, which here was as large as the gain itself.

The progressive decoder was not adopted. Its mechanism (higher hidden-pixel contrast) was real; its effect on detection was within checkpoint noise. And again, as with FT06: better reconstruction is not better detection.

Values that will drift

FT09 runs 2026-10-22, FT09c evaluated 2026-10-23 (it finished unevaluated and the GPU sat idle about 10 hours because no watcher was set). Per-scene pAUC FT09c: 0.254 / 0.131 / 0.082 / 0.219 [M: `FT09-decoder/README.md`].

change tried on FT04	chapter	pAUC change [95% interval]	verdict
cloud exclusion	36	$+0.0074$ $[+0.0047, +0.0101]$	real gain, adopted
FT06 residual/$\sqrt{b}$	41	-0.0067 $[-0.0174, +0.0037]$	tie
gated scene adaptation, $N = 50$	40	-0.0091 $[-0.0176, -0.0028]$	real loss
FT07 gated loss	this	-0.0047 $[-0.0082, -0.0020]$	real loss, small
FT08 gated loss + blocks	this	-0.0057 $[-0.0101, -0.0013]$	real loss, small
FT09a decoder only	this	-0.0006 $[-0.0056, +0.0043]$	tie
FT09b decoder, joint	this	$+0.0060$ $[+0.0002, +0.0118]$	gain by a hair
FT09c five more epochs	this	$+0.0020$ $[-0.0026, +0.0064]$	tie: FT09b did not hold

Table 42.7: Every change tried on FT04 with a paired bootstrap interval. Sources: [M: `bench-resolution/README.md`], [M: `FT07-gated/README.md`], [M: `FT09-decoder/README.md`]. FT06 residual/b (0.170 to 0.125) was not bootstrapped. FT08's row is the 2026-10-31 re-run without the evaluation leak (Section 42.4.1); the leaky run read -0.0058 $[-0.0106, -0.0010]$.

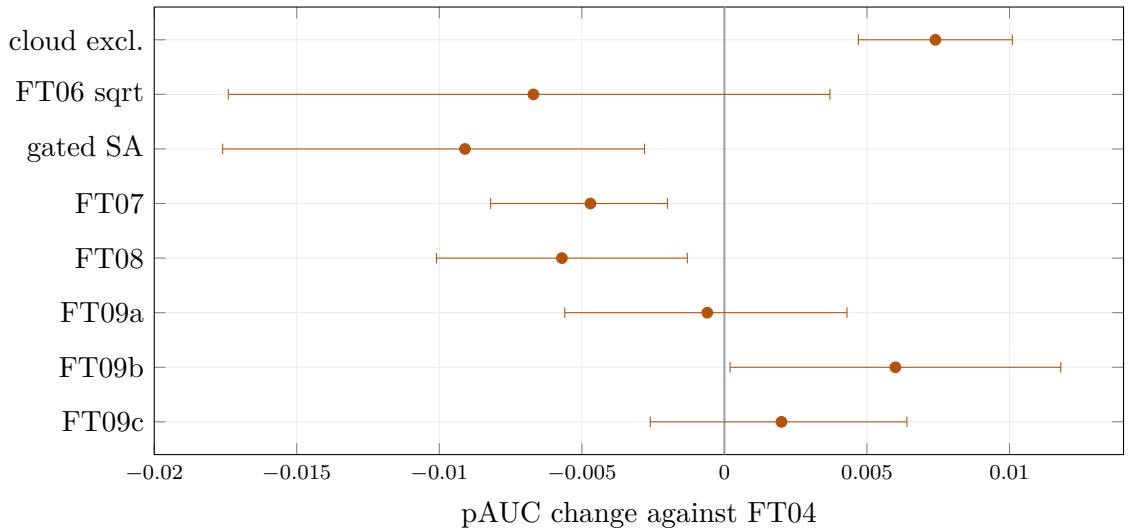


Figure 42.3: Table 42.7 drawn to scale: point values with their 95% paired-bootstrap intervals. Only cloud exclusion lies wholly right of zero; FT09b touches it, and FT09c falls back.

42.7 The cumulative lesson since FT04

Read down the last column. Since FT04, every change except cloud exclusion has been inside the bench's noise or a small, consistent cost, and the one exception from the network's shape (FT09b) did not survive further training. None of these changes altered how much the *encoder* can hold: its widths stayed 32, 64, 160, 256. That is the question Chapter 43 takes up.

42.8 Common confusions

Top 1% gated means 1% of the loss is gone.
The halo multiplies it: one isolated pixel with `gate_halo = 2` becomes a 5×5 square of 25. And the share of loss *value* removed exceeds the share of pixels, because the gated pixels are the large ones (worked example 42.2).

FT08 hides more of the tile than FT04.
Almost the same share, 664 against 665 tokens on a 128 px tile, in bigger pieces. What changes is the visible context: 1.765 against 2.807 expected visible neighbours.

The FT08 leak inflated its score, so FT08 was really much worse.
The clean re-run reads 0.1638 against the leaky 0.164, well inside what the bench resolves. The leak was real; its effect on this model was tiny, and the verdict is the same.

FT08's validation score beat FT04's, so FT08 reconstructs better.
Validation uses single-token masks, not FT08's blocks, and SAM swings by about the guard from epoch to epoch.

The decoder has 2.16 M parameters, so it cannot be under-powered.
Parameters are not the issue; paths are. Its parameters all work at 1/4 resolution on token-level vectors (worked example 42.9).

A floor share of 0.07 means the floor is 7% of the mean absolute error.
The share is of squared error (energy). In mean-absolute terms set1 background's floor is 0.27 of the error.

The progressive decoder is bigger, so the gain came from parameters.
It is smaller, 1.50 M against 2.16 M; it does more arithmetic because it works at full resolution.

The skip leaks hidden pixels.
Hidden tokens are zeroed at full and half resolution, and a test checks it to 10^{-6}. Visible neighbours do reach a hidden pixel through 3×3 convolutions; that is context.

FT09b cleared the bootstrap, so it is a real improvement.
It cleared the *object* bootstrap at one checkpoint, by 0.0002. Five more epochs gave a resolved loss against it. The bootstrap cannot see checkpoint noise.

42.9 Check yourself

Exercise 42.1. A group has 200 loss pixels and `gate_pct = 1`. How many are gated before the halo? With `gate_halo = 1`, how many loss pixels at most are removed, if the gated pixels are far apart and away from edges? What share of L1 gradient terms is that?

Exercise 42.2. A 256 px tile with `mask_block = 2` and ratio 0.65: how many blocks, how many hidden, how many tokens hidden? Then compute the expected number of visible neighbours of a hidden token, for blocks and for single tokens (where $4{,}096 \times 0.65$ is truncated).

Exercise 42.3. With one curve $\mathbf{u} = (1, 1, 1)$ and pixel $\mathbf{x} = (4, 1, 1)$: find the projection, the floor and its energy. If the model predicts $(3, 3, 3)$, what is the floor share? Verify Pythagoras.

Exercise 42.4. Count the parameters of `up_half`'s first convolution with `stage_dims = (160, 96)`, and the whole progressive decoder with that change (the second `up_half` convolution and `up_full`'s first also change).

Exercise 42.5. A colleague proposes stage-1 width 64 for FT04. What is the per-token compression, and how many numbers per pixel would the four encoder stages then deliver for a 128 px tile (other stages unchanged)?

Exercise 42.6. Set4's hidden background fell 0.139 to 0.110 and targets 0.191 to 0.162. Compute each fall as a percentage and the new ratio. Then explain in two sentences why FT09b is not shipped.

Exercise 42.7. Redo worked example 42.6 on a 6×6-token tile. How many blocks, how many tokens leak per block, and how many in all? Then explain why the answer does not depend on which pass hides which blocks.

Next: Chapter 43, Indradhanu Heavy: capacity, a fair control and the haze.

Chapter 43

Indradhanu Heavy: capacity, a fair control and the haze

The one thing this chapter teaches

Tripling the encoder's width multiplies its parameters by about nine, because almost every weight sits in a $C \times C$ matrix; the resulting 33.9 M-parameter Indradhanu Heavy, re-pretrained and fine-tuned beside a control given exactly the same recipe, beat the control and the shipped model on detection at every neighbouring checkpoint, once a thin-cloud mask that fired on haze was left out, while it reconstructed no better than the control.

This chapter closes Part VIII with the experiment that finally moved detection. It has four strands: the arithmetic of width (why parameters grow like width squared, counted to the last parameter); the pretraining run on a rented H100 and the two operational traps it hit; the logic of a fair comparison (a control, and three neighbouring checkpoints); and a mask head that called haze "cloud" and so deleted most of a scene's targets. Its lesson is the one Chapters 41 and 42 pointed towards: judge detectors on detection.

Provenance tags abbreviate paths:

- `F002-heavy/` is under `research/experiments/indradhanu-se-pretrain/`;
- `FTH1-heavy-joint/` and `FTH2-heavy-joint-more/`,
- `FTH3-heavy-joint-more2/` and `FTB1-base-joint/`: all four are under `research/experiments/indradhanu-finetune/`.

Analogy

You want to know whether a new engine makes a car faster. Last year's car won with a driver who practised the track for a month. Put the new engine in a car, hand it to a driver who has had one afternoon, and if it loses, you have learned nothing about the engine. The fair test is two cars, old engine and new, with the same driver, the same afternoon of practice, the same tyres. Here the engine is the pretrained model (Heavy against F001), the month of practice is FT04's four-stage curriculum, and the afternoon is a single joint fine-tuning run that both engines get.

43.1 Why capacity was the untested question

Look back at Table 42.7. Cloud exclusion was a real gain; FT07 and FT08 cost a little; FT09b gained by a hair and FT09c showed the gain was a lucky checkpoint. Every one of those runs *kept*

FT04's encoder at the same size: widths 32, 64, 160 and 256 across its four stages. Chapter 42 found the limit they could not move: each stage-1 token squeezes 512 numbers into 32. FT09 attacked the decoder side of that; it did not change how much the encoder can hold. As the F002 README puts it: "A wider encoder needs re-pretraining" [M: `F002-heavy/README.md`]. You cannot widen a trained encoder and keep its weights, because every weight matrix changes shape.

Definition 43.1 (Capacity and width)**.** The *capacity* of a model is how much it can represent. For a network of fixed depth it grows with its *width*: the number of channels each layer carries.

43.2 The architecture

setting	F001 / FT04	Heavy	ratio
`embed_dims` (stage widths)	32, 64, 160, 256	96, 192, 480, 768	3×
`num_heads`	2, 2, 5, 8	3, 6, 10, 12	
channels per head	16, 32, 32, 32	32, 32, 48, 64	
spectral latent (`compressed_channels`)	32	64	2×
`token_dim`	32	32	1×
decoder	SegFormer, dim 256	progressive, dim 512	
decoder stage dims	–	256 / 128	
reduction ratios; blocks	8 4 2 1; 2 2 2 2	same	
mask ratio	0.65	0.65	

Table 43.1: F001 (FT04's pretrained parent, Chapter 27) against Heavy, both built by `SpectralStemSegFormerMAE`. Source: [M: `F002-heavy/config.json`].

Source check: "4x wide" is a name, the widths are 3x

The run was launched on the user's "Go, but 4x wide". The course (part 68) records that the README title, the config's `_description` and the commit all said "4x wide". The configuration has encoder widths $96/32 = 3$ and $768/256 = 3$ times FT04's, and the spectral latent and decoder 2 times; nothing in the model is 4 times. The source now agrees: the README title reads "3x-wide pretraining", with "implemented as 3x encoder widths … which this README first called 4x (corrected 2026-10-29)", and the `_description` reads "3x-wide (encoder)" (`research/experiments/indradhanu-se-pretrain/F002-heavy/README.md`, `config.json`). Read "4x" as the launch instruction, not a measurement.

The decoder is FT09's progressive decoder (Chapter 42). It did not help FT04 on its own; here it is pretrained from the start, so it never has the warm-start problem. `token_dim` stays 32 because "the reduced stem's wavelength encoding is 32 wide".

43.2.1 The stage-1 squeeze, three times wider

$$\text{FT04}: 16 \text{ pixels} \times 32 \text{ channels} = 512 \text{ in};\ 32 \text{ out};\ 512/32 = 16 \text{ to } 1;\ 32/16 = 2 \text{ per pixel}$$
$$\text{Heavy}: 16 \times 64 = 1{,}024 \text{ in};\ 96 \text{ out};\ 1{,}024/96 = 10.7 \text{ to } 1;\ 96/16 = 6 \text{ per pixel}$$

43.3 Why parameters grow like width squared

43.3.1 One encoder block, as a formula

A transformer block (Chapter 16) is LayerNorm, efficient self-attention with spatial reduction ratio R (Chapter 15), LayerNorm, Mix-FFN with expansion 4. Count its parameters for width

C, layer by layer. A linear layer from a to b channels has $ab + b$ parameters; a LayerNorm over C channels has $2C$ (a weight and a bias each).

$$\begin{aligned}
\text{norm1} &: 2C \\
q, k, v, \text{proj} &: 4\,(C \cdot C + C) = 4C^2 + 4C \\
\text{reduction conv } (R \times R, \text{ if } R > 1) &: C \cdot C \cdot R^2 + C = R^2C^2 + C \\
\text{reduction norm (if } R > 1) &: 2C \\
\text{norm2} &: 2C \\
\text{fc1 } (C \to 4C) &: C \cdot 4C + 4C = 4C^2 + 4C \\
\text{depthwise } 3 \times 3 \text{ conv on } 4C &: 4C \cdot 1 \cdot 9 + 4C = 40C \\
\text{fc2 } (4C \to C) &: 4C \cdot C + C = 4C^2 + C
\end{aligned}$$

Collect the C^2 terms: attention $4C^2$, reduction R^2C^2, FFN $4C^2 + 4C^2 = 8C^2$. Collect the C terms: $2 + 4 + 1 + 2 + 2 + 4 + 40 + 1 = 56$. So

$$P_{\text{block}}(C, R) = (12 + R^2)\,C^2 + 56\,C \quad (R > 1), \qquad P_{\text{block}}(C, 1) = 12\,C^2 + 53\,C, \tag{43.1}$$

where for $R = 1$ the reduction convolution and its norm are absent ($56 - 1 - 2 = 53$).

Worked example 43.1: Stage-1 block, FT04 and Heavy

Stage 1 has $R = 8$, so the C^2 coefficient is $12 + 64 = 76$. Do it on paper.

$$\begin{aligned}
\text{FT04, } C = 32 &: 76 \times 32^2 + 56 \times 32 = 76 \times 1{,}024 + 1{,}792 = 77{,}824 + 1{,}792 = 79{,}616 \\
\text{Heavy, } C = 96 &: 76 \times 96^2 + 56 \times 96 = 76 \times 9{,}216 + 5{,}376 = 700{,}416 + 5{,}376 = 705{,}792 \\
\text{ratio} &: 705{,}792/79{,}616 = 8.86
\end{aligned}$$

Line by line for Heavy: norm1 192; q,k,v,proj $4 \times (9{,}216 + 96) = 37{,}248$; reduction $96 \times 96 \times 64 + 96 = 589{,}920$; reduction norm 192; norm2 192; fc1 $96 \times 384 + 384 = 37{,}248$; dwconv $384 \times 9 + 384 = 3{,}840$; fc2 $384 \times 96 + 96 = 36{,}960$. Sum: $192 + 37{,}248 + 589{,}920 + 192 + 192 + 37{,}248 + 3{,}840 + 36{,}960 = 705{,}792$. The 8×8 reduction convolution alone is $589{,}920/705{,}792 = 84\%$ of the block.

Why the ratio is a little under 9. Tripling C multiplies the C^2 term by $3^2 = 9$ and the C term by only 3:

$$\frac{P(3C)}{P(C)} = \frac{9\,aC^2 + 3 \cdot 56\,C}{aC^2 + 56\,C} = 9 - \frac{6 \cdot 56\,C}{aC^2 + 56\,C}.$$

For stage 1 of FT04 ($a = 76$, $C = 32$):

$$\frac{6 \times 56 \times 32}{79{,}616} = \frac{10{,}752}{79{,}616} = 0.135, \qquad 9 - 0.135 = 8.865.$$

The wider the layer, the smaller the correction: width scaling is "about width squared" because the C^2 terms dominate.

stage ($C_{\text{FT04}} \to C_{\text{Heavy}}$, R)	FT04 block	Heavy block	ratio	formula check (Heavy)
1 (32 → 96, 8)	79,616	705,792	8.86	$76 \cdot 9{,}216 + 5{,}376$
2 (64 → 192, 4)	118,272	1,042,944	8.82	$28 \cdot 36{,}864 + 10{,}752$
3 (160 → 480, 2)	418,560	3,713,280	8.87	$16 \cdot 230{,}400 + 26{,}880$
4 (256 → 768, 1)	800,000	7,118,592	8.90	$12 \cdot 589{,}824 + 40{,}704$

Table 43.2: One block per stage. Counts from building both models on CPU [C](`tools/compute_39-41_output.txt`); every one matches Equation (43.1).

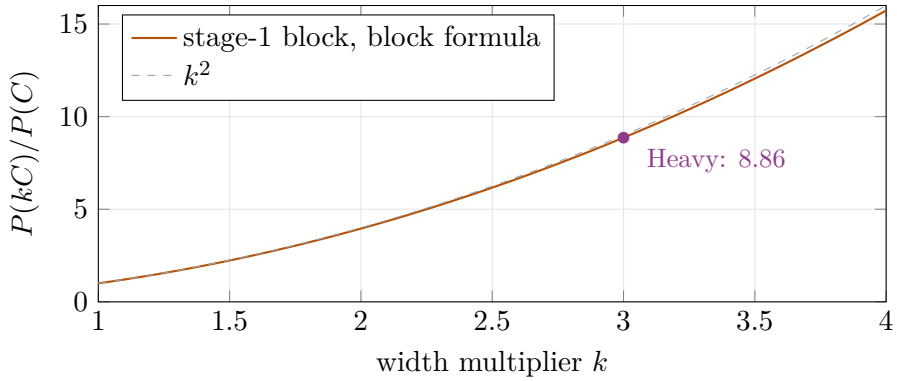


Figure 43.1: Parameter growth of a stage-1 block with width, drawn to scale. The curve hugs k^2; the gap is the linear (bias and norm) terms.

43.3.2 The whole encoder, counted

The patch embeddings (Chapter 13): stage 1 is a 4×4 convolution with stride 4 from the spectral latent; stages 2–4 are 3×3, stride 2, from the previous stage. Each is followed by a LayerNorm.

$$
\begin{aligned}
&\text{stage 1} : 64 \times 96 \times 16 + 96 + 2 \times 96 = 98{,}304 + 96 + 192 = 98{,}592 \\
&\text{stage 2} : 96 \times 192 \times 9 + 192 + 384 = 165{,}888 + 576 = 166{,}464 \\
&\text{stage 3} : 192 \times 480 \times 9 + 480 + 960 = 829{,}440 + 1{,}440 = 830{,}880 \\
&\text{stage 4} : 480 \times 768 \times 9 + 768 + 1{,}536 = 3{,}317{,}760 + 2{,}304 = 3{,}320{,}064 \\
&\quad \text{sum} : 98{,}592 + 166{,}464 = 265{,}056;\ +830{,}880 = 1{,}095{,}936;\ +3{,}320{,}064 = 4{,}416{,}000
\end{aligned}
$$

(FT04's stage-1 embedding: $32 \times 32 \times 16 + 32 + 64 = 16{,}480$.) Two blocks per stage:

$$
\begin{aligned}
2 \times 705{,}792 &= 1{,}411{,}584 \\
2 \times 1{,}042{,}944 &= 2{,}085{,}888 \\
2 \times 3{,}713{,}280 &= 7{,}426{,}560 \\
2 \times 7{,}118{,}592 &= 14{,}237{,}184 \\
\text{blocks} : 1{,}411{,}584 + 2{,}085{,}888 &= 3{,}497{,}472;\ +7{,}426{,}560 = 10{,}924{,}032 \\
10{,}924{,}032 + 14{,}237{,}184 &= 25{,}161{,}216
\end{aligned}
$$

and a LayerNorm after each stage, $2 \times (96 + 192 + 480 + 768) = 2 \times 1{,}536 = 3{,}072$. The encoder:

$$25{,}161{,}216 + 4{,}416{,}000 + 3{,}072 = 29{,}580{,}288.$$

43.3.3 The decoder and the whole model

Worked example 43.2: Heavy's progressive decoder, 4,279,616 parameters

Widths: stages $[96, 192, 480, 768]$, `decoder_dim` 512, skip 64 channels (the spectral latent), stage dims (256, 128), 64 output channels.

$$\begin{aligned}
\text{projections} &: (96+192+480+768)\times 512 + 4\times 512 = 786{,}432 + 2{,}048 = 788{,}480\\
\text{fuse} &: 2{,}048\times 512 + 512 = 1{,}048{,}576 + 512 = 1{,}049{,}088\\
\texttt{up_half}\ 1 &: (512+64)\times 256\times 9 + 256 = 1{,}327{,}104 + 256 = 1{,}327{,}360\\
\texttt{up_half}\ 2 &: 256\times 256\times 9 + 256 = 589{,}824 + 256 = 590{,}080\\
\texttt{up_full}\ 1 &: (256+64)\times 128\times 9 + 128 = 368{,}640 + 128 = 368{,}768\\
\texttt{up_full}\ 2 &: 128\times 128\times 9 + 128 = 147{,}456 + 128 = 147{,}584\\
\texttt{head} &: 128\times 64 + 64 = 8{,}192 + 64 = 8{,}256\\
\text{running total} &: 788{,}480 + 1{,}049{,}088 = 1{,}837{,}568;\ +1{,}327{,}360 = 3{,}164{,}928\\
&\quad + 590{,}080 = 3{,}755{,}008;\ +368{,}768 = 4{,}123{,}776\\
&\quad + 147{,}584 = 4{,}271{,}360;\ +8{,}256 = 4{,}279{,}616
\end{aligned}$$

part	FT04 size (no seg head)	Heavy
`spectral_in` (stem)	14,721	17,825
`norm` (BatchNorm, $2D$)	64	128
encoder	3,331,072	29,580,288
decoder	2,164,736	4,279,616
`spectral_out`	4,257	6,337
total	**5,514,850**	**33,884,194**

Table 43.3: Whole-model counts. The encoder and decoder rows are derived by hand above, Heavy's stem rows in Section 43.4.5; all match the built models [C].

Add Heavy's column: 17,825 + 128 = 17,953; +29,580,288 = 29,598,241; +4,279,616 = 33,877,857; +6,337 = 33,884,194. That is the README's "33.9 M parameters (encoder 29.6 M, decoder 4.3 M) vs FT04 5.9 M". The 5.9 M is the shipped FT04 *with* its segmentation head (5,886,451, Chapter 39); without the head, 33,884,194/5,514,850 = 6.1. **Heavy is about six times FT04, with nine times its encoder** (29,580,288/3,331,072 = 8.9).

Source check: every count verified

Building both models on CPU (`research/textbook/tools/compute_39-41.py`) gives Heavy 33,884,194 total, encoder 29,580,288, decoder 4,279,616, `encoder.patch_embeds[0]` 98,592, one stage-1 block 705,792; the FT04-sized model 5,514,850, stage-1 block 79,616 [C].

43.4 One forward pass of Heavy, at full size

The counts above are a budget. This section follows the money: one batch through the model the product serves, Indradhanu V2 `v2.1.0` (FTH3 epoch 5, Heavy fine-tuned with its segmentation head, Section 43.12), at real sizes, with every tensor's shape. It is where Chapter 19's trace of the small model, Chapter 44's trace of the stem, and the counts above meet. The sizes:

- $B = 2$ tiles of 128×128, the product's tile size (`default_patch_size` 128, `backend/`

`allotrope/foundation_models/resolver.py`);

- the native bands of PRISMA bench scene set1 after `trained_band_mask` (below);
- the training mask ratio $\rho = 0.65$ (the same file), so $\lfloor 1{,}024 \times 0.65 \rfloor = 665$ of the 1,024 stage-1 tokens are hidden and 359 kept (Chapter 18).

Every shape in this section was recorded by forward hooks on the shipped checkpoint, loaded strictly into the real class, by `research/textbook/tools/compute_heavy_trace.py`; its output is `compute_heavy_trace_output.txt` [C]. The script reads set1 exactly as the evaluator does (`load_prisma_scene(..., "native")`).

43.4.1 How many bands: 185, all kept

Set1's native cube after the product's band filter has 185 bands, 453.4 to 2,477.1 nm [C]. The model then drops any band farther than 15 nm from every wavelength it has trained on (`trained_band_mask`, Chapter 24). Which wavelengths count as "trained on" is the buffer `trained_wl`, which starts as the 202-wavelength SpectralEarth curve and grows during fine-tuning.

record used	wavelengths in the record	set1 bands kept
the pretraining curve alone (a pretrained model)	202	165 (20 dropped)
v2.1.0's `trained_wl`, after fine-tuning	733	185 (none dropped)

The 20 bands a pretrained record drops are the three past 2,445 nm (2,463.0 to 2,477.1), one at 1,958.6, seven from 1,736.5 to 1,794.0, five from 1,459.3 to 1,502.0 and four from 1,306.2 to 1,339.1, the edges of SpectralEarth's water-vapour gaps: $3 + 1 + 7 + 5 + 4 = 20$ [C]. Fine-tuning on EnMAP (224 bands), PRISMA and AVIRIS-NG (372 bands) filled those gaps in the record; it now runs to 2,500.4 nm with no gap wider than 30 nm.

Source check: 185 to 165 is a pretrained model's number

`docs/lld/spectral-stem.md` ("set1 native: 185 -> 165") and Chapter 24 quote 20 of 185 bands dropped. That is right for a model whose record is the pretraining curve. The served v2.1.0 keeps all 185, because its `trained_wl` has 733 entries [C]. The FTH3 README's CPU timing ("1186 x 1196 x 185 bands") is consistent with the served model. So $N = 185$ below.

43.4.2 The stem, at 32,768 pixels

The stem treats every pixel separately, so it flattens the batch to $P = B \times H \times W = 2 \times 128 \times 128 = 32{,}768$ rows (Chapter 44, next, walks the same code step by step at the namer's sizes). The 185 kept bands fall into 45 of the 53 bins of 40 nm, at most 6 to a bin [C].

43.4.3 The encoder

Worked example 43.3: What the masked stage 1 costs and holds

Masked, stage 1 attends among the 359 kept tokens (Chapter 15): $359^2 = 128{,}881$ scores per head, three heads, $3 \times 128{,}881 = 386{,}643$ per tile per block. Unmasked it would use 16 keys: $1{,}024 \times 16 = 16{,}384$ per head. The masked pass is the expensive one, by $128{,}881 / 16{,}384 = 7.9$ times per head.

#	operation	shape
0	reflectance in; normalised by the wavelength curve	`(2, 185, 128, 128)`
1	wavelength encoding of every band	`(185, 32)`
2	bin membership (one-hot)	`(185, 53)`
3	width-weighted values; weight per bin	`(32768, 185)`; `(32768, 53)`
4	band vectors $\mathbf{k}(\lambda)$; their matrix $\mathbf{K}$; bin tokens	`(185, 32)`; `(185, 1696)`; `(32768, 53, 32)`
4	bin position bias	`(53, 32)`
5	queries (64 of them, 4 heads of 8); keys and values	`(32768, 4, 64, 8)`; `(32768, 4, 53, 8)`
5	attention weights (never stored)	`(32768, 4, 64, 53)`
6	answers; `to_channel`; skip path	`(32768, 64, 32)`; `(32768, 64)`; `(32768, 64)`
7	sum, back to a picture; BatchNorm	**z**: `(2, 64, 128, 128)`

Table 43.4: The reduced-attention stem of Heavy, $D = 64$, token width 32, 4 heads, 40 nm bins, at $N = 185$ bands [C]. The weights of row 5, if stored, would be $32{,}768 \times 4 \times 64 \times 53 = 444{,}596{,}224$ numbers, 1,696 MiB in float32; the fused attention kernel never forms them, and 32,768 rows is exactly the stem's `SDPA_MAX_ROWS`, so one call suffices.

stage	step	shape	attention
1	patch embed $k4\,s4$, $64 \to 96$	`(2, 1024, 96)`	
	remove 665 hidden tokens	`(2, 359, 96)`	
	2 blocks; FFN $96 \to 384 \to 96$	`(2, 359, 96)`	3 heads $\times$ 32; full, 359^2
	norm, restore, reshape	F1 `(2, 96, 32, 32)`	
2	embed $k3\,s2$, $96 \to 192$; 2 blocks	`(2, 256, 192)`	6×32; 16 keys
	FFN $192 \to 768 \to 192$; reshape	F2 `(2, 192, 16, 16)`	
3	embed, $192 \to 480$; 2 blocks	`(2, 64, 480)`	10×48; 16 keys
	FFN $480 \to 1920 \to 480$; reshape	F3 `(2, 480, 8, 8)`	
4	embed, $480 \to 768$; 2 blocks	`(2, 16, 768)`	12×64; 16 keys
	FFN $768 \to 3072 \to 768$; reshape	F4 `(2, 768, 4, 4)`	

Table 43.5: Heavy's encoder on a masked pass, $B = 2$, 128×128, $\rho = 0.65$ [C]. "Keys" is the reduced key count: $(32/8)^2 = (16/4)^2 = (8/2)^2 = 16$, and 16 tokens at stage 4.

What flows where, per tile:

$$\begin{aligned} \text{input} &: 185 \times 128 \times 128 = 185 \times 16{,}384 = 3{,}031{,}040 \\ \text{stem output } \mathbf{z} &: 64 \times 16{,}384 = 1{,}048{,}576 \\ \text{F1} &: 96 \times 32 \times 32 = 96 \times 1{,}024 = 98{,}304 \\ \text{F4} &: 768 \times 4 \times 4 = 12{,}288 \end{aligned}$$

The stem cuts the values by 3,031,040/1,048,576 = 2.9; stage 1 by another 1,048,576/98,304 = 10.7, the "10.7 to 1" squeeze of Section 43.2.

43.4.4 The progressive decoder and the way back to 185 bands

#	operation	shape
1	1×1 projections of F1–F4 to 512; bilinear to 32×32	four of `(2, 512, 32, 32)`
2	concatenate; fuse $2048 \to 512$, GELU	`(2, 2048, 32, 32)` $\to$ `(2, 512, 32, 32)`
3	upsample $\times 2$	`(2, 512, 64, 64)`
4	skip: $\mathbf{z}$ with hidden tokens zeroed, 2×2 average	`(2, 64, 64, 64)`
5	concatenate; `up_half`: two 3×3 convs, GELU	`(2, 576, 64, 64)` $\to$ `(2, 256, 64, 64)`
6	upsample $\times 2$; concatenate the full-resolution skip	`(2, 256, 128, 128)`; `(2, 320, 128, 128)`
7	`up_full`: two 3×3 convs, GELU	`(2, 128, 128, 128)`
8	head, 1×1 conv $128 \to 64$	`(2, 64, 128, 128)`
9	per-band output weights $[\mathbf{u}(\lambda) \,\vert\, c(\lambda)]$	`(185, 65)`
10	$\hat{x}_n = \mathbf{u}(\lambda_n) \cdot \hat{\mathbf{z}} + c(\lambda_n)$; denormalise	`(2, 185, 128, 128)`

Table 43.6: The progressive decoder (Chapter 42) and the wavelength decoder at Heavy's widths [C]. Rows 4 and 6 are the skip that lets the decoder render detail below a token: the stem output itself, with every pixel of a hidden token set to 0 so no hidden value reaches the reconstruction.

The segmentation path. The class map comes from a second, *unmasked* pass through the same stem and encoder (`segment`), into a `SegFormerDecoder` of width 256 with five outputs per sub-pixel: four projections to `(2, 256, 32, 32)`, concatenated to `(2, 1024, 32, 32)`, fused to `(2, 256, 32, 32)`, refined, a sub-pixel convolution to $5 \times 16 = 80$ channels, `(2, 80, 32, 32)`, and the pixel shuffle to `(2, 5, 128, 128)`: one logit per class per pixel, land being "none of the five" (Chapter 28) [C].

43.4.5 The stem, the output and the segmentation head, counted by hand

Table 43.3 took its stem rows from the built model. Count them. Every small network in the stem is a two-layer perceptron of the 32-number wavelength encoding, $32 \to 64 \to$ out (`app/foundation_models/components/spectral_reduced.py`, `_hyper`), so it has $(32 \times 64 +$

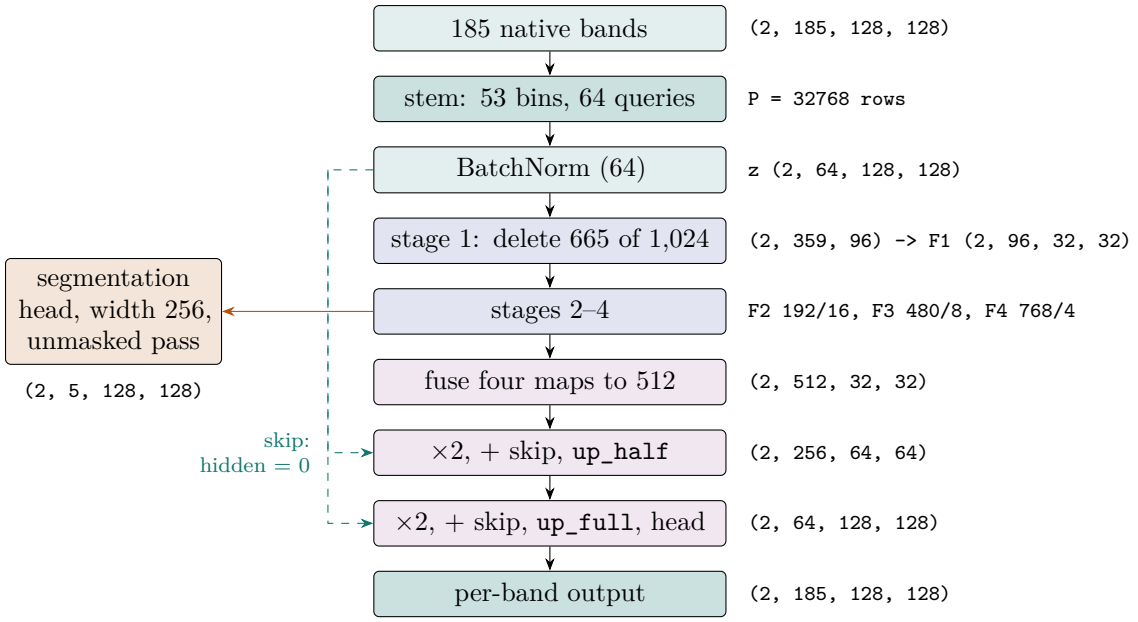


Figure 43.2: Indradhanu Heavy as served, one masked pass over two 128 × 128 PRISMA tiles with 185 bands, drawn from the shapes the forward hooks recorded [C]. Teal: spectral stem and output; blue: encoder; plum: progressive decoder; dashed teal: the skip from the stem output with hidden tokens zeroed; amber: the segmentation head, fed by a separate, unmasked encoder pass.

$64) + (64 \times \text{out} + \text{out}) = 2{,}112 + 65\,\text{out}$ parameters.

$$
\begin{aligned}
\text{band weights } \mathbf{k}(\lambda),\ \text{out} = 32 &: 2{,}112 + 65 \times 32 = 2{,}112 + 2{,}080 = 4{,}192 \\
\text{skip weights } W(\lambda),\ \text{out} = 64 &: 2{,}112 + 65 \times 64 = 2{,}112 + 4{,}160 = 6{,}272 \\
\text{bin bias, } 32 \to 32 &: 32 \times 32 + 32 = 1{,}056 \\
\text{queries} &: 64 \times 32 = 2{,}048 \\
\text{q, k, v, o, each } 32 \to 32 &: 4 \times 1{,}056 = 4{,}224 \\
\text{to_channel, } 32 \to 1 &: 32 + 1 = 33 \\
\text{stem} &: 4{,}192 + 6{,}272 = 10{,}464;\ +1{,}056 = 11{,}520;\ +2{,}048 = 13{,}568; \\
&\quad + 4{,}224 = 17{,}792;\ +33 = 17{,}825 \\
\text{output weights } [\mathbf{u} \,|\, c],\ \text{out} = 64 + 1 &: 2{,}112 + 65 \times 65 = 2{,}112 + 4{,}225 = 6{,}337 \\
\text{BatchNorm on } D = 64 &: 2 \times 64 = 128
\end{aligned}
$$

These are the 17,825, 6,337 and 128 of Table 43.3; none depends on the band count. The segmentation head is a `SegFormerDecoder` of width 256 writing $5 \times 16 = 80$ channels:

$$
\begin{aligned}
\text{projections} &: (96 + 192 + 480 + 768) \times 256 + 4 \times 256 = 1{,}536 \times 256 + 1{,}024 \\
&= 393{,}216 + 1{,}024 = 394{,}240 \\
\text{fuse} &: 1{,}024 \times 256 + 256 = 262{,}144 + 256 = 262{,}400 \\
\text{refine } 3 \times 3 &: 256 \times 256 \times 9 + 256 = 589{,}824 + 256 = 590{,}080 \\
\text{sub-pixel } 3 \times 3 &: 256 \times 80 \times 9 + 80 = 184{,}320 + 80 = 184{,}400 \\
\text{head} &: 394{,}240 + 262{,}400 = 656{,}640;\ +590{,}080 = 1{,}246{,}720; \\
&\quad + 184{,}400 = 1{,}431{,}120
\end{aligned}
$$

$$
33{,}884{,}194 + 1{,}431{,}120 = 35{,}315{,}314 \tag{43.2}
$$

trainable numbers in the served model, the documents' 35.3 M. Loading the shipped checkpoint strictly and summing its parameters gives 35,315,314 [C].

Source check: the manifest's 35,315,443

The v2.1.0 manifest gives `params` 35,315,443. `scripts/make_model_manifest.py` counts every value in the state dictionary except the `stats.*` curve and `trained_wl`, so it includes the stem BatchNorm's running mean and variance ($2 \times 64 = 128$) and its batch counter (1): $35{,}315{,}314 + 129 = 35{,}315{,}443$. The difference is buffers, not parameters, as for Indradhanu (Chapter 9) and Chakshu (Chapter 21).

43.5 Pretraining on a rented H100

The run used an *H100 capacity block*: AWS rents a GPU instance for a fixed window booked in advance, here a `p5.4xlarge` (one H100 with 80 GB, 16 vCPU) from 2026-10-28 11:30 to 2026-10-29 11:30 UTC [M: `F002-heavy/README.md`]. The window runs whether or not your job does, like a hired hall: an hour spent setting up is an hour of the booking gone.

UTC	event
10-28 11:30	block starts
12:38	launch (68 minutes into the block)
12:48	first start; crashes at the first validation, three times
13:10	the driver gives up after the third failure; GPU idle
14:05	restart with the fix, batch 32×1
22:55	80 epochs done
10-29 01:02–10:39	FTH1, FTB1, FTH2, FTB2, FTH3 (Section 43.8)
11:30	block ends

Table 43.7: Block 1. Source: [M: `F002-heavy/README.md`] and the FTH READMEs.

From 11:30 to 14:05 is 2 h 35 min = 155 minutes, 155/1,440 = 0.108 of the 24-hour booking: the README's "~2.5 h of block 1 were lost". Split by the timeline: 68 before launch, 10 of start-up, 22 in three failed starts, 55 idle; $68 + 10 + 22 + 55 = 155$.

43.5.1 The crash: 10 test shards, 12 loader workers

Training data arrives in *shards*: tar files of patches, read by `webdataset`. A data *loader worker* is a separate process that reads and decodes data while the GPU trains; the configuration asked for 12. Webdataset deals shards to workers in turn, and a worker with no shard has nothing to yield.

Worked example 43.4: Dealing shards to workers

Deal shards $0, 1, 2, \ldots$ round-robin: shard i goes to worker i mod 12.

10 shards : workers 0–9 get one each; workers 10 and 11 get none

16 shards : workers 0–3 get two (shards i, $i + 12$); workers 4–11 get one

check : $4 \times 2 + 8 \times 1 = 8 + 8 = 16$

With 10 shards, two workers are empty and webdataset raised "No samples found … fewer shards than workers". The rule: shards $\geq$ workers. The fix was `test_shards_per_size: 16`.

Why it went unseen for about 55 minutes, in the README's words: "My epoch watcher only looked for an epoch-1 row". A watcher that waits for success says nothing when the job fails; it waits. The watcher was changed to alert on any trainer exit or the driver stopping.

43.5.2 The stale log line

The second trap was in `h100_chain.sh`, the script that waits for pretraining and then runs the fine-tunes. Its committed version waited for the sentence "F002-heavy driver finished" in a log file, then checked that `metrics.csv` had 80 epochs and stopped if not. But the driver prints that sentence whenever its loop ends, including after the three failed starts at 13:10, and the log is appended across restarts. So the `grep` found the crashed run's line, counted fewer than 80 epochs, and the chain stopped itself. The fix waits on the thing it needs, the epoch count, and stops only if the driver process is gone:

```
# wait on the epoch count, not a log line: heavy.log is appended across restarts and
    keeps old "finished" lines
epochs() { echo $(( $(wc -l < $N/runs/F002-heavy/metrics.csv 2>/dev/null || echo 1) -
     1 )); }
until [ $(epochs) -ge 80 ]; do
  pgrep -f heavy_run.sh >/dev/null || { log "driver not running at $(epochs) epochs -
     chain stops"; exit 1; }
  sleep 120
done
```

Watch for failure, not only for success, and wait on the state you need (80 rows in `metrics.csv`), never on a message an earlier run may have left behind.

43.5.3 Batch size, throughput and the schedule

The first start used batch 16 with 2 accumulation steps; the restart used 32×1. The effective batch is 32 either way. At 16×2 the GPU ran 63–85% busy with 86% of the CPUs idle; at 32×1, 85–100% [M: `F002-heavy/README.md`]. Epoch 1 took 369.4 s, $30{,}000/369.4 = 81.2$ patches per second. All 80 epochs took 30,690 seconds of training, 8.5 hours.

F001 ran a 200-epoch cosine schedule (Chapter 27); Heavy ran 80, "cosine fully decayed", on the user's "we will lose steam after epoch 50-75". A cosine schedule (Chapter 26) lowers the learning rate along half a cosine wave from 3×10^{-4} to the floor 10^{-6}:

$$\eta(e) = 10^{-6} + \left(3 \times 10^{-4} - 10^{-6}\right) \frac{1 + \cos(\pi e/80)}{2}. \tag{43.3}$$

Worked example 43.5: The learning rate at epochs 40 and 60

$e = 40 : \cos(\pi/2) = 0;\ \frac{1+0}{2} = 0.5;\ 2.99 \times 10^{-4} \times 0.5 = 1.495 \times 10^{-4};\ +10^{-6} = 1.505 \times 10^{-4}$

$e = 60 : \cos(3\pi/4) = -0.7071;\ \frac{1-0.7071}{2} = 0.14645;\ 2.99 \times 10^{-4} \times 0.14645 = 4.379 \times 10^{-5};$
$+ 10^{-6} = 4.479 \times 10^{-5}$

`metrics.csv` logs 1.5050×10^{-4} and 4.4788×10^{-5} [M: `F002-heavy/metrics.csv`]: the schedule is evaluated at the epoch number itself.

The data rotated: training shards in blocks of 200, a new block every 5 epochs (F001: every 20), 16 blocks, 3,200 of the 3,419 training shards, 30,000 patches per epoch.

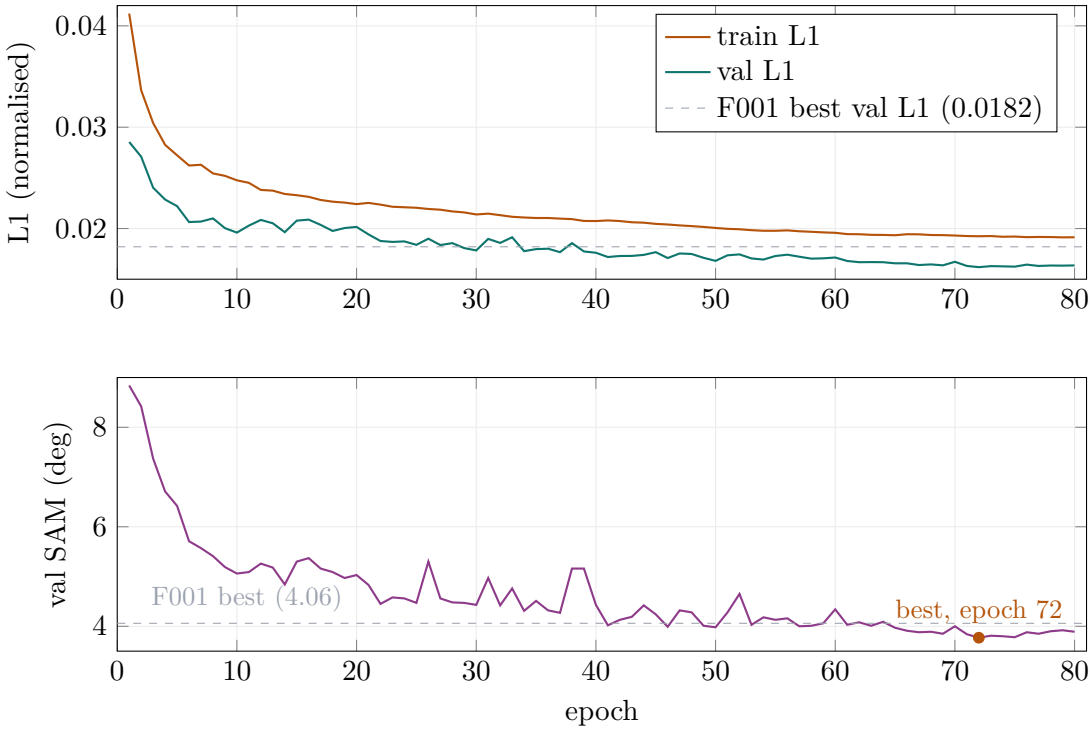


Figure 43.3: Indradhanu Heavy's pretraining, all 80 epochs drawn from `metrics.csv` [M: `F002-heavy/metrics.csv`]. Train L1 lies above validation L1 throughout (see the text), and both keep falling.

43.6 The curve: did it overfit?

Values that will drift

F002-heavy, 2026-10-28: best validation L1 / SAM 0.0162 / 3.77° at epoch 72; epoch 80 0.0164 / 3.89°. F001's best in 113 epochs: 0.0182 / 4.06° at epoch 100 [M: `F002-heavy/README.md`]. Heavy first beat F001's L1 at epoch 29 (0.0181) and its SAM at epoch 41 (4.02°).

The user's prediction holds in the numbers: from epoch 1 to 20 validation L1 fell $0.0285 - 0.0202 = 0.0083$; from 50 to 72, $0.0168 - 0.0162 = 0.0006$, fourteen times less ($0.0083/0.0006 = 13.8$).

One caveat the README does not make: F001 validated on test shards 0–9, Heavy on 16 test shards because of the crash fix. Both draw 1,500 patches per validation, but not the same 1,500. The gap is large next to epoch-to-epoch wobble, so its direction is safe; its exact size is not.

Definition 43.2 (Overfitting)**.** A model *overfits* when it learns its training examples rather than the pattern behind them: training loss keeps falling while validation loss rises.

Two pieces of evidence say Heavy did not.

1. **The train–validation gap is steady.** Train L1 sits above validation L1 throughout, with no trend. Train above validation is expected here: training hides 65% of tokens, reads a random band subset in 80% of batches and runs with dropout 0.4, while validation hides 50% twice with complementary masks and no augmentation.
2. **Nothing is memorised within a block.** Each block's 200 shards are seen for 5 epochs. A model memorising them would show train loss falling inside a block and jumping back up at the next block. Across the 15 block changes the largest rise is +0.00012 (epoch 20 to 21), and train L1 falls inside all 16 blocks; validation falls inside 13 of 16 [C].

Source check: the size of the gap

The course (part 68) gives the gap as 0.0023 to 0.0033 from epoch 20 to 80. Computing it from every row of `F002-heavy/metrics.csv` gives 0.00201 to 0.00366 [C](`research/textbook/tools/compute_39-41_output.txt`); the course's range comes from the rounded summary rows. The conclusion, a steady gap with no trend, is unchanged. The block-change jump (+0.00012) and the 16 and 13 blocks reproduce exactly.

43.7 Pretrained detection, and the decision

	pAUC	caught top 1% / 0.1%	TPR @ 0.1%
F001 (pretrained, FT04 size)	0.155	45.1% / **14.0%**	0.059
Heavy (pretrained, epoch 72)	**0.166**	**47.4%** / 12.4%	0.058
FT04 (F001 + fine-tuning)	0.170	46.0% / 15.8%	0.065

Table 43.8: Pretrained detection, real PRISMA bench, residual Mahalanobis, cloud excluded on FT04's masks. Source: [M: `F002-heavy/README.md`].

Paired bootstrap, Heavy − F001: pAUC +0.0111 [−0.0003, +0.0223], $P = 0.971$: the interval touches zero, not resolved, by a hair. Caught at the top 0.1%: −1.5 points [−3.0, −0.2], a resolved loss.

Worked example 43.6: Where the top-0.1% loss sits

Per-scene pAUC, Heavy 0.259 / 0.150 / 0.071 / 0.185, F001 0.220 / 0.126 / 0.080 / 0.195:

$$\begin{aligned} \text{Heavy} &: 0.259 + 0.150 + 0.071 + 0.185 = 0.665; \quad 0.665/4 = 0.166 \\ \text{F001} &: 0.220 + 0.126 + 0.080 + 0.195 = 0.621; \quad 0.621/4 = 0.155 \end{aligned}$$

Objects caught at the top 0.1%, Heavy 53 / 23 / 12 / 25, F001 52 / 27 / 11 / 37:

$$\begin{aligned} \text{Heavy} &: 53 + 23 + 12 + 25 = 113; \quad 113/909 = 0.124 \\ \text{F001} &: 52 + 27 + 11 + 37 = 127; \quad 127/909 = 0.140 \\ \text{loss} &: 127 - 113 = 14 \text{ objects, of which set4 } 37 - 25 = 12 \end{aligned}$$

Twelve of the fourteen lost objects are on set4, the hazy scene. FT04's lead over F001 is $0.170 - 0.155 = 0.015$; pretrained Heavy recovers $0.166 - 0.155 = 0.011$ of it, $0.011/0.015 = 0.73$ (the README rounds to "~75%").

The decision, 2026-10-29: "Indradhanu Heavy is the base pretrained model" (user: "I think heavy is our base model now"). Be clear what the evidence was at that moment: better reconstruction, a near-resolved pAUC gain as a pretrained model, and a resolved loss at the top 0.1%. It was a judgement on the better general representation, not a detection win. That came next.

43.8 The fair test: a control and three checkpoints

FT04 is F001 plus a four-stage curriculum (Chapters 29 and 38). Block 1 had about 12 hours left after pretraining, not enough to repeat four stages for Heavy. So Heavy got FT04's *final* recipe in one joint run. If that model beat FT04, you could not say whether the bigger model

won or something else in the change did; if it lost, it might be the short recipe. Two causes mixed into one comparison: that is *confounding*.

Definition 43.3 (Control)**.** A *control* is a run identical to the experiment in every way except the thing under test.

`FTB1-base-joint/config.json` says it directly: "F001 (FT04's size, SegFormer decoder) fine-tuned with exactly FTH1's one-run recipe, so Heavy vs FTB1 isolates size + decoder + pretraining from the recipe." The control separates the *model* from the *recipe*; it does not separate size alone, because Heavy also differs in decoder, pretraining run and segmentation-head width (256 against 128).

run	starts from	joint epochs	lr (cosine)	total joint epochs
FTH1	Heavy, epoch 72	10	10^{-4}	10
FTB1 (control)	F001 epoch 109	10	10^{-4}	10
FTH2	FTH1	8	5×10^{-5}	18
FTB2 (control)	FTB1	8	5×10^{-5}	18
FTH3	FTH2	6	3×10^{-5}	24 (no control)

Chapter 42 ended on a lesson: FT09b's gain cleared the object bootstrap and vanished five epochs later. So these runs kept a checkpoint after every epoch, and every comparison is made at **three neighbouring checkpoints**. A gain counts if it holds at all three.

43.8.1 FTH1 and FTB1

At epoch 10 both one-run models fall short of FT04's curriculum on PRISMA reconstruction and masks, and Heavy reconstructs *worse* than the control: SAM 3.94 / 4.74 / 5.60° against FTB1's 3.90 / 4.48 / 5.34° (FT04 3.88 / 4.11 / 5.46°) [M: `FTH1-heavy-joint/README.md`]. On these numbers alone Heavy looked like a disappointment.

Detection, with cloud excluded on FT04's masks for both, so only the detector differs:

checkpoint	FTB1 pAUC	FTH1 pAUC	FTH1 − FTB1 [95%]
epoch 8	0.162	**0.175**	+0.0132 [+0.0022, +0.0243]
epoch 9	0.162	**0.171**	+0.0093 [−0.0013, +0.0195]
epoch 10	0.161	**0.175**	+0.0145 [+0.0045, +0.0246]

Heavy ahead at all three, resolved at two. **Worse reconstruction, better detection.**

Source check: 0.182 or 0.183

Against shipped FT04 with the control's masks, the FTH1 README gives epochs 8 / 9 / 10 pAUC 0.182 / 0.178 / 0.182. Its `eval_report.md` row for epoch 8 reads 0.183 (`FTH1-heavy-joint/eval_report.md`, line 17). The two are the same number rounded differently; the bootstrap's +0.0130 over FT04's 0.1695 puts it at 0.1825.

43.9 The mask failure: haze in the thin-cloud head

Excluding cloud with FTH1's *own* head wrecked every detector it was applied to: FT04 fell from 0.170 to 0.127 pAUC with FTH1's masks, and FTH1 itself from 0.182 to 0.133 (ROC-AUC 0.742, caught top 1% 32.8%) [M: `FTH1-heavy-joint/eval_report.md`]. Yet FTH1's EnMAP validation cloud IoU, 0.717, was close to FTB1's 0.721.

The product's class map comes from `segment_full_scene`. The head predicts five independent probabilities per pixel (water, cloud, thin cloud, shadow, snow; Chapter 28), and the class map sets priorities by assignment order, cloud last so it wins:

```
cls[prob[0] > 0.5] = 1                                  # water
cls[prob[3] > 0.5] = 3                                  # shadow
cls[prob[4] > 0.5] = 4                                  # snow
cls[np.maximum(prob[1], prob[2]) > 0.5] = 2             # cloud: cloud OR thin cloud
```

A pixel is excluded as cloud if *either* cloud output passes 0.5.

	set1	set2	set3	set4
FT04	0.0% / 0	0.0% / 0	0.8% / 0	11.8% / 2
FTB1	0.0% / 0	0.0% / 0	1.7% / 0	0.3% / 0
FTH1	0.4% / 17	0.0% / 0	2.1% / 0	**75.4% / 209 of 230**

Table 43.9: Share of each bench scene called cloud, and targets inside it. Source: [M: `FTH1-heavy-joint/README.md`].

"All of it is the thin-cloud output": on set4, FTH1's thick-cloud channel covers 0.4% (no targets), its thin-cloud channel 75.4% (209 targets). That is the haze over set4. Thin cloud was always the weak label (its IoU was 0.07 for FTH1), and the validation IoU above measures the *cloud* channel, which is why it did not warn anyone.

43.9.1 Why zeroing targets wrecks pAUC

Cloud exclusion sets the score of every excluded pixel to 0 after scoring. Recall from Chapters 33 and 36 the per-target credit $c(p) = \max(0,\ 0.01 - \mathrm{FPR}(p))/0.01$, where $\mathrm{FPR}(p)$ is the share of background pixels scoring at or above target pixel p, and pAUC is the mean credit. A target scored 0 has every background pixel at or above it: $\mathrm{FPR} = 1$, so $c = \max(0, 0.01 - 1)/0.01 = 0$.

Worked example 43.7: Switching off a scene's targets

Ten one-pixel targets, each with $c = 0.5$. Zero nine of them.

$$\text{before} : (10 \times 0.5)/10 = 5.0/10 = 0.50$$
$$\text{after} : (1 \times 0.5 + 9 \times 0)/10 = 0.5/10 = 0.05$$

Now set4, as an estimate. FTH1 epoch 10 with thick-cloud-only exclusion (below) scores set4 at 0.222. Assume ([A]) the 209 excluded targets are typical of set4's 230, so the 21 kept ones carry their share:

$$\text{kept share} : 21/230 = 0.0913$$
$$\text{set4 pAUC} : 0.222 \times 0.0913 = 0.020$$
$$\text{bench, thick} : (0.253 + 0.162 + 0.086 + 0.222)/4 = 0.723/4 = 0.181$$
$$\text{bench, estimate} : (0.253 + 0.162 + 0.086 + 0.020)/4 = 0.521/4 = 0.130$$
$$\text{measured} : 0.133$$

The estimate ignores set1's 17 excluded targets and assumes the targets alike, so do not read the third decimal. But it shows the collapse is one scene's targets being switched off.

A mask head that calls haze "cloud" deletes the targets under the haze.

43.9.2 The fix: thick cloud only

The evaluator gained a switch, `ModelSpec.cloud_rule` in `app/evaluation/config.py`: `"cloud_or_thin"` (the default) or `"thick"`, described as "thin cloud fires on haze". Under the thick rule a pixel that was cloud only because of thin cloud falls to whatever it would otherwise have been: snow, shadow, water or land. With FTH1's own head and the thick rule: epoch 10 pAUC 0.181 (sets 0.253 / 0.162 / 0.086 / 0.222), the same as with FTB1's masks. Does the thick rule hurt FT04? FT04 shipped → FT04 thick: -0.0004 $[-0.0015, +0.0015]$, a tie [M: `FTH2-heavy-joint-more/bootstrap.txt`].

Source check: where the thick rule lives now

The course (part 69) shows the rule as an `np.select` in `app/evaluation/models.py` that re-assigns thin-only pixels after `segment_full_scene`. In the current source the rule has moved into `segment_full_scene` itself, as a parameter `thin_cloud` (default `True`): the cloud line becomes `cls[(np.maximum(prob[1], prob[2]) if thin_cloud else prob[1]) > 0.5] = 2`, and `models.py` calls it with `thin_cloud=self.spec.cloud_rule!="thick"` (`app/foundation_models/inferencers/spectral_stem_inferencer.py`, lines 112–141). The two give the same class map: with thin cloud ignored, a thin-only pixel keeps the class from the earlier lines, where snow overwrites shadow overwrites water, which is the course's priority snow, shadow, water, land.

43.10 FTH2, FTB2 and FTH3: a gain that holds

From here every model excludes cloud with its own thick-cloud masks.

	pAUC epoch 6 / 7 / 8	caught top 1%	caught top 0.1%
FT04 shipped (cloud or thin)	0.170	46.0%	**15.8%**
FTB2 (own thick)	0.160 / 0.163 / 0.162	46.1 / 46.4 / 46.8%	15.1 / 15.1 / 15.1%
FTH2 (own thick)	**0.185 / 0.184 / 0.186**	**48.2 / 49.0 / 49.0%**	15.3 / 15.2 / 15.7%

Table 43.10: 18 joint epochs. Source: [M: `FTH2-heavy-joint-more/README.md`].

Every pAUC interval clears zero, at every checkpoint, against both the control and FT04 (Figure 43.4). The average gap over the control:

$$(0.0245 + 0.0209 + 0.0244)/3 = 0.0698/3 = 0.0233.$$

Source check: "about twice" or "about three times"

The course (part 69) quotes the FTH2 README as calling this gap "about twice what cloud exclusion bought", and corrects it: cloud exclusion is +0.0074 (Chapter 36), and $0.0233/0.0074 = 3.1$. The current README reads "about three times what cloud exclusion bought, +0.0074" (`FTH2-heavy-joint-more/README.md`): the source was corrected after the course note.

The gap *grew* with fine-tuning: about +0.013 at 10 epochs, +0.023 at 18. And reconstruction still did not separate the models: FTH2's PRISMA SAM (4.48°) is worse than the control's (4.36°).

FTH3: the plateau. Six more joint epochs at 3×10^{-5}, Heavy only, from the source [M: `FTH3-heavy-joint-more2/README.md`]. Epochs 4, 5 and 6 score pAUC 0.184, 0.189 and 0.187. Against FTH2 epoch 8 the three differences are:

-0.0021 $[-0.0043, -0.0000]$, $+0.0028$ $[-0.0002, +0.0058]$, $+0.0011$ $[-0.0009, +0.0030]$:

one barely below zero, one barely above, one a tie. That is a *plateau*: more epochs at this recipe no longer move detection. Against FT04 all three stay resolved: $+0.0148$, $+0.0196$, $+0.0179$. The release candidate is FTH3 epoch 5: pAUC 0.189 against 0.170, caught top 1% 49.4% against 46.0%, top 0.1% 16.0% against 15.8% (a tie, $[-1.2, +1.5]$). Pick it as the plateau's representative, not as a discovery: it is the best of a stable set and the only one whose top-0.1% count is not below FT04's.

Worked example 43.8: Mean gain over FT04 at three checkpoints

FTH2 − FT04 at epochs 6, 7, 8: $+0.0151$, $+0.0143$, $+0.0168$.

$$0.0151 + 0.0143 + 0.0168 = 0.0462$$
$$0.0462/3 = 0.0154$$

The lowest lower bound is $+0.0032$ (epoch 7), so none touches zero. FTH3 at epochs 4, 5, 6: $0.0148 + 0.0196 + 0.0179 = 0.0523$, $0.0523/3 = 0.0174$.

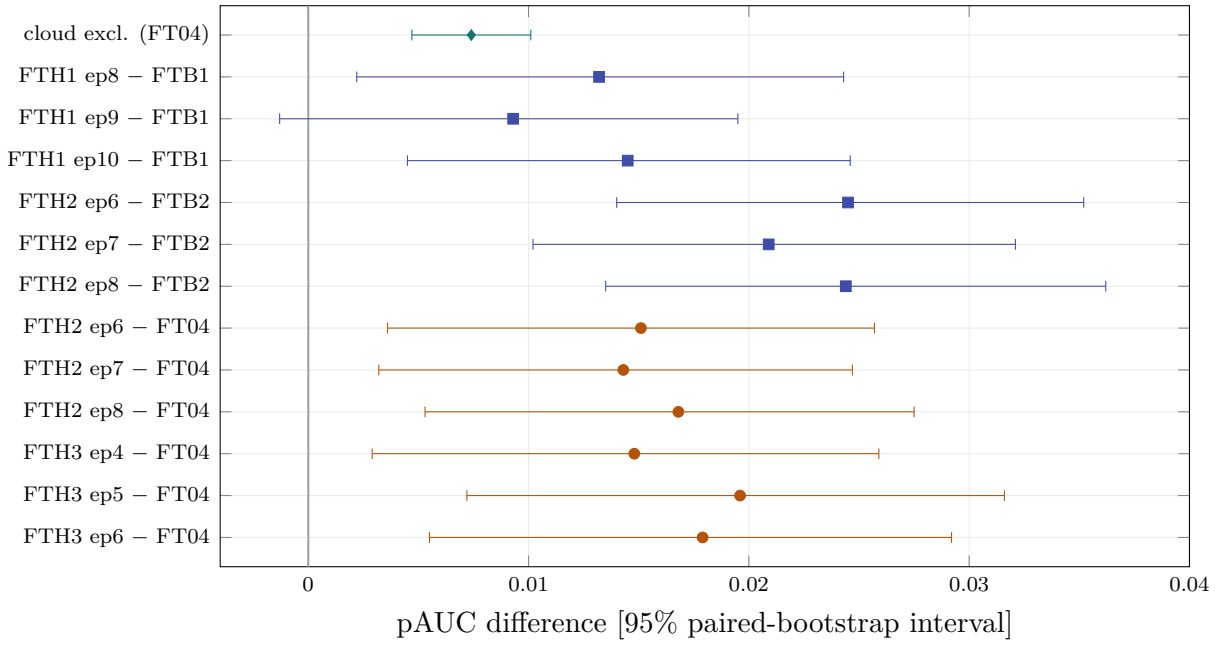


Figure 43.4: Heavy against the control (indigo squares) and against shipped FT04 (amber circles), at three neighbouring checkpoints each, drawn to scale; cloud exclusion (teal) for reference. Only FTH1 epoch 9 touches zero. Sources: [M: `FTH1-heavy-joint/README.md`], [M: `FTH2-heavy-joint-more/bootstrap.txt`], [M: `FTH3-heavy-joint-more2/bootstrap.txt`].

43.11 The lesson: reconstruction did not separate the models

	PRISMA SAM (deg)	pAUC
FTB1 (control, 10 epochs)	**4.48**	0.161–0.162 (FT04 masks)
FTH1 (Heavy, 10 epochs)	4.74	**0.171–0.175** (FT04 masks)
FTB2 (control, 18 epochs)	**4.36**	0.160–0.163 (own thick masks)
FTH2 (Heavy, 18 epochs)	4.48	**0.184–0.186** (own thick masks)

The model that reconstructs worse is the better detector, at every stage. Had

the decision been made on validation SAM, the number every run logs each epoch, Heavy would have been dropped after FTH1. The FTH2 README: "Better detection without better reconstruction." This is the third time in Part VIII: FT06's best-NLL epoch was its worst for detection, FT09c reconstructed better and detected worse, and now this.

Judge detectors on detection: reconstruction error and detection come apart, so a model must be selected on the measure the product uses.

43.12 Into the product

FTH3's verdict listed three things the release candidate needed before an analyst could use it: the thick-cloud rule, a manifest and resolver path for the progressive decoder, and a check of the worker's CPU cost. All three were done on 2026-10-29, and FTH3 epoch 5 became Indradhanu V2 `v2.1.0` the same day, behind FT04's codename and architecture key, with FT04 (`v2.0.0`) kept as the manifest's first alternative (`docs/lld/spectral-stem.md`, "Serving"). Because the manifest embeds the checkpoint's whole model configuration (Chapter 39), switching models changed files, not code.

What the product loads. The serving documents give 35.3 M parameters. Section 43.3 counted the pretrained network at 33,884,194; the fine-tuned checkpoint adds the segmentation head, 1,431,120 counted by hand in Section 43.4.5, for the 35,315,314 of Equation (43.2): the 35.3 M [C].

The thick-cloud rule in the product. The resolver's capabilities entry for `spectral_stem_finetune` sets `thin_cloud_is_cloud=False`; the worker passes it to `segment_full_scene(..., thin_cloud=False)`, so only the head's thick-cloud channel can make a pixel "cloud", and `stats.json` records `cloud_rule: "thick"` (Chapter 39, the source check on the cloud rule). Pixels that are only thin cloud fall to the next class by priority and are scored.

Worked example 43.9: Parity, to the digit

The parity run (`FTH3-heavy-joint-more2/product_parity.py`) takes the product's own path, resolver manifest to inferencer to `read_bands`, prediction, residual Mahalanobis, segmentation with the thick rule and the zeroing, and compares with the evaluator's run on the four PRISMA scenes [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/README.md`]:

scene	thick cloud excluded	pAUC product	pAUC evaluator	gap
set1	0.00%	0.2617	0.26165	+0.00005
set2	0.00%	0.1874	0.18742	−0.00002
set3	1.49%	0.0861	0.08609	+0.00001
set4	0.47%	0.2217	0.22167	+0.00003

The product column's mean:

$$0.2617 + 0.1874 = 0.4491$$
$$0.4491 + 0.0861 = 0.5352$$
$$0.5352 + 0.2217 = 0.7569$$
$$0.7569/4 = 0.1892.$$

The evaluator's: $0.26165 + 0.18742 + 0.08609 + 0.22167 = 0.75683$, and $0.75683/4 = 0.1892$. Every gap is smaller than half of the product column's last printed digit, 0.00005: the gaps

are rounding. Note set4: 0.47% thick cloud, where the old rule had called 75.4% of the scene cloud (Section 43.9).

In objects: the comparison page shown to the user on 2026-10-29 (`https://claude.ai/artifact/2dZFbGN6HhMBCx3kkZ8pQ4`) counts the 909 labelled targets of the four scenes caught in each model's top 1%, FT04 418 and Heavy 449.

$$418/909 = 0.4598 \to 46.0\%, \qquad 449/909 = 0.4939 \to 49.4\%, \qquad 449 - 418 = 31 \text{ objects},$$

the FTH3 README's caught-at-1% figures for FT04 and FTH3 epoch 5 [C].

Sixteen threads. The first CPU check (`cpu_timing.py`, inside the worker container, PRISMA set1, 1186×1196 pixels, 185 bands) gave Heavy 1,819 s and FT04 1,058 s, about 30 and 18 minutes a scene. The worker was running torch on 2 threads. A *thread* is one line of work a program runs at a time; a numerical library splits a large matrix product across as many threads as it is allowed. The cap of 2 was set in two places, the worker image's `ENV` (`backend/Dockerfile.worker`) and an `os.environ.setdefault(..., "2")` in `backend/allotrope_worker/__init__.py`, after crashes that the compose file's comment says were "Docker Desktop UI's restart button, not throttling". Since 2026-10-29 `docker/docker-compose.yml` sets `OMP_NUM_THREADS`, `MKL_NUM_THREADS`, `OPENBLAS_NUM_THREADS` and `NUMEXPR_NUM_THREADS` to `${WORKER_THREADS:-16}`: compose's `environment` replaces the image's `ENV`, and `setdefault` never overwrites a variable that is already set, so the 16 wins inside compose and the 2 still applies outside it.

Worked example 43.10: Four times faster from eight times the threads

From the FTH3 README [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/README.md`]:

step	2 threads	16 threads	speed-up
Heavy, total	1,819 s	454 s	$1819/454 = 4.01$
predict	1,629 s	402 s	$1629/402 = 4.05$
segment	185 s	47 s	$185/47 = 3.94$
score	4 s	4 s	1.00
FT04, total	1,058 s	274 s	$1058/274 = 3.86$

In minutes, Heavy goes from $1819/60 = 30.3$ to $454/60 = 7.6$, and still costs $454/274 = 1.66$ times FT04. Eight times the threads ($16/2 = 8$) bought four times the speed. A standard model for why is *Amdahl's law*: if a share p of the work splits perfectly over n threads and the rest does not, the time is proportional to $(1-p)+p/n$. It is a model, not a measurement. Set the speed-up from 2 to 16 threads to 4 and solve for p:

$$\begin{aligned}
\frac{(1-p)+p/2}{(1-p)+p/16} &= 4 && \text{the measured ratio} \\
1 - \tfrac{1}{2}p &= 4(1 - \tfrac{15}{16}p) && \text{simplify each side, multiply up} \\
1 - 0.5p &= 4 - 3.75p \\
3.25p &= 3 && \text{collect the } p \text{ terms} \\
p &= 0.923.
\end{aligned}$$

The measured speed-up behaves as if about 92% of the work were perfectly parallel [C]. The score step did not speed up at all, but at 4 s it is a small part; the rest of the serial share is reading, writing and bookkeeping, and threads also share memory bandwidth, which the law ignores.

Source check: Heavy has shipped

The course's part 69, as written on 2026-10-29, ended with "Not shipped: the product still runs Indradhanu V2 (FT04)" (corrected on 2026-10-31; the course's part 70 now covers the ship). The source had moved on the same day: FTH3 epoch 5 was shipped on 2026-10-29 as Indradhanu V2 `v2.1.0`, same codename and architecture key as FT04 (FT04 `v2.0.0` is the manifest's first alternative), and the product now excludes thick cloud only (resolver `thin_cloud_is_cloud=False`) (`FTH3-heavy-joint-more2/README.md`, "Shipped"; commit c4b3a63). The three items FTH3's verdict listed before release (the thick-cloud rule, a manifest path for the progressive decoder, a worker CPU-cost check) were done.

Values that will drift

Product checks recorded 2026-10-29 in `FTH3-heavy-joint-more2/README.md`. Parity: the product path scored PRISMA sets 1–4 at pAUC 0.2617 / 0.1874 / 0.0861 / 0.2217 against the evaluator's 0.26165 / 0.18742 / 0.08609 / 0.22167, mean 0.1892 for both. Worker CPU cost on PRISMA set1 (1186×1196×185 bands, 2 torch threads): Heavy 1,819 s against FT04 1,058 s, 1,819/1,058 = 1.7 times, far less than the 6.1 times parameter ratio; on 16 threads (compose, since 2026-10-29) 454 s against 274 s [M:`FTH3-heavy-joint-more2/README.md`]. The Dockerfile and `allotrope_worker/__init__.py` still default to 2 for anything run outside compose. The open caveats remain: four scenes, and a bootstrap that resamples objects, not scenes.

43.13 Common confusions

Heavy is 4x wide.
That was its launch name. Its encoder widths are 3 times FT04's, its latent and decoder 2 times; its parameter count is 6.1 times overall and 8.9 times for the encoder.

Three times the width should mean three times the parameters.
Almost every weight is in a $C \times C$ matrix, so the count grows like C^2: $3^2 = 9$, less a little for the biases and norms (8.86 at stage 1).

Six times the parameters should mean six times slower.
On the H100 an epoch took about 370 s; on the product's CPU worker Heavy took 1.7 times FT04's time, on 2 threads and on 16 alike. Speed depends on hardware and on how busy it is kept, not on the parameter count alone.

FTH1 − FTB1 measures the effect of size.
It measures the effect of the model: size, decoder, pretraining run and head width together. The recipe is what it holds fixed.

FTH1's masks were bad, so Heavy's segmentation head is broken.
Its thick-cloud channel works; thick-only exclusion with its own head matches the control's masks. The thin-cloud channel fires on haze, and the product's old rule took either channel. FT04's head did it too, on 11.8% of set4.

The thick rule was chosen to make Heavy look good.
It was chosen after a diagnosis showed thin cloud covering 75% of a hazy scene, and checked on FT04, where it changes nothing (-0.0004 $[-0.0015, +0.0015]$). It was still adopted after seeing results, one more reason the release needs more than this bench.

FTH3 epoch 5 beat FTH2, so the extra epochs helped.
Its gain over FTH2 is unresolved and its neighbour is slightly below: a plateau.

The served model drops 20 of PRISMA's 185 bands.
A pretrained model does; its record of trained wavelengths is the SpectralEarth curve. The served v2.1.0's record grew to 733 wavelengths during fine-tuning, and it keeps all 185.

35,315,443 parameters.
That is the manifest's count, which includes 129 BatchNorm buffers; the trainable count is 35,315,314.

Train loss above validation loss means something is broken.
Training is the harder task here: more hidden tokens, missing bands, dropout. Watch the gap's trend, not its sign.

43.14 Check yourself

Exercise 43.1. Use Equation (43.1) to count FT04's and Heavy's stage-2 block ($C = 64$ and 192, $R = 4$). Check both against Table 43.2, and compute the ratio. Then predict the ratio for a 2 times wider FT04 at stage 1.

Exercise 43.2. Count Heavy's stage-2 patch embedding (3×3, stride 2, 96 to 192 channels, plus LayerNorm), and FT04's ($32 \to 64$).

Exercise 43.3. Using Equation (43.3), what learning rate does epoch 20 log? ($\cos(\pi/4) = 0.7071$.)

Exercise 43.4. With 16 test shards, what is the largest number of loader workers that avoids an empty worker? With 12 workers, how many shards would give every worker exactly two?

Exercise 43.5. A pixel's thick-cloud probability is 0.3, thin cloud 0.8, water 0.6, and nothing else passes 0.5. What class does the old rule give it, and what does the thick rule give it?

Exercise 43.6. A scene has 8 one-pixel targets with credits 1.0, 0.8, 0.8, 0.6, 0.4, 0.4, 0.0, 0.0. Compute its pAUC; then exclusion zeroes the first three. Compute it again. Why is the drop larger than 3 of 8?

Exercise 43.7. Why could FTH1 losing to FT04 not have shown that Heavy is the worse model, and what does FTB1 fix and not fix?

Exercise 43.8. With $p = 0.923$, predict the speed-up from 2 to 24 threads with Amdahl's law. Then give two reasons not to trust the prediction on the 24-core host.

Exercise 43.9. Count by hand the stem's output network (`spectral_out`) for FT04's latent width $D = 32$, and check it against Table 43.3. Then count FT04's segmentation head: width 128, five classes, FT04's encoder widths 32, 64, 160, 256.

Exercise 43.10. At inference the product hides half the stage-1 tokens of each tile, twice. For Heavy on a 128×128 tile, how many tokens does each pass keep, and how many attention scores per head does stage 1 compute? With the product's batch of 8 tiles, how many rows does the stem see, and how many calls of the fused attention kernel does that take with `SDPA_MAX_ROWS` = 32,768?

That closes the experiments. Indradhanu Heavy is the base model, and its fine-tuned FTH3 epoch 5 is what the product's Indradhanu V2 now runs; every number behind that decision is in this chapter, with its interval.

Next: Chapter 44, The spectral namer as built.

Part IX

Naming and delivering

Chapter 44

The spectral namer as built

The one thing this chapter teaches

The spectral namer is a 347,685-parameter two-tower network on Indradhanu V2's wavelength-aware stem: one tower reads a pixel with its ring of neighbours, the other reads a laboratory spectrum, and both meet in one table of cosines; since 2026-10-31 *both* of its questions are answered from that table, the material class by pooling the cosines of each man-made class, and natural-or-man-made by comparing the pooled man-made cosines with the pooled natural ones, because a single linear head on the pixel tower sat at chance.

Section 32.4 gave the *why* of the redesigned naming step: use-based families, two questions in turn, matching instead of classifying, and the separability check as a classical baseline. The design became `docs/proposals/spectral-namer.md`, and then code, all on 2026-10-31. This chapter reads the code from end to end, with every tensor shape and every parameter, and it tells the one change made after the first run started: *level 1 by matching*.

Status when this chapter was written. Run N001 started on an H100 capacity block at 11:30 UTC on 2026-10-31 (block until 2026-11-02 11:30 UTC). After its first epoch its level-1 head was at chance; it was stopped and replaced by N001b (`configs/namer/N001b.json`), the same run with level 1 by matching, and by N002, the open-set run, also with matching. **The results of N001b and N002 are pending**: N001b's README records its first epoch only, and N002 has none yet, so nothing in this chapter is a final result of either. Every number is a setting [M: `configs/namer/N001b.json`], a count or worked number from this book's script `research/textbook/tools/compute_namer_full.py` [C], a CPU smoke test, or one of the early measurements now recorded in `research/experiments/spectral-namer/N001/README.md` and `N001b/README.md`.

Paths abbreviate: `namer/` is `app/spectral_match/namer/`, and `components/` is `app/foundation_models/components/`.

Analogy

A wine-tasting course that cannot find enough labelled wines. The teacher pours ordinary table wine, adds a measured splash of a reference wine from the cellar, and asks two questions: "was anything added that does not grow in a vineyard?" and "which grape?". The splash size varies, the glasses differ a little, the room is noisy. The student is marked right for naming the *grape*, not for recognising the exact bottle, and the bottle used for the splash is never one of the answers on the card. The student answers both questions the same way: by asking which bottles on the shelf the glass tastes most like. The table wine is a real pixel, the splash a library spectrum, the grape the material class, and the shelf the library.

44.1 What the namer answers

The namer runs *after* detection. Something else (Chapters 30 and 32) has already flagged a pixel as anomalous; the namer says what it might be. For each pixel it returns four numbers (`namer/infer.py`, `NamerResult`):

1. $P(M)$, the probability that something man-made is in the pixel: *level 1*;
2. the most likely man-made class $\hat{k}$ among seven;
3. $P(\hat{k} \mid M)$, that class's probability *if* the pixel is man-made: *level 2*;
4. $\hat{a}$, an estimate of the *fill*, the share of the pixel the material covers.

The proposal combines the two levels as $P(k) = P(M) \times P(k \mid M)$. The seven classes are materials, not uses (`namer/classes.py`, `MANMADE_CLASSES`): oil/fuel, plastic, textile/fabric, wood/paper, paint/coatings, metal, concrete/asphalt.

Definition 44.1 (Fill)**.** The *fill* a of a pixel is the fraction of its area covered by the target material, between 0 and 1. A 10 m boat in a 30 m pixel has $a \approx (10 \times 10)/(30 \times 30) = 100/900 = 0.11$ [A].

44.2 The architecture: a Siamese, two-tower network

Definition 44.2 (Encoder, embedding, tower, Siamese network)**.** An *encoder* is a network that turns an input into a short vector, its *embedding*. A *tower* is one encoder in a model that has several, drawn side by side. A *Siamese network* [3] runs two inputs through the *same* weights and compares the outputs. A *two-tower* (or dual-encoder) model has two towers whose outputs are compared, usually by a cosine; the towers may share some weights and not others.

The namer is both, in a precise sense. Its first stage, the *stem*, is Siamese: one set of 17,825 weights runs on the pixel $\mathbf{x}$, on its ring mean $\mathbf{r}$ and on every library spectrum $\mathbf{t}_j$. After the stem the two sides part into towers with different weights. The namer's own docstring (`namer/model.py`) says:

```
pixel side:   x, r (ring mean) -> stem -> [z_x, z_r, z_x - z_r] -> MLP -> h
              h -> embedding (unit, 128) | fill in [0, 1] | (level-1 linear head,
    unused with level1="matching")
library side: t -> stem -> MLP -> embedding (unit, 128)
class score:  logsumexp over the class's library embeddings of  cos(z, e_j) / tau
level 1:      "matching" (N001b on) - natural entries as one more class, level1_logit
    ; or the linear head (N001)
```

The linear head is the one N001 used; N001b computes level 1 by matching (Section 44.7), and the head is still built but no longer read.

44.2.1 The pixel tower

The pixel tower encodes the pixel $\mathbf{x}$ and its ring mean $\mathbf{r}$ separately through the stem, getting $\mathbf{z}_x$ and $\mathbf{z}_r$ of 64 numbers each, and concatenates

$$\mathbf{u} = [\mathbf{z}_x,\ \mathbf{z}_r,\ \mathbf{z}_x - \mathbf{z}_r] \in \mathbb{R}^{192}. \tag{44.1}$$

This is Section 32.3's "subtract the ring", done on features rather than on reflectance; the network also keeps $\mathbf{z}_x$ and $\mathbf{z}_r$ themselves, so it can learn when the subtraction helps and when it does not. A three-layer perceptron (`_mlp`, Linear–GELU–Linear–GELU–Linear, Chapter 16 defined GELU) maps $\mathbf{u}$ to $\mathbf{h} \in \mathbb{R}^{256}$. Three linear *heads* read $\mathbf{h}$:

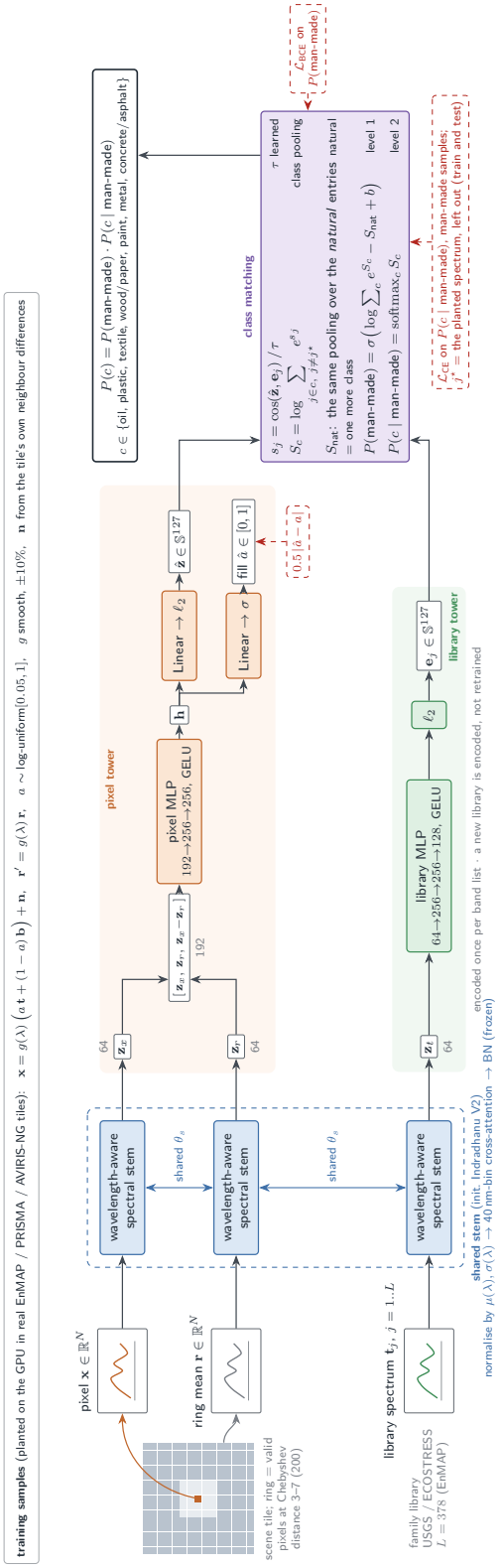


Figure 44.1: The publication schematic of the namer, from `research/experiments/spectral-namer/figures/namer_network.pdf`, redrawn on 31 October 2026 for level 1 by matching: the pixel tower has two heads (embedding and fill), and the class-matching box pools the natural library entries as one more class, $P(\text{man-made}) = \sigma(\text{LSE}_{c<7}\, S_c - S_{\text{natural}} + b)$ (Section 44.7, Figure 44.2). The old Linear level-1 head is still built in the model but not used, so it is not drawn.

- `to_embed`: $256 \to 128$, then divided by its length, so the pixel embedding $\mathbf{z}$ is a unit vector;
- `to_fill`: $256 \to 1$, then a sigmoid, so $\hat{a} \in (0, 1)$;
- `to_manmade`: $256 \to 1$, a raw score (a *logit*). This was level 1 in N001; under `"level1": "matching"` its output is overwritten before any loss sees it.

44.2.2 The library tower

The library tower runs each library spectrum $\mathbf{t}_j$ through the same stem, a second perceptron $64 \to 256 \to 256 \to 128$ (`library`), and divides by the length: a unit vector $\mathbf{e}_j \in \mathbb{R}^{128}$.

44.2.3 Where the towers meet

Both embeddings have length 1, so their dot product is their cosine. With a learned temperature τ:

$$s_j = \frac{\mathbf{z} \cdot \mathbf{e}_j}{\tau} = \frac{\cos(\mathbf{z}, \mathbf{e}_j)}{\tau}. \tag{44.2}$$

For a batch of P pixels and L library entries, all the s_j form one matrix $\mathbf{S} = \mathbf{Z}\mathbf{E}^\top/\tau$ of shape `(P, L)`, the line `s=z@lib_embed.T/self.log_tau.exp()` of `class_scores`. Everything the namer says about materials is read from this matrix.

44.2.4 Why the towers are not the same

A Siamese network often uses one tower for both inputs. The namer does not, after the stem, because the two inputs are different kinds of object:

- **The pixel is a mixture in context.** It is $a\mathbf{t}+(1-a)\mathbf{b}$ plus noise and an atmospheric error, with a often below 0.25. What identifies the target is the *difference* from the surroundings, so the pixel tower takes the ring too, and it must also estimate how much was added (the fill head).
- **The library spectrum is pure and alone.** It is a laboratory measurement of one material, $a = 1$, no background, no ring, no noise worth modelling. The library tower has no ring input and no fill head.

The pixel tower's job is to *undo* the mixing: to map a 15%-plastic pixel over grass near the point where the library tower maps pure plastic. The library tower only has to place pure spectra so that materials group together. Forcing one set of weights to do both would make the library side pretend to un-mix something that was never mixed. **The asymmetry is deliberate: the pixel tower learns to un-mix, the library tower only to arrange.**

44.2.5 Why a new library needs no retraining

Nothing in the model is tied to a library entry. The library tower is a function: spectrum in, embedding out. To add a spectrum, or a whole class, you encode it and append the row to $\mathbf{E}$; to add a class you also give its rows a new class index. Inference does exactly this every time: `name_pixels` builds the bank at the scene's own bands and calls `forward_library` on all of it (Section 44.12). A closed classifier, by contrast, has one output per class; a new class means a new output and new training. The proposal's decision record says this is why the namer matches rather than classifies ("a matching (not closed-classifier) model so a new library needs no retraining").

Worked example 44.1: The cost of a new library

A new library of 500 spectra at EnMAP's bands, encoded on the laptop. The library tower is one stem pass over `(500, N, 1, 1)` and one perceptron over `(500, 64)`. Its perceptron multiplies

$$64 \times 256 + 256 \times 256 + 256 \times 128 = 16{,}384 + 65{,}536 + 32{,}768 = 114{,}688$$

numbers per spectrum, so $500 \times 114{,}688 = 57{,}344{,}000$ multiply-adds: a fraction of a second on a CPU. Training the namer is 20,000 steps of tens of thousands of samples. The difference is the reason for the design: a new library costs one forward pass, not a training run [C].

But note what "no retraining" does not promise. The library tower was trained on the family library's spectra; a new spectrum that is unlike all of them may land somewhere unhelpful. Whether a new class is *named well* without training is exactly what the open-set run N002 measures (Section 44.10).

44.2.6 Counting the parameters

Worked example 44.2: Counting the namer's parameters

A linear layer from i to o numbers has $i \times o$ weights and o biases. The pixel perceptron is $192 \to 256 \to 256 \to 256$, the library perceptron $64 \to 256 \to 256 \to 128$:

$$\begin{aligned}
\text{pixel MLP} &= (192 \times 256 + 256) + (256 \times 256 + 256) + (256 \times 256 + 256) \\
&= 49{,}408 + 65{,}792 + 65{,}792 = 180{,}992, \\
\text{library MLP} &= (64 \times 256 + 256) + (256 \times 256 + 256) + (256 \times 128 + 128) \\
&= 16{,}640 + 65{,}792 + 32{,}896 = 115{,}328, \\
\text{heads} &= \underbrace{(256 \times 128 + 128)}_{\text{embedding}} + \underbrace{2 \times (256 + 1)}_{\text{manmade, fill}} = 32{,}896 + 514 = 33{,}410, \\
\text{BatchNorm} &= \underbrace{64}_{\text{scale}} + \underbrace{64}_{\text{shift}} = 128, \\
\text{scalars} &= \underbrace{1}_{\ln \tau} + \underbrace{1}_{\text{level-1 bias } b} = 2, \\
\text{stem} &= 17{,}825 \quad \text{(counted by hand in Section 44.3.9)}, \\
\text{total} &= 180{,}992 + 115{,}328 + 33{,}410 + 128 + 2 + 17{,}825 = 347{,}685.
\end{aligned}$$

Instantiating `SpectralNamer` with N001b's settings gives 347,685 [C]. The stem, the only part taken from Indradhanu, is 17,825/347,685 = 5.1% of it. The pixel tower (perceptron and three heads) is 180,992 + 33,410 = 214,402, the library tower 115,328.

Source check: 347,685, not 347,684; and not 1–3 M

The first version of this chapter, and the course, counted 347,684. That was right for the code before level 1 by matching: `l1_bias`, one number, was added on 2026-10-31 with `level1_logit`. The proposal's "Size: small (~1-3 M parameters)" is three to nine times the model in the code. Of the 347,685, the 257 of `to_manmade` receive no gradient under matching (Section 44.7).

44.3 The stem in full, at the namer's shapes

Section 24.4 built the reduced stem at $D = 32$ for SpectralEarth. The namer runs it at $D = 64$, Indradhanu Heavy's width, copied from the V2 checkpoint FTH3 epoch 5 by `from_spectral_stem` with the settings `compressed_channels` 64, `token_dim` 32, `attention_heads` 4, `bin_nm` 40, `band_weighting` `"width"` (FTH3's own config has the same five values [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/config.json`]). This section walks it once more, at the shapes it really runs in one namer training step: $P = 8192$ pixels (or $L = 378$ library spectra) and $N = 200$ bands [A].

The namer calls it through `SpectralNamer.encode`:

```
x = spectra[:, :, None, None]                    # (P, N) -> (P, N, 1, 1): P images
    of one pixel
z = self.stats.normalize(x, wavelengths_nm)      # per-wavelength z-score
...
return self.norm(self.stem(z, wavelengths_nm, bv)[:, :, 0, 0])   # (P, 64)
```

The stem works along wavelength inside each pixel, so a batch of P spectra is passed as P images of size 1×1. Nothing spatial happens.

44.3.1 Step 0: normalisation by wavelength

`WavelengthStats` (`components/pixel_normalization.py`) holds a mean curve $\mu(\lambda)$ and a spread curve $\sigma(\lambda)$ over wavelength, measured once on SpectralEarth's training split and stored in the checkpoint ("never recomputed"). A band at wavelength λ between two stored wavelengths $\lambda_{lo} < \lambda < \lambda_{hi}$ reads the curves by linear interpolation:

$$t = \frac{\lambda - \lambda_{lo}}{\lambda_{hi} - \lambda_{lo}}, \qquad \mu(\lambda) = \mu_{lo} + t\,(\mu_{hi} - \mu_{lo}), \qquad \sigma(\lambda) = \sigma_{lo} + t\,(\sigma_{hi} - \sigma_{lo}), \tag{44.3}$$

$$v = \frac{x - \mu(\lambda)}{\sigma(\lambda)}. \tag{44.4}$$

Outside the stored range the end values are held.

Worked example 44.3: Normalising one band

Stored: at 860 nm mean 0.30, spread 0.10; at 870 nm mean 0.32, spread 0.12 [A]. A band at 864 nm reads 0.35:

$$\begin{aligned} t &= (864 - 860)/(870 - 860) = 4/10 = 0.4, \\ \mu &= 0.30 + 0.4 \times (0.32 - 0.30) = 0.30 + 0.008 = 0.308, \\ \sigma &= 0.10 + 0.4 \times (0.12 - 0.10) = 0.10 + 0.008 = 0.108, \\ v &= (0.35 - 0.308)/0.108 = 0.042/0.108 = 0.389. \end{aligned}$$

This keeps *absolute* reflectance meaningful: a bright and a dark pixel stay different, unlike a per-scene min-max scaling. The proposal names this as a requirement ("it keeps absolute reflectance"). Invalid bands are set to 0 after normalisation (`masked_fill`) and are also excluded from every average below.

44.3.2 Step 1: the wavelength encoding

Every band's wavelength becomes a vector of $d = 32$ numbers (`components/spectral_tokens.py`, `wavelength_encoding`): 16 periods spaced geometrically from 10 to 10,000 nm, a sine and

a cosine of each,

$$p_i = 10 \times 1000^{\,i/15}\ \text{nm}, \quad i = 0, \ldots, 15; \qquad \text{enc}(\lambda) = \Big[\sin\tfrac{2\pi\lambda}{p_0}, \ldots, \sin\tfrac{2\pi\lambda}{p_{15}},\ \cos\tfrac{2\pi\lambda}{p_0}, \ldots, \cos\tfrac{2\pi\lambda}{p_{15}}\Big]. \tag{44.5}$$

Successive periods differ by the factor $1000^{1/15} = 1.5849$: 10.0, 15.8, 25.1, 39.8, …, 6309.6, 10000.0 nm [C].

Worked example 44.4: Three entries of enc(1000 nm)

$$\begin{aligned} \sin(2\pi \times 1000/10) &= \sin(200\pi) = 0, \\ \sin(2\pi \times 1000/15.85) &= \sin(396.4) = 0.5659, \\ \sin(2\pi \times 1000/10000) &= \sin(0.6283) = 0.5878, \qquad \cos(0.6283) = 0.8090. \end{aligned}$$

The code gives the same [C]. The slowest pair (period 10,000 nm) changes by only a few per cent between neighbouring bands, so it says roughly *where* in the spectrum a band sits; the fastest pair separates bands 5 nm apart. The encodings of 1000 and 1010 nm have cosine 0.6709 [C]: similar, not identical.

For the batch, enc of all N bands is one matrix of shape `(N, 32)` = `(200, 32)`, the same for every pixel; it is computed once per call.

44.3.3 Step 2: bins of 40 nm

The bins are a fixed grid from 400 nm, the same on every sensor (`BIN_ORIGIN_NM`, `BIN_END_NM` = 400, 2500):

$$G = \left\lceil \frac{2500 - 400}{40} \right\rceil = \lceil 52.5 \rceil = 53, \qquad g(\lambda) = \text{clamp}\left(\left\lfloor \frac{\lambda - 400}{40} \right\rfloor,\ 0,\ 52 \right). \tag{44.6}$$

So 405 and 439.9 nm fall in bin 0, 440 nm in bin 1, 1000 nm in bin $\lfloor 600/40 \rfloor = 15$, 2450 nm in bin 51; the last bin runs from 2480 to 2520 nm, past the grid's end, and a band below 400 nm is clamped into bin 0 [C]. Bin g has centre $420 + 40g$ nm. The membership is a one-hot matrix $\mathbf{O}$ of shape `(N, G)` = `(200, 53)`: $O_{bg} = 1$ if band b is in bin g.

On EnMAP's real 224-band list (418–2445 nm), 47 of the 53 bins hold at least one band, at most 9 in one bin; the empty ones are the two water-vapour gaps and the bins past 2445 nm [C].

44.3.4 Step 3: band widths

With `band_weighting="width"` each band counts by the nanometres of spectrum it stands for (`band_widths`): half the gap to each neighbour, the single gap at either end, every gap capped at `MAX_GAP_NM` = 20 nm.

Worked example 44.5: Widths of four bands, and of a band beside a water gap

Bands at 410, 430, 445, 470 nm. Gaps: 430 − 410 = 20, 445 − 430 = 15, 470 − 445 = 25, capped to 20, 15, 20.

$$\begin{aligned} w_{410} &= 20 && \text{one gap at the end} \\ w_{430} &= (20 + 15)/2 = 17.5, \qquad w_{445} = (15 + 20)/2 = 17.5 \\ w_{470} &= 20 && \text{one (capped) gap at the end} \end{aligned}$$

Bands at 1320, 1330, 1340, 1460, 1470 nm: the 120 nm water gap is capped to 20, so the widths are 10, 10, (10 + 20)/2 = 15, (20 + 10)/2 = 15, 10. Without the cap the 1340 nm

band would claim 65 nm of spectrum it never measured. The code gives both lists [C].

The widths, times validity, form the weight c_b of band b for each pixel, and the stem multiplies the values by it: `vals` $= c_b v_b$, shape `(P, N)`; `cnt_band` $= c_b$, shape `(P, N)`; and the total weight per bin `cnt` $= \mathbf{C}_{\text{band}}\mathbf{O}$, shape `(P, N)`$\times$`(N, G)` $=$ `(P, 53)`.

44.3.5 Step 4: bin tokens, as one matrix product

Each band also gets its own weight vector $\mathbf{k}(\lambda_b) \in \mathbb{R}^{32}$, from a small network of its encoding (`band_weight`, $32 \to 64 \to 32$). Bin g's token is the weighted mean of its bands' $v_b\,\mathbf{k}(\lambda_b)$:

$$\tau_g = \frac{\sum_{b \in g} c_b\, v_b\, \mathbf{k}(\lambda_b)}{\sum_{b \in g} c_b}. \tag{44.7}$$

Written as a loop this would build a `(P, N, 32)` tensor, 52.4 million numbers at $P = 8192$, $N = 200$. The code avoids it by folding the bin membership into the weights. Build

$$\mathbf{K} \in \mathbb{R}^{N \times (G \cdot d)}, \qquad K_{b,\,(g,i)} = O_{bg}\, k_i(\lambda_b), \tag{44.8}$$

row b holding $\mathbf{k}(\lambda_b)$ in the 32 columns of its own bin and zeros elsewhere (`kmat=(onehot[:,:,None]*self.band_weight(enc)[:,None,:]).reshape(n,G*self.d)`). Then one product `(P, N)` @ `(N, 53·32)` $=$ `(P, 1696)`, reshaped to `(P, 53, 32)` and divided by `cnt`, gives every token of every pixel. $\mathbf{K}$ has $200 \times 1696 = 339{,}200$ entries; the tokens $8192 \times 53 \times 32 = 13{,}893{,}632$ [C].

Worked example 44.6: Tokens by the matrix product, on paper

Take the four bands of worked example 44.5 with $d = 2$ instead of 32 and two bins (410, 430 in bin 0; 445, 470 in bin 1). Normalised values $v = (0.5, -0.2, 1.0, 0.4)$ and band vectors $\mathbf{k} = (1,0)$, $(0.5, 0.5)$, $(0,1)$, $(1,-1)$ [A]. Widths $c = (20, 17.5, 17.5, 20)$.

$$\begin{aligned}
c\,v &= (20 \times 0.5,\ 17.5 \times (-0.2),\ 17.5 \times 1.0,\ 20 \times 0.4) = (10,\ -3.5,\ 17.5,\ 8),\\
\text{cnt} &= (20 + 17.5,\ 17.5 + 20) = (37.5,\ 37.5),\\
\mathbf{K} &= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 0.5 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & -1 \end{pmatrix},\\
(c\,v)\,\mathbf{K} &= \big(10 \times 1 + (-3.5) \times 0.5,\quad (-3.5) \times 0.5,\\
&\qquad 8 \times 1,\quad 17.5 \times 1 + 8 \times (-1)\big)\\
&= (8.25,\ -1.75,\ 8.0,\ 9.5),\\
\tau_0 &= (8.25,\ -1.75)/37.5 = (0.2200,\ -0.0467),\\
\tau_1 &= (8.0,\ 9.5)/37.5 = (0.2133,\ 0.2533).
\end{aligned}$$

A direct loop over the bands of each bin gives the same two tokens [C].

Then every token gets its bin's position,

```
tok = tok + self.bin_bias(wavelength_encoding(centres))     # (P, 53, 32) + (53, 32)
```

a linear map $32 \to 32$ of the encoding of the bin centre, shape `(53, 32)`, added to every pixel's tokens.

44.3.6 Step 5: 64 queries attend over 53 bins, in 4 heads

The stem has 64 learned queries, a `(64, 32)` matrix, one per output channel. Attention (Chapter 14) lets each query read the bins it needs:

what	formula	shape
queries	$\mathbf{Q} = \mathrm{Lin}_q(\text{queries})$, all pixels	`(64, 32)` → 4 heads `(64, 8)`
keys, values	$\mathbf{K}_p = \mathrm{Lin}_k(\tau)$, $\mathbf{V}_p = \mathrm{Lin}_v(\tau)$	`(P, 53, 32)` → `(P, 4, 53, 8)`
weights	softmax $\left(\mathbf{Q}\mathbf{K}_p^\top/\sqrt{8}\right)$, masked	`(P, 4, 64, 53)`
answers	weights $\times\, \mathbf{V}_p$, heads joined	`(P, 64, 32)`

The mask is `cnt>0`: a bin with no valid band gets weight exactly 0 (Section 23.4). If a pixel has no valid band at all, the mask is dropped for it and its output is set to 0 at the end.

Worked example 44.7: One query, three bins, one empty

Head dimension 2 for paper: query $\mathbf{q} = (1, 0.5)$; keys $(1, 0)$, $(0, 1)$, $(2, 2)$; values $(0.2, 0.4)$, $(0.6, -0.2)$, $(9, 9)$; bin 3 has no valid band [A].

$$\begin{aligned}
\mathbf{q}\cdot\mathbf{k}/\sqrt{2} &= (1,\ 0.5,\ 3)/1.4142 = (0.7071,\ 0.3536,\ 2.1213),\\
\text{masked} &= (0.7071,\ 0.3536,\ -\infty),\\
e^{0.7071} &= 2.0281, \quad e^{0.3536} = 1.4242, \quad e^{-\infty} = 0, \quad \text{sum } 3.4523,\\
\text{weights} &= (2.0281/3.4523,\ 1.4242/3.4523,\ 0) = (0.5875,\ 0.4125,\ 0),\\
\text{answer} &= 0.5875\,(0.2, 0.4) + 0.4125\,(0.6, -0.2) = (0.3650,\ 0.1525).
\end{aligned}$$

The large value 9 in the empty bin cannot leak. PyTorch's `scaled_dot_product_attention` with the same boolean mask returns $(0.3650,\ 0.1525)$ [C].

If the attention weights were stored, they would be $8192 \times 4 \times 64 \times 53 = 111{,}149{,}056$ numbers, 424 MiB in float32, per stem call [C]. PyTorch's fused attention kernel avoids storing them, and the stem splits calls above 32,768 rows (`SDPA_MAX_ROWS`); at $P = 8192$ one call suffices.

44.3.7 Step 6: one number per channel, plus the skip path

The answers pass through Lin_o $(32 \to 32)$ and `to_channel` $(32 \to 1)$: `(P, 64, 32)` → `(P, 64, 1)` → `(P, 64)`. Channel c is query c's answer squeezed to one number.

Alongside, a per-band *skip path* lets every valid band reach the channels directly with weights computed from its wavelength (`skip_weight`, $32 \to 64 \to 64$):

$$\mathrm{skip}_c = \frac{\sum_b c_b\, v_b\, W_c(\lambda_b)}{\sum_b c_b}, \tag{44.9}$$

in code `(vals@self.skip_weight(enc))/cnt_band.sum(1)`: `(P, N)` @ `(N, 64)` = `(P, 64)`. This is the old 1×1 compressor with columns computed from wavelength; attention adds adaptivity on top, it does not carry all the detail alone (the stem's docstring, "fix 2"). The stem returns $\mathbf{z} = \text{attention} + \text{skip}$, `(P, 64)` (as `(P, 64, 1, 1)`).

44.3.8 Step 7: the frozen BatchNorm

Definition 44.3 (BatchNorm in evaluation mode)**.** A BatchNorm layer [17] with stored running mean m_c and variance s_c^2, learned scale γ_c and shift β_c, maps channel c of an input as $y_c = \gamma_c\,(z_c - m_c)/\sqrt{s_c^2 + \epsilon} + \beta_c$, $\epsilon = 10^{-5}$. In *training* mode it uses the current batch's mean and variance instead of m_c, s_c^2, and updates the stored ones.

Worked example 44.8: One channel through the frozen BatchNorm

$m = 0.4$, $s^2 = 0.25$, $\gamma = 1.5$, $\beta = -0.1$, input $z = 0.9$ [A]:

$$z - m = 0.9 - 0.4 = 0.5, \qquad \sqrt{0.25 + 0.00001} = 0.50001,$$
$$y = 1.5 \times 0.5/0.50001 - 0.1 = 1.49997 - 0.1 = 1.39997.$$

`BatchNorm1d` in evaluation mode gives 1.39997 [C].

`SpectralNamer.train()` calls the usual `train()` and then `self.norm.eval()`: the running statistics stay those of V2, estimated on real pixels of the three sensors, and normalise pixels, rings and library spectra alike, in training and evaluation; γ and β still train. Section 44.9 tells why.

44.3.9 The stem's parameters, by hand

piece	layers	parameters
`band_weight`, $\mathbf{k}(\lambda)$	$32 \to 64$, $64 \to 32$	$(32 \times 64 + 64) + (64 \times 32 + 32) = 2{,}112 + 2{,}080 = 4{,}192$
`skip_weight`, $W(\lambda)$	$32 \to 64$, $64 \to 64$	$(32 \times 64 + 64) + (64 \times 64 + 64) = 2{,}112 + 4{,}160 = 6{,}272$
`bin_bias`	$32 \to 32$	$32 \times 32 + 32 = 1{,}056$
`queries`	64×32	2,048
$\mathrm{Lin}_{q,k,v,o}$	$4 \times (32 \to 32)$	$4 \times 1{,}056 = 4{,}224$
`to_channel`	$32 \to 1$	33
total		$4{,}192 + 6{,}272 + 1{,}056 + 2{,}048 + 4{,}224 + 33 = 17{,}825$

Table 44.1: The reduced stem at $D = 64$, $d = 32$. Instantiation gives 17,825, tensor by tensor [C]; the Indradhanu course's part 71 derives the same total. No count depends on the number of bands N or on the sensor.

step	tensor	shape
input	spectra as images	`(8192, 200, 1, 1)`
0	normalised values v, masked	`(8192, 200, 1, 1)`
1	enc(λ)	`(200, 32)`
2	one-hot bins $\mathbf{O}$	`(200, 53)`
3	`vals`, `cnt_band`; `cnt`	`(8192, 200)`; `(8192, 53)`
4	$\mathbf{k}(\lambda)$; $\mathbf{K}$; tokens	`(200, 32)`; `(200, 1696)`; `(8192, 53, 32)`
4	bin bias	`(53, 32)`
5	$\mathbf{Q}$; $\mathbf{K}_p, \mathbf{V}_p$	`(1, 4, 64, 8)` expanded; `(8192, 4, 53, 8)`
5	weights (not stored); answers	`(8192, 4, 64, 53)`; `(8192, 64, 32)`
6	`to_channel`; skip; $\mathbf{z}$	`(8192, 64, 1)`; `(8192, 64)`; `(8192, 64)`
7	BatchNorm (frozen)	`(8192, 64)`

Table 44.2: The stem for the 8,192 pixels of one sensor's batch, $N = 200$ bands [A]. Forward hooks on the real modules record these shapes [C]; for the library replace 8192 by 378.

44.4 The ring mean

A pixel's surroundings are summarised by the mean of the valid pixels in a 15×15 box minus the central 5×5 box (`namer/generator.py`, `INNER,OUTER=5,15`). Box sums come for every pixel at once from an average pool times the box area:

```
s = box(v, OUTER) - box(v, INNER)      # sum of valid values in the ring, (B, N, H, W)
c = box(m, OUTER) - box(m, INNER)      # number of valid pixels in the ring
ring = s / c.clamp(min=1)
```

In the interior $c = 15^2 - 5^2 = 225 - 25 = 200$. The 5×5 box covers offsets -2 to 2, so it removes every pixel up to two steps away (a diagonal step counts as one: the *Chebyshev distance* $\max(|\Delta\text{row}|, |\Delta\text{col}|)$). The ring is the pixels at Chebyshev distance 3 to 7. Check: the square ring at distance d holds $(2d+1)^2 - (2d-1)^2 = 8d$ pixels, and $8 \times (3+4+5+6+7) = 8 \times 25 = 200$. The two-pixel gap keeps a target's halo (Chapter 34) out of its own background.

Worked example 44.9: A ring that ignores the halo

A flat tile of 0.20; the target pixel 0.50; its eight neighbours 0.35 [A]. The 15×15 box holds the target, its 8 neighbours and $225 - 9 = 216$ background pixels; the 5×5 box holds the same 9 and $25 - 9 = 16$ background pixels:

$$S_{15} = 216 \times 0.20 + 8 \times 0.35 + 0.50 = 43.2 + 2.8 + 0.5 = 46.5,$$
$$S_5 = 16 \times 0.20 + 8 \times 0.35 + 0.50 = 3.2 + 2.8 + 0.5 = 6.5,$$
$$\mathbf{r} = \frac{S_{15} - S_5}{200} = \frac{40.0}{200} = 0.200.$$

The target and its halo cancel exactly; the code gives 0.2000 with a count of 200 [C].

Samples are drawn only where every band is valid, the ring has at least 20 valid pixels and the pixel is at least 7 px from the tile's edge, so of a 128×128 tile at most $(128-14)^2 = 12{,}996$ pixels qualify [C]. Inference computes the same ring one band at a time with `scipy`'s `uniform_filter` (`namer/infer.py`, `ring_means`); a test pins the two to 10^{-5}.

Source check: 3 to 7 pixels, not 2 to 7

The proposal's diagram says "r (ring mean 2-7 px)". The code's ring is Chebyshev distance 3 to 7, as the generator's docstring and, now, `infer.py`'s docstring say ("3-7 px"); an earlier version of `infer.py` said "2-7 px".

44.5 The generator

Almost no real pixel comes with a known material, so the namer trains on samples it makes from real tiles of the three sensors' fine-tuning shards. For a sampled pixel $\mathbf{b}$ with ring mean $\mathbf{r}$ (`namer/generator.py`, `plant`):

$$\mathbf{x} = \mathbf{g} \odot (a\,\mathbf{t} + (1-a)\,\mathbf{b}) + \mathbf{n}, \qquad \mathbf{r}' = \mathbf{g} \odot \mathbf{r}, \tag{44.10}$$

where $\odot$ multiplies band by band. The kind of sample decides $\mathbf{t}$ and a:

kind	share	$\mathbf{t}$	level-1 truth
0, man-made	0.50	a man-made library spectrum of the training split	man-made
1, natural mix	0.25	a natural library spectrum (either split)	natural
2, plain	0.25	none, $a = 0$	natural

(`kinds` = 0.5, 0.25, 0.25 [M: `configs/namer/N001b.json`].) The kind is drawn per sample with `torch.multinomial`, so the counts vary: one draw of 8,192 gave 4,110, 1,984 and 2,098 [C]. Each sample records its library entry index (`entry`, -1 for plain), its class (`cls`, -1 unless man-made) and its fill.

Picking pixels. For each tile the generator draws 512 positions (`pixels_per_tile`) uniformly, *with replacement*, among the allowed pixels of Section 44.4; with 16 tiles that is $P = 16 \times 512 = 8192$ samples per sensor per step. It gathers **b** and **r** at those positions, shape `(8192, N)` each, and the tile's noise level σ, one per band, also `(8192, N)`.

The fill, log-uniform. With u uniform on $[0, 1]$, the code sets $a = \exp\big(\ln a_{\min}\,(1-u)\big) = a_{\min}^{1-u}$ with $a_{\min} = 0.05$: $u = 0$ gives $a_{\min}$, $u = 1$ gives 1. Its distribution, one step per line; remember that $\ln a_{\min} < 0$:

$$
\begin{aligned}
a \le q &\iff (1-u)\ln a_{\min} \le \ln q && \text{take logs}\\
&\iff 1-u \;\ge\; \frac{\ln q}{\ln a_{\min}} && \text{divide by } \ln a_{\min} < 0\text{: it flips}\\
&\iff u \;\le\; 1 - \frac{\ln q}{\ln a_{\min}} && \text{rearrange}\\
P(a \le q) &= 1 - \frac{\ln q}{\ln a_{\min}} && P(u \le v) = v,\ u \text{ uniform}\\
&= \frac{\ln a_{\min} - \ln q}{\ln a_{\min}} && \text{common denominator}\\
&= \frac{\ln q - \ln a_{\min}}{-\ln a_{\min}} && \text{times } \tfrac{-1}{-1}
\end{aligned}
$$

which, with $\ln q - \ln a_{\min} = \ln(q/a_{\min})$ and $-\ln a_{\min} = \ln(1/a_{\min})$, is

$$P(a \le q) = \frac{\ln(q/a_{\min})}{\ln(1/a_{\min})}. \tag{44.11}$$

A shorter way to see it: $\ln a = (1-u)\ln a_{\min}$ moves linearly from $\ln a_{\min}$ to 0 as u goes from 0 to 1, so $\ln a$ is uniform on $[\ln a_{\min}, 0]$, and the share below $\ln q$ is that interval's length fraction. That is why the distribution is called *log-uniform*.

Its density and mean. The density is the slope of Equation (44.11) in q. The slope of $\ln q$ is $1/q$, so

$$p(a) = \frac{1}{a\,\ln(1/a_{\min})} \quad \text{for } a_{\min} \le a \le 1. \tag{44.12}$$

The mean, one step per line:

$$
\begin{aligned}
\mathbb{E}[a] &= \int_{a_{\min}}^{1} a\,p(a)\,da && \text{definition of a mean}\\
&= \int_{a_{\min}}^{1} a \cdot \frac{1}{a\ln(1/a_{\min})}\,da && \text{insert the density}\\
&= \frac{1}{\ln(1/a_{\min})}\int_{a_{\min}}^{1} 1\,da && \text{the } a \text{ cancels}\\
&= \frac{1-a_{\min}}{\ln(1/a_{\min})} && \text{the integral of 1 is the interval's length.}
\end{aligned}
$$

Worked example 44.10: How much training goes to small fills

With $a_{\min} = 0.05$, $\ln(1/0.05) = \ln 20 = 2.9957$. By Equation (44.11):

$$P(a \le 0.1) = \frac{\ln(0.1/0.05)}{2.9957} = \frac{\ln 2}{2.9957} = \frac{0.6931}{2.9957} = 0.2314,$$
$$P(a \le 0.25) = \frac{\ln 5}{2.9957} = \frac{1.6094}{2.9957} = 0.5372,$$
$$P(a \le 0.5) = \frac{\ln 10}{2.9957} = \frac{2.3026}{2.9957} = 0.7686.$$

The mean: $\mathbb{E}[a] = (1 - 0.05)/2.9957 = 0.95/2.9957 = 0.3171$; the median is where $P = \frac{1}{2}$, $u = \frac{1}{2}$, so $a = 0.05^{0.5} = 0.2236$. A million samples drawn with the code's formula put 0.5368 of them at or below 0.25, median 0.2241 [C]. A uniform fill on $[0.05, 1]$ would put only $(0.25 - 0.05)/0.95 = 0.21$ there and have mean 0.525. Table 32.1 showed small fills are where ACE fails; the log-uniform spends more than half of the man-made training there.

The fill *target* of the fill head is a for kinds 0 and 1 and 0 for plain pixels, so over all samples its mean is $0.75 \times 0.3171 = 0.2378$ [C].

The gain, smooth. Three coefficients $c_0, c_1, c_2 \sim \mathcal{N}(0, 0.5^2)$ make a parabola over the band index rescaled to $t \in [-1, 1]$, clipped to $[-1, 1]$; then $g = 1 + 0.1 \times$ curve, so $0.9 \le g \le 1.1$ (`smooth_gain`, shape `(P, N)`). It stands for an atmospheric-correction error, and the same **g** multiplies the ring mean, because the atmosphere errs alike over a pixel and its neighbours. With $c = (0.2, -0.4, 0.6)$ [A]: at $t = -1$ the curve is $0.2 + 0.4 + 0.6 = 1.2$, clipped to 1, $g = 1.10$; at $t = 0$, $g = 1.02$; at $t = 1$, $0.2 - 0.4 + 0.6 = 0.4$, $g = 1.04$.

The noise, from the tile. Per tile and band, σ is the root mean square of the differences between horizontal neighbours, divided by $\sqrt{2}$ (`ring_and_noise`, shape `(B, N)`). The reason, using one rule: *the variance of a difference of independent variables is the sum of their variances.* Check it on two coins: $X, Y \in \{0, 1\}$ each with probability $\frac{1}{2}$ have variance $\frac{1}{4}$; $X - Y$ takes $-1, 0, 0, 1$ with equal chance, mean 0, variance $\frac{1}{4}(1 + 0 + 0 + 1) = \frac{1}{2} = \frac{1}{4} + \frac{1}{4}$. So if neighbours carry independent noise of variance σ^2,

$$\begin{aligned} \mathrm{Var}(n_1 - n_2) &= \sigma^2 + \sigma^2 = 2\sigma^2 && \text{the rule} \\ \mathrm{sd}(n_1 - n_2) &= \sigma\sqrt{2} && \text{square root} \\ \sigma &= \mathrm{sd}(n_1 - n_2)/\sqrt{2} && \text{divide by } \sqrt{2}. \end{aligned}$$

Worked example 44.11: Noise from four pixels

One band of a row reads $0.20, 0.23, 0.21, 0.22$ [A]:

$$\begin{aligned} \text{differences} &= 0.03,\ -0.02,\ 0.01 \\ \text{squares} &= 0.0009,\ 0.0004,\ 0.0001, && \text{sum } 0.0014 \\ \text{mean square} &= 0.0014/3 = 0.000467 \\ \text{RMS} &= \sqrt{0.000467} = 0.0216 \\ \sigma &= 0.0216/1.4142 = 0.01528. \end{aligned}$$

The code gives 0.01528; on 200,000 samples of white noise with $\sigma = 0.01$ it estimates 0.01003 [C].

Source check: an RMS, not a standard deviation

The differences' mean is not subtracted before squaring ("RMS (uncentred) … texture counts too", the docstring), so real texture, an edge or a slope, counts as noise. On textured tiles σ is an over-estimate, which errs on the hard side.

Worked example 44.12: Planting one pixel

Three bands, $\mathbf{b} = (0.10, 0.30, 0.25)$, $\mathbf{t} = (0.40, 0.35, 0.05)$, $a = 0.25$, $\mathbf{g} = (1.02, 1.00, 0.98)$, $\mathbf{n} = (0.003, -0.002, 0.001)$ [A]. By Equation (44.10):

$$
\begin{aligned}
a\mathbf{t} &= (0.1000,\ 0.0875,\ 0.0125),\\
(1-a)\mathbf{b} &= (0.0750,\ 0.2250,\ 0.1875),\\
\text{sum} &= (0.1750,\ 0.3125,\ 0.2000),\\
\mathbf{g} \odot \text{sum} &= (1.02 \times 0.1750,\ 1.00 \times 0.3125,\ 0.98 \times 0.2000) = (0.1785,\ 0.3125,\ 0.1960),\\
\mathbf{x} &= (0.1785 + 0.003,\ 0.3125 - 0.002,\ 0.1960 + 0.001) = (0.1815,\ 0.3105,\ 0.1970).
\end{aligned}
$$

What is *not* modelled matters as much: the planting is one pixel, with no point-spread function, no halo and no adjacency, and the ring is untouched by the target. The proposal's "Risks" list says so ("linear mixing, smooth gain and Gaussian noise are simplifications"). Per batch only bands valid in more than half the tiles' pixels are kept (`keep=valid.float().mean(dim=(0,2,3))>0.5`), so water-vapour bands drop out, and the library is cut to the same bands (`bank.bands(keep)`); a batch with fewer than 20 bands left is skipped.

Source check: no band-list jitter

The proposal lists "Band-list jitter as in pretraining (`band_augment_prob`)". Neither the generator nor the trainer reads such a setting; every sample of a batch has the batch's live bands.

44.6 The class score

Definition 44.4 (Logsumexp, class score)**.** For numbers $z_1, \dots, z_m$, $\mathrm{LSE}(z) = \ln \sum_j e^{z_j}$, a smooth maximum: it lies between $\max_j z_j$ and $\max_j z_j + \ln m$. With s_j from Equation (44.2), the *class score* of class k is $S_k = \mathrm{LSE}(\{s_j : j \in k\})$, and $P(k \mid M)$ is the softmax of $S_1, \dots, S_7$ (`class_scores`).

The temperature τ is learned, stored as $\ln \tau$ and started at $\ln 0.07 = -2.6593$, so $1/\tau = 14.286$ at the start: a cosine of 0.9 against 0.7 becomes a score of 12.86 against 10.00.

The class softmax is a sum of entry softmaxes. Write $q_j = e^{s_j} / \sum_i e^{s_i}$ for the softmax over all library entries that belong to some class. Then

$$
\begin{aligned}
P(k \mid M) &= \frac{e^{S_k}}{\sum_{k'} e^{S_{k'}}} && \text{definition}\\
&= \frac{\sum_{j \in k} e^{s_j}}{\sum_{k'} \sum_{j \in k'} e^{s_j}} && e^{\ln u} = u\\
&= \frac{\sum_{j \in k} e^{s_j}}{\sum_i e^{s_i}} && \text{every entry is in exactly one class}\\
&= \sum_{j \in k} q_j .
\end{aligned}
$$

So the loss treats *any* member of the right class as a right answer. That is the remedy the companion volume proposed for "false negatives" in a per-spectrum contrastive loss, two spectra of one material pushed apart; here it is built in.

Leaving the planted spectrum out. In `forward`, `class_scores(...,exclude=s["entry"])` sets the planted entry's s_j to $-\infty$ before the logsumexp, in training and in evaluation alike. Every planted sample (man-made and natural mix) has its own entry excluded; plain pixels have none. A test checks that a class whose only member is excluded scores $-\infty$.

Worked example 44.13: Recognising the sample, or naming the material

Take $\tau = 0.1$ for round numbers [A]. The pixel was planted with metal spectrum e_1; cosines: metal e_1 0.95, e_2 0.70, e_3 0.60; plastic e_4 0.75; oil e_5 0.20. Divided by τ: 9.5, 7.0, 6.0, 7.5, 2.0.
With e_1 in the library:

$$S_{\text{metal}} = \ln(e^{9.5} + e^{7} + e^{6}) = 9.6064, \quad S_{\text{plastic}} = 7.5, \quad S_{\text{oil}} = 2.0,$$
$$P = \text{softmax}(9.6064,\ 7.5,\ 2.0) = (0.8911,\ 0.1084,\ 0.0004),$$
$$L = -\ln 0.8911 = 0.1153.$$

With e_1 left out:

$$S_{\text{metal}} = \ln(e^{7} + e^{6}) = 7 + \ln(1 + e^{-1}) = 7 + \ln 1.3679 = 7.3133,$$
$$P = (0.4524,\ 0.5453,\ 0.0022), \qquad L = -\ln 0.4524 = 0.7931.$$

Check the sum-of-entries identity: over $e_2, \ldots, e_5$ the entry softmax is $(0.3308, 0.1217, 0.5453, 0.0022)$, and $0.3308 + 0.1217 = 0.4525$, the metal probability up to rounding [C].

With its own spectrum present the pixel is "metal" at 0.89 because it recognises e_1. With e_1 out it must lean on the other metals, and here plastic wins. **Leaving the planted spectrum out makes the only route to a low loss putting all metals near each other: learning the material, not the sample.** It is the separability check's leave-one-spectrum-out (`ace[e]=-1`) used as a training rule.

The gradient. Let c be the true class. The loss is $L = -S_c + \ln \sum_k e^{S_k} = -\ln \sum_{j \in c} e^{s_j} + \ln \sum_i e^{s_i}$. Differentiate with respect to one entry's s_j:

$$\frac{\partial}{\partial s_j} \ln \sum_i e^{s_i} = \frac{e^{s_j}}{\sum_i e^{s_i}} = q_j \qquad \text{chain rule}$$

$$\frac{\partial}{\partial s_j}\Big(-\ln \sum_{i \in c} e^{s_i} \Big) = -\,\mathbf{1}[j \in c]\, w_j, \quad w_j = \frac{e^{s_j}}{\sum_{i \in c} e^{s_i}} \qquad \text{the same rule, inside class } c$$

$$\frac{\partial L}{\partial s_j} = q_j - \mathbf{1}[j \in c]\, w_j.$$

Wrong-class entries are pushed down by their probability q_j, as in the per-spectrum loss. The true class's members are pulled up, but shared out by w_j: the member already closest gets most of the pull.

Worked example 44.14: Who gets the pull

In worked example 44.13 with e_1 out: within metal, $w = (e^7, e^6)/(e^7+e^6) = (0.7311,\ 0.2689)$. With $q = (0.3308, 0.1217, 0.5453, 0.0022)$ for e_2, e_3, e_4, e_5:

$$e_2 : 0.3308 - 0.7311 = -0.4003, \qquad e_3 : 0.1217 - 0.2689 = -0.1473,$$
$$e_4 : 0.5453, \qquad e_5 : 0.0022.$$

They sum to $-0.4003 - 0.1473 + 0.5453 + 0.0022 = -0.0001 \approx 0$, as they must [C]. Plastic e_4, the tempting wrong answer, is pushed hardest; metal's pull is shared 73 : 27.

Which entries take part in level 2? In training, only *training-split man-made* entries (`train_lib`, minus an open-set class). Test-split entries are still encoded (the whole bank goes through the library tower) but get class index -1, which no class selects.

Source check: class-level, not per-spectrum InfoNCE

The proposal's "Losses" (InfoNCE, 43) says "InfoNCE over the batch's library spectra. The positive is the planted spectrum". The code does what its "As built" list says instead: cross-entropy over class scores, own spectrum left out. Read the proposal's loss section as superseded.

44.7 Level 1 by matching

44.7.1 What N001 did, and what it measured

In N001, level 1 was the linear head `to_manmade`: one weight vector $\mathbf{w} \in \mathbb{R}^{256}$ and a bias on the pixel tower's $\mathbf{h}$, $\ell = \mathbf{w}\cdot\mathbf{h} + b_0$, $P(M) = \sigma(\ell)$, trained by binary cross-entropy. Two measurements, both from 2026-10-31 [M: `research/experiments/spectral-namer/N001/README.md`]:

- **N001, epoch 1, on the H100.** Level-1 accuracy was about 0.50 at every evaluation fill on all three sensors: on EnMAP 0.52 at fill 0.25 and 0.50 at fill 1.0. In the same epoch level-2 class accuracy was 0.60 to 0.68.
- **A laptop CPU test, 150 steps at full fill on the EnMAP smoke shards**, the two level-1 forms side by side:

level 1	BCE	accuracy	man-made recall at ACE's false-alarm rate
chance	$\ln 2 = 0.6931$	0.50	–
linear head on $\mathbf{h}$	0.64	0.65	0.41
by matching	0.43	0.80	0.74

Table 44.3: The laptop test that decided the switch, as recorded in N001's README [M: `research/experiments/spectral-namer/N001/README.md`]. ACE at fill 1.0 on EnMAP calls 13.36% of natural plantings man-made and recalls 0.8711 of man-made ones [C], so after 150 steps neither form is yet at ACE's level; the comparison is between the two forms.

Read the epoch-1 numbers together. At fill 1.0 a planted man-made pixel *is* a library spectrum (with gain and noise), and level 2 named its class correctly about two times in three: the embedding $\mathbf{z}$, computed from the same $\mathbf{h}$, plainly carried the information. The linear head on $\mathbf{h}$ could not read "man-made" out of it. **The information was there; one linear layer was the wrong way to read it.**

To compare the two BCEs, turn each into a probability: $e^{-\mathrm{BCE}}$ is the geometric mean of the

probability given to the truth. Chance: $e^{-0.6931} = 0.5000$; linear: $e^{-0.64} = 0.5273$; matching: $e^{-0.43} = 0.6505$ [C].

N001 was stopped. N001b is the same configuration plus `"level1":"matching"` (`configs/namer/N001b.json`, whose description says "N001's linear level-1 head sat at chance after epoch 1"), and N002 adds it too.

44.7.2 The rule

Under `"level1":"matching"` (`namer/train.py`, `forward`) the natural library entries become one more class, index $C = 7$:

```
lib_cls = torch.where(bank.manmade, lib_cls, torch.full_like(lib_cls, C))     #
    natural -> class 7
scores = self.model.class_scores(z.float(), e.float(), lib_cls, C + 1, exclude=s["
    entry"])   # (P, 8)
logit, scores = self.model.level1_logit(scores), scores[:, :C]
```

and `level1_logit` (`namer/model.py`) is

```
return torch.logsumexp(scores[:, :-1], dim=1) - scores[:, -1] + self.l1_bias
```

In symbols, with $S_0, \dots, S_6$ the man-made class scores, $S_7 = S_{\text{nat}}$ the natural one, and b the learned scalar `l1_bias` (started at 0):

$$\ell = \mathrm{LSE}(S_0, \dots, S_6) - S_{\text{nat}} + b, \qquad P(M) = \sigma(\ell) = \frac{1}{1 + e^{-\ell}}. \tag{44.13}$$

The level-2 scores are the first seven columns, exactly as before; the old head's output is discarded.

44.7.3 What the rule means

Two identities make Equation (44.13) readable.

A logsumexp of logsumexps is the logsumexp of everything. Since $S_c = \ln \sum_{j \in c} e^{s_j}$,

$$\begin{aligned} \mathrm{LSE}(S_0, \dots, S_6) &= \ln \sum_{c<7} e^{S_c} && \text{definition} \\ &= \ln \sum_{c<7} \sum_{j \in c} e^{s_j} && e^{\ln u} = u \\ &= \ln \sum_{j \in \mathcal{M}} e^{s_j} && \mathcal{M}\text{: all man-made entries in play.} \end{aligned}$$

Check on five numbers $(1, 2, 0.5, 3, -1)$ in groups $\{1,2\}, \{0.5\}, \{3,-1\}$: both ways give 3.4723 [C].

The sigmoid of a log-ratio is a share. Write $A = \sum_{j \in \mathcal{M}} e^{s_j}$ and $N = \sum_{j \in \mathcal{N}} e^{s_j}$ over the natural entries. Then $\ell = \ln A - \ln N + b$, and

$$\begin{aligned} e^{\ell} &= e^{b} A / N && \text{exponentiate} \\ \sigma(\ell) &= \frac{e^{\ell}}{e^{\ell} + 1} && \text{multiply top and bottom of } \tfrac{1}{1+e^{-\ell}} \text{ by } e^{\ell} \\ &= \frac{e^{b} A/N}{e^{b} A/N + 1} = \frac{e^{b} A}{e^{b} A + N} && \text{multiply top and bottom by } N. \end{aligned}$$

So

$$P(M) = \frac{e^b \sum_{j\in\mathcal{M}} e^{s_j}}{e^b \sum_{j\in\mathcal{M}} e^{s_j} + \sum_{j\in\mathcal{N}} e^{s_j}} : \tag{44.14}$$

$P(M)$ **is the share of the pixel's softmax mass over the library that falls on man-made entries**, with the man-made side weighted by e^b. Level 1 asks "does this pixel look more like the man-made shelf or the natural shelf?", in the same embedding level 2 uses.

44.7.4 Its gradient, through the logsumexp

The level-1 loss is $L_1 = \mathrm{BCE}(\ell, y) = -[y \ln \sigma(\ell) + (1-y)\ln(1-\sigma(\ell))]$ with $y = 1$ for man-made. Two pieces, then the chain rule.

The BCE's slope in the logit. With $p = \sigma(\ell)$ and $\sigma'(\ell) = p(1-p)$:

$$\begin{aligned} \frac{\partial L_1}{\partial \ell} &= -\Big[y\,\frac{p(1-p)}{p} - (1-y)\,\frac{p(1-p)}{1-p}\Big] && \text{slope of ln is } 1/\cdot \\ &= -[y(1-p) - (1-y)p] && \text{cancel} \\ &= -[y - yp - p + yp] = p - y && \text{expand.} \end{aligned}$$

The logit's slope in each entry score. For $j \in \mathcal{M}$, only $\ln A$ depends on s_j; for $j \in \mathcal{N}$, only $-\ln N$:

$$\begin{aligned} \frac{\partial \ell}{\partial s_j} &= \frac{e^{s_j}}{A} =: u_j && j \in \mathcal{M} \text{ (softmax within the man-made entries)} \\ \frac{\partial \ell}{\partial s_j} &= -\frac{e^{s_j}}{N} =: -v_j && j \in \mathcal{N} \text{ (softmax within the natural entries)} \\ \frac{\partial \ell}{\partial b} &= 1. \end{aligned}$$

Together, by the chain rule:

$$\frac{\partial L_1}{\partial s_j} = (p-y)\,u_j \quad (j \in \mathcal{M}), \qquad \frac{\partial L_1}{\partial s_j} = -(p-y)\,v_j \quad (j \in \mathcal{N}), \qquad \frac{\partial L_1}{\partial b} = p - y. \tag{44.15}$$

Since $\sum_{\mathcal{M}} u_j = 1$ and $\sum_{\mathcal{N}} v_j = 1$, the entry gradients sum to $(p-y) - (p-y) = 0$: like level 2's, a step moves score from one side to the other, it cannot raise everything. Finally $s_j = \mathbf{z}\cdot\mathbf{e}_j/\tau$, so $\partial s_j/\partial \mathbf{z} = \mathbf{e}_j/\tau$ and the pixel embedding moves along $\sum_j (\partial L_1/\partial s_j)\,\mathbf{e}_j/\tau$: *towards* the man-made entries it is already closest to and *away* from the nearest natural ones when it is man-made, and the reverse when it is natural.

Worked example 44.15: Level 1 by matching, three library entries

One pixel, truth man-made ($y = 1$). Three entries in play: plastic $s = 6.0$, metal $s = 5.0$ (both man-made, in different classes) and soil $s = 5.5$ (natural); $b = 0$ [A].
The logit:

$$\begin{aligned} S_{\text{plastic}} = 6.0, \quad S_{\text{metal}} = 5.0, \quad S_{\text{nat}} = 5.5 && \text{one member each} \\ \mathrm{LSE}(6.0, 5.0) = 6 + \ln(1 + e^{-1}) = 6 + \ln 1.3679 = 6 + 0.3133 &= 6.3133 \\ \ell = 6.3133 - 5.5 + 0 &= 0.8133. \end{aligned}$$

The probability, two ways:

$$p = \sigma(0.8133) = 1/(1 + e^{-0.8133}) = 1/(1 + 0.4434) = 0.6928,$$

$$\text{share} = \frac{e^6 + e^5}{e^6 + e^5 + e^{5.5}} = \frac{403.4288 + 148.4132}{403.4288 + 148.4132 + 244.6919} = \frac{551.8420}{796.5339} = 0.6928.$$

The loss: $L_1 = -\ln 0.6928 = 0.3670$. The gradient, by Equation (44.15), with $p - y = 0.6928 - 1 = -0.3072$:

$$\begin{aligned} u &= (e^6, e^5)/(e^6 + e^5) = (403.4288,\ 148.4132)/551.8420 = (0.7311,\ 0.2689), \\ \partial L_1/\partial s_{\text{plastic}} &= -0.3072 \times 0.7311 = -0.2246, \\ \partial L_1/\partial s_{\text{metal}} &= -0.3072 \times 0.2689 = -0.0826, \\ \partial L_1/\partial s_{\text{soil}} &= -(-0.3072) \times 1 = +0.3072, \\ \partial L_1/\partial b &= -0.3072. \end{aligned}$$

Sum: $-0.2246 - 0.0826 + 0.3072 = 0$. Autograd on `level1_logit` gives $(-0.2246,\ -0.0826,\ 0.3072)$ [C]. Gradient descent raises the plastic score most (the closer man-made entry), the metal score a little, lowers the soil score, and raises b.
Had the truth been natural ($y = 0$): $L_1 = -\ln(1 - 0.6928) = 1.1803$ and the gradients flip sign and scale by $p = 0.6928$: $(0.5065,\ 0.1863,\ -0.6928)$ [C].

44.7.5 Why matching helps

A hypothesis, consistent with Table 44.3 but not proven by it. Three reasons.

1. One hyperplane must hold all of nature on one side. A linear head says "man-made" when $\mathbf{w} \cdot \mathbf{h} + b_0 > 0$: one flat cut through the 256-dimensional space. Nature is not one cluster: vegetation, soils, rocks, water, snow and dry plant matter sit in different places, and man-made materials sit between them (a green tarpaulin near vegetation, concrete near rock). For a *fixed* feature space, one cut can be impossible.

Worked example 44.16: Four points no line can split

Two dimensions, unit vectors: natural at 0° and 180°, $\mathbf{n}_1 = (1, 0)$, $\mathbf{n}_2 = (-1, 0)$; man-made at 90° and 270°, $\mathbf{m}_1 = (0, 1)$, $\mathbf{m}_2 = (0, -1)$ [A]. Suppose a line $\mathbf{w} \cdot \mathbf{h} + b_0$ is negative on both natural points and positive on both man-made ones:

$$\begin{aligned} w_1 + b_0 < 0, \quad -w_1 + b_0 < 0 \ &\Rightarrow \ \text{adding: } 2b_0 < 0, \\ w_2 + b_0 > 0, \quad -w_2 + b_0 > 0 \ &\Rightarrow \ \text{adding: } 2b_0 > 0. \end{aligned}$$

Both cannot hold: no line does it. Matching does, with the four points as library entries and $\tau = 0.1$. A pixel at 80° has cosines 0.9848 to $\mathbf{m}_1$, −0.9848 to $\mathbf{m}_2$, 0.1736 to $\mathbf{n}_1$, −0.1736 to $\mathbf{n}_2$:

$$\begin{aligned} \text{LSE}(9.848,\ -9.848) = 9.848, \qquad \text{LSE}(1.736,\ -1.736) = 1.767, \\ \ell = 9.848 - 1.767 = 8.081 \ \Rightarrow \ \text{man-made}. \end{aligned}$$

A pixel at 190° gets $\ell = -8.081$, natural [C]. The nearest shelf decides.

The perceptron beneath the head *could* bend $\mathbf{h}$ until one cut works; that is what training would have to discover from one bit per sample. Matching needs no such bend: it is a nearest-prototype rule [40], and the namer has 219 natural prototypes on EnMAP's bank [C], one per natural spectrum, rather than one half-space.

2. It reuses what level 2 already learns. The class loss shapes $\mathbf{z}$ and $\mathbf{e}_j$ with a rich signal: seven classes, own spectrum left out. A separate linear head on $\mathbf{h}$ competes with that for the same features. Under matching, level 1 reads the *same* cosine table, so whatever arrangement

helps level 2 helps level 1; and level 1's gradient (Equation (44.15)) is the same kind of signal, moving the pixel toward and away from specific entries. The natural entries now also get a training signal they never had: in N001 nothing pulled a pixel toward a natural spectrum.

3. Nature gets described by examples. A new natural spectrum in the library is a new prototype at inference without retraining, as for man-made ones. A linear head's idea of nature is frozen in $\mathbf{w}$.

One built-in offset. The logsumexp of m equal scores is $s + \ln m$, so the side with more entries scores higher when nothing distinguishes them. On EnMAP's bank the training uses 219 natural and $159 - 34 = 125$ man-made entries: an offset of $\ln(219/125) = \ln 219 - \ln 125 = 5.3891 - 4.8283 = 0.5608$ towards "natural" [C]. The learned bias b can absorb it; it is one reason b exists.

Source check: what changes in the code and docs

`to_manmade` is still built and still computed by `forward_pixel`; under matching its output is replaced before the loss, so its 257 parameters receive no gradient (with `zero_grad(set_to_none=True)` AdamW skips them, weight decay included). `load_namer` reads `config["level1"]` from the checkpoint (default `"head"`) and accepts a checkpoint without `l1_bias`, so N001's checkpoints still load. The model's docstring now describes both forms, and the proposal's "As built" list records level 1 by matching; the proposal's design text ("Level 1 does not use the library at all") and the publication schematic (Figure 44.1) still show the old head. The code and `configs/namer/N001b.json` are what runs.

What level 1 by matching does not change. Natural spectra are planted in both splits, so level-1 accuracy on natural plantings is not an unseen-spectrum test; but the planted natural entry is left out of S_{nat} (`exclude`), so a natural mix cannot be recognised as its own spectrum either. And the planted man-made test spectra at evaluation are matched only against *training-split* man-made entries, because the class scores read the training library (Section 44.11).

44.8 One training step, end to end

This section is where the pieces stop being separate. One step of `NamerRun.train` loops over the three sensors; for each it draws one batch of tiles, plants, encodes, scores and back-propagates its loss divided by 3. Here is one sensor's share at illustrative EnMAP sizes: $B = 16$ tiles of 128×128, 512 pixels per tile so $P = 8192$ [M: `configs/namer/N001b.json`]; $N = 200$ live bands [A] (EnMAP's list has 224; a batch keeps those valid in more than half its pixels, and the true count depends on the tiles); $L = 378$ library entries [C]; $C = 7$; $D = 64$; $d = 32$; 4 heads; $G = 53$. The shapes in Tables 44.4 and 44.5 were recorded by forward hooks on the real code with random data at these sizes [C].

Worked example 44.17: From one row of S to both levels

Shrink the library to six entries, τ folded into the scores [A]: e_1 plastic 4.0 (the planted one), e_2 plastic 3.0, e_3 metal 2.0, e_4 concrete 1.0 (test split), e_5 soil 2.5, e_6 water 0.0. The sample

#	operation (file, function)	output shape
1	decode tiles: int16 $\times 10^{-4}$, -32768 invalid (`decode`)	`(16, 224, 128, 128)`
2	keep live bands; bank to the same bands (`forward`)	`(16, 200, 128, 128)`; bank `(378, 200)`
3	box sums: ring mean, ring count (`ring_and_noise`)	`(16, 200, 128, 128)`, `(16, 1, 128, 128)`
4	noise σ from horizontal differences	`(16, 200)`
5	allowed pixels: all bands valid, ≥ 20 ring pixels, 7 from the edge	`(16, 128, 128)` bool
6	pick 512 per tile with replacement; gather $\mathbf{b}$, $\mathbf{r}$, σ	`(8192, 200)` each
7	kind per sample (0.5, 0.25, 0.25); entry; fill	`(8192,)` each
8	$\mathbf{t}$ = bank rows (zeros for plain); gain $\mathbf{g}$; noise	`(8192, 200)` each
9	$\mathbf{x} = \mathbf{g} \odot (a\mathbf{t} + (1-a)\mathbf{b}) + \mathbf{n}$, $\mathbf{r}' = \mathbf{g} \odot \mathbf{r}$	`(8192, 200)` each

Table 44.4: One step, part 1: the generator, for one sensor. The tile tensor alone is $16 \times 200 \times 128 \times 128 = 52{,}428{,}800$ numbers, 200 MiB in float32 [C].

#	operation	output shape
10	stem + frozen BN on $\mathbf{x}$ (Table 44.2)	$\mathbf{z}_x$: `(8192, 64)`
11	stem + frozen BN on $\mathbf{r}'$	$\mathbf{z}_r$: `(8192, 64)`
12	concatenate $[\mathbf{z}_x, \mathbf{z}_r, \mathbf{z}_x - \mathbf{z}_r]$	`(8192, 192)`
13	pixel MLP	$\mathbf{h}$: `(8192, 256)`
14	`to_embed`, unit length; `to_fill`, sigmoid	$\mathbf{z}$: `(8192, 128)`; $\hat{a}$: `(8192,)`
15	stem + frozen BN on all 378 library spectra	`(378, 64)`
16	library MLP, unit length	$\mathbf{E}$: `(378, 128)`
17	$\mathbf{S} = \mathbf{Z}\mathbf{E}^\top / \tau$	`(8192, 378)`, 3,096,576 numbers
18	own entry $\to -\infty$ (every planted sample)	`(8192, 378)`
19	class index per entry: train man-made 0–6, natural 7, test -1	`(378,)`
20	pool: logsumexp over each class's columns	`(8192, 8)`
21	level 1: LSE(cols 0–6) $-$ col 7 $+ b$	ℓ: `(8192,)`
22	level 2: cols 0–6	`(8192, 7)`
23	BCE(ℓ, man-made), all samples	scalar
24	CE(level 2, class), man-made samples only (≈ 4096 rows)	scalar
25	$0.5\times$ mean $\lvert\hat{a} - a\rvert$, all samples	scalar
26	$(L_1 + L_2 + 0.5L_f)/3$, backward	–

Table 44.5: One step, part 2: the model and the losses, for one sensor. The stem runs three times per sensor (rows 10, 11, 15), nine times per step. After all three sensors: clip the gradient norm to 1, one AdamW step, one scheduler step.

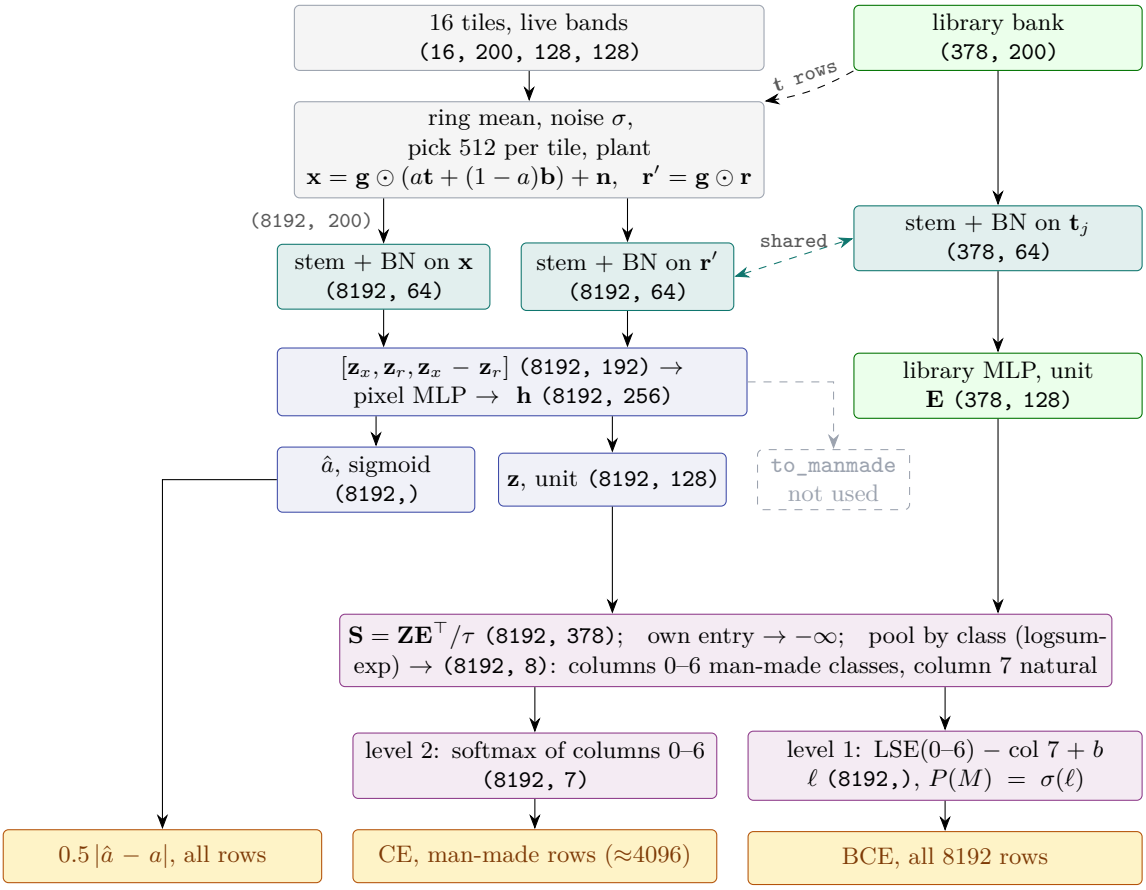


Figure 44.2: Tensor shapes through one training step for one sensor, with **level 1 by matching** (the route through column 7 of the pooled scores). Grey: the generator; teal: the shared stem, run three times; blue: the pixel tower; green: the library tower; plum: matching; amber: losses. The dashed grey head is N001's linear level 1, still built, no longer read. $B = 16$, $P = 8192$, $N = 200$ [A], $L = 378$.

is man-made, class plastic.

$$\begin{aligned}
\text{row 18 (exclude } e_1\text{)} &: (-\infty,\ 3.0,\ 2.0,\ 1.0,\ 2.5,\ 0.0)\\
\text{row 19 (classes)} &: (\text{plastic},\ \text{plastic},\ \text{metal},\ -1,\ \text{nat},\ \text{nat})\\
S_{\text{plastic}} &= \ln(e^{-\infty}+e^{3}) = 3.0, \qquad S_{\text{metal}} = 2.0, \qquad S_{\text{concrete}} = -\infty\\
S_{\text{nat}} &= \ln(e^{2.5}+e^{0}) = \ln(12.1825+1) = \ln 13.1825 = 2.5789\\
\ell &= \ln(e^{3}+e^{2}) - 2.5789 = \ln(20.0855+7.3891) - 2.5789\\
&= 3.3133 - 2.5789 = 0.7344\\
P(M) &= \sigma(0.7344) = 1/(1+e^{-0.7344}) = 1/(1+0.4798) = 0.6758\\
P(\text{plastic} \mid M) &= e^{3}/(e^{3}+e^{2}) = 20.0855/27.4746 = 0.7311
\end{aligned}$$

The test-split concrete entry e_4 is encoded but belongs to no class, so it plays no part; a class with no entry in play scores $-\infty$ and gets probability 0.

44.9 The loss, the optimiser and the schedule

$$\mathcal{L} = \underbrace{\mathrm{BCE}(\ell, M)}_{\text{every sample}} + \underbrace{\mathrm{CE}(S_{0..6}, c)}_{\text{man-made samples}} \quad +0.5 \quad \underbrace{|\hat{a} - a|}_{\text{every sample}} \tag{44.16}$$

(`namer/train.py`, `loss`; `fill_weight` 0.5). *BCE*, binary cross-entropy, is the loss for a yes/no answer, $-[y \ln p + (1-y)\ln(1-p)]$; CE is minus the log of the right class's probability. Under matching ℓ is Equation (44.13); under the old head it was `to_manmade`'s output. The fill term is on *every* sample, natural mixes and plain pixels too (plain target 0), not on man-made samples only as the proposal said; its "As built" list records the change. A fill head that learns "nothing was added" is level-1 evidence.

Two chance levels anchor any loss curve. A level-1 answer that knows nothing outputs $p = \frac{1}{2}$: BCE $= -\ln\frac{1}{2} = \ln 2 = 0.6931$. A class answer that knows nothing over 7 classes: $\ln 7 = 1.9459$. The untrained namer at the sizes of Section 44.8 gives BCE 0.7344 and CE 1.9941 on random data [C], close to both. The CPU smoke run of the first code (EnMAP and SpectralEarth smoke shards, 120 steps, linear level 1) moved the class CE from 1.81 to 1.02, level-2 accuracy at fill 1.0 from 0.62 to 0.65 on EnMAP, and left BCE "still ∼0.68 (too short to judge)" [M: `research/experiments/spectral-namer/N001/README.md`]: level 1 had not left chance, an early sign of what epoch 1 then confirmed.

Per step. Each sensor's loss is multiplied by its weight (1) and divided by the number of sensors (3) before `backward()`, so gradients from the three sensors add up within one step. Then the gradient norm is clipped to 1, AdamW [29] steps, and the scheduler steps. On a GPU the forward runs under bfloat16 autocast; the class scores are computed in float32 (`z.float()`, `e.float()`). A step sees up to $3 \times 16 \times 512 = 24{,}576$ samples, about 491.5 million over $40 \times 500 = 20{,}000$ steps.

Two learning rates. The stem and its BatchNorm (parameter names starting `stem.` or `norm.`) learn at 0.3× the base rate 3×10^{-4}, that is 9×10^{-5}; everything else, including $\ln\tau$ and b, at 3×10^{-4}. The stem already knows spectra from V2; a slower rate keeps that knowledge while the new towers, which start from random weights, catch up. Weight decay is 0.01 for both groups.

Warm-up and cosine. The scheduler multiplies both rates by

$$\lambda(s) = \min\Big(1, \frac{s+1}{500}\Big) \times \frac{1}{2}\Big(1 + \cos\Big(\pi \min\big(1, \tfrac{s}{20000}\big)\Big)\Big), \tag{44.17}$$

a linear warm-up over 500 steps times a half cosine over the whole run.

Worked example 44.18: The schedule at a few steps

$$s = 0 : \min(1, 1/500) = 0.002,\ \cos 0 = 1,\ \lambda = 0.002 \times 1 = 0.002,\ \mathrm{lr} = 6.0 \times 10^{-7},$$
$$s = 249 : 250/500 = 0.5,\ \tfrac{1}{2}(1 + \cos(0.0391)) = 0.99962,\ \lambda = 0.49981,\ \mathrm{lr} = 1.499 \times 10^{-4},$$
$$s = 10000 : 1 \times \tfrac{1}{2}(1 + \cos(\pi/2)) = \tfrac{1}{2}(1 + 0) = 0.5,\ \mathrm{lr} = 1.5 \times 10^{-4},\ \text{stem } 4.5 \times 10^{-5},$$
$$s = 15000 : \tfrac{1}{2}(1 + \cos(3\pi/4)) = \tfrac{1}{2}(1 - 0.70711) = 0.14645,\ \mathrm{lr} = 4.39 \times 10^{-5}.$$

The script prints the same [C].

One BatchNorm, nine kinds of batch: why it is frozen. In the first version of the code the post-stem BatchNorm was in training mode during training. Each sensor's `forward` encodes the pixels, the rings and the whole library in three separate calls, so it saw nine kinds of batch per step, each normalised by its own statistics. Its stored running mean is an exponential average with momentum 0.1, running $\leftarrow 0.9\,\text{running} + 0.1\,\text{batch}$; in steady state the call k places from the end of a step carries weight $0.1 \times 0.9^k/(1 - 0.9^9)$, and $1 - 0.9^9 = 1 - 0.3874 = 0.6126$. The last call, AVIRIS-NG's library, weighs $0.1/0.6126 = 0.1632$; by kind, pixels 0.299, rings 0.332, library 0.369 (`research/textbook/tools/compute_namer_submission_output.txt`) [C]. Evaluation and inference would then normalise all three by that one blend: Chapter 29's BatchNorm see-saw in a new form, a train/evaluation mismatch found by reading the code and never measured. **It was fixed before N001 ran** (code bundle `namer2`): the norm never leaves evaluation mode; its running statistics stay V2's and its scale and shift (128 numbers) still train, in the stem's group.

44.10 The library, the classes and the splits

The bank. `build_bank` (`namer/bank.py`) resamples the family library to one band list with the library's own resampler (Gaussian spectral responses of width 1.1× the band spacing), fills bands a spectrum does not cover by linear interpolation along wavelength (and the nearest end beyond its range), keeps entries with at least 90% finite bands, maps each to a class, and keeps the first entry of every repeated name. The result is a `LibraryBank`: `refl (L, N)`, `cls (L,)` with -1 for natural, `test (L,)`, the names, and the held-out class.

The classes. `namer/classes.py`: natural chapters → natural; building and pavement → concrete/asphalt; roofing dissolved by name (fiberglass, rubber, clear roof → plastic; cedar, shake, wood → wood/paper; metal, tin, steel, aluminium → metal; tar, felt, asphalt, shingle, tile → concrete/asphalt; any other roofing → concrete/asphalt); glass left out (one spectrum, "cannot be tested"). No roofing entry of the shipped library maps to metal: it holds no metal roofing.

Held-out spectra. A man-made entry is in the test split if the first 8 hexadecimal digits of the SHA-1 hash of its name, read as a number and divided by $2^{32} - 1 = 4{,}294{,}967{,}295$, fall below 0.2 (`_is_test`). Test spectra are never planted in training and are not candidates in the class

scores, in training or in evaluation. A hash rather than a random draw means every band list agrees on the split without sharing a file.

Worked example 44.19: Two metals, two sides

"Copper Metal" : `1600007b` = 369,098,875,
369,098,875/4,294,967,295 = 0.0859 < 0.2 ⇒ test;

"Aluminum Metal" : `d13908a4` = 3,510,175,908,
3,510,175,908/4,294,967,295 = 0.8173 ⇒ train.

De-duplication. Some library names repeat ("Copper Metal" three times, "Galvanized Steel Metal" twice). The hash sends repeats to one side, so the test split stays clean, but in training, leaving one "Galvanized Steel Metal" out left its namesake in the class score, a near-copy that defeats the leave-out. Since the N001 code bundle (`namer2`), `build_bank` keeps only the first entry per name.

class	entries	held out	share	train
oil/fuel	40	8	20%	32
plastic	46	8	17%	38
textile/fabric	14	3	21%	11
wood/paper	11	2	18%	9
paint/coatings	4	1	25%	3
metal	10	3	30%	7
concrete/asphalt	34	9	26%	25
all man-made	159	34	21%	125
natural	219	–	–	219
bank	378			

Table 44.6: The bank on EnMAP's 224-band list, one entry per name (AVIRIS-NG: 380 entries, 220 natural, 160 man-made, one more wood/paper entry, 34 held out) [C]. Under level 1 by matching the 219 natural entries are class 7 in training.

Source check: about 20% overall, not per class

The hash ignores the class, so the held-out share varies: 17% to 30% per class with one entry per name, as the docstring of `namer/bank.py` now says. The N001 README still says "∼20% per class", and its counts (408, 174, 41) are from before the de-duplication; the code gives 378, 159, 34 on EnMAP.

A held-out class (N002): textile/fabric. `configs/namer/N002.json` holds the whole textile/fabric class out (`"holdout_class":"textile_fabric"`). It is never planted in training and never a candidate; at evaluation it is planted, and the trainer records how often level 1 calls it man-made and how often level 2 gives it a known class with $P > 0.5$. Pass: man-made at least 70%, confident wrong at most 20%. On EnMAP's bank this leaves $125 - 11 = 114$ man-made training entries, and the evaluation pool of unseen man-made spectra is the 34 held-out spectra plus textile's 11 training-split ones, 45 [C].

Source check: textile, not paint

N002 first held out paint/coatings. After de-duplication paint has 4 spectra, too few to test anything, and on 2026-10-31 the user chose textile/fabric (14 spectra) instead; `configs/namer/N002.json` says so. Three places still say paint: the proposal's "As built" list ("Open-set class: paint/coatings (run N002)"), its "Splits" text ("e.g. paint/coatings"), and the copy `research/experiments/spectral-namer/N002/config.json`, which is the old config. Likewise `configs/namer/N001.json` now holds N001b's settings (run directory `N001b`, `"level1": "matching"`); N001's own config survives as `research/experiments/spectral-namer/N001/config.json`.

44.11 The test it must pass

Each epoch `evaluate` plants test-split spectra into test shards at fills 0.1, 0.25, 0.5 and 1.0, half man-made and half natural mix (`kinds=(0.5,0.5,0.0)`), 40 batches with a fixed seed: up to $40 \times 8192 = 327{,}680$ samples per sensor and fill [C]. Level 1 can buy recall by calling more things man-made, so the fair comparison holds false alarms equal: read ACE's natural false-alarm rate f from `separability_<sensor>.json` (1−`level1_natural_called_natural`), cut the namer's logits at the $(1-f)$ quantile of its natural plantings, and count the man-made plantings above the cut:

```
cut = torch.quantile(torch.cat(nat_logits), 1 - fa)
recall = (torch.cat(mm_logits) > cut).float().mean()
```

Worked example 44.20: Recall at a fixed false-alarm rate

Ten natural logits, sorted: $-3, -2.5, -2, -1.5, -1, -0.5, 0, 0.5, 1, 2$; $f = 0.2$ [A]. The 0.8 quantile, with linear interpolation as in Section 32.2, sits at position $0.8 \times (10-1) = 7.2$, between 0.5 and 1.0:

$$\text{cut} = 0.5 + 0.2 \times (1.0 - 0.5) = 0.5 + 0.1 = 0.6.$$

Five man-made logits $-1, 0.4, 0.7, 1.5, 3$: above 0.6 are three, so recall $= 3/5 = 0.6$ [C].

sensor	ACE natural right	f	ACE man-made recall	namer needs
PRISMA	0.8342	0.1658	0.7724	≥ 0.8724
EnMAP	0.7851	0.2149	0.6635	≥ 0.7635
AVIRIS-NG	0.8134	0.1866	0.7603	≥ 0.8603

Table 44.7: Level 1's bars at fill 0.25: ACE +0.10 at ACE's own false-alarm rate, from the three separability JSONs [C].

Level 2 must also beat ACE by 0.10, but not against these JSONs yet: their level-2 numbers were measured on the 17 old families and "must be re-scored on the v1 classes … before comparing; that re-score is the first job after launch" [M: `research/experiments/spectral-namer/N001/README.md`]. `best.pt` is the epoch with the best mean level-2 accuracy at fill 0.25 over the sensors, and every epoch also writes `ep<n>.pt` and a row of `metrics.csv` (losses, τ, learning rate, and per sensor and fill the level-1 and level-2 accuracies, the recall at ACE's rate, and for N002 the two open-set rates).

Source check: three things the proposal lists and the trainer does not do

It computes no bootstrap interval (the pass rule's "interval above zero" is a later step); it has no "unknown" output (level 2 always returns its best class and `p_cls`; "class unknown" needs a threshold not yet chosen); and natural spectra are planted in both splits, so level-1 natural accuracy is not an unseen-spectrum test ("As built").

44.12 Inference: naming the pixels of a mask

`load_namer` (`namer/infer.py`) rebuilds the model from a checkpoint (`last.pt`, `best.pt` or `ep<n>.pt`): the normalisation curve from the stored `stats.*` buffers, the sizes from the stored config's `model` entry, and the level-1 mode from its `level1` entry (default `"head"`). Then

```
name_pixels(model, cube, wavelengths, pixels, library_root, cache_dir, valid)
```

names the pixels of a boolean mask:

1. **Bands.** Keep a band if it is valid at more than half of the requested pixels (`valid[:,rows,cols].mean(axis=1)>0.5`): 3 valid of 5 pixels is 0.6, kept; 2 of 4 is exactly 0.5, dropped.
2. **Pixels and rings.** `x` is the cube at the mask's pixels, `(P, N)`; the ring means are computed one band at a time over the whole scene (`ring_means`, box sums of 15×15 minus 5×5 with `uniform_filter`), so a full scene never needs a second cube in memory. The per-pixel band validity is passed to the stem, which masks dead bands exactly.
3. **Library.** `build_bank` at the scene's bands, the *whole* library, test-split spectra included: at inference every spectrum is fair to match against. One `forward_library` call encodes it.
4. **Scores**, in batches of 4,096 pixels: `forward_pixel`, then `class_scores` with $C+1=8$ classes under matching (all man-made entries in their classes, natural as class 7), `level1_logit` for level 1 and the first seven columns for level 2; with the old head, 7 classes and the head's logit.
5. **Outputs** per pixel: $P(M)=\sigma(\ell)$; the best class and its $P(k \mid M)$ from the softmax of the seven class scores; $\hat{a}$.

No entry is excluded at inference: the pixel's own spectrum is unknown, so there is nothing to leave out.

Into the AIGC table. `scripts/aigc_object_table.py` is the first caller (Chapter 45). With `--namer-ckpt` it loads the vendable, passes its `normalized_hyperspectral_cube` (reflectance, despite the name; `docs/09-known-issues.md`), `band_cw_order` and validity cube to `name_pixels` with the committed anomaly mask, and then averages per 8-connected object: mean $P(M)$, the majority class and its share, mean $P(k \mid M)$, mean fill. The findings file labels the namer column "preliminary", and objects with mean $P(M)>0.5$ count as man-made when it is compared with the family match.

Leave the planted spectrum out of its own class score, and a two-tower network can only lower its loss by placing a whole material together; read *both* questions from the same cosine table, so that level 1 compares the man-made shelf with the natural one; and judge level 1 at the classical method's own false-alarm rate, so that recall cannot be bought.

Values that will drift

Everything in this chapter: N001b and N002's settings (`configs/namer/`), the bank counts (378 / 219 / 159 / 34 on EnMAP), the per-class shares, the 347,685 parameters, the two early level-1 measurements, the CPU smoke numbers, and the pass bars. N001b's and N002's results were pending when the chapter was written (2026-10-31).

44.13 Common confusions

The namer reuses Indradhanu V2, so it detects anomalies.
It reuses V2's stem, 5% of its parameters, as a spectral feature extractor. It names pixels something else has already flagged.

Siamese means the two sides are identical.
Only the stem is shared, 17,825 weights. After it the pixel tower (214,402 weights, ring input, fill head) and the library tower (115,328) differ on purpose.

Logsumexp is the best cosine.
A smooth maximum: $\ln(e^7 + e^6) = 7.31$, not 7. Every member counts, the closest most.

Level 1 by matching needs a new model.
It needs one new number, b, and a different reading of the same score table. N001's head is still in the model, unused.

$P(M)$ is the best man-made cosine against the best natural one.
It is a share of softmax mass (Equation (44.14)): every entry contributes, weighted by e^{s_j}, and the side with more entries starts ahead by ln of the count ratio, which b can absorb.

Leaving the planted spectrum out is an evaluation trick.
Here it is a training rule too, for man-made *and* natural plantings, and the main departure from the proposal's per-spectrum InfoNCE.

The ring is 2 to 7 pixels away.
Beyond 2: Chebyshev distance 3 to 7, 200 pixels in the interior.

20% of every class is held out.
About 21% of the man-made spectra overall on EnMAP (34 of 159); 17% to 30% per class.

N002 holds out paint.
It did in the first config. It holds out textile/fabric (14 spectra) since the user's decision of 2026-10-31.

A frozen BatchNorm does not learn.
Its running statistics are frozen; its scale and shift still train.

44.14 Check yourself

Exercise 44.1. In worked example 44.9, make the halo 0.60 and the target 0.90. What is the ring mean, and why?

Exercise 44.2. With $a_{\min} = 0.02$ instead of 0.05, what share of fills is at most 0.25, and what is the mean fill? Use Equation (44.11) and the mean formula.

Exercise 44.3. Plant $\mathbf{t} = (0.30, 0.10)$ into $\mathbf{b} = (0.20, 0.40)$ at $a = 0.5$ with $\mathbf{g} = (1.1, 0.9)$ and no noise.

Exercise 44.4. In worked example 44.13, suppose metal had only e_1 and e_2. With e_1 left out, compute S_{metal}, $P(\text{metal})$ and the loss.

Exercise 44.5. Count the parameters of a namer with `hidden` 128 and `embed` 64, everything else as N001b. (Hint: only the towers change.)

Exercise 44.6. At PRISMA fill 0.25 the namer, cut at $f = 0.1658$, recalls 85% of man-made plantings. Does level 1 pass? What recall would it need?

Exercise 44.7. Show that the level-2 gradient entries $q_j - \mathbf{1}[j \in c]w_j$ always sum to zero.

Exercise 44.8. In worked example 44.15, add a second natural entry, water, with $s = 5.5$. Compute ℓ, $P(M)$, the BCE for a man-made truth, and the four gradients. Which side gained from the extra entry?

Exercise 44.9. Bands at 1790, 1800, 1960, 1970 nm. Compute their widths, and the bins they fall in.

Exercise 44.10. Level 1 by matching with $b = 0$ on a bank of 219 natural and 125 man-made entries: a pixel has the same score s against every entry. What is $P(M)$? What b would make it $\frac{1}{2}$?

Next: Chapter 45, Producing a submission.

Chapter 45

Producing a submission

The one thing this chapter teaches

An evaluation result is a committed `anomaly_detection_prep` exported as a boolean 0/1 GeoTIFF plus a point shapefile, with no score anywhere a judge could read as a heatmap, on the product's own map grid. For PRISMA that grid now comes from the HE5's UTM corner attributes read as pixel *centres*, pitch = span/$(n-1)$ and origin = upper-left centre minus half a pixel, replacing a latitude and longitude bounding box that was a median 2.4 pixels off.

On 2026-10-31 an AIGC Stage-III evaluation circular set the rules for the deliverables, with a report due on 2 November 2026 and a presentation on 4 November. The plan and its status are `docs/proposals/aigc-evaluation-submission.md`: its changes C1–C6 were built and tested that day, and C7, the rehearsal, was done on the *local* stack ("images rebuilt, worker `running`, export rehearsed on a committed PRISMA prep: `hyper_*` bundle, uint8 {0,1} mask in the product's UTM zone"). A production or air-gapped deployment is not approved and is a separate step. This chapter is how the product meets the rules, and the one piece of geometry that had to change.

Analogy

A land registry accepts boundary maps only as surveyed lines on its own grid: not a sketch, and not a shaded map of "likely boundaries". You may file up to three claims per plot. Until this week the office drew PRISMA plots by eye from the corners of a road atlas; now it reads the surveyor's grid references off the deed.

45.1 The rules, and the one bundle that meets them

	rule (circular)	how the product meets it
R2a	boolean 0/1 anomaly map as a GeoTIFF with a real CRS	the prep commit writes uint8 0/1; the export stamps on the raw scene's georeference
R2b	the same map as a shapefile	one point per anomaly pixel, full sidecar set (.shp .shx .dbf .prj .cpg)
R2c	no heatmap-style output: it disqualifies	no score raster in the bundle, and since 2026-10-31 no score attribute in the shapefile
R2d	up to three results per scene	three committed prep actions, three exports
R2e	library matching by a foundation model and by classic SAM, compared in the report	the per-object table, Section 45.5
R1	a reflectance product from L1 data	not in the product; a writer for another module's output, Section 45.6

Only the prep bundle qualifies: the export of a committed `anomaly_detection_prep`, built by `build_thermal_bundle` in `app/anomaly_detection/export.py`. The project export zips every output tree, `anomaly_score.tif` (a heatmap) included; the materials bundle of Section 32.5 carries int16 family codes, not 0/1. Two changes were made to the prep bundle (C2):

```
prefix = "thermal" if spec.sensor_type in THERMAL_SENSORS else "hyper"   # ("landsat9
    ", "hotsat1")
folder = f"{prefix}_{_short(spec.scene_id)}_{_short(spec.action_id)}"
keep_cols = ["row", "col", "latitude", "longitude", "geometry"]           # "score"
    removed
```

A PRISMA bundle used to be named `thermal_*`, against the rule that a name says `hyper` or `thermal`. The comment on the dropped column: "No score column: the submission map is boolean, and a continuous score reads as a heatmap (AIGC rules, 2026-10-31). The CSV keeps it."

Source check: the score is still in the zip

The CSV, `<prefix>_anomaly_summary.csv`, is inside the same zip with its `score` column; the proposal calls it "not submitted", so leave it out of the upload. Two names were not updated: the function is still `build_thermal_bundle`, and the manifest's schema is still `allotrope-export-thermal/1` for a `hyper_` bundle.

45.2 The PRISMA grid

Recall Equation (32.6): $X = a\,j + b\,i + c$, $Y = d\,j + e\,i + f$, with (j, i) a position in pixel units measured from the top-left *corner* of the image, so the top-left pixel's centre is $(0.5, 0.5)$. Until 2026-10-31 a PRISMA export used the latitude and longitude bounding box of worked example 32.7. The ground-truth rasters are in UTM. On Set-1 the box put pixels a median 2.4 px from the product grid, 95% within 4.1, at most 5.6 [M: `docs/proposals/aigc-evaluation-submission.md`]: at 30 m, $2.4 \times 30 = 72$ m typical and $5.6 \times 30 = 168$ m at worst. The internal evaluation never noticed, because it compares masks in pixel space.

A PRISMA L2D product is already gridded in UTM, and the HE5's root attributes say how (`prisma_utm_grid`, `app/georef/from_scene_file.py`): `Epsg_Code`, and the easting and northing of the upper-left, upper-right and lower-left corner pixels. Its docstring: they "are the corner pixels' *centres*, so the pitch is the corner span over $(n-1)$ and the origin is the UL centre minus half a pixel".

Centres, not edges. Five fence posts 30 m apart span $4 \times 30 = 120$ m from the first post's centre to the last's. In general, n pixel centres are $n - 1$ pitches apart, so with W columns

$$\begin{aligned} \mathrm{UR}_E - \mathrm{UL}_E &= (W-1)\,\Delta x && \text{centres of the first and last column} \\ \Delta x &= \frac{\mathrm{UR}_E - \mathrm{UL}_E}{W-1} && \text{divide by } W-1 \\ c &= \mathrm{UL}_E - \tfrac{1}{2}\Delta x && \text{the image edge is half a pixel left of the first centre,} \end{aligned}$$

and the same for rows with the northings, except that rows run southwards, so $e = -\Delta y$ and $f = \mathrm{UL}_N + \frac{1}{2}\Delta y$.

Worked example 45.1: Set-1's grid by hand

The test file `tests/test_utils/test_georef_prisma.py` holds Set-1's attributes, stored as 32-bit floats like the file: `Epsg_Code` 32642; UL easting 545,022.75 and northing 2,501,422.5; UR easting 580,872.5; LL northing 2,465,872.5. The image is $H = 1186$ rows by $W = 1196$ columns.

$$\begin{aligned} \text{east span} &= 580{,}872.5 - 545{,}022.75 = 35{,}849.75 \text{ m}, \\ \Delta x &= 35{,}849.75/1195 = 29.99979 \;\rightarrow\; 30.0 \text{ m (rounded to centimetres)}, \\ \text{north span} &= 2{,}501{,}422.5 - 2{,}465{,}872.5 = 35{,}550 \text{ m}, \\ \Delta y &= 35{,}550/1185 = 30.0 \text{ m}, \\ c &= 545{,}022.75 - 15 = 545{,}007.75, \qquad f = 2{,}501{,}422.5 + 15 = 2{,}501{,}437.5. \end{aligned}$$

The transform is (30, 0, 545,007.75, 0, −30, 2,501,437.5) in EPSG:32642 (UTM zone 42 north): exactly the ground-truth raster's grid, which the test asserts. Check the top-left centre: $X = 30 \times 0.5 + 545{,}007.75 = 545{,}022.75$, the UL attribute [C].

Read the attributes as edges instead and $\Delta x = 35{,}849.75/1196 = 29.9747$ m: 0.025 m short per pixel, which over 1196 columns adds up to $1196 \times 0.0253 \approx 30$ m, one whole pixel at the far edge. The rounding to centimetres (the code's comment: "float32 attrs: 29.9998 -> 30.0") snaps the pitch to the product's 30 m; its price is that the bottom-right centre comes out at 580,872.75 instead of the attribute's 580,872.5, a quarter of a metre, under 1% of a pixel [C].

When the old method still runs. `_resolve_prisma` uses the product grid only when the attributes are present and sane (a positive EPSG code, at least 2×2 pixels, positive spans) and the vendable's shape equals the latitude array's. Otherwise it falls back to the bounding box, whose docstring now says "approximate, can be pixels off"; a test checks both refusals. The same function serves the reflectance writer's `--georef-from`.

45.3 Native bands by default

The proposal's gap 3 was "Indradhanu V2 cannot be run from the UI, because no UI control sets the band filter to native bands". Its fix, C3: every hyperspectral sensor default in `band_filter_apply.py` now reads

```
"use_common_wavelength_grid": False,   # native bands (2026-10-31): V2 reads them;
    165-grid models resample at scoring
```

The dialog prefills its form from these defaults, so the default is now V2's setting, in line with the standing design decision "One inference method: native bands" (`docs/lld/spectral-stem.md`, 2026-10-15). The old 165-band model keeps working because `anomaly_scoring` resamples for

it, per model, with the band filter's own resampler (`_to_common_grid`; a cube already on the grid is passed through):

band filter	v0.2.0 (165 grid)	V2 (native)	MNF-RX
native (the default)	runs, resampled	runs	runs
165 grid	runs	refused	runs

So one scoring action can now run both Indradhanus on a native band filter (Section 32.6).

Source check: a field default still describes the 165-band world

The pydantic field `BandFilterApplyConfig.use_common_wavelength_grid` still has `default=True`, and its description still says the grid is "Required by Indradhanu (v0.2.0)". The UI sends the sensor defaults, so the UI gets native bands; **an API call that omits the field gets the 165 grid, and V2 then fails** (`docs/09-known-issues.md` records it as deliberate). The worker's fallbacks `cfg.get("use_common_wavelength_grid",True)` agree with the field, not with the defaults. Two more comments were stale when this chapter was first written, `_check_band_grid`'s docstring and the resolver's comment on `band_grid`; both were corrected on 2026-10-31 and now describe the refusal of V2 on the 165 grid and the resampling for a common165 model.

45.4 Three results per scene

Each result is one committed prep action, so three results are a procedure, not code: three prep actions (they may share one scoring output), three commits, three exports, each zip named by its action's short id. A re-commit overwrites a prep's mask, so one prep cannot carry two results. The proposal suggests, per hyperspectral scene and all with residual Mahalanobis: Indradhanu V2 (Heavy); V2 and MNF-RX combined at the prep weights; and MNF-RX alone or V2 at a second threshold. Choosing the thresholds stays a person's job (Section 32.1); with no ground truth on the evaluation scenes, the proposal asks whether to fix a percentile per model from the four labelled PRISMA scenes.

45.5 The per-object table

The report must compare a foundation model with SAM, per object. `scripts/aigc_object_table.py` groups the committed mask into connected components with `scipy.ndimage.label` and a 3×3 structure of ones.

Definition 45.1 (4- and 8-connectivity)**.** Two flagged pixels are *4-connected* if they share an edge, and *8-connected* if they share an edge or a corner. An object is a largest set of flagged pixels linked by a chain of such neighbours.

Worked example 45.2: Two objects or three

$$\begin{pmatrix} 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The pixel at row 2, column 2 touches row 1, column 1 only at a corner. With 8-connectivity it joins the top-left group: 2 objects. With 4-connectivity it is alone: 3 objects. `label`

agrees in both cases [C].

Per object the script writes size, centroid (latitude and longitude through the scene's own georeference, so PRISMA uses Section 45.2's grid), and three opinions: SAM (`matches.parquet`, rank 0: the majority top-1 name and its share, its chapter, the median angle, the three commonest names), the family match (`family_matches.parquet`, answered pixels: majority family, group, mean confidence, answered share) and, with `--namer-ckpt`, the namer of Chapter 44 (mean $P(M)$, majority class and share, mean $P(k \mid M)$, mean fill). Outputs: `objects.csv`, `objects.geojson`, and a `findings.md` stub with agreement counts. Its docstring is honest that the family match is "calibrated Simplex ACE (classical)": the namer is the only foundation-model column, read from an N001 checkpoint (`ep<n>.pt` exists after every epoch, so a preliminary one can be used before the block ends).

45.6 The reflectance writer

R1 asks for surface reflectance from L1 data. The atmospheric correction is another session's module; `scripts/write_reflectance_product.py` only writes its result: GeoTIFF and/or ENVI BSQ with a `.hdr`, every wavelength recorded, water-absorption bands removed (default 1340–1460 and 1790–1960 nm, ends included), and reflectance ×10000 as int16 with nodata −9999:

```
data = np.where(np.isfinite(cube), np.clip(np.round(cube * SCALE), -9998, 32767),
    NODATA).astype(np.int16)
```

Worked example 45.3: Writing five values

reflectance	×10000	written	why
0.2	2000	2000	
0.12345	1234.5	1234	`np.round` sends a half to the even neighbour
0.12355	1235.5	1236	here the even neighbour is above
NaN	NaN	−9999	nodata
4.0	40000	32767	clipped to int16's top

The lower clip is −9998, so no real value collides with nodata. Bands at 1330, 1345, 1460 and 1470 nm under the default drop: 1345 and 1460 go (1460 is an end of the range), 1330 and 1470 stay [C].

Values that will drift

The submission's status (C1–C7 on the local stack, deployment not approved), the deadlines, the Set-1 offset (median 2.4 px, 95% 4.1, max 5.6), the three suggested results, and the field default and its description in Section 45.3, which a later change may align.

45.7 Common confusions

Any export from the product will do.
Only the prep bundle. The project export carries a heatmap; the materials raster is family codes.

No score in the shapefile means no score in the zip.
The CSV in the same zip keeps it. Do not upload it.

PRISMA's corner attributes are the image's corners.
They are the corner pixels' centres. Read as edges, the far edge drifts by a pixel.

The georeference error moved the benchmark.
The evaluation compares masks in pixel space. Only deliverables overlaid on a map were affected.

V2 still needs a band-filter setting.
Not from the UI since 2026-10-31. An API call that omits the field still gets the 165 grid.

The reflectance script corrects the atmosphere.
It writes another module's output. "The atmospheric correction itself is not here."

45.8 Check yourself

Exercise 45.1. A PRISMA product has $W = 1000$ columns, UL easting 400,015 and UR easting 429,985. Compute Δx and c. What pitch does dividing by W give, and how far off is the last column's centre then?

Exercise 45.2. With Set-1's transform, where is the centre of the pixel in row 10, column 20?

Exercise 45.3. A band filter was created through the API without `use_common_wavelength_grid`. A scoring action picks `["IndradhanuV2","Indradhanu"]`. What happens?

Exercise 45.4. Count the objects in $\begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{pmatrix}$ under 8- and under 4-connectivity.

Exercise 45.5. Write 0.03, -1.2 and 3.5 as the reflectance writer would.

Exercise 45.6. Why is a second commit on the same prep action not a way to make result 2?

That brings the book to 2026-10-31: Indradhanu V2 (Heavy) in the product, native bands by default, a spectral namer in training, and the first evaluation submission on the product's own grid.

Part X

A foundation model that is wrong for this job: HyperSL

Chapter 46

What HyperSL is, and what it does well

The one thing this chapter teaches

HyperSL is a 9.03-million-parameter masked autoencoder that reads *one pixel's spectrum* at a time, after every band has been min-max scaled over the scene it came from; we checked it against its own code and checkpoint and reproduced its paper's clustering result almost to the decimal, so what the next five chapters find is not a broken download but a good model built for a different job.

This chapter opens Part X. The whole part answers one question the team was asked more than once in September and October 2026: *there is a published spectral foundation model with open weights; why do we not use it?* The honest answer has two halves. The first half, this chapter, is what the model is and what it is good at, checked against its source rather than its abstract. The second half, Chapters 47 to 50, is what happened when it was tried on each of the jobs this product needs done, with every number from a recorded run. Chapter 51 puts the two halves side by side.

Paths abbreviate. `upstream/` is the authors' code as we hold it, byte-identical to their GitHub commit `e56de7e`. It is kept outside the repository, in the folder

`C:/Users/FITT/Documents/hypersl_match_data/hypersl_model/`

Experiment records that are no longer in the working tree are cited by the commit that holds them, for example `2d893e6:hypersl-validation/summary.md`, which you read with `gitshow2d893e6:research/experiments/hypersl-validation/summary.md`.

Analogy

A concert pianist who learned every piece from recordings that a studio had normalised so that the loudest note of each recording was exactly as loud as every other recording's loudest note. She plays beautifully, she can tell a Chopin nocturne from a Bach fugue with her eyes closed, and if you hum her half a phrase she finishes it. Ask her instead whether *this* hall is louder than last night's, and she cannot say: the one thing every recording she learned from had removed was the absolute level. HyperSL is the pianist. The level is reflectance.

46.1 Why look at an outside model at all

The product needs three things from spectra, and in autumn 2026 each had a weak spot.

1. **Naming a material** by comparing a pixel with a laboratory library (USGS splib07, ECOSTRESS): Chapters 32 and 44. Whole-pixel spectral angle named the right man-made family for 6% of pixels at 30% fill
   [M: `docs/lld/spectral-family-match.md`].

2. **Detecting anomalies**: Indradhanu's residual, scored by residual Mahalanobis (Chapter 31).

3. **Masking cloud**, so that detections on cloud are not reported (Chapter 28).

A *foundation model* is a network pretrained on a large, broad, unlabelled data set so that its internal representation can be reused for many tasks with little or no task-specific training. HyperSL [21] advertises exactly that for spectra: pretrained "on over 300 million spectral instances worldwide", able to process "all forms of hyperspectral data across diverse spectral bands and ranges", with code and weights public. If any outside model could cover all three weak spots at once, it was this one. That is why it was evaluated four separate times between 2026-09-20 and 2026-10-14.

Definition 46.1 (Embedding, reconstruction, residual)**.** An *embedding* is the short vector a network makes of its input (HyperSL's has 256 numbers). A *reconstruction* $\hat{\mathbf{x}}$ is the network's attempt to reproduce its input $\mathbf{x}$ from partial information. The *residual* $\mathbf{r} = \mathbf{x} - \hat{\mathbf{x}}$ is what the reconstruction got wrong (Chapter 9).

46.2 What it is, read from the code

46.2.1 One pixel in, one pixel out

HyperSL's forward pass is the method `forward(x,wave,mask_ratio)` of `SpectralSharedEncoder` in `upstream/model.py`. Its input x has the shape `(B, 1, L)`: B pixels, one channel, L bands. It has no height and no width. **Each pixel is processed alone; no pixel ever sees another.** Keep that sentence in mind through the whole part: it is the root of Chapter 48's result.

Table 46.1 traces one PRISMA pixel on the 149-band axis the experiments in this part used (the reason for 149 bands comes in Section 47.3).

Three facts in that table matter later.

- **It hides bands, not pixels** (step 5). Indradhanu hides whole pixels and rebuilds them from their neighbours (Chapter 18); HyperSL hides 142 of a pixel's own 149 bands and rebuilds them from the other 7 of the *same* pixel. Chapter 48 is about what that does to anomalies.

- **The tokenizer convolves over band index** (step 1): band i's token mixes bands $i-2 \dots i+2$, $i-4 \dots i+4$ and $i-6 \dots i+6$ whatever their wavelengths. Section 47.3 measures what that means on a band list with gaps.

- **The output is squashed into** $(0, 1)$ (step 12). That only makes sense if every input lies in $[0, 1]$, which is what the per-scene min-max scaling of Section 46.3 guarantees.

46.2.2 The wavelength code

Definition 46.2 (Sinusoidal position code)**.** A *position code* adds to each token a fixed vector that says where the token sits. HyperSL's says which wavelength a band token stands for. With $p = (\lambda - 400)/10$ and frequencies ω_j, entry $2j$ of the code is $\sin(p\,\omega_j)$ and entry $2j+1$ is $\cos(p\,\omega_j)$, for $j = 0, \dots, 127$.

step	operation	shape	where in `model.py`
0	min-max scaled spectrum and its wavelengths	`(1, 1, 149)`, `(1, 149)`	input, scaled by the caller
1	three 1-D convolutions over band index, kernels 5, 9, 13, stride 1, `padding='same'`; BatchNorm; GELU	$3 \times$ `(1, 256, 149)`	`embedding_layer_1..3`
2	concatenate, then a 1×1 convolution $768 \to 256$; BatchNorm; GELU	`(1, 256, 149)`	`embedding_layer_4`
3	transpose to tokens: one token per band	`(1, 149, 256)`	`x.transpose(2,1)`
4	add the wavelength code $\sin$ / $\cos$ of $(\lambda - 400)/10$	`(1, 149, 256)`	`pos_tokens`
5	keep a random 5%: `int(149*0.05)` = 7 tokens	`(1, 7, 256)`	`random_masking`
6	prepend a learned global token	`(1, 8, 256)`	`global_token`
7	7 transformer blocks, then one more layer that also returns its keys and values	`(1, 8, 256)`	`encoder_blocks`, `transformer_encoder_layer`
8	embedding $\mathbf{z}$ = global token + mean of the 7 band tokens	`(1, 1, 256)`	`aggregate`
9	cross-attention from $\mathbf{z}$ back onto the 7 kept tokens	`(1, 7, 256)`	`corss_attn`
10	append 142 mask tokens, unshuffle, add the wavelength code again	`(1, 149, 256)`	`decoder_forward`
11	3 transformer blocks; LayerNorm	`(1, 149, 256)`	`decoder_blocks`
12	1-D convolution $256 \to 1$, kernel 3; sigmoid	`(1, 1, 149)`	`decoder_layer`

Table 46.1: One pixel through HyperSL, shapes from the code. The reconstruction at step 12 covers all 149 bands, visible and hidden, and passes through a sigmoid, so every output lies between 0 and 1.

The code computes the frequencies as `div_term=exp(arange(0,256,2)*-(log(100.0)/256))`, that is

$$\begin{aligned}\omega_j &= \exp\left(2j \cdot \left(-\frac{\ln 100}{256}\right)\right) && \text{the line above, with } 2j \text{ for } \texttt{arange(0,256,2)}\\ &= \exp(\ln 100)^{-2j/256} && \text{since } e^{-ab} = (e^a)^{-b}\\ &= 100^{-2j/256}. && \end{aligned} \tag{46.1}$$

So $\omega_0 = 100^0 = 1$, $\omega_1 = 100^{-2/256} = 0.964662$, and $\omega_{127} = 100^{-254/256} = 0.010366$ [C].

Worked example 46.1: The wavelength code at 450 nm

$$\begin{aligned} p &= (450 - 400)/10 = 50/10 = 5.0,\\ \text{entry } 0 &= \sin(5.0 \times 1) = \sin 5.0 = -0.9589,\\ \text{entry } 1 &= \cos(5.0 \times 1) = \cos 5.0 = +0.2837,\\ \text{entry } 254 &= \sin(5.0 \times 0.010366) = \sin 0.05183 = +0.0518. \end{aligned}$$

The fastest pair turns through 5.0 radians, the slowest through 0.052: the slow entries tell 450 nm from 2400 nm ($p = 200$, entry 254 is $\sin 2.0732 = +0.8764$), the fast ones tell neighbours apart [C]. Any wavelength has a code, so the model can be fed any band list.

Source check: the base is 100 in the code, 10,000 in the paper

The paper's equations (5) and (6) divide $w_i - 400$ by $10\,000^{2j/d}$ [21]. The code divides by `100` raised to that power (`math.log(100.0)`). The released weights were trained with the code, so the code is right for them; feeding the paper's frequencies to the checkpoint would be off-distribution. A second check: the wavelengths must be in nanometres. In micrometres, 1.0 μm gives $p = (1.0 - 400)/10 = -39.9$, and the validation run measured the loss rising from 0.0024 to 0.173 on EnMAP [M: `2d893e6`: summary.md, step 2, row G].

Source check: the last encoder layer has no skip connection

`TransformerEncoderLayer.forward` reads `x,k,v=self.attn(self.norm1(x))` and then `x=x+self.drop_path(x)`. The second `x` is the attention output, not the layer's input, so the line doubles the attention output and the usual residual (skip) connection of Chapter 16 is lost in this one layer. The weights were trained this way, so "fixing" it would break the checkpoint. It does not change any conclusion in this part; it is the kind of thing you only find by reading the source.

46.2.3 Counting the parameters, by hand

Worked example 46.2: HyperSL's 9,027,841 trainable parameters

A linear layer from i to o numbers has $i \times o$ weights and o biases; a LayerNorm or BatchNorm over D numbers has $2D$ (a scale and a shift). With $D = 256$ and the feed-forward width

$4D = 1024$, one transformer block is

$$\begin{aligned}
\text{two LayerNorms} &= 2 \times (2 \times 256) = 1{,}024,\\
\text{query, key, value} &= 256 \times 768 + 768 = 196{,}608 + 768 = 197{,}376,\\
\text{output projection} &= 256 \times 256 + 256 = 65{,}792,\\
\text{feed-forward} &= (256 \times 1024 + 1024) + (1024 \times 256 + 256)\\
&= 263{,}168 + 262{,}400 = 525{,}568,\\
\text{block} &= 1{,}024 + 197{,}376 + 65{,}792 + 525{,}568 = 789{,}760.
\end{aligned}$$

The encoder has 8 such layers and the decoder 3:

$$\begin{aligned}
\text{encoder} &= 8 \times 789{,}760 = 6{,}318{,}080,\\
\text{decoder} &= 3 \times 789{,}760 = 2{,}369{,}280,\\
\text{cross-attention} &= 2 \times (256 \times 256 + 256) = 131{,}584,\\
\text{tokenizer convolutions} &= \underbrace{(5 \cdot 256 + 256)}_{1{,}536} + \underbrace{(9 \cdot 256 + 256)}_{2{,}560} + \underbrace{(13 \cdot 256 + 256)}_{3{,}584}\\
&\quad + \underbrace{(768 \cdot 256 + 256)}_{196{,}864} = 204{,}544,\\
\text{tokenizer BatchNorms} &= 4 \times (2 \times 256) = 2{,}048,\\
\text{global and mask tokens} &= 2 \times 256 = 512,\\
\text{two final LayerNorms} &= 2 \times (2 \times 256) = 1{,}024,\\
\text{output convolution} &= 256 \times 3 + 1 = 769,\\
\text{total} &= 6{,}318{,}080 + 2{,}369{,}280 + 131{,}584 + 204{,}544\\
&\quad + 2{,}048 + 512 + 1{,}024 + 769\\
&= 9{,}027{,}841.
\end{aligned}$$

The released checkpoint holds 9,158,021 numbers. The difference, $9{,}158{,}021 - 9{,}027{,}841 = 130{,}180$, is exactly its stored buffers: the position table `pe` ($500 \times 256 = 128{,}000$), `div_term` (128), and the BatchNorms' running means and variances ($4 \times 2 \times 256 = 2{,}048$) with their four step counters ($128{,}000 + 128 + 2{,}048 + 4 = 130{,}180$) [C]. The "∼9.16 M parameters" quoted in the team's notes counts the buffers too.

The detection run of Chapter 48 logged the model's `parameters()` as 9,155,969 [M: `research/experiments/hypersl-anomaly-bench/results.json`]: the trainable 9,027,841 plus `pe` and `div_term`, which the code declares as frozen parameters rather than buffers ($9{,}027{,}841 + 128{,}000 + 128 = 9{,}155{,}969$) [C]. The tokenizer is $(204{,}544 + 2{,}048)/9{,}027{,}841 = 2.29\%$ of the model, and 96% of the tokenizer is its 1×1 fusion convolution: the part that "reads the band list" is tiny [C].

Source check: the class defaults are not the checkpoint

`SpectralSharedEncoder`'s defaults are `embedding_dim=1024`, `encoder_depth=32`, `decoder_depth=16`. The checkpoint loads only into 256 / 8 / 4 with 8 heads, the configuration `upstream/train.py` builds. A strict state-dict load checks the sizes but not the head count, which was confirmed by behaviour: 4 or 16 heads load without error and raise the EnMAP test loss from 0.0024 to 0.087 or 0.008 [M: 2d893e6: summary.md, step 2]. Every upstream downstream script ships with its checkpoint-load lines commented out, so running one unmodified evaluates a randomly initialised network.

46.3 What it was trained on

46.3.1 The data path, as code

`upstream/mutihsi.py` builds the pretraining file. For an EnMAP scene, `read_EnMap` reads the L2A reflectance, multiplies by 10^{-4}, deletes band indices 128 to 134 (the paper's "bad bands [128–134]"), sets nodata to NaN, reshapes the cube to a table of pixels by bands, and calls `self.scale`, which is `sklearn.preprocessing.minmax_scale`. DESIS loses bands 0 to 5. The scaling is applied per *column* of that table, which is per band, over every pixel of the scene:

$$x'_b = \frac{x_b - \min_b}{\max_b - \min_b}, \qquad \min_b,\ \max_b \text{ taken over the scene's pixels in band } b. \tag{46.2}$$

Chapter 47 takes this formula apart. Here note only what it means for training: **HyperSL has never seen a reflectance.** Every spectrum it learned from was rescaled so that, in each band, the darkest pixel of its own scene was 0 and the brightest 1. Water-absorption bands were kept: 217 of EnMAP's 224 bands went in, and all but the first 6 of DESIS's.

Source check: the paper describes the scaling in four words

The paper says "to ensure that the features of each band are on the same scale, we performed layer-by-layer normalization" [21]. It does not say min-max, and it does not say per scene; both are visible only in the code (`mutihsi.py`, and again in every `downstream_*.py` reader). The team's earlier note that "the paper does not mention it" is slightly too strong: it mentions a normalisation, not which one.

46.3.2 What the training set is, and what it is not

claim	what the code and files show
sensors	EnMAP and DESIS L2A only (`train.py` builds two loaders, `'ENMap'` and `'DESIS'`). A GF5 reader exists in `mutihsi.py` and is never called. The paper's 439-band Shanghai airborne set is not in `train.py`.
size	"over 300 million spectral instances" (abstract); "approximately 200 million spectral data points" (section IV-A). The team's reading: about 300 one-megapixel tiles [M: memory note on HyperSL, 2026-10-04].
clouds	"cloud cover near zero"; "overall cloud coverage of the data is below 10%" (paper, Fig. 6 caption). Chapter 50 needs this.
training schedule	10 epochs, AdamW 5×10^{-4}, batch 256 per sensor, mask ratio 0.95, a checkpoint every 5 epochs (`train.py`); the released `10_base_mask95` is epoch 10 [C].
loss	MSE + (1 − cos) over the *whole* reconstructed spectrum, visible bands included (`loss.py`, `MSE_SAM_loss`).
where the data is	the ∼265 GB training corpus is on Baidu Netdisk only; Hugging Face has the 10.5 GB *test* split (about 12 million spectra) [M: memory note on HyperSL].
licence	no LICENSE file in the GitHub repository; the Hugging Face card says MIT [M: memory note on HyperSL, "verify before shipping"].

46.4 Did we have the right model? The reproduction

Before blaming a model you must show you ran it right. The validation (2026-10-02/03) did that in four steps; all numbers below are from `2d893e6:hypersl-validation/summary.md`.

46.4.1 Step 1: provenance

The checkpoint's sha256 is `3d5f1914...a74a68c`, identical to Hugging Face's `10_base_mask95_checkpoint.pt`; the model, data and downstream scripts are byte-identical to upstream `e56de7e` (2026-02-03); the configuration 256 / 8 / 4 / 8 heads matches `train.py`. This book's own script re-hashed the file and read its `epoch` field: 10 [C].

46.4.2 Step 2: the right settings, by elimination

There is no published loss on the test split, so the validation ran nine variants on 20,000 test spectra per sensor (loss MSE + (1 − cos) at mask 0.95, mean of 3 mask seeds; lower is better):

	variant	DESIS loss	EnMAP loss	DESIS SAM	EnMAP SAM
A	**ours: epoch 10, 8 heads, nm**	**0.0050**	**0.0024**	**4.4°**	**2.8°**
B	epoch 5	0.0054	0.0032	4.7°	3.3°
C	epoch 0	0.0066	0.0053	5.5°	4.1°
D	untrained	0.183	0.218	25.4°	26.7°
E	4 heads	0.054	0.087	16.2°	19.2°
F	16 heads	0.012	0.008	7.4°	5.7°
G	wavelengths in μm	0.132	0.173	24.1°	27.9°
H	wavelengths +20 nm	0.0081	0.0073	5.9°	5.3°
I	per-spectrum 0–1 scaling	0.0137	0.0063	6.8°	4.1°

Row A wins every column, and epochs 0, 5, 10 improve in order: the setup is right. Row I matters for Chapter 47: rescaling each spectrum by its *own* minimum and maximum instead of the scene's roughly doubles the loss. A second finding from the same run: with mask ratio 0 (all bands visible) the loss is *worse* than at 0.95 (0.0081 / 0.0073 against 0.0050 / 0.0024), because the model only ever trained at 0.95.

46.4.3 Step 3: the paper's own benchmarks

Zero-shot reconstruction (paper Table III). On Indian Pines, an AVIRIS scene HyperSL never saw, the reproduction was equal to or better than the paper on every measure (SAM 7.86° at mask 0.95 against the paper's 8.43°). On Botswana (Hyperion) MSE was ten times better but SAM 2–3° worse (8.84° against 5.67°), most likely because the authors' wavelength file for Botswana is unpublished.

Unsupervised classification on Chikusei (paper Table XII). Chikusei is a labelled 19-class airborne scene. Embed every labelled pixel (77,592), cluster the embeddings with K-means into 19 groups, map clusters to classes one-to-one by the best assignment, and score:

Definition 46.3 (Overall accuracy, adjusted Rand index)**.** *Overall accuracy* (OA) is the share of pixels whose mapped cluster equals their class. The *adjusted Rand index* (ARI) compares two groupings pair by pair, corrected so that a random grouping scores 0 and an identical one 1 [16].

features + K-means	OA (%) ours / paper	ARI ours / paper
raw spectra	39.60 / 42.69	0.268 / 0.289
PCA	37.68 / 41.11	0.259 / 0.276
HyperSL (band 0–1 input)	**59.60 / 60.23**	**0.5404 / 0.5405**
HyperSL (raw input)	48.19 / –	0.418 / –

HyperSL's row matches the paper to 0.63 OA points and 0.0001 ARI [C]. Look at the last row: the same embeddings from *raw* reflectance lose $59.60 - 48.19 = 11.41$ points [C]. The authors' commented-out recipe in their script feeds raw values, so "the paper's numbers were not produced by that code as written"; they need the per-band scaling. Two rows did not reproduce: agglomerative clustering on HyperSL features reached 62.23 against the paper's 73.43, unexplained; spectral clustering is not comparable because its affinity is unpublished.

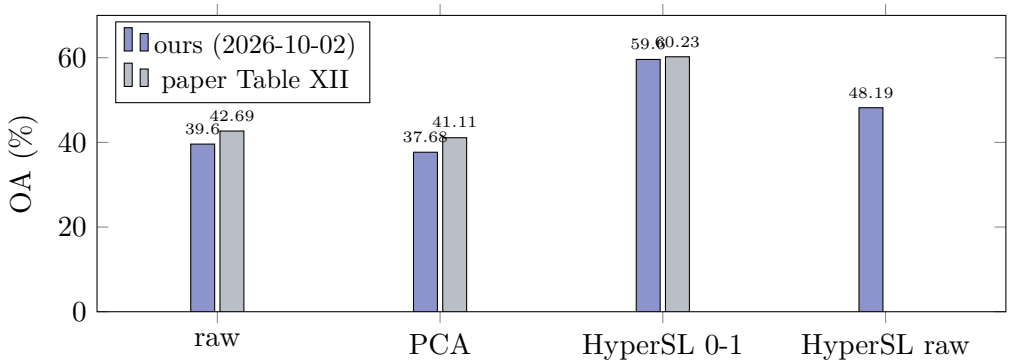


Figure 46.1: Chikusei, K-means on 19 clusters. HyperSL's features add 20 points of overall accuracy over the raw spectra, as the paper says, but only when the input is scaled per band; on raw reflectance the gain halves. Numbers from `2d893e6:hypersl-validation/summary.md`.

Fine-tuning (paper Tables VI and VII). Ten seeds on Colab, the paper's protocol (patch 5, 10% of each class, 150 epochs, AdamW 10^{-4}, weight decay 0.05):

data, initialisation	OA ours	OA paper
Botswana, pretrained	99.01 ± 0.51	99.66 ± 0.22
Botswana, random	97.55 ± 2.13	97.05 ± 1.48
Indian Pines, pretrained	97.73 ± 0.52	98.48 ± 0.39
Indian Pines, random	94.94 ± 3.05	95.04 ± 1.78

Worked example 46.3: How much does pretraining buy?

The gain is pretrained minus random.

$$\begin{aligned} \text{Botswana, ours} &= 99.01 - 97.55 = +1.46, & \text{paper} &= 99.66 - 97.05 = +2.61, \\ \text{ratio} &= 1.46/2.61 = 0.56, \\ \text{Indian Pines, ours} &= 97.73 - 94.94 = +2.79, & \text{paper} &= 98.48 - 95.04 = +3.44, \\ \text{ratio} &= 2.79/3.44 = 0.81. \end{aligned}$$

Pretraining helps on both, as claimed, by 56% and 81% of the paper's gain [C]. The random-initialisation rows match the paper, so data, split and head agree with the authors'; the remaining gap sits in unpublished fine-tuning details. (The summary's own rounding, "+1.45" and "+2.80", differs from these by 0.01 in the last digit.)

46.5 What it is good at

Put the reproduction together and HyperSL's strengths are real and specific.

- **It learns the shape of spectra.** On a scene it never saw, from a sensor it never saw (Indian Pines), it reconstructs 95% hidden bands at 7.9° from seven visible ones.

- **Its embedding groups land cover without labels**, 20 points better than raw spectra on Chikusei.
- **It is a good starting point for supervised classification**, worth 1.5 to 2.8 OA points over random initialisation on two classic scenes.
- **It accepts any band list**, because its position code is a function of wavelength, not of band number. Our own stem has the same property (Chapters 23 and 24); the idea is sound.

All four strengths share a setting: *one scene at a time, classes that cover many pixels, and a frame in which only relative differences inside that scene matter.* Classification of a labelled benchmark scene is exactly that setting.

HyperSL is well built for describing land cover inside one scene. Each of our three jobs asks something it was not built for: absolute reflectance that can be compared with a laboratory (library matching), surprise at a few pixels that do not fit their surroundings (anomaly detection), and recognising a class its training data excluded (cloud).

our job	what it needs from a model	HyperSL's design
library matching	the same spectrum gives the same input in every scene, and so does a library spectrum	input depends on the scene's extremes; a library spectrum has no scene (Chapter 47)
anomaly detection	a pixel must not help predict itself; context from neighbours	predicts a pixel's bands from its own bands; no neighbours (Chapter 48)
unmixing, sub-pixel naming	per-pixel abundances that keep a 0.1-pixel target	deep unmixing head smooths over 3×3 (Chapter 49)
cloud masking	clouds in the training data, brightness kept	trained on near-cloud-free scenes, brightness rescaled away (Chapter 50)

Values that will drift

The checkpoint (`10_base_mask95`, epoch 10), the upstream commit `e56de7e`, the licence status, the test split on Hugging Face and the reproduction numbers are as of 2026-10-04. A later HyperSL release could change any of them.

46.6 Common confusions

HyperSL looks at image patches like Indradhanu.
It looks at one pixel's spectrum. Its "tokens" are bands, and it never sees a neighbour. (Its classification fine-tune uses 5×5 patches, but that is a downstream head, not the pretrained model.)

It has 9.16 million parameters.
9,027,841 trainable; 9,158,021 counting stored buffers, of which 128,000 is the position table.

The wavelength code makes the tokenizer wavelength-aware.
The code is added *after* the convolutions. The convolutions themselves mix neighbouring band indices, whatever their wavelengths (Section 47.3).

Mask 0 (show every band) gives the best embedding.
The model never trained at mask 0; its loss there is worse than at 0.95.

The reproduction proves HyperSL suits our product.
It proves we ran the model the authors trained, as they ran it. Whether it suits a different job is a separate question, answered in the next four chapters.

The paper hides the normalisation.
It names "layer-by-layer normalization"; the code shows it is min-max per band per scene.

46.7 Check yourself

Exercise 46.1. A HyperSL with embedding width $D = 128$ and the same depths (8 encoder layers, 3 decoder blocks, feed-forward $4D$). Count one transformer block's parameters.

Exercise 46.2. Compute entries 0 and 1 of the wavelength code at 1000 nm, and say what p would be if someone passed the wavelength as 1.0 (micrometres).

Exercise 46.3. EnMAP L2A has 224 bands and HyperSL drops 7. At mask ratio 0.95, how many bands does the encoder see? How many for SpectralEarth's 202-band list?

Exercise 46.4. On Chikusei, by how many points does HyperSL beat raw spectra with band 0–1 input, and with raw input? What share of the first gain survives without the scaling?

Exercise 46.5. The checkpoint holds 9,158,021 numbers. Without looking, list the four kinds of stored buffer and their sizes, and check they sum to the difference from 9,027,841.

Exercise 46.6. Why does a sigmoid at the output require the per-scene min-max at the input? What would happen to a reflectance of 1.2 (a specular glint) fed raw?

Next: Chapter 47, Per-scene min-max, and a library with nowhere to stand.

Chapter 47

Per-scene min-max, and a library with nowhere to stand

The one thing this chapter teaches

Per-scene min-max scaling makes a pixel's input depend on the brightest and darkest pixels of its scene, so the same material reads differently in every scene, one bright anomaly changes every other pixel, and a library spectrum, which has no scene, has no defined input at all; measured on four PRISMA scenes, HyperSL's embeddings did not name man-made targets better than plain spectral angle, and one vegetation spectrum absorbed 60% of a scene's anomalies.

Library matching (Section 32.3, Chapter 44) asks whether a pixel looks like a laboratory spectrum. That question has a precondition so basic it is easy to miss: *the pixel and the library spectrum must be put on the same footing before they are compared.* This chapter shows, on numbers small enough to do by hand, that HyperSL's input scaling cannot meet that precondition, and then shows what happened when the team tried anyway.

Analogy

Marks graded on a curve. In each class the weakest student is given 0 and the strongest 100, and everyone else is placed proportionally between. A mark of 75 means one thing in a class of strong students and another in a weak one; a single brilliant newcomer pushes everyone else's mark down; and an external candidate who sat the exam alone has no curved mark at all, because there is no class to curve against. Comparing that candidate with a student in some class means inventing a class for them. The library spectrum is the external candidate.

47.1 The scaling, as a formula

Definition 47.1 (Per-scene, per-band min-max scaling)**.** For a scene with pixels $\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(N)}$ and band b, let $m_b = \min_n x_b^{(n)}$ and $M_b = \max_n x_b^{(n)}$. Each pixel's value in band b becomes

$$x'_b = \frac{x_b - m_b}{M_b - m_b}. \tag{47.1}$$

This is Equation (46.2), the operation `sklearn.preprocessing.minmax_scale` applies to each column of the pixels-by-bands table in `upstream/mutihsi.py`.

Rewrite it to see its shape:

$$\begin{aligned} x'_b &= \frac{x_b - m_b}{M_b - m_b} \\ &= \frac{1}{M_b - m_b}\, x_b - \frac{m_b}{M_b - m_b} && \text{split the fraction} \\ &= s_b\, x_b + c_b, \quad s_b = \frac{1}{M_b - m_b},\ c_b = -\frac{m_b}{M_b - m_b}. && (47.2) \end{aligned}$$

Definition 47.2 (Per-band affine map)**.** A map that multiplies band b by a number s_b and adds a number c_b, with each band allowed its own s_b and c_b, is a *per-band affine map*.

So min-max is a per-band affine map, and so is the fixed z-score our own models use (`WavelengthStats`, Section 24.2): $z_b = (x_b - \mu(\lambda_b))/\sigma(\lambda_b)$, which is Equation (47.2) with $s_b = 1/\sigma(\lambda_b)$ and $c_b = -\mu(\lambda_b)/\sigma(\lambda_b)$.

The two maps have the same form. The difference is where s_b and c_b come from: from the *scene* for min-max, from a *fixed curve* stored in the checkpoint for `WavelengthStats`. Everything in this chapter follows from that one difference.

47.2 A three-pixel scene, by hand

Two bands are enough. Scene S_1 has three pixels:

$$\mathbf{p}_1 = (0.10,\ 0.30), \qquad \mathbf{p}_2 = (0.20,\ 0.50), \qquad \mathbf{p}_3 = (0.30,\ 0.40) \quad [A].$$

Worked example 47.1: Scaling scene S_1

Band 1 holds 0.10, 0.20, 0.30, so $m_1 = 0.10$, $M_1 = 0.30$, $M_1 - m_1 = 0.20$. Band 2 holds 0.30, 0.50, 0.40, so $m_2 = 0.30$, $M_2 = 0.50$, $M_2 - m_2 = 0.20$.

$$\begin{aligned} \mathbf{p}'_1 &= \left(\frac{0.10 - 0.10}{0.20},\ \frac{0.30 - 0.30}{0.20}\right) = (0,\ 0), \\ \mathbf{p}'_2 &= \left(\frac{0.20 - 0.10}{0.20},\ \frac{0.50 - 0.30}{0.20}\right) = \left(\frac{0.10}{0.20},\ \frac{0.20}{0.20}\right) = (0.5,\ 1.0), \\ \mathbf{p}'_3 &= \left(\frac{0.30 - 0.10}{0.20},\ \frac{0.40 - 0.30}{0.20}\right) = \left(\frac{0.20}{0.20},\ \frac{0.10}{0.20}\right) = (1.0,\ 0.5). \end{aligned}$$

47.2.1 (a) The same spectrum, two different inputs

Scene S_2 contains the very same spectrum $(0.20, 0.50)$, beside two others: $(0.05, 0.20)$ and $(0.40, 0.60)$ [A].

Worked example 47.2: The same spectrum in scene S_2

Band 1: $m_1 = 0.05$, $M_1 = 0.40$, $M_1 - m_1 = 0.35$. Band 2: $m_2 = 0.20$, $M_2 = 0.60$, $M_2 - m_2 = 0.40$.

$$\begin{aligned} x'_1 &= \frac{0.20 - 0.05}{0.35} = \frac{0.15}{0.35} = 0.4286, \\ x'_2 &= \frac{0.50 - 0.20}{0.40} = \frac{0.30}{0.40} = 0.75. \end{aligned}$$

In S_1 the spectrum $(0.20, 0.50)$ entered the model as $(0.5, 1.0)$; in S_2 as $(0.4286, 0.75)$ [C].

One material, two inputs. A model fed these will produce two different embeddings for the same surface, and nothing in its training told it the two are the same thing: the scenes' extremes, not the material, decided the difference.

47.2.2 (b) One bright anomaly moves everybody

Add a bright fourth pixel, $\mathbf{a} = (0.60, 0.90)$, to S_1 [A]: a metal roof, say.

Worked example 47.3: Scene S_1 after a bright pixel arrives

The minima do not change ($m_1 = 0.10$, $m_2 = 0.30$). The maxima become $M_1 = 0.60$ and $M_2 = 0.90$, so the spans are $0.60 - 0.10 = 0.50$ and $0.90 - 0.30 = 0.60$.

$$\mathbf{p}_2' = \left(\frac{0.20 - 0.10}{0.50},\ \frac{0.50 - 0.30}{0.60}\right) = \left(\frac{0.10}{0.50},\ \frac{0.20}{0.60}\right) = (0.2,\ 0.3333),$$
$$\mathbf{p}_3' = \left(\frac{0.30 - 0.10}{0.50},\ \frac{0.40 - 0.30}{0.60}\right) = \left(\frac{0.20}{0.50},\ \frac{0.10}{0.60}\right) = (0.4,\ 0.1667),$$
$$\mathbf{a}' = \left(\frac{0.50}{0.50},\ \frac{0.60}{0.60}\right) = (1,\ 1).$$

$\mathbf{p}_2$ was $(0.5, 1.0)$ and is now $(0.2, 0.3333)$; $\mathbf{p}_3$ was $(1.0, 0.5)$ and is now $(0.4, 0.1667)$ [C]. Not one of their reflectances changed.

For detection this is perverse. The arrival of an anomaly rewrites the input of every *ordinary* pixel in the scene, and the anomaly itself is pinned to the corner $(1, 1)$ of the box the model was trained in. A real scene has a million pixels and the extremes are usually set by a handful of them: glint, a metal roof, a fire, a cloud. Those handful decide the frame for the million.

47.2.3 (c) A library spectrum has no frame

A laboratory spectrum $\mathbf{t} = (0.70, 0.20)$ [A] was measured alone. It has no scene, so Equation (47.1) has no m_b and M_b to use. There are three ways to force it in, and each fails.

Borrow the scene's frame. Use S_1's m and M:

$$t_1' = \frac{0.70 - 0.10}{0.20} = \frac{0.60}{0.20} = 3.0, \qquad t_2' = \frac{0.20 - 0.30}{0.20} = \frac{-0.10}{0.20} = -0.5.$$

Both lie outside $[0, 1]$, the range every training input occupied, and outside the range the sigmoid output can reach (Table 46.1). Clipping gives $(1, 0)$ and throws away how far outside it was. On real data this is not rare: the team's first attempt clipped 41% of all ECOSTRESS library values, and the clipping fell mostly on natural entries [M: `4bbf91d`: hypersl_model.py docstring].

Put the spectrum into the scene. Append $\mathbf{t}$ to S_1 as a fourth pixel and recompute. Now $m = (0.10, 0.20)$ and $M = (0.70, 0.50)$, $\mathbf{t}' = (1, 0)$, and $\mathbf{p}_1$ moves from $(0, 0)$ to $(0, 0.3333)$ [C]. The library entry has changed the scene it is being compared against. With 2,000 library entries there would be 2,000 different frames.

Scale each spectrum by itself. Use each spectrum's own minimum and maximum over its bands ("per-curve"). That treats pixel and library alike, but it is not what the model was trained on: the validation measured the loss roughly doubling under it (row I in Section 46.4).

Definition 47.3 (Circular)**.** A procedure is *circular* when its answer is needed to set up its own input. Borrowing the scene's frame is circular for matching: a scene's extremes in each band are typically set by its unusual pixels, the very ones being named, so the frame the library is placed in depends on what the target is.

47.2.4 (d) Spectral angle does not survive the map

Definition 47.4 (Spectral angle)**.** The angle between two spectra $\mathbf{u}$ and $\mathbf{v}$ (Chapter 7, Chapter 32) is

$$\theta = \arccos \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \, \|\mathbf{v}\|}.$$

It ignores overall brightness: $\theta(k\mathbf{u}, \mathbf{v}) = \theta(\mathbf{u}, \mathbf{v})$ for any $k > 0$. That is why library matching uses it.

A tempting hope: the angle ignores brightness, so perhaps it ignores the min-max map too. It does not.

Worked example 47.4: The angle between $\mathbf{p}_2$ and $\mathbf{p}_3$, before and after

Before, in reflectance:

$$\begin{aligned}
\mathbf{p}_2 \cdot \mathbf{p}_3 &= 0.20 \times 0.30 + 0.50 \times 0.40 = 0.06 + 0.20 = 0.26, \\
\|\mathbf{p}_2\| &= \sqrt{0.20^2 + 0.50^2} = \sqrt{0.04 + 0.25} = \sqrt{0.29} = 0.5385, \\
\|\mathbf{p}_3\| &= \sqrt{0.30^2 + 0.40^2} = \sqrt{0.09 + 0.16} = \sqrt{0.25} = 0.5000, \\
\cos\theta &= 0.26/(0.5385 \times 0.5000) = 0.26/0.26926 = 0.9656, \qquad \theta = 15.07^\circ.
\end{aligned}$$

After, in S_1's frame, $\mathbf{p}_2' = (0.5, 1.0)$ and $\mathbf{p}_3' = (1.0, 0.5)$:

$$\begin{aligned}
\mathbf{p}_2' \cdot \mathbf{p}_3' &= 0.5 \times 1.0 + 1.0 \times 0.5 = 1.0, \\
\|\mathbf{p}_2'\| &= \|\mathbf{p}_3'\| = \sqrt{0.25 + 1.0} = \sqrt{1.25} = 1.1180, \\
\cos\theta' &= 1.0/(1.1180 \times 1.1180) = 1.0/1.25 = 0.8000, \qquad \theta' = 36.87^\circ.
\end{aligned}$$

Two spectra 15.07° apart in reflectance are 36.87° apart after the map [C]. And $\mathbf{p}_1$, the darkest pixel in both bands, became $(0, 0)$: a vector of length zero, for which no angle is defined at all.

Why, in general? Take a per-band map with no offset, $x_b' = s_b x_b$. Then

$$\begin{aligned}
\cos\theta' &= \frac{\sum_b (s_b u_b)(s_b v_b)}{\sqrt{\sum_b s_b^2 u_b^2}\,\sqrt{\sum_b s_b^2 v_b^2}} && \text{substitute } u_b' = s_b u_b,\ v_b' = s_b v_b \\
&= \frac{\sum_b s_b^2 u_b v_b}{\sqrt{\sum_b s_b^2 u_b^2}\,\sqrt{\sum_b s_b^2 v_b^2}}. && \text{collect } s_b \cdot s_b
\end{aligned}$$

If every s_b equals one number s, then s^2 comes out of each sum, $s^2/\sqrt{s^2}\sqrt{s^2} = 1$, and $\cos\theta' = \cos\theta$. If the s_b differ, each band's product is weighted by its own s_b^2, and the angle changes. With $s = (1/0.20,\ 1/0.10)$, the pair above goes from 15.07° to 9.25° [C]. An offset c_b changes it again, because $\mathbf{u} + \mathbf{c}$ is no longer a multiple of anything related to $\mathbf{u}$. Min-max has both unequal scales and offsets.

The fixed z-score changes angles too: it is also a per-band affine map. What it does not do is change them *differently in each scene*. A spectrum's z-scores are a property of the spectrum alone, identical for an image pixel in any scene and for a library entry; that, not

angle preservation, is what matching needs.

47.3 A convolution over band index

The second obstacle sits in the tokenizer. Recall the convolutions of Table 46.1, step 1.

Definition 47.5 (1-D convolution over band index; kernel; span)**.** A 1-D convolution with *kernel* size k makes band i's output from bands $i-h,\dots,i+h$ with $h=(k-1)/2$, using the same k weights at every i. It knows the bands' *positions in the list*, not their wavelengths. The *span* of the window at i is $\lambda_{i+h}-\lambda_{i-h}$, the wavelength range its k bands cover.

The same k weights are used everywhere, so the convolution assumes that "two places along" means the same wavelength distance everywhere. On a list with roughly even spacing that is nearly true. On a list with a water-absorption window removed it is false at the edges of the window.

Worked example 47.5: A 5-band window on PRISMA's 149-band list

PRISMA's 149-band list (the one every HyperSL experiment in this part used) drops 1325–1495 nm. Its bands just below and above that gap are, in nm [C]:

below: …, 1250.98, 1262.53, 1273.50, 1284.49, 1295.42, 1306.22, 1317.26,
above: 1502.02, 1512.63, 1523.22, …

Window centred on 1273.50, five bands 1250.98 to 1295.42:

$$\text{span} = 1295.42 - 1250.98 = 44.44 \text{ nm}.$$

Window centred on 1317.26, five bands 1295.42, 1306.22, 1317.26, 1502.02, 1512.63. Its four steps are

$$\begin{aligned} 1306.22 - 1295.42 &= 10.80,\\ 1317.26 - 1306.22 &= 11.04,\\ 1502.02 - 1317.26 &= 184.76,\\ 1512.63 - 1502.02 &= 10.61, \end{aligned}$$

and its span is $1512.63 - 1295.42 = 217.21$ nm [C]. The kernel's third and fourth weights, which elsewhere connect two bands 10 nm apart, here connect 1317 nm to 1502 nm, across the 1.4 μm water band.

Table 47.1 gives the spans for the three kernels on four band lists, and Figure 47.1 draws the 5-band span along PRISMA's 149 bands.

Source check: the authors saw the effect and drew it

The paper's Fig. 4 caption notes that "the nonlinear segments in the figure are due to the removal of bad bands from the ENMap data" [21]. It refers to the position code, which knows wavelength and handles a gap correctly. The convolutions in front of it do not; the code (`nn.Conv1d(1,256,k,stride=1,padding='same')`) has no wavelength input.

Our own stem answers this differently: it groups bands into 40 nm wavelength *bins* (Chapter 24), so a band that is absent is simply absent, and no weight ever connects 1317 nm to 1502 nm as if they were neighbours.

band list	bands	$k = 5$: median / max	$k = 9$: max	$k = 13$: median / max
PRISMA, all 234 good bands (water kept)	234	36.4 / 45.0	88.7	108.9 / 132.7
PRISMA, 149 (water and noisy ranges cut)	149	39.7 / 280.4	319.4	122.5 / 358.9
EnMAP via SpectralEarth (water cut)	202	31.0 / 291.6	335.0	93.1 / 379.2
product 165-band grid	165	40.0 / 200.0	240.0	120.0 / 280.0

Table 47.1: How many nanometres (nm) a kernel over band index covers, for windows that fit inside the list [C]. With the water bands kept (first row; HyperSL trained on EnMAP this way) the spans are even: the design is consistent with its own data. Every band list this product uses removes the water windows, and the largest span grows six- to sevenfold.

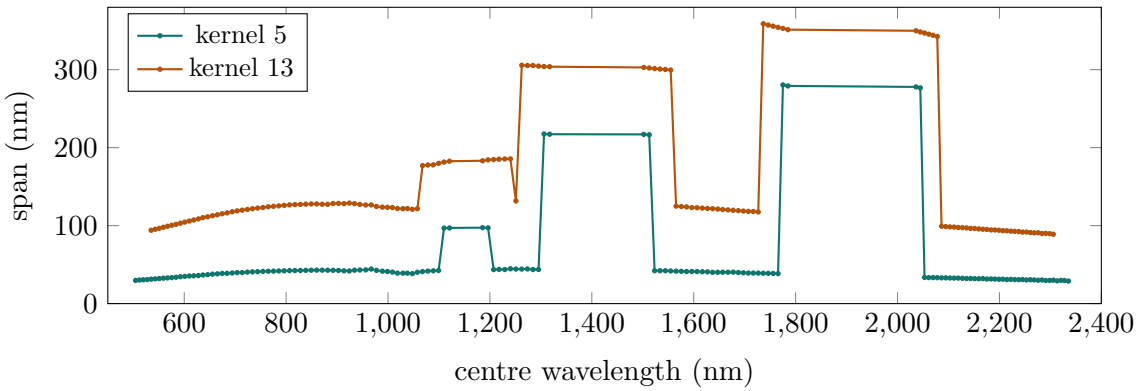


Figure 47.1: The span of HyperSL's kernels along PRISMA's 149-band list. Away from the gaps a 5-band window covers about 40 nm; next to the three removed ranges (1120–1186, 1317–1502 and 1785–2036 nm) it jumps to 100–280 nm. Data: `figdata/hsl_kernel_span_prisma149.dat`, written by `tools/compute_hypersl.py`.

47.4 What happened when it was tried

47.4.1 The four-scene matching run (2026-09-23)

The experiment `research/experiments/hypersl-splib-match/` (commits `f554fe4` to `4bbf91d`, no longer in the working tree) matched every ground-truth anomaly pixel of the four labelled PRISMA scenes against two libraries: ECOSTRESS (1,947 entries, 45 of them man-made) and USGS splib07 (518 entries)
[M: the run's `summary.json`, in `C:/Users/FITT/Documents/hypersl_match_data/results/`, 2026-09-23 22:02].
Three methods, each ranked by cosine similarity:

- **HyperSL**: the embedding of pixel and library entry, each scaled *per curve* (its own minimum and maximum over bands), mask 0. Per-curve scaling was chosen because the scene frame clipped 41% of the library and because a spectrum cannot find itself across two different scalings (the control `frame_control.py`);
- **HyperSL raw**: the embedding of unscaled reflectance;
- **SAM**: the cosine of the reflectances themselves, the classical reference.

Did anomalies match man-made entries? The measure is the share of anomaly pixels whose best ECOSTRESS match is a man-made entry, beside the same share for brightness-matched background pixels. A useful matcher should put the first well above the second.

	HyperSL, per-curve		HyperSL, raw		SAM	
scene (anomaly px)	anomaly	background	anomaly	background	anomaly	background
set1 (638)	7.2%	2.7%	0.16%	0.0%	9.6%	1.9%
set2 (992)	0.40%	0.0%	0.30%	0.0%	0.81%	0.20%
set3 (778)	0.26%	0.51%	0.39%	0.13%	2.7%	2.1%
set4 (1,165)	8.3%	33.0%	0.0%	0.0%	6.4%	24.3%

Table 47.2: Share of pixels whose top ECOSTRESS match is man-made [M: `summary.json`, `m1`]. HyperSL is below SAM on three of four scenes, and on set4 both call a third or a quarter of ordinary background man-made.

Where did the matches go? The run also recorded the single library entry that took the most anomaly pixels as its top match. With HyperSL per-curve, *Salvia leucophylla 3*, a sage leaf, was the best match for 59.6% of set2's 992 anomaly pixels, and *Dolomitic Marble* for 31.2% of set4's. Against USGS, a buckbrush spectrum took 52.5% of set2's. SAM's largest attractor on ECOSTRESS took 19.9% (set2, *Misc Grass Litter*) [M: `summary.json`, `top1_share`]. An embedding in which one leaf is the nearest neighbour of most of a scene's anomalies is describing the scene, not the anomalies.

Does the embedding at least organise the library? For each library entry, take its 5 nearest other entries and count the share in its own class:

	ECOSTRESS		USGS	
space	all classes	man-made	all classes	man-made
HyperSL, per-curve	0.965	0.124	0.827	0.373
HyperSL, raw	0.962	0.196	0.841	0.454
SAM (reflectance)	0.949	0.093	0.795	0.278

To be fair to HyperSL: its space groups library classes a little *better* than raw angle, by 1.6 to 4.6 points overall and 3 to 18 points for man-made entries [M: `summary.json, purity`]. But look at the man-made column on ECOSTRESS: 12% means that seven out of eight of a man-made entry's nearest neighbours are natural. Vegetation, minerals and soils dominate the library and the embedding; man-made materials are not a region of it.

Source check: why the USGS rows of the man-made table are all zero

The run's man-made test is `e.chapter=="manmade"`. USGS splib07 calls its man-made chapter "artificial", so the test is false for every USGS entry and every USGS share is 0 by construction (the library summary even reports `"manmade":0`). The purity measure uses `np.isin(cls,["manmade","artificial"])` and counts them. So read the USGS purity, and ignore the USGS man-made shares; Table 47.2 shows ECOSTRESS only for this reason.

47.4.2 Band sampling moves the embedding as much as material does

One more measurement shaped the decision, and it is the only one in this part whose raw output was not kept. On 2026-09-23 an ECOSTRESS entry fed at its own laboratory sampling found its *own* PRISMA-resampled copy as nearest neighbour only 28% of the time [M: `docs/proposals/spectral-matching-foundation-model.md`]. The proposal draws the conclusion: "that is the property a matching model must not have, and the reason not to start from HyperSL's checkpoint". The cloud-mask experiment of Chapter 50 took the lesson seriously and resampled every training spectrum onto PRISMA's exact bands for that reason.

Source check: the 28% has no output file

The figure is quoted in the proposal (line 63) and in the team's memory note, both citing the 2026-09-23 run. The experiment's committed scripts do not save it, and the run's `summary.json` does not contain it. Treat it as recorded by the people who ran it, not as re-checkable. The related control in `frame_control.py` (can a pixel find itself across two scalings?) printed its result to the screen and did not save it either.

47.5 What we do instead

requirement	HyperSL	this product
one input per spectrum, in every scene	min-max of the scene: input depends on the scene's extremes	`WavelengthStats`: one mean and spread curve over wavelength, fixed in the checkpoint
a library spectrum has an input	none; must borrow a frame (clips 41%) or change the scaling	the same curve read at the library's wavelengths, the same stem (Chapter 44)
brightness survives	removed per band, per scene	kept; the namer and ACE see reflectance
gaps in the band list	convolution over index joins 1317 and 1502 nm	40 nm wavelength bins; an absent band is absent

Values that will drift

The four-scene matching numbers belong to the 2026-09-23 run (ECOSTRESS 1,947 entries, USGS 518, PRISMA 149-band axis), whose outputs live outside the repository. The span table depends on the band lists: PRISMA's from Set-1's HE5, the SpectralEarth table, and

the product grid as of 2026-10-31.

47.6 Common confusions

Min-max only changes brightness, and the spectral angle ignores brightness.
It changes each band by its own scale and offset; the angle changes (15.07° became 36.87° in worked example 47.4).

Then our z-score is just as bad.
It is also affine, so it changes angles too, but identically in every scene and for every library entry. Scene-dependence, not affinity, is the problem.

Scaling each spectrum by itself fixes it.
It treats pixel and library alike, which is why the matching run used it, but it is off the training distribution (the loss roughly doubles) and it erases brightness entirely.

The position code tells the tokenizer about the gap.
The code is added after the convolutions; the convolutions have already mixed 1317 and 1502 nm.

HyperSL's embedding is worse than SAM at grouping the library.
It is slightly better (0.965 against 0.949). It is worse at the job that matters: anomalies matched to man-made entries.

The USGS man-made rate of 0% means HyperSL never matched USGS man-made entries.
It is 0 by construction: the label test looked for "manmade" and USGS calls the chapter "artificial".

47.7 Check yourself

Exercise 47.1. Scene S_3 has pixels $(0.15, 0.25)$, $(0.20, 0.50)$, $(0.25, 0.55)$. Compute the min-max input of $(0.20, 0.50)$ and compare with worked examples 47.1 and 47.2.

Exercise 47.2. In S_1 plus the bright pixel $\mathbf{a}$, what is $\mathbf{p}_1$'s input? Why did it not move?

Exercise 47.3. Compute the angle between $\mathbf{p}_2 = (0.20, 0.50)$ and $\mathbf{p}_3 = (0.30, 0.40)$ after the scaling $s = (1/0.20,\ 1/0.10)$ with no offset. (The script gives 9.25°.)

Exercise 47.4. Show that with $s_1 = s_2 = 5$ and no offset the angle between any two 2-band spectra is unchanged.

Exercise 47.5. On the product's 165-band grid the bands are 10 nm apart except at the gaps; the 1790–1960 nm gap removes the bands between. Find the span of the 5-band window whose centre is 1780 nm, and the 13-band window centred at 1740 nm.

Exercise 47.6. A library spectrum is $(0.70, 0.20)$. Write its fixed z-score with mean $(0.25, 0.35)$ and spread $(0.10, 0.12)$. Does the answer depend on which scene you are matching against?

Next: Chapter 48, Anomaly detection: a model that completes the anomaly.

Chapter 48

Anomaly detection: a model that completes the anomaly

The one thing this chapter teaches

HyperSL rebuilds a pixel's hidden bands from that same pixel's visible bands, so it learns to complete whatever spectrum it is shown, anomalies included; measured on the 909-object PRISMA bench with the evaluator's own measures, its residual found 130 of the 909 objects in its top 1% (random scores find about 93), and even scored with our own residual Mahalanobis its reconstruction made the score worse than using no reconstruction at all (pAUC 0.083 against 0.128).

Chapter 9 built anomaly detection on one rule: *a pixel must never help reconstruct itself.* Indradhanu obeys it by hiding whole pixels and rebuilding them from their neighbours (Chapter 18). HyperSL's masking hides bands of a pixel and rebuilds them from the rest of the same pixel. This chapter works out, on a three-band toy, why that is the opposite of what a detector needs, and then measures it, because a theory about a nine-million-parameter network is only a prediction until it is run.

Analogy

A cloze test, in which a reader fills in the missing words of a sentence. A good reader fills them in for *any* sentence, however strange, because the rest of the sentence says what is missing: "the giraffe ordered a _ of tea" is completed as easily as "the man ordered a cup of tea". If you want to know whether a sentence is odd, a fluent cloze reader is the wrong judge: it repairs the oddness. Ask instead whether the sentence fits the paragraph it is in. HyperSL is the fluent cloze reader of spectra; the paragraph is the pixel's neighbourhood.

48.1 Two ways to predict a pixel

Definition 48.1 (Spatial completion, spectral self-completion)**.** *Spatial completion* predicts a pixel's spectrum from *other* pixels, its neighbours. *Spectral self-completion* predicts some of a pixel's bands from its *other bands.* Indradhanu does the first; HyperSL does the second (Table 46.1, step 5).

A reconstruction detector scores a pixel by how badly it is predicted (Chapter 9). For that to flag an anomaly, the predictor must *fail* on it. Spatial completion fails on an anomaly because its neighbours are background: they carry no information about the anomaly. Self-completion uses the anomaly's own bands, which carry a great deal.

48.1.1 A three-band toy, by hand

Take spectra of three bands. Ordinary background is *smooth*: the middle band lies halfway between the outer two.

$$\begin{aligned} \text{background } \mathbf{b} &= (0.20,\ 0.30,\ 0.40)\ [A], \\ \text{smooth anomaly } \mathbf{t} &= (0.60,\ 0.50,\ 0.40)\ [A] \quad \text{(a bright roof falling with wavelength)}, \\ \text{anomaly with a dip } \mathbf{d} &= (0.60,\ 0.30,\ 0.60)\ [A] \quad \text{(an absorption feature in band 2)}. \end{aligned}$$

The self-completer. Hide band 2 and predict it from bands 1 and 3 of the same pixel. What predictor would training on background learn? Fit $\hat{x}_2 = w_1 x_1 + w_3 x_3 + c$ by least squares on many background spectra. If every background spectrum is exactly smooth, $x_2 = (x_1 + x_3)/2$, then $w_1 = w_3 = \frac{1}{2}$, $c = 0$ gives zero error on every one of them, and since a squared error cannot be negative, nothing does better. On 2,000 noisy smooth backgrounds the fit gives $w_1 = 0.4981$, $w_3 = 0.5011$, $c = -0.00018$ [C] (offsets and slopes drawn at random, noise 0.005 [A]): the average, as predicted.

Worked example 48.1: Self-completion of band 2

With $\hat{x}_2 = (x_1 + x_3)/2$:

$$\begin{aligned} \text{background:} \quad & \hat{x}_2 = (0.20 + 0.40)/2 = 0.30, \qquad r_2 = 0.30 - 0.30 = 0, \\ \text{smooth anomaly:} \quad & \hat{x}_2 = (0.60 + 0.40)/2 = 0.50, \qquad r_2 = 0.50 - 0.50 = 0, \\ \text{dip:} \quad & \hat{x}_2 = (0.60 + 0.60)/2 = 0.60, \qquad r_2 = 0.30 - 0.60 = -0.30. \end{aligned}$$

The learned predictor gives 0.4991 on the smooth anomaly (true 0.50) and 0.5994 on the dip (true 0.30) [C]. The smooth anomaly, three times brighter than the background in band 1, has *the same residual as the background: zero.*

The spatial completer. Predict every band of the pixel as the mean of its four neighbours, which are background.

Worked example 48.2: Spatial completion of the same pixels

$$\begin{aligned} \text{background:} \quad & \mathbf{r} = \mathbf{b} - \mathbf{b} = (0, 0, 0), \qquad \|\mathbf{r}\|^2 = 0, \\ \text{smooth anomaly:} \quad & \mathbf{r} = (0.60 - 0.20,\ 0.50 - 0.30,\ 0.40 - 0.40) = (0.40,\ 0.20,\ 0), \\ & \|\mathbf{r}\|^2 = 0.16 + 0.04 + 0 = 0.20, \\ \text{dip:} \quad & \mathbf{r} = (0.60 - 0.20,\ 0.30 - 0.30,\ 0.60 - 0.40) = (0.40,\ 0,\ 0.20), \\ & \|\mathbf{r}\|^2 = 0.16 + 0 + 0.04 = 0.20. \end{aligned}$$

Both anomalies stand out, by the same amount, whatever their shape [C].

A self-completer flags only the part of an anomaly it cannot infer from the anomaly itself: a feature narrower than the spacing of the bands it can see. Everything broad about the anomaly (its brightness, its slope, its overall shape) is completed, and the residual is the background's.

48.1.2 What this means for HyperSL

HyperSL is a far more flexible self-completer than an average: a transformer that learned, from 200–300 million spectra, what bands go with what. At mask 0.95 it sees 7 of 149 bands, chosen at random; those 7 carry the anomaly's overall level and shape, and the other 142 are filled in to match. Three details push it further towards completion.

- **Its loss covers visible bands too** (`MSE_SAM_loss` on the whole output), so copying what it sees is rewarded, the identity trap of Section 9.3.

- **The min-max frame pins an anomaly to the edge of** $[0, 1]$ (worked example 47.3), where a sigmoid output is comfortable.

- **It has no neighbours to disagree with.** Even a perfect model of "what spectra look like" cannot know that this spectrum does not belong *here*.

The first evidence (2026-09-23). The matching run of Section 47.4 also measured HyperSL's own reconstruction error (angle between pixel and reconstruction, mask 0, per-curve scaling) on the ground-truth anomaly pixels and on random background:

scene	anomaly, median angle	background, median angle	AUC (anomaly above background)
set1	3.18°	6.17°	0.094
set2	4.02°	5.16°	0.220
set3	3.26°	4.27°	0.242
set4	2.74°	3.99°	0.104

An AUC below 0.5 means the anomalies were reconstructed *better* than the background, on every scene; the mean is 0.165 [C] [M: `summary.json`, `recon`]. That run showed HyperSL every band (mask 0), its weakest setting, and scaled per curve, off its training distribution. The run below fixes both.

48.2 The measurement on the labelled bench (2026-10-31)

For this book the question was put to HyperSL directly, under the conditions it was trained for, and scored exactly as Indradhanu is scored. The experiment is `research/experiments/hypersl-anomaly-bench/` (README, scripts and `results.json`); its score maps are outside the repository in `C:/Users/FITT/Documents/stage1_datasets/hypersl_anomaly_bench/`.

item	setting
data	the 4 labelled PRISMA L2D scenes, 909 objects; validity and object maps exported from the evaluator's own `PrismaBench` (`step0_export_masks.py`), so every method is counted on the same pixels
HyperSL input	the 149-band PRISMA axis of every earlier HyperSL experiment; each band min-max scaled over the scene's valid pixels, as in `mutihsi.py`
model	`10_base_mask95` (sha256 `3d5f1914...a74a68c`), mask ratio 0.95 (7 of 149 bands visible), 4 random masks per pixel (seeds 1000–1003), fp16 on an RTX 4000 SFF Ada in WSL (`step1_hypersl_scores.py`)
reconstruction	each band's prediction averaged over the draws in which it was *hidden*, so every residual is a prediction of a band the encoder did not see
HyperSL scores	**L2**: mean squared residual in min-max units; **loss**: its own training objective MSE + (1 − cos) per pixel; **RM**: the product's residual Mahalanobis (30 components, ring 21/61) on the residual converted back to reflectance
baselines	**FTH3 epoch 5**, Indradhanu's release candidate (its cached score maps from the 2026-10-29 evaluation, found by the evaluator's cache key); **PCA-RX**, the evaluator's `pca_rx` (residual Mahalanobis with a *zero* reconstruction); **MNF-RX**, the product's `MNFCompressionDetector`, 10 components
cloud	every method also scored with FTH3 epoch 5's thick-cloud pixels set to 0, the release candidate's rule
measures	`app.evaluation.metrics` and `significance` (Chapters 33 and 36), via `step2_evaluate.py`

Three checks before the results. FTH3's two rows reproduce its evaluation report exactly (pAUC 0.1749 and 0.1892, 422 and 449 objects) [M: `research/experiments/indradhanu-finetune/FTH3-heavy-joint-more2/eval_report.md`]. PCA-RX gives 0.1281, the stage-3 run's 0.128 [M: `docs/lld/evaluation.md`]. And the masking did what it should:

Worked example 48.3: How many bands should never be hidden in 4 draws?

In one draw a band is visible with probability 7/149 (7 of 149 tokens kept). In 4 independent draws it is visible every time with probability

$$\left(\frac{7}{149}\right)^4 = 0.046980^4 = 0.0022071^2 = 4.8713 \times 10^{-6}.$$

Set1 has 992,708 valid pixels of 149 bands:

$$992{,}708 \times 149 = 147{,}913{,}492 \text{ (pixel, band) pairs,}$$
$$147{,}913{,}492 \times 4.8713 \times 10^{-6} = 720.5.$$

The run logged 706 [M: `research/experiments/hypersl-anomaly-bench/results.json`]: the expected count, within chance ($\sqrt{720} \approx 27$) [C]. For those 706 entries the script used the last draw's reconstruction, made with the band visible.

48.2.1 The results

1. HyperSL's own residual is close to chance. Its training loss as a score catches 63 objects, *fewer* than the 10.2% (about 93) that random scores would catch [C], with a ROC-AUC of 0.501. Its mean squared residual does somewhat better, 130 objects and pAUC 0.0175, a tenth of FTH3's. This is the toy of worked example 48.1 at full scale: the anomalies are completed.

method	caught top 1%	caught top 0.1%	pAUC	ROC-AUC	TPR at 0.001
chance	10.2%				
FTH3 ep. 5, RM	422 (46.4%)	13.9%	0.1749	0.866	0.058
PCA-RX (no model)	361 (39.7%)	8.1%	0.1281	0.839	0.029
HyperSL RM	350 (38.5%)	5.4%	0.0832	0.757	0.020
MNF-RX	249 (27.4%)	3.4%	0.0750	0.749	0.019
HyperSL L2	130 (14.3%)	4.0%	0.0175	0.624	0.007
HyperSL loss	63 (6.9%)	0.9%	0.0039	0.501	0.000
cloud excluded by FTH3 epoch 5's thick-cloud mask					
FTH3 ep. 5, RM	449 (49.4%)	16.0%	0.1892	0.869	0.065
PCA-RX	377 (41.5%)	8.5%	0.1390	0.842	0.031
HyperSL RM	369 (40.6%)	6.7%	0.0947	0.761	0.024
MNF-RX	280 (30.8%)	5.2%	0.0966	0.753	0.024
HyperSL L2	137 (15.1%)	4.3%	0.0182	0.628	0.007
HyperSL loss	63 (6.9%)	0.9%	0.0039	0.503	0.000

Table 48.1: HyperSL on the 909-object PRISMA bench, beside Indradhanu FTH3 epoch 5 and two classical detectors. pAUC is up to FPR 0.01; pixel measures are means over the four scenes; caught is the share of all 909 objects with a pixel within 1 px of the top 1% or 0.1% [M: `research/experiments/hypersl-anomaly-bench/results.json`].

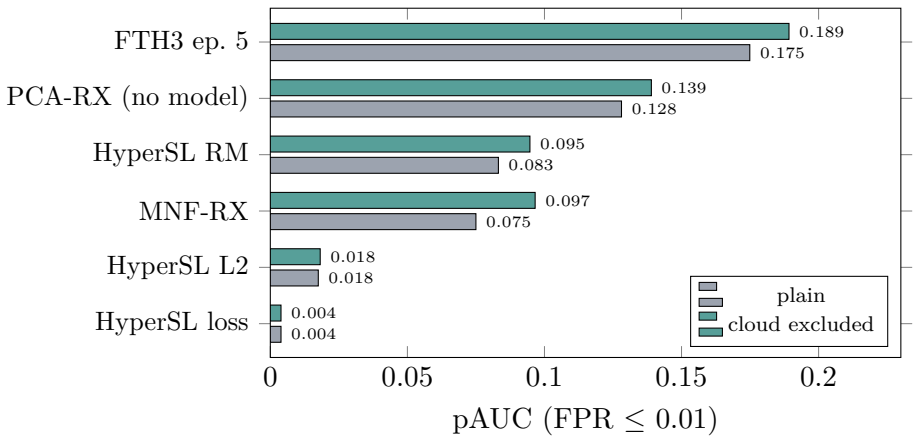


Figure 48.1: pAUC on the PRISMA bench. HyperSL's own residual (L2, loss) sits near zero; scored with residual Mahalanobis it lands below PCA-RX, which uses no model at all. Data `figdata/hsl_detect_bar.dat`.

2. Our scorer rescues part of it, but not past "no model". Feeding HyperSL's residual to the product's residual Mahalanobis gives 350 objects and pAUC 0.0832. Most of that is the scorer's work, as the next comparison shows.

3. The decisive comparison: HyperSL's reconstruction against none. PCA-RX *is* residual Mahalanobis on the cube with a zero reconstruction. So "HyperSL RM minus PCA-RX" isolates what subtracting HyperSL's reconstruction does, everything else equal. Paired object bootstrap, 2,000 draws:

difference (B − A)	pAUC [95% interval]	caught top 1% [95% interval]
HyperSL RM − PCA-RX	−0.0450 [−0.0589, −0.0318]	−1.2 [−4.0, +1.5] points
HyperSL RM − PCA-RX, cloud excl.	−0.0443 [−0.0577, −0.0320]	−0.9 [−3.5, +1.9] points
PCA-RX − FTH3 ep. 5	−0.0467 [−0.0604, −0.0327]	−6.7 [−8.8, −4.5] points
HyperSL RM − FTH3 ep. 5	−0.0917 [−0.1060, −0.0776]	−7.9 [−10.8, −5.2] points
HyperSL RM − FTH3 ep. 5, cloud excl.	−0.0944 [−0.1094, −0.0800]	−8.8 [−11.7, −6.2] points
MNF-RX − FTH3 ep. 5	−0.0999 [−0.1170, −0.0826]	−19.0 [−22.0, −16.3] points

Subtracting HyperSL's reconstruction lowers pAUC by 0.045, an interval well clear of zero, and leaves the object count statistically unchanged. Indradhanu's reconstruction does the opposite: against the same PCA-RX it *adds* 0.047 (the third row, read with its sign reversed). The two models' residuals move the same scorer by almost the same amount in opposite directions [C]. That is the measured form of Section 48.1: a spatial completer leaves the anomaly in the residual; a self-completer takes it out.

Worked example 48.4: Reading the bootstrap rows

FTH3 against PCA-RX is the third row reversed: $+0.0467$ with interval $[+0.0327, +0.0604]$. HyperSL against PCA-RX is -0.0450 with interval $[-0.0589, -0.0318]$. Add the two point values:

$$0.0467 + (-0.0450) = 0.0017.$$

Within 0.002 of zero: on this bench, what Indradhanu's residual adds to the cube's own statistics is about what HyperSL's residual removes [C]. Both intervals exclude zero, so both effects are resolved.

Per scene and by contrast tier.

	pAUC per scene				caught top 1% by tier		
method	set1	set2	set3	set4	easy	medium	subtle
FTH3 ep. 5, RM	0.2616	0.1874	0.0543	0.1964	59.2%	41.0%	39.0%
PCA-RX	0.1734	0.1171	0.0822	0.1400	48.7%	35.4%	35.0%
HyperSL RM	0.0875	0.0966	0.0286	0.1199	52.0%	33.8%	29.7%
MNF-RX	0.0871	0.1267	0.0196	0.0664	34.9%	25.2%	22.0%
HyperSL L2	0.0415	0.0225	0.0040	0.0021	21.1%	10.8%	11.0%
HyperSL loss	0.0021	0.0117	0.0019	0.0000	7.9%	4.9%	8.0%

HyperSL RM is below PCA-RX and below FTH3 on every scene. It holds up best on the easy (high-contrast) tier, 52.0% against PCA-RX's 48.7%, and falls furthest on the subtle one, 29.7% against 35.0%: the low-contrast targets are the ones a self-completer rebuilds most completely [M: `research/experiments/hypersl-anomaly-bench/results.json`].

Cost. 428–462 s of GPU time per scene for about a million pixels and four draws, 1,773 s for the bench: 438 microseconds per pixel on a 20 GB laptop GPU [C]. For comparison, Indradhanu Heavy takes 454 s per PRISMA scene on 16 CPU threads in the product worker [M: commit `72a767c` message].

Source check: what this run did not try

HyperSL read the 149-band axis of the earlier experiments, not the 186-band native axis the evaluator gives Indradhanu, and it kept no water bands, unlike its pretraining. Mask ratio and draw count were not tuned (0.95, the training ratio, and 4). These choices follow the model's training and the team's earlier runs; none was tuned on the bench, so none can have flattered the comparison either way. A different band axis could move HyperSL's numbers a little; it cannot remove self-completion, which is in the masking.

48.3 HyperSL as a false-alarm filter

A gentler use was tried on 2026-09-26: keep Indradhanu as the detector, and use HyperSL's embeddings of each candidate pixel and of its ring to tell real targets from false alarms (`research/experiments/indradhanu-fp-filter/`, scripts in commit `74df314`; results outside the repository in `C:/Users/FITT/Documents/stage1_datasets/indradhanu_fp_filter/step3/results.json`). Candidates are the top 2% of Indradhanu's residual-Mahalanobis score; a small classifier is trained on three scenes and tested on the fourth; the headline is the share of false-alarm pixels removed while keeping 90% of real objects. "Oracle" sets that cut on the test scene's own labels (an upper bound); "honest" sets it on the training scenes, as deployment would.

features (one hidden layer)	set1	set2	set3	set4	mean
oracle: share of false-alarm pixels removed at 90% object recall					
none (a stricter Indradhanu cut)	0.339	0.128	0.198	0.262	0.232
detector scores only	0.059	0.219	0.223	0.249	0.188
HyperSL pixel + ring embeddings	0.615	0.464	0.188	0.318	0.396
honest: object recall reached / false-alarm pixels removed					
none	1.00 / 0.00	0.62 / 0.56	1.00 / 0.00	0.86 / 0.31	
HyperSL embeddings	0.72 / 0.77	0.42 / 0.87	0.45 / 0.50	0.69 / 0.59	

[M: `indradhanu_fp_filter/step3/results.json`, configurations `baseline`, `scores/B`, `hypersl/B`; means [C]]

This is HyperSL's best showing in the whole part, and it deserves to be stated plainly: with the cut chosen in hindsight, its embeddings removed 40% of false-alarm pixels at 90% object recall against 23% for a stricter cut, so they carry information about what a target looks like next to its ring. But the honest cut, the only one a deployment has, did not transfer between scenes: it kept between 42% and 72% of the objects instead of 90%. The filter was not adopted.

A model that rebuilds a pixel from that pixel cannot tell you the pixel is surprising. Scored as a detector, HyperSL's residual is near chance; scored with our own residual Mahalanobis, its reconstruction makes the score worse than using no reconstruction at all.

Values that will drift

The bench run is of 2026-10-31 (`research/experiments/hypersl-anomaly-bench/`); FTH3's maps are from the 2026-10-29 evaluation; the false-alarm-filter numbers are from 2026-09-26 and were computed on candidates from the Indradhanu of that date. A new HyperSL release, a different band axis, or a new release candidate would move the rows.

48.4 Common confusions

HyperSL hides 95% of the bands, so it cannot copy.
It cannot copy those bands, but the 7 it sees carry the anomaly's level and shape, and the other 142 are completed to match. Hiding bands is not hiding the pixel.

A residual Mahalanobis of 350 objects shows HyperSL detects anomalies.
The same scorer on the raw cube, with no model, catches 361 and has a higher pAUC. The objects are the scorer's, not HyperSL's.

HyperSL's loss catching 63 objects is a little worse than chance, so it is useless but harmless.
Below chance means it ranks anomalies *lower* than background: it fits them better. That is self-completion, not noise.

Indradhanu would have the same problem with spectral masking.
It would, which is why spectral self-completion is listed as rejected in its design (`docs/lld/spectral-stem.md`) and band-list augmentation is allowed only because a hidden pixel's own bands are never an input (Section 24.2).

The false-alarm filter result shows HyperSL helps.
Only with a cut chosen on the test scene's labels. With an honest cut it lost up to 58% of the objects.

48.5 Check yourself

Exercise 48.1. A third anomaly is (0.50, 0.45, 0.40). Compute its self-completion residual in band 2 and its spatial residual and squared length against $\mathbf{b} = (0.20, 0.30, 0.40)$.

Exercise 48.2. Set2 has 1,050,895 valid pixels. How many (pixel, band) pairs do you expect never to be hidden in 4 draws? The run logged 736.

Exercise 48.3. An object covers 5 pixels after the 1-pixel dilation. What is its chance of being caught in the top 1% by random scores? (Chapter 33: $1 - (1 - q)^m$.)

Exercise 48.4. Does the bootstrap show HyperSL RM catching fewer objects than PCA-RX? Does it show a lower pAUC?

Exercise 48.5. Why can a score catch *fewer* objects than chance? What does that say about how HyperSL ranks anomalies?

Exercise 48.6. At 438 microseconds per pixel with 4 draws, how long would a 1,000 by 1,000 scene take with 4 draws, and roughly with 1 draw?

Next: Chapter 49, Unmixing and sub-pixel targets.

Chapter 49

Unmixing and sub-pixel targets

The one thing this chapter teaches

A published "endmember-free" unmixing method built on HyperSL lost to the classical VCA + FCLS on every data set with exact truth, needed VCA to start in the authors' own code, gained nothing from pretraining, took 5–13 minutes per scene on a 96 GB GPU, and, because its abundances pass through a 3×3 head and a total-variation penalty, smoothed a target a seventh of a pixel wide down to an object AUC of 0.68, where per-pixel ACE scored 0.97.

Chapter 47 asked whether HyperSL's *embedding* can name a material. A second route exists: use HyperSL as the backbone of an *unmixing* network, which splits each pixel into a few pure materials and their shares. Liu et al. [26] proposed exactly this, "endmember-free" unmixing on a pretrained HyperSL. If it worked it would answer two of the product's questions at once: which material is in the pixel, and how much of the pixel it fills. The experiment `research/experiments/hypersl-unmixing/` tested it on 2026-10-13/14 on a Colab RTX PRO 6000 (96 GB). Its README is in commit `4d292ac`; every number in this chapter is from it unless marked otherwise.

Analogy

A newspaper photograph printed through a coarse halftone screen. A crowd shot reproduces well: faces are many dots wide. A person standing far off, smaller than one dot, is averaged into the grey of the dot and vanishes. Nothing is wrong with the press; it is built for pictures whose detail is larger than its dot. A sub-pixel target is the far-off person, and HyperSL's unmixing head is the screen.

49.1 The words

Definition 49.1 (Linear mixing, endmember, abundance)**.** The *linear mixing model* writes a pixel as a weighted sum of K pure spectra, its *endmembers* $\mathbf{e}_1, \dots, \mathbf{e}_K$, plus noise:

$$\mathbf{x} = \sum_{k=1}^{K} a_k \mathbf{e}_k + \mathbf{n} = \mathbf{E}\mathbf{a} + \mathbf{n}, \qquad a_k \geq 0, \quad \sum_k a_k = 1. \tag{49.1}$$

The weights a_k are the *abundances*: the share of the pixel each material covers. With $K = 2$, background $\mathbf{b}$ and target $\mathbf{t}$, it is the fill model of Chapter 44: $\mathbf{x} = (1 - a)\mathbf{b} + a\mathbf{t}$.

Definition 49.2 (Blind unmixing, VCA, FCLS)**.** *Blind unmixing* finds both $\mathbf{E}$ and the abundances from the image alone. *VCA* (vertex component analysis, [33]) finds endmembers as the

most extreme pixels of the data cloud; *FCLS* (fully constrained least squares, [12]) then finds, for each pixel, the abundances that best fit Equation (49.1) under both constraints. VCA + FCLS takes seconds per scene.

Definition 49.3 (mSAD, mRMSE)**.** After matching each found endmember to a true one (the Hungarian method pairs them so the total error is least), *mSAD* is the mean spectral angle between pairs, in radians, and *mRMSE* the mean root-mean-square error of the abundance maps. Lower is better for both. 0.1 rad is 5.7°.

Definition 49.4 (Object AUC)**.** For one labelled object, take its highest score and rank it against every background pixel's score: the *object AUC* is the share of background pixels it beats. 1.00 means the object's best pixel outranks the whole background.

49.2 What was compared

Data. Two kinds, graded by how exact their truth is.

- **Blind unmixing**: DC1, DC2, DC3 and EGU-Net, synthetic scenes with exact truth (grade B; DC3 has no pure pixels, EGU has spectral variability, the two cases built to favour a learned method); and Samson, Jasper and Urban4, real scenes whose "truth" is FCLS from hand-picked endmembers (grade D, a reproduction check only).
- **Sub-pixel targets**: DLR HySU [4], five materials in squares of side 0.25, 0.5, 1, 2 and 3 m on grass, imaged at 0.7 m (VNIR only); and Gudalur AVIRIS-NG aggregated to 32 m, 16 m sheets, true fill at most 0.25.

HyperSL presets. The paper and the released code disagree, so both were run. "Start" is how the endmembers are initialised; R_θ is explained in Section 49.3.

preset	start	R_θ	loss	source
`paper`	random	yes	paper eq. 17	the paper
`paper_noR`	random	no	paper eq. 17	the paper without R_θ
`upstream`	**VCA**	no	SAD $+0.35\sqrt{A}$ $+0.1\sum\lvert\Delta E\rvert$	`downstream_unmixing.py`
`scratch`	random	yes	paper eq. 17	`paper` with a *random* backbone

All use the `10_base_mask95` checkpoint, AdamW, 200 epochs and 400 patches; 2 seeds for `paper_noR` and `upstream`, 1 for `paper` and `scratch`, 10 for VCA + FCLS. 112 runs in all.

Source check: the released code is not endmember-free

`downstream_unmixing.py` overwrites the decoder with VCA endmembers and prints "Endmember initialized by VCA". It also uses a different loss, patch size and weights from the paper and has no R_θ. Only this VCA-initialised version came close to a published number: Urban mSAD 0.262 ± 0.050 against the paper's 0.213. The paper's endmember-free version reached 0.323 on Jasper against a reported 0.045. An independent benchmark also failed to reproduce the method [M: `2d893e6`: summary.md, step 3].

49.3 Blind unmixing: the classical method wins where truth is exact

Read the tables in three steps.

set (grade)	VCA+FCLS (10)	upstream (2)	paper_noR (2)	paper (1)	scratch (1)
DC1 (B)	**0.000 ± 0.000**	0.092 ± 0.000	0.084 ± 0.003	0.173	0.173
DC2 (B)	**0.000 ± 0.000**	0.085 ± 0.001	0.082 ± 0.029	0.182	0.175
DC3 (B)	**0.069 ± 0.008**	0.124 ± 0.000	0.164 ± 0.043	0.199	0.153
EGU (B)	**0.004 ± 0.001**	0.106 ± 0.001	0.083 ± 0.005	0.264	0.249
Samson (D)	0.085 ± 0.062	0.278 ± 0.108	**0.074 ± 0.025**	0.531	0.493
Jasper (D)	0.397 ± 0.038	0.405 ± 0.011	**0.323 ± 0.039**	0.472	0.472
Urban4 (D)	0.418 ± 0.015	**0.262 ± 0.050**	0.322 ± 0.084	0.419	0.413

Table 49.1: mSAD in radians, mean ± standard deviation over seeds (seed count in brackets), lower is better, best per row in bold [M: `4d292ac`: hypersl-unmixing/README.md].

set (grade)	VCA+FCLS	upstream	paper_noR	paper	scratch
DC1 (B)	**0.000 ± 0.000**	0.332 ± 0.025	0.333 ± 0.001	0.380	0.388
DC2 (B)	**0.000 ± 0.000**	0.220 ± 0.010	0.209 ± 0.005	0.387	0.296
DC3 (B)	**0.103 ± 0.026**	0.324 ± 0.033	0.300 ± 0.009	0.357	0.347
EGU (B)	**0.081 ± 0.005**	0.229 ± 0.001	0.242 ± 0.004	0.453	0.438
Samson (D)	0.282 ± 0.021	0.301 ± 0.040	0.236 ± 0.000	0.308	**0.225**
Jasper (D)	0.317 ± 0.044	0.298 ± 0.026	**0.242 ± 0.113**	0.472	0.461
Urban4 (D)	0.380 ± 0.033	0.282 ± 0.119	**0.281 ± 0.021**	0.517	0.569

Table 49.2: mRMSE of the abundances, same layout [M: `4d292ac`: hypersl-unmixing/README.md].

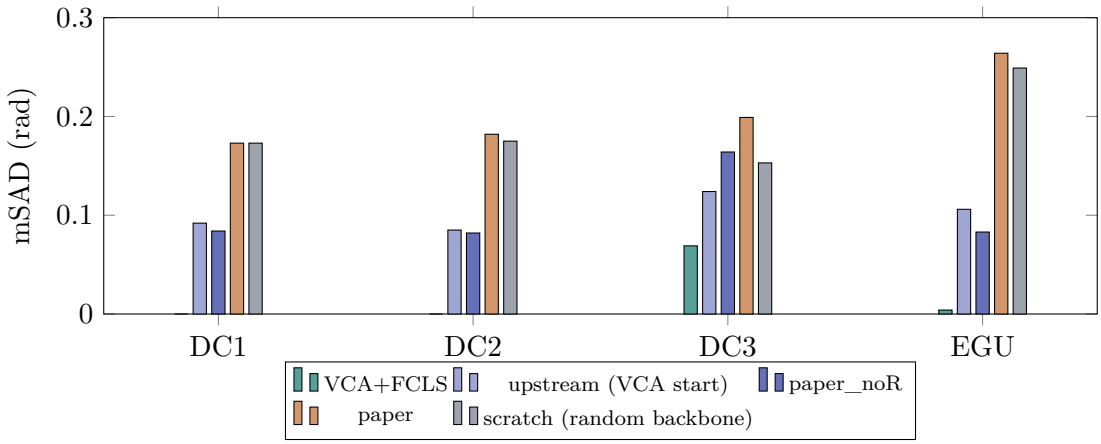


Figure 49.1: mSAD on the four exact-truth sets. The classical method is near zero on DC1, DC2 and EGU; every HyperSL preset is 0.08–0.26 rad. The last two bars, pretrained and random backbone, are nearly the same height.

1. On exact truth, VCA + FCLS wins every set on both measures , including DC3 and EGU, the cases built to favour a learned method. HyperSL's abundance error there is 0.21–0.33 against the classical ≤ 0.10.

Worked example 49.1: What 0.069 against 0.199 radians means

Convert with 1 rad $= 180/\pi = 57.296°$:

$$0.069 \times 57.296 = 3.95° \quad \text{(VCA+FCLS on DC3)},$$
$$0.199 \times 57.296 = 11.40° \quad \text{(the } \texttt{paper} \text{ preset on DC3) } [C].$$

On the hardest exact-truth set, HyperSL's endmembers sit almost three times as far from the truth.

2. HyperSL wins only where the "truth" is itself an unmixing. On Samson, Jasper and Urban the reference is FCLS from hand-picked endmembers, so a win measures agreement with that reference, not accuracy. The chain is calibrated: our VCA + FCLS matches the paper's own VCA rows on Jasper (0.397 against 0.406) and Urban (0.418 against 0.473).

3. Pretraining adds nothing in the paper's configuration. `scratch` is `paper` with a randomly initialised backbone. DC1: 0.173 against 0.173; EGU: 0.249 against 0.264. The paper's ablation claims a 3–10 times gain in mSAD from pretraining; it does not reproduce.

Source check: what was not run

The combination `paper_noR` with a random backbone was not run, so pretraining's value *without* R_θ is untested. The comparison in point 3 is for the paper's full configuration. Seeds were trimmed to 1–2 for the HyperSL presets; on the grade-B sets the spread is small next to the gaps, on grade D it is not (Samson `upstream` ±0.108).

49.3.1 The R_θ branch explains what the mixing should

The paper's model reconstructs a pixel as $\hat{\mathbf{x}} = \mathbf{E}\mathbf{a} + R_\theta(\cdot)$: the mixing term plus an unconstrained convolutional branch from HyperSL's features to the spectrum. Measured: **R_θ carries 29–56% of the reconstruction energy on every set**. Whatever R_θ explains, **Ea** does not have to, and the abundances stop being forced to account for the pixel. On the target scenes it drove the target abundance to about 1 even on empty background (floor 0.995), which breaks every area estimate. `paper` is the worst preset everywhere.

49.4 Sub-pixel targets: smoothed away

At Gudalur (32 m pixels, fill at most 0.25), ACE (0.99), the step-5 gain (0.98) and anchored HyperSL (1.00) all find the sheets; blind HyperSL reaches 0.59. And blind HyperSL never learned a target as an endmember: on HySU the nearest learned endmember is a median 18° from each target spectrum and 3 of 5 materials share one; at Gudalur 10°, with 4 of 5 sharing one. A target covering a few pixels carries too little of the scene's variance to win an endmember of its own.

target side (true area in pixels)	0.25 m (0.13)	0.5 m (0.51)	1 m (2.0)	2 m (8.2)	3 m (18.4)
ACE, our library + field spectra	**0.97**	**1.00**	1.00	1.00	1.00
step 5 gain, our library + field	**0.97**	**1.00**	1.00	1.00	1.00
ACE, our library only	0.88	0.99	1.00	1.00	1.00
HyperSL anchored (`paper_noR`)	0.68	0.77	1.00	1.00	1.00
HyperSL blind (`paper_noR`)	0.72	0.62	0.80	0.88	0.92

Table 49.3: HySU object AUC: does the object's best pixel outrank the background? [M: `4d292ac`: hypersl-unmixing/README.md] "Anchored" gives HyperSL the field spectra as fixed endmembers; "blind" lets it learn its own. ACE is the adaptive cosine estimator of the family matcher (Chapter 32); step 5 is the ring-background subspace plus a library fill of `manmade-library-unmixing`.

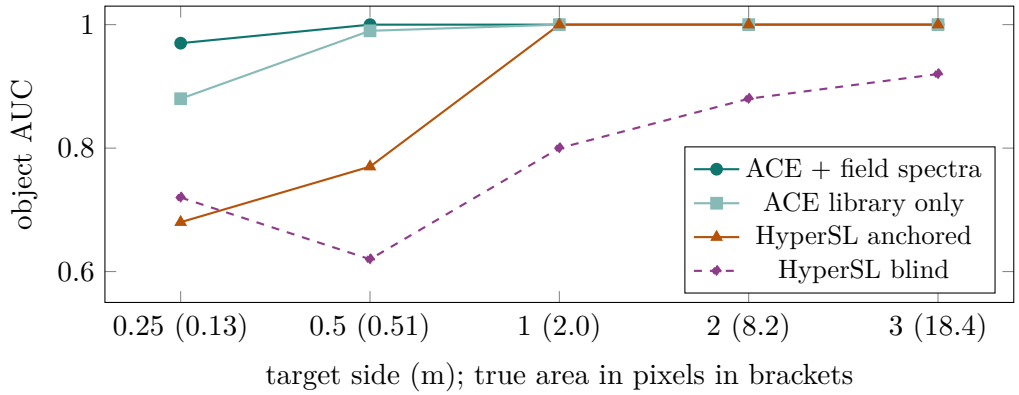


Figure 49.2: HySU object AUC against target size. Every method finds targets of 1 m and more when told the material; below one pixel, HyperSL anchored falls to 0.68 and 0.77 while per-pixel ACE stays at 0.97–1.00.

49.4.1 Why: a 3×3 head, worked by hand

The README gives the cause: HyperSL's abundances "pass through a 3×3 conv head and a TV penalty. A target smaller than a pixel is smoothed into its surroundings, so its fill drops to about 0." Both halves can be worked on paper.

Definition 49.5 (3×3 averaging)**.** A 3×3 convolution replaces each pixel's value by a weighted sum of itself and its 8 neighbours. Its simplest form is the plain average, all nine weights $\frac{1}{9}$. A learned head is not exactly this, but any head whose weights are positive and sum to 1 is a smoothing of the same kind [A].

Worked example 49.2: One target pixel at fill a, averaged over 3×3

The abundance map is 0 everywhere except one pixel, which holds a. At that pixel, the average of its nine values is

$$\frac{a+0+0+0+0+0+0+0+0}{9} = \frac{a}{9}.$$

At each of its 8 neighbours, the window again holds exactly one non-zero value, the target's, so each neighbour also becomes $a/9$. For HySU's two smallest targets [C]:

$$a = 0.13: \quad 0.13/9 = 0.0144,$$
$$a = 0.51: \quad 0.51/9 = 0.0567.$$

The peak falls ninefold and spreads over nine pixels. A detector reading the smoothed map is looking for an abundance of 0.0144 against the background's own noise in abundance, which the smoothing does not shrink by the same factor.

Definition 49.6 (Total variation)**.** The (anisotropic) *total variation* of a map is the sum, over all pairs of side-by-side pixels, of the absolute difference of their values: $\mathrm{TV}(\mathbf{a}) = \sum_{i\sim j} |a_i - a_j|$. A TV penalty adds $\lambda\,\mathrm{TV}$ to the loss, rewarding maps made of flat patches [38].

Worked example 49.3: Why a TV penalty sets small targets to zero

Take the $K = 2$ mixture: a pixel holds $\mathbf{x} = \mathbf{b} + a(\mathbf{t} - \mathbf{b})$. If the model reports fill a' instead, its reconstruction misses by $(a - a')(\mathbf{t} - \mathbf{b})$, whose squared length is $(a - a')^2 c$ with $c = \|\mathbf{t} - \mathbf{b}\|^2$. A single target pixel among zeros has 4 side neighbours, so its TV is $4a'$. The cost for that pixel is

$$J(a') = c\,(a - a')^2 + 4\lambda a'.$$

Differentiate with respect to a' and set to zero:

$$\begin{aligned}
\frac{dJ}{da'} &= -2c\,(a - a') + 4\lambda = 0 && \text{derivative of a square, and of a line}\\
2c\,(a - a') &= 4\lambda && \text{move } 4\lambda \text{ across}\\
a - a' &= \frac{2\lambda}{c} && \text{divide by } 2c\\
a' &= a - \frac{2\lambda}{c}, && \text{and since a fill cannot be negative,}\\
a' &= \max\!\left(0,\ a - \frac{2\lambda}{c}\right).
\end{aligned}$$

With $c = 0.20$ (the smooth anomaly of Chapter 48's toy, $\|(0.4, 0.2, 0)\|^2 = 0.16 + 0.04 + 0 = 0.20$) and $\lambda = 0.013$ [A], the shrinkage is $2 \times 0.013/0.20 = 0.026/0.20 = 0.13$. Then [C]

$$\begin{aligned}
a = 0.13: &\quad a' = \max(0,\ 0.13 - 0.13) = 0,\\
a = 0.51: &\quad a' = 0.51 - 0.13 = 0.38,\\
a = 1: &\quad a' = 1 - 0.13 = 0.87.
\end{aligned}$$

The penalty subtracts the same amount from every target pixel. A target whose fill is below that amount vanishes.

The amount depends on the target's shape. For a $k \times k$ block of fill a' among zeros, the data term is $k^2 c\,(a - a')^2$ (one per pixel) and the TV is $4k\,a'$ (only the $4k$ edges on its border differ). The same steps give

$$-2k^2 c\,(a - a') + 4k\lambda = 0 \quad\Longrightarrow\quad a' = a - \frac{2\lambda}{k\,c}.$$

The shrinkage falls as $1/k$: with the numbers above it is 0.13 for a single pixel, 0.065 for 2×2, 0.043 for 3×3 and 0.026 for 5×5 [C]. **TV charges by perimeter and the data pay by area, so the smallest targets lose the most.** That is the exact opposite of what a sub-pixel detector needs; ACE and the step-5 gain score each pixel on its own and keep them.

A penalty that makes abundance maps tidy is a prior that objects are large. Our targets are small. Any model that smooths across pixels, whether by a 3×3 head, TV, or a spatial decoder, has to be judged at fills of 0.1 to 0.25, where the product works.

49.4.2 Fill: accurate only when the material is known

Median relative area error on HySU (0 is perfect):

method	0.25 m	0.5 m	1 m	2 m	3 m
step 5 oracle, in-image spectrum	0.35	0.68	0.05	0.08	**0.04**
step 5 oracle, field spectrum	**0.30**	**0.51**	0.37	0.14	0.07
HyperSL anchored (`paper_noR`)	0.97	0.99	0.33	0.33	0.29
step 5 blind (best man-made entry)	19.2–19.6	0.65–0.77	0.03–0.37	0.07–0.46	0.06–0.68

HyperSL's 0.97 and 0.99 on the two smallest sizes are worked example 49.3 measured: the reported area is about 0. At Gudalur no method estimates fill well (per-pixel RMSE 0.14–0.16 for the step-5 oracle and for HyperSL, against true fills ≤ 0.25). The product's rule followed from this: report fill only when a spectrum of the actual material is available (`docs/lld/spectral-family-match.md`).

49.5 Cost, and the decision

VCA + FCLS takes seconds per scene. HyperSL took 5 minutes (`upstream`) to 13 minutes (`paper_noR`) per scene on a 96 GB GPU. The family-match design records the outcome in one row of its rejected-techniques table: "HyperSL deep 'endmember-free' unmixing … worse than classical unmixing on exact-truth sets; smooths away sub-pixel targets; 5–13 min per scene on a 96 GB GPU … rejected" [M: `docs/lld/spectral-family-match.md`].

Values that will drift

Every number here is from the 2026-10-13/14 Colab run (README in commit `4d292ac`); its data and raw outputs are on the team Drive (`MyDrive/PS_11/claude_experiments/`), not in the repository. HySU truth is the nominal target side, not the measured areas.

49.6 Common confusions

HyperSL won on Jasper and Urban, so it unmixes real scenes better.
Those references are FCLS from hand-picked endmembers; a win is agreement with that reference. On exact truth it lost every set.

"Endmember-free" means no classical step.
The authors' released script starts from VCA. The endmember-free version, as the paper describes it, missed its own published Jasper number by a factor of seven.

Pretraining is what makes it work.
With R_θ, a random backbone matched the pretrained one.

A 3×3 head only blurs edges.
For a one-pixel target the whole object is an edge: the peak falls to $a/9$.

TV removes noise, not targets.
TV cannot tell the two apart when the target is one pixel: it subtracts $2\lambda/c$ from both.

49.7 Check yourself

Exercise 49.1. A target pixel at fill 0.25 sits among zeros. After a 3×3 average, what is the value at the target and at a diagonal neighbour? At a pixel two steps away?

Exercise 49.2. Convert the mSAD of `upstream` and VCA + FCLS on EGU to degrees.

Exercise 49.3. In worked example 49.3, find the largest λ for which a single target at fill 0.13 survives with $a' > 0$, when $c = 0.20$.

Exercise 49.4. Repeat the TV shrinkage for a 2×2 target at fill 0.13 with $c = 0.20$ and $\lambda = 0.013$. Does it survive?

Exercise 49.5. R_θ carries 40% of the reconstruction energy. If a pixel's spectrum has squared length 1.0 and the reconstruction is perfect, what squared length is left for **Ea** to explain? Why does this make abundances unreliable?

Exercise 49.6. HyperSL takes 13 minutes per scene. How long for the 4 PRISMA bench scenes, and for 18 implant hosts?

Next: Chapter 50, Cloud masking.

Chapter 50

Cloud masking

The one thing this chapter teaches

A cloud classifier given HyperSL's 256-number embedding plus seven raw bands found 28.9% of the cirrus that EnMAP's own masks label, against 67.9% for the same classifier given the 148 raw bands alone; HyperSL was pretrained on scenes chosen to be nearly cloud-free, and its per-scene scaling turns the same thin cirrus into a different input in every scene.

The product excludes cloud from its detections (Chapter 28, `docs/lld/evaluation.md`, "Decisions"). In September 2026 the cloud mask came from fixed thresholds and from EnMAP's own quality layers, which are wrong over the high plateau, and PRISMA L2D carries no cloud mask at all. A learned cloud mask on PRISMA was needed, and a foundation model's embedding is the textbook way to build one with few labels. The experiment `research/experiments/cloud-mask-hypersl/` (commits `fa8c1ab` and `b452e8d`, 2026-10-01) tried it.

Analogy

A wine taster trained only on still wines, poured at the temperature of whichever cellar they came from, and asked to detect a faint sparkle. The sparkle is small, it was never in the training glasses, and how cold the glass is (which changes how a sparkle feels) is set by the cellar, not the wine. Give a second taster a thermometer and the bare glass and they do better, with no training at all on what wine is.

50.1 The experiment

Training pixels (step 1). 97 EnMAP L2A scenes (the commit message of `fa8c1ab`), every spectrum resampled onto PRISMA's band axis with a Gaussian of PRISMA's band width, because "HyperSL's embedding shifts with band sampling almost as much as with material" (Chapter 47). Classes 0 clear, 1 cloud, 2 cirrus, 3 snow, from EnMAP's `QL_QUALITY_*` masks with three corrections agreed on 2026-10-01: scenes above 2,500 m mean elevation contribute no cloud or cirrus labels (EnMAP flags 51–57% of clear plateau ground as cirrus); snow labels below 1,000 m are dropped (salt flats); pixels within 2 of a mask boundary are dropped. Cirrus is `QL_QUALITY_CIRRUS` ≥ 2; haze and shadow are left out [M: `fa8c1ab`: step1_training_set.py docstring].

Source check: the cirrus band is not there

The classic cirrus test uses the 1.38 μm water-vapour band, where low surfaces are dark and high ice cloud is bright. "EnMAP L2A blanks 1343–1390 nm … to nodata, so the 1.38 μm cirrus band is never in the training spectra" (the same docstring). PRISMA's 149-band axis

also drops 1325–1495 nm. Both variants below work without it, so the comparison between them is fair, and both are harder than they would be with it.

Embeddings (step 2). HyperSL at mask 0 on 148 bands (the 149-band axis minus 1785 nm, which EnMAP does not cover), each pixel scaled per band by its own scene's minimum and maximum, "the frame HyperSL was trained in".

Classifier (step 3). A gradient-boosted tree classifier (`HistGradientBoostingClassifier`, 300 rounds, classes weighted to balance, early stopping) with two feature sets:

- **C**, "the chosen design": HyperSL's 256-number embedding plus raw reflectance at 7 wavelengths (490, 560, 665, 865, 1240, 1610 and 2200 nm);
- **B**, "a yardstick only": raw reflectance on the 148 bands, no HyperSL.

Definition 50.1 (Grouped cross-validation)**.** *k-fold cross-validation* splits the data into k parts, trains on $k-1$ and tests on the remaining one, k times, so every sample is tested exactly once by a model that did not train on it. *Grouped* means whole groups are kept together: here an EnMAP *datatake* (one overpass, cut into several tiles that share atmosphere and sun angle) is never split between training and test.

784,120 pixels from 47 datatakes, 5 folds [M: `b452e8d`: cloud-mask-hypersl/step3_report.txt].

50.2 The results, in full

	the classifier says				
EnMAP label	clear	cloud	cirrus	snow	row total
clear	**341,395**	36,215	4,955	909	383,474
cloud	22,708	**244,246**	11,452	2,042	280,448
cirrus	21,979	28,341	**20,472**	1	70,793
snow	1,619	5,025	36	**42,725**	49,405

Table 50.1: Variant C, HyperSL embedding + 7 raw bands. Rows are EnMAP's labels, columns the classifier's out-of-fold predictions [M: `b452e8d`: step3_report.txt]; row totals [C].

	the classifier says				
EnMAP label	clear	cloud	cirrus	snow	row total
clear	**342,688**	25,158	12,901	2,727	383,474
cloud	27,357	**225,595**	25,510	1,986	280,448
cirrus	6,861	15,888	**48,037**	7	70,793
snow	406	1,135	16	**47,848**	49,405

Table 50.2: Variant B, the 148 raw bands, no HyperSL; same pixels, folds and classifier [M: `b452e8d`: step3_report.txt].

Definition 50.2 (Recall and precision of a class)**.** For a class, *recall* is the share of pixels *labelled* with it that the classifier also calls it: the diagonal entry over its row total. *Precision* is the share of pixels the classifier *calls* it that are labelled with it: the diagonal entry over its column total. The report prints them as "found x% of EnMAP's" and "right y% of the time".

Worked example 50.1: Cirrus recall and precision, from the tables

Variant C. The cirrus row is 21,979 + 28,341 + 20,472 + 1:

$$\begin{aligned} \text{row total} &= 21{,}979 + 28{,}341 = 50{,}320, \\ & \qquad 50{,}320 + 20{,}472 = 70{,}792, \quad 70{,}792 + 1 = 70{,}793, \\ \text{recall} &= 20{,}472/70{,}793 = 0.2892 = 28.9\%, \\ \text{column total} &= 4{,}955 + 11{,}452 + 20{,}472 + 36 = 36{,}915, \\ \text{precision} &= 20{,}472/36{,}915 = 0.5546 = 55.5\%. \end{aligned}$$

Variant B. Same row total, 70,793.

$$\begin{aligned} \text{recall} &= 48{,}037/70{,}793 = 0.6786 = 67.9\%, \\ \text{column total} &= 12{,}901 + 25{,}510 + 48{,}037 + 16 = 86{,}464, \\ \text{precision} &= 48{,}037/86{,}464 = 0.5556 = 55.6\%. \end{aligned}$$

At the same precision, the raw bands find 48,037/20,472 = 2.35 times as much cirrus [C]. Of C's 50,321 missed cirrus pixels, 21,979 were called clear and 28,341 cloud.

	recall (found)		precision (right)	
class	C: HyperSL + 7	B: 148 raw	C: HyperSL + 7	B: 148 raw
clear	89.0%	89.4%	88.1%	90.8%
cloud	**87.1%**	80.4%	77.8%	**84.2%**
cirrus	28.9%	**67.9%**	55.5%	55.6%
snow	86.5%	**96.8%**	93.5%	91.0%
overall accuracy	C 82.75%	B 84.70% [C]		
high-plateau clear called cloud or cirrus	C **4.34%**	B 6.62%		

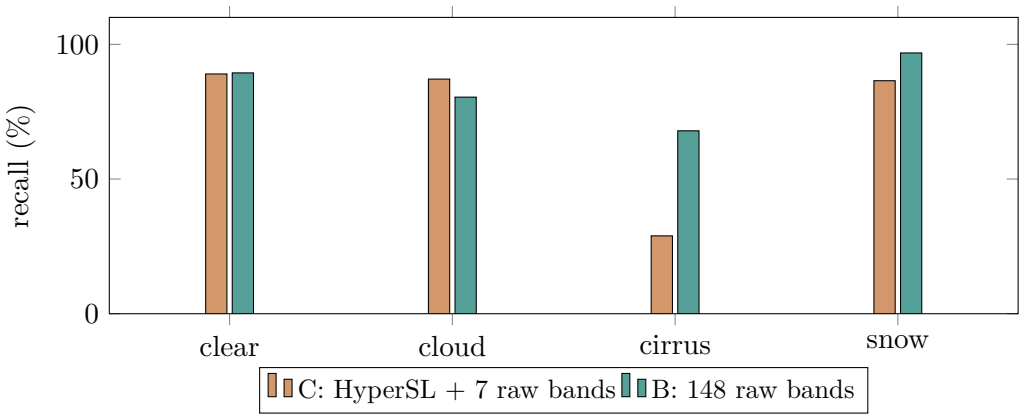


Figure 50.1: Share of each EnMAP-labelled class the classifier finds, out of fold over 47 datatakes. HyperSL's features lose most on cirrus, the thin class, and on snow.

Being fair. C is not worse everywhere. It finds more of the thick cloud (87.1% against 80.4%), and on the 59,474 clear pixels of high-plateau scenes, where EnMAP's own masks fail, it calls fewer of them cloud or cirrus (4.34% against 6.62%). Its overall accuracy is two points lower, 82.75% against 84.70% [C]. And the scores measure *agreement with EnMAP's masks*, not truth: the step-3 docstring warns that "a model can only look as good as EnMAP is right".

Source check: one comparison the run did not make

C has 7 raw bands and B has 148. The run did not test the 7 raw bands *alone*, nor HyperSL with all 148 raw bands, so it cannot say how much of B's advantage is "more raw bands" rather than "no HyperSL". What it does show is that adding HyperSL's 256 numbers to 7 bands is worth far less, on cirrus, than having the other 141 bands.

50.3 Why thin cloud is hard for this model

50.3.1 It was never shown clouds

The paper's pretraining data were chosen with "cloud cover near zero", overall below 10% (Section 46.3). A reconstruction model learns what its data contain. Cloud and cirrus were deliberately kept out, so nothing in pretraining made the embedding sensitive to them. Our own Indradhanu fine-tunes, by contrast, include cloud as a labelled class of their segmentation head (Chapter 28).

50.3.2 The same cirrus, a different input in every scene

Thin cirrus mostly adds a little brightness to whatever is beneath it [A]. Call that addition δ in one band. Under Equation (47.1) a pixel's input changes by

$$
\begin{aligned}
x'_{\text{cirrus}} - x'_{\text{clear}} &= \frac{(x+\delta)-m}{M-m} - \frac{x-m}{M-m} && \text{the scaled value with and without } \delta \\
&= \frac{x+\delta-m-x+m}{M-m} && \text{one fraction, same denominator} \\
&= \frac{\delta}{M-m}. && x \text{ and } m \text{ cancel}
\end{aligned}
$$

The step depends on the scene's span $M - m$ in that band.

Worked example 50.2: One cirrus layer, two scenes

Cirrus adds $\delta = 0.03$ reflectance at 865 nm [A]. Scene A is dark (water and forest), with $m = 0.02$ and $M = 0.22$, a span of 0.20; scene B is a bright desert with $m = 0.20$ and $M = 0.80$, a span of 0.60 [A].

$$
\begin{aligned}
\text{scene A: } 0.03/0.20 &= 0.15, \\
\text{scene B: } 0.03/0.60 &= 0.05.
\end{aligned}
$$

The same cirrus is a step of 0.15 in one scene's input and 0.05 in the other's, a factor of three. With raw reflectance the step is 0.03 in both. A classifier trained across 47 datatakes sees one physical signal at many sizes.

Every band has its own span, so the cirrus signature's *shape* across bands is distorted too, differently per scene (the angle argument of worked example 47.4). And the brightest pixels of a cloudy scene are often the cloud itself: the cloud then sets M, the frame, for every other pixel (worked example 47.3).

A cloud mask must recognise a class by an absolute property (thin ice cloud is bright, flat and high). A model that removes absolute brightness per scene and never saw clouds has to recover both from seven raw bands bolted on afterwards.

What the product does. Cloud is a class of Indradhanu's own segmentation head, trained on labelled masks from reflectance input; since FTH3 the release candidate excludes *thick* cloud only, because its thin-cloud output called haze cloud on set4 (Chapter 43). The HyperSL classifier was not adopted.

Values that will drift

The 784,120 pixels, 47 datatakes and both confusion tables belong to the 2026-10-01 run (commit `b452e8d`); its training tables lived on the GPU box (`/wt/cloudmask/`), not in the repository. A step 4 (maps on PRISMA) was planned in the scripts; no result of it is in the history.

50.4 Common confusions

HyperSL is worse at every cloud class.
It finds more thick cloud (87.1% against 80.4%) and makes fewer false calls on the high plateau. It loses on cirrus and snow, and on overall accuracy by two points.

The cirrus result is low because the 1.38 μm band was missing.
It was missing for both variants. The difference between them is not that band.

Recall and precision are the same thing seen twice.
Recall divides by the row (what EnMAP labelled), precision by the column (what the classifier called). C and B have the same cirrus precision and very different recall.

These numbers say how often the mask is right.
They say how often it agrees with EnMAP's own masks, which are themselves wrong over the plateau.

50.5 Check yourself

Exercise 50.1. From Table 50.1, compute variant C's recall and precision for snow.

Exercise 50.2. From Table 50.2, compute variant B's recall and precision for cloud.

Exercise 50.3. Compute both variants' overall accuracy from the tables (the diagonal over 784,120).

Exercise 50.4. A third scene has span 0.40 at 865 nm. What input step does the same cirrus ($\delta = 0.03$) make? What span would make it 0.10?

Exercise 50.5. In variant C, what share of the missed cirrus was called cloud? Why is that error less harmful to the product than calling it clear?

Next: Chapter 51, The verdict, and what would have to change.

Chapter 51

The verdict, and what would have to change

The one thing this chapter teaches

HyperSL is a sound model of what spectra look like inside one scene, and on every one of this product's jobs a design choice that serves that purpose (per-scene scaling, band masking within a pixel, a tokenizer over band index, smoothing heads, cloud-free pretraining) works against the job; making it fit would mean retraining it with our choices, which is what Indradhanu already is.

This chapter puts Chapters 46 to 50 side by side. It adds no new measurement; every number has its source in the chapter that measured it.

Analogy

A superb road bicycle offered to a team that delivers parcels across a ploughed field. Everything that makes it superb (thin tyres, no luggage rack, a riding position for speed) is a deliberate choice for a road, and each one is a problem in a field. You could fit fat tyres, a rack and a new frame; at that point you have built a different bicycle, and the team already owns one.

51.1 The decision table

requirement	HyperSL	this product	evidence
the same spectrum has the same input in every scene	no: per-band min-max over the scene	`WavelengthStats`, one fixed curve	worked examples 47.1 and 47.2
a library spectrum has an input	none; the scene frame clipped 41% of ECOSTRESS	the same curve, the same stem	Section 47.2
anomalies matched to man-made library entries	below SAM on 3 of 4 scenes; one leaf took 59.6% of set2	Simplex ACE (adopted), the namer (in training)	Table 47.2
band lists with removed windows	convolution over index; a 5-band window spans 217 nm	40 nm wavelength bins	Table 47.1

requirement	HyperSL	this product	evidence
a pixel must not help rebuild itself	rebuilds a pixel's bands from its own bands	hides whole pixels, rebuilds from neighbours	worked example 48.1
detection on the 909-object bench	own residual: 130 objects, pAUC 0.018; with our scorer: 350, pAUC 0.083, below PCA-RX's 0.128 with no model	FTH3 ep. 5: pAUC 0.189, 449 objects	Table 48.1
sub-pixel targets kept	0.25 m target: object AUC 0.68	ACE 0.97	Table 49.3
blind unmixing on exact truth	worse than VCA + FCLS on every set	VCA + FCLS	Table 49.1
thin cloud found	28.9% of EnMAP's cirrus	67.9% from raw bands; Indradhanu's head	worked example 50.1

51.1.1 Where HyperSL did better

A verdict that lists only losses is not a fair one. In the same experiments HyperSL was ahead on five measures:

- its embedding groups a spectral library slightly better than raw angle (5-neighbour same-class share 0.965 against 0.949 on ECOSTRESS, Section 47.4);
- the cloud classifier on its features found more thick cloud (87.1% against 80.4%) and made fewer false calls on the high plateau (4.34% against 6.62%, Chapter 50);
- as a second-stage filter on Indradhanu's candidates, its embeddings removed more false alarms at 90% object recall when the cut was set with hindsight (Section 48.3);
- on Samson, Jasper and Urban it agreed with the FCLS references better than VCA + FCLS (Table 49.1);
- it reproduces its paper's clustering and fine-tuning claims (Section 46.4).

None of these is one of the product's jobs as the product does them, and the third did not survive an honest threshold.

51.2 What it costs

cost	what the records show
compute	HyperSL needs a GPU: 1,773 s of laptop-GPU time for the four bench scenes, 438 microseconds per pixel at 4 draws (Section 48.2); Indradhanu Heavy runs at 454 s per scene on 16 CPU threads in the product worker. The unmixing variant took 5–13 minutes per scene on a 96 GB GPU, against seconds for VCA + FCLS.
licence	no LICENSE file on GitHub; the Hugging Face card says MIT. Unresolved for a product.
data	the ∼265 GB pretraining corpus is on Baidu Netdisk only. We cannot audit what it learned from, or retrain it with our scaling, without it.
paper against code	seven surprises found only by reading the code: base 100 against 10,000 in the position code; "layer-by-layer normalization" that is per-scene min-max; 300 against 200 million spectra; a 439-band airborne set not in `train.py`; the "endmember-free" code that starts from VCA; downstream scripts that never load the checkpoint; a final encoder layer with no skip connection.

51.3 What would have to change

Each obstacle has a fix, and each fix is a change to how the model is *trained*, not to how it is called.

1. **Scaling.** Train on reflectance with a fixed per-wavelength normalisation, so a pixel's input is a property of the pixel and a library spectrum has one (Chapter 47).
2. **Tokenizer.** Make the first layer read wavelengths, not band positions, so a removed window is a gap rather than a neighbour (Section 47.3).
3. **Masking and context.** Hide whole pixels and give the model their neighbours, so it cannot complete an anomaly from itself (Chapter 48).
4. **Data.** Keep clouds in pretraining and label them (Chapter 50).
5. **Heads.** Judge any per-pixel head at fills of 0.1–0.25, and do not smooth abundances across pixels (Chapter 49).
6. **Access.** Publish the training corpus somewhere it can be read, and a licence in the code repository.

Items 1–4 together describe a wavelength-aware spatial masked autoencoder pretrained on reflectance with fixed statistics: Indradhanu on its SpectralEarth stem (Chapters 24 and 27). The honest summary is not that HyperSL is bad. It is that **the changes it would need are the design we already chose, for the same reasons.**

51.3.1 What we took from it

- **Wavelength as a coordinate.** HyperSL's position code accepts any band list; our stem goes further and builds its bins from wavelength, but the idea is shared.
- **A discipline for outside models.** Check the checkpoint's hash, read the code before the paper, reproduce a published number before measuring anything new, and keep the outputs. Every claim in this part could be checked because those steps were followed, and the one that could not (the 28% of Section 47.4) is the one whose output was not kept.

Values that will drift

Every number in the decision table belongs to a dated run (2026-09-23 to 2026-10-31) and to HyperSL's `10_base_mask95` release. A new release (different scaling, spatial context, a licence) would reopen each row.

51.4 Common confusions

The part shows HyperSL is a bad model.
It shows a good model for one-scene land-cover description, measured on jobs it was not built for.

Fine-tuning would fix it.
Fine-tuning keeps the input scaling, the band masking and the tokenizer. The obstacles are in what the model was trained to do, so they need a retrain.

Its false-alarm filter result means it helps detection.
The gain needed a threshold chosen with the test scene's labels; with a threshold set on other scenes it lost objects (Section 48.3).

The licence is fine because Hugging Face says MIT.
The code repository has no licence file. For a product, that needs settling with the authors.

51.5 Check yourself

Exercise 51.1. For each of the four jobs (matching, detection, unmixing, cloud), name the single design choice of HyperSL most responsible for its result, and the chapter that shows it.

Exercise 51.2. Which of the six changes in Section 51.3 could be made without retraining? Explain.

Exercise 51.3. From Table 48.1, how many more objects does FTH3 epoch 5 catch in the top 1% than HyperSL RM, both with cloud excluded? What share of the 909 is that, and which bootstrap row agrees?

Exercise 51.4. List the five measures on which HyperSL was ahead, and say for each why it does not change the decision.

Appendix A

Answers to the Check-yourself exercises

Every exercise in the book is solved here in full, chapter by chapter. Try each one on paper before you read its solution: reading a solution is not the same as finding it.

Chapter 1: The problem: finding what does not belong

Solution to Exercise 1.1. (1) No labelled anomalies exist, because labelling a satellite pixel needs someone to have confirmed what is on the ground; this closes the route "collect training data". (2) The target class is open-ended: a classifier can only return labels it was given, so it is blind to the unanticipated anomaly that matters most; this closes the route "label a few known kinds and hope". (3) Anomalies are so rare that a rule answering "normal" every time scores near-perfect accuracy; this closes the route "train anyway and check the accuracy", because the check itself is meaningless.

Solution to Exercise 1.2. The rule is right on every normal pixel and wrong on every anomalous one.

$$\begin{aligned} \text{normal pixels} &= 1{,}000{,}000 - 10 = 999{,}990, \\ \text{accuracy} &= \frac{999{,}990}{1{,}000{,}000} = 0.99999 = 99.999\%, \\ \text{base rate} &= \frac{10}{1{,}000{,}000} = 0.00001 = 10^{-5}. \end{aligned}$$

The rule finds none of the ten anomalies and still scores 99.999%.

Solution to Exercise 1.3. Mean:

$$\begin{aligned} 10 + 12 + 14 + 20 &= 56, \\ \mu &= 56/4 = 14. \end{aligned}$$

Differences from the mean and their squares:

$$\begin{aligned} 10 - 14 &= -4, & 16, \\ 12 - 14 &= -2, & 4, \\ 14 - 14 &= 0, & 0, \\ 20 - 14 &= 6, & 36. \end{aligned}$$

Average of the squares and its root:

$$\begin{aligned} 16 + 4 + 0 + 36 &= 56, \\ \sigma^2 &= 56/4 = 14, \\ \sigma &= \sqrt{14} \approx 3.742. \end{aligned}$$

Score of the pixel reading 20:

$$s = \frac{|20 - 14|}{3.742} = \frac{6}{3.742} \approx 1.60.$$

Solution to Exercise 1.4.

$$k = 0.0005 \times 2{,}000{,}000 = 1{,}000 \text{ pixels}.$$

Doubling every score does not change the kept set. Multiplying all scores by the same positive number preserves their order: if $s_a > s_b$ then $2s_a > 2s_b$. The top 1,000 by rank are the same pixels; only the cut-off value (the 99.95 percentile) doubles with them.

Solution to Exercise 1.5. Differences band by band:

$$\begin{aligned} 0.05 - 0.07 &= -0.02, \\ 0.25 - 0.22 &= 0.03, \\ 0.40 - 0.40 &= 0.00. \end{aligned}$$

Absolute values 0.02, 0.03, 0.00; their sum is 0.05; divided by 3 the score is $0.05/3 \approx 0.0167$.

A pixel stored as 0 is not a measurement, but its values are far from every real pixel, so any score that measures distance from normal ranks it near the top. It also distorts the scene statistics (mean and spread) that the genuine pixels are scored against, as worked example 1.6 showed, so it must be excluded both from the statistics and from the ranking.

Chapter 2: Light, bands and the hyperspectral cube

Solution to Exercise 2.1. By Equation (2.8),

$$\sigma = 12/2.3548 \approx 5.096 \text{ nm}.$$

At $\Delta = 3$ nm:

$$\begin{aligned} \Delta^2 &= 9, \\ \sigma^2 &= 5.096^2 \approx 25.97, \\ 2\sigma^2 &\approx 51.94, \\ \Delta^2/(2\sigma^2) &= 9/51.94 \approx 0.1733, \\ w &= \exp(-0.1733) \approx 0.841. \end{aligned}$$

Three nanometres from the centre the band still responds at about 84% of its peak; the half point is at 6 nm, half the FWHM.

Solution to Exercise 2.2. Products of weight and reflectance:

$$\begin{aligned} 0.5 \times 0.30 &= 0.15, \\ 1 \times 0.26 &= 0.26, \\ 0.5 \times 0.18 &= 0.09. \end{aligned}$$

Sum of products $0.15 + 0.26 + 0.09 = 0.50$; sum of weights $0.5 + 1 + 0.5 = 2$; band value $0.50/2 = 0.25$. The reflectance at the centre is 0.26. The spectrum falls more steeply on the long-wavelength side ($0.26 \to 0.18$, a drop of 0.08) than it rises on the short side ($0.30 \to 0.26$, a drop of 0.04 going the other way), so the weighted average is pulled below the centre value.

Solution to Exercise 2.3.

$$\begin{aligned} 0.00341802 \times 41{,}000 &= 140.13882, \\ T_{\mathrm{K}} &= 140.13882 + 149.0 = 289.13882 \text{ K}, \\ T_{^\circ\mathrm{C}} &= 289.13882 - 273.15 = 15.98882 \approx 15.99\ ^\circ\mathrm{C}. \end{aligned}$$

About 16 degrees: ordinary ground on a mild day.

Solution to Exercise 2.4. Pixel $(1, 0)$ reads position $(1, 0)$ of each band matrix: $\mathbf{X}[0, 1, 0] = 0.05$, $\mathbf{X}[1, 1, 0] = 0.30$, $\mathbf{X}[2, 1, 0] = 0.14$, so $\mathbf{x}_{10} = (0.05,\ 0.30,\ 0.14)$. The 1,650 nm band is the third matrix,

$$\mathbf{X}[2] = \begin{pmatrix} 0.14 & 0.10 \\ 0.14 & 0.10 \end{pmatrix}.$$

Compare with $\mathbf{x}_{00} = (0.03,\ 0.20,\ 0.14)$ component by component:

$$\begin{aligned} 0.05/0.03 &\approx 1.667, \\ 0.30/0.20 &= 1.5, \\ 0.14/0.14 &= 1. \end{aligned}$$

The ratios differ, so pixel $(1, 0)$ is not a scaled copy of pixel $(0, 0)$: the two have different spectral shapes, not only different brightness.

Solution to Exercise 2.5. The denominator is unchanged, $E \cos\theta_s = 1.2990$. Numerator: $\pi \times 0.05 = 0.15708$. Reflectance: $0.15708/1.2990 \approx 0.121$, half of the first surface's 0.242, as it must be since the radiance is half.

Radiance depends on the sun's angle, the atmosphere and the instrument as well as the surface; the same field gives different radiances on different days and through different sensors. Reflectance is the surface's own ratio, so a model trained on reflectance from one sensor sees the same kind of number from another.

Chapter 3: Five sensors and the 165-band grid

Solution to Exercise 3.1. Step 10:

$$\frac{2{,}500 - 400}{10} + 1 = \frac{2{,}100}{10} + 1 = 210 + 1 = 211 \text{ rungs}.$$

Step 5:

$$\frac{2{,}500 - 400}{5} + 1 = \frac{2{,}100}{5} + 1 = 420 + 1 = 421 \text{ rungs}.$$

Halving the step does not exactly double the count, because of the fencepost +1: $2 \times 211 = 422 \neq 421$.

Solution to Exercise 3.2. By Equation (3.5):

$$\begin{aligned} \lceil 1{,}125/10 \rceil &= \lceil 112.5 \rceil = 113, \\ \lfloor 1{,}160/10 \rfloor &= \lfloor 116 \rfloor = 116, \\ 116 - 113 + 1 &= 4 \quad (1{,}130,\ 1{,}140,\ 1{,}150,\ 1{,}160). \end{aligned}$$

The windows now remove $0+6+4+11+16=37$ rungs, so the grid would have $200-37=163$ bands. Every saved model, statistics file and shard built for 165 would then be misaligned, which is the wire-format warning of Section 3.10.

Solution to Exercise 3.3. Band 70: $62 \le 70 < 81$, so $(c_0, \lambda_0) = (62, 1{,}160)$:

$$\lambda(70) = 1{,}160 + 10 \times (70 - 62) = 1{,}160 + 80 = 1{,}240 \text{ nm}.$$

Band 164: $115 \le 164$, so $(c_0, \lambda_0) = (115, 1{,}960)$:

$$\lambda(164) = 1{,}960 + 10 \times 49 = 1{,}960 + 490 = 2{,}450 \text{ nm},$$

the last band, as expected. Index of 2,210 nm (fifth segment):

$$c = 115 + (2{,}210 - 1{,}960)/10 = 115 + 25 = 140.$$

Solution to Exercise 3.4. With $(c, i, j) = (2, 1, 3)$, $C = 4$, $H = 3$, $W = 5$:

$$\begin{aligned}
\text{BSQ: } & c\,HW + i\,W + j = 2 \times 15 + 1 \times 5 + 3 = 30 + 5 + 3 = 38, \\
\text{BIL: } & i\,CW + c\,W + j = 1 \times 20 + 2 \times 5 + 3 = 20 + 10 + 3 = 33, \\
\text{BIP: } & i\,WC + j\,C + c = 1 \times 20 + 3 \times 4 + 2 = 20 + 12 + 2 = 34.
\end{aligned}$$

Solution to Exercise 3.5.

$$\begin{aligned}
21{,}840 - 16{,}384 &= 5{,}456 \quad (2^{14}), \\
5{,}456 - 4{,}096 &= 1{,}360 \quad (2^{12}), \\
1{,}360 - 1{,}024 &= 336 \quad (2^{10}), \\
336 - 256 &= 80 \quad (2^{8}), \\
80 - 64 &= 16 \quad (2^{6}), \\
16 - 16 &= 0 \quad (2^{4}).
\end{aligned}$$

Bit 4 (cloud shadow) is set, so the builder marks the pixel *invalid*. (It is 21,824 from worked example 3.4 plus $2^4 = 16$.)

EnMAP: $\rho = 1{,}830 \times 0.0001 = 0.183$, an 18.3% reflectance.

Chapter 4: The product: scenes, projects, actions and the vendable

Solution to Exercise 4.1. One Scene (the uploaded file). Two Projects, one per band-filter setting, both bound to that Scene. In each Project: one `band_filter_apply`, one `anomaly_scoring` (Indradhanu and MNF-RX are two models ticked inside the same Action, not two Actions), and two `anomaly_detection_prep` Actions, one per threshold attempt:

$$1 + 1 + 2 = 4 \text{ Actions per Project}, \qquad 2 \times 4 = 8 \text{ Actions in total}.$$

Solution to Exercise 4.2. In-degrees (number of incoming arrows):

$$B\colon 0, \quad G\colon 1, \quad S\colon 2, \quad P\colon 1, \quad Q\colon 1, \quad M\colon 1.$$

Kahn's algorithm: list B (removing its arrows makes $G{:}\,0$, $S{:}\,1$); list G ($S{:}\,0$); list S ($P{:}\,0$, $Q{:}\,0$). Now there is a choice. Two valid orders:

$$B, G, S, P, M, Q \qquad \text{and} \qquad B, G, S, Q, P, M.$$

An edge $M \to B$ would mean the band filter reads an output of the spectral match. B is created first and can only name outputs that exist at its creation, and M does not exist yet; moreover the edge would close the cycle $B \to S \to P \to M \to B$, which a DAG cannot contain.

Solution to Exercise 4.3. $H = 2$, $W = 3$, so $HW = 6$. Band 0:

$$s_0 = 100 \times \frac{1+0+1+1+1+0}{6} = 100 \times \frac{4}{6} \approx 66.67.$$

Per pixel, sum the two bands at each position:

$$\begin{pmatrix} 1+1 & 0+0 & 1+1 \\ 1+1 & 1+1 & 0+0 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 2 \\ 2 & 2 & 0 \end{pmatrix} \;\Rightarrow\; m = \begin{pmatrix} 1 & 0 & 1 \\ 1 & 1 & 0 \end{pmatrix}.$$

Valid fraction $4/6 \approx 0.667$, equal to $s_0/100$ because validity is all-or-nothing per pixel.

Solution to Exercise 4.4. Pixel $(i, j) = (0, 0)$:

$$x = 30 \times 0 + 0 + 500{,}000 = 500{,}000, \qquad y = 0 + (-30) \times 0 + 2{,}000{,}000 = 2{,}000{,}000,$$

the top-left corner, as the definition of (c, f) says. Pixel $(100, 0)$:

$$x = 500{,}000, \qquad y = -30 \times 100 + 2{,}000{,}000 = 2{,}000{,}000 - 3{,}000 = 1{,}997{,}000,$$

three kilometres south. To invert (no rotation, so $b = d = 0$), solve each line for its pixel index:

$$x = aj + c \;\Rightarrow\; j = \frac{x - c}{a} = \frac{501{,}500 - 500{,}000}{30} = \frac{1{,}500}{30} = 50,$$
$$y = ei + f \;\Rightarrow\; i = \frac{y - f}{e} = \frac{1{,}998{,}500 - 2{,}000{,}000}{-30} = \frac{-1{,}500}{-30} = 50.$$

The pixel is row 50, column 50.

Solution to Exercise 4.5. With a 60 s timeout: during a 45 s freeze no heartbeat is written. The last heartbeat before the freeze was at most 5 s before it began, so at the moment the worker resumes the heartbeat's age is at most $45 + 5 = 50$ s, below 60. Not reaped; the next heartbeat refreshes it.

With a 12 s timeout: after 12 s without a heartbeat (at most 12 s into the freeze) the reaper marks the job `failed` although the worker is alive. When the worker resumes, it is working on a job the system has already declared failed. This is why the source asks for a timeout of at least about three heartbeat intervals ($3 \times 5 = 15$ s), and why the default (60 s, twelve intervals) is generous.

Chapter 5: Making sensors comparable: band filtering, interpolation and normalisation

Solution to Exercise 5.1. Weight form. The fraction of the way:

$$t = \frac{1{,}610 - 1{,}600}{1{,}612 - 1{,}600} = \frac{10}{12} \approx 0.8333.$$

Weights $1 - t \approx 0.1667$ and $t \approx 0.8333$:

$$\begin{aligned} y &= 0.1667 \times 0.20 + 0.8333 \times 0.26 \\ &= 0.03333 + 0.21667 \\ &= 0.25000. \end{aligned}$$

Two-point form. Slope:

$$m = \frac{0.26 - 0.20}{1{,}612 - 1{,}600} = \frac{0.06}{12} = 0.005 \text{ per nm.}$$

Then

$$y = 0.20 + 0.005 \times (1{,}610 - 1{,}600) = 0.20 + 0.05 = 0.25.$$

Both give 0.25.

Solution to Exercise 5.2. A pixel is wiped when $u/C' > 0.4$, so it is kept when $u \leq 0.4 \times 200 = 80$. The largest number of invalid bands for a kept pixel is 80. A pixel with exactly 80 invalid bands has $f = 80/200 = 0.4$, and $0.4 > 0.4$ is false, so it is kept and its 80 holes are filled at stage 7. Had the code used "$\geq$", the same pixel would be wiped: exactly at the threshold, the choice of comparison decides.

Solution to Exercise 5.3. `np.interp` assumes ascending wavelengths and first checks whether the query is at or beyond the *last listed* wavelength, here 700. Since $750 \geq 700$, it answers "beyond the end" and returns the last listed value, 0.1. The correct answer: sort to $(700, 800, 900)$ with values $(0.1, 0.2, 0.3)$; 750 lies between 700 and 800 with $t = 50/100 = 0.5$, so $y = 0.5 \times 0.1 + 0.5 \times 0.2 = 0.05 + 0.10 = 0.15$.

Solution to Exercise 5.4. With $\mu = 0.25$, $\sigma = 0.05$:

$$\begin{aligned} (0.15 - 0.25)/0.05 &= -0.10/0.05 = -2, \\ (0.25 - 0.25)/0.05 &= 0/0.05 = 0, \\ (0.40 - 0.25)/0.05 &= 0.15/0.05 = 3, \\ (0 - 0.25)/0.05 &= -0.25/0.05 = -5. \end{aligned}$$

The invalid pixel lands at -5, far outside the typical range, because the code masks before it normalises. Denormalise $z = -0.6$:

$$x = \sigma z + \mu = 0.05 \times (-0.6) + 0.25 = -0.03 + 0.25 = 0.22.$$

Round trip for 0.40: $N(0.40) = 3$, and $D(3) = 0.05 \times 3 + 0.25 = 0.15 + 0.25 = 0.40$. Algebraically, $D(N(x)) = \sigma \frac{x-\mu}{\sigma} + \mu = x - \mu + \mu = x$ for any x.

Solution to Exercise 5.5. Valid positions are 1 (0.20) and 5 (0.35).

- Position 0: distance 1 to position 1, 5 to position 5: 0.20.
- Position 2: distance 1 to position 1, 3 to position 5: 0.20.
- Position 3: distance 2 to position 1 and 2 to position 5: a *tie*; the implementation chooses one of 0.20 and 0.35.
- Position 4: distance 3 to position 1, 1 to position 5: 0.35.

Result $0.20, 0.20, 0.20, (0.20 \text{ or } 0.35), 0.35, 0.35$.

The fill copies a real measurement so that every value the network's first layer sees is physically plausible. An average of two neighbours could be a spectrum that exists nowhere in the scene, and in a system that hunts for values that do not fit, an invented value could itself look anomalous.

Chapter 6: The mathematical toolkit: vectors, projection, covariance, eigenvectors, PCA and whitening

Solution to Exercise 6.1. Let $\mathbf{a} = (1, 2, 2)$ and $\mathbf{b} = (2, 1, 2)$.

$$\begin{aligned}
\mathbf{a}^\top \mathbf{b} &= 1 \times 2 + 2 \times 1 + 2 \times 2 = 2 + 2 + 4 = 8 \\
\|\mathbf{a}\| &= \sqrt{1 + 4 + 4} = \sqrt{9} = 3 \\
\|\mathbf{b}\| &= \sqrt{4 + 1 + 4} = \sqrt{9} = 3 \\
\cos\theta &= \frac{8}{3 \times 3} = \frac{8}{9} = 0.8889 \\
\theta &= \arccos(0.8889) = 0.4759 \text{ rad} = 0.4759 \times 180/3.14159 = 27.3^\circ.
\end{aligned}$$

Projection of $\mathbf{a}$ onto $\mathbf{b}$:

$$\begin{aligned}
t^\star &= \frac{\mathbf{b}^\top \mathbf{a}}{\mathbf{b}^\top \mathbf{b}} = \frac{8}{9} \\
\mathbf{p} &= \tfrac{8}{9}(2, 1, 2) = (\tfrac{16}{9}, \tfrac{8}{9}, \tfrac{16}{9}) = (1.778, 0.889, 1.778) \\
\mathbf{r} &= \mathbf{a} - \mathbf{p} = (1 - \tfrac{16}{9},\ 2 - \tfrac{8}{9},\ 2 - \tfrac{16}{9}) = (-\tfrac{7}{9}, \tfrac{10}{9}, \tfrac{2}{9}).
\end{aligned}$$

Orthogonality: $\mathbf{b}^\top \mathbf{r} = 2 \times (-\frac{7}{9}) + 1 \times \frac{10}{9} + 2 \times \frac{2}{9} = \frac{-14+10+4}{9} = 0$.

Solution to Exercise 6.2. Mean: band 1 $(1 + 3 + 2)/3 = 6/3 = 2$; band 2 $(2 + 2 + 5)/3 = 9/3 = 3$. Deviations: $(1 - 2, 2 - 3) = (-1, -1)$, $(3 - 2, 2 - 3) = (1, -1)$, $(2 - 2, 5 - 3) = (0, 2)$. Outer products:

$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \qquad \begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}, \qquad \begin{pmatrix} 0 & 0 \\ 0 & 4 \end{pmatrix}.$$

Sum: $(1, 1)$ entry $1 + 1 + 0 = 2$; $(1, 2)$ entry $1 - 1 + 0 = 0$; $(2, 2)$ entry $1 + 1 + 4 = 6$. Divide by $N - 1 = 2$:

$$\boldsymbol{\Sigma} = \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 0 & 6 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 3 \end{pmatrix}.$$

The covariance is 0, so the correlation $0/(\sqrt{1} \times \sqrt{3})$ is 0: in this sample the two bands are uncorrelated, and the principal directions are the band axes themselves.

Solution to Exercise 6.3. Trace $5 + 2 = 7$, determinant $5 \times 2 - 2 \times 2 = 10 - 4 = 6$. Characteristic polynomial $\lambda^2 - 7\lambda + 6 = 0$, which factorises as $(\lambda - 6)(\lambda - 1)$, so $\lambda_1 = 6$, $\lambda_2 = 1$.

For $\lambda_1 = 6$: $(5 - 6)u_1 + 2u_2 = 0$, so $u_1 = 2u_2$; take $(2, 1)$, length $\sqrt{5}$, $\mathbf{u}_1 = (2, 1)/\sqrt{5} = (0.8944, 0.4472)$. Check: $(5 \times 2 + 2 \times 1, 2 \times 2 + 2 \times 1) = (12, 6) = 6 \times (2, 1)$.

For $\lambda_2 = 1$: $(5 - 1)u_1 + 2u_2 = 0$, so $u_2 = -2u_1$; take $(1, -2)$, $\mathbf{u}_2 = (1, -2)/\sqrt{5}$. Check: $(5 - 4, 2 - 4) = (1, -2)$.

Explained variance ratio of the first component: $6/(6 + 1) = 6/7 = 0.857$.

Rebuild:

$$6 \times \frac{1}{5}\begin{pmatrix} 4 & 2 \\ 2 & 1 \end{pmatrix} + 1 \times \frac{1}{5}\begin{pmatrix} 1 & -2 \\ -2 & 4 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 24 + 1 & 12 - 2 \\ 12 - 2 & 6 + 4 \end{pmatrix} = \frac{1}{5}\begin{pmatrix} 25 & 10 \\ 10 & 10 \end{pmatrix} = \begin{pmatrix} 5 & 2 \\ 2 & 2 \end{pmatrix}.$$

Solution to Exercise 6.4.

$$\mathbf{W} = \begin{pmatrix} 1/\sqrt{6} & 0 \\ 0 & 1 \end{pmatrix} \frac{1}{\sqrt{5}}\begin{pmatrix} 2 & 1 \\ 1 & -2 \end{pmatrix} = \begin{pmatrix} 2/\sqrt{30} & 1/\sqrt{30} \\ 1/\sqrt{5} & -2/\sqrt{5} \end{pmatrix}.$$

First $\boldsymbol{\Sigma}\mathbf{W}^\top$, column by column. Column 1 of $\mathbf{W}^\top$ is $(2,1)/\sqrt{30}$: $\boldsymbol{\Sigma}(2,1) = (10{+}2, 4{+}2) = (12,6)$, so the column is $(12,6)/\sqrt{30}$. Column 2 is $(1,-2)/\sqrt{5}$: $\boldsymbol{\Sigma}(1,-2) = (5-4, 2-4) = (1,-2)$, so the column is $(1,-2)/\sqrt{5}$. Now multiply by $\mathbf{W}$:

$$\begin{aligned}
(1,1): &\quad \frac{2\times 12 + 1\times 6}{\sqrt{30}\sqrt{30}} = \frac{30}{30} = 1 \\
(1,2): &\quad \frac{2\times 1 + 1\times(-2)}{\sqrt{30}\sqrt{5}} = \frac{0}{\sqrt{150}} = 0 \\
(2,1): &\quad \frac{1\times 12 + (-2)\times 6}{\sqrt{5}\sqrt{30}} = \frac{0}{\sqrt{150}} = 0 \\
(2,2): &\quad \frac{1\times 1 + (-2)\times(-2)}{\sqrt{5}\sqrt{5}} = \frac{5}{5} = 1.
\end{aligned}$$

So $\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^\top = \mathbf{I}$.

Solution to Exercise 6.5. By Equation (6.13), $\boldsymbol{\Sigma}_z = \boldsymbol{\Lambda}_d^{-1/2}\mathbf{U}_d^\top\boldsymbol{\Sigma}\mathbf{U}_d\boldsymbol{\Lambda}_d^{-1/2}$. Replace $\boldsymbol{\Sigma}$ by $\mathbf{U}\boldsymbol{\Lambda}\mathbf{U}^\top$. The product $\mathbf{U}^\top\mathbf{U}_d$ is a $C\times d$ matrix whose entry (j,k) is $\mathbf{u}_j^\top\mathbf{u}_k$ for $k \le d$; because the eigenvectors are orthogonal unit vectors this is 1 when $j=k$ and 0 otherwise, so $\mathbf{U}^\top\mathbf{U}_d = \left(\begin{smallmatrix}\mathbf{I}_d\\ \mathbf{0}\end{smallmatrix}\right)$. Likewise $\mathbf{U}_d^\top\mathbf{U} = (\mathbf{I}_d \;\; \mathbf{0})$. Then

$$\mathbf{U}_d^\top\boldsymbol{\Sigma}\mathbf{U}_d = (\mathbf{I}_d \;\; \mathbf{0})\,\boldsymbol{\Lambda}\begin{pmatrix}\mathbf{I}_d\\ \mathbf{0}\end{pmatrix} = \boldsymbol{\Lambda}_d,$$

the top-left $d\times d$ block of $\boldsymbol{\Lambda}$. Finally $\boldsymbol{\Lambda}_d^{-1/2}\boldsymbol{\Lambda}_d\boldsymbol{\Lambda}_d^{-1/2} = \mathbf{I}_d$. The proof uses the orthonormality of the eigenvectors, $\mathbf{u}_j^\top\mathbf{u}_k = 1$ if $j=k$ and 0 otherwise.

Solution to Exercise 6.6. Group the product as $(\mathbf{AB})\mathbf{C}$ and apply Equation (6.8) with $\mathbf{AB}$ in place of $\mathbf{A}$: $((\mathbf{AB})\mathbf{C})^\top = \mathbf{C}^\top(\mathbf{AB})^\top$. Apply the rule again to $(\mathbf{AB})^\top = \mathbf{B}^\top\mathbf{A}^\top$, giving $\mathbf{C}^\top\mathbf{B}^\top\mathbf{A}^\top$.

With $\mathbf{B} = \boldsymbol{\Sigma}$ and $\mathbf{C} = \mathbf{A}^\top$: $(\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)^\top = (\mathbf{A}^\top)^\top\boldsymbol{\Sigma}^\top\mathbf{A}^\top = \mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$, because transposing twice gives back the original and $\boldsymbol{\Sigma}^\top = \boldsymbol{\Sigma}$. So the transformed covariance is symmetric whenever the original is.

Chapter 7: Anomaly as surprise: the Gaussian, Mahalanobis distance and the RX detector

Solution to Exercise 7.1.

$$\begin{aligned}
z &= \frac{61 - 24.58}{13.57} = \frac{36.42}{13.57} = 2.684 \\
\mathrm{RX} &= z^2 = 2.684 \times 2.684 = 7.20.
\end{aligned}$$

The χ^2_1 threshold for a 1% false-alarm rate is 6.63 (the square of 2.576). Since $7.20 > 6.63$ the pixel is beyond it; its tail probability is $P(|z| > 2.684) = 0.0073$, about one pixel in 140. One-band RX is the squared z-score, which is why `ThermalGRXDetector` computes it directly instead of calling a matrix routine.

Solution to Exercise 7.2. $\det\boldsymbol{\Sigma} = 5\times 2 - 2\times 2 = 6$, so $\boldsymbol{\Sigma}^{-1} = \frac{1}{6}\left(\begin{smallmatrix}2 & -2\\ -2 & 5\end{smallmatrix}\right)$.

For $\mathbf{d} = (1,-2)$: $\boldsymbol{\Sigma}^{-1}\mathbf{d} = \frac{1}{6}(2\times 1 - 2\times(-2),\; -2\times 1 + 5\times(-2)) = \frac{1}{6}(6,-12) = (1,-2)$, and $D^2 = 1\times 1 + (-2)\times(-2) = 1 + 4 = 5$.

For $\mathbf{d} = (2, 1)$: $\boldsymbol{\Sigma}^{-1}\mathbf{d} = \frac{1}{6}(4 - 2,\ -4 + 5) = \frac{1}{6}(2, 1) = (\frac{1}{3}, \frac{1}{6})$, and $D^2 = 2 \times \frac{1}{3} + 1 \times \frac{1}{6} = \frac{4}{6} + \frac{1}{6} = \frac{5}{6} = 0.833$.

From Exercise 6.3 the eigenvectors are $(2, 1)/\sqrt{5}$ with eigenvalue 6 and $(1, -2)/\sqrt{5}$ with eigenvalue 1. The first pixel lies exactly along the low-variance eigenvector, so $D^2 = \|\mathbf{d}\|^2 / 1 = 5$; the second lies along the high-variance one, so $D^2 = \|\mathbf{d}\|^2 / 6 = 5/6$. Same length, six times the score.

Solution to Exercise 7.3.

$$\begin{aligned}\boldsymbol{\Sigma}^{-1}\mathbf{d} &= \tfrac{1}{4}\big(3 \times 0 - 2 \times 2 + 1 \times 0,\ -2 \times 0 + 4 \times 2 - 2 \times 0,\\ &\qquad 1 \times 0 - 2 \times 2 + 3 \times 0\big)\\ &= \tfrac{1}{4}(-4, 8, -4) = (-1, 2, -1),\end{aligned}$$

so $D^2 = 0 \times (-1) + 2 \times 2 + 0 \times (-1) = 4$. The per-band score is $2^2/2 = 2$. Mahalanobis scores it twice as high because the middle band rose while its two correlated neighbours did not. Through the eigenvectors of worked example 7.4: $\mathbf{u}_1^\top \mathbf{d} = 2 \times 0.7071 = 1.414$, $\mathbf{u}_2^\top \mathbf{d} = 0$, $\mathbf{u}_3^\top \mathbf{d} = -1.414$, so $D^2 = 2/3.414 + 0 + 2/0.586 = 0.586 + 3.414 = 4.0$. A score of 4 is ordinary for χ^2_3 (tail probability 0.26).

Solution to Exercise 7.4. With variances 1 and correlation $\rho = 0.99$ the eigenvalues are $1 \pm \rho$: 1.99 along $(1, 1)/\sqrt{2}$ and 0.01 along $(1, -1)/\sqrt{2}$. Condition number $1.99/0.01 = 199$. For $\mathbf{d} = (0.1, -0.1)$ the coordinate along $(1, -1)/\sqrt{2}$ is $0.2/\sqrt{2} = 0.1414$, squared 0.02, and along $(1, 1)/\sqrt{2}$ it is 0. So $D^2 = 0.02/0.01 = 2$.

After loading with $\epsilon = 0.01$ the eigenvalues are $1.99 + 0.01 = 2.00$ and $0.01 + 0.01 = 0.02$, the condition number is $2.00/0.02 = 100$, and $D^2 = 0.02/0.02 = 1$. Loading halved the score of a pixel that differs only along the low-variance direction.

Solution to Exercise 7.5. 23.21 is the χ^2_{10} threshold for a 1% false-alarm rate, so a Gaussian background of $1000 \times 1000 = 1{,}000{,}000$ pixels would put $0.01 \times 1{,}000{,}000 = 10{,}000$ pixels above it. Real scenes exceed it far more often because (1) they are mixtures of surfaces, not one Gaussian cloud, so pixels of a minority surface sit far from the global mean (worked example 7.5); (2) they have heavy tails from edges, detector stripes, glint and haze; and (3) the mean and covariance are themselves estimates, contaminated by the anomalies and other outliers. This is why the product thresholds by percentile.

Solution to Exercise 7.6. From Section 7.7, for $C = 2$, $P(D^2 > t) = e^{-t/2}$. Set it equal to α:

$$\begin{aligned} e^{-t/2} &= \alpha \\ -t/2 &= \ln \alpha && \text{take natural logarithms} \\ t &= -2 \ln \alpha = 2 \ln(1/\alpha) && \text{multiply by } -2;\ -\ln \alpha = \ln(1/\alpha). \end{aligned}$$

For $\alpha = 0.001$: $t = 2 \ln 1000 = 2 \times 6.9078 = 13.82$, matching the table.

Chapter 8: Local RX, regularisation and MNF-RX

Solution to Exercise 8.1. Directly:

$$\begin{aligned} \text{outer} &= (2 \times 15 + 1)^2 = 31^2 = 961 \\ \text{guard} &= (2 \times 3 + 1)^2 = 7^2 = 49 \\ n_{\text{ring}} &= 961 - 49 = 912. \end{aligned}$$

With Equation (8.1): $4 \times (15-3) \times (15+3+1) = 4 \times 12 \times 19 = 912$. At a corner only rows and columns 0 to 15 of the outer window lie inside the image, a $16 \times 16 = 256$ square, and the guard keeps rows and columns 0 to 3, $4 \times 4 = 16$. The corner ring has $256 - 16 = 240$ pixels, about a quarter of 912.

Solution to Exercise 8.2. With $i = 0$ the ring is all 24 pixels except the centre: the 16 border pixels (eight 20s, eight 22s) and the 8 inner neighbours (four 20s, four 22s), so twelve 20s and twelve 22s.

$$\begin{aligned} \mu &= \frac{12 \times 20 + 12 \times 22}{24} = \frac{240 + 264}{24} = \frac{504}{24} = 21 \\ \textstyle\sum (x-\mu)^2 &= 12 \times 1 + 12 \times 1 = 24 \\ \sigma^2 &= 24/23 = 1.0435 \\ \mathrm{RX} &= \frac{(25-21)^2}{1.0435} = \frac{16}{1.0435} = 15.33. \end{aligned}$$

A one-pixel target scores as high without a guard as worked example 8.2's target did with one (slightly higher, because 24 background pixels estimate the variance a little better than 16). The guard is needed when the target, or the blur around it, extends beyond the pixel under test; its size should match the largest target you expect.

Solution to Exercise 8.3. Sum of the real values: $0.20 + 0.22 + 0.24 = 0.66$. Correct mean $0.66/3 = 0.22$; dividing by the padded length gives $0.66/6 = 0.11$, exactly half, because half the slots are padding.

Variance with divisor $n - 1 = 2$: deviations $-0.02, 0, 0.02$; squares $0.0004, 0, 0.0004$; sum 0.0008; $\sigma^2 = 0.0008/2 = 0.0004$. Score of 0.30:

$$\frac{(0.30 - 0.22)^2}{0.0004} = \frac{0.0064}{0.0004} = 16.$$

(With the wrong mean 0.11 the score would be $(0.19)^2/\sigma^2_{\text{wrong}}$ with a variance also corrupted by the padded zeros: a meaningless number at exactly the pixels near edges.)

Solution to Exercise 8.4. $\mathbf{W}_n = \mathrm{diag}(1/\sqrt{2}, 1/\sqrt{0.5}) = \mathrm{diag}(0.7071, 1.4142)$. Entry (i, j) of $\mathbf{W}_n \boldsymbol{\Sigma} \mathbf{W}_n$ is Σ_{ij} times the i-th and j-th diagonal entries:

$$\begin{aligned} (1,1): &\quad 3 \times 0.7071 \times 0.7071 = 3 \times 0.5 = 1.5 \\ (1,2): &\quad 1 \times 0.7071 \times 1.4142 = 1 \\ (2,2): &\quad 1.5 \times 1.4142 \times 1.4142 = 1.5 \times 2 = 3. \end{aligned}$$

Trace 4.5, determinant $1.5 \times 3 - 1 = 3.5$; $\lambda^2 - 4.5\lambda + 3.5 = 0$ gives $\lambda = (4.5 \pm \sqrt{20.25 - 14})/2 = (4.5 \pm 2.5)/2$, so $\lambda_1 = 3.5$ and $\lambda_2 = 1$. The first MNF component has $1 + \mathrm{SNR} = 3.5$, $\mathrm{SNR} = 2.5$; the second is pure noise.

Eigenvector for 3.5: $(1.5 - 3.5)v_1 + v_2 = 0$, so $v_2 = 2v_1$, $\mathbf{v}_1 = (1, 2)/\sqrt{5}$. In band space $\mathbf{a}_1 = \mathbf{W}_n \mathbf{v}_1 = (0.7071, 2.8284)/\sqrt{5} = (0.316, 1.265)$: band 2, the cleaner band (noise variance 0.5 against 2), is weighted four times as heavily. Check: the signal covariance is $\boldsymbol{\Sigma} - \boldsymbol{\Sigma}_n = \left(\begin{smallmatrix} 1 & 1 \\ 1 & 1 \end{smallmatrix}\right)$, so $\mathbf{a}_1^\top \boldsymbol{\Sigma}_s \mathbf{a}_1 = (0.316 + 1.265)^2 = 1.581^2 = 2.5$, and the noise variance is $2 \times 0.316^2 + 0.5 \times 1.265^2 = 2 \times 0.1 + 0.5 \times 1.6 = 0.2 + 0.8 = 1$: SNR 2.5.

Solution to Exercise 8.5. Let $\mathbf{d} = \mathbf{x} - \boldsymbol{\mu}$ and $\mathbf{y} = \mathbf{A}\mathbf{d}$. The mean of $\mathbf{y}$ is $\mathbf{A} \times \mathbf{0} = \mathbf{0}$ and its covariance is $\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top$. Its Mahalanobis distance is

$$\begin{aligned} \mathbf{y}^\top (\mathbf{A}\boldsymbol{\Sigma}\mathbf{A}^\top)^{-1} \mathbf{y} &= \mathbf{d}^\top \mathbf{A}^\top (\mathbf{A}^\top)^{-1} \boldsymbol{\Sigma}^{-1} \mathbf{A}^{-1} \mathbf{A}\mathbf{d} \\ &= \mathbf{d}^\top \boldsymbol{\Sigma}^{-1} \mathbf{d} \end{aligned} \qquad \begin{aligned} &(\mathbf{A}\mathbf{d})^\top = \mathbf{d}^\top \mathbf{A}^\top;\ (\mathbf{P}\mathbf{Q}\mathbf{R})^{-1} = \mathbf{R}^{-1}\mathbf{Q}^{-1}\mathbf{P}^{-1} \\ &\mathbf{A}^\top (\mathbf{A}^\top)^{-1} = \mathbf{I},\ \mathbf{A}^{-1}\mathbf{A} = \mathbf{I}. \end{aligned}$$

The full MNF transform is such an $\mathbf{A}$ (rows $\mathbf{a}_k^\top = \mathbf{v}_k^\top \mathbf{W}_n$, a product of an orthogonal matrix and an invertible whitening matrix), so full MNF-RX equals plain RX.

MNF-RX helps because it keeps only the first 10 components. (1) Numerically, the discarded directions include those whose estimates are dominated by noise and rounding; the 10 kept have eigenvalues $1 + \mathrm{SNR}$ well above the noise level, so the 10×10 covariance is well conditioned and well estimated. (2) Statistically, each discarded near-noise component would add a χ_1^2 term to every score and nothing to a target's, so dropping them shrinks the background spread $\sqrt{2C}$ against which a target must stand out.

Solution to Exercise 8.6. A symmetric 30×30 matrix has 30 diagonal entries and $(30 \times 30 - 30)/2 = 435$ distinct off-diagonal ones: $30 + 435 = 465 = 30 \times 31/2$ numbers. The global sample of 200,000 pixels gives $200{,}000/465 = 430$ pixels per number. A local covariance from a 3280-pixel ring would have $3280/465 = 7.1$ pixels per number, far fewer, and every pixel would need its own 30×30 solve. That is one reason PCA-RX and the residual score take only the *mean* from the ring and keep one global covariance.

Chapter 9: Reconstruct, then subtract: why a pixel must never help reconstruct itself

Solution to Exercise 9.1. Reproducing the visible input has a perfect solution, the identity $\mathrm{id}(\mathbf{x}) = \mathbf{x}$, whose loss is exactly zero; training heads towards it, and the identity returns every pixel unchanged, so the residual $\mathbf{x} - \hat{\mathbf{x}}$ is zero everywhere, anomalies included, and the score map is empty. The failure is dangerous because it is silent: nothing crashes and nothing warns, and an empty score map looks exactly like a scene with nothing unusual in it.

Solution to Exercise 9.2. Directly. First the reconstruction:

$$\begin{aligned}\hat{x} &= 0.25 \times 0.80 + 0.75 \times 0.20 \\ &= 0.20 + 0.15 \\ &= 0.35.\end{aligned}$$

Then the residual:

$$r = x - \hat{x} = 0.80 - 0.35 = 0.45.$$

With Equation (9.10), $w = 0.25$ so $1 - w = 0.75$, and the honest surprise is $x - c = 0.80 - 0.20 = 0.60$:

$$r = (1 - w)(x - c) = 0.75 \times 0.60 = 0.45.$$

The two agree. The model reports $0.45/0.60 = 0.75$, that is 75% of the honest surprise; the quarter it lost is exactly the self-weight.

Solution to Exercise 9.3. Predictions with the leave-one-out neighbour average (ends copy their only neighbour):

$$\begin{aligned}\hat{x}_1 &= x_2 = 2.0, \\ \hat{x}_2 &= \tfrac{1}{2}(2.0 + 2.0) = \tfrac{1}{2}(4.0) = 2.0, \\ \hat{x}_3 &= \tfrac{1}{2}(2.0 + 2.0) = 2.0, \\ \hat{x}_4 &= \tfrac{1}{2}(x_3 + x_5) = \tfrac{1}{2}(2.0 + 0.4) = \tfrac{1}{2}(2.4) = 1.2, \\ \hat{x}_5 &= \tfrac{1}{2}(x_4 + x_6) = \tfrac{1}{2}(2.0 + 2.0) = 2.0, \\ \hat{x}_6 &= x_5 = 0.4.\end{aligned}$$

Residuals:

$$r_1 = 2.0 - 2.0 = 0, \qquad r_4 = 2.0 - 1.2 = 0.8,$$
$$r_2 = 2.0 - 2.0 = 0, \qquad r_5 = 0.4 - 2.0 = -1.6,$$
$$r_3 = 2.0 - 2.0 = 0, \qquad r_6 = 2.0 - 0.4 = 1.6.$$

Scores: 0, 0, 0, 0.8, 1.6, 1.6. The anomaly (pixel 5, a dark pixel) scores 1.6, but pixel 6 ties with it. Pixel 6 sits at the end of the signal and has only one neighbour, the anomaly itself, so its whole prediction is the anomalous value and its halo is as large as the anomaly's own score. The halo is pixels 4 (0.8) and 6 (1.6). The lesson: near the edge of the valid data a prediction depends on fewer neighbours and can be badly contaminated, which is why the training code erodes the validity mask near borders (Chapter 18).

Solution to Exercise 9.4. L2 is minimised by the mean. Add the values one at a time:

$$0.9 + 1.0 = 1.9,$$
$$1.9 + 1.1 = 3.0,$$
$$3.0 + 6.0 = 9.0,$$

and divide by 4: $9.0/4 = 2.25$.

L1 is minimised by any median. Sorted, the values are 0.9, 1.0, 1.1, 6.0; the two middle values are 1.0 and 1.1, and the midpoint is

$$\tfrac{1}{2}(1.0 + 1.1) = \tfrac{1}{2}(2.1) = 1.05.$$

(Any guess between 1.0 and 1.1 has two values above and two below, so the slope of the L1 loss is zero there.)

The mean is pulled towards the anomaly. The three ordinary values average $(0.9 + 1.0 + 1.1)/3 = 3.0/3 = 1.0$; the L2 guess 2.25 is $2.25 - 1.05 = 1.2$ higher than the L1 guess and more than double the typical ordinary value. A model trained with L2 would predict about 2.25 behind this context and so report a residual of only $6.0 - 2.25 = 3.75$ at the anomaly, against $6.0 - 1.05 = 4.95$ under L1.

Solution to Exercise 9.5. Row 3 of $\mathbf{m}^{(1)}$ is $(0, 1, 0, 1)$. At $(3, 0)$, $m^{(1)} = 0$: hidden in pass 1, so $p^{(1)} = 1$ and the value comes from pass 1 ($i + j = 3$ is odd, matching the pattern). At $(3, 3)$, $m^{(1)} = 1$ and $m^{(2)} = 0$: hidden in pass 2, so the value comes from pass 2 ($i + j = 6$ is even).

No mixing. At any pixel $p^{(1)} = 1 - m^{(1)}$ and $p^{(2)} = 1 - m^{(2)} = m^{(1)}$. So $p^{(1)} + p^{(2)} = 1$ and $p^{(1)}p^{(2)} = (1 - m^{(1)})m^{(1)} = 0$, because $m^{(1)}$ is 0 or 1 and one of the two factors is then 0. Two numbers that are each 0 or 1, sum to 1 and multiply to 0 must be one 1 and one 0. Equation (9.13) then reads either $\hat{x}^{(1)} \cdot 1 + \hat{x}^{(2)} \cdot 0 = \hat{x}^{(1)}$ or $\hat{x}^{(1)} \cdot 0 + \hat{x}^{(2)} \cdot 1 = \hat{x}^{(2)}$: one pass, whole, never a blend.

Solution to Exercise 9.6. (1) Token deletion at the encoder input: without it the hidden tokens' content would reach the encoder and could be copied into the output, a self-weight $w > 0$. (2) Two complementary passes at inference: with one pass, the pixels that were visible in it would be scored on reconstructions made while they were visible, so their anomalies would be shrunk by $(1 - w)$, possibly to zero. (3) The loss mask restricting the grade to hidden positions: if visible positions were graded, training would reward copying them and pull the network towards the identity, teaching it a skill that hurts detection.

Chapter 10: Tensors, shapes, layers and the arithmetic of convolutions

Solution to Exercise 10.1. "Eight patches, each with 165 bands, each band 128 rows by 128 columns."

For `(4, 64, 16, 16)`: `flatten(2)` merges the two spatial axes, $16 \times 16 = 256$, giving `(4, 64, 256)`; `transpose(1,2)` swaps the last two axes, giving `(4, 256, 64)`. So $N = 256$ tokens, each with 64 features. This is the stage-2 token sequence of the encoder.

Solution to Exercise 10.2. The output is $(3-2+1)\times(3-2+1) = 2\times 2$. Pair $K = \begin{pmatrix} 0 & 1 \\ 1 & -1 \end{pmatrix}$ with each 2×2 block.

$(0,0)$, block $\begin{pmatrix} 2 & 0 \\ 1 & 3 \end{pmatrix}$:

$$y_{0,0} = 0 \times 2 + 1 \times 0 + 1 \times 1 + (-1) \times 3 = 0 + 0 + 1 - 3 = -2.$$

$(0,1)$, block $\begin{pmatrix} 0 & 1 \\ 3 & 0 \end{pmatrix}$:

$$y_{0,1} = 0 \times 0 + 1 \times 1 + 1 \times 3 + (-1) \times 0 = 0 + 1 + 3 + 0 = 4.$$

$(1,0)$, block $\begin{pmatrix} 1 & 3 \\ 0 & 1 \end{pmatrix}$:

$$y_{1,0} = 0 \times 1 + 1 \times 3 + 1 \times 0 + (-1) \times 1 = 0 + 3 + 0 - 1 = 2.$$

$(1,1)$, block $\begin{pmatrix} 3 & 0 \\ 1 & 2 \end{pmatrix}$:

$$y_{1,1} = 0 \times 3 + 1 \times 0 + 1 \times 1 + (-1) \times 2 = 0 + 0 + 1 - 2 = -1.$$

So $y = \begin{pmatrix} -2 & 4 \\ 2 & -1 \end{pmatrix}$, which is what `F.conv2d` returns.

Solution to Exercise 10.3. $k = 7$, $s = 4$, $p = 3$, $H = 128$:

$$\begin{aligned} 128 + 2 \times 3 - 7 &= 128 + 6 - 7 = 127, \\ 127/4 &= 31.75, \quad \lfloor 31.75 \rfloor = 31, \\ H_{\text{out}} &= 31 + 1 = 32. \end{aligned}$$

The stale design gives the same 32×32 grid as the real one, which is one reason the stale comment never caused a visible failure; the difference is that its windows overlap by $7 - 4 = 3$ pixels.

$k = 3$, $s = 2$, $p = 1$, $H = 31$:

$$\begin{aligned} 31 + 2 - 3 &= 30, \\ 30/2 &= 15, \quad \lfloor 15 \rfloor = 15, \\ H_{\text{out}} &= 15 + 1 = 16. \end{aligned}$$

But $31//2 = 15$. The two disagree: for odd inputs this convolution rounds up, so the docstring rule $H_{\text{out}} = H_{\text{in}}//s$ is wrong there.

Solution to Exercise 10.4. `nn.Conv2d(64,160,kernel_size=3)`, from Equation (10.7):

$$64 \times 160 = 10{,}240,$$
$$10{,}240 \times 3 \times 3 = 10{,}240 \times 9 = 92{,}160,$$
$$92{,}160 + 160 = 92{,}320.$$

This is stage 3's patch-embedding convolution (Chapter 13).

`nn.Conv2d(32,165,kernel_size=1)`:

$$32 \times 165 \times 1 \times 1 = 5{,}280,$$
$$5{,}280 + 165 = 5{,}445.$$

This is Indradhanu's spectral decompressor (Chapter 12).

Solution to Exercise 10.5. Start with $r_0 = 1$, $J_0 = 1$.

$$\begin{aligned}
&\text{layer 1 } (3,1): && r_1 = 1 + (3-1) \times 1 = 3, && J_1 = 1 \times 1 = 1,\\
&\text{layer 2 } (3,2): && r_2 = 3 + (3-1) \times 1 = 5, && J_2 = 1 \times 2 = 2,\\
&\text{layer 3 } (3,1): && r_3 = 5 + (3-1) \times 2 = 9, && J_3 = 2 \times 1 = 2.
\end{aligned}$$

The third layer adds twice as much as the first, although it has the same kernel, because after the stride-2 layer its neighbouring inputs are two pixels apart.

Solution to Exercise 10.6. `transpose(1,2)` turns the `(1, 4, 2)` tokens into `(1, 2, 4)`: $\begin{pmatrix}1&2&3&4\\5&6&7&8\end{pmatrix}$. Laid out contiguously that is 1 2 3 4 5 6 7 8, and `reshape(1,2,2,2)` cuts it into channel 0 $\begin{pmatrix}1&2\\3&4\end{pmatrix}$ and channel 1 $\begin{pmatrix}5&6\\7&8\end{pmatrix}$: the original cube.

Without the transpose, the memory of the sequence tensor is 1 5 2 6 3 7 4 8, and the reshape gives channel 0 $\begin{pmatrix}1&5\\2&6\end{pmatrix}$ and channel 1 $\begin{pmatrix}3&7\\4&8\end{pmatrix}$: each "channel" now mixes both real channels. The shape is right and the data are scrambled.

Chapter 11: The five convolutional autoencoders

Solution to Exercise 11.1. With $k = 4$: $(16-1) \times 2 - 2 \times 1 + 4 = 30 - 2 + 4 = 32$, an exact doubling. With $k = 3$: $30 - 2 + 3 = 31$, one short. That is why the code uses $k = 4$ for the transposed convolutions (and its comment says $k = 3$ "would require output_padding"). A CPU check gives 32×32 and 31×31 [C].

Solution to Exercise 11.2. Encoder channels $1 \to 16 \to 32 \to 64$, by Equation (11.2):

$$1 \to 16 : 16 \times 1 \times 16 + 3 \times 16 = 256 + 48 = 304$$
$$16 \to 32 : 16 \times 16 \times 32 + 3 \times 32 = 8{,}192 + 96 = 8{,}288$$
$$32 \to 64 : 16 \times 32 \times 64 + 3 \times 64 = 32{,}768 + 192 = 32{,}960$$
$$\text{encoder} : 304 + 8{,}288 + 32{,}960 = 41{,}552$$

Decoder channels $64 \to 32 \to 16 \to 1$, the last without BatchNorm:

$$64 \to 32 : 32{,}768 + 96 = 32{,}864$$
$$32 \to 16 : 8{,}192 + 48 = 8{,}240$$
$$16 \to 1 : 16 \times 16 \times 1 + 1 = 257$$
$$\text{decoder} : 32{,}864 + 8{,}240 + 257 = 41{,}361$$

Total 41,552 + 41,361 = 82,913; building the class gives the same [C].

Solution to Exercise 11.3. Four stages divide each side by $2^4 = 16$: $256/16 = 16$. The bottleneck is $256 \times 16 \times 16 = 65{,}536$ values; the input is $256 \times 256 = 65{,}536$. Exactly as many, at every patch size: the first stage multiplies the count by 8 (1 channel becomes 32 on a quarter of the area) and each later stage halves it (channels double, area quarters), so $8 \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} = 1$. The bottleneck is no compression at all.

Solution to Exercise 11.4. A valid, hidden pixel has $m_{\text{valid}} = 1$, $m_{\text{input}} = 0$. Asanskrita: channel 0 is $30 \times 1 \times 0 = 0$, so $(0, 1, 0)$. Drashta: the masks are applied first, then the normalisation, so channel 0 is $(0 - 24.5756)/13.5744 = -1.8104$: $(-1.8104, 1, 0)$. The true 30 °C appears in neither.

Solution to Exercise 11.5. Pratibimba was trained to reproduce whole patches with nothing hidden, so it never learned what to do with pixels set to 0 °C. The product's checkerboard hides every other pixel at inference, a patch unlike anything it trained on, and it reconstructs the whole field badly.

Solution to Exercise 11.6. $x_0 = 3$ at position 0 adds $(3, 6, 6, 3)$ to positions 0–3; $x_1 = -1$ at position 2 adds $(-1, -2, -2, -1)$ to positions 2–5. Sum: $(3,\ 6,\ 6-1,\ 3-2,\ -2,\ -1) = (3, 6, 5, 1, -2, -1)$. Crop one from each end: $(6, 5, 1, -2)$. PyTorch's `ConvTranspose1d` gives the same [C].

Chapter 12: The spectral compressor

Solution to Exercise 12.1. `Conv2d(165,24,kernel_size=1)`:

$$165 \times 24 = 3{,}960,$$
$$3{,}960 + 24 = 3{,}984.$$

`BatchNorm2d(24)` adds $2 \times 24 = 48$. The v0.1.0 compressor has

$$3{,}984 + 48 = 4{,}032$$

parameters, against 5,376 at $D = 32$.

Solution to Exercise 12.2. Mean:

$$2 + 4 + 4 + 6 = 16,$$
$$\mu = 16/4 = 4.$$

Deviations: $2 - 4 = -2$, $4 - 4 = 0$, $4 - 4 = 0$, $6 - 4 = 2$. Squares: $4, 0, 0, 4$; sum 8. Biased variance $\sigma^2 = 8/4 = 2$, so the denominator is $\sqrt{2} = 1.41421$. Outputs:

$$-2/1.41421 = -1.4142, \qquad 0/1.41421 = 0,$$
$$0/1.41421 = 0, \qquad 2/1.41421 = 1.4142.$$

Running statistics. The unbiased variance divides the same sum by $M - 1 = 3$: $s^2 = 8/3 = 2.66667$.

$$\text{running_mean} = 0.9 \times 0 + 0.1 \times 4 = 0.4,$$
$$\text{running_var} = 0.9 \times 1 + 0.1 \times 2.66667 = 0.9 + 0.26667 = 1.16667.$$

Solution to Exercise 12.3. (1) A BatchNorm would force every output band to the same batch mean β, erasing information the batch shares: two dark-water pixels both correctly predicted at -1.5 would both come out as β, "average brightness", and the result would depend on which patches shared the batch. (2) An activation such as ReLU would clip negative values: a band predicted at -0.8 (below its average) would come out as 0, so nothing could be reconstructed darker than its band's average. The decompressor's output feeds `PixelDenormalize` and must be free to take any value.

Solution to Exercise 12.4. Compress:

$$z_1 = 1 \times 0.3 + 0 \times 0.5 + 0 \times 0.5 = 0.3,$$
$$z_2 = 0 \times 0.3 + 0.5 \times 0.5 + 0.5 \times 0.5 = 0.25 + 0.25 = 0.5.$$

Decompress: $\hat{x}_1 = 0.3$, $\hat{x}_2 = 0.5$, $\hat{x}_3 = 0.5$. Residual $(0.3 - 0.3,\ 0.5 - 0.5,\ 0.5 - 0.5) = (0, 0, 0)$.

This pixel's bands 2 and 3 are equal, so it has no component along the lost direction $\mathbf{n} = (0, 1, -1)^\top$: it already lies in the space $\{\mathbf{Vz}\}$ that the decompressor can produce, and the bottleneck reproduces it exactly. Only pixels with a band-2-versus-band-3 difference lose anything.

Solution to Exercise 12.5. With $b = 10$ the values are $11, 12, 13, 16$. Mean:

$$11 + 12 + 13 + 16 = 52,$$
$$52/4 = 13.$$

Deviations: $11 - 13 = -2$, $12 - 13 = -1$, $13 - 13 = 0$, $16 - 13 = 3$, the same as in worked example 12.2. The variance is built from the deviations, so it is again 3.5, and the outputs are again -1.0690, -0.5345, 0, 1.6036. In general, the mean shifts by b and the shift cancels in every deviation.

Solution to Exercise 12.6. Like MNF, the compressor is a linear projection from many bands to few, a matrix of shape (few, many). Unlike MNF, it is found by gradient descent rather than eigen-decomposition, optimises reconstruction loss rather than signal-to-noise, is learned once for all scenes rather than computed per scene, and is trained jointly with the rest of the network.

The symptom of a too-small D is a spectral-angle (SAM) error that stays high, above 0.3 rad after epoch 30 according to the diagnosis rule in the training configuration. The fix was to raise `compressed_channels` from 24 to 32 between v0.1.0 and v0.2.0, and validation loss fell from 0.077 to 0.043.

Chapter 13: Patch embedding

Solution to Exercise 13.1. Stage 1 ($k = s = 4$, $p = 0$): $\lfloor (96 - 4)/4 \rfloor + 1 = \lfloor 23 \rfloor + 1 = 24$, so a 24×24 grid and $24 \times 24 = 576$ tokens.

Stages 2 to 4 ($k = 3$, $s = 2$, $p = 1$):

$$\text{stage 2: } \lfloor (24 + 2 - 3)/2 \rfloor + 1 = \lfloor 11.5 \rfloor + 1 = 12, \qquad 12^2 = 144,$$
$$\text{stage 3: } \lfloor (12 + 2 - 3)/2 \rfloor + 1 = \lfloor 5.5 \rfloor + 1 = 6, \qquad 6^2 = 36,$$
$$\text{stage 4: } \lfloor (6 + 2 - 3)/2 \rfloor + 1 = \lfloor 2.5 \rfloor + 1 = 3, \qquad 3^2 = 9.$$

Solution to Exercise 13.2. `Conv2d(64,160,kernel_size=3)`:

$$64 \times 160 \times 9 = 10{,}240 \times 9 = 92{,}160,$$
$$92{,}160 + 160 = 92{,}320,$$

plus LayerNorm $2 \times 160 = 320$: total 92,640, stage 3's figure.

Stage 1 reading 24 channels:

$$24 \times 32 \times 16 = 768 \times 16 = 12{,}288,$$
$$12{,}288 + 32 = 12{,}320,$$
$$12{,}320 + 64 = 12{,}384.$$

Solution to Exercise 13.3. $(1, 3, 5, 7)$: mean $(1 + 3 + 5 + 7)/4 = 16/4 = 4$. Deviations $-3, -1, 1, 3$; squares $9, 1, 1, 9$; sum 20; variance $20/4 = 5$; standard deviation $\sqrt{5} = 2.23607$. Outputs:

$$-3/2.23607 = -1.3416, \qquad -1/2.23607 = -0.4472,$$
$$1/2.23607 = 0.4472, \qquad 3/2.23607 = 1.3416.$$

$(10, 30, 50, 70)$: mean $160/4 = 40$; deviations $-30, -10, 10, 30$; squares $900, 100, 100, 900$; sum 2000; variance 500; standard deviation $\sqrt{500} = 22.3607$. Outputs $-30/22.3607 = -1.3416$, -0.4472, 0.4472, 1.3416: identical. The second token is ten times the first; LayerNorm divides out each token's own scale (and subtracts its own mean), so only the pattern across features survives.

Solution to Exercise 13.4. Top-left block $\begin{pmatrix} 4 & 0 \\ 4 & 0 \end{pmatrix}$: average $0.25 \times (4+0+4+0) = 0.25 \times 8 = 2$; left minus right $(4 + 4) - (0 + 0) = 8$.

Top-right $\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}$: average $0.25 \times 4 = 1$; left minus right $(1 + 1) - (1 + 1) = 0$.

Bottom-left $\begin{pmatrix} 2 & 2 \\ 2 & 2 \end{pmatrix}$: average $0.25 \times 8 = 2$; left minus right 0.

Bottom-right $\begin{pmatrix} 0 & 3 \\ 0 & 3 \end{pmatrix}$: average $0.25 \times 6 = 1.5$; left minus right $(0 + 0) - (3 + 3) = -6$.

Token sequence, shape `(1, 4, 2)`:

$$\begin{pmatrix} 2 & 8 \\ 1 & 0 \\ 2 & 0 \\ 1.5 & -6 \end{pmatrix}.$$

Solution to Exercise 13.5. Window n starts at original pixel $4n - 1$ and covers six pixels, $4n - 1$ to $4n + 4$:

$$n = 0 : -1..4 \ \rightarrow \ 0..4 \text{ real},$$
$$n = 1 : 3..8,$$
$$n = 2 : 7..12 \ \rightarrow \ 7..11 \text{ shown},$$
$$n = 3 : 11..16 \ \rightarrow \ 11 \text{ shown}.$$

Counts for pixels 0..11:

$$1\ 1\ 1\ 2 \quad 2\ 1\ 1\ 2 \quad 2\ 1\ 1\ 2.$$

Token 1's block is pixels 4..7. Pixel 4 is also in window 0 and pixel 7 in window 2; pixels 5 and 6 are only in window 1. So two of the four pixels per row leak if both neighbours are visible. (The grid is still $\lfloor (128 + 2 - 6)/4 \rfloor + 1 = 31 + 1 = 32$ on a side, so a size check would not catch the leak.)

Solution to Exercise 13.6. After `flatten(2)` the token tensor no longer records that its N tokens came from an $H' \times W'$ grid, and later modules (attention's spatial reduction, the Mix-FFN depthwise convolution) must reshape the tokens back into that grid; so the grid size is returned and carried explicitly. LayerNorm normalises each token over its own features, so a token's output does not depend on other examples in the batch or on how many tokens masking deleted; BatchNorm would couple tokens across the batch and change behaviour between training and evaluation.

Chapter 14: Attention, from one query to many heads

Solution to Exercise 14.1. *Directly.* Exponentiate each score:

$$e^3 = 20.0855, \qquad e^1 = 2.7183, \qquad e^0 = 1.0000.$$

Add: $20.0855 + 2.7183 + 1.0000 = 23.8038$. Divide:

$$20.0855/23.8038 = 0.8438, \qquad 2.7183/23.8038 = 0.1142, \qquad 1.0000/23.8038 = 0.0420.$$

Stably. The maximum is 3. Subtract it: $(3-3, 1-3, 0-3) = (0, -2, -3)$. Exponentiate: $e^0 = 1.0000$, $e^{-2} = 0.1353$, $e^{-3} = 0.0498$. Add: $1.0000 + 0.1353 + 0.0498 = 1.1851$. Divide: $1.0000/1.1851 = 0.8438$, $0.1353/1.1851 = 0.1142$, $0.0498/1.1851 = 0.0420$.

The weights agree, as Section 14.3.2 proves they must. Check the sum: $0.8438 + 0.1142 + 0.0420 = 1.0000$.

Solution to Exercise 14.2. Scores of $\mathbf{q} = (0, 1)$ against $(1, 0), (0, 1), (1, 1)$:

$$0 \cdot 1 + 1 \cdot 0 = 0, \qquad 0 \cdot 0 + 1 \cdot 1 = 1, \qquad 0 \cdot 1 + 1 \cdot 1 = 1.$$

Scaled by $1/\sqrt{2} = 0.7071$: $(0, 0.7071, 0.7071)$. Exponentials: $(1.0000, 2.0281, 2.0281)$. Sum: $1.0000 + 2.0281 + 2.0281 = 5.0562$. Weights: $1.0000/5.0562 = 0.1978$, $2.0281/5.0562 = 0.4011$, $2.0281/5.0562 = 0.4011$.

Output, first feature: $0.1978 \cdot 1 + 0.4011 \cdot 0 + 0.4011 \cdot 1 = 0.1978 + 0 + 0.4011 = 0.5989$. Second feature: $0.1978 \cdot 0 + 0.4011 \cdot 1 + 0.4011 \cdot 1 = 0 + 0.4011 + 0.4011 = 0.8022$. So $\mathbf{o}_2 = (0.5989, 0.8022)$: the mirror image of token 1's output, as the symmetry of the input (swap the two features and tokens 1 and 2 swap) demands.

Solution to Exercise 14.3. With two scores,

$$\begin{aligned} \operatorname{softmax}(s_1, s_2)_1 &= \frac{e^{s_1}}{e^{s_1} + e^{s_2}} && \text{definition} \\ &= \frac{e^{s_1}/e^{s_1}}{e^{s_1}/e^{s_1} + e^{s_2}/e^{s_1}} && \text{divide top and bottom by } e^{s_1} \\ &= \frac{1}{1 + e^{s_2 - s_1}} = \frac{1}{1 + e^{-(s_1 - s_2)}} && e^x/e^y = e^{x-y}. \end{aligned}$$

This is the logistic sigmoid of the difference $s_1 - s_2$. For a lead of 2: $e^{-2} = 0.1353$, $1 + 0.1353 = 1.1353$, $1/1.1353 = 0.8808$. The leader takes 88% of the attention.

Solution to Exercise 14.4. *One term, mean.* $\mathbb{E}[q_i k_i] = \mathbb{E}[q_i]\mathbb{E}[k_i] = 0 \cdot 0 = 0$ (independence).

One term, variance. $\mathbb{E}[q_i^2] = \operatorname{Var}(q_i) + (\mathbb{E}[q_i])^2 = 4 + 0 = 4$, and likewise $\mathbb{E}[k_i^2] = 4$. So

$$\operatorname{Var}(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - 0^2 = \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] = 4 \cdot 4 = 16.$$

Sum of 16 independent terms. $\mathrm{Var}(\mathbf{q}\cdot\mathbf{k}) = 16+16+\cdots+16$ (16 terms) $= 16\times 16 = 256$, a standard deviation of $\sqrt{256} = 16$.

Divisor. Dividing by a divides the variance by a^2; we need $256/a^2 = 1$, so $a^2 = 256$ and $a = 16$. In general the divisor is $\sigma^2\sqrt{d} = 4\times 4 = 16$; the usual $\sqrt{d} = 4$ would leave a variance of $256/16 = 16$. This is one reason the inputs to attention are normalised first (Chapter 16): the $\sqrt{d}$ rule assumes unit-variance components.

Solution to Exercise 14.5. (a) $d = 160/5 = 32$; scale $= 1/\sqrt{32} = 1/5.6569 = 0.1768$.

(b) One projection: $160\times 160 + 160 = 25{,}600 + 160 = 25{,}760$. Four: $4\times 25{,}760 = 103{,}040$. (Equivalently $4\cdot 160^2 + 4\cdot 160 = 102{,}400 + 640 = 103{,}040$.)

(c) $205{,}920 - 103{,}040 = 102{,}880$. These belong to the spatial-reduction convolution and its LayerNorm of Chapter 15: $160\times 160\times 2\times 2 + 160 = 102{,}400 + 160 = 102{,}560$ and $2\times 160 = 320$; $102{,}560 + 320 = 102{,}880$.

Solution to Exercise 14.6. $d = 256/8 = 32$. After the split and transpose, $\mathbf{Q}$ is `(B, 8, 16, 32)`. The scores are `(B, 8, 16, 32) @ (B, 8, 32, 16)` = `(B, 8, 16, 16)`. After `weights@V` the output is `(B, 8, 16, 32)`; transposing gives `(B, 16, 8, 32)` and concatenating the heads gives `(B, 16, 256)`, which the output projection keeps at `(B, 16, 256)`. Per patch there are $16\times 16 = 256$ scores per head and $8\times 256 = 2{,}048$ over all heads.

Chapter 15: Efficient self-attention with spatial reduction

Solution to Exercise 15.1. Here $d = 64/2 = 32$ and $N' = 256/4^2 = 256/16 = 16$.

$$
\begin{aligned}
\text{input } \mathbf{x} &: (B, 256, 64)\\
\mathbf{Q} = \mathrm{q}(\mathbf{x}) &: (B, 256, 64) \to \text{reshape } (B, 256, 2, 32) \to \text{transpose } (B, 2, 256, 32)\\
\text{transpose for the grid} &: (B, 64, 256)\\
\text{reshape} &: (B, 64, 16, 16)\\
\text{conv } k = s = 4 &: (B, 64, 16/4, 16/4) = (B, 64, 4, 4)\\
\text{flatten}(2) &: (B, 64, 16)\\
\text{transpose} &: (B, 16, 64)\\
\text{LayerNorm} &: (B, 16, 64)\\
\mathbf{K}, \mathbf{V} &: (B, 16, 64) \to (B, 16, 2, 32) \to (B, 2, 16, 32)\\
\mathbf{S} = \mathbf{Q}\mathbf{K}^\top/\sqrt{32} &: (B, 2, 256, 32)\,(B, 2, 32, 16) \to (B, 2, 256, 16)\\
\mathbf{A} = \mathrm{softmax}(\mathbf{S}) &: (B, 2, 256, 16)\\
\mathbf{A}\mathbf{V} &: (B, 2, 256, 16)\,(B, 2, 16, 32) \to (B, 2, 256, 32)\\
\text{transpose} &: (B, 256, 2, 32)\\
\text{reshape (concatenate heads)} &: (B, 256, 64)\\
\text{proj} &: (B, 256, 64).
\end{aligned}
$$

Solution to Exercise 15.2. Reduction convolution:

$$
\begin{aligned}
160\times 160 &= 25{,}600\\
25{,}600\times 2\times 2 &= 25{,}600\times 4 = 102{,}400\\
102{,}400 + 160 &= 102{,}560.
\end{aligned}
$$

Reduction LayerNorm: $2\times 160 = 320$. Four projections: $4\times(160\times 160 + 160) = 4\times 25{,}760 = 103{,}040$. Total:

$$103{,}040 + 102{,}560 + 320 = 205{,}920,$$

exactly the built module's count. At this stage the convolution ($CR = 320$) is about as large as all four projections together.

Solution to Exercise 15.3. Stage 1 divides the patch side by 4 and each later stage by 2 again:

stage 1: $256/4 = 64$,	$N = 64 \times 64 = 4096$,	$64/8 = 8$,	$N' = 8 \times 8 = 64$,	$4096 \times 64 = 262{,}144$
stage 2: $64/2 = 32$,	$N = 1024$,	$32/4 = 8$,	$N' = 64$,	$1024 \times 64 = 65{,}536$
stage 3: $32/2 = 16$,	$N = 256$,	$16/2 = 8$,	$N' = 64$,	$256 \times 64 = 16{,}384$
stage 4: $16/2 = 8$,	$N = 64$,	$8/1 = 8$,	$N' = 64$,	$64 \times 64 = 4{,}096$.

Every stage attends over an 8×8 summary of 64 tokens. Stage 1's score matrix grows from 16,384 to 262,144, a factor of $262{,}144/16{,}384 = 16$: four times the queries and four times the keys. (Plain attention would grow from 1024^2 to 4096^2, also a factor of 16, but from a base 64 times larger: $4096^2 = 16{,}777{,}216$ entries per head.)

Solution to Exercise 15.4. $C^2 = 64 \times 64 = 4{,}096$.

Plain.

$$\begin{aligned} 4NC^2 &= 4 \times 256 \times 4{,}096 = 1{,}024 \times 4{,}096 = 4{,}194{,}304 \\ N^2 &= 256 \times 256 = 65{,}536 \\ 2N^2C &= 2 \times 65{,}536 \times 64 = 2 \times 4{,}194{,}304 = 8{,}388{,}608 \\ \text{total} &= 4{,}194{,}304 + 8{,}388{,}608 = 12{,}582{,}912. \end{aligned}$$

Reduced.

$$\begin{aligned} 3NC^2 &= 3 \times 256 \times 4{,}096 = 768 \times 4{,}096 = 3{,}145{,}728 \\ 2N'C^2 &= 2 \times 16 \times 4{,}096 = 131{,}072 \\ 2NN'C &= 2 \times 256 \times 16 \times 64 = 2 \times 262{,}144 = 524{,}288 \\ \text{total} &= 3{,}145{,}728 + 131{,}072 + 524{,}288 = 3{,}801{,}088. \end{aligned}$$

Both match the table; the ratio is $12{,}582{,}912/3{,}801{,}088 = 3.3$.

Solution to Exercise 15.5. With one channel the token is a single number x. Its mean is $\mu = x/1 = x$. Its variance is $\sigma^2 = (x - \mu)^2/1 = 0$. LayerNorm computes

$$\gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta = \gamma \frac{0}{\sqrt{0 + \epsilon}} + \beta = 0 + \beta = \beta,$$

whatever x is (the small ϵ of Chapter 16 only prevents division by zero). The input is erased. It does not matter for the real model because its narrowest tokens have $C = 32$ channels, and LayerNorm only removes one number's worth of information (the mean) and one scale; the pattern across the channels survives. The toy example skipped LayerNorm for exactly this reason.

Solution to Exercise 15.6. Survivors: $1024 \times (1 - 0.5) = 1024 \times 0.5 = 512$. Fallback score entries per head: $512 \times 512 = 262{,}144$ (the "262K" of the source comment). Fraction of plain attention over all tokens: $262{,}144/1{,}048{,}576 = 0.25$, a quarter, because halving N quarters N^2.

Chapter 16: Mix-FFN, LayerNorm, residuals and the transformer block

Solution to Exercise 16.1. *Mean.* $(2.0 + 4.0 + 4.0 + 6.0)/4 = 16.0/4 = 4.0$.

Deviations and squares.

$$\begin{aligned} 2.0 - 4.0 &= -2.0, & (-2.0)^2 &= 4.0 \\ 4.0 - 4.0 &= 0.0, & 0.0^2 &= 0.0 \\ 4.0 - 4.0 &= 0.0, & 0.0^2 &= 0.0 \\ 6.0 - 4.0 &= 2.0, & 2.0^2 &= 4.0 \end{aligned}$$

Variance. $(4.0 + 0.0 + 0.0 + 4.0)/4 = 8.0/4 = 2.0$. *Standard deviation.* $\sqrt{2.0} = 1.4142$.

Outputs. $-2.0/1.4142 = -1.4142$, $0/1.4142 = 0$, 0, $2.0/1.4142 = 1.4142$. So LN $= (-1.4142, 0, 0, 1.4142)$. Check: sum 0; squares $2 + 0 + 0 + 2 = 4$, average 1.

$(10, 20, 20, 30) = 5 \times (2, 4, 4, 6)$, a positive rescaling, so by the invariance of Section 16.2 its LayerNorm is the same $(-1.4142, 0, 0, 1.4142)$.

Solution to Exercise 16.2. $\text{GELU}(2) = 2 \times \Phi(2) = 2 \times 0.9772 = 1.9545$ (to four places from $\Phi(2) = 0.97725$). By symmetry $\Phi(-2) = 1 - 0.9772 = 0.0228$, so $\text{GELU}(-2) = -2 \times 0.0228 = -0.0455$.

The fraction passed is $\Phi(x)$ itself: 97.7% at $x = 2$ and 2.3% at $x = -2$. ReLU would pass 100% and 0%.

Solution to Exercise 16.3. *Without residuals*: $(-0.1)^{16} = 0.1^{16} = 10^{-16}$ (an even power, so positive). The signal reaching the first layer is one part in ten million billion: nothing.

With residuals: each factor is $1 + f' = 1 - 0.1 = 0.9$, and 0.9^{16}:

$$0.9^2 = 0.81, \quad 0.9^4 = 0.81^2 = 0.6561, \quad 0.9^8 = 0.6561^2 = 0.4305, \quad 0.9^{16} = 0.4305^2 = 0.1853.$$

About a fifth of the signal arrives. Only the residual stack lets the first layer learn.

Solution to Exercise 16.4. $C = 64$, hidden width $4 \times 64 = 256$.

$$\begin{aligned} \text{fc1: } & 64 \times 256 + 256 = 16{,}384 + 256 = 16{,}640 \\ \text{dwconv: } & 256 \times 9 + 256 = 2{,}304 + 256 = 2{,}560 \\ \text{fc2: } & 256 \times 64 + 64 = 16{,}384 + 64 = 16{,}448 \\ \text{total: } & 16{,}640 + 2{,}560 + 16{,}448 = 35{,}648. \end{aligned}$$

Check: $8 \times 64^2 + 45 \times 64 = 8 \times 4{,}096 + 2{,}880 = 32{,}768 + 2{,}880 = 35{,}648$.

Solution to Exercise 16.5. $C = 96$, hidden width $4 \times 96 = 384$.

$$\begin{aligned} \text{norm1: } & 2 \times 96 = 192 \\ \text{q, k, v, proj: } & 4 \times (96 \times 96 + 96) = 4 \times (9{,}216 + 96) = 4 \times 9{,}312 = 37{,}248 \\ \text{reduction conv: } & 96 \times 96 \times 64 + 96 = 589{,}824 + 96 = 589{,}920 \\ \text{reduction norm: } & 2 \times 96 = 192 \\ \text{norm2: } & 2 \times 96 = 192 \\ \text{fc1: } & 96 \times 384 + 384 = 36{,}864 + 384 = 37{,}248 \\ \text{dwconv: } & 384 \times 9 + 384 = 3{,}456 + 384 = 3{,}840 \\ \text{fc2: } & 384 \times 96 + 96 = 36{,}864 + 96 = 36{,}960 \end{aligned}$$

Sum: $192 + 37{,}248 = 37{,}440$; $+589{,}920 = 627{,}360$; $+192 = 627{,}552$; $+192 = 627{,}744$; $+37{,}248 = 664{,}992$; $+3{,}840 = 668{,}832$; $+36{,}960 = 705{,}792$. Confirmed.

Ratio: $705{,}792/79{,}616 = 8.86$. Every weight matrix is $C \times C$ (or $C \times C \times k \times k$), so tripling C multiplies it by $3^2 = 9$; but biases, LayerNorms and the depthwise weights are proportional to C and grow only threefold, which pulls the ratio a little under 9.

Solution to Exercise 16.6. With $x = 1$ and $1/(1-p) = 1/0.6 = 1.6667$, the dropped value is 1.6667 with probability 0.6 and 0 with probability 0.4.

$$\begin{aligned}\mathbb{E} &= 0.6 \times 1.6667 + 0.4 \times 0 = 1.0000\\ \mathbb{E}[\cdot^2] &= 0.6 \times 1.6667^2 + 0.4 \times 0 = 0.6 \times 2.7778 = 1.6667\\ \mathrm{Var} &= 1.6667 - 1.0000^2 = 0.6667.\end{aligned}$$

In general the variance is $x^2\, p/(1-p)$: the mean is preserved but dropout adds noise, which is what forces the network not to rely on any single value.

Chapter 17: The four-stage encoder

Solution to Exercise 17.1. Use $H_{\text{out}} = \lfloor (H_{\text{in}} + 2p - k)/s \rfloor + 1$.

$$\begin{aligned}\text{stage 1 } (k=4,\ s=4,\ p=0): &\quad H_1 = \lfloor (256+0-4)/4 \rfloor + 1 = \lfloor 252/4 \rfloor + 1 = 63 + 1 = 64,\\ \text{stage 2 } (k=3,\ s=2,\ p=1): &\quad H_2 = \lfloor (64+2-3)/2 \rfloor + 1 = \lfloor 63/2 \rfloor + 1 = 31 + 1 = 32,\\ \text{stage 3:} &\quad H_3 = \lfloor (32+2-3)/2 \rfloor + 1 = \lfloor 31/2 \rfloor + 1 = 15 + 1 = 16,\\ \text{stage 4:} &\quad H_4 = \lfloor (16+2-3)/2 \rfloor + 1 = \lfloor 15/2 \rfloor + 1 = 7 + 1 = 8.\end{aligned}$$

Token counts: $64 \times 64 = 4096$, $32 \times 32 = 1024$, $16 \times 16 = 256$, $8 \times 8 = 64$. Each is four times the 128-pixel figure, because the patch has four times the area.

Solution to Exercise 17.2. Keys are N_i/R_i^2: stage 1 $1024/64 = 16$, stage 2 $256/16 = 16$, stage 3 $64/4 = 16$, stage 4 $16/1 = 16$. Stage 2 has 256 queries and 16 keys, so its score table per head has $256 \times 16 = 4{,}096$ entries. Stage 2 has 2 heads, so $2 \times 4{,}096 = 8{,}192$ entries per block and per patch.

Solution to Exercise 17.3. Start with $r_0 = 1$, $j_0 = 1$. Stage 1 with $k = 7$, $s = 4$:

$$r_1 = 1 + (7-1) \cdot 1 = 7, \qquad j_1 = 1 \cdot 4 = 4.$$

Stage 2 with $k = 3$, $s = 2$:

$$r_2 = 7 + (3-1) \cdot 4 = 7 + 8 = 15, \qquad j_2 = 4 \cdot 2 = 8.$$

The problem is at stage 1: $r_1 = 7 > j_1 = 4$, so neighbouring stage-1 tokens share $7 - 4 = 3$ pixels (in each direction). A hidden token's pixels would then also feed its visible neighbours, and the encoder would see part of what it must predict: the leakage that Chapter 9 forbids.

Solution to Exercise 17.4. With $\rho = 0.5$:

$$\begin{aligned}\rho^4 &= 0.5^4 = 0.0625,\\ \rho^6 &= 0.5^6 = 0.015625,\\ \rho^9 &= 0.5^9 = 0.001953125.\end{aligned}$$

The window counts are unchanged (1 window of 4 tokens, 30 of 6, 225 of 9). By linearity of expectation:

$$\begin{aligned}\mathbb{E}[\text{empty}] &= 1 \times 0.0625 + 30 \times 0.015625 + 225 \times 0.001953125\\ &= 0.0625 + 0.46875 + 0.439453\\ &\approx 0.97.\end{aligned}$$

About one empty stage-2 token per patch, compared with about seven at $\rho = 0.65$.

Solution to Exercise 17.5. Patch embed; remove tokens (stage 1 only, when a keep mask is given); the blocks; the stage LayerNorm; restore tokens (stage 1 only, same condition); reshape to (B, C_i, H_i, W_i). Removal and restore happen only at stage 1 because hiding is done once at the finest scale and the stride-2 patch embeddings dilute the holes. The reshaped tensor is appended to the feature list for the decoder and also stays in the loop variable `x`, so it is the next stage's input.

Chapter 18: Masked autoencoding: validity, targets, remove-and-restore, erosion

Solution to Exercise 18.1. With 9 valid pixels: $f_t = 9/16 = 0.5625$. Since $0.5625 > 0.5$, the token is valid. With 8 valid pixels: $f_t = 8/16 = 0.5$. The test is strict, $0.5 > 0.5$ is false, so the token is invalid. Ties go to invalid.

Solution to Exercise 18.2.

$$\begin{aligned} \rho\, n_{\text{valid}} &= 0.65 \times 37 = 24.05, \\ n_{\text{hide}} &= \lfloor 24.05 \rfloor = 24, \\ \text{tokens seen by the encoder} &= 1024 - 24 = 1000, \\ \text{effective ratio} &= 24/37 \approx 0.6486. \end{aligned}$$

The encoder sees the 13 visible valid tokens and all $1024 - 37 = 987$ invalid tokens: $13 + 987 = 1000$. (In practice the trainer would have dropped this patch, since 37 valid tokens is far below 40% valid pixels.)

Solution to Exercise 18.3. Token 3 is invalid. Add 2 there:

$$\begin{aligned} &[0.9,\ 0.1,\ 0.05,\ 0.4,\ 0.6,\ 0.02] \\ +\ &[0,\ 0,\ 0,\ 2,\ 0,\ 0] \\ =\ &[0.9,\ 0.1,\ 0.05,\ 2.4,\ 0.6,\ 0.02]. \end{aligned}$$

Count: $n_{\text{valid}} = 1+1+1+0+1+1 = 5$, $n_{\text{hide}} = \lfloor 0.65 \times 5 \rfloor = \lfloor 3.25 \rfloor = 3$.

Sort ascending: $0.02\,(t_5) < 0.05\,(t_2) < 0.1\,(t_1) < 0.6\,(t_4) < 0.9\,(t_0) < 2.4\,(t_3)$, so $\pi = [5, 2, 1, 4, 0, 3]$. Flags $s = [1, 1, 1, 0, 0, 0]$.

Scatter $q_{\pi_k} = s_k$: $q_5 = 1$, $q_2 = 1$, $q_1 = 1$, $q_4 = 0$, $q_0 = 0$, $q_3 = 0$. So $q = [0, 1, 1, 0, 0, 1]$ and $k = 1 - q = [1, 0, 0, 1, 1, 0]$. The invalid token 3 is kept and not hidden.

Inverse: $\pi_0 = 5 \Rightarrow \pi_5^{-1} = 0$; $\pi_1 = 2 \Rightarrow \pi_2^{-1} = 1$; $\pi_2 = 1 \Rightarrow \pi_1^{-1} = 2$; $\pi_3 = 4 \Rightarrow \pi_4^{-1} = 3$; $\pi_4 = 0 \Rightarrow \pi_0^{-1} = 4$; $\pi_5 = 3 \Rightarrow \pi_3^{-1} = 5$. In token order $\pi^{-1} = [4, 2, 1, 5, 3, 0]$.

Gather: $q_t = s_{\pi_t^{-1}}$, so $q = [s_4, s_2, s_1, s_5, s_3, s_0] = [0, 1, 1, 0, 0, 1]$, the same as the scatter. All of these values are reproduced by `research/textbook/tools/compute_masking.py`.

Solution to Exercise 18.4. num_kept $= 1+0+1+1 = 3$, so max_kept $= 3$. The descending argsort puts the three 1s first, for example $[0, 2, 3, 1]$; the gather indices are $g = [0, 2, 3]$ and the kept tokens $[t_0, t_2, t_3]$, shape $(1, 3, C)$.

$$\mathbf{S} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Restore starts from four zero vectors and writes t'_0 at 0, t'_2 at 2, t'_3 at 3: $[t'_0, \mathbf{0}, t'_2, t'_3]$. Position 1, the hidden token, stays zero.

Solution to Exercise 18.5. First mask, kernel 3, radius 1. Flip: invalid $= [1, 0, 0, 0, 0, 1]$. Maximum over $[i-1, i+1]$, clipped at the ends:

$$\begin{aligned} &i = 0 : \{1, 0\} \to 1, && i = 1 : \{1, 0, 0\} \to 1, && i = 2 : \{0, 0, 0\} \to 0, \\ &i = 3 : \{0, 0, 0\} \to 0, && i = 4 : \{0, 0, 1\} \to 1, && i = 5 : \{0, 1\} \to 1. \end{aligned}$$

Dilated $[1, 1, 0, 0, 1, 1]$; flip back: $[0, 0, 1, 1, 0, 0]$. One pixel removed from each end of the valid run.

Second mask, kernel 5, radius 2. Flip: invalid only at index 4. Indices within 2 of index 4 are 2–6, so the dilated invalid is $[0, 0, 1, 1, 1, 1, 1, 0, 0]$ and the eroded mask $[1, 1, 0, 0, 0, 0, 0, 1, 1]$. The single bad pixel grew to a hole of 5. The two ends stay valid: the max pool pads with $-\infty$, so the array edge never counts as invalid.

Solution to Exercise 18.6. Patch A: $n_{\text{hide}} = \lfloor 0.65 \times 1024 \rfloor = \lfloor 665.6 \rfloor = 665$, kept $1024 - 665 = 359$.

Patch B: $n_{\text{hide}} = \lfloor 0.65 \times 960 \rfloor = \lfloor 624 \rfloor = 624$, kept $1024 - 624 = 400$ (336 visible valid plus 64 invalid).

max_kept $= \max(359, 400) = 400$. Patch B gathers exactly its 400 kept tokens. Patch A gathers 400, of which the last $400 - 359 = 41$ come from the tail of its sort: 41 of its hidden tokens leak into the encoder while still being graded. The fully valid patch is the one that leaks.

Chapter 19: The decoder, pixel shuffle and one full forward trace

Solution to Exercise 19.1. Scale $s = 2/4 = 0.5$, so the positions are the same as in worked example 19.2:

$$\begin{aligned} &d = 0 : \quad u = -0.25 \to 0, && \text{value} = 2, \\ &d = 1 : \quad u = 0.25, && \text{value} = 0.75 \cdot 2 + 0.25 \cdot 10 = 1.5 + 2.5 = 4, \\ &d = 2 : \quad u = 0.75, && \text{value} = 0.25 \cdot 2 + 0.75 \cdot 10 = 0.5 + 7.5 = 8, \\ &d = 3 : \quad u = 1.25, && \text{value} = 10 \text{ (last value)}. \end{aligned}$$

Row: $[2, 4, 8, 10]$.

For the grid, first each row: $[0, 4] \to [0, 1, 3, 4]$ (as in the chapter) and $[8, 12] \to [8, 9, 11, 12]$ (since $0.75 \cdot 8 + 0.25 \cdot 12 = 6 + 3 = 9$ and $0.25 \cdot 8 + 0.75 \cdot 12 = 2 + 9 = 11$). Then down each column with weights $0, 0.25, 0.75, 1$ on the lower row:

$$\begin{aligned} \text{out row } 1 &= 0.75 \cdot [0, 1, 3, 4] + 0.25 \cdot [8, 9, 11, 12] \\ &= [0 + 2,\ 0.75 + 2.25,\ 2.25 + 2.75,\ 3 + 3] = [2, 3, 5, 6], \\ \text{out row } 2 &= 0.25 \cdot [0, 1, 3, 4] + 0.75 \cdot [8, 9, 11, 12] \\ &= [0 + 6,\ 0.25 + 6.75,\ 0.75 + 8.25,\ 1 + 9] = [6, 7, 9, 10]. \end{aligned}$$

$$\begin{pmatrix} 0 & 1 & 3 & 4 \\ 2 & 3 & 5 & 6 \\ 6 & 7 & 9 & 10 \\ 8 & 9 & 11 & 12 \end{pmatrix}.$$

Solution to Exercise 19.2. $C = 1$, $r = 2$: output channel 0 takes input channels 0..3, placed row-major in a 2×2 block: offset 0 at $(0,0)$, 1 at $(0,1)$, 2 at $(1,0)$, 3 at $(1,1)$. So the output is $\begin{pmatrix} 9 & 8 \\ 7 & 6 \end{pmatrix}$. Nothing was computed; the four numbers were moved.

Solution to Exercise 19.3. Output $(31, 127, 0)$: $h = \lfloor 127/4 \rfloor = 31$, $i = 127 \bmod 4 = 3$, $w = \lfloor 0/4 \rfloor = 0$, $j = 0 \bmod 4 = 0$. Channel $= 31 \cdot 16 + 4 \cdot 3 + 0 = 496 + 12 = 508$, at position $(31, 0)$.

Input channel 37 at $(2, 3)$: output channel $c = \lfloor 37/16 \rfloor = 2$, offset $37 - 32 = 5$, so $i = \lfloor 5/4 \rfloor = 1$ and $j = 5 \bmod 4 = 1$. Destination: $(c, Y, X) = (2,\ 4 \cdot 2 + 1,\ 4 \cdot 3 + 1) = (2, 9, 13)$.

Solution to Exercise 19.4. The last convolution outputs $D \times 16 = 24 \times 16 = 384$ channels. Before pixel shuffle one patch holds $384 \times 32 \times 32 = 384 \times 1024 = 393{,}216$ numbers. The decoder's output is $24 \times 128 \times 128 = 24 \times 16{,}384 = 393{,}216$ numbers: the same, as a permutation requires.

Solution to Exercise 19.5. With 256×256 the stage grids are $64, 32, 16, 8$ (Exercise 17.1).

1. mask: `(2, 165, 256, 256)`;
2. normalise: `(2, 165, 256, 256)`;
3. compress: `(2, 32, 256, 256)`;
4. encode: stage 1 has $64 \times 64 = 4096$ tokens, of which $\lfloor 0.65 \times 4096 \rfloor = \lfloor 2662.4 \rfloor = 2662$ are hidden and 1434 kept; F1 `(2, 32, 64, 64)`, F2 `(2, 64, 32, 32)`, F3 `(2, 160, 16, 16)`, F4 `(2, 256, 8, 8)`;
5. decode: everything at F1's 64×64, concatenation `(2, 1024, 64, 64)`, last convolution `(2, 512, 64, 64)`, pixel shuffle `(2, 32, 256, 256)`;
6. decompress: `(2, 165, 256, 256)`;
7. denormalise: `(2, 165, 256, 256)`.

The parameter count is the same as for 128-pixel patches; only the shapes and the work change.

Solution to Exercise 19.6.

$$\hat{x} = \sigma\hat{z} + \mu = 0.1 \times 1.3 + 0.25 = 0.13 + 0.25 = 0.38,$$
$$|\hat{x} - x| = |0.38 - 0.36| = 0.02.$$

An error of 0.02 reflectance units in this band.

Chapter 20: The parameter budget, counted by hand

Solution to Exercise 20.1. Full convolution: $64 \times 128 \times 9 + 128 = 73{,}728 + 128 = 73{,}856$. Depthwise: $64 \times 9 + 64 = 576 + 64 = 640$. Ratio: $73{,}856/640 \approx 115$ times fewer.

Solution to Exercise 20.2. Formula, $C = 128$, $R = 4$:

$$12C^2 = 12 \times 16{,}384 = 196{,}608, \qquad 53C = 53 \times 128 = 6{,}784,$$
$$R^2C^2 = 16 \times 16{,}384 = 262{,}144, \qquad 3C = 384,$$
$$P = 196{,}608 + 6{,}784 + 262{,}144 + 384 = 465{,}920.$$

Piece by piece:

$$\begin{aligned}
\text{q, k, v, out} &= 4 \times (128 \times 128 + 128) = 4 \times 16{,}512 = 66{,}048 \\
\text{reduction conv} &= 128 \times 128 \times 16 + 128 = 262{,}144 + 128 = 262{,}272 \\
\text{reduction norm} &= 2 \times 128 = 256 \\
\text{fc1} &= 128 \times 512 + 512 = 65{,}536 + 512 = 66{,}048 \\
\text{depthwise} &= 512 \times 9 + 512 = 4{,}608 + 512 = 5{,}120 \\
\text{fc2} &= 512 \times 128 + 128 = 65{,}536 + 128 = 65{,}664 \\
\text{two LayerNorms} &= 4 \times 128 = 512
\end{aligned}$$

Sum: 66,048 + 262,272 = 328,320; +256 = 328,576; +66,048 = 394,624; +5,120 = 399,744; +65,664 = 465,408; +512 = 465,920. The two agree.

Solution to Exercise 20.3. One more stage-4 block adds $P_{\text{block}}(256, 1) = 800{,}000$ parameters. The model grows from 5,506,629 to 5,506,629 + 800,000 = 6,306,629; the addition is 800,000/5,506,629 ≈ 0.145, about 14.5% of the original model, for a block that processes 16 tokens.

Solution to Exercise 20.4. With decoder width 128 the decoder becomes:

$$\begin{aligned}
\text{projections} &= (32 + 64 + 160 + 256) \times 128 + 4 \times 128 = 65{,}536 + 512 = 66{,}048 \\
\text{fuse} &= (4 \times 128) \times 128 + 128 = 65{,}536 + 128 = 65{,}664 \\
\text{refine 1} &= 128 \times 128 \times 9 + 128 = 147{,}456 + 128 = 147{,}584 \\
\text{refine 2} &= 128 \times 512 \times 9 + 512 = 589{,}824 + 512 = 590{,}336
\end{aligned}$$

Decoder: 66,048+65,664+147,584+590,336 = 869,632. Model: 5,506,629−2,164,736+869,632 = 4,211,525. The largest single saving is refine 2, 1,180,160 − 590,336 = 589,824; refine 1 saves 590,080 − 147,584 = 442,496 (it shrinks by a factor 4 because both its widths halve, refine 2 only by 2 because its output stays at 512).

Solution to Exercise 20.5. Normaliser mean and standard deviation, denormaliser mean and standard deviation: 4×165 = 660. The compressor's BatchNorm: running mean 32, running variance 32, batch counter 1, total 65. 660 + 65 = 725 and 5,506,629 + 725 = 5,507,354.

Solution to Exercise 20.6. Compressor: 165 × 32 = 5,280 multiply-adds per pixel, ×16,384 pixels = 86,507,520. Fuse: 1,024 × 256 = 262,144 per position, ×1,024 positions = 268,435,456. The fuse does more, by 268,435,456/86,507,520 ≈ 3.1. It has about 49 times the compressor's parameters but runs at 16 times fewer positions, so it does about 49/16 ≈ 3 times the work.

Chapter 21: Chakshu: the same machine for one thermal band

Solution to Exercise 21.1. With $E = 2$, Equation (21.3) has coefficients $4+2\times2+16 = 24$ and $12 + 11 \times 2 = 34$:

$$24 \times 32^2 + 34 \times 32 = 24 \times 1{,}024 + 1{,}088 = 24{,}576 + 1{,}088 = 25{,}664$$

With $E = 4$ the coefficients are $4 + 8 + 16 = 28$ and $12 + 44 = 56$:

$$28 \times 1{,}024 + 56 \times 32 = 28{,}672 + 1{,}792 = 30{,}464.$$

Doubling the expansion adds 30,464 − 25,664 = 4,800. Raising E by $\Delta E = 2$ adds $2\Delta E\, C^2 + 11\Delta E\, C = 4\times1{,}024+22\times32 = 4{,}096+704 = 4{,}800$. The C^2 part is $2\Delta E\, C^2$ because two layers,

fc1 and fc2, each hold EC^2 weights; the linear part collects fc1's EC biases and the depthwise convolution's $10EC$.

Solution to Exercise 21.2. The stage-1 grid is $128/4 = 32$, then 16, 8 and 4. Tokens: $32^2 = 1{,}024$, $16^2 = 256$, $8^2 = 64$, $4^2 = 16$. Keys: $(32/8)^2 = 4^2 = 16$, $(16/4)^2 = 16$, $(8/2)^2 = 16$, and at stage 4 all 16 tokens: 16 keys at every stage. A training pass hides $\lfloor 1{,}024 \times 0.7 \rfloor = \lfloor 716.8 \rfloor = 716$ and keeps $1{,}024 - 716 = 308$. The parameter count does not change: no layer's size depends on H or W (Chapter 10). That is why the product can offer 64, 128 and 256.

Solution to Exercise 21.3.

$$
\begin{aligned}
0.00341802 \times 48{,}000 &= 164.06496 \\
164.06496 + 149.0 &= 313.06496 \text{ K} \\
313.06496 - 273.15 &= 39.91 \text{ °C} \\
z &= (39.9150 - 24.5756)/13.5744 = 15.3394/13.5744 = 1.1300.
\end{aligned}
$$

Solution to Exercise 21.4. Sum: three 0.1s make 0.3, three 0.2s make 0.6, two 0.3s make 0.6, then 0.4 and 5.0: $0.3 + 0.6 + 0.6 + 0.4 + 5.0 = 6.9$; plain mean $6.9/10 = 0.69$. At $\tau = 0.05$ the code keeps $\lfloor 10 \times 0.95 \rfloor = \lfloor 9.5 \rfloor = 9$, dropping the 5.0: $(6.9 - 5.0)/9 = 1.9/9 = 0.2111$. At $\tau = 0.1$ it keeps $\lfloor 9.0 \rfloor = 9$, the same 0.2111. With only ten numbers the floor turns a 5% trim into a 10% one: it cannot drop half a number, so it drops one. On a real batch of millions of pixels the rounding is negligible.

Solution to Exercise 21.5. Rows: $(1{,}000 - 256)/128 = 5.8125$, $\lceil 5.8125 \rceil + 1 = 7$ (worked example 21.7). Columns: $(600 - 256)/128 = 344/128 = 2.6875$, $\lceil 2.6875 \rceil + 1 = 3 + 1 = 4$, starting at 0, 128, 256 and the snapped $600 - 256 = 344$. Tiles: $7 \times 4 = 28$; forward passes: $2 \times 28 = 56$.

Solution to Exercise 21.6. Baked: $(4{,}600 - 24.5756)/13.5744 = 4{,}575.4244/13.5744 = 337.06$. Override: $(4{,}600 - 5{,}000)/400 = -400/400 = -1$. The product uses the baked statistics, for two reasons (Section 21.9): the vendable loses its `units` field, so the override is never built; and if it were, loading the checkpoint would write the baked buffers back over it.

Solution to Exercise 21.7. Pixel 127 lies in token row (and column) $\lfloor 127/4 \rfloor = 31$, pixel 128 in $\lfloor 128/4 \rfloor = 32$. The spot touches tokens $(31, 31)$, $(31, 32)$, $(32, 31)$ and $(32, 32)$, one hot pixel each, with colours $62 \bmod 2 = 0$, $63 \bmod 2 = 1$, 1 and 0. In each pass two of its four hot pixels are visible, diagonally next to the two hidden ones, so expect a residual nearer the straddling 3×3 spot's than the one-token spot's. This case was not run; the expectation follows from the mechanism of Section 21.8.2, and you could check it by adding it to `tools/compute_chakshu.py`.

Solution to Exercise 21.8. `encoder_dims` sets the four widths and `decoder_dim` the decoder width; everything else is fixed in `_model_config_from_manifest`. A two-blocks-per-stage checkpoint would be rebuilt with one block per stage, so its `state_dict` would hold keys such as `encoder.blocks.0.1.norm1.weight` that the model lacks. The strict `load_state_dict` in `FoundationInferencer._load_weights` would raise, and the scoring action would fail at its `load_checkpoint` step. The fix belongs in the manifest (embed the full `model_config`, as the spectral-stem models already do) or in the resolver.

Chapter 22: Why the compressor is a ceiling: cross-attention and a spectrum as tokens

Solution to Exercise 22.1. The broken pixel is $\mathbf{x}_{\text{broken}} = [1.2, 1.0, -0.4, 0.0]$. Row 1 of $\mathbf{W}$ is $[0, 0, 0.5, 0.5]$, so directly

$$\begin{aligned} z_1 &= 0 \times 1.2 + 0 \times 1.0 + 0.5 \times (-0.4) + 0.5 \times 0.0 \\ &= 0 + 0 - 0.20 + 0 \\ &= -0.20. \end{aligned}$$

With Equation (22.5): the fill is $f = 0.0$ and the true value $x_3 = -0.6$, so $f - x_3 = 0.0 - (-0.6) = 0.6$. Column 3 of $\mathbf{W}$ is $[0, 0.5]$. The output error is

$$0.6 \times [0, 0.5] = [0, 0.30],$$

so z_1 moves from -0.50 to $-0.50 + 0.30 = -0.20$, as computed directly. As a share of the true value:

$$\frac{0.30}{|-0.50|} = 0.60 = 60\%.$$

Output z_0 is unchanged because column 3 has a zero in row 0.

Solution to Exercise 22.2. The convolution has one weight per (output, input) pair plus one bias per output:

$$165 \times 32 + 32 = 5{,}280 + 32 = 5{,}312.$$

BatchNorm has a scale and a shift per channel:

$$2 \times 32 = 64.$$

Total: $5{,}312 + 64 = 5{,}376$.

The weight matrix is 32×165, so its rank is at most 32. By rank–nullity, the null space has dimension $165 - \text{rank} \geq 165 - 32 = 133$. At $D = 24$ the rank is at most 24 and the bound becomes $165 - 24 = 141$.

Solution to Exercise 22.3. Raw dot products with $\mathbf{q} = [0, 1]$:

$$\begin{aligned} \mathbf{q} \cdot \mathbf{k}_1 &= 0 \times 2 + 1 \times 0 = 0, \\ \mathbf{q} \cdot \mathbf{k}_2 &= 0 \times 0 + 1 \times 1 = 1, \\ \mathbf{q} \cdot \mathbf{k}_3 &= 0 \times 1 + 1 \times 1 = 1. \end{aligned}$$

Scaled by 0.7071: 0, 0.7071, 0.7071. Exponentials:

$$e^0 = 1.0000, \qquad e^{0.7071} = 2.0281, \qquad e^{0.7071} = 2.0281.$$

Sum: $1.0000 + 2.0281 + 2.0281 = 5.0562$. Weights:

$$\begin{aligned} 1.0000/5.0562 &= 0.1978, \\ 2.0281/5.0562 &= 0.4011, \\ 2.0281/5.0562 &= 0.4011. \end{aligned}$$

Check: $0.1978 + 0.4011 + 0.4011 = 1.0000$. Answer:

$$0.1978 \times 10 = 1.978,$$
$$0.4011 \times 20 = 8.022,$$
$$0.4011 \times 30 = 12.033,$$
$$1.978 + 8.022 + 12.033 = 22.033.$$

The query now asks about feature 1, which tokens 2 and 3 have equally and token 1 lacks, so tokens 2 and 3 share most of the weight. PyTorch gives 22.0334.

Solution to Exercise 22.4. Value part:

$$0.6 \times 0.5 + 0.1 = 0.30 + 0.10 = 0.40,$$
$$0.6 \times (-1.0) + 0.2 = -0.60 + 0.20 = -0.40.$$

So the value part is $[0.40, -0.40]$. Tag at period 1000 nm: $\theta = 2\pi \times 500/1000 = \pi$, and $\sin \pi = 0$, $\cos \pi = -1$, so the tag is $[0, -1]$. The token is

$$[0.40 + 0,\ -0.40 + (-1)] = [0.40,\ -1.40].$$

Solution to Exercise 22.5. Every term of the sum in Equation (22.11) is a cosine, and a cosine is at most 1. Sixteen terms each at most 1 sum to at most 16, and dividing by 16 gives at most 1. Equality needs every cosine to equal 1 at once, which happens at $\Delta = 0$.

For $\Delta = 10{,}000$ nm, look at each period. At $P_0 = 10$ the angle is $2\pi \times 1000$, a whole number of turns, so the cosine is 1. The same happens at $P_5 = 100$ (100 turns), $P_{10} = 1000$ (10 turns) and $P_{15} = 10{,}000$ (one turn). But the other twelve periods are not whole divisors of 10,000 (for example $10{,}000/15.85 = 631.0$ is close to whole, while $10{,}000/63.10 = 158.5$ is half-way, giving a cosine near -1). Those terms scatter between -1 and 1. The book's script gives a sum of 6.4063, so $s(10{,}000) = 6.4063/16 = 0.4004$: far from 1. Even the longest period completing one full turn does not make the tags alike, because the other periods have not.

Solution to Exercise 22.6. Cross-attention returns one answer per query: 32 answers. The score matrix is queries by keys:

$$32 \times 224 = 7{,}168 \text{ scores per head.}$$

For 425 bands, self-attention compares every band with every band:

$$425 \times 425 = 180{,}625,$$

and cross-attention with 32 queries:

$$32 \times 425 = 13{,}600.$$

Chapter 23: The cross-attention compressor, dead bands and what it costs

Solution to Exercise 23.1. With $d = 16$, $D = 32$, apply $n_{\text{out}} \times n_{\text{in}} + n_{\text{out}}$ to each layer:

$$\begin{aligned}
\text{value stretch, Linear(1, 16)}: &\quad 16 \times 1 + 16 = 32 \\
\text{queries, table (32, 16)}: &\quad 32 \times 16 = 512 \\
\text{Q, K, V, O, each Linear(16, 16)}: &\quad 16 \times 16 + 16 = 256 + 16 = 272 \\
\text{four of them}: &\quad 4 \times 272 = 1{,}088 \\
\text{to channel, Linear(16, 1)}: &\quad 16 + 1 = 17
\end{aligned}$$

Total:

$$32 + 512 + 1{,}088 + 17 = 544 + 1{,}088 + 17 = 1{,}632 + 17 = 1{,}649.$$

Formula check, $4d^2 + Dd + 7d + 1$ at $d = 16$, $D = 32$:

$$4 \times 256 + 32 \times 16 + 7 \times 16 + 1 = 1{,}024 + 512 + 112 + 1 = 1{,}649.$$

Instantiating the real compressor with $D = 32$, $d = 16$ and 4 heads also reports 1,649.

Solution to Exercise 23.2. Masked scores $[3, -\infty, 1]$. Exponentials:

$$e^3 = 20.0855, \qquad e^{-\infty} = 0, \qquad e^1 = 2.7183.$$

Sum: $20.0855 + 0 + 2.7183 = 22.8038$. Weights:

$$\begin{aligned} 20.0855/22.8038 &= 0.8808, \\ 0/22.8038 &= 0, \\ 2.7183/22.8038 &= 0.1192. \end{aligned}$$

They sum to $0.8808+0+0.1192 = 1.0000$. Before masking the exponentials were 20.0855, 2.7183 and 2.7183, with sum 25.5221, giving weights 0.7870, 0.1065 and 0.1065. The ratio of first to third surviving weight: before, $0.7870/0.1065 = 7.389$; after, $0.8808/0.1192 = 7.389$. Both equal $e^{3-1} = e^2 = 7.389$.

Solution to Exercise 23.3.

$$\begin{aligned} \text{K, V :} \quad & 2 \times 224 \times 1{,}024 = 448 \times 1{,}024 = 458{,}752 \\ \text{scores :} \quad & 32 \times 224 \times 32 = 7{,}168 \times 32 = 229{,}376 \\ \text{values :} \quad & 32 \times 224 \times 32 = 229{,}376 \\ \text{output + to channel :} \quad & 32{,}768 + 1{,}024 = 33{,}792 \end{aligned}$$

Total:

$$\begin{aligned} 458{,}752 + 229{,}376 + 229{,}376 + 33{,}792 &= 688{,}128 + 229{,}376 + 33{,}792 \\ &= 917{,}504 + 33{,}792 = 951{,}296. \end{aligned}$$

The output stage does not depend on the band count; the other three grow linearly with it.

Solution to Exercise 23.4.

$$\begin{aligned} 16{,}384 \times 2 &= 32{,}768, \\ 32{,}768 \times 224 &= 7{,}340{,}032, \\ 7{,}340{,}032 \times 32 &= 234{,}881{,}024 \text{ numbers}, \\ 234{,}881{,}024 \times 2 \text{ bytes} &= 469{,}762{,}048 \text{ bytes}, \\ 469{,}762{,}048/1{,}048{,}576 &= 448 \text{ MiB}. \end{aligned}$$

Solution to Exercise 23.5. Available memory: $48 - 4 = 44$ GB. Patches: $44/3.7 = 11.9$, and a batch must be a whole number of patches that fits, so 11. Pixel rows: $11 \times 16{,}384 = 180{,}224$, far over the 65,535 limit. `_attend` splits them into chunks of 32,768: $180{,}224/32{,}768 = 5.5$, rounded up to 6 calls (five full chunks of 32,768 and one of $180{,}224 - 5 \times 32{,}768 = 16{,}384$ rows). (Strictly, 48 GB and 3.7 GiB use different units; using 44 GiB instead gives the same answer, 11.)

Solution to Exercise 23.6. With one head and frozen weights, Equation (23.3) without offsets is $y_c = \kappa(a_0 v_0 + a_1 v_1)$ with $a_0, a_1 > 0$ and $a_0 + a_1 = 1$. To equal $v_0 - v_1$ for every v_0, v_1 we need $\kappa a_0 = 1$ and $\kappa a_1 = -1$. Dividing, $a_0/a_1 = -1$, which is impossible because both are positive. So a single head cannot form a difference.

With two heads, take head 1 weights $[0.9, 0.1]$, head 2 weights $[0.1, 0.9]$, $\kappa_1 = 1.25$, $\kappa_2 = -1.25$:

$$\begin{aligned} y_c &= 1.25(0.9v_0 + 0.1v_1) - 1.25(0.1v_0 + 0.9v_1) \\ &= (1.125 - 0.125)v_0 + (0.125 - 1.125)v_1 \\ &= 1.0\,v_0 - 1.0\,v_1. \end{aligned}$$

The difference of two weighted averages with opposite-signed κ gives the slope.

Chapter 24: The spectral decoder, the traps, per-pixel gating and spectral reduction

Solution to Exercise 24.1. With $d = 16$, $D = 32$:

$$\begin{aligned} \text{query from wavelength, Linear(16, 16)}: &\quad 16 \times 16 + 16 = 272 \\ \text{channel value stretch, Linear(1, 16)}: &\quad 16 + 16 = 32 \\ \text{channel identities, table (32, 16)}: &\quad 32 \times 16 = 512 \\ \text{Q, K, V, O}: &\quad 4 \times 272 = 1{,}088 \\ \text{to reflectance, Linear(16, 1)}: &\quad 16 + 1 = 17 \end{aligned}$$

Total: $272+32+512+1{,}088+17 = 304+512+1{,}088+17 = 816+1{,}088+17 = 1{,}904+17 = 1{,}921$. Formula $5d^2 + Dd + 8d + 1$: $5 \times 256 + 32 \times 16 + 8 \times 16 + 1 = 1{,}280 + 512 + 128 + 1 = 1{,}921$.

Solution to Exercise 24.2.

$$\begin{aligned} \sigma(1) &= 1/(1 + e^{-1}) = 1/(1 + 0.3679) = 1/1.3679 = 0.7311, \\ \sigma(-1) &= 1/(1 + e^{1}) = 1/(1 + 2.7183) = 1/3.7183 = 0.2689. \end{aligned}$$

They sum to 1, as $\sigma(z) + \sigma(-z)$ always does.

Gates $[0.88, 0.88, 0.12, 0.12]$:

$$\begin{aligned} \mathbf{x}' &= [0.88 \times 1.2,\ 0.88 \times 1.0,\ 0.12 \times (-0.4),\ 0.12 \times (-0.6)] \\ &= [1.056,\ 0.880,\ -0.048,\ -0.072], \\ y_0 &= 0.5 \times 1.056 + 0.5 \times 0.880 = 0.528 + 0.440 = 0.968, \\ y_1 &= 0.5 \times (-0.048) + 0.5 \times (-0.072) = -0.024 - 0.036 = -0.060. \end{aligned}$$

Now the second output barely listens to bands 2 and 3.

Solution to Exercise 24.3.

$$\begin{aligned} \text{first layer}, 165 \to 32: &\quad 165 \times 32 + 32 = 5{,}280 + 32 = 5{,}312 \\ \text{second layer}, 32 \to 165: &\quad 32 \times 165 + 165 = 5{,}280 + 165 = 5{,}445 \\ \text{total}: &\quad 5{,}312 + 5{,}445 = 10{,}757. \end{aligned}$$

The real module reports 10,757. Leak:

$$\begin{aligned} \sigma(-4) &= 1/(1 + e^{4}) = 1/(1 + 54.598) = 1/55.598 = 0.0180, \\ 0.0180 &\times 50 = 0.90. \end{aligned}$$

The cross-attention compressor gives a masked band weight exactly 0, so nothing leaks.

Solution to Exercise 24.4.

$$(2500 - 400)/50 = 2100/50 = 42, \qquad \lceil 42 \rceil = 42 \text{ bins.}$$

Here 2100 is a whole multiple of 50, so the last bin is full width. For 864 nm:

$$(864 - 400)/50 = 464/50 = 9.28, \qquad \lfloor 9.28 \rfloor = 9.$$

Bin 9 covers $400 + 50 \times 9 = 850$ to $400 + 50 \times 10 = 900$ nm.

Solution to Exercise 24.5. Spectrum $[0.3, 0.5]$:

$$\tfrac{1}{2}(0.3[1,0] + 0.5[1,1]) = \tfrac{1}{2}([0.3,0] + [0.5,0.5]) = \tfrac{1}{2}[0.8,0.5] = [0.4,0.25].$$

Spectrum $[0.5, 0.3]$:

$$\tfrac{1}{2}(0.5[1,0] + 0.3[1,1]) = \tfrac{1}{2}([0.5,0] + [0.3,0.3]) = \tfrac{1}{2}[0.8,0.3] = [0.4,0.15].$$

Different, so the order of the values survives. The first component is the sum (both 0.8, halved to 0.4), which is all a plain average keeps; the second component, carried only by band 2, tells the two apart. With a shared weight both spectra would have given the same token.

Solution to Exercise 24.6. Band 0 is dead, so its value is multiplied by validity 0:

$$\begin{aligned} \text{vals} &= [99 \times 0,\ 1.0,\ -0.4,\ -0.6] = [0,\ 1.0,\ -0.4,\ -0.6], \\ \text{sum} &= 1.0[0.5,0] + (-0.4)[0,0.5] + (-0.6)[0,0.5] \\ &= [0.5,0] + [0,-0.2] + [0,-0.3] = [0.5,-0.5], \\ \text{mean} &= [0.5,-0.5]/3 = [0.1667,-0.1667]. \end{aligned}$$

Three valid bands, so divide by 3.

Solution to Exercise 24.7.

$$\begin{aligned} (2.8 - 0.4)/(32 - 4) = 2.4/28 = 0.086 \text{ GiB per extra patch,} \\ 0.39/0.086 = 4.5. \end{aligned}$$

The reduced stem's extra patch costs about 4.5 times the linear stem's, the same factor by which the per-band stem's extra patch costs more than the reduced stem's.

Chapter 25: Losses: L1, the spectral angle and the ramp

Solution to Exercise 25.1. **Errors** $e = \hat{x} - x$:

$$\begin{aligned} &\text{A, band 1: } 0.13 - 0.10 = +0.03 \\ &\text{A, band 2: } 0.16 - 0.20 = -0.04 \\ &\text{B, band 1: } 0.30 - 0.30 = \ \ 0.00 \\ &\text{B, band 2: } 0.09 - 0.05 = +0.04 \end{aligned}$$

Absolute values: $0.03, 0.04, 0.00, 0.04$.

Band means per pixel:

$$\begin{aligned} &\text{A: } (0.03 + 0.04)/2 = 0.07/2 = 0.035 \\ &\text{B: } (0.00 + 0.04)/2 = 0.04/2 = 0.020 \end{aligned}$$

Mean over the two graded pixels: $(0.035 + 0.020)/2 = 0.055/2 = 0.0275$.

Check with the second form: the sum of all four absolute errors is $0.03 + 0.04 + 0.00 + 0.04 = 0.11$, and $n = C\,|M| = 2 \times 2 = 4$, so $0.11/4 = 0.0275$.

Gradient. Every graded entry gets $\text{sign}(e)/n$ with $1/n = 1/4 = 0.25$:

$$\frac{\partial L_1}{\partial \hat{\mathbf{x}}_A} = (+0.25,\ -0.25), \qquad \frac{\partial L_1}{\partial \hat{\mathbf{x}}_B} = (0,\ +0.25).$$

Band 1 of B is exact, so $\text{sign}(0) = 0$ and it gets no push.

Solution to Exercise 25.2. **The angle.** $\langle \mathbf{x}, \hat{\mathbf{x}} \rangle = 1 \times 1 + 0 \times 1 = 1$. $a = \|\mathbf{x}\| = \sqrt{1 + 0} = 1$. $b = \|\hat{\mathbf{x}}\| = \sqrt{1 + 1} = \sqrt{2} = 1.414214$.

$$\begin{aligned} u &= 1/(1 \times 1.414214) = 0.707107 \\ \theta &= \arccos(0.707107) = \pi/4 = 0.785398 \text{ rad} \\ \theta &= 0.785398 \times 180/\pi = 45^\circ \end{aligned}$$

The cosine's gradient, Equation (25.4): $\partial u/\partial \hat{\mathbf{x}} = \frac{1}{b}(\mathbf{x}/a - u\,\hat{\mathbf{x}}/b)$.

$$\begin{aligned} \mathbf{x}/a &= (1, 0) \\ \hat{\mathbf{x}}/b &= (1/1.414214,\ 1/1.414214) = (0.707107,\ 0.707107) \\ u\,\hat{\mathbf{x}}/b &= 0.707107 \times (0.707107,\ 0.707107) = (0.5,\ 0.5) \\ \mathbf{x}/a - u\,\hat{\mathbf{x}}/b &= (1 - 0.5,\ 0 - 0.5) = (0.5,\ -0.5) \\ \partial u/\partial \hat{\mathbf{x}} &= (0.5,\ -0.5)/1.414214 = (0.353553,\ -0.353553) \end{aligned}$$

The arccos factor. $1 - u^2 = 1 - 0.5 = 0.5$; $\sqrt{0.5} = 0.707107$; $-1/0.707107 = -1.414214$.

Multiply.

$$\frac{\partial \theta}{\partial \hat{\mathbf{x}}} = -1.414214 \times (0.353553,\ -0.353553) = (-0.5,\ +0.5).$$

Perpendicular to $\hat{\mathbf{x}}$: $(-0.5)(1) + (0.5)(1) = 0$.

Length: $\sqrt{0.25 + 0.25} = \sqrt{0.5} = 0.707107$, and $1/\|\hat{\mathbf{x}}\| = 1/1.414214 = 0.707107$. Equal.

Gradient descent moves $\hat{\mathbf{x}}$ against the gradient, by $(+0.5, -0.5)$ times the step size: it raises band 1 and lowers band 2, rotating $(1, 1)$ towards $(1, 0)$, as it should.

Solution to Exercise 25.3. The derivative of arccos is $-1/\sqrt{1 - u^2}$, which grows without bound as $u \to 1$: -22.4 at $u = 0.999$, -70.7 at 0.9999, $-\infty$ at 1. A well-trained model's pixels have angles of 0.02–0.05 rad, that is $u \geq 0.9987$, so the better the model, the larger this factor.

The true gradient does not blow up because the other factor, $\partial u/\partial \hat{\mathbf{x}}$, has length $\sqrt{1 - u^2}/\|\hat{\mathbf{x}}\|$, which shrinks at exactly the same rate. Their product has length $1/\|\hat{\mathbf{x}}\|$ at every nonzero angle.

What goes wrong is the computation. Automatic differentiation evaluates the two factors separately in float32, where the gap below 1 is $2^{-24} \approx 6 \times 10^{-8}$, so for angles below about 2.4×10^{-4} rad the computed u is 1 or a hair above it. Then arccos returns NaN (for $u > 1$) or the factor is $-\infty$ while $\partial u/\partial \hat{\mathbf{x}}$ is 0, and $\infty \times 0$ is NaN. A clamp of u to $[-1, 1]$ turns some of these NaNs into a silent zero gradient but not all of them.

`SAMLoss` avoids arccos and computes $\theta = \text{atan2}(\sin\text{-term}, \cos\text{-term})$. The guard that makes the derivative finite is the $+\varepsilon$ inside the square root of the sine term (guard 3): it keeps that term at least $\sqrt{10^{-8}} = 10^{-4}$, so the square root's derivative is bounded. Guard 2 (clamping the dot product to $[-N, N]$) keeps the quantity under the root non-negative when rounding makes the dot product exceed the product of lengths.

Solution to Exercise 25.4. **Epoch index 4.**

$$\begin{aligned} t/R &= 4/10 = 0.4 \\ \min(1, 0.4) &= 0.4 \\ \lambda(4) &= 0.5 \times 0.4 = 0.20 \end{aligned}$$

Epoch index 50. $50/10 = 5$, $\min(1, 5) = 1$, $\lambda = 0.5 \times 1 = 0.5$.

$$\begin{aligned} L &= 0.02 + 0.5 \times 0.12 \\ &= 0.02 + 0.06 \\ &= 0.08 \end{aligned}$$

The angle contributes 0.06 against L1's 0.02: $0.06/0.02 = 3$ times more, 75% of the total.

Solution to Exercise 25.5. **At $c = 0.12$.** Differences $y_i - c$: $-0.02, -0.01, 0.00, +0.01, +0.78$.

$$\begin{aligned} S_1 &= 0.02 + 0.01 + 0.00 + 0.01 + 0.78 = 0.82 \\ S_2 &= 0.0004 + 0.0001 + 0 + 0.0001 + 0.6084 = 0.6090 \end{aligned}$$

At $c = 0.272$. Differences: $-0.172, -0.162, -0.152, -0.142, +0.628$.

$$\begin{aligned} S_1 &= 0.172 + 0.162 + 0.152 + 0.142 + 0.628 = 1.256 \\ S_2 &= 0.029584 + 0.026244 + 0.023104 + 0.020164 + 0.394384 = 0.493480 \end{aligned}$$

S_1 is smaller at $c = 0.12$ (the median): $0.82 < 1.256$. S_2 is smaller at $c = 0.272$ (the mean): $0.4935 < 0.6090$. Squared error moves the best prediction towards the glitch; absolute error does not.

Solution to Exercise 25.6. Each prediction-mask flag lives on a token that covers a 4×4 block of pixels, and the mask must stay strictly 0 or 1 so that `loss_mask==1` selects whole blocks. Nearest-neighbour upsampling copies each flag to its 16 pixels. Bilinear upsampling averages neighbouring flags, so pixels near the boundary between a hidden and a visible token get fractional values such as 0.25, 0.5 or 0.75. The test `loss_mask==1` rejects them silently, so a lattice of pixels along every block edge is never graded, and the product with the eroded mask would no longer be a clean logical AND.

Chapter 26: Training data and the training loop: gradients, AdamW, warmup and cosine schedules

Solution to Exercise 26.1. $L'(w) = 2(w - 3)$, $\eta = 0.25$, $w_0 = 0$.

Step 1: $g = 2(0 - 3) = -6$,	$w_1 = 0 - 0.25 \times (-6) = 1.5$
Step 2: $g = 2(1.5 - 3) = -3$,	$w_2 = 1.5 - 0.25 \times (-3) = 2.25$
Step 3: $g = 2(2.25 - 3) = -1.5$,	$w_3 = 2.25 - 0.25 \times (-1.5) = 2.625$

The distances to 3 are $-3, -1.5, -0.75, -0.375$: each is multiplied by $1 - 2\eta = 1 - 0.5 = 0.5$. One step lands exactly on 3 when the factor is 0, that is $1 - 2\eta = 0$, $\eta = 0.5$: from $w_0 = 0$, $w_1 = 0 - 0.5 \times (-6) = 3$. (For this loss the curvature is $L'' = 2$, and the one-step learning rate is $1/L''$.)

Solution to Exercise 26.2. $g = 2$, $g^2 = 4$, $\beta_1 = 0.9$, $\beta_2 = 0.999$.

$t = 1$**.**

$$\begin{aligned} m_1 &= 0.9 \times 0 + 0.1 \times 2 = 0.2 \\ v_1 &= 0.999 \times 0 + 0.001 \times 4 = 0.004 \\ \hat{m}_1 &= 0.2/0.1 = 2 \\ \hat{v}_1 &= 0.004/0.001 = 4, \quad \sqrt{4} = 2 \\ \text{step} &= 10^{-3} \times 2/2 = 10^{-3} \end{aligned}$$

$t = 2$**.**

$$\begin{aligned} m_2 &= 0.9 \times 0.2 + 0.1 \times 2 = 0.18 + 0.2 = 0.38 \\ v_2 &= 0.999 \times 0.004 + 0.001 \times 4 = 0.003996 + 0.004 = 0.007996 \\ \hat{m}_2 &= 0.38/0.19 = 2 \\ \hat{v}_2 &= 0.007996/0.001999 = 4, \quad \sqrt{4} = 2 \\ \text{step} &= 10^{-3} \times 2/2 = 10^{-3} \end{aligned}$$

Bias correction recovers the true mean (2) and mean square (4) exactly for a constant gradient, and the step is η.

Without correction at $t = 1$**:** $m_1/\sqrt{v_1} = 0.2/\sqrt{0.004} = 0.2/0.063246 = 3.1623$, so the step would be 3.16×10^{-3}, more than three times too large ($\sqrt{1000}/10 = 31.62/10 = 3.162$).

Solution to Exercise 26.3. $\eta(t) = 10^{-6} + 0.0004995\,(1 + \cos(\pi t/200))$.

$t = 25$**:** $\pi \times 25/200 = \pi/8$, $\cos(\pi/8) = 0.923880$.

$$\begin{aligned} 1 + 0.923880 &= 1.923880 \\ 0.0004995 \times 1.923880 &= 0.00096098 \\ 0.000001 + 0.00096098 &= 0.00096198 = 9.620 \times 10^{-4} \end{aligned}$$

$t = 175$**:** $\pi \times 175/200 = 7\pi/8$, $\cos(7\pi/8) = -0.923880$.

$$\begin{aligned} 1 - 0.923880 &= 0.076120 \\ 0.0004995 \times 0.076120 &= 0.000038022 \\ 0.000001 + 0.000038022 &= 0.000039022 = 3.902 \times 10^{-5} \end{aligned}$$

Rate of change. $d\eta/dt = -\frac{\pi}{400}(0.000999)\sin(\pi t/200)$. Since $\sin(\pi/8) = \sin(7\pi/8) = 0.382683$, the rate is the same at both epochs: $-0.0078540 \times 0.000999 \times 0.382683 = -3.00 \times 10^{-6}$ per epoch. The cosine curve is symmetric about $t = 100$. In *relative* terms, though, the change at $t = 175$ is far larger: 3.0×10^{-6} is 0.3% of 9.6×10^{-4} but 7.7% of 3.9×10^{-5}.

Solution to Exercise 26.4. $\|(1, 2, 2)\| = \sqrt{1 + 4 + 4} = \sqrt{9} = 3$.

To 1.5: $\min(1, 1.5/3) = 0.5$, so $\mathbf{g} \to (0.5, 1, 1)$, whose length is $\sqrt{0.25 + 1 + 1} = 1.5$.

To 5: $\min(1, 5/3) = 1$: unchanged, $(1, 2, 2)$.

Would clipping at 1.0 change this repository's training? Adam divides each parameter's gradient by a running estimate of its size, so if every step's gradient were clipped by the same factor, $\hat{m}/\sqrt{\hat{v}}$ would be unchanged and so would training. Clipping changes things only on the steps where the gradient norm is unusually large: it shrinks those gradients before they enter m and v, so one wild batch moves the parameters and inflates v less. Whether a threshold of 1.0 is ever active depends on the typical whole-model gradient norm, which the repository does not log; you would measure it before choosing a threshold.

Solution to Exercise 26.5. **Rows** (height 400):

$$0 + 128 = 128 \leq 400 \text{ keep } 0; \quad 64 + 128 = 192 \text{ keep } 64; \quad 128 + 128 = 256 \text{ keep } 128;$$
$$192 + 128 = 320 \text{ keep } 192; \quad 256 + 128 = 384 \leq 400 \text{ keep } 256;$$
$$320 + 128 = 448 > 400\text{: snap to } 400 - 128 = 272, \text{ stop.}$$

Rows $[0, 64, 128, 192, 256, 272]$: 6.

Columns (width 300): as in worked example 26.9, $[0, 64, 128, 172]$: 4.

Patches: every row with every column, $6 \times 4 = 24$. The real generator returns the same [C].

A duplicate: any height where (height-128) is a multiple of 64, for example 192: rows 0 and 64 are kept ($64 + 128 = 192 \leq 192$), then row 128 gives $256 > 192$ and snaps to $192 - 128 = 64$, already present. Heights 128, 256 and 320 do the same.

Solution to Exercise 26.6. Near 0.2 a bfloat16 number has 8 significant bits, so neighbouring representable values are $2^{-3} \times 2^{-7} = 2^{-10} = 0.000977$ apart: 0.2001953, then 0.2011719. Both 0.2 and 0.2004 round to 0.2001953, so their difference, 0.0004, becomes exactly 0. A good model's errors are of this size, so a loss computed in bfloat16 would be blind to precisely the errors training is trying to remove.

The SpectralEarth trainer runs only the model's forward pass under bfloat16 autocast and converts the output to float32 (`out.float()`) before the loss. The truth is never rounded, and the subtraction, absolute value, angle and averaging run in float32. (The prediction itself still carries the rounding of the network's last bfloat16 operations, in the network's normalised units.)

Chapter 27: Pretraining on SpectralEarth

Solution to Exercise 27.1. With the ramp:

$$S = 0.050 + 0.05 \times 0.18 = 0.050 + 0.009 = 0.059.$$

At the full weight:

$$S = 0.050 + 0.5 \times 0.18 = 0.050 + 0.090 = 0.140.$$

Change from epoch 1 under the ramp, term by term (epoch 1: L1 = 0.055, SAM = 0.20, $\lambda = 0$):

$$\begin{aligned} \Delta L1 &= 0.050 - 0.055 = -0.005 \\ \lambda_1 \, \Delta \,\text{SAM} &= 0 \times (0.18 - 0.20) = 0 \\ \Delta\lambda \,\text{SAM} &= (0.05 - 0) \times 0.18 = 0.009 \\ \text{sum} &= -0.005 + 0 + 0.009 = +0.004. \end{aligned}$$

Check: $0.059 - 0.055 = 0.004$. The ramp term ($+0.009$) outweighs the L1 improvement (-0.005), so the ramped score rises although both measurements improved. At the full weight the score fell from 0.155 to 0.140.

Solution to Exercise 27.2. A later epoch must score below $0.155 \times (1 - 0.01) = 0.155 \times 0.99 = 0.15345$. A score of 0.1540 is lower than 0.155 but not lower than 0.15345, so it is *not* an improvement: every later epoch adds one bad epoch. After epoch e the counter is $e - 1$. Patience 10 is reached at epoch 11 ($11 - 1 = 10$), but the minimum is 20, so the run continues. At epoch 20 the counter is 19: $20 \geq 20$ and $19 \geq 10$, so the run stops at epoch 20. The 1% threshold exists to stop a slow creep of sub-1% gains from keeping a run alive for ever.

Solution to Exercise 27.3. Plain average, $N = 4$:

$$s = \frac{2 + 2 + 2 + (-1)}{4} = \frac{5}{4} = 1.25,$$

different again from A's 1.0 and B's 0.5. Width-weighted:

$$s_w = \frac{2 \times 5 + 2 \times 5 + 2 \times 5 + (-1) \times 10}{5 + 5 + 5 + 10} = \frac{10 + 10 + 10 - 10}{25} = \frac{20}{25} = 0.8.$$

It is not 0.5 because C covers 15 nm of visible where A and B cover 10 nm. Width weighting makes the answer depend on *how much spectrum* is covered, as an integral should, not on how many bands cover it. A spectrum that is 2 over a longer stretch genuinely has a higher mean value over wavelength.

Solution to Exercise 27.4.

$$\begin{aligned} \text{done} &= 128 - 1 = 127 \\ k &= \lfloor 127/20 \rfloor = 6 \quad (6 \times 20 = 120 \le 127 < 140) \\ \text{stop_at} &= 20 \times (6 + 1) = 140 \\ \text{shards} &= 200 \times 6 = 1200 \text{ to } 1200 + 199 = 1399. \end{aligned}$$

The run resumes at epoch 128 on block 6 and stops at 140. Learning rate at $t = 100$:

$$\begin{aligned} \cos(\pi \times 100/200) &= \cos(\pi/2) = 0 \\ \eta &= 0.000001 + 0.5 \times 0.000499 \times (1 + 0) = 0.000001 + 0.0002495 = 0.0002505. \end{aligned}$$

Halfway through the schedule the rate is halfway between η_0 and $\eta_{\min}$.

Solution to Exercise 27.5. Deltas (a zero band in front):

$$2000 - 0 = 2000, \quad 2010 - 2000 = 10, \quad 1990 - 2010 = -20, \quad 1995 - 1990 = 5.$$

Decode by running sum: 2000; $2000 + 10 = 2010$; $2010 - 20 = 1990$; $1990 + 5 = 1995$.

Wrapped pair. First delta: $-32{,}768 - 0 = -32{,}768$ (in range). Second:

$$\begin{aligned} 30{,}000 - (-32{,}768) &= 62{,}768 \quad \text{(too big for int16)} \\ 62{,}768 - 65{,}536 &= -2{,}768 \quad \text{(stored)}. \end{aligned}$$

Decode: first value $-32{,}768$; second

$$\begin{aligned} -32{,}768 + (-2{,}768) &= -35{,}536 \quad \text{(too small)} \\ -35{,}536 + 65{,}536 &= 30{,}000 \quad \text{(exact)}. \end{aligned}$$

Solution to Exercise 27.6.

- Target 500: nearest source 500.3, distance $0.3 \le 0.5$: take it.
- Target 510: nearest source 509.6, distance $|510 - 509.6| = 0.4 \le 0.5$: take it.
- Target 520: nearest source 525.0, distance $5.0 > 0.5$: nodata.
- Source 525.0 matches no target (it is 5 nm from 520): dropped.

Kept: 500.3 and 509.6 (as 500 and 510); nodata: 520; dropped: 525.0.

Chapter 28: The segmentation head, BCE plus Dice, and auditing labels

Solution to Exercise 28.1. Closed form with $d = 64$, $K = 5$:

$$\begin{aligned} 13d^2 &= 13 \times 4{,}096 = 53{,}248 \\ (518 + 144 \times 5)\, d &= (518 + 720) \times 64 = 1{,}238 \times 64 = 79{,}232 \\ 16K &= 80 \\ P &= 53{,}248 + 79{,}232 + 80 = 132{,}560. \end{aligned}$$

Layer by layer:

$$\begin{aligned} 32 \to 64: &\quad 32 \times 64 + 64 = 2{,}048 + 64 = 2{,}112 \\ 64 \to 64: &\quad 64 \times 64 + 64 = 4{,}096 + 64 = 4{,}160 \\ 160 \to 64: &\quad 160 \times 64 + 64 = 10{,}240 + 64 = 10{,}304 \\ 256 \to 64: &\quad 256 \times 64 + 64 = 16{,}384 + 64 = 16{,}448 \\ \text{fuse } 256 \to 64: &\quad 256 \times 64 + 64 = 16{,}448 \\ \text{refine 1}: &\quad 9 \times 64 \times 64 + 64 = 36{,}864 + 64 = 36{,}928 \\ \text{refine 2}: &\quad 9 \times 64 \times 80 + 80 = 46{,}080 + 80 = 46{,}160. \end{aligned}$$

Sum: 2,112+4,160 = 6,272; +10,304 = 16,576; +16,448 = 33,024; +16,448 = 49,472; +36,928 = 86,400; +46,160 = 132,560. The two agree.

Solution to Exercise 28.2. Sigmoid for water: $1/(1 + e^{-1}) = 1/(1 + 0.368) = 1/1.368 = 0.731$, and the same for thin cloud, independently.

Softmax: the exponentials are $e^{1} = 2.718$ (twice) and $e^{-3} = 0.0498$ (three times).

$$\begin{aligned} \text{sum} &= 2.718 + 2.718 + 0.0498 + 0.0498 + 0.0498 = 5.436 + 0.149 = 5.585 \\ \text{water} &= 2.718/5.585 = 0.487. \end{aligned}$$

The softmax must split the belief between water and thin cloud; the sigmoids give each 0.731.

Solution to Exercise 28.3. For $t = 1$, $p = 0.2$: $\ell = -\ln 0.2 = 1.6094$; gradient $p - t = 0.2 - 1 = -0.8$ (push the logit up). For $t = 0$, $p = 0.2$: $\ell = -\ln(1 - 0.2) = -\ln 0.8 = 0.2231$; gradient $0.2 - 0 = 0.2$ (push down, gently).

Numerical check for the first pixel. With $t = 1$, $\ell(z) = -\ln \sigma(z) = \ln(1 + e^{-z})$.

$$\begin{aligned} z = -1.376: &\quad e^{1.376} = 3.959; \quad \ell = \ln(4.959) = 1.6014 \\ z = -1.396: &\quad e^{1.396} = 4.039; \quad \ell = \ln(5.039) = 1.6174 \\ &\text{slope} = (1.6014 - 1.6174)/0.02 = -0.0160/0.02 = -0.80, \end{aligned}$$

matching $p - t = -0.8$.

Solution to Exercise 28.4. Water term with pixel 3 at $p = 0.2$ (label 0):

$$\begin{aligned} \text{BCE terms} &= 0.1054,\ 0.1054,\ -\ln(0.8) = 0.2231 \\ \text{BCE} &= (0.1054 + 0.1054 + 0.2231)/3 = 0.4339/3 = 0.1446 \\ \textstyle\sum p &= 0.9 + 0.1 + 0.2 = 1.2; \quad \textstyle\sum pt = 0.9; \quad \textstyle\sum t = 1 \\ \mathcal{L}_{\text{Dice}} &= 1 - \frac{2.8}{1.2 + 1 + 1} = 1 - \frac{2.8}{3.2} = 1 - 0.875 = 0.125 \\ \text{water term} &= 0.1446 + 0.125 = 0.2696. \end{aligned}$$

Soft Dice of the 2×2 mask with the top-right probability at 0.9:

$$\begin{aligned} \textstyle\sum pt &= 0.8 + 0.9 = 1.7 \\ \textstyle\sum p &= 0.8 + 0.9 + 0.3 + 0.1 = 2.1; \quad \textstyle\sum t = 2 \\ Q &= \frac{2 \times 1.7 + 1}{2.1 + 2 + 1} = \frac{4.4}{5.1} = 0.863 \\ \mathcal{L}_{\text{Dice}} &= 1 - 0.863 = 0.137, \end{aligned}$$

down from 0.261, as the positive gradient of Section 28.5 predicted.

Solution to Exercise 28.5.

$$\begin{aligned} \text{union} &= 8 + 10 - 6 = 12 \\ J &= 6/12 = 0.50 \\ D &= \frac{2 \times 6}{8 + 10} = \frac{12}{18} = 0.667 \\ \frac{2J}{1 + J} &= \frac{2 \times 0.5}{1 + 0.5} = \frac{1}{1.5} = 0.667. \quad \checkmark \end{aligned}$$

Solution to Exercise 28.6. Flatness $0.22/0.36 = 0.61$, below 0.7: not flat, so not plausible, although it is bright ($0.24 > 0.2$). It looks more like bright soil than cloud.

$812/1000 = 0.812\,\text{g/cm}^2$, not below 0.75: the scene is not dry and its cloud labels are used.

"Water vapour below 1.0" achieves $66.6/15.7 = 4.2$ suspect labels per plausible one, slightly worse than the adopted 4.6. It removes 4 points more suspect labels (66.6% against 62.6%) but touches 15.2% of tiles instead of 5.6% and removes more plausible labels (15.7% against 13.6%).

Chapter 29: Fine-tuning: the curriculum, the BatchNorm see-saw and mixed batches

Solution to Exercise 29.1. Apply $r \leftarrow 0.9\,r + 0.1 \times 0.30$, where $0.1 \times 0.30 = 0.03$:

$$\begin{aligned} r_1 &= 0.9 \times 0.10 + 0.03 = 0.09 + 0.03 = 0.12 \\ r_2 &= 0.9 \times 0.12 + 0.03 = 0.108 + 0.03 = 0.138 \\ r_3 &= 0.9 \times 0.138 + 0.03 = 0.1242 + 0.03 = 0.1542. \end{aligned}$$

Closed form: $0.9^3 = 0.729$, so $r_3 = 0.729 \times 0.10 + (1 - 0.729) \times 0.30 = 0.0729 + 0.271 \times 0.30 = 0.0729 + 0.0813 = 0.1542$. The two agree.

Solution to Exercise 29.2. The last 5 micro-batches carry $1 - 0.9^5 = 1 - 0.590 = 0.41$, about 41%.

For 95%: $1 - 0.9^K = 0.95$, so $0.9^K = 0.05$ and

$$K = \frac{\ln 0.05}{\ln 0.9} = \frac{-2.996}{-0.1054} = 28.4,$$

so 29 micro-batches carry 95% of the stored mean: still under 1% of a 3,750-micro-batch epoch.

Solution to Exercise 29.3.

- 999.8 is $1000 - 999.8 = 0.2\,\text{nm}$ from 1000: already known, skipped.
- 1004.6 is 4.6 nm from 1000 and 5.4 nm from 1010: new.

- 1005.0 is new against the record but $1005.0 - 1004.6 = 0.4$ nm from 1004.6: counted once with it.
- 1030.0 is 20 nm from 1010: new.

Record: 1000, 1004.6, 1010, 1030; the call returns 2. A band at 1018 nm is $1018 - 1010 = 8$ nm from the nearest recorded wavelength, within 15 nm: accepted by `trained_band_mask`.

Solution to Exercise 29.4.

$$\begin{aligned} \text{EnMAP:} &\quad \operatorname{round}(0.40 \times 4) = \operatorname{round}(1.6) = 2 \\ \text{PRISMA:} &\quad \operatorname{round}(0.35 \times 4) = \operatorname{round}(1.4) = 1 \\ \text{AVIRIS-NG:} &\quad \max(1, \operatorname{round}(0.25 \times 4)) = \max(1, 1) = 1 \\ \text{total:} &\quad 2 + 1 + 1 = 4. \end{aligned}$$

Effective shares $2/4, 1/4, 1/4 = 0.50, 0.25, 0.25$: EnMAP gains 10 points, PRISMA loses 10. Small batches round coarsely.

Toy problem from $w = 1$, using the maps $B : w \mapsto 0.5w - 0.5$ and $A : w \mapsto 0.5w + 0.5$:

$$\begin{aligned} \text{B:} &\quad 0.5 \times 1 - 0.5 = 0 \\ \text{A:} &\quad 0.5 \times 0 + 0.5 = 0.5 \\ \text{B:} &\quad 0.5 \times 0.5 - 0.5 = -0.25. \end{aligned}$$

Solution to Exercise 29.5. With $a = 0.1$ and $b = 0.6$:

$$\begin{aligned} \text{after the mixed pass:} &\quad 0.9\, r^* + 0.01 \\ \text{after the EnMAP-only pass:} &\quad 0.9(0.9\, r^* + 0.01) + 0.06 \\ &\quad = 0.81\, r^* + 0.009 + 0.06 = 0.81\, r^* + 0.069 \\ &\quad r^* = 0.81\, r^* + 0.069 \;\Rightarrow\; 0.19\, r^* = 0.069 \;\Rightarrow\; r^* = 0.363. \end{aligned}$$

With `segment_mixed` both passes see the mix's 0.1, so the fixed point is $(0.09 \times 0.1 + 0.1 \times 0.1)/0.19 = 0.019/0.19 = 0.1$: the stored mean stays at the mix's value.

Solution to Exercise 29.6.

$$\begin{aligned} 4.5^\circ &= 4.5 \times 3.1416/180 = 0.0785 \text{ rad} \\ \text{reconstruction} &= 0.015 + 0.5 \times 0.0785 = 0.015 + 0.0393 = 0.0543. \end{aligned}$$

Segmentation shares with segmentation 0.3 and reconstruction 0.06:

$$\begin{aligned} w_s = 1.0: &\quad 0.3/(0.06 + 0.3) = 0.3/0.36 = 83\% \\ w_s = 0.1: &\quad 0.03/(0.06 + 0.03) = 0.03/0.09 = 33\% \\ w_s = 0.2: &\quad 0.06/(0.06 + 0.06) = 0.06/0.12 = 50\%. \end{aligned}$$

Chapter 30: Inference on a whole scene

Solution to Exercise 30.1. One pass with a 50% mask hides only half the tokens, so only half the pixels get a prediction made without seeing themselves; the other half were visible and their predictions may be copies, so they have no honest prediction at all. One pass with no mask is worse: every pixel is visible, the model can copy every one of them, and the residual is near zero everywhere, including at the anomalies you are looking for.

Solution to Exercise 30.2. Rows, $W = 1200$:

$$\begin{aligned} W - P &= 1200 - 128 = 1072, \\ \frac{1072}{32} &= 33.5, \\ \lceil 33.5 \rceil + 1 &= 34 + 1 = 35. \end{aligned}$$

Columns, $W = 900$:

$$\begin{aligned} W - P &= 900 - 128 = 772, \\ \frac{772}{32} &= 24.125, \\ \lceil 24.125 \rceil + 1 &= 25 + 1 = 26. \end{aligned}$$

Neither quotient is a whole number, so no duplicate tile arises. Tiles:

$$35 \times 26 = 910.$$

Batches of 8:

$$\begin{aligned} \frac{910}{8} &= 113.75, \\ \lceil 113.75 \rceil &= 114 \text{ batches (the last holds } 910 - 113 \times 8 = 910 - 904 = 6 \text{ tiles}). \end{aligned}$$

Forward passes, two per batch:

$$114 \times 2 = 228.$$

The book's script gives the same: 910 tiles, 114 batches, 228 forward passes.

Solution to Exercise 30.3. With `cell_size=2`, rows and columns $0, 1, 2, 3$ become $0, 0, 1, 1$ after integer division by 2. The sum table is

$$\begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 2 & 2 \\ 1 & 1 & 2 & 2 \end{pmatrix}, \qquad \text{and modulo 2:} \qquad \text{grid} = \begin{pmatrix} 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 \end{pmatrix}.$$

In the grid 1 is visible and 0 hidden. Pass 1 hides the 0 entries: the top-left and bottom-right 2×2 blocks. Pass 2 uses $1 - \text{grid}$ and hides the top-right and bottom-left blocks. Together they hide every token exactly once.

Solution to Exercise 30.4. Weighted sum and count:

$$\begin{aligned} \text{sum} &= 1 \times 0.31 + 1 \times 0.29 + 1 \times 0.30 + 0 \times 0.70 \\ &= 0.31 + 0.29 + 0.30 + 0 \\ &= 0.90, \\ \text{count} &= 1 + 1 + 1 + 0 = 3, \\ \hat{x} &= \frac{0.90}{3} = 0.30. \end{aligned}$$

Residual:

$$r = x - \hat{x} = 0.36 - 0.30 = 0.06.$$

If all four weights were 0, the count would be 0, the code would set $\hat{x} = x = 0.36$, and the residual would be $0.36 - 0.36 = 0$: the pixel could never be flagged.

Solution to Exercise 30.5. Fitting starts satisfy $32k \le 600 - 128 = 472$, so $k \le 472/32 = 14.75$, $k \le 14$, and the last fitting start is $14 \times 32 = 448$. It covers $448, \ldots, 575$, leaving $576, \ldots, 599$ uncovered, so the snapped start $600 - 128 = 472$ is added. The last three starts are 416, 448, 472.

Column 500 lies in a tile starting at s when $s \le 500 \le s + 127$, that is $373 \le s \le 500$. The starts in that range are 384 ($= 12 \times 32$), 416, 448 and 472. Coverage: 4 tiles.

Solution to Exercise 30.6. A checkpoint holds numbers, not an architecture. `load_state_dict` copies each saved tensor into the slot of the same name in a model you have already built, and by default it insists that every slot exists with exactly the same shape. The spectral compressor's weights have shapes that depend on D (for example $165 \times D$ entries in its first layer), so a model built with $D = 24$ has slots of the wrong shape for a $D = 32$ checkpoint, and loading stops at once with a shape-mismatch error. That loud failure is better than a silently wrong score.

Chapter 31: From residual to score: residual Mahalanobis, the ring and integral images

Solution to Exercise 31.1. With the inverse from worked example 31.1,

$$\boldsymbol{\Sigma}_r^{-1} = \begin{pmatrix} 2/3 & -1/3 & 0 \\ -1/3 & 2/3 & 0 \\ 0 & 0 & 1/4 \end{pmatrix},$$

and $\mathbf{r} = (1, 1, 2)$:

$$\boldsymbol{\Sigma}_r^{-1}\mathbf{r} = \left(\tfrac{2}{3}\cdot 1 - \tfrac{1}{3}\cdot 1,\ -\tfrac{1}{3}\cdot 1 + \tfrac{2}{3}\cdot 1,\ \tfrac{1}{4}\cdot 2\right) = \left(\tfrac{1}{3},\ \tfrac{1}{3},\ \tfrac{1}{2}\right),$$
$$D^2 = 1\cdot\tfrac{1}{3} + 1\cdot\tfrac{1}{3} + 2\cdot\tfrac{1}{2} = \tfrac{1}{3} + \tfrac{1}{3} + 1 = \tfrac{5}{3} \approx 1.667.$$

Rotate and divide, with $0.7071 = 1/\sqrt{2}$:

$$y_1 = 0.7071\,(1+1) = 1.414, \qquad z_1 = 1.414/1.732 = 0.8165,$$
$$y_2 = 0.7071\,(1-1) = 0, \qquad z_2 = 0,$$
$$y_3 = 2, \qquad z_3 = 2/2 = 1,$$
$$D^2 = 0.8165^2 + 0^2 + 1^2 = 0.6667 + 0 + 1 = 1.667.$$

L1 is $(1 + 1 + 2)/3 = 4/3 = 1.333$, the same as **Q**'s $(2 + 2 + 0)/3 = 1.333$. By size the two are equal; by direction the new pixel scores 1.667, below even the ordinary pixels' 2.25, while **Q** scores 8. Its error lies along "A and B together" and along band C, both directions the background already takes.

Solution to Exercise 31.2. Box rows 0–1, columns 2–3: $r_0 = 0$, $r_1 = 1$, $c_0 = 2$, $c_1 = 3$.

$$S = I[2][4] - I[0][4] - I[2][2] + I[0][2] = 10 - 0 - 7 + 0 = 3.$$

Check: the values are 0, 1 (row 0) and 2, 0 (row 1): $0 + 1 + 2 + 0 = 3$.

Ring around value row 1, column 2. Outer box rows 0–2, columns 1–3:

$$S_{\text{outer}} = I[3][4] - I[0][4] - I[3][1] + I[0][1] = 16 - 0 - 4 + 0 = 12.$$

Guard, the single value at row 1, column 2:

$$S_{\text{guard}} = I[2][3] - I[1][3] - I[2][2] + I[1][2] = 9 - 3 - 7 + 3 = 2.$$

Ring sum $12-2=10$, count $9-1=8$, mean $10/8=1.25$. Check: the eight neighbours are $2,0,1$ (row 0), $1,0$ (row 1, either side of the centre) and $2,1,3$ (row 2): $2+0+1+1+0+2+1+3=10$.

Solution to Exercise 31.3. Mean:

$$m_a = 8/4 = 2, \qquad m_b = 4/4 = 1.$$

Covariance, $\frac{1}{n}\sum \mathbf{x}\mathbf{x}^\top - \mathbf{m}\mathbf{m}^\top$, entry by entry:

$$\begin{aligned}\sigma_{aa} &= \frac{20}{4} - 2\cdot 2 = 5 - 4 = 1,\\ \sigma_{ab} &= \frac{6}{4} - 2\cdot 1 = 1.5 - 2 = -0.5,\\ \sigma_{bb} &= \frac{6}{4} - 1\cdot 1 = 1.5 - 1 = 0.5.\end{aligned}$$

(One set of pixels with these sums is $a=(1,3,1,3)$, $b=(2,0,1,1)$; you can check the result directly.)

Solution to Exercise 31.4. Ring:

$$31\times 31 - 11\times 11 = 961 - 121 = 840 \text{ pixels.}$$

Fallback threshold: `MIN_RING_FACTOR` $\times d = 2\times 20 = 40$ valid ring pixels. Cost per pixel and channel: direct, 840 additions; with integral images, two boxes of four look-ups, 8. The reduction is $840/8 = 105$ times.

Solution to Exercise 31.5. The top two directions are $\mathbf{u}_3$ ($\lambda=4$) and $\mathbf{u}_1$ ($\lambda=3$). For $(1,1,2)$: $z_3 = 2/2 = 1$ and $z_1 = 1.414/1.732 = 0.8165$, so $D^2 = 1 + 0.6667 = 1.667$, unchanged because it had no component along the dropped $\mathbf{u}_2$. For $\mathbf{Q}=(2,-2,0)$: $z_3 = 0$ and $z_1 = 0.7071\,(2-2)/1.732 = 0$, so $D^2 = 0$. Truncation gave up all sensitivity along the quietest direction, which is exactly where $\mathbf{Q}$, the most unusual pixel, differed from the background.

Solution to Exercise 31.6. Five independent coin flips:

$$P(\text{all 5 to D}) = \left(\tfrac{1}{2}\right)^5 = \tfrac{1}{32},$$
$$P(\text{all 5 one way or the other}) = 2\times\tfrac{1}{32} = \tfrac{2}{32} = 0.0625.$$

Still above 0.05: even five out of five would not pass a conventional two-sided test.

Chapter 32: Naming the material, export, and using Indradhanu V2 in a chain

Solution to Exercise 32.1. Position of the 99.75th percentile:

$$\begin{aligned}N - 1 &= 2{,}400{,}000 - 1 = 2{,}399{,}999,\\ \text{pos} &= 0.9975 \times 2{,}399{,}999\\ &= 2{,}399{,}999 - 0.0025 \times 2{,}399{,}999\\ &= 2{,}399{,}999 - 5{,}999.9975\\ &= 2{,}393{,}999.0025,\\ \lceil \text{pos} \rceil &= 2{,}394{,}000,\\ n_{\text{flag}} &= 2{,}400{,}000 - 2{,}394{,}000 = 6{,}000.\end{aligned}$$

That is 0.25% of 2,400,000, as intended.

Solution to Exercise 32.2. Rescale each map:

$$\tilde{s}_1 = \frac{0.4 - 0.1}{1.6 - 0.1} = \frac{0.3}{1.5} = 0.20,$$
$$\tilde{s}_2 = \frac{12 - 0}{60 - 0} = \frac{12}{60} = 0.20.$$

Normalise the weights: $\omega_1 = 1/(1+3) = 0.25$, $\omega_2 = 3/(1+3) = 0.75$. Combine:

$$c = 0.25 \times 0.20 + 0.75 \times 0.20 = 0.05 + 0.15 = 0.20.$$

Solution to Exercise 32.3.

$$\frac{\alpha}{F} = \frac{0.02}{15} = 0.0013333,$$
$$p^* = 1 - 0.0013333 = 0.9986667.$$

By the union bound an ordinary pixel gets some man-made name with probability at most $15 \times 0.0013333 = 0.02 = \alpha$. Adding families raised the bar so that α keeps its meaning.

Solution to Exercise 32.4. The pixel centre is at column $j = 20.5$, row $i = 10.5$. With Equation (32.6):

$$X = 30 \times 20.5 + 0 \times 10.5 + 500{,}000 = 615 + 0 + 500{,}000 = 500{,}615 \text{ m},$$
$$Y = 0 \times 20.5 + (-30) \times 10.5 + 1{,}400{,}000 = 0 - 315 + 1{,}400{,}000 = 1{,}399{,}685 \text{ m}.$$

The identity transform would give $X = 20.5$, $Y = 10.5$: pixel coordinates, which a GIS would place near the origin of the coordinate system, nowhere near the scene. That is why the CRS and transform must be recovered at export.

Solution to Exercise 32.5. The action is accepted, but in the worker `_check_band_grid` finds that the vendable's wavelengths equal the 165-band grid while V2 reads native bands, and the action fails with "model 'Indradhanu V2' reads native bands: run `band_filter_apply` with `use_common_wavelength_grid=false`". Set `use_common_wavelength_grid:false` on the band filter. MNF-RX, a classical detector, runs on either grid, so it can stay in the same action. With today's default band filter (native bands, since 2026-10-31) the action runs as it is.

Solution to Exercise 32.6.

$$\text{total} = 800 \times 900 = 720{,}000,$$
$$\text{valid} = 720{,}000 - 20{,}000 = 700{,}000,$$
$$\texttt{excluded_percent} = \frac{100 \times 35{,}800}{700{,}000} = \frac{3{,}580{,}000}{700{,}000} = 5.114.$$

Only cloud is excluded. Shadow is a class in the map but is not zeroed, so a vessel in cloud shadow is scored normally by residual Mahalanobis; its score is not 0.

Solution to Exercise 32.7.

$$P(\text{bare metal}) = 0.7 \times 0.5 = 0.35, \qquad P(\text{painted metal}) = 0.7 \times 0.3 = 0.21,$$
$$P(\text{tarp}) = 0.7 \times 0.2 = 0.14, \qquad P(\text{natural}) = 1 - 0.7 = 0.30,$$

and $0.35 + 0.21 + 0.14 + 0.30 = 1.00$.

Solution to Exercise 32.8. $\mathbf{d} = 1.0 \times (12.5, -2, 10) = (12.5, -2, 10)$. Against soil: $\mathbf{s} \cdot \mathbf{d} = 7.5 \times 12.5 + 3.75 \times 2 + 5 \times 10 = 93.75 + 7.5 + 50 = 151.25$; $\mathbf{s} \cdot \mathbf{s} = 95.3125$; $\mathbf{d} \cdot \mathbf{d} = 260.25$; ACE $= 151.25^2/(95.3125 \times 260.25) = 22876.56/24804.58 = 0.9223$. Without arithmetic: the fill only scales $\mathbf{d}$, and ACE does not change when $\mathbf{d}$ is scaled. At fill 0.25: $0.7724 \times 0.7151 = 0.5523$, the table's 0.55 for level 2 right; it must agree, because a class can be right only when level 1 was.

Chapter 33: Detection measures: ROC, pAUC, TPR at a fixed FPR, objects caught and chance

Solution to Exercise 33.1. $P = 2$ targets (0.6, 0.3), $N = 2$ background pixels (0.5, 0.4). Sweep the threshold down:

threshold	TP	FP	TPR	FPR
above 0.6	0	0	0	0
0.6 (T)	1	0	1/2	0
0.5 (B)	1	1	1/2	1/2
0.4 (B)	1	2	1/2	1
0.3 (T)	2	2	1	1

ROC points: $(0,0), (0,\frac{1}{2}), (\frac{1}{2},\frac{1}{2}), (1,\frac{1}{2}), (1,1)$.

Trapezoids:

$$\begin{aligned}(0,0) \to (0,\tfrac{1}{2}) &: \text{width } 0 \to 0\\ (0,\tfrac{1}{2}) \to (\tfrac{1}{2},\tfrac{1}{2}) &: \tfrac{1}{2} \times \tfrac{\frac{1}{2}+\frac{1}{2}}{2} = \tfrac{1}{2} \times \tfrac{1}{2} = \tfrac{1}{4}\\ (\tfrac{1}{2},\tfrac{1}{2}) \to (1,\tfrac{1}{2}) &: \tfrac{1}{2} \times \tfrac{1}{2} = \tfrac{1}{4}\\ (1,\tfrac{1}{2}) \to (1,1) &: \text{width } 0 \to 0\\ \text{AUC} &= \tfrac{1}{4} + \tfrac{1}{4} = \tfrac{1}{2}.\end{aligned}$$

Pairs ($2 \times 2 = 4$): target 0.6 beats 0.5 and 0.4 (2 wins); target 0.3 beats neither (0 wins). AUC $= 2/4 = 0.5$.

$\text{pAUC}_{0.01}$: the curve is at TPR $\frac{1}{2}$ from FPR 0 to $\frac{1}{2}$, so the area to 0.01 is $0.01 \times \frac{1}{2} = 0.005$, and $0.005/0.01 = 0.5$. TPR@0.001: the points with FPR ≤ 0.001 are $(0,0)$ and $(0,\frac{1}{2})$, so the best TPR is $\frac{1}{2}$.

AUC (0.5) says "random"; pAUC (0.5, a hundred times its chance of 0.005) says the top of the list is excellent. Half the targets are at the very top and half at the very bottom: AUC averages the two, the top-of-list measures see only the first.

Solution to Exercise 33.2. Chance: the diagonal gives $\frac{1}{m}\int_0^m f\,df = \frac{1}{m}\cdot\frac{m^2}{2} = \frac{m}{2} = \frac{0.05}{2} = 0.025$.

McClish for $A = 0.02$:

$$\begin{aligned}A_{\min} &= m^2/2 = 0.05^2/2 = 0.0025/2 = 0.00125\\ A_{\max} &= m = 0.05\\ \frac{A - A_{\min}}{A_{\max} - A_{\min}} &= \frac{0.02 - 0.00125}{0.05 - 0.00125} = \frac{0.01875}{0.04875} = 0.3846\\ \text{pAUC}^{\text{McClish}} &= \tfrac{1}{2}(1 + 0.3846) = \tfrac{1}{2} \times 1.3846 = 0.6923.\end{aligned}$$

On the repository's scale the same area is $0.02/0.05 = 0.40$ (chance 0.025).

Solution to Exercise 33.3. Cells (0,0), (1,0), (1,1). Their 4-neighbours not in the object:

- from (0,0): $(-1,0)$, $(0,-1)$, $(0,1)$ [$(1,0)$ is in the object];
- from (1,0): $(2,0)$, $(1,-1)$ [$(0,0)$ and $(1,1)$ are in the object];
- from (1,1): $(2,1)$, $(1,2)$ [$(0,1)$ already counted; $(1,0)$ in the object].

That is $3 + 2 + 2 = 7$ neighbours, so $m = 3 + 7 = 10$.

$$0.99^{10} = 0.99^5 \times 0.99^5 = 0.95099 \times 0.95099 = 0.90438$$
$$P(\text{caught}) = 1 - 0.90438 = 0.0956.$$

About 9.6%; the straight three-in-a-row (m = 11) gets 10.5% because a straight line exposes one more neighbour.

Solution to Exercise 33.4.

$$n_{\text{pts}} = \min(256, \max(20, 600{,}000//10{,}000 + 64)) = \min(256, \max(20, 60 + 64)) = 124$$
$$\text{spacing} = 100/(124 - 1) = 100/123 = 0.813 \text{ percentile.}$$

The sampled thresholds below the maximum flag the top 0.813%, then 1.626%, … So only one sampled point lies between 0 and 1% of the pixels, and the first-1% area is a single straight-line guess.

Solution to Exercise 33.5.

$$\text{chance, 1 pixel} = 1 - 0.99^5 = 0.049$$
$$\text{chance, domino} = 1 - 0.99^8 = 0.077$$
$$\text{mean} = (0.049 + 0.077)/2 = 0.126/2 = 0.063$$
$$12\%/6.3\% = 0.12/0.063 = 1.9.$$

The model catches 1.9 times as many objects as random scores would.

Solution to Exercise 33.6. Background pixels in descending order: 0.5 and 0.4. When the walk reaches 0.5, one target (0.6) has been passed: TPR $\frac{1}{2}$. When it reaches 0.4, still one target: TPR $\frac{1}{2}$. Each rectangle has width $1/N = \frac{1}{2}$:

$$\text{AUC} = \tfrac{1}{2} \times \tfrac{1}{2} + \tfrac{1}{2} \times \tfrac{1}{2} = \tfrac{1}{4} + \tfrac{1}{4} = \tfrac{1}{2}.$$

The same 0.5 as the trapezoids and the pair count.

Chapter 34: Benchmarks: the public sets and the implant bench (contrast z, foreignness, PSF)

Solution to Exercise 34.1. The ring is everything within 3 steps of the object minus everything within 1 step. Put the object in row 0, columns 0 and 1, and count the cells within 3 steps of either object pixel, row by row. In row Δy a cell is within 3 steps when its column is within $3 - |\Delta y|$ of column 0 or of column 1:

$$\begin{aligned}
\text{row } \pm 3: &\ \text{columns } 0 \text{ to } 1 \to 2 \text{ cells each} \\
\text{row } \pm 2: &\ \text{columns } -1 \text{ to } 2 \to 4 \text{ cells each} \\
\text{row } \pm 1: &\ \text{columns } -2 \text{ to } 3 \to 6 \text{ cells each} \\
\text{row } 0: &\ \text{columns } -3 \text{ to } 4 \to 8 \text{ cells} \\
\text{within 3 steps} &= 2 + 4 + 6 + 8 + 6 + 4 + 2 = 32.
\end{aligned}$$

Within 1 step: the 2 object pixels plus their 6 neighbours (1 left, 1 right, 2 above, 2 below) = 8. Ring = $32 - 8 = 24$. The book's script confirms 24.

Solution to Exercise 34.2.

$$
\begin{aligned}
|0.36 - 0.30| = 0.06, \quad |0.27 - 0.30| &= 0.03 \\
\text{mean } |o - r| &= (0.06 + 0.03)/2 = 0.09/2 = 0.045 \\
\bar{\sigma} &= (0.06 + 0.06)/2 = 0.06 \\
z &= 0.045/0.06 = 0.75.
\end{aligned}
$$

$0.502 \le 0.75 < 0.934$: medium.

Solution to Exercise 34.3. $o_0 - r = 0.43 - 0.40 = 0.03$, so $z(s) = |0.03 - 0.20\,s|/0.10$.

$$
\begin{aligned}
z(0) = 0.03/0.10 = 0.3 < 1.5 & \qquad \text{solvable} \\
\text{minimum} : 0.03 - 0.20\,s = 0 &\Rightarrow s = 0.15 \\
\text{rising branch} : 0.03 - 0.20\,s &= -1.5 \times 0.10 = -0.15 \\
0.20\,s = 0.18 &\Rightarrow s = 0.9 \\
\text{check} : |0.03 - 0.18|/0.10 &= 0.15/0.10 = 1.5.
\end{aligned}
$$

Simple formula: $s = 1.5 \times 0.10/0.20 = 0.75$; realised $z = |0.03 - 0.15|/0.10 = 0.12/0.10 = 1.2$, still easy but short of the target. `solve_strength` returns $s = 0.9$.

Solution to Exercise 34.4. $\mathbf{d}_{\text{in}} = (3, 0, 0)$, $A = 9$; $\mathbf{d}_{\text{out}} = (0, 0, 3)$, $C = 9$.

$$
\begin{aligned}
\beta^2 = \frac{0.1}{0.9} \times \frac{9}{9} = 0.1111, \quad \beta &= 0.3333 \\
\mathbf{d}' = (3, 0, 0) + 0.3333 \times (0, 0, 3) &= (3, 0, 1) \\
\phi(\mathbf{d}') = \frac{1^2}{3^2 + 1^2} = \frac{1}{10} &= 0.1.
\end{aligned}
$$

Solution to Exercise 34.5. With $\sigma = 1$, $2\sigma^2 = 2$, so $w(\Delta y, \Delta x) = \exp(-(\Delta y^2 + \Delta x^2)/2)$.

$$
\begin{aligned}
w(2, 0) &= e^{-4/2} = e^{-2} = 0.1353 \\
w(1, 1) &= e^{-2/2} = e^{-1} = 0.3679 \\
w(3, 0) &= e^{-9/2} = e^{-4.5} = 0.0111 \\
w(2, 1) &= e^{-5/2} = e^{-2.5} = 0.0821 \\
\text{ring sum} &= 4 \times 0.1353 + 4 \times 0.3679 + 4 \times 0.0111 + 8 \times 0.0821 \\
&= 0.5413 + 1.4715 + 0.0444 + 0.6567 = 2.7139 \\
\bar{w}_{\text{ring}} &= 2.7139/20 = 0.1357 \\
w_{\text{halo}} &= w(1, 0) = e^{-1/2} = 0.6065 \\
\text{halo ratio} &= \frac{0.6065 - 0.1357}{1 - 0.1357} = \frac{0.4708}{0.8643} = 0.545.
\end{aligned}
$$

The blur is sharper than the build's $\sigma = 1.4$: less light reaches the halo (0.6065 against 0.7748), so the flat-ground halo ratio falls from 0.667 to 0.545, below the real median of 0.643. The script gives 0.1357, 0.6065 and 0.5448.

Solution to Exercise 34.6.

$$
\begin{aligned}
\text{realised} &= (0.00 + 0.06,\ 0.02 + 0.00) = (0.06, 0.02) \\
\|\cdot\|^2 &= 0.0036 + 0.0004 = 0.0040, \quad \text{outside } 0.0004 \\
\phi &= 0.0004/0.0040 = 0.10.
\end{aligned}
$$

The signal alone, $(0.06, 0)$, lies wholly inside: foreignness 0. Matching the signal would report 0 while the evaluator sees 0.10, fourteen times the real median of 0.0071.

Chapter 35: Running an evaluation, and what a mask costs

Solution to Exercise 35.1.

$$\begin{aligned} \text{score maps per scene} &= 3 \times 3 + 1 = 9 + 1 = 10 \\ \text{score maps in all} &= 5 \times 10 = 50 \\ \text{reconstructions} &= 5 \text{ scenes} \times 3 \text{ models} = 15. \end{aligned}$$

A second run after adding a measure computes no score map and no reconstruction: all 50 maps come from the cache, because the measure is not part of any cache key.

Solution to Exercise 35.2. After excluding water: glint $\to 0$, the 0.1 water pixel $\to 0$; A stays 0.9 and B stays 0.6 (on land).

$$\begin{aligned} \text{A beats } 0, 0.5, 0.2, 0 &\to 4 \text{ of } 4 \\ \text{B beats } 0, 0.5, 0.2, 0 &\to 4 \text{ of } 4 \\ \text{AUC} &= (4 + 4)/8 = 1.0. \end{aligned}$$

The top pixel is A. With no target inside the mask, exclusion can only help: the fall in stage 3 comes from targets inside it.

Solution to Exercise 35.3.

$$\begin{aligned} \text{scan: } 0.07 < 0.04\text{? No} &\to \text{land} \\ \text{NDVI} &= (0.02 - 0.05)/(0.02 + 0.05) = -0.03/0.07 = -0.43 \\ \text{builder: NIR } 0.02 < 0.08 \text{ and NDVI} < 0 &\to \text{water}. \end{aligned}$$

Solution to Exercise 35.4.

$$\begin{aligned} \text{plain} &: 101 + 103 + 48 + 161 = 413, \quad 413/909 = 0.454 = 45.4\% \\ \text{cloud only} &: 101 + 103 + 54 + 161 = 419, \quad 419/909 = 0.461 = 46.1\% \\ \text{water + cloud} &: 98 + 107 + 53 + 161 = 419, \quad 419/909 = 46.1\%. \end{aligned}$$

Same total, different objects: compared with cloud only, water exclusion loses 3 on set1 ($101 \to 98$) and 1 on set3 ($54 \to 53$), and gains 4 on set2 ($103 \to 107$).

Solution to Exercise 35.5. The largest finite float16 is 65,504, so 70,000 becomes infinity, which would sit at the top of every ranking computed from the cache. The runner checks `np.abs(s).max()<6e4` and stores such a map as float32 instead. Uncompressed sizes: float16, $1{,}000{,}000 \times 2 = 2{,}000{,}000$ bytes (2 MB); float32, $1{,}000{,}000 \times 4 = 4{,}000{,}000$ bytes (4 MB).

Solution to Exercise 35.6.

1. Scope: is anything the product must find ever on snow (for example vehicles or structures in snowfields)?
2. Price: how many labelled objects lie wholly inside the snow mask on the real bench? Count them by hand.
3. Mask quality: where does the mask fire wrongly (bright rock, cloud)? Check against a true-colour view.
4. Measure: judge on pAUC, TPR at 0.1% and objects caught, with ROC-AUC reported beside them, knowing that ties at 0 lower it by construction.

Chapter 36: How small a difference can we see: the paired object bootstrap

Solution to Exercise 36.1.

$$\begin{aligned} \mathrm{FPR}(d) &= 5/1{,}000 = 0.005 \\ c(d) &= (0.01 - 0.005)/0.01 = 0.005/0.01 = 0.5. \end{aligned}$$

Object 3's parts become $(0.5, 1)$, and

$$\mathrm{pAUC}_A = \frac{1.2 + 1.0 + 0.5}{2 + 1 + 1} = \frac{2.7}{4} = 0.675.$$

Solution to Exercise 36.2. The data $x = (1, 3, 8)$ have mean $12/3 = 4$. The four draws:

$$\begin{aligned} (1,1,2) &\to (1,1,3), && \text{mean } 5/3 = 1.667 \\ (3,2,3) &\to (8,3,8), && \text{mean } 19/3 = 6.333 \\ (2,2,1) &\to (3,3,1), && \text{mean } 7/3 = 2.333 \\ (3,3,3) &\to (8,8,8), && \text{mean } 24/3 = 8.000 \end{aligned}$$

Mean of the four: $(1.667 + 6.333 + 2.333 + 8.000)/4 = 18.333/4 = 4.583$.

Sorted: $1.667, 2.333, 6.333, 8.000$ at positions 0–3.

$$\begin{aligned} \text{2.5th: position } 3 \times 0.025 = 0.075: &\quad 1.667 + 0.075 \times (2.333 - 1.667) = 1.667 + 0.050 = 1.717 \\ \text{97.5th: position } 3 \times 0.975 = 2.925: &\quad 6.333 + 0.925 \times (8.000 - 6.333) = 6.333 + 1.542 = 7.875 \end{aligned}$$

Interval $[1.717, 7.875]$ - very wide, as four draws of three numbers must be.

Solution to Exercise 36.3. No. The interval $[-0.001, +0.013]$ contains zero, so under the rule it is a tie whatever the point value says. Half-width $(0.013 - (-0.001))/2 = 0.014/2 = 0.007$.

Solution to Exercise 36.4.

$$\begin{aligned} \mu_A &= (0.8 + 0.3 + 0.5 + 0.4)/4 = 2.0/4 = 0.5 \\ A - \mu_A &= (0.3, -0.2, 0, -0.1) \\ \mathrm{Var}\, A &= (0.09 + 0.04 + 0 + 0.01)/4 = 0.14/4 = 0.035 \\ \mu_B &= (0.9 + 0.3 + 0.6 + 0.6)/4 = 2.4/4 = 0.6 \\ B - \mu_B &= (0.3, -0.3, 0, 0) \\ \mathrm{Var}\, B &= (0.09 + 0.09 + 0 + 0)/4 = 0.18/4 = 0.045 \\ \mathrm{Cov}(A, B) &= (0.3 \times 0.3 + (-0.2)(-0.3) + 0 + 0)/4 \\ &= (0.09 + 0.06)/4 = 0.15/4 = 0.0375 \\ \mathrm{Var}(A - B) &= 0.035 + 0.045 - 2 \times 0.0375 = 0.080 - 0.075 = 0.005. \end{aligned}$$

Directly: $A - B = (-0.1, 0, -0.1, -0.2)$, mean $-0.4/4 = -0.1$, deviations $(0, 0.1, 0, -0.1)$, variance $(0 + 0.01 + 0 + 0.01)/4 = 0.02/4 = 0.005$. Agreed; unpaired would be 0.080, sixteen times larger.

Solution to Exercise 36.5.

$$\begin{aligned} \text{pixels} &= 909 \times 2 = 1{,}818 \\ \text{design effect} &= 1 + (2 - 1) \times 0.9 = 1.9 \\ N_{\text{eff}} &= 1{,}818/1.9 = 956.8 \\ \text{understatement factor} &= \sqrt{1.9} = 1.378. \end{aligned}$$

The 1,818 pixels carry about as much evidence as 957 independent ones; a pixel bootstrap's standard error would be about 1.38 times too small.

Solution to Exercise 36.6.

$$\begin{aligned} A &= (1.2 + 1.0 + 0.0)/(2 + 1 + 1) = 2.2/4 = 0.55 \\ B &= (1.0 + 0.9 + 0.2)/(2 + 1 + 1) = 2.1/4 = 0.525 \\ B - A &= -0.025. \end{aligned}$$

Drawing every object exactly once reproduces the original bench, so these are the point values. `paired_bootstrap` computes the point values exactly this way, with `np.arange` as the "draw".

Solution to Exercise 36.7.

$$\begin{aligned} &2.5\text{th} : (2{,}000 - 1) \times 0.025 = 1{,}999 \times 0.025 = 49.975 \\ &97.5\text{th} : 1{,}999 \times 0.975 = 1{,}949.025. \end{aligned}$$

Position 49.975 lies between 0-based positions 49 and 50, the 50th and 51st smallest draws (counting from 1), taking 97.5% of the step from the 50th to the 51st. Position 1,949.025 lies between 0-based 1,949 and 1,950, the 1,950th and 1,951st smallest, taking 2.5% of that step.

Chapter 37: The sorting bug that rewrote the scoreboard

Solution to Exercise 37.1. *Correct value at 650 nm.* The neighbours are $(600, 0.10)$ and $(700, 0.20)$.

$$\begin{aligned} t &= \frac{650 - 600}{700 - 600} = \frac{50}{100} = 0.5 \\ y &= 0.10 + 0.5 \times (0.20 - 0.10) \\ &= 0.10 + 0.05 \\ &= 0.15 \end{aligned}$$

Listed 800, 700, 600. The last listed sample is 600 nm with value 0.10; the first is 800 nm with value 0.30. Check 1: is $650 > 600$? Yes, so the last listed value is returned: 0.10.

At 590 nm. Check 1: is $590 > 600$? No. Check 2: is $590 < 800$? Yes, so the first listed value is returned: 0.30. The correct answer is the held end value below the smallest sample, 0.10. The descending list gives the value at the *other* end of the spectrum.

Solution to Exercise 37.2. *Correct row.* 620 nm lies between 600 and 700 nm:

$$t = \frac{620 - 600}{700 - 600} = \frac{20}{100} = 0.2.$$

By $y = (1 - t)\, y_{600} + t\, y_{700}$, the 700 nm band gets weight $t = 0.2$ and the 600 nm band $1 - t = 0.8$; 500 nm gets 0. Row (listed order 700, 600, 500): $(0.2, 0.8, 0.0)$. Applied to $(0.40, 0.30, 0.10)$:

$$\begin{aligned} &0.2 \times 0.40 + 0.8 \times 0.30 + 0 \times 0.10 \\ &= 0.08 + 0.24 + 0 \\ &= 0.32 \end{aligned}$$

Unsorted row. $620 > 500$, the last listed wavelength, so every column returns the last entry of its unit vector: $(0, 0, 1)$. Applied to the pixel: $0 \times 0.40 + 0 \times 0.30 + 1 \times 0.10 = 0.10$.

Solution to Exercise 37.3. *Sort.* `wl` $= (600, 800, 700)$. The smallest, 600, is at position 0; next, 700, at position 2; the largest, 800, at position 1. So `argsort` $= (0, 2, 1)$ and the sorted list is $(600, 700, 800)$.

Columns on the sorted list. Grid 650 lies halfway between 600 and 700 ($t = 0.5$); grid 750 halfway between 700 and 800 ($t = 0.5$).

	sorted 0 (600)	sorted 1 (700)	sorted 2 (800)
650 nm	0.5	0.5	0
750 nm	0	0.5	0.5

Write back. Sorted column i goes to original column `order`$[i]$: sorted 0 → original 0 (600), sorted 1 → original 2 (700), sorted 2 → original 1 (800). In the listed order (600, 800, 700):

$$\mathbf{M} = \begin{pmatrix} 0.5 & 0 & 0.5 \\ 0 & 0.5 & 0.5 \end{pmatrix}.$$

Apply to $(1, 3, 2)$.

$$\begin{aligned} \text{row 1:} \quad & 0.5 \times 1 + 0 \times 3 + 0.5 \times 2 = 0.5 + 0 + 1.0 = 1.5 \\ \text{row 2:} \quad & 0 \times 1 + 0.5 \times 3 + 0.5 \times 2 = 0 + 1.5 + 1.0 = 2.5 \end{aligned}$$

Check against the function itself: the pairs are 600 → 1, 700 → 2, 800 → 3, a straight line, so 650 gives 1.5 and 750 gives 2.5. The sort recovered the right answer from an unsorted list.

Solution to Exercise 37.4. *bench-v2.1.* With $p = 0.34$, $n = 2{,}817$:

$$\begin{aligned} p(1-p) &= 0.34 \times 0.66 = 0.2244 \\ 0.2244/2{,}817 &= 0.0000797 \\ \sqrt{0.0000797} &= 0.0089 \quad (0.89 \text{ points}) \\ 35.1 - 33.1 &= 2.0 \text{ points} \\ 2.0/0.89 &= 2.2 \text{ standard errors.} \end{aligned}$$

Real bench. With $p = 0.30$, $n = 909$:

$$\begin{aligned} p(1-p) &= 0.30 \times 0.70 = 0.21 \\ 0.21/909 &= 0.000231 \\ \sqrt{0.000231} &= 0.0152 \quad (1.52 \text{ points}) \\ 44.2 - 15.8 &= 28.4 \text{ points} \\ 28.4/1.52 &= 18.7 \text{ standard errors.} \end{aligned}$$

The bug's effect is about eight times larger, in standard errors, than the largest model-to-model step on bench-v2.1. A gap of nearly 19 standard errors between two readings of one model on the same scenes cannot be noise; it can only be a difference in the data path.

Solution to Exercise 37.5. The tests fed ascending wavelength lists, such as $[500, 600, 700]$. On an ascending list the precondition of `np.interp` holds, so the sorted and unsorted code compute the same matrix and every test passes. The bug only appears on real PRISMA files, which list their bands descending.

An assertion that makes the old code fail loudly, placed before the `np.interp` calls:

```
assert np.all(np.diff(wl) > 0), "np.interp needs ascending wavelengths; sort wl first
    "
```

`np.diff(wl)` is the list of differences between neighbouring entries; all of them are positive exactly when the list is strictly increasing. On $(700, 600, 500)$ the differences are $(-100, -100)$ and the assertion fails. (The repository chose to sort instead of asserting, which also makes the function correct for any order; `test_descending_band_lists` pins that behaviour.)

Chapter 38: FT01 to FT05: what each fine-tuning run taught

Solution to Exercise 38.1. Same four layers, with width 64 and stage channels 16, 32, 80, 128.

1. Projections: weights $64 \times (16 + 32 + 80 + 128) = 64 \times 256 = 16{,}384$; biases $4 \times 64 = 256$; total $16{,}384 + 256 = 16{,}640$.
2. Fuse, 1×1 from $4 \times 64 = 256$ channels to 64: $64 \times 256 + 64 = 16{,}384 + 64 = 16{,}448$.
3. Refine, 3×3, 64 to 64: $64 \times 64 \times 9 + 64 = 36{,}864 + 64 = 36{,}928$.
4. Output, 3×3, 64 to $5 \times 16 = 80$: $80 \times 64 \times 9 + 80 = 46{,}080 + 80 = 46{,}160$.

Sum: $16{,}640 + 16{,}448 = 33{,}088$; $33{,}088 + 36{,}928 = 70{,}016$; $70{,}016 + 46{,}160 = 116{,}176$ parameters.

Solution to Exercise 38.2. *Edge strength.* Differences to the neighbours inside the patch:

$$\begin{aligned} |0.30 - 0.30| &= 0.00 \quad \text{(up)} \\ |0.30 - 0.31| &= 0.01 \quad \text{(down)} \\ |0.30 - 0.12| &= 0.18 \quad \text{(left)} \end{aligned}$$

Maximum: $e = 0.18$.

Central differences. With the missing right neighbour taken equal to the pixel (0.30):

$$\begin{aligned} g_x &= (0.30 - 0.12)/2 = 0.18/2 = 0.09 \\ g_y &= (0.31 - 0.30)/2 = 0.01/2 = 0.005 \\ |\nabla f| &= \sqrt{0.09^2 + 0.005^2} = \sqrt{0.0081 + 0.000025} = \sqrt{0.008125} = 0.0901 \end{aligned}$$

Both measures call the pixel next to the boundary an edge; the one-sided maximum reports the full step (0.18), the central difference half of it spread over two pixels.

Solution to Exercise 38.3. Input $0.10, 0.10, 0.50, 0.50$; reconstruction $0.10, 0.10, 0.45, 0.50$.

Value L1. Differences $0, 0, 0.05, 0$; sum 0.05; mean $0.05/4 = 0.0125$.

Gradient L1. Input differences $0, 0.40, 0$. Reconstruction differences $0.10 - 0.10 = 0$, $0.45 - 0.10 = 0.35$, $0.50 - 0.45 = 0.05$. Mismatches $|0 - 0| = 0$, $|0.35 - 0.40| = 0.05$, $|0.05 - 0| = 0.05$; sum 0.10; mean $0.10/3 = 0.033$.

A sharper reconstruction (most of the step in one pixel) cuts the gradient loss from 0.133 to 0.033.

Solution to Exercise 38.4. Hidden near pixels: $n = 0.65 \times 32 = 20.8$. With weight 5:

$$\begin{aligned} w\,n &= 5 \times 20.8 = 104 \\ (N - n) + w\,n &= 10{,}650 - 20.8 + 104 = 10{,}733.2 \\ \text{share} &= 104/10{,}733.2 = 0.0097 \quad \text{(about 1.0\%)} \end{aligned}$$

Weight for a 2% share. Set $\frac{wn}{(N-n)+wn} = 0.02$. Multiply out: $wn = 0.02(N - n) + 0.02\,wn$. Collect: $0.98\,wn = 0.02(N - n)$. Divide:

$$w = \frac{0.02 \times (10{,}650 - 20.8)}{0.98 \times 20.8} = \frac{0.02 \times 10{,}629.2}{20.384} = \frac{212.58}{20.384} = 10.4.$$

Solution to Exercise 38.5.

- 199: $199/255 = 0.780$, between 0.1 and 0.9: unknown (no loss).
- 231: $231/255 = 0.906 > 0.9$: known, target 1 - but its NIR is $0.18 > 0.15$, so the guard sets the target to 0: a known not-water.
- 24: $24/255 = 0.094 < 0.1$: known, target 0.
- 26: $26/255 = 0.102$, not below 0.1 and not above 0.9: unknown.

In bytes: "yes" needs 230 or more, "no" 25 or less, so 24 is known and 26 is not.

Solution to Exercise 38.6.

$$\begin{aligned} \text{unweighted:} \quad & (0.2 + 0.2 + 0.4 + 0.6)/4 = 1.4/4 = 0.35 \\ \text{weighted:} \quad & (0.2 + 0.2 + 0.4 + 3 \times 0.6)/(1 + 1 + 1 + 3) = (0.8 + 1.8)/6 = 2.6/6 = 0.433 \end{aligned}$$

Shadow's share of the gradient is $w_{\text{shadow}}/\sum w$: $1/4 = 0.25$ unweighted, $3/6 = 0.50$ weighted.

Solution to Exercise 38.7. From (38.8)'s derivation the gap after n steps is 0.999^n times the starting gap. Closing 90% leaves 10%: $0.999^n = 0.1$. Take natural logarithms: $n \ln 0.999 = \ln 0.1$. Since $\ln 0.999 = -0.0010005$ and $\ln 0.1 = -2.3026$,

$$n = \frac{-2.3026}{-0.0010005} = 2{,}301 \text{ steps.}$$

Chapter 39: Shipping Indradhanu V2

Solution to Exercise 39.1. Compare first characters: `a` is 97, `h` is 104. Since $97 < 104$, `a_spectral_stem` sorts first and `hyperspectral_segformer_mae` last. The dictionary keeps the last value written for "indradhanu", so it routes to v0.2.0. Directory names deciding routing is exactly why the codenames must differ: the same two models would route differently depending only on what a folder happened to be called.

Solution to Exercise 39.2.

- 1400 nm: $|1400 - 1385| = 15$, $|1460 - 1400| = 60$; minimum 15, and $15 \leq 15$: kept.
- 1420 nm: $|1420 - 1385| = 35$, $|1460 - 1420| = 40$; minimum $35 > 15$: dropped.
- 1445 nm: $|1445 - 1385| = 60$, $|1460 - 1445| = 15$; minimum 15: kept.

The rule keeps a 15 nm fringe on each side of every recorded wavelength; a gap wider than 30 nm in the record leaves bands in its middle unread.

Solution to Exercise 39.3. Every band must pass (39.1); the tightest tolerance is at the smallest wavelength.

$$\begin{aligned} \text{at 460 nm:} \quad & 10^{-8} + 10^{-5} \times 460 = 0.0046\,\text{nm} \\ \text{at 2450 nm:} \quad & 10^{-8} + 10^{-5} \times 2450 = 0.0245\,\text{nm} \end{aligned}$$

A shift of 0.003 nm is below 0.0046 at 460 nm and below every larger tolerance, so `np.allclose` is true: the list counts as the grid (and a native-band model would be refused). A shift of 0.03 nm exceeds 0.0046 at 460 nm (and even 0.0245 at 2450 nm), so `np.allclose` is false: the list counts as native.

Solution to Exercise 39.4.

$$100 \times 4/26 = 400/26 = 15.3846\ldots$$
$$\text{rounded to 3 places} = 15.385$$

For $n = 200$: $n - T = 200 - 128 = 72$. The next multiple of 64 at or above 72 is 128, so the pad is $128 - 72 = 56$ (the code's $(-72) \bmod 64 = 56$). The padded side is $200 + 56 = 256$, and windows start at $0, 64, 128$ (the last start is $256 - 128 = 128$).

Solution to Exercise 39.5. Seven residuals: four land at ± 0.02 and three cloud at ± 0.06 magnitude.

$$\begin{aligned}
\text{squares:} &\quad 4 \times 0.0004 + 3 \times 0.0036 = 0.0016 + 0.0108 = 0.0124 \\
\text{variance:} &\quad 0.0124/7 = 0.001771 \\
\text{spread:} &\quad \sqrt{0.001771} = 0.0421 \\
\text{target:} &\quad 0.08/0.0421 = 1.90 \text{ spreads}
\end{aligned}$$

More cloud in the fit widens the spread and lowers the target's score (from 2.09 to 1.90). Which pixels enter the fit changes every score, which is why the product must use the order the benchmark measured.

Solution to Exercise 39.6. The action fails at run time. The default band filter resamples to the 165-band grid, so V2's grid check raises "reads native bands: run band_filter_apply with use_common_wavelength_grid=false". Instead, run two scoring actions: v0.2.0 on a default band filter, and V2 on a second band filter with `use_common_wavelength_grid=false`.

Chapter 40: Improving the score on a fixed model, and gated scene adaptation

Solution to Exercise 40.1. *Invert.* $ad - bc = 1 \times 1 - 0.5 \times 0.5 = 0.75$, so

$$\boldsymbol{\Sigma}^{-1} = \frac{1}{0.75}\begin{pmatrix} 1 & -0.5 \\ -0.5 & 1 \end{pmatrix} = \begin{pmatrix} 1.333 & -0.667 \\ -0.667 & 1.333 \end{pmatrix}.$$

Apply to $\mathbf{t} = (1, 1)$. $\boldsymbol{\Sigma}^{-1}\mathbf{t} = (1.333 - 0.667,\ -0.667 + 1.333) = (0.667, 0.667)$; $\mathbf{t}^\top\boldsymbol{\Sigma}^{-1}\mathbf{t} = 0.667 + 0.667 = 1.333$.

Filter and SNR. $\mathbf{w}^\star = (0.667, 0.667)/1.333 = (0.5, 0.5)$; SNR $= \sqrt{1.333} = 1.155$.

Naive $\mathbf{w} = \mathbf{t}$. $\mathbf{w}^\top\mathbf{t} = 1 + 1 = 2$; $\mathbf{w}^\top\boldsymbol{\Sigma}\mathbf{w} = 1 + 0.5 + 0.5 + 1 = 3$; SNR $= 2/\sqrt{3} = 1.155$. The same. Here $\mathbf{t}$ is an eigenvector of $\boldsymbol{\Sigma}$ ($\boldsymbol{\Sigma}\mathbf{t} = (1.5, 1.5) = 1.5\,\mathbf{t}$), so $\boldsymbol{\Sigma}^{-1}\mathbf{t} = \mathbf{t}/1.5$ points the same way as $\mathbf{t}$: the matched filter is the naive filter, rescaled. Matching only helps when the noise's correlations point away from the signal, as in worked example 40.1.

Solution to Exercise 40.2. $b_{11} = 0.25 \times 0 + 0.5 \times 5.2 + 0.25 \times 0 = 2.6$, score $2.6^2 = 6.76$. The blurred ranking becomes 18.06 (pixel 7), 16 (6), 7.56 (8), 6.76 (11), 6.25 (5): the target is fourth and pixel 5 drops to fifth. Both objects are now caught - the big one through pixels 6–8 - so objects caught returns to 2. Four tenths of a unit of residual decided whether the one-pixel target survived blurring; a filter that puts a small target this close to the cut-off loses many of them on a real scene.

Solution to Exercise 40.3. New PCA-RX scores: P1 11, P2 1, P3 2, P4 10, P5 3. Sorted: P2 (1), P3 (2), P5 (3), P4 (10), P1 (11), so the PCA-RX ranks are P2 1/5, P3 2/5, P5 3/5, P4

4/5, P1 5/5. RM ranks are unchanged: P1 4/5, P4 2/5, P2 5/5.

$$\begin{aligned}
\text{P1:}&\quad \text{mean } (4/5+5/5)/2 = 9/10 = 0.9, \quad \text{max } 1.0\\
\text{P4:}&\quad \text{mean } (2/5+4/5)/2 = 6/10 = 0.6, \quad \text{max } 0.8\\
\text{P2:}&\quad \text{mean } (5/5+1/5)/2 = 6/10 = 0.6, \quad \text{max } 1.0
\end{aligned}$$

Rank mean puts P1 first (0.9). Rank max ties P1 with P2, RM's false alarm, at 1.0.

Solution to Exercise 40.4.

$$\begin{aligned}
\mathcal{N}(2;0,1) &= 0.3989 \times e^{-2} = 0.3989 \times 0.1353 = 0.05399\\
\mathcal{N}(2;5,4) &= 0.1995 \times e^{-9/8} = 0.1995 \times 0.3247 = 0.06476\\
\pi_1\mathcal{N}_1 &= 0.8 \times 0.05399 = 0.04319, \qquad \pi_2\mathcal{N}_2 = 0.2 \times 0.06476 = 0.01295\\
r_1 &= 0.04319/(0.04319+0.01295) = 0.04319/0.05614 = 0.769, \qquad r_2 = 0.231
\end{aligned}$$

Squared distances: field $(2-0)^2/1 = 4$; roofs $(2-5)^2/4 = 9/4 = 2.25$. Weighted score $0.769 \times 4 + 0.231 \times 2.25 = 3.077 + 0.519 = 3.60$. (Script: 3.596.)

Solution to Exercise 40.5. Eight built-up pixels: six roofs at ± 2 and two targets at ± 6.

$$\begin{aligned}
\text{sum of squares} &= 6 \times 4 + 2 \times 36 = 24 + 72 = 96\\
\text{variance} &= 96/8 = 12\\
\text{target score} &= 36/12 = 3.0
\end{aligned}$$

Still below the field edges' 3.77: the targets lose. Diluting them with more ordinary roofs helps, but not enough; on set4 one type holds 60% of the target pixels, far more concentrated than this.

Solution to Exercise 40.6. Only $N = 50$ and $N = 200$:

- set1 held out: 0.1393 against 0.1360, picks 50, reports 0.223.
- set2 held out: 0.1663 against 0.1647, picks 50, reports 0.142.
- set3 held out: 0.1857 against 0.1850, picks 50, reports 0.084.
- set4 held out: 0.1497 against 0.1503, picks 200, reports 0.185.

Headline: $(0.223+0.142+0.084+0.185)/4 = 0.634/4 = 0.1585$, a gain of $0.1585-0.1695 = -0.011$. Different candidates, different picks - and still below FT04. The choice of N is unstable; the verdict is not.

Solution to Exercise 40.7. With $a = 1, b = 4$ in (40.11): $1\times4+4\times(1+4)+4 = 4+20+4 = 28$ pixels (SciPy agrees). Growth $28/4 = 7$, so a top 1% made of such lines would gate about $1\% \times 7 = 7\%$ of the scene - between the 2×2 clumps' 6% and isolated pixels' 13%.

Chapter 41: The uncertainty head and the Laplace likelihood

Solution to Exercise 41.1. The loss at each scale:

$$\begin{aligned}
b = 0.25: &\quad 0.5/0.25 + \ln 0.25 = 2.0 + (-1.386) = 0.614\\
b = 0.5: &\quad 0.5/0.5 + \ln 0.5 = 1.0 + (-0.693) = 0.307\\
b = 1.0: &\quad 0.5/1.0 + \ln 1.0 = 0.5 + 0 = 0.500
\end{aligned}$$

The lowest is at $b = 0.5$, the error itself. The derivative $d\ell/db = -|r|/b^2 + 1/b$:

$$\begin{aligned} b = 0.25: &\quad -0.5/0.0625 + 1/0.25 = -8 + 4 = -4 \\ b = 0.5: &\quad -0.5/0.25 + 1/0.5 = -2 + 2 = 0 \\ b = 1.0: &\quad -0.5/1 + 1/1 = -0.5 + 1 = 0.5 \end{aligned}$$

Negative to the left of the minimum, zero at it, positive to the right; and steeper on the overconfident side.

Solution to Exercise 41.2. Ranks (0 = smallest). b sorted: $0.1 < 0.2 < 0.3 < 0.4 < 0.5$, so $u = (2, 0, 4, 1, 3)$. e sorted: $0.05 < 0.25 < 0.30 < 0.40 < 0.45$, so $v = (1, 0, 2, 3, 4)$. Both means are 2.

$$u - \bar{u} = (0, -2, 2, -1, 1), \qquad v - \bar{v} = (-1, -2, 0, 1, 2).$$

Dot product: $(0)(-1) + (-2)(-2) + (2)(0) + (-1)(1) + (1)(2) = 0 + 4 + 0 - 1 + 2 = 5$. Lengths: each is $\sqrt{4 + 1 + 0 + 1 + 4} = \sqrt{10}$, product 10. So $\rho = 5/10 = 0.50$.

Check: $d = u - v = (1, 0, 2, -2, -1)$, $\sum d^2 = 1 + 0 + 4 + 4 + 1 = 10$, $\rho = 1 - \frac{6 \times 10}{5 \times 24} = 1 - \frac{60}{120} = 1 - 0.5 = 0.50$. A head this good would pass FT06's criterion 1 ($\rho \geq 0.5$) with nothing to spare.

Solution to Exercise 41.3. With width 32:

$$\begin{aligned} \text{projections}: &\ 32 \times 32 + 32 = 1{,}056; \quad 64 \times 32 + 32 = 2{,}080 \\ &\ 160 \times 32 + 32 = 5{,}152; \quad 256 \times 32 + 32 = 8{,}224 \\ \text{sum of projections}: &\ 1{,}056 + 2{,}080 = 3{,}136; \ 3{,}136 + 5{,}152 = 8{,}288 \\ &\ 8{,}288 + 8{,}224 = 16{,}512 \\ \text{fuse } (128 \to 32): &\ 128 \times 32 + 32 = 4{,}096 + 32 = 4{,}128 \\ \text{refine}: &\ 32 \times 32 \times 9 + 32 = 9{,}216 + 32 = 9{,}248 \\ \text{sub-pixel}: &\ 32 \times 16 \times 9 + 16 = 4{,}608 + 16 = 4{,}624 \\ \text{total}: &\ 16{,}512 + 4{,}128 = 20{,}640; \ 20{,}640 + 9{,}248 = 29{,}888 \\ &\ 29{,}888 + 4{,}624 = 34{,}512. \end{aligned}$$

Solution to Exercise 41.4. Ratio 1.07: $3.0/1.07 = 2.804$, and $2.804^2 = 7.86$. That is above 6.76, so the target keeps its place in this example, consistent with set1 losing less than set2.

Ratio 1.19 with the square root: the residual is divided by $\sqrt{1.19} = 1.091$, so the squared score is divided by $1.091 \times 1.091 = 1.19$, about 16% lower ($1 - 1/1.19 = 0.16$). With residual/$b$ it was divided by $1.19^2 = 1.416$, about 29% lower.

Solution to Exercise 41.5. One error: $\ell(\sigma) = r^2/(2\sigma^2) + \log\sigma$. Differentiate:

$$\frac{d\ell}{d\sigma} = \frac{r^2}{2} \cdot (-2)\sigma^{-3} + \frac{1}{\sigma} = -\frac{r^2}{\sigma^3} + \frac{1}{\sigma}.$$

Set to zero and multiply by σ^3:

$$0 = -r^2 + \sigma^2 \quad \Rightarrow \quad \sigma^\star = |r|.$$

N errors sharing σ: the average loss is $\frac{1}{2\sigma^2} \cdot \frac{1}{N}\sum r_i^2 + \log\sigma$, so by the same steps $\sigma^{\star 2} = \frac{1}{N}\sum_i r_i^2$: the root-mean-square error. The Laplace gives the *mean absolute* error; the Gaussian gives the *root mean square*. For errors $0.1, 0.3, 0.2, 0.4$: mean absolute $1.0/4 = 0.25$; mean square $(0.01 + 0.09 + 0.04 + 0.16)/4 = 0.30/4 = 0.075$, root 0.274. The RMS is pulled up more by the largest error.

Solution to Exercise 41.6. The target/ring ratio of b grew with training (set2 1.12 to 1.19): the head learned more of the "something odd next door" signal, which predicts errors well, so Spearman rose, and divides targets away, so detection fell. The epoch-1 square-root row is not evidence because the square-root variant was picked after seeing epoch-1 results on the same bench that grades it (selection risk), and its pAUC of 0.176 is still below the 0.180 bar anyway.

Chapter 42: Gated losses, block masking, the decoder bottleneck and FT09

Solution to Exercise 42.1. Quantile level 0.99; position $0.99 \times (200 - 1) = 197.01$, so the cut lies 1% of the way from $x_{(197)}$ to $x_{(198)}$ (counting from 0). The values at or above it are $x_{(198)}$ and $x_{(199)}$: 2 pixels, 1% of 200. With `gate_halo = 1` the kernel is $2 \times 1 + 1 = 3$, so each becomes a 3×3 square of 9 pixels: $9 + 9 = 18$ loss pixels removed at most. Under L1 every loss pixel carries a gradient of the same size, so the share of gradient terms removed is $18/200 = 0.09$, 9%.

Solution to Exercise 42.2. Tokens per side $256/4 = 64$, tokens $64 \times 64 = 4{,}096$. Blocks per side $64/2 = 32$, blocks $32 \times 32 = 1{,}024$. Hidden blocks $1{,}024 \times 0.65 = 665.6 \to 665$; hidden tokens $665 \times 4 = 2{,}660$, a share $2{,}660/4{,}096 = 0.649$.

Blocks: given our block is hidden, the other 1,023 blocks contain $1{,}024 - 665 = 359$ visible ones. Five outside neighbours: $5 \times 359/1{,}023 = 5 \times 0.3509 = 1.755$.

Single tokens: hidden $4{,}096 \times 0.65 = 2{,}662.4 \to 2{,}662$, visible $4{,}096 - 2{,}662 = 1{,}434$. Given ours is hidden, the other 4,095 contain 1,434 visible: $8 \times 1{,}434/4{,}095 = 8 \times 0.3502 = 2.801$. The ratio $1.755/2.801 = 0.63$, as for the 128 px tile.

Solution to Exercise 42.3. $\mathbf{x} \cdot \mathbf{u} = 4 + 1 + 1 = 6$, $\mathbf{u} \cdot \mathbf{u} = 3$, $a^\star = 6/3 = 2$, projection $(2, 2, 2)$. Floor $(4 - 2, 1 - 2, 1 - 2) = (2, -1, -1)$, energy $4 + 1 + 1 = 6$. Residual for $(3, 3, 3)$: $(3-4, 3-1, 3-1) = (-1, 2, 2)$, energy $1+4+4 = 9$. Share $6/9 = 0.67$. Pythagoras: the in-span part is $(3, 3, 3) - (2, 2, 2) = (1, 1, 1)$, energy $1 + 1 + 1 = 3$, and $3 + 6 = 9$. ✓

Solution to Exercise 42.4.

$$\texttt{up_half}\ 1 : 288 \times 160 \times 9 + 160 = 414{,}720 + 160 = 414{,}880$$
$$\texttt{up_half}\ 2 : 160 \times 160 \times 9 + 160 = 230{,}400 + 160 = 230{,}560$$
$$\texttt{up_full}\ 1 : (160 + 32) \times 96 \times 9 + 96 = 192 \times 864 + 96 = 165{,}888 + 96 = 165{,}984$$

Unchanged: projections 132,096, fuse 262,400, `up_full` 2 83,040, head 3,104. Total:

$$\begin{aligned} 132{,}096 + 262{,}400 &= 394{,}496 \\ 394{,}496 + 414{,}880 &= 809{,}376 \\ 809{,}376 + 230{,}560 &= 1{,}039{,}936 \\ 1{,}039{,}936 + 165{,}984 &= 1{,}205{,}920 \\ 1{,}205{,}920 + 83{,}040 &= 1{,}288{,}960 \\ 1{,}288{,}960 + 3{,}104 &= 1{,}292{,}064 \end{aligned}$$

Solution to Exercise 42.5. Per token: $4 \times 4 \times 32 = 512$ numbers in, 64 out, $512/64 = 8$ to 1 (4 numbers per pixel). Stage 1 now delivers $32 \times 32 \times 64 = 65{,}536$; the total is $65{,}536 + 16{,}384 + 10{,}240 + 4{,}096 = 96{,}256$, and $96{,}256/16{,}384 = 5.875$ numbers per pixel, against 3.875 before and 32 to produce.

Solution to Exercise 42.6. Background: $0.139 - 0.110 = 0.029$, $0.029/0.139 = 0.209$ (20.9%). Targets: $0.191 - 0.162 = 0.029$, $0.029/0.191 = 0.152$ (15.2%). Ratio $0.162/0.110 = 1.47$.

FT09b's interval cleared zero by only 0.0002, the bootstrap resamples objects and not scenes or checkpoints, the top-0.1% count fell by 4 objects, and criterion 5 failed as written. Then FT09c, five more epochs of the same training, lost the gain (a resolved -0.0040 against FT09b), showing it was a favourable checkpoint.

Solution to Exercise 42.7. Blocks per side $6/2 = 3$, blocks $3 \times 3 = 9$. In each block h is the same for all 4 tokens and s takes each value twice, so 2 tokens leak per block: $9 \times 2 = 18$ of $6 \times 6 = 36$ tokens, $18/36 = 0.5$. Swapping which pass hides which blocks flips h in every block from 0 to 1 or back; s still takes each value on 2 of the 4 tokens, so still 2 agree and 2 disagree. Only the positions of the leaks move, not their number.

Chapter 43: Indradhanu Heavy: capacity, a fair control and the haze

Solution to Exercise 43.1. Stage 2 has $R = 4$, so the C^2 coefficient is $12 + 16 = 28$.

$$\text{FT04, } C = 64 : 28 \times 64^2 + 56 \times 64 = 28 \times 4{,}096 + 3{,}584 = 114{,}688 + 3{,}584 = 118{,}272$$

$$\text{Heavy, } C = 192 : 28 \times 192^2 + 56 \times 192 = 28 \times 36{,}864 + 10{,}752 = 1{,}032{,}192 + 10{,}752 = 1{,}042{,}944$$

$$\text{ratio} : 1{,}042{,}944/118{,}272 = 8.82$$

Both match the table. Doubling stage 1 ($C = 32 \to 64$): the C^2 term $77{,}824 \times 4 = 311{,}296$, the C term $1{,}792 \times 2 = 3{,}584$, total 314,880; ratio $314{,}880/79{,}616 = 3.95$, a little under $2^2 = 4$.

Solution to Exercise 43.2. Heavy: $96 \times 192 \times 3 \times 3 = 165{,}888$; plus bias 192 gives 166,080; plus LayerNorm $2 \times 192 = 384$ gives 166,464, what the built model reports for `encoder.patch_embeds[1]`. FT04: $32 \times 64 \times 9 = 18{,}432$; $+64 = 18{,}496$; $+128 = 18{,}624$. Ratio $166{,}464/18{,}624 = 8.94$.

Solution to Exercise 43.3. $\cos(20\pi/80) = \cos(\pi/4) = 0.7071$. $(1 + 0.7071)/2 = 1.7071/2 = 0.85355$. $2.99 \times 10^{-4} \times 0.85355 = 2.5521 \times 10^{-4}$. Add 10^{-6}: 2.5621×10^{-4}. The CSV logs 2.56×10^{-4} at epoch 20.

Solution to Exercise 43.4. Every worker needs at least one shard, so at most 16 workers; a 17th would get none. For exactly two shards each with 12 workers you need $12 \times 2 = 24$ shards.

Solution to Exercise 43.5. Old rule: $\max(0.3, 0.8) = 0.8 > 0.5$, so the cloud line assigns class 2, cloud, overriding water. Thick rule: only prob[1] = 0.3 counts, and $0.3 \leq 0.5$, so the cloud line does not fire; the pixel keeps the class from the earlier lines: shadow and snow do not pass, water does (0.6), so class 1, water.

Solution to Exercise 43.6. Before: $1.0 + 0.8 + 0.8 + 0.6 + 0.4 + 0.4 + 0.0 + 0.0 = 4.0$; $4.0/8 = 0.50$. After zeroing the first three: $0.6 + 0.4 + 0.4 + 0 + 0 = 1.4$; $1.4/8 = 0.175$. The pAUC fell by $(0.50 - 0.175)/0.50 = 0.65$, 65%, although only 3 of 8 targets (37.5%) were removed, because the removed targets were the best-ranked ones, carrying 2.6 of the 4.0 credit.

Solution to Exercise 43.7. FT04 had a four-stage curriculum and FTH1 one short run, so a loss could be the recipe's fault: the comparison is confounded. FTB1 gives F001 exactly FTH1's recipe, so FTH1 − FTB1 holds the recipe fixed and isolates the model. It does not separate size from the decoder, the pretraining run or the segmentation-head width; that would need further controls.

Solution to Exercise 43.8. On 2 threads: $(1-0.923)+0.923/2 = 0.077+0.4615 = 0.5385$. On 24: $0.077+0.923/24 = 0.077+0.0385 = 0.1155$. Speed-up $0.5385/0.1155 = 4.66$ (4.67 with p unrounded), against 4.0 at 16. Reasons for doubt: p was fitted to a single measurement, and the law ignores memory bandwidth, which threads share; and 24 threads on a 24-core host leave nothing for the api, the database and the operating system, which is why the compose file chose 16 for headroom.

Solution to Exercise 43.9. The output network is $32 \to 64 \to (D+1)$ on the wavelength encoding. With $D = 32$:

$$\begin{aligned} 32\times 64+64 &= 2{,}048+64 = 2{,}112\\ 64\times 33+33 &= 2{,}112+33 = 2{,}145\\ 2{,}112+2{,}145 &= 4{,}257, \end{aligned}$$

FT04's `spectral_out` in Table 43.3. FT04's segmentation head, width 128, $5\times 16 = 80$ sub-pixel channels:

$$\begin{aligned} \text{projections} &: (32+64+160+256)\times 128+4\times 128 = 512\times 128+512 = 65{,}536+512 = 66{,}048\\ \text{fuse} &: 512\times 128+128 = 65{,}536+128 = 65{,}664\\ \text{refine} &: 128\times 128\times 9+128 = 147{,}456+128 = 147{,}584\\ \text{sub-pixel} &: 128\times 80\times 9+80 = 92{,}160+80 = 92{,}240\\ \text{total} &: 66{,}048+65{,}664 = 131{,}712;\ +147{,}584 = 279{,}296;\ +92{,}240 = 371{,}536, \end{aligned}$$

the head counted in Chapter 39.

Solution to Exercise 43.10. Each pass hides half of the 1,024 valid tokens and keeps 512. Stage 1 then attends among the kept tokens: $512^2 = 262{,}144$ scores per head, 16 times the unmasked $1{,}024\times 16 = 16{,}384$. A batch of 8 tiles gives the stem $8\times 128\times 128 = 8\times 16{,}384 = 131{,}072$ rows, and $131{,}072/32{,}768 = 4$ calls of the attention kernel.

Chapter 44: The spectral namer as built

Solution to Exercise 44.1. Still 0.20. The halo and the target lie within Chebyshev distance 2 of the centre, so they are inside both the 15×15 and the 5×5 boxes and cancel in the difference:

$$\begin{aligned} S_{15}-S_5 &= (216\times 0.20+8\times 0.60+0.90)-(16\times 0.20+8\times 0.60+0.90)\\ &= 216\times 0.20-16\times 0.20 = 200\times 0.20 = 40.0, \qquad \mathbf{r} = 40.0/200 = 0.20. \end{aligned}$$

Solution to Exercise 44.2.

$$P(a\le 0.25) = \frac{\ln(0.25/0.02)}{\ln(1/0.02)} = \frac{\ln 12.5}{\ln 50} = \frac{2.5257}{3.9120} = 0.6456,$$
$$\mathbb{E}[a] = \frac{1-0.02}{\ln 50} = \frac{0.98}{3.9120} = 0.2505.$$

Lowering $a_{\min}$ pushes even more of the training to small fills: from 0.5372 to 0.6456 at or below 0.25, and the mean falls from 0.3171 to 0.2505 [C].

Solution to Exercise 44.3.

$$\begin{aligned} a\mathbf{t} &= (0.5\times 0.30,\ 0.5\times 0.10) = (0.15,\ 0.05),\\ (1-a)\mathbf{b} &= (0.5\times 0.20,\ 0.5\times 0.40) = (0.10,\ 0.20),\\ \text{sum} &= (0.25,\ 0.25),\\ \mathbf{x} &= (1.1\times 0.25,\ 0.9\times 0.25) = (0.275,\ 0.225). \end{aligned}$$

Solution to Exercise 44.4. Metal's only remaining member is e_2, and the logsumexp of one number is that number: $S_{\text{metal}} = 7.0$.

$$e^{7.0} = 1096.63, \quad e^{7.5} = 1808.04, \quad e^{2.0} = 7.39, \quad \text{sum} = 2912.06,$$
$$P(\text{metal}) = 1096.63/2912.06 = 0.3766, \qquad L = -\ln 0.3766 = 0.9766.$$

Fewer class members, less evidence: the loss is higher than the 0.7931 of worked example 44.13.

Solution to Exercise 44.5. The stem (17,825), the BatchNorm (128) and the two scalars (2) do not depend on `hidden` or `embed`.

$$\begin{aligned}
\text{pixel MLP} &= (192 \times 128 + 128) + (128 \times 128 + 128) + (128 \times 128 + 128) \\
&= 24{,}704 + 16{,}512 + 16{,}512 = 57{,}728, \\
\text{heads} &= (128 \times 64 + 64) + 2 \times (128 + 1) = 8{,}256 + 258 = 8{,}514, \\
\text{library MLP} &= (64 \times 128 + 128) + (128 \times 128 + 128) + (128 \times 64 + 64) \\
&= 8{,}320 + 16{,}512 + 8{,}256 = 33{,}088, \\
\text{total} &= 57{,}728 + 8{,}514 + 33{,}088 + 17{,}825 + 128 + 2 = 117{,}285.
\end{aligned}$$

Instantiation gives 117,285 [C]. The stem is then 15% of the model.

Solution to Exercise 44.6. It needs ACE's recall plus 0.10, $0.7724 + 0.10 = 0.8724$ (Table 44.7). At 0.85 it does not pass on PRISMA, even though its false alarms equal ACE's.

Solution to Exercise 44.7.

$$\sum_j \left(q_j - \mathbf{1}[j \in c]\, w_j\right) = \sum_j q_j - \sum_{j \in c} w_j = 1 - 1 = 0,$$

because the q_j are a softmax over all entries and the w_j a softmax over the true class's entries. A step redistributes score; it cannot raise all of them.

Solution to Exercise 44.8. The natural class now has two members of score 5.5:

$$\begin{aligned}
S_{\text{nat}} &= \ln(e^{5.5} + e^{5.5}) = 5.5 + \ln 2 = 5.5 + 0.6931 = 6.1931, \\
\ell &= 6.3133 - 6.1931 = 0.1201 \quad \text{(from unrounded values)}, \\
p &= \sigma(0.1201) = 1/(1 + e^{-0.1201}) = 1/(1 + 0.8868) = 0.5300, \\
L_1 &= -\ln 0.5300 = 0.6349, \qquad p - y = -0.4700.
\end{aligned}$$

Gradients by Equation (44.15): within the man-made side $u = (0.7311, 0.2689)$ as before; within the natural side $v = (0.5, 0.5)$:

$$\begin{aligned}
\text{plastic} &: -0.4700 \times 0.7311 = -0.3436, & \text{metal} &: -0.4700 \times 0.2689 = -0.1264, \\
\text{soil} &: +0.4700 \times 0.5 = +0.2350, & \text{water} &: +0.4700 \times 0.5 = +0.2350.
\end{aligned}$$

They sum to zero; autograd agrees [C]. The natural side gained: $P(M)$ fell from 0.6928 to 0.5300 although no score changed. One more natural entry at the same score adds $\ln 2$ to S_{nat}. This is the count offset of Section 44.7, and the bias b is what can undo it.

Solution to Exercise 44.9. Gaps: $1800 - 1790 = 10$; $1960 - 1800 = 160$, capped to 20; $1970 - 1960 = 10$. Widths:

$$w_{1790} = 10, \quad w_{1800} = (10 + 20)/2 = 15, \quad w_{1960} = (20 + 10)/2 = 15, \quad w_{1970} = 10.$$

Bins: $\lfloor (1790 - 400)/40 \rfloor = \lfloor 34.75 \rfloor = 34$; $\lfloor 1400/40 \rfloor = 35$; $\lfloor 1560/40 \rfloor = 39$; $\lfloor 1570/40 \rfloor = \lfloor 39.25 \rfloor = 39$. The code gives the same [C]. Bins 36 to 38 hold none of these bands and would be masked out of attention.

Solution to Exercise 44.10. With every $s_j = s$, $\sum_{\mathcal{M}} e^{s_j} = 125\,e^{s}$ and $\sum_{\mathcal{N}} e^{s_j} = 219\,e^{s}$. By Equation (44.14) with $b = 0$:

$$P(M) = \frac{125\,e^{s}}{125\,e^{s} + 219\,e^{s}} = \frac{125}{344} = 0.3634.$$

For $\frac{1}{2}$ the two sides must weigh the same: $e^{b} \times 125 = 219$, so $b = \ln(219/125) = 0.5608$ [C]. A pixel that looks like nothing in particular starts out "natural", by the count of entries alone, until b learns otherwise.

Chapter 45: Producing a submission

Solution to Exercise 45.1.

$$\begin{aligned} \text{span} &= 429{,}985 - 400{,}015 = 29{,}970 \text{ m}, \\ \Delta x &= 29{,}970/(1000 - 1) = 29{,}970/999 = 30.0 \text{ m}, \\ c &= 400{,}015 - 15 = 400{,}000. \end{aligned}$$

Dividing by W: $29{,}970/1000 = 29.97\,\text{m}$. If the UL value is also taken as the image's left edge ($c = 400{,}015$), the last column's centre lands at

$$400{,}015 + 999.5 \times 29.97 = 400{,}015 + 29{,}955.015 = 429{,}970.015,$$

against the attribute's 429,985: 14.985 m, half a pixel, off. The first column's centre, $400{,}015 + 0.5 \times 29.97 = 400{,}029.985$, is off by the same amount: the whole grid slides half a pixel.

Solution to Exercise 45.2. The centre is at $j = 20.5$, $i = 10.5$:

$$\begin{aligned} X &= 30 \times 20.5 + 545{,}007.75 = 615 + 545{,}007.75 = 545{,}622.75, \\ Y &= -30 \times 10.5 + 2{,}501{,}437.5 = -315 + 2{,}501{,}437.5 = 2{,}501{,}122.5, \end{aligned}$$

in EPSG:32642.

Solution to Exercise 45.3. The omitted field takes the pydantic default `True`, so the vendable is on the 165-band grid. v0.2.0 could run on it, but the worker checks the models in list order, so V2 comes first and `_check_band_grid` refuses it ("reads native bands: run band_filter_apply with use_common_wavelength_grid=false"): the action fails. Fix: a band filter with the field `false`, which is what the UI's defaults give.

Solution to Exercise 45.4. 8-connected: the centre pixel touches all four corners, so all five pixels form **1** object. 4-connected: no two pixels share an edge, so there are **5** objects.

Solution to Exercise 45.5. $0.03 \times 10000 = 300 \to 300$. $-1.2 \times 10000 = -12{,}000 \to -9998$ (the lower clip, so it cannot be read as nodata). $3.5 \times 10000 = 35{,}000 \to 32{,}767$ (int16's largest value).

Solution to Exercise 45.6. A commit overwrites the prep's `anomaly_mask.tif`, so result 1 would be lost, and the export names the zip by the action's short id, so two results from one action would not be told apart. Three results need three prep actions.

Chapter 46: What HyperSL is, and what it does well

Solution to Exercise 46.1. With $D = 128$ and feed-forward width 512:

$$\begin{aligned}
\text{LayerNorms} &= 2 \times (2 \times 128) = 512,\\
\text{q, k, v} &= 128 \times 384 + 384 = 49{,}152 + 384 = 49{,}536,\\
\text{projection} &= 128 \times 128 + 128 = 16{,}512,\\
\text{feed-forward} &= (128 \times 512 + 512) + (512 \times 128 + 128) = 66{,}048 + 65{,}664 = 131{,}712,\\
\text{block} &= 512 + 49{,}536 + 16{,}512 + 131{,}712 = 198{,}272.
\end{aligned}$$

About a quarter of 789,760: halving D quarters every $D \times D$ matrix.

Solution to Exercise 46.2. $p = (1000 - 400)/10 = 60.0$. Entry 0 is $\sin(60.0) = -0.3048$, entry 1 is $\cos(60.0) = -0.9524$. Passed as 1.0, $p = (1.0 - 400)/10 = -39.9$: a wavelength 400 nm *below* the code's zero, one the model has never seen.

Solution to Exercise 46.3. $224 - 7 = 217$ bands; `int(217*0.05)` = `int(10.85)` = 10 visible. For 202 bands, `int(10.1)` = 10.

Solution to Exercise 46.4. With band 0–1 input: $59.60 - 39.60 = 20.00$ points. Raw input: $48.19 - 39.60 = 8.59$ points. Share kept: $8.59/20.00 = 0.43$, so 43% of the gain survives without the scaling.

Solution to Exercise 46.5. `pe` $500 \times 256 = 128{,}000$; `div_term` 128; BatchNorm running means and variances $4 \times 2 \times 256 = 2{,}048$; four `num_batches_tracked` counters, 4. Sum $128{,}000 + 128 + 2{,}048 + 4 = 130{,}180$, and $9{,}158{,}021 - 9{,}027{,}841 = 130{,}180$.

Solution to Exercise 46.6. A sigmoid's output lies strictly between 0 and 1, so the model can only reproduce inputs in that range; per-scene min-max puts every input in $[0, 1]$ by construction. A raw 1.2 is a target the output can never reach: the reconstruction saturates near 1 and leaves a residual of at least 0.2 there however well the model understands the spectrum.

Chapter 47: Per-scene min-max, and a library with nowhere to stand

Solution to Exercise 47.1. Band 1 holds 0.15, 0.20, 0.25: $m_1 = 0.15$, $M_1 - m_1 = 0.10$. Band 2 holds 0.25, 0.50, 0.55: $m_2 = 0.25$, $M_2 - m_2 = 0.30$.

$$\begin{aligned}
x'_1 &= (0.20 - 0.15)/0.10 = 0.05/0.10 = 0.5,\\
x'_2 &= (0.50 - 0.25)/0.30 = 0.25/0.30 = 0.8333.
\end{aligned}$$

A third input for the same spectrum: $(0.5, 1.0)$ in S_1, $(0.4286, 0.75)$ in S_2, $(0.5, 0.8333)$ in S_3.

Solution to Exercise 47.2. Still $(0, 0)$: $\mathbf{p}_1$ holds the minimum of both bands, and the bright pixel changed only the maxima, so $x'_b = (m_b - m_b)/(M_b - m_b) = 0$ whatever M_b is.

Solution to Exercise 47.3.

$$\begin{aligned}
\mathbf{u}' &= (0.20/0.20,\ 0.50/0.10) = (1,\ 5),\\
\mathbf{v}' &= (0.30/0.20,\ 0.40/0.10) = (1.5,\ 4),\\
\mathbf{u}' \cdot \mathbf{v}' &= 1 \times 1.5 + 5 \times 4 = 1.5 + 20 = 21.5,\\
\|\mathbf{u}'\| &= \sqrt{1 + 25} = \sqrt{26} = 5.0990, \qquad \|\mathbf{v}'\| = \sqrt{2.25 + 16} = \sqrt{18.25} = 4.2720,\\
\cos\theta' &= 21.5/(5.0990 \times 4.2720) = 21.5/21.783 = 0.9870, \qquad \theta' = 9.25^\circ.
\end{aligned}$$

Solution to Exercise 47.4.

$$\cos\theta' = \frac{25u_1v_1 + 25u_2v_2}{\sqrt{25u_1^2 + 25u_2^2}\sqrt{25v_1^2 + 25v_2^2}} = \frac{25(u_1v_1 + u_2v_2)}{5\sqrt{u_1^2 + u_2^2}\cdot 5\sqrt{v_1^2 + v_2^2}} = \cos\theta.$$

Solution to Exercise 47.5. Around 1780 nm the grid runs 1760, 1770, 1780, 1790, then 1960 (the next band after the gap). The 5-band window centred on 1780 is 1760, 1770, 1780, 1790, 1960: span $1960 - 1760 = 200$ nm. The 13-band window centred on 1740 runs from $1740 - 6 \times 10 = 1680$ up to 1790 (five steps) and then 1960: span $1960 - 1680 = 280$ nm. Both match the table's maxima.

Solution to Exercise 47.6.

$$z_1 = (0.70 - 0.25)/0.10 = 0.45/0.10 = 4.5,$$
$$z_2 = (0.20 - 0.35)/0.12 = -0.15/0.12 = -1.25.$$

No: the mean and spread come from the checkpoint's fixed curve, so the answer is the same against every scene.

Chapter 48: Anomaly detection: a model that completes the anomaly

Solution to Exercise 48.1. Self-completion: $\hat{x}_2 = (0.50 + 0.40)/2 = 0.45$, residual $0.45 - 0.45 = 0$. Spatial:

$$\mathbf{r} = (0.50 - 0.20,\ 0.45 - 0.30,\ 0.40 - 0.40) = (0.30,\ 0.15,\ 0),$$
$$\|\mathbf{r}\|^2 = 0.09 + 0.0225 + 0 = 0.1125.$$

Invisible to the self-completer, plainly visible to the spatial one.

Solution to Exercise 48.2.

$$1{,}050{,}895 \times 149 = 156{,}583{,}355,$$
$$156{,}583{,}355 \times 4.8713 \times 10^{-6} = 762.8 \approx 763.$$

The run logged 736, 27 fewer: about one standard deviation ($\sqrt{763} \approx 27.6$).

Solution to Exercise 48.3. $1 - (1 - 0.01)^5 = 1 - 0.99^5 = 1 - 0.95099 = 0.0490$, about 4.9%.

Solution to Exercise 48.4. Objects: the interval $[-4.0, +1.5]$ points contains zero, so no difference is shown. pAUC: $[-0.0589, -0.0318]$ lies entirely below zero, so HyperSL RM's pAUC is lower, resolved.

Solution to Exercise 48.5. Chance is what scores unrelated to the objects would catch. Catching fewer means the objects' scores are systematically *lower* than the background's: HyperSL's loss is smaller on the anomalies, so it reconstructs them better than ordinary ground. That is self-completion, and it matches the AUCs below 0.5 of the 2026-09-23 run.

Solution to Exercise 48.6. $10^6 \times 438 \times 10^{-6} = 438$ s, 7.3 minutes. With one draw, about a quarter, ≈ 110 s, assuming the time is spent per draw [A] (each draw runs the tokenizer, encoder and decoder once).

Chapter 49: Unmixing and sub-pixel targets

Solution to Exercise 49.1. At the target, $(0.25 + 8 \times 0)/9 = 0.25/9 = 0.0278$. A diagonal neighbour's window also contains the target once, so it is 0.0278 too. A pixel two steps away has a window that does not reach the target: 0.

Solution to Exercise 49.2.

$$\text{upstream: } 0.106 \times 57.296 = 6.07^\circ,$$
$$\text{VCA + FCLS: } 0.004 \times 57.296 = 0.23^\circ.$$

Solution to Exercise 49.3. The target survives while $0.13 - 2\lambda/0.20 > 0$, that is $2\lambda < 0.13 \times 0.20 = 0.026$, so $\lambda < 0.013$. The $\lambda = 0.013$ of the worked example is exactly the boundary, which is why the target landed on 0.

Solution to Exercise 49.4. For $k = 2$ the shrinkage is $2\lambda/(kc) = 0.026/(2 \times 0.20) = 0.026/0.40 = 0.065$, so $a' = 0.13 - 0.065 = 0.065$. It survives, at half its fill: four pixels share a perimeter of 8 edges, half the perimeter per pixel of a lone pixel.

Solution to Exercise 49.5. If the two parts are at right angles to each other [A], squared lengths add, and **Ea** is left with $1.0 - 0.40 = 0.60$. The abundances need not account for the whole pixel: whatever the unconstrained branch can absorb, including a target's signature, no longer has to show up as a fill.

Solution to Exercise 49.6. $4 \times 13 = 52$ minutes for the bench; $18 \times 13 = 234$ minutes, 3.9 hours, for the implant hosts, against seconds per scene for VCA + FCLS.

Chapter 50: Cloud masking

Solution to Exercise 50.1.

$$\text{recall} = 42{,}725/49{,}405 = 0.8648 = 86.5\%,$$
$$\text{column total} = 909 + 2{,}042 + 1 + 42{,}725 = 45{,}677,$$
$$\text{precision} = 42{,}725/45{,}677 = 0.9354 = 93.5\%.$$

Solution to Exercise 50.2.

$$\text{recall} = 225{,}595/280{,}448 = 0.8044 = 80.4\%,$$
$$\text{column total} = 25{,}158 + 225{,}595 + 15{,}888 + 1{,}135 = 267{,}776,$$
$$\text{precision} = 225{,}595/267{,}776 = 0.8425 = 84.2\%.$$

Solution to Exercise 50.3.

$$\text{C: } 341{,}395 + 244{,}246 + 20{,}472 + 42{,}725 = 648{,}838, \qquad 648{,}838/784{,}120 = 0.8275,$$
$$\text{B: } 342{,}688 + 225{,}595 + 48{,}037 + 47{,}848 = 664{,}168, \qquad 664{,}168/784{,}120 = 0.8470.$$

Solution to Exercise 50.4. $0.03/0.40 = 0.075$. For a step of 0.10 the span must be $0.03/0.10 = 0.30$.

Solution to Exercise 50.5. C missed $70{,}793 - 20{,}472 = 50{,}321$ cirrus pixels, of which 28,341 were called cloud: $28{,}341/50{,}321 = 0.563$, 56.3%. The product excludes cloud, so a cirrus pixel called cloud is still excluded; one called clear stays in the scene, and any false alarm on it is reported.

Chapter 51: The verdict, and what would have to change

Solution to Exercise 51.1. Matching: per-scene min-max scaling, which gives a library spectrum no input (Chapter 47). Detection: band masking within a pixel, so the pixel completes itself (Chapter 48). Unmixing: the 3×3 head and the TV penalty, which shrink small targets to zero (Chapter 49). Cloud: pretraining on near-cloud-free scenes, compounded by min-max turning one cirrus layer into a different step per scene (Chapter 50).

Solution to Exercise 51.2. Only item 6 (publishing the data and a licence), and that needs the authors, not us. Items 1–5 change what the weights were trained to read or do: the input scaling, the first layer, the masking, the data, the heads. Weights trained under the old choices are off-distribution under new ones; Section 46.4 measured how far (wavelengths in micrometres raised the loss from 0.0024 to 0.173; per-spectrum scaling roughly doubled it).

Solution to Exercise 51.3. $449 - 369 = 80$ objects; $80/909 = 0.088$, 8.8 points. The bootstrap row "HyperSL RM − FTH3 ep. 5, cloud excl." gives -8.8 points, interval $[-11.7, -6.2]$: the same number, resolved.

Solution to Exercise 51.4. (1) Library grouping, 0.965 against 0.949: grouping entries is not naming anomalies, where it lost. (2) Thick-cloud recall 87.1% against 80.4%: thick cloud is the easy class, and cirrus, the hard one, fell to 28.9%. (3) Fewer plateau false calls, 4.34% against 6.62%: a smaller error on a corrected subset does not outweigh losing 39 points of cirrus. (4) The false-alarm filter with a hindsight cut: the honest cut lost up to 58% of objects. (5) Agreement with FCLS references on Samson, Jasper and Urban: those references are themselves unmixings, and on exact truth it lost every set.

Appendix B

Glossary

Each term is defined where it first appears; the chapter is given in brackets.

1 by 1 convolution
A convolution that looks at one position at a time and mixes only channels: a linear layer applied at every pixel with shared weights. (Chapter 19)

1-pixel tolerance
The rule that a flagged pixel adjacent to an object also catches it, allowing for ground truth about one pixel off the grid. (Chapter 33)

1x1 convolution
A convolution with a one-pixel kernel: a linear layer applied independently at every pixel, mixing channels only. (Chapter 10)

3D ROC
Plots of detection and false-alarm rates against a min-max normalised threshold; its areas reduce to mean normalised scores and are degenerate for heavy-tailed scores. (Chapter 33)

4-connectivity
Pixels are neighbours only if they share an edge (up, down, left, right), not a corner. (Chapter 33)

40 percent rule
The trainer's rule that drops any patch with less than 40 percent valid pixels before the loss is computed. (Chapter 25)

ABU
The Airport-Beach-Urban benchmark of 13 small scenes (2017); radiance of unknown gain and no published wavelengths. (Chapter 34)

Abundance
The fraction of a pixel's footprint covered by a material; the a of the linear mixing model. (Chapter 34)

Accuracy
The fraction of answers a yes/no rule gets right; meaningless when one class is extremely rare. (Chapter 1)

Action
One processing step inside a Project; reads earlier Actions' outputs and writes its own to its own folder. (Chapter 4)

Action chain
The sequence of product actions from upload to export: band filter, segmentation, scoring,

detection prep, spectral match, export. (Chapter 30)

ActionOutput
The folder of files one Action writes, referred to by later Actions. (Chapter 4)

Activation function
A nonlinear function applied after a linear layer, such as GELU or ReLU. (Chapter 10)

Activation memory
Memory held by intermediate tensors of the forward pass that the backward pass needs; for the per-band stem about 19 tensors of 202 MiB per patch in bf16. (Chapter 23)

AdaBN
Adaptive batch normalisation: replacing BatchNorm's stored statistics with statistics measured on the data being scored. (Chapter 29)

Adam
The optimiser that divides an EMA of each parameter's gradient by the square root of an EMA of its squared gradient, giving every parameter a step of about the learning rate when its gradient is consistent. (Chapter 26)

AdamW
Adam with decoupled weight decay: the decay is subtracted separately rather than passed through Adam's per-parameter normaliser. (Chapter 26)

Adapt stage
Fine-tuning reconstruction only, on all three sensors at their own bands, at a low learning rate. (Chapter 29)

Adjusted Rand index
A pair-counting agreement between two groupings, corrected so a random grouping scores 0 and an identical one 1. (Chapter 46)

Affine map (LayerNorm)
The learned per-feature scale gamma and shift beta applied after standardisation. (Chapter 16)

Affine subspace
A subspace shifted by a fixed vector; Indradhanu's reconstructed spectra lie in a 32-dimensional one. (Chapter 12)

Affine transform
Six numbers mapping a pixel's column and row to map coordinates: X = a col + b row + c, Y = d col + e row + f. (Chapter 32)

Aliasing
Two different inputs giving the same encoding because a wave has completed a whole number of periods between them. (Chapter 22)

align_corners=False
PyTorch's resizing convention in which pixel centres sit at half-integer positions and the outer edges of input and output grids line up. (Chapter 19)

All-MLP decoder
SegFormer's decoder design: per-scale 1 by 1 projections, upsampling to one size, concatenation and a 1 by 1 fuse; 1 by 1 convolutions are per-pixel MLP layers. (Chapter 19)

Allotrope
The web application this book describes: users upload satellite scenes, run chains of analysis steps, pick a threshold and export a bundle of candidate anomalies. (Chapter 1)

Angle between vectors
The angle whose cosine is the dot product divided by both lengths; it measures shape and ignores brightness. (Chapter 6)

Annulus
Another name for the dual-window background ring. (Chapter 8)

Anomaly
A pixel or small group of pixels unlike the rest of its scene; the word says nothing about what the pixel is. (Chapter 1)

Anomaly mask
The binary raster (anomaly_mask.tif) written when a threshold is committed: 1 where the composite is at or above the threshold. (Chapter 32)

Anomaly score
A non-negative number per pixel, larger meaning the pixel fits the model of normal worse. (Chapter 1)

Antardhana
The convolutional autoencoder trained to fill 13 to 25 percent hidden pixels with an MSE loss; 264,961 parameters. (Chapter 11)

api
The web-server process: handles requests and the database, hands out work, never computes. (Chapter 4)

Argsort
The permutation that sorts a list: its k-th entry is the index of the k-th smallest value. (Chapter 18)

Array
A block of numbers indexed by whole-number positions; its shape lists the size of each index. (Chapter 2)

Asanskrita
The mask-aware convolutional autoencoder with no normalisation: three input channels (temperature, validity, input mask); 267,009 parameters. (Chapter 11)

atan2
The two-argument arctangent, giving the angle of the point (x, y) from the positive x-axis; used by SAMLoss to compute the spectral angle with finite derivatives. (Chapter 25)

Atmospheric window (absorption)
A wavelength range where water vapour or carbon dioxide absorbs so strongly that a band measures noise; removed from the grid. (Chapter 3)

Attention mask
A boolean tensor passed to PyTorch's fused attention function; True means attention is allowed, False means the score becomes minus infinity. (Chapter 23)

Attention matrix
The N-by-N (or N-by-N-reduced) matrix of softmax weights; row i says how query i shares its attention among the keys, and every row sums to one. (Chapter 14)

Attention weights
The softmax of scaled query-key scores; each row is positive and sums to 1. (Chapter 22)

AUC
The area under the ROC curve; 1 is perfect, 0.5 is random. Equal to the probability that a random target outscores a random background pixel, ties counting half. (Chapter 33)

Autocast
PyTorch's context that runs eligible operations in a lower-precision format automatically. (Chapter 26)

Autoencoder
A reconstruction model built from an encoder that summarises the input and a decoder that rebuilds the input from the summary. (Chapter 9)

Average pooling
Replacing each block of a grid by the plain average of its entries; a special case of the learned reduction convolution. (Chapter 15)

AVIRIS-NG
Airborne hyperspectral sensor with about 425 bands, delivered as ENVI files in a folder. (Chapter 3)

Axis
One direction along which a tensor is indexed; also called a dimension. (Chapter 10)

Background subspace
The span of a scene's top ten principal directions. (Chapter 34)

Backpropagation
Computing the gradient of the loss with respect to every parameter by applying the chain rule backwards from the loss through each layer. (Chapter 25)

Band
One measurement channel of a sensor, recording light in one narrow range of wavelengths; called a channel in deep-learning code. (Chapter 2)

Band alignment
Matching a sample's bands to a target list by wavelength within 0.5 nm; unmatched targets become nodata and unmatched sources are dropped. (Chapter 27)

Band density
How many bands a sensor places in a region of the spectrum; with a one-vote-per-band average it changes the stem's output for the same spectrum. (Chapter 27)

Band grouping
Making one token from several adjacent bands to cut the token count (202 to 51), at the cost of mixing fine spectral detail early. (Chapter 23)

Band pipeline
The eight stages that turn a sensor's native bands into a complete spectrum per valid pixel on the 165-band common grid. (Chapter 5)

Band response
The weighting over wavelengths with which a sensor band integrates light, roughly Gaussian with a given full width at half maximum. (Chapter 32)

Band token
One band of one pixel represented as a d-number vector: its value stretched by Linear(1, d), plus the encoding of its wavelength. (Chapter 22)

Band validity mask
The tensor `band_valid`: one boolean per band per pixel, False for a dead or missing band. (Chapter 23)

Band weighting by width
An option of the reduced stem in which each band counts by the nanometres of spectrum it covers, making its means approximate integrals over wavelength. (Chapter 24)

Band-delta compression
Storing each band as its int16 difference from the previous band, with wrap-around, then compressing with zstd; exact and about 2 times smaller. (Chapter 27)

Band-grid check
The worker's test that a model's expected grid matches the vendable's wavelengths, failing the action with a message naming the band-filter setting. (Chapter 32)

Band-list augmentation
Training on a random subset of a batch's bands while reconstructing all of them; safe because the loss is taken only on hidden pixels. (Chapter 24)

BandFilterConfig
The configuration object that drives the eight-stage band pipeline: exclusion windows, edge trimming, coverage threshold, voxel-fraction threshold, quality masks and target grid. (Chapter 5)

Base rate
The fraction of all cases that belong to the rare class, here the fraction of pixels that are anomalous. (Chapter 1)

Base width
The channel count of a convolutional autoencoder's first stage; later stages double it. (Chapter 11)

Batch
A group of image patches processed together in one step of training or inference; its size is written B. (Chapter 2)

Batch (mini-batch)
The group of patches processed together in one forward pass; its gradient is a noisy estimate of the whole training set's gradient. (Chapter 26)

Batch normalisation
A layer that standardises each channel with the mean and variance computed over the batch and all pixels, then applies a learned scale and shift. (Chapter 12)

BatchNorm
Normalisation of one channel using statistics gathered across the batch, with running statistics at evaluation; its output depends on the other samples in the batch. (Chapter 16)

BatchNorm running statistics
A running mean and variance per channel and one batch counter, kept as buffers for evaluation. (Chapter 20)

Bench
The mean ROC-AUC on the four labelled PRISMA scenes under residual Mahalanobis, the detection measure every fine-tuning gate reads. (Chapter 29)

Bench resolution
The smallest difference a bench can tell from noise: about plus or minus 0.005 to 0.01 pAUC on the real PRISMA bench. (Chapter 36)

bench-v1, bench-v2, bench-v2.1
The implant bench before foreignness and halo were matched; after (accepted 2026-10-17); and rebuilt after the descending-wavelength fix with a tighter foreignness tolerance (2026-10-18). (Chapter 34)

bench-v2.1
The implant benchmark rebuilt on 2026-10-18 with sorted interpolation and foreignness tolerance 1.2: 18 hosts, 2,817 objects. (Chapter 37)

Bernoulli distribution
The distribution of a single yes/no outcome with probability p of yes: $P(t) = p^t(1-p)^{1-t}$. (Chapter 28)

Bernoulli variable
A random variable that is 1 with probability p and 0 otherwise; its mean is p and its variance p(1 - p). (Chapter 37)

best.pt
The checkpoint a run keeps as its best, chosen by validation reconstruction (or segmentation) loss, not by detection. (Chapter 38)

bfloat16
A 16-bit floating-point format with float32's 8 exponent bits and 7 mantissa bits: float32's range at about 2 to 3 significant digits. (Chapter 26)

Bias
A learned constant added to a neuron's or channel's output. (Chapter 10)

Bias correction
Dividing Adam's moment estimates by 1 minus beta to the power t, which removes the bias towards zero caused by starting the averages at zero. (Chapter 26)

Biased variance
A variance that divides the sum of squared deviations by the number of values M; BatchNorm normalises with it. (Chapter 12)

BIL
Band-interleaved-by-line layout (H, C, W): PRISMA's native layout. (Chapter 3)

Bilinear interpolation
Linear interpolation along the rows and then between the rows; each output is a weighted average of four neighbouring samples with weights that sum to 1. (Chapter 19)

Bin token
The mean over a bin's valid bands of each band value times its own wavelength-computed vector k(lambda), plus a bias from the bin centre's tag. (Chapter 24)

Binary cross-entropy
The negative log-likelihood of 0/1 labels under predicted probabilities, averaged over pixels; its gradient with respect to the logit is p - t. (Chapter 28)

Binary cross-entropy (BCE)
The loss -(y ln p + (1 - y) ln(1 - p)) for a yes/no target y and predicted probability p. (Chapter 38)

Binary dilation
Adding every pixel within the radius of the set; grows the invalid region. Computed as a stride-1 max pool. (Chapter 18)

Binary erosion
Keeping a pixel only when its whole square neighbourhood is in the set; shrinks the valid region by the radius. (Chapter 18)

Binary search
Finding a value's position in a sorted list by repeated halving; requires ascending order, which is why wavelengths must be sorted before np.interp or np.searchsorted. (Chapter 5)

Binary validity
The state after stage 7 in which every pixel is valid in all bands or in none. (Chapter 5)

Binomial standard error
The spread of a share caught over n independent objects, the square root of p(1 - p)/n; a rough ruler for whether two shares differ. (Chapter 37)

BIP
Band-interleaved-by-pixel layout (H, W, C): each pixel's spectrum is contiguous. (Chapter 3)

Bit flag
A single binary digit of an integer used as a yes/no answer; bit k of q is (q » k) and 1. (Chapter 3)

Blind spot
The arrangement at inference in which each pixel is reconstructed while its own token is hidden, so it cannot copy itself. (Chapter 38)

Blind unmixing
Finding both the endmembers and the abundances from the image alone. (Chapter 49)

Block
In F001, 200 consecutive shards (30,000 patches, one epoch's worth) trained for 20 epochs before the next block replaces it. (Chapter 27)

Block masking
Hiding tokens in b x b groups rather than singly (FT08: 2 x 2 tokens, 8 px), so a target and its halo vanish together. (Chapter 42)

Block parameter formula
One transformer block of width C, reduction ratio R and expansion 4 has 12 C squared + 53 C parameters, plus R squared C squared + 3C when R is above 1. (Chapter 20)

Block-diagonal matrix
A square matrix whose non-zero entries sit in square blocks along the diagonal; its inverse is the inverse of each block in place. (Chapter 31)

Boolean indexing
Selecting the entries of a tensor where a True/False tensor is True, producing a flat list; used to pull out the graded pixels. (Chapter 25)

Bootstrap
Estimating a statistic's sampling distribution by recomputing it on many samples drawn with replacement from the data. (Chapter 36)

Bootstrap sample
One draw of n items with replacement from the n data items. (Chapter 36)

Bottleneck
The internal summary between encoder and decoder; it constrains what the model can carry from input to output. (Chapter 9)

Box sum
The sum of a rectangle of values, obtained from an integral image with four look-ups: far corner minus top strip minus left strip plus the doubly removed corner. (Chapter 31)

Broadcasting
Repeating an array along axes of size 1 so that arrays of shapes such as (B, 165, H, W) and (1, 165, 1, 1) combine element by element. (Chapter 5)

BSQ
Band-sequential layout (C, H, W): each band image is contiguous; the layout every model expects. (Chapter 3)

Buffer
A tensor stored in a model's state but not trained, such as normalisation statistics or BatchNorm running averages; it travels inside the checkpoint. (Chapter 9)

Buffer (PyTorch)
A tensor registered on a module that is saved in the checkpoint and moved with the model but never trained; how the normalisation statistics are stored. (Chapter 5)

Cache key
The path of a cached map: dataset, scene, model id, and a SHA-1 of the scorer's settings joined to the scene fingerprint. (Chapter 35)

Capacity
How much a model can represent; for fixed depth it grows with width. (Chapter 43)

Capacity block
A GPU instance rented for a fixed, pre-booked time window that runs whether or not a job is running. (Chapter 43)

Capacity limit
Error that remains even when the network can see the pixel, because it cannot render the detail. (Chapter 42)

Catastrophic cancellation
Loss of precision when subtracting two large, nearly equal floating-point numbers, as in E[x squared] minus (E[x]) squared. (Chapter 5)

Centre wavelength
The wavelength at which a band responds most strongly; stored per band as band_cw_order. (Chapter 2)

Centring
Subtracting the mean from every vector, so that the deviations add up to zero. (Chapter 6)

Chain rule
The derivative of a composition is the product of the derivatives along the way: d g(f(x))/dx = g'(f(x)) f'(x). (Chapter 16)

Chakshu
The thermal foundation model ("the eye"), the one-band sibling of Indradhanu. (Chapter 1)

Chance level
What a random scorer would get on a measure: 0.5 for AUC, half the limit for pAUC, the FPR itself for TPR at FPR, and 1 - (1 - q)^m per object for objects caught. (Chapter 33)

Chance row
The first row of every report table: the mean over objects of each object's chance of being caught by random scores. (Chapter 35)

Channel identity
A learned 32-number vector per latent channel, added to its stretched value, because channels have no wavelength to encode. (Chapter 24)

Characteristic polynomial
det(S - lambda I) as a polynomial in lambda; its roots are the eigenvalues. (Chapter 6)

Chebyshev distance
The larger of the row and column offsets, so a diagonal step counts as one. (Chapter 44)

Checkerboard cell size
The size, in stage-1 tokens, of the cells of the two complementary inference masks. (Chapter 42)

Checkerboard mask
The default inference masking pattern, (row + column) mod 2 over the token grid; pass 1 hides the even tokens and pass 2 the odd ones. (Chapter 30)

Checkpoint
A saved file holding the model's parameters, the optimiser and scheduler state, the epoch and the configuration, from which training can resume. (Chapter 26)

Checkpoint noise
Variation in a measured score from one epoch's weights to the next, invisible to an object bootstrap. (Chapter 42)

Chi-square distribution
The distribution of a sum of C squared independent standard normals; mean C, standard deviation sqrt(2C). The RX score of a Gaussian background pixel follows it. (Chapter 7)

Circular procedure
A procedure whose answer is needed to set up its own input; borrowing a scene's min-max frame to place a library spectrum is one. (Chapter 47)

Cirrus
Thin, high ice cloud; EnMAP grades it in levels 0 to 3, and level 1 is treated as unknown. (Chapter 28)

Class imbalance
A class covering a tiny share of pixels; per-pixel losses then let a model ignore it almost for free. (Chapter 28)

Class map
Indradhanu V2's per-pixel classes: 0 land, 1 water, 2 cloud, 3 shadow, 4 snow, chosen by priority from the segmentation head's outputs. (Chapter 32)

Class priority
The order in which classes overwrite each other in the class map: cloud over snow over shadow over water over land. (Chapter 39)

Class score
The logsumexp of a class's scaled cosines; its softmax equals the sum of the per-entry softmax probabilities of the class's members. (Chapter 44)

Class weights
Per-class factors in the weighted mean of class losses; [1, 1, 1, 3, 3] doubles the rare classes' share of the gradient. (Chapter 38)

Class-aware residual Mahalanobis
Residual Mahalanobis run separately per class or type, each with its own whitening and a ring of same-type neighbours. (Chapter 40)

Classical detector
An anomaly detector that computes statistics from the scene itself and has no trained weights, such as RX or MNF-RX. (Chapter 1)

Classifier
A model that answers with one of a fixed list of labels; it needs labelled examples and cannot return a label it was never given. (Chapter 1)

Cloud exclusion
Setting to 0, after scoring, the score of pixels Indradhanu V2's segmentation head calls (thick) cloud. (Chapter 32)

Cloze test
A reading exercise in which words are blanked out and must be filled in; the analogy for masked training. (Chapter 18)

Codename
The Indic name (Indradhanu, Chakshu, ...) by which the backend selects a foundation model. (Chapter 4)

Codename routing
Turning a codename into a model through a dictionary built from the catalog; a repeated codename routes to the last manifest loaded. (Chapter 39)

Cofactor
The signed determinant of the block left after deleting one row and one column of a matrix; cofactors build the inverse of a 3 x 3 matrix. (Chapter 7)

Combined loss
Indradhanu's training objective, L1 plus lambda(t) times the mean spectral angle over graded pixels. (Chapter 25)

Combined score
A size-based score: 0.5 times L1 divided by its scene maximum plus 0.5 times SAM divided by its scene maximum; Indradhanu's default until 2026-10-15. (Chapter 30)

Common wavelength grid
The 165 wavelengths, 10 nm apart from 460 to 2450 nm with four atmospheric windows removed, onto which every hyperspectral sensor is resampled (DEFAULT_COMMON_WAVELENGTH_GRID). (Chapter 3)

Composite score
The weighted sum of each model's score map after min-max rescaling to [0, 1], with weights divided by their sum. (Chapter 32)

Compressed channels (D)
The width of the spectral bottleneck; 24 in v0.1.0 and 32 from v0.2.0. (Chapter 12)

computed mask
The pixels that actually received a local score, as opposed to the spatial mask of pixels eligible for one; statistics must be reduced over it. (Chapter 8)

Concatenation
Joining tensors along an axis; here four 256-channel maps along the channel axis into one 1024-channel map. (Chapter 19)

Condition number
The ratio of a matrix's largest to smallest eigenvalue; inverting loses roughly log10 of it in significant digits. (Chapter 7)

Confidence threshold
The rule that a pseudo-label counts only where the teacher's probability is above 0.9 or below 0.1. (Chapter 38)

Confounding
Two possible causes mixed into one comparison, so the result cannot be attributed to either. (Chapter 43)

Confusion matrix
The two-by-two table of true positives, false positives, false negatives and true negatives at one threshold. (Chapter 33)

Connectivity (4 and 8)
Whether pixels that touch only at a corner count as neighbours (8) or not (4); the object table uses 8. (Chapter 45)

Constant extrapolation
Filling values beyond the last sample by repeating the nearest end value. (Chapter 5)

Context
The visible surroundings from which a hidden value is predicted. (Chapter 9)

Contract
What a function promises to return, provided its inputs meet stated conditions. (Chapter 37)

Contrast (hidden-pixel)
Mean error on hidden target pixels divided by mean error on hidden background pixels. (Chapter 42)

Contrast z
Mean over bands of the absolute object-minus-ring difference, divided by the mean over bands of the ring's standard deviation. (Chapter 34)

Control run
A run identical to the experiment except for the thing under test (FTB1: F001 with Heavy's fine-tuning recipe). (Chapter 43)

Convex combination
A weighted sum whose weights are non-negative and sum to 1; it always lies between the smallest and largest of the values combined. (Chapter 22)

Convolution
A layer that slides a small kernel of shared weights over the input and takes a weighted sum at each position; in deep learning, a cross-correlation. (Chapter 10)

Convolution parameter rule
A convolution has c_in times c_out times k squared weights plus c_out biases. (Chapter 20)

Coordinate reference system
The definition of what map coordinates mean, such as longitude and latitude (EPSG:4326) or metres in a UTM zone. (Chapter 32)

Correlation
A covariance divided by both standard deviations, between -1 and 1. (Chapter 6)

Cosine annealing
The schedule eta(t) = eta-min + (eta-max - eta-min)(1 + cos(pi t / T))/2: flat at the start, steepest in the middle, flat near the floor. (Chapter 26)

Cosine schedule
A learning rate that falls along half a cosine wave from its start value to a floor. (Chapter 43)

Cosine similarity
The dot product of two vectors divided by the product of their lengths; for two wavelength tags it depends only on their wavelength difference. (Chapter 22)

Count channel
A channel of ones on valid pixels carried through the integral image, whose box sums give the number of valid pixels in each ring. (Chapter 31)

Covariance
The expected product of two quantities' deviations from their means; positive when they rise and fall together. (Chapter 36)

Covariance matrix
The average outer product of centred vectors (divided by N - 1 in the code); variances on the diagonal, pairwise covariances off it. (Chapter 6)

Coverage count
The number of convolution windows that contain a given pixel; k/s on average along one axis. (Chapter 13)

Coverage pruning
Dropping bands valid in fewer than min_valid_pixel_pct (default 20) percent of pixels. (Chapter 5)

Cross-attention
Attention in which the queries come from one set and the keys and values from another. (Chapter 14)

Cross-attention compressor
SpectralCrossAttentionCompressor: 32 learned queries attend over a pixel's band tokens and each answer becomes one of the 32 output channels; 5,345 parameters at d = D = 32, whatever the band count. (Chapter 23)

Cross-attention decoder
SpectralCrossAttentionDecoder: each requested wavelength becomes a query against the pixel's 32 channel tokens; 6,401 parameters at d = D = 32, for any band list. (Chapter 24)

Cross-check (of data paths)
Scoring one model through two different reading paths on data where both exist; any gap is a bug until explained. (Chapter 37)

Cross-correlation
Sliding a kernel over an input and summing products without flipping the kernel. (Chapter 10)

Cross-validation
Estimating how a setting chosen on some data will do on new data by holding data out of the choice. (Chapter 40)

crs_missing
The 422 error the export endpoint returns when no georeferencing can be recovered from the raw scene file. (Chapter 32)

Cubic spline
A smooth curve of joined cubic pieces through samples; can overshoot and invent dips or bumps. (Chapter 5)

Curriculum
A sequence of training stages, each changing one thing and each judged before the next starts: probe, adapt, joint. (Chapter 29)

Data-dependent weights
Mixing weights computed from the input itself, here a softmax of query-key scores, as opposed to the fixed weights of a convolution. (Chapter 23)

DatasetBuilder
The class that turns one sensor's raw file into a vendable. (Chapter 4)

Datatake
One continuous EnMAP recording; its consecutive scenes border each other, so it is kept on one side of a split. (Chapter 27)

Decision table
A table of requirement, candidate, alternative and evidence, used in Part X to set HyperSL beside this product's models job by job. (Chapter 51)

Decoder
The half of an autoencoder that rebuilds the input from the internal summary. (Chapter 9)

Decoder warm-up
Training only the new decoder with everything else frozen (mode decoder, FT09a) before joint training. (Chapter 42)

Decoder width
The common channel count, 256, to which all four encoder maps are projected before they are combined. (Chapter 19)

Decompressor
The bare 1 by 1 convolution from 32 compressed channels back to the band count, with no normalisation and no activation. (Chapter 19)

Degrees of freedom
The number C of squared standard normals summed in a chi-square variable; for RX, the number of dimensions scored. (Chapter 7)

Denormalisation
The inverse map x = std times z + mean (PixelDenormalize), returning model output to reflectance. (Chapter 5)

Depthwise convolution
A convolution in which each channel has its own private filter and no channels are mixed (groups equal to channels); 9 weights plus a bias per channel for a 3-by-3 kernel. (Chapter 16)

Depthwise parameter rule
A depthwise convolution over c channels has c times k squared weights plus c biases. (Chapter 20)

Descending band list
A band list ordered from long to short wavelength, as PRISMA writes its files within each detector. (Chapter 37)

Design effect
1 + (m - 1) rho: how much within-group correlation inflates the variance of a mean over groups of m correlated values. (Chapter 36)

Detached tensor
A tensor cut out of the gradient computation, so a loss that uses it cannot change what produced it. (Chapter 41)

Detector stripe
A column of pixels that all err the same way because one detector element of a push-broom spectrometer is calibrated slightly off. (Chapter 31)

Determinant
A number computed from a square matrix (ad - bc for 2 x 2) that is zero exactly when the matrix squashes some direction flat. (Chapter 6)

Diagonal loading
Adding a small multiple of the identity to a covariance before inverting (also called a ridge); every eigenvalue rises by that amount, eigenvectors unchanged. (Chapter 7)

Dice coefficient
Twice the overlap of two sets divided by the sum of their sizes; 1 for identical sets, 0 for disjoint ones. (Chapter 28)

Difference-of-Gaussians boost
A post-filter that subtracts a wide blur from a narrow one to favour compact blobs; it caught slightly more objects but lost much AUC. (Chapter 31)

Digital number
The raw integer a sensor file stores (0 to 65,535 for PRISMA), converted to reflectance with per-detector scale factors. (Chapter 27)

Digital number (DN)
The integer a sensor stores, proportional to detected energy; must be converted to a physical quantity before comparison. (Chapter 2)

Dilation
Growing a binary mask by a square structuring element, optionally applied before metrics and commit. (Chapter 32)

Directed acyclic graph (DAG)
A set of nodes joined by one-way arrows with no path that returns to its start; the shape of a Project's Actions. (Chapter 4)

Dot product
The sum of the products of two vectors' entries, position by position; a single number, large when the vectors are long and point the same way. (Chapter 6)

Drashta
Asanskrita with z-score normalisation in and out, hiding 50 to 75 percent; 267,009 parameters. (Chapter 11)

Drop path
Stochastic depth: dropping a whole sublayer's contribution for a whole sample; not used in this codebase. (Chapter 16)

Dropout
During training, zeroing each value with probability p and scaling the survivors by 1/(1-p); off at evaluation. (Chapter 16)

Dual encoder
A matching model with one network for the pixel and one for library spectra, scoring by the cosine of their embeddings, so new library entries need no retraining. (Chapter 32)

Dual window
The square ring used by local RX: an outer window around the pixel with a smaller inner (guard) window removed; also called an annulus. (Chapter 8)

Dual-window RX
The classical local detector that takes both mean and covariance from the ring; worse than the local mean on the residual. (Chapter 31)

Early stopping
Stopping a run once its validation score has not improved by a minimum relative amount for a number of epochs called the patience. (Chapter 27)

Edge band trimming
Dropping the first and last bands (default 3) of each detector, counted on the full list of the detector's band indices. (Chapter 5)

Edge snapping
The patch planner's rule that moves the last patch back to end exactly at the scene edge instead of running past it. (Chapter 26)

Edge strength
In the edge diagnosis, the largest absolute difference between a pixel's band mean and that of any valid four-neighbour. (Chapter 38)

Effective mask ratio
The hidden count divided by the number of valid tokens; at most the mask ratio because of the rounding down. (Chapter 18)

Effective sample size
The number of independent values that would give the same variance: n m / (1 + (m - 1) rho) for n groups of m. (Chapter 36)

Efficient self-attention (ESA)
SegFormer's attention, in which queries come from every token while keys and values come from a spatially reduced summary of the tokens. (Chapter 15)

Eigen-decomposition
Writing a symmetric matrix as U Lambda U transpose, with orthonormal eigenvectors in U and eigenvalues on the diagonal of Lambda. (Chapter 31)

Eigenvalue floor
Raising any eigenvalue below 1e-12 to 1e-12 before dividing, which prevents division by zero but does not otherwise regularise. (Chapter 31)

Eigenvector
A non-zero vector that a matrix only stretches, not turns; the stretch factor is its eigenvalue. (Chapter 6)

Embedding
A vector of numbers that represents something that is not itself a vector, such as a word or a block of pixels. (Chapter 13)

Embedding dimension
The number of features per token at a stage: 32, 64, 160 and 256 for stages 1 to 4. (Chapter 13)

Embedding width
The number of features each token carries at a stage; 32, 64, 160 and 256 for stages 1 to 4. (Chapter 17)

Empty token
A stage-2 token whose whole window of stage-1 tokens was hidden; about 7 of 256 at a 65 percent hiding rate. (Chapter 17)

Encoder
The half of an autoencoder that turns the input into an internal summary. (Chapter 9)

Encoder stage
One level of the four-level SegFormer encoder: a patch embedding, two transformer blocks and a LayerNorm. (Chapter 13)

Endmember
One of the pure spectra in the linear mixing model. (Chapter 49)

EnMAP
German hyperspectral satellite; 224 bands delivered as a folder, with ready-made cloud, cirrus, haze, shadow and snow masks. (Chapter 3)

Epoch (in this project)
A fixed budget of surviving training samples (40,000 for v0.2.0) followed by validation; the interval at which the learning rate, SAM weight and checkpoints advance. Not a full pass over the data. (Chapter 26)

Epsilon (normalisation)
A small constant, 1e-5 by default in PyTorch, added to the variance before the square root so that a constant input does not cause division by zero. (Chapter 16)

Erosion
Shrinking a validity mask so that a pixel survives only if every pixel in a square around it is valid; inference uses a 15 x 15 square. (Chapter 30)

Erosion radius
Half the erosion kernel, rounded down: kernel 15 removes a 7-pixel border strip, kernel 1 removes nothing. (Chapter 18)

Eval mode
PyTorch's model.eval(): BatchNorm uses its stored statistics and dropout is off; gradients and weight updates still work. (Chapter 40)

Evaluation config
One JSON file naming the models, datasets, scorers, measures and groupings of an evaluation; run by scripts/evaluate.py. (Chapter 35)

Evaluation mode
The mode in which BatchNorm normalises with its stored running statistics, so each output depends only on its own input. (Chapter 12)

Exact ROC
The ROC built from every distinct score as a threshold, one point at the end of each run of tied scores (exact_roc). (Chapter 33)

Exclude after scoring
Fitting the score's whitening on all valid pixels and then setting excluded pixels to 0, the order the benchmark measured. (Chapter 39)

Exclude scorer
A wrapper that runs a base scorer and sets chosen surface classes to 0 using a segmentation head's class map. (Chapter 35)

excluded_percent
The stats.json key giving 100 times the excluded (cloud) pixels divided by the spatially valid pixels. (Chapter 32)

Expansion ratio
The factor by which the feed-forward network widens the token in its middle; 4 here. (Chapter 16)

Explained variance ratio
A component's eigenvalue divided by the sum of all eigenvalues: its share of the total variance. (Chapter 6)

Exponential moving average
A running average in which each older value's weight shrinks by a factor beta per step. (Chapter 26)

Exponential moving average (EMA)
A running average updated as new = 0.999 old + 0.001 current; its gap to a fixed value halves in about 693 steps. (Chapter 38)

Export
The deliverable zip (GeoTIFF, Shapefile, CSV) of candidate anomalies with map coordinates. (Chapter 4)

Export bundle
The zip produced by POST /actions/{id}/export: a georeferenced GeoTIFF, a Shapefile with one point per anomaly pixel, legends, a CSV and a manifest. (Chapter 32)

Factorial design
An experiment that varies two or more factors in every combination, so each factor's main effect can be read with the others fixed. (Chapter 38)

Fallback path (ESA)
The branch taken when the token count is not H times W (tokens were removed at stage 1): no reduction, plain full attention over the surviving tokens. (Chapter 15)

False-alarm rate
The fraction of normal pixels a threshold flags by mistake. (Chapter 7)

False-positive rate (FPR)
The share of background pixels flagged at a threshold; also called the false-alarm rate. (Chapter 33)

Family matcher
The matcher that subtracts a ring background, scores whole families of spectra, calibrates against the scene's ordinary pixels and names a man-made family only above p*. (Chapter 32)

FCLS
Fully constrained least squares: per pixel, the abundances that best fit the mixing model with both constraints. (Chapter 49)

Feature map
The output of one encoder stage reshaped to (B, C, H, W): one C-number vector per grid position. (Chapter 17)

Feature pyramid
A set of feature maps of the same ground at falling resolution and rising channel width; here F1 to F4 at 1/4 to 1/32 of the patch side. (Chapter 17)

Feed-forward network
The per-token two-layer network of a transformer block, Linear, nonlinearity, Linear, with a wider middle. (Chapter 16)

Fencepost formula
The number of evenly spaced points from first to last inclusive is (last - first)/step + 1. (Chapter 3)

FileHelper
A class wrapping one physical file format (HE5Helper, TIFHelper, EnmapHelper, ENVIHelper, HotSatHelper) that returns raw arrays and metadata. (Chapter 3)

Fill
The fraction of a pixel's area covered by the target material, between 0 and 1. (Chapter 44)

Fill value
A substitute number written into a dead or missing band so that a fixed-size layer can accept the pixel; it is not a measurement. (Chapter 22)

Filtered vendable
The vendable built by band_filter_apply on the common grid and pickled to filtered_vendable.pkl. (Chapter 4)

Fine-tuning
Continuing to train a pretrained model on new data or a new task, starting from its learned weights. (Chapter 29)

Finite difference
An estimate of a derivative from neighbouring grid values, such as (f(i, j+1) - f(i, j-1))/2. (Chapter 38)

Fixed recipe
This book's name for a fixed linear compressor's mixing weights: the same weighted sum of bands for every pixel. (Chapter 22)

Fixed-grid model
A model that reads only the 165-band common grid (the shipped v0.2.0), so every scene must be resampled onto that grid before it reaches the model. (Chapter 37)

Flatten
An operation that merges several neighbouring axes into one without moving any number in memory. (Chapter 10)

Float16
A 16-bit floating-point number; half the storage of float32 but with a largest finite value of 65,504. (Chapter 35)

Float32
The 32-bit floating-point number format; its largest value is about 3.4 times 10 to the 38, so exp of anything above 88.72 overflows. (Chapter 14)

Floor and ceiling
The floor of u is the largest whole number not above u; the ceiling is the smallest whole number not below u. (Chapter 24)

Foreign fraction
The share of an object's squared difference from its ring that lies outside the background subspace. (Chapter 34)

forward_mixed
The model method that runs one stem pass per band list, then one BatchNorm, encoder and decoder pass over the concatenated batch, and splits the output by sensor. (Chapter 29)

Foundation model
In this repository, a network trained once to reconstruct ordinary imagery, whose reconstruction error is used as the anomaly score. (Chapter 1)

fp32 and bf16
32-bit floating point (4 bytes per number) and 16-bit bfloat16 (2 bytes per number). (Chapter 23)

Frozen backbone
Training only a new head while every other weight, and its BatchNorm statistics, stays fixed. (Chapter 41)

Frozen encoder
An encoder whose parameters and BatchNorm statistics are not updated during training. (Chapter 38)

Full width at half maximum
The width of a peaked response at half its height; for a Gaussian it is about 2.3548 times its standard deviation. (Chapter 32)

Full width at half maximum (FWHM)
The width of a band's response curve measured where the response is half its peak; for a Gaussian, about 2.3548 times sigma. (Chapter 2)

Full-attention fallback
What efficient self-attention does when tokens have been removed: it cannot rebuild the grid, so it uses all kept tokens as keys. (Chapter 17)

Full-resolution skip
A second input that bypasses the encoder and reaches the decoder at full size; in FT09 it has every hidden pixel zeroed. (Chapter 42)

Fuse convolution
The 1 by 1 convolution from 1024 to 256 channels that learns how much of each scale to use at each position. (Chapter 19)

Fused attention
One GPU routine that computes scores, softmax and the weighted sum tile by tile, never storing the full weight matrix; exact, not approximate. (Chapter 23)

Gate
A pass criterion written in a run's README before launch and never moved; the next stage starts only if it is met. (Chapter 29)

Gate halo
The square dilation around each gated pixel; halo 2 turns one pixel into a 5 x 5 square. (Chapter 42)

Gated loss
A training loss that leaves out the pixels with the highest current error (FT07: the top 1% per source group, plus a halo). (Chapter 42)

Gather
Reading values from named positions: output k is input at index k of the index list. (Chapter 18)

Gating
Leaving pixels that already look anomalous out of what a network learns from. (Chapter 40)

Gaussian cumulative distribution function
Phi(x), the probability that a standard Gaussian random number is below x. (Chapter 16)

Gaussian distribution
The bell-shaped distribution whose density falls off as the exponential of minus half the squared number of standard deviations from the mean. (Chapter 7)

Gaussian mixture
A density made of K weighted Gaussian components, each modelling one background type. (Chapter 40)

GELU
Gaussian Error Linear Unit, x times Phi(x), a smooth relative of ReLU; GELU(-1) = -0.1587, GELU(0) = 0, GELU(1) = 0.8413. (Chapter 16)

Generalised eigenproblem
Sigma a = lambda (noise Sigma) a: find directions where one covariance is a multiple of another; solved by whitening with the second matrix and then taking ordinary eigenvectors. (Chapter 8)

Geotransform (affine)
The map from a pixel's column and row to map coordinates: x = a j + b i + c, y = d j + e i + f. (Chapter 4)

Global RX
RX with one mean and covariance estimated from all valid pixels of the scene. (Chapter 7)

Graded pixel
A pixel whose loss-mask value is 1, so that it contributes to the loss. (Chapter 25)

Gradient
The vector of partial derivatives of the loss with respect to every parameter; it points in the direction of steepest increase of the loss. (Chapter 26)

Gradient accumulation
Summing the gradients of several small batches, each loss divided by their number, before one optimiser step, to imitate a larger batch. (Chapter 26)

Gradient clipping
Scaling the whole gradient down to a maximum length when it exceeds it, keeping its direction; not used in this repository. (Chapter 26)

Gradient descent
The update that moves every parameter a small step against its gradient: w becomes w minus the learning rate times the gradient. (Chapter 26)

Gradient magnitude
The length of the vector of an image's horizontal and vertical derivatives; large on edges. (Chapter 38)

Gradient-matching loss
The edge term gradient_l1: the L1 difference between the neighbour differences of the reconstruction and of the input, on hidden pixels. (Chapter 38)

GT-HAD
A gated-transformer hyperspectral anomaly detector trained from scratch on the scene it scores, with gating and early stopping as defences against learning the anomalies. (Chapter 40)

Guard window
The inner square excluded from the local background, so that a target larger than one pixel does not contaminate its own statistics. (Chapter 8)

HAD100
A public hyperspectral anomaly benchmark of 100 AVIRIS-NG scenes in at-sensor radiance, published with AETNet (2023). (Chapter 34)

Half-size
The window parameter used by LocalRXDetector: a half-size of 25 means a 51 x 51 square. The residual score and PCA-RX use full widths instead. (Chapter 8)

Halo
Raised scores around an anomaly caused by the anomaly contaminating its neighbours' predictions. (Chapter 9)

Halo ratio
The halo's contrast z over the object's; 0 for a hard edge. (Chapter 34)

Hash
A fixed-length code computed from text that changes completely when any character changes; used to build cache keys. (Chapter 35)

Hash split
Holding out a library spectrum when the first 8 hex digits of the SHA-1 of its name, over 2 to the 32 minus 1, fall below 0.2. (Chapter 44)

Haze colour test
Near-infrared over blue reflectance: above 1 for red bare ground, below 1 for bluish haze. (Chapter 35)

Head
One of several parallel attention computations, each on its own slice of the key, query and value vectors; the stem uses 4 heads of 8 numbers. (Chapter 23)

Head dimension
The number of features each attention head works with, d = C / h. (Chapter 14)

Heartbeat
A timestamp a worker writes to its running job at a fixed interval to show it is alive. (Chapter 4)

Heatmap rule
The evaluation rule that a continuous score map, or a score attribute that reads as one, disqualifies a submission. (Chapter 45)

Hidden-pixel rule
Train and apply a per-pixel prediction only where that pixel was hidden from the encoder, so it is judged from context alone. (Chapter 41)

Hierarchical encoder
An encoder made of stages at successively coarser resolutions, each returning its own feature map, as opposed to a plain Vision Transformer that keeps one token grid throughout. (Chapter 17)

Host
A real held-out scene that receives implanted objects. (Chapter 34)

Hot storage
The trainer's local-disk cache of shards, synced once and reused for every epoch. (Chapter 26)

HotSat-1
Thermal sensor delivering one band of uncalibrated digital numbers around 5000. (Chapter 3)

Hypernetwork
A small network whose outputs are weights for another computation; here a two-layer network maps a wavelength encoding to k, W or U. (Chapter 24)

HyperSL
An external spectral foundation model (Kong et al., TGRS 2025): a per-pixel masked autoencoder of 9,027,841 trainable parameters, pretrained on EnMAP and DESIS L2A spectra min-max scaled per band per scene; evaluated and not adopted (Part X). (Chapter 46)

Hyperspectral
Recording hundreds of narrow bands per pixel; high spectral, not spatial, resolution. (Chapter 2)

Hyperspectral cube
An array of shape (C, H, W) holding one reflectance image per band; equivalently, one spectrum per pixel. (Chapter 2)

Identity function
The function that returns its input unchanged; the perfect but useless solution to unmasked reconstruction. (Chapter 9)

Identity matrix
The square matrix with ones on the diagonal and zeros elsewhere; multiplying by it changes nothing. (Chapter 6)

Identity transform
The geotransform with a = e = 1 and all else 0; written on every Action's GeoTIFF as a placeholder. (Chapter 4)

Idle parameters
The 133,824 stage-1 weights (reduction convolutions, their norms and the depthwise convolutions) that receive no gradient in a masked pass because their layers are skipped. (Chapter 20)

Implant bench
A benchmark made by adding synthetic objects of known position and difficulty to real held-out scenes. (Chapter 34)

Implant-denoising
Training with synthetic anomalies in the input and the clean tile as the target, so the loss rewards reconstructing the background beneath an anomaly (FT05). (Chapter 38)

In-degree
The number of arrows entering a node of a directed graph. (Chapter 4)

Inclusion-exclusion
Counting the union of two overlapping sets as the sum of their sizes minus the size of their overlap. (Chapter 31)

Indradhanu
The hyperspectral foundation model ("rainbow"), a masked SegFormer autoencoder. (Chapter 1)

Indradhanu Heavy
The widened model (stage widths 96 to 768, progressive decoder) shipped as Indradhanu V2 v2.1.0; about 35.3 million parameters. (Chapter 20)

Inference
Running a trained model on new data to obtain its predictions, with no learning taking place. (Chapter 30)

Inference batch size
The number of tiles passed through the network together; 8 by default in the scoring action, the main memory control on a CPU worker. (Chapter 30)

Information leakage
A hidden token's pixels reaching the encoder through a visible neighbour whose window overlaps them. (Chapter 13)

Input mask
In the convolutional masked models, a 0/1 channel marking pixels that are valid and visible. (Chapter 9)

Integral image
A table in which each entry holds the sum of every value above and to the left of it, with a leading row and column of zeros. (Chapter 31)

Intersection over union
The size of the overlap of predicted and target sets divided by the size of their union; related to Dice by D = 2J/(1+J). (Chapter 28)

Invariant 7
The evaluation rule that excluded pixels stay in the measures as negatives tied at 0, and labelled objects inside them stay as unfindable positives. (Chapter 35)

Inverse
The matrix that undoes a square matrix, so that their product is the identity; it exists only when the determinant is non-zero. (Chapter 6)

Inverse map
A map that undoes another: D(N(x)) = x and N(D(z)) = z. (Chapter 5)

Inverse permutation
The permutation that undoes another: it gives, for each token, the place it occupies in the sorted order (its rank). Called ids_restore in the original masked autoencoder. (Chapter 18)

Irradiance
Power of light arriving on a surface per square metre per nanometre. (Chapter 2)

Jacobian
The matrix of partial derivatives of a vector function with respect to its vector input; the vector form of a derivative. (Chapter 16)

Joint stage
Training reconstruction and segmentation together: loss = reconstruction + seg weight x segmentation. (Chapter 29)

Jump
The distance in input pixels between neighbouring outputs of a layer; the product of the strides so far. (Chapter 10)

k-means
Lloyd's clustering recipe: assign each point to its nearest centre, move each centre to the mean of its points, repeat; the hard-assignment limit of a mixture. (Chapter 40)

Keep mask
The per-token mask saying which stage-1 tokens are shown to the encoder; when it removes tokens, stage 1 attention takes the fallback path. (Chapter 15)

Kernel
The small block of weights a convolution slides over its input; its side length is the kernel size k. (Chapter 10)

Kernel span
The wavelength range covered by a convolution window over band index: the last band's wavelength minus the first's. (Chapter 47)

Key
The learned projection of a token that expresses what it advertises about itself; compared against queries. (Chapter 14)

Key count
The number of keys efficient self-attention uses, N divided by R squared; 16 at every stage of a 128-pixel patch. (Chapter 17)

kmat
The matrix of shape (bands, bins x 32) that places each band's k(lambda) in its bin's slot, so one matrix product forms all bin tokens without a per-band tensor. (Chapter 24)

Known mask
A per-class, per-pixel flag saying whether a label is trusted; unknown pixels are left out of the loss and the IoU. (Chapter 28)

L1 loss
The average absolute error; its best constant guess is the median, so it is robust to rare extreme values. (Chapter 9)

L1 norm
The sum of the absolute values of a vector's entries. (Chapter 25)

L2 loss
The average squared error; its best constant guess is the mean, so rare extreme values pull predictions towards them. (Chapter 9)

L2 regularisation
Adding half lambda times the squared length of the weights to the loss; equal to weight decay under plain gradient descent but not under Adam. (Chapter 26)

L2-SP
A regulariser that adds alpha/2 times the squared distance of the weights from their pretrained values, pulling them back during fine-tuning. (Chapter 38)

Label
The correct answer attached to a training example. (Chapter 1)

Label audit
Checking labels without ground truth by testing whether what is labelled looks physically like its class, and finding what predicts failures. (Chapter 28)

Label ceiling
The limit on a class's accuracy set by wrong labels: a class that stays flat while every other class improves. (Chapter 38)

Label policy
The fixed rules (no-data, cloud class, cirrus level 1, overlaps, dry scenes) that decide which EnMAP labels are known; set from the audit before training. (Chapter 28)

Lagrange multiplier
A number introduced to maximise a function under a constraint; at the optimum the function's gradient is that multiple of the constraint's gradient. (Chapter 6)

Lambertian surface
An idealised matt surface that looks equally bright from every direction, for which total exitance equals pi times radiance. (Chapter 2)

Land surface temperature
The temperature of the ground's skin inferred from the heat it radiates; Chakshu's input, in degrees Celsius for Landsat 9. (Chapter 21)

Landsat 9
Thermal source in this product: one band (B10) delivered as a GeoTIFF, converted to degrees Celsius. (Chapter 3)

Laplace distribution
The probability distribution with density (1/2b) exp(-|r|/b); sharp-peaked at zero with exponential tails, and with mean absolute value equal to its scale b. (Chapter 41)

Last-wins
The rule that assigning to an existing dictionary key, or to an array element, overwrites the earlier value. (Chapter 39)

Law of total variance
Total variance equals the mean of the within-class variances plus the variance of the class means. (Chapter 29)

Layer normalisation (LayerNorm)
Standardising each token over its own features, then applying a learned scale and shift per feature; independent of the batch. (Chapter 13)

LayerNorm
Normalisation of one token across its own C features: subtract their mean, divide by their standard deviation, then apply a learned per-feature scale and shift; 2C parameters. (Chapter 16)

LayerNorm parameters
One learned scale and one learned shift per feature: 2c numbers. (Chapter 20)

Layout
The order in which a multi-index array is flattened into memory. (Chapter 3)

Lazy-import rule
Action modules may import app, torch or rasterio only inside functions, keeping the api's start-up light. (Chapter 4)

Leakage
A benchmark scene overlapping or neighbouring training data, so a model can score by memory. (Chapter 34)

Learned query
A query vector stored as a trained parameter rather than computed from the input; the spectral compressor has 32 of them. (Chapter 22)

Learning rate
The step-size multiplier eta in a gradient update; too small crawls, too large diverges. (Chapter 26)

Learning-rate schedule
A rule that changes the learning rate over the course of training. (Chapter 26)

Leave-one-scene-out
Cross-validation with scenes as units: each scene is reported at the setting chosen on the other scenes only. (Chapter 40)

Leave-one-spectrum-out
Matching a planted library spectrum against every other library spectrum, never itself, so the test asks whether the nearest other spectrum shares its family. (Chapter 32)

Level 1 by matching
The namer's natural-or-man-made answer since run N001b: the natural library entries pooled as an eighth class, and P(man-made) = sigmoid(LSE of the seven man-made class scores - the natural score + b), the share of softmax mass on man-made entries. (Chapter 44)

Lexicographic order
Sorting strings by comparing character codes from the left; a (97) before h (104) before s (115). (Chapter 39)

Likelihood
The probability a model gives to the observed labels, viewed as a function of the model's predictions. (Chapter 28)

Likelihood ratio
The probability density of the data under one hypothesis divided by that under another; RX is the generalised likelihood ratio for an unknown shift from a Gaussian background. (Chapter 7)

Linear bottleneck
A pair of linear maps through a narrow width; every output lies in a subspace whose dimension is at most that width. (Chapter 12)

Linear interpolation
Estimating a value between two samples on the straight line through them: y = (1 - t) y0 + t y1 with t the fraction of the way. (Chapter 5)

Linear interpolation (percentiles)
np.percentile's rule: the q-th percentile of n sorted values sits at position (n - 1) q / 100, interpolated between neighbours. (Chapter 36)

Linear layer
A layer computing y = W x + b; it has in times out plus out parameters. (Chapter 10)

Linear map
A map f with f(ax + cy) = a f(x) + c f(y); its sensitivity to each input is a constant, the same for every input vector. (Chapter 22)

Linear mixing model
A pixel covering fraction a of material t and 1 - a of background b reads (1 - a) b + a t. (Chapter 34)

Linear mixture
A synthetic pixel made as a fraction of a target spectrum plus the rest of the background spectrum. (Chapter 38)

Linear-interpolation percentile
numpy's default percentile: the value at position q/100 times (N - 1) in the sorted list, interpolated between neighbours. (Chapter 32)

Linearity of expectation
The expected value of a sum equals the sum of the expected values, whether or not the terms are independent. (Chapter 17)

Loader worker
A separate process that reads and decodes data while the GPU trains; each needs at least one shard. (Chapter 43)

Local mean
The average of a quantity over a ring of neighbouring pixels, used in place of the scene-wide mean. (Chapter 31)

Local RX
RX with the mean and covariance estimated separately for each pixel from a ring of its neighbours, so a pixel is judged against its surroundings rather than the whole scene. (Chapter 8)

Local z-score
A spatial post-filter that re-expresses each score against its neighbourhood's mean and spread; it lowered AUC in the comparison. (Chapter 31)

Local-mean Mahalanobis distance
The Mahalanobis distance measured from the ring mean but with the scene-wide covariance; the default with a 21/61 window. (Chapter 31)

Log parameterisation
Letting a network output s = log b and using b = exp(s), so b is always positive and gradients near zero stay moderate. (Chapter 41)

Log-odds
The logarithm of p/(1-p); it runs over all real numbers as p runs from 0 to 1. (Chapter 28)

Log-uniform distribution
A distribution whose logarithm is uniform; the namer's fill on [0.05, 1], with P(a <= q) = ln(q / 0.05) / ln 20 and mean 0.95 / ln 20 = 0.3171. (Chapter 44)

Logit
A raw real-valued score, read as the log-odds of yes; the sigmoid turns it into a probability. (Chapter 28)

Logsumexp
ln of the sum of exponentials, a smooth maximum; the namer's class score over a class's library cosines divided by tau. (Chapter 44)

Loss
A single number measuring how bad a model's output is; training lowers it. (Chapter 9)

Loss mask
The pixel prediction mask times the eroded validity mask: the pixels the loss is averaged over. (Chapter 18)

Machine epsilon
The relative rounding step of a floating-point format: about 1.2e-7 for float32 and 2.2e-16 for float64. (Chapter 7)

Machine epsilon (float32)
The relative spacing of float32 numbers near 1, 2 to the power -24 below 1; it makes cosines of angles under about 0.00024 rad round to exactly 1. (Chapter 25)

Mahalanobis distance
(x - mean)T inverse-covariance (x - mean): the squared distance from the mean after whitening, so each direction is measured in its own standard deviations. (Chapter 7)

Manhattan distance
Rows apart plus columns apart; the number of 4-neighbour steps between two pixels. (Chapter 34)

Manifest (current.json)
The file beside a model's weights naming which checkpoint is current and recording facts about it. (Chapter 4)

Manifest parameter figure
The count recorded with a checkpoint; it includes buffers, so it exceeds the trainable count (5,507,354 against 5,506,629). (Chapter 20)

Mann-Whitney count
The number of (target, background) pairs in which the target scores higher, ties counting half; divided by the number of pairs it equals the AUC. (Chapter 33)

Mask
An array of zeros and ones selecting positions; in masked reconstruction 1 means visible and 0 means hidden. (Chapter 9)

Mask ratio
The fraction of tokens hidden in a training step; 0.65 in Indradhanu's v0.2.0 training configuration. (Chapter 9)

Mask token
In the original masked autoencoder, a learned vector placed at hidden positions; this encoder uses zeros instead. (Chapter 17)

Masked autoencoder (MAE)
An autoencoder that hides part of its input and is graded only on the hidden part, so it cannot copy. (Chapter 9)

Masked mean
A mean over padded rows that sums only the real entries and divides by their count, not by the padded length. (Chapter 8)

Masked mean absolute error (L1 loss)
The mean of the absolute errors over all bands of all graded pixels; Indradhanu's brightness term, in reflectance units. (Chapter 25)

Masked softmax
A softmax in which invalid positions have their scores set to minus infinity, giving them weight exactly 0 and renormalising the rest. (Chapter 23)

Matched filter
The linear filter w = Sigma-inverse t / (t-transpose Sigma-inverse t) that maximises signal-to-noise for a known signal shape t in noise of covariance Sigma. (Chapter 40)

Matrix
A rectangular table of numbers; multiplying a vector by it forms one weighted mixture of the inputs per row. (Chapter 6)

Max invalid voxel fraction
The stage-6 threshold (default 0.4): a pixel invalid in more than this fraction of the surviving bands is invalidated in every band. (Chapter 5)

Max pooling
Sliding a window over a grid and outputting the maximum of each window; on a 0/1 mask it asks whether any 1 is in reach. (Chapter 18)

Max pooling as dilation
Taking the maximum of a binary mask over a sliding window, which grows every marked pixel into a square. (Chapter 42)

Max-normalisation
Dividing a map by its largest valid value; fragile, because one extreme pixel compresses all the others towards zero. (Chapter 30)

Maximum likelihood
Choosing the parameter value that makes the observed data most probable. (Chapter 41)

Maximum noise fraction
The MNF transform (Green et al. 1988): the directions ordered by noise fraction, equivalently by signal-to-noise ratio; often called minimum noise fraction. (Chapter 8)

max_kept leakage
When patches in a batch keep different numbers of tokens, every patch gathers the batch maximum, so patches that keep fewer let a few hidden tokens into the encoder. (Chapter 18)

McClish standardisation
A rescaling of the raw partial area that maps random to 0.5 and perfect to 1; not used by this repository. (Chapter 33)

Mean
The sum of a set of numbers divided by how many there are. (Chapter 1)

Mean squared error (MSE)
The mean of the squared errors; its gradient is proportional to the error, so large errors dominate it. (Chapter 25)

Mean vector
The band-by-band average of a set of spectra. (Chapter 6)

Mebibyte (MiB)
2 to the power 20 = 1,048,576 bytes; a gibibyte (GiB) is 1,073,741,824 bytes. (Chapter 23)

Median
The middle value of sorted observations; the constant prediction that minimises the sum of absolute errors. (Chapter 25)

Memory per extra patch
The slope of peak memory against batch size; it removes fixed costs such as weights and optimiser state from a per-patch comparison. (Chapter 24)

Memory wall
The point at which memory, not arithmetic or parameters, limits the batch size and speed of training. (Chapter 23)

MiB
A mebibyte, 2 to the 20 = 1,048,576 bytes; a float32 number takes 4 bytes. (Chapter 15)

Micro-batch
The batch of one forward pass (8 tiles here); BatchNorm statistics are computed per micro-batch. (Chapter 29)

min bg pixels
The least number of valid ring pixels needed to score a pixel locally, by default the number of bands plus one, because a covariance from n points has rank at most n - 1. (Chapter 8)

Min-max rescaling
Mapping a map's values to [0, 1] by subtracting its minimum and dividing by its range. (Chapter 32)

Minimum Noise Fraction (MNF)
A classical linear projection to few bands chosen by eigen-decomposition to maximise signal-to-noise ratio. (Chapter 12)

Mix-FFN
SegFormer's feed-forward network with a 3-by-3 depthwise convolution on the token grid between the expansion and the GELU. (Chapter 16)

Mixdown
The analogy for compression: each output channel is a learned blend of every input band. (Chapter 12)

Mixed background
A scene made of several distinct surfaces, whose global mean and covariance describe none of them well. (Chapter 7)

Mixed batch
A micro-batch holding tiles from every sensor, each sensor's sub-batch passed through the stem alone and then concatenated. (Chapter 29)

Mixed pixel
A pixel whose spectrum is a combination of several materials because each covers part of the ground it sees. (Chapter 32)

Mixed precision
Running the expensive parts of a network in a 16-bit floating-point format and the precision-critical parts, such as the loss, in float32. (Chapter 26)

Mixing line
The straight segment in spectral space from the background spectrum to the material spectrum, on which every two-part mixture lies. (Chapter 34)

MNF component
A pixel's coordinate along an MNF direction; MNF components are uncorrelated, with variance 1 + SNR. (Chapter 8)

MNF eigenvalue diagnostic
Reading the MNF eigenvalues: values well above 1 are signal components, values near 1 are noise; a flat list means no dominant signal. (Chapter 8)

MNF-RX
RX computed after a Minimum Noise Fraction transform into a smaller, better-conditioned space; the shipped classical hyperspectral detector. (Chapter 1)

Mode scale
The fine-tuning trainer mode that trains only the uncertainty head with the Laplace NLL and ranks epochs by that NLL. (Chapter 41)

Model-free baseline
A detector with no learned model, run beside the models to check that a benchmark rewards what a model adds. (Chapter 35)

Momentum
An optimiser that steps along a running sum of past gradients, so consistent directions accumulate and alternating ones cancel. (Chapter 26)

Momentum (BatchNorm)
The weight m given to the newest batch statistic when updating the running statistics; 0.1 by default. (Chapter 12)

mRMSE
Mean root-mean-square error of the abundance maps after matching. (Chapter 49)

mSAD
Mean spectral angle, in radians, between matched found and true endmembers. (Chapter 49)

Multi-head attention
Several attentions run side by side on separate slices of the features, concatenated and mixed by an output projection. (Chapter 14)

Multi-label
A classification problem in which any number of classes can be true at once, such as thin cloud over water. (Chapter 28)

Multiply-add
One multiplication followed by one addition; the unit in which convolution work is counted. (Chapter 19)

Multiply-add (MAC)
One multiplication followed by one addition, the unit used here to count the work of matrix products and convolutions. (Chapter 15)

Multiply-adds
The work of a layer: its weights per output position times the number of positions. Not the same as its parameter count. (Chapter 20)

Multivariate Gaussian
The Gaussian over vectors, set by a mean vector and covariance matrix; its exponent is minus half the Mahalanobis distance. (Chapter 7)

NaN
Not a number: the floating-point result of 0/0, produced by a softmax over all-masked scores unless guarded against. (Chapter 23)

Nanometre
One billionth of a metre (10 to the power -9 m); the unit of wavelength used throughout the codebase. (Chapter 2)

Native band list
A sensor's own band centres and pixel grid, untouched by interpolation. (Chapter 27)

Native bands
The wavelengths a sensor actually measured, as opposed to the 165-band common grid; the band-filter default since 2026-10-31. (Chapter 32)

NDSI
Normalised difference snow index, (green - SWIR)/(green + SWIR); high for snow, which is dark near 1600 nm where cloud stays bright. (Chapter 28)

NDVI
Normalised difference vegetation index, (NIR - red) / (NIR + red) or with a visible mean; positive for vegetation, negative for water. (Chapter 35)

Near infrared (NIR)
Wavelengths from about 700 to 1000 nm, immediately beyond red; vegetation reflects strongly here. (Chapter 2)

Near weight
The factor (5 in FT05) applied to hidden pixels within 3 px of an implant in the loss mask; it changes shares of a weighted mean, not its size. (Chapter 38)

Nearest-valid fill
Replacing each invalid pixel's values with those of the nearest valid pixel by Euclidean distance, leaving the validity masks unchanged. (Chapter 5)

needs_threshold
The status in which anomaly_detection_prep waits for a person to choose and commit a threshold. (Chapter 4)

Negative (background)
A valid pixel the ground truth does not mark as anomalous. (Chapter 33)

Negative log-likelihood
Minus the logarithm of the likelihood; minimising it is the same as maximising the likelihood, and it turns products over pixels into sums. (Chapter 41)

Neighbouring checkpoints
Consecutive epochs' weights; a gain counts only if it holds at all three compared. (Chapter 43)

Neural network
A function built from many simple layers whose behaviour is set by a large list of learned numbers called parameters. (Chapter 1)

Neuron
A weighted sum of inputs plus a bias, usually followed by an activation function. (Chapter 10)

Nodata
The value -32768 in a shard, marking a voxel with no valid reading; the stem skips it rather than reading it as zero. (Chapter 27)

Noise fraction
Along a direction, the share of the total variance that is noise. (Chapter 8)

Noise whitening
Transforming data so the noise covariance becomes the identity, the first step of MNF. (Chapter 8)

Non-overlapping windows
Convolution windows with kernel equal to stride and no padding, so each pixel feeds exactly one token; Indradhanu's stage 1. (Chapter 13)

Norm
The length of a vector, the square root of the sum of its squared entries (Pythagoras in many dimensions). For a spectrum, roughly its brightness. (Chapter 6)

Normalised convolution
Blurring data with nodata set to 0 and dividing by the blurred validity mask, so missing pixels do not drag values down. (Chapter 40)

np.allclose
NumPy's test that two arrays agree within atol + rtol times the size of each value (defaults 1e-8 and 1e-5). (Chapter 39)

np.interp
NumPy's one-dimensional linear interpolator, np.interp(x, xp, fp); it requires the sample positions xp to be increasing and does not check. (Chapter 37)

Null space
The set of input directions a matrix sends to zero; changes along them are invisible after the matrix. (Chapter 12)

Numerical guard
A clamp or small added constant (epsilon) that keeps a computation away from a division by zero, a negative square root or an infinite derivative. (Chapter 25)

Object
A 4-connected component of the ground-truth mask. (Chapter 33)

Object AUC
The share of background pixels an object's best-scoring pixel outranks. (Chapter 49)

Object bootstrap
Resampling labelled objects rather than pixels, because an object's pixels are not independent evidence. (Chapter 36)

Object measure
A detection measure that counts each object once, such as objects caught at the top 1 percent. (Chapter 40)

Object pAUC parts
The pair (sum of c over an object's pixels, its pixel count) from which a bootstrap rebuilds pAUC. (Chapter 36)

Objects caught
The share of objects with a pixel within one pixel of the top q percent of valid scores. (Chapter 33)

Onboarding vendable
The vendable built at upload, with the sensor's native bands. (Chapter 4)

Open-set recognition
Recognition that may meet classes it never saw in training and say so; here, level 1 calls man-made whatever its model of nature cannot explain. (Chapter 32)

Opt-in model
A model available to choose but not used by any default. (Chapter 39)

Ordering test
Acceptance of a benchmark only if it reproduces a ranking the real labels already showed: fine-tuned models above a model-free PCA-RX. (Chapter 34)

Orthogonal
At right angles; two vectors are orthogonal when their dot product is zero. (Chapter 6)

Orthogonal matrix
A square matrix whose columns are orthogonal unit vectors; it rotates or reflects without stretching, and its inverse is its transpose. (Chapter 6)

Orthogonal projection
The closest point of a span to a vector; the leftover is at right angles to the span. (Chapter 42)

Outer product
The matrix d dT whose entry (i, j) is the product of entries i and j; averaging outer products of deviations gives the covariance. (Chapter 6)

Output projection
The final C-by-C linear layer of an attention module, which lets the separate heads' outputs interact. (Chapter 14)

Output-size formula
For kernel k, stride s and padding p, an axis of length H becomes floor((H + 2p - k)/s) + 1. (Chapter 10)

Overall accuracy
The share of pixels whose predicted (or cluster-mapped) class equals their true class. (Chapter 46)

Overfitting
Learning the training examples rather than the pattern: training loss falls while validation loss rises. (Chapter 43)

Overlap
The number of pixels two neighbouring tiles share along an axis, patch size minus stride: 96 for a 128 tile at stride 32. (Chapter 30)

Overlap averaging
Combining the predictions of all tiles that cover a pixel by summing them, weighted by the eroded validity mask, and dividing by the sum of weights. (Chapter 30)

OverlapPatchEmbedding
The repository's patch-embedding module: Conv2d, flatten, transpose, LayerNorm; returns the tokens and the grid height and width. (Chapter 13)

Overlapping windows
Convolution windows with kernel larger than stride, so neighbouring tokens share pixels; Indradhanu's stages 2 to 4. (Chapter 13)

Overshoot
An interpolated curve passing below the lowest or above the highest nearby sample. (Chapter 5)

P(B>A)
The share of bootstrap draws in which scorer B's measure beat scorer A's. (Chapter 36)

p*
The percentile bar 1 - alpha / F for giving a man-made name, chosen so that an ordinary pixel gets any man-made name with probability at most alpha. (Chapter 32)

Padding
A border of zeros added around the input before a convolution. (Chapter 10)

Paired bootstrap
A bootstrap that applies the same draw to both scorers being compared, so that which objects were drawn cancels in the difference. (Chapter 36)

Parameter
One adjustable number inside a model, set by training. (Chapter 1)

Parameter count
The total number of parameters; 5,506,629 for the 165-band Indradhanu v0.2.0. (Chapter 20)

Parity run
Running the shipped checkpoint through the product's own code path on benchmark scenes and comparing with the evaluator's numbers. (Chapter 39)

Partial AUC (pAUC)
The area under the ROC from FPR 0 to a limit, here divided by the limit so that perfect is 1 and random is half the limit (0.005 at 1%). (Chapter 33)

Partition
A split of a set into pieces that do not overlap and together cover the whole set; at inference, the hidden token sets of the passes form a partition. (Chapter 30)

Pass criterion (gate)
A condition a run must meet, written in its README before launch and not moved afterwards. (Chapter 38)

Patch
A small square cut from a scene, here 128 by 128 pixels with every band, the unit a training batch is built from. (Chapter 27)

Patch embedding
The operation that cuts a feature map into blocks and computes one vector, a token, per block; in this model a strided convolution followed by LayerNorm. (Chapter 13)

Patience
The number of epochs without sufficient improvement after which early stopping ends a run. (Chapter 27)

PCA-RX
The evaluation's classical baseline: whiten the top 30 principal components of the cube with a global covariance and score the squared distance from a ring mean (full widths 21 and 61). (Chapter 8)

PCHIP
Piecewise cubic Hermite interpolating polynomial: cubic pieces with shape-preserving slopes, never creating an extremum absent from the data; used for stage-7 gap filling. (Chapter 5)

Per-band affine map
A map that multiplies each band by its own number and adds its own number; min-max and the fixed z-score are both of this form. (Chapter 47)

Per-band normalisation
Rescaling each band by its own corpus mean and standard deviation (PixelNormalize), done inside the model. (Chapter 5)

Per-curve scaling
Rescaling one spectrum by its own minimum and maximum over its bands; used in the HyperSL matching run so that pixel and library were treated alike. (Chapter 47)

Per-pixel contribution c(p)
max(0, m - FPR(p)) / m, with FPR(p) the share of background scoring at or above target pixel p; its mean over target pixels is the pAUC. (Chapter 36)

Per-pixel gate
SpectralGate: a small network computes one sigmoid volume per band from the pixel's own spectrum before the fixed 1x1 compressor; 13,162 parameters at 202 bands. (Chapter 24)

Per-scene min-max normalisation
Rescaling each scene by its own minimum and maximum; avoided because it destroys absolute reflectance. (Chapter 24)

Per-scene min-max scaling
Rescaling each band so that the scene's darkest pixel in that band is 0 and its brightest 1: x' = (x - min) / (max - min), with min and max taken over the scene. HyperSL's input scaling. (Chapter 47)

Per-scene override
Replacing a model's stored normalisation mean and spread with the scene's own (PixelStatsOverride); meant for HotSat-1, but never built, and overwritten by the checkpoint if it were. (Chapter 21)

Percentile interval
The 2.5th to 97.5th percentiles of the bootstrap values: a 95 percent interval. (Chapter 36)

Percentile threshold
A cut that keeps a fixed fraction of a scene's highest scores, so it adapts to the scene's own score scale. (Chapter 1)

Period
The distance, in nm, after which a sine or cosine repeats; the encoding uses five periods per decade. (Chapter 22)

Permutation
A reordering of the indices 0 to N-1 in which each index appears exactly once. (Chapter 18)

Permutation equivariance
The property that reordering the input tokens reorders the outputs the same way and changes nothing else; plain attention has it, so it cannot tell where a token is. (Chapter 14)

Permutation invariance
The property that reordering the keys and values (together) does not change an attention output. (Chapter 22)

Pickle
Python's object serialisation; stores the class's import path, which makes class names a wire format. (Chapter 4)

Pixel
One cell of an image grid, standing for a patch of ground; named by its row and column, counted from 0. (Chapter 1)

Pixel centre
The middle of a pixel, half a pixel in from its edges; PRISMA's corner attributes give the corner pixels' centres. (Chapter 45)

Pixel checkerboard
The inference mask of the convolutional models: one pass hides pixels whose row plus column is odd, the other those where it is even. (Chapter 11)

Pixel measure
A detection measure that counts target pixels, such as pAUC or TPR; a large object weighs more than a one-pixel target. (Chapter 40)

Pixel prediction mask
The token prediction mask copied to each token's 4 by 4 pixels by nearest-neighbour upsampling. (Chapter 18)

Pixel shuffle
A parameter-free rearrangement from (B, C r squared, H, W) to (B, C, rH, rW): out[c, rh + i, rw + j] = in[c r squared + r i + j, h, w]. (Chapter 19)

Pixel validity mask
A (B, 1, H, W) array of ones and zeros: 1 where the pixel is a real measurement, 0 for cloud, no-data or swath edge. (Chapter 18)

Pixel with no valid band
A pixel whose every band is masked; the code lets it attend to everything and then sets its output to exactly 0. (Chapter 23)

Pixel-shuffle bottleneck
The decoder's step that turns one quarter-resolution feature vector into a 4 x 4 block of pixels, so sub-block detail must be carried in that one vector. (Chapter 38)

Pixels as a batch
Treating each pixel of a patch as an independent row (16,384 rows per 128 x 128 patch) because nothing in the spectral stem moves information between pixels. (Chapter 23)

PixelStatsOverride
Per-scene replacement normalisation statistics for sensors whose units differ from the model's training data; makes scores scene-relative. (Chapter 5)

Placement bias
A benchmark bias in which synthetic targets are placed on surfaces where real targets do not occur, so a scorer can win on the benchmark and lose on real scenes. (Chapter 40)

Plain RX
The Mahalanobis distance of each pixel of the raw cube over all bands; dropped from the product in 2026-05 because its near-singular covariance gave distances near 1e11. (Chapter 31)

Planted-sample generator
The namer's source of training data: x = g (a t + (1 - a) b) + n, a library spectrum mixed into a real pixel with a gain curve and noise. (Chapter 44)

Plateau
A stretch of training in which further epochs no longer change the measured result beyond noise. (Chapter 43)

Plausible cloud
A labelled cloud pixel that is bright (visible mean above 0.2) and flat (min/max of four window means at least 0.7). (Chapter 28)

Point-spread function (PSF)
The image of a single bright point; here a Gaussian of standard deviation 1.4 pixels. (Chapter 34)

Position code
A fixed vector added to each token to say where it sits; HyperSL's is sin and cos of (wavelength - 400) / 10 at 128 frequencies 100 to the power -2j/256. (Chapter 46)

Positional encoding
A vector added to each token to tell a transformer where the token sits; Indradhanu has none and gets position from a depthwise convolution instead. (Chapter 14)

Positive (target)
A valid pixel the ground truth marks as anomalous. (Chapter 33)

Positive semi-definite
A symmetric matrix with no negative eigenvalues; every covariance matrix is one, because its eigenvalues are variances. (Chapter 6)

Post-norm
Placing LayerNorm after the residual addition, LN(x + f(x)), as in the original 2017 transformer. (Chapter 16)

Pratibimba
The plain convolutional autoencoder (reflection): four stages, base 32, trained with nothing hidden; 1,379,393 parameters. (Chapter 11)

Pre-norm
Placing LayerNorm inside the residual branch, x + f(LN(x)), so that the residual path is never normalised. (Chapter 16)

Precision
For a class, the share of the pixels the classifier calls it that are labelled with it: diagonal entry over column total. (Chapter 50)

Precondition
A condition the inputs of a function must satisfy for its contract to hold; when it is violated, the function promises nothing. (Chapter 37)

Prediction mask
The (B, N) array of hidden tokens, the ones the model is graded on. Only valid tokens can be in it. (Chapter 18)

Prep bundle
The export of a committed anomaly_detection_prep: a 0/1 mask GeoTIFF, a point shapefile without a score attribute, a CSV and a manifest, named hyper_ or thermal_ by sensor; the only bundle fit for submission. (Chapter 45)

Pretraining
Training a model on a large general data set before adapting it (fine-tuning) to the data and tasks of the product. (Chapter 27)

Principal component analysis
Finding the orthogonal directions of greatest variance of a cloud of vectors: the eigenvectors of its covariance, ordered by eigenvalue. (Chapter 6)

PRISMA
Italian Space Agency hyperspectral satellite; 239 bands (66 VNIR, 173 SWIR) delivered as one HDF5 (.he5) file in BIL layout. (Chapter 3)

Probability density
A curve (or surface) whose area (or volume) over a region is the probability of landing in that region. (Chapter 7)

Probe
Training only a small head on frozen features, to measure what the features already know. (Chapter 29)

Product grid (PRISMA)
The UTM grid of a PRISMA L2D product, read from its EPSG code and the corner pixels' centres: pitch = span / (n - 1), origin = upper-left centre minus half a pixel. (Chapter 45)

Product parity
Checking that the product's code path reproduces the evaluator's scores on the same scenes. (Chapter 43)

Progressive decoder
The FT09 decoder: SegFormer fusion at 1/4 resolution, then two x2 upsampling stages that each see a masked skip of the stem output. (Chapter 42)

Project
A piece of work bound to exactly one Scene; it holds that work's Actions. (Chapter 4)

Projection
The multiple of a direction vector closest to a given vector, found by minimising the squared distance; for a unit direction u it is (u.x) u. (Chapter 6)

Prototype
A stored example in embedding space that an input is compared with; under level 1 by matching every library entry is one. (Chapter 44)

Pseudo-label
A training label produced by a model (the teacher) rather than a person. (Chapter 38)

PSF halo
The part of a small target's light that the sensor's point-spread function spreads into neighbouring pixels. (Chapter 41)

QA PIXEL
Landsat's bit-packed quality layer; the builder rejects pixels with the fill, dilated cloud, cloud, cloud shadow or high-cirrus bits set. (Chapter 3)

QR decomposition
Writing a matrix as an orthonormal-column matrix Q times a triangular R; Q spans the same space. (Chapter 42)

Quality layer
A per-pixel mask the provider's processor computes with automatic tests, such as EnMAP's cloud or cirrus layer; not drawn by people. (Chapter 28)

Quantile
The value below which a given fraction of a list lies; torch.quantile interpolates linearly between neighbouring sorted values. (Chapter 42)

Query
The learned projection of a token that expresses what it is looking for; compared against every key. (Chapter 14)

Quotient rule
The derivative of a/b is (a'b - ab')/b squared. (Chapter 25)

R-theta branch
The unconstrained convolutional path in the HyperSL unmixing paper's reconstruction; it carried 29-56 percent of the reconstruction energy. (Chapter 49)

Radiance
Power of light leaving a surface in one direction, per square metre, per steradian and per nanometre; what a sensor measures. (Chapter 2)

Ramp
The schedule lambda(t) = lambda-max times min(1, t/R): a linear rise over R epochs, held constant within each epoch, then flat. (Chapter 25)

Ramped weight
A loss weight that grows over the first epochs (here the SAM weight, from 0 to 0.5 over 10 epochs); fine for training, wrong for ranking epochs. (Chapter 27)

Random inference mask
The alternative strategy: a random half of the tokens and its exact complement, which scatters token-boundary artefacts differently in every tile. (Chapter 30)

Rank
The number of independent directions a matrix's outputs fill; for a covariance, the number of non-zero eigenvalues. (Chapter 8)

Rank (of a matrix)
The number of linearly independent columns of a matrix; the dimension of the set of vectors it can produce. (Chapter 12)

Rank (of a tensor)
The number of axes a tensor has. (Chapter 10)

Rank fusion
Combining several rank maps pixel by pixel by their mean (needs agreement) or their maximum (lets either scorer's false alarms through). (Chapter 40)

Rank map
A score map converted to the fraction of valid pixels scoring at or below each pixel, so scorers with different scales can be combined. (Chapter 40)

Rank of a matrix
The dimension of the set of outputs a linear map can produce; a map through 32 numbers has rank at most 32. (Chapter 42)

Rank-nullity theorem
For a matrix with N columns, rank plus the dimension of the null space equals N. (Chapter 22)

read_bands
The inferencer helper that keeps only bands within 15 nm of the trained-wavelength record and tells the model those wavelengths. (Chapter 39)

Reading
How a model reads a scene's bands: its own native bands by wavelength, or a resample onto a fixed grid (product165, se202). (Chapter 35)

Reading path
The chain of code that turns a stored scene into the cube a model reads; two paths for the same scene must give the same cube. (Chapter 37)

Realised contrast
The contrast z measured on the object after implanting, including the spot's own natural difference from its ring. (Chapter 34)

Reaper
The routine that marks running jobs failed when their heartbeat is older than a timeout. (Chapter 4)

Recall
For a class, the share of the pixels labelled with it that the classifier also calls it: diagonal entry over row total. (Chapter 50)

Recall at a fixed false-alarm rate
The share of man-made plantings above the cut that calls a given share of natural plantings man-made; the namer's level-1 pass test uses ACE's rate. (Chapter 44)

Receptive field
The region of input pixels that can affect one output value. (Chapter 10)

Reconstruction
The model's output, written x hat; its estimate of the recorded cube. (Chapter 9)

Reconstruction model
A function from an input cube to an output cube of the same shape, trained so that its output, the reconstruction, is close to its input on ordinary data. (Chapter 9)

Red edge
The sharp rise in vegetation reflectance between about 680 and 760 nm, caused by chlorophyll absorbing red and leaf structure scattering near-infrared light. (Chapter 2)

Reduced stem
The reduced-attention stem: attention over fixed 40 nm wavelength bins plus three per-band paths; 14,721 compressor and 4,257 decoder parameters. (Chapter 24)

Reduction LayerNorm
The LayerNorm applied to the summary tokens before they are projected into keys and values. (Chapter 15)

Reduction ratio
The factor R by which ESA shrinks the token grid in each direction before forming keys and values; 8, 4, 2 and 1 at the four stages. (Chapter 15)

Refine convolutions
The decoder's two 3 by 3 convolutions at quarter resolution, its only spatial layers; the second outputs 512 channels and has no activation. (Chapter 19)

Reflectance
The fraction of arriving light that a surface returns at a given wavelength; unitless, usually between 0.02 and 0.30 for real ground. (Chapter 2)

Regular expression
A text pattern, such as asphalt|concrete, matched against names; the family library's curation assigns families with them, first match wins. (Chapter 32)

Release candidate
The model proposed for shipping; here FT04 with cloud exclusion on its own masks. (Chapter 38)

ReLU
Rectified Linear Unit, max(0, x). (Chapter 16)

Remove tokens
Deleting hidden tokens from the sequence by a gather, so the encoder's blocks never see them. (Chapter 18)

Renormalisation
The rescaling of surviving softmax weights so they again sum to 1 after some positions are masked; ratios between survivors are unchanged. (Chapter 23)

Resample matrix
The (G, C) matrix that maps a spectrum on C scene bands to G grid wavelengths; its b-th column is the interpolation of the b-th unit vector. (Chapter 37)

Resampled band list
Values interpolated onto a band list other than the sensor's own, such as the 165-band grid. (Chapter 27)

Resampling
Estimating a spectrum's values at a fixed list of wavelengths from its values at the sensor's own wavelengths. (Chapter 3)

Reshape
An operation that changes a tensor's shape while keeping its numbers in the same memory order. (Chapter 10)

Residual
The recorded value minus the reconstructed value, x minus x hat, one number per band per pixel. (Chapter 9)

Residual (error)
The difference between the reconstruction and the truth, band by band, at one pixel. (Chapter 25)

Residual connection
Adding a sublayer's output to its input, x + f(x), so that the derivative is 1 + f' and an untouched path always exists; also called a skip connection. (Chapter 16)

Residual covariance
The covariance matrix of the residuals of a scene's valid pixels, estimated in the product from up to 200,000 pixels with a fixed seed. (Chapter 31)

Residual Mahalanobis distance
The squared Mahalanobis distance of a pixel's residual from the scene's mean residual, using the residual covariance; Indradhanu's default score since 2026-10-15. (Chapter 31)

Resolved gain
A difference whose 95 percent paired-bootstrap interval lies wholly above zero; the repository's rule since 2026-10-21. (Chapter 36)

Resolver
The static table of what each model architecture can do, combined with its manifest to build the model picker. (Chapter 4)

Responsibility
The probability that a pixel came from one mixture component, given the pixel, by Bayes' rule. (Chapter 40)

Responsibility-weighted Mahalanobis distance
The sum over components of each responsibility times the pixel's squared Mahalanobis distance to that component. (Chapter 40)

Restore tokens
Putting the encoded kept tokens back at their positions by a scatter into zeros; hidden positions stay zero. (Chapter 18)

Resume mode
Whether a checkpoint restores the full training state (resume) or only the weights with a fresh optimiser and schedule (finetune). (Chapter 26)

Ring
The pixels inside the outer window but outside the guard: 61 x 61 minus 21 x 21 = 3,280 pixels by default. (Chapter 31)

Ring fallback
The rule that a pixel whose ring holds fewer than 2 d valid pixels (60 at d = 30) is scored against the global mean instead. (Chapter 31)

Ring mean
The mean of the valid pixels at Chebyshev distance 3 to 7 from a pixel (15 x 15 box minus 5 x 5), 200 pixels in the interior. (Chapter 44)

Robust loss
A loss whose pull from any single observation is bounded, so outliers cannot dominate it; L1 is robust, squared error is not. (Chapter 25)

ROC curve
The path of (FPR, TPR) points as the threshold slides from the top score to the bottom score. (Chapter 33)

Rotation
Swapping one block of training data for the next while the learning-rate schedule and optimiser state continue unbroken. (Chapter 27)

Row-major order
Memory order in which the last index changes fastest; offset of (a, b, d) in shape (A, B, D) is a B D + b D + d. (Chapter 3)

Running mean
BatchNorm's stored estimate of a channel's mean, updated in training mode as 0.9 x old + 0.1 x batch mean. (Chapter 29)

Running statistics
BatchNorm's exponential moving averages of batch mean and variance, kept as buffers and used in evaluation mode. (Chapter 12)

RX detector
The Reed-Xiaoli detector: scores each pixel by its Mahalanobis distance from the scene mean. (Chapter 1)

SAM dark fade
An opt-in patch that multiplies the spectral angle by clip(mean reflectance / t, 0, 1) so that dark pixels such as open water do not dominate; superseded by residual Mahalanobis. (Chapter 31)

SAM weight (lambda)
The multiplier on the spectral-angle term; it both schedules and converts radians into the scale of reflectance. (Chapter 25)

Sample-weighted average
An average over batches in which each batch's mean is weighted by the number of samples that survived filtering. (Chapter 26)

Sampled ROC
The product's ROC from evenly spaced percentiles of the scores, at most 256 of them (compute_roc). (Chapter 33)

Sampling distribution
The spread of values a statistic would take across many fresh datasets of the same kind. (Chapter 36)

Saturation (softmax)
The state in which one score is so far above the others that it takes almost all the weight and the softmax's derivatives are almost zero. (Chapter 14)

Scale and shift (gamma, beta)
BatchNorm's two learned parameters per channel, applied after standardising. (Chapter 12)

Scale floor
The per-scene lower limit on b (its 5th percentile by default) that stops very confident pixels from inflating the score. (Chapter 41)

Scale invariance
The property that multiplying a spectrum by a positive constant leaves its spectral angle to any other spectrum unchanged. (Chapter 25)

Scale invariance (of residual Mahalanobis)
The property that shrinking every residual by the same factor leaves every score unchanged, because the fitted covariance shrinks by the square of that factor. (Chapter 38)

Scale parameter
A positive number that stretches a distribution horizontally; for the Laplace it equals the mean absolute error. (Chapter 41)

Scaled dot-product attention
softmax of Q times K-transpose divided by the square root of the head dimension, multiplied by V. (Chapter 14)

Scaled RM
The scorer scaled_rm: residual Mahalanobis on the residual divided by the head's floored b (or its square root). (Chapter 41)

Scatter
Writing values into named positions of a starting array; unnamed positions keep their starting value. (Chapter 18)

Scene
One complete satellite acquisition: one patch of ground at one moment, as one file or folder. (Chapter 1)

Scene (product)
One uploaded raw satellite acquisition, onboarded into a vendable. (Chapter 4)

Scene adaptation
Briefly retraining a pretrained model on the scene being scored, then restoring its weights. (Chapter 40)

Scene fingerprint
An identity of a scene's source files (file size and time, or SHA-256s) included in the cache key since commit 229d423. (Chapter 35)

Scene-level uncertainty
Luck in which scenes are on the bench; not covered by the object bootstrap, so its intervals are lower bounds. (Chapter 36)

Scene-major loop
Loading one scene at a time and running every model on it before the next scene, so memory holds one scene and one reconstruction. (Chapter 35)

Scope
The set of surfaces and objects the product is meant to examine; excluding a class removes it from scope. (Chapter 35)

Score map
The anomaly scores laid out on the pixel grid, one number per pixel. (Chapter 1)

Score tensor
The tensor of query-key scores, shape (B, h, N, N / R-squared); its size sets the memory cost of attention. (Chapter 15)

Score-map cache
Saved score maps under a key built from the scene, the model file and the scorer settings, so new measures need no model run. (Chapter 35)

Score-map correction
Any post-processing applied to a finished anomaly score map, such as ranking each pixel within its edge-strength bin; none beat plain residual Mahalanobis. (Chapter 38)

Scorer
A rule turning a cube and its reconstruction into a score map, such as residual Mahalanobis or exclude. (Chapter 35)

SDPA row chunking
Splitting the pixel rows of a fused-attention call into chunks of at most 32,768 to stay under CUDA's 65,535 launch-grid limit; exact because pixels are independent. (Chapter 23)

See-saw
FT02's per-sensor validation swinging between epochs, one sensor improving while others worsen, consistent with BatchNorm drift under sensor-pure batches. (Chapter 29)

SegFormer
A hierarchical vision transformer with four scales and a light fusion decoder; the architecture behind Chakshu and Indradhanu. (Chapter 9)

SegFormer block
The repeating unit of the encoder: pre-norm efficient self-attention with a residual, then pre-norm Mix-FFN with a residual; shape (B, N, C) in and out. (Chapter 16)

SegFormer-B0
The smallest published SegFormer configuration, with stage widths 32, 64, 160 and 256, which this encoder follows. (Chapter 17)

Segmentation
Giving every pixel a label (water, cloud, ...) rather than one label per image. (Chapter 28)

Segmentation head
A small network on the shared encoder's features that outputs one logit per class per pixel; in Indradhanu a SegFormerDecoder of width 128 with 5 outputs, 371,536 parameters. (Chapter 28)

Segmentation weight
The factor seg_weight multiplying the segmentation loss in joint training; 0.1 in FT03. (Chapter 29)

segment_mixed
The segmentation counterpart of forward_mixed: one unmasked pass over the whole mixed batch, so BatchNorm sees every sensor. (Chapter 29)

Selection effect
A shift in a population caused by which members survive a filter, not by any change to the members themselves. (Chapter 37)

Selection matrix
A matrix with one 1 per row that performs a gather by multiplication; its transpose performs the matching scatter into zeros. (Chapter 18)

Selection risk
The optimism that enters when a variant is chosen after seeing results on the same data that then grades it. (Chapter 41)

Self-attention
Attention in which queries, keys and values all come from the same set of tokens. (Chapter 14)

Self-weight
How much a pixel contributes to its own reconstruction; a self-weight w shrinks every residual by the factor 1 minus w. (Chapter 9)

Sensor share
The fixed probability (or sub-batch fraction) with which each sensor is drawn: EnMAP 0.40, PRISMA 0.35, AVIRIS-NG 0.25. (Chapter 29)

Sequence layout
The arrangement (B, N, C): a list of N tokens of C numbers each, used by attention, LayerNorm and linear layers. (Chapter 10)

set_wavelengths
The call that tells a spectral-stem model which wavelengths its next input holds; forgetting it leaves the previous scene's wavelengths in place. (Chapter 39)

Shape
The list of sizes of an array's axes, such as (C, H, W). (Chapter 2)

Shape-preserving model
A model whose output has exactly the input's shape, so that the two can be subtracted to give a residual. (Chapter 19)

Shard
A tar archive packing many patches as named members, streamed sequentially; the webdataset convention. (Chapter 26)

Shard rotation
Training on a block of shards for a few epochs, then moving to a fresh block (Heavy: 200 shards every 5 epochs). (Chapter 43)

Shift difference
Estimating the noise covariance as half the covariance of differences between neighbouring pixels, assuming neighbours share the same signal. (Chapter 8)

Shift invariance
The property of the wavelength encoding that the similarity of two tags depends only on the difference of their wavelengths. (Chapter 22)

Shortwave infrared (SWIR)
Wavelengths from about 1000 to 2500 nm, where water, clays and minerals have absorption features. (Chapter 2)

Shrink
One minus the ring's mean PSF weight (0.676 for a one-pixel object at sigma 1.4): the share of an implant's signal that survives as contrast against the ring. (Chapter 34)

Shrinkage
Blending a covariance with its average eigenvalue times the identity, pulling all eigenvalues towards their mean; not used in this repository. (Chapter 7)

Siamese network
A network that runs two inputs through the same weights and compares the outputs; the

namer's stem is shared this way by the pixel, its ring mean and every library spectrum. (Chapter 44)

Sigmoid
The function 1 / (1 + exp(-z)), which maps any number into (0, 1) and never reaches either end. (Chapter 24)

Sign function
The function returning +1 for a positive number, -1 for a negative one and 0 for zero; the derivative of the absolute value away from zero. (Chapter 25)

Sign test
A test that counts in how many paired comparisons one method wins; with four scenes the smallest possible two-sided p-value is 0.125. (Chapter 31)

Signal-to-noise ratio
Along a direction, the signal variance divided by the noise variance. (Chapter 8)

Signal-to-noise ratio (SNR)
The filtered signal divided by the spread of the filtered noise, w-transpose t over the square root of w-transpose Sigma w. (Chapter 40)

Signature implant
An additive implant x + a d, with d a real object's difference from its ring. (Chapter 34)

Singular matrix
A square matrix with determinant zero, which has no inverse; a covariance is singular when the data do not vary at all along some direction. (Chapter 7)

Singular value decomposition
The factorisation X = U S V-transpose; the columns of V are the eigenvectors of X-transpose X, with eigenvalues the squared singular values. (Chapter 38)

Size score
Any score that measures how large a residual is (L1, MSE, SAM, combined), regardless of its direction. (Chapter 31)

SKIP LOCKED
A Postgres row-locking clause that lets several workers claim different queued jobs without blocking or double-claiming. (Chapter 4)

Skip path
The reduced stem's per-band route, the mean over valid bands of each value times W(lambda), straight to the 32 channels: a 1x1 compressor whose columns are computed from wavelength. (Chapter 24)

Sliding window
Covering a large image with fixed-size tiles placed at regular offsets, here 128 x 128 tiles every 32 pixels. (Chapter 30)

Slope
Rise divided by run of a straight line: (y1 - y0)/(x1 - x0). (Chapter 5)

Snapped tile
The last tile along an axis, moved back so that it ends exactly at the scene edge; it overlaps its neighbour by more than the usual overlap. (Chapter 30)

Soft Dice loss
One minus a differentiable Dice coefficient computed from probabilities, with +1 smoothing; it gives each class a full-sized term however rare. (Chapter 28)

Softmax
The function that turns a list of scores into positive weights summing to one, by exponentiating each score and dividing by the total. (Chapter 14)

Span
All weighted sums of a set of vectors. (Chapter 42)

Spatial completion
Predicting a pixel's spectrum from other pixels, its neighbours; how Indradhanu reconstructs a hidden pixel. (Chapter 48)

Spatial layout
The arrangement (B, C, H, W): a picture with C channels, used by convolutions. (Chapter 10)

Spatial matched filter
A blur shaped like a target's point-spread footprint, applied to the residual before whitening (scoring-gains idea A1). (Chapter 40)

Spatial-reduction attention
The general name, from the Pyramid Vision Transformer, for attention whose keys and values are computed on a downsampled grid. (Chapter 15)

Spearman rank correlation
The Pearson correlation of two lists' ranks; 1 for the same order, -1 for the reverse, near 0 for unrelated orders. (Chapter 41)

Spectral angle
The angle between two spectra treated as vectors, arccos of their dot product over the product of their lengths, in radians; it measures shape and ignores brightness. (Chapter 25)

Spectral angle mapper
Matching by the angle between a pixel's spectrum and a library spectrum, which ignores overall brightness. (Chapter 32)

Spectral Angle Mapper (SAM)
The remote-sensing method that classifies a spectrum by its spectral angle to library spectra; in this book also the name of the angle used as a loss. (Chapter 25)

Spectral compressor
Indradhanu's first learned module: a 1x1 convolution from 165 bands to D = 32 channels followed by BatchNorm, applied at every pixel. (Chapter 12)

Spectral decompressor
The 1x1 convolution from 32 channels back to 165 bands at the end of Indradhanu, with no normalisation and no activation. (Chapter 12)

Spectral family
In the code, the detector a band came from (the SpectralFamily enum: VNIR, SWIR, PANCHROMATIC); not the five grid segments. (Chapter 3)

Spectral floor
The part of a spectrum outside the span of the spectral output layer's curves; no decoder can remove it. (Chapter 42)

Spectral library
A collection of laboratory reflectance spectra of known materials; the product uses a curated subset of USGS splib07. (Chapter 32)

Spectral namer
The two-tower network built on 2026-10-31 (app/spectral_match/namer/) that names flagged pixels: P(man-made), a man-made class and a fill estimate. (Chapter 44)

Spectral response function (SRF)
The curve describing how strongly a band responds at each wavelength; a band's value is a weighted average of the spectrum with these weights. (Chapter 2)

Spectral self-completion
Training a pixel to fill its own hidden bands from its other bands; rejected because an anomaly could then rebuild itself. (Chapter 24)

Spectral theorem
Every real symmetric matrix can be written U Lambda UT with orthogonal unit eigenvectors in U and real eigenvalues in Lambda. (Chapter 6)

Spectral tokenizer
HyperSL's front end: three 1-D convolutions over band index (kernels 5, 9, 13, stride 1) fused by a 1 x 1 convolution, giving one 256-number token per band. (Chapter 46)

Spectral-stem model
The wavelength-aware variant of Indradhanu (V2) that replaces the fixed 165-band compressor and decompressor; same encoder and decoder classes. (Chapter 20)

SpectralEarth
DLR's collection of 539,755 EnMAP patches, 128 by 128 pixels on 202 bands, published for pretraining hyperspectral models; Indradhanu's pretraining set. (Chapter 27)

Spectrum
One pixel's values across all bands, read in wavelength order; treated as a vector in R to the C. (Chapter 2)

Split by site
Dividing data into training and test so that every image of one place lands on the same side, preventing the test score from flattering the model. (Chapter 27)

Square-root stretch
Displaying the square root of a score so that a heavy-tailed distribution of squared distances spreads across the colour scale. (Chapter 30)

Squeeze-and-excitation
A per-image (here per-patch) channel gate computed from a pooled summary; it gives every pixel of the patch the same recipe. (Chapter 24)

Stable softmax
Computing the softmax after subtracting the largest score from all scores, which leaves the result unchanged and prevents overflow. (Chapter 14)

Stage
One level of the encoder: a patch embedding, a number of transformer blocks and a LayerNorm, at one grid size and width. (Chapter 17)

Stage 3
The reference evaluation of 2026-10-17 (configs/eval/stage3.json) that reproduced earlier numbers and exposed bench-v1's bias. (Chapter 35)

Standard deviation
The square root of the average squared distance from the mean; the typical spread of a set of numbers, in their own units. (Chapter 1)

Standard error
The standard deviation of a statistic's sampling distribution; sigma / sqrt(n) for the mean of n independent values. (Chapter 36)

Standard normal
The Gaussian with mean 0 and standard deviation 1. (Chapter 7)

Statistical ensembler
A notebook-only detector that fuses global and local RX score maps by their ranks. (Chapter 8)

Step (iteration)
One update of the parameters by the optimiser. (Chapter 26)

Stochastic gradient descent
Gradient descent using batch gradients, which are cheap, noisy estimates of the full gradient. (Chapter 26)

Strength
The single number (a, f or s) that scales an implant; solved so the realised contrast hits its target. (Chapter 34)

Strict loading
The default behaviour of PyTorch's load_state_dict: every saved tensor must find a slot of the same name and shape, or loading fails with an error. (Chapter 30)

Stride
In local RX, scoring only every n-th row and column and interpolating the rest. (Chapter 8)

Stride (of a convolution)
How far the kernel jumps between neighbouring output positions. (Chapter 10)

Stride (of a tensor)
The memory distance between neighbouring elements along one axis. (Chapter 10)

Stride (sliding window)
The distance between the starts of neighbouring tiles; 32 pixels for Indradhanu at inference. (Chapter 30)

Strided convolution
A convolution whose window moves more than one position at a time; with kernel equal to stride the windows tile the grid without overlap. (Chapter 15)

Sub-pixel convolution
A convolution that outputs r squared channels per output channel so that a pixel shuffle can turn them into an r by r block of pixels. (Chapter 19)

Sublayer
One of the two halves of a block, attention or Mix-FFN, each with its own LayerNorm, dropout and residual. (Chapter 16)

Summary token
One of the N / R-squared reduced tokens of ESA, each standing for an R-by-R block of the token grid; there are 16 at every stage for a 128-by-128 patch. (Chapter 15)

Summed-area table
Another name for an integral image. (Chapter 31)

Surface-driven label
A cirrus label whose pixels differ from clear land mainly in the shortwave infrared, so it follows the ground rather than a cloud. (Chapter 28)

Symmetric matrix
A square matrix equal to its own transpose, such as every covariance matrix. (Chapter 6)

Target / ring ratio
Median b on a target's pixels divided by median b on pixels 4 to 8 city-block steps away; above 1 means the head expects more error on the target than on its own surroundings. (Chapter 41)

Teacher-student training
Training a student model to match the output of a teacher that sees more; here the teacher would be an EMA of the student. (Chapter 38)

Template (file)
A dictionary, injected into a FileHelper, mapping each logical component to where it lives in the file. (Chapter 3)

Tensor
In this book, a multi-index array of numbers. (Chapter 2)

The 39
The 39 of 909 real labelled objects lying wholly inside FT03's water / cloud mask; 35 are on water or shorelines, 4 under haze in set4. (Chapter 35)

Thermal sensor
A sensor recording emitted heat in one thermal-infrared band, giving a cube with C = 1. (Chapter 2)

Thick-cloud rule
Since 2026-10-29, V2 excludes only the head's thick-cloud output, not thin cloud, because the Heavy model's thin-cloud output fires on haze. (Chapter 39)

Thin-cloud channel
The segmentation head's third output; it fires on haze, which made it delete targets under set4's haze. (Chapter 43)

Threshold
A score above which a pixel is flagged; chosen from the false-alarm rate one accepts. (Chapter 7)

Tier
Subtle, medium or easy: contrast z below 0.502, up to 0.934, or above; the terciles of the 909 real objects. (Chapter 34)

Tile validity filter
The rule that skips any tile with less than 10 percent valid pixels at inference (training used 40 percent). (Chapter 30)

Tiled segmentation
Running the segmentation head in overlapping 128 px windows at stride 64 and averaging the probabilities where windows overlap. (Chapter 39)

Tirohita
Antardhana's L1 sibling, smaller per stage, deeper, and hiding 50 to 75 percent; 329,665 parameters. (Chapter 11)

To-channel layer
The Linear(32, 1) shared by all 32 answers that turns each answer vector into one output channel. (Chapter 23)

Token
A small block of the image, 4 by 4 pixels at Indradhanu's first stage, summarised as one vector. (Chapter 9)

Token bottleneck
Stage 1's squeeze of 4 x 4 pixels x 32 channels = 512 numbers into one 32-wide token vector. (Chapter 42)

Token grid
The H' by W' arrangement of tokens produced by a patch embedding; 32 by 32 at stage 1 for a 128 by 128 patch. (Chapter 13)

Token mask
The (B, N) array of token validities: whether each token holds real data. A fact about the sensor, not a choice. (Chapter 18)

Token straddling
An object that crosses stage-1 token borders, so in each masked pass part of it stays visible and helps reconstruct the hidden part, lowering its residual. (Chapter 21)

Token validity
A token is valid when its valid fraction is strictly greater than one half; exactly half counts as invalid. (Chapter 18)

Token width
The number of components d in each token; 32 in the spectral stem. (Chapter 22)

Top-of-atmosphere reflectance
pi L / (E0 cos of the solar zenith): a crude conversion of radiance that removes the sun but not the atmosphere. (Chapter 34)

Topological order
An ordering of a DAG's nodes in which every node comes after all of its parents; found with Kahn's algorithm. (Chapter 4)

torchinfo
A PyTorch library that prints a per-layer table of shapes, parameter counts and multiply-adds. (Chapter 20)

Total variation
The sum over side-by-side pixel pairs of the absolute difference of their values; as a penalty it shrinks every target by an amount proportional to perimeter over area. (Chapter 49)

Tower
One encoder in a model that has several side by side; the namer has a pixel tower and a library tower. (Chapter 44)

TPR at a fixed FPR
The best detection rate at any exact ROC point whose false-alarm rate is at most the given value. (Chapter 33)

Trace
The sum of a square matrix's diagonal entries; equal to the sum of its eigenvalues. (Chapter 7)

Trained-band mask
The rule that keeps only bands within 15 nm of a wavelength the model has trained on, guarding against extrapolation. (Chapter 24)

Trained-wavelength record
The buffer trained_wl listing every wavelength the model has trained on; it grows during fine-tuning and decides which bands are accepted at load time. (Chapter 29)

Training mode
The mode in which BatchNorm normalises with the current batch's statistics and updates its running statistics. (Chapter 12)

Transpose
A matrix with rows and columns swapped; the transpose of a product reverses the order, (AB)T = BT AT. (Chapter 6)

Transpose (axes)
Reordering an array's axes without changing any value, such as BIL to BSQ. (Chapter 3)

Transposed convolution
The learnable upsampling layer that spreads each input value over a kernel-sized stretch of the output; with kernel 4, stride 2, padding 1 it doubles height and width. (Chapter 11)

Trapezoid rule
Area of a straight segment as width times the average of its two heights; summed over segments it gives the area under a piecewise-linear curve. (Chapter 33)

Trim fraction
The share tau of the largest per-pixel errors a trimmed loss discards; 0.05 for Chakshu, 0 for Indradhanu. (Chapter 21)

Trimmed loss
A loss that sorts the per-pixel errors and averages only the smallest fraction 1 minus tau of them, so the worst (presumed anomalous) pixels give no gradient; Chakshu trims 5 percent. (Chapter 21)

True-positive rate (TPR)
The share of targets flagged at a threshold; also called the detection rate. (Chapter 33)

Truncation (PCA)
Keeping only the d directions with the largest eigenvalues (d = 30 here) and discarding the rest, which stabilises the score at the cost of sensitivity in the quietest directions. (Chapter 31)

Two-level namer
A naming design (2026-10-31; built the same day as the spectral namer, Chapter 44): level 1 asks whether nature explains a pixel, level 2 which man-made class it is; P(class) = P(man-made) x P(class | man-made). (Chapter 32)

Two-pass inference
Running the model twice with complementary masks and keeping each pixel's prediction from the pass in which it was hidden. (Chapter 9)

Two-pass masking
Running the model twice on a tile with complementary token masks and taking each pixel's prediction from the pass in which it was hidden, so that no pixel is ever predicted while visible. (Chapter 30)

Two-pass validation
Validation that hides a random half of the tokens, then the complementary half, so every valid pixel is graded exactly once from a prediction that did not see it. (Chapter 25)

Two-point form
The equation of a line through two known points: y = y0 + slope times (x - x0). (Chapter 5)

Two-tower model
A model with two encoders whose outputs are compared, usually by a cosine; also called a dual encoder. (Chapter 44)

Unbiased variance
A variance that divides by M minus 1; PyTorch feeds it to BatchNorm's running variance. (Chapter 12)

Uncertainty head
A small third decoder on Indradhanu's encoder features that predicts, per pixel, the log of the reconstruction error the model expects there (FT06, 95,632 parameters). (Chapter 41)

Undefined behaviour (by contract)
Output that a function's documentation declares meaningless for some inputs; whatever a given version returns cannot be relied on. (Chapter 37)

Union bound
The chance that at least one of several events happens is at most the sum of their chances. (Chapter 32)

Unit vector
A vector of length 1, used to represent a pure direction. (Chapter 6)

Unpredictability
Error that remains because a hidden pixel's value is not fixed by its context; it is what makes targets detectable. (Chapter 42)

Upscale factor
The factor r of a pixel shuffle; 4 here, taking 32 by 32 to 128 by 128. (Chapter 19)

UUID
A 128-bit random identifier; on the wire always written with a type prefix such as scene_ or action_. (Chapter 4)

Valid fraction
The proportion of a token's 16 pixels that are valid, computed as the average of the 0/1 pixel mask over the token's block. (Chapter 18)

Valid-band loss
A reconstruction loss computed only over bands that were measured; dead and padded bands are excluded from the L1 mean and zeroed before SAM. (Chapter 24)

Validity cube
A 0/1 array of the cube's shape marking which band values of which pixels are real measurements. (Chapter 2)

Validity mask
An array on the image grid holding 1 for a genuine measurement and 0 for dead detectors, cloud or off-swath pixels. (Chapter 1)

Value
The learned projection of a token that it hands over, in proportion to its attention weight, when another token attends to it. (Chapter 14)

Value stretch
The Linear(1, d) layer that turns one reflectance number into a d-number vector; 64 parameters at d = 32. (Chapter 22)

Vanishing gradient
The shrinking of the error signal to almost nothing after it is multiplied by many factors smaller than one on its way back through a deep network. (Chapter 16)

Variance
The average squared distance of a random number from its mean; its square root is the standard deviation. (Chapter 14)

Variance of a difference
Var(A - B) = Var A + Var B - 2 Cov(A, B); small when A and B move together. (Chapter 36)

VCA
Vertex component analysis: finds endmembers as the most extreme pixels of the data cloud. (Chapter 49)

Vector
An ordered list of real numbers, written as a column; a spectrum on the 165-band grid is a vector in R to the 165. (Chapter 2)

Vendable
The standard in-memory form of a scene: cube in physical units, validity cube and per-band metadata; consumed by everything downstream. (Chapter 4)

View
A tensor operation that reinterprets the shape of stored numbers without copying them, used to split 32 numbers into 4 heads of 8. (Chapter 23)

Visible neighbours
Of the 8 tokens around a hidden token, those the encoder can see; expected 2.81 for random single tokens and 1.77 for 2 x 2 blocks at ratio 0.65. (Chapter 42)

Vision Transformer (ViT)
A transformer for images that cuts the image into non-overlapping patches and embeds each by a linear projection. (Chapter 13)

VNIR and SWIR detectors
The two detector arrays of a hyperspectral instrument, for the visible/near-infrared and the shortwave-infrared ranges. (Chapter 3)

Warm start
Initialising a component from previously trained weights rather than at random, as FT03's head from FT01's probe. (Chapter 29)

Warmup
A schedule phase that raises the learning rate gradually from a small value at the start of training; described by warmup-epochs in this repository's config but never implemented. (Chapter 26)

Water guard
The rule that a confident water pseudo-label over a mean 800-900 nm reflectance above 0.15 becomes a known not-water. (Chapter 38)

Water vapour column
The mass of water in the air above each square centimetre of ground, in g/cm2; 0.75 g/cm2 would condense to a 7.5 mm layer. (Chapter 28)

Wavelength
The distance between successive crests of a light wave, written lambda and measured in nanometres; different wavelengths are different colours, most of them invisible. (Chapter 2)

Wavelength bin
One cell of the fixed grid of 40 nm bins from 400 nm; 53 bins up to 2500 nm, the same on every sensor. (Chapter 24)

Wavelength encoding
A fixed vector of sines and cosines of 2 pi lambda / P over 16 geometric periods P from 10 to 10,000 nm; sines first, then cosines. (Chapter 22)

Wavelength query
A decoder query built from the wavelength encoding of a band to be reconstructed, through a Linear(32, 32). (Chapter 24)

Wavelength statistics
WavelengthStats: mean and standard deviation curves over wavelength, measured once on SpectralEarth and read at any band by linear interpolation. (Chapter 24)

Wavelength-linear decoder
WavelengthLinearDecoder: each requested band is U(lambda) . z + c(lambda), with U and c from one hypernetwork. (Chapter 24)

Wavelength-reading model
A model with a spectral stem that reads a scene's own bands at their stated wavelengths (F001 and the FT runs), so it needs no resample. (Chapter 37)

WavelengthStats
The normalisation of Indradhanu V2: mean and spread stored as curves over wavelength and read at each sensor's own band wavelengths. (Chapter 5)

Weight
A learned multiplier applied to one input of a neuron or one entry of a kernel. (Chapter 10)

Weight decay
Shrinking every weight towards zero by a fixed fraction each step, a form of regularisation. (Chapter 26)

Weight sharing
Using the same kernel weights at every position, which makes a convolution's parameter count independent of image size. (Chapter 10)

Weighted mean
A sum of values times weights divided by the sum of the weights. (Chapter 38)

Whitened coordinate
One component of a whitened vector; the sum of their squares is the Mahalanobis distance. (Chapter 31)

Whitening
Rotating vectors onto the principal directions and dividing each coordinate by its standard deviation, so the cloud has identity covariance. (Chapter 6)

Width
The number of channels a layer carries. (Chapter 43)

Width weighting
Weighting each band by the nanometres of spectrum it stands for, so an average over bands approximates an integral over wavelength. (Chapter 27)

Width-squared scaling
Because most weights sit in C x C matrices, multiplying width by k multiplies parameters by about k squared (8.86 for k = 3 at stage 1). (Chapter 43)

Wire format
A data layout that stored files and saved models depend on by position, so that changing it silently invalidates them. (Chapter 3)

With replacement
Drawing so that each pick can be any item, including ones already picked; some items appear several times, some not at all. (Chapter 36)

Worker
The process that claims jobs from the jobs table and runs the analysis. (Chapter 4)

z-score
A value expressed as standard deviations from its band's mean: z = (x - mean)/std. (Chapter 5)

Zero padding
Treating positions outside the grid as zeros so that a convolution keeps the grid size; it also makes border positions distinguishable. (Chapter 16)

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Nassim Ait Ali Braham, Conrad M. Albrecht, Julien Mairal, Jocelyn Chanussot, Yi Wang, and Xiao Xiang Zhu. SpectralEarth: Training hyperspectral foundation models at scale. arXiv preprint arXiv:2408.08447, 2024. A later journal version may exist; the arXiv preprint is cited here.

[3] Jane Bromley, Isabelle Guyon, Yann LeCun, Eduard Säckinger, and Roopak Shah. Signature verification using a "Siamese" time delay neural network. In *Advances in Neural Information Processing Systems 6 (NIPS 1993)*, 1993. The original Siamese network: two inputs through shared weights, compared by distance. Page numbers not checked for this book.

[4] Daniele Cerra et al. DLR HySU: A benchmark dataset for spectral unmixing. *Remote Sensing*, 2021. The HySU sub-pixel target set (0.25–3 m squares at 0.7 m). Cited in the experiment README as "Cerra et al. 2021"; volume, article number and full author list not checked for this book.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[6] Bradley Efron. Bootstrap methods: another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

[7] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia, and Kaiming He. Accurate, large minibatch SGD: Training ImageNet in 1 hour. *arXiv preprint arXiv:1706.02677*, 2017.

[8] Andrew A. Green, Mark Berman, Paul Switzer, and Maurice D. Craig. A transformation for ordering multispectral data in terms of image quality with implications for noise removal. *IEEE Transactions on Geoscience and Remote Sensing*, 26(1):65–74, 1988. doi: 10.1109/36.3001.

[9] James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. *Radiology*, 143(1):29–36, 1982.

[10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[11] Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.

[12] Daniel C. Heinz and Chein-I Chang. Fully constrained least squares linear spectral mixture analysis method for material quantification in hyperspectral imagery. *IEEE Transactions on Geoscience and Remote Sensing*, 39(3):529–545, 2001. FCLS. Volume and pages from memory, not checked for this book.

[13] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (GELUs). arXiv preprint arXiv:1606.08415, 2016.

[14] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7132–7141, 2018.

[15] Peter J. Huber. Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35(1):73–101, 1964.

[16] Lawrence Hubert and Phipps Arabie. Comparing partitions. *Journal of Classification*, 2: 193–218, 1985. The adjusted Rand index. Volume and pages from memory, not checked for this book.

[17] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning (ICML)*, volume 37 of *Proceedings of Machine Learning Research*, pages 448–456, 2015.

[18] Xudong Kang, Xiangping Zhang, Shutao Li, Kenli Li, Jun Li, and Jón Atli Benediktsson. Hyperspectral anomaly detection with attribute and edge-preserving filters. *IEEE Transactions on Geoscience and Remote Sensing*, 55(10):5600–5611, 2017. Author list beyond the first author not verified against the paper.

[19] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[20] Raymond F. Kokaly, Roger N. Clark, Gregg A. Swayze, K. Eric Livo, Todd M. Hoefen, Neil C. Pearson, Richard A. Wise, William M. Benzel, Heather A. Lowers, Rhonda L. Driscoll, and Anna J. Klein. USGS spectral library version 7. Data Series 1035, U.S. Geological Survey, 2017. 61 pages; author list as published by USGS, verify against the report's title page.

[21] Weili Kong, Baisen Liu, Xiaojun Bi, Changdong Yu, Xinyao Li, and Yushi Chen. HyperSL: A spectral foundation model for hyperspectral image interpretation. *IEEE Transactions on Geoscience and Remote Sensing*, 63:5513119, 2025. doi: 10.1109/TGRS.2025.3566205. The model of Part X. Code github.com/kkweil/HyperSL (commit e56de7e), weights huggingface.co/WeilKon/HyperSL. Authors, volume and article number read from the PDF's first page; the paper describes the per-band scaling only as "layer-by-layer normalization".

[22] F. A. Kruse, A. B. Lefkoff, J. W. Boardman, K. B. Heidebrecht, A. T. Shapiro, P. J. Barloon, and A. F. H. Goetz. The spectral image processing system (SIPS) - interactive visualization and analysis of imaging spectrometer data. *Remote Sensing of Environment*, 44(2–3):145–163, 1993.

[23] Yanghao Li, Naiyan Wang, Jianping Shi, Jiaying Liu, and Xiaodi Hou. Revisiting batch normalization for practical domain adaptation. arXiv preprint arXiv:1603.04779, 2016. Introduces AdaBN; a journal version appeared later, details not verified here.

[24] Zhaoxu Li, Yingqian Wang, Chao Xiao, Qiang Ling, Zaiping Lin, and Wei An. You only train once: Learning a general anomaly enhancement network with random masks for

hyperspectral anomaly detection. *IEEE Transactions on Geoscience and Remote Sensing*, 61, 2023. doi: 10.1109/TGRS.2023.3258067. The AETNet paper that published HAD100; volume and DOI from research/experiments/benchmarks/README.md; article number not verified.

[25] Jie Lian, Lizhi Wang, He Sun, and Hua Huang. GT-HAD: Gated transformer for hyperspectral anomaly detection. *IEEE Transactions on Neural Networks and Learning Systems*, 36 (2), 2025. Volume and issue as cited in research/experiments/gt-had-prisma/summary.md; author list and page range not verified against the publisher.

[26] Liu et al. Endmember-free hyperspectral unmixing. *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing*, 18, 2025. HyperSL backbone plus an unmixing head. Cited from the experiment README (commit 4d292ac) as "Liu et al., Endmember-Free Hyperspectral Unmixing (JSTARS 18, 2025)"; full author list, exact title wording and pages not checked for this book.

[27] Stuart P. Lloyd. Least squares quantization in PCM. *IEEE Transactions on Information Theory*, 28(2):129–137, 1982.

[28] Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017. arXiv:1608.03983.

[29] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019. arXiv:1711.05101.

[30] Donna Katzman McClish. Analyzing a portion of the ROC curve. *Medical Decision Making*, 9(3):190–195, 1989.

[31] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. Mixed precision training. In *International Conference on Learning Representations (ICLR)*, 2018. arXiv:1710.03740.

[32] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *Fourth International Conference on 3D Vision (3DV)*, pages 565–571, 2016.

[33] José M. P. Nascimento and José M. Bioucas-Dias. Vertex component analysis: A fast algorithm to unmix hyperspectral data. *IEEE Transactions on Geoscience and Remote Sensing*, 43(4):898–910, 2005. VCA. Volume and pages from memory, not checked for this book.

[34] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning (ICML)*, volume 28 of *Proceedings of Machine Learning Research*, pages 1310–1318, 2013.

[35] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

[36] Irving S. Reed and Xiaoli Yu. Adaptive multiple-band CFAR detection of an optical pattern with unknown spectral distribution. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 38(10):1760–1770, 1990. doi: 10.1109/29.60107.

[37] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*, volume 9351 of *Lecture Notes in Computer Science*, pages 234–241. Springer, 2015.

[38] Leonid I. Rudin, Stanley Osher, and Emad Fatemi. Nonlinear total variation based noise removal algorithms. *Physica D*, 60:259–268, 1992. Total variation as a penalty. Volume and pages from memory, not checked for this book.

[39] Wenzhe Shi, Jose Caballero, Ferenc Huszár, Johannes Totz, Andrew P. Aitken, Rob Bishop, Daniel Rueckert, and Zehan Wang. Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 1874–1883, 2016.

[40] Jake Snell, Kevin Swersky, and Richard Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. Classification by distance to class prototypes in an embedding space; the idea behind level 1 by matching. Page numbers not checked for this book.

[41] C. Spearman. The proof and measurement of association between two things. *The American Journal of Psychology*, 15(1):72–101, 1904.

[42] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.

[43] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018. The InfoNCE loss.

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*, pages 5998–6008, 2017.

[45] Wenhai Wang, Enze Xie, Xiang Li, Deng-Ping Fan, Kaitao Song, Ding Liang, Tong Lu, Ping Luo, and Ling Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 568–578, 2021.

[46] Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M. Alvarez, and Ping Luo. SegFormer: Simple and efficient design for semantic segmentation with transformers. In *Advances in Neural Information Processing Systems*, volume 34, pages 12077–12090, 2021.

Index

Symbols
1-pixel tolerance, 508

A
abundance, 516, 733
accuracy, 3
Action, 36
activation function, 137
AdaBN, 432
Adam, 369
AdamW, 372
adjusted Rand index, 712
affine transform, 45, 490
all-MLP decoder, 250
Allotrope, 2
Amdahl's law, 666
angle between vectors, 68
annulus, 103
anomaly, 2
anomaly score, 4, 119
argsort, 233, 554
array, 18
attention
    cross, 190, 289
    efficient self-attention, 197
    multi-head, 188
    scaled dot-product, 182
    scaling by root d, 186
    self, 190, 289
    weights, 290
attention matrix, 182
AUC, 500
autoencoder, 119
average pooling, 230
axis, 133

B
background subspace, 519
backpropagation, 355
band, 15
band grid check, 590
band width weighting, 394
band-delta compression, 402
band-grid check, 493
band-list augmentation, 393
base rate, 4
batch, 22, 134, 303, 367
batch normalisation, 156, 427
    scale and shift, 157
BatchNorm, 148, 207, 678
    buffers, 258
Bernoulli distribution, 410
best checkpoint, 387
bfloat16, 376
bias, 137
binary cross-entropy, 411
    weighted, 582
binary dilation, 507
binomial distribution, 429
bit flag, 27
blind spot, 569
blind unmixing, 733
block (of shards), 395
bootstrap, 538
bottleneck, 119
box sum, 472
broadcasting, 59, 136, 447
buffer, 257
buffer (PyTorch), 59, 426

C
cache key, 530
capacity, 649
capacity block, 657
catastrophic cancellation, 60
ceiling function, 330
centre wavelength, 16
centring, 74
chain rule, 207, 354
Chakshu, 6
chance row, 530
channel, 15

characteristic polynomial, 77
Chebyshev distance, 680
checkerboard mask, 446
checkpoint, 444
checkpoint selection, 566
chi-square distribution, 92
    thresholds, 92
circular procedure, 719
class priority, 592
classical detector, 5
classifier, 3
cloud exclusion, 494
codename, 492
codename routing, 588
cofactor, 95
combined loss, 360
composite score, 481
compute versus parameters, 263
computed mask, 106
concatenation, 250
condition number, 97
confounding, 661
confusion matrix, 499
connected component, 507
connectivity, 702
constant extrapolation, 53
context, 121
contract (of a function), 551
control (experiment), 661
convex combination, 292
convolution, 138
    1×1, 143
    1x1, 247
    depthwise, 211
    output size, 219
    parameter count, 258
    strided, 195
coordinate reference system, 490
correlation, 76
cosine annealing, 373
cosine similarity, 297
covariance, 74, 538
covariance matrix, 74
    rank of a sample, 105
coverage count, 451
cross-attention, 289
cross-check, 557
cross-correlation, 138
cross-validation, 609, 742
cubic spline, 55
curriculum, 424

D
dataset pooling, 531
DatasetBuilder, 42
datatake, 400
decoder, 119
decoder width, 247
decompressor, 253
denormalisation, 253
depthwise convolution, 211
    parameter count, 258
detach, 620
determinant, 73
diagonal loading, 98
Dice coefficient, 413
digital number, 21, 269, 401
dilation, 240, 482, 576, 608
directed acyclic graph, 37
dot product, 67, 176
dropout, 209
dual encoder, 488, 671
dual window, 103

E
early stopping, 387, 607
edge strength, 567
effective sample size, 540
efficient self-attention, 197
    fallback, 202
eigen-decomposition, 465
eigenvalue, 77
eigenvector, 77
embedding, 165, 671, 707
embedding width, 220
encoder, 119, 671
endmember, 733
epoch, 367
erosion, 240, 452
evaluation config, 528
exclude scorer, 528
expansion ratio, 210
expected value, 225
explained variance ratio, 80
exponential moving average, 369, 427, 571
Export, 36
export bundle, 491

F
factorial design, 583
false-alarm rate, 87
false-positive rate, 499
family matcher, 484
FCLS, 734

feature pyramid, 220
feed-forward network, 210
fencepost formula, 30
FileHelper, 28
fill, 671
float16, 530, 607
float32 and float64, 98
floor, 140, 330
foreign fraction, 519
foreignness, 574
forward pass, 253
foundation model, 6, 707
full width at half maximum, 16
fuse convolution, 250
fused attention, 313

G
gate (fine-tuning), 425
gather, 236
gating, 607
Gaussian distribution, 86
    multivariate, 89
Gaussian mixture, 604
GELU, 212
generalised eigenproblem, 108
generalised likelihood ratio, 91
geometric series, 428
gradient, 367
    finite difference, 567
gradient accumulation, 432
gradient clipping, 375
gradient descent, 367
gradient-matching loss, 570
graph, 37
ground truth, 498
guard window, 103, 470

H
halo, 122, 520
hash, 530
haze colour test, 534
head (network), 406
head dimension, 188, 221
heartbeat, 40
hypernetwork, 332
hyperspectral, 19
hyperspectral cube, 18

I
identity function, 120
identity matrix, 73
identity transform, 490
idle parameters, 264
implant
    signature, 555
implant bench, 514
implant-denoising, 575
inclusion-exclusion, 472
Indradhanu, 6
Indradhanu Heavy
    parameter count, 266
integral image, 471
interpolation
    bilinear, 249
    linear, 248, 550
intersection over union, 415, 564
invariant 7, 532
inverse
    of a 2x2 matrix, 73
    of a 3x3 matrix, 95
inverse map, 58
irradiance, 13

J
jump, 142, 223

K
k-means, 605
keep mask, 445
kernel, 138
key, 181, 289
known mask, 416

L
L1 loss, 346
L1 norm, 346
L2-SP, 572
label, 3
Lagrange multiplier, 79, 600
Lambertian surface, 13
land surface temperature, 269
Laplace distribution, 615
law of total variance, 439
layer normalisation, 168
LayerNorm, 196, 205
    parameter count, 258
layout
    sequence, 135
    spatial, 135
layout (array), 25
learned query, 292
learning rate, 367
learning-rate schedule, 372
leave-one-scene-out, 609

lexicographic order, 588
likelihood, 411, 617
likelihood ratio, 91
linear interpolation, 324
  two-point form, 51
  weight form, 52
linear layer, 137, 181, 304
  parameter count, 258
linear map, 285
linear mixing, 516
linear mixing model, 733
linear mixture, 572
linear solve, 100
linearity of expectation, 225, 633
loader worker, 657
local RX, 102
  batching, 105
  cost, 107
local-mean Mahalanobis distance, 469
locality, 138
log-uniform distribution, 681
logit, 407
logsumexp, 683
loss, 120, 344
  L1, 120
loss mask, 345

M
machine epsilon, 98
Mahalanobis distance, 89
  invariance, 111
Manhattan distance, 514
manifest, 589
Mann–Whitney statistic, 501
mask, 126
mask ratio, 232, 445
  effective, 232
mask token, 227
masked autoencoder, 127
masked mean, 105
masking
  attention, 308
matched filter, 600
matrix, 72
matrix product, 72
matrix-vector product, 72
max pooling, 241
maximum likelihood, 617
maximum noise fraction, 108
  eigenvalues, 111
  noise estimate, 110
mean, 5, 124
mean absolute error, 346
mean squared error, 349
mean vector, 74
mebibyte, 312
median, 125
Mix-FFN, 210
mixed background, 96
mixed batch, 432
mixed precision, 376
mixing line, 516
MNF local RX, 112
MNF-RX, 6, 111
modular arithmetic, 402
momentum, 368
mRMSE, 734
mSAD, 734
multi-label classification, 409
multiply-add, 193, 252, 263, 311

N
nanometre, 12
native band list, 399
native bands, 492
NDSI, 419
nearest-valid fill, 61
negative, 499
negative log-likelihood, 617
neural network, 6
neuron, 137
noise covariance, 110
noise fraction, 108
norm, 68
normalisation
  per-band, 57
normalisation curve, 387
normalised convolution, 602
np.allclose, 591
np.interp, 551
null space, 286

O
object, 507
object AUC, 734
object measure, 602
open-set recognition, 487
orthogonal, 69
orthogonal matrix, 78
outer product, 74
output projection, 188
overall accuracy, 712
overfitting, 395, 659
overlap, 449

P
padding, 139, 310
parameter, 6, 118, 257, 284, 344
  versus buffer, 589
parameter (model), 426
parity run, 594
partial AUC, 504
partition, 445
pass criterion, 562
patch, 386
patch embedding, 165
patch size, 449
patience, 387
PCA-RX, 113, 529
PCHIP, 55
per-band affine map, 717
per-pixel contribution, 541
percentile, 8
percentile interval, 539
percentile threshold, 480
period, 294
permutation, 233
  inverse, 234
permutation equivariance, 187
pixel, 2
pixel measure, 602
pixel shuffle, 246, 571
planted-sample generator, 680
point-spread function, 521
position code, 707
positional encoding, 187
  sinusoidal, 294
positive, 499
positive semi-definite, 78
post-norm, 208
pre-norm, 208
precision, 742
prediction mask, 445
principal component analysis, 79
principal direction, 79
probability density, 86, 615
probe, 426, 563
Project, 36
projection, 70, 637
pseudo-label, 581
PSF halo, 625

Q
quantile, 630
query, 181, 289
quotient rule, 355

R
radiance, 13, 512
ramp, 361
rank, 105, 286
rank (of a matrix), 160, 640
rank (of a tensor), 133
rank (ordering), 622
rank fusion, 603
rank–nullity theorem, 286
reading (of a scene), 528
reaper, 40
recall, 742
receptive field, 142, 223
reconstruction, 118
reconstruction model, 118
red edge, 18
reduction ratio, 222
reflectance, 13, 512
regular expression, 486
regularisation, 98, 371, 467
renormalisation, 308
resampling, 29
  as a matrix, 552
residual, 119, 460
residual connection, 207
residual covariance, 462
residual Mahalanobis
  scale invariance, 584
residual Mahalanobis distance, 462
responsibility, 604
Riemann sum, 394
ring, 470, 514
ring (around a target), 625
ring mean, 679
robust loss, 350
ROC curve, 499
row-major order, 25, 134, 231
running statistics, 157
RX detector, 6, 90
  global, 96
  plain, dropped, 467

S
SAM, *see* spectral angle
SAM weight, 361
sampling distribution, 538
scale parameter, 615
scatter, 236
scene, 2, 36
scene-major loop, 529
scope, 533
score map, 4, 85

score-map cache, 530
score-map correction, 568
scoring methods
  comparison, 475
SegFormer, 129
SegFormer block, 214
segmentation, 406
  tiled, 592
selection matrix, 237
selection risk, 627
shape, 18, 133
shape-preserving model, 253
shard, 378, 386, 657
shift difference, 110
shrinkage, 99, 468
Siamese network, 671
sigmoid, 326, 409
sign function, 348
sign test, 478
signal-to-noise ratio, 108
single-link clustering, 401
singular matrix, 97
singular value decomposition, 577
skip connection
  full-resolution, 640
sliding window, 449
slope, 51
softmax, 177, 308, 410
  masked, 308
  numerical stability, 178
  saturation, 179
span, 637
spatial completion, 725
spatial reduction, 195
Spearman rank correlation, 622
spectral angle, 351
Spectral Angle Mapper, 351, 484
spectral floor, 637
spectral response function, 15
spectral self-completion, 324, 725
spectral theorem, 78
SpectralEarth, 385
spectrum, 17
standard deviation, 5
standard error, 538
  binomial, 559
standard normal, 86
statistical ensembler, 114
step, 367
stochastic gradient descent, 367
stride, 377
  of a convolution, 139
  of a tensor, 134
  sliding window, 449
sub-pixel convolution, 250
summed-area table, 471
surface-driven label, 420
symmetric matrix, 72

T

teacher-student, 571
temperature, 683
template (file), 28
tensor, 18, 118, 133
thermal RX, 100
thermal sensor, 20
thread, 666
threshold, 8, 87, 499
tier, 516
token, 126, 135, 165, 175, 289, 445
token validity, 230, 445
token width, 294
top-of-atmosphere reflectance, 512
topological order, 39
total variation, 738
tower, 671
TPR at FPR, 507
trace, 93
trained band mask, 591
trained-wavelength record, 429
training, 120
transformer block, 214
  parameter count, 259
transpose, 72
transpose (axes), 26
transposed convolution, 148
trapezoid rule, 500
trimmed loss, 276
true-positive rate, 499
truncation, 468
two-level namer, 486
two-pass masking, 446

U

uncertainty head, 620
union bound, 485
unit vector, 69
UUID, 37

V

valid fraction, 230
validity cube, 21
validity mask, 9

value, 181, 289
vanishing gradient, 208
variance, 74, 185
VCA, 733
vector, 17, 67
vendable, 42

W
warm start, 438
warmup, 372
water guard, 581
water vapour, 420
wavelength, 12
wavelength encoding, 294
weight, 137
weight decay, 371
weight sharing, 138
weighted average, 452
weighted mean, 576
whitening, 81, 465
width (of a layer), 649

Z
z-score, 57, 86